UIKit Playground

10 мини-приложений и каталог UI-паттернов от LaunchScreen до App Store

Учебник

Содержание

Введение **1**
Чем эта книга отличается . . . 2
Кому подойдёт . . . 2
Что мы построим — учебный проект `UIKitPlayground` . . . 3
0.1 Создаём проект в Xcode 26 . . . 4
0.2 Что значат эти три настройки . . . 5
Стек, на котором пишем . . . 6
Как читать . . . 6
Условные обозначения . . . 6
Спасибо . . . 7
Apple Developer Documentation . . . 7

Глава 0. Карта UIKit — кто от кого наследуется и что это даёт **8**
0.1 Карта . . . 8
Бытовая аналогия . . . 10
0.2 `UIResponder` — кто получает события . . . 10
0.3 `UIView` — прямоугольник на экране . . . 11
`frame`, `bounds`, `center` . . . 11
Дерево: `addSubview` . . . 11
Раскладка: что такое Auto Layout и почему он так называется . . . 11
Слой . . . 13
Кто получает касание . . . 13
Инициализаторы: когда писать свой . . . 13
0.4 `UIControl` — view, которое умеет «нажаться» . . . 14
Кто контрол, а кто нет . . . 15
0.5 `UIScrollView` и его наследники . . . 16
0.6 `UIViewController` — экран, а не view . . . 16
`view` создаётся лениво . . . 16
Сколько раз что вызывается . . . 17
Контейнеры . . . 17
`navigationItem` — чей он . . . 17
Модальный показ . . . 17
0.7 Сцена, окно и первый экран кодом . . . 18
Три объекта над экраном . . . 18
Кто кого создаёт при запуске . . . 18
`Info.plist`: Application Scene Manifest . . . 19
`AppDelegate` — отвечает на один вопрос . . . 19
`SceneDelegate` — четыре строки, которые создают интерфейс . . . 20

Первый экран целиком кодом . . . 22
Как из любого view добраться до окна, сцены и экрана устройства . . . 24
Если вместо экрана — чёрный . . . 24
0.8 Что не view . . . 25
0.9 Делегат и источник данных . . . 25
0.10 Проверь сам . . . 26
Ответы к упражнениям . . . 27
Что мы выучили . . . 27
Apple Developer Documentation . . . 28

Глава 1. LaunchScreen vs AnimatedSplash — два экрана, оба «splash» 29
1.1 Что происходит между тапом и первым экраном . . . 29
1.2 LaunchScreen — почему storyboard и почему без кода . . . 31
Каким должен быть launch screen . . . 31
1.3 AnimatedSplash — это уже твой view controller . . . 32
Свойства и конструктор . . . 32
viewDidLoad — один раз, когда view создан . . . 33
setupLayout — элементы и Auto Layout . . . 34
viewDidAppear — когда запускать анимацию . . . 36
1.4 Анимация — по очереди и с пружиной . . . 37
1.5 Бытовая аналогия . . . 39
1.6 Почему оба, а не только AnimatedSplash . . . 39
1.7 Кто решает, когда AnimatedSplash закончился . . . 40
Ответы к упражнениям . . . 40
Что мы выучили . . . 41
Apple Developer Documentation . . . 41

Глава 2. AppManifest — конфиг одного mini-app 43
2.1 Манифест как «паспорт» mini-app . . . 43
2.2 Флаги гейтов запуска . . . 44
Типы для гейтов: `PermissionKind` и `OnboardingPage` . . . 46
2.3 Фабрика `makeMain` — как создавать main-экран . . . 46
2.4 Заготовка для ещё не готовых mini-app . . . 48
Экран-заглушка . . . 49
2.5 AppRegistry — реестр всех mini-app . . . 51
2.6 Кто читает манифест . . . 52
2.7 Бытовая аналогия . . . 52
2.8 Что манифест не описывает . . . 53
Ответы к упражнениям . . . 53
Что мы выучили . . . 54
Apple Developer Documentation . . . 54

Глава 3. PlaygroundWindow — UIWindow с shake-detect 56
3.1 Что такое UIWindow и почему его обычно не трогают . . . 56
3.2 Минимальный наследник . . . 57
3.3 Responder chain — коротко . . . 58
3.4 Кто устанавливает `onShake` . . . 59
3.5 Где создаётся окно . . . 59

3.6 Как проверить встряхивание в симуляторе . . . 60
3.7 Бытовая аналогия . . . 61
3.8 Что можно было бы сделать вместо встряхивания . . . 61
Ответы к упражнениям . . . 61
Что мы выучили . . . 62
Apple Developer Documentation . . . 62

Глава 4. BootCoordinator — оркестратор гейтов **64**
4.1 Зачем отдельный класс . . . 64
4.2 Скелет — конструктор и зависимости . . . 65
4.3 Точка входа — `start()` . . . 66
4.4 Цепочка гейтов: один гейт — три метода . . . 67
4.5 `setRoot` — как меняем экран в окне . . . 69
Модальные экраны при выходе . . . 70
4.6 Кто держит координатор. Лаунчер целиком . . . 71
Data source — что показать . . . 72
Delegate — что делать по тапу . . . 73
Запуск mini-app . . . 74
4.7 Гейт может сам **выйти** в лаунчер . . . 75
4.8 Бытовая аналогия . . . 75
4.9 Что можно улучшить . . . 75
Ответы к упражнениям . . . 76
Что мы выучили . . . 77
Apple Developer Documentation . . . 77

Глава 5. Lifecycle App → Scene → VC и @MainActor под капотом **79**
5.1 App → Scene → VC: три уровня . . . 79
5.2 Info.plist: как iOS находит наши делегаты . . . 80
5.3 AppDelegate — что туда класть, а что нет . . . 81
5.4 SceneDelegate — где собирается всё дерево . . . 82
5.5 Порядок событий при запуске . . . 84
5.6 Lifecycle-уведомления, которые мы используем . . . 84
5.7 MainActor под капотом . . . 85
Когда нужен `nonisolated` . . . 86
Task: наследует или нет . . . 86
5.8 `MainActor.assumeIsolated` — мост к старым API . . . 88
5.9 Бытовая аналогия . . . 89
5.10 Что мы пропустили . . . 89
5.11 Собираем фундамент . . . 90
Ответы к упражнениям . . . 92
Что мы выучили . . . 92
Apple Developer Documentation . . . 93

Глава 6. Onboarding — UIPageViewController с dots indicator **94**
6.1 Что показывать на этих экранах . . . 94
6.2 UIPageViewController vs альтернативы . . . 96
6.3 Параметры контейнера . . . 97
6.4 DataSource — два метода . . . 99

6.5 PageControl и синхронизация со свайпом . . . 100
6.6 Кнопки «Пропустить» и «Дальше» . . . 102
6.7 «Уже видел» — флаг в UserDefaults . . . 103
6.8 Бытовая аналогия . . . 104
6.9 Краевой случай: одна страница . . . 104
6.10 Что можно добавить . . . 104
6.11 Экран целиком . . . 105
Ответы к упражнениям . . . 111
Что мы выучили . . . 111
Apple Developer Documentation . . . 112

Глава 7. Permission primer — объяснение перед системным диалогом 113
7.1 Что показываем . . . 113
7.2 Текст под каждый тип — `switch` по enum . . . 115
7.3 Кнопка — одна, и она ведёт к системному запросу . . . 116
7.4 Вёрстка . . . 118
7.5 `PermissionService` — обёртка над системными API . . . 119
CoreLocation — делегат через `withCheckedContinuation` . . . 120
Фото, камера и уведомления — готовые async-версии . . . 121
7.6 `shouldShow` — когда primer не нужен . . . 122
7.7 Info.plist: строки-пояснения . . . 124
7.8 Бывает и без разрешения . . . 125
7.9 Бытовая аналогия . . . 125
7.10 Экран целиком . . . 125
7.11 Краевые случаи . . . 131
Ответы к упражнениям . . . 132
Что мы выучили . . . 132
Apple Developer Documentation . . . 133

Глава 8. Auth gate — Login / Register / Forgot 134
8.1 Архитектура: контейнер + три экрана . . . 134
8.2 Делегирование событий — почему `delegate` вместо замыканий . . . 137
8.3 Keychain — где жить токену . . . 138
8.4 MockAuthService — почему без реального бэкенда . . . 141
8.5 Login — что внутри . . . 143
8.6 Register — индикатор надёжности пароля . . . 146
8.7 Forgot password — минимальный экран . . . 148
8.8 `shouldShow` — когда auth не нужен . . . 149
8.9 Бытовая аналогия . . . 149
8.10 Что мы не делаем (но в production стоит) . . . 149
8.11 Гейт целиком . . . 150
Ответы к упражнениям . . . 159
Что мы выучили . . . 159
Apple Developer Documentation . . . 160

Глава 9. Force-update + Maintenance — серверные гейты 162
9.1 Зачем force-update вообще . . . 162
9.2 AppConfigService — мок «remote config» . . . 164

9.3 Логика в BootCoordinator . . . 165
9.4 Обработка результата и сравнение версий . . . 167
Почему версии нельзя сравнивать как строки . . . 168
9.5 ForceUpdateViewController — блокирующий . . . 170
9.6 MaintenanceViewController — с таймером и повтором . . . 172
9.7 Что считать «hard» и «soft» update . . . 174
9.8 Demo-сценарии в реестре . . . 175
9.9 Бытовая аналогия . . . 175
9.10 Кеширование remote-config . . . 175
9.11 Экраны целиком . . . 176
Ответы к упражнениям . . . 178
Что мы выучили . . . 179
Apple Developer Documentation . . . 180

Глава 10. Region + Age gates — фильтры по локации и возрасту 181
10.1 Region picker . . . 181
Как из «KZ» получается флаг . . . 183
Хранение выбора . . . 183
10.2 Догадка по локали . . . 184
10.3 Список — UITableView с галочкой . . . 185
10.4 Age gate — когда он действительно нужен . . . 186
10.5 UIDatePicker — стиль «барабаны» . . . 188
10.6 Русские окончания — год / года / лет . . . 189
10.7 «Пока рано» — блокирующее состояние . . . 190
10.8 Хранение даты . . . 191
10.9 `shouldShow` для каждого гейта . . . 193
10.10 Бытовая аналогия . . . 193
10.11 Что мы пропустили в этой главе . . . 193
10.12 Экраны целиком . . . 194
Ответы к упражнениям . . . 199
Что мы выучили . . . 199
Apple Developer Documentation . . . 200

Глава 11. Privacy blur + Biometric on resume 201
11.1 Жизненный цикл сцены и снимок экрана . . . 201
11.2 Контроллер «жизненного» гейта . . . 202
11.3 NotificationCenter — подписка . . . 203
11.4 Размытие . . . 205
11.5 Возвращение: с биометрией или без . . . 206
11.6 Экран блокировки . . . 207
11.7 Биометрия — LocalAuthentication . . . 209
11.8 Что биометрия гарантирует, а что нет . . . 211
11.9 Проверка в симуляторе . . . 213
11.10 Бытовая аналогия . . . 214
11.11 Краевые случаи . . . 214
11.12 Код целиком . . . 214
11.13 Что можно добавить . . . 219
Ответы к упражнениям . . . 219

Что мы выучили . . . 220
Apple Developer Documentation . . . 220

Глава 12. Todo — UITableView, ячейка-чек, UserDefaults+Codable, dummyjson 222
Перед началом: проект и режим Swift . . . 222
12.1 Модель данных . . . 225
12.2 Группировка по датам . . . 227
12.3 Хранилище — UserDefaults через JSON . . . 229
12.4 Наблюдатели — чтобы перерисовать экран . . . 232
12.5 Первое заполнение из dummyjson . . . 234
12.6 TodoListViewController — список с секциями . . . 236
12.7 Своя ячейка — `TodoCell` . . . 242
12.8 Действия по свайпу . . . 247
12.9 Плавающая кнопка «плюс» . . . 248
12.10 Редактор в выдвижном листе . . . 250
12.11 Бытовая аналогия . . . 256
12.12 Что мы пропустили . . . 256
Ответы к упражнениям . . . 257
Что мы выучили . . . 257
Apple Developer Documentation . . . 258

Глава 13. Notes — UITextView, FileManager, UISearchController 259
13.1 Почему FileManager, а не UserDefaults . . . 261
13.2 Модель заметки . . . 262
13.3 Хранилище через FileManager . . . 263
13.4 Сохранение, загрузка, удаление . . . 264
13.5 Поиск через UISearchController . . . 267
13.6 Группировка «закреплённые / обычные» . . . 272
13.7 Свайпы — закрепить и удалить . . . 273
13.8 Редактор — UITextView во весь экран и автосохранение . . . 274
13.9 Клавиатура — keyboardLayoutGuide и ручной способ . . . 279
13.10 Меню действий в навигационной панели — UIMenu . . . 281
13.11 Поделиться — UIActivityViewController . . . 282
13.12 Бытовая аналогия . . . 283
13.13 Что мы пропустили . . . 283
Ответы к упражнениям . . . 284
Что мы выучили . . . 284
Apple Developer Documentation . . . 285

Глава 14. Calculator — UIStackView grid, state machine, haptics 286
14.1 Архитектура — движок отдельно, экран отдельно . . . 288
14.2 Состояния — как «думает» калькулятор . . . 290
14.3 Главный метод и переходы . . . 291
14.4 Форматирование и точность Double . . . 297
Проверяем движок без симулятора . . . 299
14.5 Интерфейс — сетка из UIStackView . . . 300
14.6 Кнопки через `UIButton.Configuration` . . . 305
14.7 Подсветка активного оператора и отрисовка . . . 307

14.8 Тактильный отклик . . . 308
14.9 Всегда тёмная тема . . . 309
14.10 Бытовая аналогия . . . 310
14.11 Что мы пропустили . . . 310
Ответы к упражнениям . . . 310
Что мы выучили . . . 311
Apple Developer Documentation . . . 312

Глава 15. Weather — open-meteo, pull-to-refresh, skeleton, offline 313
15.1 Почему open-meteo . . . 315
15.2 Модели — DTO и доменные модели . . . 316
15.3 `WeatherCondition` — коды погоды WMO . . . 319
15.4 Сетевой клиент — async/await и ошибки . . . 321
15.5 WeatherStore — кеш и дедупликация . . . 324
15.6 Экран списка — параллельные запросы . . . 326
15.7 Ячейки — три состояния . . . 329
15.8 Skeleton — бегущий блик . . . 334
15.9 Pull-to-refresh . . . 337
15.10 Офлайн-баннер через NWPathMonitor . . . 337
15.11 Детальный экран — блоки и параметры . . . 340
15.12 Почасовая полоса — горизонтальная прокрутка внутри вертикальной . . . 345
15.13 Бытовая аналогия . . . 349
15.14 Что мы пропустили . . . 350
Ответы к упражнениям . . . 350
Что мы выучили . . . 351
Apple Developer Documentation . . . 352

Глава 16. Gallery — UICollectionView compositional, пагинация, photo viewer 353
16.1 picsum.photos — какие данные . . . 355
16.2 API — пагинация . . . 356
16.3 ImageCache — кеш картинок в памяти . . . 357
16.4 Compositional Layout — адаптивная сетка . . . 360
16.5 Пагинация — `willDisplay` . . . 362
16.6 Footer-loader . . . 364
16.7 GalleryCell — асинхронная загрузка картинки . . . 366
16.8 PhotoViewerViewController — pinch-to-zoom . . . 369
16.9 Context menu — долгое нажатие . . . 373
16.10 «Сохранить в Фото» — какое разрешение нужно в действительности . . . 374
16.11 Собираем GalleryViewController . . . 376
16.12 Бытовая аналогия . . . 378
16.13 Что мы пропустили . . . 378
Ответы к упражнениям . . . 379
Что мы выучили . . . 380
Apple Developer Documentation . . . 380

Глава 17. Music Player — AVPlayer, mini-player → full sheet, haptic slider 382
17.1 Треки — публичные mp3 . . . 384
17.2 AVPlayer vs AVAudioPlayer . . . 385

17.3 Один плеер на всё приложение и аудиосессия . . . 385
Аудиосессия . . . 386
Периодический наблюдатель времени . . . 387
17.4 Подписчики — как экраны узнают о переменах . . . 388
17.5 Play / Pause / Skip . . . 389
17.6 `load` — смена трека . . . 390
17.7 Seek — перемотка . . . 392
17.8 MusicLibraryViewController — список + mini-player . . . 392
17.9 NowPlaying — лист с «ручкой» . . . 398
17.10 HapticSlider — ползунок с тактильными щелчками . . . 398
17.11 Scrubbing — перемотка ползунком . . . 400
17.12 Автопереход и чужие паузы . . . 404
17.13 Фоновое воспроизведение — Background Modes . . . 404
17.14 Бытовая аналогия . . . 404
17.15 Что мы пропустили . . . 405
Ответы к упражнениям . . . 405
Что мы выучили . . . 406
Apple Developer Documentation . . . 407

Глава 18. Chat — двусторонние ячейки, keyboardLayoutGuide, typing indicator 409
18.1 Модель сообщения . . . 411
18.2 ChatBot — эхо с задержкой . . . 412
18.3 Optimistic UI — показать сразу, подтвердить потом . . . 412
18.4 Индикатор «бот печатает» . . . 413
18.5 TypingIndicatorCell — три точки в «пузыре» . . . 415
18.6 keyboardLayoutGuide — поле ввода едет за клавиатурой . . . 418
18.7 ComposerView — растущее поле ввода . . . 419
18.8 MessageCell — пузырь слева или справа . . . 422
18.9 Статусы — галочки . . . 425
18.10 Прокрутка к низу . . . 426
18.11 Собираем ChatViewController . . . 426
18.12 Бытовая аналогия . . . 428
18.13 Что мы пропустили . . . 428
Ответы к упражнениям . . . 429
Что мы выучили . . . 430
Apple Developer Documentation . . . 430

Глава 19. Profile / Settings — insetGrouped с разными типами ячеек 432
19.1 Декларативный массив секций . . . 434
19.2 Строка аккаунта — картинка и два текста . . . 436
19.3 Переключатель — UISwitch в accessoryView . . . 437
19.4 Счётчик «+/−» — UIStepper для целых чисел . . . 438
19.5 Ползунок — отдельная ячейка . . . 439
19.6 Выбор из списка — UIMenu на кнопке . . . 442
19.7 Строка-переход — disclosure . . . 443
19.8 Строка-кнопка — обычная и опасная . . . 444
19.9 Опасные действия — подтверждение . . . 445
19.10 Заголовок и пояснение секции . . . 447

19.11 Аватар через PHPicker — без доступа к медиатеке . . . 447
19.12 Собираем ProfileViewController . . . 450
19.13 Бытовая аналогия . . . 452
19.14 Что мы пропустили . . . 452
Ответы к упражнениям . . . 453
Что мы выучили . . . 454
Apple Developer Documentation . . . 454

Глава 20. Custom Tab Bar — три стиля контейнера 456
20.1 Container view controller — паттерн . . . 456
20.2 Tab model — общая для трёх стилей . . . 460
20.3 Style 1 — Floating capsule . . . 461
20.4 Style 2 — Top Tabs . . . 466
20.5 Style 3 — Drawer (боковое меню) . . . 470
Жесты . . . 474
20.6 Все три в одном демо — Segmented control . . . 477
20.7 Бытовая аналогия . . . 478
20.8 Что мы пропустили . . . 479
Ответы к упражнениям . . . 479
Что мы выучили . . . 480
Apple Developer Documentation . . . 481

Глава 21. Complex Layouts — параллакс, sticky header, stretchy 482
21.1 Архитектура . . . 482
21.2 Сетап — два «слоя» . . . 484
21.3 ScrollViewDelegate — главная логика . . . 486
Тянут вниз (offset > 0) . . . 488
Листают вверх (offset < 0) . . . 488
21.4 Шапка не должна ловить касания . . . 488
21.5 StretchyHeaderView — градиент, аватар, текст . . . 489
21.6 Auto Layout внутри шапки . . . 491
21.7 Параллакс — текст «отстаёт» от шапки . . . 492
21.8 Sticky-заголовки разделов — почему отказались . . . 493
21.9 Safe area и навигационная панель . . . 494
21.10 Бытовая аналогия . . . 494
21.11 Липкие заголовки на compositional layout . . . 494
21.12 Когда оно нужно . . . 496
21.13 Что мы пропустили . . . 497
Ответы к упражнениям . . . 497
Что мы выучили . . . 499
Apple Developer Documentation . . . 499

Глава 22. Anatomy — тур по всем гейтам через modal preview 501
22.1 Структура UI . . . 501
22.2 Entry — описание одного гейта . . . 503
22.3 Presenter — общая функция . . . 504
22.4 TableView render . . . 505
22.5 Закрытие — не координатор, а callback . . . 506

22.6 Anatomy — учебный, а не рабочий экран . . . 507
22.7 Скрытый debug-экран — паттерн . . . 507
22.8 Анатомия UIKit: кто от кого наследуется . . . 508
22.9 Бытовая аналогия . . . 510
22.10 Бонус-идеи . . . 510
Ответы к упражнениям . . . 510
Что мы выучили . . . 511
Apple Developer Documentation . . . 511

Глава 23. Cookbook — загрузка 513
23.1 Pull-to-refresh . . . 513
23.2 Infinite scroll (пагинация) . . . 515
23.3 Skeleton-плейсхолдер . . . 517
23.4 Spinner внутри кнопки . . . 520
23.5 Progressive image loading . . . 521
23.6 Toast / Snackbar . . . 523
23.7 Loading overlay (полноэкранный) . . . 525
Ответы к упражнениям . . . 528
Что мы выучили . . . 528
Apple Developer Documentation . . . 529

Глава 24. Cookbook — empty / error / offline states 530
24.1 EmptyStateView — общий компонент . . . 530
24.2 Empty state с call-to-action . . . 533
24.3 Error state — иконка ошибки + retry . . . 534
24.4 Offline баннер — top-of-screen . . . 537
24.5 Skeleton vs spinner vs blank — что когда . . . 539
24.6 Empty state для search . . . 540
24.7 iOS 17+: UIContentUnavailableConfiguration . . . 540
Ответы к упражнениям . . . 541
Что мы выучили . . . 542
Apple Developer Documentation . . . 542

Глава 25. Cookbook — поиск и фильтры 543
Заготовка: список товаров . . . 543
25.1 UISearchController . . . 544
25.2 Debounced search . . . 547
25.3 Scope buttons . . . 548
25.4 Recent searches . . . 550
25.5 Filter chips . . . 551
25.6 Sort sheet . . . 554
25.7 Live API search (autocomplete) . . . 556
25.8 Search highlight . . . 559
Ответы к упражнениям . . . 560
Что мы выучили . . . 560
Apple Developer Documentation . . . 561

Глава 26. Cookbook — навигация и заголовки 562

26.1 Large titles + scroll fade . . . 562
26.2 Tab bar badges . . . 564
26.3 Step indicator . . . 565
26.4 Breadcrumbs . . . 566
26.5 Custom back button . . . 568
26.6 Modal vs push — когда какой . . . 570
26.7 Custom transitions . . . 571
26.8 Title view с двумя строками . . . 573
26.9 Nav bar appearance — кастомизация . . . 574
Ответы к упражнениям . . . 576
Что мы выучили . . . 576
Apple Developer Documentation . . . 576

Глава 27. Cookbook — типы ячеек 578
27.1 Default content configuration . . . 578
27.2 Switch cell . . . 580
27.3 Stepper cell . . . 580
27.4 Slider cell (полный label + slider) . . . 581
27.5 Inline picker (UIMenu) . . . 584
27.6 Disclosure value cell . . . 585
27.7 Multi-line cell . . . 585
27.8 Segmented cell . . . 585
27.9 Picker cell (full width) . . . 587
27.10 Inline editing cell . . . 587
27.11 Variable-height cells . . . 589
27.12 Cell selection style . . . 590
27.13 Avatar в ячейке . . . 590
27.14 Drag handles (editing mode) . . . 591
Ответы к упражнениям . . . 592
Что мы выучили . . . 593
Apple Developer Documentation . . . 593

Глава 28. Cookbook — модалки и листы 594
28.1 Bottom sheet с детентами . . . 594
28.2 Full-screen modal . . . 597
28.3 Page sheet и стиль по умолчанию . . . 598
28.4 Form sheet (iPad) . . . 599
28.5 Popover . . . 599
28.6 UIAlertController — alert . . . 600
28.7 UIAlertController — action sheet . . . 601
28.8 Alert с UITextField . . . 602
28.9 Custom alert (без UIAlertController) . . . 602
28.10 In-app banner . . . 604
28.11 HUD — Heads-Up Display . . . 606
28.12 Modal flow с UIViewControllerTransitioningDelegate . . . 607
Ответы к упражнениям . . . 610
Что мы выучили . . . 610
Apple Developer Documentation . . . 611

Глава 29. Cookbook — жесты **612**
29.1 UIContextMenu — long-press с превью . . . 613
29.2 Swipe actions (table) . . . 614
29.3 Pinch to zoom . . . 615
29.4 Pan для draggable view . . . 617
29.5 Tap gesture . . . 618
29.6 Long press . . . 619
29.7 Swipe gesture (не actions) . . . 620
29.8 Pan-to-dismiss modal . . . 621
29.9 Edge swipe (back gesture) . . . 622
29.10 Конфликт gesture'ов . . . 623
29.11 Gesture inside UITableView . . . 625
29.12 Бытовая аналогия . . . 626
Ответы к упражнениям . . . 626
Что мы выучили . . . 626
Apple Developer Documentation . . . 627

Глава 30. Cookbook — формы и валидация **628**
30.1 Inline validation . . . 628
30.2 Submit button enabled только при валидной форме . . . 630
30.3 Password show/hide . . . 631
30.4 Phone mask +7 (___) ___-__-__ . . . 632
30.5 Password strength . . . 635
30.6 Auto-grow UITextView . . . 636
30.7 Form wizard (multi-step) . . . 638
30.8 Save as draft . . . 640
30.9 Keyboard avoidance . . . 641
30.10 Return key flow . . . 643
30.11 TextField suggestions / textContentType . . . 644
30.12 Disable autocorrect . . . 645
Ответы к упражнениям . . . 646
Что мы выучили . . . 647
Apple Developer Documentation . . . 647

Глава 31. Cookbook — дата, время, деньги **649**
31.1 DateFormatter — база . . . 649
31.2 RelativeDateTimeFormatter — «5 минут назад» . . . 651
31.3 DateComponentsFormatter — длительности . . . 652
31.4 Календарные операции . . . 652
31.5 NumberFormatter — деньги . . . 653
31.6 NumberFormatter — другие стили . . . 655
31.7 Анимация числа . . . 655
31.8 UIDatePicker . . . 657
31.9 Часовые пояса . . . 658
31.10 Тонкости русского плюрала . . . 659
31.11 Размер файлов . . . 660
31.12 Сравнение строк с учётом локали . . . 661
31.13 Деньги: Decimal вместо Double . . . 661

31.14 FormatStyle — современный синтаксис (iOS 15+) . . . 662
Упражнения . . . 663
Ответы к упражнениям . . . 664
Что мы выучили . . . 665
Apple Developer Documentation . . . 665

Глава 32. Cookbook — анимации **666**
32.1 UIView.animate — базовый . . . 666
Кривые: разгон и торможение . . . 667
32.2 Spring — пружина . . . 668
32.3 Последовательность . . . 669
32.4 Появление ячеек . . . 670
32.5 Анимации SF Symbols (iOS 17+) . . . 671
32.6 CABasicAnimation — анимация слоя . . . 671
32.7 CAKeyframeAnimation — несколько точек . . . 672
32.8 Анимация констрейнта . . . 673
32.9 Смена одного view на другой . . . 673
32.10 Конфетти через CAEmitterLayer . . . 674
32.11 Hero-анимация (перелёт элемента между экранами) . . . 675
32.12 Вращение . . . 676
32.13 UIViewPropertyAnimator — анимация под управлением . . . 676
32.14 Бытовая аналогия . . . 677
Упражнения . . . 677
Ответы к упражнениям . . . 678
Что мы выучили . . . 678
Apple Developer Documentation . . . 679

Глава 33. Cookbook — haptics **680**
33.1 Impact (удар) . . . 680
33.2 Notification (успех / предупреждение / ошибка) . . . 681
33.3 Selection (короткий тик) . . . 681
33.4 prepare() — прогрев . . . 682
33.5 Один генератор или новый каждый раз . . . 682
33.6 Свои узоры — Core Haptics (iOS 13+) . . . 683
33.7 Когда haptics **не нужны** . . . 684
33.8 Отдача для ползунка . . . 685
33.9 Отдача при достижении цели . . . 686
33.10 Отдача при перетаскивании . . . 686
Упражнения . . . 687
Ответы к упражнениям . . . 687
Что мы выучили . . . 688
Apple Developer Documentation . . . 688

Глава 34. Cookbook — accessibility **689**
34.1 VoiceOver — accessibilityLabel и hint . . . 689
34.2 Скрыть от VoiceOver . . . 690
34.3 Группировка элементов . . . 690
34.4 Dynamic Type . . . 691

34.5 Reduce Motion — «Уменьшение движения» . . . 692
34.6 Зона нажатия — минимум 44 × 44 точки . . . 692
34.7 Контраст и Increase Contrast . . . 693
34.8 Accessibility traits . . . 694
34.9 accessibilityValue — текущее значение . . . 695
34.10 Проверка через VoiceOver . . . 695
34.11 Accessibility Inspector (Xcode) . . . 695
34.12 Dynamic Type в ячейках таблицы . . . 695
34.13 Картинки . . . 696
34.14 Custom actions — свои действия . . . 696
34.15 Бытовая аналогия . . . 697
Упражнения . . . 697
Ответы к упражнениям . . . 697
Что мы выучили . . . 698
Apple Developer Documentation . . . 698

Глава 35. Cookbook — темы и цвета 700
35.1 Системные цвета (адаптивные) . . . 700
35.2 Tint color — цвет акцента . . . 701
35.3 Свой адаптивный цвет в коде . . . 702
35.4 Цвета в Asset Catalog . . . 703
35.5 Принудительная тема . . . 703
35.6 Реакция на смену темы . . . 703
35.7 Выбор темы в настройках приложения . . . 705
35.8 Палитра бренда . . . 706
35.9 Контраст цветов . . . 707
35.10 Цвета в Storyboard . . . 708
35.11 Плавная смена темы . . . 708
35.12 Тема у дочерних и модальных экранов . . . 708
Упражнения . . . 709
Ответы к упражнениям . . . 709
Что мы выучили . . . 710
Apple Developer Documentation . . . 711

Глава 36. Cookbook — индикаторы статуса 712
36.1 Точка «в сети» . . . 712
36.2 «Печатает…» — три прыгающие точки . . . 714
36.3 Бейдж с числом . . . 715
36.4 Полоса прогресса . . . 717
36.5 Круговой прогресс . . . 717
36.6 Состояния экрана — один enum . . . 720
36.7 Бегущее число . . . 720
36.8 Цвета и иконки статусов . . . 721
36.9 «Подпрыгивание» бейджа при смене числа . . . 722
36.10 Индикатор страниц . . . 722
36.11 Галочки доставки . . . 723
36.12 Качество связи . . . 724
Упражнения . . . 724

Ответы к упражнениям . . . 725
Что мы выучили . . . 725
Apple Developer Documentation . . . 726

Глава 37. Cookbook — photo viewer 727
37.1 Одна страница: фото с зумом . . . 727
37.2 Двойной тап — приблизить к точке . . . 731
37.3 Смахнуть вниз, чтобы закрыть . . . 732
37.4 Кнопка «Поделиться» . . . 733
37.5 Сохранить в Фото . . . 734
37.6 Листание фото — UIPageViewController . . . 735
37.7 Hero-анимация: миниатюра вырастает в фото . . . 737
37.8 Панель навигации над фото . . . 741
37.9 Спрятать панель тапом . . . 742
37.10 Счётчик «N из M» . . . 743
37.11 Подгрузка соседних фото . . . 743
37.12 Что должно быть в хорошем просмотрщике . . . 744
Упражнения . . . 744
Ответы к упражнениям . . . 744
Что мы выучили . . . 745
Apple Developer Documentation . . . 745

Глава 38. Production: screenshots для App Store 747
38.1 Зачем скриншоты . . . 747
38.2 Требования Apple: размеры и количество . . . 747
38.3 Как делать скриншоты . . . 748
38.4 Статус-бар 9:41 . . . 749
38.5 Что Guidelines говорят о содержимом скриншотов . . . 749
38.6 Стиль скриншотов . . . 750
Слабо: голый скриншот . . . 750
Лучше: скриншот + подпись . . . 750
Сильнее всего: рамка устройства + подпись + фон . . . 750
38.7 Что показывать . . . 751
38.8 Локализация скриншотов . . . 751
38.9 Видео-превью (App Preview) . . . 752
38.10 A/B-тесты страницы (Product Page Optimization) . . . 752
38.11 Где обычно ошибаются . . . 753
38.12 Ответы к упражнениям . . . 753
Что мы выучили . . . 754
Apple Developer Documentation . . . 754

Глава 39. Production: App Privacy + Privacy Manifest 755
39.1 Анкета App Privacy в App Store Connect . . . 755
39.2 Privacy Manifest — `PrivacyInfo.xcprivacy` . . . 757
39.3 Required reason API — что это и какие коды выбрать . . . 758
39.4 Собираемые данные в манифесте . . . 761
39.5 Манифесты сторонних SDK . . . 761
39.6 Usage description в Info.plist . . . 762

39.7 App Tracking Transparency (ATT) . . . 762
39.8 Где хранить данные на устройстве . . . 764
39.9 Законы: GDPR, CCPA, казахстанский закон о персональных данных . . . 764
39.10 Что запрещено . . . 765
39.11 App Transport Security . . . 765
39.12 Ответы к упражнениям . . . 766
Что мы выучили . . . 766
Apple Developer Documentation . . . 767

Глава 40. Production: account deletion flow 768
40.1 Что нужно реализовать . . . 768
40.2 Кнопка в Profile . . . 769
40.3 Подтверждение и удаление . . . 769
40.4 Серверная часть: что значит «удалить» . . . 773
40.5 Мягкое и жёсткое удаление . . . 774
40.6 Восстановление в течение отсрочки . . . 774
40.7 Sign in with Apple — отзыв токенов . . . 775
40.8 Подписки . . . 776
40.9 Что Apple не пропустит . . . 777
40.10 Тестирование . . . 778
40.11 Политика конфиденциальности . . . 778
40.12 Ответы к упражнениям . . . 778
Что мы выучили . . . 779
Apple Developer Documentation . . . 779

Глава 41. Production: push notifications + deep links 781
41.1 Как устроена доставка пуша . . . 781
41.2 Настройка в Apple Developer и ключ `.p8` . . . 782
41.3 Две среды APNs: sandbox и production . . . 782
41.4 Xcode: capability . . . 783
41.5 Регистрация и device token . . . 783
41.6 Объяснение перед запросом разрешения . . . 785
41.7 Содержимое пуша (payload) . . . 785
41.8 Получение: приложение открыто и тап по уведомлению . . . 786
41.9 Тихие (фоновые) пуши . . . 787
41.10 Deep links через URL-схему . . . 788
41.11 Universal Links . . . 791
41.12 Что когда использовать . . . 793
41.13 Кнопки в уведомлении: категории . . . 793
41.14 Notification Service Extension . . . 794
41.15 Тестирование пушей . . . 795
41.16 Практика хороших пушей . . . 796
41.17 Ответы к упражнениям . . . 796
Что мы выучили . . . 797
Apple Developer Documentation . . . 798

Глава 42. Production: widgets + App Intents 799
42.1 WidgetKit — что такое виджет . . . 799

42.2 Таргет виджета . . . 800
42.3 Модель, запись в таймлайне и провайдер . . . 800
42.4 Общие данные — App Group . . . 802
42.5 Интерфейс виджета . . . 803
42.6 Как посмотреть виджет . . . 805
42.7 Тап по виджету — deep link . . . 805
42.8 App Intents — действия для Siri и «Команд» . . . 806
42.9 Экран блокировки (iOS 16+) . . . 807
42.10 Интерактивный виджет (iOS 17+) . . . 809
42.11 Live Activities (iOS 16.1+) . . . 810
42.12 Что когда использовать . . . 813
42.13 Приватность виджетов . . . 813
42.14 Нужен ли виджет . . . 813
42.15 Ответы к упражнениям . . . 813
Что мы выучили . . . 814
Apple Developer Documentation . . . 814

Глава 43. Production: accessibility audit перед релизом 815
43.1 Accessibility Nutrition Labels . . . 815
43.2 Полный чек-лист . . . 816
VoiceOver . . . 816
Dynamic Type . . . 816
Контраст . . . 816
Зоны нажатия . . . 817
Движение . . . 817
Не только цветом . . . 817
Клавиатура и Switch Control . . . 817
43.3 Accessibility Inspector . . . 817
43.4 Автоматический аудит в UI-тестах (iOS 17+) . . . 818
43.5 Сценарии для VoiceOver на устройстве . . . 819
43.6 Dynamic Type: крайние размеры . . . 819
43.7 Reduce Motion . . . 820
43.8 Цветовые фильтры . . . 820
43.9 На реальном устройстве . . . 820
43.10 Локализация . . . 820
43.11 Производительность . . . 821
43.12 Финальный чек-лист перед отправкой . . . 821
Сборка . . . 821
Разрешения . . . 821
Страница в App Store . . . 821
Аккаунт и данные . . . 822
Сеть . . . 822
Проверка . . . 822
43.13 После релиза . . . 822
43.14 Ответы к упражнениям . . . 822
43.15 Что мы прошли за книгу . . . 823
Что мы выучили в этой главе . . . 823
Apple Developer Documentation . . . 823

Приложение. Словарь терминов 825
Карта UIKit: классы и наследование . . . 825
Приложение и жизненный цикл . . . 826
Экраны и навигация . . . 828
Раскладка . . . 830
Списки и ячейки . . . 833
Ввод, касания и жесты . . . 835
Графика, цвет и анимации . . . 837
Конкурентность и память . . . 840
Данные, файлы и сеть . . . 842
Дата, время, числа и локализация . . . 844
Безопасность и разрешения . . . 846
Доступность . . . 848
Публикация, пуши и расширения . . . 850
Жаргон и паттерны . . . 853

Введение

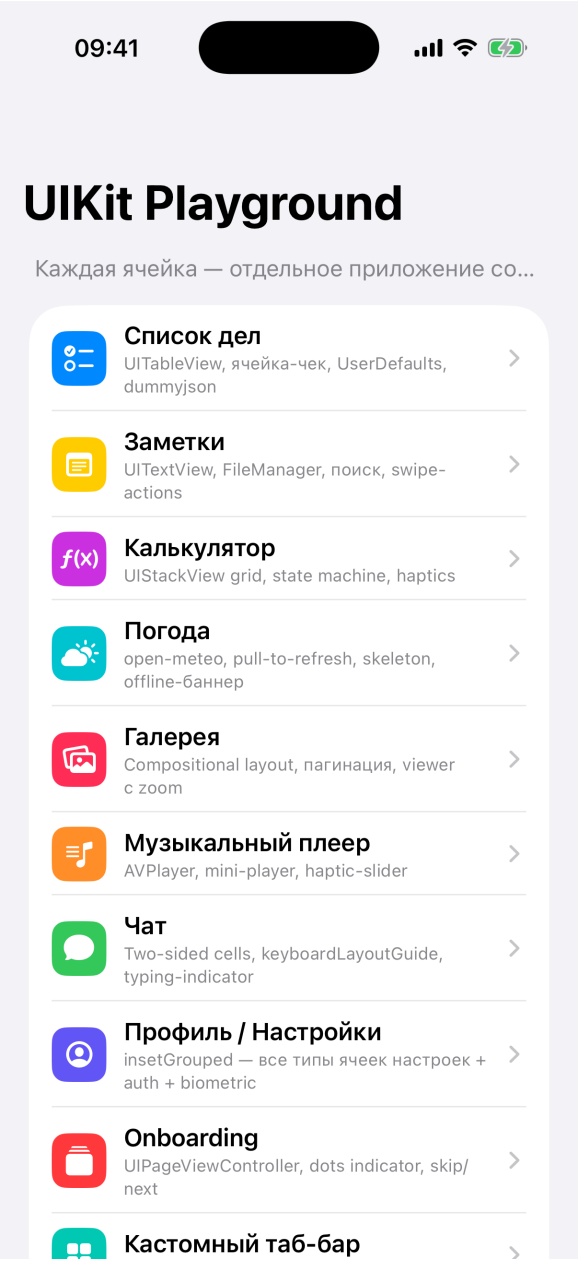


Рис. 1: Лаунчер UIKit Playground

Эта книга — практикум по UIKit. Не словарь и не пересказ документации Apple. Мы собираем в одном Xcode-проекте десять мини-приложений и по дороге разбираем приёмы, из которых состоит настоящее приложение из App Store, а не учебный пример «одна кнопка, один лейбл».

Каждое мини-приложение (дальше — **mini-app**) проходит путь от пустой папки до экрана, который не стыдно показать на собеседовании. Вокруг него — всё то, что обычно остаётся за кадром в учебниках:

- **splash** — заставка, которую приложение показывает само, пока готовится к работе (глава 1);
- **onboarding** — несколько вводных экранов при первом запуске: «вот что умеет приложение» (глава 6);
- **permission primer** — наш собственный экран-подводка, который объясняет, *зачем* нужно разрешение, ещё до системного вопроса «Разрешить доступ к геопозиции?» (глава 7);
- **pull-to-refresh** — «потяни список вниз, чтобы обновить»;
- **skeleton** — серые плашки-заготовки на месте будущего контента, пока данные грузятся;
- **photo viewer** — просмотр фото на весь экран с увеличением пальцами;
- **haptics** — лёгкие вибро-отклики на действия (тот самый «щелчок» при переключении тумблера).

Чем эта книга отличается

У автора есть и другая книга — «ShopApp Beginner»: линейная история «строим один интернет-магазин от Hello World до App Store». Там глава 15 опирается на главу 14, и читать её вразбивку не получится.

Эта книга — **каталог-практикум**. Мини-приложения **независимы**: можно взять «Список дел», потом «Галерею», потом «Чат», и ничего не потеряется. **UI Cookbook** (часть IV) — справочник: пришёл за конкретным приёмом (например, «как сделать pull-to-refresh со своим текстом»), прочитал, забрал в свой проект.

Общее у всех мини-приложений одно — **фундамент** из части I. Это каркас playground'а: лаунчер, окно, координатор запуска. Его мы строим один раз, в главах 1–5, и дальше на нём держится всё остальное.

Кому подойдёт

- **Junior**, который сделал хотя бы один экран на UIKit и хочет двигаться дальше: splash, onboarding, экраны разрешений, тёмная тема, haptics.
- **Middle**, который пишет давно, но хочет освежить «как делается X в 2026 году»: здесь всё на актуальных API (iOS 15+, Xcode 26).
- **Тот, кто возвращается в UIKit после SwiftUI**: увидит современный UIKit-стиль — `@MainActor`, async/await, без старых вложенных completion handler'ов (функций обратного вызова, которые передают в метод, чтобы он «позвонил», когда закончит).

Кому не подойдёт: тем, кто открыл Xcode впервые. Что такое класс, опционал или замыкание, здесь не объясняем — книга предполагает, что Swift ты уже знаешь. Зато каждый термин iOS и UIKit (сцена, окно, view controller, Auto Layout и так далее) объясняем простыми словами там, где он встречается впервые.

Что мы построим — учебный проект `UIKitPlayground`

Один Xcode-проект, который ведёт себя как **playground** (песочница): на главном экране — список мини-приложений. Этот экран мы называем **лаунчером** — он «запускает» остальные, как рабочий стол iPhone запускает приложения. Тап по строке — и выбранное mini-app стартует так, будто это отдельное приложение из App Store: со своим splash, своим фирменным цветом, своими экранами запуска. Встряхнул телефон (в симуляторе — ^⌘Z) — вернулся в лаунчер.

Экраны, через которые mini-app проходит до основного экрана (onboarding, вход в аккаунт, проверка возраста и т. п.), мы называем **гейтами** (от англ. *gate* — ворота, проходная): пользователь проходит их по очереди, как турникеты на входе в офис. Основной экран, ради которого mini-app существует, — **main**.

Это не монолит, а **полигон**. Каждое mini-app живёт в своей папке (`Apps/Todo/`, `Apps/Notes/`…), общий слой запуска — в `App/`, общие цвета и отступы — в `Common/`.

Вот что лежит в проекте после части I (фундамента):

```
UIKitPlayground/
├── App/                                  ← фундамент (главы 1–5)
│   ├── AppDelegate.swift                 ← события процесса (глава 5)
│   ├── SceneDelegate.swift               ← создаёт окно и лаунчер (глава 5)
│   ├── PlaygroundWindow.swift            ← окно, которое ловит встряхивание (глава 3)
│   ├── AppManifest.swift                 ← описание одного mini-app (глава 2)
│   ├── AppRegistry.swift                 ← список всех mini-app (глава 2)
│   ├── PlaceholderViewController.swift ← заглушка «ещё не готово» (глава 2)
│   ├── AnimatedSplashViewController.swift ← наш splash (глава 1)
│   ├── BootCoordinator.swift             ← ведёт mini-app через гейты (глава 4)
│   └── AppListViewController.swift       ← лаунчер (глава 4)
├── Common/
│   └── DesignSystem.swift                ← общие цвета и отступы (глава 5)
├── Resources/
│   ├── Assets.xcassets
│   └── Base.lproj/LaunchScreen.storyboard  ← системная заставка (глава 1)
└── Info.plist
```

Дальше проект растёт так. Часть II добавляет гейты — по файлу-два на главу (где их держать, решаешь сам; удобно завести папку `Gates/`) — и вписывает их в `BootCoordinator`. Часть III добавляет папки `Apps/<Имя>/` и в `AppRegistry` меняет заглушку «СКОРО» на настоящее mini-app. Например, «Список дел» из главы 12 выглядит так:

```
Apps/Todo/
├── Models/   Todo.swift, TodoStorage.swift, TodoAPI.swift
└── UI/       TodoListViewController.swift, TodoCell.swift, …
```

Каждый файл фундамента в книге приведён **целиком**, так что проект собирается и запускается уже после главы 5 — с лаунчером, splash'ем и заглушками на месте будущих мини-приложений.

0.1 Создаём проект в Xcode 26

Эти шаги нужны один раз, до главы 1. Они выглядят скучно, но от них зависит, в каком **режиме языка** будет собираться весь код книги, — поэтому разберём их подробно.

Шаг 1. Шаблон. File → New → Project → iOS → **App**. В окне параметров:

- Product Name: `UIKitPlayground`;
- **Interface: Storyboard** (по умолчанию стоит SwiftUI — поменяй!);
- Language: Swift;
- Testing System: None (тесты в этой книге не пишем).

Шаблон Xcode 26.5 с такими параметрами создаёт:

- `AppDelegate.swift` и `SceneDelegate.swift` — делегаты приложения и сцены (что это, разберём в главе 5);
- `ViewController.swift` и `Main.storyboard` — стартовый экран, нарисованный в Interface Builder;
- `LaunchScreen.storyboard` — системная заставка (глава 1);
- `Assets.xcassets` — каталог картинок и цветов;
- `Info.plist` — файл настроек приложения, который iOS читает до запуска твоего кода (подробнее — в главе 5).

Шаг 2. Убираем `Main.storyboard`. В этой книге все экраны создаются кодом, а storyboard остаётся только для `LaunchScreen`. Поэтому:

1. Удали `ViewController.swift` и `Main.storyboard` (Move to Trash).
2. Открой `Info.plist` → Application Scene Manifest → Scene Configuration → Application Session Role → Item 0 и удали строку **Storyboard Name** (ключ `UISceneStoryboardFile`). Строку **Delegate Class Name** оставь — по ней iOS найдёт наш `SceneDelegate`.
3. Target → Build Settings, найди поиском `UIMainStoryboardFile` (в списке — «UIKit Main Storyboard File Base Name») и удали значение `Main`.

Если пропустить шаг 2.2 или 2.3, iOS при запуске попробует загрузить `Main.storyboard`, не найдёт его и приложение упадёт. Мы проверили оба случая на iOS 26.5: с оставшимся `UIMainStoryboardFile` — с ошибкой «Could not find a storyboard named 'Main' in bundle», с оставшимся `UISceneStoryboardFile` — тоже сразу при запуске.

Шаг 3. Минимальная версия iOS. Шаблон ставит минимальную версию, равную самой свежей (в Xcode 26.5 — iOS 26.5). Книга рассчитана на **iOS 15.0**: Target → General → Minimum Deployments → iOS **15.0**. Эта настройка называется **deployment target** — самая старая версия iOS, на которую можно установить приложение. Всё, что появилось в iOS позже 15-й, в книге помечено `@available` или обёрнуто в `if #available(...)`.

Шаг 4. Режим языка. Target → Build Settings, раздел **Swift Compiler – Language** / **Swift Compiler – Concurrency**. Вот что там ставит шаблон Xcode 26.5 и что ставим мы:

Настройка (ключ)	Шаблон Xcode 26.5	В этой книге
Swift Language Version (`SWIFT_VERSION`)	Swift 5	**Swift 6**
Default Actor Isolation (`SWIFT_DEFAULT_ACTOR_ISOLATION`)	MainActor	MainActor

Настройка (ключ)	Шаблон Xcode 26.5	В этой книге
Approachable Concurrency (`SWIFT_APPROACHABLE_CONCURRENCY`)	Yes	Yes

Меняем одно: **Swift Language Version** → **Swift 6**. Две остальные настройки шаблон уже выставил так, как нам нужно.

0.2 Что значат эти три настройки

Все три — про **конкурентность**, то есть про код, который может выполняться одновременно в разных потоках. Разберём на бытовом примере.

Главный поток (main thread) — это единственный «сотрудник», которому разрешено трогать интерфейс: менять текст лейбла, двигать кнопку, показывать экран. Если интерфейс поменяет кто-то другой (фоновый поток, который, скажем, скачивал картинку), приложение может отрисовать мусор или упасть — причём не всегда, а раз в сто запусков. Такая ошибка называется **гонкой данных** (data race), и ловить её отладчиком — мучение.

Актор (actor) — объект, у которого есть «своя очередь»: к его данным обращаются строго по одному. `MainActor` — особый актор, чья «очередь» — это и есть главный поток. Пометка `@MainActor` на классе или функции значит: «этот код выполняется только на главном потоке». Весь UIKit (`UIView`, `UIViewController`, `UILabel`…) Apple уже пометила `@MainActor`.

Default Actor Isolation = MainActor. Обычно код без пометок ни к какому актору не привязан (*nonisolated*), и компилятор не знает, на каком потоке он окажется. С этой настройкой всё, что ты объявил в модуле без явной пометки — свои классы, структуры, функции, — считается `@MainActor`. Раз по умолчанию всё на главном потоке, гонки «обновил лейбл из фона» просто негде взяться. Если какой-то код нужно выполнять вне главного потока, его явно помечают `nonisolated` — это разберём в главе 5.

Swift Language Version = Swift 6. В режиме Swift 5 почти все нарушения правил конкурентности — только **предупреждения**: проект соберётся, даже если ты трогаешь интерфейс из фона. В режиме Swift 6 большинство таких нарушений — **ошибки**, и проект не соберётся, пока их не исправишь. Мы выбираем строгий режим: пусть компилятор ловит ошибки до запуска, а не пользователь — после.

Approachable Concurrency = Yes («доступная конкурентность») включает набор послаблений, которые делают правила Swift 6 предсказуемее. Главное для нас: асинхронная функция без пометок выполняется там же, где её вызвали, а не «где-то в фоне». Вызвал её с главного потока — она и работает на главном, пока сама явно не уйдёт в фон.

> **В каком режиме код этой книги.** Все листинги проверены компилятором Swift 6.3 (Xcode 26.5) в режиме **Swift 6** с **Default Actor Isolation = MainActor** и **Approachable Concurrency = Yes**, с минимальной версией **iOS 15.0**. Если оставить шаблонный Swift 5, код тоже соберётся, но часть ошибок, о которых говорит книга, превратится в предупреждения — и их легко пропустить.

Стек, на котором пишем

- **Swift 6** в режиме, описанном выше (Swift 6 + MainActor по умолчанию + Approachable Concurrency).
- **UIKit**, экраны — **кодом**. Storyboard используется ровно для одной вещи — `LaunchScreen`: системная заставка, которую iOS показывает до запуска нашего кода, не умеет исполнять Swift (подробно — в главе 1).
- **Auto Layout** — система правил, по которым UIKit сам вычисляет размеры и положение элементов («лейбл на 20 точек ниже иконки, по центру экрана»). Каждое такое правило называется **constraint** (ограничение). В коде книги ты увидишь их постоянно — начиная с главы 1.
- **iOS 15.0+** как минимальная версия. Более свежие API помечаем `@available`.
- **Никакого SwiftUI**. Это книга про UIKit.

Как читать

- **Глава 0. Карта UIKit** — прочитай **первой**, даже если UIKit тебе не нов. В ней разложено, кто от кого наследуется (`NSObject` → `UIResponder` → `UIView` → `UIControl` → `UIButton`…), что каждый «этаж» этой лестницы даёт классу, и чем view controller отличается от view, а `UIBarButtonItem` — от кнопки. Остальные главы ссылаются на неё, когда поведение класса объясняется его родителем.
- **Часть I. Фундамент** (главы 1–5) — читай **по порядку**. Вместе они дают работающий каркас, на котором держится всё остальное.
- **Часть II. Launch-гейты** (главы 6–11) — тоже по порядку, но гейты, которые тебе не нужны, можно пропускать (например, проверка возраста нужна немногим).
- **Часть III. Мини-приложения** (главы 12–22) — **выбирай**. Не нужно читать все подряд: берёшь то, что хочешь сделать. Десять mini-app и в конце — тур «Anatomy» по всем гейтам.
- **Часть IV. UI Cookbook** (главы 23–37) — **справочник**. Открываешь, когда нужен конкретный приём.
- **Часть V. Production checklist** (главы 38–43) — перед первым релизом в App Store: скриншоты, приватность, удаление аккаунта, пуши и ссылки, виджеты, аудит доступности.
- **Приложение. Словарь терминов** — все термины книги с короткими определениями и номером главы, где термин разобран подробно. Встретил незнакомое слово в середине книги — загляни туда.

Условные обозначения

> **Идея.** Самое важное вынесено в такие цитаты-врезки.

```
let example = "это код, который можно (и нужно) набрать"
```

Упражнение 1.1. Маленькое задание: первое число — номер главы, второе — номер задания в ней. Делай его **до того**, как читать дальше. Ответы и решения — в конце главы, в разделе «Ответы к упражнениям». В методической части V задания без номеров, но ответы к ним лежат там же.

Что мы выучили — краткий список фактов в конце каждой главы.

Apple Developer Documentation — в самом конце главы: ссылки на первоисточники по всему, что в ней встретилось.

Спасибо

Если ты читаешь эту книгу — спасибо. Нашёл опечатку или непонятное место — открой issue в репозитории книги sirserik/uikit-playground-book.

Поехали.

Apple Developer Documentation

- The Swift Programming Language — официальный справочник по языку. Книга предполагает, что базовые конструкции отсюда уже знакомы.
- Concurrency — Swift Book — акторы, `async`/`await`, изоляция: то, что стоит за настройками из раздела 0.1.
- UIKit — каталог фреймворка, на котором построены все mini-app.
- Human Interface Guidelines — продуктовые правила Apple. На них опираемся, когда обсуждаем UX-решения (splash, onboarding, permission primer).
- Xcode — справочник по самой IDE, настройкам сборки и схемам.

→ Глава 1. LaunchScreen vs AnimatedSplash — два экрана, оба «splash»

Глава 0. Карта UIKit — кто от кого наследуется и что это даёт

Эта глава короткая, и её задача одна: чтобы в любой следующей главе ты понимал, почему код написан именно так. Почему кнопке можно сказать `addAction`, а лейблу — нет. Почему `UIBarButtonItem` нельзя положить на экран через `addSubview`. Почему у экрана (`UIViewController`) нет `frame`, а у его `view` — есть. Почему таблица умеет `contentOffset`, хотя она «список».

Ответ на все эти «почему» один: **класс умеет то, что умеют его предки, плюс своё**. Почти весь UIKit стоит на лестнице из четырёх ступеней:

```
NSObject                  ← любой объект: его можно сравнить и напечатать
 └─ UIResponder           ← умеет получать события (касания, клавиатуру)
     └─ UIView            ← прямоугольник на экране: frame, subviews, слой
         └─ UIControl     ← прямоугольник, который умеет «нажаться»
             └─ UIButton, UISwitch, UISlider, UITextField, …
```

Если знаешь, на какой ступени стоит класс, ты знаешь, что с ним можно делать, ещё до того, как открыл документацию. Карту ниже мы не вспоминали по памяти, а распечатали в симуляторе iOS 26.5 — как это сделать самому, показано в конце главы.

> **Как читать главу.** Не заучивай. Прочитай один раз, запомни четыре ступени, таблицу «кто контрол, а кто нет» из раздела 0.4 и `SceneDelegate` из раздела 0.7. Остальное — справка: главы книги будут ссылаться сюда словами «глава 0, раздел 0.N», и тогда вернёшься.

0.1 Карта

Строка `A → B` читается «A наследует B».

```
NSObject
├── UIResponder                  получает события
│   ├── UIApplication            приложение, один на процесс
│   ├── UIScene → UIWindowScene  сцена: один экземпляр интерфейса
│   ├── UIViewController         экран (не view!)
│   │   ├── UINavigationController, UITabBarController
│   │   ├── UIPageViewController, UISplitViewController
│   │   ├── UIAlertController, UIActivityViewController
│   │   ├── UIImagePickerController → UINavigationController
```

```
│   │   └── UITableViewController, UICollectionViewController
│   └── UIView                    прямоугольник на экране
│       ├── UIWindow              окно — тоже view
│       ├── UILabel, UIImageView, UIStackView
│       ├── UIActivityIndicatorView, UIProgressView, UIPickerView
│       ├── UISearchBar, UINavigationBar, UITabBar, UIToolbar
│       ├── UITableViewCell
│       ├── UICollectionReusableView → UICollectionViewCell
│       ├── UIScrollView
│       │   ├── UITableView
│       │   ├── UICollectionView
│       │   └── UITextView
│       └── UIControl             умеет target-action
│           ├── UIButton, UISwitch, UISlider, UIStepper
│           ├── UISegmentedControl, UIPageControl
│           ├── UIDatePicker, UIColorWell, UIRefreshControl
│           └── UITextField → UISearchTextField
│
├── UIBarItem → UIBarButtonItem, UITabBarItem     описание кнопки панели
├── UINavigationItem                              как экран виден в панели
├── UIGestureRecognizer → UITap…, UIPan…, UIPinch…, UILongPress…
├── UIPresentationController → UISheet…, UIPopover…
├── UIMenuElement → UIAction, UIMenu
├── UIFeedbackGenerator → UIImpact…, UINotification…, UISelection…
├── UICollectionViewLayout → …FlowLayout, …CompositionalLayout
├── NSLayoutConstraint, UILayoutGuide
├── UIColor, UIFont, UIImage, UIBezierPath
├── UITraitCollection, UIScreen, UIDevice, UISceneSession
├── CALayer → CAGradientLayer, CAShapeLayer, CATextLayer
└── CAAnimation → CABasicAnimation, CAKeyframeAnimation, CATransition
```

Карта делится пополам. **Верхняя половина** — всё, что растёт из `UIResponder`: оно получает события, а ниже `UIView` ещё и видно на экране. **Нижняя половина** — объекты, которые наследуют `NSObject` напрямую. Они нужны интерфейсу, но **не view**: у них нет `frame`, их нельзя `addSubview`, они не ловят касания. Большая часть ошибок «почему не компилируется» в UIKit — попытка обращаться с нижней половиной как с верхней.

Что добавляет каждая ступень, одной строкой:

Ступень	Что даёт	Самое нужное
`NSObject`	объект можно сравнить (==), напечатать, позвать по имени через `#selector`	метод для `#selector` помечай `@objc`
`UIResponder`	получает события и передаёт дальше, если не обработал	`becomeFirstResponder()` поднимает клавиатуру
`UIView`	прямоугольник: `frame`, `addSubview`, слой `layer`, Auto Layout	`translatesAutoresizingMaskIntoConstraints = false`
`UIControl`	«нажатие» и состояния	`addAction(_:for:)`, `isEnabled`

Ступень	Что даёт	Самое нужное
конкретный класс	содержимое	`text` у лейбла, `image` у картинки

Из `NSObject` на практике нужны две вещи. Первая: метод, который передаёшь через `#selector`, помечай `@objc` — иначе компилятор скажет «instance method 'tap()' is not exposed to Objective-C». Вторая: класс, который становится делегатом таблицы или поля, должен наследовать `NSObject` (`final class Source: NSObject, UITableViewDataSource`), иначе «should inherit 'NSObject' instead». Экранам и view это не нужно — они и так наследники `NSObject`.

Бытовая аналогия

Должностные инструкции в компании. Любой **сотрудник** (`NSObject`) имеет пропуск, по которому его можно отличить от других. Сотрудник **приёмной** (`UIResponder`) обязан принимать обращения и передавать выше, если вопрос не его. Сотрудник **за стойкой в зале** (`UIView`) — тот же приёмщик, но у него есть рабочее место, которое видно посетителям. **Кассир** (`UIControl`) — работник за стойкой, который по нажатию кнопки выдаёт результат.

Кассир умеет всё, что умеют трое выше, плюс своё. А **табличка на двери** (`UIBarButtonItem`) — не сотрудник: у неё нет рабочего места и она ничего не принимает. Дверь (`UINavigationBar`) показывает табличку; хочешь другую надпись — меняешь табличку, а не дверь.

0.2 `UIResponder` — кто получает события

Ответчик (responder) — объект, который умеет получать касания, клавиатуру, встряхивание. На этой ступени стоят приложение, сцена, окно, все view и все экраны, а ещё `AppDelegate` и `SceneDelegate`, которые шаблон Xcode объявляет наследниками `UIResponder`.

Два факта, которые нужны в работе.

Цепочка ответчиков. Событие, которое объект не обработал, идёт дальше: view → его экран → superview → … → окно → сцена → приложение → `AppDelegate`. Поэтому в главе 3 окно ловит встряхивание, которое не обработал ни один экран: до окна оно доходит само. Распечатка цепочки от кнопки в нашем приложении — в главе 3.

Первый ответчик. Для событий без координат — клавиатуры, встряхивания — цепочка начинается с **первого ответчика**: обычно это текстовое поле, в котором мигает курсор. `textField.becomeFirstResponder()` поднимает клавиатуру, `resignFirstResponder()` или `view.endEditing(true)` — убирает. По умолчанию стать первым ответчиком могут `UITextField` и `UITextView`; у обычного `UIView` и у экрана `canBecomeFirstResponder` возвращает `false`.

Чего на этой ступени **нет**: слоёв `CALayer`, распознавателей жестов, `UIBarButtonItem`. Они участвуют в обработке событий, но сами события не получают — получает view, к которой они прикреплены.

0.3 `UIView` — прямоугольник на экране

Всё, что ты видишь в приложении, — view. Эта ступень даёт четыре вещи: геометрию, дерево, раскладку и слой. Разберём с числами из симулятора.

`frame`, `bounds`, `center`

```
let box = UIView(frame: CGRect(x: 10, y: 20, width: 100, height: 50))
print(box.frame)    // (10.0, 20.0, 100.0, 50.0)
print(box.bounds)   // (0.0, 0.0, 100.0, 50.0)
print(box.center)   // (60.0, 45.0)
```

- **`frame`** — где view лежит **внутри родителя**: угол в (10, 20), размер 100 × 50.
- **`bounds`** — тот же прямоугольник **в собственных координатах**: начало всегда (0, 0). Когда раскладываешь subviews внутри view, нужен именно `bounds`.
- **`center`** — центр `frame`: 10 + 100 / 2 = 60, 20 + 50 / 2 = 45.

Единица — **точка**, не пиксель: у iPhone 16 экран 393 × 852 точки при масштабе 3 (1179 × 2556 пикселей). Один код даёт одинаковый размер на экранах разной плотности.

Одно правило про `transform`: после поворота или масштаба `frame` Apple объявляет **неопределённым** — не читай его и не меняй, двигай view через `center`, меняй размер через `bounds` (главы 18, 32).

Упражнение 0.1. У view `frame = (40, 100, 200, 80)`. Чему равны `bounds` и `center`? Если присвоить `center = (100, 100)`, каким станет `frame`? Ответ — в конце главы.

Дерево: `addSubview`

View вкладываются друг в друга: у каждого одно `superview` и массив `subviews`. Корень дерева — окно. Последний добавленный subview рисуется **поверх** предыдущих (мы проверили: из двух одинаковых view касание достаётся тому, что добавили вторым). Поменять порядок — `bringSubviewToFront(_:)`, `sendSubviewToBack(_:)`.

`addSubview` принимает только `UIView`. Попробуй положить элемент панели:

```
view.addSubview(UIBarButtonItem())
// error: cannot convert value of type 'UIBarButtonItem'
//        to expected argument type 'UIView'
```

Так компилятор напоминает про нижнюю половину карты: у `UIBarButtonItem` нет `frame`, рисовать нечего.

Перевести точку из координат одной view в координаты другой — `convert(_:to:)`; `nil` вместо view означает координаты окна (глава 37).

Раскладка: что такое Auto Layout и почему он так называется

Раскладка (layout) — это ответ на вопрос «где и какого размера каждый view». Самый прямой способ ответить — задать `frame` числами:

```
titleLabel.frame = CGRect(x: 16, y: 120, width: 361, height: 24)
```

Цифры подобраны под iPhone 16 шириной 393 точки. Поверни телефон — ширина станет 852, и лейбл займёт меньше половины экрана. Запусти на iPhone SE шириной 375 — лейбл вылезет за правый край. Пользователь увеличит шрифт в настройках — текст не влезет в 24 точки высоты. С каждым новым экраном Apple такие числа приходится пересчитывать, и их десятки на каждом экране приложения.

Auto Layout — «автоматическая раскладка»: вместо чисел ты пишешь **правила**, а числа UIKit считает сам, каждый раз заново — при повороте, на другом устройстве, при смене шрифта. Отсюда и название: *layout* — раскладка, *auto* — её делают за тебя. Правило называется **constraint** (ограничение): «левый край лейбла на 16 точек правее правого края иконки», «лейбл по вертикали по центру иконки», «правый край лейбла не дальше края экрана». Из набора таких ограничений получается система уравнений, UIKit решает её и выставляет `frame` всем view разом. Ты описываешь **отношения** между view, а не их координаты, — и отношения верны на любом экране.

Бытовой пример: инструкция «повесь картину на 20 сантиметров ниже потолка, по центру стены» работает в любой комнате, а «повесь на высоте 2 метра 30 сантиметров, в 1 метре 85 сантиметрах от левого угла» — только в той, где её измеряли.

Так выглядят те же правила в коде:

```
label.translatesAutoresizingMaskIntoConstraints = false
view.addSubview(label)
NSLayoutConstraint.activate([
    label.leadingAnchor.constraint(equalTo: icon.trailingAnchor, constant: 16),
    label.centerYAnchor.constraint(equalTo: icon.centerYAnchor),
])
```

Что здесь важно знать.

- **`translatesAutoresizingMaskIntoConstraints = false`** — первая строка для любого view, которое ты расставляешь правилами в коде. По умолчанию свойство `true`, и UIKit сам создаёт правила из `frame`; они столкнутся с твоими, и в консоли появится «Unable to simultaneously satisfy constraints». Ставь его subview, а не корневому `view` экрана. Элементам `UIStackView` ставить не нужно — стек делает это сам.
- Правило — объект `NSLayoutConstraint` (не view). Пока не активировано, ни на что не влияет.
- **Приоритеты**: 1000 — обязательно, 750 — высокий, 250 — низкий. Когда правила спорят, побеждает более важное.
- **Собственный размер.** `UILabel`, `UIButton`, `UISwitch` сами знают, сколько места хотят (`intrinsicContentSize`): лейбл «Привет» 17-м шрифтом просит 58 × 20 точек. Поэтому лейблу хватает правил на позицию, без ширины и высоты. У голого `UIView` такого размера нет.
- **Направляющие** — невидимые прямоугольники с якорями: `safeAreaLayoutGuide` (не под вырезом и индикатором Home: на iPhone 16 это 59 точек сверху и 34 снизу), `layoutMarginsGuide` (поля: 8 точек у обычного view, у корневого view экрана — 16 по бокам плюс safe area), `keyboardLayoutGuide` (iOS 15, верх клавиатуры; глава 30).

Когда размеры известны. Не сразу. UIKit считает раскладку один раз перед отрисовкой кадра, поэтому в `viewDidLoad` и сразу после `addSubview` ширина subview равна **нулю**. Окончательные размеры — в `viewDidLayoutSubviews` экрана или в `layoutSubviews` самого view. Если `frame` нужен прямо сейчас, например внутри анимации, — `view.layoutIfNeeded()` (глава 32).

Слой

У каждого view ровно один слой `CALayer` (`view.layer`) — то, что железо реально рисует. Скругление, рамка и тень живут в нём: `layer.cornerRadius`, `layer.borderWidth`, `layer.shadowOpacity`. Два правила: `layer.masksToBounds` и `view.clipsToBounds` — одно и то же свойство; тень рисуется **за** краем слоя, поэтому вместе с обрезкой по краю исчезает (решение — глава 32). Слой не view: касаний не получает, Auto Layout его не видит, размер задаёшь сам в `layoutSubviews` (глава 21).

Кто получает касание

Палец касается экрана — UIKit ищет самый верхний view под ним. Три условия выключают view из поиска вместе со всеми потомками: `isHidden`, `alpha < 0.01`, `isUserInteractionEnabled == false`.

Два класса выключены из касаний **по умолчанию**: у `UILabel` и `UIImageView` `isUserInteractionEnabled == false`. Поэтому жест на лейбле или картинке не сработает, пока не написать `label.isUserInteractionEnabl = true` (главы 14, 22, 29).

Инициализаторы: когда писать свой

Инициализатор — это код, который выполняется один раз при создании объекта и заполняет всё, без чего объект не имеет смысла. Когда ты пишешь `UILabel()`, `UIView(frame: …)` или `HelloViewController()`, ты вызываешь инициализатор. Бытовой пример: бланк заявления. Чтобы бланк стал заявлением, в нём обязаны быть заполнены ФИО и дата — это инициализатор; остальное (телефон, комментарий) можно вписать потом — это обычные свойства.

У классов UIKit свои инициализаторы уже есть, и в большинстве случаев **свой писать не нужно**: создаёшь `UIViewController()` без аргументов, а всё содержимое настраиваешь в `viewDidLoad`. Так сделан `HelloViewController` в разделе 0.7.

Свой инициализатор нужен ровно тогда, когда объекту **на входе** нужны данные, без которых он не может работать: экран сплэша должен знать, какое mini-app показывать, карточка — какой заголовок рисовать. Тогда данные принимают параметром и кладут в константу:

```
final class Card: UIView {
    private let titleLabel = UILabel()

    init(title: String) {
        super.init(frame: .zero)      // сначала создаём «родительскую часть»
        titleLabel.text = title       // потом настраиваем своё
        addSubview(titleLabel)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) is not supported") }
}
```

```
let card = Card(title: "Заголовок")
```

Три правила, по которым это пишется.

Сначала `super.init`, потом своё. Наследник обязан дать родителю создать его часть объекта. У `UIView` для этого есть `init(frame:)` — передаём `.zero`, потому что размер всё равно задаст Auto Layout. У `UIViewController` родительский инициализатор — `init(nibName: nil, bundle: nil)` («без xib-файла»). Только после `super.init` можно обращаться к `self` и к своим свойствам, кроме присваивания констант с `let`, которые Swift разрешает заполнить до `super.init` — так в главе 1 `AnimatedSplashViewController` сохраняет `manifest` до вызова родителя.

Откуда берётся `init?(coder:)`. У `UIView` и `UIViewController` два инициализатора: обычный (`init(frame:)` / `init(nibName:bundle:)`) и `init?(coder:)` — им UIKit воссоздаёт объект из storyboard или xib. Пока ты не объявил ни одного своего инициализатора, оба достаются по наследству, и писать ничего не надо. Как только объявил свой, Swift перестаёт наследовать родительские, а `init?(coder:)` помечен `required` — «у каждого наследника он обязан быть». Отсюда ошибка компилятора:

```
error: 'required' initializer 'init(coder:)' must be provided
       by subclass of 'UIView'
```

В книге storyboard для экранов не используется, поэтому реализовать его по-настоящему нечем, и везде стоит одна и та же строка-заглушка:

```
    required init?(coder: NSCoder) { fatalError("init(coder:) is not supported") }
```

`fatalError` честно говорит: «из storyboard меня создавать нельзя». До этой строки дело никогда не дойдёт — мы не создаём экраны из storyboard.

Как понять, нужен ли свой. Задай вопрос: «может ли объект появиться без этого значения?». Если нет (экран ничего не покажет без манифеста, ячейка-карточка — без данных) — параметр инициализатора. Если да (цвет фона, текст лейбла, который потом поменяется) — обычное свойство или `viewDidLoad`. Второй случай встречается чаще, и в книге большинство экранов создаются просто `ИмяЭкрана()`.

0.4 `UIControl` — view, которое умеет «нажаться»

`UIControl` добавляет к view **target-action**: ты не следишь за касаниями сам, а говоришь «при таком событии вызови вот это»:

```
button.addAction(UIAction { [weak self] _ in self?.save() }, for: .touchUpInside)
```

`addAction` появился в iOS 14; старый способ — `addTarget(self, action: #selector(didTap), for:)` с `@objc`-методом. События (`UIControl.Event`):

Событие	Когда	У кого
`.touchUpInside`	палец поднят внутри — обычный «тап»	`UIButton`

Событие	Когда	У кого
`.valueChanged`	значение изменилось	`UISwitch`, `UISlider`, `UIStepper`, `UISegmentedControl`, `UIDatePicker`, `UIPageControl`, `UIRefreshControl`
`.editingChanged`	изменился текст	`UITextField`
`.primaryActionTriggered`	«основное действие» любого контрола	все; к нему привязан `UIButton(primaryAction:)`

Плюс состояния: `isEnabled` (выключенный контрол серый и не реагирует), `isSelected`, `isHighlighted`.

Кто контрол, а кто нет

Самая практичная таблица главы. Если класс — наследник `UIControl`, у него есть `addAction`. Если нет — о событии сообщает **делегат**, уведомление или жест.

Класс	Ступень	Как узнать о событии
`UIButton`	`UIControl`	`.touchUpInside`
`UISwitch`, `UISlider`, `UIStepper`, `UISegmentedControl`, `UIDatePicker`, `UIPageControl`	`UIControl`	`.valueChanged`
`UITextField`	`UIControl`	`.editingChanged` или `UITextFieldDelegate`
`UIRefreshControl`	`UIControl`	`.valueChanged` — список потянули
`UILabel`, `UIImageView`	`UIView`	не контрол: `UITapGestureRecognizer` + `isUserInteractionEnabled = true`
`UISearchBar`	`UIView`	не контрол: `UISearchBarDelegate`
`UITextView`	`UIScrollView`	не контрол: `UITextViewDelegate`
`UIPickerView`	`UIView`	не контрол: `UIPickerViewDelegate`
`UIProgressView`, `UIActivityIndicatorView`	`UIView`	только показывают
`UIBarButtonItem`, `UITabBarItem`	`UIBarItem` (не view)	`primaryAction` или `target/action`

Попытка повесить действие на лейбл не компилируется:

```
label.addAction(UIAction { _ in }, for: .touchUpInside)
// error: value of type 'UILabel' has no member 'addAction'
```

Обратный случай — `UIRefreshControl`: внешне крутилка над списком, но по карте контрол, и подписка такая же, как на тумблер (глава 23).

Упражнение 0.2. Не глядя в таблицу: у каких из классов есть `addAction(_:for:)` — `UISlider`, `UISearchBar`, `UIPageControl`, `UIImageView`, `UIDatePicker`, `UITextView`? Для остальных назови, как узнать об изменении. Ответ — в конце главы.

0.5 `UIScrollView` и его наследники

`UIScrollView` — view, содержимое которого больше него самого и сдвигается пальцем. На этой ступени стоят **три** класса, которыми ты будешь пользоваться постоянно:

```
UIScrollView
├── UITableView       список строк
├── UICollectionView  сетка или лента
└── UITextView        многострочный текст
```

Из наследования следует то, что иначе выглядит магией:

- у таблицы и коллекции есть `contentOffset` (на сколько прокручено), `contentInset` (поля — ими поднимают низ списка над клавиатурой), `refreshControl` («потяни, чтобы обновить»), `keyboardDismissMode` — это всё свойства scroll view, а не таблицы;
- делегат таблицы `UITableViewDelegate` наследует `UIScrollViewDelegate`, поэтому `scrollViewDidScroll(_:` можно написать прямо в делегате таблицы (глава 21);
- `UITextView` прокручивается, а при `isScrollEnabled = false` растёт по тексту (глава 13).

Ячейки — обычные view, не контролы: `UITableViewCell → UIView`, `UICollectionViewCell → UICollectionReusableView → UIView`. Тап по строке обрабатывает таблица и сообщает делегату через `didSelectRowAt`. Своё содержимое кладут в **`contentView`** ячейки, а не в саму ячейку — иначе в режиме редактирования оно останется на месте и перекроет кнопки (глава 12). Ячейки **переиспользуются**: `register` → `dequeueReusableCell` → `prepareForReuse` (глава 19). `dataSource` и `delegate` у таблицы — слабые ссылки: ими должен владеть кто-то ещё, обычно сам экран.

0.6 `UIViewController` — экран, а не view

Самое частое недоразумение: считать view controller «большой view». По карте `UIViewController → UIResponder`, он стоит **рядом** с `UIView`, а не под ним. У него нет `frame` и `addSubview`:

```
let vc = UIViewController()
vc.addSubview(UIView())
// error: value of type 'UIViewController' has no member 'addSubview'
```

View controller управляет одним деревом view: создаёт его, наполняет, знает, когда оно появилось на экране. Книга называет его **экраном**, потому что обычно один view controller — один экран.

`view` создаётся лениво

Свойство `view` пустое, пока к нему не обратились. При первом обращении UIKit вызывает `loadView()` (создаёт пустой `UIView` размером с экран), затем один раз `viewDidLoad()`. Отсюда

два правила: интерфейс собираем в `viewDidLoad`, а в инициализаторе `view` **не трогаем** — иначе дерево создастся раньше времени. Узнать, создан ли view, не создавая его, — `isViewLoaded`.

Сколько раз что вызывается

Метод	Когда	Сколько раз
`viewDidLoad()`	view создан	один раз
`viewWillAppear(_:)`	экран сейчас покажут	каждый показ
`viewDidLayoutSubviews()`	размеры посчитаны	много раз
`viewDidAppear(_:)`	экран показан	каждый показ
`viewWillDisappear(_:)`, `viewDidDisappear(_:)`	экран уходит	каждый уход

«Каждый показ» значит: вернулся по «назад» — снова `viewWillAppear` и `viewDidAppear`. Поэтому анимацию в `viewDidAppear` защищают флажком (глава 1), а подписку из `viewWillAppear` снимают в `viewWillDisappear`. Модальный лист **не** убирает нижний экран, поэтому его `viewWillDisappear` не вызывается; `.fullScreen` — убирает и вызывает (глава 28).

Контейнеры

Экран может содержать другие экраны — так устроены `UINavigationController`, `UITabBarController`, `UIPageViewController`. У них есть `children`, у ребёнка — `parent`. Свой контейнер строится в три шага: `addChild(child)` → добавить `child.view` в своё дерево → `child.didMove(toParent: self)`. `addChild` сам view **не** добавляет (глава 20).

Отсюда же `navigationController`, `tabBarController`, `parent` у любого экрана: UIKit ищет их вверх по родителям, и у экрана, показанного модально без обёртки, `navigationController` — `nil`.

`navigationItem` — чей он

Кнопки в панели навигации задаются **у экрана**, а не у панели: `navigationItem.title`, `rightBarButtonItem`, `searchController`. `navigationItem` — объект `UINavigationItem` (не view), панель лишь показывает его для текущего экрана. Поэтому и кнопка «назад» настраивается через `backBarButtonItem` **предыдущего** экрана (глава 26). Так же `tabBarItem` описывает экран во вкладках.

Модальный показ

`present(_:animated:)` показывает экран поверх, `dismiss(animated:)` закрывает — вызывать можно у любого из двух, UIKit перешлёт показывающему. Стиль — `modalPresentationStyle`: по умолчанию `.automatic`, что с iOS 18 означает `.formSheet` (до iOS 18 — `.pageSheet`); на iPhone это карточка, которую можно смахнуть вниз. `.fullScreen` — на весь экран. Показом управляет отдельный объект `UIPresentationController` (не view); у листа это `UISheetPresentationController` с высотами-`detents` (глава 28).

Готовые экраны — тоже `UIViewController`: `UIAlertController`, `UIActivityViewController` («Поделиться»), `UIImagePickerController`. Их показывают через `present`, а не `addSubview`.

0.7 Сцена, окно и первый экран кодом

Этот раздел подробнее остальных, потому что именно здесь начинается любое приложение на UIKit без storyboard: пока не понятно, кто создаёт окно и откуда берётся первый экран, всё остальное висит в воздухе. Разберём цепочку от тапа по иконке до лейбла на экране и соберём крошечное приложение из трёх файлов. Оно собрано и запущено в симуляторе iPhone 16 с iOS 26.5 — скриншоты ниже оттуда.

Три объекта над экраном

```
UIApplication        → UIResponder → NSObject    приложение: один на процесс
UIWindowScene        → UIScene → UIResponder     сцена: один экземпляр интерфейса
UIWindow             → UIView → UIResponder      окно: корневой view
   └── rootViewController.view                   дерево view первого экрана
```

`UIApplication` — сам процесс приложения. Он один, доступен как `UIApplication.shared`, и у него есть делегат — твой `AppDelegate`. Приложение знает свои сцены (`connectedScenes`), умеет открыть URL и запросить пуши. Экранов и окон оно само не создаёт.

`UIScene` — документация Apple называет её «один экземпляр интерфейса приложения». Представь iPad, на котором «Заметки» открыты в двух окнах рядом: это одно приложение (`UIApplication`), но **две сцены**, у каждой свои окна и свой первый экран. На iPhone сцена одна, но устройство то же: с iOS 13 UIKit всегда создаёт сцену, даже если она единственная. `UIScene` — базовый класс; на деле UIKit всегда создаёт её наследника **`UIWindowScene`** — сцену, которая умеет держать окна. Документация так и пишет: «обычно UIKit создаёт `UIWindowScene`, а не `UIScene`». Разница для тебя одна: у `UIWindowScene` есть `windows`, `screen`, `interfaceOrientation` и инициализатор окна `UIWindow(windowScene:)`; у голого `UIScene` их нет. Поэтому в делегате сцена приходит как `UIScene`, и первым делом её приводят к `UIWindowScene`.

Сцена — наследник `UIResponder`, но **не view**: нарисовать её нельзя, `frame` у неё нет. Это контейнер для окон и источник событий «стала активной», «ушла в фон» (глава 5). Сцены не создают руками — их создаёт система по описанию из `Info.plist`.

`UIWindow` — окно, и по карте это **view**, самый верхний в дереве. Окно держит один корневой экран — `rootViewController` — и само добавляет его `view` в своё дерево. Всё, что видно на экране, лежит внутри окна. Окно привязано к сцене: `UIWindow(windowScene:)` (документация: «создаёт окно и автоматически связывает его с указанной сценой; окно потом доступно через `windows` сцены»).

Ещё два объекта рядом, оба `NSObject`: **`UISceneSession`** — «карточка» сцены, которую система хранит, пока сцена существует (идентификатор, роль, конфигурация), и **`UISceneConfiguration`** — описание, как сцену создать: класс сцены, класс делегата и, если есть, storyboard.

Кто кого создаёт при запуске

```
[тап по иконке]
 → iOS запускает процесс и создаёт UIApplication
 → UIApplication создаёт AppDelegate (класс помечен @main)
 → AppDelegate.application(_:didFinishLaunchingWithOptions:)
 → UIKit читает Info.plist → Application Scene Manifest
 → AppDelegate.application(_:configurationForConnecting:options:)
```

```
      ← отдаём UISceneConfiguration с именем из Info.plist
 → UIKit создаёт UIWindowScene и UISceneSession
 → UIKit создаёт SceneDelegate (класс из конфигурации)
 → SceneDelegate.scene(_:willConnectTo:options:)
      ← здесь МЫ создаём UIWindow и первый экран
 → сцена активна, окно на экране
```

Два места в этой цепочке — твои: `AppDelegate` отвечает, какую конфигурацию использовать, `SceneDelegate` строит окно. Всё остальное делает UIKit. Со storyboard последний шаг тоже делал бы UIKit: ключ `UISceneStoryboardFile` в конфигурации велит ему загрузить первый экран из `Main.storyboard`, создать окно и поставить `rootViewController` самому. Без storyboard этот ключ убирают, и строить окно приходится руками — ровно четыре строки, их разберём ниже.

`Info.plist`: Application Scene Manifest

UIKit узнаёт про сцены из словаря `UIApplicationSceneManifest` (в редакторе Xcode — Application Scene Manifest). В проекте без storyboard он выглядит так:

```
Application Scene Manifest
├── Enable Multiple Windows: NO          (UIApplicationSupportsMultipleScenes)
└── Scene Configuration                  (UISceneConfigurations)
    └── Application Session Role         (UIWindowSceneSessionRoleApplication)
        └── Item 0
            ├── Configuration Name: Default Configuration
            └── Delegate Class Name: $(PRODUCT_MODULE_NAME).SceneDelegate
```

- **Enable Multiple Windows** — можно ли открыть приложение в нескольких окнах на iPad. Шаблон ставит `NO`: одна сцена.
- **Application Session Role** — роль «обычное окно приложения». Есть ещё роль внешнего экрана; нам она не нужна.
- **Configuration Name** — имя конфигурации. Его же вернёт `AppDelegate`.
- **Delegate Class Name** — какой класс создать делегатом сцены. `$(PRODUCT_MODULE_NAME)` Xcode заменит на имя модуля: Swift-классы живут внутри модуля, и полное имя — `МойМодуль.SceneDelegate`.
- Строки **Storyboard Name** (`UISceneStoryboardFile`) здесь **нет** — её удаляют (введение, раздел 0.1). Если оставить и удалить сам `Main.storyboard`, приложение упадёт при запуске: «Could not find a storyboard named 'Main'».

`AppDelegate` — отвечает на один вопрос

```swift
import UIKit

@main
final class AppDelegate: UIResponder, UIApplicationDelegate {

    func application(_ application: UIApplication,
                     didFinishLaunchingWithOptions launchOptions:
                         [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        true
```

```
    }

    func application(_ application: UIApplication,
                     configurationForConnecting connectingSceneSession: UISceneSession,
                     options: UIScene.ConnectionOptions) -> UISceneConfiguration {
        UISceneConfiguration(name: "Default Configuration",
                             sessionRole: connectingSceneSession.role)
    }
}
```

`@main` — точка входа: компилятор сам напишет функцию `main`, которая запустит `UIApplication` и назначит этот класс делегатом. Класс наследует `UIResponder` (последнее звено цепочки ответчиков, раздел 0.2) и реализует протокол `UIApplicationDelegate`.

`didFinishLaunchingWithOptions` — «процесс запущен». Возвращаем `true`. Окно здесь **не создают** — это работа сцены.

`configurationForConnecting` — UIKit собирается создать сцену и спрашивает: «с какой конфигурацией?». Отвечаем именем из `Info.plist` и той же ролью, что у сессии. UIKit найдёт в манифесте запись с этим именем и возьмёт из неё класс делегата.

`SceneDelegate` — четыре строки, которые создают интерфейс

```
import UIKit

final class SceneDelegate: UIResponder, UIWindowSceneDelegate {

    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }

        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = UINavigationController(
            rootViewController: HelloViewController())
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

Разберём каждую строку — каждая отвечает за свою типичную ошибку.

`UIWindowSceneDelegate` — протокол делегата **оконной** сцены. Он наследует `UISceneDelegate` (общие методы любой сцены: подключилась, активна, в фоне) и добавляет оконные. Документация: «не создавайте объекты `UIWindowSceneDelegate` сами — укажите имя класса в конфигурации сцены».

`var window: UIWindow?` — **сильная ссылка на окно**. Это не формальность: сцена окно не удерживает. Убери эту строку (или присваивание ниже) — и окно освободится сразу после выхода из метода, а экран останется чёрным. Мы проверили это в главе 3: без ссылки `windowScene.windows` пустеет.

`scene(_:willConnectTo:options:)` — UIKit зовёт его, когда сцена подключается к приложению: при холодном запуске или когда пользователь открывает новое окно на iPad. Документация: «используйте этот метод, чтобы отреагировать на добавление сцены и начать загрузку данных, которые ей нужны». Третий параметр `connectionOptions` хранит, **из-за чего** сцена открылась: по тапу на уведомление, по ссылке, по Handoff (глава 41).

`guard let windowScene = scene as? UIWindowScene` — сцена приходит как базовый `UIScene`; окна умеет только `UIWindowScene`, приводим тип.

`UIWindow(windowScene: windowScene)` — создаём окно **с привязкой к сцене**. Старый инициализатор `UIWindow(frame:)` для приложения со сценами не подходит: документация просит создавать окно именно через `init(windowScene:)`, иначе у него `windowScene == nil` и связь со сценой придётся налаживать вручную.

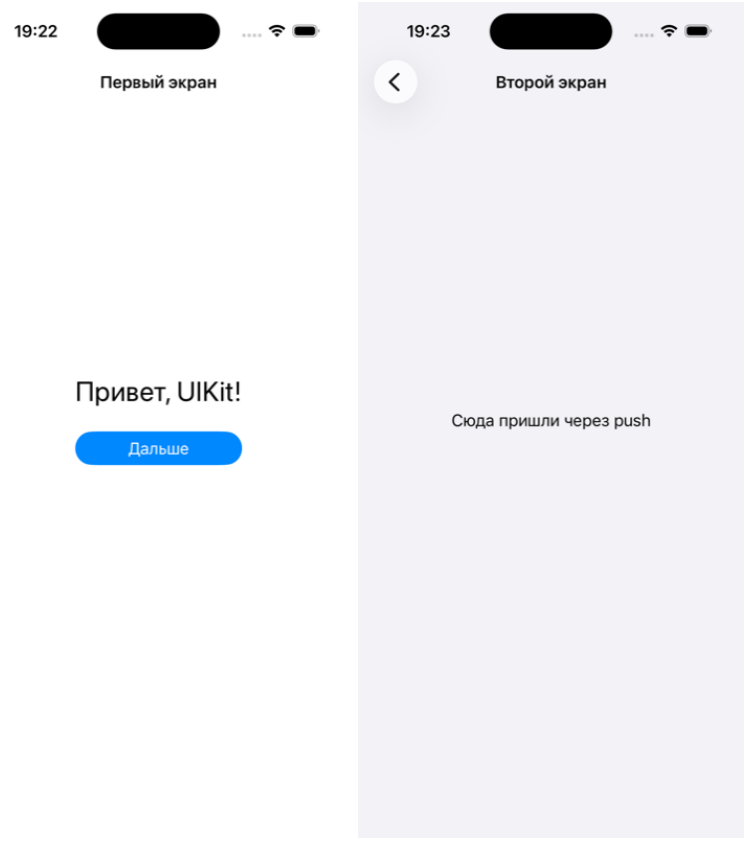
`window.rootViewController = …` — первый экран. Здесь он обёрнут в `UINavigationController`, чтобы была панель навигации и можно было делать `push`. Окно само добавит `view` этого контроллера в своё дерево и растянет на весь размер — вручную `addSubview` не нужен.

`window.makeKeyAndVisible()` — показать окно и сделать его **ключевым** (получающим клавиатуру). Документация: это удобная замена двум действиям, `isHidden = false` и `makeKey()`.

`self.window = window` — та самая сильная ссылка.

Других методов делегату сцены пока не нужно. Когда понадобятся — `sceneDidBecomeActive`, `sceneWillResignActive`, `sceneDidEnterBackground` — порядок их вызовов и что в них делать разобраны в главе 5.

Первый экран целиком кодом




```swift
import UIKit

final class HelloViewController: UIViewController {

    private let titleLabel = UILabel()
    private let nextButton = UIButton(configuration: .filled())

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Первый экран"
        view.backgroundColor = .systemBackground

        titleLabel.text = "Привет, UIKit!"
        titleLabel.font = .preferredFont(forTextStyle: .title1)
        titleLabel.textAlignment = .center

        nextButton.configuration?.title = "Дальше"
        nextButton.addAction(UIAction { [weak self] _ in
            self?.navigationController?.pushViewController(
                DetailViewController(), animated: true)
        }, for: .touchUpInside)
```

```
        let stack = UIStackView(arrangedSubviews: [titleLabel, nextButton])
        stack.axis = .vertical
        stack.spacing = 24
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)

        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: view.centerYAnchor),
            stack.leadingAnchor.constraint(
                greaterThanOrEqualTo: view.safeAreaLayoutGuide.leadingAnchor, constant:
↪   16),
        ])
    }
}

final class DetailViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Второй экран"
        view.backgroundColor = .systemGroupedBackground
        let label = UILabel()
        label.text = "Сюда пришли через push"
        label.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(label)
        NSLayoutConstraint.activate([
            label.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            label.centerYAnchor.constraint(equalTo: view.centerYAnchor),
        ])
    }
}
```

Это шаблон каждого экрана книги; в нём собраны все ступени карты.

Экран — наследник `UIViewController` (раздел 0.6). Своих инициализаторов нет, поэтому `HelloViewController()` создаётся без аргументов, и `init?(coder:)` писать не нужно. Всё строится в `viewDidLoad`: к этому моменту `view` уже создан UIKit'ом (пустой `UIView` размером с экран), а раньше трогать его нельзя.

`title` — свойство экрана; панель навигации возьмёт его в `navigationItem` и покажет (раздел 0.6).

`view.backgroundColor = .systemBackground` — без этой строки корневой view прозрачный, и под ним будет чёрное окно. `.systemBackground` — динамический цвет: белый в светлой теме, чёрный в тёмной.

Лейбл и кнопка — ступени `UIView` и `UIControl` (разделы 0.3, 0.4): у лейбла только содержимое (`text`, `font`), у кнопки — ещё и `addAction`. `UIButton(configuration: .filled())` — кнопка нового образца (iOS 15) с синей заливкой; заголовок задаётся через `configuration?.title`. В замы-

кании `[weak self]`, потому что кнопка хранит замыкание, а экран хранит кнопку — без `weak` получился бы цикл.

`navigationController?.pushViewController` — экран находит свой `UINavigationController`, поднимаясь по родителям (раздел 0.6). Он есть, потому что в `SceneDelegate` мы обернули первый экран в навигационный контроллер. Без обёртки свойство вернуло бы `nil`, и тап ничего бы не сделал. Второй экран получает кнопку «назад» сам — её рисует панель.

`UIStackView` — складывает лейбл и кнопку в столбик с промежутком 24 точки и сам создаёт правила раскладки для своих элементов. Ему, как любому subview с правилами, нужна строка `translatesAutoresizingMaskIntoConstraints = false` (раздел 0.3).

Три правила Auto Layout: стек по центру экрана, а его левый край — не ближе 16 точек к безопасной зоне. Ширину и высоту не задаём — их дадут `intrinsicContentSize` лейбла и кнопки. Правило с `greaterThanOrEqualTo` сработает только на узком экране или с крупным шрифтом: тогда стек не вылезет за край, а лейбл перенесёт текст.

Второй экран — тот же шаблон в минимальной форме: заголовок, фон, один лейбл по центру.

Как из любого view добраться до окна, сцены и экрана устройства

В `viewDidAppear` второго экрана мы напечатали, что видно «снизу вверх»:

```
print(view.window.map { NSStringFromClass(type(of: $0)) } ?? "nil",
      view.window?.windowScene.map { NSStringFromClass(type(of: $0)) } ?? "nil",
      view.window?.windowScene?.screen.bounds ?? .zero,
      view.window?.windowScene?.session.configuration.name ?? "nil")
```

Вывод в симуляторе iPhone 16:

```
UIWindow UIWindowScene (0.0, 0.0, 393.0, 852.0) Default Configuration
```

То есть путь такой: `view.window` → `windowScene` → `screen` или `session`. Это и есть замена устаревшему `UIScreen.main`: размер экрана берут у сцены, в которой живёт view. Два предупреждения: `view.window` равен `nil`, пока view не в окне — в `viewDidLoad` мы получили `nil` (тот же вывод до перестановки печати в `viewDidAppear`); и у окна, созданного без сцены, `windowScene` тоже `nil`.

Если вместо экрана — чёрный

Четыре причины, по порядку проверки:

1. В `Info.plist` остался **Storyboard Name** или в Build Settings — `UIMainStoryboardFile`, а самого storyboard нет: приложение падает при запуске с «Could not find a storyboard named ‘Main’».
2. Нет `self.window = window` — окно освободилось.
3. Нет `window.makeKeyAndVisible()` — окно создано, но скрыто.
4. В **Delegate Class Name** опечатка или нет `$(PRODUCT_MODULE_NAME).` — UIKit не найдёт класс делегата, `willConnectTo` не вызовется. Когда конфигурация одна, имя в `configurationForConnecting` может даже не совпадать с манифестом — UIKit возьмёт единственную (проверено в главе 5), а вот класс делегата должен быть найден.

И одна причина не чёрного, а **прозрачного** экрана: забытый `view.backgroundColor` у корневого view.

0.8 Что не view

Нижняя половина карты — объекты без `frame`. Их не кладут на экран, они не ловят касания. Как с ними работать:

Класс	Что это	Как с ним работать
`CALayer` и наследники	то, что рисует железо	`view.layer.addSublayer`, `frame` задаёшь в `layoutSubviews`
`CABasicAnimation`, `CAKeyframeAnimation`	описание анимации слоя	`layer.add(_:forKey:)`; значение свойства меняй отдельно
`UIGestureRecognizer` и наследники	распознаватель жеста	`view.addGestureRecognizer`; один жест — одна view
`UIBarButtonItem`, `UITabBarItem`	описание кнопки для панели	`navigationItem.rightBarButtonItem = …`
`UINavigationItem`	как экран виден в панели	`vc.navigationItem.title = …`
`UIAction`, `UIMenu`	действие или меню	`addAction`, `button.menu`
`UIPresentationController`	ведёт модальный показ	`vc.sheetPresentationController?.detents`
`UIFeedbackGenerator`	хаптика	`prepare()`, `impactOccurred()` (глава 33)
`UICollectionViewLayout`	раскладка коллекции	передаётся в `UICollectionView(frame:collectionViewLayo`
`NSLayoutConstraint`, `UILayoutGuide`	правило раскладки, направляющая	`activate`, `isActive`, `constant`
`UIColor`, `UIFont`, `UIImage`	цвет, шрифт, картинка	значения; `UIColor` может быть динамическим

Три пары, которые путают чаще всего.

`UIBarButtonItem` и `UIButton`. Кнопка — view. Элемент панели — **описание** кнопки: заголовок, картинка, действие. Рисует его панель. Нужен свой view в панели — `UIBarButtonItem(customView:)`.

`CALayer` и `UIView`. Слой дешевле view и умеет градиент, тень, фигуру по контуру, но не знает про Auto Layout и касания. Его `frame` выставляют руками в `layoutSubviews` (глава 21).

`UIGestureRecognizer` и касания. Распознаватель прикрепляют к одной view, и он сообщает о жесте через `addTarget` или `UIAction`, как контрол. По умолчанию он отменяет касание для самой view (`cancelsTouchesInView`), поэтому на одной view редко совмещают жесты и `touchesBegan` (глава 29).

Правило большого пальца: **нет `frame` — не view**. Нужно показать — найди view, которая это отобразит: панель для `UIBarButtonItem`, view для `CALayer`, `UIImageView` для `UIImage`.

0.9 Делегат и источник данных

Наследование — первый скелет UIKit, **протоколы** — второй. Два протокола встречаются на каждом шагу.

Делегат (delegate) — объект, которому класс UIKit сообщает о событиях и задаёт вопросы: «тапнули строку», «какой высоты строка?». **Источник данных** (data source) — объект, у которого класс запрашивает содержимое: «сколько строк?», «дай ячейку для строки 5». Таблица не хранит твоих данных и не знает твоей логики — она только рисует и спрашивает. Поэтому один `UITableView` годится и для списка дел, и для чата.

Три практических правила:

- делегат — всегда **слабая** ссылка, чтобы экран и таблица не держали друг друга вечно. Значит, делегатом должен владеть кто-то ещё; обычно это сам экран — проблем нет. Свой протокол для `weak var delegate` объявляй с `: AnyObject`;
- класс-делегат наследует `NSObject` (раздел 0.1);
- большинство методов делегата **необязательны**: реализуешь три из тридцати, остальное UIKit делает по умолчанию.

0.10 Проверь сам

Карту не нужно принимать на веру. Функция ниже спрашивает у рантайма родителя класса, потом родителя родителя — до корня:

```
import UIKit

/// «Родословная» класса: сам класс → родитель → … → NSObject.
func superclassChain(of cls: AnyClass) -> String {
    var names = [NSStringFromClass(cls)]
    var current: AnyClass? = class_getSuperclass(cls)
    while let next = current {
        names.append(NSStringFromClass(next))
        current = class_getSuperclass(next)
    }
    return names.joined(separator: " → ")
}
```

`class_getSuperclass` — «дай родителя класса»; у `NSObject` родителя нет, вернётся `nil`, и цикл закончится. Вызови из `viewDidLoad` любого экрана книги:

```
let classes: [AnyClass] = [UITableView.self, UIBarButtonItem.self, UIViewController.self]
for cls in classes {
    print(superclassChain(of: cls))
}
```

В симуляторе iOS 26.5 мы получили:

```
UITableView → UIScrollView → UIView → UIResponder → NSObject
UIBarButtonItem → UIBarItem → NSObject
UIViewController → UIResponder → NSObject
```

Тем же способом снята вся карта из 0.1 и все числа этой главы: `print(UILabel().isUserInteractionEnabled)`, `print(UIView().frame)`. Сомневаешься в утверждении про UIKit — напечатай и посмотри. Это надёжнее любой статьи, включая эту главу.

Упражнение 0.3. Распечатай цепочки для `UISearchBar`, `UITextView`, `UIImagePickerController` и `UIAlertAction`. Для каждого ответь: можно ли его `addSubview`, есть ли у него `addAction(_:for:)`, как он попадает на экран. Ответ — в конце главы.

Ответы к упражнениям

0.1. `bounds` = (0, 0, 200, 80): начало в своих координатах всегда (0, 0), размер тот же. `center` = (40 + 100, 100 + 40) = (140, 140). После `center = (100, 100)` размер не меняется, а угол сдвигается так, чтобы центр оказался в (100, 100): `frame` = (0, 60, 200, 80). Проверь: создай view с таким `frame`, присвой `center` и напечатай `frame`.

0.2. `addAction(_:for:)` есть у `UISlider`, `UIPageControl` и `UIDatePicker` — все три контролы, событие `.valueChanged`. Нет у `UISearchBar` (`UISearchBarDelegate.searchBar(_:textDidChange:)`), `UIImageView` (`UITapGestureRecognizer` + `isUserInteractionEnabled = true`) и `UITextView` (`UITextViewDelegate.textViewDidChange(_:)`).

0.3. В симуляторе iOS 26.5:

```
UISearchBar → UIView → UIResponder → NSObject
UITextView → UIScrollView → UIView → UIResponder → NSObject
UIImagePickerController → UINavigationController → UIViewController
                        → UIResponder → NSObject
UIAlertAction → NSObject
```

`UISearchBar` — view: `addSubview` можно, `addAction` нет, на экран попадает как subview или через `navigationItem.searchController`. `UITextView` — view и scroll view: `addSubview` можно, `addAction` нет, события через делегат. `UIImagePickerController` — экран, причём целый навигационный стек: ни `addSubview`, ни `addAction`, на экран — через `present`. `UIAlertAction` — ни view, ни экран, а описание кнопки алерта; его рисует `UIAlertController` после `alert.addAction(_:)`.

Что мы выучили

- Почти весь UIKit стоит на лестнице `NSObject → UIResponder → UIView → UIControl → конкретный класс`; класс умеет то, что умеют предки, плюс своё.
- Из `NSObject` на практике: `@objc` для `#selector`, `NSObject` в предках у класса-делегата.
- `UIResponder` получает события и передаёт по цепочке до окна и `AppDelegate`; первый ответчик — поле с курсором, `becomeFirstResponder()` поднимает клавиатуру.
- `UIView`: `frame` в координатах родителя, `bounds` в своих; при `transform` `frame` не определён. Для правил раскладки — `translatesAutoresizingMaskIntoConstraints = false`; размеры subviews известны не в `viewDidLoad`, а в `viewDidLayoutSubviews`. У `UILabel` и `UIImageView` касания выключены по умолчанию.
- `UIControl` даёт `addAction(_:for:)` и события `.touchUpInside`, `.valueChanged`, `.editingChanged`. У `UILabel`, `UISearchBar`, `UITextView` этого нет — там жесты и делегаты.
- `UITableView`, `UICollectionView`, `UITextView` — scroll view: у них есть `contentOffset`, `contentInset`, `refreshControl`. Ячейки — view, своё кладём в `contentView`.
- `UIViewController` — не view: нет `frame` и `addSubview`; `view` создаётся лениво, `viewDidLoad` один раз, `viewWillAppear` — каждый показ; `navigationItem` принадлежит экрану; `.automatic` — это `.formSheet` с iOS 18.

- Запуск без storyboard: `Info.plist` → `AppDelegate` отдаёт конфигурацию → UIKit создаёт `UIWindowScene` и `SceneDelegate` → в `willConnectTo` мы создаём `UIWindow(windowScene:)`, ставим `rootViewController`, зовём `makeKeyAndVisible()` и храним окно в `self.window`.
- `UIWindow` — view (можно `addSubview`), сцена и приложение — ответчики, но не view; до них из любого view: `view.window?.windowScene`.
- Слои, жесты, `UIBarButtonItem`, `UINavigationItem`, `UIAction`, цвета, constraints — без `frame`: их нельзя `addSubview`, нужно найти view, которая их покажет.
- Делегат сообщает о событиях, источник данных отдаёт содержимое; обе ссылки слабые.
- Любое утверждение про UIKit проверяется печатью: `class_getSuperclass` и значения свойств у свежего объекта.

Apple Developer Documentation

- UIKit — каталог фреймворка; у каждого класса на странице есть раздел Inherits From.
- UIResponder и Using responders and the responder chain to handle events — ответчики и цепочка.
- UIView, frame, bounds — геометрия и предупреждение про `transform`.
- hitTest(_:with:) — кто получает касание и три условия пропуска.
- translatesAutoresizingMaskIntoConstraints, intrinsicContentSize, layoutIfNeeded() — Auto Layout.
- layer и CALayer — слой view.
- UIControl, UIControl.Event, addAction(_:for:) — target-action.
- UIScrollView, UITableViewCell.contentView — прокрутка и ячейки.
- UIViewController, view, loadView() — экран и его view.
- addChild(_:) и navigationItem — контейнеры и элемент навигации.
- UIModalPresentationStyle.automatic и dismiss(animated:completion:) — модальный показ.
- UIWindow, init(windowScene:), makeKeyAndVisible() — окно и его привязка к сцене.
- UIScene, UIWindowScene, UIWindowSceneDelegate, scene(_:willConnectTo:options:) — сцена и её делегат.
- UISceneConfiguration, UIApplicationSceneManifest — конфигурация сцены и ключ `Info.plist`.
- UIApplication и UIApplicationDelegate — приложение и его делегат.
- UIBarItem, UIGestureRecognizer — объекты нижней половины карты.
- UITableViewDelegate и UITableViewDataSource — делегат и источник данных.
- class_getSuperclass(_:) — функция рантайма из раздела 0.10.

→ Глава 1. LaunchScreen vs AnimatedSplash — два экрана, оба «splash»

Глава 1. LaunchScreen vs AnimatedSplash — два экрана, оба «splash»

Когда пользователь тапает иконку приложения на домашнем экране, между этим моментом и первым рабочим экраном проходят две стадии. На первой система iOS показывает картинку, которую ты подготовил заранее. На второй уже твоё приложение рисует что-то своим кодом.

Обе стадии часто называют одним словом **splash** (англ. «всплеск», здесь — заставка). Но это разные механизмы, и задачи у них разные. В этой главе разбираем оба и заодно понимаем, почему в нашем playground'е лежат и `LaunchScreen.storyboard`, и `AnimatedSplashViewController.swift`.

Два слова, которые встретятся сразу.

View controller (VC) — объект, который управляет одним экраном: создаёт его содержимое, реагирует на появление и исчезновение экрана, на нажатия. Все экраны в UIKit — наследники `UIViewController`. Сам view controller — не view: его родословная `UIViewController → UIResponder → NSObject`, а прямоугольник на экране — это его свойство `view` (подробно — глава 0, раздел 0.6). Аналогия: экран — комната, view controller — её хозяин, который расставляет мебель и открывает дверь гостям.

Storyboard — файл Xcode, в котором экраны не пишут кодом, а рисуют мышкой в редакторе Interface Builder. В этой книге storyboard нужен ровно для одного экрана — `LaunchScreen`.

1.1 Что происходит между тапом и первым экраном

Упрощённо запуск выглядит так:

```
[тап по иконке]
        │
        ▼
[iOS показывает LaunchScreen]   ← статичная картинка, на неё уходят доли секунды
        │
        ▼
[процесс приложения запустился]
        │
        ▼
[наш код получил управление]    ← здесь начинается AnimatedSplash
```

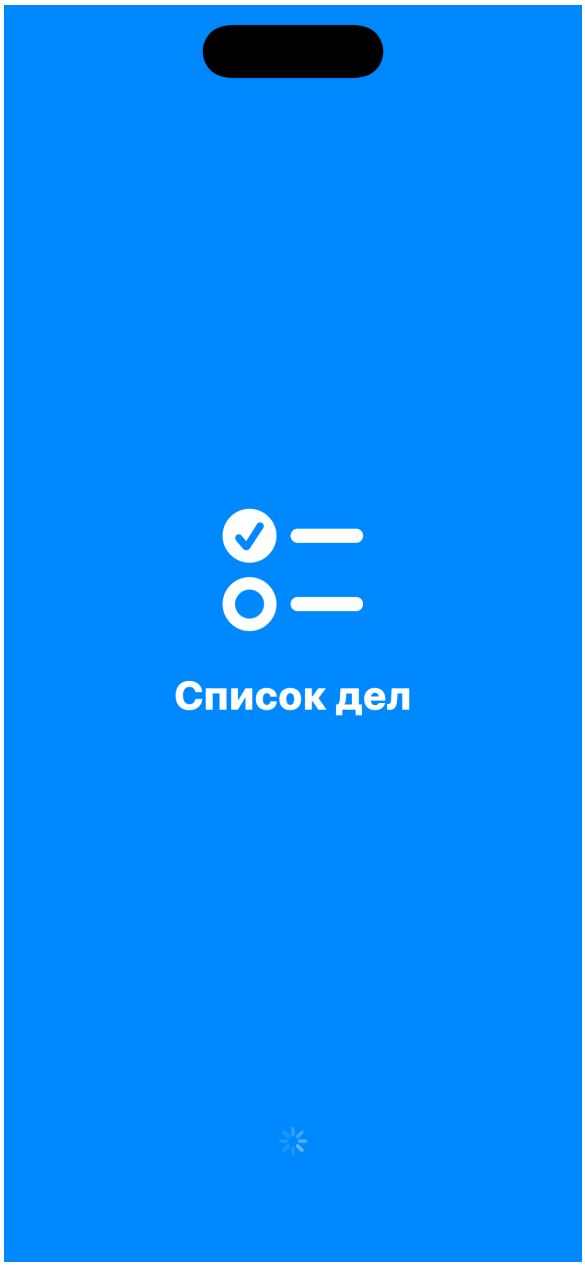


Рис. 2: Анимированный splash mini-app

Между «тапнул» и «процесс запустился» проходит время, которым ты не управляешь: система загружает исполняемый файл и фреймворки, готовит среду выполнения Swift. На свежем устройстве это обычно десятые доли секунды, на старом при **холодном старте** (приложения нет в памяти, его запускают с нуля) может доходить до секунды-двух. И всё это время твой код ещё не работает.

Что-то показать всё равно надо. Поэтому iOS показывает картинку, которую ты описал заранее. Это и есть **launch screen** — экран запуска.

> **Идея.** `LaunchScreen` — **картинка для системы**: чтобы, пока приложение не загрузилось, экран не был пустым. `AnimatedSplash` — уже **твой код**: в нём можно показать анимацию, сходить в сеть, проверить токен входа.

1.2 LaunchScreen — почему storyboard и почему без кода

Когда iOS показывает launch screen, ни одна строчка твоего Swift ещё не выполнилась: нет ни `viewDidLoad`, ни `init`. Поэтому launch screen описывают так, чтобы система могла нарисовать его **сама, без исполнения кода**. Способов два:

1. **Storyboard** — файл `LaunchScreen.storyboard`. Его создаёт шаблон Xcode, и имя файла записано в настройку `UILaunchStoryboardName` (в Build Settings она видна как «Launch Screen Interface File Base Name»).
2. **Ключ `UILaunchScreen` в `Info.plist`** (с iOS 14) — вообще без storyboard: указываешь цвет фона, картинку, наличие панели навигации, и система собирает экран сама.

Мы остаёмся на storyboard: его создал шаблон, и в нём видно, что получится. Но помни, что второй способ существует: «Apple требует именно storyboard» — не совсем так. С 2020 года App Store требует, чтобы launch screen был задан одним из этих двух способов, а не старыми картинками `LaunchImage`.

У launch storyboard строгие ограничения. Документация Apple («Specifying your app's launch screen») перечисляет их так:

- только стандартные классы UIKit (`UIView`, `UIImageView`, `UILabel`…);
- один корневой view или view controller;
- никаких связей с кодом — ни `@IBOutlet`, ни `@IBAction`;
- никаких своих классов (`class MyView: UIView` сюда не подставить);
- никаких runtime-атрибутов (значений, которые Interface Builder присваивает свойствам при загрузке);
- никаких устаревших view вроде `UIWebView`.

Отсюда же ещё одна особенность, о которую спотыкаются все: iOS **кэширует** launch screen — делает с него снимок и дальше показывает снимок. Поменял storyboard, запустил — а на экране старая картинка. Лечится удалением приложения с симулятора (долгое нажатие на иконку → Удалить приложение) и повторным запуском.

Каким должен быть launch screen

Здесь важно не путать launch screen с заставкой. Human Interface Guidelines (HIG — продуктовые правила Apple, раздел «Launching») говорят прямо:

- launch screen должен быть **почти копией первого экрана** приложения, чтобы переход был незаметен и казалось, что приложение открылось мгновенно;
- **без текста**: launch screen статичен, и текст на нём не будет переведён на язык пользователя;
- **без рекламы и логотипов**: это не место для брендинга. Если нужна заставка с логотипом — делай её отдельным экраном уже в коде.

Наш первый экран — лаунчер: список на сером «сгруппированном» фоне (его разберём в главе 4). Поэтому наш `LaunchScreen.storyboard` — пустой view с фоном **System Grouped Background Color**. Это системный цвет: светло-серый в светлой теме и почти чёрный в тёмной, и iOS сама берёт нужный вариант.

Как это сделать: открой `LaunchScreen.storyboard` (он в группе `Base.lproj` или в корне проекта, зависит от версии шаблона), выдели **View** в списке слева, справа в Attributes Inspector (иконка ползунков) у **Background** выбери **System Grouped Background Color**. Больше ничего на экран не ставим.

А логотип, название и анимация будут в **нашем** splash — уже кодом.

Упражнение 1.1. Поставь в `LaunchScreen.storyboard` фон **System Red Color** и запусти приложение. Что увидишь в первую долю секунды? Потом верни серый фон и запусти снова. Если красный не исчез — почему, и как это исправить?

1.3 AnimatedSplash — это уже твой view controller

Когда процесс запустился и `SceneDelegate` (объект, который готовит окно приложения, — разберём в главе 5) поставил первый экран, можно показать **второй** splash, уже своими руками. У него три задачи:

1. **Создать ощущение продукта.** Launch screen статичен. Короткая анимация говорит «сейчас всё будет».
2. **Дать время.** Пока идёт анимация, можно асинхронно проверить токен, скачать настройки с сервера, прогреть кэш.
3. **Брендировать.** В нашем playground'е splash окрашен в фирменный цвет mini-app: у «Списка дел» он синий, у «Заметок» жёлтый, у «Чата» зелёный. Launch screen один на всё приложение, splash у каждого mini-app свой.

Файл: `App/AnimatedSplashViewController.swift`. Разберём его целиком, по частям.

Свойства и конструктор

```
import UIKit

/// Второй, «наш» splash: анимация в цвете mini-app, потом onFinish.
final class AnimatedSplashViewController: UIViewController {

    private let manifest: AppManifest
    private let onFinish: () -> Void

    private let iconView = UIImageView()
```

```
    private let titleLabel = UILabel()
    private let activityIndicator = UIActivityIndicatorView(style: .large)
    private var didStartAnimation = false

    init(manifest: AppManifest, onFinish: @escaping () -> Void) {
        self.manifest = manifest
        self.onFinish = onFinish
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется: экран создаётся только кодом")
    }
```

Splash получает `manifest` — описание mini-app: имя, иконку, цвет, длительность заставки. Подробно `AppManifest` разберём в главе 2, пока достаточно знать, что это структура с этими полями.

Второй параметр — `onFinish`, **callback** (функция обратного вызова): замыкание, которое splash вызовет, когда закончит. Сам splash не знает, что будет дальше, — это решает координатор (глава 4). Аналогия: ты отдаёшь курьеру посылку и номер телефона — «позвони, когда доставишь». Что ты сделаешь после звонка, курьера не касается.

Три UI-элемента создаём сразу, в объявлении свойств: иконку (`UIImageView`), название (`UILabel`) и `UIActivityIndicatorView` — системный крутящийся индикатор загрузки, стиль `.large` — крупный.

`didStartAnimation` — флажок «анимацию уже запускали». Зачем он, станет понятно в разделе про `viewDidAppear`.

`super.init(nibName: nil, bundle: nil)` — так создаётся view controller без storyboard и без xib-файла: «интерфейс я соберу кодом сам».

`required init?(coder:)` требует не UIKit в рантайме, а компилятор Swift: в `UIViewController` этот инициализатор объявлен как `required` (он нужен протоколу `NSCoding`, по которому экран создаётся из storyboard или xib), а раз мы добавили свой инициализатор, обязаны повторить и его. Мы экран из storyboard не создаём, поэтому тело — просто `fatalError` с объяснением. Если когда-нибудь кто-то попробует — сразу увидит понятное сообщение, а не загадочный сбой.

viewDidLoad — один раз, когда view создан

```
    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = manifest.brandColor
        setupLayout()
    }
```

`viewDidLoad` UIKit вызывает один раз — когда корневой view экрана создан, но ещё **не показан**. Здесь собирают интерфейс.

Два слова о том, откуда берётся `view`. Он создаётся **лениво**: пока к свойству `view` никто не обратился, его нет (`isViewLoaded == false`). При первом обращении UIKit зовёт `loadView()` — тот без storyboard создаёт пустой `UIView`, — и сразу после него `viewDidLoad`. Мы проверили это на симуляторе: у свежего view controller'а `isViewLoaded` равен `false` даже после того, как его поставили в `window.rootViewController`, и становится `true` при первом чтении `view`.

Что в `viewDidLoad` **ещё неизвестно**: экран пока не в окне (`view.window == nil`), `safeAreaInsets` и `layoutMargins` у корневого view равны нулю, а `view.bounds` — лишь предварительный размер (на симуляторе мы увидели размер экрана, но внутри навигации или на iPad он изменится при раскладке). Поэтому здесь ставят constraints, а не читают координаты. Окончательные рамки и safe area известны начиная с `viewDidLayoutSubviews()` и `viewSafeAreaInsetsDidChange()` (подробно — глава 0, раздел 0.6).

Цвет фона берём из манифеста — `brandColor`. Никакого жёстко прописанного `.systemBlue`, иначе все mini-app выглядели бы одинаково.

setupLayout — элементы и Auto Layout

```
    private func setupLayout() {
        let config = UIImage.SymbolConfiguration(pointSize: 80, weight: .semibold)
        iconView.image = UIImage(systemName: manifest.symbolName, withConfiguration:
↪   config)
        iconView.tintColor = .white
        iconView.contentMode = .scaleAspectFit

        titleLabel.text = manifest.name
        titleLabel.textColor = .white
        titleLabel.font = .preferredFont(forTextStyle: .title1)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.textAlignment = .center
        titleLabel.numberOfLines = 0

        activityIndicator.color = .white
        activityIndicator.startAnimating()
```

Иконка — это **SF Symbol**: одна из нескольких тысяч векторных иконок Apple, встроенных в систему. Их берут по имени — `UIImage(systemName: "checklist")`. `SymbolConfiguration(pointSize: 80)` задаёт размер: символ рисуется как буква шрифта кеглем 80 точек.

Точка (point, pt) — единица размеров в UIKit. Это не пиксель: на iPhone с экраном «@3x» одна точка — это 3×3 = 9 физических пикселей, на «@2x» — 2×2 = 4. Ты пишешь размеры в точках, а система сама пересчитывает их в пиксели конкретного экрана.

`tintColor = .white` — SF Symbol по умолчанию одноцветный и перекрашивается в `tintColor`.

У лейбла шрифт `preferredFont(forTextStyle: .title1)` — это не фиксированный размер, а **текстовый стиль**. Он подстраивается под **Dynamic Type** — настройку «Размер текста» в iOS (Настройки → Экран и яркость → Размер текста). Если человек плохо видит и поставил крупный текст, заголовок тоже станет крупнее. `adjustsFontForContentSizeCategory = true` велит лейблу

обновиться сразу, если настройку поменяли, пока экран открыт. `numberOfLines = 0` — «сколько угодно строк»: длинное имя при крупном шрифте перенесётся, а не обрежется многоточием.

`startAnimating()` запускает вращение индикатора. Пока он прозрачный, вращение не видно, — покажем его позже анимацией.

Дальше — расстановка элементов:

```
        for subview in [iconView, titleLabel, activityIndicator] {
            subview.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview(subview)
        }

        NSLayoutConstraint.activate([
            iconView.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            iconView.centerYAnchor.constraint(equalTo: view.centerYAnchor, constant: -40),

            titleLabel.topAnchor.constraint(equalTo: iconView.bottomAnchor, constant: 20),
            titleLabel.leadingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.leadingAnchor),
            titleLabel.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor),

            activityIndicator.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            activityIndicator.bottomAnchor.constraint(
                equalTo: view.safeAreaLayoutGuide.bottomAnchor, constant: -48),
        ])
```

Это **Auto Layout** — система, в которой ты не задаёшь координаты вручную («иконка в точке x = 156, y = 380»), а описываешь правила: «иконка по центру по горизонтали, на 40 точек выше центра по вертикали». Каждое правило — **constraint** (ограничение). По этим правилам UIKit сам вычисляет координаты — на любом iPhone, iPad, в любой ориентации.

Прочитаем правила словами:

- иконка: центр по горизонтали = центру экрана; центр по вертикали на 40 точек выше центра экрана (`constant: -40` — вверх, потому что ось y в UIKit растёт **вниз**);
- заголовок: верх на 20 точек ниже низа иконки; левый и правый края прижаты к полям экрана;
- индикатор: по центру по горизонтали; низ на 48 точек выше нижней границы safe area.

Два понятия отсюда:

- **Layout margins** (`layoutMarginsGuide`) — стандартные поля по краям view. У обычного `UIView` они по умолчанию 8 точек со всех сторон, а у корневого view экрана их задаёт система: на iPhone 16 в портретной ориентации мы получили 16 точек слева и справа (на широких моделях бывает 20), сверху и снизу поля включают safe area. Текст, прижатый к полям, не липнет к краю экрана.
- **Safe area** (`safeAreaLayoutGuide`) — часть экрана, которую ничего не перекрывает: ни «чёлка» или Dynamic Island сверху, ни полоска «домой» (home indicator) снизу. Привязываем индикатор к safe area, иначе на iPhone без кнопки «Домой» он залез бы под полоску.

Оба guide — `UILayoutGuide`, прямоугольники для constraints, а не view: их нет в дереве subviews, и `addSubview` им не сделать (глава 0, раздел 0.8).

`translatesAutoresizingMaskIntoConstraints = false` — обязательная строка для каждого view, которое расставляешь через constraints в коде. По умолчанию у view, созданного кодом, свойство равно `true`, и UIKit сам создаёт для него constraints, которые повторяют его autoresizing mask — закрепляют текущий `frame` (прямоугольник с координатами). Эти автоматические правила столкнутся с твоими, и в консоли появится длинное сообщение «Unable to simultaneously satisfy constraints» — конфликт правил. Забыть эту строку — самая частая ошибка при вёрстке кодом. Исключение — view, раскладкой которых управляет сам UIKit: например, `arrangedSubviews` стека (глава 2) — им это свойство трогать не нужно.

`NSLayoutConstraint.activate([...])` включает все правила разом — это быстрее, чем включать по одному.

Последний кусок `setupLayout` — стартовое состояние для анимации:

```
        // Стартовое состояние: всё прозрачное, иконка уменьшена до 60%.
        iconView.alpha = 0
        iconView.transform = CGAffineTransform(scaleX: 0.6, y: 0.6)
        titleLabel.alpha = 0
        activityIndicator.alpha = 0
    }
```

`alpha` — непрозрачность: 0 — полностью прозрачный, 1 — полностью видимый, 0.5 — наполовину.

`transform` — геометрическое преобразование view поверх его обычного положения: масштаб, поворот, сдвиг. `CGAffineTransform(scaleX: 0.6, y: 0.6)` — масштаб 0.6 по обеим осям, то есть иконка рисуется размером **60% от обычного**. Для иконки 80×80 точек это 48×48 точек. Constraints при этом не меняются: transform — это «как нарисовать», а не «где стоять», поэтому соседей иконка не толкает. Auto Layout работает с `center` и `bounds`, а они от transform не зависят — мы проверили: у view 80×80 после масштаба 0.6 `bounds` остаётся 80×80, `center` тот же, и только `frame` становится 48×48. Документация Apple про `frame` при transform, отличном от identity, говорит прямо: значение не определено, читать его не стоит (глава 0, раздел 0.3).

Анимация потом вернёт `alpha` к 1, а `transform` — к `.identity` («без преобразования», обычный размер, 100%).

viewDidAppear — когда запускать анимацию

```
    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        guard !didStartAnimation else { return }
        didStartAnimation = true
        runAnimation()
    }
```

`viewDidAppear` UIKit вызывает, когда экран **уже на экране**. Это надёжное место для анимации появления: пользователь увидит её с первого кадра.

Почему не `viewDidLoad`? В `viewDidLoad` экран ещё не показан. Если запустить анимацию там, её начало может уйти «в пустоту» — пока экран добирается до окна, часть анимации уже проиграется, и появление получится рваным.

Флажок `didStartAnimation` нужен, потому что `viewDidAppear`, в отличие от `viewDidLoad`, может вызваться **несколько раз**: например, если поверх splash показали экран на весь экран (`modalPresentationStyle = .fullScreen`) и закрыли его, или ушли по навигации вперёд и вернулись назад. Без флажка анимация и таймер запустились бы заново, и `onFinish` вызвался бы дважды. Одна тонкость: обычный «лист» (sheet, стиль `.automatic`, то есть `.formSheet` в iOS 18+ и `.pageSheet` раньше) нижний экран из окна не убирает, поэтому `viewWillDisappear` и повторный `viewDidAppear` у него **не** вызываются — мы это проверили на iOS 26.5: после показа и закрытия листа счётчик `viewDidAppear` не изменился, а после `.fullScreen` вырос на единицу.

Коротко о **жизненном цикле** экрана (подробно — глава 0, раздел 0.6): это порядок, в котором UIKit вызывает методы view controller'а. Для одного показа он такой: `viewDidLoad` (один раз за всю жизнь экрана) → `viewWillAppear` (сейчас появится) → `viewWillLayoutSubviews` / `viewDidLayoutSubviews` (раскладка; здесь уже известны рамки и safe area) → `viewDidAppear` (появился) → … → `viewWillDisappear` → `viewDidDisappear`. Всё, кроме `viewDidLoad`, может повторяться: каждый раз, когда экран уходит и возвращается, а методы раскладки — ещё и при повороте и смене размера.

1.4 Анимация — по очереди и с пружиной

Метод `runAnimation()` запускает три анимации с разной задержкой, чтобы элементы появлялись по очереди: сначала иконка, потом название, потом индикатор.

Иконка:

```
    private func runAnimation() {
        UIView.animate(withDuration: 0.5,
                       delay: 0.05,
                       usingSpringWithDamping: 0.65,
                       initialSpringVelocity: 0.4,
                       options: []) {
            self.iconView.alpha = 1.0
            self.iconView.transform = .identity
        }
```

`UIView.animate` работает так: ты в замыкании пишешь, каким view должен **стать** (прозрачность 1, обычный размер), а UIKit плавно проводит его из текущего состояния в новое за `withDuration` секунд — здесь за полсекунды. `delay: 0.05` — подождать 0.05 секунды (пять сотых) перед стартом.

Главное здесь — **пружина**: `usingSpringWithDamping` и `initialSpringVelocity`. Иконка не просто растёт с 60% до 100%, а чуть **перелетает** 100%, становится на мгновение крупнее и возвращается назад — как мячик на резинке.

- **`damping` (затухание) 0.65.** Число от 0 до 1: насколько быстро гасятся колебания. При 1.0 пружины нет вовсе — иконка плавно подъезжает к 100% и останавливается. Чем меньше число, тем сильнее перелёт и тем больше «качаний». 0.65 — заметный, но один короткий

перелёт: чуть перелетит и вернётся. 0.3 — иконка качнётся туда-сюда несколько раз. Для заставки обычно берут 0.6–0.8.

- **`velocity` (начальная скорость) 0.4.** С какой скоростью иконка «стартует». Единица здесь относительная: 1 означает «с такой скоростью, чтобы пройти весь путь анимации за одну секунду». Наш путь — от 60% до 100% размера, то есть 40 процентных пунктов; 0.4 — это старт со скоростью 0.4 × 40 = 16 пунктов в секунду, мягкий толчок. 0 — старт с места, 2–3 — резкий «выстрел».

`options: []` — никаких дополнительных опций. Кривую разгона и торможения у пружинной анимации задаёт сама пружина, поэтому кривые вроде `.curveEaseOut` здесь не нужны.

Заголовок и индикатор появляются обычным плавным проявлением (**fade-in**), без пружины:

```
        UIView.animate(withDuration: 0.4, delay: 0.35, options: [.curveEaseOut]) {
            self.titleLabel.alpha = 1.0
        }
        UIView.animate(withDuration: 0.3, delay: 0.7, options: [.curveEaseOut]) {
            self.activityIndicator.alpha = 1.0
        }
```

`.curveEaseOut` — **кривая замедления**: анимация стартует быстро и тормозит к концу, как машина, которая подъезжает к светофору. Для появления это выглядит естественнее, чем равномерное (линейное) изменение: элемент «выплывает» и мягко встаёт на место. Бывает и обратная кривая — `.curveEaseIn`, разгон с места, и `.curveEaseInOut` — разгон, потом торможение.

Задержки 0.05 → 0.35 → 0.7 секунды дают **stagger** (англ. «вразнобой») — элементы появляются друг за другом, а не одновременно. Вот расписание в секундах от запуска `runAnimation()`:

Элемент	Начало	Конец
иконка (пружина)	0.05	0.55
заголовок	0.35	0.75
индикатор	0.70	1.00

Анимации перекрываются: заголовок начинает проявляться, пока иконка ещё «качается». Так появление выглядит цельным, а не как три отдельных шага.

И завершение:

```
        DispatchQueue.main.asyncAfter(deadline: .now() + manifest.splashDuration) { [weak
↪  self] in
            guard let self else { return }
            UIView.animate(withDuration: 0.25, animations: {
                self.activityIndicator.alpha = 0
            }, completion: { _ in
                self.onFinish()
            })
        }
    }
}
```

`DispatchQueue.main.asyncAfter` — «выполни этот код на главном потоке через столько-то секунд». Через `manifest.splashDuration` секунд (по умолчанию 1.4) прячем индикатор за четверть секунды (0.25) и в `completion` — замыкании, которое UIKit вызывает по окончании анимации, — зовём `onFinish`. Итого splash живёт на экране около 1.4 + 0.25 ≈ 1.65 секунды.

Про `[weak self]`. Это **список захвата**: замыкание держит splash не сильной, а слабой ссылкой. Сильной ссылки тут было бы достаточно, чтобы не случилось утечки памяти: очередь держит замыкание, замыкание держит splash, но splash не держит ни очередь, ни замыкание, так что цикла нет — через 1.4 секунды замыкание выполнится, отпустит splash, и всё освободится. Но с сильной ссылкой splash прожил бы эти 1.4 секунды **обязательно**, даже если пользователь уже встряхнул телефон и ушёл в лаунчер, и потом ещё и вызвал бы `onFinish` для mini-app, которого на экране уже нет. Со `[weak self]` всё проще: если splash уже никому не нужен и освобождён, `self` будет `nil`, и `guard` тихо выходит.

Упражнение 1.2. В `runAnimation()` поменяй `usingSpringWithDamping: 0.65` на `0.3`, запусти любое mini-app и посмотри на иконку. Потом поставь `1.0`. Опиши словами разницу между тремя вариантами.

Упражнение 1.3. Запусти «Заметки» — splash у них жёлтый (`.systemYellow`), а текст белый. Прочитать название трудно. Напиши расширение `UIColor` со свойством `readableForeground`, которое для светлого цвета возвращает `.black`, а для тёмного — `.white`, и используй его в `setupLayout()` для заголовка, иконки и индикатора.

1.5 Бытовая аналогия

`LaunchScreen` — это **витрина магазина до открытия**. Её оформили заранее, и она не меняется: подходишь к двери, видишь зал через стекло. Хорошая витрина показывает то, что внутри, — поэтому, когда дверь открывается, ты не удивлён.

`AnimatedSplash` — это **продавец, который встречает тебя у входа**: «Здравствуйте, сейчас всё покажу». Он уже живой, может что-то сказать и сделать, пока в подсобке готовят твой заказ.

Витрина нужна всегда: без неё закрытый магазин выглядит заброшенным. Встречающий — по желанию, но с ним вход приятнее.

1.6 Почему оба, а не только AnimatedSplash

Резонный вопрос: если у нас есть свой экран с анимацией, зачем launch screen? Нельзя ли удалить storyboard и обойтись одним?

Нельзя, по двум причинам.

Первая — техническая. Документация Apple («Specifying your app's launch screen») начинается со слов «Every iOS app must provide a launch screen» — каждое приложение обязано его иметь. Исторически приложение без launch screen iOS считала старым, созданным до появления экранов разных размеров, и запускала в **режиме совместимости**: интерфейс рисовался в маленьком окне размером со старый iPhone, с чёрными полосами вокруг. Такое приложение и App Store не примет — с 2020 года launch screen обязателен.

Вторая — про ощущения. Пока процесс запускается, твоего кода ещё нет, и AnimatedSplash эту фазу прикрыть не может: его view controller ещё не существует. Экран в это время покажет либо launch screen, либо ничего.

> **Итог.** `LaunchScreen` — для системы: ты задаёшь его заранее и после сборки на него не влияешь. Он должен быть похож на первый экран. `AnimatedSplash` — для пользователя: ты решаешь, что в нём показать и когда он закончится.

1.7 Кто решает, когда AnimatedSplash закончился

Это **не** забота самого `AnimatedSplashViewController`. Он знает только «прошло `splashDuration` секунд → вызови `onFinish`». Что делать дальше, решает объект уровнем выше.

В нашем playground'е это `BootCoordinator` (глава 4). Вот как он создаёт splash:

```
private func showSplash() {
    let splash = AnimatedSplashViewController(manifest: manifest) { [weak self] in
        self?.proceedAfterSplash()
    }
    setRoot(splash, animated: false)
}
```

Координатор передаёт splash'у callback. Когда splash его вызовет, координатор пойдёт дальше — к onboarding, экрану разрешений, входу в аккаунт и так далее. Splash **ничего не знает** про эти экраны.

Эту схему — «координатор владеет цепочкой, экраны зовут callback» — подробно разберём в главе 4. Сейчас важно одно: **splash должен быть простым**. Он умеет показать анимацию и сообщить «я закончил». Что дальше — не его забота.

Ответы к упражнениям

1.1. В первую долю секунды после тапа экран будет красным — это launch screen, — а потом появится лаунчер. Если после возврата серого фона красный не исчез, дело в кэше: iOS показывает сохранённый снимок старого launch screen. Удали приложение с симулятора и запусти заново; если и это не помогло — перезапусти симулятор (Device → Restart).

1.2. При `0.3` иконка заметно перелетает 100%, возвращается, снова чуть перелетает — качается несколько раз. При `0.65` — один короткий перелёт. При `1.0` перелёта нет: иконка плавно дорастает до 100% и останавливается.

1.3. Яркость цвета удобно оценить по формуле, которой пользуются в телевидении: зелёный вносит в воспринимаемую яркость больше всего, синий — меньше всего. Каждый канал цвета — число от 0 до 1; берём 30% красного, 59% зелёного и 11% синего и складываем:

```
import UIKit

extension UIColor {
    /// Чёрный или белый — что лучше читается поверх этого цвета.
    var readableForeground: UIColor {
```

```
        var red: CGFloat = 0, green: CGFloat = 0, blue: CGFloat = 0, alpha: CGFloat = 0
        getRed(&red, green: &green, blue: &blue, alpha: &alpha)
        let brightness = 0.299 * red + 0.587 * green + 0.114 * blue
        return brightness > 0.6 ? .black : .white
    }
}
```

Проверим на числах. `systemYellow` в светлой теме — это примерно красный 1.0, зелёный 0.8, синий 0.0. Яркость: 0.299 × 1.0 + 0.587 × 0.8 + 0.114 × 0 ≈ 0.77 — больше 0.6, значит текст чёрный. `systemBlue` — примерно 0.0, 0.48, 1.0: яркость ≈ 0.40, текст белый.

В `setupLayout()` вместо `.white` пиши `manifest.brandColor.readableForeground` — для `iconView.tintColor`, `titleLabel.textColor` и `activityIndicator.color`.

Порог 0.6 подобран на глаз. Точная проверка читаемости — коэффициент контраста из стандарта доступности WCAG, о нём подробно в главе 34, раздел 34.7.

Что мы выучили

- **Launch screen** — статичная картинка для **системы**, её iOS показывает, пока запускается процесс. Задаётся storyboard'ом или ключом `UILaunchScreen` в `Info.plist`, без кода и своих классов. iOS кэширует её снимок.
- По HIG launch screen должен быть **похож на первый экран**, без текста и логотипов. Логотип и анимация — в своём splash.
- Без launch screen приложение запускается в режиме совместимости, и App Store его не примет.
- `AnimatedSplashViewController` — обычный view controller. Он появляется после того, как `SceneDelegate` поставил первый экран, и может анимировать, ходить в сеть — что угодно.
- `view` создаётся лениво (`loadView` → `viewDidLoad`, один раз). В `viewDidLoad` экран ещё не в окне, safe area и поля нулевые — ставим constraints, а не читаем координаты.
- Интерфейс собираем в `viewDidLoad`, анимацию появления запускаем в `viewDidAppear` и защищаем флажком от повторного запуска: он повторяется после `.fullScreen`-экрана и возврата по навигации (но не после обычного листа).
- Auto Layout: правила-constraints вместо координат; у каждого view из кода — `translatesAutoresizingMa = false`. `transform` на раскладку не влияет: `bounds` и `center` те же, а `frame` после transform не определён.
- Пружина (`usingSpringWithDamping`): чем меньше damping, тем сильнее перелёт; при 1.0 пружины нет. Stagger — задержки между элементами.
- `[weak self]` в отложенном замыкании не даёт splash'у жить дольше нужного и звать `onFinish` для закрытого экрана.
- Splash сам не знает, что идёт после него. Решает координатор.

Apple Developer Documentation

- Specifying your app's launch screen — два способа задать launch screen и список ограничений launch storyboard.

- Launching — Human Interface Guidelines — каким должен быть launch screen и чем он отличается от splash.
- `UILaunchStoryboardName` и `UILaunchScreen` — ключи `Info.plist` для storyboard и для экрана без storyboard.
- `UIViewController` — базовый класс `AnimatedSplashViewController`; методы `viewDidLoad` / `viewDidAppear`.
- `loadView()` и `isViewLoaded` — ленивое создание `view`.
- `viewDidLayoutSubviews()` и `viewSafeAreaInsetsDidChange()` — где уже известны рамки и safe area.
- `translatesAutoresizingMaskIntoConstraints` — почему `true` по умолчанию конфликтует с твоими constraints.
- `UIView.transform` — transform не влияет на Auto Layout, а `frame` при нём не определён.
- `UIView.animate(withDuration:delay:usingSpringWithDamping:initialSpringVelocity:options:animations` — пружинная анимация из `runAnimation()`.
- `NSLayoutConstraint` — constraints Auto Layout.
- `UIImage.SymbolConfiguration` — размер и толщина SF Symbol.
- Closures — Swift Book — списки захвата и `[weak self]`.

→ Глава 2. AppManifest — конфиг одного mini-app

Глава 2. AppManifest — конфиг одного mini-app

В нашем playground'е десять мини-приложений и тур «Anatomy». У каждого своё имя, иконка, цвет и свой набор **гейтов запуска** — экранов, которые пользователь проходит до main-экрана: одному нужен onboarding, другому — проверка возраста, третьему — Face ID при возврате из фона.

Можно было бы описать каждое mini-app отдельным классом:

```
// Так делать НЕ будем
class TodoApp { /* всё про Todo */ }
class NotesApp { /* всё про Notes */ }
class CalculatorApp { /* всё про Калькулятор */ }
```

Но это путь к десятку почти одинаковых классов со скопированной логикой — и к неизбежным расхождениям: где-то splash длится 1.4 секунды, где-то 1.2, где-то иконка 80 точек, где-то 84. Через полгода в этом не разберётся даже автор.

Поэтому мы описываем mini-app **данными, а не кодом**. Один тип `AppManifest` с полями «имя, иконка, цвет, флаги» и один реестр `AppRegistry` с массивом таких описаний. Код, который по этим данным что-то делает, пишется один раз — в координаторе (глава 4) и лаунчере.

В этой главе разбираем оба файла и заглушку для ещё не готовых mini-app.

2.1 Манифест как «паспорт» mini-app

> **Идея.** Манифест — это структура, которая **описывает** mini-app: что показать в лаунчере, какие гейты пройти, какой экран сделать главным. Сам манифест ничего не **делает**. Делают другие — `BootCoordinator` и `AppListViewController`.

Слово «манифест» взято из мира сборки программ: так называют файл с описанием пакета — имя, версия, зависимости. Наш манифест — такое же описание, только для mini-app.

Файл: `App/AppManifest.swift`. Начнём с блока «кто я» (identity):

```
import UIKit

struct AppManifest {

```

```
    // MARK: Identity — задаются один раз
    let id: String          // "todo", "notes", "calculator"
    let name: String        // "Список дел"
    let subtitle: String    // "UITableView, ячейка-чек, UserDefaults"
    let symbolName: String  // SF Symbol: "checklist"
    let brandColor: UIColor // .systemBlue
    let isReady: Bool       // false → в лаунчере пометка «СКОРО»
```

`id` — стабильный идентификатор, короткая строка латиницей. По нему гейты хранят свои отметки в **UserDefaults** — маленьком встроенном хранилище настроек вида «ключ → значение», которое переживает перезапуск приложения. Например, онбординг из главы 6 запоминает «этот mini-app онбординг уже показывал» под ключом `onboarding.seen.todo`. Поэтому **менять `id` после релиза нельзя**: ключи станут другими, и у всех пользователей «забудутся» пройденные шаги.

`name`, `subtitle`, `symbolName`, `brandColor` — то, что пользователь видит в лаунчере. Иконка — SF Symbol по имени (`UIImage(systemName:)`, см. главу 1). Цвет работает трижды: иконка в строке лаунчера, фон splash'а и цвет кнопок на панели навигации внутри mini-app.

`isReady` — готово ли mini-app. Если `false`, в строке лаунчера стоит пометка «СКОРО», а по тапу откроется не настоящий экран, а заглушка (раздел 2.4).

`// MARK:` — не код, а комментарий-закладка: Xcode показывает такие заголовки в выпадающем меню над редактором, по ним удобно прыгать по большому файлу.

2.2 Флаги гейтов запуска

Дальше — поля, которые включают и выключают каждый гейт:

```
    // MARK: Флаги гейтов запуска — у каждого значение по умолчанию
    var hasAnimatedSplash: Bool = true
    var splashDuration: TimeInterval = 1.4

    var hasOnboarding: Bool = false
    var onboardingPages: [OnboardingPage] = []

    var hasAuthGate: Bool = false
    var checksForceUpdate: Bool = false
    var hasMaintenanceCheck: Bool = false
    var requiresPermission: PermissionKind? = nil
    var hasPrivacyBlurOnBackground: Bool = false
    var requiresBiometricOnResume: Bool = false

    var requiresRegionPick: Bool = false
    var requiresAgeGate: Bool = false
    var minAgeYears: Int = 13
```

`TimeInterval` — это просто `Double`, число секунд: 1.4 — одна целая и четыре десятых секунды.

Каждое поле — **одно решение**, которое координатор принимает при запуске mini-app. Вот что за ними стоит и где это разбирается:

Поле	Что включает	Глава
`hasAnimatedSplash`, `splashDuration`	наш splash и его длительность	1
`hasOnboarding`, `onboardingPages`	вводные экраны при первом запуске	6
`requiresPermission`	экран «зачем нам разрешение» до системного вопроса	7
`hasAuthGate`	вход в аккаунт, если пользователь не вошёл	8
`checksForceUpdate`, `hasMaintenanceCheck`	«обновите приложение» и «ведутся работы» по ответу сервера	9
`requiresRegionPick`, `requiresAgeGate`, `minAgeYears`	выбор региона и проверка возраста	10
`hasPrivacyBlurOnBackground`, `requiresBiometricOnResume`	размытие в переключателе приложений и Face ID при возврате	11

Все флаги — `var` со значениями по умолчанию. Их тринадцать, и перечислять все при создании манифеста не нужно — только те, что **отличают** этот mini-app от обычного.

Как это работает. Для структуры Swift сам создаёт **поэлементный инициализатор** (memberwise initializer): по параметру на каждое хранимое свойство, **в том порядке, в каком свойства объявлены**. У свойств со значением по умолчанию параметр становится необязательным. Поэтому можно написать так:

```
AppManifest(
    id: "profile",
    name: "Профиль / Настройки",
    subtitle: "insetGrouped, разные типы ячеек",
    symbolName: "person.crop.circle",
    brandColor: .systemPurple,
    isReady: true,
    hasAuthGate: true,
    hasPrivacyBlurOnBackground: true,
    requiresBiometricOnResume: true,
    makeMain: { ProfileViewController() }
)
```

Пропущенные флаги возьмут значения по умолчанию. Одно правило: указанные параметры должны идти в том же порядке, что и свойства в структуре. Напишешь `requiresBiometricOnResume` раньше `hasAuthGate` — компилятор скажет, что аргумент стоит не на своём месте.

(`ProfileViewController` появится в главе 19 — пока это просто пример того, как будет выглядеть манифест с несколькими гейтами.)

Почему `var`, а не `let`. Флаги удобно менять после создания манифеста — «создал, потом поправил одно поле»:

```
var m = AppManifest.placeholder(id: "music", name: "Музыка",
                                subtitle: "AVPlayer", symbolName: "music.note",
                                brandColor: .systemRed)
m.checksForceUpdate = true
m.requiresAgeGate = true
m.minAgeYears = 16
```

Будь флаги `let`, так бы не получилось. А вот поля «кто я» (`id`, `name`…) — `let`: после создания их менять нельзя.

`requiresPermission` — опциональный enum, а не `Bool`. Разрешения бывают разные (геолокация, фото, камера, уведомления), и нам важно не «нужно ли», а «какое именно»: `nil` — никакое, `.location` — геолокация. Mini-app просит максимум **одно** основное разрешение. Если когда-нибудь понадобится несколько сразу, поле превратится в `Set<PermissionKind>`. Пока это лишнее: ни одному нашему mini-app больше одного не нужно.

Типы для гейтов: `PermissionKind` и `OnboardingPage`

Флаги ссылаются на два типа, которых ещё нет. Объявим их в том же файле, над `AppManifest`:

```
/// Какое системное разрешение mini-app просит до main-экрана (глава 7).
enum PermissionKind: Sendable {
    case location
    case photoLibrary
    case camera
    case notifications
}

/// Одна страница онбординга (глава 6).
struct OnboardingPage {
    let symbolName: String
    let title: String
    let body: String
}
```

Экраны, которые эти типы используют, появятся в главах 6 и 7, а сами типы нужны уже сейчас: без них не соберётся манифест. Держим их рядом с `AppManifest`, потому что это часть его «словаря».

`Sendable` — пометка «значение можно безопасно передавать между потоками». Простой enum без связанных значений и так безопасен, а пометка делает это явным: в главе 7 значение `PermissionKind` уйдёт в асинхронный запрос разрешения.

2.3 Фабрика `makeMain` — как создавать main-экран

Последнее поле манифеста — самое интересное:

```
    // MARK: Фабрика main-экрана
    let makeMain: @MainActor () -> UIViewController
}
```

Это **функция**, которая создаёт и возвращает main-экран. Манифест хранит не готовый view controller, а замыкание, которое умеет его сделать. Такой приём называется **фабрикой**: не сам товар, а цех, который выпускает товар по заказу.

Зачем так? Две причины.

Экран создаётся только когда нужен. `AppRegistry.allApps` создаётся при первом обращении к нему — когда создаётся лаунчер. Если бы каждый манифест держал готовый экран, при запуске playground'а сразу создались бы десять view controller'ов со своими данными — хотя пользователь, может, не зайдёт ни в один. С замыканием экран создаётся **в момент** запуска mini-app. Зашёл в Калькулятор — создался `CalculatorViewController`. Вышел в лаунчер — на него больше никто не ссылается, и он освобождается из памяти. В памяти живёт только то mini-app, в котором пользователь сейчас.

Каждый вход — с чистого листа. Зашёл в «Список дел», вышел, зашёл снова — хотим **свежий** экран. Если бы манифест держал один и тот же экземпляр, при втором входе ты увидел бы его в том состоянии, в каком оставил: прокрученный список, открытое меню. С фабрикой каждый вход — новый экземпляр.

`@MainActor` в типе замыкания говорит: «вызывать это замыкание можно только на главном потоке». Вызывать его в другом месте всё равно нельзя: внутри создаётся view controller, а весь UIKit работает на главном потоке (см. раздел 0.1 введения).

Строго говоря, в нашем проекте эту пометку можно не писать: с Default Actor Isolation = MainActor замыкание, написанное в нашем коде, и так выполняется на главном потоке. Но стоит перенести `AppManifest` в модуль без этой настройки (например, в отдельный Swift-пакет), и без `@MainActor` компилятор Swift 6 откажется собирать реестр: «static property ‘allApps’ is not concurrency-safe» — глобальная константа не может хранить значение, которое нельзя безопасно передавать между потоками. С пометкой такое замыкание разрешено передавать между потоками (всё равно выполнится оно только на главном), и ошибки нет: остальные поля манифеста — `String`, `Bool`, `UIColor` — и так `Sendable` (`UIColor` Apple пометила `NS_SWIFT_SENDABLE` в заголовке `UIColor.h`), так что вся структура становится безопасной для передачи. Мы проверили оба варианта компилятором Swift 6.3 в модуле без Default Actor Isolation: без `@MainActor` — именно эта ошибка, с ним — чисто. Мы пишем `@MainActor` явно, чтобы код не зависел от настройки проекта.

Когда mini-app будет готово (например, «Список дел» после главы 12), его манифест будет выглядеть так:

```
AppManifest(
    id: "todo",
    name: "Список дел",
    subtitle: "UITableView, ячейка-чек, UserDefaults, dummyjson",
    symbolName: "checklist",
    brandColor: .systemBlue,
    isReady: true,
    makeMain: { TodoListViewController() }
```

```
)
```

`makeMain: { TodoListViewController() }` — короткое замыкание, которое просто создаёт экран. Внутри может быть что угодно: передача зависимостей, настройка, выбор экрана по условию.

2.4 Заготовка для ещё не готовых mini-app

В лаунчере должны быть **все** mini-app — и готовые, и те, что пока только в плане («СКОРО»). Чтобы не повторять одинаковые поля у каждой заглушки, заведём **фабричный метод** — статическую функцию, которая собирает манифест. Файл `App/AppManifest+Placeholder.swift` (плюс в имени файла — принятое в Swift обозначение «расширение такого-то типа»):

```
import UIKit

extension AppManifest {
    static func placeholder(id: String,
                            name: String,
                            subtitle: String,
                            symbolName: String,
                            brandColor: UIColor) -> AppManifest {
        AppManifest(
            id: id,
            name: name,
            subtitle: subtitle,
            symbolName: symbolName,
            brandColor: brandColor,
            isReady: false,
            hasAnimatedSplash: true,
            makeMain: { PlaceholderViewController(name: name) }
        )
    }
}
```

Заглушка — это манифест с `isReady: false` и экраном `PlaceholderViewController`. Обрати внимание: замыкание `makeMain` **захватывает** параметр `name` — запоминает его значение, чтобы использовать позже, когда замыкание вызовут.

Используется так (тип `AppManifest` Swift выведет из контекста, поэтому достаточно точки):

```
.placeholder(
    id: "calculator",
    name: "Калькулятор",
    subtitle: "UIStackView grid, состояние, haptics",
    symbolName: "function",
    brandColor: .systemIndigo
)
```

Когда в главе 14 появится настоящий Калькулятор, эту запись заменим на полный `AppManifest(...)` с `isReady: true` и `makeMain: { CalculatorViewController() }`. Лаунчер подхватит это сам: пометка «СКОРО» пропадёт.

Экран-заглушка

`App/PlaceholderViewController.swift` — простой экран: иконка молотка по центру и текст «ещё не реализовано». Это лучше пустого экрана: пользователь понимает, что попал куда хотел, просто здесь пока ничего нет.

```swift
import UIKit

/// Заглушка для mini-app, которое ещё не реализовано.
final class PlaceholderViewController: UIViewController {

    private let name: String

    init(name: String) {
        self.name = name
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется: экран создаётся только кодом")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = name
        view.backgroundColor = .systemBackground

        let config = UIImage.SymbolConfiguration(pointSize: 56)
        let icon = UIImageView(image: UIImage(systemName: "hammer.fill",
                                              withConfiguration: config))
        icon.tintColor = .secondaryLabel

        let label = UILabel()
        label.text = "«\(name)» ещё не реализовано.\n"
            + "Встряхни устройство, чтобы вернуться в лаунчер."
        label.numberOfLines = 0
        label.textAlignment = .center
        label.font = .preferredFont(forTextStyle: .body)
        label.adjustsFontForContentSizeCategory = true
        label.textColor = .secondaryLabel

        let stack = UIStackView(arrangedSubviews: [icon, label])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 16
```

```
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)

        let guide = view.readableContentGuide
        NSLayoutConstraint.activate([
            stack.centerYAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.centerYAnchor),
            stack.leadingAnchor.constraint(equalTo: guide.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: guide.trailingAnchor),
        ])
    }
}
```

Что здесь нового по сравнению со splash'ем из главы 1.

`title = name` — заголовок экрана. Координатор покажет main-экран внутри панели навигации (глава 4), и заголовок появится на ней. Как это устроено: у каждого view controller'а есть свой `navigationItem` (класс `UINavigationItem`, потомок `NSObject`, а не view), и присваивание `title` обновляет его `navigationItem.title`; панель `UINavigationBar` лишь показывает item того экрана, который сейчас сверху. Мы проверили на симуляторе: после `title = "T"` у `navigationItem.title` то же значение, а `navigationItem` в `UINavigationController` совпадает с `navigationBar.topItem` (подробно — глава 0, разделы 0.6 и 0.8).

`.systemBackground`, `.secondaryLabel` — **семантические цвета**: их имя говорит не «какой цвет», а «для чего». `systemBackground` — фон (белый в светлой теме, чёрный в тёмной), `secondaryLabel` — второстепенный текст (серый, в обеих темах читаемый). Пишешь семантический цвет — и тёмная тема работает сама.

`UIStackView` — контейнер, который сам выстраивает вложенные view в столбик (`axis = .vertical`) или в строку, с промежутком `spacing` — здесь 16 точек. `alignment = .center` — каждый элемент по центру столбика. Иконке и лейблу constraints не нужны: их расставляет стек, а constraints задаём только самому стеку.

Стек — обычный `UIView` (`UIStackView → UIView`), но его работа — раскладка, а не рисование. Он держит два списка: `arrangedSubviews` (те, кого расставляет) и обычные `subviews`; первый всегда входит во второй. `addArrangedSubview` (и `init(arrangedSubviews:)`) добавляет view и в тот, и в другой, а простой `addSubview` — только в `subviews`: такой view стек не расставляет. Для каждого arranged subview стек сам выставляет `translatesAutoresizingMaskIntoConstraints = false` и строит constraints по его `intrinsicContentSize` — мы проверили это на симуляторе, поэтому иконке и лейблу эта строка не нужна (подробно — глава 0, раздел 0.3).

`readableContentGuide` — полоса «удобной для чтения ширины». На iPhone она почти совпадает с полями экрана, а на широком iPad ограничивает ширину текста, чтобы строка не растянулась на весь экран (длинные строки читать тяжело). Как и `safeAreaLayoutGuide` из главы 1, это `UILayoutGuide` — прямоугольник для constraints, а не view. Стек привязан к ней слева и справа и стоит по центру safe area по вертикали.

Текст склеен из двух строк через `+`, а `\n` внутри — перевод строки.

2.5 AppRegistry — реестр всех mini-app

Все манифесты лежат в одном массиве, в `App/AppRegistry.swift`. Пока ни одно mini-app не готово, реестр состоит из заглушек:

```
import UIKit

enum AppRegistry {

    static let allApps: [AppManifest] = [
        .placeholder(id: "todo", name: "Список дел",
                     subtitle: "UITableView, ячейка-чек, UserDefaults, dummyjson",
                     symbolName: "checklist", brandColor: .systemBlue),
        .placeholder(id: "notes", name: "Заметки",
                     subtitle: "UITextView, файловое хранилище, поиск",
                     symbolName: "note.text", brandColor: .systemYellow),
        .placeholder(id: "calculator", name: "Калькулятор",
                     subtitle: "UIStackView grid, состояние, haptics",
                     symbolName: "function", brandColor: .systemIndigo),
        .placeholder(id: "weather", name: "Погода",
                     subtitle: "URLSession, pull-to-refresh, skeleton",
                     symbolName: "cloud.sun.fill", brandColor: .systemTeal),
        .placeholder(id: "gallery", name: "Галерея",
                     subtitle: "Compositional layout, пагинация, photo viewer",
                     symbolName: "photo.on.rectangle", brandColor: .systemPink),
        .placeholder(id: "music", name: "Музыка",
                     subtitle: "AVPlayer, mini-player, full sheet",
                     symbolName: "music.note", brandColor: .systemRed),
        .placeholder(id: "chat", name: "Чат",
                     subtitle: "Двусторонние ячейки, клавиатура",
                     symbolName: "bubble.left.and.bubble.right.fill", brandColor:
↪   .systemGreen),
        .placeholder(id: "profile", name: "Профиль / Настройки",
                     subtitle: "insetGrouped, разные типы ячеек",
                     symbolName: "person.crop.circle", brandColor: .systemPurple),
        .placeholder(id: "tabbar", name: "Custom Tab Bar",
                     subtitle: "Свой контейнер вместо UITabBarController",
                     symbolName: "square.split.bottomrightquarter", brandColor:
↪   .systemBrown),
        .placeholder(id: "layouts", name: "Сложные экраны",
                     subtitle: "Stretchy header, sticky, parallax",
                     symbolName: "rectangle.3.group", brandColor: .systemCyan),
        .placeholder(id: "anatomy", name: "Anatomy",
                     subtitle: "Тур по всем гейтам запуска",
                     symbolName: "list.bullet.rectangle", brandColor: .systemGray),
    ]
}
```

Три детали, которые легко проглядеть.

Это `enum` без `case`'ов, а не `struct` или `class`. Экземпляр такого enum создать нельзя вообще: `AppRegistry()` не скомпилируется. Значит, никто не заведёт «свой» реестр с отдельным состоянием — реестр один на всё приложение. Пустой enum как «папка» для статических свойств и функций — обычный приём в Swift, его называют **namespace enum** (enum-пространство имён).

Про изоляцию. В нашем режиме (Default Actor Isolation = MainActor) `AppRegistry` и его `allApps` автоматически принадлежат главному потоку, поэтому отдельная пометка `@MainActor` здесь не нужна. Лаунчер тоже работает на главном потоке, и читает массив без всяких `await`.

Порядок имеет значение. Это массив (`Array`), а не множество (`Set`): в каком порядке перечислены манифесты, в таком они и появятся в лаунчере.

Один цвет выбран не случайно: `.systemCyan` появился только в iOS 15. На iOS 14 такого свойства нет, но наш минимум — iOS 15, так что его можно использовать без проверок версии.

2.6 Кто читает манифест

Лаунчер (`AppListViewController`, глава 4) при создании берёт массив:

```
private let apps: [AppManifest] = AppRegistry.allApps
```

и показывает его в таблице — по строке на манифест. По тапу отдаёт манифест координатору, а тот смотрит на флаги:

```
if manifest.hasAnimatedSplash {
    showSplash()
} else {
    proceedAfterSplash()
}
```

И так — для каждого гейта по очереди. То есть **манифест полностью описывает поведение mini-app при запуске**. Координатор без манифеста не знает, что делать; манифест без координатора — мёртвые данные. Работают они только в паре.

> **Идея.** Манифест и координатор — это разделение «**конфигурация и исполнение**». Конфигурация (`AppManifest`) пассивна, легко читается, ничего не делает. Исполнение (`BootCoordinator`) активно: читает конфигурацию и решает, что показать.

2.7 Бытовая аналогия

Манифест — **меню в кафе**. В нём написано: «Капучино — 900 ₸, средний, на обычном молоке». Меню само ничего не готовит.

Бариста (координатор) читает меню, смотрит на твой заказ и **делает**. Если бы бариста работал без меню, каждый раз решал бы сам, что варить, и кофе был бы каждый день разный.

`isReady: false` — пометка «скоро в меню». Пункт ты видишь, но заказать пока нельзя: принесут табличку «готовим».

Упражнение 2.1. Докажи, что при выходе в лаунчер экран mini-app действительно освобождается из памяти, а при повторном входе создаётся новый. Добавь в `PlaceholderViewController` что-нибудь, что напечатает сообщение при создании экрана и при его освобождении. Проверять удобнее после главы 5, когда фундамент соберётся целиком.

2.8 Что манифест не описывает

Манифест не описывает **внутреннюю** работу mini-app. Если у Калькулятора есть «текущее число» и «отложенная операция», это живёт **внутри** калькулятора, а не в манифесте. Манифест знает «как запустить», а не «как работать».

Это различие полезно держать в голове. Хочешь добавить в манифест поле — спроси себя: это про **запуск** или про **работу**? Если про работу — полю место внутри самого mini-app.

Упражнение 2.2. В `AppRegistry.swift` поменяй местами «Список дел» и «Заметки» (перенеси целиком строки `.placeholder(...)`). Что изменится в лаунчере? Какой код таблицы для этого пришлось поменять?

Упражнение 2.3. Сделай так, чтобы «Чат» открывался **без** splash — сразу main-экран. Меняй только `AppRegistry.swift`.

Ответы к упражнениям

2.1. Добавь в `PlaceholderViewController` строку в конец `init` и метод `deinit` — его Swift вызывает прямо перед освобождением объекта:

```
    init(name: String) {
        self.name = name
        super.init(nibName: nil, bundle: nil)
        print("Создан экран «\(name)»")
    }

    deinit {
        print("Освобождён экран «\(name)»")
    }
```

Зайди в любое mini-app, встряхни (^⌘Z), зайди снова. В консоли Xcode появится «Создан экран…», после встряхивания — «Освобождён экран…», и при повторном входе — снова «Создан экран…». Два «Создан» подряд с «Освобождён» между ними и доказывают, что `makeMain` каждый раз создаёт новый экран, а старый не задерживается в памяти.

2.2. «Заметки» станут первой строкой, «Список дел» — второй. Код таблицы менять не пришлось: лаунчер показывает массив в том порядке, в каком он записан. В этом и смысл описания данными — поведение меняется правкой данных.

2.3. Заглушку создаёт `.placeholder(...)`, а флаги — `var`, поэтому можно создать манифест и поправить его прямо внутри массива через замыкание, которое сразу же вызывается:

```
        {
            var chat = AppManifest.placeholder(
```

```
            id: "chat", name: "Чат",
            subtitle: "Двусторонние ячейки, клавиатура",
            symbolName: "bubble.left.and.bubble.right.fill",
            brandColor: .systemGreen)
        chat.hasAnimatedSplash = false
        return chat
    }(),
```

Запись `{ ... }()` — замыкание, которое объявили и тут же вызвали: его результат (готовый манифест) и попадает в массив. При запуске Чата координатор увидит `hasAnimatedSplash == false`, пропустит `showSplash()` и сразу вызовет `proceedAfterSplash()`, а оттуда цепочка дойдёт до main-экрана.

Что мы выучили

- `AppManifest` — структура, которая **описывает** mini-app. Сама ничего не делает.
- Поля «кто я» (`id`, `name`, `symbolName`, `brandColor`…) — `let`. `id` после релиза менять нельзя: на нём держатся ключи в UserDefaults.
- Флаги гейтов — `var` со значениями по умолчанию: в поэлементном инициализаторе указываешь только отличия, в порядке объявления свойств.
- `PermissionKind` и `OnboardingPage` объявлены рядом с манифестом: они нужны ему уже сейчас, а экраны для них появятся в главах 6–7.
- `makeMain: @MainActor () -> UIViewController` — **фабрика** main-экрана: экран создаётся лениво и заново при каждом входе. `@MainActor` делает код независимым от настройки изоляции проекта.
- `AppManifest.placeholder(...)` — короткая запись для заглушек «СКОРО»; экран-заглушка — `PlaceholderViewController`.
- `title` экрана живёт в его `navigationItem` (не view), панель лишь показывает его. `UIStackView` расставляет только `arrangedSubviews` и сам снимает у них `translatesAutoresizingMaskIntoCo`
- `AppRegistry` — пустой enum-«папка» с `static let allApps`. Порядок массива = порядок в лаунчере.
- Манифест + координатор = конфигурация + исполнение. Манифест описывает **запуск**, а не **работу** mini-app.

Apple Developer Documentation

- Structures and Classes — Swift Book — почему `AppManifest` — структура и откуда берётся поэлементный инициализатор.
- Properties — Swift Book — `let` и `var`, значения по умолчанию у свойств.
- Deinitialization — Swift Book — `deinit` из упражнения 2.1.
- `UIColor` — тип `brandColor`; системные цвета сами подстраиваются под светлую и тёмную тему.
- `UIStackView` и `addArrangedSubview(_:)` — контейнер, который выстраивает элементы заглушки в столбик; `arrangedSubviews` как подмножество `subviews`.
- `UINavigationItem` — объект, в который попадает `title` экрана.

- `UserDefaults` — хранилище «ключ → значение», в котором гейты запоминают свои отметки по `id`.
- SF Symbols — каталог имён, которые можно класть в `symbolName`.

→ Глава 3. PlaygroundWindow — UIWindow с shake-detect

Глава 3. PlaygroundWindow — UIWindow с shake-detect

Когда пользователь зашёл в mini-app — например, в Калькулятор, — он оказывается «внутри отдельного приложения». Кнопки «← Назад» к лаунчеру сверху нет: mini-app не кладётся поверх лаунчера в общую стопку экранов, а **целиком заменяет** его в окне (как именно — в главе 4). По той же причине не сработает и жест «смахни от левого края»: он возвращает на предыдущий экран стопки, а предыдущего экрана у mini-app нет.

Нужен другой способ вернуться. В iOS он уже есть — **встряхивание** устройства (shake, англ. «трясти»). Система сама использует его для отмены ввода: набрал текст, встряхнул — iOS спросит «Отменить ввод?». В симуляторе встряхивание делается сочетанием **^⌘Z** (Control + Command + Z) или через меню **Device** → **Shake**.

В этой главе делаем своё окно — наследника `UIWindow`, который ловит встряхивание и сообщает о нём. Никаких сторонних библиотек, десяток строк кода — плюс понимание **responder chain**, о которой ниже.

3.1 Что такое UIWindow и почему его обычно не трогают

`UIWindow` — самый верхний контейнер интерфейса. Все экраны приложения рисуются внутри окна: у окна есть `rootViewController` (корневой view controller), а у него — свои вложенные экраны и view. Аналогия: окно — рама картины, `rootViewController` — сама картина. Картину можно заменить, рама остаётся.

При этом окно — само view: `UIWindow → UIView → UIResponder → NSObject`. У него есть всё, что есть у view (слой, `addSubview`, `backgroundColor`), только нет superview — оно и есть корень дерева (подробно — глава 0, раздел 0.7).

Окно принадлежит **сцене** (`UIWindowScene`) — одному «экземпляру» интерфейса приложения. На iPhone сцена обычно одна, на iPad приложение может открыть несколько сцен рядом (подробнее — в главе 5). Кроме твоего окна в сцене бывают и другие: например, клавиатура живёт в отдельном окне поверх твоего (на iOS 26.5 в `windowScene.windows` рядом с нашим окном мы увидели `UITextEffectsWindow`). Порядок окон задаёт `UIWindow.Level`: `.normal` = 0, `.statusBar` = 1000, `.alert` = 2000 — окно с большим уровнем рисуется выше.

В большинстве проектов окно создают одной строкой и забывают:

```
window = UIWindow(windowScene: windowScene)
```

Дальше с окном не работают — все экраны живут внутри `rootViewController`.

Нам этого мало: мы хотим, чтобы окно само замечало встряхивание. Поэтому делаем наследника — `PlaygroundWindow`.

3.2 Минимальный наследник

Файл: `App/PlaygroundWindow.swift`. Весь класс — десять строк:

```swift
import UIKit

/// Окно, которое сообщает о встряхивании устройства.
final class PlaygroundWindow: UIWindow {

    var onShake: (() -> Void)?

    override func motionEnded(_ motion: UIEvent.EventSubtype, with event: UIEvent?) {
        super.motionEnded(motion, with: event)
        guard motion == .motionShake else { return }
        onShake?()
    }
}
```

Разберём.

- `var onShake: (() -> Void)?` — callback: опциональное замыкание без параметров. Кто хочет реагировать на встряхивание, кладёт сюда функцию. Никто не положил — `nil`, и встряхивание ничего не делает.
- `motionEnded(_:with:)` — метод класса `UIResponder` (о нём — в разделе 3.3). UIKit вызывает его, когда **движение закончилось**: человек встряхнул телефон и перестал. Есть и парный `motionBegan(_:with:)` — «начал трясти», — но нам важен момент, когда жест завершён.
- `super.motionEnded(motion, with: event)` — передаём событие дальше, как сделал бы `UIWindow` без нашего наследника. Зачем — чуть ниже.
- `guard motion == .motionShake` — проверяем, что это именно встряхивание. Тип `UIEvent.EventSubtype` описывает не только движения: в нём есть и подтипы для кнопок гарнитуры и пульта (`remoteControlPlay`, `remoteControlPause`…). Из «движений» встряхивание — единственное, но проверка ничего не стоит и защищает от сюрпризов.
- `onShake?()` — вызываем callback, если он есть. Знак `?` — вызов опционального замыкания: если `onShake == nil`, ничего не произойдёт.

Про `super`. Встряхивание нужно не только нам: на нём работает системная отмена ввода. Её обрабатывает объект `UIApplication`, который стоит в цепочке **после** окна. Если не вызвать `super`, событие на окне и закончится, и «встряхни, чтобы отменить» перестанет работать во всём приложении.

> **Идея.** Окно — одно из последних звеньев responder chain. Всё, что не обработали экраны, доходит до него. Поэтому окно — удобное место для глобальных реакций: встряхивание, сочетания клавиш на iPad с клавиатурой и тому подобное.

3.3 Responder chain — коротко

Responder (англ. «отвечающий») — любой объект-наследник `UIResponder`, который умеет получать события: касания, нажатия клавиш, движения. Наследники `UIResponder` — это `UIView`, `UIViewController`, `UIWindow`, `UIApplication` и даже `UIWindowScene`.

Responder chain (цепочка ответчиков) — порядок, в котором событие передаётся от одного ответчика к другому, если первый его не обработал. У каждого ответчика есть свойство `next` — «кому передать дальше». Правила такие (документация Apple, «Using responders and the responder chain»):

- у view дальше идёт его родительский view; а если view — корневой view экрана, то сам view controller;
- у view controller'а — родительский view его view (или окно, если экран — корневой в окне; или экран, который его показал, если он показан модально);
- у окна — сцена, у сцены — `UIApplication` (в документации написано «у окна — `UIApplication`»: текст старше сцен; на iOS 26.5 `window.next` — это `UIWindowScene`, мы проверили);
- у `UIApplication` — делегат приложения (`AppDelegate`), если он наследник `UIResponder` и при этом сам не view, не view controller и не `UIApplication`.

First responder (первый ответчик) — тот, кому событие достаётся первым. Для касания это view, по которому попал палец (его находит hit-testing — глава 0, раздел 0.3). Для клавиатуры и встряхивания — объект, который сам объявил себя первым ответчиком через `becomeFirstResponder()` (и ответил `true` в `canBecomeFirstResponder`): например, текстовое поле, в котором мигает курсор (глава 0, раздел 0.2). Когда такого объекта нет, UIKit отдаёт встряхивание view controller'у текущего экрана, и дальше оно идёт по цепочке вверх — к окну. Мы проверили на iOS 26.5: с полем ввода в фокусе `motionEnded` получили по очереди текстовое поле, view controller и окно; без первого ответчика — view controller и окно.

Чтобы не гадать, мы распечатали цепочку в нашем собранном playground'е (как — в упражнении 3.2). От таблицы лаунчера она такая:

```
UITableView
 → AppListViewController
 → UIViewControllerWrapperView       ┐
 → UINavigationTransitionView        │ служебные view
 → UILayoutContainerView             ┘ UINavigationController
 → UINavigationController
 → UIDropShadowView                  ┐ служебные view,
 → UITransitionView                  ┘ которые окно держит под root
 → PlaygroundWindow                  ← здесь ловим мы
 → UIWindowScene
 → UIApplication                     ← здесь работает «встряхни для отмены»
 → AppDelegate
```

Классы с незнакомыми именами (`UIViewControllerWrapperView`, `UITransitionView`…) — внутренние view UIKit, их не создают вручную. Важно другое: какой бы экран ни был открыт, событие, которое никто не обработал, **обязательно дойдёт до окна**.

Каждое звено может «съесть» событие или передать дальше. Мы ловим **в окне**, потому что:

- окно одно и существует всё время, пока приложение открыто, — а конкретный экран mini-app появляется и исчезает;
- подписываться нужно в одном месте, а не в каждом из десятка экранов.

Альтернативы есть: рассылать «было встряхивание» через `NotificationCenter` (общую «доску объявлений» приложения) или сделать протокол, который реализует каждый экран. Оба варианта работают, но требуют дисциплины: забыл подписать экран — жест на нём не работает. С окном забыть нельзя.

Упражнение 3.1. Убери из `motionEnded` строку `super.motionEnded(motion, with: event)`. Что сломается и в какой ситуации это заметит пользователь?

3.4 Кто устанавливает `onShake`

Сам `PlaygroundWindow` не знает, что делать при встряхивании. Решает **`BootCoordinator`** (глава 4): когда запускает mini-app, он кладёт в окно callback:

```
// фрагмент из BootCoordinator.start()
window?.onShake = { [weak self] in
    self?.exitToLauncher()
}
```

А при выходе в лаунчер — убирает его:

```
// фрагмент из BootCoordinator
private func exitToLauncher() {
    window?.onShake = nil
    // ...
    onExit()
}
```

После этого встряхивание в лаунчере ничего не делает. Так и задумано: из лаунчера выходить некуда.

> **Внимание: `[weak self]` здесь обязателен.** Окно держит замыкание (свойство `onShake`), замыкание держит координатор. Если координатор захвачен **сильно**, то, пока окно живо (а оно живо всё время), координатор не освободится, даже когда его никто больше не использует. Это **утечка памяти**: объект висит в памяти, хотя уже не нужен. С `[weak self]` замыкание не удерживает координатор: когда его отпустят, `self` внутри станет `nil`, и вызов `self?.exitToLauncher()` просто ничего не сделает.

3.5 Где создаётся окно

В `SceneDelegate.swift` (целиком разберём его в главе 5):

```
func scene(_ scene: UIScene,
           willConnectTo session: UISceneSession,
           options connectionOptions: UIScene.ConnectionOptions) {
```

```
    guard let windowScene = scene as? UIWindowScene else { return }

    let window = PlaygroundWindow(windowScene: windowScene)
    // ... здесь создаём лаунчер и ставим его rootViewController'ом ...
    window.makeKeyAndVisible()
    self.window = window
}
```

`PlaygroundWindow(windowScene:)` — инициализатор `UIWindow`, который сразу привязывает окно к сцене. Наш класс его не переопределяет, а наследует, поэтому интерфейс тот же, что у обычного окна.

`makeKeyAndVisible()` — делает окно **ключевым** (key window: именно оно получает ввод с клавиатуры и события без конкретной точки касания) и показывает его. Без этого вызова окно останется невидимым.

`self.window = window` — сохраняем **сильную ссылку** на окно в свойстве `SceneDelegate`. UIKit сам окно не удерживает: сцена хранит свои окна слабо. Если не сохранить ссылку, окно освободится сразу после выхода из метода — приложение не упадёт, но экран останется чёрным. Мы проверили это на iOS 26.5: без `self.window = window` через полторы секунды после запуска `windowScene.windows` пуст, `keyWindow` равен `nil`, а на снимке экрана — чернота.

3.6 Как проверить встряхивание в симуляторе

Физически потрясти симулятор нельзя, поэтому:

- меню симулятора **Device** → **Shake**;
- сочетание клавиш **^⌘Z** (Control + Command + Z).

Фокус должен быть на окне симулятора. Если нажать ⌘Z в Xcode — это будет «Отменить» в редакторе кода, а не встряхивание.

На устройстве — просто встряхни iPhone.

Два нюанса этого жеста.

Его никто не знает. Пользователь не догадается, что для выхода надо трясти телефон. Поэтому в лаунчере под списком мы пишем подсказку (глава 4), а в заглушке «ещё не реализовано» — прямо текстом.

Он пересекается со «встряхни для отмены». Если в mini-app открыто текстовое поле с набранным текстом, встряхивание вызовет и нашу реакцию, и системную: окно уйдёт в лаунчер, а система предложит отменить ввод. Для учебного playground'а это приемлемо. В настоящем приложении встряхивание для навигации лучше не использовать — только для отладочных функций вроде «отправить отчёт об ошибке».

Упражнение 3.2. Напиши функцию, которая печатает цепочку ответчиков от любого view до конца — через свойство `next`, — и вызови её из `viewDidAppear` заглушки `PlaceholderViewController` для её `view`. Что окажется между экраном и окном, если mini-app показано внутри `UINavigationController`?

3.7 Бытовая аналогия

Окно — **ресепшен в офисном здании**. Экраны — **кабинеты**. Позвонили в здание — звонок сначала попадает в тот кабинет, куда звонили. Если там не берут, звонок переводят выше по цепочке: начальнику отдела, потом на ресепшен. Ресепшен — одна из последних инстанций: он есть всегда, какие бы кабинеты ни открывались и закрывались.

Поэтому общую для всего здания инструкцию «если позвонили с кодовым словом — проводить гостя к выходу» даём ресепшену, а не каждому кабинету.

3.8 Что можно было бы сделать вместо встряхивания

Способов «вернуться в лаунчер» несколько:

- **Системный жест «домой»** (свайп вверх от нижнего края) — сворачивает приложение целиком, а не возвращает на наш экран. Перехватить его нельзя.
- **Свайп от левого края** — работает только внутри стопки `UINavigationController`, а mini-app у нас — отдельный корневой экран.
- **Кнопка в углу** поверх каждого mini-app — работает и понятна, но вмешивается в дизайн каждого экрана.
- **Встряхивание** — не требует элементов на экране, не конфликтует с интерфейсом mini-app, в симуляторе есть сочетание клавиш.

Встряхивание — компромисс. В приложении для App Store я бы так не делал: жест неочевиден. Для учебного playground'а он подходит лучше всего: не требует UI-элементов и не портит дизайн mini-app.

Ответы к упражнениям

3.1. Встряхивание перестанет доходить до `UIApplication`, а именно он показывает системное «Отменить ввод?». Пользователь заметит это, когда наберёт текст в любом поле ввода и встряхнёт телефон: алерта с отменой не будет. Наш выход в лаунчер при этом работать продолжит — `onShake` вызывается после `super`, и от него не зависит. Мы проверили это на iOS 26.5 в тестовом приложении: с `super` после встряхивания над полем ввода появляется алерт «Undo Typing» (`UIAlertController`, показанный поверх корневого экрана), без `super` — ничего.

3.2. Функция — обход по `next`, пока он не станет `nil`:

```
func printResponderChain(from responder: UIResponder) {
    let chain = sequence(first: responder, next: { $0.next })
        .map { String(describing: type(of: $0)) }
        .joined(separator: " → ")
    print(chain)
}
```

`sequence(first:next:)` — стандартная функция Swift: берёт первый элемент и правило «как получить очередной» и выдаёт последовательность до первого `nil`. `type(of:)` возвращает класс объекта, а `String(describing:)` превращает его в имя.

Вызов в `PlaceholderViewController`:

```
    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        printResponderChain(from: view)
    }
```

В нашем playground'е печатается:

```
UIView → PlaceholderViewController → UIViewControllerWrapperView
→ UINavigationTransitionView → UILayoutContainerView
→ UINavigationController → UIDropShadowView → UITransitionView
→ PlaygroundWindow → UIWindowScene → UIApplication → AppDelegate
```

Между экраном и окном — `UINavigationController` и его служебные view (`UIViewControllerWrapperView`, `UINavigationTransitionView`, `UILayoutContainerView`), а также `UIDropShadowView` и `UITransitionView`, которые окно держит под корневым экраном. Точный набор служебных классов зависит от версии iOS — это внутреннее устройство UIKit, полагаться на него в коде нельзя. Но порядок «экран → контроллер навигации → окно → сцена → приложение» стабилен.

Что мы выучили

- `UIWindow` — самый верхний контейнер интерфейса, «рама» для `rootViewController`; сам при этом — view (`UIWindow → UIView`). Принадлежит сцене, а порядок окон задаёт `UIWindow.Level`. Обычно его не трогают, но для глобальных жестов это самое надёжное место.
- **Responder chain** — цепочка передачи событий через `next`: view → view controller → … → окно → сцена → `UIApplication` → `AppDelegate`. **First responder** получает событие первым; встряхивание без первого ответчика начинается с view controller'а экрана и всё равно доходит до окна.
- `motionEnded(_:with:)` — метод `UIResponder`, который вызывается по окончании встряхивания. Проверяем `motion == .motionShake`.
- Всегда вызывай `super.motionEnded(...)` — иначе сломается системное «встряхни для отмены».
- `onShake` ставит `BootCoordinator` при входе в mini-app и сбрасывает при выходе. `[weak self]` в этом замыкании обязателен — иначе окно удержит координатор навсегда.
- Окно нужно сохранить в `SceneDelegate.window` и показать через `makeKeyAndVisible()`, иначе экран будет чёрным.
- В симуляторе встряхивание — ^⌘Z или Device → Shake.

Apple Developer Documentation

- `UIWindow` — базовый класс окна, наследник которого — `PlaygroundWindow`.
- `UIWindowScene` — сцена, к которой привязано окно; инициализатор `UIWindow(windowScene:)`.
- Using responders and the responder chain to handle events — правила передачи событий по цепочке и first responder.
- `UIResponder.motionEnded(_:with:)` — метод, в котором ловим встряхивание.
- `becomeFirstResponder()` — как объект становится первым ответчиком.
- `UIEvent.EventSubtype.motionShake` — подтип события «встряхивание».

- `UIApplication.applicationSupportsShakeToEdit` — системная функция «встряхни для отмены», которую не должен ломать наш `motionEnded`.

→ Глава 4. BootCoordinator — оркестратор гейтов

Глава 4. BootCoordinator — оркестратор гейтов

В главах 1–3 появились три детали. `AnimatedSplashViewController` — экран, который показывает анимацию и зовёт `onFinish`. `AppManifest` — структура с набором флагов: какие гейты нужны этому mini-app. `PlaygroundWindow` — окно, которое сообщает о встряхивании.

Самих **гейтов** — экранов, которые пользователь проходит до main, — в части II появится несколько: onboarding, экран разрешения, выбор региона, проверка возраста, «обновите приложение», «ведутся работы», вход в аккаунт, защита при возврате из фона. Каждый — отдельный экран. Каждый зовёт callback, когда закончил.

Кто-то должен собрать их в правильную цепочку: посмотреть в манифест, решить, какой гейт нужен сейчас, показать его, дождаться callback'а и перейти к очередному. Этот «кто-то» — **BootCoordinator** (boot — «загрузка, запуск»). А раз уж он запускает mini-app, то и лаунчер, который его создаёт, мы соберём в этой же главе.

4.1 Зачем отдельный класс

Когда я впервые делал подобную систему, всё сидело в `SceneDelegate`:

```
// Так выглядит код, который мы НЕ пишем
func scene(_ scene: UIScene, willConnectTo ...) {
    if needsOnboarding { showOnboarding(...) }
    else if needsPermission { showPermission(...) }
    else if needsAuth { showAuth(...) }
    else { showMain() }
}
```

И каждый `show*` был длинным замыканием, которое заканчивалось вызовом очередного `show*`. `SceneDelegate` раздулся, стал нечитаемым, любое добавление гейта ломало два соседних.

Решение — выделить **отдельный объект**, который держит:

1. **Правила** — какой гейт идёт после какого и когда он нужен.
2. **Зависимости** — окно (куда ставить экран) и манифест (что показывать).
3. **Выход** — что сделать, когда mini-app закрывают.

Такой объект называют **координатором** (coordinator). Приём популяризовал iOS-разработчик Соруш Ханлу (Soroush Khanlou) в статье «The Coordinator» в 2015 году. Идея простая: экраны

ничего не знают друг о друге и не решают, куда идти дальше, — они только сообщают «я закончил». Порядок знает координатор.

4.2 Скелет — конструктор и зависимости

Файл: `App/BootCoordinator.swift`. Начало:

```swift
import UIKit

/// Проводит mini-app через цепочку гейтов: splash → … → main.
final class BootCoordinator {

    private let manifest: AppManifest
    private weak var window: PlaygroundWindow?
    private let onExit: () -> Void

    init(manifest: AppManifest,
         window: PlaygroundWindow,
         onExit: @escaping () -> Void) {
        self.manifest = manifest
        self.window = window
        self.onExit = onExit
    }
```

Три зависимости.

- `manifest` — что показывать (глава 2). `let`: по ходу запуска не меняется.
- `window` — куда ставить экраны. Ссылка **`weak`** (слабая): окно держит `SceneDelegate`, и координатору незачем продлевать ему жизнь. Если окно исчезло, координатору всё равно нечего делать.
- `onExit` — callback «mini-app закрыли, верни лаунчер». Его передаёт лаунчер (раздел 4.6).

`@escaping` у параметра-замыкания значит: «замыкание переживёт вызов `init`» — мы сохраняем его в свойство и вызовем позже. Без этой пометки Swift не разрешил бы сохранить замыкание.

Координатор работает только с UIKit, поэтому должен жить на главном потоке. Писать `@MainActor` над классом в нашем проекте не нужно: с Default Actor Isolation = MainActor (введение, раздел 0.1) это получается автоматически. В проекте без этой настройки пометку пришлось бы добавить. Мы проверили такой вариант компилятором Swift 6.3 в режиме Swift 6: вызов `manifest.makeMain()` — нашего `@MainActor`-замыкания — даёт ошибку «call to main actor-isolated function in a synchronous nonisolated context», а обращения к свойствам и методам UIKit (`window.rootViewController = …`, `UINavigationController(rootViewController:)`) — предупреждения с тем же смыслом. Почему только предупреждения: UIKit импортирован из Objective-C, и его `@MainActor`-пометки компилятор считает «унаследованными» (preconcurrency), поэтому смягчает нарушение до предупреждения; подробнее — глава 5, раздел 5.7.

В главе 11 у координатора появится ещё одно свойство — контроллер защиты при сворачивании (`LifecycleSecurityController`). Пока его нет.

4.3 Точка входа — `start()`

```
    // MARK: Вход и выход

    func start() {
        window?.onShake = { [weak self] in
            self?.exitToLauncher()
        }

        if manifest.hasAnimatedSplash {
            showSplash()
        } else {
            proceedAfterSplash()
        }
    }
```

Две вещи. Первая — подписываемся на встряхивание: пользователь встряхнул — выходим в лаунчер (глава 3). `[weak self]` — чтобы окно не удерживало координатор (подробно — в главе 3, раздел 3.4).

Вторая — запускаем цепочку. Если в манифесте включён splash — идём через него. Если нет — сразу к тому, что после splash'а.

И выход:

```
    private func exitToLauncher() {
        window?.onShake = nil
        // Если открыт модальный экран (лист, алерт) — закрываем его
        // сами, без анимации, до смены корневого экрана.
        window?.rootViewController?.dismiss(animated: false)
        onExit()
    }
```

Снимаем реакцию на встряхивание, закрываем модальный экран, если он открыт (подробнее — в конце раздела 4.5), и сообщаем лаунчеру «я всё».

Splash:

```
    // MARK: Splash

    private func showSplash() {
        let splash = AnimatedSplashViewController(manifest: manifest) { [weak self] in
            self?.proceedAfterSplash()
        }
        setRoot(splash, animated: false)
    }
```

Создаём splash из главы 1 и передаём ему callback: «когда закончишь — вызови мой `proceedAfterSplash()`». Ставим splash в окно без анимации: пользователь только что тапнул строку, и splash должен появиться сразу.

> **Идея.** Каждый гейт **необязателен**. Для каждого есть два пути: «нужен — показываем» и «не нужен — пропускаем». Координатор для каждого гейта выбирает один из двух методов: `showX()` или `proceedAfterX()`. Получается единый понятный механизм переходов.

4.4 Цепочка гейтов: один гейт — три метода

Сейчас, в фундаменте, гейтов ещё нет. Но место для каждого мы подготовим сразу, чтобы в части II только вписывать новое:

```
    // MARK: Цепочка гейтов (её заполняет часть II)

    private func proceedAfterSplash() {
        // Глава 6: если нужен онбординг — showOnboarding()
        proceedAfterOnboarding()
    }

    private func proceedAfterOnboarding() {
        // Глава 7: если нужен permission primer — showPermissionPrimer()
        proceedAfterPermission()
    }

    private func proceedAfterPermission() {
        // Глава 10: если нужен выбор региона — showRegionPicker()
        proceedAfterRegion()
    }

    private func proceedAfterRegion() {
        // Глава 10: если нужна проверка возраста — showAgeGate()
        proceedAfterAgeGate()
    }

    private func proceedAfterAgeGate() {
        // Глава 9: если нужна проверка серверного конфига — checkRemoteConfig()
        proceedAfterRemoteConfig()
    }

    private func proceedAfterRemoteConfig() {
        // Глава 8: если нужен вход в аккаунт — showAuthGate()
        showMain()
    }
```

Сейчас каждый метод просто передаёт управление дальше по цепочке, и после splash'а цепочка сразу доходит до `showMain()`. Методы-«проходы» выглядят лишними, но у каждого есть своё место в очереди: в главах 6–10 мы будем менять тело одного метода, не трогая соседей.

Вот как будет выглядеть блок онбординга после главы 6:

```swift
private func proceedAfterSplash() {
    if OnboardingViewController.shouldShow(for: manifest) {
        showOnboarding()
    } else {
        proceedAfterOnboarding()
    }
}

private func showOnboarding() {
    let onboarding = OnboardingViewController(manifest: manifest) { [weak self] in
        self?.proceedAfterOnboarding()
    }
    setRoot(onboarding, animated: true)
}

private func proceedAfterOnboarding() {
    // ... решение про очередной гейт ...
}
```

Три метода вокруг одного гейта:

1. **`proceedAfterSplash()`** — развилка перед онбордингом: нужен этот гейт или нет. Если да — `showOnboarding()`. Если нет — сразу `proceedAfterOnboarding()`.
2. **`showOnboarding()`** — создаёт экран гейта и ставит его в окно. В callback передаёт «когда закончишь — `proceedAfterOnboarding()`».
3. **`proceedAfterOnboarding()`** — развилка перед очередным гейтом (экраном разрешения). Та же логика: нужен — показать, не нужен — пропустить.

Тройка повторяется для **каждого** гейта. Решение не самое короткое, зато **читаемое** и **расширяемое**: новый гейт — это новая развилка и новый `show`-метод между существующими.

Полная цепочка после части II, если у mini-app включены все гейты:

```
start()
 └ showSplash()                          [если hasAnimatedSplash]
     └ proceedAfterSplash()
         └ showOnboarding()              [если hasOnboarding и ещё не показывали]
             └ proceedAfterOnboarding()
                 └ showPermissionPrimer() [если requiresPermission и ещё не спрашивали]
                     └ proceedAfterPermission()
                         └ showRegionPicker() [если requiresRegionPick и регион не выбран]
                             └ proceedAfterRegion()
                                 └ showAgeGate() [если requiresAgeGate и возраст не
                                 ↪ подтверждён]
                                     └ proceedAfterAgeGate()
                                         └ checkRemoteConfig() [если checksForceUpdate или
                                           ↪ hasMaintenanceCheck]
                                             └ proceedAfterRemoteConfig()
                                                 └ showAuthGate() [если hasAuthGate и не
                                                   ↪ вошли]
```

```
                                    └─ showMain()
```

Лесенка глубокая, но каждый узел простой: проверить флаг и перейти к одному из двух соседей. На практике у mini-app включены один-два гейта: у «Профиля» — вход и защита при сворачивании, у «Погоды» — разрешение на геопозицию.

Main-экран:

```
    // MARK: Main

    private func showMain() {
        let main = manifest.makeMain()
        let nav = UINavigationController(rootViewController: main)
        nav.navigationBar.tintColor = manifest.brandColor
        setRoot(nav, animated: true)
        // Глава 11: здесь запустим LifecycleSecurityController
    }
```

`manifest.makeMain()` — вызываем фабрику из главы 2 и получаем свежий main-экран. Оборачиваем его в **`UINavigationController`** — **контейнерный view controller** (container view controller): сам наследник `UIViewController`, который держит другие экраны как дочерние (`children`) и показывает их **стопкой**: `push` кладёт новый экран сверху, кнопка «Назад» снимает верхний. Сверху у него **панель навигации** — `UINavigationBar`, это view. Заголовок и кнопки на ней берутся не из панели, а из `navigationItem` того экрана, который сейчас сверху (глава 2, раздел 2.4; подробно — глава 0, раздел 0.6). Многие mini-app открывают из main-экрана вложенные экраны (карточку заметки, настройки), и стопка им нужна.

`navigationBar.tintColor` — цвет кнопок на панели. Берём фирменный цвет mini-app: кнопки в «Списке дел» будут синими, в «Чате» — зелёными. Сами кнопки — `UIBarButtonItem`, потомки `NSObject`, а не view: их нельзя добавить через `addSubview`, они лишь описывают, что панели нарисовать (глава 0, раздел 0.8).

Упражнение 4.1. Добавь в начало каждого метода цепочки (`start`, `showSplash`, все `proceedAfter…`, `showMain`) строку `print(#function)` и запусти любое mini-app. Что появится в консоли и в каком порядке? Что изменится для mini-app с `hasAnimatedSplash = false`?

4.5 `setRoot` — как меняем экран в окне

Координатор не кладёт экраны в стопку через `push`. Он **меняет корневой экран окна целиком**: splash → онбординг → экран разрешения → main. Каждый раз `rootViewController` окна — другой экран.

Почему так? Потому что пройденный гейт **своё отработал**, и возвращаться к нему нельзя. Будь гейты в одной стопке, пользователь мог бы смахнуть назад из онбординга обратно в splash — бессмыслица. К тому же у гейтов разный «верх»: у splash'а нет панели навигации, у онбординга свои кнопки, у экрана разрешения — ничего. В общей стопке панель навигации пришлось бы то прятать, то показывать.

Меняем корневой экран так:

```
    // MARK: Смена root

    private func setRoot(_ vc: UIViewController, animated: Bool) {
        guard let window else { return }
        if animated {
            UIView.transition(with: window,
                              duration: 0.35,
                              options: .transitionCrossDissolve,
                              animations: {
                                  window.rootViewController = vc
                              })
        } else {
            window.rootViewController = vc
        }
    }
}
```

`guard let window` — окно слабое, и его может уже не быть; тогда тихо выходим.

`UIView.transition(with:duration:options:animations:)` — анимированная смена содержимого **контейнера**: по документации Apple, изменения дерева subviews, сделанные в `animations` (добавить, убрать, скрыть view), UIKit показывает как переход. Контейнер у нас — окно, а присваивание `rootViewController` как раз убирает view старого экрана и вставляет view нового. `.transitionCrossDissolve` — **перекрёстное растворение**: старый экран за 0.35 секунды становится прозрачным, новый одновременно проявляется. Около трети секунды — достаточно, чтобы глаз заметил смену, и недостаточно, чтобы заскучать.

Без анимации (`animated: false`) экран меняется мгновенно — так ставим splash.

> **Замена root — не бесплатна.** Старый корневой экран убирается из окна со всеми вложенными view, новый загружается, его view вставляется в окно и заново раскладывается. Для смены гейта — события раз в несколько секунд — это пустяк. Но делать `setRoot` в ответ на каждое нажатие — плохая идея.

Модальные экраны при выходе

Если в mini-app открыт **модальный** экран — лист снизу (sheet), алерт, полноэкранное окно поверх, — а пользователь встряхнул телефон, то в `exitToLauncher()` мы сначала закрываем его:

```
window?.rootViewController?.dismiss(animated: false)
```

`dismiss` на экране, который сам что-то показал, закрывает всё, что показано поверх него. Если поверх ничего нет — вызов ничего не делает.

Строго говоря, на iOS 26.5 эта строка не обязательна: мы проверили в собранном playground'е — открыли лист поверх заглушки и встряхнули **без** `dismiss`, и UIKit сам убрал лист вместе со старым корневым экраном, а в памяти не осталось ни листа, ни экранов mini-app. На старых версиях iOS мы это не проверяли, поэтому оставляем `dismiss` как явное намерение и дешёвую страховку: закрыть модальный экран до смены корня ничего не ломает.

4.6 Кто держит координатор. Лаунчер целиком

Координатор — обычный объект. Если на него нет ни одной **сильной ссылки**, система освобождает его сразу после `start()`. Тогда все callback'и с `[weak self]` получат `nil`, и цепочка оборвётся: splash доиграет анимацию и… ничего не случится.

Держит координатор **лаунчер** — `AppListViewController`. Разберём его целиком: он показывает список mini-app и запускает выбранное.

Файл: `App/AppListViewController.swift`.

```
import UIKit

/// Лаунчер: список всех mini-app из AppRegistry.
final class AppListViewController: UITableViewController {

    /// Как добраться до окна — даёт SceneDelegate.
    var windowProvider: (() -> PlaygroundWindow?)?
    /// Что сделать, когда mini-app закрылось, — тоже даёт SceneDelegate.
    var onReturnToLauncher: (() -> Void)?

    private let apps: [AppManifest] = AppRegistry.allApps
    private var activeCoordinator: BootCoordinator?
    private let cellID = "AppCell"

    init() {
        super.init(style: .insetGrouped)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется: экран создаётся только кодом")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "UIKit Playground"
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: cellID)
    }
```

`UITableViewController` — готовый view controller, у которого корневой view — **таблица** (`UITableView`): вертикальный прокручиваемый список строк. Таблица — наследник `UIScrollView` (`UITableView → UIScrollView → UIView`), отсюда её прокрутка, `contentOffset` и инерция (глава 0, раздел 0.5). Стиль `.insetGrouped` — строки собраны в скруглённую «карточку» с отступами по бокам на сером фоне, как в приложении «Настройки». Этот серый фон мы и повторили в launch screen (глава 1).

Два свойства-замыкания — `windowProvider` и `onReturnToLauncher` — лаунчер получает от `SceneDelegate` (глава 5). Самому лаунчеру окно не принадлежит, но координатору окно нужно. Поэтому `SceneDelegate` даёт лаунчеру «способ добраться до окна» и «способ вернуть лаунчер

на экран». Такой приём — передать объекту то, что ему нужно, снаружи, а не заставлять его искать самому, — называется **внедрением зависимостей** (dependency injection).

`activeCoordinator` — та самая сильная ссылка на координатор.

`tableView.register(_:forCellReuseIdentifier:)` — сообщаем таблице: «ячейки с идентификатором `AppCell` — это обычные `UITableViewCell`». Зачем идентификатор, — сразу ниже.

Data source — что показать

```
    // MARK: Data source — что показать

    override func tableView(_ tableView: UITableView,
                            numberOfRowsInSection section: Int) -> Int {
        apps.count
    }

    override func tableView(_ tableView: UITableView,
                            cellForRowAt indexPath: IndexPath) -> UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: cellID, for: indexPath)
        let app = apps[indexPath.row]

        var content = UIListContentConfiguration.subtitleCell()
        content.text = app.name
        content.secondaryText = app.subtitle
        content.secondaryTextProperties.color = .secondaryLabel
        content.image = UIImage(systemName: app.symbolName)
        content.imageProperties.tintColor = app.brandColor
        cell.contentConfiguration = content

        // Ячейка переиспользуется: задаём ОБА варианта, а не только один.
        cell.accessoryView = app.isReady ? nil : makeSoonBadge()
        cell.accessoryType = app.isReady ? .disclosureIndicator : .none
        return cell
    }
```

Таблица сама не знает, что показывать. Она спрашивает у своего **data source** (источника данных) — объекта, который отвечает на вопросы «сколько строк?» и «что в строке номер N?». Это протокол `UITableViewDataSource`, а свойства `dataSource` и `delegate` у таблицы **weak**: таблица не владеет своими помощниками, владеть ими должен кто-то другой (в заголовке `UITableView.h` оба объявлены `weak`; подробно — глава 0, раздел 0.9). `UITableViewController` назначает себя и делегатом, и источником данных своей таблицы, поэтому мы просто переопределяем эти два метода.

`numberOfRowsInSection` — сколько строк: столько, сколько манифестов.

`cellForRowAt` — строка номер `indexPath.row`. Здесь важное понятие — **переиспользование ячеек**. Таблица не создаёт ячейку на каждую строку: если строк тысяча, а на экране помещается десять, ячеек будет около десяти. Строка ушла за верх экрана — её ячейка возвращается в «запас» и тут же используется для строки, появившейся снизу.

`dequeueReusableCell(withIdentifier:for:)` — «дай ячейку из запаса, а если там пусто — создай новую» (класс для создания мы указали в `register`). Перед выдачей ячейки из запаса таблица зовёт у неё `prepareForReuse()`, но, по документации Apple, этот метод сбрасывает только служебное состояние — выделение, подсветку, `alpha`, — а не содержимое.

Отсюда правило: в `cellForRowAt` нужно **заново задать всё**, что отличается между строками. Ячейка может прийти из запаса с данными другой строки. Поэтому у пометки «СКОРО» мы задаём оба варианта: для готового mini-app явно убираем её (`accessoryView = nil`) и ставим стрелку `›` (`disclosureIndicator`), для заглушки — наоборот. Если бы мы писали только `if !app.isReady { cell.accessoryView = … }`, то после прокрутки пометка «СКОРО» из чужой строки могла бы остаться у готового mini-app.

`UIListContentConfiguration` (iOS 14+) — готовая раскладка содержимого ячейки: картинка слева, основной текст, дополнительный текст под ним. Вариант `.subtitleCell()` — дополнительный текст второй строкой. Это **content configuration** — структура-описание (протокол `UIContentConfiguration`), а не view: заполняем поля и присваиваем конфигурацию ячейке, а view под неё ячейка создаёт сама в своём `contentView`; раскладку, отступы и поддержку Dynamic Type UIKit делает сам (глава 0, раздел 0.9).

```
    override func tableView(_ tableView: UITableView,
                            titleForFooterInSection section: Int) -> String? {
        "Внутри mini-app встряхни устройство (в симуляторе ^⌘Z), чтобы вернуться сюда."
    }
```

Подпись под списком — подсказка про встряхивание. Без неё жест никто не найдёт (глава 3).

Delegate — что делать по тапу

```
    // MARK: Delegate — что делать по тапу

    override func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath)
     ↪ {
        tableView.deselectRow(at: indexPath, animated: true)
        launch(apps[indexPath.row])
    }
```

Второй помощник таблицы — **delegate** (делегат): объект, которому таблица сообщает о событиях — «тапнули строку», «строка сейчас появится». Data source отвечает на вопрос «что показать», delegate — «что делать». Аналогия: data source — повар, который готовит блюда, delegate — официант, которому говорят «вот этот столик позвал».

По тапу снимаем выделение со строки и запускаем mini-app. Строго говоря, `UITableViewController` снял бы выделение и сам: у него есть свойство `clearsSelectionOnViewWillAppear`, по умолчанию `true` (заголовок `UITableViewController.h`: «defaults to YES»), и при возврате лаунчера в окно его `viewWillAppear` очистит выделение. Но снимать его сразу, с анимацией, — принятый приём: строка коротко «вспыхивает» в ответ на тап и тут же гаснет, не дожидаясь возврата.

Запуск mini-app

```
    // MARK: Запуск mini-app

    private func launch(_ manifest: AppManifest) {
        guard let window = windowProvider?() else { return }
        let coordinator = BootCoordinator(
            manifest: manifest,
            window: window,
            onExit: { [weak self] in
                self?.activeCoordinator = nil
                self?.onReturnToLauncher?()
            }
        )
        activeCoordinator = coordinator
        coordinator.start()
    }

    private func makeSoonBadge() -> UIView {
        let label = UILabel()
        label.text = "СКОРО"
        label.font = .preferredFont(forTextStyle: .caption2)
        label.textColor = .secondaryLabel
        label.sizeToFit()
        return label
    }
}
```

`windowProvider?()` — вызываем опциональное замыкание: получаем окно (или `nil`, если замыкание не задано или окна уже нет).

Создаём координатор и **сохраняем** его в `activeCoordinator` — лаунчер держит его, **пока пользователь в mini-app**. Когда сработает `onExit` (встряхнули или гейт сам попросил выйти, как проверка возраста в разделе 4.7), лаунчер обнуляет ссылку: `self?.activeCoordinator = nil`. На координатор больше никто не ссылается, и **ARC** (автоматический подсчёт ссылок Swift) освобождает его. Вместе с ним уходят и экраны mini-app.

В `onExit` — `[weak self]`: координатор хранит это замыкание, а лаунчер хранит координатор. Сильный захват лаунчера замкнул бы круг: лаунчер → координатор → замыкание → лаунчер. Это **цикл сильных ссылок** (retain cycle): объекты держат друг друга и не освобождаются никогда.

`makeSoonBadge()` — пометка «СКОРО»: маленький серый лейбл. `sizeToFit()` подгоняет размер лейбла под текст — `accessoryView` раскладывается без Auto Layout, по размеру самого view.

Мы проверили цепочку жизни объектов в собранном playground'е: после встряхивания и возврата в лаунчер не остаётся в памяти ни координатора, ни splash'а, ни main-экрана.

> **Признак «забыли удержать».** Запустил mini-app, splash доиграл — и ничего не происходит, callback'и не срабатывают. Почти всегда это значит, что координатор

освободился посередине. Проверь, что у кого-то есть сильная ссылка вида `var coordinator: ...`.

Упражнение 4.2. Замени в `launch(_:)` строку `activeCoordinator = coordinator` на `_ = coordinator` (то есть не сохраняй координатор). Что произойдёт, когда ты запустишь mini-app? Объясни по шагам.

4.7 Гейт может сам выйти в лаунчер

Большинство гейтов пропускают пользователя дальше. Но проверка возраста (глава 10) особая: если пользователь слишком молод, дальше его не пускаем, и кнопка «Вернуться» ведёт в лаунчер. Поэтому у этого гейта **два** callback'а — `onPass` и `onTooYoung`. Так этот блок будет выглядеть в координаторе после главы 10:

```
private func showAgeGate() {
    let gate = AgeGateViewController(
        manifestId: manifest.id,
        brandColor: manifest.brandColor,
        minAge: manifest.minAgeYears,
        onPass: { [weak self] in self?.proceedAfterAgeGate() },
        onTooYoung: { [weak self] in self?.exitToLauncher() }
    )
    setRoot(gate, animated: true)
}
```

`onPass` → идём дальше по цепочке. `onTooYoung` → сразу в лаунчер.

Полезный приём: у гейта может быть **больше одного выхода**, и координатор решает, куда ведёт каждый. Сам гейт про лаунчер не знает — он только сообщает, чем всё закончилось.

4.8 Бытовая аналогия

Координатор — **администратор поликлиники**. Пациент приходит, администратор смотрит в карту (манифест): «нужна справка — в кабинет 3, нужны анализы — в кабинет 7, всё в порядке — к врачу».

Кабинеты (splash, онбординг, экран разрешения) друг о друге не знают. Каждый делает своё и отправляет пациента обратно к администратору: «у нас всё». Куда дальше — решает администратор.

Если бы кабинеты сами отправляли пациента в другой кабинет, поменять порядок было бы мучением: пришлось бы переучивать каждый кабинет. С администратором достаточно поменять пару строк в его инструкции.

4.9 Что можно улучшить

Текущая реализация практичная, но не единственная. Если цепочка станет длиннее, есть варианты.

- **Конечный автомат** (state machine). Каждый гейт — состояние, координатор — таблица переходов между состояниями. Порядок виден целиком в одном месте.
- **Цепочка через общий протокол «гейт»**:

  ```
  Chain
      .start(SplashGate())
      .then(OnboardingGate.if(manifest.hasOnboarding))
      .then(PermissionGate.if(manifest.requiresPermission != nil))
      .terminate(MainGate())
  ```

  Это псевдокод — такого API нет, его пришлось бы написать. Выглядит чище, но требует общей абстракции «гейт», а наши гейты слишком разные: у проверки возраста два выхода, у проверки конфига — асинхронный запрос к серверу.

- **Цепочка на async/await**:

  ```
  await splashGate.show(in: window)
  if shouldShowOnboarding { await onboardingGate.show(in: window) }
  ```

  Линейный код вместо лесенки callback'ов. Но каждый гейт должен уметь «показаться и дождаться окончания» как `async`-функция.

Всё это можно прикрутить позже. На нашем масштабе — около восьми гейтов — «простой» подход с тройками методов даёт лучший баланс читаемости и простоты.

Упражнение 4.3. Допустим, после части II ты решил, что вход в аккаунт должен идти **раньше** экрана разрешения: сначала войти, потом просить геопозицию. Какие методы координатора придётся поменять и как? Ответь, не запуская код.

Ответы к упражнениям

4.1. Для mini-app со splash'ем в консоли будет:

```
start()
showSplash()
proceedAfterSplash()
proceedAfterOnboarding()
proceedAfterPermission()
proceedAfterRegion()
proceedAfterAgeGate()
proceedAfterRemoteConfig()
showMain()
```

Причём `start()` и `showSplash()` — сразу по тапу, а остальное — одной пачкой примерно через 1.65 секунды, когда splash вызовет `onFinish`. Все `proceedAfter…` пока только передают управление дальше, поэтому идут подряд без пауз. `#function` — специальный литерал Swift, который подставляет имя текущей функции.

Для mini-app с `hasAnimatedSplash = false` строки `showSplash()` не будет, и вся цепочка напечатается сразу после тапа.

4.2. Координатор создастся, `start()` поставит splash и повесит `onShake`, а после выхода из `launch(_:)` на координатор не останется ни одной сильной ссылки — ARC освободит его. Splash доиграет анимацию и вызовет `onFinish`, но в этом замыкании `self` (координатор) уже `nil`, и `proceedAfterSplash()` не вызовется. Пользователь застрянет на splash'е. Встряхивание тоже не поможет: замыкание `onShake` захватило координатор слабо и теперь ничего не делает.

4.3. Сейчас порядок такой: … → `proceedAfterOnboarding()` → экран разрешения → `proceedAfterPermission()` → … → `proceedAfterRemoteConfig()` → вход → `showMain()`. Чтобы вход шёл раньше разрешения:

1. В `proceedAfterOnboarding()` вместо развилки «разрешение» поставить развилку «вход»: если нужен — `showAuthGate()`, иначе — сразу то, что после входа.
2. Callback экрана входа (и ветка «вход не нужен») теперь ведёт не в `showMain()`, а в развилку «разрешение» — то, что раньше было телом `proceedAfterOnboarding()`.
3. В `proceedAfterRemoteConfig()` вместо развилки «вход» — сразу `showMain()`.

Сами экраны гейтов не меняются вовсе: они по-прежнему сообщают только «я закончил». Весь порядок — в координаторе.

Что мы выучили

- `BootCoordinator` — отдельный класс, который знает порядок гейтов. Без него вся логика запуска осела бы в `SceneDelegate`.
- Зависимости: `manifest` (что показывать), `window` (куда, `weak`), `onExit` (как вернуть лаунчер).
- Один гейт = три метода: развилка `proceedAfter…` (нужен или нет), `show…` (создать экран), callback экрана → очередная развилка. В фундаменте развилки пока пустые — их заполняет часть II.
- `showMain()` оборачивает main-экран в `UINavigationController` с фирменным цветом кнопок.
- `setRoot` меняет `window.rootViewController` целиком, с перекрёстным растворением за 0.35 секунды. Открытый модальный экран перед выходом закрываем через `dismiss`.
- Лаунчер (`AppListViewController`) — таблица (`UITableView` → `UIScrollView`): data source отвечает «что показать», delegate — «что делать по тапу»; оба свойства у таблицы weak. Ячейки переиспользуются (`prepareForReuse` сбрасывает только служебное состояние), поэтому в `cellForRowAt` задаём все отличающиеся свойства заново.
- `UINavigationController` — контейнерный view controller; заголовок и кнопки на `UINavigationBar` берутся из `navigationItem` экрана, `UIBarButtonItem` — не view.
- Координатор держит лаунчер (`activeCoordinator`); при выходе ссылку обнуляем, и ARC освобождает координатор вместе с экранами mini-app.
- У гейта может быть несколько выходов (`onPass` / `onTooYoung`), а куда какой ведёт — решает координатор.

Apple Developer Documentation

Паттерна «координатор» в документации Apple нет — это приём сообщества разработчиков. Но все API, на которых он стоит, — обычные UIKit и Swift.

- `UIWindow.rootViewController` — корневой экран окна, который меняет `setRoot`.

- `UIView.transition(with:duration:options:animations:completion:)` — анимация перекрёстного растворения при смене корневого экрана.
- `UINavigationController` — стопка экранов с панелью навигации, в которую `showMain()` кладёт main-экран.
- `UITableViewController` и `UITableViewDataSource` — основа лаунчера.
- `dequeueReusableCell(withIdentifier:for:)` и `prepareForReuse()` — переиспользование ячеек и что при нём сбрасывается.
- `UITableView.dataSource` — weak-свойство: таблица не владеет источником данных.
- `clearsSelectionOnViewWillAppear` — почему `UITableViewController` и сам снял бы выделение.
- `UIListContentConfiguration` — готовая раскладка содержимого ячейки.
- `dismiss(animated:completion:)` — закрытие модальных экранов перед выходом в лаунчер.
- Automatic Reference Counting — Swift Book — сильные и слабые ссылки, циклы сильных ссылок и почему координатор должен кто-то держать.

→ Глава 5. Lifecycle App → Scene → VC и @MainActor под капотом

Глава 5. Lifecycle App → Scene → VC и @MainActor под капотом

В четырёх предыдущих главах мы строили цепочку запуска mini-app: splash, манифест, окно, координатор, лаунчер. Но под всем этим лежит цепочка **более фундаментальная** — та, что запускается ещё до того, как выполнится хоть одна наша строчка.

Когда пользователь тапает иконку, iOS не сразу зовёт твой `SceneDelegate`. Сначала запускается процесс, создаётся объект `UIApplication`, вызывается делегат приложения, система читает настройки сцен из `Info.plist`, создаёт сцену — и **только потом** вызывает `scene(_:willConnectTo:options:)`, где мы создаём окно.

В этой главе разбираем эту цепочку — **жизненный цикл** (lifecycle) приложения: порядок состояний, через которые оно проходит («запускается → активно → в фоне → ...»), и методы, которые iOS вызывает на каждом переходе. Заодно разберём, почему весь код книги работает на главном потоке и что для этого делает настройка Xcode 26 Default Actor Isolation. А в конце соберём фундамент в работающий проект.

5.1 App → Scene → VC: три уровня

С iOS 13 у приложения на UIKit **три** уровня сверху вниз:

```
UIApplication                  ← процесс, один на всё приложение
   │
   └─ UIWindowScene            ← сцена: один «экземпляр» интерфейса
        │                          (запись о ней — UISceneSession)
        └─ PlaygroundWindow    ← наше окно (глава 3)
             │
             └─ rootViewController (лаунчер, splash, main mini-app)
                  │
                  └─ вложенные экраны и view…
```

Сцена (`UIScene`, для экранов — `UIWindowScene`) — это один «экземпляр» интерфейса приложения со своими окнами. До iOS 13 сцен не было: окно принадлежало прямо `UIApplication`, и у приложения мог быть ровно один интерфейс. Сцены Apple ввела ради iPad: там одно приложение может быть открыто в нескольких окнах сразу — две заметки рядом в Split View, несколько окон в Stage Manager. Каждое такое окно — своя сцена.

`UISceneSession` — «карточка» сцены, которую хранит система: какой тип сцены, с какой конфигурацией создана, какое состояние сохранено. Сцену система может выгрузить из памяти

(например, давно не открывавшееся окно на iPad), а карточка останется — и по ней сцену потом восстановят.

На iPhone сцена обычно одна, но устройство то же самое. Шаблон Xcode так и настраивает: в `Info.plist` ключ **Enable Multiple Windows** (`UIApplicationSupportsMultipleScenes`) = `NO` — у нашего приложения будет одна сцена даже на iPad.

Сцены — уже не выбор, а требование. В технической заметке TN3187 Apple пишет: в первом крупном выпуске iOS после iOS 26 приложение, собранное с новым SDK и не перешедшее на сцены, **не запустится**. Поэтому книга с самого начала использует `SceneDelegate`.

Кто здесь кто по родословной: `UIApplication` и `UIScene` (а значит, и `UIWindowScene`) — наследники `UIResponder`, то есть звенья responder chain, но не view; `UISceneSession` — просто `NSObject`; окно — view (`UIWindow → UIView`). Подробно — глава 0, раздел 0.7.

На двух верхних уровнях — свой **делегат**. Делегат — объект, которому система сообщает о событиях и у которого спрашивает решения («что сделать при запуске?», «приложение ушло в фон»). Мы встречали делегата у таблицы в главе 4: там таблица сообщала лаунчеру «тапнули строку». Здесь так же:

- `AppDelegate` — делегат `UIApplication`, события процесса: запуск, получение токена push-уведомлений, конфигурация новых сцен;
- `SceneDelegate` — делегат сцены, события одной сцены: подключилась, стала активной, ушла в фон.

У экрана делегата нет: события «появился», «исчез» `UIViewController` получает сам, в своих методах `viewDidLoad`, `viewWillAppear`, `viewDidDisappear` (глава 1, раздел 1.3).

5.2 Info.plist: как iOS находит наши делегаты

`Info.plist` — файл-анкета приложения: название, версия, минимальная версия iOS, разрешения, настройки сцен. iOS читает его **до** запуска твоего кода. Часть ключей Xcode 26 генерирует из настроек сборки (они начинаются с `INFOPLIST_KEY_…`), часть лежит в самом файле `Info.plist`.

За сцены отвечает ключ **Application Scene Manifest** (`UIApplicationSceneManifest`). После того как мы во введении удалили строку со storyboard (раздел 0.1), в нём осталось:

```
Application Scene Manifest
├── Enable Multiple Windows: NO
└── Scene Configuration
    └── Application Session Role
        └── Item 0
            ├── Configuration Name: Default Configuration
            └── Delegate Class Name: $(PRODUCT_MODULE_NAME).SceneDelegate
```

`Delegate Class Name` — какой класс создать делегатом сцены. `$(PRODUCT_MODULE_NAME)` Xcode при сборке заменит на имя модуля (название цели, где пробелы и дефисы заменены на _): Swift-классы живут внутри модуля, и полное имя класса — `Модуль.SceneDelegate`.

`Configuration Name` — название этой конфигурации. Его же возвращает `AppDelegate` (сразу ниже). По документации, по этому имени UIKit ищет запись в манифесте и берёт из неё класс делегата. Держи имена одинаковыми буква в букву. Правда, пока конфигурация в манифесте од-

на, UIKit прощает опечатку: мы проверили на iOS 26.5 — с именем «Wrong Name» в `AppDelegate` конфигурация всё равно пришла с `delegateClass = SceneDelegate`, и приложение запустилось. Но полагаться на это не стоит: как только конфигураций станет несколько (например, отдельная для внешнего экрана), выбор пойдёт по имени.

5.3 AppDelegate — что туда класть, а что нет

В нашем проекте `App/AppDelegate.swift` минимальный:

```
import UIKit

@main
final class AppDelegate: UIResponder, UIApplicationDelegate {

    func application(_ application: UIApplication,
                     didFinishLaunchingWithOptions launchOptions:
                         [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        true
    }

    func application(_ application: UIApplication,
                     configurationForConnecting connectingSceneSession: UISceneSession,
                     options: UIScene.ConnectionOptions) -> UISceneConfiguration {
        UISceneConfiguration(name: "Default Configuration",
                             sessionRole: connectingSceneSession.role)
    }
}
```

`@main` — атрибут, который говорит компилятору: «это точка входа в программу». Для класса-делегата приложения компилятор сам создаст функцию `main()`, которая запустит `UIApplication` и назначит этот класс его делегатом.

`AppDelegate` наследует `UIResponder` — поэтому он и стоит последним в responder chain (глава 3).

`didFinishLaunchingWithOptions` — «процесс запущен, приложение готово к работе». Возвращаем `true` — «всё в порядке». Шаблон Xcode пишет тут `return true`; у нас тело из одного выражения, и Swift разрешает не писать `return`.

`configurationForConnecting` — iOS создаёт новую сцену и спрашивает: «с какой конфигурацией?». Отвечаем: с той, что в `Info.plist` называется «Default Configuration», для той же роли (`sessionRole` — роль сцены: обычное окно приложения, внешний экран и т. п.).

Что **обычно** живёт в `AppDelegate` (а не в `SceneDelegate`):

- регистрация push-уведомлений (`registerForRemoteNotifications`) и получение **device token** — адреса устройства, по которому сервер Apple доставит push (глава 41);
- регистрация фоновых задач `BGTaskScheduler` — её нужно сделать до конца запуска приложения;
- инициализация SDK аналитики и отчётов о сбоях (Firebase, Sentry и подобных).

Что **не должно**: создание окна и интерфейса — это работа `SceneDelegate`. Туда же в приложении со сценами приходят и ссылки: открытие приложения по URL и по **Universal Link** (обычной ссылке `https://…`, которую iOS открывает в приложении, а не в браузере). Их получает `SceneDelegate` — в `scene(_:openURLContexts:)`, `scene(_:continue:)`, а при холодном старте — в `connectionOptions` метода `willConnectTo` (подробно — в главе 41). Методы `AppDelegate` для ссылок в приложении со сценами не вызываются.

Наш `AppDelegate` почти пуст, потому что ничего из списка мы пока не делаем. Метод `application(_:didDiscardSceneSessions:)`, который создаёт шаблон, мы удалили: он нужен, чтобы подчистить данные закрытых сцен, а у нас таких данных нет.

5.4 SceneDelegate — где собирается всё дерево

С точки зрения интерфейса жизнь приложения начинается здесь: создаётся окно и ставится первый экран. `App/SceneDelegate.swift` целиком:

```
import UIKit

final class SceneDelegate: UIResponder, UIWindowSceneDelegate {

    var window: UIWindow?
    private var launcherRoot: UIViewController?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }

        let window = PlaygroundWindow(windowScene: windowScene)
        let launcher = AppListViewController()
        let nav = UINavigationController(rootViewController: launcher)
        nav.navigationBar.prefersLargeTitles = true

        launcher.windowProvider = { [weak window] in window }
        launcher.onReturnToLauncher = { [weak self, weak window] in
            guard let self, let window, let root = self.launcherRoot else { return }
            UIView.transition(with: window, duration: 0.35,
                              options: .transitionCrossDissolve,
                              animations: { window.rootViewController = root })
        }

        launcherRoot = nav
        window.rootViewController = nav
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

Разберём по частям.

`willConnectTo` iOS вызывает, когда сцена подключается к приложению: при запуске или когда пользователь открывает новое окно на iPad.

`guard let windowScene = scene as? UIWindowScene` — сцена приходит как общий тип `UIScene`. Окна бывают только у `UIWindowScene`, поэтому приводим тип; если это какая-то другая сцена — выходим.

```
let window = PlaygroundWindow(windowScene: windowScene)
```

`PlaygroundWindow` — наше окно из главы 3. Параметр `windowScene` привязывает окно к сцене: без привязки окно не появится на экране.

`AppListViewController()` — лаунчер из главы 4. Оборачиваем его в `UINavigationController` ради панели навигации с **большим заголовком** (`prefersLargeTitles = true`): «UIKit Playground» крупным шрифтом, как в «Настройках», который при прокрутке списка сжимается в обычный.

Этот `UINavigationController` мы **сохраняем** в `launcherRoot`:

```
launcherRoot = nav
```

`launcherRoot` — приватное свойство `SceneDelegate`. Когда пользователь зайдёт в mini-app, координатор заменит `window.rootViewController` на splash, потом на main. Лаунчер уйдёт из окна, но **не из памяти**: на него по-прежнему указывает сильная ссылка `launcherRoot`.

> **Зачем сохранять.** Без `launcherRoot` после первой замены корневого экрана лаунчер освободился бы, и при возврате пришлось бы создавать новый: прокрутка списка сбросилась бы. С сохранённой ссылкой возвращаемся к **тому же** лаунчеру, в том же состоянии. Есть и вторая причина: лаунчер держит активный координатор (глава 4). Если освободить лаунчер, освободится и координатор — и mini-app «зависнет» посреди запуска.

`launcher.windowProvider` и `launcher.onReturnToLauncher` — те самые замыкания, которые лаунчер ждёт снаружи (глава 4, раздел 4.6): «вот так добраться до окна» и «вот так вернуть лаунчер на экран». Возврат — та же анимация перекрёстного растворения, что у `setRoot` в координаторе: 0.35 секунды.

Списки захвата здесь такие.

- `{ [weak window] in window }` — лаунчер хранит это замыкание всё время. Со слабой ссылкой замыкание не продлевает окну жизнь: если сцена закроется и окно освободится, замыкание вернёт `nil`, а не «воскресит» ненужное окно. К тому же окно косвенно держит лаунчер (через `rootViewController`), а сильная ссылка обратно замкнула бы круг.
- `[weak self, weak window]` в `onReturnToLauncher` — та же логика: `SceneDelegate` держит лаунчер (через `launcherRoot`), лаунчер держит замыкание, и сильный захват `self` дал бы цикл сильных ссылок. `guard let self, let window, let root` — если чего-то уже нет, просто выходим.

`window.makeKeyAndVisible()` — окно становится ключевым и видимым (глава 3). `self.window = window` — сохраняем окно, иначе оно освободится и экран будет чёрным: сцена свои окна не удерживает (мы проверили — глава 3, раздел 3.5).

Упражнение 5.1. Чему равен `window.rootViewController` в тот момент, когда пользователь находится внутри main-экрана mini-app? А `launcherRoot`? Что из этого показано на экране?

5.5 Порядок событий при запуске

Упрощённый таймлайн **холодного старта** (приложения нет в памяти):

```
[тап по иконке]
 → iOS показывает launch screen (глава 1)
 → процесс запускается, создаётся UIApplication и AppDelegate
 → AppDelegate.application(_:didFinishLaunchingWithOptions:)
 → AppDelegate.application(_:configurationForConnecting:options:)
 → SceneDelegate.scene(_:willConnectTo:options:)  ← создаём окно и лаунчер
 → SceneDelegate.sceneWillEnterForeground(_:)
 → SceneDelegate.sceneDidBecomeActive(_:)          ← приложение готово к касаниям
 → launch screen исчезает, виден лаунчер
```

Сколько миллисекунд уходит на каждый шаг, зависит от устройства, размера приложения и того, что ты делаешь в этих методах. Порядок же одинаков всегда: делегат приложения → делегат сцены → первый экран → активная сцена. Поэтому в `didFinishLaunching` и `willConnectTo` не делают ничего долгого (скачивание, тяжёлые вычисления): пока они не вернут управление, пользователь смотрит на launch screen.

Тёплый старт — приложение уже в памяти, пользователь вернулся к нему из фона. Процесс и сцена уже есть, поэтому `didFinishLaunching` и `willConnectTo` не вызываются — сразу `sceneWillEnterForeground` и `sceneDidBecomeActive`.

Состояния сцены стоит различать:

- **active** (активна) — на экране и получает касания;
- **inactive** (неактивна) — на экране, но касания не получает: пользователь открыл переключатель приложений, опустил Центр уведомлений или пришёл входящий звонок;
- **background** (в фоне) — не на экране; через несколько секунд iOS обычно приостанавливает такое приложение.

5.6 Lifecycle-уведомления, которые мы используем

В главе 11 появится `LifecycleSecurityController` — он размывает экран mini-app, когда приложение уходит в фон. Подписывается он не на методы `SceneDelegate`, а на **уведомления** `NotificationCenter`.

`NotificationCenter` — общая «доска объявлений» приложения. Одни объекты вешают объявления («сцена сейчас станет неактивной»), другие подписываются на нужные и получают их. Отправитель не знает, кто подписан, подписчик не знает, кто отправил. Системные события сцены UIKit публикует там сам.

```swift
let bgObs = NotificationCenter.default.addObserver(
    forName: UIScene.willDeactivateNotification,
    object: nil,
    queue: .main
) { [weak self] _ in
    MainActor.assumeIsolated { self?.handleEnterBackground() }
}
```

- `forName:` — на какое объявление подписываемся.
- `object: nil` — от любого отправителя, то есть от любой сцены.
- `queue: .main` — выполнить замыкание на главной очереди (главном потоке).
- Замыкание — что сделать. Про `MainActor.assumeIsolated` — в разделе 5.8.
- Возвращённое значение `bgObs` — **жетон подписки**. Его надо сохранить и, когда подписка больше не нужна, отдать в `NotificationCenter.default.removeObserver(bgObs)`. Иначе замыкание продолжит вызываться — даже если объект, для которого подписывались, давно не нужен.

Почему уведомления, а не методы `SceneDelegate`? Контроллер защиты живёт **внутри координатора**, а не в `SceneDelegate`. Если бы `SceneDelegate` знал про координатор и сам звал его методы, два независимых объекта оказались бы связаны. Уведомления развязывают их: контроллер слушает «когда что произойдёт», не зная, кто сообщает.

Полезные уведомления жизненного цикла:

- `UIScene.willDeactivateNotification` — сцена сейчас станет неактивной: пользователь открывает переключатель приложений, уходит на домашний экран, пришёл звонок;
- `UIScene.didActivateNotification` — сцена снова активна;
- `UIScene.didEnterBackgroundNotification` / `UIScene.willEnterForegroundNotification` — ушла в фон / вот-вот вернётся на экран;
- `UIApplication.didReceiveMemoryWarningNotification` — системе не хватает памяти. Самое время освободить кэши: приложение, которое занимает много памяти, iOS может завершить.

Любой объект может на них подписаться — `SceneDelegate` этого не контролирует.

5.7 MainActor под капотом

Во введении (раздел 0.1) мы включили в проекте три настройки. Теперь посмотрим, что они меняют в коде.

Весь UIKit помечен `@MainActor` самой Apple. `UIView`, `UIViewController`, `UILabel` — все их свойства и методы можно трогать только с главного потока. Твой `class TodoListViewController: UIViewController` наследует эту пометку **независимо от настроек проекта**: экраны всегда на главном потоке.

Default Actor Isolation = MainActor меняет всё остальное — твои собственные типы, которые ни от чего из UIKit не наследуют: модели, сервисы, хранилища, свободные функции.

```
final class Counter {
    var value = 0
}
```

- Без настройки `Counter` ни к какому актору не привязан (nonisolated). Его можно трогать откуда угодно — и из двух потоков одновременно, получив гонку данных.
- С настройкой `Counter` — `@MainActor`. Компилятор следит, чтобы с ним работали только с главного потока.

То есть с этой настройкой **по умолчанию** весь твой код живёт там же, где интерфейс, и гонка «обновил лейбл из фона» становится ошибкой компиляции, а не редким падением у пользо-

вателя. В режиме Swift 6 большинство таких нарушений — ошибки, в режиме Swift 5 — предупреждения (введение, раздел 0.1).

Когда нужен `nonisolated`

Иногда код **нужно** выполнять не на главном потоке: сортировать большой массив, разбирать JSON, декодировать картинку — чтобы интерфейс в это время не замирал. Такой код помечают `nonisolated` — «не привязан ни к какому актору, можно вызывать откуда угодно»:

```
struct Todo {
    var title: String
    var priority: Int
}

enum TodoSorting {
    nonisolated static func byPriority(_ a: Todo, _ b: Todo) -> Bool {
        a.priority > b.priority
    }
}
```

Функция сравнивает два дела по приоритету: `true`, если первое важнее. Она «чистая» — не трогает интерфейс и не читает общее изменяемое состояние, работает только с тем, что ей передали. Такую функцию безопасно вызывать с любого потока, и `nonisolated` сообщает об этом компилятору.

Проверим. Отсортируем список в фоне:

```
func sortOffMain(_ todos: [Todo]) {
    Task.detached {
        let sorted = todos.sorted(by: TodoSorting.byPriority)
        print(sorted.count)
    }
}
```

С `nonisolated` это собирается. Без него Swift 6 выдаст ошибку: «converting function value of type ‘@MainActor (Todo, Todo) throws -> Bool’ to ‘(Todo, Todo) throws -> Bool’ loses global actor ‘MainActor’». По-человечески: функция привязана к главному потоку, а `sorted(by:)` в фоне вызвал бы её не на главном — так нельзя.

> **Когда нужен `nonisolated`.** Чистые функции и типы, которые не трогают UIKit и не используют состояние главного потока: сортировка, форматирование, валидация, разбор данных. Помечаем их `nonisolated`, чтобы звать из фоновых задач и других акторов. Частый случай — модель с `Codable`, которую разбирают в фоне: её объявляют `nonisolated struct` (подробнее — в главах про сеть, например в главе 15).

Task: наследует или нет

`Task { … }` — запуск асинхронной работы. Если `Task` создан в коде, привязанном к главному потоку (а у нас это почти весь код), он **наследует** эту привязку. Поэтому такой код работает:

```swift
struct Post: Decodable {
    let title: String
}

struct PostAPI {
    func fetch() async throws -> Post { Post(title: "Привет") }
}

final class DemoViewController: UIViewController {
    private let label = UILabel()
    private let api = PostAPI()

    func loadInherited() {
        Task {
            let post = try await api.fetch()
            self.label.text = post.title   // мы на главном потоке
        }
    }
}
```

`await` — точка ожидания: пока `fetch()` работает, главный поток не заблокирован и рисует интерфейс. Когда результат готов, задача продолжается — снова на главном потоке, и лейбл можно менять.

`Task.detached { … }` — «отсоединённая» задача: ничего не наследует от места, где её создали, и выполняется не на главном потоке. Трогать из неё интерфейс напрямую нельзя — на главный поток переходим явно:

```swift
    func loadDetached() {
        let api = self.api
        Task.detached {
            let post = try await api.fetch()
            await MainActor.run {
                self.label.text = post.title
            }
        }
    }
```

`MainActor.run { … }` — «выполни это замыкание на главном потоке». `await` перед ним — потому что главный поток может быть занят, и придётся подождать своей очереди.

Если забыть `MainActor.run` и вызвать метод экрана прямо из `Task.detached` — например, `self.show(post)`, где `show` обновляет лейбл, — Swift 6 выдаст ошибку «main actor-isolated instance method ‘show’ cannot be called from outside of the actor». Любопытная деталь, которую мы увидели при проверке: присваивание свойствам UIKit (`self.label.text = …`) из отсоединённой задачи Swift 6.3 отмечает только **предупреждением** «main actor-isolated property ‘text’ can not be mutated from a nonisolated context». Причина — в происхождении кода. `show` объявлен в Swift, и его `@MainActor` компилятор считает точным. А `UILabel.text` импортирован из Objective-C, и пометки `@MainActor` в заголовках UIKit компилятор трактует

как «унаследованные» (preconcurrency) — нарушения с ними он смягчает до предупреждения, чтобы старый код продолжал собираться. Мы проверили это на паре классов: тот же код с `@MainActor`-классом, написанным на Swift, даёт ошибку, с `UILabel` — предупреждение. В рантайме разницы нет: и то и другое — обращение к интерфейсу не с главного потока. Поэтому предупреждения о конкурентности не игнорируй — это те же ошибки, только компилятор пока не настаивает.

`Task.detached` в UIKit-коде нужен редко. Если хочется просто выполнить тяжёлую функцию в фоне, с Approachable Concurrency для этого есть пометка `@concurrent`:

```
final class ImageDecoder {
    @concurrent
    nonisolated func decode(_ data: Data) async -> UIImage? {
        UIImage(data: data)
    }
}
```

`nonisolated` снимает привязку к главному потоку, а `@concurrent` велит выполнять функцию в фоновом пуле потоков. Без `@concurrent` асинхронная `nonisolated`-функция с Approachable Concurrency выполняется там же, откуда её вызвали, — на главном потоке, если вызвали из экрана. Вызов из экрана — `let image = await decoder.decode(data)`: главный поток ждёт, не блокируясь, а декодирование идёт в фоне.

5.8 `MainActor.assumeIsolated` — мост к старым API

Вернёмся к подписке из раздела 5.6:

```
let bgObs = NotificationCenter.default.addObserver(
    forName: UIScene.willDeactivateNotification,
    object: nil,
    queue: .main
) { [weak self] _ in
    MainActor.assumeIsolated { self?.handleEnterBackground() }
}
```

`addObserver(forName:object:queue:using:)` — API из времён Objective-C. Его замыкание помечено как `@Sendable` (в заголовке `NSNotification.h` — `NS_SWIFT_SENDABLE`) — «может выполниться на любом потоке», и компилятор не знает про `queue: .main`. Мы-то уверены, что замыкание придёт на главный поток — мы сами передали `.main`. А компилятор считает, что это может быть любой поток, и на прямой вызов `self?.handleEnterBackground()` — метода экрана или контроллера, привязанного к главному потоку, — отвечает предупреждением «call to main actor-isolated instance method ‘handleEnterBackground()’ in a synchronous nonisolated context». Именно предупреждением, а не ошибкой, даже в режиме Swift 6: замыкание уходит в API, импортированный из Objective-C, а для таких API Swift 6.3 смягчает нарушения изоляции (раздел 5.7). Мы проверили это компилятором в обоих режимах языка; то же замыкание, переданное в функцию с параметром `@Sendable`, объявленную на Swift, даёт ошибку. Оставлять такое предупреждение нельзя: оно означает вызов кода главного потока без гарантии главного потока.

`MainActor.assumeIsolated { … }` — это **утверждение** для компилятора: «мы уже на главном потоке, выполни замыкание сразу, синхронно». Оно проверяется во время работы: если окажется, что мы **не** на главном потоке, приложение аварийно завершится с понятной ошибкой. Так лучше, чем тихо тронуть интерфейс из фона.

Это компромисс между Swift Concurrency и старыми API UIKit. Используй его, только когда **точно знаешь**, что окружение правильное. Здесь это гарантирует `queue: .main`.

`MainActor.assumeIsolated` доступен начиная с iOS 13: так указывает документация Apple, и то же стоит в интерфейсе модуля `_Concurrency` (`@available(iOS 13.0, *)` с `@_alwaysEmitIntoClient` — тело вкомпилируется в твоё приложение, поэтому новый рантайм Swift не нужен). Мы проверили: листинг проходит `swiftc -typecheck` с целью iOS 15.0. Так что с нашим минимумом iOS 15 никаких проверок версии не нужно. Альтернатива без проверки потока — `Task { @MainActor in self?.handleEnterBackground() }`: тоже работает, но выполняет код **асинхронно**, чуть позже. Для размытия экрана перед уходом в фон это плохо: система может успеть сделать снимок экрана для переключателя приложений до того, как размытие появится.

5.9 Бытовая аналогия

`AppDelegate` — **управляющий бизнес-центром**. Он знает, когда здание открылось, принимает почту на всё здание (push-уведомления) и решает, какой офис выделить новому арендатору.

`SceneDelegate` — **администратор одного офиса** в этом здании. Он расставляет мебель (окно, первый экран), знает, когда офис открылся и когда в нём погасили свет.

`UIViewController` — **сотрудник за конкретным столом**. Он знает, что лежит у него на столе и кто к нему заходил.

На iPhone в здании один офис. На iPad их может быть несколько, и у каждого свой администратор.

`@MainActor` — правило **«с документами работаем только в офисе»**. Сотрудник не может поменять бумаги из дома (с фонового потока) — охрана не пустит к шкафу. А `nonisolated` — черновики, с которыми можно работать где угодно, потому что они ничьи.

5.10 Что мы пропустили

Несколько тем жизненного цикла, которые playground'у не нужны и в этой книге не разбираются:

- **Восстановление состояния** (state restoration) — сохранить, на каком экране был пользователь, и вернуть его туда после того, как система выгрузила приложение. В приложениях со сценами это делается через `NSUserActivity` сцены (метод `stateRestorationActivity(for:)` делегата сцены).
- **Фоновые задачи** — `BGTaskScheduler`: периодически обновлять данные, даже когда приложение свёрнуто.
- **Несколько сцен на iPad** — если включить Enable Multiple Windows, `willConnectTo` будет вызываться для каждого окна, и всё, что сейчас хранится в одном экземпляре (как наш реестр), придётся продумать под несколько окон сразу.

Ссылки для самостоятельного изучения — в конце главы.

Упражнение 5.2. Добавь в `SceneDelegate` четыре метода и запусти приложение в симуляторе:

```
    func sceneWillEnterForeground(_ scene: UIScene) {
        print("→ scene will enter foreground")
    }
    func sceneDidBecomeActive(_ scene: UIScene) {
        print("→ scene did become active")
    }
    func sceneWillResignActive(_ scene: UIScene) {
        print("→ scene will resign active")
    }
    func sceneDidEnterBackground(_ scene: UIScene) {
        print("→ scene did enter background")
    }
```

Сверни приложение (в симуляторе — ⇧⌘H, Device → Home), потом открой снова. Какие строки и в каком порядке появятся в консоли Xcode при запуске, при сворачивании и при возвращении?

5.11 Собираем фундамент

Все файлы части I готовы. Осталась мелочь и проверка.

Общие цвета и отступы. Заведём одно место для общих цветов и отступов playground'а. Листинги mini-app из части III ради самостоятельности пишут системные цвета прямо в коде (`.systemGreen`, `.systemRed`), а когда захочешь единый стиль, их удобно заменить на `Palette.success`, `Palette.danger` и т. д. Создай `Common/DesignSystem.swift`:

```
import UIKit

/// Общие цвета playground'а. Системные цвета сами подстраиваются
/// под светлую и тёмную тему.
enum Palette {
    static let tint: UIColor = .systemBlue
    static let success: UIColor = .systemGreen
    static let danger: UIColor = .systemRed
}

/// Отступы: все кратны 4 точкам.
enum Spacing {
    static let xs: CGFloat = 4
    static let s: CGFloat = 8
    static let m: CGFloat = 16
    static let l: CGFloat = 24
}
```

Это тоже namespace enum'ы (глава 2): только статические константы. Цвета — системные, поэтому тёмная тема работает сама. Отступы кратны 4 точкам — так интерфейс выглядит ровнее:

соседние расстояния 8 и 16 заметно различаются, а 15 и 16 — нет. Глава 35 покажет, как завести свои фирменные цвета в каталоге ассетов.

Проверь, что в проекте есть всё:

Файл	Глава
`App/AnimatedSplashViewController.swift`	1
`Base.lproj/LaunchScreen.storyboard` (фон System Grouped Background)	1
`App/AppManifest.swift` (с `PermissionKind` и `OnboardingPage`)	2
`App/AppManifest+Placeholder.swift`	2
`App/PlaceholderViewController.swift`	2
`App/AppRegistry.swift`	2
`App/PlaygroundWindow.swift`	3
`App/BootCoordinator.swift`	4
`App/AppListViewController.swift`	4
`App/AppDelegate.swift`, `App/SceneDelegate.swift`	5
`Common/DesignSystem.swift`	5

И настройки из введения (раздел 0.1): нет `Main.storyboard` и ссылок на него, минимальная версия iOS 15.0, Swift 6.

Если перемещаешь файлы по папкам в Xcode — перетаскивай их внутри Project Navigator, а не в Finder: так Xcode не потеряет их.

Запусти (⌘R) на симуляторе iPhone. Мы собрали этот фундамент в Xcode 26.5 (Swift 6, iOS 15.0) и прогнали на симуляторе iPhone 16 с iOS 26.5. Что ты должен увидеть:

1. На долю секунды — серый фон launch screen.
2. Лаунчер: большой заголовок «UIKit Playground», одиннадцать строк с цветными иконками, у каждой справа серая пометка «СКОРО», под списком — подсказка про встряхивание.
3. Тап по любой строке — экран окрашивается в цвет mini-app, иконка «выпрыгивает» с лёгким перелётом, следом проявляется название, за ним — индикатор загрузки.
4. Примерно через полторы секунды — плавный переход к заглушке: молоток и текст «ещё не реализовано», сверху — панель навигации с названием mini-app.
5. ^⌘Z — плавный возврат в лаунчер, в том же положении прокрутки.

Консоль Xcode при этом должна оставаться чистой: никаких сообщений о конфликтах constraints («Unable to simultaneously satisfy constraints»).

Если вместо лаунчера — чёрный экран, проверь по порядку: имя конфигурации «Default Configuration» в `Info.plist` и в `AppDelegate` совпадает; в `Delegate Class Name` стоит `$(PRODUCT_MODULE_NAME).SceneDelegate`; в `willConnectTo` есть `makeKeyAndVisible()` и `self.window = window`.

Фундамент готов. С части II мы начнём вписывать гейты в `BootCoordinator`, а с части III — заменять заглушки настоящими mini-app.

Ответы к упражнениям

5.1. Внутри main-экрана `window.rootViewController` — это `UINavigationController`, который создал `showMain()` координатора, с main-экраном mini-app внутри. `launcherRoot` — другой `UINavigationController`, с лаунчером; он в памяти, но не в окне. На экране — только то, что в `window.rootViewController`, то есть mini-app.

5.2. При запуске мы увидели в консоли:

```
→ scene will enter foreground
→ scene did become active
```

Первое уведомление приходит и при самом первом появлении сцены: для iOS запуск — это тоже «выход на передний план». При сворачивании:

```
→ scene will resign active
→ scene did enter background
```

Сначала сцена перестаёт получать касания (неактивна), потом уходит с экрана. При возвращении — снова `will enter foreground` и `did become active` (эти шесть строк мы получили, запустив playground на симуляторе iPhone 16 с iOS 26.5). Если вместо сворачивания открыть переключатель приложений (в симуляторе — Device → App Switcher) и вернуться, не выбирая другое приложение, сцена остаётся на экране, просто без касаний, — поэтому ожидай только пару `will resign active` / `did become active`. Проверь это сам.

Что мы выучили

- С iOS 13 у приложения три уровня: `UIApplication` → сцена (`UIWindowScene`, её запись — `UISceneSession`) → окно и экраны. `UIApplication` и `UIScene` — наследники `UIResponder`, окно — view. У приложения и сцены свой делегат; экран получает события сам. После iOS 26 приложения без сцен перестанут запускаться (TN3187).
- `Info.plist` → Application Scene Manifest говорит iOS, какой `SceneDelegate` создать; имя конфигурации должно совпадать с тем, что возвращает `AppDelegate` (с одной конфигурацией UIKit опечатку прощает, с несколькими — выбирает по имени).
- `AppDelegate` — для процесса: push, фоновые задачи, SDK. `SceneDelegate` — для интерфейса сцены: окно, первый экран, а также входящие URL и Universal Links.
- Окно держится в `SceneDelegate.window`. Лаунчер держим в `launcherRoot`, чтобы возвращаться к тому же экрану — и чтобы не освободился активный координатор.
- Порядок при запуске: `didFinishLaunching` → `configurationForConnecting` → `willConnectTo` → `willEnterForeground` → `didBecomeActive`. Долгой работы в этих методах быть не должно.
- Для событий жизненного цикла вне `SceneDelegate` удобны уведомления `NotificationCenter` (`UIScene.willDeactivateNotification` и другие). Жетон подписки храним и снимаем через `removeObserver`.
- UIKit помечен `@MainActor` всегда. Default Actor Isolation = MainActor делает `@MainActor` и твои собственные типы. Для работы в фоне — `nonisolated` (и `@concurrent` для асинхронных функций).
- `Task { }` наследует главный поток, `Task.detached { }` — нет; из него на главный поток переходим через `await MainActor.run { }`. Нарушения изоляции с API UIKit (импорт из

Objective-C) Swift 6.3 показывает как предупреждения, со своими `@MainActor`-типами — как ошибки; относись к обоим одинаково.
- `MainActor.assumeIsolated { }` (iOS 13+) — мост к старым API, которые точно вызываются на главном потоке; если это не так — аварийное завершение.

Apple Developer Documentation

- TN3187: Migrating to the UIKit scene-based life cycle — переход на сцены и требование после iOS 26.
- Managing your app's life cycle — состояния active / inactive / background и переходы между ними.
- `UIApplicationDelegate` — что обрабатывает делегат приложения.
- `UISceneDelegate` и `UIWindowSceneDelegate` — события сцены, `scene(_:willConnectTo:options:)`.
- `UIApplicationSceneManifest` — ключ `Info.plist` с конфигурациями сцен.
- `UISceneSession` — запись о сцене, по которой система её восстанавливает.
- `UIScene.willDeactivateNotification` и `didActivateNotification` — уведомления, на которые подписан `LifecycleSecurityController`.
- `MainActor.assumeIsolated(_:file:line:)` — проверка «мы на главном потоке» с доступностью iOS 13+.
- Concurrency — Swift Book — async/await, акторы, `Task` и изоляция.

Это конец части I. Дальше — часть II про гейты запуска: onboarding, экран разрешения, вход в аккаунт, «обновите приложение», регион и возраст, защита при сворачивании. Каждый гейт — отдельная глава с разбором UX и кода.

→ Глава 6. Onboarding — UIPageViewController с dots indicator

Глава 6. Onboarding — UIPageViewController с dots indicator

С Части II начинаются **гейты запуска**. Гейт (от англ. *gate* — «ворота», «проход с проверкой») — экран, через который пользователь проходит по пути к главному экрану mini-app. Каждый гейт что-то проверяет или спрашивает: видел ли человек вводный тур, дал ли разрешение, вошёл ли в аккаунт. Кто за кем идёт, решает `BootCoordinator` из главы 4.

Onboarding (по-русски — вводный тур, «знакомство с приложением») — самый безобидный из гейтов. Он показывается **один раз** при первом запуске mini-app и больше не появляется. Цель — за 3–4 экрана объяснить, что человек сейчас увидит и какие разрешения приложение попросит.

В этой главе делаем гейт целиком: контейнер `UIPageViewController`, индикатор-точки, кнопки «Пропустить» / «Дальше» / «Поехали» и флаг «уже видел» в `UserDefaults`, отдельный для каждого mini-app. В конце главы (раздел 6.11) лежит полный файл экрана — его можно скопировать в проект и запустить.

> **Режим компилятора.** Весь код этой части проверен в том режиме, который описан во введении: iOS 15 как минимальная версия, язык Swift 6, **Approachable Concurrency** и **Default Actor Isolation = MainActor**. Последняя настройка значит, что каждый класс и функция без явной пометки считаются работающими на главном потоке. Поэтому в листингах UIKit-классов нет `@MainActor` — он подразумевается.

6.1 Что показывать на этих экранах

Хороший onboarding отвечает на три вопроса пользователя:

1. **Что это за приложение?** Одно предложение, не три.
2. **Чем оно мне полезно?** Конкретно, без маркетинговой ваты.
3. **Что от меня потребуется?** Особенно — какие разрешения мы попросим (геолокация, фото, уведомления). Когда человек заранее понимает, зачем приложению доступ, системный запрос не застаёт его врасплох.

Хороший onboarding **не** делает: не требует email до того, как человек увидел продукт, не заставляет регистрироваться, не рассказывает «о нашей миссии».

Apple в Human Interface Guidelines (HIG — сборник правил оформления интерфейсов от Apple) формулирует похожие правила. Вводный тур должен быть коротким и, по возможности, необя-

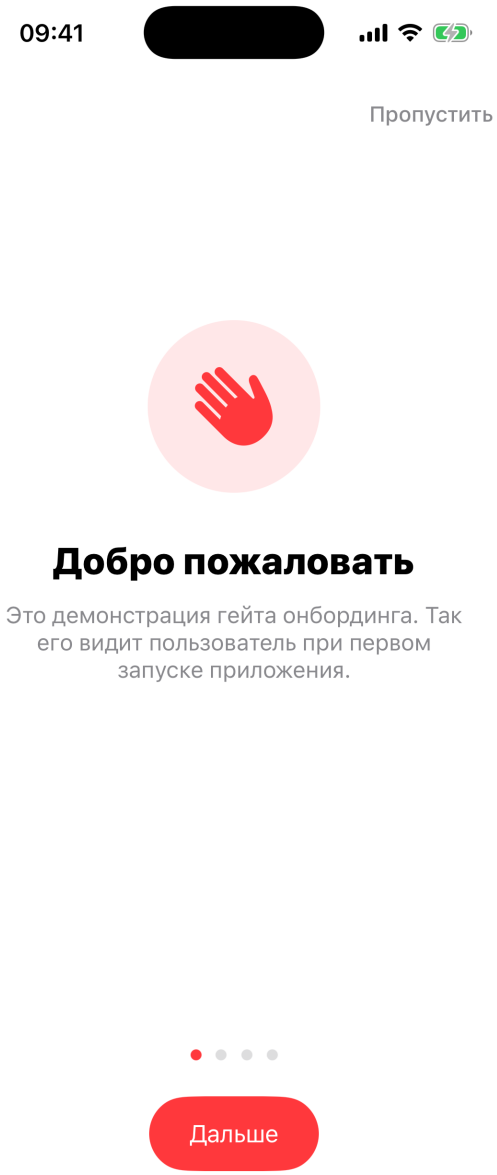


Рис. 3: Первая страница онбординга

зательным. Если человек его пропустил, при новых запусках тур не показывают, но дают найти его позже — например, пунктом «Показать обучение» в настройках. И ещё: тур объясняет **твоё** приложение, а не то, как пользоваться iPhone.

Демо-onboarding в playground'е повесим на «Список дел» (id `todo` — отсюда ключ `onboarding.seen.todo` из главы 2). В его манифесте в `AppRegistry` включаем `hasOnboarding: true` и перечисляем четыре страницы (заглушку «СКОРО» правят так же, как Чат в упражнении 2.3: `todo.hasOnboarding = true`, `todo.onboardingPages = [...]`). Тексты страниц — демонстрация механики, а не рассказ о настоящем приложении:

```
onboardingPages: [
    OnboardingPage(symbolName: "hand.wave.fill",
                   title: "Добро пожаловать",
                   body: "Это демонстрация гейта онбординга..."),
    OnboardingPage(symbolName: "bolt.fill",
                   title: "Расскажи о ключевых фишках",
                   body: "На таких страницах обычно перечисляют 2–3 главные..."),
    OnboardingPage(symbolName: "lock.shield.fill",
                   title: "Подготовь к разрешениям",
                   body: "..."),
    OnboardingPage(symbolName: "checkmark.seal.fill",
                   title: "Поехали",
                   body: "Нажми «Поехали» — ты увидишь main-экран..."),
]
```

Каждая страница — это иконка, заголовок и текст. Иконка берётся из **SF Symbols** — встроенной в iOS библиотеки из нескольких тысяч значков. Их не нужно добавлять в проект картинками: пишешь имя (`"bolt.fill"` — молния) и получаешь векторную иконку, которая масштабируется вместе с размером шрифта. Имена удобно искать в бесплатном приложении SF Symbols с сайта Apple.

Сам тип страницы — обычная структура с тремя строками:

```
struct OnboardingPage {
    let symbolName: String
    let title: String
    let body: String
}
```

Шаблон годится для любого реального приложения: заменил тексты и иконки — получил свой тур.

6.2 UIPageViewController vs альтернативы

Горизонтальное листание страниц в UIKit можно сделать тремя способами:

- **UIScrollView** с `isPagingEnabled = true`. Прокрутка сама «защёлкивается» на границах страниц, но всё остальное — вручную: раскладывать страницы, по `contentOffset` (насколько сдвинуто содержимое) вычислять номер текущей. Например, ширина экрана 390 точек, сдвиг 780 — значит, открыта третья страница (780 / 390 = 2, счёт с нуля).

- **UICollectionView** с горизонтальной раскладкой и постраничной прокруткой. Удобен, если страниц **много** (десятки): коллекция переиспользует ячейки и держит в памяти только видимые. Для 3–4 страниц это лишняя сложность.
- **UIPageViewController** — готовый контейнер, который Apple сделала именно для листания страниц. Ты отдаёшь ему «предыдущую» и «соседнюю справа» страницу, он сам рисует свайп и анимацию.

Мы берём третий: кода меньше, ошибиться сложнее.

> **Контейнер, data source, delegate.** `UIPageViewController` — это **контейнер**: view controller, который показывает внутри себя другие view controller'ы, как `UINavigationController` показывает стопку экранов. Данные он получает через **data source** (источник данных) — объект, который отвечает на вопрос «какой экран слева или справа от текущего?». О событиях («листание закончилось») он сообщает **делегату** — объекту, которому поручено реагировать. Оба — просто ссылки на объекты, реализующие нужный протокол. У нас обе роли играет сам экран онбординга. Обе ссылки — `weak` (в заголовке UIKit: `@property (nullable, nonatomic, weak) id <UIPageViewControllerDataSource> dataSource`), поэтому контейнер не держит экран онбординга и цикла ссылок не возникает. Оставишь `dataSource` пустым — листание жестами просто отключится. Почему делегат и data source в UIKit всегда weak — глава 0, раздел 0.9; семейство контейнеров — раздел 0.6.

6.3 Параметры контейнера

Инициализатор `UIPageViewController` берёт два главных аргумента:

```
private let pageController = UIPageViewController(
    transitionStyle: .scroll,
    navigationOrientation: .horizontal
)
```

- `transitionStyle: .scroll` — страницы едут вбок, как в галерее фото. Альтернатива `.pageCurl` — страница загибается, как лист бумажной книги. Выглядит старомодно, в современных приложениях почти не встречается.
- `navigationOrientation: .horizontal` — листать влево-вправо. `.vertical` — вверх-вниз; так иногда делают ленты в духе «историй».

Контейнер нужно встроить в наш экран как **дочерний view controller** (child view controller). Это отдельный механизм UIKit — **containment** («вложение» view controller'ов): родитель не просто кладёт чужую view к себе, а берёт на себя ответственность за дочерний контроллер целиком — держит его в памяти, передаёт ему события появления и смены размеров, делится safe area. Так устроены и `UINavigationController`, и `UITabBarController`, и сам `UIPageViewController` по отношению к своим страницам (глава 0, раздел 0.6).

```
private func embedPageController() {
    addChild(pageController)
    view.insertSubview(pageController.view, at: 0)
    pageController.view.translatesAutoresizingMaskIntoConstraints = false
    NSLayoutConstraint.activate([
        pageController.view.topAnchor.constraint(equalTo: view.topAnchor),
```

```
        pageController.view.leadingAnchor.constraint(equalTo: view.leadingAnchor),
        pageController.view.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        pageController.view.bottomAnchor.constraint(equalTo: view.bottomAnchor),
    ])
    pageController.didMove(toParent: self)
    pageController.dataSource = self
    pageController.delegate = self
}
```

Встраивание — три шага, это стандартный порядок из документации Apple про контейнерные view controller'ы:

1. `addChild(pageController)` — создаём связь «родитель — ребёнок»: у `pageController.parent` появляется наш экран, а сам он попадает в массив `children`. Внутри `addChild` UIKit сам зовёт у ребёнка `willMove(toParent:)`.
2. Кладём `pageController.view` в свою иерархию и растягиваем на весь экран **констрейнтами**. Констрейнт (constraint) — правило раскладки Auto Layout вида «верхний край этой view равен верхнему краю той». Четыре правила «край к краю» делают дочернюю view ровно размером с экран. Строка `translatesAutoresizingMaskIntoConstraints = false` выключает старый механизм автоматической раскладки (по умолчанию он включён у любой view), иначе его правила конфликтуют с нашими.
3. `pageController.didMove(toParent: self)` — сообщаем ребёнку, что встраивание закончено. Этот вызов UIKit за нас **не** делает: `addChild` зовёт только `willMove`, а `didMove` документация требует вызвать самим — сразу после `addChild`, если перехода с анимацией нет.

Что ломается без первого шага, видно в эксперименте: мы положили view «сироты» — контроллера без `addChild` — рядом с настоящим ребёнком и показали окно (симулятор iOS 26.5). У ребёнка `view.safeAreaInsets` = (59, 0, 34, 0), как у родителя; у сироты — нули, и `layoutMargins` тоже нули: safe area — область экрана, не закрытая чёлкой, статус-баром и полоской «домой», — ребёнку сообщает родитель, а у сироты родителя нет. `parent` и `navigationController` у сироты — `nil`: цепочка `parent` разорвана, и он не найдёт ни навигационный контроллер, ни tab bar. Методы появления (`viewWillAppear`, `viewDidAppear`) сироте всё же приходят, но не согласованно с родителем: его `viewDidAppear` сработал раньше, чем у родителя. И никто не держит сам контроллер: не будь `pageController` свойством экрана, он освободился бы сразу после `viewDidLoad`, а его view осталась бы на экране без хозяина.

Третий шаг на поведение нашего экрана не влияет: `didMove(toParent:)` — уведомление для ребёнка, который может его переопределить и, например, начать работу только после встраивания. Но это часть контракта контейнера, и пропускать его нельзя. Зеркальная операция — убрать ребёнка — делается в обратном порядке: `child.willMove(toParent: nil)` (здесь UIKit за тебя его не вызовет), затем `child.view.removeFromSuperview()`, затем `child.removeFromParent()` — последний сам зовёт `didMove(toParent: nil)`. В онбординге убирать ничего не нужно: страницы контейнер убирает сам. Подробно про containment — глава 0, раздел 0.6.

`insertSubview(_:at: 0)` кладёт страницы **под** всё остальное: индекс 0 — самый нижний слой. Кнопки и точки мы добавим позже, и они окажутся сверху.

6.4 DataSource — два метода

`UIPageViewController` спрашивает у data source ровно две вещи:

```
func pageViewController(_ pageViewController: UIPageViewController,
                        viewControllerBefore viewController: UIViewController) ->
                         ↪ UIViewController? {
    guard let current = viewController as? OnboardingPageViewController else { return nil }
    return makePage(at: current.index - 1)
}

func pageViewController(_ pageViewController: UIPageViewController,
                        viewControllerAfter viewController: UIViewController) ->
                         ↪ UIViewController? {
    guard let current = viewController as? OnboardingPageViewController else { return nil }
    return makePage(at: current.index + 1)
}
```

Контейнер передаёт текущую страницу и спрашивает соседнюю. Чтобы ответить, нужно знать **номер** текущей страницы. Можно было бы прятать номер в `view.tag`, но это неочевидно для читающего код. Надёжнее — отдельный класс страницы со свойством `index`:

```
private final class OnboardingPageViewController: UIViewController {
    let index: Int
    private let page: OnboardingPage
    private let brandColor: UIColor
    // ...
}
```

`dataSource` приводит пришедший контроллер к этому типу через `as?`, читает `index` и строит соседа через `makePage(at:)`. Если сосед за пределами массива, `makePage` возвращает `nil`, и контейнер понимает: дальше листать некуда. На первой странице свайп вправо просто пружинит и возвращается, на последней — то же с другой стороны.

`makePage(at:)` — фабрика страниц:

```
private func makePage(at index: Int) -> UIViewController? {
    guard pages.indices.contains(index) else { return nil }
    let page = pages[index]
    return OnboardingPageViewController(
        page: page,
        brandColor: manifest.brandColor,
        index: index
    )
}
```

`pages.indices.contains(index)` — безопасная проверка «такой номер в массиве есть». Для четырёх страниц `indices` — это диапазон 0…3, поэтому `-1` и `4` отсекаются, и до обращения `pages[index]` (которое на неверном индексе уронило бы приложение) дело не доходит.

Каждый запрос создаёт **новый** контроллер страницы. Контейнер держит у себя текущую страницу и, возможно, соседние, а ушедшие из вида освобождает сам. Кешировать страницы вручную не нужно: их мало, и каждая лёгкая.

Страница, которую мы отдали контейнеру, становится его дочерним контроллером: после `setViewControllers` у неё `parent === pageController`, и в `children` она тоже есть (проверено на симуляторе). То есть `UIPageViewController` делает с нашими страницами ровно то, что мы в 6.3 сделали с ним самим — `addChild`, view в иерархию, `didMove(toParent:)`.

6.5 PageControl и синхронизация со свайпом

Точки внизу — это `UIPageControl`:

```
pageControl.numberOfPages = pages.count
pageControl.currentPage = 0
pageControl.currentPageIndicatorTintColor = manifest.brandColor
pageControl.pageIndicatorTintColor = .quaternaryLabel
pageControl.hidesForSinglePage = true
pageControl.addTarget(self, action: #selector(pageControlChanged), for: .valueChanged)
```

Разбор по строкам:

- `numberOfPages` — сколько точек рисовать; `currentPage` — какая подсвечена.
- `currentPageIndicatorTintColor` — цвет активной точки, у нас это фирменный цвет mini-app из манифеста.
- `pageIndicatorTintColor = .quaternaryLabel` — неактивные точки. `.quaternaryLabel` — системный «очень бледный» цвет текста; он сам подстраивается под светлую и тёмную тему.
- `hidesForSinglePage = true` — если страница одна, точки прячутся (по умолчанию `false`; вернёмся к этому в 6.9).
- `addTarget(_:action:for: .valueChanged)` — подписка на событие «пользователь сменил страницу через точки».

`addTarget` здесь есть потому, что `UIPageControl` — контрол: `UIPageControl → UIControl → UIView`. Точки — не просто картинка, а view, которая умеет отслеживать касания и рассылать события target-action. У `UILabel` или `UIImageView` такого метода нет — они не наследуют `UIControl` (таблица «кто контрол, а кто нет» — глава 0, раздел 0.4).

`UIPageControl` сам по себе интерактивный. Тап справа от активной точки переключает на страницу вперёд, слева — назад — ровно на одну. С iOS 14 по точкам можно ещё и вести пальцем, листая страницы подряд; это поведение включено по умолчанию (`allowsContinuousInteraction == true`, проверено на симуляторе). Мы его не трогаем. Значение `currentPage` контрол сам держит в допустимых пределах: присвоишь 99 при четырёх страницах — прочитаешь 3.

Синхронизация точек и страниц идёт в обе стороны.

Свайп пальцем → точки. Когда листание закончилось, контейнер зовёт метод делегата. Мы читаем, какая страница теперь на экране, и двигаем точку:

```
func pageViewController(_ pageViewController: UIPageViewController,
                        didFinishAnimating finished: Bool,
                        previousViewControllers: [UIViewController],
                        transitionCompleted completed: Bool) {
    guard completed,
          let current = pageController.viewControllers?.first as?
          ↪ OnboardingPageViewController
    else { return }
    currentIndex = current.index
    pageControl.currentPage = current.index
    updateNextButtonTitle()
}
```

Проверка `completed` здесь обязательна. Если пользователь начал свайп, передумал и отпустил палец, метод всё равно вызовется, но с `completed == false`: страница осталась прежней. Без проверки точка убежала бы на соседнюю страницу, а на экране осталась бы старая.

`pageController.viewControllers?.first` — контейнер хранит массив показанных страниц; в режиме `.scroll` на экране всегда одна, поэтому берём первую.

Тап по точкам → страница. При тапе `UIPageControl` сначала сам меняет `currentPage`, потом шлёт `.valueChanged`. Мы ловим событие и листаем контейнер:

```
@objc private func pageControlChanged() {
    let target = pageControl.currentPage
    guard target != currentIndex else { return }
    let direction: UIPageViewController.NavigationDirection =
        target > currentIndex ? .forward : .reverse
    showPage(at: target, direction: direction, animated: true)
}
```

Направление анимации зависит от того, куда листаем: номер больше текущего — едем вперёд (`.forward`), меньше — назад (`.reverse`).

Сам переход делает общий метод `showPage`:

```
private func showPage(at index: Int,
                      direction: UIPageViewController.NavigationDirection,
                      animated: Bool) {
    guard let page = makePage(at: index) else { return }
    pageController.setViewControllers([page], direction: direction, animated: animated)
    currentIndex = index
    pageControl.currentPage = index
    updateNextButtonTitle()
}
```

`setViewControllers(_:direction:animated:)` — способ программно поставить в контейнер нужную страницу. Сразу после этого обновляем `currentIndex`, точки и заголовок кнопки. Метод `didFinishAnimating` при программной смене **не** вызывается — в заголовке UIKit он описан как

«sent when a gesture-initiated transition ends», то есть только для жестов пальцем, поэтому состояние обновляем здесь сами.

Обрати внимание на `pageControl.currentPage = index`: в обратную сторону зацикливания не будет. Контролы UIKit рассылают действия только в ответ на действия пользователя; когда значение меняет код, `.valueChanged` не уходит (у `UISwitch.setOn(_:animated:)` Apple пишет это прямо: «doesn't result in an action message being sent»). Поэтому наш `pageControlChanged` от этой строки не сработает.

6.6 Кнопки «Пропустить» и «Дальше»

```
@objc private func skipTapped() {
    finish()
}

@objc private func nextTapped() {
    if currentIndex < pages.count - 1 {
        showPage(at: currentIndex + 1, direction: .forward, animated: true)
    } else {
        finish()
    }
}
```

«Пропустить» закрывает онбординг целиком — сразу `finish()`.

«Дальше» ведёт себя по-разному. Если впереди есть страницы — листает. Если страница последняя (`currentIndex` равен `pages.count - 1`, то есть 3 при четырёх страницах) — завершает тур.

Это отражается на подписях кнопок:

```
private func updateNextButtonTitle() {
    let isLast = currentIndex == pages.count - 1
    var config = nextButton.configuration
    config?.title = isLast ? "Поехали" : "Дальше"
    nextButton.configuration = config
    skipButton.isHidden = isLast
}
```

`UIButton.Configuration` (iOS 15+) — современный способ описать кнопку целиком: текст, цвета, скругление, отступы. Это структура (значимый тип), а не объект: `nextButton.configuration` возвращает копию. Поэтому читаем её в переменную, меняем заголовок и присваиваем обратно — на присваивание кнопка перерисовывается. Короткая запись `nextButton.configuration?.title = "Поехали"` тоже компилируется и делает то же самое: Swift сам берёт копию, меняет поле и записывает структуру назад.

На последней странице кнопка называется «Поехали» (призыв к действию), а «Пропустить» прячется: пропускать уже нечего. Мелочь, но без неё на последней странице рядом стоят «Дальше» и «Пропустить», и непонятно, чем они отличаются.

6.7 «Уже видел» — флаг в UserDefaults

Гейт показывается только при первом запуске mini-app. Где хранить отметку «уже видел»? В `UserDefaults` — простом хранилище настроек «ключ → значение», которое iOS сохраняет в файл внутри **песочницы** приложения. Песочница (sandbox) — личная папка приложения на устройстве: другие приложения в неё заглянуть не могут, а при удалении приложения она стирается вместе со всеми настройками. Для булевых флагов вроде нашего `UserDefaults` подходит идеально. Для паролей и токенов — нет, про это глава 8.

Реализация — несколько статических методов и один обычный:

```
static func key(for manifest: AppManifest) -> String {
    "onboarding.seen.\(manifest.id)"
}

static func shouldShow(for manifest: AppManifest) -> Bool {
    guard manifest.hasOnboarding,
          !manifest.onboardingPages.isEmpty else { return false }
    return !UserDefaults.standard.bool(forKey: key(for: manifest))
}

static func reset(for manifest: AppManifest) {
    UserDefaults.standard.removeObject(forKey: key(for: manifest))
}

private func markSeen() {
    UserDefaults.standard.set(true, forKey: Self.key(for: manifest))
}
```

`key(for:)` строит ключ **для каждого mini-app свой**: `onboarding.seen.todo`, `onboarding.seen.notes`. Будь ключ общим, пройдя тур в Todo, человек никогда не увидел бы тур Notes.

`shouldShow(for:)` читает флаг. `UserDefaults.bool(forKey:)` для несуществующего ключа возвращает `false`, поэтому при первом запуске «не видел» получается само собой, без отдельной проверки.

`shouldShow(for:)` сделан **статическим**, чтобы координатор мог спросить «нужно ли показывать?», не создавая сам экран. Создавать контроллер ради одной проверки расточительно: пришлось бы собирать кнопки, констрейнты, контейнер — и выбросить. Решение принимается статическим методом, а экран создаётся, только если ответ «да».

Этот приём повторяется во всех гейтах книги: у каждого есть `shouldShow(for: AppManifest) -> Bool`. Координатор зовёт их по очереди и ничего не знает об их устройстве.

`markSeen()` вызывается внутри `finish()` — и при «Пропустить», и при «Поехали»:

```
private func finish() {
    markSeen()
    onFinish()
}
```

Оба случая считаем как «пользователь видел онбординг». `onFinish` — замыкание, которое передал координатор; через него гейт говорит «я закончил, веди дальше» (см. главу 4).

`reset(for:)` удаляет ключ. Он нужен в двух местах: разработчику — чтобы увидеть тур заново, не переустанавливая приложение, и пользователю — если в настройках mini-app есть пункт «Показать обучение ещё раз», как советует HIG. Такой пункт вызывает `OnboardingViewController.reset(for: manifest)` и возвращает в лаунчер; при новом входе тур покажется снова.

6.8 Бытовая аналогия

Onboarding — **инструктаж в начале экскурсии**. Гид собрал группу и сказал три главные вещи: «маршрут такой, фотографировать можно, без вспышки». Дальше инструктаж не повторяется — все знают правила.

Если бы перед каждым залом музея правила пересказывали заново, это было бы невыносимо. Но **один раз** — нужно, иначе люди не поймут, где можно фотографировать. А если кто-то прослушал, он может подойти к гиду и спросить ещё раз — это наш пункт «Показать обучение».

6.9 Краевой случай: одна страница

Что если в манифесте `onboardingPages.count == 1`?

- Одна точка в индикаторе выглядит странно: на неё нечего нажимать.
- Свайп не работает: соседних страниц нет.
- Кнопка сразу называется «Поехали».

Точки решает одна строка, которую мы уже поставили в 6.5:

```
pageControl.hidesForSinglePage = true
```

У `UIPageControl` это встроенное свойство: при `numberOfPages == 1` контрол прячется сам. Второй вариант — вообще запретить тур из одной страницы: добавить в `shouldShow` условие `manifest.onboardingPages.count >= 2`. Тогда одностраничный «онбординг» лучше сделать обычным экраном или всплывающей подсказкой.

Упражнение 6.1. В `AppRegistry.swift` (глава 2) найди манифест «Списка дел» и уменьши массив `onboardingPages` до одной страницы. Запусти mini-app и убедись, что точек нет, свайп пружинит, а кнопка называется «Поехали». Затем закомментируй `hidesForSinglePage = true` и посмотри на разницу. Какой вариант выберешь ты — спрятать точки или запретить тур из одной страницы в `shouldShow`? Реши и реализуй. Ответ — в конце главы.

6.10 Что можно добавить

Production-онбординги часто содержат:

- **Видео или анимацию** вместо SF Symbol. Анимированная иллюстрация (например, через библиотеку Lottie) выглядит живее статичной иконки.

- **Запрос разрешения прямо со страницы.** На странице «Подготовь к разрешениям» можно сразу показать системный запрос. HIG прямо предлагает так делать, если приложение без разрешения не работает. Как правильно оформить такой экран, разберём в главе 7.
- **A/B-тестирование** контента — разный текст для разных групп пользователей, чтобы сравнить, какой лучше. Тексты тогда приходят с сервера через удалённую конфигурацию (глава 9).
- **Показ по событию** — не только при первом запуске, но и после крупного обновления, чтобы рассказать о новых возможностях. Для этого в ключ добавляют версию: `onboarding.seen.todo.v2`.

Всё это строится поверх той же базы: `UIPageViewController`, флаг «видел» и кнопки «Пропустить» / «Дальше».

Упражнение 6.2. Сделай так, чтобы после обновления приложения до версии 2.0 тур показался снова, даже если человек уже видел тур версии 1. Подсказка: достаточно поменять одну функцию. Ответ — в конце главы.

6.11 Экран целиком

Ниже — полный файл `OnboardingViewController.swift` со всеми кусками из этой главы и вёрсткой, которую мы в тексте пропускали. Он собирается и запускается в режиме, описанном в начале главы; тип `AppManifest` — из главы 2.

```swift
import UIKit

final class OnboardingViewController: UIViewController {

    static func key(for manifest: AppManifest) -> String {
        "onboarding.seen.\(manifest.id)"
    }

    static func shouldShow(for manifest: AppManifest) -> Bool {
        guard manifest.hasOnboarding,
              !manifest.onboardingPages.isEmpty else { return false }
        return !UserDefaults.standard.bool(forKey: key(for: manifest))
    }

    static func reset(for manifest: AppManifest) {
        UserDefaults.standard.removeObject(forKey: key(for: manifest))
    }

    private let manifest: AppManifest
    private let onFinish: () -> Void
    private var pages: [OnboardingPage] { manifest.onboardingPages }
    private var currentIndex = 0

    private let pageController = UIPageViewController(
        transitionStyle: .scroll,
        navigationOrientation: .horizontal
```

```
    )
    private let pageControl = UIPageControl()
    private let skipButton = UIButton(type: .system)
    private let nextButton = UIButton(configuration: .filled())

    init(manifest: AppManifest, onFinish: @escaping () -> Void) {
        self.manifest = manifest
        self.onFinish = onFinish
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется: экран собирается кодом")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        embedPageController()
        setupControls()
        showPage(at: 0, direction: .forward, animated: false)
    }

    private func embedPageController() {
        addChild(pageController)
        view.insertSubview(pageController.view, at: 0)
        pageController.view.translatesAutoresizingMaskIntoConstraints = false
        NSLayoutConstraint.activate([
            pageController.view.topAnchor.constraint(equalTo: view.topAnchor),
            pageController.view.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            pageController.view.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            pageController.view.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        ])
        pageController.didMove(toParent: self)
        pageController.dataSource = self
        pageController.delegate = self
    }

    private func setupControls() {
        pageControl.numberOfPages = pages.count
        pageControl.currentPage = 0
        pageControl.currentPageIndicatorTintColor = manifest.brandColor
        pageControl.pageIndicatorTintColor = .quaternaryLabel
        pageControl.hidesForSinglePage = true
        pageControl.addTarget(self, action: #selector(pageControlChanged), for:
↪   .valueChanged)

        skipButton.setTitle("Пропустить", for: .normal)
```

```
        skipButton.tintColor = .secondaryLabel
        skipButton.addTarget(self, action: #selector(skipTapped), for: .touchUpInside)

        var config = UIButton.Configuration.filled()
        config.baseBackgroundColor = manifest.brandColor
        config.cornerStyle = .capsule
        config.title = "Дальше"
        nextButton.configuration = config
        nextButton.addTarget(self, action: #selector(nextTapped), for: .touchUpInside)

        [pageControl, skipButton, nextButton].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview($0)
        }
        let guide = view.safeAreaLayoutGuide
        NSLayoutConstraint.activate([
            skipButton.topAnchor.constraint(equalTo: guide.topAnchor, constant: 8),
            skipButton.trailingAnchor.constraint(equalTo: guide.trailingAnchor, constant:
↪ -16),

            nextButton.leadingAnchor.constraint(equalTo: guide.leadingAnchor, constant:
↪ 24),
            nextButton.trailingAnchor.constraint(equalTo: guide.trailingAnchor, constant:
↪ -24),
           nextButton.bottomAnchor.constraint(equalTo: guide.bottomAnchor, constant: -16),
            nextButton.heightAnchor.constraint(greaterThanOrEqualToConstant: 50),

            pageControl.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            pageControl.bottomAnchor.constraint(equalTo: nextButton.topAnchor, constant:
↪ -16),
        ])
    }

    private func makePage(at index: Int) -> UIViewController? {
        guard pages.indices.contains(index) else { return nil }
        let page = pages[index]
        return OnboardingPageViewController(
            page: page,
            brandColor: manifest.brandColor,
            index: index
        )
    }

    private func showPage(at index: Int,
                          direction: UIPageViewController.NavigationDirection,
                          animated: Bool) {
        guard let page = makePage(at: index) else { return }
        pageController.setViewControllers([page], direction: direction, animated:
↪ animated)
```

```
        currentIndex = index
        pageControl.currentPage = index
        updateNextButtonTitle()
    }

    @objc private func pageControlChanged() {
        let target = pageControl.currentPage
        guard target != currentIndex else { return }
        let direction: UIPageViewController.NavigationDirection =
            target > currentIndex ? .forward : .reverse
        showPage(at: target, direction: direction, animated: true)
    }

    @objc private func skipTapped() {
        finish()
    }

    @objc private func nextTapped() {
        if currentIndex < pages.count - 1 {
            showPage(at: currentIndex + 1, direction: .forward, animated: true)
        } else {
            finish()
        }
    }

    private func updateNextButtonTitle() {
        let isLast = currentIndex == pages.count - 1
        var config = nextButton.configuration
        config?.title = isLast ? "Поехали" : "Дальше"
        nextButton.configuration = config
        skipButton.isHidden = isLast
    }

    private func finish() {
        markSeen()
        onFinish()
    }

    private func markSeen() {
        UserDefaults.standard.set(true, forKey: Self.key(for: manifest))
    }
}

extension OnboardingViewController: UIPageViewControllerDataSource,
↪ UIPageViewControllerDelegate {

    func pageViewController(_ pageViewController: UIPageViewController,
                            viewControllerBefore viewController: UIViewController) ->
                             ↪ UIViewController? {
```

```
        guard let current = viewController as? OnboardingPageViewController else { return
        ↪ nil }
        return makePage(at: current.index - 1)
    }

    func pageViewController(_ pageViewController: UIPageViewController,
                            viewControllerAfter viewController: UIViewController) ->
                            ↪ UIViewController? {
        guard let current = viewController as? OnboardingPageViewController else { return
        ↪ nil }
        return makePage(at: current.index + 1)
    }

    func pageViewController(_ pageViewController: UIPageViewController,
                            didFinishAnimating finished: Bool,
                            previousViewControllers: [UIViewController],
                            transitionCompleted completed: Bool) {
        guard completed,
              let current = pageController.viewControllers?.first as?
              ↪ OnboardingPageViewController
        else { return }
        currentIndex = current.index
        pageControl.currentPage = current.index
        updateNextButtonTitle()
    }
}

private final class OnboardingPageViewController: UIViewController {
    let index: Int
    private let page: OnboardingPage
    private let brandColor: UIColor

    init(page: OnboardingPage, brandColor: UIColor, index: Int) {
        self.page = page
        self.brandColor = brandColor
        self.index = index
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground

        let icon = UIImageView(image: UIImage(systemName: page.symbolName))
        icon.tintColor = brandColor
        icon.contentMode = .scaleAspectFit
```

```
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 72,
↪  weight: .semibold)

        let titleLabel = UILabel()
        titleLabel.text = page.title
        titleLabel.font = .preferredFont(forTextStyle: .title1)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.textAlignment = .center
        titleLabel.numberOfLines = 0

        let bodyLabel = UILabel()
        bodyLabel.text = page.body
        bodyLabel.font = .preferredFont(forTextStyle: .body)
        bodyLabel.adjustsFontForContentSizeCategory = true
        bodyLabel.textColor = .secondaryLabel
        bodyLabel.textAlignment = .center
        bodyLabel.numberOfLines = 0

        let stack = UIStackView(arrangedSubviews: [icon, titleLabel, bodyLabel])
        stack.axis = .vertical
        stack.spacing = 16
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerYAnchor.constraint(equalTo:
↪  view.safeAreaLayoutGuide.centerYAnchor, constant: -40),
            stack.leadingAnchor.constraint(equalTo: view.layoutMarginsGuide.leadingAnchor,
↪  constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪  view.layoutMarginsGuide.trailingAnchor, constant: -16),
        ])
    }
}
```

Что в файле нового по сравнению с кусками из разделов:

- `required init?(coder:)` с `fatalError`. Этого требует не UIKit, а Swift: в `UIViewController` инициализатор `init?(coder:)` объявлен как `required` (он приходит из протокола `NSCoding`, через него экран создаётся из storyboard), а класс, который добавил свой `init`, обязан повторить все `required`-инициализаторы родителя. Мы собираем экран кодом, поэтому честно падаем, если кто-то попробует загрузить его из storyboard.
- В `setupControls()` кнопки привязаны к `safeAreaLayoutGuide` — прямоугольнику safe area. Поэтому «Пропустить» не залезает под чёлку, а «Дальше» — под полоску «домой».
- `nextButton.heightAnchor ... greaterThanOrEqualToConstant: 50` — высота **не меньше** 50 точек. Не «ровно 50»: если человек включил крупный шрифт, кнопка вырастет вместе с текстом.

- Шрифты страницы — `preferredFont(forTextStyle:)` вместе с `adjustsFontForContentSizeCategory = true`. Это **Dynamic Type**: размер текста следует системной настройке «Размер текста» и меняется на лету, без перезапуска экрана.
- `numberOfLines = 0` у надписей — «сколько угодно строк». Длинный текст переносится, а не обрезается многоточием.
- Страница центрируется по safe area со сдвигом на 40 точек вверх (`constant: -40`), чтобы не наезжать на кнопки внизу. Отступы по бокам — от `layoutMarginsGuide`: это «поля» view (layout margins). У обычной `UIView` они по 8 точек со всех сторон, а корневая view контроллера уважает системный минимум `systemMinimumLayoutMargins`: на iPhone 16 это 16 точек слева и справа, а сверху и снизу в поля входит ещё и safe area (59 и 34 точки — проверено на симуляторе). Подробнее про поля и safe area — глава 0, раздел 0.3.
- Цвета фона и текста — системные (`.systemBackground`, `.secondaryLabel`), поэтому тёмная тема работает без единой строки кода.

Ответы к упражнениям

6.1. Оба варианта рабочие. Если оставляешь одностраничный тур — достаточно `pageControl.hidesForSingleP = true` (строка уже есть в 6.11). Если запрещаешь — меняешь `shouldShow`:

```
static func shouldShow(for manifest: AppManifest) -> Bool {
    guard manifest.hasOnboarding,
          manifest.onboardingPages.count >= 2 else { return false }
    return !UserDefaults.standard.bool(forKey: key(for: manifest))
}
```

Проверка: с одной страницей mini-app сразу открывает главный экран, тура нет.

6.2. Добавь версию в ключ — старый флаг просто перестанет учитываться:

```
static func key(for manifest: AppManifest) -> String {
    "onboarding.seen.\(manifest.id).v2"
}
```

Проверка: пройди тур, поменяй `v2` на `v3`, запусти снова — тур появится, хотя флаг `...v2` в `UserDefaults` остался. Старые ключи можно удалить при запуске, но хранить один лишний `Bool` дёшево.

Что мы выучили

- Onboarding — **разовый** гейт: показываем при первом запуске, дальше не показываем, но даём найти тур в настройках.
- `UIPageViewController(transitionStyle: .scroll, navigationOrientation: .horizontal)` — стандартный контейнер для горизонтального листания страниц.
- Data source отдаёт соседние страницы через `viewControllerBefore` и `viewControllerAfter`. Номер страницы храним в свойстве `index` отдельного класса страницы.
- Встраивание дочернего контроллера (containment): `addChild` → view в иерархию → `didMove(toParent:)`. Без `addChild` у ребёнка нет `parent`, нулевая safe area и никто его не держит; `didMove` UIKit сам не зовёт — это обязанность контейнера по контракту.

- `UIPageControl` — контрол (`UIControl`), поэтому у него есть `addTarget`; данные и события `UIPageViewController` получает через weak-ссылки `dataSource` и `delegate`.
- Точки и страницы синхронизируем вручную: свайп — через `didFinishAnimating` с проверкой `completed`, тап по точкам — через `.valueChanged` и `setViewControllers(...)`. Программная установка `currentPage` событие не шлёт.
- На последней странице «Дальше» превращается в «Поехали», а «Пропустить» прячется.
- `shouldShow(for:) -> Bool` — статический метод гейта; координатор зовёт его, не создавая экран.
- Флаг «видел» — отдельный для каждого mini-app ключ в `UserDefaults` (`"onboarding.seen.\(manifest.id)`). `UserDefaults` — для настроек, не для секретов.
- `hidesForSinglePage` прячет точки, когда страница одна.

Apple Developer Documentation

- Human Interface Guidelines — Onboarding — вводный тур должен быть коротким и необязательным, не повторяться при новых запусках и оставаться доступным из настроек.
- `UIPageViewController` — контейнер для листания страниц; `transitionStyle: .scroll`, `navigationOrientation: .horizontal` — наша конфигурация.
- `UIPageViewControllerDataSource` и `UIPageViewControllerDelegate` — соседние страницы по запросу и уведомления о жестовых переходах; обе ссылки у контейнера weak.
- Creating a custom container view controller — порядок `addChild` → view → `didMove(toParent:)` и зеркальное удаление; то, что мы делаем в 6.3.
- `UIPageControl` — индикатор-точки; `hidesForSinglePage`, с iOS 14 — `allowsContinuousInteraction` и `preferredIndicatorImage` для своих иконок вместо точек.
- `UIScrollView` — низкоуровневая альтернатива (`isPagingEnabled = true`), когда возможностей `UIPageViewController` уже не хватает.
- `UserDefaults` — простое хранилище под флаг «онбординг пройден»; годится для настроек, но не для секретов.

→ Глава 7. Permission primer — объяснение перед системным диалогом

Глава 7. Permission primer — объяснение перед системным диалогом

Часть данных и устройств на iPhone защищена **разрешениями** (permissions): геолокация, фото, камера, микрофон, контакты, уведомления. Приложение не может просто взять их — оно должно спросить. Когда приложение впервые вызывает, например, `CLLocationManager.requestWhenInUseAuthorization()` или `PHPhotoLibrary.requestAuthorization(for:)`, iOS **сама** показывает системный alert: «Разрешить приложению доступ к геолокации?» с кнопками. Заголовок и кнопки этого окна ты не контролируешь. Твоя в нём только одна строка пояснения — о ней в разделе 7.7.

Проблема в том, что человек видит этот alert **без контекста**. Он только открыл приложение, а его уже о чём-то спрашивают. Многие по привычке жмут «Не разрешать» — на всякий случай.

А после отказа второй раз системный alert **не появится**: iOS запоминает ответ, и вернуть доступ человек может только сам, в Настройках. Для приложения это почти финал.

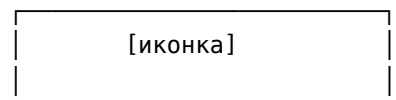
Решение — показать **свой** экран **до** системного alert'а: иконка, объяснение «зачем», пара пунктов про приватность. Такой экран называют **permission primer** (буквально «подготовка к разрешению»; по-русски — «экран-пояснение перед запросом»). В гайдлайнах Apple он называется pre-alert screen.

Насколько primer поднимает долю согласий, точно сказать нельзя: публичные цифры разных команд сильно расходятся и зависят от приложения. Но и логика, и опыт сходятся в одном: человек, который понимает, зачем нужен доступ, соглашается охотнее. В этой главе делаем такой экран — и делаем его так, чтобы его пропустила проверка App Store. Это важнее, чем кажется: у Apple есть конкретные правила для таких экранов, и многие приложения их нарушают.

Полный код главы — в разделе 7.10.

7.1 Что показываем

Структура primer'а одинакова для всех типов разрешений:



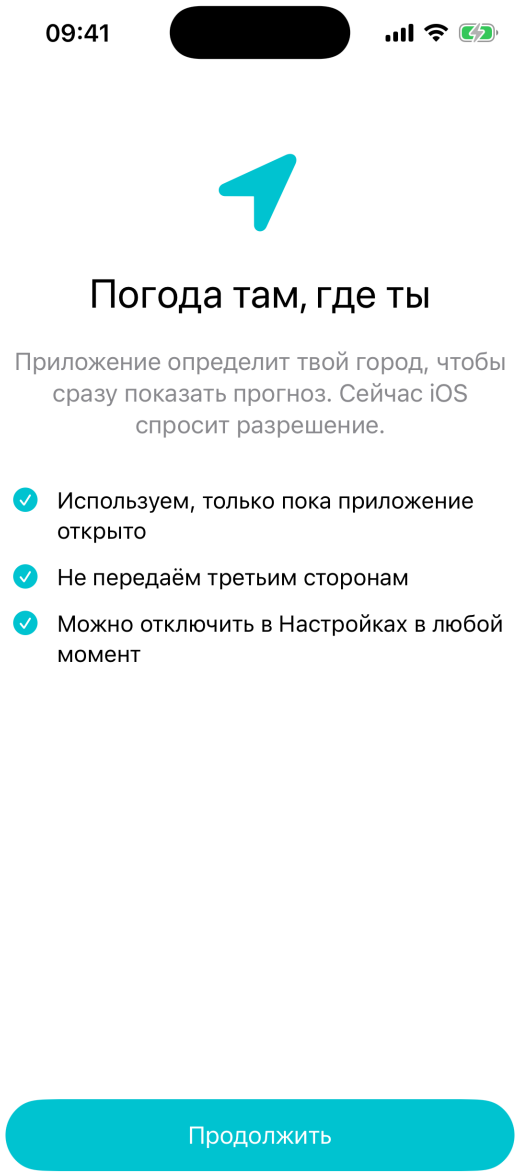
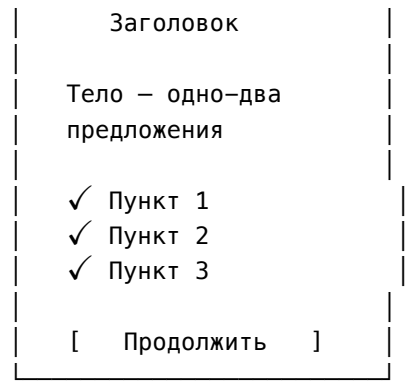


Рис. 4: Экран-пояснение перед запросом геолокации: одна кнопка «Продолжить»



Иконка — по теме (геолокация, фото, камера). Заголовок — какую **пользу** человек получит: не «Нужна геолокация», а «Погода там, где ты». Тело — одно-два предложения, и в конце — честное предупреждение «Сейчас iOS спросит разрешение». Пункты — почему это безопасно: «используем, только пока приложение открыто», «не передаём третьим сторонам», «можно отключить в Настройках».

Кнопка **одна**. Почему не две, как часто делают, — в разделе 7.3.

Меняется только **текст**. Раскладка одна на все разрешения.

7.2 Текст под каждый тип — `switch` по enum

В главе 2 мы объявили `PermissionKind` — какие разрешения бывают у наших mini-app. Напомним объявление: четыре случая, и у каждого свой способ запроса, все четыре разберём в этой главе:

```
enum PermissionKind: Sendable {
    case location, photoLibrary, camera, notifications
}
```

`PermissionPrimerViewController` хранит текст экрана в вычисляемом свойстве:

```
private var content: (icon: String, title: String, body: String, bullets: [String]) {
    switch kind {
    case .location:
        return (
            icon: "location.fill",
            title: "Погода там, где ты",
            body: "Приложение определит твой город, чтобы сразу показать прогноз. Сейчас
↪  iOS спросит разрешение.",
            bullets: [
                "Используем, только пока приложение открыто",
                "Не передаём третьим сторонам",
                "Можно отключить в Настройках в любой момент",
            ]
        )
    case .photoLibrary:
        return (...)
```

```
    case .camera:
        return (...)
    case .notifications:
        return (...)
    }
}
```

(Здесь и ниже многоточия — сокращение для книги, полный текст всех четырёх вариантов — в 7.10.)

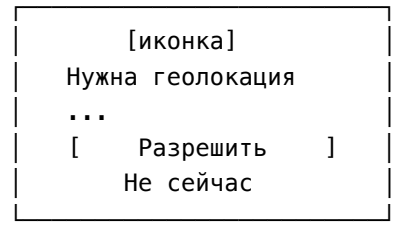
Один `switch` по `PermissionKind`, каждый случай возвращает кортеж из иконки, заголовка, тела и списка пунктов. Кортеж с именованными элементами — как маленькая безымянная структура: `content.title`, `content.bullets`. Интерфейс ничего не знает о конкретном разрешении — он просто читает эти четыре поля.

Добавишь в `PermissionKind` новый случай, например `.microphone`, — компилятор сразу укажет на этот `switch`: он перестанет быть полным. Это удобная страховка, текст для нового разрешения не забудется.

> **Почему не подкласс на каждый тип.** Можно было бы сделать `LocationPrimerViewController`, `PhotoPrimerViewController` и так далее — у каждого своя раскладка, своя анимация. Но раскладка у нас одна, и заводить четыре класса ради разного текста — лишняя сложность. Один контроллер и вычисляемое свойство проще читать и поддерживать. Понадобится особый экран для одного разрешения — вынесешь его в отдельный класс, это десять минут работы.

7.3 Кнопка — одна, и она ведёт к системному запросу

Первая версия нашего экрана выглядела так:



Две кнопки: «Разрешить» и «Не сейчас». Так делали многие приложения, и так делать **нельзя**. В Human Interface Guidelines (HIG, раздел Privacy → «Pre-alert screens, windows, or views») у Apple прямые правила для экранов перед системным запросом:

- **Только одна кнопка, и понятно, что она открывает системный alert.** Вторая кнопка, которая к alert'у не ведёт, уводит человека от решения — Apple считает это манипуляцией.
- **Не называть кнопку «Разрешить»** или похоже. На экране-пояснении человек ничего не разрешает — разрешают только в системном окне. Если кнопка primer'а по смыслу и виду похожа на «Разрешить» в alert'е, человек по инерции нажмёт «Разрешить» и там. Apple предлагает «Продолжить» или «Далее».

- **Никаких дополнительных действий**, кроме случаев, когда этого требует закон (например, юридическое согласие). В частности — никакого способа закрыть экран, не увидев системный alert: ни «Отмена», ни «Не сейчас», ни крестика.

Кроме того, правило App Review (проверки, которую сотрудники Apple проводят перед публикацией каждой версии в App Store; правила собраны в App Review Guidelines) **5.1.1(iv)** запрещает «манипулировать, обманывать или принуждать» человека к согласию, а HIG предупреждает, что экраны, которые так делают, отклоняются при проверке. Поэтому наша кнопка — «Продолжить», и она одна:

```
@objc private func continueTapped() {
    guard !requestInFlight else { return }
    requestInFlight = true
    continueButton.isEnabled = false
    let kind = self.kind
    Task { [weak self] in
        let outcome = await PermissionService.shared.request(kind)
        guard let self else { return }
        self.requestInFlight = false
        self.onResult(outcome)
    }
}
```

Что здесь происходит:

1. `requestInFlight` — защита от двойного тапа. Если человек быстро нажмёт дважды, второй тап ничего не сделает, пока первый запрос не закончится. Кнопка ещё и выключается (`isEnabled = false`) — теперь второй тап физически некуда сделать: по документации выключенный `UIControl` «ignores touch events», а в его `state` добавляется флаг `.disabled`, по которому кнопка на `UIButton.Configuration` сама перерисовывается бледной (состояния контролов — глава 0, раздел 0.4).
2. `let kind = self.kind` — копируем тип разрешения до запуска задачи, чтобы задаче не понадобилась сильная ссылка на экран.
3. `PermissionService.shared.request(kind)` — наша обёртка над системными API (разберём в 7.5). Внутри неё iOS покажет системный alert, а `await` дождётся ответа человека.
4. `guard let self` после `await` — как в главе 8: пока ждём, экран держится слабой ссылкой.
5. `onResult(outcome)` — **callback** (обратный вызов): замыкание, которое передал координатор, чтобы узнать результат.

`PermissionService.request` возвращает наш enum `Outcome`: `granted` (разрешил), `denied` (отказал), `limited` (для фото — человек дал доступ только к **некоторым** снимкам, а не ко всей библиотеке) и `notSupported` (на устройстве нет нужного железа — например, камеры в симуляторе).

Координатор **не** смотрит на конкретный ответ (см. главу 4): он идёт дальше и при `granted`, и при `denied`. Как жить без разрешения, решает само mini-app — и это тоже правило. Пункт **5.1.1(iv)** говорит: где возможно, дай альтернативу тем, кто отказался. Например, если человек не дал геолокацию, Погода должна позволить ввести город вручную, а не показывать пустой экран.

> **А если человек не хочет?** Он откажет в системном alert'е — там есть «Не разрешать». Primer не лишает его выбора, он лишь объясняет, о чём сейчас спросят. А

> «отложить решение на потом» правильнее делать иначе: просить разрешение не на старте, а в тот момент, когда человек сам открыл функцию, которой оно нужно. Об этом — в 7.6.

7.4 Вёрстка

Разметка экрана — вертикальный стек в верхней части и кнопка внизу:

```
let stack = UIStackView(arrangedSubviews: [icon, titleLabel, bodyLabel, bullets])
stack.axis = .vertical
stack.alignment = .center
stack.spacing = 20
stack.setCustomSpacing(32, after: bodyLabel)
```

`spacing = 20` — 20 точек между элементами, а после текста — 32 (`setCustomSpacing`), чтобы список пунктов визуально отделился от объяснения.

Каждый пункт — строка из галочки и текста:

```
private func makeBullet(_ text: String) -> UIView {
    let check = UIImageView(image: UIImage(systemName: "checkmark.circle.fill"))
    check.tintColor = brandColor
    check.setContentHuggingPriority(.required, for: .horizontal)
    let label = UILabel()
    label.text = text
    label.font = .preferredFont(forTextStyle: .callout)
    label.adjustsFontForContentSizeCategory = true
    label.numberOfLines = 0
    let row = UIStackView(arrangedSubviews: [check, label])
    row.spacing = 12
    row.alignment = .firstBaseline
    return row
}
```

Две строки здесь не очевидны.

`setContentHuggingPriority(.required, for: .horizontal)` — **hugging** («обнимание») говорит Auto Layout, насколько view сопротивляется растягиванию шире своего естественного размера. Естественный размер (intrinsic content size) у картинки — размер самой иконки, у надписи — размер текста. В горизонтальном стеке лишнее место достаётся тому, у кого hugging ниже. По умолчанию hugging у всех view одинаковый — 250 (`.defaultLow`), поэтому без подсказки стек растянул бы кого захочет. Ставим иконке максимальный (`.required`, то есть 1000) — она остаётся размером с иконку, а всю ширину забирает текст. Обратный приоритет, compression resistance (сопротивление сжатию), по умолчанию 750 — поэтому надпись скорее перенесётся на вторую строку, чем даст себя обрезать. Приоритеты и `intrinsicContentSize` подробно — глава 0, раздел 0.3.

`alignment = .firstBaseline` — выравнивание по базовой линии первой строки: галочка стоит ровно на уровне первой строки текста, даже если текст переносится на две строки. Этот ва-

риант имеет смысл только для горизонтального стека — в заголовке UIKit он так и помечен: «valid for horizontal axis only».

Шрифты — через `preferredFont(forTextStyle:)` и `adjustsFontForContentSizeCategory = true`: это Dynamic Type (см. главу 6), размер текста следует системной настройке.

7.5 `PermissionService` — обёртка над системными API

Системные API разрешений писались в разные годы и выглядят по-разному:

- **CoreLocation** — через делегата: ответ приходит вызовом метода `locationManagerDidChangeAuthorizati`. Готовой async-версии запроса нет.
- **Photos** и **AVCaptureDevice** (камера) — исторически через completion handler (замыкание, которое вызывается по завершении). Swift автоматически создаёт для таких Objective-C-методов async-версии, и мы пользуемся ими.
- **UserNotifications** — тоже async-версия того же происхождения.

Чтобы интерфейс не возился с этим зоопарком, делаем один сервис с одним методом:

```
final class PermissionService: NSObject {

    static let shared = PermissionService()
    private override init() {}

    enum Outcome: Sendable, Equatable {
        case granted
        case limited
        case denied
        case notSupported
    }

    func request(_ kind: PermissionKind) async -> Outcome {
        switch kind {
        case .location: return await requestLocation()
        case .photoLibrary: return await requestPhotos()
        case .camera: return await requestCamera()
        case .notifications: return await requestNotifications()
        }
    }
}
```

Интерфейс вызывает `await PermissionService.shared.request(.camera)` и получает `Outcome`. Что внутри — забота сервиса. `NSObject` в предках нужен, потому что сервис станет делегатом `CLLocationManager`: протокол `CLLocationManagerDelegate`, как почти все протоколы-делегаты в UIKit и соседних фреймворках Apple, наследует `NSObjectProtocol`, а значит, класс-делегат должен быть потомком `NSObject` (глава 0, разделы 0.1 и 0.9). При этом сервису не нужны ни view, ни `UIViewController` — у него нет интерфейса, только логика. `private override init()` — чтобы никто не создал второй экземпляр мимо `shared`.

CoreLocation — делегат через `withCheckedContinuation`

CoreLocation — самая хитрая обёртка. Метод запроса ничего не возвращает, ответ приходит в делегат:

```
private var locationManager: CLLocationManager?
private var locationContinuation: CheckedContinuation<Outcome, Never>?

private func requestLocation() async -> Outcome {
    let manager = CLLocationManager()
    switch manager.authorizationStatus {
    case .authorizedAlways, .authorizedWhenInUse: return .granted
    case .denied, .restricted: return .denied
    default: break
    }
    guard locationContinuation == nil else { return .denied }
    locationManager = manager
    return await withCheckedContinuation { cont in
        locationContinuation = cont
        manager.delegate = self
        manager.requestWhenInUseAuthorization()
    }
}

private func resolveLocation(_ outcome: Outcome) {
    guard let cont = locationContinuation else { return }
    locationContinuation = nil
    locationManager?.delegate = nil
    locationManager = nil
    cont.resume(returning: outcome)
}
```

По шагам:

1. Создаём `CLLocationManager` и **сначала** смотрим текущий статус (`authorizationStatus`, iOS 14+). Если ответ уже есть — возвращаем его сразу. Вызов `requestWhenInUseAuthorization()` при уже известном статусе ничего не показывает и делегата не зовёт — мы бы ждали вечно.
2. `guard locationContinuation == nil` — если запрос уже идёт, второй не начинаем. Защита на уровне сервиса, в дополнение к флагу экрана.
3. Менеджер сохраняем в свойстве. Если бы он был только локальной переменной, после выхода из функции его освободил бы ARC (автоматический подсчёт ссылок), и делегат никогда бы не сработал.
4. `withCheckedContinuation` — мост между «ответ придёт в делегат» и `async`-функцией. Он приостанавливает функцию и даёт объект **continuation** (`cont`) — «пульт», которым её потом можно продолжить. Пульт сохраняем в свойство.
5. `requestWhenInUseAuthorization()` — система показывает alert.
6. Когда человек ответил, iOS вызывает делегат, делегат зовёт `resolveLocation`, и `cont.resume(returning:)` продолжает функцию с нашим результатом.

Делегат:

```
extension PermissionService: CLLocationManagerDelegate {
    nonisolated func locationManagerDidChangeAuthorization(_ manager: CLLocationManager) {
        let status = manager.authorizationStatus
        MainActor.assumeIsolated {
            switch status {
            case .notDetermined:
                break
            case .authorizedAlways, .authorizedWhenInUse:
                resolveLocation(.granted)
            case .denied, .restricted:
                resolveLocation(.denied)
            @unknown default:
                resolveLocation(.denied)
            }
        }
    }
}
```

- `nonisolated` — протокол `CLLocationManagerDelegate` объявлен в Objective-C и ничего не знает о главном акторе. Метод, который его реализует, должен быть неизолированным, иначе Swift 6 не примет соответствие протоколу.
- `MainActor.assumeIsolated { ... }` — CoreLocation вызывает методы делегата в том потоке, где был создан менеджер. Мы создали его на главном потоке, значит, и вызов будет на главном. `assumeIsolated` сообщает это компилятору и разрешает трогать `resolveLocation` — метод главного актора. Если бы обещание оказалось ложным, приложение остановилось бы с понятной ошибкой. Функция доступна с iOS 13.
- `case .notDetermined: break` — важная строка. С iOS 14 этот метод вызывается ещё и **сразу после создания менеджера** (и назначения делегата), со статусом «пока не определено». Если принять этот вызов за ответ, запрос закончится раньше, чем человек что-то нажмёт. Поэтому «не определено» пропускаем и ждём настоящего ответа.
- `@unknown default` — на случай, если в будущих iOS появятся новые статусы.

Continuation — одноразовая. Если вызвать `resume` у одной и той же continuation **дважды**, приложение упадёт. Если **ни разу** — функция будет ждать вечно (а `withCheckedContinuation` напишет об утечке в консоль). Поэтому `resolveLocation` первым делом забирает continuation и обнуляет свойство: второй вызов найдёт `nil` и ничего не сделает.

Фото, камера и уведомления — готовые async-версии

Здесь continuation не нужна: Swift сам превращает Objective-C-методы с completion handler'ом в `async`-функции.

```
private func requestPhotos() async -> Outcome {
    let status = await PHPhotoLibrary.requestAuthorization(for: .readWrite)
    switch status {
    case .authorized: return .granted
    case .limited: return .limited
```

```
    case .denied, .restricted, .notDetermined: return .denied
    @unknown default: return .denied
    }
}

private func requestCamera() async -> Outcome {
    guard AVCaptureDevice.default(for: .video) != nil else { return .notSupported }
    let granted = await AVCaptureDevice.requestAccess(for: .video)
    return granted ? .granted : .denied
}

private func requestNotifications() async -> Outcome {
    UserDefaults.standard.set(true, forKey: Self.notificationsAskedKey)
    do {
        let granted = try await UNUserNotificationCenter.current()
            .requestAuthorization(options: [.alert, .sound, .badge])
        return granted ? .granted : .denied
    } catch {
        return .denied
    }
}
```

Разбор:

- `PHPhotoLibrary.requestAuthorization(for: .readWrite)` (iOS 14+) — полный доступ на чтение и запись. Бывает ещё `.addOnly` — только добавлять снимки, не видя библиотеку. Статус `.restricted` значит, что доступ запрещён не человеком, а ограничениями устройства (например, родительским контролем).
- `AVCaptureDevice.default(for: .video) == nil` — у устройства нет камеры. В симуляторе камеры нет, отсюда наш случай `.notSupported`.
- Уведомления: `options` — что именно просим: баннеры (`.alert`), звук, значок-счётчик на иконке (`.badge`). Метод может бросить ошибку, поэтому `try` и `do/catch`. Про флаг `notificationsAskedKey` — в 7.6.

  Почему не continuation и здесь. Можно написать и по-старому: `withCheckedContinuation { cont in PHPhotoLibrary.requestAuthorization(for: .readWrite) { status in cont.resume(returning: ...) } }`. Это работает, но длиннее, и в замыкании легко забыть, что Photos вызывает его на своей очереди, а не на главном потоке (так сказано в документации к этому методу). Тронешь там интерфейс — получишь ошибку. Async-версия возвращает результат прямо в код главного актора.

7.6 `shouldShow` — когда primer не нужен

Primer показываем, только если человек ещё **ни разу** не отвечал на системный запрос:

```
static func shouldShow(for manifest: AppManifest) -> Bool {
    guard let kind = manifest.requiresPermission else { return false }
    return PermissionService.shared.isNotDetermined(kind)
```

```
}
```

А в сервисе:

```
func isNotDetermined(_ kind: PermissionKind) -> Bool {
    switch kind {
    case .location:
        return CLLocationManager().authorizationStatus == .notDetermined
    case .photoLibrary:
        return PHPhotoLibrary.authorizationStatus(for: .readWrite) == .notDetermined
    case .camera:
        return AVCaptureDevice.authorizationStatus(for: .video) == .notDetermined
    case .notifications:
        return !UserDefaults.standard.bool(forKey: Self.notificationsAskedKey)
    }
}

private static let notificationsAskedKey = "permission.asked.notifications"
```

Три случая:

- **Ещё не спрашивали** (`.notDetermined`) → показываем primer.
- **Уже разрешено** (в том числе частично, `.limited`) → primer не нужен, просить повторно бессмысленно.
- **Уже отказано** → primer тоже не нужен. Системный alert больше не появится, и кнопка «Продолжить» ни к чему не приведёт. Правильнее, чтобы mini-app в нужный момент показало своё сообщение с кнопкой «Открыть Настройки» (`UIApplication.openSettingsURLString` открывает страницу настроек твоего приложения) или предложило обходной путь.

С уведомлениями особый случай. Их статус iOS отдаёт только асинхронно (`notificationSettings()`), а `shouldShow` — синхронный: координатор спрашивает и сразу получает ответ. Поэтому для уведомлений запоминаем сами: в `requestNotifications` ставим флаг «уже спрашивали». Системный запрос уведомлений тоже показывается только один раз, так что флаг отражает правду. При удалении приложения `UserDefaults` стирается вместе с разрешениями — флаг не соврёт и после переустановки.

`shouldShow` сильно упрощает координатор: он ничего не знает о разрешениях, просто зовёт `PermissionPrimerViewController.shouldShow(for: manifest)`.

> **Когда спрашивать.** HIG советует не просить разрешения при запуске, если приложение может работать без них, а спрашивать в момент, когда человек сам открыл функцию. Исключение — когда разрешение нужно для самой сути приложения, и это понятно без слов: навигатору без геолокации делать нечего. Наш гейт спрашивает на входе в mini-app, поэтому включай его только для таких случаев (Погода без города бесполезна). Для «вставить фото в заметку» лучше спросить в момент тапа по кнопке «Добавить фото» — а ещё лучше, не спрашивать вовсе, см. 7.8.

Упражнение 7.1. Открой mini-app «Погода» (`requiresPermission = .location`). Увидишь primer. Нажми «Продолжить» — появится системный alert геолокации. Выбери «Разрешить при использовании». Зайди в Погоду снова — primer не появится. Теперь сбрось разрешение и про-

верь, что primer вернулся. Как сбросить разрешение двумя способами — через Настройки и через терминал? Ответ — в конце главы.

7.7 Info.plist: строки-пояснения

Info.plist — файл настроек приложения, который лежит внутри его **bundle** (пакета приложения: папки `.app` с кодом, картинками и настройками). В нём хранятся имя, версия и, среди прочего, **строки- пояснения** для разрешений (purpose string, usage description). В Xcode 26 эти ключи редактируются на вкладке **Info** настроек таргета; в новых проектах отдельного файла `Info.plist` может и не быть — Xcode собирает его из настроек сборки.

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Показываем погоду в твоём городе.</string>

<key>NSPhotoLibraryUsageDescription</key>
<string>Вставляем выбранные тобой фото в заметки.</string>

<key>NSCameraUsageDescription</key>
<string>Делаем фото для аватара.</string>
```

Имя ключа говорит, к какому разрешению он относится: геолокация «при использовании», фотобиблиотека, камера. Для уведомлений ключ **не нужен** — их alert обходится без пояснения.

Эту строку iOS показывает в системном alert'е после названия приложения, над кнопками. Это твоя последняя возможность объяснить «зачем» прямо в момент выбора. HIG советует: короткое законченное предложение, конкретно, без пассивного залога, с точкой в конце. Правило App Review **5.1.1(ii)** требует, чтобы строка ясно и полностью описывала, как используются данные. Размытое «Приложению нужен доступ к фото» — частая причина отказа.

Что будет, если строки нет:

- **Фото, камера, микрофон, контакты и другие данные под защитой TCC** (подсистемы iOS, которая следит за доступом к личным данным) — приложение **падает** при первом запросе. В консоли Xcode появляется сообщение вида: «This app has crashed because it attempted to access privacy-sensitive data without a usage description. The app's Info.plist must contain an NSPhotoLibraryUsageDescription key…».
- **Геолокация** — запрос молча игнорируется: alert не появляется, статус остаётся «не определён», а в консоль пишется предупреждение «This app has attempted to access privacy-sensitive data without a usage description…». Это коварнее падения — легко не заметить. А наш `requestLocation()` в такой ситуации будет ждать ответа делегата вечно, и primer останется на экране с выключенной кнопкой.

Оба поведения мы проверили запуском в симуляторе iOS 26.5: без `NSPhotoLibraryUsageDescription` приложение завершилось с сообщением выше, без `NSLocationWhenInUseUsageDescription` — только напечатало предупреждение.

Как выглядит alert с нашей строкой (iOS 26, запрос полного доступа к фото): заголовок «Приложение „…“ запрашивает полный доступ к медиатеке.», под ним — наша строка из Info.plist, превью нескольких снимков и три кнопки: «Ограничить доступ…», «Разрешить полный доступ», «Не разрешать».

При загрузке сборки в App Store Connect (веб-кабинет разработчика, через который приложение отправляют на проверку и публикуют) автоматическая проверка тоже ругается на отсутствующие строки для API, которые использует код, и сборка не проходит.

> **Primer и строка-пояснение — об одном и том же.** Текст на нашем экране и в `NSPhotoLibraryUsageDescription` могут отличаться словами, но не смыслом. Если на primer'е написано «вставлять фото в заметки», а в Info.plist — «сменить аватар», это выглядит странно и для человека, и для проверяющего App Review.

7.8 Бывает и без разрешения

Лучшее разрешение — то, которое не нужно спрашивать. Правило **5.1.1(iii)** прямо советует по возможности брать «внепроцессный» выбор или стандартные окна вместо полного доступа к фото или контактам.

- **`PHPickerViewController`** (iOS 14+) — системное окно выбора фото. Оно работает в отдельном процессе iOS: приложение получает только те снимки, которые человек выбрал, и **никакого разрешения не требуется**. Для «вставить фото в заметку» или «выбрать аватар» это лучший выбор: ни primer'а, ни alert'а, ни строки в Info.plist.
- **`CLLocationButton`** из CoreLocationUI (iOS 15+) — системная кнопка «поделиться геолокацией». Нажатие даёт разовый доступ к местоположению без alert'а. Подходит для «найти магазины рядом».
- **Точная и примерная геолокация.** С iOS 14 человек может дать доступ только к **примерному** местоположению (с точностью до нескольких километров). Для погоды этого хватает; проверить, что выбрал человек, можно через `manager.accuracyAuthorization`.

7.9 Бытовая аналогия

Системный alert — **пограничник** с двумя штампами: «разрешить» и «не разрешать». Ответил невнятно — получил «не разрешать», и пересдать нельзя.

Permission primer — **стюардесса перед посадкой**, которая раздаёт миграционные карточки и объясняет: «на границе спросят цель поездки, вот что нужно показать». Подготовленный человек проходит границу спокойнее. Но стюардесса не ставит штамп сама и не уводит тебя в обход пограничника — она просто объясняет и провожает к окошку. Это и есть наша единственная кнопка «Продолжить».

7.10 Экран целиком

`PermissionService` целиком — это куски из 7.5 и 7.6, собранные в один класс (ниже он приведён полностью). Затем — сам экран.

```
import UIKit
import CoreLocation
import Photos
import AVFoundation
import UserNotifications
```

```
final class PermissionService: NSObject {

    static let shared = PermissionService()
    private override init() {}

    enum Outcome: Sendable, Equatable {
        case granted
        case limited
        case denied
        case notSupported
    }

    func isNotDetermined(_ kind: PermissionKind) -> Bool {
        switch kind {
        case .location:
            return CLLocationManager().authorizationStatus == .notDetermined
        case .photoLibrary:
            return PHPhotoLibrary.authorizationStatus(for: .readWrite) == .notDetermined
        case .camera:
            return AVCaptureDevice.authorizationStatus(for: .video) == .notDetermined
        case .notifications:
            return !UserDefaults.standard.bool(forKey: Self.notificationsAskedKey)
        }
    }

    private static let notificationsAskedKey = "permission.asked.notifications"

    func request(_ kind: PermissionKind) async -> Outcome {
        switch kind {
        case .location: return await requestLocation()
        case .photoLibrary: return await requestPhotos()
        case .camera: return await requestCamera()
        case .notifications: return await requestNotifications()
        }
    }

    private var locationManager: CLLocationManager?
    private var locationContinuation: CheckedContinuation<Outcome, Never>?

    private func requestLocation() async -> Outcome {
        let manager = CLLocationManager()
        switch manager.authorizationStatus {
        case .authorizedAlways, .authorizedWhenInUse: return .granted
        case .denied, .restricted: return .denied
        default: break
        }
        guard locationContinuation == nil else { return .denied }
        locationManager = manager
```

```
        return await withCheckedContinuation { cont in
            locationContinuation = cont
            manager.delegate = self
            manager.requestWhenInUseAuthorization()
        }
    }

    private func resolveLocation(_ outcome: Outcome) {
        guard let cont = locationContinuation else { return }
        locationContinuation = nil
        locationManager?.delegate = nil
        locationManager = nil
        cont.resume(returning: outcome)
    }

    private func requestPhotos() async -> Outcome {
        let status = await PHPhotoLibrary.requestAuthorization(for: .readWrite)
        switch status {
        case .authorized: return .granted
        case .limited: return .limited
        case .denied, .restricted, .notDetermined: return .denied
        @unknown default: return .denied
        }
    }

    private func requestCamera() async -> Outcome {
        guard AVCaptureDevice.default(for: .video) != nil else { return .notSupported }
        let granted = await AVCaptureDevice.requestAccess(for: .video)
        return granted ? .granted : .denied
    }

    private func requestNotifications() async -> Outcome {
        UserDefaults.standard.set(true, forKey: Self.notificationsAskedKey)
        do {
            let granted = try await UNUserNotificationCenter.current()
                .requestAuthorization(options: [.alert, .sound, .badge])
            return granted ? .granted : .denied
        } catch {
            return .denied
        }
    }
}

extension PermissionService: CLLocationManagerDelegate {
    nonisolated func locationManagerDidChangeAuthorization(_ manager: CLLocationManager) {
        let status = manager.authorizationStatus
        MainActor.assumeIsolated {
            switch status {
```

```
            case .notDetermined:
                break
            case .authorizedAlways, .authorizedWhenInUse:
                resolveLocation(.granted)
            case .denied, .restricted:
                resolveLocation(.denied)
            @unknown default:
                resolveLocation(.denied)
            }
        }
    }
}
```

```
import UIKit

final class PermissionPrimerViewController: UIViewController {

    static func shouldShow(for manifest: AppManifest) -> Bool {
        guard let kind = manifest.requiresPermission else { return false }
        return PermissionService.shared.isNotDetermined(kind)
    }

    private let kind: PermissionKind
    private let brandColor: UIColor
    private let onResult: (PermissionService.Outcome) -> Void
    private var requestInFlight = false
    private let continueButton = UIButton(configuration: .filled())

    init(kind: PermissionKind,
         brandColor: UIColor,
         onResult: @escaping (PermissionService.Outcome) -> Void) {
        self.kind = kind
        self.brandColor = brandColor
        self.onResult = onResult
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private var content: (icon: String, title: String, body: String, bullets: [String]) {
        switch kind {
        case .location:
            return (
                icon: "location.fill",
                title: "Погода там, где ты",
                body: "Приложение определит твой город, чтобы сразу показать прогноз.
↪  Сейчас iOS спросит разрешение.",
                bullets: [
```

```
                    "Используем, только пока приложение открыто",
                    "Не передаём третьим сторонам",
                    "Можно отключить в Настройках в любой момент",
                ]
            )
        case .photoLibrary:
            return (
                icon: "photo.on.rectangle",
                title: "Фото для заметок",
                body: "Чтобы вставлять снимки в заметки, нужен доступ к фото. Сейчас iOS
↪  спросит разрешение.",
                bullets: [
                    "Можно открыть доступ только к выбранным снимкам",
                    "Ничего не загружаем без твоего действия",
                    "Можно отключить в Настройках в любой момент",
                ]
            )
        case .camera:
            return (
                icon: "camera.fill",
                title: "Снимок для аватара",
                body: "Камера нужна, чтобы сделать фото профиля. Сейчас iOS спросит
↪  разрешение.",
                bullets: [
                    "Камера включается только по твоей кнопке",
                    "Снимок остаётся на устройстве",
                    "Можно отключить в Настройках в любой момент",
                ]
            )
        case .notifications:
            return (
                icon: "bell.badge.fill",
                title: "Напоминания о задачах",
                body: "Пришлём уведомление, когда подойдёт срок задачи. Сейчас iOS спросит
↪  разрешение.",
                bullets: [
                    "Только о твоих задачах, без рекламы",
                    "Не чаще, чем ты сам запланировал",
                    "Можно отключить в Настройках в любой момент",
                ]
            )
        }
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        let c = content
```

```
        let icon = UIImageView(image: UIImage(systemName: c.icon))
        icon.tintColor = brandColor
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 56,
↪  weight: .semibold)

        let titleLabel = UILabel()
        titleLabel.text = c.title
        titleLabel.font = .preferredFont(forTextStyle: .title1)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.textAlignment = .center
        titleLabel.numberOfLines = 0

        let bodyLabel = UILabel()
        bodyLabel.text = c.body
        bodyLabel.font = .preferredFont(forTextStyle: .body)
        bodyLabel.adjustsFontForContentSizeCategory = true
        bodyLabel.textColor = .secondaryLabel
        bodyLabel.textAlignment = .center
        bodyLabel.numberOfLines = 0

        let bullets = UIStackView(arrangedSubviews: c.bullets.map(makeBullet))
        bullets.axis = .vertical
        bullets.spacing = 12

        let stack = UIStackView(arrangedSubviews: [icon, titleLabel, bodyLabel, bullets])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 20
        stack.setCustomSpacing(32, after: bodyLabel)

        var config = UIButton.Configuration.filled()
        config.title = "Продолжить"
        config.baseBackgroundColor = brandColor
        config.cornerStyle = .capsule
        continueButton.configuration = config
        continueButton.addTarget(self, action: #selector(continueTapped), for:
↪  .touchUpInside)

        [stack, continueButton].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview($0)
        }
        let margins = view.layoutMarginsGuide
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪  constant: 48),
            stack.leadingAnchor.constraint(equalTo: margins.leadingAnchor, constant: 8),
```

```
            stack.trailingAnchor.constraint(equalTo: margins.trailingAnchor, constant: -8),

            continueButton.leadingAnchor.constraint(equalTo: margins.leadingAnchor,
↪   constant: 8),
            continueButton.trailingAnchor.constraint(equalTo: margins.trailingAnchor,
↪   constant: -8),
            continueButton.bottomAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.bottomAnchor, constant: -16),
            continueButton.heightAnchor.constraint(greaterThanOrEqualToConstant: 50),
        ])
    }

    private func makeBullet(_ text: String) -> UIView {
        let check = UIImageView(image: UIImage(systemName: "checkmark.circle.fill"))
        check.tintColor = brandColor
        check.setContentHuggingPriority(.required, for: .horizontal)
        let label = UILabel()
        label.text = text
        label.font = .preferredFont(forTextStyle: .callout)
        label.adjustsFontForContentSizeCategory = true
        label.numberOfLines = 0
        let row = UIStackView(arrangedSubviews: [check, label])
        row.spacing = 12
        row.alignment = .firstBaseline
        return row
    }

    @objc private func continueTapped() {
        guard !requestInFlight else { return }
        requestInFlight = true
        continueButton.isEnabled = false
        let kind = self.kind
        Task { [weak self] in
            let outcome = await PermissionService.shared.request(kind)
            guard let self else { return }
            self.requestInFlight = false
            self.onResult(outcome)
        }
    }
}
```

7.11 Краевые случаи

Симулятор и устройство. В симуляторе часть разрешений ведёт себя не так, как на iPhone. Камеры нет — отсюда `.notSupported`. Разрешение на уведомления запрашивается как обычно, а вот push-уведомления с сервера доходят до симулятора только в ограниченном виде; проверять их лучше на устройстве (глава 41).

«Разрешить один раз». В alert'е геолокации есть вариант «Разрешить один раз». Он даёт доступ только до конца текущего использования приложения; потом статус снова становится «не определён», и при новом запросе alert появится опять. Для нашего гейта это значит: primer покажется снова. Это честное поведение — человек сам выбрал «один раз».

Фоновая геолокация. Доступ «Всегда» (`authorizedAlways`) Apple выдаёт неохотно: нужен отдельный ключ `NSLocationAlwaysAndWhenInUseUsageDescription`, а система спустя время сама переспрашивает человека, показывая, как часто приложение брало геолокацию в фоне. Мы просим только «при использовании» — этого хватает большинству приложений.

Частичный доступ к фото. С iOS 14 человек может выбрать «Ограниченный доступ» и отметить несколько снимков. Остальная библиотека приложению не видна. Это полноценный сценарий, к которому нужно готовиться, поэтому в `Outcome` есть отдельный `.limited`. (Снова: если полная библиотека не нужна, `PHPickerViewController` из 7.8 снимает вопрос целиком.)

Отслеживание (App Tracking Transparency). Если приложение отслеживает человека между приложениями и сайтами других компаний (обычно для рекламы), оно обязано спросить через `ATTrackingManager.requestTrackingAuthorization` и указать ключ `NSUserTrackingUsageDescription`. Для экрана перед этим запросом HIG строже: нельзя обещать награду за согласие, нельзя показывать картинку системного alert'а или стрелки к кнопке «Разрешить», нельзя делать экран, похожий на сам запрос.

Упражнение 7.2. Переделай первую версию экрана (схема в начале раздела 7.3) так, чтобы она соответствовала правилам из 7.3. Что нужно убрать и что переименовать? Ответ — в конце главы.

Ответы к упражнениям

7.1. Через Настройки: «Настройки → Конфиденциальность и безопасность → Службы геолокации → название приложения», выбрать вариант «Спросить в следующий раз» (в свежих версиях iOS у него может быть более длинное название) — статус вернётся в «не определён». (Вариант «Никогда» — это отказ, после него primer не покажется: `shouldShow` вернёт `false`.) Через терминал, для симулятора:

```
xcrun simctl privacy booted reset location <bundle id приложения>
```

`booted` — «запущенный симулятор», bundle id — идентификатор приложения из настроек таргета. Учти, что при смене разрешения iOS может завершить запущенное приложение — это нормально. После сброса зайди в Погоду: primer появится снова.

7.2. Убрать кнопку «Не сейчас» совсем: на экране перед системным запросом не должно быть способа уйти, не увидев alert. Кнопку «Разрешить» переименовать в «Продолжить» (или «Далее»): на этом экране человек ничего не разрешает. В тексте тела полезно добавить «Сейчас iOS спросит разрешение» — так понятно, что будет дальше. Заголовок «Нужна геолокация» лучше заменить на пользу для человека: «Погода там, где ты». Итог — экран из 7.10.

Что мы выучили

- Системный alert разрешения человек видит без контекста; после отказа второй раз он не появится — только Настройки.

- **Permission primer** — свой экран **до** системного alert'а: иконка, польза, пара пунктов про приватность.
- По HIG у такого экрана **одна** кнопка, «Продолжить» или «Далее», а не «Разрешить», и никаких «Не сейчас» или «Отмена». Нарушение — типичная причина отказа в App Review (5.1.1(iv)).
- Текст под тип разрешения — `switch` по `PermissionKind` с кортежем `(icon, title, body, bullets)`.
- Выключенный контрол (`isEnabled = false`) игнорирует касания и получает флаг `.disabled` в `state`; hugging у view по умолчанию 250, compression resistance — 750, `.required` — 1000.
- `PermissionService` прячет разные системные API за одним `async` методом и возвращает `Outcome` (granted / limited / denied / notSupported).
- CoreLocation: делегат + `withCheckedContinuation`; сначала проверить текущий статус, в делегате пропустить `.notDetermined`, continuation возобновлять ровно один раз.
- Фото, камера, уведомления — готовые async-версии: короче, чем continuation, и результат сразу приходит на главный актор.
- В Info.plist нужна строка-пояснение для каждого разрешения. Без неё фото/камера падают, геолокация молча не спрашивает; уведомлениям строка не нужна.
- Primer показываем, только если статус «не определён».
- Лучше всего — вообще не спрашивать: `PHPickerViewController`, `CLLocationButton`, примерная геолокация. И всегда давать обходной путь тем, кто отказал.

Apple Developer Documentation

- Human Interface Guidelines — Privacy — раздел «Pre-alert screens»: одна кнопка «Продолжить»/«Далее», без «Разрешить» и без способа уйти мимо системного alert'а; как писать строки-пояснения.
- App Review Guidelines — пункт 5.1.1: согласие и строки-пояснения (ii), минимум данных и системные окна выбора (iii), запрет манипуляций и обходной путь для отказавших (iv).
- `CLLocationManager.requestWhenInUseAuthorization()` — геолокация «при использовании»; ответ приходит делегату, поэтому в `PermissionService` обёрнут в `withCheckedContinuation`.
- `PHPhotoLibrary` — доступ к фото: `.readWrite` / `.addOnly` и отдельное состояние `.limited`.
- `PHPickerViewController` — выбор фото без разрешения на библиотеку.
- `AVCaptureDevice` — `requestAccess(for:)` для камеры и микрофона, у метода есть async-версия.
- `UNUserNotificationCenter` — `requestAuthorization(options:)` для уведомлений; статус — только асинхронно через `notificationSettings()`.
- `UIControl` — `isEnabled`, `state` и target-action: почему выключенная кнопка «Продолжить» не реагирует на второй тап.
- `ATTrackingManager.requestTrackingAuthorization` — App Tracking Transparency; для экрана перед этим запросом правила ещё строже.
- Bundle resources — Information Property List — справочник ключей `NS...UsageDescription`.

→ Глава 8. Auth gate — Login / Register / Forgot

Глава 8. Auth gate — Login / Register / Forgot

Auth gate (гейт авторизации) показывается, если mini-app требует входа в аккаунт (`manifest.hasAuthGate == true`), а пользователь ещё не вошёл. В нашем playground'е такой mini-app один — Profile / Настройки.

Гейт состоит из трёх экранов:

- **Login** — корневой. Email, пароль, кнопка «Войти», ссылки «Забыли пароль?» и «Создать аккаунт».
- **Register** — открывается из Login. Email, пароль, повтор пароля, галочка «согласен с условиями», индикатор надёжности пароля.
- **Forgot password** — тоже открывается из Login. Поле email, кнопка «Отправить ссылку», сообщение об успехе.

В этой главе разбираем архитектуру (как экраны собраны в `UINavigationController` и как сообщают о событиях наверх), мок-сервис вместо настоящего сервера и **Keychain** — место, где живёт токен. Полный код гейта — в разделе 8.11.

Два слова, которые встретятся на каждой странице:

- **Токен** (token) — строка, которую сервер выдаёт после успешного входа. Это «пропуск»: приложение прикладывает его к каждому запросу, и сервер понимает, кто пришёл, без повторного ввода пароля. Пароль после входа хранить не нужно — хранится только токен.
- **Keychain** (связка ключей) — системное зашифрованное хранилище секретов на устройстве. Пароли Safari, данные Wi-Fi и токены приложений лежат там. Подробно — в 8.3.

8.1 Архитектура: контейнер + три экрана

Auth gate — это **отдельный** `UINavigationController` со своим стеком экранов. `UINavigationController` — контейнер (container view controller), который показывает экраны стопкой: новый экран «кладётся сверху» (push) и выезжает справа, кнопка «Назад» или свайп от левого края снимают его (pop). Сам он ничего не рисует, кроме панели `navigationBar` (`UINavigationBar → UIView`), а экраны держит в массиве `viewControllers` как дочерние контроллеры — тем же механизмом containment, что мы разбирали в главе 6. Заголовок «Вход», который каждый экран задаёт через `title`, панель читает из его `navigationItem` — объекта, который принадлежит экрану, а не панели (глава 0, разделы 0.6 и 0.8). Стек гейта не связан ни с лаунчером, ни с главным экраном mini-app.

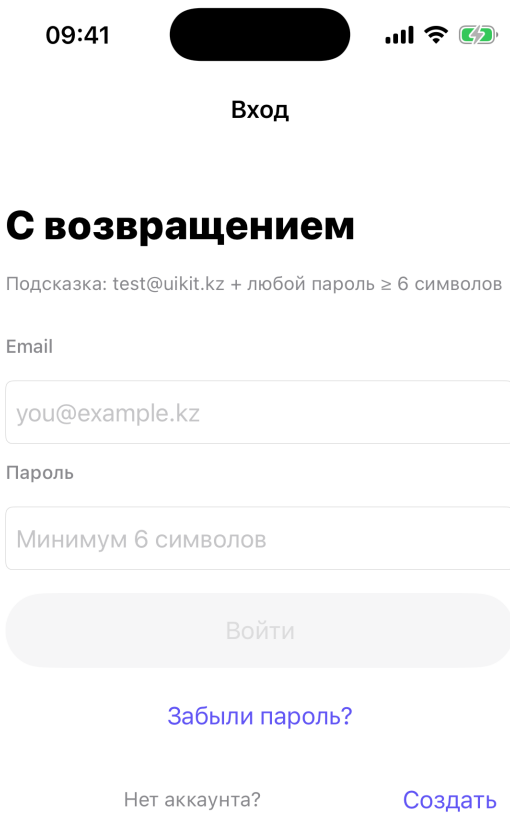


Рис. 5: Login-экран auth-гейта

```swift
final class AuthGateContainerViewController: UINavigationController {

    private let brandColor: UIColor
    private let onAuthSuccess: () -> Void

    init(brandColor: UIColor, onAuthSuccess: @escaping () -> Void) {
        self.brandColor = brandColor
        self.onAuthSuccess = onAuthSuccess
        let login = LoginViewController(brandColor: brandColor)
        super.init(rootViewController: login)
        login.delegate = self
        navigationBar.tintColor = brandColor
        navigationBar.prefersLargeTitles = false
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }
}
```

Разбор:

- Контейнер — наследник `UINavigationController`. `@MainActor` писать не нужно: в нашем режиме (Default Actor Isolation = MainActor, см. введение, раздел 0.1) это подразумевается.
- В `init` сначала заполняем собственные свойства, потом создаём экран логина и отдаём его родителю как корневой (`super.init(rootViewController:)`). Порядок важен: Swift требует инициализировать все свои свойства **до** вызова `super.init`.
- `login.delegate = self` — после `super.init` объект готов, и его уже можно передавать как делегата.
- `required init?(coder:)` — обязательный инициализатор для загрузки из storyboard; мы им не пользуемся.

Почему отдельный стек, а не тот, что вокруг главного экрана? В момент показа гейта главного экрана ещё не существует. Координатор только **решает**, нужен ли вход, и если да — ставит этот контейнер корневым экраном окна. Главный экран создастся позже, **после** успешного входа.

В стеке три экрана:

```
[NavigationController]
        │
        ├─ LoginViewController       ← rootViewController
        │       │
        │       ├─ push → RegisterViewController
        │       │
        │       └─ push → ForgotPasswordViewController
```

Login — корневой. Register и Forgot открываются push'ем, закрываются кнопкой «Назад» или свайпом от левого края.

`prefersLargeTitles = false` — на экранах входа крупные заголовки не нужны, место занимает форма.

`navigationBar.tintColor = brandColor` — кнопки панели навигации окрашены в цвет mini-app. В Profile (индиго) кнопка «Назад» будет индиго. Цвет приходит из манифеста (см. главу 2).

8.2 Делегирование событий — почему `delegate` вместо замыканий

Экран логина должен сообщать контейнеру о событиях: «успешно вошёл», «хочу на регистрацию», «хочу на восстановление». Два варианта.

На замыканиях (closure):

```
let login = LoginViewController(brandColor: brandColor)
login.onLoginSuccess = { [weak self] token in self?.didAuthenticate(token: token) }
login.onGoToRegister = { [weak self] in self?.goToRegister(from: login) }
login.onGoToForgot = { [weak self] in self?.goToForgot(from: login) }
```

На делегате (классический стиль UIKit):

```
protocol LoginViewControllerDelegate: AnyObject {
    func login(_ vc: LoginViewController, didSucceedWith token: String)
    func loginRequestsRegister(_ vc: LoginViewController)
    func loginRequestsForgot(_ vc: LoginViewController)
}
```

И в Login:

```
weak var delegate: LoginViewControllerDelegate?
```

Делегат — объект, которому экран поручает реагировать на свои события. Экран знает только протокол, а не конкретный класс: сегодня делегат — наш контейнер, завтра — тестовый объект.

В нашем коде — второй вариант. Не потому что он «правильнее», а потому что:

- **Событий три.** Три замыкания-свойства и три присваивания читаются хуже, чем один протокол, где все события собраны вместе.
- **Меньше мест для утечки памяти.** Каждое замыкание, которое обращается к контейнеру, нужно не забыть написать с `[weak self]`, иначе контейнер и экран будут держать друг друга сильными ссылками и никогда не освободятся. С делегатом `weak` пишется один раз — в объявлении свойства `delegate`. Ограничение `AnyObject` в протоколе нужно, чтобы `weak` вообще было разрешено (слабой бывает только ссылка на класс). Заставить написать `weak` компилятор не может — это остаётся на тебе, но место одно, и его легко проверить. Так же устроены делегаты самого UIKit: у `UITextField.delegate` в заголовке стоит `weak`, и в главе 0 (раздел 0.9) показано, что `tableView.dataSource` обнуляется, как только его владелец освобождён.
- **Стиль UIKit.** Apple использует делегатов повсюду: `UITableViewDelegate`, `UITextFieldDelegate`, `UIPageViewControllerDelegate`. Книга показывает обычный UIKit-код.

  Когда замыкание лучше. Если событие одно (`onFinish` у онбординга) или два (`onPass` / `onTooYoung` у age gate в главе 10), замыкания короче. Делегат начинает выигрывать, когда событий три и больше.

Контейнер реализует оба протокола — и логина, и регистрации:

```
extension AuthGateContainerViewController: LoginViewControllerDelegate,
                                           RegisterViewControllerDelegate {
    func login(_ vc: LoginViewController, didSucceedWith token: String) {
        didAuthenticate(token: token)
    }
    func loginRequestsRegister(_ vc: LoginViewController) {
        goToRegister(from: vc)
    }
    func loginRequestsForgot(_ vc: LoginViewController) {
        goToForgot(from: vc)
    }
    func register(_ vc: RegisterViewController, didSucceedWith token: String) {
        didAuthenticate(token: token)
    }
}
```

Login и Register сообщают об успехе одинаково — «получен токен». Контейнер сохраняет его и сообщает координатору:

```
private func didAuthenticate(token: String) {
    guard AuthStorage.shared.save(token: token) else {
        let alert = UIAlertController(title: "Не удалось сохранить вход",
                                      message: "Попробуй ещё раз.",
                                      preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "OK", style: .default))
        present(alert, animated: true)
        return
    }
    onAuthSuccess()
}
```

Если сохранить токен не удалось (Keychain вернул ошибку), молча идти дальше нельзя: при новом запуске приложение «забудет» вход. Лучше сразу сказать об этом человеку.

8.3 Keychain — где жить токену

Наш `AuthStorage` умеет три вещи: сохранить, прочитать, стереть. Базовый Keychain без групп доступа, без синхронизации через iCloud и без биометрии — просто защищённое хранилище на устройстве.

```
import Foundation
import Security

final class AuthStorage {

    static let shared = AuthStorage()
    private init() {}
```

```
    private let service = "kz.waid.uikitplayground.auth"
    private let account = "token"

    var token: String? { read() }
    var isLoggedIn: Bool { token != nil }

    private var baseQuery: [String: Any] {
        [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrService as String: service,
            kSecAttrAccount as String: account,
        ]
    }

    @discardableResult
    func save(token: String) -> Bool {
        let data = Data(token.utf8)
        var addQuery = baseQuery
        addQuery[kSecValueData as String] = data
        addQuery[kSecAttrAccessible as String] =
↪   kSecAttrAccessibleWhenUnlockedThisDeviceOnly

        let status = SecItemAdd(addQuery as CFDictionary, nil)
        if status == errSecDuplicateItem {
            // Запись уже есть — меняем только данные.
            let changes: [String: Any] = [kSecValueData as String: data]
            return SecItemUpdate(baseQuery as CFDictionary, changes as CFDictionary) ==
             ↪  errSecSuccess
        }
        return status == errSecSuccess
    }

    func read() -> String? {
        var query = baseQuery
        query[kSecReturnData as String] = true
        query[kSecMatchLimit as String] = kSecMatchLimitOne

        var result: CFTypeRef?
        let status = SecItemCopyMatching(query as CFDictionary, &result)
        guard status == errSecSuccess,
              let data = result as? Data else { return nil }
        return String(data: data, encoding: .utf8)
    }

    func clear() {
        SecItemDelete(baseQuery as CFDictionary)
    }
```

```
}
```

Разберём по частям.

Почему не UserDefaults. `UserDefaults` — это обычный plist-файл в песочнице приложения, данные лежат в нём открытым текстом. Он попадает в резервные копии, а на взломанном (jailbreak) устройстве его может прочитать кто угодно. Для флага «онбординг пройден» это неважно, для токена — недопустимо: с токеном можно войти в аккаунт без пароля. Apple в HIG прямо советует хранить чувствительные данные в Keychain.

Что такое Keychain. Это база записей, которой управляет система, а не приложение. Записи зашифрованы ключами, которые привязаны к устройству и к его код-паролю; приложение видит только свои записи. Каждая запись — словарь атрибутов плюс данные.

Запрос — словарь. API Keychain — функции на языке C из фреймворка Security. Параметры передаются словарём `[String: Any]`, который приводится к `CFDictionary`. Ключи словаря — константы вроде `kSecClass`, значения — другие константы или `Data`. Привыкнуть можно, но в больших проектах часто берут библиотеку-обёртку (KeychainAccess, SimpleKeychain) или пишут свою, как наш `AuthStorage`.

`baseQuery` — как найти нашу запись. Три атрибута:

- `kSecClass: kSecClassGenericPassword` — тип записи «произвольный секрет». Есть ещё интернет-пароли, сертификаты и ключи.
- `kSecAttrService` — «имя сервиса». Обычно bundle id приложения плюс назначение: `kz.waid.uikitplayground.auth`.
- `kSecAttrAccount` — «имя слота» внутри сервиса. У нас `token`. Если бы мы хранили два токена (access и refresh), были бы два слота: `access` и `refresh`.

Пара `service + account` однозначно определяет запись.

`save` — добавить, а если есть, обновить. Так рекомендует Apple в статье «Updating and deleting keychain items»:

1. `SecItemAdd` пытается создать запись. Если её ещё нет — готово, функция вернёт `errSecSuccess`.
2. Если запись уже есть, `SecItemAdd` вернёт `errSecDuplicateItem`. Тогда зовём `SecItemUpdate`: первый словарь — какую запись искать (`baseQuery`), второй — что в ней поменять (новые данные `kSecValueData`).

Встречается и вариант «сначала `SecItemDelete`, потом `SecItemAdd`». Он тоже работает, но между двумя вызовами записи нет совсем, и если второй вызов упадёт, токен потерян.

Каждая функция Keychain возвращает `OSStatus` — числовой код результата. `errSecSuccess` (0) — успех, остальное — ошибка. Наш `save` возвращает `Bool`, чтобы вызывающий код мог отреагировать (как в `didAuthenticate` выше). `@discardableResult` разрешает игнорировать результат без предупреждения компилятора — пригодится в тестах.

`kSecAttrAccessible` — когда запись доступна. Мы ставим `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. Название читается по частям:

- `WhenUnlocked` — прочитать запись можно, только пока устройство разблокировано. Токен нужен, когда человек пользуется приложением, — это наш случай. Если бы прило-

жение обновляло данные в фоне при заблокированном экране, понадобился бы вариант `AfterFirstUnlock` («доступно после первой разблокировки с момента включения»).
- `ThisDeviceOnly` — запись не переедет на другое устройство. Записи Keychain попадают в зашифрованные резервные копии; без этого суффикса при восстановлении копии на **новый** iPhone токен переехал бы вместе с ней. С суффиксом он восстанавливается только на тот же самый аппарат.

Если атрибут не указать, по умолчанию будет `kSecAttrAccessibleWhenUnlocked` — то же, но без запрета переезда. Apple советует выбирать самый строгий вариант, который подходит приложению.

Синхронизация через iCloud Keychain — отдельный атрибут `kSecAttrSynchronizable`; по умолчанию он выключен, и токен остаётся на устройстве.

`read` — поиск записи. К `baseQuery` добавляем `kSecReturnData: true` («верни данные, а не только факт наличия») и `kSecMatchLimitOne` («нужна одна запись»). `SecItemCopyMatching` кладёт результат в `result` типа `CFTypeRef?`; приводим его к `Data` и декодируем строку. Любая ошибка, включая «записи нет» (`errSecItemNotFound`), превращается в `nil` — для «вошёл ли пользователь» этого достаточно.

Главный поток. Вызовы Keychain синхронные: пока система ищет запись, поток ждёт. Для одной маленькой записи это доли миллисекунды, поэтому `AuthStorage` работает на главном потоке, как и весь остальной код. Для записей, защищённых Face ID (глава 11), чтение ждёт, пока человек пройдёт проверку, — такие вызовы уводят с главного потока.

Удаление приложения. На практике записи Keychain в iOS **не** стираются при удалении приложения, в отличие от `UserDefaults`. Apple не обещает это поведение в документации, но на него стоит рассчитывать: человек удалил приложение, поставил заново — и вдруг оказался «залогинен» старым токеном.

Упражнение 8.1. Сделай так, чтобы после переустановки приложение не оказывалось «залогиненным» старым токеном. Подсказка: `UserDefaults` при удалении приложения стирается, а Keychain — нет. Ответ — в конце главы.

Упражнение 8.2. Запусти Profile mini-app и войди с `test@uikit.kz` и любым паролем от 6 символов. Выйди на домашний экран (⇧⌘H в симуляторе), затем полностью закрой приложение: Stop в Xcode или смахни его в переключателе приложений. Запусти снова и зайди в Profile. Что ты увидишь и почему? Ответ — в конце главы.

8.4 MockAuthService — почему без реального бэкенда

Настоящий сервер для книги не нужен: мы пишем про интерфейс и гейты. Мок-сервис (mock — «подделка», объект, который притворяется настоящим сервисом) делает три вещи:

- **Задержка.** `try await Task.sleep(nanoseconds: 600_000_000)` — 600 миллионов наносекунд, то есть 0,6 секунды. Это имитация сети: без задержки кнопка «Войти» срабатывала бы мгновенно, и ты не увидел бы спиннер. Более удобная запись `Task.sleep(for: .milliseconds(600))` появилась только в iOS 16, а мы поддерживаем iOS 15.
- **Проверка данных.** Email — регулярным выражением, пароль — от 6 символов, для входа подходит только учётка `test@uikit.kz` (регистрация принимает любой адрес).
- **Ошибки** — свой enum с понятными человеку сообщениями.

```
import Foundation

enum MockAuthService {
    enum Error: Swift.Error, LocalizedError {
        case invalidCredentials
        case weakPassword
        case invalidEmail
        case network

        var errorDescription: String? {
            switch self {
            case .invalidCredentials: "Неверный email или пароль."
            case .weakPassword: "Пароль должен быть не короче 6 символов."
            case .invalidEmail: "Проверь адрес почты: похоже, в нём опечатка."
            case .network: "Не удалось подключиться к серверу."
            }
        }
    }

    static func isValidEmail(_ email: String) -> Bool {
        let pattern = #"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"#
        return email.range(of: pattern, options: .regularExpression) != nil
    }

    static func login(email: String, password: String) async throws -> String {
        try await Task.sleep(nanoseconds: 600_000_000)
        guard isValidEmail(email) else { throw Error.invalidEmail }
        guard email.lowercased() == "test@uikit.kz", password.count >= 6 else {
            throw Error.invalidCredentials
        }
        return "mock-token-\(UUID().uuidString)"
    }

    static func register(email: String, password: String) async throws -> String {
        try await Task.sleep(nanoseconds: 600_000_000)
        guard isValidEmail(email) else { throw Error.invalidEmail }
        guard password.count >= 6 else { throw Error.weakPassword }
        return "mock-token-\(UUID().uuidString)"
    }

    static func sendResetEmail(email: String) async throws {
        try await Task.sleep(nanoseconds: 600_000_000)
        guard isValidEmail(email) else { throw Error.invalidEmail }
    }
}
```

Что здесь стоит разобрать:

- `enum Error: Swift.Error` — вложенный тип называется `Error`, как стандартный протокол. Чтобы не перепутать, протокол указан полным именем `Swift.Error`.
- `LocalizedError` — протокол с одним главным свойством `errorDescription`. Реализуешь его — и стандартное `error.localizedDescription` вернёт твою строку, а не безликое «The operation couldn't be completed».
- `switch` без `return` в каждой ветке — с Swift 5.9 `switch` может быть выражением, и значение ветки становится результатом.
- При **входе** неверный пароль и неизвестный email дают **одну и ту же** ошибку `invalidCredentials`. Это сознательно. Если на «нет такого email» отвечать иначе, чем на «неверный пароль», злоумышленник может перебором выяснить, какие адреса зарегистрированы. Короткий пароль при входе — тоже просто «неверный пароль»: требования к длине проверяются при регистрации, а не при входе.
- Токен — `"mock-token-"` плюс случайный UUID, каждый вход даёт новый.

Удобство `LocalizedError` для интерфейса — любую ошибку можно сразу показать человеку:

```
do {
    let token = try await MockAuthService.login(email: email, password: password)
    // ...
} catch {
    showError(error.localizedDescription)  // наша строка
}
```

> **Регулярка для email.** Выражение `^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$` читается так: «одна или больше латинских букв, цифр или символов `._%+-`, затем `@`, затем имя домена, точка и зона из двух и более букв». Оно намеренно простое: адрес `ivan.petrov+shop@mail.kz` пройдёт, а `ivan@mail` — нет. Полная проверка по стандарту RFC 5322 огромна и всё равно не гарантирует, что ящик существует. Настоящая проверка email — письмо со ссылкой подтверждения. Имей в виду, что регулярка не пропустит адреса с кириллицей в имени (`иван@почта.рф`); если они нужны, условие придётся ослабить.
>
> `#"..."#` — «сырая» строка: обратный слеш в ней не нужно удваивать, и `\.` попадает в регулярку как есть.

8.5 Login — что внутри

Экран логина — `UIScrollView`, внутри вертикальный `UIStackView` с полями и кнопками. Прокрутка нужна из-за клавиатуры: она занимает нижнюю треть экрана, и поля под ней надо иметь возможность подвинуть вверх. Сам по себе scroll view о клавиатуре ничего не знает: прокручивать он даёт ровно настолько, насколько содержимое больше его рамки, и если форма помещалась на экран целиком, под клавиатурой она просто останется закрытой. Поэтому нижний край scroll view мы привязываем не к низу экрана, а к `view.keyboardLayoutGuide.topAnchor` (iOS 15+) — это layout guide, который следит за клавиатурой. Пока клавиатуры нет, его верх совпадает с низом safe area (так сказано в заголовке `UIKeyboardLayoutGuide.h`); когда клавиатура выезжает, guide поднимается вместе с ней, рамка scroll view укорачивается, и содержимое снова можно прокрутить до последнего поля. До iOS 15 то же делали вручную: подписывались на `UIResponder.keyboardWillChangeFrameNotification` и по рамке клавиатуры из `userInfo` меняли `contentInset` scroll view. `keyboardDismissMode = .interactive` — клавиатура уезжает вниз вслед

за пальцем, когда прокручиваешь форму (по умолчанию `.none`: прокрутка клавиатуру не трогает).

Ключевые куски.

Поля с автозаполнением паролей. Поля создаёт общая функция:

```
func makeAuthField(placeholder: String, contentType: UITextContentType, secure: Bool) ->
↪  UITextField {
    let field = UITextField()
    field.placeholder = placeholder
    field.borderStyle = .roundedRect
    field.font = .preferredFont(forTextStyle: .body)
    field.adjustsFontForContentSizeCategory = true
    field.textContentType = contentType
    field.isSecureTextEntry = secure
    field.autocapitalizationType = .none
    field.autocorrectionType = .no
    if !secure {
        field.keyboardType = .emailAddress
    }
    field.heightAnchor.constraint(greaterThanOrEqualToConstant: 44).isActive = true
    return field
}
```

Самая важная строка — `textContentType`. Она говорит системе, что это за поле: `.username` — логин, `.password` — существующий пароль, `.newPassword` — пароль при регистрации. По этим подсказкам iOS включает **Password AutoFill**: над клавиатурой появляется предложение подставить сохранённые логин и пароль, а на экране регистрации — предложение сгенерировать надёжный пароль и сохранить его в связку ключей. Человеку не нужно ничего запоминать, а тебе — писать ни строчки кода хранения пароля. (Чтобы iOS предлагала пароль именно от твоего сайта, настраивают Associated Domains — связку приложения с доменом; это тема за пределами главы.)

Остальное:

- `isSecureTextEntry = true` — символы пароля скрываются точками, текст из поля нельзя скопировать, а в некоторых случаях система не даёт записать его и при записи экрана (документация говорит осторожно: «in some cases»).
- `autocapitalizationType = .none`, `autocorrectionType = .no` — клавиатура не делает первую букву email заглавной и не «исправляет» адрес по словарю.
- `.emailAddress` — клавиатура с `@` и точкой на виду.
- Высота поля не меньше 44 точек — минимальный размер области касания по HIG, в неё уверенно попадает палец. Свой `intrinsicContentSize` у пустого поля без рамки всего 22 точки в высоту, поэтому без констрейнта поля получились бы низкими.

Проверка на лету — на каждое изменение текста. `UITextField` — контрол (`UITextField → UIControl → UIView`), поэтому о правках он сообщает через target-action, как кнопка о нажатии. События у него свои: `.editingDidBegin` (поле стало first responder), `.editingChanged` (текст изменился), `.editingDidEnd` и `.editingDidEndOnExit` (нажали Return). Для тонкого контроля есть ещё

делегат `UITextFieldDelegate` — например, `textField(_:shouldChangeCharactersIn:replacementString:)` может запретить символ до того, как он попадёт в поле. Нам хватает `.editingChanged`:

```
emailField.addTarget(self, action: #selector(textChanged), for: .editingChanged)
passwordField.addTarget(self, action: #selector(textChanged), for: .editingChanged)

@objc private func textChanged() {
    let emailOk = MockAuthService.isValidEmail(emailField.text ?? "")
    let passOk = (passwordField.text?.count ?? 0) >= 6
    loginButton.isEnabled = emailOk && passOk
    errorLabel.isHidden = true
}
```

Кнопка «Войти» активна, только когда оба поля выглядят правдоподобно. Иначе человек тапает «Войти» с пустым полем и получает ошибку — лишнее раздражение. Внешний вид неактивной кнопки делать не нужно: кнопка на `UIButton.Configuration` сама становится бледной при `isEnabled = false`. Заодно прячем прошлую ошибку — человек уже исправляет ввод.

`.editingChanged` приходит на каждую правку **пользователя**. Когда ты сам присваиваешь `field.text` в коде, событие не рассылается: контролы UIKit шлют действия только в ответ на действия человека (глава 0, раздел 0.4). Поэтому, если поле когда-нибудь будет заполняться кодом, `textChanged()` после этого придётся вызвать руками. В `stopLoading()` ниже мы зовём его по соседней причине: на время запроса кнопка была выключена, и после ответа её состояние надо пересчитать по текущему вводу.

Состояние загрузки:

```
@objc private func loginTapped() {
    let email = emailField.text ?? ""
    let password = passwordField.text ?? ""
    startLoading()
    Task { [weak self] in
        do {
            let token = try await MockAuthService.login(email: email, password: password)
            guard let self else { return }
            self.stopLoading()
            self.delegate?.login(self, didSucceedWith: token)
        } catch {
            guard let self else { return }
            self.stopLoading()
            self.showError(error.localizedDescription)
        }
    }
}

private func startLoading() {
    view.endEditing(true)
    loginButton.isEnabled = false
    var cfg = loginButton.configuration
    cfg?.showsActivityIndicator = true
```

```
    cfg?.title = "Входим…"
    loginButton.configuration = cfg
}
```

Порядок действий:

1. Текст полей читаем **до** запуска задачи, пока мы на главном потоке и поля точно те, что ввёл человек.
2. `startLoading()` прячет клавиатуру. Клавиатура на экране принадлежит **first responder** — тому `UITextField`, в котором сейчас стоит курсор; поле становится first responder, когда человек его тапает, и система сама показывает клавиатуру (глава 0, раздел 0.2). `view.endEditing(true)` ищет среди subview текущий first responder и заставляет его отказаться от статуса (`resignFirstResponder()`), после чего клавиатура уезжает; `true` значит «не спрашивать разрешения у делегата, снимать принудительно». Затем метод выключает кнопку и включает спиннер прямо внутри неё: `UIButton.Configuration` умеет это свойством `showsActivityIndicator` (iOS 15+), отдельный `UIActivityIndicatorView` не нужен.
3. `Task { [weak self] in ... }` запускает асинхронную работу. Внутри — `await` на мок-сервис.
4. `guard let self` стоит **после** `await`, а не в начале задачи. Разница важна: пока идёт запрос, экран держится только слабой ссылкой. Если человек за эти 0,6 секунды ушёл с экрана, контроллер освободится, и `guard` просто выйдет. Если бы `guard let self` стоял первой строкой, задача держала бы экран сильной ссылкой до конца запроса.
5. Задача, созданная в коде главного актора, продолжает работу на главном потоке, поэтому трогать интерфейс после `await` можно.

`stopLoading()` возвращает кнопке заголовок и снова вызывает `textChanged()`, чтобы кнопка стала активной или нет по текущему вводу.

Ошибка под формой:

```
private func showError(_ message: String) {
    errorLabel.text = message
    errorLabel.isHidden = false
    UIAccessibility.post(notification: .announcement, argument: message)
}
```

Красная надпись под полями заметнее alert'а и не требует лишнего тапа. Последняя строка — для VoiceOver, экранного диктора для незрячих: он произнесёт текст ошибки вслух. Без этого человек с VoiceOver не узнал бы, что вход не удался.

8.6 Register — индикатор надёжности пароля

Register — то же самое плюс поле «повтори пароль», галочка «согласен с условиями» и индикатор надёжности пароля.

Индикатор — небольшая функция:

```
enum Strength { case weak, medium, strong }

static func passwordStrength(_ s: String) -> Strength {
```

```
    var score = 0
    if s.count >= 6 { score += 1 }
    if s.count >= 10 { score += 1 }
    if s.contains(where: \.isNumber) { score += 1 }
    if s.contains(where: { $0.isUppercase }) { score += 1 }
    if s.contains(where: { !$0.isLetter && !$0.isNumber }) { score += 1 }
    switch score {
    case 0...1: return .weak
    case 2...3: return .medium
    default: return .strong
    }
}
```

За каждое выполненное условие — балл: длина от 6 символов, длина от 10, есть цифра, есть заглавная буква, есть символ, который не буква и не цифра. 0–1 балл — слабый, 2–3 — средний, 4–5 — хороший. Примеры на числах (проверены запуском):

Пароль	Баллы	Итог
`pass`	0	слабый
`qwerty`	1 (длина от 6)	слабый
`qwerty12`	2 (длина, цифра)	средний
`password123`	3 (длина от 6, от 10, цифра)	средний
`password123Aa`	4 (плюс заглавная)	хороший
`password123!Aa`	5 (плюс !)	хороший

Цветная подпись под полем: красная «Слабый пароль», оранжевая «Средний», зелёная «Хороший».

Таблица заодно показывает слабость такой эвристики: `qwerty12` — один из самых популярных паролей в утечках, а функция называет его средним. Серьёзные оценщики вроде библиотеки zxcvbn от Dropbox сверяют пароль со словарями популярных паролей и раскладок клавиатуры. Современные рекомендации по паролям (например, американский стандарт NIST SP 800-63B) ставят длину и проверку по базам утечек выше обязательных «цифра + заглавная + спецсимвол». Для демонстрации интерфейса нашей функции хватает, для настоящего сервиса окончательную проверку делает сервер.

Для **сгенерированного** системой пароля можно подсказать правила через `passwordRules`:

```
passwordField.passwordRules = UITextInputPasswordRules(
    descriptor: "minlength: 10; required: lower; required: upper; required: digit;"
)
```

Когда iOS предлагает «надёжный пароль» в поле с `.newPassword`, она учтёт эти правила: не короче 10 символов, обязательно строчная, заглавная и цифра.

Галочка на `UIButton`:

```
agreeButton.setImage(UIImage(systemName: "square"), for: .normal)
agreeButton.setImage(UIImage(systemName: "checkmark.square.fill"), for: .selected)
agreeButton.addTarget(self, action: #selector(toggleAgree), for: .touchUpInside)

@objc private func toggleAgree() {
    agreed.toggle()
    agreeButton.isSelected = agreed
    validateForm()
}
```

Ни `UISwitch`, ни `UISegmentedControl` — просто `UIButton` с двумя картинками из SF Symbols: пустой квадрат для обычного состояния (`.normal`) и квадрат с галочкой для выбранного (`.selected`). Сам `UIControl` флаг `isSelected` по тапу **не** переключает — это делаем мы в `toggleAgree`; по документации «most controls don't modify their appearance or behavior when selected». Зато картинку кнопка подбирает сама: `state` — набор флагов (`.highlighted`, `.selected`, `.disabled`; `.normal` — когда ни один не поднят), и `UIButton` при каждой перерисовке берёт изображение для текущего набора. Бонус: VoiceOver обычно озвучивает такую кнопку как «выбрано» — слой доступности UIKit добавляет признак по `isSelected`; проверить это можно только с включённым VoiceOver или в Accessibility Inspector.

`validateForm()` включает кнопку «Создать аккаунт», только если email правдоподобен, пароль от 6 символов, повтор совпадает и галочка стоит.

Упражнение 8.3. Не запуская приложение, посчитай баллы и итог для паролей `Pass1`, `Qwerty12` и `Пароль1!`. Затем открой Register и проверь себя. Совпадает ли с тем, что проверка длины в `validateForm` требует от 6 символов? Ответ — в конце главы.

8.7 Forgot password — минимальный экран

Самый простой экран гейта: email, кнопка «Отправить ссылку» и сообщение.

```
@objc private func sendTapped() {
    let email = emailField.text ?? ""
    view.endEditing(true)
    var cfg = sendButton.configuration
    cfg?.showsActivityIndicator = true
    sendButton.configuration = cfg
    sendButton.isEnabled = false
    Task { [weak self] in
        try? await MockAuthService.sendResetEmail(email: email)
        guard let self else { return }
        var cfg = self.sendButton.configuration
        cfg?.showsActivityIndicator = false
        self.sendButton.configuration = cfg
        self.sendButton.isEnabled = true
        self.successLabel.isHidden = false
    }
}
```

`try?` превращает любую ошибку в `nil`: результат нам не важен. Текст сообщения — «Если такой адрес зарегистрирован, мы отправили на него письмо». Не «письмо отправлено» и не «такого адреса нет» — по той же причине, что и в 8.4: экран восстановления не должен подсказывать, какие email есть в базе.

В реальности сервер пришлёт письмо со ссылкой, по которой откроется страница с формой нового пароля. Эта часть обычно живёт **вне** приложения, на сайте. Мок просто ждёт 0,6 секунды.

8.8 `shouldShow` — когда auth не нужен

```
static func shouldShow(for manifest: AppManifest) -> Bool {
    manifest.hasAuthGate && !AuthStorage.shared.isLoggedIn
}
```

Два условия:

- mini-app **требует** входа (`hasAuthGate == true` в манифесте);
- пользователь **ещё не вошёл** (в Keychain нет токена).

Если токен есть — гейт пропускается, координатор идёт прямо к главному экрану. Чтобы вернуть гейт, нужно стереть токен. В Profile это делает кнопка «Выйти из аккаунта» — она вызывает `AuthStorage.shared.clear()`.

Заметь: `isLoggedIn` проверяет только **наличие** токена на устройстве, а не то, что сервер его ещё принимает. Токен мог истечь или быть отозван. Настоящее приложение узнаёт об этом при первом запросе к API (ответ 401) — см. 8.10.

8.9 Бытовая аналогия

Auth gate — **стойка регистрации в отеле**. Прежде чем попасть в номер (главный экран mini-app), нужно заселиться. Администратор умеет три вещи: «у меня бронь» (login), «я новый гость» (register), «забыл номер брони» (forgot).

После заселения тебе дают ключ-карту (токен), и ты идёшь в номер. Пароль от брони больше не нужен — дверь открывает карта. Карта работает, пока ты её не сдашь (выход — `clear()`) или пока отель её не заблокирует (сервер отозвал токен). Карту ты носишь во внутреннем кармане на молнии (Keychain), а не в прозрачном бейдже на шее (`UserDefaults`).

Системные запросы разрешений iOS из главы 7 — это **охрана здания**. Она пускает к ресурсам устройства (фото, камера, геолокация), но не в твой номер. Разные уровни доступа: auth gate — про **аккаунт**, разрешения — про **устройство**.

8.10 Что мы не делаем (но в production стоит)

- **Refresh token.** У мока один токен на сессию. В production их обычно два: короткоживущий access (минуты-часы) и долгоживущий refresh. Когда access истёк, приложение отправляет refresh и получает новый access, не спрашивая пароль.

- **Реакция на 401.** Если сервер во время работы ответил «401 Unauthorized» (токен истёк или отозван), нужно стереть токен из Keychain, показать «Сессия истекла» и вернуть человека на вход.
- **Вход по коду из SMS или письма** — человек вводит телефон или email и получает одноразовый код. Для поля кода есть `textContentType = .oneTimeCode`: iOS сама подставит код из пришедшего сообщения.
- **Passkeys.** Вход без пароля вообще: ключ хранится в связке ключей и подтверждается Face ID. HIG советует passkeys, если ты не используешь Sign in with Apple.
- **Вход через сторонние сервисы** (Google, Telegram и т.д.). Здесь есть правило App Review **4.8 Login Services**: если основной аккаунт в приложении создаётся через сторонний сервис входа, приложение обязано предложить ещё один равноценный вариант, который собирает только имя и email, позволяет скрыть email и не отслеживает человека для рекламы без согласия. Sign in with Apple этим требованиям отвечает, поэтому его обычно и добавляют. Если же вход только через твою собственную систему аккаунтов (как у нас: email + пароль), дополнительный вариант не требуется.
- **Удаление аккаунта.** Правило **5.1.1(v)**: если в приложении можно создать аккаунт, в нём же должна быть возможность этот аккаунт удалить — не просто деактивировать. Подробно — в главе 40.
- **Биометрия.** Face ID вместо ввода пароля каждый раз. Мы делаем биометрию для **возврата из фона** в главе 11, там же — как привязать к Face ID сам токен в Keychain.

Всё это строится поверх той же базы: хранилище + три экрана + делегаты.

8.11 Гейт целиком

`AuthStorage` и `MockAuthService` целиком приведены в 8.3 и 8.4. Ниже — файл `AuthGate.swift` со всеми экранами. Он собирается и запускается вместе с ними; `AppManifest` — из главы 2.

```
import UIKit

protocol LoginViewControllerDelegate: AnyObject {
    func login(_ vc: LoginViewController, didSucceedWith token: String)
    func loginRequestsRegister(_ vc: LoginViewController)
    func loginRequestsForgot(_ vc: LoginViewController)
}

protocol RegisterViewControllerDelegate: AnyObject {
    func register(_ vc: RegisterViewController, didSucceedWith token: String)
}

final class AuthGateContainerViewController: UINavigationController {

    static func shouldShow(for manifest: AppManifest) -> Bool {
        manifest.hasAuthGate && !AuthStorage.shared.isLoggedIn
    }

    private let brandColor: UIColor
    private let onAuthSuccess: () -> Void
```

```
    init(brandColor: UIColor, onAuthSuccess: @escaping () -> Void) {
        self.brandColor = brandColor
        self.onAuthSuccess = onAuthSuccess
        let login = LoginViewController(brandColor: brandColor)
        super.init(rootViewController: login)
        login.delegate = self
        navigationBar.tintColor = brandColor
        navigationBar.prefersLargeTitles = false
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private func goToRegister(from vc: UIViewController) {
        let register = RegisterViewController(brandColor: brandColor)
        register.delegate = self
        pushViewController(register, animated: true)
    }

    private func goToForgot(from vc: UIViewController) {
        pushViewController(ForgotPasswordViewController(brandColor: brandColor),
↪   animated: true)
    }

    private func didAuthenticate(token: String) {
        guard AuthStorage.shared.save(token: token) else {
            let alert = UIAlertController(title: "Не удалось сохранить вход",
                                          message: "Попробуй ещё раз.",
                                          preferredStyle: .alert)
            alert.addAction(UIAlertAction(title: "OK", style: .default))
            present(alert, animated: true)
            return
        }
        onAuthSuccess()
    }
}

extension AuthGateContainerViewController: LoginViewControllerDelegate,
                                           RegisterViewControllerDelegate {
    func login(_ vc: LoginViewController, didSucceedWith token: String) {
        didAuthenticate(token: token)
    }
    func loginRequestsRegister(_ vc: LoginViewController) {
        goToRegister(from: vc)
    }
    func loginRequestsForgot(_ vc: LoginViewController) {
        goToForgot(from: vc)
    }
```

```
    func register(_ vc: RegisterViewController, didSucceedWith token: String) {
        didAuthenticate(token: token)
    }
}

func makeAuthField(placeholder: String, contentType: UITextContentType, secure: Bool) ->
↪  UITextField {
    let field = UITextField()
    field.placeholder = placeholder
    field.borderStyle = .roundedRect
    field.font = .preferredFont(forTextStyle: .body)
    field.adjustsFontForContentSizeCategory = true
    field.textContentType = contentType
    field.isSecureTextEntry = secure
    field.autocapitalizationType = .none
    field.autocorrectionType = .no
    if !secure {
        field.keyboardType = .emailAddress
    }
    field.heightAnchor.constraint(greaterThanOrEqualToConstant: 44).isActive = true
    return field
}

func makeScrollingForm(in view: UIView, arrangedSubviews: [UIView]) -> UIScrollView {
    let scroll = UIScrollView()
    scroll.keyboardDismissMode = .interactive
    scroll.alwaysBounceVertical = true
    let stack = UIStackView(arrangedSubviews: arrangedSubviews)
    stack.axis = .vertical
    stack.spacing = 12
    scroll.translatesAutoresizingMaskIntoConstraints = false
    stack.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(scroll)
    scroll.addSubview(stack)
    NSLayoutConstraint.activate([
        scroll.topAnchor.constraint(equalTo: view.topAnchor),
        scroll.leadingAnchor.constraint(equalTo: view.leadingAnchor),
        scroll.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        scroll.bottomAnchor.constraint(equalTo: view.keyboardLayoutGuide.topAnchor),

        stack.topAnchor.constraint(equalTo: scroll.contentLayoutGuide.topAnchor,
↪  constant: 32),
        stack.bottomAnchor.constraint(equalTo: scroll.contentLayoutGuide.bottomAnchor,
↪  constant: -32),
        stack.leadingAnchor.constraint(equalTo: scroll.frameLayoutGuide.leadingAnchor,
↪  constant: 24),
        stack.trailingAnchor.constraint(equalTo: scroll.frameLayoutGuide.trailingAnchor,
↪  constant: -24),
```

```
    ])
    return scroll
}

final class LoginViewController: UIViewController {

    weak var delegate: LoginViewControllerDelegate?
    private let brandColor: UIColor

    private let emailField = makeAuthField(placeholder: "Email", contentType: .username,
    ↪  secure: false)
    private let passwordField = makeAuthField(placeholder: "Пароль", contentType:
    ↪  .password, secure: true)
    private let loginButton = UIButton(configuration: .filled())
    private let errorLabel = UILabel()

    init(brandColor: UIColor) {
        self.brandColor = brandColor
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Вход"
        view.backgroundColor = .systemBackground

        var cfg = UIButton.Configuration.filled()
        cfg.title = "Войти"
        cfg.baseBackgroundColor = brandColor
        cfg.cornerStyle = .large
        loginButton.configuration = cfg
        loginButton.isEnabled = false
        loginButton.addTarget(self, action: #selector(loginTapped), for: .touchUpInside)

        errorLabel.textColor = .systemRed
        errorLabel.font = .preferredFont(forTextStyle: .footnote)
        errorLabel.adjustsFontForContentSizeCategory = true
        errorLabel.numberOfLines = 0
        errorLabel.isHidden = true

        let forgot = UIButton(type: .system)
        forgot.setTitle("Забыли пароль?", for: .normal)
        forgot.addTarget(self, action: #selector(forgotTapped), for: .touchUpInside)
        let register = UIButton(type: .system)
        register.setTitle("Создать аккаунт", for: .normal)
        register.addTarget(self, action: #selector(registerTapped), for: .touchUpInside)
```

```
        emailField.addTarget(self, action: #selector(textChanged), for: .editingChanged)
        passwordField.addTarget(self, action: #selector(textChanged), for:
↪   .editingChanged)

        _ = makeScrollingForm(in: view, arrangedSubviews: [
            emailField, passwordField, errorLabel, loginButton, forgot, register,
        ])
    }

    @objc private func textChanged() {
        let emailOk = MockAuthService.isValidEmail(emailField.text ?? "")
        let passOk = (passwordField.text?.count ?? 0) >= 6
        loginButton.isEnabled = emailOk && passOk
        errorLabel.isHidden = true
    }

    @objc private func loginTapped() {
        let email = emailField.text ?? ""
        let password = passwordField.text ?? ""
        startLoading()
        Task { [weak self] in
            do {
                let token = try await MockAuthService.login(email: email, password:
                ↪  password)
                guard let self else { return }
                self.stopLoading()
                self.delegate?.login(self, didSucceedWith: token)
            } catch {
                guard let self else { return }
                self.stopLoading()
                self.showError(error.localizedDescription)
            }
        }
    }

    private func startLoading() {
        view.endEditing(true)
        loginButton.isEnabled = false
        var cfg = loginButton.configuration
        cfg?.showsActivityIndicator = true
        cfg?.title = "Входим…"
        loginButton.configuration = cfg
    }

    private func stopLoading() {
        var cfg = loginButton.configuration
        cfg?.showsActivityIndicator = false
```

```
        cfg?.title = "Войти"
        loginButton.configuration = cfg
        textChanged()
    }

    private func showError(_ message: String) {
        errorLabel.text = message
        errorLabel.isHidden = false
        UIAccessibility.post(notification: .announcement, argument: message)
    }

    @objc private func forgotTapped() { delegate?.loginRequestsForgot(self) }
    @objc private func registerTapped() { delegate?.loginRequestsRegister(self) }
}

final class RegisterViewController: UIViewController {

    enum Strength { case weak, medium, strong }

    static func passwordStrength(_ s: String) -> Strength {
        var score = 0
        if s.count >= 6 { score += 1 }
        if s.count >= 10 { score += 1 }
        if s.contains(where: \.isNumber) { score += 1 }
        if s.contains(where: { $0.isUppercase }) { score += 1 }
        if s.contains(where: { !$0.isLetter && !$0.isNumber }) { score += 1 }
        switch score {
        case 0...1: return .weak
        case 2...3: return .medium
        default: return .strong
        }
    }

    weak var delegate: RegisterViewControllerDelegate?
    private let brandColor: UIColor
    private var agreed = false

    private let emailField = makeAuthField(placeholder: "Email", contentType: .username,
    ↪  secure: false)
    private let passwordField = makeAuthField(placeholder: "Пароль", contentType:
    ↪  .newPassword, secure: true)
    private let confirmField = makeAuthField(placeholder: "Повтори пароль", contentType:
    ↪  .newPassword, secure: true)
    private let strengthLabel = UILabel()
    private let agreeButton = UIButton(type: .system)
    private let submitButton = UIButton(configuration: .filled())

    init(brandColor: UIColor) {
```

```
        self.brandColor = brandColor
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Регистрация"
        view.backgroundColor = .systemBackground

        passwordField.passwordRules = UITextInputPasswordRules(
            descriptor: "minlength: 10; required: lower; required: upper; required: digit;"
        )

        strengthLabel.font = .preferredFont(forTextStyle: .footnote)
        strengthLabel.adjustsFontForContentSizeCategory = true
        strengthLabel.isHidden = true

        agreeButton.setImage(UIImage(systemName: "square"), for: .normal)
        agreeButton.setImage(UIImage(systemName: "checkmark.square.fill"), for: .selected)
        agreeButton.setTitle("  Согласен с условиями", for: .normal)
        agreeButton.contentHorizontalAlignment = .leading
        agreeButton.addTarget(self, action: #selector(toggleAgree), for: .touchUpInside)

        var cfg = UIButton.Configuration.filled()
        cfg.title = "Создать аккаунт"
        cfg.baseBackgroundColor = brandColor
        submitButton.configuration = cfg
        submitButton.isEnabled = false
        submitButton.addTarget(self, action: #selector(submitTapped), for: .touchUpInside)

        [emailField, passwordField, confirmField].forEach {
            $0.addTarget(self, action: #selector(validateForm), for: .editingChanged)
        }

        _ = makeScrollingForm(in: view, arrangedSubviews: [
            emailField, passwordField, strengthLabel, confirmField, agreeButton,
↪   submitButton,
        ])
    }

    @objc private func toggleAgree() {
        agreed.toggle()
        agreeButton.isSelected = agreed
        validateForm()
    }
```

```
    @objc private func validateForm() {
        let password = passwordField.text ?? ""
        switch Self.passwordStrength(password) {
        case .weak:
            strengthLabel.text = "Слабый пароль"
            strengthLabel.textColor = .systemRed
        case .medium:
            strengthLabel.text = "Средний пароль"
            strengthLabel.textColor = .systemOrange
        case .strong:
            strengthLabel.text = "Хороший пароль"
            strengthLabel.textColor = .systemGreen
        }
        strengthLabel.isHidden = password.isEmpty
        submitButton.isEnabled = MockAuthService.isValidEmail(emailField.text ?? "")
            && password.count >= 6
            && password == confirmField.text
            && agreed
    }

    @objc private func submitTapped() {
        let email = emailField.text ?? ""
        let password = passwordField.text ?? ""
        view.endEditing(true)
        submitButton.isEnabled = false
        Task { [weak self] in
            do {
                let token = try await MockAuthService.register(email: email, password:
                 ↪  password)
                guard let self else { return }
                self.delegate?.register(self, didSucceedWith: token)
            } catch {
                self?.validateForm()
            }
        }
    }
}

final class ForgotPasswordViewController: UIViewController {
    private let brandColor: UIColor
    private let emailField = makeAuthField(placeholder: "Email", contentType: .username,
     ↪  secure: false)
    private let sendButton = UIButton(configuration: .filled())
    private let successLabel = UILabel()

    init(brandColor: UIColor) {
        self.brandColor = brandColor
        super.init(nibName: nil, bundle: nil)
```

```
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Восстановление"
        view.backgroundColor = .systemBackground
        var cfg = UIButton.Configuration.filled()
        cfg.title = "Отправить ссылку"
        cfg.baseBackgroundColor = brandColor
        sendButton.configuration = cfg
        sendButton.addTarget(self, action: #selector(sendTapped), for: .touchUpInside)
        successLabel.text = "Если такой адрес зарегистрирован, мы отправили на него
↪   письмо."
        successLabel.numberOfLines = 0
        successLabel.isHidden = true
        _ = makeScrollingForm(in: view, arrangedSubviews: [emailField, sendButton,
↪   successLabel])
    }

    @objc private func sendTapped() {
        let email = emailField.text ?? ""
        view.endEditing(true)
        var cfg = sendButton.configuration
        cfg?.showsActivityIndicator = true
        sendButton.configuration = cfg
        sendButton.isEnabled = false
        Task { [weak self] in
            try? await MockAuthService.sendResetEmail(email: email)
            guard let self else { return }
            var cfg = self.sendButton.configuration
            cfg?.showsActivityIndicator = false
            self.sendButton.configuration = cfg
            self.sendButton.isEnabled = true
            self.successLabel.isHidden = false
        }
    }
}
```

Про `makeScrollingForm` — общую функцию раскладки формы — стоит сказать отдельно. У `UIScrollView` два «прямоугольника», и оба — layout guide, а не view (глава 0, раздел 0.5):

- `frameLayoutGuide` — рамка самой прокрутки на экране;
- `contentLayoutGuide` — всё прокручиваемое содержимое; его размер и становится `contentSize`.

Стек привязан боками к **рамке** (ширина формы = ширина экрана минус по 24 точки с каждой стороны), а верхом и низом — к **содержимому**. Поэтому высота содержимого равна высоте

стека плюс по 32 точки сверху и снизу, а по горизонтали к `contentLayoutGuide` не привязано ничего — ширина содержимого остаётся нулевой, и горизонтальная прокрутка не включается. Мы проверили это на симуляторе: при трёх надписях в стеке высотой 85 точек `contentSize` получился (0, 149) — 85 + 32 + 32. Нижний край scroll view — `keyboardLayoutGuide.topAnchor` (8.5): когда клавиатура выезжает, рамка укорачивается, содержимое становится выше рамки, и форму можно прокрутить до последнего поля. То же происходит и без клавиатуры при крупном шрифте.

Ответы к упражнениям

8.1. Решение — флаг первого запуска в `UserDefaults`, который, в отличие от Keychain, при удалении стирается:

```
func clearKeychainOnFirstLaunch() {
    let key = "app.hasLaunchedBefore"
    guard !UserDefaults.standard.bool(forKey: key) else { return }
    AuthStorage.shared.clear()
    UserDefaults.standard.set(true, forKey: key)
}
```

Вызывай его в самом начале, до первого `shouldShow` — например, в `scene(_:willConnectTo:options:)`. Проверка: войди, удали приложение с симулятора, поставь заново — должен показаться экран входа.

8.2. Profile откроется сразу, без экрана входа. Токен лежит в Keychain, а Keychain переживает перезапуск приложения: `AuthGateContainerViewController.shouldShow(for:)` нашла токен и вернула `false`. Чтобы снова увидеть вход, нажми «Выйти из аккаунта» в Profile.

8.3. `Pass1` — 5 символов: длина от 6 не набрана, от 10 тоже; есть цифра (+1) и заглавная (+1) — 2 балла, «средний». `Qwerty12` — длина от 6 (+1), цифра (+1), заглавная (+1) — 3 балла, «средний». `Пароль1!` — длина от 6 (+1), цифра (+1), заглавная `П` (+1), `!` (+1) — 4 балла, «хороший»: `isUppercase` и `isLetter` понимают кириллицу. Нестыковка есть: `Pass1` индикатор называет средним, а кнопка «Создать аккаунт» остаётся неактивной, потому что паролю меньше 6 символов. Честнее считать «слабым» любой пароль короче минимума — например, первой строкой функции: `guard s.count >= 6 else { return .weak }`.

Что мы выучили

- Auth gate — отдельный `UINavigationController` с тремя экранами: Login, Register, ForgotPassword.
- Login → Register / Forgot — обычный `pushViewController`; кнопка и свайп «Назад» работают из коробки.
- Делегат-протокол против замыканий: при трёх и более событиях протокол читается лучше, а `weak` пишется один раз.
- Токен — в Keychain, не в `UserDefaults`. Запрос к Keychain — словарь атрибутов; `service` + `account` определяют запись.
- Сохранение: `SecItemAdd`, а при `errSecDuplicateItem` — `SecItemUpdate`. Каждый `OSStatus` проверяем.

- `kSecAttrAccessibleWhenUnlockedThisDeviceOnly` — токен доступен только при разблокированном устройстве и не переезжает на другой iPhone.
- Записи Keychain на практике переживают удаление приложения — нужен флаг первого запуска.
- `textContentType` (`.username`, `.password`, `.newPassword`) включает Password AutoFill и генерацию паролей.
- `UITextField` — контрол: о правках сообщает через `.editingChanged` и другие `.editing…`-события; программное присваивание `text` событий не шлёт. Тонкий контроль — через `UITextFieldDelegate`.
- Клавиатура принадлежит first responder; `view.endEditing(true)` снимает этот статус с поля, и клавиатура уезжает.
- Форма в `UIScrollView`: бока — к `frameLayoutGuide`, верх и низ — к `contentLayoutGuide`, низ scroll view — к `view.keyboardLayoutGuide.topAnchor` (iOS 15+), иначе клавиатура закроет поля, а прокрутить к ним будет нельзя.
- `UIButton.Configuration.showsActivityIndicator = true` — спиннер внутри кнопки.
- `LocalizedError` даёт человекочитаемый `localizedDescription`.
- Ошибки входа и восстановления не должны подсказывать, какие email зарегистрированы.
- Правила App Review: 4.8 (сторонний вход требует равноценной приватной альтернативы, например Sign in with Apple) и 5.1.1(v) (создание аккаунта в приложении → удаление аккаунта тоже в приложении).

Apple Developer Documentation

- Human Interface Guidelines — Managing accounts — просить вход только когда без него нельзя, откладывать его как можно дольше, предпочитать passkeys или Sign in with Apple; правила удаления аккаунта.
- App Review Guidelines — пункты 4.8 Login Services и 5.1.1(v) Account Sign-In.
- `UINavigationController` — стек гейта: login как корневой экран, register/forgot — `pushViewController` со штатным жестом «назад».
- `UIButton.Configuration` — `showsActivityIndicator = true` даёт встроенный спиннер без отдельного `UIActivityIndicatorView`.
- `UITextContentType` — подсказки полям для автозаполнения: `.username`, `.password`, `.newPassword`, `.oneTimeCode`.
- `UITextField` — контрол для ввода текста: события `.editing…`, делегат, клавиатура и first responder.
- `UIView.endEditing(_:)` — найти в иерархии first responder и снять с него статус; так мы прячем клавиатуру перед запросом.
- `UIKeyboardLayoutGuide` — layout guide, который следует за клавиатурой (iOS 15+); к его `topAnchor` привязан низ scroll view формы.
- `UIResponder.keyboardWillChangeFrameNotification` — старый способ узнать рамку клавиатуры и подвинуть `contentInset`, если нужно поддерживать iOS младше 15.
- `LocalizedError` — протокол, благодаря которому `error.localizedDescription` возвращает нашу строку.
- Keychain Services — `SecItemAdd` / `SecItemUpdate` / `SecItemCopyMatching` / `SecItemDelete`; сюда кладём токен, а не в `UserDefaults`.

- Updating and deleting keychain items — почему повторное сохранение делают через `SecItemUpdate`.
- `kSecAttrAccessible` — когда запись доступна; варианты с `ThisDeviceOnly` не переезжают на другое устройство.
- `ASAuthorizationAppleIDProvider` — Sign in with Apple; типичный способ выполнить правило 4.8, если в приложении есть вход через сторонние сервисы.
- `URLSession` — когда мок заменится настоящим сервером, отправная точка — `data(for:)` с async/await.
- The Swift Programming Language — Concurrency — `async`/`await` и `Task`: почему `Task { [weak self] in ... }` и где ставить `guard let self`.

→ Глава 9. Force-update + Maintenance — серверные гейты

Глава 9. Force-update + Maintenance — серверные гейты

Эти два гейта решает **сервер**. Они отвечают на вопрос «можно ли пускать пользователя в приложение прямо сейчас?». Если ответ «нет, обновись» — показываем экран-блокатор с кнопкой «Обновить в App Store». Это **force update** (принудительное обновление). Если ответ «нет, у нас технические работы» — показываем экран «Возвращаемся через 45 минут». Это **maintenance** (режим обслуживания).

От предыдущих гейтов (онбординг, разрешения, вход) эти отличаются принципиально: они **асинхронные**. Чтобы решить, нужны ли они, мы делаем сетевой запрос и ждём ответа. Пока запрос идёт, показываем маленький экран загрузки.

Полный код всех экранов главы собран в разделе 9.11.

9.1 Зачем force-update вообще

Сервер и приложение договариваются о формате данных — это **API** (программный интерфейс): какие адреса (endpoint'ы) у сервера, какие поля в запросах и ответах. Иногда формат приходится менять так, что старые версии приложения перестают его понимать. Такое изменение называют **breaking change** («ломающее изменение»): например, поле `price` было числом, а стало объектом `{ amount, currency }`.

Варианты, что делать со старыми версиями приложения:

1. **Поддерживать старый и новый формат одновременно.** Дорого: каждый endpoint живёт в двух версиях, бэкенд обрастает ветками «если клиент старый».
2. **Закрыть старый формат** и ждать, пока все обновятся сами. Не у всех включены автообновления; у кого нет — приложение начнёт показывать ошибки или вообще падать на непонятном ответе.
3. **Force update.** При запуске приложение спрашивает у сервера «удалённую конфигурацию» (remote config) и получает: «версии ниже 2.0 не поддерживаются». Старое приложение показывает блокатор, человек обновляется и продолжает работать.

Force-update гейт — последний рубеж между ломающим изменением на сервере и сломанным приложением у пользователей. Встроить его стоит **в первую же версию**: версии без проверки остановить уже не получится — они о ней не знают.

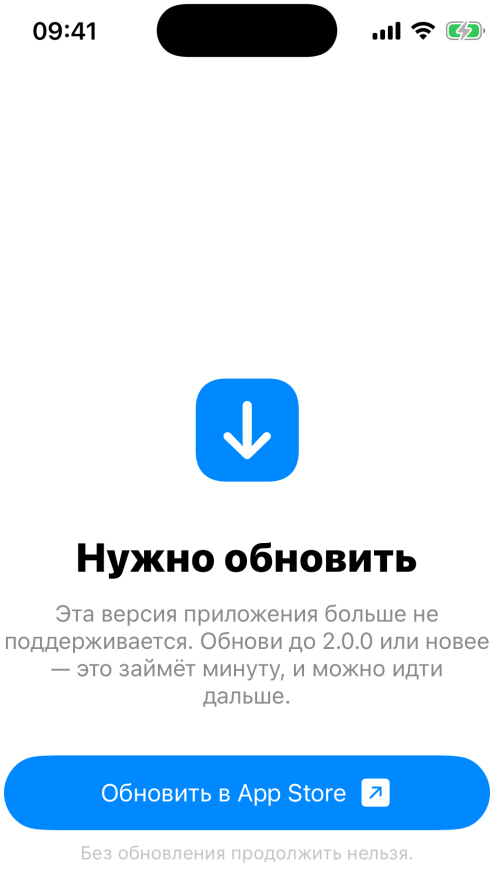


Рис. 6: Force-update гейт

9.2 AppConfigService — мок «remote config»

В реальности конфигурация приходит сетевым запросом: собственный endpoint `/config`, Firebase Remote Config или похожий сервис. У нас — мок, который притворяется сервером:

```
enum AppConfigStatus: Sendable {
    case ok
    case forceUpdate(minVersion: String, storeURL: URL)
    case maintenance(message: String, until: Date?)
}

final class AppConfigService {
    static let shared = AppConfigService()
    private init() {}

    private var scenarios: [String: AppConfigStatus] = [:]

    func setScenario(_ status: AppConfigStatus, for manifestId: String) {
        scenarios[manifestId] = status
    }

    func fetch(for manifestId: String) async -> AppConfigStatus {
        try? await Task.sleep(nanoseconds: 400_000_000)
        return scenarios[manifestId] ?? .ok
    }
}
```

`AppConfigStatus` — три варианта ответа: всё в порядке, нужно обновиться, технические работы. Вместе с `forceUpdate` приходят `minVersion` (минимальная поддерживаемая версия, строка вида `"2.0.0"`) и `storeURL` (куда ведёт кнопка). Вместе с `maintenance` — текст `message` и необязательное `until` — когда работы закончатся.

`Sendable` у enum означает «значение можно безопасно передавать между потоками». Для enum из строк, URL и дат Swift и так вывел бы это сам, но явная пометка документирует намерение: ответ сервера рождается в сетевом слое и едет в интерфейс.

`setScenario(_:for:)` нужен для **демонстрации**. Если бы мок отвечал «обновись» всем подряд, playground перестал бы работать. Поэтому сценарии регистрируются для конкретных mini-app по их `id`. В `SceneDelegate` (глава 5) при старте вызывается `AppRegistry.registerDemoConfigScenarios()`:

```
extension AppRegistry {
    static func registerDemoConfigScenarios() {
        let service = AppConfigService.shared
        service.setScenario(
            .forceUpdate(minVersion: "2.0.0",
                         storeURL: URL(string: "https://apps.apple.com/app/id1234567890")!),
            for: "music"
        )
        service.setScenario(
            .maintenance(message: "Обновляем серверы чата. Это ненадолго.",
```

```
                        until: Date().addingTimeInterval(45 * 60)),
            for: "chat"
        )
    }
}
```

Mini-app Music получает «обновись до 2.0.0», Chat — «технические работы ещё 45 минут» (`45 * 60` = 2700 секунд; `addingTimeInterval` прибавляет к текущему моменту секунды). Восклицательный знак после `URL(string:)` здесь допустим: строка написана вручную и заведомо корректна, упасть этот код не может. Для адреса, пришедшего из сети, так делать нельзя. В настоящем приложении этой функции нет вовсе — сценарий выбирает сервер.

`fetch(for:)` асинхронный: ждёт 0,4 секунды (400 миллионов наносекунд), имитируя сеть, и возвращает зарегистрированный сценарий или `.ok`. Сервис, как и весь наш код, работает на главном акторе — это удобно для UIKit, и настоящий сетевой запрос через `URLSession` всё равно выполняется в фоне без нашего участия.

> **Про 0,4 секунды.** Цифра условная — примерно столько отвечает лёгкий запрос по хорошей сети. На медленном мобильном интернете будет дольше, поэтому экран загрузки и кеш (9.10) обязательны.

9.3 Логика в BootCoordinator

В `BootCoordinator` (глава 4) этот гейт стоит **после** региона и возраста, но **до** входа в аккаунт. Это те методы координатора, которые добавляет глава:

```
private var configTask: Task<Void, Never>?

private func proceedAfterAgeGate() {
    if manifest.checksForceUpdate || manifest.hasMaintenanceCheck {
        checkRemoteConfig()
    } else {
        proceedAfterRemoteConfig()
    }
}

private func checkRemoteConfig() {
    let loader = RemoteConfigLoadingViewController(brandColor: manifest.brandColor)
    setRoot(loader, animated: true)
    configTask?.cancel()
    configTask = Task { [weak self, manifestId = manifest.id] in
        let status = await AppConfigService.shared.fetch(for: manifestId)
        guard let self, !Task.isCancelled else { return }
        self.apply(remoteConfig: status)
    }
}
```

Сначала ставим экран загрузки, потом запускаем асинхронный запрос. Когда придёт ответ, `apply(remoteConfig:)` решит, что показать.

Две детали в `checkRemoteConfig` защищают от реального бага. Представь: запрос идёт, а пользователь встряхнул телефон и вышел в лаунчер (глава 3). Лаунчер обнуляет ссылку на координатор, тот должен освободиться. Но если задача держит координатор **сильной** ссылкой, через 0,4 секунды она выполнит `apply` и поставит экран «Нужно обновить» корневым экраном окна — **поверх лаунчера**.

- `[weak self, manifestId = manifest.id]` — задача держит координатор слабой ссылкой, а `id` манифеста копирует заранее, чтобы для запроса координатор вообще не понадобился.
- `guard let self` стоит **после** `await`. Если бы он стоял первой строкой задачи, `self` стал бы сильной ссылкой на всё время ожидания, и слабый захват ничего бы не дал. После `await` проверка ловит случай «координатор уже освобождён».
- `configTask` хранит задачу, а `!Task.isCancelled` — вторая страховка. При выходе в лаунчер координатор отменяет задачу:

```
private func exitToLauncher() {
    configTask?.cancel()
    configTask = nil
    // ... остальное из главы 4 ...
}
```

Отмена в Swift **кооперативная**: `cancel()` не обрывает задачу, а ставит ей флаг «тебя отменили». Наш `Task.sleep` на флаг реагирует и заканчивается раньше, а `guard !Task.isCancelled` не даёт дойти до `apply`. `configTask?.cancel()` в начале `checkRemoteConfig` нужен для кнопки «Проверить ещё раз» (9.6): новая проверка отменяет старую, если та ещё не закончилась.

Экран загрузки без него выглядел бы как «застывший» предыдущий экран на 0,4 секунды, и человек решил бы, что приложение зависло. Со спиннером видно, что работа идёт:

```
final class RemoteConfigLoadingViewController: UIViewController {
    private let brandColor: UIColor

    init(brandColor: UIColor) {
        self.brandColor = brandColor
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        let spinner = UIActivityIndicatorView(style: .medium)
        spinner.color = brandColor
        spinner.startAnimating()
        let label = UILabel()
        label.text = "Проверяем доступность…"
        label.font = .preferredFont(forTextStyle: .footnote)
        label.adjustsFontForContentSizeCategory = true
        label.textColor = .secondaryLabel
        let stack = UIStackView(arrangedSubviews: [spinner, label])
```

```
        stack.axis = .vertical
        stack.spacing = 12
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: view.centerYAnchor),
        ])
    }
}
```

Спиннер и подпись друг под другом, стек выровнен по центру экрана двумя констрейнтами. Больше ничего: пользователь не должен успеть здесь что-то сделать. Цвет спиннера приходит через инициализатор — отсюда свойство `brandColor` и `init(brandColor:)`.

`UIActivityIndicatorView` — обычная view, не контрол: нажимать на неё нечего, и `addTarget` у неё нет (глава 0, раздел 0.3). У неё `hidesWhenStopped` по умолчанию `true` (проверено на симуляторе): после `stopAnimating()` она сама скроется, прятать её вручную не нужно. Нам это не понадобится — экран загрузки целиком заменяется следующим.

9.4 Обработка результата и сравнение версий

```
private func apply(remoteConfig: AppConfigStatus) {
    switch remoteConfig {
    case .ok:
        proceedAfterRemoteConfig()
    case .forceUpdate(let minVersion, let storeURL):
        guard manifest.checksForceUpdate,
              let required = AppVersion(minVersion),
              let current = AppVersion.current,
              current < required else {
            proceedAfterRemoteConfig()
            return
        }
        let vc = ForceUpdateViewController(
            brandColor: manifest.brandColor,
            minVersion: minVersion,
            storeURL: storeURL
        )
        setRoot(vc, animated: true)
    case .maintenance(let message, let until):
        guard manifest.hasMaintenanceCheck else {
            proceedAfterRemoteConfig()
            return
        }
        let vc = MaintenanceViewController(
```

```
            brandColor: manifest.brandColor,
            message: message,
            until: until,
            onRetry: { [weak self] in self?.checkRemoteConfig() }
        )
        setRoot(vc, animated: true)
    }
}
```

Блокатор обновления показывается, только если выполнены **все** три условия в `guard`:

1. У mini-app включён флаг `checksForceUpdate`. Сервер может прислать force update кому угодно, но показываем мы его только тем, кому он нужен: офлайн-калькулятору, например, ломающие изменения API безразличны.
2. Обе версии удалось разобрать (`AppVersion(minVersion)` и `AppVersion.current` не `nil`). Если сервер прислал мусор, лучше пропустить человека, чем запереть его навсегда.
3. Текущая версия **меньше** минимальной. Сервер сообщает порог, а сравнивает клиент. Так серверу не нужно знать версию каждого клиента: один и тот же ответ «минимум 2.0.0» заблокирует 1.9 и пропустит 2.0.1.

Для maintenance логика проще — нужен только флаг `hasMaintenanceCheck`. `onRetry` ведёт обратно в `checkRemoteConfig()`, с `[weak self]` по той же причине, что и в 9.3.

Почему версии нельзя сравнивать как строки

Версия приложения — строка вида `"1.10.2"`: три числа через точку (мажорная, минорная, патч). Хочется написать `current < minVersion` для строк, но строки сравниваются **посимвольно**, как слова в словаре. Посмотрим на числах, что получится:

- `"1.9"` против `"1.10"`: первые два символа `1.` совпадают, дальше `9` против `1`. Символ `9` «больше» символа `1`, значит, строка `"1.9"` считается больше `"1.10"`. А по смыслу 1.9 — это версия **до** 1.10. Человек с 1.9 не увидит требование «обновись до 1.10».
- `"2.0.0"` против `"10.0.0"`: `2` больше `1`, строка `"2.0.0"` «больше» — хотя 2 меньше 10.

Встречается совет сравнивать через `compare(_:options: .numeric)` — он сравнивает группы цифр как числа, и оба примера выше решает правильно. Но у него своя ловушка: `"2.0".compare("2.0.0", options: .numeric)` возвращает «меньше». Для людей 2.0 и 2.0.0 — одна и та же версия, а блокатор покажется пользователю, у которого уже всё обновлено. Мы проверили все эти случаи запуском.

Надёжнее разобрать версию на числа и сравнить по частям, дополняя недостающие части нулями:

```
struct AppVersion: Comparable, Sendable {
    let parts: [Int]

    init?(_ string: String) {
        var parts: [Int] = []
        for piece in string.split(separator: ".", omittingEmptySubsequences: false) {
            guard let number = Int(piece), number >= 0 else { return nil }
```

```
            parts.append(number)
        }
        guard !parts.isEmpty else { return nil }
        self.parts = parts
    }

    static func < (lhs: AppVersion, rhs: AppVersion) -> Bool {
        let count = max(lhs.parts.count, rhs.parts.count)
        for i in 0..<count {
            let l = i < lhs.parts.count ? lhs.parts[i] : 0
            let r = i < rhs.parts.count ? rhs.parts[i] : 0
            if l != r { return l < r }
        }
        return false
    }

    static func == (lhs: AppVersion, rhs: AppVersion) -> Bool {
        !(lhs < rhs) && !(rhs < lhs)
    }

    static var current: AppVersion? {
        let string = Bundle.main.object(forInfoDictionaryKey:
        ↪  "CFBundleShortVersionString") as? String
        return string.flatMap(AppVersion.init)
    }
}
```

Как это работает:

- `init?` режет строку по точкам и превращает каждый кусок в `Int`. Если кусок не число (`"1.0-beta"`) или пустой (`"1..2"`, `omittingEmptySubsequences: false` сохраняет пустые куски, чтобы их поймать), инициализатор возвращает `nil` — такую версию мы не понимаем.
- `<` идёт по позициям слева направо. Для `1.9` и `1.10`: части `[1, 9]` и `[1, 10]`, первая позиция 1 = 1, вторая 9 < 10 — ответ «меньше», верно. Для `2.0` и `2.0.0`: части `[2, 0]` и `[2, 0, 0]`, недостающая третья часть считается нулём, все позиции равны — «не меньше», а `==` даёт «равны». Верно.
- `Comparable` требует только `<`; остальные операторы (`>`, `<=`, `>=`) Swift выводит сам. `==` мы задали вручную, иначе Swift сравнил бы массивы `[2, 0]` и `[2, 0, 0]` напрямую и счёл бы их разными.
- `current` читает версию приложения из `Info.plist` по ключу `CFBundleShortVersionString`. Это «маркетинговая» версия — та, что видна в App Store (`2.0.1`). Есть ещё `CFBundleVersion` — номер сборки (build number), он растёт при каждой загрузке в App Store Connect и для этой проверки не подходит.

Проверенные запуском результаты:

Сравнение	Строки `<`	`.numeric`	`AppVersion`
`1.9` и `1.10`	1.9 больше (неверно)	меньше	меньше

Сравнение	Строки `<`	`.numeric`	`AppVersion`
`2.0.0` и `10.0.0`	2.0.0 больше (неверно)	меньше	меньше
`2.0` и `2.0.0`	меньше (неверно)	меньше (неверно)	равны
`1.2.3` и `1.2`	больше	больше	больше

Упражнение 9.1. Добавь в `AppVersion` поддержку версий с суффиксом вроде `"2.1.0-beta"`: суффикс отбрасывается, сравниваются только числа. Проверь, что `AppVersion("2.1.0-beta")! < AppVersion("2.1.1")!` даёт `true`. Ответ — в конце главы.

9.5 ForceUpdateViewController — блокирующий

```
final class ForceUpdateViewController: UIViewController {
    private let brandColor: UIColor
    private let minVersion: String
    private let storeURL: URL

    init(brandColor: UIColor, minVersion: String, storeURL: URL) {
        self.brandColor = brandColor
        self.minVersion = minVersion
        self.storeURL = storeURL
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        setupLayout()
    }

    private func setupLayout() {
        let stack = makeGateStack(
            symbol: "arrow.down.app.fill",
            color: brandColor,
            title: "Нужно обновить",
            body: "Эта версия приложения больше не поддерживается. "
                + "Обнови до \(minVersion) или новее — это займёт минуту."
        )
        let updateButton = UIButton(configuration: makeUpdateConfig())
        updateButton.addTarget(self, action: #selector(updateTapped), for: .touchUpInside)

        let noteLabel = UILabel()
        noteLabel.text = "Без обновления продолжить нельзя."
        noteLabel.font = .preferredFont(forTextStyle: .footnote)
        noteLabel.adjustsFontForContentSizeCategory = true
```

```
        noteLabel.textColor = .secondaryLabel
        noteLabel.textAlignment = .center

        stack.addArrangedSubview(updateButton)
        stack.addArrangedSubview(noteLabel)
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerYAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.centerYAnchor),
           stack.leadingAnchor.constraint(equalTo: view.layoutMarginsGuide.leadingAnchor,
↪   constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor, constant: -16),
        ])
    }

    @objc private func updateTapped() {
        UIApplication.shared.open(storeURL)
    }
}
```

Иконка, заголовок и текст собираются общей функцией `makeGateStack` — она же пригодится экрану обслуживания (полный код — в 9.11). Под ними — кнопка и мелкая приписка.

Никаких «продолжить позже». Одна кнопка — «Обновить в App Store». Тап вызывает `UIApplication.shared.open(storeURL)`: система открывает ссылку. Ссылка вида `https://apps.apple.com/app/i` где число — Apple ID приложения из App Store Connect, на iPhone открывается сразу в приложении App Store. В симуляторе приложения App Store нет, поэтому там ссылка откроется в Safari; проверять переход в магазин нужно на устройстве. Встречается и схема `itms-apps://`, но обычная `https://`-ссылка работает везде.

Если хочется показать страницу приложения, не выходя из него, есть `SKStoreProductViewController` из StoreKit: он открывает карточку App Store прямо поверх твоего экрана.

Кнопка — `UIButton.Configuration.filled()` со стрелкой справа:

```
private func makeUpdateConfig() -> UIButton.Configuration {
    var cfg = UIButton.Configuration.filled()
    cfg.title = "Обновить в App Store"
    cfg.image = UIImage(systemName: "arrow.up.right.square.fill")
    cfg.imagePlacement = .trailing
    cfg.imagePadding = 8
    cfg.cornerStyle = .capsule
    cfg.baseBackgroundColor = brandColor
    cfg.baseForegroundColor = .white
    cfg.contentInsets = NSDirectionalEdgeInsets(top: 14, leading: 24, bottom: 14,
↪   trailing: 24)
    return cfg
}
```

- `imagePlacement = .trailing` ставит иконку **после** текста. «Trailing» — «со стороны конца строки»: в русском это справа, а в арабском интерфейсе система сама перенесёт иконку влево. Стрелка вверх-вправо намекает, что откроется что-то внешнее.
- `imagePadding = 8` — 8 точек между текстом и иконкой.
- `cornerStyle = .capsule` — скругление радиусом в половину высоты: кнопка превращается в «таблетку».
- `baseBackgroundColor` / `baseForegroundColor` — цвет заливки и текста.
- `contentInsets` — внутренние отступы: по 14 точек сверху и снизу, по 24 — слева и справа. `NSDirectionalEdgeInsets` тоже говорит `leading` / `trailing`, а не `left` / `right`, по той же причине.

9.6 MaintenanceViewController — с таймером и повтором

Maintenance отличается двумя вещами:

1. **Обратный отсчёт.** Если сервер прислал `until`, показываем «Возвращаемся через ~44 мин 59 с» и обновляем каждую секунду.
2. **Кнопка «Проверить ещё раз».** Работы иногда заканчиваются раньше; не заставляй человека перезапускать приложение.

Таймер:

```
private var timer: Timer?

override func viewWillAppear(_ animated: Bool) {
    super.viewWillAppear(animated)
    startCountdown()
}

override func viewDidDisappear(_ animated: Bool) {
    super.viewDidDisappear(animated)
    timer?.invalidate()
    timer = nil
}

private func startCountdown() {
    guard let until else {
        countdownLabel.isHidden = true
        return
    }
    timer?.invalidate()
    updateCountdown(until: until)
    timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
        MainActor.assumeIsolated {
            self?.updateCountdown(until: until)
        }
    }
```

```
}

private func updateCountdown(until: Date) {
    let remaining = until.timeIntervalSinceNow
    if remaining <= 0 {
        countdownLabel.text = "Похоже, работы завершены — проверь ещё раз."
        timer?.invalidate()
        timer = nil
        return
    }
    countdownLabel.text = "Возвращаемся через ~\(formatter.string(from: remaining) ??
↪   "—")"
}
```

Разберём.

Когда запускать и останавливать. Таймер стартует в `viewWillAppear` (экран вот-вот появится) и останавливается в `viewDidDisappear` (экран ушёл). Отсчёт нужен, только пока его видно. Это парные методы жизненного цикла: в отличие от `viewDidLoad`, который приходит один раз, UIKit зовёт их столько раз, сколько экран показывается и уходит (глава 0, раздел 0.6) — отсюда и `timer?.invalidate()` перед созданием нового, защита от двух таймеров сразу, если экран появится повторно. Координатор убирает этот экран, меняя `window.rootViewController`; старому корневому экрану при этом приходят `viewWillDisappear` и `viewDidDisappear` (проверено на симуляторе iOS 26.5), так что таймер остановится.

Почему `[weak self]`. `Timer.scheduledTimer` регистрирует таймер в **run loop** — цикле событий главного потока, который крутится всё время жизни приложения. Run loop держит таймер, таймер держит замыкание. Если бы замыкание держало `self` сильно, экран не освободился бы, пока кто-нибудь не вызовет `invalidate()`: цепочка «run loop → таймер → замыкание → экран» сама не рвётся, и забытый `invalidate()` означал бы утечку навсегда. С `[weak self]` экран освобождается в любом случае, а остановку таймера мы делаем явно в `viewDidDisappear`.

Зачем `MainActor.assumeIsolated`. Замыкание таймера для компилятора — обычное, не привязанное к главному актору, а `updateCountdown` принадлежит главному актору (как весь класс). Swift 6 предупреждает о таком вызове — мы проверили сборкой без `assumeIsolated`: `warning: call to main actor-isolated instance method 'updateCountdown(until:)' in a synchronous nonisolated context`. Это предупреждение, а не ошибка, но оставлять его нельзя. Мы знаем, что таймер, созданный `scheduledTimer` на главном потоке, срабатывает на главном потоке. `MainActor.assumeIsolated { ... }` сообщает это компилятору: «я уже на главном акторе, выполняй». Если обещание окажется ложным, приложение остановится с понятной ошибкой, а не испортит данные тихо. `assumeIsolated` доступен с iOS 13, подходит для нашего iOS 15.

Отсчёт на числах. `until.timeIntervalSinceNow` — сколько секунд осталось до `until`. Для «45 минут от старта» в первую секунду это 2699,9…; когда время вышло, число становится нулём или отрицательным — тогда пишем «Похоже, работы завершены» и останавливаем таймер.

`DateComponentsFormatter` — форматтер длительностей из Foundation. Он превращает секунды в текст на языке устройства:

```
private let formatter: DateComponentsFormatter = {
    let f = DateComponentsFormatter()
    f.allowedUnits = [.hour, .minute, .second]
    f.unitsStyle = .abbreviated
    return f
}()
```

`allowedUnits` — какие единицы использовать, `unitsStyle = .abbreviated` — сокращения. Проверено запуском: 2699 секунд на русском дают «44 мин 59 с», 3723 секунды — «1 ч 2 мин 3 с», на английском — «1h 2m 3s». Склонять «минута / минуты / минут» самим не нужно. Форматтер создаётся один раз и хранится в свойстве: создавать его каждую секунду заново — лишняя работа.

Кнопка повтора:

```
@objc private func retryTapped() {
    var cfg = retryButton.configuration
    cfg?.showsActivityIndicator = true
    cfg?.title = "Проверяем…"
    retryButton.configuration = cfg
    retryButton.isEnabled = false
    onRetry()
}
```

Кнопка показывает спиннер, выключается (защита от серии тапов) и зовёт `onRetry` — замыкание от координатора, которое ведёт обратно в `checkRemoteConfig()`. Координатор ставит экран загрузки, снова спрашивает сервер. Если работы ещё идут — появится новый экран обслуживания. Если закончились — цепочка пойдёт дальше, к входу и главному экрану.

9.7 Что считать «hard» и «soft» update

Наш force update — **жёсткий** (hard): одна кнопка, обойти нельзя. В реальности бывают и мягкие варианты:

- **Рекомендованное обновление** — окно с двумя кнопками: «Обновить» и «Напомнить позже». Старой версией пока можно пользоваться.
- **Ненавязчивое** — баннер в углу «Доступна новая версия», ничего не блокирует.
- **Смешанное** — мягкое до определённой даты («старая версия поддерживается ещё две недели»), потом жёсткое.

В playground'е — жёсткий вариант для простоты. В настоящем приложении выбор зависит от того, что случилось:

- сервер больше не понимает старые версии → жёсткий;
- появилась новая функция, без которой можно жить → мягкий;
- обновился дизайн без изменений API → ненавязчивый или никакой: автообновления сделают своё дело.

Для мягкого варианта в `AppConfigStatus` добавляют ещё один случай, например `.recommendUpdate(latestVers String, storeURL: URL)`, и экран с кнопкой «Позже», которая ведёт дальше по цепочке.

9.8 Demo-сценарии в реестре

Чтобы увидеть оба экрана, `registerDemoConfigScenarios` из 9.2 регистрирует сценарии для Music (force update до 2.0.0) и Chat (обслуживание 45 минут). Но сервер — это только половина: экран покажется, если у mini-app в манифесте включён соответствующий флаг.

```
var m = AppManifest.placeholder(id: "music", name: "Музыка",
                                subtitle: "AVPlayer", symbolName: "music.note",
                                brandColor: .systemRed)
m.checksForceUpdate = true
```

И ещё одно условие для force update из 9.4: версия приложения должна быть **меньше** 2.0.0. У нового проекта в Xcode версия по умолчанию 1.0 (поле **Version** на вкладке **General** настроек таргета), так что условие выполняется.

Упражнение 9.2. Включи Music флаг `checksForceUpdate = true`. Зайди в Music: сначала splash, потом на 0,4 секунды экран загрузки, потом «Нужно обновить». Тапни «Обновить в App Store». Встряхни симулятор (^⌘Z), чтобы вернуться в лаунчер. Теперь поменяй версию приложения на 2.0 в настройках таргета и запусти снова. Что покажет Music и почему? Ответ — в конце главы.

9.9 Бытовая аналогия

Force update — **техосмотр автомобиля**. Приезжаешь на стоянку торгового центра, а охрана говорит: «у машины просрочен техосмотр, сюда нельзя». Вариант один — пройти техосмотр (обновить приложение). Поэтому экран блокирующий.

Maintenance — **табличка «Закрыто на инвентаризацию» на двери магазина**. Дверь заперта на 45 минут: сходи за кофе, возвращайся. Иногда инвентаризация заканчивается раньше, и табличку снимают — но чтобы это заметить, нужно подёргать дверь. Это наша кнопка «Проверить ещё раз».

9.10 Кеширование remote-config

В нашей реализации каждый вход в mini-app делает свежий запрос. В реальности конфигурация меняется редко — раз в день или реже. Стандартный приём:

- хранить последний ответ вместе с временем получения;
- при запуске, если ответу меньше N минут (например, 5), брать его из кеша без ожидания;
- иначе — запрашивать сервер и обновлять кеш.

Реализация — порядка 30 строк поверх `AppConfigService`: ответ кодируется в JSON и кладётся в `UserDefaults` вместе с датой. Если сервер недоступен, а кеш есть — разумно взять кеш, даже старый: лучше пустить человека, чем показать «нет сети» на ровном месте.

В книге кеш не делаем, но в production он нужен: без него каждый запуск ждёт сеть.

9.11 Экраны целиком

Все типы главы в одном файле. `AppManifest` — из главы 2, методы координатора — в 9.3 и 9.4, `AppVersion` и `AppConfigService` — выше.

```
import UIKit

func makeGateStack(symbol: String, color: UIColor, title: String, body: String) ->
↪  UIStackView {
    let icon = UIImageView(image: UIImage(systemName: symbol))
    icon.tintColor = color
    icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 64,
↪  weight: .semibold)
    let titleLabel = UILabel()
    titleLabel.text = title
    titleLabel.font = .preferredFont(forTextStyle: .title2)
    titleLabel.adjustsFontForContentSizeCategory = true
    titleLabel.textAlignment = .center
    titleLabel.numberOfLines = 0
    let bodyLabel = UILabel()
    bodyLabel.text = body
    bodyLabel.font = .preferredFont(forTextStyle: .body)
    bodyLabel.adjustsFontForContentSizeCategory = true
    bodyLabel.textColor = .secondaryLabel
    bodyLabel.textAlignment = .center
    bodyLabel.numberOfLines = 0
    let stack = UIStackView(arrangedSubviews: [icon, titleLabel, bodyLabel])
    stack.axis = .vertical
    stack.alignment = .center
    stack.spacing = 16
    return stack
}

final class MaintenanceViewController: UIViewController {
    private let brandColor: UIColor
    private let message: String
    private let until: Date?
    private let onRetry: () -> Void

    private let countdownLabel = UILabel()
    private let retryButton = UIButton(configuration: .gray())
    private var timer: Timer?

    private let formatter: DateComponentsFormatter = {
        let f = DateComponentsFormatter()
        f.allowedUnits = [.hour, .minute, .second]
        f.unitsStyle = .abbreviated
        return f
    }()
```

```
    init(brandColor: UIColor, message: String, until: Date?, onRetry: @escaping () ->
    ↪  Void) {
        self.brandColor = brandColor
        self.message = message
        self.until = until
        self.onRetry = onRetry
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
       let stack = makeGateStack(symbol: "wrench.and.screwdriver.fill", color: brandColor,
                                 title: "Технические работы", body: message)
        countdownLabel.font = .monospacedDigitSystemFont(ofSize: 17, weight: .medium)
        countdownLabel.textAlignment = .center
        countdownLabel.numberOfLines = 0
        var cfg = UIButton.Configuration.gray()
        cfg.title = "Проверить ещё раз"
        retryButton.configuration = cfg
        retryButton.addTarget(self, action: #selector(retryTapped), for: .touchUpInside)
        stack.addArrangedSubview(countdownLabel)
        stack.addArrangedSubview(retryButton)
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerYAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.centerYAnchor),
           stack.leadingAnchor.constraint(equalTo: view.layoutMarginsGuide.leadingAnchor,
↪   constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor, constant: -16),
        ])
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        startCountdown()
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        timer?.invalidate()
        timer = nil
    }
```

```
    private func startCountdown() {
        guard let until else {
            countdownLabel.isHidden = true
            return
        }
        timer?.invalidate()
        updateCountdown(until: until)
       timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
            MainActor.assumeIsolated {
                self?.updateCountdown(until: until)
            }
        }
    }

    private func updateCountdown(until: Date) {
        let remaining = until.timeIntervalSinceNow
        if remaining <= 0 {
            countdownLabel.text = "Похоже, работы завершены — проверь ещё раз."
            timer?.invalidate()
            timer = nil
            return
        }
        countdownLabel.text = "Возвращаемся через ~\(formatter.string(from: remaining) ??
↪   "—")"
    }

    @objc private func retryTapped() {
        var cfg = retryButton.configuration
        cfg?.showsActivityIndicator = true
        cfg?.title = "Проверяем…"
        retryButton.configuration = cfg
        retryButton.isEnabled = false
        onRetry()
    }
}
```

Пара деталей, которых не было в разделах:

- `monospacedDigitSystemFont` — системный шрифт, в котором все цифры одной ширины. Без него при отсчёте «44 мин 59 с» → «44 мин 58 с» текст подёргивался бы: у обычного шрифта «1» уже, чем «8».
- `ForceUpdateViewController` целиком приведён в 9.5, а `makeUpdateConfig()` — сразу после него; в файле они идут подряд.

Ответы к упражнениям

9.1. Отрезаем всё после первого – до разбора:

```
init?(_ string: String) {
    let numeric = string.split(separator: "-", maxSplits: 1).first.map(String.init) ??
    ↪ string
    var parts: [Int] = []
    for piece in numeric.split(separator: ".", omittingEmptySubsequences: false) {
        guard let number = Int(piece), number >= 0 else { return nil }
        parts.append(number)
    }
    guard !parts.isEmpty else { return nil }
    self.parts = parts
}
```

`"2.1.0-beta"` превращается в `"2.1.0"`, части `[2, 1, 0]`; против `[2, 1, 1]` на третьей позиции 0 < 1 — `true`. Для пустой строки `split` вернёт пустой массив, `.first` будет `nil`, сработает `?? string`, и дальше `Int("")` вернёт `nil` — как и раньше.

9.2. Music откроется без блокатора. Сервер по-прежнему отвечает «минимум 2.0.0», но `AppVersion("2.0")` и `AppVersion("2.0.0")` равны, условие `current < required` ложно, и `apply` уходит в `proceedAfterRemoteConfig()`. Со строковым сравнением через `.numeric` блокатор показался бы и здесь.

Что мы выучили

- Force update и maintenance — **серверные** гейты: чтобы решить, что показывать, нужен сетевой запрос.
- Пока запрос идёт, показываем экран загрузки (`RemoteConfigLoadingViewController`).
- Асинхронная задача координатора держит его слабо, `guard let self` стоит после `await`, а при выходе в лаунчер задача отменяется — иначе поздний ответ сервера перекроет лаунчер.
- `AppConfigStatus` — enum с тремя случаями: `.ok`, `.forceUpdate`, `.maintenance`, каждый со своими данными.
- Гейт показывается, только если в манифесте включён нужный флаг.
- Версии сравниваем по числам, дополняя недостающие части нулями: `1.9 < 1.10`, `2.0 == 2.0.0`. Строки и `.numeric` здесь ошибаются.
- Версия приложения — `CFBundleShortVersionString`, а не `CFBundleVersion` (номер сборки).
- Force update — **блокирующий** экран с одной кнопкой `UIApplication.shared.open(storeURL)`.
- Maintenance — отсчёт на `Timer` с `[weak self]`, остановкой в `viewDidDisappear` и `MainActor.assumeIsolated`, плюс кнопка повтора. `viewWillAppear`/`viewDidDisappear` приходят при каждом показе и уходе экрана, в том числе при смене `window.rootViewController`.
- `UIActivityIndicatorView` — view, не контрол; `hidesWhenStopped` по умолчанию `true`.
- `DateComponentsFormatter` форматирует длительность на языке устройства: «44 мин 59 с».
- В production ответ remote config кешируют.

Apple Developer Documentation

- Human Interface Guidelines — Patterns — раздел о типовых сценариях, в том числе запуске приложения: блокирующий экран оправдан, только когда без него приложение не может работать.
- `Bundle.main` — доступ к `Info.plist` приложения; отсюда читаем текущую версию.
- `CFBundleShortVersionString` — «маркетинговая» версия («1.2.3»); с ней и сравниваем `minVersion`, а не с `CFBundleVersion` (номер сборки).
- `URLSession` — настоящий remote config обычно запрашивают через `URLSession.shared.data(for:)`; наш мок повторяет ту же async-форму.
- `UIApplication.open(_:options:completionHandler:)` — открывает ссылку `https://apps.apple.com/...` в App Store.
- `SKStoreProductViewController` — карточка приложения из App Store поверх своего экрана, без выхода из приложения.
- `Timer.scheduledTimer(withTimeInterval:repeats:block:)` — таймер для отсчёта; `[weak self]` и явный `invalidate()`.
- `DateComponentsFormatter` — форматирует длительности с учётом языка («1 ч 2 мин 3 с»).
- `OperationQueue` — старая альтернатива async/await, если сетевой слой проекта построен на `Operation`.

→ Глава 10. Region + Age gates — фильтры по локации и возрасту

Глава 10. Region + Age gates — фильтры по локации и возрасту

Эти два гейта собраны в одну главу, потому что устроены похоже:

- оба спрашивают **один раз** (как онбординг) и сохраняют ответ в `UserDefaults`;
- оба собирают данные **до** того, как человек увидит главный экран;
- оба могут **не пустить** дальше: age gate — если человек слишком молод; region — теоретически, если регион не поддерживается (у нас такого случая нет).

Но есть и **принципиальное отличие**: у age gate **два** выхода — `onPass` («проходи») и `onTooYoung` («рано»). Координатор по-разному реагирует на каждый (см. главу 4).

Полный код обоих гейтов — в разделе 10.12.

10.1 Region picker

Зачем вообще спрашивать регион. От него часто зависят:

- **валюта** в ценах;
- **контент** — какие фильмы, песни, товары доступны в стране;
- **способы оплаты** — где-то есть Apple Pay, где-то только карта;
- **законы** — GDPR в Евросоюзе, COPPA в США и местные правила о персональных данных.

Сначала два понятия, которые легко спутать.

- **Локаль** (locale) — набор региональных настроек устройства: язык интерфейса, регион, формат дат и чисел, валюта. Человек выбирает её в «Настройки → Основные → Язык и регион». Например, локаль `ru_KZ` — русский язык, регион Казахстан: даты вида «27.09.2026», деньги в тенге.
- **Местоположение** — где телефон физически находится. Это геолокация, для неё нужно разрешение (глава 7).

Регион в локали — это **выбор человека в настройках**, а не место, где он сейчас. Казахстанец в командировке в Стамбуле по-прежнему видит регион «Казахстан». Поэтому регион из локали — хорошая **догадка**, но не истина, и мы даём явный выбор.

Страна в нашем списке:

```
struct Region: Hashable, Sendable {
    let code: String      // ISO 3166-1 alpha-2: "KZ"
```

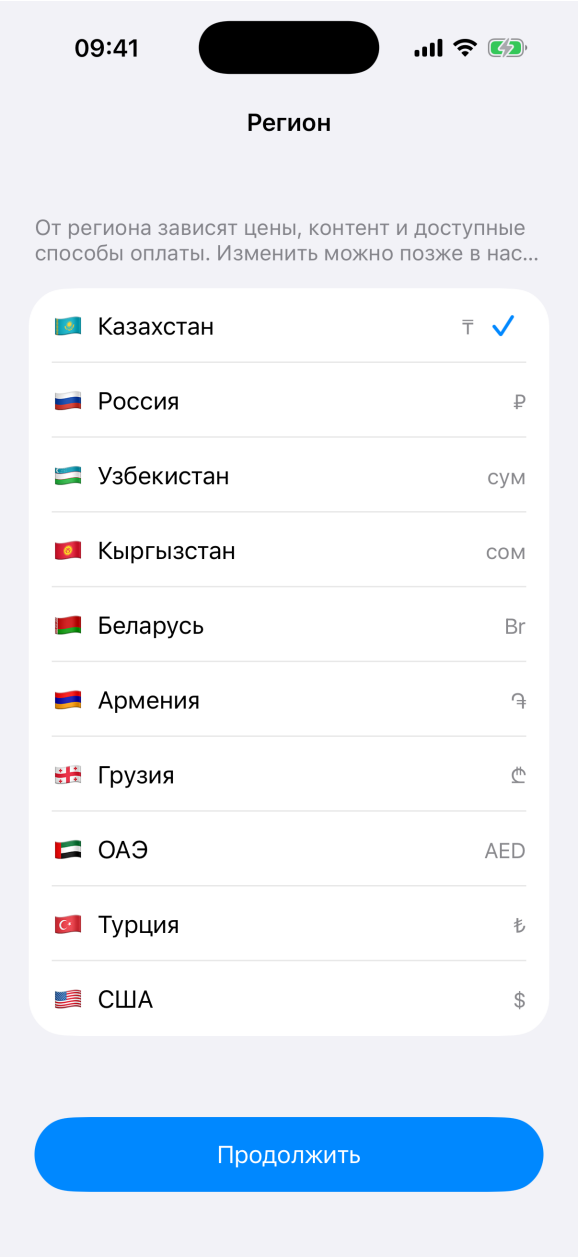


Рис. 7: Выбор региона: список стран с флагами и валютой

```
    let name: String
    let currency: String

    var flag: String {
        let base: UInt32 = 0x1F1E6 - 0x41
        var result = ""
        for scalar in code.uppercased().unicodeScalars {
            guard let flagScalar = Unicode.Scalar(base + scalar.value) else { return "" }
            result.unicodeScalars.append(flagScalar)
        }
        return result
    }

    static let all: [Region] = [
        Region(code: "KZ", name: "Казахстан", currency: "₸"),
        Region(code: "RU", name: "Россия", currency: "₽"),
        Region(code: "UZ", name: "Узбекистан", currency: "сум"),
        Region(code: "KG", name: "Кыргызстан", currency: "сом"),
    ]
}
```

`code` — двухбуквенный код страны по стандарту ISO 3166-1 alpha-2: `KZ`, `RU`, `UZ`. Этот же код iOS возвращает в локали, поэтому по нему удобно сравнивать.

`Hashable` нужен, чтобы регионы можно было сравнивать и класть в множества, `Sendable` — чтобы значение можно было передавать между потоками.

Как из «KZ» получается флаг

Эмодзи-флаг — не отдельная картинка, а **пара особых символов**. В Unicode есть 26 «региональных букв» — от «региональной A» с кодом `0x1F1E6` до «региональной Z». Две такие буквы подряд iOS рисует как флаг страны с этим кодом. Код обычной латинской «A» — `0x41` (65 в десятичной записи).

Считаем на числах для «K»:

- код буквы «K» — `0x4B` (75);
- «K» отстоит от «A» на 75 − 65 = 10 позиций;
- «региональная K» — это `0x1F1E6` + 10 = `0x1F1F0`.

`base` в коде — это заранее вычтенное `0x1F1E6 - 0x41`, поэтому `base + scalar.value` сразу даёт региональную букву. «KZ» → «региональная K» + «региональная Z» → флаг Казахстана. Никаких картинок в проекте и никаких эмодзи в исходнике. `Unicode.Scalar(...)` возвращает опционал: если бы в коде оказался не латинский символ, получился бы недопустимый код, и мы вернули бы пустую строку.

Хранение выбора

```
final class RegionStorage {
```

```
    static let shared = RegionStorage()
    private init() {}

    private func key(for manifestId: String) -> String {
        "region.selected.\(manifestId)"
    }

    func region(for manifestId: String) -> Region? {
        guard let code = UserDefaults.standard.string(forKey: key(for: manifestId)) else {
        ↪  return nil }
        return Region.all.first { $0.code == code }
    }

    func setRegion(_ region: Region, for manifestId: String) {
        UserDefaults.standard.set(region.code, forKey: key(for: manifestId))
    }

    func reset(for manifestId: String) {
        UserDefaults.standard.removeObject(forKey: key(for: manifestId))
    }
}
```

Ключ свой для каждого mini-app: `region.selected.music`, `region.selected.weather`. В настоящем приложении регион обычно один на всё приложение, но в playground'е каждое mini-app независимо.

Храним только код («KZ»), а не всю структуру. При чтении ищем регион с этим кодом в `Region.all`. Если его нет (например, страну убрали из списка в новой версии) — получаем `nil`, и гейт спросит заново. Это защита от устаревших данных.

10.2 Догадка по локали

При показе списка сразу отмечаем регион из локали:

```
private func guessDefaultRegion() {
    let code: String?
    if #available(iOS 16, *) {
        code = Locale.current.region?.identifier
    } else {
        code = Locale.current.regionCode
    }
    if let code, let match = regions.first(where: { $0.code == code }) {
        selected = match
    }
    tableView.reloadData()
    updateContinueState()
}
```

`Locale.current` — текущая локаль. Код региона в ней в iOS 16 и новее читается через `region?.identifier`, а в iOS 15 — через старое свойство `regionCode`, которое с iOS 16 помечено устаревшим. `if #available(iOS 16, *)` выбирает нужный путь во время выполнения: на iOS 16+ — новый, на iOS 15 — старый. Компилятор не ругается на устаревшее свойство, потому что оно вызывается только в ветке для старых систем.

Если кода нет в нашем списке (телефон настроен на Польшу, а Польши нет), `selected` остаётся `nil`: человек выбирает сам, а кнопка «Продолжить» выключена, пока выбора нет.

> **Догадка ≠ решение за человека.** Мы **отмечаем** угаданный регион, но не сохраняем его сами. Человек видит догадку и одним тапом подтверждает или меняет. Это лучше, чем «угадал и сохранил молча», особенно для тех, кто путешествует.

10.3 Список — UITableView с галочкой

Список стран — `UITableView` в стиле `.insetGrouped`: карточка со скруглёнными углами и отступами от краёв, как в «Настройках». В каждой строке флаг, название и валюта; у выбранной — галочка.

```
extension RegionPickerViewController: UITableViewDataSource, UITableViewDelegate {
    func tableView(_ tableView: UITableView, titleForHeaderInSection section: Int) ->
    ↪ String? {
        "Выбери регион"
    }

    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        regions.count
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "cell", for: indexPath)
        let region = regions[indexPath.row]
        var content = cell.defaultContentConfiguration()
        content.text = "\(region.flag)  \(region.name)"
        content.secondaryText = region.currency
        content.prefersSideBySideTextAndSecondaryText = true
        cell.contentConfiguration = content
        cell.accessoryType = (selected?.code == region.code) ? .checkmark : .none
        cell.tintColor = brandColor
        return cell
    }

    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        selected = regions[indexPath.row]
        tableView.reloadData()
        updateContinueState()
    }
```

```
}
```

Переиспользование ячеек. `dequeueReusableCell(withIdentifier:for:)` не создаёт ячейку каждый раз. Таблица держит небольшой запас ячеек: когда строка уезжает за край экрана, её ячейка возвращается в запас и потом приходит заново — уже для другой строки. Поэтому в `cellForRowAt` нужно выставлять **всё** состояние ячейки: и текст, и галочку. Если бы мы ставили `.checkmark` только выбранной строке и никогда не ставили `.none` остальным, переиспользованная ячейка могла бы «принести» чужую галочку. Строка с тернарным оператором как раз выставляет оба варианта. Идентификатор `"cell"` мы зарегистрировали заранее: `tableView.register(UITableViewCell.self, forCellReuseIdentifier: "cell")` (см. 10.12). Раз класс зарегистрирован, `dequeueReusableCell(withIdentifier:for:)` всегда возвращает ячейку: из запаса — тогда перед выдачей таблица зовёт у неё `prepareForReuse()` — или новую, созданную через `init(style:reuseIdentifier:)`. Сама ячейка — обычная view (`UITableViewCell → UIView`), а не контрол: нажатие на строку обрабатывает таблица, а не ячейка. Подробно про таблицу, ячейки и reuse — глава 0, раздел 0.5.

`UIListContentConfiguration` (iOS 14+) — современный способ задать содержимое стандартной ячейки. `defaultContentConfiguration()` даёт заготовку со стилем ячейки; заполняешь `text`, `secondaryText`, при желании `image`, и присваиваешь в `cell.contentConfiguration`. Раньше для того же брали `UITableViewCell(style: .value1)` и трогали `textLabel` / `detailTextLabel` напрямую. В SDK эти свойства помечены «будут объявлены устаревшими в будущем выпуске» (`API_TO_BE_DEPRECATED`): компилятор пока не предупреждает, но Apple просит новый код писать через `UIListContentConfiguration`.

`prefersSideBySideTextAndSecondaryText = true` — название и валюта **в одну строку**: название слева, валюта справа. Без этого валюта оказалась бы под названием.

`accessoryType = .checkmark` — стандартная галочка в правой части ячейки. Её цвет берётся из `tintColor` ячейки, поэтому `cell.tintColor = brandColor` красит галочку в цвет mini-app.

`didSelectRowAt` — запомнили выбор, перерисовали таблицу (галочка переехала), обновили кнопку. `deselectRow(at:animated:)` снимает серую подсветку нажатой строки — иначе она осталась бы «залипшей». `reloadData()` перерисовывает все строки; для четырёх стран это мгновенно. Для длинного списка лучше перерисовать только две строки — старую и новую — через `reloadRows(at:with:)`.

Упражнение 10.1. Включи mini-app Music выбор региона (`m.requiresRegionPick = true` в `AppRegistry`, как флаги в главе 2). Зайди в Music — после splash появится «Выбери регион», и, если локаль симулятора казахстанская, Казахстан уже будет отмечен. Выбери другую страну, нажми «Продолжить». Зайди в Music ещё раз. Что увидишь и почему? Как вернуть экран выбора, не переустанавливая приложение? Ответ — в конце главы.

10.4 Age gate — когда он действительно нужен

Сначала факты, потому что вокруг возрастных гейтов много мифов.

Возрастной рейтинг в App Store ставит не разработчик напрямую: ты отвечаешь на анкету в App Store Connect (есть ли насилие, азартные игры, медицинские темы, общение с незнакомцами), и система выставляет рейтинг. С 2025 года шкала такая: **4+, 9+, 13+, 16+, 18+** (раньше были 12+ и 17+). Рейтинг может отличаться по странам.

Соблюдает рейтинг **сама iOS**: если родители включили в «Экранном времени» ограничение «до 13+», приложение 16+ ребёнок не скачает и не откроет. Поэтому правило «поставил 17+ — обязан сделать age gate» неверно: общего требования делать в приложении свою проверку возраста у Apple нет.

Когда проверка возраста внутри приложения всё-таки нужна:

- **Правила App Review.** Приложения с контентом, созданным пользователями (1.2.1(a)), и приложения, которые запускают внутри себя чужие мини-игры и мини-приложения (4.7.5), должны давать способ пометить контент старше рейтинга приложения и ограничивать к нему доступ по **подтверждённому или заявленному** возрасту.
- **Законы.** Ставки, алкоголь, знакомства, работа с данными детей — во многих странах законы требуют проверять возраст, иногда строже, чем просто спросить дату рождения.
- **Собственная политика продукта** — например, сервис только для взрослых.

Apple в iOS 26 добавила фреймворк **DeclaredAgeRange**: приложение может попросить человека поделиться возрастным диапазоном (например, «до 13», «13–15», «16+»), не спрашивая дату рождения. Данные указал сам человек или, для детей в «Семейном доступе», родитель; решает, делиться ли ими, тоже родитель. Чтобы пользоваться API, у таргета включают **capability** Declared Age Range. Capability — это «разрешённая возможность» приложения: включаешь её в Xcode на вкладке **Signing & Capabilities**, и в подпись приложения добавляется **entitlement** — запись «этому приложению можно пользоваться такой-то функцией». Это API только для iOS 26+, поэтому в книге с iOS 15 мы делаем гейт с вводом даты — а на новых системах его можно заменить системным запросом через `if #available(iOS 26, *)`.

Наш гейт — простая **самодекларация**: человек сам указывает дату рождения. Обойти её легко — достаточно соврать. Это не защита, а отметка «мы спросили». Для учебного примера этого хватает.

Структура гейта:

```
final class AgeGateViewController: UIViewController {
    private let manifestId: String
    private let brandColor: UIColor
    private let minAge: Int
    private let onPass: () -> Void
    private let onTooYoung: () -> Void

    init(manifestId: String,
         brandColor: UIColor,
         minAge: Int,
         onPass: @escaping () -> Void,
         onTooYoung: @escaping () -> Void) {
        self.manifestId = manifestId
        self.brandColor = brandColor
        self.minAge = minAge
        self.onPass = onPass
        self.onTooYoung = onTooYoung
        super.init(nibName: nil, bundle: nil)
    }
```

```
    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }
}
```

Главное — **два** замыкания:

- `onPass` — возраст не меньше `minAge`, идём дальше;
- `onTooYoung` — возраст меньше, возвращаемся в лаунчер.

Это пример **гейта с несколькими выходами** из главы 4. Какой выход куда ведёт, решает координатор.

10.5 UIDatePicker — стиль «барабаны»

```
datePicker.datePickerMode = .date
datePicker.preferredDatePickerStyle = .wheels
datePicker.maximumDate = Date()
datePicker.minimumDate = Calendar.current.date(byAdding: .year, value: -120, to: Date())
datePicker.date = Date()
datePicker.addTarget(self, action: #selector(dateChanged), for: .valueChanged)
```

Разбор:

- `datePickerMode = .date` — только дата, без времени.
- `preferredDatePickerStyle = .wheels` — три крутящихся барабана: день, месяц, год. По умолчанию стиль `.automatic`: конкретный вид система подбирает сама по платформе и режиму, и для режима `.date` на iOS это компактный вариант — строка с датой, по тапу на которую открывается календарь (в симуляторе `datePickerStyle` свежего пикера читается как `.compact`). Для даты рождения это неудобно: до 1990 года пришлось бы листать календарь месяц за месяцем. Барабан года прокручивается за пару взмахов.
- `maximumDate = Date()` — нельзя выбрать **будущее**: родиться в 2030 году не получится.
- `minimumDate` — 120 лет назад. `Calendar.current.date(byAdding: .year, value: -120, to: Date())` отнимает 120 календарных лет от сегодняшней даты. Функция возвращает опционал, и `minimumDate` тоже опционал, так что их можно присвоить напрямую.
- `date = Date()` — стартовое значение **сегодня**. Почему не «20 лет назад», как иногда делают? Потому что такая дата уже проходит проверку 16+: ребёнку достаточно нажать «Подтвердить», ничего не трогая. Проверка возраста должна быть **нейтральной**: не подсказывать «правильный» ответ и не делать проходящий вариант выбором по умолчанию. По той же причине на экране не написано, какой возраст нужен. Так советуют, например, разъяснения американского регулятора FTC к закону о детской приватности COPPA.

`addTarget(_:action:for: .valueChanged)` работает, потому что `UIDatePicker` — наследник `UIControl` (`UIDatePicker → UIControl → UIView`): у него есть target-action и событие `.valueChanged`, как у `UISwitch` или `UISlider`. У `UILabel` или `UIPickerView` такого механизма нет — они не контролы (глава 0, раздел 0.4).

`dateChanged` вызывается при каждом повороте барабана. В нём пересчитываем возраст и обновляем подпись:

```
@objc private func dateChanged() {
    updateAgeLabel()
}

private func updateAgeLabel() {
    let years = AgeStorage.shared.ageYears(for: datePicker.date)
    ageLabel.text = "Тебе \(years) " + russianYearsSuffix(years)
}
```

10.6 Русские окончания — год / года / лет

Маленькая, но заметная деталь. В английском у чисел две формы: `1 year`, `2 years`. В русском три: «1 год», «2 года», «5 лет».

```
func russianYearsSuffix(_ years: Int) -> String {
    let mod100 = years % 100
    let mod10 = years % 10
    if mod100 >= 11 && mod100 <= 14 { return "лет" }
    switch mod10 {
    case 1: return "год"
    case 2, 3, 4: return "года"
    default: return "лет"
    }
}
```

`%` — остаток от деления. `years % 10` — последняя цифра числа, `years % 100` — две последние. Правило словами:

- если две последние цифры — от 11 до 14, всегда «лет»: 11 лет, 12 лет, 112 лет;
- иначе смотрим на последнюю цифру: 1 → «год» (1, 21, 101 год); 2, 3, 4 → «года» (2, 23, 104 года); 0 и 5–9 → «лет» (0, 5, 20, 25 лет).

Проверим на 21: две последние цифры — 21, это не 11–14; последняя цифра 1 → «21 год». На 111: две последние — 11, попадаем в исключение → «111 лет».

Как это делают по-взрослому. Для локализованных приложений правильный путь — не функция в коде, а **правила множественного числа** в файлах локализации: String Catalog (`.xcstrings`, Xcode 15+) или старый `.stringsdict`. Там для строки задаются варианты по категориям Unicode CLDR: для русского это `one` (1, 21, 31…), `few` (2–4, 22–24…), `many` (0, 5–20, 25…) и `other` (дробные числа). Система сама выбирает вариант по числу, а переводчик на казахский или английский задаст свои формы. В коде тогда одна строка:

```
let text = String.localizedStringWithFormat(
    NSLocalizedString("age.years", comment: "Возраст на экране проверки"),
    years
)
```

`NSLocalizedString` находит строку по ключу `age.years` в ресурсах приложения, `localizedStringWithFormat` подставляет число и выбирает нужную форму. В String Catalog варианты задаются прямо в редакторе Xcode: у строки выбираешь **Vary by Plural** и заполняешь формы.

Есть ещё «автоматическое грамматическое согласование» в `AttributedString(localized:)` с разметкой `^[...](inflect: true)`. Но оно работает не для всех языков: `InflectionRule.canInflect(language:)` подскажет, поддерживается ли язык. Мы проверили в симуляторе iOS 26.5: для `"en"` метод возвращает `true`, для `"ru"` — `false`. Для русского остаются правила множественного числа. Их мы тоже проверили запуском с `.stringsdict` из четырёх форм: 1 → «1 год», 22 → «22 года», 11 → «11 лет», 111 → «111 лет» — ровно как наша функция.

Наша функция — честный вариант для приложения на одном языке. Если окончания нужны в нескольких местах, вынеси её в общий файл (`Common/`), а не копируй в каждый экран.

Упражнение 10.2. Не запуская код, посчитай, что вернёт `russianYearsSuffix` для 0, 4, 12, 22, 104 и 1011. Потом проверь себя. Ответ — в конце главы.

10.7 «Пока рано» — блокирующее состояние

При тапе «Подтвердить»:

```
@objc private func confirmTapped() {
    let date = datePicker.date
    AgeStorage.shared.setBirthDate(date, for: manifestId)
    if AgeStorage.shared.ageYears(for: date) < minAge {
        showBlocker()
    } else {
        onPass()
    }
}
```

Дату сохраняем **в любом случае** — и когда человек прошёл, и когда нет. Зачем сохранять «слишком молодую» дату? Иначе проверку легко обойти: получил отказ, вышел, зашёл снова — и ввёл другую дату. С сохранённой датой при новом входе гейт сразу покажет блокер (см. `viewDidLoad` в 10.12), а когда человек подрастёт до нужного возраста, `shouldShow` сам пропустит его (10.9).

`showBlocker()` **не** меняет корневой экран и не показывает alert. Он перестраивает текущий экран в блокирующее состояние:

```
private func showBlocker() {
    let icon = UIImageView(image: UIImage(systemName: "hand.raised.fill"))
    icon.tintColor = .systemRed
    icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 56)

    let titleLabel = UILabel()
    titleLabel.text = "Пока рано"
    titleLabel.font = .preferredFont(forTextStyle: .title2)
    titleLabel.adjustsFontForContentSizeCategory = true
```

```
    let bodyLabel = UILabel()
    bodyLabel.text = "Это приложение для пользователей постарше."
    bodyLabel.font = .preferredFont(forTextStyle: .body)
    bodyLabel.adjustsFontForContentSizeCategory = true
    bodyLabel.numberOfLines = 0
    bodyLabel.textAlignment = .center

    var cfg = UIButton.Configuration.gray()
    cfg.title = "Вернуться в лаунчер"
    let exitButton = UIButton(configuration: cfg, primaryAction: UIAction { [weak self] _
    ↪  in
        self?.onTooYoung()
    })

    blockerStack.arrangedSubviews.forEach { $0.removeFromSuperview() }
    [icon, titleLabel, bodyLabel, exitButton].forEach(blockerStack.addArrangedSubview)
    formStack.isHidden = true
    blockerStack.isHidden = false
    UIAccessibility.post(notification: .screenChanged, argument: titleLabel)
}
```

Что здесь происходит:

- Экран с самого начала содержит **два** стека: `formStack` (заголовок, барабаны, подпись, кнопка) и пустой скрытый `blockerStack`. `showBlocker` наполняет второй, прячет первый и показывает второй. Прятать каждую view по отдельности не нужно — достаточно спрятать стек целиком.
- `blockerStack.arrangedSubviews.forEach { $0.removeFromSuperview() }` — на случай повторного вызова: старое содержимое убираем, чтобы не получить две иконки.
- `UIButton(configuration:primaryAction:)` (iOS 15+ — вместе с самим `UIButton.Configuration`; с iOS 14 есть только `UIButton(type:primaryAction:)`) — кнопка сразу с действием в виде `UIAction`, без `addTarget` и `@objc`-метода. `[weak self]` — потому что кнопка принадлежит экрану, а замыкание без `weak` держало бы экран сильно: экран → кнопка → действие → экран, цикл.
- Текст «для пользователей постарше» без конкретного возраста — чтобы не подсказывать, какую дату ввести в другой раз.
- `UIAccessibility.post(notification: .screenChanged, argument:)` — сообщение для VoiceOver: «экран сильно изменился», и фокус диктора переходит на заголовок «Пока рано». Без этого незрячий человек не узнал бы, что форма исчезла.

Можно было бы вместо этого поставить в окно отдельный экран-блокер, но это лишнее: блокирующее состояние — часть того же гейта.

10.8 Хранение даты

```
final class AgeStorage {
    static let shared = AgeStorage()
```

```
    private init() {}

    private func key(for manifestId: String) -> String {
        "agegate.birthday.\(manifestId)"
    }

    func birthDate(for manifestId: String) -> Date? {
        UserDefaults.standard.object(forKey: key(for: manifestId)) as? Date
    }

    func setBirthDate(_ date: Date, for manifestId: String) {
        UserDefaults.standard.set(date, forKey: key(for: manifestId))
    }

    func reset(for manifestId: String) {
        UserDefaults.standard.removeObject(forKey: key(for: manifestId))
    }

    func ageYears(for date: Date, now: Date = Date()) -> Int {
        Calendar.current.dateComponents([.year], from: date, to: now).year ?? 0
    }
}
```

`UserDefaults` умеет хранить `Date` напрямую — не нужно превращать дату в строку. Сохраняем через `set`, читаем через `object(forKey:)` с приведением `as? Date`.

`ageYears` через `Calendar.current.dateComponents([.year], from:to:)` — **правильный** способ посчитать полные годы. Календарь считает, сколько **полных календарных лет** прошло между датами. Пример: родился 28.09.2010, сегодня 27.09.2026 — это 15 лет, шестнадцатилетие наступит только завтра. Календарь так и ответит: 15.

Почему не «разделить секунды на длину года»? Дата — это момент времени в секундах. Можно взять разницу и разделить на 365,25 суток по 86 400 секунд. Но четверть суток в «среднем годе» — это усреднение високосных лет, и в дни рядом с днём рождения такое деление ошибается на день в ту или иную сторону. Для гейта «16+» ошибка на день — пропустить человека за день до шестнадцатилетия. Календарь считает по настоящим датам и таких ошибок не делает.

Про **часовой пояс**: дата из `UIDatePicker` в режиме `.date` — это полночь выбранного дня по часовому поясу устройства, а `Calendar.current` считает в том же поясе. Пока пояс не меняется, всё сходится. Если человек переехал из Алматы (UTC+5) в Лондон (UTC+0 или +1 летом), сохранённая «полночь по Алматы» окажется вечером предыдущего дня по Лондону — и в день рождения возраст может «подрасти» на несколько часов позже. Для учебного гейта это несущественно; в серьёзной системе дату рождения хранят как три числа (год, месяц, день) без времени и пояса.

Параметр `now: Date = Date()` — для **тестирования**. В проверке можно подставить конкретную дату «сегодня» и убедиться, что функция права, не завися от реальных часов.

10.9 `shouldShow` для каждого гейта

```swift
// Region
static func shouldShow(for manifest: AppManifest) -> Bool {
    manifest.requiresRegionPick && RegionStorage.shared.region(for: manifest.id) == nil
}

// Age
static func shouldShow(for manifest: AppManifest) -> Bool {
    guard manifest.requiresAgeGate else { return false }
    if let date = AgeStorage.shared.birthDate(for: manifest.id) {
        return AgeStorage.shared.ageYears(for: date) < manifest.minAgeYears
    }
    return true
}
```

Region — простой: показываем, если флаг включён и регион ещё не выбран.

Age — хитрее. Показываем, если флаг включён **и** либо дата ещё не введена, либо по сохранённой дате возраст пока меньше `minAgeYears`. Второй случай — человек уже получил отказ. Гейт покажется снова, но сразу в состоянии «Пока рано» (проверка в `viewDidLoad`, 10.12), без возможности ввести другую дату. А в день, когда человеку исполнится нужный возраст, условие станет ложным, и гейт пропустит его сам.

Упражнение 10.3. Включи Music проверку возраста, как в примере из главы 2: `m.requiresAgeGate = true`, `m.minAgeYears = 16`. Зайди в Music и, не трогая барабаны, нажми «Подтвердить». Что увидишь? Выйди в лаунчер и зайди снова. Затем сделай так, чтобы гейт снова спросил дату. Ответ — в конце главы.

10.10 Бытовая аналогия

Выбор региона — **выбор языка в банкомате**. Спросили один раз, запомнили в карточке. Потом банкомат сразу говорит по-русски (или по-казахски, или по-английски). А подсказка «похоже, вы из Казахстана» — это банкомат, который по карте догадался о стране, но всё равно даёт нажать кнопку самому.

Проверка возраста — **кассир на входе в кинозал на фильм 18+**. Он просит назвать год рождения и не подсказывает, какой нужен. Ответ записывает в журнал: если сегодня не пустил, то завтра, назвав другой год, не пройдёшь. А через несколько лет, когда подрастёшь, журнал сам тебя пропустит.

10.11 Что мы пропустили в этой главе

- **Выбор языка.** Отдельный гейт мы не делаем: язык интерфейса лучше оставить системным настройкам. С iOS 13 у каждого приложения есть свой пункт выбора языка в «Настройки → [Приложение] → Язык», если приложение переведено больше чем на один язык. Список языков человека в порядке предпочтения — `Locale.preferredLanguages`.

- **Код страны для телефона** (+7, +1, +44) — отдельная задача формы регистрации, обычно свой список с поиском.
- **Согласие на обработку данных** (GDPR и похожие законы) — отдельный гейт. По устройству похож на primer из главы 7, по содержанию — длиннее и юридически строже. Что именно из этого нужно App Store, разбирается в главе 39.

10.12 Экраны целиком

`Region`, `RegionStorage` и `AgeStorage` целиком приведены выше, как и функция `russianYearsSuffix`. Здесь — оба экрана. `AppManifest` — из главы 2.

```
import UIKit

final class RegionPickerViewController: UIViewController {

    static func shouldShow(for manifest: AppManifest) -> Bool {
        manifest.requiresRegionPick && RegionStorage.shared.region(for: manifest.id) ==
↪   nil
    }

    private let manifestId: String
    private let brandColor: UIColor
    private let onDone: () -> Void
    private let regions = Region.all
    private var selected: Region?

    private let tableView = UITableView(frame: .zero, style: .insetGrouped)
    private let continueButton = UIButton(configuration: .filled())

    init(manifestId: String, brandColor: UIColor, onDone: @escaping () -> Void) {
        self.manifestId = manifestId
        self.brandColor = brandColor
        self.onDone = onDone
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemGroupedBackground
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "cell")
        tableView.dataSource = self
        tableView.delegate = self

        var cfg = UIButton.Configuration.filled()
        cfg.title = "Продолжить"
        cfg.baseBackgroundColor = brandColor
```

```
        cfg.cornerStyle = .capsule
        continueButton.configuration = cfg
        continueButton.addTarget(self, action: #selector(continueTapped), for:
↪   .touchUpInside)

        [tableView, continueButton].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview($0)
        }
        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
           tableView.bottomAnchor.constraint(equalTo: continueButton.topAnchor, constant:
↪   -12),
            continueButton.leadingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.leadingAnchor),
            continueButton.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor),
            continueButton.bottomAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.bottomAnchor, constant: -16),
            continueButton.heightAnchor.constraint(greaterThanOrEqualToConstant: 50),
        ])
        guessDefaultRegion()
    }

    private func guessDefaultRegion() {
        let code: String?
        if #available(iOS 16, *) {
            code = Locale.current.region?.identifier
        } else {
            code = Locale.current.regionCode
        }
        if let code, let match = regions.first(where: { $0.code == code }) {
            selected = match
        }
        tableView.reloadData()
        updateContinueState()
    }

    private func updateContinueState() {
        continueButton.isEnabled = selected != nil
    }

    @objc private func continueTapped() {
        guard let selected else { return }
        RegionStorage.shared.setRegion(selected, for: manifestId)
        onDone()
```

```
    }
}

final class AgeGateViewController: UIViewController {

    static func shouldShow(for manifest: AppManifest) -> Bool {
        guard manifest.requiresAgeGate else { return false }
        if let date = AgeStorage.shared.birthDate(for: manifest.id) {
            return AgeStorage.shared.ageYears(for: date) < manifest.minAgeYears
        }
        return true
    }

    private let manifestId: String
    private let brandColor: UIColor
    private let minAge: Int
    private let onPass: () -> Void
    private let onTooYoung: () -> Void

    private let datePicker = UIDatePicker()
    private let ageLabel = UILabel()
    private let formStack = UIStackView()
    private let blockerStack = UIStackView()

    init(manifestId: String,
         brandColor: UIColor,
         minAge: Int,
         onPass: @escaping () -> Void,
         onTooYoung: @escaping () -> Void) {
        self.manifestId = manifestId
        self.brandColor = brandColor
        self.minAge = minAge
        self.onPass = onPass
        self.onTooYoung = onTooYoung
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground

        let titleLabel = UILabel()
        titleLabel.text = "Укажи дату рождения"
        titleLabel.font = .preferredFont(forTextStyle: .title2)
        titleLabel.adjustsFontForContentSizeCategory = true
```

```
        datePicker.datePickerMode = .date
        datePicker.preferredDatePickerStyle = .wheels
        datePicker.maximumDate = Date()
        datePicker.minimumDate = Calendar.current.date(byAdding: .year, value: -120, to:
↪  Date())
        datePicker.date = Date()
        datePicker.addTarget(self, action: #selector(dateChanged), for: .valueChanged)

        ageLabel.font = .preferredFont(forTextStyle: .body)
        ageLabel.adjustsFontForContentSizeCategory = true
        ageLabel.textColor = .secondaryLabel

        var cfg = UIButton.Configuration.filled()
        cfg.title = "Подтвердить"
        cfg.baseBackgroundColor = brandColor
        cfg.cornerStyle = .capsule
        let confirm = UIButton(configuration: cfg)
        confirm.addTarget(self, action: #selector(confirmTapped), for: .touchUpInside)

        [titleLabel, datePicker, ageLabel, confirm].forEach(formStack.addArrangedSubview)
        formStack.axis = .vertical
        formStack.alignment = .center
        formStack.spacing = 16

        blockerStack.axis = .vertical
        blockerStack.alignment = .center
        blockerStack.spacing = 16
        blockerStack.isHidden = true

        for stack in [formStack, blockerStack] {
            stack.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview(stack)
            NSLayoutConstraint.activate([
                stack.centerYAnchor.constraint(equalTo:
↪  view.safeAreaLayoutGuide.centerYAnchor),
                stack.leadingAnchor.constraint(equalTo:
↪  view.layoutMarginsGuide.leadingAnchor),
                stack.trailingAnchor.constraint(equalTo:
↪  view.layoutMarginsGuide.trailingAnchor),
            ])
        }
        updateAgeLabel()

        if let saved = AgeStorage.shared.birthDate(for: manifestId),
           AgeStorage.shared.ageYears(for: saved) < minAge {
            showBlocker()
        }
    }
```

```
    @objc private func dateChanged() {
        updateAgeLabel()
    }

    private func updateAgeLabel() {
        let years = AgeStorage.shared.ageYears(for: datePicker.date)
        ageLabel.text = "Тебе \(years) " + russianYearsSuffix(years)
    }

    @objc private func confirmTapped() {
        let date = datePicker.date
        AgeStorage.shared.setBirthDate(date, for: manifestId)
        if AgeStorage.shared.ageYears(for: date) < minAge {
            showBlocker()
        } else {
            onPass()
        }
    }

    private func showBlocker() {
        let icon = UIImageView(image: UIImage(systemName: "hand.raised.fill"))
        icon.tintColor = .systemRed
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 56)

        let titleLabel = UILabel()
        titleLabel.text = "Пока рано"
        titleLabel.font = .preferredFont(forTextStyle: .title2)
        titleLabel.adjustsFontForContentSizeCategory = true

        let bodyLabel = UILabel()
        bodyLabel.text = "Это приложение для пользователей постарше."
        bodyLabel.font = .preferredFont(forTextStyle: .body)
        bodyLabel.adjustsFontForContentSizeCategory = true
        bodyLabel.numberOfLines = 0
        bodyLabel.textAlignment = .center

        var cfg = UIButton.Configuration.gray()
        cfg.title = "Вернуться в лаунчер"
        let exitButton = UIButton(configuration: cfg, primaryAction: UIAction { [weak
        ↪  self] _ in
            self?.onTooYoung()
        })

        blockerStack.arrangedSubviews.forEach { $0.removeFromSuperview() }
        [icon, titleLabel, bodyLabel, exitButton].forEach(blockerStack.addArrangedSubview)
        formStack.isHidden = true
        blockerStack.isHidden = false
```

```
        UIAccessibility.post(notification: .screenChanged, argument: titleLabel)
    }
}
```

Расширение с data source и delegate таблицы — в 10.3; в файле оно идёт сразу после `RegionPickerViewController`.

Пара деталей вёрстки:

- Таблица занимает всё от верха экрана до кнопки (`tableView.bottomAnchor` = верх кнопки минус 12 точек), а кнопка прижата к низу safe area. Таблица сама прокручивается, если стран станет много: `UITableView` — наследник `UIScrollView` (глава 0, раздел 0.5).
- Оба стека возрастного гейта центрируются по вертикали одинаково: они лежат друг на друге, а виден в каждый момент только один.

Ответы к упражнениям

10.1. Второй раз экран выбора не появится: код региона сохранён в `UserDefaults` под ключом `region.selected.music`, и `shouldShow` вернёт `false`. Вернуть экран можно вызовом `RegionStorage.shared.reset(for: "music")` — например, из отладочной кнопки или из точки останова в Xcode командой `expr RegionStorage.shared.reset(for: "music")`. Второй способ — удалить приложение с симулятора: вместе с ним сотрутся и настройки.

10.2. 0 → «лет» (последняя цифра 0); 4 → «года»; 12 → «лет» (две последние цифры 12 — исключение 11–14); 22 → «года» (22 — не исключение, последняя цифра 2); 104 → «года» (две последние — 04, не исключение, последняя 4); 1011 → «лет» (две последние цифры 11).

10.3. Барабаны стоят на сегодняшней дате, возраст 0 лет, поэтому «Подтвердить» сразу покажет «Пока рано». Дата сохранена, и при повторном входе гейт откроется уже с блокером, без барабанов. Чтобы гейт снова спросил дату, удали сохранённую дату: `AgeStorage.shared.reset(for: "music")` (из отладчика командой `expr`, как в 10.1) или удали приложение с симулятора.

Что мы выучили

- Region и Age — разовые гейты, ответ хранится в `UserDefaults`.
- Локаль — региональные настройки устройства (язык, регион, форматы), а не местоположение. Регион из локали — догадка, которую человек подтверждает.
- `Locale.current.region?.identifier` в iOS 16+ и `regionCode` в iOS 15 — через `if #available`.
- Флаг страны — две «региональные буквы» Unicode; их можно вычислить из кода «KZ», без картинок и эмодзи в исходнике.
- Список — `UITableView(.insetGrouped)` + `UIListContentConfiguration`
  - `accessoryType = .checkmark`; в `cellForRowAt` выставляем всё состояние ячейки, потому что ячейки переиспользуются.
- Возрастной рейтинг App Store (4+, 9+, 13+, 16+, 18+) соблюдает сама iOS; своя проверка возраста нужна по правилам 1.2.1(a) и 4.7.5, по законам или по политике продукта. В iOS 26 есть DeclaredAgeRange.

- Проверка возраста должна быть нейтральной: барабаны на сегодняшней дате, порог не показан, неудачная попытка запоминается.
- Возраст считаем `Calendar.dateComponents([.year], from:to:)` — полными календарными годами, а не делением секунд.
- Окончания «год / года / лет»: своя функция на `% 100` и `% 10` или, правильнее для локализации, правила множественного числа в String Catalog / `.stringsdict`.
- У age gate **два** выхода: `onPass` и `onTooYoung`.
- Блокирующее состояние — второй стек на том же экране и уведомление VoiceOver `.screenChanged`.

Apple Developer Documentation

- Human Interface Guidelines — Right to left — если поддерживаешь арабский или иврит, экраны должны зеркалиться; UIKit делает это сам, если раскладка на `leading`/`trailing`, а не на `left`/`right`.
- `Locale` — региональные настройки; `Locale.current` отражает выбор в «Настройки → Основные → Язык и регион».
- `Locale.current` — текущая локаль; в iOS 16+ регион читаем через `region?.identifier`.
- `Locale.preferredLanguages` — языки пользователя в порядке предпочтения.
- `Bundle.preferredLocalizations(from:forPreferences:)` — пересечение языков приложения и предпочтений пользователя.
- `NSLocalizedString` — загрузка локализованных строк, в том числе с правилами множественного числа; в Swift есть и `String(localized:)` (iOS 15+).
- `UIDatePicker` — `preferredDatePickerStyle = .wheels`, `minimumDate` / `maximumDate`.
- `Calendar.dateComponents(_:from:to:)` — разница между датами в календарных единицах; так считаем полные годы.
- `UIListContentConfiguration` — содержимое стандартной ячейки (iOS 14+), замена `textLabel` / `detailTextLabel`.
- `UITableView.dequeueReusableCell(withIdentifier:for:)` — всегда возвращает ячейку: из запаса (с вызовом `prepareForReuse()`) или новую из зарегистрированного класса.
- Age ratings values and definitions — шкала возрастных рейтингов App Store и вопросы анкеты.
- App Review Guidelines — пункты 1.2.1(a) и 4.7.5 о проверке возраста для пользовательского контента и мини-приложений.

→ Глава 11. Privacy blur + Biometric on resume

Глава 11. Privacy blur + Biometric on resume

Последний гейт нашей цепочки запуска. От всех предыдущих он отличается принципиально: он работает **не только в начале**. Splash, онбординг, экран-пояснение для разрешения показались — и всё. А этот гейт живёт **всю сессию** и реагирует на события жизненного цикла: приложение уходит в фон — закрываем экран размытием; возвращается — просим Face ID.

В этой главе два связанных, но независимых механизма:

1. **Privacy blur** (размытие для приватности) — пока приложение не на переднем плане, вместо содержимого видно размытие. Соседи в очереди или коллеги через плечо не увидят баланс в переключателе приложений.
2. **Biometric on resume** (биометрия при возврате) — когда человек возвращается в приложение из фона, спросить Face ID или Touch ID, прежде чем показать данные.

Оба нужны приложениям с чувствительными данными: банкам, мессенджерам, заметкам с паролями, медицинским картам.

Главное, что стоит вынести из главы, — **что биометрия гарантирует, а что нет**. Экран «приложите лицо» сам по себе — это замок на двери комнаты, а не сейф. Настоящий сейф — запись в Keychain, которую система отдаёт только после Face ID. Разберём оба в разделе 11.8.

Полный код — в разделе 11.12.

11.1 Жизненный цикл сцены и снимок экрана

Напомню, как iOS сообщает приложению о смене состояний (подробно — в главе 5). Приложение на iOS 13+ состоит из **сцен** (scene): сцена (`UIWindowScene`) — один экземпляр интерфейса приложения, обычно с одним окном; формально сцена управляет одним **или несколькими** окнами (`windowScene.windows`). На iPhone сцена обычно одна, на iPad их может быть несколько. Сцена — не view и не окно: `UIWindowScene → UIScene → UIResponder` (глава 0, раздел 0.7). У сцены есть состояния:

- **активна** (foreground active) — на экране, принимает касания;
- **неактивна** (foreground inactive) — видна, но касания не получает: поверх открыт Пункт управления, Центр уведомлений, системный alert, окно Face ID или переключатель приложений;
- **в фоне** (background) — не видна, скоро будет «заморожена».

Переходы приходят уведомлениями `NotificationCenter`:

- `UIScene.willDeactivateNotification` — сцена **сейчас перестанет** быть активной. Apple в документации перечисляет причины: временные прерывания (системные alert'ы) и уход в фон.
- `UIScene.didEnterBackgroundNotification` — сцена ушла в фон.
- `UIScene.didActivateNotification` — сцена снова активна.

Каждому уведомлению соответствует метод делегата сцены: `sceneWillResignActive(_:)`, `sceneDidEnterBackground(_:)`, `sceneDidBecomeActive(_:)`. UIKit зовёт метод и рассылает уведомление в один и тот же момент; мы берём уведомления, потому что наш контроллер — не делегат сцены.

Теперь о **снимке экрана** (snapshot). Документ Apple «Preparing your UI to run in the background» описывает так: после того как приложение ушло в фон и обработчик ухода в фон вернул управление, UIKit делает снимок текущего интерфейса. Этот снимок показывается в переключателе приложений и на мгновение — при возвращении в приложение. Та же статья прямо требует: в интерфейсе не должно остаться чувствительных данных — паролей, номеров карт.

Отсюда план:

- на `willDeactivate` накрываем окно размытием. Так закрыты сразу три случая: переключатель приложений (там сцена неактивна), системные шторки поверх приложения и снимок при уходе в фон — к моменту снимка размытие уже лежит сверху;
- на `didActivate` убираем размытие — если не нужна биометрия.

11.2 Контроллер «жизненного» гейта

В отличие от остальных гейтов, этот **не view controller**. Это обычный класс, который владеет двумя view и подписан на уведомления:

```
final class LifecycleSecurityController {

    private weak var window: UIWindow?
    private let manifest: AppManifest

    private var blurView: UIVisualEffectView?
    private var biometricPromptView: UIView?

    private var observers: [NSObjectProtocol] = []
    private var isAuthenticating = false
    private var needsUnlock = false
    private var didAutoPrompt = false

    init(window: UIWindow, manifest: AppManifest) {
        self.window = window
        self.manifest = manifest
    }
}
```

Свойства по порядку:

- `window` — окно, которое закрываем. `weak`, потому что окно принадлежит `SceneDelegate`; контроллер не должен продлевать ему жизнь.
- `blurView` и `biometricPromptView` — наши две накладки: размытие и экран блокировки поверх него. `nil` — накладки сейчас нет.
- `observers` — «токены» подписок на уведомления, чтобы потом отписаться.
- `isAuthenticating` — идёт ли проверка Face ID прямо сейчас.
- `needsUnlock` — «приложение побывало в фоне, на входе нужна проверка».
- `didAutoPrompt` — «в этот раз Face ID уже спрашивали автоматически».

Зачем последние два флага и почему их нельзя заменить одним, — главный сюжет раздела 11.5.

Почему не view controller? Этот гейт не становится корневым экраном — он накладывается **поверх любого** экрана mini-app, включая модальные окна и alert'ы. Для этого накладки кладутся прямо в окно (`UIWindow`), выше всей иерархии view controller'ов. View controller для этого не нужен.

Это возможно, потому что окно — тоже view: `UIWindow → UIView → UIResponder`. У него есть `addSubview`, `bounds`, `subviews`, и корневой экран (`rootViewController.view`) — лишь один из его подвидов. Само окно ничего не рисует: по документации Apple оно — «подложка», которая держит view и раздаёт им события. Подробно — глава 0, раздел 0.7.

Контроллер принадлежит `BootCoordinator` (глава 4). Когда координатор показывает главный экран, он создаёт и запускает его:

```
private func startLifecycleSecurity() {
    guard let window else { return }
    let controller = LifecycleSecurityController(window: window, manifest: manifest)
    controller.start()
    lifecycleSecurity = controller
}
```

и держит сильной ссылкой (`lifecycleSecurity`) до выхода в лаунчер. При выходе `stop()` снимает подписки и убирает накладки:

```
private func exitToLauncher() {
    window?.onShake = nil
    lifecycleSecurity?.stop()
    lifecycleSecurity = nil
    onExit()
}
```

11.3 NotificationCenter — подписка

```
func start() {
    guard manifest.hasPrivacyBlurOnBackground || manifest.requiresBiometricOnResume else
    ↪  { return }
    let nc = NotificationCenter.default
    let scene = window?.windowScene
```

```
    let resign = nc.addObserver(forName: UIScene.willDeactivateNotification,
                                object: scene, queue: .main) { [weak self] _ in
        MainActor.assumeIsolated { self?.handleWillDeactivate() }
    }
    let background = nc.addObserver(forName: UIScene.didEnterBackgroundNotification,
                                    object: scene, queue: .main) { [weak self] _ in
        MainActor.assumeIsolated { self?.handleDidEnterBackground() }
    }
    let active = nc.addObserver(forName: UIScene.didActivateNotification,
                                object: scene, queue: .main) { [weak self] _ in
        MainActor.assumeIsolated { self?.handleDidActivate() }
    }
    observers = [resign, background, active]
}

func stop() {
    observers.forEach { NotificationCenter.default.removeObserver($0) }
    observers.removeAll()
    removeBlur()
    removeBiometricPrompt()
}
```

Разбор:

- `guard` в начале — если у mini-app нет ни размытия, ни биометрии, подписываться незачем.
- `addObserver(forName:object:queue:using:)` — подписка «блоком»: передаёшь замыкание, и NotificationCenter вызывает его при каждом уведомлении. Метод возвращает **токен** (`NSObjectProtocol`). Чтобы отписаться, токен передают в `removeObserver(_:)` — это делает `stop()`.
- `object: scene` — слушаем уведомления **только своей** сцены. Если передать `nil`, придут уведомления от всех сцен приложения: на iPad с двумя окнами сворачивание одного окна закрыло бы размытием и второе. `window?.windowScene` — сцена, к которой относится окно.
- `queue: .main` — замыкание гарантированно выполняется на главной очереди. Это важно: трогать UIKit можно только с главного потока.
- `MainActor.assumeIsolated { ... }` — мост между старым API и нашим кодом главного актора. Замыкание NotificationCenter для компилятора не привязано к главному актору, а методы `handle...` — привязаны (как весь класс в нашем режиме). Мы знаем, что блок выполнится на главном потоке (`queue: .main`), но компилятор этого не знает. `assumeIsolated` говорит ему: «я уже на главном акторе, выполняй», а если это окажется неправдой, приложение остановится с понятной ошибкой. Доступен с iOS 13 — в нашем iOS 15 пользоваться можно без `@available`. (Альтернатива — `Task { @MainActor in self?.handleWillDeactivate() }`, но она выполнит код **позже**, на одном из новых проходов цикла событий, а нам нужно успеть поставить размытие до снимка.)
- `[weak self]` — обязательно. NotificationCenter держит замыкание, пока мы не отписались. Сильный `self` внутри удерживал бы контроллер — и он жил бы, даже когда координатор его отпустил.

11.4 Размытие

`UIVisualEffectView` с `UIBlurEffect` — стандартный способ получить размытие «как в системе»:

```
private func installBlur(on window: UIWindow) {
    guard blurView == nil else { return }
    let effect = UIBlurEffect(style: .systemMaterial)
    let view = UIVisualEffectView(effect: effect)
    view.frame = window.bounds
    view.autoresizingMask = [.flexibleWidth, .flexibleHeight]
    window.addSubview(view)
    blurView = view
}

private func removeBlur() {
    blurView?.removeFromSuperview()
    blurView = nil
}
```

По строкам:

- `guard blurView == nil` — защита от двойной установки. Уведомление о деактивации приходит и при открытии Пункта управления, и перед уходом в фон; без проверки мы положили бы два размытия друг на друга.
- `UIBlurEffect.Style.systemMaterial` — «материал» из системной палитры, сам подстраивается под светлую и тёмную тему. Есть тоньше (`.systemUltraThinMaterial`, `.systemThinMaterial` — сквозь них угадывается содержимое) и плотнее (`.systemThickMaterial`, `.systemChromeMaterial`). Для приватности нужен такой, через который не прочитать цифры. `.systemMaterial` — разумная середина; для совсем секретных экранов можно положить сверху ещё и непрозрачный фон с логотипом.
- `view.frame = window.bounds` — размытие размером с окно. `frame` задаётся в координатах superview — здесь окна, — а `bounds` окна начинается с (0, 0): на iPhone 16 это (0, 0, 393, 852) точки. Поэтому «frame = bounds родителя» всегда означает «накрыть родителя целиком» (глава 0, раздел 0.3).
- `autoresizingMask = [.flexibleWidth, .flexibleHeight]` — если окно изменит размер (поворот на iPad, изменение окна в многозадачности), размытие растянется вместе с ним. Это старый механизм «автоматического растягивания»: без констрейнтов, одной строкой. По умолчанию маска пустая (`UIViewAutoresizingNone`). Работает она, пока `translatesAutoresizingMaskIntoConstraints` у view остаётся `true` (так и есть по умолчанию) — UIKit превращает маску в констрейнты сам; ставить здесь `false` нельзя. Проверено в симуляторе: окно 393×852 растянули до 500×900 — подвид стал 500×900.
- `window.addSubview(view)` — кладём прямо в окно, последним, то есть **поверх** всего, что лежит в этом окне: корневого экрана, модальных экранов и alert'ов, которые показало само приложение. Чужие окна размытие не накрывает: системный запрос разрешения или окно Face ID живут в собственных окнах и процессах.

Сам `UIVisualEffectView` — наследник `UIView`, но устроен из двух внутренних подвидов: «фон» с эффектом и `contentView`. Отсюда правила Apple: свои подвиды класть только в `contentView`, а `alpha` у самой view и у её superview оставлять 1, иначе эффект ломается или исчезает. Нам это важно в 11.6.

11.5 Возвращение: с биометрией или без

Здесь прячется главный подвох главы. Вот наивная версия:

```
// Так НЕ делаем
private func handleEnterForeground() {
    guard let window else { return }
    if manifest.requiresBiometricOnResume {
        showBiometricPrompt(on: window)   // и сразу запрос Face ID
    } else {
        removeBlur()
    }
}
```

Кажется логичным: вернулись — спросили Face ID. Но вспомни 11.1 (и это мы видели в журнале запуска: как только появилось окно проверки, делегат сцены получил `sceneWillResignActive(_:)` — тот же момент, что наше `willDeactivateNotification`): сцена становится неактивной не только при уходе в фон, но и при **любом системном окне поверх приложения**. А системное окно Face ID — как раз такое. Получается цепочка:

1. Приложение вернулось → `didActivate` → показываем Face ID.
2. Окно Face ID открылось → сцена неактивна → `willDeactivate`.
3. Человек посмотрел в камеру, окно закрылось → `didActivate`.
4. Мы снова показываем Face ID → и по кругу.

Ещё хуже с Пунктом управления: человек лишь опустил шторку, чтобы сделать экран ярче, — и при возврате получил требование Face ID, хотя приложение никуда не уходило.

Решение — различать «побывали в фоне» и «на секунду прикрылись шторкой»:

```
private func handleWillDeactivate() {
    guard let window else { return }
    installBlur(on: window)
}

private func handleDidEnterBackground() {
    guard manifest.requiresBiometricOnResume else { return }
    needsUnlock = true
    didAutoPrompt = false
}

private func handleDidActivate() {
    guard let window else { return }
    if needsUnlock {
        showBiometricPrompt(on: window)
        if !didAutoPrompt {
            didAutoPrompt = true
            authenticate()
        }
    } else if biometricPromptView == nil {
        removeBlur()
```

```
    }
}
```

Как это работает:

- **Размытие** ставим на любую деактивацию — и для шторки, и для фона.
- **Потребность в проверке** (`needsUnlock = true`) появляется только при настоящем уходе в фон. Окно Face ID или Пункт управления фон не вызывают — значит, и новой проверки не будет.
- **При возвращении:** если проверка нужна — показываем экран блокировки и **один раз** автоматически запускаем Face ID (`didAutoPrompt`). Если человек отменил Face ID, окно закрылось, пришёл новый `didActivate` — но `didAutoPrompt` уже `true`, и повторного запроса нет. Остаётся экран блокировки с кнопкой «Разблокировать»: человек сам решит, когда попробовать снова.
- **Если проверка не нужна** — убираем размытие. Но только когда нет экрана блокировки (`biometricPromptView == nil`): пока человек не прошёл проверку, размытие должно оставаться под экраном блокировки.

Проверь сценарии в голове:

Что случилось	willDeactivate	didEnterBackground	didActivate
Опустил Пункт управления и вернул	размытие	—	убрали размытие
Свернул и вернулся	размытие	`needsUnlock = true`	экран блокировки + Face ID
Окно Face ID открылось и закрылось	размытие уже есть	—	экран блокировки остаётся, повторного запроса нет

Упражнение 11.1. Сейчас Face ID спрашивается после **любого** ухода в фон, даже на две секунды. Сделай «льготный период»: если человек вернулся быстрее чем через 30 секунд, проверку не спрашивать. Ответ — в конце главы.

11.6 Экран блокировки

```
private func showBiometricPrompt(on window: UIWindow) {
    guard biometricPromptView == nil else { return }
    installBlur(on: window)

    let container = UIView(frame: window.bounds)
    container.autoresizingMask = [.flexibleWidth, .flexibleHeight]
    container.accessibilityViewIsModal = true

    let icon = UIImageView(image: UIImage(systemName: "lock.fill"))
    icon.tintColor = manifest.brandColor
    icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 48)
```

```
    var cfg = UIButton.Configuration.filled()
    cfg.title = unlockButtonTitle()
    cfg.baseBackgroundColor = manifest.brandColor
    cfg.cornerStyle = .capsule
    let button = UIButton(configuration: cfg, primaryAction: UIAction { [weak self] _ in
        self?.authenticate()
    })

    let stack = UIStackView(arrangedSubviews: [icon, button])
    stack.axis = .vertical
    stack.spacing = 24
    stack.alignment = .center
    stack.translatesAutoresizingMaskIntoConstraints = false
    container.addSubview(stack)
    NSLayoutConstraint.activate([
        stack.centerXAnchor.constraint(equalTo: container.centerXAnchor),
        stack.centerYAnchor.constraint(equalTo: container.centerYAnchor),
    ])
    window.addSubview(container)
    biometricPromptView = container
}
```

- Сначала `installBlur` — на случай, если размытия почему-то нет. Наш guard внутри не даст положить второе.
- `container` — прозрачная view на всё окно поверх размытия. Иконка замка и кнопка — по центру. Почему это отдельная view-сосед в окне, а не подвид размытия: у `UIVisualEffectView` подвиды разрешено добавлять только в его `contentView`, а менять `alpha` — нельзя (11.4). Отдельная view обходит оба ограничения и живёт своей жизнью: размытие остаётся, когда экран блокировки уже убран.
- `container` размечен через `frame` и `autoresizingMask`, а стек внутри него — констрейнтами. Так смешивать можно: важно лишь, чтобы у каждой view `translatesAutoresizingMaskIntoConstraints` соответствовал её способу раскладки — у стека `false`, у `container` по умолчанию `true`.
- `accessibilityViewIsModal = true` — для VoiceOver: «за этой view ничего нет». Без этого незрячий человек мог бы пролистать диктором элементы экрана **под** размытием и услышать баланс вслух.
- Кнопка вызывает `authenticate()` — ручная повторная попытка.

Подпись кнопки зависит от того, какая биометрия есть на устройстве:

```
private func unlockButtonTitle() -> String {
    let context = LAContext()
    _ = context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error: nil)
    switch context.biometryType {
    case .faceID: return "Разблокировать с Face ID"
    case .touchID: return "Разблокировать с Touch ID"
    default: return "Разблокировать"
    }
}
```

HIG советует называть способ входа прямо («с Face ID», а не просто «Войти») и не упоминать Face ID на устройстве, где его нет. `biometryType` заполняется только после вызова `canEvaluatePolicy`, поэтому вызываем его, даже если результат нам не нужен (`_ =`).

11.7 Биометрия — LocalAuthentication

LocalAuthentication — фреймворк Apple для проверки владельца устройства: Face ID, Touch ID или код-пароль. Главный класс — `LAContext`, «контекст проверки»: один объект на одну проверку. Правило App Review **2.5.13** требует для входа по лицу использовать именно LocalAuthentication, а не свою распознавалку лиц.

```
private func authenticate() {
    guard !isAuthenticating else { return }
    let context = LAContext()
    let policy: LAPolicy = .deviceOwnerAuthentication
    var error: NSError?
    guard context.canEvaluatePolicy(policy, error: &error) else {
        if error?.code == LAError.passcodeNotSet.rawValue {
            // На устройстве нет даже код-пароля: проверять нечем.
            unlock()
        }
        return
    }
    isAuthenticating = true
    Task { [weak self] in
        let success: Bool
        do {
            success = try await context.evaluatePolicy(
                policy,
                localizedReason: "Подтверди, что это ты, чтобы открыть профиль"
            )
        } catch {
            success = false
        }
        guard let self else { return }
        self.isAuthenticating = false
        if success {
            self.unlock()
        }
    }
}

private func unlock() {
    needsUnlock = false
    removeBiometricPrompt()
    removeBlur()
}
```

Разберём.

`isAuthenticating` — защита от двух проверок одновременно: пока окно Face ID открыто, повторный тап по кнопке ничего не сделает.

Новый `LAContext` на каждую проверку. Документация предупреждает: не рассчитывай, что прошлая успешная проверка означает успех новой. Контекст хранит состояние своей проверки; для новой проверки создаём новый контекст — так ничего не «прилипнет» от прошлого раза.

Политика (`LAPolicy`) — что считается успешной проверкой:

- `.deviceOwnerAuthenticationWithBiometrics` — **только** Face ID или Touch ID. Если лицо не распозналось несколько раз подряд, биометрия блокируется, и войти будет нечем.
- `.deviceOwnerAuthentication` — биометрия, а если она не удалась или её нет — **код-пароль устройства**. Мы берём её: человек с повреждённым пальцем или в маске не останется запертым вне своего приложения.

`canEvaluatePolicy(_:error:)` — «можно ли сейчас проверить?». Для нашей политики ответ «нет» бывает в основном по одной причине: на устройстве не задан код-пароль (`LAError.passcodeNotSet`). Тогда проверять нечем: без код-пароля нельзя включить и Face ID. Мы пропускаем человека — иначе владелец никогда не попадёт в своё приложение. Серьёзные банковские приложения в такой ситуации просят завести собственный PIN приложения; для учебного примера достаточно пропустить.

`evaluatePolicy(_:localizedReason:)` — сама проверка. Исходный метод устроен на completion handler, который, как сказано в документации, вызывается **на внутренней очереди фреймворка** — не на главном потоке. Мы берём его async-версию, которую Swift создаёт автоматически: `await` возвращает нас в код главного актора, и после него можно спокойно трогать интерфейс. Ошибка (человек нажал «Отменить», система прервала проверку) приходит как `throw` — считаем это неуспехом.

> **Почему не completion handler.** Старый вариант — `context.evaluatePolicy(policy, localizedReason: ...) { success, error in ... }`. Мы проверили запуском в симуляторе: это замыкание действительно вызывается **не** на главном потоке (`Thread.isMainThread` внутри него — `false`). Значит, любую работу с интерфейсом внутри нужно вручную переносить на главный поток (`DispatchQueue.main.async`), и забыть об этом легко. Async-версия делает переход сама.

`localizedReason` — пояснение в системном окне проверки: зачем просим. Документация Apple уточняет: коротко и ясно, на языке пользователя, и **без названия приложения** — оно и так видно в окне (в iOS — в подзаголовке). В симуляторе окно ввода код-пароля так и выглядит: заголовок «Введите код-пароль iPhone для приложения „G2Gates“», а под ним наша строка. Поэтому «Подтверди, что это ты, чтобы открыть профиль», а не «Подтверди вход в Профиль».

`NSFaceIDUsageDescription` — отдельная строка в Info.plist, которую iOS показывает, когда приложение **впервые** пытается использовать Face ID: «Разрешить приложению использовать Face ID?». Документация Apple: без этого ключа система не позволит приложению использовать Face ID. Для Touch ID такой строки не требуется.

```
<key>NSFaceIDUsageDescription</key>
<string>Face ID защищает профиль, когда приложение возвращается из фона.</string>
```

При успехе — `unlock()`: снимаем флаг, экран блокировки и размытие. При неудаче — всё остаётся как есть, человек может нажать «Разблокировать» ещё раз.

> **Про повторное использование разблокировки.** У `LAContext` есть свойство `touchIDAuthenticationAllowableReuseDuration`. Оно не про «запомнить прошлую проверку в приложении», как часто думают, а про **разблокировку устройства**: если человек только что разблокировал iPhone Touch ID и это было не раньше чем указанное число секунд назад, проверка пройдёт сразу, без второго прикладывания пальца. По умолчанию 0 — повторное использование выключено; максимум задан константой `LATouchIDAuthenticationMaximumAllowableReuseDuration` — в iOS 26 это 300 секунд, то есть 5 минут (значение напечатано запуском).

11.8 Что биометрия гарантирует, а что нет

`evaluatePolicy` возвращает приложению **одно логическое значение**: «да, это владелец» или «нет». Проверку лица делает защищённый сопроцессор (Secure Enclave), и подделать само распознавание нельзя. Но решение «пускать или нет» принимает **наш код**, в памяти нашего процесса. На взломанном устройстве этот `if success` можно подменить — и экран блокировки откроется без лица.

Поэтому экран блокировки из этой главы — защита от **случайного** человека: от ребёнка, взявшего телефон, от коллеги, заглянувшего через плечо. Он **не** защищает данные, если телефон попал в руки подготовленному злоумышленнику.

Настоящая защита — привязать к биометрии **сам секрет**. В главе 8 токен лежал в Keychain и был доступен при разблокированном устройстве. Можно положить его так, чтобы система выдавала его **только** после Face ID — проверку тогда делает не наш `if`, а сама iOS вместе с Secure Enclave:

```
import Foundation
import LocalAuthentication
import Security

nonisolated enum ProtectedTokenStore {
    private static let service = "kz.waid.uikitplayground.auth.protected"
    private static let account = "token"

    static func save(_ token: String) -> Bool {
        guard let access = SecAccessControlCreateWithFlags(
            nil,
            kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly,
            .biometryCurrentSet,
            nil
        ) else { return false }

        let base: [String: Any] = [
```

```
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrService as String: service,
            kSecAttrAccount as String: account,
        ]
        SecItemDelete(base as CFDictionary)
        var add = base
        add[kSecValueData as String] = Data(token.utf8)
        add[kSecAttrAccessControl as String] = access
        return SecItemAdd(add as CFDictionary, nil) == errSecSuccess
    }

    static func read(reason: String) -> String? {
        let context = LAContext()
        context.localizedReason = reason
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrService as String: service,
            kSecAttrAccount as String: account,
            kSecReturnData as String: true,
            kSecMatchLimit as String: kSecMatchLimitOne,
            kSecUseAuthenticationContext as String: context,
        ]
        var result: CFTypeRef?
        guard SecItemCopyMatching(query as CFDictionary, &result) == errSecSuccess,
              let data = result as? Data else { return nil }
        return String(data: data, encoding: .utf8)
    }
}

func loadProtectedToken() async -> String? {
    await Task.detached {
        ProtectedTokenStore.read(reason: "Подтверди вход в аккаунт")
    }.value
}
```

Что здесь нового по сравнению с `AuthStorage` из главы 8:

- **`SecAccessControlCreateWithFlags`** создаёт правило доступа к записи. Два условия:
  - `kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly` — запись существует, только пока на устройстве задан код-пароль, и не переезжает на другое устройство. Если человек снимет код-пароль, система **удалит** такие записи — токен пропадёт, и придётся войти заново;
  - `.biometryCurrentSet` — прочитать запись можно только после Face ID или Touch ID, и только с **текущим** набором лиц и пальцев. Если кто-то добавит в телефон своё лицо или палец, запись станет недоступной. Это защита от сценария «узнал код-пароль, добавил свой палец, открыл банк». Мягче — `.biometryAny` (любые, в том числе добавленные позже) или `.userPresence` (биометрия или код-пароль — пусть решит система).

- Правило кладётся в запись атрибутом `kSecAttrAccessControl` — вместо `kSecAttrAccessible` из главы 8.
- **Чтение** — тот же `SecItemCopyMatching`, но iOS сама показывает окно Face ID и отдаёт данные только после успеха. `LAContext` с `localizedReason` задаёт текст этого окна и передаётся атрибутом `kSecUseAuthenticationContext`.
- `SecItemCopyMatching` для такой записи **ждёт**, пока человек пройдёт проверку, — секунды. Держать главный поток всё это время нельзя: интерфейс замрёт. Поэтому `ProtectedTokenStore` объявлен `nonisolated` (не привязан к главному актору), а читают его через `Task.detached` — задачу, которая выполняется в фоне, вне главного актора. `await ... .value` дожидается результата, не блокируя главный поток.
- `SecItemDelete` + `SecItemAdd` вместо «добавить или обновить» из главы 8. Здесь это осознанно: обновить запись с биометрической защитой без самой проверки нельзя, поэтому старую удаляем, новую кладём.

С таким хранилищем экран блокировки из 11.6 становится просто удобной оболочкой, а стеной работает iOS: без лица владельца токена в памяти приложения просто нет.

Упражнение 11.2. Что произойдёт с токеном, сохранённым через `ProtectedTokenStore`, в каждом случае: (а) человек добавил второе лицо в «Альтернативный внешний вид» Face ID; (б) человек выключил код-пароль; (в) человек перенёс данные на новый iPhone через резервную копию? Ответ — в конце главы.

11.9 Проверка в симуляторе

В симуляторе нет камеры TrueDepth, но Face ID можно изобразить через меню **Features** → **Face ID** (меню в окне приложения Simulator):

- **Enrolled** — «лицо записано». Пока пункт не отмечен, у симулятора нет биометрии.
- **Matching Face** — нажимаешь, когда открыто окно Face ID, и проверка проходит.
- **Non-matching Face** — проверка не проходит.

Когда отмечено **Enrolled**, `canEvaluatePolicy` для нашей политики возвращает `true`, и появляется системное окно Face ID. После этого выбираешь в меню Matching или Non-matching — и получаешь нужный результат.

А если **Enrolled** не отмечено? Мы проверили запуском: политика «только биометрия» (`.deviceOwnerAuthenticationWithBiometrics`) даёт `false` с ошибкой «биометрия не настроена» (`biometryNotEnrolled`), а наша `.deviceOwnerAuthentication` даёт `true` — и iOS показывает окно ввода код-пароля. Так что без Enrolled ты увидишь запасной путь, а не Face ID. Для проверки главы сначала включи Enrolled.

Упражнение 11.3. Запусти Profile, войди (`test@uikit.kz`, глава 8). Включи Features → Face ID → Enrolled. Сверни приложение (⇧⌘H). Открой переключатель приложений (Device → App Switcher). Что видно на карточке Profile? Вернись в приложение. Что появится? Выбери Non-matching Face, потом нажми «Разблокировать» и выбери Matching Face. Ответ — в конце главы.

11.10 Бытовая аналогия

Размытие — **жалюзи на окнах банка**. Прохожий (переключатель приложений) заглядывает в окно и видит размытые силуэты, а не суммы на экранах. Жалюзи опускают, даже когда сотрудник просто отвернулся к шкафу (шторка Пункта управления), — так надёжнее.

Экран блокировки — **охранник у входа**. Он проверяет пропуск, только если ты выходил из здания (ушёл в фон), а не когда ты отвернулся к окну. И если ты не нашёл пропуск с первого раза, он не будет требовать его каждые две секунды — подождёт, пока ты сам подойдёшь снова (кнопка «Разблокировать»).

А Keychain с `.biometryCurrentSet` — **банковская ячейка**, которая открывается только отпечатком владельца. Даже если охранника обманули, ячейку без пальца не открыть.

11.11 Краевые случаи

Системные окна поверх приложения. Документ-пикер, окно «Поделиться», alert разрешения — некоторые из них показываются в отдельном процессе и деактивируют сцену. Размытие на мгновение появится. Это плата за то, что мы закрываем переключатель приложений. Банковские приложения обычно мирятся с этим.

Фоновые задачи. Если приложение выполняет фоновую работу (`BGTaskScheduler`), интерфейс ему в фоне не нужен. Нашему контроллеру это не мешает: он реагирует только на смену состояний сцены.

iPad и несколько окон. Контроллер привязан к одному окну и слушает только его сцену (`object: scene` в 11.3). Если у приложения несколько окон с чувствительным содержимым — нужно по контроллеру на каждое.

Скриншоты и запись экрана. Размытие не защищает от снимка экрана, который делает сам человек. iOS не даёт приложению запретить снимок экрана: уведомление `UIApplication.userDidTakeScreenshotNotification` приходит уже **после** снимка. Запись экрана и трансляцию (AirPlay) можно заметить: в iOS 17+ — через `traitCollection.sceneCaptureState`, в более старых — через `isCaptured` у экрана и уведомление `UIScreen.capturedDidChangeNotification`. Экран берём у сцены окна: `window.windowScene?.screen.isCaptured`. `UIScreen.main` Apple не рекомендует с iOS 16, а в SDK iOS 26 он помечен устаревшим: экран нужно брать «по контексту» — через `windowScene.screen`. При записи можно закрыть чувствительные элементы.

11.12 Код целиком

`ProtectedTokenStore` приведён полностью в 11.8. Ниже — `LifecycleSecurityController` одним файлом. `AppManifest` — из главы 2.

```
import UIKit
import LocalAuthentication

final class LifecycleSecurityController {

    private weak var window: UIWindow?
```

```
    private let manifest: AppManifest

    private var blurView: UIVisualEffectView?
    private var biometricPromptView: UIView?

    private var observers: [NSObjectProtocol] = []
    private var isAuthenticating = false
    private var needsUnlock = false
    private var didAutoPrompt = false

    init(window: UIWindow, manifest: AppManifest) {
        self.window = window
        self.manifest = manifest
    }

    func start() {
        guard manifest.hasPrivacyBlurOnBackground || manifest.requiresBiometricOnResume
        ↪  else { return }
        let nc = NotificationCenter.default
        let scene = window?.windowScene

        let resign = nc.addObserver(forName: UIScene.willDeactivateNotification,
                                    object: scene, queue: .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.handleWillDeactivate() }
        }
        let background = nc.addObserver(forName: UIScene.didEnterBackgroundNotification,
                                        object: scene, queue: .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.handleDidEnterBackground() }
        }
        let active = nc.addObserver(forName: UIScene.didActivateNotification,
                                    object: scene, queue: .main) { [weak self] _ in
            MainActor.assumeIsolated { self?.handleDidActivate() }
        }
        observers = [resign, background, active]
    }

    func stop() {
        observers.forEach { NotificationCenter.default.removeObserver($0) }
        observers.removeAll()
        removeBlur()
        removeBiometricPrompt()
    }

    private func handleWillDeactivate() {
        guard let window else { return }
        installBlur(on: window)
    }
```

```
    private func handleDidEnterBackground() {
        guard manifest.requiresBiometricOnResume else { return }
        needsUnlock = true
        didAutoPrompt = false
    }

    private func handleDidActivate() {
        guard let window else { return }
        if needsUnlock {
            showBiometricPrompt(on: window)
            if !didAutoPrompt {
                didAutoPrompt = true
                authenticate()
            }
        } else if biometricPromptView == nil {
            removeBlur()
        }
    }

    private func installBlur(on window: UIWindow) {
        guard blurView == nil else { return }
        let effect = UIBlurEffect(style: .systemMaterial)
        let view = UIVisualEffectView(effect: effect)
        view.frame = window.bounds
        view.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        window.addSubview(view)
        blurView = view
    }

    private func removeBlur() {
        blurView?.removeFromSuperview()
        blurView = nil
    }

    private func showBiometricPrompt(on window: UIWindow) {
        guard biometricPromptView == nil else { return }
        installBlur(on: window)

        let container = UIView(frame: window.bounds)
        container.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        container.accessibilityViewIsModal = true

        let icon = UIImageView(image: UIImage(systemName: "lock.fill"))
        icon.tintColor = manifest.brandColor
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 48)

        var cfg = UIButton.Configuration.filled()
        cfg.title = unlockButtonTitle()
```

```
        cfg.baseBackgroundColor = manifest.brandColor
        cfg.cornerStyle = .capsule
        let button = UIButton(configuration: cfg, primaryAction: UIAction { [weak self] _
        ↪ in
            self?.authenticate()
        })

        let stack = UIStackView(arrangedSubviews: [icon, button])
        stack.axis = .vertical
        stack.spacing = 24
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        container.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: container.centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: container.centerYAnchor),
        ])
        window.addSubview(container)
        biometricPromptView = container
    }

    private func removeBiometricPrompt() {
        biometricPromptView?.removeFromSuperview()
        biometricPromptView = nil
    }

    private func unlockButtonTitle() -> String {
        let context = LAContext()
        _ = context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error:
↪   nil)
        switch context.biometryType {
        case .faceID: return "Разблокировать с Face ID"
        case .touchID: return "Разблокировать с Touch ID"
        default: return "Разблокировать"
        }
    }

    private func authenticate() {
        guard !isAuthenticating else { return }
        let context = LAContext()
        let policy: LAPolicy = .deviceOwnerAuthentication
        var error: NSError?
        guard context.canEvaluatePolicy(policy, error: &error) else {
            if error?.code == LAError.passcodeNotSet.rawValue {
                // На устройстве нет даже код-пароля: проверять нечем.
                unlock()
            }
            return
```

```
        }
        isAuthenticating = true
        Task { [weak self] in
            let success: Bool
            do {
                success = try await context.evaluatePolicy(
                    policy,
                    localizedReason: "Подтверди, что это ты, чтобы открыть профиль"
                )
            } catch {
                success = false
            }
            guard let self else { return }
            self.isAuthenticating = false
            if success {
                self.unlock()
            }
        }
    }

    private func unlock() {
        needsUnlock = false
        removeBiometricPrompt()
        removeBlur()
    }
}
```

В нашем playground'е оба механизма включены у Profile — вместе с auth gate из главы 8:

```
AppManifest(
    id: "profile",
    name: "Профиль / Настройки",
    subtitle: "insetGrouped, разные типы ячеек",
    symbolName: "person.crop.circle",
    brandColor: .systemPurple,
    isReady: true,
    hasAuthGate: true,
    hasPrivacyBlurOnBackground: true,
    requiresBiometricOnResume: true,
    makeMain: { ProfileViewController() }
)
```

Это полный набор «осторожного» приложения: вход при первом открытии, размытие в переключателе, Face ID при возвращении. Так же ведут себя многие банковские приложения.

11.13 Что можно добавить

- **Льготный период** — не спрашивать Face ID, если человек вернулся быстро (упражнение 11.1).
- **Своё изображение вместо размытия** — логотип на фоне фирменного цвета. Выглядит аккуратнее размытого пятна и точно ничего не просвечивает.
- **Реакция на запись экрана** — закрывать суммы и номера карт, пока идёт запись или трансляция (11.11).

Всё это — поверх той же основы: подписка на уведомления сцены и view поверх окна.

Ответы к упражнениям

11.1. Запоминаем время ухода в фон и решаем о проверке при возвращении:

```
private var backgroundedAt: Date?
private let gracePeriod: TimeInterval = 30

private func handleDidEnterBackground() {
    guard manifest.requiresBiometricOnResume else { return }
    backgroundedAt = Date()
    didAutoPrompt = false
}

private func handleDidActivate() {
    guard let window else { return }
    if let backgroundedAt {
        self.backgroundedAt = nil
        if Date().timeIntervalSince(backgroundedAt) > gracePeriod {
            needsUnlock = true
        }
    }
    if needsUnlock {
        showBiometricPrompt(on: window)
        if !didAutoPrompt {
            didAutoPrompt = true
            authenticate()
        }
    } else if biometricPromptView == nil {
        removeBlur()
    }
}
```

`Date().timeIntervalSince(backgroundedAt)` — сколько секунд прошло с ухода в фон. Вернулся через 12 секунд — 12 меньше 30, проверки нет. Через 45 — больше 30, `needsUnlock = true`. `backgroundedAt` обнуляем сразу, чтобы очередной `didActivate` (например, после Пункта управления) не пересчитывал старое время. Проверка: сверни и сразу разверни — экран без Face ID; сверни, подожди минуту — Face ID появится.

Учти: часы устройства человек может перевести, и тогда разница окажется неверной. Для льготного периода в 30 секунд это не страшно; для серьёзных сроков используют монотонное время, которое не зависит от настроек часов.

11.2. (а) Запись станет недоступной: `.biometryCurrentSet` привязан к набору лиц и пальцев на момент сохранения, а новое лицо этот набор меняет. Чтение вернёт ошибку, приложению нужно будет попросить войти заново и сохранить токен снова. (б) Система удалит запись: у неё `kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly`, а такие записи живут, только пока задан код-пароль. (в) Запись не переедет: суффикс `ThisDeviceOnly` запрещает восстановление на другом устройстве. На новом iPhone человек войдёт заново.

11.3. На карточке Profile в переключателе — размытие вместо содержимого (его положил `willDeactivate`). При возвращении появится экран блокировки с замком и кнопкой «Разблокировать с Face ID», и сразу откроется окно Face ID. После Non-matching Face окно проверки предложит повторить попытку или ввести код-пароль; если отменить, останется экран блокировки, и Face ID сам больше не появится. Тап «Разблокировать» и Matching Face — экран блокировки и размытие исчезнут, виден Profile.

Что мы выучили

- «Жизненный» гейт — не view controller, а обычный класс с подпиской на уведомления **своей** сцены: `willDeactivate`, `didEnterBackground`, `didActivate`.
- Снимок для переключателя приложений iOS делает после ухода в фон; размытие ставим уже на `willDeactivate` — оно закрывает и переключатель, и шторки, и снимок.
- Размытие — `UIVisualEffectView` с `UIBlurEffect(style: .systemMaterial)`, прямо в окне, поверх всего в этом окне: окно — тоже `UIView`, а `frame = window.bounds` плюс `autoresizingMask` держат накладку размером с окно.
- Сцена (`UIWindowScene → UIScene → UIResponder`) — не окно и не view; она управляет окнами и сообщает о смене состояния уведомлениями и методами делегата одновременно.
- `MainActor.assumeIsolated` (iOS 13+) — мост между замыканием NotificationCenter на `.main` и методами главного актора.
- `[weak self]` в замыканиях подписок и `removeObserver` в `stop()` — иначе контроллер не освободится.
- Проверку Face ID требуем только после настоящего ухода в фон, иначе окно Face ID и Пункт управления запускают бесконечный круг.
- `LAContext` — новый на каждую проверку; политика `.deviceOwnerAuthentication` даёт запасной путь через код-пароль; `localizedReason` — без названия приложения.
- `evaluatePolicy` берём в async-версии: completion handler вызывается не на главном потоке, а `await` возвращает нас на главный актор сам.
- `NSFaceIDUsageDescription` обязателен: без него Face ID приложению недоступен.
- Экран блокировки защищает от случайного человека, а не от взлома. Настоящая защита — Keychain с `SecAccessControl` и `.biometryCurrentSet`.
- Симулятор: Features → Face ID → Enrolled / Matching Face / Non-matching Face.

Apple Developer Documentation

- Human Interface Guidelines — Privacy — хранить чувствительное в Keychain, не изобретать свои схемы проверки, предпочитать системные (Face ID, Touch ID, passkeys).

- Preparing your UI to run in the background — когда UIKit делает снимок для переключателя приложений и почему из интерфейса нужно убрать чувствительные данные.
- `UIBlurEffect` — стили размытия; `.systemMaterial` подстраивается под светлую и тёмную тему.
- `UIVisualEffectView` — view с эффектом; кладём её прямо в окно, поверх всех экранов. Подвиды — только в `contentView`, `alpha` — не меньше 1.
- `UIWindowScene` — сцена управляет одним или несколькими окнами приложения; её состояния и уведомления мы слушаем.
- `UIScene.willDeactivateNotification` — сцена перестаёт быть активной: системные окна поверх и уход в фон.
- `UIScene.didActivateNotification` — сцена снова активна.
- `UIApplication.willResignActiveNotification` — то же на уровне всего приложения, без различия сцен.
- `UIApplication.didBecomeActiveNotification` — парное событие для всего приложения.
- `LAContext` — проверка владельца устройства; новый контекст на каждую проверку.
- Logging a user into your app with Face ID or Touch ID — официальный пример: `NSFaceIDUsageDescription`, контекст, политики.
- `LAPolicy.deviceOwnerAuthenticationWithBiometrics` — только Face ID / Touch ID, без запасного код-пароля.
- `LAPolicy.deviceOwnerAuthentication` — биометрия с запасным код-паролем; её берём для экрана блокировки.
- `NSFaceIDUsageDescription` — обязательная строка в Info.plist для Face ID.
- Restricting keychain item accessibility — `SecAccessControl`, `.userPresence` и биометрические флаги для записей Keychain.

Это конец Части II. Дальше — Часть III, mini-приложения: Todo, Notes, Calculator, Weather, Gallery, Music, Chat, Profile, Tab Bar, Layouts, Anatomy. Каждое — отдельная глава с разбором кода.

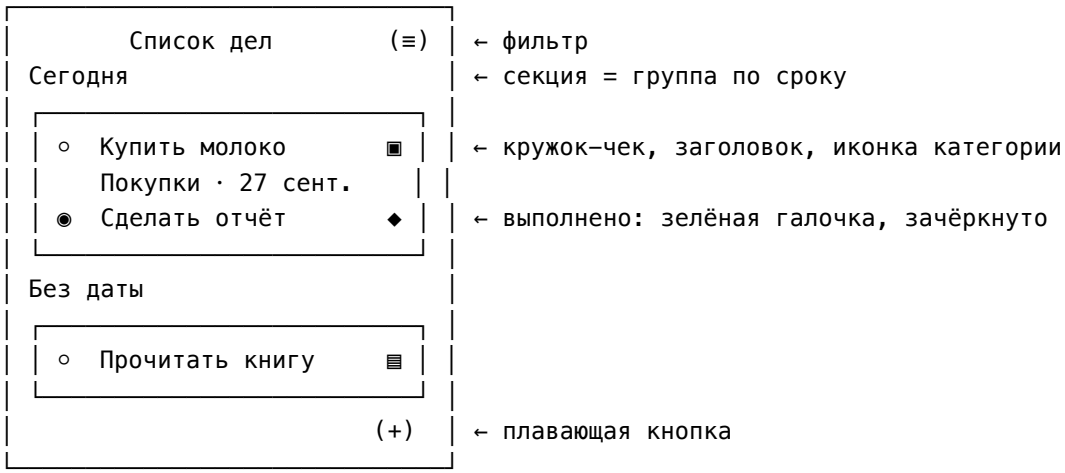
→ Глава 12. Todo — UITableView, ячейка-чек, UserDefaults+Codable, dummyjson

Глава 12. Todo — UITableView, ячейка-чек, UserDefaults+Codable, dummyjson

С этой главы начинается Часть III — **мини-приложения**. Каждое самодостаточно: можно прочитать одну главу, забрать приём в свой проект и не вникать в остальные. Но Todo я советую прочитать целиком — здесь закладываются базовые приёмы (хранилище, своя ячейка, кнопка добавления, лист-редактор), которые повторяются в других мини-приложениях.

«Список дел» — классический учебный проект iOS-разработчика. И не зря: он включает почти всё, что нужно понимать про `UITableView`. Мы делаем его в стиле playground: ячейка с кружком-галочкой, действия по свайпу для удаления и завершения, плавающая кнопка «плюс», секции по датам («Сегодня», «Завтра», «Ближайшие 7 дней»), фильтр «все / активные / завершённые» и заполнение демо-задачами с сервера dummyjson при первом запуске.

Что строим, одним экраном:



Перед началом: проект и режим Swift

Весь код главы собирается в отдельное приложение — так его проще проверить. В Xcode: **File** → **New** → **Project** → **iOS** → **App**, интерфейс **Storyboard**, язык **Swift**. Шаблон Xcode 26 ставит в Build Settings две настройки, о которых надо знать:

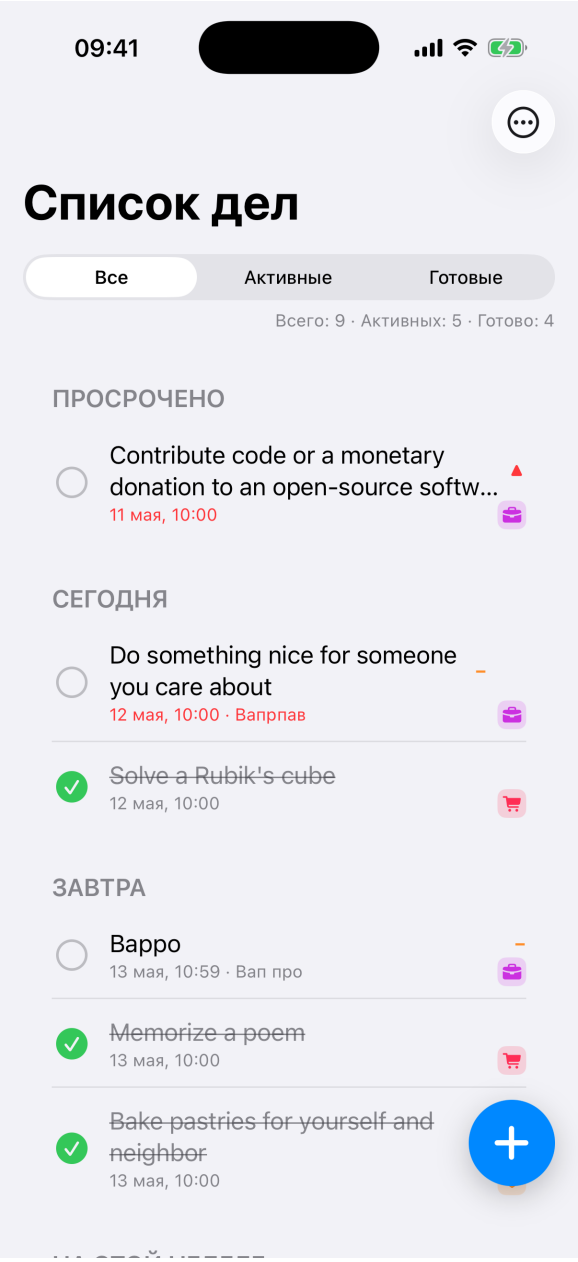


Рис. 8: Список дел с группировкой по датам

- **Swift Language Version = Swift 5**. Код книги проверен в режиме **Swift 6** — переключи эту настройку на Swift 6. В этом режиме компилятор строже проверяет работу с потоками: ошибку «UI трогают не с главного потока» ты увидишь при сборке, а не в отчёте о падении.
- **Default Actor Isolation = MainActor**. Это значит: всё, что ты объявил без пометок, по умолчанию живёт на главном потоке (*main actor* — «исполнитель», который выполняет код строго на главном потоке, по одному куску за раз). Для UIKit это ровно то, что нужно: `UIView` и `UIViewController` всё равно можно трогать только с главного потока.

Третья настройка шаблона, **Approachable Concurrency = Yes**, остаётся включённой. Все три подробно разобраны во введении (шаг 4 «Режим языка»).

Deployment target (минимальная версия iOS, на которой запустится приложение) — **iOS 15.0**. Всё, что появилось позже, в коде помечено проверкой `if #available`.

Шаблон создаёт `Main.storyboard` со своим контроллером. Нам он не нужен: экран соберём кодом. Удали `Main.storyboard` и `ViewController.swift` и убери две ссылки на storyboard: в Info.plist — строку **Storyboard Name** внутри *Application Scene Manifest* → *Scene Configuration* → *Application Session Role* → *Item 0*, а в Build Settings таргета — значение **UIKit Main Storyboard File Base Name** (ключ `INFOPLIST_KEY_UIMainStoryboardFile`). Если забудешь, приложение упадёт при старте с «Could not find a storyboard named 'Main'». `AppDelegate.swift` из шаблона оставь как есть, а содержимое `SceneDelegate.swift` замени нашим. *Сцена* (`UIWindowScene`) — один экземпляр интерфейса приложения: обычно одно окно, формально — одно или несколько (`windowScene.windows`); на iPhone сцена одна, на iPad их может быть несколько. Сцена — не view (`UIWindowScene → UIScene → UIResponder`). `SceneDelegate` получает событие «сцена подключилась» и создаёт окно (`UIWindow`) — подробно об этом в главе 5 и в главе 0, раздел 0.7.

```
import UIKit

final class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }
        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = UINavigationController(
            rootViewController: TodoListViewController()
        )
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

`UINavigationController` — *container view controller* (контейнерный view controller): своего содержимого у него нет, он управляет стопкой дочерних экранов (`viewControllers`) и рисует сверху навигационную панель `UINavigationBar`, в которую выводит `navigationItem` верхнего экрана (глава 0, раздел 0.6). Нам он нужен ради панели: туда встанет кнопка фильтра. `window` мы сохраняем в свойство: сцена окно сильно не держит, и если на него нет ни одной сильной ссылки,

оно освобождается сразу (проверено в симуляторе: слабая ссылка на такое окно становится `nil`), а экран остаётся чёрным.

В нашем playground Todo открывается из лаунчера через манифест (глава 2), а не из `SceneDelegate`. Разница только в том, кто создаёт первый экран; код самого Todo от этого не меняется. Если подключаешь Todo в playground, пиши просто `makeMain: { TodoListViewController() }`: `showMain()` координатора (глава 4, раздел 4.4) сам кладёт main-экран в `UINavigationController`. Обернёшь ещё раз — получишь навигационный контроллер внутри навигационного, а такое вложение UIKit не поддерживает: приложение упадёт с исключением «Pushing a navigation controller is not supported» (проверено в симуляторе iOS 26.5).

12.1 Модель данных

Файл `Todo.swift`. Структура простая, но с двумя нюансами — категория и приоритет:

```
import Foundation

struct Todo: Identifiable, Codable, Hashable, Sendable {
    enum Category: String, Codable, CaseIterable, Sendable {
        case personal = "Личное"
        case work = "Работа"
        case shopping = "Покупки"
        case health = "Здоровье"
        case study = "Учёба"
    }

    enum Priority: Int, Codable, CaseIterable, Sendable {
        case low = 0
        case medium = 1
        case high = 2
    }

    let id: UUID
    var title: String
    var note: String
    var category: Category
    var priority: Priority
    var dueDate: Date?
    var isCompleted: Bool
    let createdAt: Date

    init(id: UUID = UUID(),
         title: String,
         note: String = "",
         category: Category = .personal,
         priority: Priority = .medium,
         dueDate: Date? = nil,
```

```
         isCompleted: Bool = false,
         createdAt: Date = Date()) {
        self.id = id
        self.title = title
        self.note = note
        self.category = category
        self.priority = priority
        self.dueDate = dueDate
        self.isCompleted = isCompleted
        self.createdAt = createdAt
    }
}
```

Четыре протокола в объявлении. Зачем каждый:

- **`Identifiable`** — у задачи есть свойство `id`, которое однозначно её называет. Нам это нужно, чтобы находить задачу в массиве после любых перестановок: позиция в таблице меняется, `id` — никогда. Если позже перейдёшь на *diffable data source* (таблица, которая сама вычисляет разницу между старым и новым списком, см. ссылки в конце главы), `id` станет ключом, по которому она узнаёт «ту же» строку.
- **`Codable`** — задачу можно превратить в JSON и обратно. Для структуры, у которой все поля сами `Codable` (строки, даты, перечисления с `rawValue`), компилятор пишет код кодирования сам. Без этого протокола пришлось бы вручную писать `init(from:)` и `encode(to:)`.
- **`Hashable`** — задачу можно положить в `Set` или сделать ключом словаря. В этой главе это нужно для группировки ниже и пригодится для diffable data source: он требует `Hashable` от элементов.
- **`Sendable`** — «значение можно безопасно передать в другой поток». Для структуры из простых полей Swift выводит это сам, если тип не публичный; явная пометка — документация для читателя и защита: если кто-то добавит в `Todo` небезопасное поле (например, ссылку на изменяемый класс), компилятор сразу скажет.

`init` с значениями по умолчанию. Автоматический инициализатор структуры требовал бы все восемь полей. Свой `init` позволяет писать коротко: `Todo(title: "Купить молоко", category: .shopping)`. Новый `id` генерируется вызовом `UUID()` — это 128-битное случайное число вида `79CB4E81-5A0F-4E3B-…`. Вероятность, что два таких числа совпадут, настолько мала, что на практике ею пренебрегают: не нужен ни сервер, ни счётчик, чтобы выдать задаче уникальный номер.

`dueDate: Date?` — опциональный. Не все задачи привязаны к дате («Когда-нибудь прочитать книгу»). Без даты задача попадает в группу «Без даты».

`let` и `var`. Только `id` и `createdAt` — `let`: они не меняются после создания. Остальные — `var`, потому что пользователь их редактирует.

`rawValue` у категорий — русские названия. Это удобно (подпись в интерфейсе бесплатно), но у решения есть цена: `rawValue` попадает в сохранённый JSON. Переименуешь «Учёба» в «Обучение» — старые данные перестанут читаться. В настоящем приложении `rawValue` делают латинским (`"study"`), а подпись — отдельным свойством. Здесь оставим как есть ради краткости, но помни об этом.

Иконки и цвета категорий — это уже вопрос интерфейса, модели они не касаются. Держим их в отдельном файле-расширении, чтобы `Todo.swift` не импортировал UIKit:

```
import UIKit

extension Todo.Category {
    var symbolName: String {
        switch self {
        case .personal: return "person.fill"
        case .work:     return "briefcase.fill"
        case .shopping: return "cart.fill"
        case .health:   return "heart.fill"
        case .study:    return "book.fill"
        }
    }

    var color: UIColor {
        switch self {
        case .personal: return .systemBlue
        case .work:     return .systemIndigo
        case .shopping: return .systemOrange
        case .health:   return .systemPink
        case .study:    return .systemTeal
        }
    }
}

extension Todo.Priority {
    var title: String {
        switch self {
        case .low:    return "Низкий"
        case .medium: return "Средний"
        case .high:   return "Высокий"
        }
    }
}
```

`symbolName` — имя картинки из *SF Symbols*, встроенной библиотеки иконок Apple (их больше 5000, названия смотри в бесплатном приложении SF Symbols для Mac). `UIImage(systemName: "cart.fill")` достаёт иконку тележки. `.systemBlue`, `.systemPink` — системные цвета: они сами чуть меняют оттенок в тёмной теме, чтобы оставаться читаемыми.

12.2 Группировка по датам

Приём: вычисляемое свойство `group`, которое смотрит на `dueDate` и возвращает одну из шести «корзин»:

```
import Foundation
```

```
extension Todo {
    enum Group: CaseIterable, Hashable, Sendable {
        case overdue, today, tomorrow, thisWeek, later, noDate

        var title: String {
            switch self {
            case .overdue:  return "Просрочено"
            case .today:    return "Сегодня"
            case .tomorrow: return "Завтра"
            case .thisWeek: return "Ближайшие 7 дней"
            case .later:    return "Позже"
            case .noDate:   return "Без даты"
            }
        }

        var sortOrder: Int {
            switch self {
            case .overdue:  return 0
            case .today:    return 1
            case .tomorrow: return 2
            case .thisWeek: return 3
            case .later:    return 4
            case .noDate:   return 5
            }
        }
    }

    var group: Group {
        guard let due = dueDate else { return .noDate }
        let calendar = Calendar.current
        let today = calendar.startOfDay(for: .now)
        let dueDay = calendar.startOfDay(for: due)
        if dueDay < today { return .overdue }
        if dueDay == today { return .today }
        if let tomorrow = calendar.date(byAdding: .day, value: 1, to: today),
           dueDay == tomorrow { return .tomorrow }
        if let weekEnd = calendar.date(byAdding: .day, value: 7, to: today),
           dueDay < weekEnd { return .thisWeek }
        return .later
    }
}
```

`CaseIterable` на `Group` даёт свойство `Group.allCases` — массив всех шести вариантов в порядке объявления. Оно понадобится в списке, чтобы пройти по группам.

`Calendar.current.startOfDay(for:)` — отбрасывает время суток и возвращает полночь того же дня. Зачем: `Date` — это точный момент («27 сентября, 14:00:00»), а нам нужно сравнивать **дни**. Пусть сейчас 27 сентября, 16:00, а срок задачи — 27 сентября, 14:00. Сравнение моментов ска-

жет «срок меньше текущего, просрочено», хотя задача на сегодня. После `startOfDay` обе даты становятся «27 сентября, 00:00», равны — значит, «Сегодня».

Разберём на числах. Пусть сегодня 27 сентября:

- срок 26 сентября → меньше «сегодня» → **Просрочено**;
- срок 27 сентября → равен → **Сегодня**;
- `today + 1 день` = 28 сентября; срок 28 → **Завтра**;
- `today + 7 дней` = 4 октября; срок 29 сентября … 3 октября → **Ближайшие 7 дней** (строго меньше 4 октября);
- срок 4 октября и дальше → **Позже**.

`calendar.date(byAdding: .day, value: 1, to: today)` — «прибавь один календарный день». Почему не `today + 86400` секунд? Потому что не каждый день длится 24 часа: в странах с переходом на летнее время бывают дни по 23 и 25 часов. Календарь это знает, простое сложение секунд — нет. Метод возвращает опционал, потому что теоретически прибавление может не получиться (например, в экзотическом календаре); поэтому `if let`.

`Calendar.current` — календарь и часовой пояс, выбранные на телефоне. «Сегодня» для пользователя в Алматы и в Берлине начинается в разные моменты, и `startOfDay` это учитывает: полночь считается по поясу телефона.

`group` пересчитывается **при каждом обращении**. Для сотни задач это микросекунды. На десятках тысяч стоило бы кешировать, но это не наш случай.

В таблице мы потом разложим массив по этому свойству:

```
let grouped = Dictionary(grouping: storage.items, by: \.group)
```

и отсортируем секции по `sortOrder` (просроченное наверху, «без даты» внизу).

> **`Dictionary(grouping:by:)`** — инициализатор словаря из стандартной библиотеки Swift. Принимает массив и правило (замыкание или key path — «путь к свойству», `\.group`), возвращает `[Ключ: [Элемент]]`. Из `[молоко(сегодня), отчёт(завтра), хлеб(сегодня)]` получится `[.today: [молоко, хлеб], .tomorrow: [отчёт]]`. Порядок внутри каждой группы сохраняется.

12.3 Хранилище — UserDefaults через JSON

UserDefaults — простое хранилище «ключ → значение», которое iOS держит для каждого приложения в отдельном файле (plist) внутри его *песочницы* (sandbox — личной папки приложения, куда другие приложения доступа не имеют). Туда удобно класть настройки и небольшие данные. Строки, числа и `Bool` он хранит напрямую, а массив наших структур — нет. Поэтому превращаем задачи в JSON (`Data`) и кладём уже байты.

`TodoStorage` — единственный экземпляр на приложение (*синглтон*, `static let shared`), который держит массив задач и сохраняет его:

```
import Foundation

@MainActor
final class TodoStorage {
```

```
    static let shared = TodoStorage()

    private let defaults: UserDefaults
    private let key = "todo.items.v1"
    private let firstLaunchKey = "todo.firstLaunchSeeded.v1"

    private(set) var items: [Todo] = []
    private var observers: [ObjectIdentifier: Observer] = [:]

    init(defaults: UserDefaults = .standard) {
        self.defaults = defaults
        load()
    }

    private func save() {
        do {
            let encoder = JSONEncoder()
            encoder.dateEncodingStrategy = .iso8601
            let data = try encoder.encode(items)
            defaults.set(data, forKey: key)
        } catch {
            assertionFailure("TodoStorage: не удалось закодировать задачи: \(error)")
        }
        notify()
    }

    private func load() {
        guard let data = defaults.data(forKey: key) else { return }
        do {
            let decoder = JSONDecoder()
            decoder.dateDecodingStrategy = .iso8601
            items = try decoder.decode([Todo].self, from: data)
        } catch {
            items = []
        }
    }
}
```

Разберём по строкам.

`private(set) var items` — читать массив может кто угодно, менять — только сам `TodoStorage`. Так никто снаружи не сделает `storage.items.append(...)` в обход сохранения на диск.

Зачем `v1` в ключе. Если завтра поменяешь модель `Todo` (добавишь обязательное поле `assignee`), `JSONDecoder` не сможет прочитать старые данные — поля `assignee` в них нет, и `decode` бросит ошибку. У нас `catch` в `load()` в этом случае тихо начнёт с пустого списка — то есть пользователь потеряет задачи. Версия в ключе даёт место для миграции: новая версия приложения пишет в `todo.items.v2`, а при первом запуске читает старый `v1`, переводит в новый формат и сохраняет.

Сама миграция в этой главе не нужна, но ключ с версией стоит ничего, а переименовывать ключ задним числом — больно.

`init` принимает `defaults`. По умолчанию `.standard` — обычное хранилище приложения. Но можно передать другой `UserDefaults`: например, `UserDefaults(suiteName: "group.kz.example.todo")` — общий для приложения и его виджета (для этого нужна настройка App Groups в Signing & Capabilities). Или отдельный набор для тестов: `UserDefaults(suiteName: "tests")`, чтобы тесты не портили настоящие данные. Передача зависимости через параметр называется *dependency injection* («внедрение зависимости»): объект не сам решает, с чем работать, ему это дают снаружи.

`@MainActor` на классе. В проекте с Default Actor Isolation = MainActor пометка лишняя — класс и так на главном потоке. Мы всё равно пишем её явно: читающему сразу видно, что хранилище рассчитано на главный поток, и код не сломается, если его скопируют в проект без этой настройки. Почему главный поток: сам `UserDefaults` потокобезопасен (так написано в документации Apple), но наше хранилище после каждого изменения зовёт подписчиков (`notify()`), а они перерисовывают таблицу. Таблицу можно трогать только с главного потока — проще закрепить за ним весь класс, и компилятор сам не даст позвать его из фона.

`dateEncodingStrategy = .iso8601`. В JSON нет типа «дата». По умолчанию `JSONEncoder` пишет `Date` числом — количеством секунд от 1 января **2001** года (это «эпоха» Apple, не путать с Unix-эпохой 1970 года). Момент 27 сентября 2026, 10:00 UTC станет числом `812196000`. С `.iso8601` получится строка `"2026-09-27T10:00:00Z"`, которую понимает любой сервер и любой язык. Обрати внимание: `.iso8601` сохраняет время с точностью до секунды, доли секунды пропадают. Для списка дел это неважно.

Сами байты в UserDefaults лежат блоком `Data`, а не текстом: если открыть plist приложения, увидишь закодированные байты, а не читаемый JSON. Читаемость ISO-строк пригодится, когда ты захочешь выгрузить задачи на сервер или отладить их, напечатав JSON в консоль.

`assertionFailure` в `catch`. Закодировать массив из наших простых типов практически невозможно не суметь, поэтому ошибка здесь — баг программиста. `assertionFailure` остановит приложение в отладке (ты сразу увидишь проблему), а в релизной сборке ничего не сделает. Молча проглатывать такую ошибку пустым `catch {}` — плохая привычка.

Сколько можно хранить в UserDefaults. Жёсткого лимита на iOS Apple не называет, но файл целиком читается в память при старте и переписывается при изменениях. Сотня задач — это килобайты, нормально. Мегабайты данных (фото, большие документы) туда не кладут — для них файлы, как в главе 13 (Notes).

> **Privacy manifest.** `UserDefaults` входит в список API, для которых Apple требует указать причину использования в файле `PrivacyInfo.xcprivacy` (privacy manifest — «декларация» того, какие данные и системные API использует приложение). Для хранения своих данных подходит причина `CA92.1`. Без этого App Store Connect не примет сборку. Подробно — в главе 39.

Теперь операции, которыми пользуется экран. Их кладём в расширение того же класса:

```
extension TodoStorage {
    func add(_ todo: Todo) {
        items.append(todo)
        save()
```

```
    }

    func update(_ todo: Todo) {
        guard let index = items.firstIndex(where: { $0.id == todo.id }) else { return }
        items[index] = todo
        save()
    }

    func remove(id: UUID) {
        items.removeAll { $0.id == id }
        save()
    }

    func toggleCompleted(id: UUID) {
        guard let index = items.firstIndex(where: { $0.id == id }) else { return }
        items[index].isCompleted.toggle()
        save()
    }
}
```

Все операции принимают `id`, а не номер строки в таблице. Номер строки — это позиция в **отсортированном и сгруппированном** виде, она не совпадает с позицией в `items`. `id` один и тот же везде.

`firstIndex(where:)` — «первый индекс, для которого условие истинно». Если задачи с таким `id` нет (её уже удалили), `guard` тихо выходит — это нормально, например, если пользователь быстро нажал два раза.

Каждая операция заканчивается `save()`, а `save()` — `notify()`. Так невозможно изменить данные и забыть сохранить или перерисовать экран.

12.4 Наблюдатели — чтобы перерисовать экран

Когда модель изменилась, экран надо обновить. Вариантов несколько:

1. **`NotificationCenter`** — рассылать уведомление `Notification.Name("todosChanged")`. Работает, но тип данных в уведомлении не проверяется компилятором, а подписку легко забыть снять.
2. **Combine и `@Published`** — современно, но требует знания Combine и хранения подписок (`AnyCancellable`) в каждом подписчике.
3. **Макрос `@Observable` (iOS 17+)** — самый чистый, но нам нужна поддержка iOS 15.
4. **Свой список обработчиков.** Простой, понятный, работает везде.

Выбираем 4-й. *Наблюдатель* (observer) — это «кто-то, кто хочет знать об изменениях». Хранилище держит список наблюдателей и после каждого изменения зовёт их по очереди:

```
extension TodoStorage {
    struct Observer {
        weak var owner: AnyObject?
```

```
        let handler: () -> Void
    }

    func addObserver(_ owner: AnyObject, handler: @escaping () -> Void) {
        observers[ObjectIdentifier(owner)] = Observer(owner: owner, handler: handler)
    }

    func removeObserver(_ owner: AnyObject) {
        observers.removeValue(forKey: ObjectIdentifier(owner))
    }

    private func notify() {
        // Выбрасываем тех, чей владелец уже освободился из памяти.
        observers = observers.filter { $0.value.owner != nil }
        for observer in observers.values {
            observer.handler()
        }
    }
}
```

Здесь три решения, и у каждого своя причина.

Ключ — `ObjectIdentifier(owner)`. Это «адрес» объекта в памяти, завёрнутый в тип, который можно сделать ключом словаря. Один и тот же контроллер всегда даёт один и тот же ключ. Если он подпишется второй раз, новый обработчик заменит старый, а не встанет рядом дубликатом. А два **разных** экземпляра одного класса получат разные ключи — в отличие от строкового ключа вроде `"TodoListVC"`, где второй экран молча выкинул бы подписку первого.

`weak var owner`. Хранилище живёт всё время работы приложения, а экран — нет. Если бы хранилище держало экран сильной ссылкой, экран никогда бы не освободился. `weak` — «смотрю, но не держу»: когда экран закроют, `owner` сам станет `nil`, и `notify()` выбросит эту запись.

Почему не отписываться в `deinit`. Классический код выглядит так:

```
deinit {
    TodoStorage.shared.removeObserver(self)   // ошибка компиляции в Swift 6
}
```

В режиме Swift 6 он **не собирается**: «main actor-isolated static property ‘shared’ can not be referenced from a nonisolated context». `deinit` не привязан к главному потоку — объект может освободиться на любом потоке, — а хранилище закреплено за главным. Можно было бы написать `isolated deinit`, но эта возможность требует iOS 18.4. Слабая ссылка на владельца решает задачу без `deinit` вообще: отписываться не нужно, мёртвые подписки убираются сами. `removeObserver` оставлен на случай, когда экран хочет перестать слушать, продолжая жить.

Подписчик — контроллер списка, в `viewDidLoad`:

```
storage.addObserver(self) { [weak self] in
    self?.reloadGroups()
}
```

`[weak self]` в замыкании обязателен по той же причине: замыкание хранится в хранилище, и сильный `self` внутри него снова держал бы экран вечно. Слабая ссылка на владельца в `Observer` не спасла бы — её бы «подпирало» само замыкание.

12.5 Первое заполнение из dummyjson

Когда пользователь впервые открывает Todo, список пустой — видно только надпись «Задач нет». Для учебного приложения полезнее **заполнить** его при первом запуске несколькими демо-задачами с сервера (по-английски это называют *seeding*, «посев»).

Источник — `https://dummyjson.com/todos`, публичный тестовый API без ключа и регистрации. Проверь сам в терминале:

```
curl "https://dummyjson.com/todos?limit=2"
```

Ответ (сокращён до двух задач):

```
{"todos":[{"id":1,"todo":"Do something nice for someone you care about",
           "completed":false,"userId":152},
          {"id":2,"todo":"Memorize a poem","completed":true,"userId":13}],
 "total":254,"skip":0,"limit":2}
```

Задачи там на английском — это нормально для демо. Клиент:

```
import Foundation

struct TodoAPI {
    enum APIError: Error {
        case badResponse
    }

    private struct RemoteTodo: Decodable {
        let id: Int
        let todo: String
        let completed: Bool
    }

    private struct RemoteTodoListResponse: Decodable {
        let todos: [RemoteTodo]
    }

    private let baseURL = URL(string: "https://dummyjson.com")!
    private let session: URLSession

    init(session: URLSession = .shared) {
        self.session = session
    }

    func fetchSampleTodos(limit: Int = 8) async throws -> [Todo] {
        var components = URLComponents(
```

```
            url: baseURL.appendingPathComponent("todos"),
            resolvingAgainstBaseURL: false
        )
        components?.queryItems = [URLQueryItem(name: "limit", value: String(limit))]
        guard let url = components?.url else { throw APIError.badResponse }

        var request = URLRequest(url: url)
        request.timeoutInterval = 10

        let (data, response) = try await session.data(for: request)
        guard let http = response as? HTTPURLResponse,
              (200..<300).contains(http.statusCode) else {
            throw APIError.badResponse
        }
        let decoded = try JSONDecoder().decode(RemoteTodoListResponse.self, from: data)

        let categories = Todo.Category.allCases
        let priorities = Todo.Priority.allCases
        let today = Calendar.current.startOfDay(for: .now)
        return decoded.todos.map { remote in
            // Остаток от деления id: одинаковый id всегда даёт одинаковый результат.
            let dayOffset: Int? = remote.id % 4 == 3 ? nil : remote.id % 4
            return Todo(
                title: remote.todo,
                category: categories[remote.id % categories.count],
                priority: priorities[remote.id % priorities.count],
                dueDate: dayOffset.flatMap {
                    Calendar.current.date(byAdding: .day, value: $0, to: today)
                },
                isCompleted: remote.completed
            )
        }
    }
}
```

Две модели. `RemoteTodo` зеркалит JSON сервера: поля `id`, `todo`, `completed` (поле `userId` нам не нужно — `Decodable` просто пропускает лишние ключи). Наш `Todo` устроен иначе: другие имена, `UUID` вместо `Int`, категории, даты. Держать две модели и переводить одну в другую — нормальная практика: сервер может поменять формат, а наш код изменится только в этом файле. Обе вложены в `TodoAPI` как `private` — снаружи о них никто не знает.

`URLComponents` собирает адрес из частей и сам экранирует параметры (заменяет пробелы и кириллицу на `%20`, `%D0%…`). Склеивать URL строками опасно: `"?q=" + "молоко и хлеб"` даст неверный адрес.

`try await session.data(for:)` — асинхронный запрос (iOS 15+). `await` означает «здесь функция может приостановиться, пока ждёт сеть». Главный поток при этом не блокируется: пока идёт запрос, интерфейс продолжает реагировать на пальцы. Когда ответ придёт, выполнение про-

должится со строки ниже. `timeoutInterval = 10` — если сервер молчит больше 10 секунд, запрос завершится ошибкой.

Проверка статуса. `URLSession` не считает ответ 404 или 500 ошибкой — для неё это успешно полученный ответ. Проверять код (`200..<300` — «успех» в HTTP) приходится самим.

Раскладка по категориям через остаток от деления. `remote.id % 5` — остаток от деления `id` на 5, всегда число от 0 до 4. Задача с `id = 7` получит `7 % 5 = 2` → третью категорию, «Покупки», при каждом запуске. Если бы мы выбирали категорию случайно, каждый первый запуск выглядел бы по-разному, и баг «на моём телефоне задача не в той секции» было бы трудно повторить. Тот же приём для приоритета (`id % 3`) и для срока: `id % 4` даёт 0, 1, 2 или 3 — первые три превращаем в «сегодня», «завтра», «послезавтра», а 3 — в «без даты».

Где это работает. `TodoAPI` в нашем режиме изолирован на главном потоке, как и всё остальное. Сетевое ожидание от этого не страдает — `await` отпускает поток. Разбор JSON из восьми задач занимает доли миллисекунды, так что делать его в фоне нет смысла.

Теперь хранилище. Заполняем **один раз**, и запоминаем это флагом:

```
extension TodoStorage {
    var needsFirstLaunchSeed: Bool {
        !defaults.bool(forKey: firstLaunchKey) && items.isEmpty
    }

    func seedIfNeeded(with remoteTodos: [Todo]) {
        guard needsFirstLaunchSeed else { return }
        items = remoteTodos
        defaults.set(true, forKey: firstLaunchKey)
        save()
    }
}
```

Условие двойное: флаг **не** установлен **и** список пустой. Флаг защищает от повторного заполнения: если пользователь сам удалил все задачи, демо-задачи не вернутся — у него была причина очистить список. Проверка `items.isEmpty` защищает от гонки: пока шёл запрос, человек мог успеть добавить свою задачу, и затирать её демо-данными нельзя.

Флаг ставится только при **успешном** заполнении. Если первый запуск был без интернета, запрос упадёт, флаг не встанет, и попытка повторится при новом запуске.

12.6 TodoListViewController — список с секциями

Главный экран. Начнём со свойств и `viewDidLoad`:

```
import UIKit

final class TodoListViewController: UIViewController {

    enum Filter: CaseIterable {
        case all, active, completed
```

```
        var title: String {
            switch self {
            case .all:       return "Все"
            case .active:    return "Активные"
            case .completed: return "Завершённые"
            }
        }

        func includes(_ todo: Todo) -> Bool {
            switch self {
            case .all:       return true
            case .active:    return !todo.isCompleted
            case .completed: return todo.isCompleted
            }
        }
    }

    let storage: TodoStorage
    let api: TodoAPI

    let tableView = UITableView(frame: .zero, style: .insetGrouped)
    let emptyLabel = UILabel()

    var filter: Filter = .all
    var sections: [Todo.Group] = []
    var sectionItems: [[Todo]] = []

    init(storage: TodoStorage = .shared, api: TodoAPI = TodoAPI()) {
        self.storage = storage
        self.api = api
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Список дел"
        view.backgroundColor = .systemGroupedBackground
        setupTable()
        setupAddButton()
        updateFilterMenu()

        storage.addObserver(self) { [weak self] in
            self?.reloadGroups()
        }
```

```
        reloadGroups()
        seedIfNeeded()
    }
}
```

`UIViewController` — «контроллер экрана»: объект, который владеет одним экраном (его корневым `view`), создаёт на нём элементы и реагирует на события. Сам он не view (`UIViewController → UIResponder`), а его `view` создаётся лениво — при первом обращении к свойству (до этого `isViewLoaded == false`). `viewDidLoad` вызывается один раз, сразу после того как `view` создана, — место для настройки иерархии и констрейнтов. Но размеры здесь ещё не окончательные: view пока не в окне (`view.window == nil`), safe area и настоящие рамки известны только начиная с `viewDidLayoutSubviews`. Поэтому в `viewDidLoad` мы описываем правила раскладки, а не считаем точки. Подробно — глава 0, раздел 0.6.

`UITableView(frame: .zero, style: .insetGrouped)` — таблица в стиле «Настроек»: секции-карточки со скруглёнными углами и отступом от краёв. `frame: .zero` — размер пока нулевой, настоящий зададут *констрейнты* (constraints — правила вида «верх таблицы = верх экрана», по которым система *Auto Layout* сама вычисляет размеры и положение).

Хранилище и API приходят через `init` с значениями по умолчанию — тот же приём внедрения зависимостей, что в 12.3: в тестах можно подсунуть хранилище с отдельным `UserDefaults`. Если оставишь в проекте Swift 5, на `storage: TodoStorage = .shared` компилятор выдаст предупреждение «main actor-isolated static property ‘shared’ can not be referenced from a nonisolated context»: в режиме Swift 5 значения по умолчанию вычисляются вне главного потока. В Swift 6 это правило смягчили (значение по умолчанию вычисляется там же, где изолирован инициализатор), и предупреждения нет — ещё одна причина переключиться.

`required init?(coder:)` — инициализатор для создания из storyboard. Мы экран из storyboard не создаём, а Swift всё равно требует его объявить; `fatalError` честно говорит «так не бывает».

Свойства и методы экрана в листингах — без `private`, чтобы код можно было показать частями в нескольких расширениях. Расширения в том же файле видят и `private`-члены, так что в своём проекте смело помечай всё внутреннее `private` — соберётся так же.

Разметка — таблица на весь экран и надпись для пустого списка:

```
extension TodoListViewController {
    func setupTable() {
        tableView.translatesAutoresizingMaskIntoConstraints = false
        tableView.dataSource = self
        tableView.delegate = self
        tableView.register(TodoCell.self, forCellReuseIdentifier: TodoCell.reuseID)
        // Запас снизу, чтобы последняя строка не пряталась под круглой кнопкой.
        tableView.contentInset.bottom = 88
        view.addSubview(tableView)

        emptyLabel.text = "Задач нет.\nНажми «+», чтобы добавить первую."
        emptyLabel.numberOfLines = 0
        emptyLabel.textAlignment = .center
        emptyLabel.textColor = .secondaryLabel
```

```
        emptyLabel.font = .preferredFont(forTextStyle: .body)
        emptyLabel.adjustsFontForContentSizeCategory = true

        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            tableView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        ])
    }
}
```

`translatesAutoresizingMaskIntoConstraints = false` — обязательная строка для каждого view, которое ты расставляешь констрейнтами. У любой view, созданной кодом, это свойство по умолчанию `true`: UIKit превращает её `frame` и `autoresizingMask` в констрейнты (у нас — из нулевого `frame`). Эти констрейнты обязательные, как и наши, и они начнут конфликтовать: в консоли появится «Unable to simultaneously satisfy constraints», Auto Layout разорвёт одно из правил — какое, заранее не скажешь, и раскладка станет непредсказуемой. Трогать это свойство нужно только у своих view: у `view` контроллера, ячеек и `arrangedSubviews` стека им управляет UIKit (документация `translatesAutoresizingMaskIntoConstraints`; подробно — глава 0, раздел 0.3).

`dataSource` и `delegate` — два помощника таблицы. *Data source* («источник данных») отвечает на вопросы «сколько секций и строк» и «какую ячейку показать в строке N». *Delegate* («делегат») получает события: тап по строке, свайп. Оба — наш контроллер; сами методы ниже. Обе ссылки у таблицы **слабые** (в заголовке UITableView.h — `@property (nonatomic, weak)`, в документации — «the data source isn't retained»): таблица контроллер не удерживает, он должен жить сам — у нас его держит навигационный контроллер. Протокол `UITableViewDelegate` наследует `UIScrollViewDelegate`, поэтому через тот же делегат приходят и события прокрутки (глава 0, раздел 0.9).

`register(_:forCellReuseIdentifier:)` — говорим таблице, какой класс создавать для ячеек с идентификатором `"TodoCell"`. Таблица не создаёт ячейку на каждую задачу: на экране помещается строк десять, и таблица держит примерно столько ячеек, *переиспользуя* их — ячейка, уехавшая за верхний край, возвращается снизу с новыми данными. Поэтому ячейка — это только «рамка», которую каждый раз заново заполняют. Сама ячейка — `UIView` (`UITableViewCell → UIView`), не контрол: нажатия на строку обрабатывает таблица, которая, в свою очередь, — `UIScrollView` (глава 0, раздел 0.5).

`contentInset.bottom = 88` — дополнительная прокрутка снизу на 88 точек (*точка*, point — единица размеров в iOS; на экране @3x одна точка — это 3×3 физических пикселя). Кнопка «плюс» высотой около 58 точек (12.9) стоит в 20 точках от низа: 58 + 20 = 78, плюс небольшой запас — 88. Без этого последнюю задачу нельзя было бы прокрутить выше кнопки.

`preferredFont(forTextStyle: .body)` + `adjustsFontForContentSizeCategory = true` — поддержка *Dynamic Type*: размер шрифта берётся из системной настройки «Размер текста», и меняется на лету, если пользователь её подвинет. Если ставить фиксированный `.systemFont(ofSize: 17)`, людям с плохим зрением будет нечем помочь.

`backgroundView` таблицы — view, которое она рисует под всеми строками, заголовками и подвалами секций. Туда удобно класть надпись «пусто»: таблица сама растягивает эту view до своих `bounds`, и она не прокручивается вместе с содержимым.

Данные — самое интересное: связывание секций и задач.

```
extension TodoListViewController {
    func reloadGroups() {
        let visible = storage.items.filter(filter.includes)
        let grouped = Dictionary(grouping: visible, by: \.group)
        sections = Todo.Group.allCases
            .filter { grouped[$0] != nil }
            .sorted { $0.sortOrder < $1.sortOrder }
        sectionItems = sections.map { group in
            (grouped[group] ?? []).sorted(by: Self.todoSorter)
        }
        tableView.reloadData()
        tableView.backgroundView = visible.isEmpty ? emptyLabel : nil
    }

    static func todoSorter(_ a: Todo, _ b: Todo) -> Bool {
        if a.isCompleted != b.isCompleted { return !a.isCompleted }
        if a.priority != b.priority { return a.priority.rawValue > b.priority.rawValue }
        return a.createdAt > b.createdAt
    }

    func todoAt(_ indexPath: IndexPath) -> Todo {
        sectionItems[indexPath.section][indexPath.row]
    }

    func seedIfNeeded() {
        guard storage.needsFirstLaunchSeed else { return }
        Task { [weak self, api] in
            guard let todos = try? await api.fetchSampleTodos() else { return }
            self?.storage.seedIfNeeded(with: todos)
        }
    }
}
```

Что происходит в `reloadGroups()`:

1. **Фильтруем** по выбранному фильтру. `filter(filter.includes)` — передаём метод как функцию-условие, без лишнего замыкания.
2. **Группируем** по `\.group` (свойство из 12.2). Получаем `[Todo.Group: [Todo]]`.
3. **Выбираем секции** — только те группы, где есть задачи, в порядке `sortOrder`. Пустую секцию «Завтра» без строк показывать незачем. `sections` и `sectionItems` — два параллельных массива: секция номер 2 — это `sections[2]`, её задачи — `sectionItems[2]`.
4. **Сортируем внутри секции** `todoSorter`: сначала невыполненные, потом по приоритету (высокий выше), потом новые выше.

5. **`reloadData()`** — таблица заново спросит data source обо всём и перерисует видимые строки.

Разберём `todoSorter` на примере. Функция отвечает на вопрос «должна ли `a` стоять выше `b`». Пусть `a` — выполненная, `b` — нет: первое условие сработает (`isCompleted` разные) и вернёт `!a.isCompleted`, то есть `false` — `a` ниже. Если обе невыполненные, но у `a` приоритет высокий (2), а у `b` средний (1): `2 > 1` → `true`, `a` выше. Если и приоритеты равны — сравниваем даты создания.

`todoSorter` — `static`: функция не трогает свойства экземпляра, ей нужны только две задачи. Такую функцию можно передать в `sorted(by:)` просто по имени: `Self.todoSorter`.

`todoAt(_:)` — одна точка, где номер секции и строки (`IndexPath` — пара «секция, строка») превращаются в задачу. Все методы таблицы пользуются ею.

Загрузка демо-задач. `Task { … }` запускает асинхронную работу из обычного (синхронного) метода. Список захвата `[weak self, api]`: `self` — слабо (если экран закроют раньше, чем придёт ответ, держать его незачем), `api` — копией (это структура). `try?` превращает ошибку сети в `nil`: для демо-данных молча промолчать — нормально, пользователь просто увидит пустой список. Результат кладём в хранилище, оно позовёт наблюдателей, и таблица обновится сама.

Протоколы таблицы — сколько секций, строк, какой заголовок и какая ячейка:

```
extension TodoListViewController: UITableViewDataSource {
    func numberOfSections(in tableView: UITableView) -> Int {
        sections.count
    }

    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        sectionItems[section].count
    }

    func tableView(_ tableView: UITableView, titleForHeaderInSection section: Int) ->
    ↪ String? {
        sections[section].title
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let cell = tableView.dequeueReusableCell(
            withIdentifier: TodoCell.reuseID, for: indexPath
        ) as! TodoCell
        let todo = todoAt(indexPath)
        cell.configure(with: todo)
        let id = todo.id
        cell.onCheckTap = { [weak self] in
            self?.storage.toggleCompleted(id: id)
        }
        return cell
    }
}
```

`dequeueReusableCell(withIdentifier:for:)` — «дай ячейку для переиспользования»: либо уехавшую с экрана (тогда таблица перед выдачей зовёт у неё `prepareForReuse()`), либо новую — через `init(style:reuseIdentifier:)` зарегистрированного класса. Вариант с `for: indexPath` всегда возвращает ячейку, но вызывать его можно только из `cellForRowAt`. `as! TodoCell` — принудительное приведение типа. Обычно `as!` — повод насторожиться, но здесь это осознанно: мы сами зарегистрировали под этим идентификатором `TodoCell`, и другой класс там оказаться не может. Если всё же окажется (опечатка в идентификаторе), лучше упасть сразу в отладке, чем показать пустую строку.

Зачем замыкание `onCheckTap` и что оно захватывает — в разделе 12.7.

> **Упражнение 12.1.** Добавь в `reloadGroups()` подсчёт невыполненных задач и выводи его в заголовке экрана: «Список дел (3)». Проверь: отметь задачу галочкой — число уменьшится на 1.

12.7 Своя ячейка — `TodoCell`

`UITableViewCell` — строка таблицы: обычная view (`UITableViewCell → UIView → UIResponder`) со служебными подвидами, которые таблица добавляет сама, — `contentView` для содержимого, фон (`backgroundView`, `selectedBackgroundView`), аксессуар и разделитель. В симуляторе у ячейки с галочкой `subviews` — это `UITableViewCellContentView`, `_UITableCellAccessoryButton`, `_UITableViewCellSeparatorView`. Своя ячейка — подкласс с собственной разметкой внутри `contentView`: кружок-чек (`UIButton`), заголовок, подпись (категория, приоритет, дата) и иконка категории справа. Подробно про ячейки — глава 0, раздел 0.5.

Главное решение — **чек через `UIButton`, а не `UISwitch`**. Переключатель выглядит тяжело для одной строки списка; кружок, который по тапу становится галочкой, привычнее — так сделано в «Напоминаниях».

```
import UIKit

final class TodoCell: UITableViewCell {
    static let reuseID = "TodoCell"

    var onCheckTap: (() -> Void)?

    let checkButton = UIButton(type: .custom)
    let titleLabel = UILabel()
    let detailLabel = UILabel()
    let categoryIcon = UIImageView()

    static let dateFormatter: DateFormatter = {
        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.setLocalizedDateFormatFromTemplate("d MMM")
        return formatter
    }()

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
```

```
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        setupLayout()
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func prepareForReuse() {
        super.prepareForReuse()
        onCheckTap = nil
    }

    func configure(with todo: Todo) {
        checkButton.isSelected = todo.isCompleted
        checkButton.tintColor = todo.isCompleted ? .systemGreen : .tertiaryLabel
        checkButton.accessibilityLabel = todo.isCompleted
            ? "Отметить невыполненной" : "Отметить выполненной"

        let attributes: [NSAttributedString.Key: Any] = todo.isCompleted
            ? [.strikethroughStyle: NSUnderlineStyle.single.rawValue,
               .foregroundColor: UIColor.secondaryLabel]
            : [.foregroundColor: UIColor.label]
        titleLabel.attributedText = NSAttributedString(string: todo.title, attributes:
↪   attributes)

        var parts = [todo.category.rawValue, "приоритет:
        ↪  \(todo.priority.title.lowercased())"]
        if let due = todo.dueDate {
            parts.append(Self.dateFormatter.string(from: due))
        }
        detailLabel.text = parts.joined(separator: " · ")

        categoryIcon.image = UIImage(systemName: todo.category.symbolName)
        categoryIcon.tintColor = todo.category.color
    }

    @objc func checkboxTapped() {
        onCheckTap?()
    }
}
```

`UIButton(type: .custom)`, а не `.system`. Это настоящая ловушка. Кнопка типа `.system` в выбранном состоянии (`isSelected = true`) рисует под картинкой залитую подложку цвета `tintColor`. Выполненная задача показала бы галочку в **зелёном квадрате** вместо аккуратной зелёной галочки — именно так выглядела первая сборка этой главы. Тип `.custom` ничего не добавляет от себя: только наши картинки для `.normal` и `.selected`.

`configure(with:)` заполняет ячейку данными. Он должен выставлять **каждое** свойство, которое зависит от задачи, в обе стороны: не только «зачеркнуть, если выполнено», но и «не зачёркивать, если нет». Ячейка переиспользуется: если бы мы только зачёркивали, строка с выполненной задачей, уехавшая наверх и вернувшаяся снизу с новой задачей, так и осталась бы зачёркнутой.

`prepareForReuse()` — таблица вызывает его прямо перед тем, как вернуть ячейку из `dequeueReusableCell` для новой строки. Только для ячейки из запаса: у только что созданной его нет. Apple советует сбрасывать здесь то, что не относится к содержимому (прозрачность, выделение, режим редактирования), а всё содержимое выставлять заново в `cellForRowAt` — что и делает `configure`. Мы обнуляем замыкание: старое относилось к другой задаче. `configure` всё равно поставит новое, но обнулять в `prepareForReuse` — хорошая привычка: если когда-нибудь забудешь поставить замыкание, тап не отметит чужую задачу. Переопределяя метод, обязательно зови `super`.

Зачёркивание делаем через `NSAttributedString` — строку с атрибутами: `.strikethroughStyle` (линия поверх текста) и цвет `.secondaryLabel` (приглушённый серый).

`DateFormatter` создаём один раз, `static let`: создание форматтера — сравнительно дорогая операция, а `cellForRowAt` вызывается на каждую строку при прокрутке. `setLocalizedDateFormatFromTemplate("d MMM")` просит «день и сокращённый месяц» и сам расставляет их по правилам локали: для русской получится «27 сент.».

`accessibilityLabel` — то, что прочитает *VoiceOver* (экранный диктор для незрячих пользователей). Без подписи кнопка с одной картинкой прозвучит как «кнопка» — непонятно, что она делает.

Разметка ячейки:

```
extension TodoCell {
    func setupLayout() {
        let symbolConfig = UIImage.SymbolConfiguration(pointSize: 22, weight: .regular)
        checkButton.setImage(UIImage(systemName: "circle"), for: .normal)
        checkButton.setImage(UIImage(systemName: "checkmark.circle.fill"), for: .selected)
        checkButton.setPreferredSymbolConfiguration(symbolConfig, forImageIn: .normal)
        checkButton.setPreferredSymbolConfiguration(symbolConfig, forImageIn: .selected)
        checkButton.addTarget(self, action: #selector(checkboxTapped), for:
↪   .touchUpInside)

        titleLabel.font = .preferredFont(forTextStyle: .body)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.numberOfLines = 0

        detailLabel.font = .preferredFont(forTextStyle: .footnote)
        detailLabel.adjustsFontForContentSizeCategory = true
        detailLabel.textColor = .secondaryLabel
        detailLabel.numberOfLines = 0

        categoryIcon.contentMode = .scaleAspectFit
        categoryIcon.preferredSymbolConfiguration =
            UIImage.SymbolConfiguration(textStyle: .footnote)
```

```
        let textStack = UIStackView(arrangedSubviews: [titleLabel, detailLabel])
        textStack.axis = .vertical
        textStack.spacing = 2

        [checkButton, textStack, categoryIcon].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            contentView.addSubview($0)
        }

        NSLayoutConstraint.activate([
            checkButton.leadingAnchor.constraint(equalTo:
↪   contentView.layoutMarginsGuide.leadingAnchor),
            checkButton.centerYAnchor.constraint(equalTo: contentView.centerYAnchor),
            checkButton.widthAnchor.constraint(equalToConstant: 44),
            checkButton.heightAnchor.constraint(equalToConstant: 44),
            checkButton.topAnchor.constraint(greaterThanOrEqualTo: contentView.topAnchor,
↪   constant: 4),
            checkButton.bottomAnchor.constraint(lessThanOrEqualTo:
↪   contentView.bottomAnchor, constant: -4),

            textStack.leadingAnchor.constraint(equalTo: checkButton.trailingAnchor,
↪   constant: 8),
            textStack.topAnchor.constraint(equalTo:
↪   contentView.layoutMarginsGuide.topAnchor),
            textStack.bottomAnchor.constraint(equalTo:
↪   contentView.layoutMarginsGuide.bottomAnchor),

            categoryIcon.leadingAnchor.constraint(equalTo: textStack.trailingAnchor,
↪   constant: 8),
            categoryIcon.trailingAnchor.constraint(equalTo:
↪   contentView.layoutMarginsGuide.trailingAnchor),
            categoryIcon.centerYAnchor.constraint(equalTo: contentView.centerYAnchor),
            categoryIcon.widthAnchor.constraint(equalToConstant: 20),
        ])
    }
}
```

Картинки для состояний. `setImage(_:for:)` задаёт картинку для каждого состояния кнопки: `.normal` — пустой кружок, `.selected` — кружок с галочкой. Дальше достаточно менять `isSelected`, кнопка сама покажет нужную. `setPreferredSymbolConfiguration` задаёт размер символа (22 точки) — тоже для каждого состояния отдельно; если задать только для `.normal`, выбранная галочка нарисуется в размере по умолчанию и строка «прыгнет».

`addTarget(_:action:for:)` — «когда палец отпущен внутри кнопки (`.touchUpInside`), вызови у `self` метод `checkboxTapped`». Метод помечен `@objc`, потому что этот механизм пришёл из Objective-C и ищет метод по имени.

Всё добавляем в `contentView`, а не в саму ячейку. `contentView` — область содержимого ячейки, её «штатный» superview для твоих view (документация Apple: «custom subviews should be added to the content view»). Ячейка подстраивает размер этой области под свои нужды: в режиме редактирования (`isEditing`) слева появляется кнопка удаления, справа — ручка перестановки, а аксессуар — галочка или стрелка — занимает место у trailing-края (для русского языка — справа). Вместе с `contentView` сдвигается и всё, что в ней лежит. Элементы, добавленные прямо в ячейку, остались бы на месте и наехали на эти системные кнопки.

Кнопка 44×44 точки при значке 22 точки. 44 точки — минимальный размер области нажатия, который рекомендует Apple в Human Interface Guidelines: меньшую цель трудно попасть пальцем. Картинка маленькая, а «мишень» вокруг неё большая.

`layoutMarginsGuide` — рамка внутри ячейки с системными отступами от краёв (*layout margins* — «поля»). Привязываясь к ней, а не к краю, мы получаем те же отступы, что у системных ячеек, в том числе на iPad и в альбомной ориентации.

Высота строки. Мы не задаём её числом. Текстовый стек привязан к верхнему и нижнему полям — значит, высота ячейки = высота текста + поля. Если название задачи длинное и занимает три строки (`numberOfLines = 0` — «сколько угодно строк»), ячейка вырастет сама. Для кнопки — два неравенства «не ближе 4 точек к верху и низу»: строка с одной короткой строчкой текста всё равно будет не ниже 44 + 4 + 4 = 52 точек, чтобы кнопка поместилась.

Это работает, потому что у таблицы `rowHeight` по умолчанию равен `UITableView.automaticDimension` — константе −1 (в симуляторе `UITableView().rowHeight` печатает `-1.0`); то же значение у `estimatedRowHeight`. Таблица не знает высоту заранее и спрашивает её у Auto Layout ячейки (*self-sizing cells*, «самоизмеряющиеся ячейки»). Для этого констрейнты должны связать верх `contentView` с низом непрерывной цепочкой через содержимое — у нас: верхнее поле → стек → нижнее поле. Если бы мы написали `tableView.rowHeight = 60`, цепочка стала бы не нужна, но длинные названия обрезались бы. А ставить `estimatedRowHeight = 0` при self-sizing нельзя — это выключает оценку высоты (документация `estimatedRowHeight`).

`UIStackView` — контейнер, который сам выстраивает вложенные view в столбик (`.vertical`) или строку (`.horizontal`) с заданным промежутком (`spacing = 2`). Заголовок и подпись — в столбик, без отдельных констрейнтов для каждого. `addArrangedSubview` добавляет view и в `arrangedSubviews`, и в обычные `subviews`; простой `addSubview` кладёт её только в `subviews`, и стек такую view не раскладывает. Детям стека `translatesAutoresizingMaskIntoConstraints` выставлять не нужно — он делает это сам (проверено: после `addArrangedSubview` у подвида значение `false`).

Когда пользователь тапает чек-бокс, ячейка **не** должна сама переключать задачу. Ячейка — это «отображалка»: она не знает ни о хранилище, ни о том, что строка вообще чья-то. Переключение должно пройти через `TodoStorage`. Поэтому ячейка только сообщает «по мне тапнули» через замыкание `onCheckTap` (*callback* — «функция обратного вызова»), а решение принимает контроллер.

Вспомни, что он туда кладёт:

```
let id = todo.id
cell.onCheckTap = { [weak self] in
    self?.storage.toggleCompleted(id: id)
}
```

Захватываем `id`, а не `indexPath`. Соблазнительно написать `self.todoAt(indexPath)` внутри замыкания — но замыкание выполнится позже, по тапу, а к тому моменту строки могли переехать: задача выполнена → уехала вниз секции; наступила полночь → поменялись группы. Старый `indexPath` укажет на **другую** задачу или вообще выйдет за границы массива (падение). `id` задачи не меняется никогда.

`[weak self]`: ячейка хранит замыкание, таблица хранит ячейки, контроллер хранит таблицу. Сильный `self` в замыкании замкнул бы круг (*retain cycle* — два объекта держат друг друга и никогда не освобождаются).

После переключения хранилище позовёт наблюдателей, `reloadGroups()` перерисует таблицу, и кнопка покажет новое состояние.

12.8 Действия по свайпу

```
extension TodoListViewController: UITableViewDelegate {
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        presentEditor(TodoEditorViewController(mode: .edit(todoAt(indexPath))))
    }

    func tableView(_ tableView: UITableView,
                   trailingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
        let id = todoAt(indexPath).id
        let delete = UIContextualAction(style: .destructive, title: "Удалить") { [weak
        ↪  self] _, _, completion in
            self?.storage.remove(id: id)
            completion(true)
        }
        delete.image = UIImage(systemName: "trash")
        return UISwipeActionsConfiguration(actions: [delete])
    }

    func tableView(_ tableView: UITableView,
                   leadingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
        let todo = todoAt(indexPath)
        let toggle = UIContextualAction(
            style: .normal,
            title: todo.isCompleted ? "Вернуть" : "Готово"
        ) { [weak self] _, _, completion in
            self?.storage.toggleCompleted(id: todo.id)
            completion(true)
        }
        toggle.backgroundColor = todo.isCompleted ? .systemGray : .systemGreen
        toggle.image = UIImage(systemName: todo.isCompleted ? "arrow.uturn.backward" :
↪  "checkmark")
```

```
        return UISwipeActionsConfiguration(actions: [toggle])
    }
}
```

`didSelectRowAt` — тап по строке: снимаем выделение (иначе строка останется серой) и открываем редактор этой задачи (12.10).

Направления свайпа легко перепутать, поэтому точно:

- **trailing** («замыкающие») действия появляются у **правого** края строки, когда ты ведёшь палец **справа налево**. Так в «Почте» удаляют письма. У нас там «Удалить».
- **leading** («ведущие») — у **левого** края, палец ведёшь **слева направо**. У нас там «Готово» / «Вернуть».

(Для языков, которые пишут справа налево, например арабского, iOS сама зеркально меняет стороны — поэтому в API «leading/trailing», а не «left/right».)

`UIContextualAction` — одна кнопка в свайпе, точнее её **описание**: и `UIContextualAction`, и `UISwipeActionsConfiguration` — наследники `NSObject`, а не view; сами кнопки по этому описанию рисует таблица (глава 0, раздел 0.8). Стиль `.destructive` — «действие удаляет данные или делает что-то необратимое», фон по умолчанию красный (`systemRed`). Стиль `.normal` — серый по умолчанию (`systemGray3`), мы перекрашиваем его в зелёный через `backgroundColor`. Цвета сняты в симуляторе iOS 26.5.

Полный свайп. Если протянуть палец через всю строку, iOS сразу выполнит **первое** действие в списке, не дожидаясь тапа по кнопке. Это работает для обоих стилей и включено по умолчанию (`performsFirstActionWithFullSwipe = true` у `UISwipeActionsConfiguration`). Для удаления это удобно, но опасно: если хочешь подстраховаться, поставь свойству `false` — тогда кнопку придётся нажать явно.

`completion(true)` — обязательный вызов: так ты сообщаешь таблице, что действие завершилось. Аргумент — выполнено ли оно: `true` — да, `false` — не удалось. Пока ты его не вызвал, таблица считает действие незавершённым.

Опять захватываем `id`, а не `indexPath` — по той же причине, что в ячейке.

12.9 Плавающая кнопка «плюс»

Круглая кнопка в правом нижнем углу (в Material Design от Google её называют *FAB*, floating action button, «плавающая кнопка действия») — не системный элемент iOS, у Apple кнопку «добавить» обычно ставят в навигационную панель или внизу. Но в списках дел плавающая кнопка встречается часто, и на ней удобно показать, как положить view поверх таблицы.

```
extension TodoListViewController {
    func setupAddButton() {
        var config = UIButton.Configuration.filled()
        config.image = UIImage(systemName: "plus")
        config.preferredSymbolConfigurationForImage =
            UIImage.SymbolConfiguration(pointSize: 22, weight: .semibold)
        config.cornerStyle = .capsule
        config.baseBackgroundColor = .systemBlue
```

```
        config.baseForegroundColor = .white
        config.contentInsets = NSDirectionalEdgeInsets(top: 16, leading: 16, bottom: 16,
↪  trailing: 16)

        let button = UIButton(configuration: config)
        button.accessibilityLabel = "Добавить задачу"
        button.addTarget(self, action: #selector(addTapped), for: .touchUpInside)
        button.layer.shadowColor = UIColor.black.cgColor
        button.layer.shadowOpacity = 0.2
        button.layer.shadowRadius = 8
        button.layer.shadowOffset = CGSize(width: 0, height: 4)

        button.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(button)
        NSLayoutConstraint.activate([
            button.trailingAnchor.constraint(equalTo:
↪  view.safeAreaLayoutGuide.trailingAnchor, constant: -20),
            button.bottomAnchor.constraint(equalTo: view.safeAreaLayoutGuide.bottomAnchor,
↪  constant: -20),
        ])
    }

    @objc func addTapped() {
        presentEditor(TodoEditorViewController(mode: .create))
    }
}
```

`UIButton.Configuration.filled()` (iOS 15+) — «рецепт» кнопки: залитый фон, цвет, форма, отступы. `.capsule` — форма «капсулы»: углы скругляются на половину высоты. У квадратной кнопки капсула — это круг.

Размер. Отдельные констрейнты на размер не нужны — кнопка знает свой «естественный» размер (*intrinsic content size* — размер, который view хочет занимать по своему содержимому), и Auto Layout его использует. Прикинем: картинка «plus» при `pointSize: 22` выходит около 24×22 точки (SF Symbols не равны ровно pointSize), плюс отступы по 16 с каждой стороны — около 56–58 точек. Симулятор печатает `intrinsicContentSize ≈ 58×58`.

Тень. `layer` — слой Core Animation, через который в итоге рисуется любая view; у каждой view ровно один такой слой (глава 0, раздел 0.3). Тень — свойство слоя, а не view:

- `shadowOpacity = 0.2` — тень чёрная на 20% (почти прозрачная);
- `shadowRadius = 8` — размытие на 8 точек: край тени мягкий;
- `shadowOffset = (0, 4)` — тень сдвинута на 4 точки вниз, как от лампы сверху.

Тень даёт ощущение, что кнопка «парит» над списком. Без неё синий круг сливается с синими иконками в строках. Тень несовместима с обрезкой по границе на том же слое (`layer.masksToBounds = true`, то же самое, что `clipsToBounds`): обрезка срезала бы и тень. Здесь конфликта нет: `cornerStyle = .capsule` скругляет фоновую view конфигурации, а `masksToBounds` у слоя кнопки остаётся `false` (проверено в симуляторе).

`safeAreaLayoutGuide` — *безопасная зона*: часть экрана, которую не перекрывают «чёлка» или Dynamic Island сверху, полоска home-индикатора снизу и скругления углов. На iPhone 16 Pro нижний отступ безопасной зоны — 34 точки. Привязав кнопку к низу safe area с отступом 20, получим 34 + 20 = 54 точки от физического края — кнопка не залезет под полоску, которой пользователь сворачивает приложение.

Кнопку добавляем в `view` **после** таблицы: у view, добавленного позже, больший «порядок наложения», он рисуется поверх.

12.10 Редактор в выдвижном листе

По тапу на «плюс» или на строку открываем редактор задачи — лист, выезжающий снизу (*sheet*), с полем названия, выбором категории и приоритета и датой:

```
extension TodoListViewController {
    func presentEditor(_ editor: TodoEditorViewController) {
        editor.onSave = { [weak self] todo in
            guard let self else { return }
            if self.storage.items.contains(where: { $0.id == todo.id }) {
                self.storage.update(todo)
            } else {
                self.storage.add(todo)
            }
        }
        let nav = UINavigationController(rootViewController: editor)
        if let sheet = nav.sheetPresentationController {
            sheet.detents = [.medium(), .large()]
            sheet.prefersGrabberVisible = true
        }
        present(nav, animated: true)
    }

    func updateFilterMenu() {
        let actions = Filter.allCases.map { option in
            UIAction(title: option.title, state: option == filter ? .on : .off) { [weak
↪   self] _ in
                self?.filter = option
                self?.updateFilterMenu()
                self?.reloadGroups()
            }
        }
        navigationItem.rightBarButtonItem = UIBarButtonItem(
            image: UIImage(systemName: "line.3.horizontal.decrease.circle"),
            menu: UIMenu(title: "Показывать", children: actions)
        )
        navigationItem.rightBarButtonItem?.accessibilityLabel = "Фильтр"
    }
}
```

`sheetPresentationController` (iOS 15+) — настройки листа. Это объект класса `UISheetPresentationController` → `UIPresentationController` → `NSObject`: не view и не view controller, а «управляющий показом» (глава 0, раздел 0.6). Он не `nil`, потому что у view controller'а `modalPresentationStyle` по умолчанию — лист (`.automatic`: form sheet в iOS 18+, page sheet раньше), а не `.fullScreen`. `detents` («упоры») — высоты, на которых лист может остановиться: `.medium()` — примерно половина экрана, `.large()` — почти весь экран. Пользователь тянет лист пальцем между ними. Открывается лист на **самом низком** из перечисленных упоров: `selectedDetentIdentifier` по умолчанию `nil`, что по документации означает «наименьший detent» — у нас это половина экрана.

`prefersGrabberVisible = true` — серая полоска-«ручка» сверху листа: подсказка, что его можно тянуть.

Редактор обёрнут в `UINavigationController` ради панели сверху с кнопками «Отменить» и «Сохранить».

Как редактор возвращает результат. Он ничего не знает о хранилище — только вызывает `onSave(задача)`. Решение «добавить или обновить» принимает список: есть задача с таким `id` — обновляем, нет — добавляем. Это та же идея, что у ячейки: экран-помощник сообщает, главный решает.

Меню фильтра. `UIBarButtonItem(image:menu:)` (iOS 14+) — кнопка в навигационной панели, которая по тапу сразу открывает меню. `UIBarButtonItem` — не view (`UIBarButtonItem` → `UIBarItem` → `NSObject`), а описание кнопки: у него нет `frame` и `addSubview`, рисует его сама панель. Кладём мы его в `navigationItem` — свойство нашего view controller'а (`UINavigationItem` → `NSObject`), а `UINavigationBar` лишь показывает элементы верхнего экрана стопки (глава 0, разделы 0.6 и 0.8). `UIAction` — пункт меню; `state: .on` рисует галочку у текущего фильтра. После выбора пересоздаём меню (чтобы галочка переехала) и перегружаем список.

Сам редактор:

```
import UIKit

final class TodoEditorViewController: UIViewController {

    enum Mode {
        case create
        case edit(Todo)
    }

    var onSave: ((Todo) -> Void)?

    let mode: Mode
    var draft: Todo

    let titleField = UITextField()
    let categoryControl = UISegmentedControl(
        items: Todo.Category.allCases.map { UIImage(systemName: $0.symbolName) as Any }
    )
    let priorityControl = UISegmentedControl(
        items: Todo.Priority.allCases.map(\.title)
```

```
    )
    let dueSwitch = UISwitch()
    let datePicker = UIDatePicker()

    init(mode: Mode) {
        self.mode = mode
        switch mode {
        case .create:
            draft = Todo(title: "")
        case .edit(let todo):
            draft = todo
        }
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        setupNavigation()
        setupForm()
        fillForm()
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        if case .create = mode {
            titleField.becomeFirstResponder()
        }
    }
}
```

Режим `Mode` — перечисление с *ассоциированным значением*: `.edit(Todo)` несёт с собой задачу, которую правим. Один экран обслуживает оба случая. `draft` («черновик») — копия задачи, которую редактор меняет. Это структура, значит, копия настоящая: пока пользователь не нажал «Сохранить», хранилище ничего не знает об изменениях. «Отменить» просто закрывает лист — черновик исчезает.

`UISegmentedControl` — ряд сегментов, из которых выбран один. Для категорий — иконки, для приоритетов — слова.

`becomeFirstResponder()` в `viewDidAppear`. *First responder* («первый отвечающий») — responder, которому идут события без координат: ввод с клавиатуры, команды меню (глава 0, раздел 0.2). Сделать поле первым отвечающим — значит поставить в него курсор и показать клавиатуру. Для новой задачи это экономит тап. Почему в `viewDidAppear`, а не в `viewDidLoad`: документация Apple разрешает звать `becomeFirstResponder()` только у view, которая уже в активной иерархии

(`view.window != nil`), и обещает клавиатуру, только если её окно — key window. В `viewDidLoad` корневая view ещё не в окне (`view.window` там равен `nil` — проверено), и вызов вернёт `false`. `viewDidAppear` — первый момент, когда лист показан целиком и всё на месте.

Разметка и сохранение:

```
extension TodoEditorViewController {
    func setupNavigation() {
        if case .create = mode {
            title = "Новая задача"
        } else {
            title = "Задача"
        }
        navigationItem.leftBarButtonItem = UIBarButtonItem(
            systemItem: .cancel,
            primaryAction: UIAction { [weak self] _ in self?.dismiss(animated: true) }
        )
        navigationItem.rightBarButtonItem = UIBarButtonItem(
            systemItem: .save,
            primaryAction: UIAction { [weak self] _ in self?.saveTapped() }
        )
    }

    func setupForm() {
        titleField.placeholder = "Что сделать?"
        titleField.borderStyle = .roundedRect
        titleField.font = .preferredFont(forTextStyle: .body)
        titleField.adjustsFontForContentSizeCategory = true
        titleField.returnKeyType = .done
        titleField.addAction(UIAction { [weak self] _ in self?.updateSaveButton() },
                             for: .editingChanged)
        titleField.addAction(UIAction { [weak self] _ in
↪   self?.titleField.resignFirstResponder() },
                             for: .editingDidEndOnExit)

        for (index, category) in Todo.Category.allCases.enumerated() {
            categoryControl.imageForSegment(at: index)?.accessibilityLabel =
↪   category.rawValue
        }

        datePicker.datePickerMode = .date
        datePicker.preferredDatePickerStyle = .compact
        dueSwitch.addAction(UIAction { [weak self] _ in
            guard let self else { return }
            self.datePicker.isEnabled = self.dueSwitch.isOn
        }, for: .valueChanged)

        let dueRow = UIStackView(arrangedSubviews: [makeCaption("Срок"), UIView(),
         ↪  datePicker, dueSwitch])
```

```
        dueRow.spacing = 12
        dueRow.alignment = .center

        let stack = UIStackView(arrangedSubviews: [
            titleField,
            makeCaption("Категория"), categoryControl,
            makeCaption("Приоритет"), priorityControl,
            dueRow,
        ])
        stack.axis = .vertical
        stack.spacing = 12
        stack.setCustomSpacing(20, after: titleField)
        stack.setCustomSpacing(20, after: categoryControl)
        stack.setCustomSpacing(20, after: priorityControl)
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)

        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪   constant: 16),
            stack.leadingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor),
        ])
    }

    func makeCaption(_ text: String) -> UILabel {
        let label = UILabel()
        label.text = text
        label.font = .preferredFont(forTextStyle: .subheadline)
        label.adjustsFontForContentSizeCategory = true
        label.textColor = .secondaryLabel
        return label
    }

    func fillForm() {
        titleField.text = draft.title
        categoryControl.selectedSegmentIndex =
            Todo.Category.allCases.firstIndex(of: draft.category) ?? 0
        priorityControl.selectedSegmentIndex = draft.priority.rawValue
        dueSwitch.isOn = draft.dueDate != nil
        datePicker.date = draft.dueDate ?? .now
        datePicker.isEnabled = dueSwitch.isOn
        updateSaveButton()
    }

    func updateSaveButton() {
```

```
        let text = titleField.text ?? ""
        navigationItem.rightBarButtonItem?.isEnabled =
            !text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
    }

    func saveTapped() {
        draft.title = (titleField.text ?? "").trimmingCharacters(in:
↪   .whitespacesAndNewlines)
        draft.category = Todo.Category.allCases[categoryControl.selectedSegmentIndex]
        draft.priority = Todo.Priority(rawValue: priorityControl.selectedSegmentIndex) ??
↪   .medium
        draft.dueDate = dueSwitch.isOn ? datePicker.date : nil
        onSave?(draft)
        dismiss(animated: true)
    }
}
```

Кнопки панели через `UIAction`. `UIBarButtonItem(systemItem:primaryAction:)` (iOS 14+) создаёт системную кнопку («Отменить», «Сохранить» — подписи система переведёт на язык телефона сама) и сразу вешает на неё замыкание. Не нужен отдельный метод с `@objc`.

Кнопка «Сохранить» выключена, пока название пустое. `titleField.addAction(_:for:)` есть у поля потому, что `UITextField` — `UIControl` (`UITextField → UIControl → UIView`): у него target-action и события вроде `.editingChanged`. У `UITextView` и `UILabel` такого нет — они наследуют `UIView` напрямую (точнее, `UITextView` — через `UIScrollView`), и события от них получают через делегат (глава 0, раздел 0.4). `.editingChanged` срабатывает на каждую введённую букву; `updateSaveButton()` проверяет, есть ли в поле что-то кроме пробелов (`trimmingCharacters(in: .whitespacesAndNewlines)` обрезает пробелы и переводы строк по краям). Задачу « » сохранить нельзя.

`.editingDidEndOnExit` — нажата клавиша «Готово» на клавиатуре (её подпись задаёт `returnKeyType = .done`). Убираем клавиатуру `resignFirstResponder()`.

Срок — переключатель плюс дата. `UIDatePicker` не умеет быть «пустым», поэтому «без даты» выражаем переключателем. Когда переключатель выключен, выбор даты неактивен (`isEnabled = false`), а при сохранении `dueDate` станет `nil`. Стиль `.compact` (iOS 14+) — дата показывается компактной кнопкой, по тапу раскрывается календарь. Язык календаря берётся из настроек телефона; на русском iOS он будет русским без дополнительных строк.

`UIView()` между подписью «Срок» и датой — пустой «распорный» view: в горизонтальном стеке он забирает всё свободное место, прижимая дату и переключатель вправо. Почему именно он, а не подпись: у подписи, пикера и переключателя есть `intrinsicContentSize`, и стек держит их у этого размера констрейнтами с приоритетом hugging (250 по умолчанию). У пустого `UIView` естественного размера нет (`intrinsicContentSize` равен `(-1, -1)`, то есть `UIView.noIntrinsicMetric`) — ему нечего «обнимать», и он растягивается без сопротивления. Проверено: в стеке шириной 300 подпись заняла 41 точку, переключатель 63, распорка — остальные 198 (глава 0, раздел 0.3).

`setCustomSpacing(_:after:)` — промежуток после конкретного элемента стека больше, чем общий `spacing`: так подпись «Категория» визуально прилипает к своему контролу, а не к предыдущему.

> **Упражнение 12.2.** Сейчас лист открывается на половинной высоте, и на iPhone SE клавиатура закрывает выбор приоритета. Сделай так, чтобы при создании новой задачи лист открывался сразу во всю высоту, а при редактировании — на половину. Подсказка: у `UISheetPresentationController` есть свойство `selectedDetentIdentifier`.

12.11 Бытовая аналогия

Todo — это **холодильник с магнитиками**. Каждая задача — магнитик с запиской. Категории — цвета магнитиков, приоритет — насколько крупно написано. Свайп — рука, которая снимает магнитик или зачёркивает запись.

Когда ты добавляешь задачу, магнитик прикрепляется к холодильнику (хранилище). Когда смотришь на холодильник (таблица), видишь все магнитики, разложенные по зонам: «Просрочено», «Сегодня», «Когда-нибудь».

Наблюдатели — это **члены семьи, которые попросили сказать им, если на холодильнике что-то поменялось**. Хранилище не ходит за ними и не держит их за руку (`weak`): кто ушёл из дома, того просто вычёркивают из списка.

12.12 Что мы пропустили

- **Напоминания.** Задача со сроком «через час» должна напомнить о себе. Делается через `UNUserNotificationCenter`: при сохранении задачи планируется локальное уведомление на её срок (нужно разрешение пользователя — см. главу 7 про объяснение перед системным диалогом).
- **Синхронизация.** Сейчас задачи живут только на одном устройстве. Чтобы они были и на iPhone, и на iPad, нужен CloudKit или свой сервер.
- **Поиск по тексту.** Фильтр «все / активные / завершённые» есть, а поиска нет. `UISearchController` в `navigationItem.searchController` — как в главе 13 про заметки.
- **Повторяющиеся задачи.** «Каждый понедельник» требует правил повтора и расчёта очередного срока с учётом календаря. В этом учебнике пропускаем.
- **Анимированные изменения.** `reloadData()` перерисовывает таблицу мгновенно, без анимации. Diffable data source вычислит разницу и плавно переставит строки.

> **Упражнение 12.3.** Запусти приложение (первый запуск — с интернетом). Проверь по шагам:
>
> 1. Появились восемь демо-задач, разложенных по секциям «Сегодня», «Завтра», «Ближайшие 7 дней», «Без даты».
> 2. Создай «Купить молоко» с категорией «Покупки» без срока — она окажется в секции «Без даты».
> 3. Отметь её выполненной, проведя пальцем по строке **слева направо** и нажав «Готово» (или тапнув кружок) — задача зачеркнётся и опустится вниз секции.
> 4. Удали другую задачу, проведя пальцем **справа налево**.

5. Полностью закрой приложение (смахни его в переключателе приложений) и открой снова — всё на месте, демо-задачи повторно не появились.

Ответы к упражнениям

Упражнение 12.1. В конец `reloadGroups()`:

```
let activeCount = storage.items.filter { !$0.isCompleted }.count
title = activeCount == 0 ? "Список дел" : "Список дел (\(activeCount))"
```

Считаем по всем задачам (`storage.items`), а не по отфильтрованным: иначе при фильтре «Завершённые» заголовок показал бы 0. Проверка: задач 3 активных → «Список дел (3)», отметил одну → «(2)».

Упражнение 12.2. В `presentEditor(_:)`, после строки с `detents`:

```
if case .create = editor.mode {
    sheet.selectedDetentIdentifier = .large
}
```

`selectedDetentIdentifier` — на какой высоте лист стоит сейчас (`.medium` или `.large`). Если задать его до `present`, лист сразу откроется на этой высоте. Проверка: «плюс» — лист во весь экран, тап по задаче — на половину.

Упражнение 12.3. Ожидаемый результат описан в самих шагах. Если демо-задачи не появились — проверь, что у симулятора есть интернет (открой в нём Safari). Если после перезапуска они появились снова — значит, флаг `todo.firstLaunchSeeded.v1` не сохранился; проверь, что `seedIfNeeded(with:)` вызывает `defaults.set(true, forKey:)`.

Что мы выучили

- Модель `Todo` — `Identifiable, Codable, Hashable, Sendable`. Каждый протокол нужен для своей цели: стабильный `id`, JSON, `Set`/словари, безопасная передача между потоками.
- Группировка по датам — вычисляемое свойство `group` + `Dictionary(grouping:by: \.group)`. Дни сравниваем через `startOfDay`, дни прибавляем через `Calendar`, а не секундами.
- Хранилище — `UserDefaults` + JSON. Версия в ключе (`v1`) оставляет место для миграции. `.iso8601` — даты строками, понятными любому серверу. `UserDefaults` требует причины в privacy manifest.
- Наблюдатели — словарь `[ObjectIdentifier: Observer]` со слабой ссылкой на владельца: без дубликатов и без `deinit`, который в Swift 6 не может трогать main-actor хранилище.
- Первое заполнение из dummyjson — отдельная модель для ответа сервера, детерминированная раскладка через остаток от деления, флаг `todo.firstLaunchSeeded.v1`.
- `UITableView` — `UIScrollView`; `dataSource` и `delegate` у неё слабые, контроллер должен жить сам. `UITableViewCell` — `UIView` со служебными подвидами; своё содержимое кладём в `contentView`, чтобы оно сдвигалось вместе с ним в режиме редактирования и под аксессуаром.
- Своя ячейка: `UIButton(type: .custom)` с картинками для `.normal` и `.selected`, всё в `contentView`, высота от содержимого — потому что `rowHeight` и `estimatedRowHeight` по умолчанию равны `UITableView.automaticDimension` (−1) и констрейнты связывают верх

`contentView` с низом; `prepareForReuse` (только для ячеек из запаса) сбрасывает замыкание, `configure` выставляет каждое свойство в обе стороны.
- В замыканиях ячеек и свайпов захватываем `id` задачи, а не `indexPath`, и `[weak self]`.
- Свайпы: trailing (справа налево) — удалить, leading (слева направо) — отметить. Первое действие срабатывает полным свайпом.
- Плавающая кнопка — `UIButton.Configuration.filled()` с тенью на слое, привязана к safe area.
- Редактор — лист `UISheetPresentationController` (не view и не view controller) с `[.medium(), .large()]`; открывается на самом низком упоре; черновик-копия структуры, результат через `onSave`. `UIBarButtonItem` — не view, а описание кнопки в `navigationItem` нашего контроллера.

Apple Developer Documentation

- UITableView — список с секциями и переиспользуемыми ячейками, основа экрана Todo.
- UITableViewDataSource — протокол, отвечающий за количество секций/строк и `cellForRowAt`.
- UITableViewDelegate — отдельный протокол про взаимодействие (выбор строки, свайпы, высоты); наследует `UIScrollViewDelegate`.
- `UITableViewCell.contentView` — штатный superview для своего содержимого ячейки; сдвигается в режиме редактирования.
- `UITableViewCell.prepareForReuse()` — вызывается перед выдачей ячейки из запаса; сбрасывать здесь только то, что не относится к содержимому.
- `UITableView.rowHeight` — по умолчанию `automaticDimension`: высоту строки считает Auto Layout ячейки.
- `UIView.translatesAutoresizingMaskIntoConstraints` — `true` у любой view, созданной кодом; `false` — для своих констрейнтов.
- `UISheetPresentationController.selectedDetentIdentifier` — текущий упор листа; `nil` означает наименьший из `detents`.
- UITableViewDiffableDataSource — альтернатива классическому data source через снимки данных; пригодится, если уйдёшь от `reloadData()` к анимированным изменениям.
- UISwipeActionsConfiguration — конфигурация trailing/leading свайпов с `UIContextualAction`.
- UserDefaults — простое key-value хранилище, в нашей главе хранит JSON задач.
- Codable — type alias `Encodable & Decodable`, делает `Todo` сериализуемым в JSON.
- JSONEncoder и JSONDecoder — сериализация в JSON и обратно, с настройкой стратегии дат.
- URLSession — стандартный сетевой клиент iOS; используем для запроса демо-задач в dummyjson.

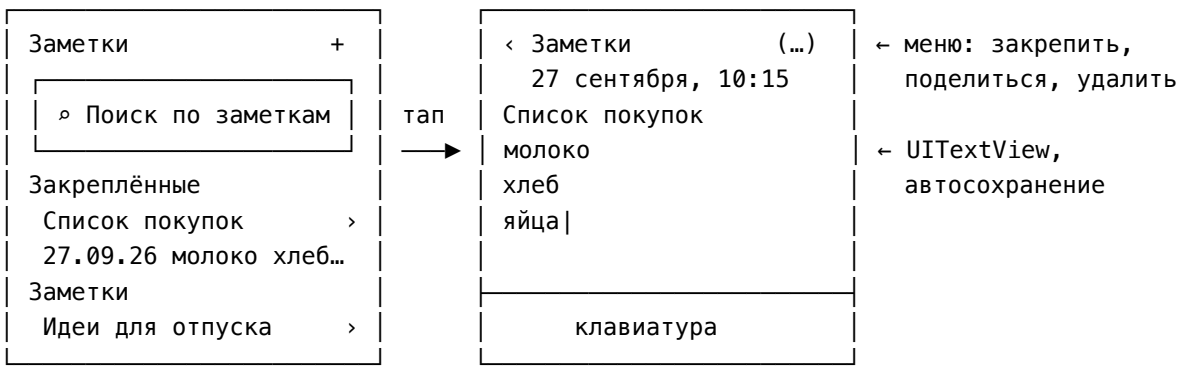
→ Глава 13. Notes — UITextView, FileManager, UISearchController

Глава 13. Notes — UITextView, FileManager, UISearchController

Заметки — второй после Todo «канонический» учебный проект iOS. Если в Todo главный экран — таблица с действиями, то в Notes главное — экран **редактирования**: свободный текст без полей и кнопок «Сохранить».

Технически от Todo три отличия: хранилище (вместо UserDefaults — отдельные файлы через FileManager), главный элемент (вместо ячеек с кнопками — `UITextView` во весь экран) и поиск через `UISearchController`.

Что строим — два экрана:



Проект — такой же, как в главе 12 (раздел «Перед началом»): App, Swift 6, Default Actor Isolation = MainActor, iOS 15+, без `Main.storyboard`. `SceneDelegate` отличается одной строкой — первым экраном:

```
import UIKit

final class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }
        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = UINavigationController(
```

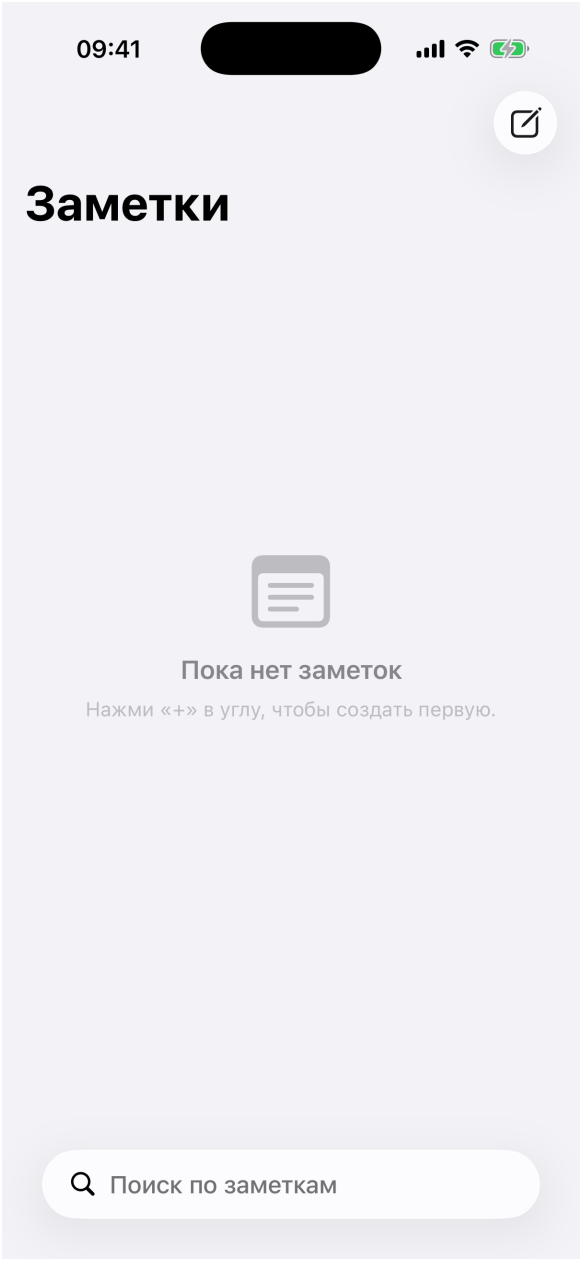


Рис. 9: Список заметок

```
            rootViewController: NoteListViewController()
        )
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

Здесь навигационный контроллер нужен не только ради панели: из списка мы будем **переходить** в редактор (`push`), и кнопка «‹ Заметки» для возврата появится сама.

13.1 Почему FileManager, а не UserDefaults

`UserDefaults` — это один plist-файл на всё приложение (см. главу 12). Он хорош для:

- настроек (`Bool`, `Int`, `String`);
- небольших структур (JSON в `Data`, как задачи в Todo);
- флагов «уже запускали», «онбординг пройден».

И плохо подходит для:

- **больших объёмов.** Жёсткого лимита на iOS Apple не называет, но весь файл целиком читается в память при старте приложения и целиком переписывается при изменениях. Сто заметок по 20 КБ — это 2 МБ, которые будут висеть в памяти, даже если пользователь открыл одну;
- **частых записей большого массива.** Поменял одну букву в одной заметке — перекодировал и переписал все;
- **данных, которые удобно видеть по отдельности** — например, удалить одну испорченную заметку, не трогая остальные.

Заметка — это **один документ**. Она может быть на 10 байт, а может на тысячи строк. Заметки независимы: правка одной не должна перезаписывать остальные.

Решение — **файл на заметку** в папке Documents:

```
Documents/
└── Notes/
    ├── 79CB4E81-5A0F-4E3B-9D0C-1F2A3B4C5D6E.json
    ├── 8FAD7C12-….json
    └── A2D5F801-….json
```

Documents — одна из папок в песочнице приложения (sandbox — личная файловая область, которую видит только это приложение). Её содержимое попадает в резервную копию iCloud/компьютера, поэтому там хранят именно пользовательские данные. Для временного и того, что можно скачать заново, есть папка Caches — её система может очистить сама, когда кончается место.

Имя файла — `id` заметки (UUID), расширение `.json`. Формат JSON выбран не ради совместимости — просто при отладке файл можно открыть и прочитать глазами. Где лежит песочница симулятора, узнаешь командой `xcrun simctl get_app_container booted <bundle id> data`.

13.2 Модель заметки

```
import Foundation

struct Note: Codable, Equatable, Sendable, Identifiable {
    let id: UUID
    var body: String
    let createdAt: Date
    var updatedAt: Date
    var pinned: Bool

    init(id: UUID = UUID(), body: String = "", createdAt: Date = Date(), pinned: Bool =
    ↪  false) {
        self.id = id
        self.body = body
        self.createdAt = createdAt
        self.updatedAt = createdAt
        self.pinned = pinned
    }

    var title: String {
        let firstLine = body
            .split(separator: "\n", omittingEmptySubsequences: false)
            .first.map(String.init) ?? ""
        let trimmed = firstLine.trimmingCharacters(in: .whitespaces)
        return trimmed.isEmpty ? "Новая заметка" : trimmed
    }

    var preview: String {
        let lines = body.split(separator: "\n", omittingEmptySubsequences: false)
        let rest = lines.dropFirst()
            .joined(separator: " ")
            .trimmingCharacters(in: .whitespaces)
        if rest.isEmpty { return "Нет дополнительного текста" }
        return rest.count > 100 ? String(rest.prefix(100)) + "…" : rest
    }
}
```

Что в этой модели важно.

`body` — это всё. Заголовок и текст не разделены: так устроены «Заметки» Apple и многие другие редакторы. Первая строка текста автоматически становится заголовком. В интерфейсе одно поле (`UITextView`), а не два, и пользователь сам решает, нужен ли заголовок: написал короткую первую строку — она и есть заголовок.

`title` и `preview` — вычисляемые свойства, в файл не пишутся. `Codable` кодирует только хранимые свойства, поэтому JSON содержит `id`, `body`, две даты и `pinned`.

Как работает `title` на примере. Пусть `body` = `"Список покупок\nмолоко\nхлеб"`. `split(separator: "\n")` режет строку по переводам строки: `["Список покупок", "молоко", "хлеб"]`. `.first` — `"Список покупок"`. `trimmingCharacters(in: .whitespaces)` обрезает пробелы по краям. Не пусто — это и есть заголовок.

`omittingEmptySubsequences: false` — важная деталь. По умолчанию `split` выбрасывает пустые куски. Для текста `"\nмолоко"` (первая строка пустая) без этого флага первым куском оказалось бы «молоко», и заголовок «украл» бы вторую строку. С флагом первый кусок — пустая строка, и заголовок честно станет «Новая заметка».

`preview` — всё, кроме первой строки, склеенное через пробел и обрезанное до 100 символов. Для примера выше: `"молоко хлеб"`.

`updatedAt` в `init` равен `createdAt`: у только что созданной заметки это одно и то же время.

`pinned` — закреплённые заметки показываются в отдельной секции сверху, как закреплённые чаты в мессенджерах.

13.3 Хранилище через FileManager

```
import Foundation

@MainActor
final class NoteStorage {
    static let shared = NoteStorage()

    private let fileManager = FileManager.default
    private let folderURL: URL
    private let encoder = JSONEncoder()
    private let decoder = JSONDecoder()

    private(set) var notes: [Note] = []
    private var observers: [ObjectIdentifier: Observer] = [:]

    init(folderURL: URL? = nil) {
        if let folderURL {
            self.folderURL = folderURL
        } else {
            let docs = fileManager.urls(for: .documentDirectory, in: .userDomainMask)[0]
            self.folderURL = docs.appendingPathComponent("Notes", isDirectory: true)
        }
        do {
            try fileManager.createDirectory(at: self.folderURL,
            ↪  withIntermediateDirectories: true)
        } catch {
            assertionFailure("NoteStorage: не удалось создать папку: \(error)")
        }
        encoder.dateEncodingStrategy = .iso8601
        encoder.outputFormatting = [.prettyPrinted, .sortedKeys]
```

```
        decoder.dateDecodingStrategy = .iso8601
        load()
    }
}
```

`fileManager.urls(for: .documentDirectory, in: .userDomainMask)` — стандартный способ получить адрес папки Documents. Возвращает массив (API общий с macOS, где папок может быть несколько), на iOS в нём всегда один элемент — берём `[0]`.

`appendingPathComponent("Notes", isDirectory: true)` — добавляем к адресу имя подпапки. `isDirectory: true` говорит, что это папка: URL получит завершающий `/` (`…/Documents/Notes/`), и все пути, собранные от него дальше, будут правильными. В iOS 16 появился более короткий `appending(path:)`, но нам нужна iOS 15.

`createDirectory(at:withIntermediateDirectories: true)` — создать папку, если её нет. С `withIntermediateDirectories: true` метод создаёт и недостающих «родителей» по пути и **не** считает ошибкой, что папка уже существует — поэтому его можно звать при каждом запуске. Ошибка возможна только в настоящей беде (например, диск переполнен); на неё — `assertionFailure`, чтобы увидеть в отладке.

`init(folderURL:)` — адрес папки можно передать снаружи. В тестах передают временную папку, чтобы не трогать настоящие заметки.

`outputFormatting = [.prettyPrinted, .sortedKeys]` — JSON с отступами и ключами по алфавиту. Файл чуть больше, зато его приятно читать при отладке. Сохранённая заметка выглядит так:

```
{
  "body" : "Список покупок\nмолоко",
  "createdAt" : "2026-09-27T05:15:00Z",
  "id" : "79CB4E81-5A0F-4E3B-9D0C-1F2A3B4C5D6E",
  "pinned" : false,
  "updatedAt" : "2026-09-27T05:16:12Z"
}
```

Время в файле — по UTC (буква `Z` в конце — «Zulu», нулевой часовой пояс). 05:15 UTC — это 10:15 в Алматы (UTC+5). Хранить время в UTC и переводить в местное только при показе — правило, которое спасает от путаницы, когда человек летит в другой часовой пояс.

`@MainActor` — по той же причине, что в Todo: после изменений хранилище зовёт подписчиков, а они трогают интерфейс. Запись одного небольшого файла занимает доли миллисекунды, так что делать её на главном потоке допустимо. Если бы заметки весили мегабайты (с картинками), запись стоило бы увести в фон.

13.4 Сохранение, загрузка, удаление

```
extension NoteStorage {
    func note(with id: UUID) -> Note? {
        notes.first { $0.id == id }
    }
```

```
    /// Сохраняет заметку. `touch: true` — это правка текста, двигаем дату изменения.
    @discardableResult
    func save(_ note: Note, touch: Bool = true) -> Note {
        var updated = note
        if touch { updated.updatedAt = Date() }
        do {
            let data = try encoder.encode(updated)
            try data.write(to: fileURL(for: updated.id), options: .atomic)
        } catch {
            assertionFailure("NoteStorage: не удалось сохранить: \(error)")
            return note
        }
        if let index = notes.firstIndex(where: { $0.id == updated.id }) {
            notes[index] = updated
        } else {
            notes.append(updated)
        }
        sortAndNotify()
        return updated
    }

    func delete(_ id: UUID) {
        try? fileManager.removeItem(at: fileURL(for: id))
        notes.removeAll { $0.id == id }
        sortAndNotify()
    }

    func togglePin(_ id: UUID) {
        guard var note = note(with: id) else { return }
        note.pinned.toggle()
        save(note, touch: false)
    }

    private func fileURL(for id: UUID) -> URL {
        folderURL.appendingPathComponent("\(id.uuidString).json")
    }
}
```

`@discardableResult` — разрешает вызвать `save` и не использовать результат. Без атрибута Swift выдаст предупреждение «result of call to ‘save’ is unused». Результат нужен редактору (ему важна новая `updatedAt`), а `togglePin` — нет.

`updatedAt` обновляется внутри `save`, чтобы вызывающему не нужно было об этом помнить. Но не всегда: параметр `touch`. Закрепить заметку — не значит изменить её текст. Если бы закрепление двигало дату, старая заметка после закрепления «помолодела» бы и прыгнула наверх списка как только что изменённая. Поэтому `togglePin` сохраняет с `touch: false`.

`options: .atomic` — атомарная запись: данные сначала пишутся во временный файл рядом, а потом одним действием файловой системы подменяют старый. Если приложение упадёт или

разрядится телефон посередине записи, на диске останется либо старая полная версия, либо новая полная — никогда половина. Для пользовательских данных это обязательно.

Порядок: сначала диск, потом память. Если запись на диск не удалась, мы выходим, не меняя массив `notes`. Иначе на экране было бы одно, а на диске другое, и после перезапуска правка молча исчезла бы.

`delete` с `try?`: если файла уже нет (удаляли дважды), ошибка нам не важна — результат тот же, файла нет.

Загрузка при старте и уведомление подписчиков:

```
extension NoteStorage {
    func load() {
        let entries = (try? fileManager.contentsOfDirectory(
            at: folderURL, includingPropertiesForKeys: nil
        )) ?? []
        notes = entries
            .filter { $0.pathExtension == "json" }
            .compactMap { url in
                guard let data = try? Data(contentsOf: url) else { return nil }
                return try? decoder.decode(Note.self, from: data)
            }
        sortAndNotify()
    }

    func sortAndNotify() {
        notes.sort { $0.updatedAt > $1.updatedAt }
        notify()
    }
}
```

`contentsOfDirectory(at:includingPropertiesForKeys:)` — список адресов файлов в папке. `includingPropertiesForKeys: nil` — не запрашивать заранее размер, дату изменения и прочее; нам хватит содержимого. Это ещё и плюс для приватности: API, читающие даты файлов, входят в список «required reason API», для которых нужно указывать причину в privacy manifest (глава 39). Мы их не трогаем.

`.filter { $0.pathExtension == "json" }` — пропускаем чужие файлы. iOS или сам пользователь (если включить доступ к Documents через приложение «Файлы») могут положить в папку что-то своё.

`compactMap` — `map`, который выбрасывает `nil`. Если один файл повреждён и не читается, `try?` даст `nil`, и этот файл просто не попадёт в список, а остальные загрузятся. С `map` и принудительным разворачиванием (`try!`) одна испорченная заметка роняла бы всё приложение при каждом запуске.

Сортировка по `updatedAt` по убыванию — свежие заметки сверху. Порядок файлов, который вернула файловая система, не определён, поэтому сортируем всегда сами.

Наблюдатели — тот же приём, что в главе 12, раздел 12.4: словарь по `ObjectIdentifier` со слабой ссылкой на владельца, без `deinit`:

```
extension NoteStorage {
    struct Observer {
        weak var owner: AnyObject?
        let handler: () -> Void
    }

    func addObserver(_ owner: AnyObject, handler: @escaping () -> Void) {
        observers[ObjectIdentifier(owner)] = Observer(owner: owner, handler: handler)
    }

    private func notify() {
        observers = observers.filter { $0.value.owner != nil }
        for observer in observers.values {
            observer.handler()
        }
    }
}
```

Атомарная запись. `.atomic` гарантирует, что ты видишь либо старую полную версию файла, либо новую полную. Никогда — половину. Используй её везде, где пишешь ценные данные: цена — одна лишняя операция переименования, то есть почти ничего.

Упражнение 13.1. Запусти приложение в симуляторе, создай две заметки. В терминале выполни `xcrun simctl get_app_container booted <твой bundle id> data` и загляни в `Documents/Notes/` по напечатанному пути. Сколько там файлов и что внутри? Удали один файл из Finder и перезапусти приложение — что станет со списком?

13.5 Поиск через UISearchController

`UISearchController` — системный контроллер поиска: строка поиска с кнопкой «Отменить» и логикой её появления и скрытия. Он встраивается в навигационную панель через `navigationItem.searchController` (iOS 11+).

Кто здесь кто. `UISearchController` — наследник `UIViewController`, а не view: его `searchBar` (`UISearchBar → UIView`) — это view, а сам контроллер при активации **показывает себя** поверх твоего экрана как presentation — отсюда настройка `definesPresentationContext` ниже. `navigationItem` — свойство самого view controller'а (класс `UINavigationItem`, потомок `NSObject`); навигационная панель лишь читает его и рисует то, что там лежит (глава 0, раздел 0.6).

Начнём со свойств и `viewDidLoad` экрана списка:

```
import UIKit

final class NoteListViewController: UIViewController {

    enum Section: Int, CaseIterable { case pinned, regular }

    let storage: NoteStorage
```

```
    let tableView = UITableView(frame: .zero, style: .insetGrouped)
    let searchController = UISearchController(searchResultsController: nil)
    let emptyLabel = UILabel()

    var pinnedNotes: [Note] = []
    var regularNotes: [Note] = []

    static let dateFormatter: DateFormatter = {
        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.dateStyle = .short
        formatter.timeStyle = .short
        return formatter
    }()

    init(storage: NoteStorage = .shared) {
        self.storage = storage
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Заметки"
        navigationController?.navigationBar.prefersLargeTitles = true
        view.backgroundColor = .systemGroupedBackground
        navigationItem.rightBarButtonItem = UIBarButtonItem(
            systemItem: .compose,
            primaryAction: UIAction { [weak self] _ in self?.openEditor(for: nil) }
        )
        navigationItem.rightBarButtonItem?.accessibilityLabel = "Новая заметка"
        setupTable()
        setupSearch()

        storage.addObserver(self) { [weak self] in
            self?.applyFilter()
        }
        applyFilter()
    }
}
```

`UISearchController(searchResultsController: nil)` — `nil` значит «результаты показывай в этом же экране». Можно передать отдельный контроллер результатов (так сделан поиск в «Настройках»), но нам проще фильтровать ту же таблицу.

`prefersLargeTitles = true` — крупный заголовок «Заметки», как в системных приложениях. При прокрутке он сам уменьшается до обычного.

`.compose` — системная иконка «карандаш над листом», ею во всех приложениях Apple обозначают «новый документ».

`DateFormatter` со стилями `.short` — «27.09.2026, 10:15» в русской локали. Порядок дня и месяца форматтер берёт из правил локали, мы его не задаём.

Разметка и настройка поиска:

```
extension NoteListViewController {
    func setupTable() {
        tableView.translatesAutoresizingMaskIntoConstraints = false
        tableView.dataSource = self
        tableView.delegate = self
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "NoteCell")
        tableView.keyboardDismissMode = .onDrag
        view.addSubview(tableView)
        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            tableView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        ])

        emptyLabel.numberOfLines = 0
        emptyLabel.textAlignment = .center
        emptyLabel.textColor = .secondaryLabel
        emptyLabel.font = .preferredFont(forTextStyle: .body)
        emptyLabel.adjustsFontForContentSizeCategory = true
    }

    func setupSearch() {
        searchController.searchResultsUpdater = self
        searchController.obscuresBackgroundDuringPresentation = false
        searchController.searchBar.placeholder = "Поиск по заметкам"
        navigationItem.searchController = searchController
        navigationItem.hidesSearchBarWhenScrolling = false
        definesPresentationContext = true
    }
}
```

Что делают параметры поиска:

- **`searchResultsUpdater = self`** — кому сообщать об изменении текста в строке поиска. Наш контроллер реализует протокол `UISearchResultsUpdating` (ниже).
- **`obscuresBackgroundDuringPresentation = false`** — не затемнять экран во время поиска. Затемнение нужно, когда результаты показываются в отдельном контроллере поверх; мы фильтруем прямо в той же таблице, и затемнённые строки нельзя было бы нажать.

- **`hidesSearchBarWhenScrolling = false`** — строка поиска видна всегда. По умолчанию iOS прячет её при прокрутке вниз и показывает, если потянуть список. Для приложения, где поиск — основной способ найти заметку, лучше не прятать.
- **`definesPresentationContext = true`** — «этот view controller — граница контекста показа». Поиск показывает себя в стиле *current context*: UIKit идёт от presenting'а вверх по иерархии и накрывает view первого контроллера, у которого этот флаг `true`. По умолчанию он `false`, но `UINavigationController` ставит его себе в `true`, так что без нашей строки контекстом стал бы весь навигационный контроллер — и строка поиска, активная в момент перехода в редактор, осталась бы висеть поверх нового экрана.

Где окажется строка поиска, решает система. На iOS 15–18 она стоит в навигационной панели под крупным заголовком, как на схеме в начале главы. На iOS 26 на iPhone та же настройка рисует поле поиска **внизу экрана**, над home-индикатором, — так выглядят новые системные приложения (проверено в симуляторе iOS 26.5). Код для обоих вариантов один и тот же, менять ничего не нужно.

`keyboardDismissMode = .onDrag` — клавиатура поиска прячется, как только пользователь начинает прокручивать список. Это свойство `UIScrollView`, а `UITableView` — его наследник (глава 0, раздел 0.5); по умолчанию стоит `.none`, и клавиатура сама не уходит.

Сам протокол и фильтр:

```
extension NoteListViewController: UISearchResultsUpdating {
    func updateSearchResults(for searchController: UISearchController) {
        applyFilter()
    }
}

extension NoteListViewController {
    func applyFilter() {
        let query = searchController.searchBar.text ?? ""
        let found = storage.search(query: query)
        pinnedNotes = found.filter { $0.pinned }
        regularNotes = found.filter { !$0.pinned }

        if found.isEmpty {
            emptyLabel.text = query.isEmpty
                ? "Заметок пока нет.\nНажми на карандаш справа вверху."
                : "Ничего не нашлось по запросу «\(query)»."
            tableView.backgroundView = emptyLabel
        } else {
            tableView.backgroundView = nil
        }
        tableView.reloadData()
    }

    func notes(in section: Int) -> [Note] {
        Section(rawValue: section) == .pinned ? pinnedNotes : regularNotes
    }
```

```
    func note(at indexPath: IndexPath) -> Note {
        notes(in: indexPath.section)[indexPath.row]
    }

    func openEditor(for note: Note?) {
        let editor = NoteEditorViewController(note: note, storage: storage)
        navigationController?.pushViewController(editor, animated: true)
    }
}
```

`updateSearchResults(for:)` вызывается, когда строка поиска становится first responder (поле получило фокус), и дальше **на каждое изменение текста** — каждая буква, стирание, вставка. Отмена поиска очищает текст, так что метод придёт ещё раз и вернёт полный список.

Пустое состояние различает два случая: заметок нет вообще («нажми на карандаш») и ничего не нашлось по запросу. Одинаковая надпись для обоих сбивала бы с толку: человек ищет «отпуск», видит «заметок пока нет» и пугается, что всё пропало.

Поиск в хранилище:

```
extension NoteStorage {
    func search(query: String) -> [Note] {
        let q = query.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !q.isEmpty else { return notes }
        return notes.filter {
           $0.body.range(of: q, options: [.caseInsensitive, .diacriticInsensitive]) != nil
        }
    }
}
```

`range(of:options:)` ищет подстроку с настройками сравнения. `.caseInsensitive` — без учёта регистра («ПОКУПОК» найдёт «покупок»). `.diacriticInsensitive` — без учёта диакритических знаков. Для русского это важно из-за буквы «ё»: по запросу «елка» найдётся «Ёлка». Простое `body.lowercased().contains(q)` так не умеет — проверено: `"Ёлка и мёд".lowercased().contains("елка")` даёт `false`, а `range(of:options:)` с этими флагами находит совпадение.

Для 50 заметок перебор мгновенный. Для тысяч длинных заметок стоит подумать о *debounce* («подавлении дребезга» — запускать поиск не на каждую букву, а через 200–300 мс после того, как человек перестал печатать) и о полнотекстовом индексе. Для нашего приложения хватит.

> **Debounce для тяжёлых запросов.** Если поиск идёт по большому объёму или ходит на сервер, debounce обязателен. Иначе слово «покупки» из 7 букв — это 7 запросов подряд, из которых нужен только последний. Как сделать — через `DispatchWorkItem` — показано ниже, в 13.8: там тем же приёмом откладывается сохранение. Подробнее о поиске и фильтрах — глава 25.

13.6 Группировка «закреплённые / обычные»

Список разбит на две секции:

```
extension NoteListViewController: UITableViewDataSource {
    func numberOfSections(in tableView: UITableView) -> Int {
        Section.allCases.count
    }

    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        notes(in: section).count
    }

    func tableView(_ tableView: UITableView, titleForHeaderInSection section: Int) ->
    ↪ String? {
        // Заголовки нужны, только когда есть закреплённые: иначе список один.
        guard !pinnedNotes.isEmpty else { return nil }
        switch Section(rawValue: section) {
        case .pinned:  return "Закреплённые"
        case .regular: return regularNotes.isEmpty ? nil : "Заметки"
        case .none:    return nil
        }
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "NoteCell", for:
        ↪ indexPath)
        let note = note(at: indexPath)
        var content = UIListContentConfiguration.subtitleCell()
        content.text = note.title
        content.textProperties.font = .preferredFont(forTextStyle: .headline)
        content.textProperties.numberOfLines = 1
        let date = Self.dateFormatter.string(from: note.updatedAt)
        content.secondaryText = "\(date)  \(note.preview)"
        content.secondaryTextProperties.color = .secondaryLabel
        content.secondaryTextProperties.numberOfLines = 2
        content.textToSecondaryTextVerticalPadding = 4
        cell.contentConfiguration = content
        cell.accessoryType = .disclosureIndicator
        return cell
    }
}
```

Секций всегда две, даже если одна пустая. Так номер секции однозначно говорит, что в ней: 0 — закреплённые, 1 — обычные. `Section(rawValue: section)` превращает число в перечисление. Пустая секция без заголовка и без строк совсем невидимой не становится: в grouped-таблице у каждой секции есть системные поля сверху и снизу, и пустая первая секция сдвигает список вниз примерно на 35 точек (измерено в симуляторе iOS 26.5: первая строка на

`y = 70` вместо `35`). Для нашего списка это небольшой зазор под строкой поиска, он не мешает. Если захочешь убрать его совсем — верни для пустой секции `.leastNonzeroMagnitude` из `tableView(_:heightForHeaderInSection:)` и `tableView(_:heightForFooterInSection:)`: ноль в grouped-стиле UIKit не принимает.

`titleForHeaderInSection` возвращает заголовки только если есть закреплённые. Логика по случаям:

- закреплённых нет → заголовков нет вообще, просто список;
- есть и те, и другие → «Закреплённые» и «Заметки»;
- все закреплённые → только «Закреплённые».

Это убирает шумный заголовок «Заметки» над единственным списком.

Ячейка без своего класса. Здесь хватает системной разметки «заголовок + подзаголовок». `UIListContentConfiguration` (iOS 14+) — «рецепт» содержимого стандартной ячейки: текст, второй текст, картинка и их стили. Это структура, а не view: она реализует протокол `UIContentConfiguration`, а ячейка по ней сама создаёт и обновляет свою content view (глава 0, раздел 0.9). Мы заполняем рецепт и отдаём ячейке (`cell.contentConfiguration = content`). Удобно для переиспользования: каждый раз задаём рецепт целиком, и от прошлой строки ничего не остаётся — `prepareForReuse` не нужен.

`numberOfLines = 1` у заголовка — длинный заголовок обрежется многоточием. У превью — 2 строки. `disclosureIndicator` — серая стрелка «›» справа: подсказка, что по тапу откроется новый экран.

13.7 Свайпы — закрепить и удалить

```
extension NoteListViewController: UITableViewDelegate {
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        openEditor(for: note(at: indexPath))
    }

    func tableView(_ tableView: UITableView,
                   leadingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
        let target = note(at: indexPath)
        let pin = UIContextualAction(
            style: .normal,
            title: target.pinned ? "Открепить" : "Закрепить"
        ) { [weak self] _, _, done in
            self?.storage.togglePin(target.id)
            done(true)
        }
        pin.image = UIImage(systemName: target.pinned ? "pin.slash.fill" : "pin.fill")
        pin.backgroundColor = .systemYellow
        return UISwipeActionsConfiguration(actions: [pin])
    }
```

```
    func tableView(_ tableView: UITableView,
                   trailingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
        let id = note(at: indexPath).id
        let delete = UIContextualAction(style: .destructive, title: "Удалить") { [weak
        ↪ self] _, _, done in
            self?.storage.delete(id)
            done(true)
        }
        delete.image = UIImage(systemName: "trash")
        return UISwipeActionsConfiguration(actions: [delete])
    }
}
```

Свайп **слева направо** (leading, «ведущие» действия у левого края) — закрепить или открепить. Жёлтый фон, иконка `pin.fill` или `pin.slash.fill` в зависимости от текущего состояния. Свайп **справа налево** (trailing) — удалить. Подробно о сторонах свайпа и полном свайпе — глава 12, раздел 12.8.

В замыкания захватываем заметку (`target`) или её `id`, а не `indexPath`: к моменту тапа по кнопке строки могли переехать.

`togglePin` в хранилище (раздел 13.4) переключает флаг и сохраняет с `touch: false`. Хранилище позовёт подписчика, `applyFilter()` перераспределит заметки по секциям, и строка переедет в «Закреплённые».

> **Упражнение 13.2.** Сейчас внутри каждой секции заметки идут по дате изменения. Сделай так, чтобы закреплённые шли в том порядке, в котором их закрепляли: последняя закреплённая — сверху. Подсказка: понадобится новое поле в модели. Что будет со старыми файлами заметок, в которых этого поля нет?

13.8 Редактор — UITextView во весь экран и автосохранение

`NoteEditorViewController` — один большой `UITextView` и надпись с датой изменения сверху. Кнопки «Сохранить» нет: текст сохраняется сам.

```
import UIKit

final class NoteEditorViewController: UIViewController, UITextViewDelegate {

    let storage: NoteStorage
    var note: Note
    let isNew: Bool

    let textView = UITextView()
    let dateLabel = UILabel()

    var saveWorkItem: DispatchWorkItem?
```

```
    var didEverEdit = false

    static let dateFormatter: DateFormatter = {
        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.dateStyle = .long
        formatter.timeStyle = .short
        return formatter
    }()

    init(note: Note?, storage: NoteStorage = .shared) {
        self.storage = storage
        self.note = note ?? Note()
        self.isNew = (note == nil)
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        navigationItem.largeTitleDisplayMode = .never
        title = isNew ? "Новая заметка" : note.title
        setupLayout()
        setupNavigation()
        updateDateLabel()

        NotificationCenter.default.addObserver(
            self,
            selector: #selector(appWillResignActive),
            name: UIApplication.willResignActiveNotification,
            object: nil
        )
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        if isNew && !didEverEdit {
            textView.becomeFirstResponder()
        }
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        flushSave()
```

```
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        // isMovingFromParent — экран действительно сняли со стека
        // («Назад» или свайп от края довели до конца), а не перекрыли.
        guard isMovingFromParent, isNew else { return }
        if note.body.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
            storage.delete(note.id)
        }
    }
}
```

Редактор получает `Note?`: `nil` — создать новую, иначе править существующую. `isNew` запоминает, какой из случаев, — это понадобится при выходе.

`largeTitleDisplayMode = .never` — в редакторе обычный маленький заголовок, крупный остаётся только у списка.

Сохранение при уходе приложения в фон. `UIApplication.willResignActiveNotification` приходит, когда приложение перестаёт быть активным: пользователь свернул его, открыл Пункт управления, пришёл звонок. Если в этот момент последняя правка ещё ждёт своих 0,4 секунды (см. ниже), а систему потом выгрузит приложение из памяти — буквы пропадут. Поэтому по этому сигналу сохраняем немедленно. Отписываться в `deinit` не нужно: с iOS 9 `NotificationCenter` сам перестаёт звать освобождённые объекты, подписанные через `addObserver(_:selector:name:object:)`.

Жизненный цикл экрана. Контроллер получает события по порядку: `viewDidLoad` (один раз, после создания `view`) → `viewWillAppear` → раскладка (`viewWillLayoutSubviews` / `viewDidLayoutSubviews`) → `viewDidAppear` (экран показан) → … → `viewWillDisappear` → `viewDidDisappear` (экран ушёл). Всё, кроме `viewDidLoad`, может повторяться: вернулся на экран — снова `viewWillAppear`. Окончательные размеры view известны не раньше `viewDidLayoutSubviews` (подробно — глава 0, раздел 0.6). Мы используем три метода:

- `viewDidAppear` — ставим курсор в текст новой заметки (почему не раньше — см. главу 12, раздел 12.10);
- `viewWillDisappear` — сохраняем всё, что не успело сохраниться;
- `viewDidDisappear` — удаляем пустую новую заметку.

Почему удаление в `viewDidDisappear`, а не в `viewWillDisappear`, — тонкость, на которой спотыкаются. Свайп от левого края назад можно **не довести** до конца: человек потянул, передумал и отпустил. При этом `viewWillDisappear` **вызывается** (экран начал уходить), а потом экран возвращается. Если удалять в `viewWillDisappear`, пустая новая заметка исчезла бы из хранилища, хотя пользователь остался в ней. `viewDidDisappear` приходит только когда экран действительно скрылся, а `isMovingFromParent` уточняет, что его убрали из стека навигации (наш контроллер — child у `UINavigationController`), а не перекрыли другим экраном. Перекрыть можно по-разному. Лист «Поделиться» из 13.11 — sheet: он не убирает нижний экран из иерархии, и `viewWillDisappear` / `viewDidDisappear` редактора при нём вообще не вызываются. А вот экран с `modalPresentationStyle = .fullScreen` их вызовет, и тогда `isMovingFromParent` будет `false` — редактор остался в стеке, его только накрыли. Для модального случая есть парное свойство

`isBeingDismissed`; Apple в заголовке `UIViewController.h` советует проверять `isBeingDismissed || isMovingFromParent`, если нужно поймать «экран уходит навсегда» (глава 0, раздел 0.6).

Разметка:

```
extension NoteEditorViewController {
    func setupLayout() {
        dateLabel.font = .preferredFont(forTextStyle: .caption1)
        dateLabel.adjustsFontForContentSizeCategory = true
        dateLabel.textColor = .secondaryLabel
        dateLabel.textAlignment = .center

        textView.text = note.body
        textView.font = .preferredFont(forTextStyle: .body)
        textView.adjustsFontForContentSizeCategory = true
        textView.delegate = self
        textView.alwaysBounceVertical = true
        textView.keyboardDismissMode = .interactive
       textView.textContainerInset = UIEdgeInsets(top: 8, left: 12, bottom: 16, right: 12)

        [dateLabel, textView].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview($0)
        }

        NSLayoutConstraint.activate([
            dateLabel.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪   constant: 4),
            dateLabel.leadingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.leadingAnchor),
            dateLabel.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor),

            textView.topAnchor.constraint(equalTo: dateLabel.bottomAnchor, constant: 4),
            textView.leadingAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.leadingAnchor),
            textView.trailingAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.trailingAnchor),
            // Низ текста — к верху клавиатуры (подробно в 13.9).
           textView.bottomAnchor.constraint(equalTo: view.keyboardLayoutGuide.topAnchor),
        ])
    }

    func updateDateLabel() {
        dateLabel.text = Self.dateFormatter.string(from: note.updatedAt)
    }
}
```

`UITextView` — многострочное редактируемое поле. По родословной это прокрутка: `UITextView → UIScrollView → UIView` (глава 0, раздел 0.5). Текст лежит в прокручиваемом содержимом, а само поле — окошко над ним; поэтому у текста есть всё, что есть у `UIScrollView`: `contentOffset`, `contentInset`, `alwaysBounceVertical`, `keyboardDismissMode`, а протокол `UITextViewDelegate` наследует `UIScrollViewDelegate` — делегат текста получит и `scrollViewDidScroll`. Однострочный `UITextField` устроен иначе: это `UIControl`, и об изменениях он сообщает через target-action (событие `.editingChanged`). У `UITextView` ни `addTarget`, ни `addAction` нет — только делегат (`textViewDidChange`) или уведомление `UITextView.textDidChangeNotification`. Настройки:

- `delegate = self` — редактор будет получать события текста (`textViewDidChange`), для этого контроллер объявлен как `UITextViewDelegate`. Метод приходит только на правки пользователя: наше `textView.text = note.body` при настройке его не вызовет, и `didEverEdit` не поднимется зря;
- `alwaysBounceVertical = true` — текст «пружинит» при прокрутке, даже если он короткий и помещается на экран; без этого короткая заметка ощущается «приклеенной» (по умолчанию `false`);
- `keyboardDismissMode = .interactive` — клавиатуру можно утянуть пальцем вниз, прокручивая текст, как в «Сообщениях»;
- `textContainerInset` — внутренние поля вокруг текста: 12 точек слева и справа, чтобы буквы не липли к краю экрана. По умолчанию поля `(8, 0, 8, 0)`, и к горизонтальным всегда добавляются ещё 5 точек `textContainer.lineFragmentPadding` — так что от края поля до первой буквы будет 12 + 5 = 17 точек.

Дата — `.long` + `.short`: «27 сентября 2026 г., 10:15».

Автосохранение с задержкой:

```
extension NoteEditorViewController {
    func textViewDidChange(_ textView: UITextView) {
        didEverEdit = true
        scheduleSave()
    }

    func scheduleSave() {
        saveWorkItem?.cancel()
        let work = DispatchWorkItem { [weak self] in self?.flushSave() }
        saveWorkItem = work
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.4, execute: work)
    }

    func flushSave() {
        saveWorkItem?.cancel()
        saveWorkItem = nil
        guard didEverEdit, note.body != textView.text else { return }
        note.body = textView.text
        note = storage.save(note)
        updateDateLabel()
        title = note.title
    }
```

```
    @objc func appWillResignActive() {
        flushSave()
    }
}
```

Что происходит по шагам:

1. Пользователь печатает букву — вызывается `textViewDidChange`.
2. `scheduleSave` отменяет ранее запланированное сохранение и планирует новое через 0,4 секунды.
3. Пока человек печатает быстрее, чем раз в 0,4 секунды, каждое сохранение отменяется новой буквой и не успевает выполниться.
4. Как только он замер на 0,4 секунды — срабатывает `flushSave`.
5. Берём текст из `UITextView`, сохраняем, обновляем дату и заголовок.

На числах: человек набирает слово «молоко» — 6 букв с интервалом 0,15 секунды, то есть за 0,75 секунды. Без задержки было бы 6 записей файла. С задержкой — одна, через 0,4 секунды после последней буквы.

`DispatchWorkItem` — кусок работы, который можно отменить. `cancel()` не даёт ему запуститься, если время ещё не пришло. Альтернатива — `Timer`, но с ним больше кода.

`[weak self]` в работе: если пользователь ушёл с экрана и контроллер освободился, отложенное сохранение не будет держать его в памяти. (Не страшно и потерять его: `viewWillDisappear` уже вызвал `flushSave`.)

`didEverEdit` — «пользователь хоть раз менял текст». Без этого флага простое открытие и закрытие заметки обновило бы её дату изменения, и она прыгнула бы наверх списка. Вторая проверка, `note.body != textView.text`, отсекает случай «набрал букву и стёр» — текст тот же, писать на диск нечего.

`note = storage.save(note)` — забираем из хранилища обновлённую версию: у неё новая `updatedAt`, и надпись с датой покажет правильное время.

> **Зачем `flushSave` при выходе.** Пользователь может не дать 0,4 секунды задержке сработать: напечатал и сразу нажал «Назад». Без `flushSave` в `viewWillDisappear` последние буквы потерялись бы.

13.9 Клавиатура — keyboardLayoutGuide и ручной способ

Когда появляется клавиатура, она перекрывает нижнюю часть экрана — около 300 точек на iPhone. Если ничего не делать, последние строки заметки окажутся под клавиатурой, и человек будет печатать вслепую.

В iOS 15 у каждого view появился `keyboardLayoutGuide` — невидимая направляющая, которая всегда стоит там, где верх клавиатуры. В разметке выше мы привязали к ней низ текста:

```
textView.bottomAnchor.constraint(equalTo: view.keyboardLayoutGuide.topAnchor)
```

Одна строка — и больше ничего: клавиатура выезжает — направляющая поднимается — `UITextView` становится ниже, система анимирует это синхронно с клавиатурой. Когда клавиа-

туры нет, направляющая прижата к низу окна и по высоте равна нижнему отступу safe area — то есть её верх совпадает с нижней границей safe area, и текст идёт до низа экрана (над home-индикатором). Сама направляющая — не view: `UIKeyboardLayoutGuide → UITrackingLayoutGuide → UILayoutGuide → NSObject`, прямоугольник-ориентир для констрейнтов, который ничего не рисует и не получает касаний (глава 0, раздел 0.8).

До iOS 15 то же самое делали вручную, через уведомления о клавиатуре. Этот способ полезно знать: он встречается в старом коде, а ещё в нём хорошо видно, что происходит с координатами.

```
// Старый способ, для сравнения. keyboardConstraint —
// textView.bottomAnchor == view.safeAreaLayoutGuide.bottomAnchor.
@objc func keyboardWillChange(_ note: Notification) {
    guard let frame = note.userInfo?[UIResponder.keyboardFrameEndUserInfoKey] as? CGRect
    else { return }
    let keyboardTop = view.convert(frame, from: nil).minY
    let overlap = view.bounds.height - keyboardTop
    keyboardConstraint.constant = -max(0, overlap - view.safeAreaInsets.bottom)
    UIView.animate(withDuration: 0.25) { self.view.layoutIfNeeded() }
}
```

Разберём на числах для iPhone 16 Pro (экран 402 × 874 точки):

1. `keyboardFrameEndUserInfoKey` — где клавиатура **окажется** после анимации, в координатах экрана. Пусть её верх — на `y = 538`, а высота — 336 точек (538 + 336 = 874, низ экрана).
2. `view.convert(frame, from: nil)` переводит рамку из координат экрана в координаты нашего `view` (`nil` — «из координат окна»). У полноэкранного приложения окно совпадает с экраном, а наш `view` занимает всё окно, поэтому числа те же: верх клавиатуры = 538. В Split View на iPad окно меньше экрана, и без `convert` расчёт поехал бы — Apple в описании `keyboardFrameEndUserInfoKey` прямо требует этого перевода.
3. `overlap = 874 - 538 = 336` — сколько точек снизу закрыто клавиатурой.
4. Наш констрейнт привязан к низу safe area, а она уже на 34 точки выше края экрана (полоска home-индикатора). Значит, текст надо поднять на 336 − 34 = 302 точки: `constant = -302`. Минус — потому что «вверх» в UIKit это уменьшение `y`.
5. `max(0, …)` — защита: когда клавиатура уезжает, её рамка уходит за низ экрана, `overlap` становится отрицательным, и подъём должен стать 0, а не отрицательным.
6. `UIView.animate(withDuration: 0.25) { layoutIfNeeded() }` — плавно за четверть секунды перестроить разметку. Почему именно `layoutIfNeeded`: новое значение `constant` само по себе ничего не двигает, оно лишь помечает разметку устаревшей, а рамки Auto Layout пересчитает в ближайшем проходе layout — уже **после** того, как блок анимации закончится, и текст прыгнул бы мгновенно. `layoutIfNeeded()` внутри блока заставляет пройти раскладку сейчас: рамки меняются внутри блока, и UIKit анимирует именно их. Это стандартный приём для анимации любых констрейнтов (глава 0, раздел 0.3).

Длительность 0,25 — приближение. Точная длительность и кривая анимации клавиатуры приходят в том же `userInfo` (`keyboardAnimationDurationUserInfoKey`, `keyboardAnimationCurveUserInfoKey`), и аккуратный код берёт их оттуда. `keyboardLayoutGuide` делает всё это сам — поэтому в новом коде используй его.

13.10 Меню действий в навигационной панели — UIMenu

В правом углу панели редактора — кнопка `ellipsis.circle` («···» в кружке) с меню:

```
extension NoteEditorViewController {
    func setupNavigation() {
        let pinAction = UIAction(
            title: note.pinned ? "Открепить" : "Закрепить",
            image: UIImage(systemName: note.pinned ? "pin.slash" : "pin")
        ) { [weak self] _ in
            self?.togglePin()
        }
        let shareAction = UIAction(
            title: "Поделиться",
            image: UIImage(systemName: "square.and.arrow.up")
        ) { [weak self] _ in
            self?.shareNote()
        }
        let deleteAction = UIAction(
            title: "Удалить",
            image: UIImage(systemName: "trash"),
            attributes: .destructive
        ) { [weak self] _ in
            self?.deleteNote()
        }
        let menu = UIMenu(children: [pinAction, shareAction, deleteAction])
        navigationItem.rightBarButtonItem = UIBarButtonItem(
            image: UIImage(systemName: "ellipsis.circle"),
            menu: menu
        )
        navigationItem.rightBarButtonItem?.accessibilityLabel = "Действия"
    }

    func togglePin() {
        flushSave()
        note.pinned.toggle()
        note = storage.save(note, touch: false)
        setupNavigation()
    }

    func deleteNote() {
        saveWorkItem?.cancel()
        didEverEdit = false
        storage.delete(note.id)
        navigationController?.popViewController(animated: true)
    }
}
```

`UIBarButtonItem(image:menu:)` (iOS 14+) — кнопка, которая по тапу сразу открывает меню. Не нужен отдельный `UIAlertController` в стиле `.actionSheet`: меню появляется рядом с кнопкой, как в системных приложениях. `UIBarButtonItem` при этом — не view (`UIBarButtonItem → UIBarItem → NSObject`), а описание кнопки, по которому панель сама рисует настоящую view. Поэтому у него нет `frame`, `addSubview` и `addAction` — только `target/action`, `primaryAction` или `menu` (глава 0, раздел 0.8).

`attributes: .destructive` — красная подпись и иконка у «Удалить». Системный сигнал «осторожно, необратимо».

Меню пересобирается после закрепления. После `togglePin` подпись должна смениться с «Закрепить» на «Открепить». У `UIAction` свойства `title` и `image` изменяемые, так что можно было бы хранить `pinAction` и править его на месте, но проще и нагляднее собрать меню заново — поэтому `setupNavigation()` вызывается ещё раз.

`deleteNote` сначала отменяет отложенное сохранение и сбрасывает `didEverEdit`. Иначе `viewWillDisappear` при уходе с экрана вызвал бы `flushSave`, и тот заново записал бы только что удалённую заметку на диск. `popViewController` — вернуться на предыдущий экран стека.

13.11 Поделиться — UIActivityViewController

```
extension NoteEditorViewController {
    func shareNote() {
        flushSave()
        let text = note.body.isEmpty ? note.title : note.body
        let activity = UIActivityViewController(activityItems: [text],
        ↪  applicationActivities: nil)
        activity.popoverPresentationController?.barButtonItem =
↪   navigationItem.rightBarButtonItem
        present(activity, animated: true)
    }
}
```

`UIActivityViewController` — стандартный лист «Поделиться» (*share sheet*). Передаёшь массив объектов (`activityItems`), iOS сама показывает подходящие приложения и действия: «Сообщения», «Почта», AirDrop, «Скопировать». Мы передаём строку — все эти приложения умеют принимать текст.

`flushSave()` в начале — чтобы поделиться тем, что на экране прямо сейчас, а не версией полсекунды назад.

`popoverPresentationController?.barButtonItem` — для iPad. На нём лист «Поделиться» показывается как всплывающее окно со стрелкой (*popover*), и ему нужно знать, **откуда** стрелка. Без этой строки приложение на iPad упадёт при показе с исключением `NSGenericException` («UIPopoverPresentationController should have a non-nil sourceView or barButtonItem»). На iPhone свойство тоже не `nil`: у `UIActivityViewController` `modalPresentationStyle` сразу стоит `.popover` (проверено в симуляторе iOS 26.5), просто в компактной ширине popover сам адаптируется в sheet, и `barButtonItem` не используется. `?.` нужен потому, что свойство опциональное: у обычного view controller'а без стиля `.popover` оно `nil`. Сам `UIPopoverPresentationController`

— не view и не view controller (`→ UIPresentationController → NSObject`), это объект, который управляет показом (глава 0, раздел 0.6).

> **Упражнение 13.3.** Добавь в меню редактора пункт «Скопировать» (иконка `doc.on.doc`), который кладёт текст заметки в буфер обмена без листа «Поделиться». Проверка: скопируй, открой новую заметку, нажми в тексте и выбери «Вставить» — появится тот же текст.

13.12 Бытовая аналогия

Notes — это **записная книжка на кольцах**. Каждая страница — один файл: её можно вынуть, заменить или выбросить, не трогая остальные (в отличие от UserDefaults, который больше похож на одну длинную тетрадь — чтобы поправить строчку, её переписывают целиком). Закреплённые заметки — страницы с закладкой, книжка открывается на них первыми. Все страницы лежат в одной папке на полке — `Documents/Notes/`.

Поиск — это **листание в поисках нужной фразы**. Для сотни страниц пролистать быстро. Для десяти тысяч понадобится предметный указатель — поисковый индекс.

Автосохранение — как **секретарь, который записывает за тобой**: ты диктуешь, а он переносит в журнал не каждое слово по отдельности, а целую фразу, когда ты делаешь паузу. Говорить «запиши» не нужно.

13.13 Что мы пропустили

- **Форматированный текст и Markdown.** У нас обычный текст. Жирный, списки, заголовки — через `NSAttributedString` и свой парсер Markdown, это отдельная большая тема.
- **Вложения.** Фото внутри заметки — через `NSTextAttachment`.
- **Папки и теги.** Группировка заметок.
- **Синхронизация через iCloud**, чтобы заметки были и на iPad. Делается через CloudKit или контейнер iCloud Documents.
- **Отмена изменений.** У `UITextView` есть встроенный `undoManager`: встряхивание телефона предлагает «Отменить ввод». Для текста работает само, но его стоит проверить после наших программных правок.

Упражнение 13.4. Пройди сценарий целиком:

1. Создай заметку: первая строка «Список покупок», дальше «молоко», «хлеб».
2. Вернись назад. В списке — заголовок «Список покупок» и превью «молоко хлеб».
3. Проведи по строке слева направо и нажми «Закрепить» — заметка переехала в секцию «Закреплённые».
4. Введи в поиске «ПОКУПОК» — заметка находится, регистр не важен.
5. Создай новую заметку и сразу нажми «Назад», ничего не написав — в списке она не появилась.

Ответы к упражнениям

Упражнение 13.1. Файлов столько же, сколько заметок, — два. Внутри — JSON с отступами, как в разделе 13.3. Удалённый вручную файл после перезапуска пропадёт из списка: хранилище при старте читает папку заново, а о заметке, которой нет на диске, ничего не знает.

Упражнение 13.2. Добавь в `Note` поле `var pinnedAt: Date?`, в `togglePin` ставь `note.pinnedAt = note.pinned ? Date() : nil`, а в `applyFilter()` сортируй закреплённые отдельно:

```
pinnedNotes = found.filter { $0.pinned }
    .sorted { ($0.pinnedAt ?? .distantPast) > ($1.pinnedAt ?? .distantPast) }
```

Со старыми файлами ничего страшного не случится: у опционального поля автоматический `Decodable` считает отсутствие ключа значением `nil`. Будь поле необязательным без `?` — старые заметки перестали бы читаться, и `compactMap` в `load()` молча выбросил бы их все. `.distantPast` — «очень давно»: заметки без даты закрепления уйдут в конец.

Упражнение 13.3. В `setupNavigation()`:

```
let copyAction = UIAction(
    title: "Скопировать",
    image: UIImage(systemName: "doc.on.doc")
) { [weak self] _ in
    guard let self else { return }
    UIPasteboard.general.string = self.textView.text
}
```

и добавь `copyAction` в `children` меню. `UIPasteboard.general` — системный буфер обмена. На iOS 14+ при **чтении** чужого буфера система показывает уведомление «Приложение вставило из …»; запись в буфер уведомлений не вызывает.

Упражнение 13.4. Ожидаемый результат — в самих шагах. Если в пункте 5 пустая заметка появилась, проверь, что удаление стоит в `viewDidDisappear` и что условие `isNew` верно передаётся из списка (`openEditor(for: nil)` для новой).

Что мы выучили

- Заметки храним **файлом на запись** в `Documents/Notes/<UUID>.json` через `FileManager`. Documents — для пользовательских данных, Caches — для того, что можно восстановить.
- Атомарная запись `data.write(to:options: .atomic)` — на диске никогда не окажется половина файла. Сначала пишем диск, потом меняем массив в памяти.
- Вычисляемые `title` и `preview` не сохраняются: первая строка текста становится заголовком. `split(omittingEmptySubsequences: false)` не теряет пустую первую строку.
- Закрепление не меняет дату изменения (`save(touch: false)`).
- `UISearchController` встраивается в `navigationItem.searchController`. `updateSearchResults(for:)` вызывается на каждое изменение текста. Поиск через `range(of:options: [.caseInsensitive, .diacriticInsensitive])` находит «Ёлку» по «елка».
- `UIListContentConfiguration` — стандартная ячейка без своего класса.

- Автосохранение с задержкой через `DispatchWorkItem`: каждая буква отменяет прошлую запись и планирует новую через 0,4 секунды. Плюс принудительное сохранение при уходе с экрана и при сворачивании приложения.
- `UITextView` — это `UIScrollView`: у него есть `contentOffset`, `alwaysBounceVertical`, `keyboardDismissMode`, а делегат наследует `UIScrollViewDelegate`. Он не `UIControl`, поэтому об изменениях сообщает делегатом, а не `addAction`. К `textContainerInset` всегда добавляются 5 точек `lineFragmentPadding`.
- Пустую новую заметку удаляем в `viewDidDisappear` с проверкой `isMovingFromParent` — не в `viewWillDisappear`, который вызывается и при недотянутом свайпе назад. Sheet поверх экрана эти методы не вызывает вовсе, `.fullScreen` — вызывает.
- Клавиатура — `view.keyboardLayoutGuide` (iOS 15+) одной строкой; ручной способ через `keyboardFrameEndUserInfoKey` — для старого кода. Анимировать констрейнт — значит менять `constant` и звать `layoutIfNeeded()` внутри блока анимации.
- `UIBarButtonItem(image:menu:)` — кнопка с меню (сам item — не view); `UIActivityViewController` — «Поделиться», на iPad обязателен `popoverPresentationController?.barButtonItem`; на iPhone он тоже не `nil`, popover просто адаптируется в sheet.

Apple Developer Documentation

- UITextView — многострочное редактируемое поле, основной экран редактора заметок.
- UISearchController — встраивается в `navigationItem.searchController`, даёт системную строку поиска.
- UISearchResultsUpdating — протокол с `updateSearchResults(for:)`, вызывается при каждом изменении текста поиска.
- FileManager — единая точка доступа к файловой системе, через неё получаем `Documents/` и создаём папки.
- URL (filesystem) — тип-«адрес» файла; `appendingPathComponent(_:isDirectory:)` собирает путь корректно для папки.
- Data — байтовый буфер; `data.write(to:options: .atomic)` даёт атомарную запись.
- String.Encoding — кодировки для конвертации `String ↔ Data`; `JSONEncoder` всегда пишет UTF-8.

→ Глава 14. Calculator — UIStackView grid, state machine, haptics

Глава 14. Calculator — UIStackView grid, state machine, haptics

Калькулятор — на удивление поучительное мини-приложение. Он не работает ни с сервером, ни с хранилищем, зато даёт три ценных навыка:

1. **Сетка кнопок** из вложенных `UIStackView`. Подстраивается под размер экрана, обходится без `UICollectionView`.
2. **Конечный автомат** (*state machine*) — логика вычислений отдельно от интерфейса и покрыта тестами.
3. **Тактильный отклик** (*haptics* — лёгкая вибрация под пальцем) на каждое нажатие и отдельный — для ошибки.

А ещё калькулятор — лучший способ на своей шкуре узнать, как компьютер считает дробные числа и почему `0.1 + 0.2` у него не равно `0.3`. В этой главе разбираем всё это.

Что строим:

```
┌──────────────────────────┐
│                          │
│                   1 234  │ ← дисплей: сжимается, если число длинное
│  (AC) (±)  (%)  (÷)      │ ← AC превращается в C, когда есть что стирать
│  (7)  (8)  (9)  (×)      │
│  (4)  (5)  (6)  (−)      │ ← нажатый оператор подсвечивается белым
│  (1)  (2)  (3)  (+)      │
│  (   0   ) (,)  (=)      │ ← «0» — на две клетки; «,» или «.» по локали
└──────────────────────────┘
```

Проект — как в главе 12 (раздел «Перед началом»): App, Swift 6, Default Actor Isolation = MainActor, iOS 15+, без `Main.storyboard`. Калькулятору не нужна навигационная панель, поэтому контроллер ставим корнем окна напрямую:

```swift
import UIKit

final class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }
```

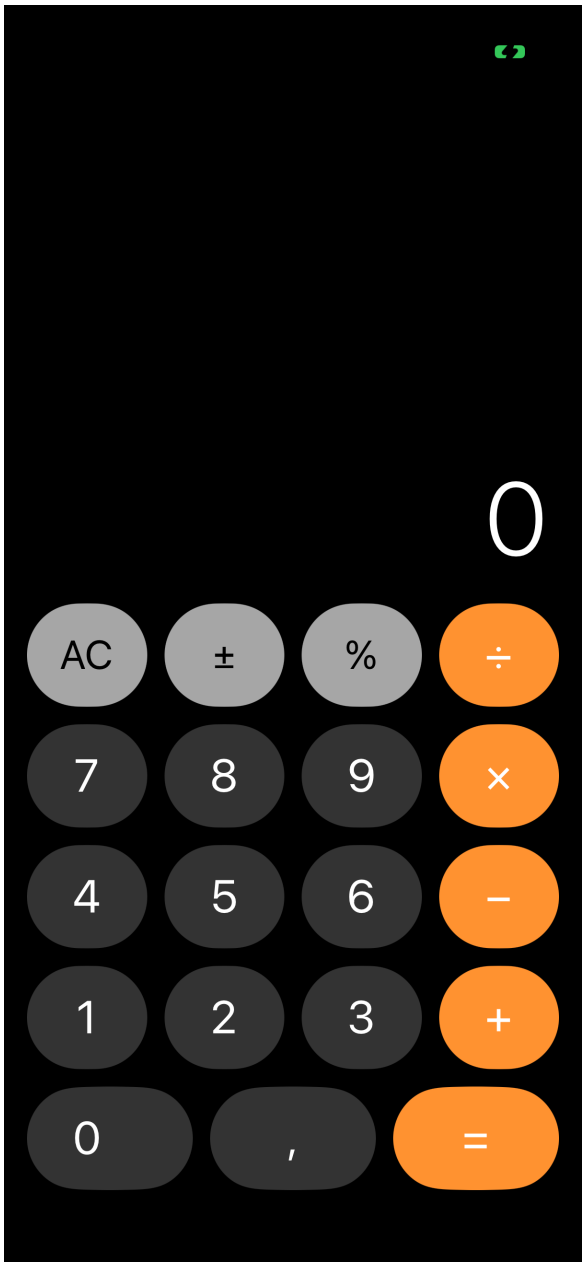


Рис. 10: Калькулятор в стиле Apple

```
        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = CalculatorViewController()
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

14.1 Архитектура — движок отдельно, экран отдельно

Главное правило калькулятора: **логика отделена от интерфейса**. Логику держит `CalculatorEngine` — структура, которая ничего не знает про UIKit. Ей говорят «нажата клавиша 5», она меняет своё состояние и отвечает на вопрос «что сейчас на дисплее».

```
import Foundation

nonisolated struct CalculatorEngine: Sendable {

    enum Key: Equatable, Sendable {
        case digit(Int)
        case dot
        case binary(BinaryOp)
        case equals
        case clear
        case negate
        case percent
    }

    enum BinaryOp: String, Equatable, Hashable, Sendable {
        case add = "+", sub = "−", mul = "×", div = "÷"

        /// Умножение и деление «сильнее» сложения и вычитания.
        var isHighPriority: Bool { self == .mul || self == .div }

        func apply(_ a: Double, _ b: Double) -> Double {
            switch self {
            case .add: return a + b
            case .sub: return a - b
            case .mul: return a * b
            case .div: return b == 0 ? .nan : a / b
            }
        }
    }

    struct Pending: Sendable {
        let lhs: Double
        let op: BinaryOp
    }
```

```
    static let maxDigits = 12

    private var input = "0"               // текст дисплея (внутри всегда с точкой)
    private var shownResult: Double?      // точное значение, если на дисплее результат
    private var pendingLow: Pending?      // ждущее «+» или «−»
    private var pendingHigh: Pending?     // ждущее «×» или «÷»
    private var lastOp: BinaryOp?         // последний нажатый оператор
    private var repeatOperation: Pending? // для повторного «=»
    private var startsNewInput = true     // очередная цифра начнёт новое число
    private var justPressedOp = false     // оператор нажат только что
    private var hasError = false

    init() {}

    var display: String { hasError ? "Ошибка" : input }
    var isError: Bool { hasError }

    /// Оператор, который надо подсветить: нажат, а второе число ещё не начато.
    var activeOp: BinaryOp? { justPressedOp ? lastOp : nil }

    /// true — кнопка сброса «AC» (стереть всё), false — «C» (стереть число).
    var showsAllClear: Bool {
        hasError || startsNewInput || input == "0"
    }

    mutating func input(_ key: Key) {
        if hasError {
            guard key == .clear else { return }
        }
        switch key {
        case .digit(let d):   appendDigit(d)
        case .dot:            appendDot()
        case .binary(let op): setBinaryOp(op)
        case .equals:         evaluate()
        case .clear:          clear()
        case .negate:         negate()
        case .percent:        percent()
        }
    }
}
```

Почему `struct`. Структура — значение: её можно скопировать, и у копии будет своё независимое состояние. Для калькулятора это удобно: «каким был калькулятор до этого нажатия» — просто старая копия. Каждое нажатие — метод `mutating`, который меняет состояние. Движок не рассылает уведомлений, не трогает UIKit и не зависит от потоков.

Почему `nonisolated`. В проекте с Default Actor Isolation = MainActor всё, что объявлено без пометок, привязано к главному потоку — и эта структура тоже была бы. Движку это не нужно: он просто считает. `nonisolated` снимает привязку, и движок можно использовать где угодно: из теста, из фоновой задачи, из командной строки на Mac (так мы его и проверим в 14.4). `Sendable` — «можно передавать между потоками»; для структуры из чисел и строк это правда, и компилятор с этим согласен.

`Key` — все клавиши калькулятора одним перечислением. `.digit(Int)` — цифра с *ассоциированным значением* (какая именно), `.binary(BinaryOp)` — один из четырёх операторов. Экран не зовёт отдельные методы `appendDigit`, `setOperator` — он передаёт клавишу в единственный `input(_:)`. Так у движка одна «дверь», и её легко тестировать: «нажми вот эти клавиши, проверь дисплей».

`BinaryOp.apply` — сама арифметика. Деление на ноль возвращает `.nan` (*Not a Number*, «не число» — специальное значение `Double` для бессмысленных результатов), а движок превратит его в «Ошибка». Почему не довериться `Double`: `5 / 0` в `Double` даёт не ошибку, а +∞ (бесконечность), а `0 / 0` — `nan`. Показывать пользователю «inf» странно, поэтому все «нечисла» ловим в одном месте (14.3, метод `show`).

`rawValue` операторов — символы `−`, `×`, `÷`. Обрати внимание: `-` — это типографский минус (U+2212), а не дефис `-` с клавиатуры. Он шире и стоит по центру цифр, как в настоящих калькуляторах.

`display` — единственное, что экрану нужно показать. `activeOp` и `showsAllClear` — подсказки для подсветки оператора и подписи кнопки сброса (14.7).

Цикл «событие → состояние → отрисовка». Экран зовёт `engine.input(.digit(5))`, потом читает `engine.display` и рисует. Данные идут в одну сторону: нажатие меняет движок, движок определяет экран, и никогда наоборот. Эту идею называют *однонаправленным потоком данных* (unidirectional data flow); на ней построены Redux, Elm и SwiftUI.

> **Зачем разделять.** Без разделения логика расползается по обработчикам кнопок. Через полгода калькулятор внезапно выдаёт ерунду на `5 + 5 =`, и непонятно, где искать. С отдельным движком ты пишешь тесты, отлаживаешь без симулятора и спокойно меняешь интерфейс.

14.2 Состояния — как «думает» калькулятор

Любой калькулятор внутри — конечный автомат: у него есть конечный набор ситуаций, и каждое нажатие переводит из одной в другую. Ситуации:

- **Ввод первого числа.** Нажимаешь цифры — они дописываются.
- **Оператор нажат.** Ждём второго числа; очередная цифра начнёт его.
- **Ввод второго числа.**
- **Результат.** Нажал `=`, видишь ответ; очередная цифра начнёт новое вычисление, а оператор продолжит с результатом.
- **Ошибка.** Деление на ноль или переполнение.

Эти ситуации не хранятся одним `enum State`. Они получаются из нескольких полей:

- `input` — текст на дисплее, например `"12.5"`. Храним **текст**, а не число: пока человек набирает `12.` (с точкой в конце) или `0.00`, это ещё не число, а заготовка, и нули после точки нельзя потерять.
- `shownResult` — если на дисплее результат вычисления, здесь его **точное** значение. Дисплей показывает 12 значащих цифр, а `Double` хранит около 16; для дальнейших операций берём точное.
- `pendingLow` / `pendingHigh` — отложенные операции: «левое число и оператор», которые ждут правого числа. Две ячейки — чтобы соблюдать порядок действий (14.3).
- `startsNewInput` — «очередная цифра начнёт новое число, а не допишется».
- `justPressedOp` — «оператор нажат только что»; нужен, чтобы заменить оператор, если нажали другой, и чтобы подсветить кнопку.

`startsNewInput` — ключевой флаг. Без него ввод `5 + 3` сломался бы:

```
[5]  → input = "5"
[+]  → запомнили «5 +»
[3]  → input = "5" + "3" = "53"   ← неправильно
```

С флагом:

```
[5]  → input = "5"
[+]  → запомнили «5 +», startsNewInput = true
[3]  → startsNewInput, поэтому input = "3", флаг снят
[=]  → 5 + 3 = 8
```

14.3 Главный метод и переходы

`input(_:)` из 14.1 — диспетчер: первой строкой он защищает от ошибки (если на дисплее «Ошибка», работает только сброс — иначе пользователь нажимал бы цифры поверх «Ошибки» и запутался), а дальше передаёт клавишу своему методу. Каждый метод — один переход состояния. Все они в расширении того же файла:

```
nonisolated extension CalculatorEngine {
    private var currentValue: Double {
        shownResult ?? Double(input) ?? 0
    }

    private mutating func beginTyping(with text: String) {
        input = text
        shownResult = nil
        startsNewInput = false
        justPressedOp = false
    }

    private mutating func appendDigit(_ d: Int) {
        if startsNewInput {
            beginTyping(with: "\(d)")
            return
        }
        if input == "0" { input = "\(d)"; return }
```

```
        if input == "-0" { input = "-\(d)"; return }
        let digitCount = input.filter(\.isNumber).count
        guard digitCount < Self.maxDigits else { return }
        input.append("\(d)")
    }

    private mutating func appendDot() {
        if startsNewInput {
            beginTyping(with: "0.")
            return
        }
        guard !input.contains(".") else { return }
        input.append(".")
    }
}
```

`nonisolated extension`. Пометка `nonisolated` на самой структуре **не** распространяется на её расширения: при Default Actor Isolation = MainActor расширение без пометки снова привязано к главному потоку. Тогда `input(_:)` (он в основном объявлении, свободный) не смог бы вызвать `appendDigit` из расширения — компилятор скажет «call to main actor-isolated instance method ‘appendDigit’ in a synchronous nonisolated context». Поэтому каждое расширение движка помечаем так же, как саму структуру. Первая версия этой главы на этом и споткнулась: программа-проверка на Mac (14.4) собиралась, потому что там нет настройки MainActor по умолчанию, а проект в Xcode — нет.

`currentValue` — число, с которым сейчас работаем: точный результат, если он на дисплее, иначе набранный текст, переведённый в `Double`. `Double("12.")` вполне понимает точку в конце и даёт 12.

`appendDigit` — четыре ветки:

1. `startsNewInput` — начинаем новое число с этой цифры.
2. На дисплее `"0"` — заменяем, чтобы не получилось `"05"`.
3. На дисплее `"-0"` (нажали «±» до цифр) — то же, со знаком: `"-5"`.
4. Иначе дописываем в конец, но не больше 12 цифр.

`input.filter(\.isNumber).count` — считаем только цифры: минус и точка в лимит не входят. `"-1234567890.12"` — 14 символов, но 12 цифр, больше дописать нельзя. Лимит в 12 цифр — это примерно столько, сколько `Double` хранит точно (о точности — в 14.4).

`appendDot` — точка дважды не ставится: `1..5` даст `1.5`, `1.2.3` — `1.23` (вторая точка просто игнорируется). Если точку нажали в начале нового числа, получаем `"0."` — так привычнее, чем `"."`.

Операторы — самое интересное. Как калькулятор iPhone, наш соблюдает **порядок действий**: умножение и деление выполняются раньше сложения и вычитания. `2 + 3 × 4 = 14`, а не 20.

```
nonisolated extension CalculatorEngine {
    private mutating func setBinaryOp(_ op: BinaryOp) {
        var value: Double
        if justPressedOp {
```

```
            // Оператор нажат дважды подряд: забираем то, что положили
            // прошлым оператором, и переигрываем с новым.
            if let high = pendingHigh {
                value = high.lhs
                pendingHigh = nil
            } else if let low = pendingLow {
                value = low.lhs
                pendingLow = nil
            } else {
                value = currentValue
            }
        } else {
            value = currentValue
        }

        if let high = pendingHigh {
            value = high.op.apply(high.lhs, value)
            pendingHigh = nil
        }
        if op.isHighPriority {
            pendingHigh = Pending(lhs: value, op: op)
        } else {
            if let low = pendingLow {
                value = low.op.apply(low.lhs, value)
            }
            pendingLow = Pending(lhs: value, op: op)
        }
        lastOp = op
        repeatOperation = nil
        guard show(value) else { return }
        startsNewInput = true
        justPressedOp = true
    }
}
```

Идея: две «полки» для отложенных операций. На нижней (`pendingLow`) ждут сложение и вычитание, на верхней (`pendingHigh`) — умножение и деление. Правило одно: **верхняя полка всегда сворачивается первой**.

Пройдём `2 + 3 × 4 =` по шагам:

Нажато	Что происходит	Нижняя полка	Верхняя	Дисплей
2	набираем	—	—	2
+	«+» слабый: кладём `2 +` на нижнюю	`2 +`	—	2
3	набираем	`2 +`	—	3

Нажато	Что происходит	Нижняя полка	Верхняя	Дисплей
×	«×» сильный: кладём 3 × на верхнюю, нижнюю не трогаем	2 +	3 ×	3
4	набираем	2 +	3 ×	4
=	сворачиваем верхнюю: 3 × 4 = 12, потом нижнюю: 2 + 12 = 14	—	—	14

А вот 2 × 3 + 4 =: при нажатии + верхняя полка (2 ×) сворачивается сразу — 2 × 3 = 6, на дисплее 6 — и 6 + уходит на нижнюю. Итог 6 + 4 = 10.

Когда приходит слабый оператор и на нижней полке уже что-то есть, оно тоже сворачивается: 10 − 2 − 3 считается слева направо, (10 − 2) − 3 = 5. Этого хватает, потому что уровней приоритета всего два. Для скобок и степеней понадобился бы настоящий разбор выражений — это уже другая программа.

Замена оператора. Если оператор нажат сразу после другого (`justPressedOp`), пользователь ошибся клавишей. Прошлый оператор положил левое число на полку — забираем его обратно и переигрываем с новым. 5 + × 3 = — это 5 × 3 = 15. Без этой ветки получилось бы 5 + 5 × 3, потому что «правым числом» для + стало бы то же 5.

`repeatOperation = nil` — любой новый оператор сбрасывает память для повторного = (о нём ниже).

Вычисление, сброс, смена знака, процент:

```
nonisolated extension CalculatorEngine {
    private mutating func evaluate() {
        let rhs = currentValue
        var value = rhs
        if pendingHigh == nil && pendingLow == nil {
            // Повторное «=»: 2 + 3 = = → 5, потом 8.
            guard let repeatOp = repeatOperation else {
                startsNewInput = true
                return
            }
            value = repeatOp.op.apply(value, repeatOp.lhs)
        } else {
            if let lastOp {
                repeatOperation = Pending(lhs: rhs, op: lastOp)
            }
            if let high = pendingHigh {
                value = high.op.apply(high.lhs, value)
            }
```

```
            if let low = pendingLow {
                value = low.op.apply(low.lhs, value)
            }
            pendingHigh = nil
            pendingLow = nil
        }
        guard show(value) else { return }
        startsNewInput = true
        justPressedOp = false
    }

    private mutating func clear() {
        if showsAllClear {
            self = CalculatorEngine()
        } else {
            input = "0"
            shownResult = nil
        }
    }

    private mutating func negate() {
        if justPressedOp || (startsNewInput && shownResult == nil) {
            beginTyping(with: "-0")
            return
        }
        if let result = shownResult {
            _ = show(-result)
            return
        }
        if input.hasPrefix("-") {
            input.removeFirst()
        } else {
            input = "-" + input
        }
    }

    private mutating func percent() {
        let value = currentValue
        let result: Double
        if let low = pendingLow, pendingHigh == nil {
            // 50 + 10 % → 10 процентов от 50 = 5
            result = low.lhs * value / 100
        } else {
            result = value / 100
        }
        guard show(result) else { return }
        startsNewInput = true
        justPressedOp = false
```

```
    }

    /// Кладёт вычисленное значение на дисплей. false — это ошибка.
    private mutating func show(_ value: Double) -> Bool {
        guard value.isFinite else {
            hasError = true
            return false
        }
        shownResult = value
        input = Self.format(value)
        return true
    }
}
```

`evaluate` сворачивает обе полки — сначала верхнюю, потом нижнюю. Перед этим запоминает `repeatOperation`: последний оператор и **правое** число. Это для **повторного =**, как в калькуляторе iPhone: `2 + 3 =` даёт 5, ещё раз `=` — 8 (прибавили те же 3), ещё раз — 11. Удобно для «прибавь ещё столько же» и сложных процентов: `1000 × 1.1 = = =` — сумма после трёх лет роста на 10% (1331).

Если нажать `=` без всякого оператора (`9 =`), ничего не считается: на дисплее остаётся 9, только `startsNewInput` — очередная цифра начнёт новое число.

После = очередная цифра начинает новое вычисление, а оператор продолжает с результатом. `7 × 8 = + 2 =` — это 56 + 2 = 58. А `8 = 2` — на дисплее 2, а не «82».

`show(_:)` — единая точка, через которую результат попадает на дисплей. `value.isFinite` — «обычное число»: не бесконечность и не `nan`. Сюда попадают деление на ноль (`nan` из `apply`) и переполнение: `Double` хранит числа примерно до $1{,}8 \times 10^{308}$ (единица и 308 нулей), а дальше — $+\infty$. Возьми 999 999 999 999 и 25 раз подряд умножь результат на него же (`× 999999999999 =`): на 25-м шаге число перевалит за 10^{308}, и дисплей покажет «Ошибка», а не уронит приложение.

`clear` — одна кнопка, два смысла, как у калькулятора iPhone:

- **C** («Clear») — стереть только набираемое число. `12 + 34, C` ⟶ на дисплее 0, но «12 +» помнится: набери `5 =` — будет 17.
- **AC** («All Clear») — сбросить всё. `self = CalculatorEngine()` — у структуры можно просто заменить себя свежим экземпляром.

Подпись на кнопке зависит от `showsAllClear`: если стирать нечего (на дисплее результат, только что нажат оператор или уже 0), кнопка показывает «AC».

`negate` (±) — три случая:

1. Сразу после оператора или в самом начале — начинаем ввод отрицательного числа: `5 + ± 3 =` ⟶ 5 + (−3) = 2. Дисплей покажет `-0`, пока не нажмёшь цифру.
2. На дисплее результат — меняем знак результата.
3. Иначе — добавляем или убираем минус у набираемого текста.

`percent` (%) ведёт себя по-разному в зависимости от контекста — тоже как на iPhone:

- просто `50 %` ⟶ 0,5 (50 делить на 100);

- `50 + 10 %` → 5, то есть «10 процентов от 50». Нажмёшь = — получишь 55: «50 плюс 10 процентов». Именно так считают наценку и скидку;
- `200 × 10 %` → 0,1, и = даёт 20: «200 умножить на 10 процентов».

14.4 Форматирование и точность Double

Остался последний кусок движка — превращение числа в текст:

```
nonisolated extension CalculatorEngine {
    private static let plainFormatter: NumberFormatter = {
        let f = NumberFormatter()
        f.locale = Locale(identifier: "en_US_POSIX")
        f.numberStyle = .decimal
        f.usesGroupingSeparator = false
        f.usesSignificantDigits = true
        f.maximumSignificantDigits = maxDigits
        return f
    }()

    private static let mantissaFormatter: NumberFormatter = {
        let f = NumberFormatter()
        f.locale = Locale(identifier: "en_US_POSIX")
        f.numberStyle = .decimal
        f.usesSignificantDigits = true
        f.maximumSignificantDigits = 8
        return f
    }()

    static func format(_ value: Double) -> String {
        if value == 0 { return "0" }
        let magnitude = abs(value)
        if magnitude < 1e12 && magnitude >= 1e-8 {
            return plainFormatter.string(from: NSNumber(value: value)) ?? String(value)
        }
        // Слишком большое или слишком маленькое — пишем как 1.5e20:
        // «1,5 умножить на 10 в двадцатой степени».
        var exponent = Int(log10(magnitude).rounded(.down))
        var mantissa = value / pow(10, Double(exponent))
        var text = mantissaFormatter.string(from: NSNumber(value: mantissa)) ??
        ↪  String(mantissa)
        if text == "10" || text == "-10" {
            // 9,99999999… после округления стало 10 — сдвигаем порядок.
            exponent += 1
            mantissa /= 10
            text = mantissaFormatter.string(from: NSNumber(value: mantissa)) ??
↪  String(mantissa)
        }
        return "\(text)e\(exponent)"
```

```
    }
}
```

Чтобы понять, зачем всё это, нужно знать, как компьютер хранит дробные числа.

Почему `0.1 + 0.2` не равно `0.3`. `Double` хранит число в двоичной системе — как сумму половинок, четвертинок, восьмушек и так далее. Одна десятая в двоичной системе — бесконечная дробь, как 1/3 = 0,3333… в десятичной. Её приходится обрезать, и в памяти оказывается не 0,1, а 0,1000000000000000055511151231257827… Сложи две такие «почти десятых» с «почти двумя десятыми» — ошибки сложатся:

```
print(0.1 + 0.2)          // 0.30000000000000004
print(0.1 + 0.2 == 0.3)   // false
```

Точность `Double` — около 15–16 значащих десятичных цифр. «Мусор» появляется в 17-й. Если показывать не больше 12 значащих цифр, мусор округлится и исчезнет: 0,30000000000000004 → 0,3. Отсюда `maximumSignificantDigits = 12`.

Значащие цифры — все цифры числа, начиная с первой ненулевой. У 1234,5 их пять; у 0,000123 — три (1, 2, 3; нули впереди только показывают, где запятая). Ограничивать именно значащие, а не «цифры после запятой», правильно: у 1/3 = 0,333333333333 мы покажем 12 троек, а у 123456,789 — все цифры без потерь.

Проверь сам: `1 ÷ 3 × 3` = даст ровно 1. Здесь повезло — ошибки округления в `Double` взаимно погасились. А `0.1 × 3` = в `Double` — 0,30000000000000004, но на дисплее — 0,3.

Когда `Double` не годится. Для денег. Там важна каждая копейка, а «округлим при показе» не спасает: ошибки накапливаются при суммировании тысяч платежей, и итог в отчёте разойдётся с банковской выпиской на тиын. Для денег есть `Decimal` — он считает в десятичной системе, и 0,1 + 0,2 у него ровно 0,3. Подробно — в главе 31 («дата, время, деньги»). Для калькулятора общего назначения `Double` хватает — так же поступает большинство калькуляторов.

Теперь по строкам.

`locale = en_US_POSIX` — специальная «техническая» локаль: точка как десятичный разделитель, никаких пробелов в тысячах, латинские цифры — на любом телефоне. Движок всегда думает с точкой. Заменять точку на запятую для русского пользователя будет экран (14.6): это вопрос отображения, а не логики.

`usesGroupingSeparator = false` — без разделителей тысяч: «1234567», а не «1,234,567». Разделители сбивали бы с толку при дальнейшем вводе.

Экспоненциальная запись. Если число больше или равно 10^{12} (триллиону) или меньше 10^{-8}, в 12 цифр оно не влезет. Тогда пишем его как `1.5e20`: «1,5, умноженное на 10 в двадцатой степени». Как это вычисляется, на числе 150 000 000 000 000 000 000:

1. `log10(1.5e20)` ≈ 20,176 — «10 в какой степени даёт это число». Округляем вниз: `exponent = 20`.
2. `mantissa = value / 10²⁰ = 1,5` — число от 1 до 10.
3. Форматируем мантиссу с 8 значащими цифрами: «1.5».
4. Склеиваем: «1.5e20».

Особый случай — `999 999 999 999 × 999 999 999 999`. Это 999 999 999 998 000 000 000 001, примерно 9,99999999998 × 10^{23}. Мантисса 9,99999999998 при округлении до 8 цифр становится «10», а «10e23» выглядит странно. Поэтому сдвигаем: порядок +1, мантисса делится на 10 — получаем «1e24».

Была ли проблема в простом решении. Первая версия этой главы форматировала так: «если число целое — `String(Int(value))`». На `999999999999 × 999999999999` это **роняло приложение**: `Int` хранит числа только до 9,2 × 10^{18}, и `Int(1e24)` — фатальная ошибка «Double value cannot be converted to Int because the result would be greater than Int.max». Любое преобразование `Double` → `Int` без проверки диапазона — мина.

Проверяем движок без симулятора

Движок не зависит от UIKit — значит, его можно проверить обычной программой на Mac, без симулятора и без Xcode-проекта. Положи рядом с `CalculatorEngine.swift` файл `main.swift`:

```
import Foundation

func run(_ keys: String) -> String {
    var engine = CalculatorEngine()
    for ch in keys {
        switch ch {
        case "0"..."9": engine.input(.digit(Int(String(ch))!))
        case ".": engine.input(.dot)
        case "+": engine.input(.binary(.add))
        case "−": engine.input(.binary(.sub))
        case "*": engine.input(.binary(.mul))
        case "/": engine.input(.binary(.div))
        case "=": engine.input(.equals)
        case "C": engine.input(.clear)
        case "n": engine.input(.negate)
        case "%": engine.input(.percent)
        default: break
        }
    }
    return engine.display
}

var failed = 0

@MainActor
func check(_ keys: String, _ expected: String) {
    let got = run(keys)
    if got != expected {
        failed += 1
        print("ОШИБКА: \(keys) → \(got), ждали \(expected)")
    }
}
```

```
check("2+3*4=", "14")       // порядок действий
check("2*3+4=", "10")
check("10-2-3=", "5")
check("0.1+0.2=", "0.3")    // мусор Double не виден
check("1/3*3=", "1")
check("5/0=", "Ошибка")     // деление на ноль
check("5/0=7", "Ошибка")    // после ошибки цифры не работают
check("5/0=C2+2=", "4")     // AC возвращает к жизни
check("2+3==", "8")         // повторное «=»
check("7*8=+2=", "58")      // продолжение с результатом
check("8=2", "2")           // после «=» цифра начинает заново
check("5++3=", "8")         // замена оператора
check("2+*3=", "6")
check("5+n3=", "2")         // ± после оператора
check("2+3=n", "-5")        // ± у результата
check("1..5", "1.5")        // точка дважды
check(".5+.5=", "1")
check("1234567890123", "123456789012")   // лимит 12 цифр
check("999999999999*999999999999=", "1e24")
check("1/1000000000=", "1e-9")
check("50+10%=", "55")      // процент от первого числа
check("12+34C5=", "17")     // C стирает только число

print(failed == 0 ? "Все проверки прошли" : "Провалено: \(failed)")
```

Собери и запусти в терминале:

```
swiftc -swift-version 6 CalculatorEngine.swift main.swift -o calc-tests
./calc-tests
```

Программа напечатает «Все проверки прошли». Сломай что-нибудь в движке — например, убери ветку `justPressedOp` в `setBinaryOp` — и увидишь, какие проверки упали. При проверке этой главы движок прогнан на 66 таких сценариях, включая все выше.

`main.swift` — особое имя: код верхнего уровня в нём выполняется как программа, без функции `main`. `@MainActor` у `check` нужен из-за Swift 6: глобальная переменная `failed` в `main.swift` принадлежит главному потоку, и менять её может только функция, тоже привязанная к нему. Без пометки компилятор скажет «main actor-isolated var ‘failed’ can not be mutated from a nonisolated context». В настоящем проекте те же проверки удобнее оформить тестами XCTest (File → New → Target → Unit Testing Bundle) — там `check` превратится в `XCTAssertEqual`, а Xcode покажет результаты зелёными и красными значками.

> **Упражнение 14.1.** Не запуская программу, предскажи, что покажет дисплей после каждой из последовательностей, а потом добавь их в `main.swift` и проверь: `2 + 3 × 4 − 5 =`, `9 = =`, `5 + =`, `200 × 10 % =`.

14.5 Интерфейс — сетка из UIStackView

5 рядов по 4 кнопки. Последний ряд — «0» на две клетки, разделитель, «=».

```
import UIKit

final class CalculatorViewController: UIViewController {

    struct ButtonSpec {
        let title: String
        let key: CalculatorEngine.Key
        let background: UIColor
        let foreground: UIColor
        let wide: Bool

        static func gray(_ title: String, _ key: CalculatorEngine.Key) -> ButtonSpec {
            ButtonSpec(title: title, key: key, background: .lightGray, foreground: .black,
↪   wide: false)
        }

        static func orange(_ title: String, _ key: CalculatorEngine.Key) -> ButtonSpec {
            ButtonSpec(title: title, key: key, background: .systemOrange, foreground:
↪   .white, wide: false)
        }

        static func dark(_ title: String, _ key: CalculatorEngine.Key, wide: Bool = false)
         ↪  -> ButtonSpec {
            ButtonSpec(title: title, key: key, background: .darkGray, foreground: .white,
↪   wide: wide)
        }
    }

    var engine = CalculatorEngine()

    let displayLabel = UILabel()
    var operatorButtons: [CalculatorEngine.BinaryOp: UIButton] = [:]
    var clearButton: UIButton?

    let haptic = UIImpactFeedbackGenerator(style: .medium)
    let errorHaptic = UINotificationFeedbackGenerator()

    /// Десятичный разделитель для экрана: «,» для русской локали, «.» для английской.
    let decimalSeparator = Locale.current.decimalSeparator ?? "."

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .black
        overrideUserInterfaceStyle = .dark
        setupLayout()
        render()
    }
}
```

`ButtonSpec` — описание одной кнопки: подпись, клавиша движка, цвета и «широкая ли». Три фабричных метода (`gray`, `orange`, `dark`) — это маленький язык описания: ряд пишется в одну строку, `[.gray("AC", .clear), .gray("±", .negate), …]`, и сразу видно, как выглядит клавиатура. Серые — служебные (AC, ±, %), оранжевые — операторы и «=», тёмные — цифры.

`engine` — `var`: движок структура, и `engine.input(...)` меняет само свойство контроллера.

Разделитель по локали. `Locale.current.decimalSeparator` — какой разделитель дробной части принят в регионе пользователя. В России и Казахстане это запятая: «2,5». В США и Великобритании — точка. Движок внутри всегда считает с точкой (14.4), а на экране мы покажем то, к чему привык человек. *Локаль* — набор региональных правил: разделители, формат дат, первый день недели. Она задаётся в «Настройки → Основные → Язык и регион».

Разметка — дисплей и сетка:

```
extension CalculatorViewController {
    func setupLayout() {
        displayLabel.textColor = .white
        displayLabel.textAlignment = .right
        displayLabel.font = .monospacedDigitSystemFont(ofSize: 80, weight: .light)
        displayLabel.adjustsFontSizeToFitWidth = true
        displayLabel.minimumScaleFactor = 0.4
        displayLabel.accessibilityTraits.insert(.updatesFrequently)

        let grid = buildGrid()
        let stack = UIStackView(arrangedSubviews: [displayLabel, grid])
        stack.axis = .vertical
        stack.spacing = 12
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)

        let margins = view.safeAreaLayoutGuide
        // Сетка хочет быть 4 × 5 квадратных клеток. Приоритет ниже обязательного:
        // в альбомной ориентации места по высоте не хватит, и тогда пусть лучше
        // кнопки сплющатся, чем Auto Layout начнёт ругаться на конфликт.
        let gridRatio = grid.heightAnchor.constraint(equalTo: grid.widthAnchor,
        ↪  multiplier: 5.0 / 4.0)
        gridRatio.priority = .defaultHigh
        NSLayoutConstraint.activate([
            stack.leadingAnchor.constraint(equalTo: margins.leadingAnchor, constant: 16),
            stack.trailingAnchor.constraint(equalTo: margins.trailingAnchor, constant:
↪  -16),
            stack.bottomAnchor.constraint(equalTo: margins.bottomAnchor, constant: -16),
            stack.topAnchor.constraint(greaterThanOrEqualTo: margins.topAnchor, constant:
↪  16),
            gridRatio,
        ])
    }
}
```

Дисплей. Шрифт 80 точек, `.light` — тонкий, как у системного калькулятора. `monospacedDigitSystemFont` — все цифры одной ширины: иначе при наборе «1111» → «1118» текст дёргался бы, потому что «1» у обычного шрифта уже «8». `adjustsFontSizeToFitWidth` + `minimumScaleFactor = 0.4` — если число не влезает в ширину, шрифт уменьшается, но не меньше чем до 40% (80 × 0,4 = 32 точки). 12 цифр в 32 точки на iPhone влезают.

`.updatesFrequently` — подсказка VoiceOver, что текст часто меняется: диктор будет зачитывать его аккуратнее, не перебивая сам себя.

Сетка квадратных клеток. Констрейнт «высота сетки = ширина × 5/4»: 4 колонки и 5 рядов. Пусть ширина сетки 370 точек, промежутки по 12. Ширина клетки: (370 − 3 × 12) / 4 = 83,5 точки. Высота сетки = 370 × 1,25 = 462,5; высота ряда: (462,5 − 4 × 12) / 5 = 82,9 — почти квадрат (разница из-за того, что промежутков по вертикали на один больше, чем по горизонтали). Дисплей при этом остаётся своей естественной высоты — высоты `intrinsicContentSize` надписи, — а всё, что выше, просто пустой чёрный фон: у стека нижний край прибит, а верхний лишь ограничен «не выше safe area» (`greaterThanOrEqualTo`), поэтому растягиваться ему незачем.

Приоритет констрейнта. Каждый констрейнт имеет приоритет от 1 до 1000. 1000 (`.required`) — обязательный; `.defaultHigh` = 750 — «очень хочу, но могу уступить». В альбомной ориентации на iPhone высота экрана ~400 точек, и сетка 4 × 5 из квадратов не влезает. Будь констрейнт обязательным — в консоли появилось бы «Unable to simultaneously satisfy constraints», и система сама выбрала бы, какой выкинуть. С приоритетом 750 он уступает, кнопки становятся ниже, но всё помещается. Одна тонкость: у надписи дисплея вертикальное сопротивление сжатию тоже 750 — это значение по умолчанию у любого view (глава 0, раздел 0.3), так что при нехватке места Auto Layout вправе ужать и её. Если хочешь, чтобы дисплей никогда не сжимался, подними ему вертикальный compression resistance до `.required` через `setContentCompressionResistancePriority(_:for:)`.

Построение сетки:

```
extension CalculatorViewController {
    func buildGrid() -> UIStackView {
        let rows: [[ButtonSpec]] = [
            [.gray("AC", .clear), .gray("±", .negate), .gray("%", .percent), .orange("÷",
            ↪  .binary(.div))],
            [.dark("7", .digit(7)), .dark("8", .digit(8)), .dark("9", .digit(9)),
            ↪  .orange("×", .binary(.mul))],
            [.dark("4", .digit(4)), .dark("5", .digit(5)), .dark("6", .digit(6)),
            ↪  .orange("−", .binary(.sub))],
            [.dark("1", .digit(1)), .dark("2", .digit(2)), .dark("3", .digit(3)),
            ↪  .orange("+", .binary(.add))],
            [.dark("0", .digit(0), wide: true), .dark(decimalSeparator, .dot),
            ↪  .orange("=", .equals)],
        ]

        let vStack = UIStackView()
        vStack.axis = .vertical
        vStack.spacing = 12
        vStack.distribution = .fillEqually
```

```
        for row in rows {
            let hStack = UIStackView()
            hStack.axis = .horizontal
            hStack.spacing = 12
            hStack.distribution = .fill

            var normalButtons: [UIButton] = []
            var wideButton: UIButton?
            for spec in row {
                let button = makeButton(spec: spec)
                hStack.addArrangedSubview(button)
                if spec.wide { wideButton = button } else { normalButtons.append(button) }
            }
            for button in normalButtons.dropFirst() {
                button.widthAnchor.constraint(equalTo:
↪   normalButtons[0].widthAnchor).isActive = true
            }
            if let wideButton, let first = normalButtons.first {
                // Широкая = две обычные + один промежуток между ними.
                wideButton.widthAnchor.constraint(
                    equalTo: first.widthAnchor, multiplier: 2, constant: hStack.spacing
                ).isActive = true
            }
            vStack.addArrangedSubview(hStack)
        }
        return vStack
    }
}
```

Устройство:

- Внешний `UIStackView` вертикальный, `distribution = .fillEqually` — все пять рядов одинаковой высоты.
- Внутренние горизонтальные — с `distribution = .fill`, **не** `.fillEqually`. С `.fillEqually` все три кнопки нижнего ряда стали бы одинаковыми, и «0» была бы размером с «=».
- Ширины задаём сами: все обычные кнопки ряда равны первой, а широкая — **две обычные плюс промежуток**.

Почему «плюс промежуток». Нижний ряд должен совпасть по сетке с рядами выше: «0» занимает место под «1» и «2», включая щель между ними. Если клетка 83,5 точки, а щель 12, то «0» = 2 × 83,5 + 12 = 179 точек, и её правый край точно встаёт под правый край «2». Констрейнт `equalTo: first.widthAnchor, multiplier: 2, constant: 12` так и читается: «ширина = ширина обычной × 2 + 12».

Стек не делит ширину «сам»: с `.fill` он ставит констрейнты «первая кнопка — к левому краю, последняя — к правому, между соседями 12 точек», а равенства ширин добавляем мы. Дальше работает обычный Auto Layout: в верхних рядах 4 равные кнопки, в нижнем — одна двойная и две обычные, и сетка совпадает.

Что такое `UIStackView`. Это `UIView` (`UIStackView → UIView`, глава 0, раздел 0.3), который сам создаёт констрейнты для своих *arranged subviews* — «расставленных» view. Список `arrangedSubviews` — всегда подмножество `subviews`: `addArrangedSubview` добавляет view и как subview, а обычный `addSubview` положит view внутрь стека, но расставлять её стек не станет (проверено: после `addSubview` массив `arrangedSubviews` пуст). `removeArrangedSubview` снимает view с расстановки, но из иерархии не убирает — чтобы она исчезла, нужен `removeFromSuperview()`. А если arranged view скрыть (`isHidden = true`), стек перестанет отводить ей место и уберёт промежуток после неё — этим мы воспользуемся в главе 15 для баннера. Четыре свойства определяют всё: `axis` (ось), `spacing` (промежуток), `distribution` (как делить место **вдоль** оси) и `alignment` (как ставить **поперёк** оси).

Значения `distribution`:

- `.fill` (по умолчанию) — view заполняют ось своими естественными размерами; если места не хватает, сжимается та, у кого ниже compression resistance, если места больше — растягивается та, у кого ниже hugging; при равенстве приоритетов — по порядку в `arrangedSubviews`;
- `.fillEqually` — все одного размера вдоль оси, `intrinsicContentSize` не учитывается;
- `.fillProportionally` — размеры пропорциональны `intrinsicContentSize`;
- `.equalSpacing` — view своих естественных размеров, лишнее место — в равные промежутки;
- `.equalCentering` — равные расстояния между центрами соседей, `spacing` остаётся минимальным зазором.

Значения `alignment`: `.fill` (по умолчанию) растягивает каждую view на всю толщину стека поперёк оси; `.leading`/`.top`, `.center`, `.trailing`/`.bottom` прижимают к краю или центру; `.firstBaseline` и `.lastBaseline` выравнивают текст по базовой линии и работают только в горизонтальном стеке. У наших горизонтальных рядов `alignment` остаётся `.fill` — поэтому кнопкам не нужен констрейнт высоты: каждая растянута на высоту ряда.

14.6 Кнопки через `UIButton.Configuration`

```
extension CalculatorViewController {
    func makeButton(spec: ButtonSpec) -> UIButton {
        var config = UIButton.Configuration.filled()
        config.baseBackgroundColor = spec.background
        config.baseForegroundColor = spec.foreground
        config.cornerStyle = .capsule
        config.contentInsets = .zero
        config.attributedTitle = Self.title(spec.title)

        let button = UIButton(configuration: config)
        button.accessibilityLabel = Self.accessibilityName(for: spec.key) ?? spec.title
        button.addAction(UIAction { [weak self] _ in
            self?.handleTap(key: spec.key)
        }, for: .touchUpInside)

        if case .binary(let op) = spec.key {
```

```
            operatorButtons[op] = button
        }
        if spec.key == .clear {
            clearButton = button
        }
        return button
    }

    static func title(_ text: String) -> AttributedString {
        var title = AttributedString(text)
        title.font = .systemFont(ofSize: 34, weight: .regular)
        return title
    }

    static func accessibilityName(for key: CalculatorEngine.Key) -> String? {
        switch key {
        case .binary(.add): return "плюс"
        case .binary(.sub): return "минус"
        case .binary(.mul): return "умножить"
        case .binary(.div): return "разделить"
        case .equals:       return "равно"
        case .negate:       return "сменить знак"
        case .percent:      return "процент"
        case .dot:          return "запятая"
        case .clear, .digit: return nil
        }
    }
}
```

`UIButton.Configuration.filled()` (iOS 15+) — современный способ настроить кнопку: структура с полями `baseBackgroundColor`, `baseForegroundColor`, `cornerStyle`, `contentInsets`, `image`/`title`/`subtitle`, `showsActivityIndicator` и другими. Кнопка перерисовывается, когда ты присваиваешь ей новую конфигурацию.

До iOS 15 кнопку настраивали по состояниям: `setTitle(_:for:)`, `setTitleColor(_:for:)`, `backgroundColor`, `layer.cornerRadius`, `contentEdgeInsets`. Этот API никуда не делся, но как только у кнопки появляется `configuration`, часть старых свойств она игнорирует: `contentEdgeInsets`, `titleEdgeInsets`, `imageEdgeInsets`, `adjustsImageWhenHighlighted` в SDK помечены deprecated с пометкой «ignored when using UIButtonConfiguration». Главный выигрыш конфигурации — состояния: `.filled()` сам затемняет фон при нажатии и делает кнопку серой при `isEnabled = false`. `baseBackgroundColor` — «исходный» цвет, который конфигурация преобразует под текущее состояние (так его и описывает Apple), а не цвет «как есть». Нужно своё поведение по состояниям — есть `configurationUpdateHandler`, UIKit зовёт его при каждой смене `state`. Ещё у кнопки есть `init(configuration:primaryAction:)`: переданный `UIAction` вешается на событие `.primaryActionTriggered` — «основное действие» контрола, у кнопки оно совпадает с обычным нажатием. Мы вешаем действие сами на `.touchUpInside`, чтобы событие было видно явно.

`cornerStyle = .capsule` — радиус скругления равен половине меньшей стороны. У квадратной кнопки 83 × 83 получится круг, у широкой 179 × 83 — «таблетка». Радиус не нужно считать и пересчитывать при повороте — стиль делает это сам.

`contentInsets = .zero` — без внутренних отступов. По умолчанию у `filled()` они есть (7 точек сверху и снизу, 12 по бокам — проверено в симуляторе iOS 26.5), и на маленьком экране подпись могла бы не влезть.

`AttributedString` (iOS 15+) — строка с атрибутами: шрифт, цвет. Конфигурация по умолчанию берёт шрифт из системного стиля кнопки; чтобы поставить ровно 34 точки, задаём заголовок с атрибутом `font`. Другой способ — `titleTextAttributesTransformer`, функция, которая меняет атрибуты заголовка; для простого случая `attributedTitle` нагляднее.

`addAction(UIAction { … }, for: .touchUpInside)` (iOS 14+) — обработчик нажатия замыканием, без `@objc`-метода и селектора. `spec.key` захватывается в замыкание: каждая кнопка помнит свою клавишу. `[weak self]` — кнопка принадлежит экрану, и сильный `self` в её замыкании замкнул бы круг ссылок.

`.touchUpInside` — палец поднят **внутри** границ кнопки; это и есть «нажатие». Увёл палец за край и отпустил — придёт `.touchUpOutside`, и нажатие не засчитается, как у физических кнопок. Это одно из событий `UIControl.Event`; другие: `.touchDown` — палец коснулся, `.valueChanged` — у слайдеров, переключателей и `UIRefreshControl`, `.editingChanged` — у `UITextField`, `.primaryActionTriggered` — «основное действие», одно для любого контрола. Вся эта механика (target-action) живёт в `UIControl`: `UIButton → UIControl → UIView`, и `addAction`/`addTarget` есть у любого контрола, а у `UILabel` или `UIImageView` их нет — они не контролы (глава 0, раздел 0.4). Поэтому в упражнении 14.2 дисплею понадобится gesture recognizer, а не действие.

> **`addAction` и `addTarget`.** Старый `addTarget(self, action: #selector(...), for:)` работает по-прежнему (мы использовали его в главе 12), но `addAction` короче и не требует `@objc`. В новом коде выбирай его.

`accessibilityLabel`. VoiceOver прочитает кнопку «÷» как «division sign» или вообще никак — поэтому словами: «разделить». Цифры и «AC» оставляем как есть, VoiceOver их прочитает сам.

Словари кнопок. Кнопки операторов складываем в `operatorButtons` по ключу-оператору, а кнопку сброса — отдельно. Они понадобятся для подсветки и смены подписи AC/C.

14.7 Подсветка активного оператора и отрисовка

Когда ты нажал +, калькулятор iPhone подсвечивает эту кнопку — оранжевая становится **белой** с оранжевым символом. Так видно, что оператор принят и ждёт второе число. Как только начнёшь набирать цифры — подсветка гаснет.

```
extension CalculatorViewController {
    func handleTap(key: CalculatorEngine.Key) {
        engine.input(key)
        render()
        if engine.isError {
            errorHaptic.notificationOccurred(.error)
        } else {
```

```
            haptic.impactOccurred(intensity: 0.6)
        }
    }

    func render() {
        let text = engine.display.replacingOccurrences(of: ".", with: decimalSeparator)
        displayLabel.text = text
        displayLabel.accessibilityLabel = text

        for (op, button) in operatorButtons {
            let active = engine.activeOp == op
            button.configuration?.baseBackgroundColor = active ? .white : .systemOrange
            button.configuration?.baseForegroundColor = active ? .systemOrange : .white
            button.accessibilityTraits = active ? [.button, .selected] : .button
        }

        let clearTitle = engine.showsAllClear ? "AC" : "C"
        clearButton?.configuration?.attributedTitle = Self.title(clearTitle)
        clearButton?.accessibilityLabel = engine.showsAllClear ? "стереть всё" : "стереть
↪ число"
    }
}
```

`render()` после каждого нажатия приводит весь экран в соответствие с движком: дисплей, подсветку, подпись сброса. Экран ничего не помнит сам — всё берёт из `engine`. Это и есть однонаправленный поток из 14.1: нельзя «забыть» погасить подсветку, потому что она каждый раз вычисляется заново.

Замена разделителя. Движок отдаёт `"2.5"`, а на русском телефоне показываем `"2,5"`. Одна строка, и только при показе.

`button.configuration?.baseBackgroundColor = …` — меняем одно поле конфигурации прямо на кнопке. Под капотом Swift берёт копию структуры, меняет поле и присваивает обратно, а кнопка при присваивании перерисовывается. Заголовок при этом не трогаем — он остаётся тем же `attributedTitle`.

`engine.activeOp` — `nil`, если оператор не нажат или уже начат ввод второго числа. Тогда ни одна кнопка не подсвечена.

`.selected` в `accessibilityTraits` — VoiceOver скажет «выбрано» у активного оператора: незрячий пользователь тоже узнает, что + ждёт второе число.

14.8 Тактильный отклик

Два генератора: для обычного нажатия и для ошибки (код — в `handleTap` выше).

`UIImpactFeedbackGenerator(style: .medium)` — короткий «толчок», как будто нажал физическую кнопку. `impactOccurred(intensity: 0.6)` (iOS 13+) — 60% от полной силы: калькулятор нажимают часто, и полный удар на каждую цифру утомляет. Генераторы — не view

(`UIImpactFeedbackGenerator → UIFeedbackGenerator → NSObject`): это обычные объекты, которые просят систему дать отклик, и держать их можно где угодно (глава 0, раздел 0.8).

`UINotificationFeedbackGenerator` — отклик-«сообщение», три вида: `.success`, `.warning`, `.error`. Каждый — короткая последовательность импульсов; `.error` ощущается как быстрое «тук-тук-тук», его трудно спутать с обычным нажатием. Играем его, когда на дисплее «Ошибка».

Проверять отклик можно только **на реальном устройстве** с Taptic Engine (iPhone 7 и новее): симулятор вибрацию не передаёт, и вызовы там ничего не делают. Если в «Настройки → Звуки, тактильные сигналы» системные тактильные сигналы выключены, телефон тоже промолчит — это выбор пользователя, и так и должно быть.

> **`prepare()` заранее.** Генератору нужно немного времени, чтобы «разбудить» вибромотор, поэтому первый отклик после простоя может чуть опоздать. Вызов `haptic.prepare()` незадолго до ожидаемого нажатия (например, когда палец коснулся кнопки — событие `.touchDown`) подготавливает мотор, и отклик приходит вовремя. Apple советует звать `prepare()` заранее, но не впритык (вызов непосредственно перед откликом ничего не даёт): подготовленное состояние держится несколько секунд и сбрасывается после отклика, а продлить его можно повторным `prepare()`. Подробнее — глава 33.

14.9 Всегда тёмная тема

Калькулятор всегда тёмный:

```
view.backgroundColor = .black
overrideUserInterfaceStyle = .dark
```

(эти две строки — в `viewDidLoad` из 14.5).

`overrideUserInterfaceStyle = .dark` — этот контроллер и всё внутри него всегда в тёмной теме, независимо от системной настройки. Действует на его `view` со всеми subviews и на child view controller'ы; на родителей и на приложение целиком — нет (так сказано в заголовке `UIViewController.h`). Экран, показанный из него модально, — не child, тему он возьмёт как обычно, от окна. Системные динамические цвета (`.label`, `.secondarySystemBackground` и другие) внутри будут брать тёмные варианты. Нам это важно, потому что дизайн «под калькулятор Apple» держится на чёрном фоне: белые цифры на светлом фоне просто исчезли бы.

Можно было бы поддержать обе темы, подобрав светлую палитру, но это больше работа дизайнера, чем программиста. Для нашего приложения упростили. Как устроены динамические цвета — глава 35 про темы.

> **Упражнение 14.2.** Добавь удаление последней цифры свайпом по дисплею — так сделано в калькуляторе iPhone: провёл пальцем по числу влево или вправо — стёрлась последняя цифра. Нужен новый `Key.backspace` в движке и `UISwipeGestureRecognizer` на дисплее. Проверка в `main.swift`: `123⌫ → 12`, `5⌫ → 0`, `2+3=⌫ → 5` (результат стирать нельзя).

14.10 Бытовая аналогия

Движок — это **бухгалтер с тетрадью**. На полях тетради у него две отложенные записи (нижняя и верхняя полка), в строке — число, которое сейчас диктуют. Рук у него нет, он ничего не показывает — только пишет и считает.

Экран — **табло**, которое показывает то, что бухгалтер записал. Человек нажимает кнопку — экран говорит бухгалтеру «получи цифру 5», бухгалтер пишет, экран читает обратно «что у тебя в строке» и показывает.

Такое разделение позволяет:

- **Проверять бухгалтера**: дать ему сто последовательностей нажатий и сверить ответы — как мы сделали в 14.4.
- **Менять табло**: кнопки круглые, квадратные, с иконками — бухгалтер не заметит.
- **Учить бухгалтера новому**: добавил синус и логарифм — экран просто получает новые кнопки.

14.11 Что мы пропустили

- **Память** (M+, M−, MR, MC) — отдельный регистр в движке.
- **Инженерный режим** — sin, cos, log, π, скобки. В калькуляторе iPhone он открывается поворотом в альбомную ориентацию. Для скобок двух «полок» не хватит — понадобится стек операций или разбор выражения.
- **История** — список прошлых вычислений.
- **Копирование результата** долгим нажатием на дисплей (`UIPasteboard.general.string`).
- **Альбомная раскладка.** Сейчас сетка просто сплющивается. Лучше было бы в альбомной ориентации показывать дисплей слева, а кнопки справа — это делается сменой `axis` у внешнего стека по `traitCollection`.

Все эти добавки ложатся на ту же основу — движок плюс экран.

> **Упражнение 14.3.** Запусти калькулятор и проверь руками: `7 × 8 =` даёт 56, сразу `+ 2 =` — 58. `5 + + 3 =` — 8 (второй «+» заменил первый). `2 + 3 × 4 =` — 14. `1 ÷ 0 =` — «Ошибка», и все кнопки, кроме AC, перестали работать. Если на телефоне русский регион, кнопка разделителя — «,», и `0,1 + 0,2 =` покажет «0,3».

Ответы к упражнениям

Упражнение 14.1.

- `2 + 3 × 4 − 5 =` → **9**. При нажатии − верхняя полка (3 × 4) сворачивается в 12, нижняя (2 +) — в 14, и `14` − ложится на нижнюю; 14 − 5 = 9.
- `9 = =` → **9**. Оператора не было — повторять нечего.
- `5 + =` → **10**. Правого числа не набрали, поэтому правым стало то, что на дисплее, — 5.
- `200 × 10 % =` → **20**. `%` при ожидающем умножении даёт 10 / 100 = 0,1, а `=` — 200 × 0,1 = 20.

Упражнение 14.2. В `Key` добавь `case backspace`, в `input(_:)` — `case .backspace: backspace()`, а в расширение движка:

```
nonisolated extension CalculatorEngine {
    private mutating func backspace() {
        guard !startsNewInput else { return }   // результат и новое число не стираем
        input.removeLast()
        if input.isEmpty || input == "−" {
            input = "0"
        }
    }
}
```

В `run` из `main.swift` добавь `case "<": engine.input(.backspace)` и проверки `check("123<", "12")`, `check("5<", "0")`, `check("2+3=<", "5")`. В `accessibilityName(for:)` добавь `case .backspace: return "удалить цифру"` — иначе `switch` не соберётся: он должен перечислить все варианты. На экране — в `setupLayout()`:

```
for direction in [UISwipeGestureRecognizer.Direction.left, .right] {
    let swipe = UISwipeGestureRecognizer(target: self, action: #selector(displaySwiped))
    swipe.direction = direction
    displayLabel.addGestureRecognizer(swipe)
}
displayLabel.isUserInteractionEnabled = true
```

и метод `@objc func displaySwiped() { handleTap(key: .backspace) }`. `isUserInteractionEnabled = true` обязателен: у `UILabel` он по умолчанию выключен (у обычного `UIView` — включён), а view с выключенным флагом hit-testing пропускает, и касания до её жестов не доходят (глава 0, раздел 0.3). Сам `UISwipeGestureRecognizer` — не view (→ `UIGestureRecognizer` → `NSObject`): распознаватель прикреплён ровно к одной view и получает её касания; поэтому мы вешаем его на `displayLabel`, а не на `view` контроллера (глава 0, раздел 0.8).

Упражнение 14.3. Ожидаемые результаты — в самом задании. Если разделитель «.», а не «,», проверь регион в настройках симулятора: «Settings → General → Language & Region».

Что мы выучили

- **Движок отдельно от интерфейса.** `CalculatorEngine` — структура без UIKit, `nonisolated`, с одним входом `input(_:)`. Её можно проверить обычной программой на Mac.
- Состояние — текст ввода, точный результат, две полки отложенных операций и флаги `startsNewInput` / `justPressedOp`.
- **Порядок действий** на двух полках: `×` и `÷` сворачиваются раньше `+` и `−`, поэтому `2 + 3 × 4 = 14`, как в калькуляторе iPhone.
- Повторное `=` повторяет последнюю операцию; `%` после `+` берёт процент от первого числа.
- Деление на ноль и переполнение ловим одной проверкой `isFinite` в `show(_:)`. Никаких `Int(double)` без проверки диапазона.
- `Double` не хранит 0,1 точно; показываем 12 значащих цифр — мусор округляется. Для денег — `Decimal`.
- `UIStackView` — `UIView`, который сам ставит констрейнты своим `arrangedSubviews` (подмножество `subviews`); скрытая arranged view места не занимает. `distribution` делит место

вдоль оси (`.fill` по умолчанию, `.fillEqually` — все равны), `alignment` — поперёк (`.fill` по умолчанию).
- Сетка — вертикальный `.fillEqually` из горизонтальных `.fill`, ширина «0» = 2 × клетка + промежуток. Констрейнт пропорций с приоритетом 750, чтобы не конфликтовать в альбомной ориентации.
- `UIButton.Configuration` (iOS 15+) заменяет `setTitle(_:for:)` и `contentEdgeInsets` и сам обслуживает состояния; `.capsule`.
- `addAction(UIAction, for: .touchUpInside)` — target-action из `UIControl`; у `UILabel` его нет, нужен gesture recognizer.
- `render()` после каждого нажатия пересобирает экран из движка: подсветка оператора, AC/C, разделитель по локали.
- Тактильный отклик: `UIImpactFeedbackGenerator` на нажатие, `UINotificationFeedbackGenerator(.error)` на ошибку; в симуляторе не ощущается.
- `overrideUserInterfaceStyle = .dark` — тёмная тема только для этого экрана.

Apple Developer Documentation

- UIStackView — контейнер для горизонтальных/вертикальных раскладок, основа сетки кнопок калькулятора.
- UIButton — кнопка; `UIButton.Configuration.filled()` (iOS 15+) задаёт фон, форму и `attributedTitle`.
- UIImpactFeedbackGenerator — тактильный «толчок» с настраиваемой `intensity` для обычных нажатий.
- UISelectionFeedbackGenerator — тонкий «щелчок», уместен для смены выбора (например, прокрутки барабана).
- Decimal — точная десятичная арифметика без двоичных погрешностей `Double`; выбор для денег.
- NumberFormatter — форматирование чисел; в главе ограничивает значащие цифры результата.
- Enumerations — Swift Book про enum, на котором держатся наши `Key` и `BinaryOp`.

→ Глава 15. Weather — open-meteo, pull-to-refresh, skeleton, offline

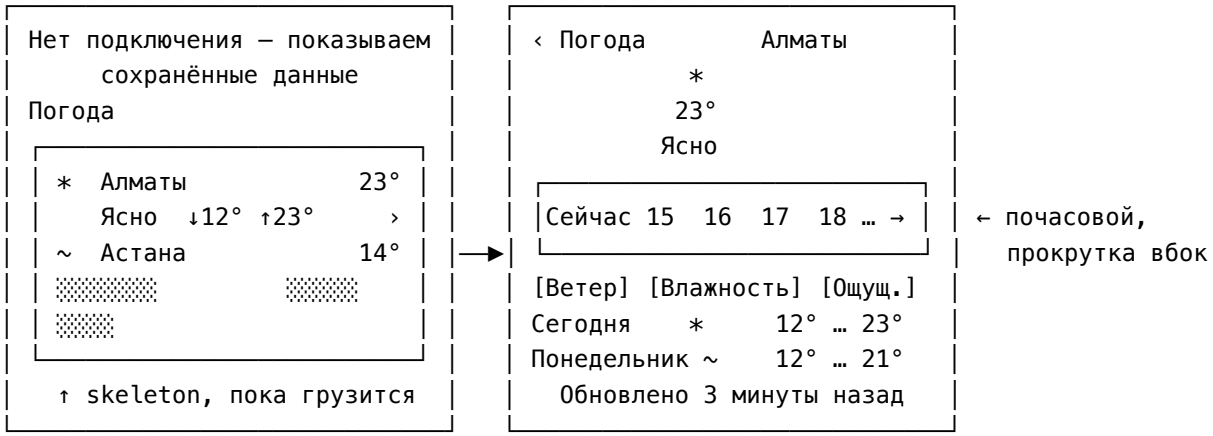
Глава 15. Weather — open-meteo, pull-to-refresh, skeleton, offline

Это самое «взрослое» мини-приложение книги. В нём собрано то, что отличает учебный пример от приложения, которым пользуются каждый день:

- **настоящий API** без ключа — open-meteo.com;
- **pull-to-refresh** («потяни, чтобы обновить») через `UIRefreshControl`;
- **skeleton** — серые «заготовки» строк с бегущим бликом, пока данные грузятся (по-английски *skeleton* — «скелет», а бегущий блик — *shimmer*, «мерцание»);
- **офлайн-баннер** через `NWPathMonitor` и кеш на диске: без сети приложение показывает последние сохранённые данные;
- **кеш с дедупликацией**: два экрана, одновременно попросившие погоду одного города, получат результат одного запроса;
- **детальный экран** с прогнозом по часам и по дням.

Глава длинная, но идти можно не подряд: 15.1–15.5 — данные и сеть, 15.6–15.9 — список, 15.10 — работа без сети, 15.11–15.12 — детальный экран.

Что строим:



Проект — как в главе 12 (раздел «Перед началом»): App, Swift 6, Default Actor Isolation = MainActor, Approachable Concurrency, iOS 15+, без `Main.storyboard`. Первый экран — список в навигационном контроллере:

```
import UIKit
```



Рис. 11: Погода для 5 городов Казахстана

```
final class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }
        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = UINavigationController(
            rootViewController: WeatherListViewController()
        )
        window.makeKeyAndVisible()
        self.window = window
    }
}
```

15.1 Почему open-meteo

Для учебника нужен погодный API, с которым можно начать за минуту. **open-meteo.com** подходит:

- **Без ключа и регистрации.** Обычный GET-запрос.
- **Бесплатно для некоммерческого использования** — с лимитами: меньше 10 000 запросов в сутки, 5 000 в час и 600 в минуту (так написано в условиях использования на open-meteo.com/en/terms). Учебный проект, личное приложение, образовательный контент — подходят. Приложение с подпиской или рекламой — уже коммерческое использование, для него нужен платный тариф.
- **Данные под лицензией CC BY 4.0** — это значит, что в приложении надо указать источник («Weather data by Open-Meteo.com»). В упражнении 15.3 мы добавим такую подпись.
- **Простой JSON** и параметры «запроси только то, что нужно».

Проверь сам в терминале, до всякого Swift:

```
curl "https://api.open-meteo.com/v1/forecast?latitude=43.2567&longitude=76.9286&timezone
↪  =auto&timeformat=unixtime&current=temperature_2m,weather_code"
```

Ответ (сокращён):

```
{"latitude":43.26889,"longitude":76.95067,"utc_offset_seconds":18000,
 "timezone":"Asia/Almaty","timezone_abbreviation":"GMT+5",
 "current_units":{"time":"unixtime","temperature_2m":"°C","weather_code":"wmo code"},
 "current":{"time":1790501400,"interval":900,"temperature_2m":23.3,"weather_code":0}}
```

Ключа нет — запрос прошёл. Координаты в ответе чуть отличаются от запрошенных: модель считает погоду по сетке, и API вернул ближайшую к Алматы точку сетки.

Альтернативы:

- **OpenWeatherMap**, **WeatherAPI.com** — нужен ключ, есть бесплатные тарифы с лимитами.

- **WeatherKit** от Apple — фреймворк для iOS 16+ (нам нужна iOS 15, поэтому мимо) и REST API. Нужна платная подписка Apple Developer Program; в неё входит 500 000 запросов в месяц, больше — за отдельную плату. Тоже требует показывать атрибуцию источника.

Для учебника open-meteo удобнее всего: никаких ключей в git, никаких аккаунтов.

15.2 Модели — DTO и доменные модели

Начнём с города. Список городов зашит в код:

```
import Foundation

nonisolated struct City: Hashable, Codable, Sendable {
    let name: String
    let latitude: Double
    let longitude: Double

    static let all: [City] = [
        City(name: "Алматы", latitude: 43.2567, longitude: 76.9286),
        City(name: "Астана", latitude: 51.1694, longitude: 71.4491),
        City(name: "Шымкент", latitude: 42.3417, longitude: 69.5901),
        City(name: "Караганда", latitude: 49.8047, longitude: 73.1094),
        City(name: "Актобе", latitude: 50.2839, longitude: 57.1670),
    ]
}
```

Координаты — широта и долгота центра города в градусах. `Hashable` нужен, чтобы город был ключом словаря («погода по городу»), `Codable` — чтобы сохранить погоду на диск вместе с городом.

Зачем `nonisolated`. В проекте с Default Actor Isolation = MainActor всё без пометок привязано к главному потоку — и эта структура тоже, вместе с её реализацией `Codable` и `Hashable`. Для модели данных это лишнее ограничение: декодировать JSON или сравнивать города можно из любого потока. С `nonisolated` модель свободна. Это ловушка, на которую часто натыкаются: пока ты декодируешь на главном потоке, всё собирается, а стоит перенести разбор ответа в фоновую задачу или в `actor` — компилятор скажет, что реализация `Decodable` «принадлежит главному актору» и вызвать её отсюда нельзя. Все модели этой главы помечены `nonisolated` сразу.

Сеть возвращает JSON. Мы декодируем его в `OpenMeteoResponse` — это **DTO** (*data transfer object*, «объект для передачи данных»): он повторяет структуру JSON один в один.

```
import Foundation

nonisolated struct OpenMeteoResponse: Decodable, Sendable {
    struct Current: Decodable, Sendable {
        let time: TimeInterval
        let temperature_2m: Double
        let apparent_temperature: Double
        let weather_code: Int
```

```
        let wind_speed_10m: Double
        let relative_humidity_2m: Double
        let is_day: Int
    }

    struct Hourly: Decodable, Sendable {
        let time: [TimeInterval]
        let temperature_2m: [Double]
        let weather_code: [Int]
        let is_day: [Int]
    }

    struct Daily: Decodable, Sendable {
        let time: [TimeInterval]
        let temperature_2m_max: [Double]
        let temperature_2m_min: [Double]
        let weather_code: [Int]
        let precipitation_probability_max: [Int?]
    }

    let timezone: String
    let current: Current
    let hourly: Hourly
    let daily: Daily
}
```

Имена с подчёркиваниями (`temperature_2m`) — такие же, как в JSON. Можно было переименовать их через `CodingKeys` или `keyDecodingStrategy = .convertFromSnakeCase`, но DTO всё равно никто, кроме маппинга, не видит — пусть остаётся зеркалом ответа, так его проще сверять с документацией API.

Несколько деталей, которые важно заметить:

- **`time: TimeInterval`** — мы просим время в виде Unix-времени (`timeformat=unixtime`): число секунд, прошедших с 1 января 1970 года, 00:00 по UTC. Это число однозначно, в каком бы часовом поясе ни был телефон. Строки вида `"2026-09-27T14:30"` без пояса пришлось бы ещё правильно разобрать.
- **`relative_humidity_2m: Double`** — в JSON влажность приходит целым числом (`26`), но `JSONDecoder` спокойно читает целое в `Double`.
- **`precipitation_probability_max: [Int?]`** — вероятность осадков может прийти как `null` для дней, на которые модель её не считает. Если объявить `[Int]`, один `null` уронит декодирование **всего** ответа. Для полей, в которых API обещает возможный `null`, лучше сразу опционал.
- **`timezone`** — имя часового пояса города (`"Asia/Almaty"`), которое API определил по координатам, потому что мы передали `timezone=auto`.

Часовой пояс и UTC на числах. В ответе `"current":{"time":1790501400}`. Это 27 сентября 2026, 09:30 по UTC (всемирное координированное время, «нулевой» пояс). В Алматы `utc_offset_seconds = 18000`: 18 000 секунд = 5 часов, значит, местное время — 14:30. С марта

2024 года весь Казахстан живёт в одном поясе UTC+5, поэтому и Алматы, и Актобе вернут смещение 5 часов, хотя имена поясов разные (`Asia/Almaty`, `Asia/Aqtobe`). `Date` в Swift хранит момент без пояса; пояс нужен только при **показе** — «который час в этом городе».

А вот модели, которыми пользуется интерфейс:

```
import Foundation

nonisolated struct HourForecast: Codable, Sendable {
    let time: Date
    let temperature: Double
    let condition: WeatherCondition
    let isDay: Bool
}

nonisolated struct DayForecast: Codable, Sendable {
    let date: Date
    let minTemperature: Double
    let maxTemperature: Double
    let condition: WeatherCondition
    let precipitationChance: Int?
}

nonisolated struct CityWeather: Codable, Sendable {
    let city: City
    let timeZoneID: String
    let temperature: Double
    let apparentTemperature: Double
    let condition: WeatherCondition
    let isDay: Bool
    let windSpeed: Double      // метры в секунду
    let humidity: Double       // проценты
    let updatedAt: Date
    let hourly: [HourForecast]
    let daily: [DayForecast]

    var timeZone: TimeZone {
        TimeZone(identifier: timeZoneID) ?? .current
    }
}

nonisolated enum TemperatureFormat {
    /// 23.4 → «23°», -0.4 → «0°» (а не «-0°»).
    static func string(_ celsius: Double) -> String {
        "\(Int(celsius.rounded()))°"
    }
}
```

`CityWeather` — всё, что нужно экранам про один город: текущая погода, 24 часа, 7 дней. Имена — в привычном Swift стиле (`apparentTemperature`), единицы — те, что покажем пользователю: ветер в метрах в секунду, а не в километрах в час, как у API.

DTO живёт только внутри сетевого слоя. Экраны работают с `CityWeather`. Если завтра API поменяет формат, мы поправим DTO и маппинг, а экраны ничего не заметят. Без такого разделения структура чужого JSON просачивается в интерфейс, и любая смена API превращается в переписывание всего приложения.

`timeZoneID` храним строкой, а `TimeZone` вычисляем: сам `TimeZone` не `Codable`-совместим «из коробки» так, как хочется, а строку можно сохранить на диск. `?? .current` — запасной вариант (*fallback*), если имя пояса вдруг окажется незнакомым.

`TemperatureFormat.string` — одно место, где число превращается в «23°». `rounded()` округляет по школьному правилу: 23,4 → 23, 23,5 → 24, −2,5 → −3 (половинки — от нуля). Ловушка, которую обходит `Int(...)`: если форматировать `Double` напрямую, −0,4 округлится в «−0» — такое бывает, и выглядит глупо. `Int` не знает «минус нуля», и получается честный «0°».

15.3 `WeatherCondition` — коды погоды WMO

Погода приходит числом — кодом по стандарту WMO (Всемирной метеорологической организации). Таблица кодов есть в документации open-meteo: 0 — ясно, 1–3 — облачность разной плотности, 45 и 48 — туман, 51–57 — морось, 61–67 — дождь, 71–77 — снег, 80–82 — ливни, 85–86 — снегопады, 95–99 — грозы.

Для интерфейса 28 кодов не нужны — хватит 11 укрупнённых вариантов:

```
import Foundation

nonisolated enum WeatherCondition: String, Codable, Sendable {
    case clear, mainlyClear, partlyCloudy, overcast, fog
    case drizzle, rain, heavyRain, snow, heavySnow, thunderstorm

    init(wmoCode: Int) {
        switch wmoCode {
        case 0:                  self = .clear
        case 1:                  self = .mainlyClear
        case 2:                  self = .partlyCloudy
        case 3:                  self = .overcast
        case 45, 48:             self = .fog
        case 51, 53, 55, 56, 57: self = .drizzle
        case 61, 63, 66, 80, 81: self = .rain
        case 65, 67, 82:         self = .heavyRain
        case 71, 73, 77, 85:     self = .snow
        case 75, 86:             self = .heavySnow
        case 95, 96, 97, 99:     self = .thunderstorm
        default:                 self = .overcast
        }
    }
```

```
    var label: String {
        switch self {
        case .clear:        return "Ясно"
        case .mainlyClear:  return "Малооблачно"
        case .partlyCloudy: return "Переменная облачность"
        case .overcast:     return "Пасмурно"
        case .fog:          return "Туман"
        case .drizzle:      return "Морось"
        case .rain:         return "Дождь"
        case .heavyRain:    return "Ливень"
        case .snow:         return "Снег"
        case .heavySnow:    return "Сильный снег"
        case .thunderstorm: return "Гроза"
        }
    }

    func systemImage(isDay: Bool) -> String {
        switch self {
        case .clear:        return isDay ? "sun.max.fill" : "moon.stars.fill"
        case .mainlyClear:  return isDay ? "sun.min.fill" : "moon.fill"
        case .partlyCloudy: return isDay ? "cloud.sun.fill" : "cloud.moon.fill"
        case .overcast:     return "cloud.fill"
        case .fog:          return "cloud.fog.fill"
        case .drizzle:      return "cloud.drizzle.fill"
        case .rain:         return "cloud.rain.fill"
        case .heavyRain:    return "cloud.heavyrain.fill"
        case .snow:         return "cloud.snow.fill"
        case .heavySnow:    return "snowflake"
        case .thunderstorm: return "cloud.bolt.rain.fill"
        }
    }
}
```

`init(wmoCode:)` — инициализатор-переводчик. Все коды из документации разложены по вариантам, а неизвестный (если API добавит новый) станет `.overcast` — безопасный вариант по умолчанию, который не соврёт сильно. `String` в качестве `rawValue` — чтобы перечисление сохранялось на диск читаемым словом.

Два кода, которые легко разложить неправильно: 66–67 — «ледяной дождь», его честнее показывать дождём, а не пропускать в `default`; 77 — «снежные зёрна», мелкая крупа, это обычный снег, а не сильный.

`systemImage(isDay:)` — имя SF Symbol с учётом времени суток: ясной ночью солнце на иконке выглядит странно, поэтому для ночи — луна. Признак «день или ночь» тоже приходит из API (`is_day`: 1 или 0) — он учитывает настоящий восход и закат в этом городе.

> **Зачем перечисление, а не `Int`.** Можно было хранить `weatherCode` в модели и в интерфейсе писать `switch` по числам. Но тогда логика «какая иконка у кода 65» раз-

множилась бы по списку, детальному экрану и почасовой полосе. С перечислением перевод в одном месте, и компилятор проверяет, что каждый вариант обработан.

15.4 Сетевой клиент — async/await и ошибки

```
import Foundation

enum WeatherAPI {
    enum Error: Swift.Error {
        case badStatus(Int)
        case decoding
        case transport(Swift.Error)
    }

    static func fetch(for city: City) async throws -> CityWeather {
        var components = URLComponents(string: "https://api.open-meteo.com/v1/forecast")
        components?.queryItems = [
            URLQueryItem(name: "latitude", value: String(city.latitude)),
            URLQueryItem(name: "longitude", value: String(city.longitude)),
            URLQueryItem(name: "timezone", value: "auto"),
            URLQueryItem(name: "timeformat", value: "unixtime"),
            URLQueryItem(name: "current", value:
↪  "temperature_2m,apparent_temperature,weather_code,wind_speed_10m,relative_humidity_2m,is_day"
↪  ),
            URLQueryItem(name: "hourly", value: "temperature_2m,weather_code,is_day"),
            URLQueryItem(name: "daily", value:
↪  "temperature_2m_max,temperature_2m_min,weather_code,precipitation_probability_max"),
            URLQueryItem(name: "forecast_days", value: "7"),
        ]
        guard let url = components?.url else { throw Error.decoding }

        let request = URLRequest(url: url, cachePolicy: .reloadIgnoringLocalCacheData,
         ↪  timeoutInterval: 20)
        let data: Data
        let response: URLResponse
        do {
            (data, response) = try await URLSession.shared.data(for: request)
        } catch {
            throw Error.transport(error)
        }
        guard let http = response as? HTTPURLResponse else { throw Error.decoding }
        guard 200..<300 ~= http.statusCode else { throw Error.badStatus(http.statusCode) }

        let dto: OpenMeteoResponse
        do {
            dto = try JSONDecoder().decode(OpenMeteoResponse.self, from: data)
        } catch {
```

```
            throw Error.decoding
        }
        return map(dto: dto, city: city, now: Date())
    }

    static func map(dto: OpenMeteoResponse, city: City, now: Date) -> CityWeather {
        let current = dto.current

        // Почасовой прогноз: 24 часа, начиная с текущего.
        let hourStart = current.time - current.time.truncatingRemainder(dividingBy: 3600)
        let firstHour = dto.hourly.time.firstIndex { $0 >= hourStart } ?? 0
        let lastHour = min(firstHour + 24, dto.hourly.time.count)
        let hourly = (firstHour..<lastHour).map { i in
            HourForecast(
                time: Date(timeIntervalSince1970: dto.hourly.time[i]),
                temperature: dto.hourly.temperature_2m[i],
                condition: WeatherCondition(wmoCode: dto.hourly.weather_code[i]),
                isDay: dto.hourly.is_day[i] == 1
            )
        }

        let daily = dto.daily.time.indices.map { i in
            DayForecast(
                date: Date(timeIntervalSince1970: dto.daily.time[i]),
                minTemperature: dto.daily.temperature_2m_min[i],
                maxTemperature: dto.daily.temperature_2m_max[i],
                condition: WeatherCondition(wmoCode: dto.daily.weather_code[i]),
                precipitationChance: dto.daily.precipitation_probability_max[i]
            )
        }

        return CityWeather(
            city: city,
            timeZoneID: dto.timezone,
            temperature: current.temperature_2m,
            apparentTemperature: current.apparent_temperature,
            condition: WeatherCondition(wmoCode: current.weather_code),
            isDay: current.is_day == 1,
            windSpeed: current.wind_speed_10m / 3.6,
            humidity: current.relative_humidity_2m,
            updatedAt: now,
            hourly: hourly,
            daily: daily
        )
    }
}
```

Шаги `fetch`:

1. **Адрес** собираем через `URLComponents`: он сам экранирует параметры. Запятые в `current=temperature_2m,weather_code` останутся запятыми — так API и ждёт.
2. **Параметры.** `timezone=auto` — пусть API сам определит пояс по координатам. `timeformat=unixtime` — время числами (15.2). `current`, `hourly`, `daily` — какие величины нужны в каждой части ответа. `forecast_days=7` — неделя. Весь ответ — около 4,5 КБ (а сжатым при передаче — около 1,3 КБ), это мало.
3. **Запрос** — `cachePolicy: .reloadIgnoringLocalCacheData` отключает стандартный HTTP-кеш `URLSession`: мы держим **свой** кеш в `WeatherStore` и хотим сами решать, когда идти в сеть. `timeoutInterval: 20` — не ждать ответа больше 20 секунд.
4. **Сеть** — `URLSession.shared.data(for:)` (iOS 15+). Ошибку сети (нет интернета, таймаут) заворачиваем в `Error.transport`, чтобы снаружи было видно, что именно сломалось.
5. **Код ответа** — `200..<300 ~= http.statusCode`. Оператор `~=` — это «сопоставление с образцом»: «попадает ли код в диапазон». Та же проверка, что `(200..<300).contains(code)`, просто короче. Если передать невозможные координаты, open-meteo ответит кодом 400 и JSON `{"error":true,"reason":"Latitude must be in range of −90 to 90°…"}` — мы превратим это в `Error.badStatus(400)`.
6. **Декодирование** DTO. Ошибка — `Error.decoding`.
7. **Маппинг** DTO → `CityWeather`.

`enum WeatherAPI` без вариантов — «пространство имён»: все методы `static`, создавать экземпляр незачем.

Где это выполняется. `WeatherAPI` в нашем режиме изолирован на главном потоке. Это не значит, что главный поток ждёт сеть: на `await` функция приостанавливается и отпускает поток, интерфейс продолжает жить. Декодирование 4,5 КБ JSON — доля миллисекунды, его можно делать и на главном. Если бы ответы были мегабайтными, разбор стоило бы увести с главного потока — и тут бы пригодилось, что модели уже `nonisolated`.

Маппинг, `map(dto:city:now:)`, — отдельная функция, а не часть `fetch`: её можно проверить без сети, подсунув сохранённый JSON (так этот код и проверялся: ответ API, записанный `curl`'ом, раскладывается в 24 часа и 7 дней). `now` передаётся параметром по той же причине — в проверке можно подставить любое «сейчас».

Какие 24 часа показывать. Почасовой прогноз приходит на всю неделю — 7 × 24 = 168 значений, начиная с полуночи сегодняшнего дня. Нам нужны 24, начиная с текущего часа. Разберём на числах:

- текущее время `current.time = 1790501400` (14:30 по Алматы);
- `truncatingRemainder(dividingBy: 3600)` — остаток от деления на 3600 секунд (час): 1790501400 делится на 3600 с остатком 1800, то есть 30 минут после начала часа;
- вычитаем остаток: 1790501400 − 1800 = 1790499600 — это 14:00;
- `firstIndex { $0 >= hourStart }` — первый час прогноза не раньше 14:00, то есть 14:00;
- берём 24 значения с этого места: с 14:00 сегодня до 13:00 завтра. `min(firstHour + 24, count)` защищает от выхода за конец массива.

Ветер. API отдаёт скорость в километрах в час. В Казахстане привычнее метры в секунду. Перевод: в часе 3600 секунд, в километре 1000 метров, значит, 1 км/ч = 1000 / 3600 = 1/3,6 м/с. Делим на 3,6: 18 км/ч = 5 м/с, 3,4 км/ч ≈ 0,9 м/с. (Можно было попросить API сразу вернуть м/с параметром `wind_speed_unit=ms`, но полезно один раз увидеть перевод своими глазами.)

Упражнение 15.1. Добавь в список город Актау (широта 43.65, долгота 51.17). Не запуская приложение, ответь: какое имя часового пояса вернёт API и какое смещение от UTC? Проверь `curl`'ом.

15.5 WeatherStore — кеш и дедупликация

Прямые вызовы `WeatherAPI.fetch` из экранов создают три проблемы:

1. **Двойные запросы.** Если список и детальный экран одновременно попросят погоду Алматы, уйдут два одинаковых запроса.
2. **Лишние запросы.** Данные пришли минуту назад — идти за ними снова незачем: open-meteo обновляет текущую погоду раз в 15 минут (поле `interval: 900` в ответе — 900 секунд).
3. **Жизнь без сети.** Закрыл приложение, открыл в метро — хочется увидеть хоть последние данные, а не пустой экран.

`WeatherStore` решает все три:

```
import Foundation

@MainActor
final class WeatherStore {
    static let shared = WeatherStore()

    /// Сколько секунд данные считаются свежими: 10 минут.
    static let freshness: TimeInterval = 10 * 60

    private var cache: [City: CityWeather] = [:]
    private var inflight: [City: Task<CityWeather, Error>] = [:]
    private let fileURL: URL

    init(fileURL: URL? = nil) {
        let caches = FileManager.default.urls(for: .cachesDirectory, in:
        ↪  .userDomainMask)[0]
        self.fileURL = fileURL ?? caches.appendingPathComponent("weather-cache.json")
        loadFromDisk()
    }

    func cached(for city: City) -> CityWeather? {
        cache[city]
    }

    func fetch(for city: City, force: Bool = false) async throws -> CityWeather {
        if !force, let cached = cache[city],
           Date().timeIntervalSince(cached.updatedAt) < Self.freshness {
            return cached
        }
        if let existing = inflight[city] {
            return try await existing.value
```

```
        }

        let task = Task {
            try await WeatherAPI.fetch(for: city)
        }
        inflight[city] = task
        defer { inflight[city] = nil }

        let weather = try await task.value
        cache[city] = weather
        saveToDisk()
        return weather
    }

    private func saveToDisk() {
        do {
            let data = try JSONEncoder().encode(Array(cache.values))
            try data.write(to: fileURL, options: .atomic)
        } catch {
            assertionFailure("WeatherStore: не удалось сохранить кеш: \(error)")
        }
    }

    private func loadFromDisk() {
        guard let data = try? Data(contentsOf: fileURL),
              let saved = try? JSONDecoder().decode([CityWeather].self, from: data)
        else { return }
        for weather in saved {
            cache[weather.city] = weather
        }
    }
}
```

Логика `fetch` по шагам:

1. Если это **не** принудительное обновление (`force == false`) и в кеше есть данные моложе 10 минут — отдаём их без сети.
2. Если запрос этого города **уже идёт** — не начинаем второй, а ждём результата первого: `try await existing.value`. Сколько бы мест ни попросили Алматы одновременно, в сеть уйдёт один запрос.
3. Иначе создаём `Task` с запросом и запоминаем его в `inflight` («в полёте»).
4. Ждём результат, кладём в кеш, сохраняем кеш на диск.

`Task` и `task.value`. `Task { … }` запускает асинхронную работу и возвращает «ручку» к ней. `value` — «дождаться результата» (или ошибки). Два вызова `fetch` могут ждать `value` одной и той же задачи — оба получат один результат.

`defer { inflight[city] = nil }` — код в `defer` выполнится при выходе из функции **любым** путём: и после успеха, и если `task.value` бросит ошибку. Без `defer` упавший запрос навсегда

остался бы в `inflight`, и все дальнейшие вызовы ждали бы его результат — то есть получали бы ту же старую ошибку.

Почему 10 минут. Считаем нагрузку: 5 городов, в худшем случае обновление каждые 10 минут — 6 раз в час × 5 = 30 запросов в час на одного пользователя. Лимит бесплатного open-meteo — 5 000 в час с одного адреса. Потянуть «обновить» пользователь может когда угодно — это `force`, мимо кеша.

Гонки здесь нет, хотя проверка «есть ли уже задача» и запись в `inflight` разнесены. `WeatherStore` — `@MainActor`: весь его код выполняется на главном потоке по очереди, а между `if let existing` и `inflight[city] = task` нет `await`, значит, никто не вклинится. Приостановка возможна только на `await`.

Кеш на диске. После каждого удачного запроса весь кеш (массив `CityWeather`) пишется JSON-файлом в **Caches** — папку песочницы для данных, которые можно получить заново. В отличие от Documents, она не попадает в резервную копию, и система может очистить её, когда на телефоне кончается место. Для кеша погоды это ровно то, что нужно: пропал — скачаем. При создании хранилище читает файл, и первый экран может показать погоду ещё до ответа сети.

> **Дедупликация — обязательная часть сетевого слоя.** Без неё легко отправить пять одинаковых запросов из разных мест экрана. С ней — один запрос на ресурс, сколько бы желающих ни было.

15.6 Экран списка — параллельные запросы

`WeatherListViewController` — таблица городов:

```
import UIKit

final class WeatherListViewController: UIViewController {

    let cities = City.all
    var weather: [City: CityWeather] = [:]
    var failedCities: Set<City> = []
    var isLoading = false

    let tableView = UITableView(frame: .zero, style: .insetGrouped)
    let refreshControl = UIRefreshControl()
    let offlineBanner = OfflineBannerView()

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Погода"
        view.backgroundColor = .systemGroupedBackground
        setupLayout()

        // Сначала — то, что сохранилось с прошлого раза.
        for city in cities {
            weather[city] = WeatherStore.shared.cached(for: city)
        }
```

```
        ConnectivityMonitor.shared.subscribe(self) { [weak self] online in
            guard let self else { return }
            UIView.animate(withDuration: 0.25) {
                self.offlineBanner.isHidden = online
            }
            if online { self.loadAll(force: false) }
        }
    }

    func setupLayout() {
        tableView.dataSource = self
        tableView.delegate = self
        tableView.register(WeatherCityCell.self, forCellReuseIdentifier:
↪   WeatherCityCell.reuseID)
        tableView.register(WeatherSkeletonCell.self, forCellReuseIdentifier:
↪   WeatherSkeletonCell.reuseID)
        tableView.refreshControl = refreshControl
        refreshControl.addTarget(self, action: #selector(pulledToRefresh), for:
↪   .valueChanged)

        offlineBanner.isHidden = true
        let stack = UIStackView(arrangedSubviews: [offlineBanner, tableView])
        stack.axis = .vertical
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor),
            stack.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            stack.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        ])
    }

    @objc func pulledToRefresh() {
        loadAll(force: true)
    }
}
```

Состояние экрана — три свойства: `weather` (что уже знаем), `failedCities` (где не получилось) и `isLoading` (идёт ли загрузка). По ним каждая строка решает, как себя показать (15.7).

В `viewDidLoad` мы **сначала** берём из хранилища то, что сохранилось на диске с прошлого запуска. Если данные есть, пользователь сразу видит погоду, а свежая догрузится следом.

Загрузка запускается из подписки на состояние сети (15.10): монитор сразу сообщает текущее состояние, и если сеть есть — зовём `loadAll(force: false)`.

Баннер и таблица в одном вертикальном стеке. У `UIStackView` есть удобное свойство: скрытый (`isHidden = true`) arranged subview не занимает места, и промежуток после него тоже исчезает — остальные сдвигаются. Это особенность именно стека: он сам строит констрейнты для `arrangedSubviews` и пересобирает их при смене `isHidden`; у обычного `UIView` скрытый subview своё место не отдаёт. Показываем баннер — таблица уезжает вниз на его высоту; прячем — возвращается. Apple прямо разрешает анимировать `isHidden` arranged subview внутри блока `UIView.animate` — этим и пользуемся. Подробно о стеке — глава 14, раздел 14.5.

Сама загрузка:

```
import UIKit

extension WeatherListViewController {
    func loadAll(force: Bool) {
        guard !isLoading else {
            refreshControl.endRefreshing()
            return
        }
        isLoading = true
        failedCities.removeAll()
        tableView.reloadData()

        Task { [weak self] in
            guard let cities = self?.cities else { return }
            await withTaskGroup(of: (City, Result<CityWeather, Error>).self) { group in
                for city in cities {
                    group.addTask {
                        do {
                            let result = try await WeatherStore.shared.fetch(for: city,
                            ↪  force: force)
                            return (city, .success(result))
                        } catch {
                            return (city, .failure(error))
                        }
                    }
                }
                for await (city, result) in group {
                    self?.apply(result, for: city)
                }
            }
            self?.isLoading = false
            self?.refreshControl.endRefreshing()
        }
    }

    func apply(_ result: Result<CityWeather, Error>, for city: City) {
        switch result {
        case .success(let value):
            weather[city] = value
```

```
            failedCities.remove(city)
        case .failure:
            // Старые данные не выбрасываем: лучше вчерашняя погода, чем пустота.
            if weather[city] == nil {
                failedCities.insert(city)
            }
        }
        if let row = cities.firstIndex(of: city) {
            tableView.reloadRows(at: [IndexPath(row: row, section: 0)], with: .fade)
        }
    }
}
```

`withTaskGroup` — группа параллельных задач: запускаем по задаче на город и получаем результаты **по мере готовности**, в любом порядке. Все пять запросов уходят в сеть одновременно. Если бы мы ждали города по очереди (`for city in cities { try await … }`), общее время было бы суммой: пять запросов по 0,3 секунды — полторы секунды. Параллельно — примерно время самого медленного, около 0,3–0,5 секунды.

«Но `WeatherStore` на главном потоке — как они могут идти параллельно?» Параллельно идёт **ожидание сети**. Каждая задача доходит до `await` на запросе и отпускает главный поток; пока пять запросов в пути, поток свободен. Главный поток нужен только на короткие моменты — отправить запрос и разложить ответ.

`Result<CityWeather, Error>` — «либо погода, либо ошибка» одним значением. Каждая дочерняя задача ловит свою ошибку сама и возвращает `.failure`. Если бы ошибка вылетала из задачи наружу (в `withThrowingTaskGroup`), первая же упавшая задача прервала бы обработку остальных результатов. А нам нужно показать Алматы, даже если Астана не ответила.

`for await (city, result) in group` — перебор результатов по мере того, как задачи заканчиваются. Каждый результат сразу применяем: `reloadRows` обновляет одну строку, а не всю таблицу. Строки «проявляются» по одной, с анимацией `.fade` (плавное появление).

`[weak self]` и `self?.` внутри: если пользователь ушёл с экрана, пока грузится погода, экран не будет удерживаться в памяти до конца загрузки.

Защита от двойной загрузки. `guard !isLoading` — если загрузка уже идёт (например, человек дважды подряд потянул список), вторую не начинаем, только гасим индикатор обновления.

Старые данные не выбрасываем. Если обновление города не удалось, а сохранённые данные есть — показываем их. «Ошибка» без данных хуже, чем погода двадцатиминутной давности.

15.7 Ячейки — три состояния

Каждая строка бывает в одном из трёх состояний:

1. **Загрузка** — данных ещё нет, показываем `WeatherSkeletonCell` с бегущим бликом.
2. **Данные есть** — `WeatherCityCell` с температурой, иконкой и описанием.
3. **Ошибка** — та же `WeatherCityCell`, но серая, с прочерком и подсказкой «потяни, чтобы повторить».

Выбор — в data source:

```
import UIKit

extension WeatherListViewController: UITableViewDataSource {
    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        cities.count
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪  UITableViewCell {
        let city = cities[indexPath.row]
        if let weather = weather[city] {
            let cell = tableView.dequeueReusableCell(
                withIdentifier: WeatherCityCell.reuseID, for: indexPath
            ) as! WeatherCityCell
            cell.configure(with: weather)
            return cell
        }
        if failedCities.contains(city) {
            let cell = tableView.dequeueReusableCell(
                withIdentifier: WeatherCityCell.reuseID, for: indexPath
            ) as! WeatherCityCell
            cell.configureFailed(city: city)
            return cell
        }
        return tableView.dequeueReusableCell(withIdentifier: WeatherSkeletonCell.reuseID,
        ↪  for: indexPath)
    }
}

extension WeatherListViewController: UITableViewDelegate {
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        guard let weather = weather[cities[indexPath.row]] else { return }
        navigationController?.pushViewController(
            WeatherDetailViewController(weather: weather), animated: true
        )
    }
}
```

Регистрируем **два** класса ячеек (в `setupLayout` выше) и выбираем по состоянию. Переиспользование работает отдельно для каждого идентификатора: ячейка-заготовка никогда не превратится в ячейку города и наоборот.

По тапу открываем детальный экран, но только если данные есть — у заготовки и у ячейки с ошибкой показывать нечего.

Ячейка-заготовка — три серых прямоугольника на месте будущих названия, описания и температуры:

```
import UIKit

final class WeatherSkeletonCell: UITableViewCell {
    static let reuseID = "WeatherSkeletonCell"

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        isAccessibilityElement = true
        accessibilityLabel = "Загрузка"

        let name = SkeletonView()
        let condition = SkeletonView()
        let temperature = SkeletonView()
        [name, condition, temperature].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            contentView.addSubview($0)
        }
        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            name.topAnchor.constraint(equalTo: margins.topAnchor, constant: 4),
            name.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            name.widthAnchor.constraint(equalTo: margins.widthAnchor, multiplier: 0.4),
            name.heightAnchor.constraint(equalToConstant: 20),

            condition.topAnchor.constraint(equalTo: name.bottomAnchor, constant: 10),
            condition.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            condition.widthAnchor.constraint(equalTo: margins.widthAnchor, multiplier:
↪  0.3),
            condition.heightAnchor.constraint(equalToConstant: 14),
           condition.bottomAnchor.constraint(equalTo: margins.bottomAnchor, constant: -4),

            temperature.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
            temperature.centerYAnchor.constraint(equalTo: margins.centerYAnchor),
            temperature.widthAnchor.constraint(equalToConstant: 64),
            temperature.heightAnchor.constraint(equalToConstant: 36),
        ])
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }
}
```

Ширины заданы долями: `multiplier: 0.4` — 40% ширины полей ячейки. На iPhone шириной 402 точки поля ячейки `.insetGrouped` — около 330 точек, значит, «название» будет ~132 точки. Заготовка повторяет **форму** настоящей строки, поэтому, когда данные придут, строка не «прыгнет».

`accessibilityLabel = "Загрузка"` — VoiceOver не станет перечислять три безымянных прямоугольника, а скажет одно понятное слово.

Ячейка города:

```
import UIKit

final class WeatherCityCell: UITableViewCell {
    static let reuseID = "WeatherCityCell"

    private let nameLabel = UILabel()
    private let conditionLabel = UILabel()
    private let temperatureLabel = UILabel()
    private let iconView = UIImageView()

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        setupLayout()
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    func configure(with weather: CityWeather) {
        nameLabel.text = weather.city.name
        nameLabel.textColor = .label
        let today = weather.daily.first
        let range = today.map {
            "  ↓\(TemperatureFormat.string($0.minTemperature))
             ↑\(TemperatureFormat.string($0.maxTemperature))"
        } ?? ""
        conditionLabel.text = weather.condition.label + range
        temperatureLabel.text = TemperatureFormat.string(weather.temperature)
        iconView.image = UIImage(systemName: weather.condition.systemImage(isDay:
↪  weather.isDay))
        iconView.tintColor = weather.isDay ? .systemOrange : .systemIndigo
        accessoryType = .disclosureIndicator
        selectionStyle = .default
    }

    func configureFailed(city: City) {
        nameLabel.text = city.name
        nameLabel.textColor = .secondaryLabel
        conditionLabel.text = "Нет данных. Потяни список вниз, чтобы повторить"
        temperatureLabel.text = "—"
        iconView.image = UIImage(systemName: "exclamationmark.icloud")
        iconView.tintColor = .secondaryLabel
        accessoryType = .none
```

```
        selectionStyle = .none
    }

    private func setupLayout() {
        nameLabel.font = .preferredFont(forTextStyle: .headline)
        conditionLabel.font = .preferredFont(forTextStyle: .subheadline)
        conditionLabel.textColor = .secondaryLabel
        conditionLabel.numberOfLines = 0
        temperatureLabel.font = UIFontMetrics(forTextStyle: .largeTitle)
            .scaledFont(for: .systemFont(ofSize: 34, weight: .light))
        [nameLabel, conditionLabel, temperatureLabel].forEach {
            $0.adjustsFontForContentSizeCategory = true
        }
        iconView.contentMode = .scaleAspectFit
        iconView.preferredSymbolConfiguration = UIImage.SymbolConfiguration(textStyle:
↪   .title2)

        temperatureLabel.setContentHuggingPriority(.required, for: .horizontal)
        temperatureLabel.setContentCompressionResistancePriority(.required, for:
↪   .horizontal)

        let textStack = UIStackView(arrangedSubviews: [nameLabel, conditionLabel])
        textStack.axis = .vertical
        textStack.spacing = 4

        let row = UIStackView(arrangedSubviews: [iconView, textStack, temperatureLabel])
        row.spacing = 12
        row.alignment = .center
        row.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(row)

        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            iconView.widthAnchor.constraint(equalToConstant: 36),
            row.topAnchor.constraint(equalTo: margins.topAnchor),
            row.bottomAnchor.constraint(equalTo: margins.bottomAnchor),
            row.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            row.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
        ])
    }
}
```

Две настройки ячейки города, которые стоит разобрать.

Hugging и compression resistance. Когда в ряду несколько элементов, Auto Layout должен решить, кого растянуть, если места много, и кого сжать, если мало. Для этого у каждого view два приоритета:

- *content hugging* («обнимание содержимого») — насколько view сопротивляется **растяжению** больше своего естественного размера;
- *compression resistance* («сопротивление сжатию») — насколько сопротивляется **сжатию** меньше естественного размера.

По умолчанию у любого view hugging 250 и compression resistance 750 (проверено в симуляторе iOS 26.5; глава 0, раздел 0.3) — у названия, описания и температуры одинаковые, то есть ничья, и кого резать при нехватке места, Auto Layout решал бы сам. У температуры мы ставим оба на максимум (`.required`): «23°» всегда ровно такой ширины, как текст, не шире и не уже. Растягиваться и переносить строки будет описание слева. Без этого на узком экране с крупным шрифтом Auto Layout мог бы обрезать температуру до «2…».

`UIFontMetrics.scaledFont(for:)` — свой шрифт (34 точки, тонкий), который всё равно подстраивается под Dynamic Type: при системной настройке «крупнее» он вырастет пропорционально стилю `.largeTitle`. `preferredFont(forTextStyle:)` даёт только системные размеры, а `UIFontMetrics` — любой свой, но масштабируемый.

`configure` и `configureFailed` выставляют **все** свойства, включая цвет названия, стрелку справа и стиль выделения. Ячейка переиспользуется: если бы `configureFailed` делал название серым, а `configure` не возвращал бы `.label`, после ошибки и обновления название осталось бы серым.

15.8 Skeleton — бегущий блик

`SkeletonView` — серый прямоугольник, по которому слева направо пробегает светлая полоса:

```
import UIKit

final class SkeletonView: UIView {
    private let gradient = CAGradientLayer()

    override init(frame: CGRect) {
        super.init(frame: frame)
        layer.cornerRadius = 6
        clipsToBounds = true
        isAccessibilityElement = false

        gradient.startPoint = CGPoint(x: 0, y: 0.5)
        gradient.endPoint = CGPoint(x: 1, y: 0.5)
        gradient.locations = [0, 0, 0]
        layer.addSublayer(gradient)
        updateColors()
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func layoutSubviews() {
        super.layoutSubviews()
```

```
        gradient.frame = bounds
    }

    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateColors()
        }
    }

    override func didMoveToWindow() {
        super.didMoveToWindow()
        if window != nil {
            startAnimating()
        } else {
            gradient.removeAnimation(forKey: "shimmer")
        }
    }

    private func updateColors() {
        let base = UIColor.systemGray5.resolvedColor(with: traitCollection)
        let shine = UIColor.systemGray4.resolvedColor(with: traitCollection)
        backgroundColor = base
        gradient.colors = [base.cgColor, shine.cgColor, base.cgColor]
    }

    private func startAnimating() {
        guard !UIAccessibility.isReduceMotionEnabled else { return }
        let animation = CABasicAnimation(keyPath: "locations")
        animation.fromValue = [-1.0, -0.5, 0.0]
        animation.toValue = [1.0, 1.5, 2.0]
        animation.duration = 1.2
        animation.repeatCount = .infinity
        animation.isRemovedOnCompletion = false
        gradient.add(animation, forKey: "shimmer")
    }
}
```

Как это устроено:

- Сам view — серая подложка (`systemGray5`).
- Сверху — `CAGradientLayer`, слой-градиент из трёх цветов: серый → светлее → серый. Светлая середина и есть блик. `CAGradientLayer → CALayer → NSObject` — это слой, а не view: он не участвует в Auto Layout и не получает касаний, поэтому его рамку мы задаём руками, `gradient.frame = bounds`, и делаем это в `layoutSubviews` — там `bounds` view уже окончательные (глава 0, разделы 0.3 и 0.8). `clipsToBounds = true` — то же самое, что

`layer.masksToBounds` (одно свойство под двумя именами); без него градиент вылез бы за скруглённые углы.

- **`locations`** — где стоит каждый из трёх цветов, в долях ширины слоя: 0 — левый край, 1 — правый. Анимация двигает их от `[-1, -0.5, 0]` к `[1, 1.5, 2]`. В начале весь градиент левее view (блик в точке −0,5, за левым краем), в конце — правее (блик на 1,5, за правым краем). Между ними блик проезжает через видимую часть от 0 до 1. Одна пробежка — 1,2 секунды, затем сначала, бесконечно (`repeatCount = .infinity`).

Анимация идёт через Core Animation — она работает со слоями (`CALayer`), а не с view. `CABasicAnimation → CAPropertyAnimation → CAAnimation → NSObject` — описание анимации одного свойства слоя. Такую анимацию отрисовывает системный процесс рендеринга, и нашему коду не нужно ничего делать на каждом кадре. Она меняет только то, что видно на экране (presentation layer), а модельное значение `locations` так и остаётся `[0, 0, 0]`: когда анимация снята, три цвета стоят в одной точке, и блика просто нет — для нашего случая это то, что нужно (подробно о модельном и отображаемом слое — глава 32).

Первая версия этой главы анимировала сдвиг `transform.translation.x` от `−bounds.width` до `bounds.width` и запускала анимацию прямо в `cellForRowAt`. В этот момент ячейку только что создали или достали из очереди, и её subviews ещё не разложены: `bounds` у `SkeletonView` нулевые (проверено в симуляторе iOS 26.5: у subview свежей ячейки `bounds` = 0 × 0, и только после прохода layout появляется настоящая ширина). Размеры становятся известны в проходе раскладки — в `layoutSubviews` у view и в `viewDidLayoutSubviews` у view controller'а; в `init`, `viewDidLoad` и `cellForRowAt` их ещё нет (глава 0, раздел 0.3). Так анимация «от 0 до 0» не двигалась вовсе. Анимация `locations` не зависит от размеров: доли остаются долями при любой ширине.

Остальные детали:

- **`didMoveToWindow`** — вызывается, когда view попал в окно (стал видимым) или ушёл из него. Запускаем анимацию только когда view на экране, и снимаем, когда ушёл: невидимым заготовкам мигать незачем.
- **`isRemovedOnCompletion = false`** — по умолчанию (`true`) Core Animation снимает анимацию со слоя, когда истекает её длительность. У бесконечной анимации конца нет, так что для обычной работы флаг не нужен — мы оставляем его как страховку. Чего он не решает: после ухода приложения в фон и возврата бегущая анимация может оказаться снятой со слоя, а `didMoveToWindow` при этом не вызывается — view окно не покидала. Надёжный способ — перезапускать `startAnimating()` по `UIApplication.willEnterForegroundNotification`; для заготовок, которые живут секунду-другую, мы этого не делаем.
- **Тёмная тема.** `cgColor` — это «застывший» цвет: динамический `UIColor.systemGray5` при переводе в `CGColor` выбирает один вариант, светлый или тёмный, и дальше не меняется. Поэтому в `updateColors` берём цвет для текущей темы через `resolvedColor(with: traitCollection)`, а при смене темы (`traitCollectionDidChange`, *trait collection* — набор характеристик среды: тема, размер шрифта, класс размера экрана) перекрашиваем. Иначе переключишь телефон в тёмную тему — и заготовки останутся светло-серыми пятнами.
- **Reduce Motion.** Если в настройках универсального доступа включено «Уменьшение движения», бегущий блик — лишнее движение. Проверяем `UIAccessibility.isReduceMotionEnabled` и оставляем статичную серую заготовку.

`traitCollectionDidChange` в iOS 17 объявлен устаревшим в пользу `registerForTraitChanges`, но нового API нет в iOS 15. Для приложения с минимальной версией iOS 15 старый метод — правильный выбор, и компилятор не предупреждает.

Подробнее о состояниях загрузки — глава 23 (cookbook «загрузка»).

> **Упражнение 15.2.** Временно добавь в начало `WeatherAPI.fetch(for:)` строку `print("Запрос: \(city.name)")`. Запусти приложение и сразу, пока строки ещё грузятся, дважды потяни список вниз. Сколько строк «Запрос:» будет в консоли и почему? А если через минуту закрыть и снова открыть приложение?

15.9 Pull-to-refresh

«Потяни, чтобы обновить» — стандартный `UIRefreshControl`. Он уже подключён в `setupLayout()`:

```
tableView.refreshControl = refreshControl
refreshControl.addTarget(self, action: #selector(pulledToRefresh), for: .valueChanged)
```

и обработчик:

```
@objc func pulledToRefresh() {
    loadAll(force: true)
}
```

Свойство `refreshControl` есть у любого `UIScrollView` (и значит, у таблицы) с iOS 10: присвоил — и при оттягивании списка вниз появится крутящийся индикатор. Когда список оттянут достаточно далеко, контрол посылает событие `.valueChanged`. Это обычный контрол: `UIRefreshControl → UIControl → UIView`, поэтому у него target-action, и `addTarget` можно заменить на `addAction(UIAction { … }, for: .valueChanged)`; в заголовке `UIRefreshControl.h` так и написано — «when a user has pulled-to-refresh, the UIRefreshControl fires its UIControlEventValueChanged event» (глава 0, раздел 0.4).

`force: true` — мимо 10-минутного кеша, гарантированно в сеть. Это то, чего пользователь ждёт от жеста «обновить».

`refreshControl.endRefreshing()` — обязательно в конце загрузки (он стоит в `loadAll` после группы задач и в ветке `guard !isLoading`). Без него индикатор будет крутиться вечно: контрол не знает, когда твоя загрузка закончилась.

15.10 Офлайн-баннер через NWPathMonitor

Фреймворк Network даёт `NWPathMonitor` — «наблюдатель за маршрутом в сеть». Он сообщает, когда меняется доступность сети:

```
import Foundation
import Network

@MainActor
final class ConnectivityMonitor {
    static let shared = ConnectivityMonitor()
```

```
    private let monitor = NWPathMonitor()
    private(set) var isOnline = true
    private var observers: [ObjectIdentifier: Observer] = [:]

    private struct Observer {
        weak var owner: AnyObject?
        let handler: (Bool) -> Void
    }

    private init() {
        monitor.pathUpdateHandler = { [weak self] path in
            // Этот код выполняется на фоновой очереди монитора.
            let online = path.status == .satisfied
            DispatchQueue.main.async {
                self?.update(online: online)
            }
        }
        monitor.start(queue: DispatchQueue(label: "ConnectivityMonitor"))
    }

    func subscribe(_ owner: AnyObject, handler: @escaping (Bool) -> Void) {
        observers[ObjectIdentifier(owner)] = Observer(owner: owner, handler: handler)
        handler(isOnline)
    }

    private func update(online: Bool) {
        guard online != isOnline else { return }
        isOnline = online
        observers = observers.filter { $0.value.owner != nil }
        for observer in observers.values {
            observer.handler(online)
        }
    }
}
```

Потоки. `monitor.start(queue:)` запускает наблюдение на своей фоновой очереди, и `pathUpdateHandler` вызывается **на ней**. Трогать отсюда состояние монитора (он `@MainActor`) и тем более интерфейс нельзя. Поэтому внутри обработчика только вычисляем `online` и перекидываем работу на главный поток через `DispatchQueue.main.async`.

В режиме Swift 6 компилятор это проверяет: тип обработчика в SDK — `@Sendable (NWPath) -> Void`, то есть «может выполняться в любом потоке», и прямой вызов `self?.update(online:)` из него не соберётся (проверено: ошибка «call to main actor-isolated instance method ‘update(online:)’ in a synchronous nonisolated context»). `DispatchQueue.main.async` компилятор понимает как «дальше — главный поток», и вызов внутри разрешён — вариант из листинга собирается без замечаний. Очередь главного потока выполняет блоки строго по порядку, так что «сеть пропала» и «сеть вернулась» не перепутаются.

`path.status` бывает `.satisfied` (сеть есть), `.unsatisfied` (нет) и `.requiresConnection` (сети нет, но может появиться по запросу — например, VPN «по требованию»). Сетью считаем только `.satisfied`.

`guard online != isOnline` — пропускаем повторы. Монитор сообщает о любом изменении маршрута, например при переходе с Wi-Fi на мобильную сеть. Для нас это «было онлайн — осталось онлайн», и подписчикам незачем об этом знать.

`subscribe` сразу отдаёт текущее значение. Без этого подписчик ждал бы **первого изменения**, которого может и не случиться, если сеть стабильна. Полезная привычка для любых наблюдателей: подписался — сразу получи текущее состояние. Наблюдатели хранятся так же, как в главе 12 (12.4): слабая ссылка на владельца, без `deinit`.

Одна тонкость: до первого сообщения от монитора `isOnline = true` — это предположение. Если приложение запустили без сети, список сначала попробует загрузиться (и получит ошибки), а через мгновение монитор сообщит «сети нет» — появится баннер. Сохранённые данные при этом показаны с самого начала.

Подписка в списке (из `viewDidLoad`):

```
ConnectivityMonitor.shared.subscribe(self) { [weak self] online in
    guard let self else { return }
    UIView.animate(withDuration: 0.25) {
        self.offlineBanner.isHidden = online
    }
    if online { self.loadAll(force: false) }
}
```

Сеть **пропала** — показываем баннер. Сеть **вернулась** — баннер прячется, и сразу загружаем данные: без `force`, так что свежие города останутся из кеша, а устаревшие обновятся.

Сам баннер — красная плашка с текстом:

```
import UIKit

final class OfflineBannerView: UIView {
    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .systemRed

        let label = UILabel()
        label.text = "Нет подключения — показываем сохранённые данные"
        label.textColor = .white
        label.font = .preferredFont(forTextStyle: .footnote)
        label.adjustsFontForContentSizeCategory = true
        label.numberOfLines = 0
        label.textAlignment = .center
        label.translatesAutoresizingMaskIntoConstraints = false
        addSubview(label)

        NSLayoutConstraint.activate([
```

```
            label.topAnchor.constraint(equalTo: topAnchor, constant: 8),
            label.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -8),
            label.leadingAnchor.constraint(equalTo: layoutMarginsGuide.leadingAnchor),
            label.trailingAnchor.constraint(equalTo: layoutMarginsGuide.trailingAnchor),
        ])
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }
}
```

`numberOfLines = 0` — при крупном шрифте текст перенесётся на две строки, а плашка вырастет по высоте, потому что метка привязана к её верху и низу.

NWPathMonitor не обещает, что сервер доступен. `.satisfied` значит «есть маршрут в сеть», а не «open-meteo ответит». Wi-Fi в кафе с экраном входа, заблокированный домен — монитор скажет «онлайн», а запросы упадут. Поэтому ячейки с ошибкой (15.7) нужны всё равно, баннер — только подсказка.

15.11 Детальный экран — блоки и параметры

`WeatherDetailViewController` — прокручиваемый экран с пятью блоками друг под другом:

1. **Заголовок** — большая иконка, температура, описание.
2. **Полоса по часам** — горизонтальная прокрутка на 24 часа.
3. **Три карточки** — ветер, влажность, «ощущается как».
4. **Прогноз на 7 дней.**
5. **«Обновлено 3 минуты назад».**

```
import UIKit

final class WeatherDetailViewController: UIViewController {

    let weather: CityWeather
    let updatedLabel = UILabel()
    var timer: Timer?

    init(weather: CityWeather) {
        self.weather = weather
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
```

```
        super.viewDidLoad()
        title = weather.city.name
        navigationItem.largeTitleDisplayMode = .never
        view.backgroundColor = .systemBackground

        let scrollView = UIScrollView()
        scrollView.alwaysBounceVertical = true
        scrollView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(scrollView)

        let content = UIStackView(arrangedSubviews: [
            makeHeader(),
            HourlyStripView(hours: weather.hourly, timeZone: weather.timeZone),
            makeMetrics(),
            DailyForecastView(days: weather.daily, timeZone: weather.timeZone),
            updatedLabel,
        ])
        content.axis = .vertical
        content.spacing = 16
        content.translatesAutoresizingMaskIntoConstraints = false
        scrollView.addSubview(content)

        updatedLabel.font = .preferredFont(forTextStyle: .footnote)
        updatedLabel.adjustsFontForContentSizeCategory = true
        updatedLabel.textColor = .secondaryLabel
        updatedLabel.textAlignment = .center

        NSLayoutConstraint.activate([
            scrollView.topAnchor.constraint(equalTo: view.topAnchor),
            scrollView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            scrollView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            scrollView.bottomAnchor.constraint(equalTo: view.bottomAnchor),

            content.topAnchor.constraint(equalTo: scrollView.contentLayoutGuide.topAnchor,
↪   constant: 16),
            content.bottomAnchor.constraint(equalTo:
↪   scrollView.contentLayoutGuide.bottomAnchor, constant: -16),
            content.leadingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.leadingAnchor),
            content.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor),
        ])
        updateRelativeTime()
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        // Раз в 30 секунд обновляем «N минут назад».
```

```
        timer = Timer.scheduledTimer(withTimeInterval: 30, repeats: true) { [weak self] _
↪   in
            MainActor.assumeIsolated {
                self?.updateRelativeTime()
            }
        }
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        timer?.invalidate()
        timer = nil
    }

    func updateRelativeTime() {
        let seconds = Date().timeIntervalSince(weather.updatedAt)
        if seconds < 60 {
            updatedLabel.text = "Обновлено только что"
            return
        }
        let formatter = RelativeDateTimeFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.unitsStyle = .full
        updatedLabel.text = "Обновлено " + formatter.localizedString(for:
↪   weather.updatedAt, relativeTo: Date())
    }
}
```

Прокрутка через два layout guide. У `UIScrollView` (iOS 11+) есть две направляющие:

- `frameLayoutGuide` — **видимая** рамка прокрутки на экране;
- `contentLayoutGuide` — **содержимое**, которое прокручивается; его размер определяет, насколько далеко можно листать.

Обе — обычные `UILayoutGuide`, не view: прямоугольники-ориентиры для констрейнтов (глава 0, раздел 0.8). Верх и низ стека привязаны к `contentLayoutGuide`: высота содержимого = высота стека + отступы 16 сверху и снизу — так Auto Layout вычисляет `contentSize`. Если стек выше экрана — появляется прокрутка. Лево и право стека привязаны не к `contentLayoutGuide`, а к полям основного `view`: ширина стека задана снаружи, а ширине содержимого прокрутки не из чего вырасти больше рамки — и горизонтальной прокрутки нет. (Чаще в таких случаях приравнивают `content.widthAnchor` к `scrollView.frameLayoutGuide.widthAnchor` — смысл тот же.) Сам scroll view прижат к краям `view`, а не к safe area, но под навигационную панель содержимое не уедет: `contentInsetAdjustmentBehavior` по умолчанию `.automatic`, и прокрутка сама добавляет safe area в `contentInset`. Прокрутка как таковая — это сдвиг `bounds.origin` scroll view, он же `contentOffset`; подробно — глава 0, раздел 0.5.

`RelativeDateTimeFormatter` (iOS 13+) — «N минут назад»:

- `unitsStyle = .full` → «3 минуты назад», «21 минуту назад»;
- `.short` → «3 мин. назад»;

- `.abbreviated` → «-3 мин» (в русской локали выглядит неожиданно — проверь, прежде чем выбрать).

Русские окончания («1 минуту», «3 минуты», «5 минут», «21 минуту») форматтер ставит сам — в отличие от «год / года / лет» в главе 10, где мы писали свою функцию. Все эти варианты проверены на macOS с локалью `ru_RU`.

Особый случай — меньше минуты. Для «только что» форматтер выдаёт странное: на один и тот же момент — «через 0 секунд», а на 20 секунд назад — «20 секунд назад». Поэтому до 60 секунд пишем сами: «Обновлено только что».

Таймер. Надпись «3 минуты назад» через пять минут станет враньём. Раз в 30 секунд пересчитываем её. Таймер создаём в `viewWillAppear`, останавливаем (`invalidate`) в `viewWillDisappear`: невидимому экрану тикать незачем, а главное — `Timer` держит своё замыкание, и забытый таймер жил бы вечно.

`MainActor.assumeIsolated { … }` внутри таймера — подсказка компилятору. `Timer.scheduledTimer` из главного потока срабатывает на главном потоке, но тип его замыкания в SDK об этом не говорит, и на вызов метода контроллера Swift 6 выдаёт предупреждение «call to main actor-isolated instance method … in a synchronous nonisolated context» (проверено: именно предупреждение, а не ошибка, как у `NWPathMonitor` выше — для API, импортированных из Objective-C, компилятор смягчает диагностику). `assumeIsolated` говорит: «я знаю, что здесь главный поток» — и проверяет это во время выполнения (подробнее — глава 5, раздел 5.7).

Блоки экрана:

```
import UIKit

extension WeatherDetailViewController {
    func makeHeader() -> UIView {
        let icon = UIImageView(image: UIImage(systemName:
        ↪  weather.condition.systemImage(isDay: weather.isDay)))
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 64)
        icon.tintColor = weather.isDay ? .systemOrange : .systemIndigo
        icon.contentMode = .scaleAspectFit

        let temperature = UILabel()
        temperature.text = TemperatureFormat.string(weather.temperature)
        temperature.font = UIFontMetrics(forTextStyle: .largeTitle)
            .scaledFont(for: .systemFont(ofSize: 72, weight: .thin))
        temperature.adjustsFontForContentSizeCategory = true

        let condition = UILabel()
        condition.text = weather.condition.label
        condition.font = .preferredFont(forTextStyle: .title3)
        condition.adjustsFontForContentSizeCategory = true

        let stack = UIStackView(arrangedSubviews: [icon, temperature, condition])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 4
```

```
        return stack
    }

    func makeMetrics() -> UIView {
        let wind = String(format: "%.0f м/с", weather.windSpeed)
        let humidity = "\(Int(weather.humidity.rounded()))%"
        let feels = TemperatureFormat.string(weather.apparentTemperature)
        let cards = [
            makeCard(symbol: "wind", title: "Ветер", value: wind),
            makeCard(symbol: "humidity", title: "Влажность", value: humidity),
            makeCard(symbol: "thermometer.medium", title: "Ощущается", value: feels),
        ]
        let row = UIStackView(arrangedSubviews: cards)
        row.distribution = .fillEqually
        row.spacing = 8
        return row
    }

    private func makeCard(symbol: String, title: String, value: String) -> UIView {
        let icon = UIImageView(image: UIImage(systemName: symbol))
        icon.tintColor = .secondaryLabel
        icon.contentMode = .scaleAspectFit

        let titleLabel = UILabel()
        titleLabel.text = title
        titleLabel.font = .preferredFont(forTextStyle: .caption1)
        titleLabel.textColor = .secondaryLabel

        let valueLabel = UILabel()
        valueLabel.text = value
        valueLabel.font = .preferredFont(forTextStyle: .title3)

        [titleLabel, valueLabel].forEach {
            $0.adjustsFontForContentSizeCategory = true
            $0.adjustsFontSizeToFitWidth = true
            $0.minimumScaleFactor = 0.7
        }

        let stack = UIStackView(arrangedSubviews: [icon, titleLabel, valueLabel])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 4
        stack.isLayoutMarginsRelativeArrangement = true
        stack.directionalLayoutMargins = NSDirectionalEdgeInsets(top: 12, leading: 8,
↪   bottom: 12, trailing: 8)
        stack.backgroundColor = .secondarySystemBackground
        stack.layer.cornerRadius = 12
        stack.isAccessibilityElement = true
```

```
        stack.accessibilityLabel = "\(title): \(value)"
        return stack
    }
}
```

Три карточки одинаковой ширины — горизонтальный стек с `.fillEqually`. У каждой карточки фон и скругление задаются прямо на `UIStackView`. До iOS 14 это не работало: слой стека был особым, нерисующим (`CATransformLayer`), и `backgroundColor`, `layer.cornerRadius`, `layer.borderWidth` на нём пропадали — фон приходилось вешать на отдельный `UIView` под стеком. С iOS 14 у стека обычный `CALayer` (на iOS 26.5 `UIStackView.layerClass` возвращает `CALayer`), и фон со скруглением рисуются. В `UIStackView.h` стек по-прежнему назван «non-rendering»: переопределять у него `layerClass` и рисовать в `draw(_:)` нельзя, он только расставляет. `isLayoutMarginsRelativeArrangement` + `directionalLayoutMargins` — внутренние отступы стека: 12 точек сверху и снизу, 8 по бокам.

`String(format: "%.0f м/с", …)` — число без знаков после запятой: 0,94 → «1 м/с».

Доступность карточки. `isAccessibilityElement = true` на всей карточке + `accessibilityLabel = "Ветер: 1 м/с"` — VoiceOver прочитает карточку одной фразой, а не тремя отдельными элементами (иконка, «Ветер», «1 м/с»).

15.12 Почасовая полоса — горизонтальная прокрутка внутри вертикальной

Прокрутка внутри прокрутки звучит страшно, но работает: UIKit сам понимает, куда ведёт палец. Движение вбок достаётся внутренней (горизонтальной) полосе, вверх-вниз — внешней. Получается это потому, что внутренняя полоса по вертикали прокручиваться не может — её содержимое ровно по высоте рамки (см. констрейнт ниже), и вертикальный pan её распознаватель не забирает, он достаётся внешней прокрутке.

```
import UIKit

final class HourlyStripView: UIView {
    private let scrollView = UIScrollView()
    private let stack = UIStackView()

    init(hours: [HourForecast], timeZone: TimeZone) {
        super.init(frame: .zero)
        backgroundColor = .secondarySystemBackground
        layer.cornerRadius = 16

        scrollView.showsHorizontalScrollIndicator = false
        scrollView.translatesAutoresizingMaskIntoConstraints = false
        addSubview(scrollView)

        stack.axis = .horizontal
        stack.spacing = 20
        stack.translatesAutoresizingMaskIntoConstraints = false
```

```
        scrollView.addSubview(stack)

        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.timeZone = timeZone
        formatter.dateFormat = "HH"

        for (index, hour) in hours.enumerated() {
            let time = index == 0 ? "Сейчас" : formatter.string(from: hour.time)
            stack.addArrangedSubview(Self.makeHourCell(time: time, hour: hour))
        }

        let content = scrollView.contentLayoutGuide
        let frame = scrollView.frameLayoutGuide
        NSLayoutConstraint.activate([
            scrollView.topAnchor.constraint(equalTo: topAnchor),
            scrollView.bottomAnchor.constraint(equalTo: bottomAnchor),
            scrollView.leadingAnchor.constraint(equalTo: leadingAnchor),
            scrollView.trailingAnchor.constraint(equalTo: trailingAnchor),

            stack.topAnchor.constraint(equalTo: content.topAnchor, constant: 12),
            stack.bottomAnchor.constraint(equalTo: content.bottomAnchor, constant: -12),
            stack.leadingAnchor.constraint(equalTo: content.leadingAnchor, constant: 16),
            stack.trailingAnchor.constraint(equalTo: content.trailingAnchor, constant:
↪  -16),
            // Высота содержимого = высота видимой области минус отступы:
            // прокрутка остаётся только горизонтальной.
            stack.heightAnchor.constraint(equalTo: frame.heightAnchor, constant: -24),
        ])
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    private static func makeHourCell(time: String, hour: HourForecast) -> UIView {
        let timeLabel = UILabel()
        timeLabel.text = time
        timeLabel.font = .preferredFont(forTextStyle: .caption1)
        timeLabel.textColor = .secondaryLabel

        let icon = UIImageView(image: UIImage(systemName:
         ↪  hour.condition.systemImage(isDay: hour.isDay)))
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(textStyle:
↪  .title3)
        icon.tintColor = .label
        icon.contentMode = .scaleAspectFit
```

```
        let temperatureLabel = UILabel()
        temperatureLabel.text = TemperatureFormat.string(hour.temperature)
        temperatureLabel.font = .preferredFont(forTextStyle: .body)

        let cell = UIStackView(arrangedSubviews: [timeLabel, icon, temperatureLabel])
        cell.axis = .vertical
        cell.alignment = .center
        cell.spacing = 6
        cell.isAccessibilityElement = true
        cell.accessibilityLabel = "\(time): \(TemperatureFormat.string(hour.temperature)),
↪  \(hour.condition.label)"
        return cell
    }
}
```

Устройство:

- Прокрутка занимает весь блок.
- Горизонтальный стек часов привязан к `contentLayoutGuide` со всех сторон — его ширина (24 часа × ~50 точек) и задаёт, насколько далеко можно листать.
- **Высота** стека приравнена к высоте `frameLayoutGuide` минус 24 (по 12 сверху и снизу). Так содержимое ровно по высоте видимой области, и вертикально полоса не прокручивается — только вбок. Высоту всему блоку при этом задаёт содержимое: часовые ячейки через свой `intrinsicContentSize` определяют высоту стека, стек — высоту `contentLayoutGuide`, а этот констрейнт приравнивает к ней высоту рамки, то есть самого scroll view. Отдельный констрейнт высоты блоку не нужен.
- `showsHorizontalScrollIndicator = false` — без полоски-индикатора внизу, как в «Погоде» Apple.

Каждый час — маленький вертикальный стек, а не `UICollectionViewCell`. Их всего 24, все создаются сразу; переиспользование нужно, когда элементов сотни.

Часы в поясе города. `formatter.timeZone = timeZone` — главное в этом блоке. Если смотреть погоду Алматы с телефона, настроенного на Москву (UTC+3), без этой строки полоса показала бы московское время: «12» вместо «14». Прогноз города должен показывать **местное** время города.

Прогноз по дням:

```
import UIKit

final class DailyForecastView: UIView {

    init(days: [DayForecast], timeZone: TimeZone) {
        super.init(frame: .zero)
        backgroundColor = .secondarySystemBackground
        layer.cornerRadius = 16

        var calendar = Calendar(identifier: .gregorian)
        calendar.timeZone = timeZone
```

```
        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.timeZone = timeZone
        formatter.setLocalizedDateFormatFromTemplate("EEEE")

        let stack = UIStackView()
        stack.axis = .vertical
        stack.spacing = 12
        stack.translatesAutoresizingMaskIntoConstraints = false
        addSubview(stack)

        for day in days {
            let name = calendar.isDate(day.date, inSameDayAs: Date())
                ? "Сегодня"
                : formatter.string(from: day.date).capitalized
            stack.addArrangedSubview(Self.makeRow(name: name, day: day))
        }

        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: topAnchor, constant: 12),
            stack.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -12),
            stack.leadingAnchor.constraint(equalTo: leadingAnchor, constant: 16),
            stack.trailingAnchor.constraint(equalTo: trailingAnchor, constant: -16),
        ])
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    private static func makeRow(name: String, day: DayForecast) -> UIView {
        let nameLabel = UILabel()
        nameLabel.text = name
        nameLabel.font = .preferredFont(forTextStyle: .body)

        let icon = UIImageView(image: UIImage(systemName: day.condition.systemImage(isDay:
        ↪  true)))
        icon.tintColor = .label
        icon.contentMode = .scaleAspectFit

        let chanceLabel = UILabel()
        chanceLabel.font = .preferredFont(forTextStyle: .footnote)
        chanceLabel.textColor = .systemBlue
        if let chance = day.precipitationChance, chance >= 20 {
            chanceLabel.text = "\(chance)%"
        }

        let rangeLabel = UILabel()
```

```
        rangeLabel.text = "\(TemperatureFormat.string(day.minTemperature)) …
↪  \(TemperatureFormat.string(day.maxTemperature))"
        rangeLabel.font = .monospacedDigitSystemFont(
            ofSize: UIFont.preferredFont(forTextStyle: .body).pointSize, weight: .regular
        )
        rangeLabel.textAlignment = .right

        [nameLabel, chanceLabel].forEach { $0.adjustsFontForContentSizeCategory = true }
        nameLabel.setContentHuggingPriority(.defaultLow, for: .horizontal)
        rangeLabel.setContentHuggingPriority(.required, for: .horizontal)

       let row = UIStackView(arrangedSubviews: [nameLabel, chanceLabel, icon, rangeLabel])
        row.spacing = 8
        row.alignment = .center
        NSLayoutConstraint.activate([
            icon.widthAnchor.constraint(equalToConstant: 28),
            chanceLabel.widthAnchor.constraint(equalToConstant: 40),
        ])
        row.isAccessibilityElement = true
        var spoken = "\(name): от \(TemperatureFormat.string(day.minTemperature)) до
        ↪  \(TemperatureFormat.string(day.maxTemperature)), \(day.condition.label)"
        if let text = chanceLabel.text { spoken += ", вероятность осадков \(text)" }
        row.accessibilityLabel = spoken
        return row
    }
}
```

`calendar.timeZone = timeZone` — то же для дат: «сегодня» в Алматы начинается в 00:00 по UTC+5. Дата дня из API (1790449200) — это полночь 27 сентября **по Алматы**, то есть 19:00 26 сентября по UTC. Календарь в поясе UTC назвал бы этот день «26 сентября, суббота», и вся неделя съехала бы на день.

`setLocalizedDateFormatFromTemplate("EEEE")` — полное название дня недели по правилам локали: «понедельник». `.capitalized` делает первую букву заглавной — в русском дни недели пишутся со строчной, но в начале строки таблицы заглавная выглядит аккуратнее.

Вероятность осадков показываем, только если она 20% и больше: «0%» и «8%» — шум, на который глаз зря тратит внимание.

`monospacedDigitSystemFont` для диапазона — цифры одной ширины, и колонка «12° … 23°» ровная во всех строках.

15.13 Бытовая аналогия

`WeatherListViewController` — **диспетчер**, который раздаёт поручения курьерам: «один едет в Алматы, второй в Астану, третий в Шымкент — все одновременно». Каждый курьер возвращается со сводкой, и диспетчер вывешивает её на доску, не дожидаясь остальных.

`WeatherStore` — **архивариус** диспетчера. Помнит, что курьер из Алматы вернулся пять минут назад, — второй раз не посылает, отдаёт из архива. Если двое одновременно спросят про Алматы, а курьер ещё в пути, архивариус скажет обоим: «ждите, он уже едет». И копию архива держит в шкафу (на диске), чтобы утром не начинать с пустых полок.

`NWPathMonitor` — **наблюдатель** за дорогами. Если дороги закрыты, диспетчер вешает табличку «Связи нет, на доске — вчерашние сводки». Но наблюдатель видит только дороги: открыта ли контора в Астане, он не знает.

15.14 Что мы пропустили

- **Геолокация.** Сейчас города зашиты в код. В настоящем приложении `CLLocationManager` определяет, где пользователь, и добавляет «Моё местоположение» (нужно разрешение — см. главу 7).
- **Поиск города.** У open-meteo есть отдельный сервис геокодинга (`geocoding-api.open-meteo.com`) — поиск координат по названию.
- **Предупреждения о непогоде** и push-уведомления — нужен сервер или WeatherKit.
- **Виджет** на главном экране — WidgetKit, отдельный таргет.
- **Детальный экран не обновляется сам.** Он показывает данные на момент открытия. Можно подписать его на обновления хранилища тем же приёмом наблюдателей.

  Упражнение 15.3. Проверь приложение целиком:

  1. Первый запуск: сначала строки-заготовки с бегущим бликом, потом пять городов с температурой.
  2. Потяни список вниз — индикатор покрутится и исчезнет.
  3. Отключи интернет у Mac (в симуляторе нет своего переключателя сети — он пользуется сетью Mac) и потяни снова — появится красный баннер, данные останутся на месте.
  4. Включи интернет — баннер исчезнет.
  5. Добавь внизу детального экрана подпись «Weather data by Open-Meteo.com», которую требует лицензия CC BY 4.0.

Ответы к упражнениям

Упражнение 15.1. Добавь в `City.all`:

```
City(name: "Актау", latitude: 43.65, longitude: 51.17),
```

API вернёт `"timezone":"Asia/Aqtau"` и `"utc_offset_seconds":18000` — 5 часов, как и у остальных городов Казахстана. Проверка:

```
curl "https://api.open-meteo.com/v1/forecast?latitude=43.65&longitude=51.17&timezone=aut
↪  o&current=temperature_2m"
```

Упражнение 15.2. На старте — пять строк, по одной на город. Первое оттягивание во время загрузки ничего не добавит: `loadAll` увидит `isLoading == true` и только погасит индикатор. Второе — тоже, если загрузка ещё идёт; если уже закончилась — ещё пять строк (`force: true`

идёт мимо кеша). Через минуту после перезапуска строк не будет вовсе: кеш прочитан с диска, данные моложе 10 минут, и `fetch(force: false)` отдаёт их без сети.

Упражнение 15.3. Пункты 1–4 — ожидаемое поведение описано в них. Если баннер не появляется, проверь, что Mac действительно без сети (откроется ли сайт в Safari симулятора). Для пункта 5 — в `WeatherDetailViewController.viewDidLoad` добавь в стек `content` ещё одну метку:

```
let attribution = UILabel()
attribution.text = "Weather data by Open-Meteo.com"
attribution.font = .preferredFont(forTextStyle: .caption2)
attribution.textColor = .tertiaryLabel
attribution.textAlignment = .center
content.addArrangedSubview(attribution)
```

Что мы выучили

- **DTO** повторяет JSON, **доменная модель** — то, чем пользуется интерфейс. Маппинг — отдельная функция, её можно проверить без сети.
- Модели помечаем `nonisolated`, иначе в режиме MainActor по умолчанию их `Codable` привязан к главному потоку.
- Время — Unix-секундами (`timeformat=unixtime`), пояс — из ответа (`timezone=auto`). Показываем в поясе **города**: `formatter.timeZone`, `calendar.timeZone`.
- Опциональные поля DTO (`[Int?]`) — там, где API может прислать `null`, иначе один `null` роняет весь ответ.
- WMO-коды → `WeatherCondition` с подписью и иконкой (днём и ночью).
- `URLComponents` для адреса, свой код проверки статуса, ошибки разделены на сеть, статус и декодирование.
- `WeatherStore`: свежесть 10 минут, дедупликация через `inflight: [City: Task]` и `defer`, кеш на диске в Caches.
- `withTaskGroup` + `Result` — параллельные запросы, результаты по мере готовности, ошибка одного города не мешает остальным.
- Три состояния строки: заготовка, данные, ошибка. Старые данные при ошибке не выбрасываем.
- Skeleton — `CAGradientLayer` с анимацией `locations` (не зависит от размера), перекраска при смене темы, уважение к Reduce Motion. Слой — не view: его `frame` задаём сами в `layoutSubviews`, где `bounds` уже известны; в `init`, `viewDidLoad` и `cellForRowAt` они ещё нули.
- `UIRefreshControl` — `UIControl`, событие `.valueChanged`; `endRefreshing()` — обязательно в конце.
- Фон и скругление на `UIStackView` работают с iOS 14, когда у стека появился обычный `CALayer`.
- `NWPathMonitor` зовёт обработчик на фоновой очереди — переводим на главный поток через `DispatchQueue.main.async`. «Онлайн» не значит «сервер ответит».
- `RelativeDateTimeFormatter` сам склоняет «минуту / минуты / минут», но «только что» лучше написать самому; таймер — в `viewWillAppear`/`viewWillDisappear`.
- Прокрутка через `contentLayoutGuide` и `frameLayoutGuide`, вложенная горизонтальная полоса — без коллекции.

Apple Developer Documentation

- URLSession — стандартный сетевой клиент, через `URLSession.shared` тянем погоду.
- URLSession.data(for:) — асинхронный сетевой запрос (iOS 15+), возвращает `(Data, URLResponse)`.
- URLRequest — запрос с `cachePolicy` и `timeoutInterval`.
- URLCache — системный кеш HTTP-ответов; мы его обходим, потому что держим свой.
- NWPathMonitor — наблюдение за доступностью сети.
- UIRefreshControl — стандартный pull-to-refresh, привязывается к `tableView.refreshControl`.
- CAGradientLayer — слой-градиент, основа бегущего блика.
- RelativeDateTimeFormatter — «N минут назад» / «через N часов» с русскими окончаниями.
- JSONDecoder — декодер JSON, превращает ответ open-meteo в DTO.
- Concurrency — Swift Book про async/await, `Task`, `TaskGroup`, акторы.

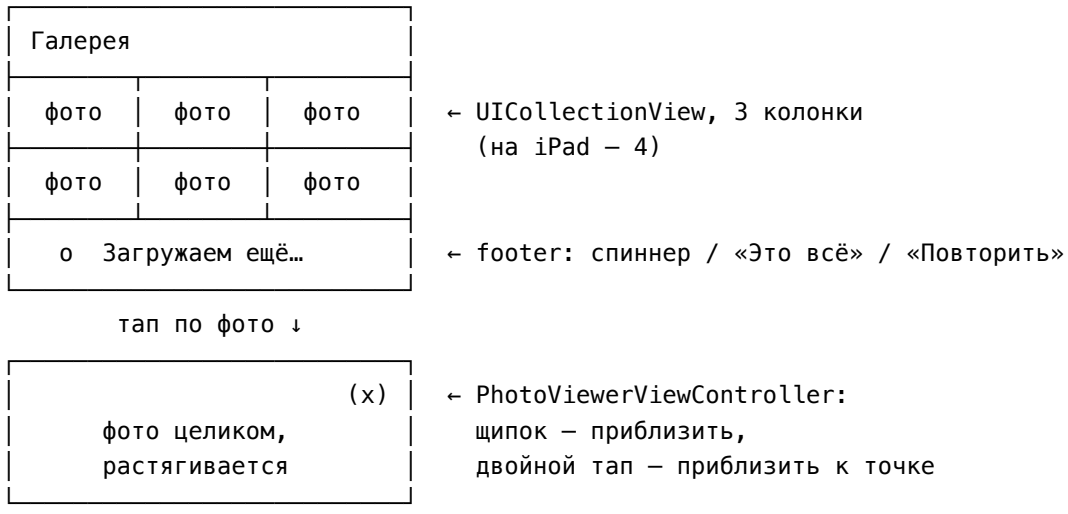
→ Глава 16. Gallery — UICollectionView compositional, пагинация, photo viewer

Глава 16. Gallery — UICollectionView compositional, пагинация, photo viewer

Галерея — это сетка картинок. Звучит просто, но внутри прячется много тонкостей: сетка должна подстраиваться под ширину экрана, картинки грузятся из сети не сразу, новые страницы подтягиваются по мере прокрутки, всё это надо где-то держать в памяти, а по тапу фото открывается на весь экран, и его можно растянуть двумя пальцами.

В этой главе строим всё это целиком. Источник картинок — **picsum.photos**, публичный сервис с фотографиями, которому не нужен ключ API.

Что получится:



Файлы мини-приложения:

```
Apps/Gallery/
├── PicsumPhoto.swift              ← модель одной фотографии
├── GalleryAPI.swift               ← запрос страницы списка
├── ImageCache.swift               ← кеш картинок в памяти
├── GalleryCell.swift              ← ячейка сетки
├── FooterLoaderView.swift         ← нижний «загружаем ещё»
├── PhotoViewerViewController.swift ← полноэкранный просмотр
```



Рис. 12: Галерея: сетка превью в три колонки

```
├── PhotoSaver.swift                ← сохранение в «Фото»
└── GalleryViewController.swift     ← экран с сеткой
```

> **Режим сборки.** Весь код главы проверен в тех настройках, что описаны во введении: Swift 6, `Default Actor Isolation = MainActor`, deployment target iOS 15. В этом режиме любой класс и функция без пометки живут на главном потоке (main actor), поэтому UIKit-код можно писать без `@MainActor` на каждом классе. Там, где тип должен работать и вне главного потока, мы явно пишем `nonisolated` — и объясняем почему.

16.1 picsum.photos — какие данные

picsum.photos даёт два полезных адреса (endpoint'а — так называют конкретный URL, который отвечает на один вид запроса):

- `https://picsum.photos/v2/list?page=1&limit=30` — JSON со списком фотографий: id, автор, исходный размер, ссылка на оригинал.
- `https://picsum.photos/id/{id}/{width}/{height}` — сама картинка нужного размера. Сервер сам уменьшит и обрежет оригинал.

Так выглядит один элемент ответа (реальный ответ сервиса, первая запись):

```
{
  "id": "0",
  "author": "Alejandro Escamilla",
  "width": 5000,
  "height": 3333,
  "url": "https://unsplash.com/photos/yC-Yzbqy7PY",
  "download_url": "https://picsum.photos/id/0/5000/3333"
}
```

Модель:

```
import Foundation

nonisolated struct PicsumPhoto: Decodable, Hashable, Sendable {
    let id: String
    let author: String
    let width: Int
    let height: Int
    let downloadURL: String

    enum CodingKeys: String, CodingKey {
        case id, author, width, height
        case downloadURL = "download_url"
    }

    func thumbnailURL(side: Int) -> URL? {
        URL(string: "https://picsum.photos/id/\(id)/\(side)/\(side)")
    }
```

```
    func fullURL(maxSide: Int = 1200) -> URL? {
        let aspect = Double(height) / Double(max(width, 1))
        let w = width >= height ? maxSide : Int((Double(maxSide) / aspect).rounded())
        let h = width >= height ? Int((Double(maxSide) * aspect).rounded()) : maxSide
        return URL(string: "https://picsum.photos/id/\(id)/\(w)/\(h)")
    }
}
```

Разберём по частям.

`nonisolated struct`. В режиме «всё на main actor» структура без пометки тоже считается привязанной к главному потоку, и её соответствие `Decodable` и `Hashable` — тоже. Пока мы декодируем её на главном потоке, это не мешает. Но `Hashable`-модели часто уходят в API, которые требуют «можно пользоваться из любого потока» (например, идентификаторы diffable data source), а декодирование иногда хочется увести в фон. `nonisolated` снимает привязку: структура из одних `let`-констант ни от какого потока не зависит, и это честно. `Sendable` подтверждает то же самое для компилятора: значение можно безопасно передавать между потоками.

`CodingKeys`. В JSON ключ называется `download_url` (через подчёркивание — стиль snake_case), а в Swift принято `downloadURL` (camelCase). Сопоставление ключей в `Codable` точное и регистрозависимое, поэтому без подсказки декодер искал бы в JSON ключ `downloadURL` и не нашёл бы. `enum CodingKeys` говорит: свойство `downloadURL` читай из ключа `download_url`. Остальные ключи совпадают с именами свойств и перечислены просто так. Другой путь — `decoder.keyDecodingStrategy = .convertFromSnakeCase`, тогда декодер сам превратит `download_url` в `downloadUrl`. Обрати внимание: именно в `downloadUrl` с маленькими `rl`, а не `downloadURL`. Из-за таких сюрпризов явные `CodingKeys` надёжнее.

`downloadURL` — оригинал, 5000×3333 пикселей и несколько мегабайт. Мы никогда не грузим его в сетку. Для превью есть `thumbnailURL(side:)` — квадрат нужной стороны. Какой именно стороны, посчитаем в разделе 16.7.

`fullURL(maxSide:)` — картинка для полноэкранного просмотра. Если попросить у picsum квадрат 1200×1200, сервис **обрежет** фото до квадрата, и в просмотре пропадут края. Поэтому сохраняем пропорции оригинала. На числах для первой записи: ширина 5000, высота 3333, значит высота составляет 3333 / 5000 ≈ 0,67 ширины (это `aspect`). Фото горизонтальное (ширина больше высоты), поэтому длинную сторону делаем 1200, а короткую — 1200 × 0,67 ≈ 800. Получаем 1200×800 — та же форма, что у оригинала. Для вертикального фото наоборот: высота 1200, ширина 1200 / aspect. `max(width, 1)` защищает от деления на ноль, если сервер вдруг пришлёт ширину 0.

16.2 API — пагинация

Список фото сервис отдаёт кусками — страницами. Это и есть пагинация (pagination, «постраничная выдача»): вместо тысячи записей за раз просим по 30, а когда пользователь докрутит до конца — ещё 30.

```
import Foundation
```

```
enum GalleryAPI {
    enum APIError: Error {
        case badURL
        case badStatus(Int)
    }

    static let pageSize = 30

    static func fetch(page: Int, limit: Int) async throws -> [PicsumPhoto] {
        var components = URLComponents(string: "https://picsum.photos/v2/list")
        components?.queryItems = [
            URLQueryItem(name: "page", value: String(page)),
            URLQueryItem(name: "limit", value: String(limit)),
        ]
        guard let url = components?.url else { throw APIError.badURL }

        let (data, response) = try await URLSession.shared.data(from: url)
        let status = (response as? HTTPURLResponse)?.statusCode ?? 0
        guard 200..<300 ~= status else { throw APIError.badStatus(status) }
        return try JSONDecoder().decode([PicsumPhoto].self, from: data)
    }
}
```

Что здесь происходит:

- **`URLComponents`** собирает URL из частей и сам экранирует значения параметров. Писать `"...?page=\(page)"` руками работает, пока в значении не окажется пробел или `&`.
- **`URLSession.shared.data(from:)`** — асинхронная загрузка (iOS 15+). Пока ответ идёт по сети, главный поток свободен: `await` не блокирует интерфейс, а отпускает поток и возвращается, когда данные пришли.
- **Статус ответа.** `200..<300 ~= status` читается как «status лежит в диапазоне от 200 до 299». Сервер может ответить 404 или 500 с HTML-страницей ошибки вместо JSON; без этой проверки мы получили бы непонятную ошибку декодирования.
- **`enum APIError`** — свои ошибки с понятными именами. В исходной версии этой главы код бросал `Error.decoding`, но такого типа нигде не было, и листинг не собирался.

Когда страницы кончаются? Мы проверили на живом сервисе: при `limit=30` страницы с 1-й по 33-ю полные, на 34-й всего 3 фото, 35-я возвращает пустой массив `[]`. Отсюда правило: **если пришло меньше, чем просили, — это последняя страница**. Ждать пустой страницы не нужно, это лишний запрос.

16.3 ImageCache — кеш картинок в памяти

Когда пользователь листает сетку вверх-вниз, одни и те же превью появляются снова и снова. Качать их каждый раз заново — трата трафика и батареи. Поэтому скачанные картинки держим в кеше (cache — «заначка», быстрое хранилище того, что уже получили).

```swift
import UIKit

@MainActor
final class ImageCache {
    static let shared = ImageCache()

    private let cache = NSCache<NSURL, UIImage>()
    private var inflight: [URL: Task<UIImage?, Never>] = [:]

    private init() {
        cache.totalCostLimit = 80 * 1024 * 1024   // 80 МБ раскодированных пикселей
    }

    func cachedImage(for url: URL) -> UIImage? {
        cache.object(forKey: url as NSURL)
    }

    func image(for url: URL) async -> UIImage? {
        if let cached = cachedImage(for: url) { return cached }
        if let existing = inflight[url] { return await existing.value }

        let task = Task<UIImage?, Never> {
            guard let (data, _) = try? await URLSession.shared.data(from: url),
                  let image = UIImage(data: data) else { return nil }
            return await image.byPreparingForDisplay() ?? image
        }
        inflight[url] = task
        let image = await task.value
        inflight[url] = nil

        if let image {
            cache.setObject(image, forKey: url as NSURL, cost: image.memoryCost)
        }
        return image
    }
}

private extension UIImage {
    /// Сколько байт занимают раскодированные пиксели.
    var memoryCost: Int {
        guard let cgImage else { return 1 }
        return cgImage.bytesPerRow * cgImage.height
    }
}
```

Разберём по шагам.

`NSCache` — класс Apple для кешей. Внешне он похож на словарь «ключ → значение», но умеет сам выбрасывать содержимое, когда памяти становится мало. Обычный словарь `[URL: UIImage]` держал бы все картинки до конца жизни приложения: пролистал 500 фото — все 500 в памяти, и система в какой-то момент просто завершит приложение.

Ключ — `NSURL`, а не `URL`: `NSCache` пришёл из Objective-C, и оба его типа-параметра должны быть классами (`NSCache<KeyType: AnyObject, ObjectType: AnyObject>`) — структуры туда не подставить. `URL` — структура Swift, поэтому пишем `url as NSURL`: это бесплатное преобразование в «объектный» двойник.

`totalCostLimit` и `cost`. Лимит можно задать двумя способами: числом объектов (`countLimit`) или суммарной «стоимостью» (`totalCostLimit`). Картинки бывают очень разного размера, поэтому считаем стоимость в байтах. Раскодированная картинка в памяти — это таблица пикселей, на каждый пиксель обычно 4 байта (красный, зелёный, синий, прозрачность). Превью 400×400 пикселей: 400 × 400 × 4 = 640 000 байт, около 0,6 МБ. Полноэкранное фото 1200×800: 1200 × 800 × 4 ≈ 3,8 МБ. Обрати внимание: JPEG-файл на диске весит в несколько раз меньше, но в памяти лежит именно распакованная таблица пикселей. При лимите 80 МБ в кеш поместится примерно 130 превью. `bytesPerRow × height` — точное число байт этой таблицы (`bytesPerRow` иногда чуть больше ширины × 4 из-за выравнивания строк). Лимит — не жёсткая граница, а ориентир: документация Apple прямо говорит, что `NSCache` может выбрасывать объекты и раньше, и позже.

`byPreparingForDisplay()` (iOS 15+). `UIImage(data:)` не распаковывает JPEG сразу, это происходит при первом показе — на главном потоке, в момент прокрутки. Тридцать распаковок подряд дают заметное подёргивание сетки. `byPreparingForDisplay()` распаковывает картинку заранее в фоне и возвращает готовую к показу копию. Если распаковать не вышло (вернулся `nil`), отдаём исходную картинку.

`inflight` («в полёте») — словарь уже запущенных загрузок. Сценарий: ячейка попросила превью A, через 50 мс пользователь прокрутил туда- обратно, и другая ячейка просит то же A. Без `inflight` ушло бы два одинаковых запроса. С ним второй вызов находит в словаре задачу первого и ждёт её результат: `await existing.value`. Тот же приём был в `WeatherStore` (глава 15).

Почему класс на `@MainActor`. Словарь `inflight` меняется из разных вызовов, и без защиты два вызова могли бы менять его одновременно. Привязка к главному потоку гарантирует, что код класса выполняется по очереди. В нашем режиме сборки класс и так попадёт на main actor, но явная пометка документирует намерение и оставляет код рабочим, если кто-то соберёт его без этой настройки. Сама загрузка при этом главный поток не держит: на `await` поток свободен.

`Task { ... }` внутри. Задача, созданная из кода на main actor, тоже начинает работу на нём, но всё тяжёлое внутри — сеть и `byPreparingForDisplay()` — асинхронное и выполняется вне главного потока. Задача не привязана к вызывающей ячейке: если ячейка передумает (о отмене — в разделе 16.7), загрузка всё равно доведётся до конца и положит картинку в кеш. Для галереи это полезно: пролистанное фото, скорее всего, понадобится при прокрутке обратно.

`cachedImage(for:)` — синхронная проверка кеша. Она нужна ячейке, чтобы показать уже скачанную картинку сразу, без мигания пустого квадрата (подробнее в 16.7).

16.4 Compositional Layout — адаптивная сетка

`UICollectionView` — вид (view) для коллекции элементов с любой раскладкой: сетка, карусель, список. Сам он не решает, где стоит каждая ячейка; это делает отдельный объект — layout («раскладка»). Раскладка — не view: `UICollectionViewCompositionalLayout → UICollectionViewLayout → NSObject`. У неё нет ни `frame`, ни subviews, она ничего не рисует — только отвечает коллекции на вопрос «где и какого размера элемент с таким индексом», а расставляет ячейки уже сама коллекция (подробно — глава 0, раздел 0.8). `UICollectionViewCompositionalLayout` (iOS 13+) описывает раскладку как матрёшку из трёх уровней:

- **item** — одна ячейка;
- **group** — ряд (или столбец) из нескольких item;
- **section** — секция, в которой группы повторяются, пока не кончатся элементы.

Размеры задаются относительно контейнера: `.fractionalWidth(0.5)` — «половина ширины контейнера», `.absolute(60)` — ровно 60 точек.

Наша сетка: 3 колонки на телефоне, 4 на широком экране:

```
import UIKit

extension GalleryViewController {
    static func columns(forWidth width: CGFloat) -> Int {
        width > 600 ? 4 : 3
    }

    func makeLayout() -> UICollectionViewLayout {
        let inset: CGFloat = 2
        return UICollectionViewCompositionalLayout { _, environment in
            let columns = Self.columns(forWidth:
             ↪  environment.container.effectiveContentSize.width)

            let item = NSCollectionLayoutItem(layoutSize: NSCollectionLayoutSize(
                widthDimension: .fractionalWidth(1.0 / CGFloat(columns)),
                heightDimension: .fractionalHeight(1.0)
            ))
            item.contentInsets = NSDirectionalEdgeInsets(top: inset, leading: inset,
                                                         bottom: inset, trailing: inset)

            let group = NSCollectionLayoutGroup.horizontal(
                layoutSize: NSCollectionLayoutSize(
                    widthDimension: .fractionalWidth(1.0),
                    heightDimension: .fractionalWidth(1.0 / CGFloat(columns))
                ),
                subitems: [item]
            )

            let section = NSCollectionLayoutSection(group: group)
            section.contentInsets = NSDirectionalEdgeInsets(top: 0, leading: inset,
                                                            bottom: 0, trailing: inset)
```

```
            let footer = NSCollectionLayoutBoundarySupplementaryItem(
                layoutSize: NSCollectionLayoutSize(widthDimension: .fractionalWidth(1.0),
                                                   heightDimension: .absolute(60)),
                elementKind: UICollectionView.elementKindSectionFooter,
                alignment: .bottom
            )
            section.boundarySupplementaryItems = [footer]
            return section
        }
    }
}
```

Раскладку мы вынесли в расширение `GalleryViewController` — сам контроллер соберём в разделе 16.11.

1. **Замыкание с `environment`.** Раскладка создаётся не один раз, а каждый раз, когда меняется размер контейнера: поворот телефона, Split View на iPad. `environment.container.effectiveContentSize` — текущий размер. Ширина больше 600 точек (iPad, или iPhone Pro Max в альбомной ориентации) — 4 колонки, иначе 3. Правило вынесено в `columns(forWidth:)`, потому что оно понадобится ещё раз — при расчёте размера превью.
2. **Item.** Ширина `.fractionalWidth(1.0 / columns)` — треть ширины группы при трёх колонках. Высота `.fractionalHeight(1.0)` — вся высота группы.
3. **Group.** Горизонтальная группа на всю ширину секции (`.fractionalWidth(1.0)`). Высота — `.fractionalWidth(1/columns)`: да, высота задана через **ширину**. Это законно и значит «высота ряда равна трети ширины». Раз ширина ячейки тоже треть ширины — ячейки получаются квадратными. `horizontal(layoutSize:subitems:)` кладёт item в ряд столько раз, сколько поместится: три трети — три ячейки.
4. **Section** повторяет группу, пока не кончатся фото, и добавляет снизу footer — дополнительный вид (supplementary view) для индикатора загрузки. Дополнительный вид — это элемент коллекции, который не является ячейкой с данными: заголовок, подвал, значок. Класс у него тот же базовый, что и у ячейки, — `UICollectionReusableView`, и переиспользуется он так же (16.6).

   Частая ошибка. Дать item ширину `.fractionalWidth(1.0)`, то есть 100% ширины группы. Тогда в ряд помещается ровно один такой item, и вместо сетки получается столбец широких прямоугольников. Если нужно, чтобы количество задавала группа, а не item, есть `NSCollectionLayoutGroup.horizontal(layoutSize:repeatingSubitem:count:)`, но он появился только в iOS 16. Для iOS 15 есть старый `horizontal(layoutSize:subitem:count:)`, который в iOS 16 объявлен устаревшим. Дробная ширина item работает везде одинаково.

Отступы на числах. У каждой ячейки `contentInsets` по 2 точки со всех сторон, и у секции ещё 2 точки слева и справа. Между соседними фото получается 2 + 2 = 4 точки, а от края экрана тоже 2 + 2 = 4. Промежутки везде одинаковые. `contentInsets` отнимается от места, которое выделено item, поэтому ячейка становится меньше, а сетка не разъезжается.

`NSDirectionalEdgeInsets` вместо `UIEdgeInsets` — отступы `leading`/`trailing` («от начала»/«до конца» строки) вместо `left`/`right`. Для языков с письмом справа налево (RTL — арабский, иврит) система зеркалит интерфейс, и `leading` сам станет правой стороной.

> **Подсказка.** Раскладку не нужно пересчитывать вручную в `traitCollectionDidChange` или `viewWillTransition`. Compositional layout сам вызовет замыкание снова при повороте или изменении размера окна.

> **Упражнение 16.1.** Маленькие телефоны в режиме «Увеличенный» (Настройки → Экран и яркость → Вид экрана) дают ширину экрана около 320 точек, и три колонки там получаются мелкими. Измени правило так, чтобы при ширине меньше 350 точек было 2 колонки, от 350 до 600 — 3, больше 600 — 4. Решение — в конце главы.

16.5 Пагинация — `willDisplay`

Как узнать, что пользователь докрутил почти до конца? Коллекция сообщает своему делегату (delegate — объект, которому вид «докладывает» о событиях и которого спрашивает о решениях) о каждой ячейке, которая вот-вот появится на экране:

```
import UIKit

extension GalleryViewController: UICollectionViewDelegate {
    func collectionView(_ collectionView: UICollectionView, willDisplay cell:
    ↪ UICollectionViewCell,
                        forItemAt indexPath: IndexPath) {
        if indexPath.item >= photos.count - 6, !lastLoadFailed {
            loadMore()
        }
    }

    func collectionView(_ collectionView: UICollectionView, didSelectItemAt indexPath:
    ↪ IndexPath) {
        openPhoto(at: indexPath.item)
    }
}
```

Когда на экран выходит одна из шести последних ячеек, начинаем грузить очередную страницу. Шесть ячеек — это два ряда при трёх колонках: запас, чтобы страница успела прийти, пока пользователь докручивает. При 30 фото загрузка стартует на ячейке с номером 24 (нумерация с нуля: 30 − 6 = 24).

`!lastLoadFailed` — если прошлая загрузка упала (нет сети), не повторяем её автоматически на каждую ячейку. Иначе без сети мы бы долбили сервер при каждом движении пальца. Повтор — по кнопке в footer'е (16.6).

Сама загрузка:

```
import UIKit

extension GalleryViewController {
```

```
    func loadMore() {
        guard hasMore, !isLoadingPage else { return }
        isLoadingPage = true
        lastLoadFailed = false
        reloadFooter()

        let pageToLoad = nextPage
        Task { [weak self] in
            do {
                let newPhotos = try await GalleryAPI.fetch(page: pageToLoad, limit:
                ↪  GalleryAPI.pageSize)
                self?.appendPage(newPhotos)
            } catch {
                self?.lastLoadFailed = true
            }
            self?.isLoadingPage = false
            self?.reloadFooter()
        }
    }

    func appendPage(_ newPhotos: [PicsumPhoto]) {
        let start = photos.count
        photos.append(contentsOf: newPhotos)
        nextPage += 1
        if newPhotos.count < GalleryAPI.pageSize { hasMore = false }

        let indexPaths = (start..<photos.count).map { IndexPath(item: $0, section: 0) }
        collectionView.performBatchUpdates {
            collectionView.insertItems(at: indexPaths)
        }
    }
}
```

Четыре переменных состояния:

- `hasMore` — есть ли ещё страницы. Станет `false`, когда придёт неполная страница (меньше 30 фото), как мы выяснили в 16.2.
- `isLoadingPage` — идёт ли сейчас загрузка. `willDisplay` вызывается на каждую появившуюся ячейку, и без этого флага одна прокрутка запустила бы шесть одинаковых запросов.
- `nextPage` — номер страницы, которую грузить дальше.
- `lastLoadFailed` — упала ли последняя попытка.

`[weak self]` в задаче — контроллер держится задачей слабо. Если пользователь закрыл галерею, пока страница грузится, контроллер спокойно освободится, а `self?.` превратит оставшиеся строки в «ничего не делать».

`insertItems` вместо `reloadData()`. `reloadData()` выбрасывает и заново строит все видимые ячейки: они мигают, а ячейки, которые уже грузили превью, начинают сначала. `insertItems(at:)` внутри `performBatchUpdates` говорит коллекции ровно то, что случилось: «в конец добавлено 30

элементов, с номера 30 по 59». Видимые ячейки не трогаются, новые появляются с анимацией. Правило одно: к моменту вызова `insertItems` массив `photos` уже должен содержать новые элементы, иначе коллекция упадёт с ошибкой «количество элементов не сходится».

16.6 Footer-loader

Снизу секции — подвал (footer) с тремя состояниями: «грузим» со спиннером, «Это всё» и «Не загрузилось» с кнопкой повтора.

```
import UIKit

final class FooterLoaderView: UICollectionReusableView {
    static let reuseID = "FooterLoaderView"

    enum Mode {
        case loading
        case finished
        case failed
    }

    var onRetry: (() -> Void)?

    private let spinner = UIActivityIndicatorView(style: .medium)
    private let label = UILabel()
    private let retryButton = UIButton(type: .system)

    override init(frame: CGRect) {
        super.init(frame: frame)
        label.font = .preferredFont(forTextStyle: .footnote)
        label.textColor = .secondaryLabel
        label.adjustsFontForContentSizeCategory = true
        retryButton.setTitle("Повторить", for: .normal)
        retryButton.addAction(UIAction { [weak self] _ in self?.onRetry?() }, for:
↪   .touchUpInside)

        let stack = UIStackView(arrangedSubviews: [spinner, label, retryButton])
        stack.spacing = 8
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: centerYAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }
```

```
    func apply(_ mode: Mode) {
        switch mode {
        case .loading:
            spinner.startAnimating()
            label.text = "Загружаем ещё…"
            retryButton.isHidden = true
        case .finished:
            spinner.stopAnimating()
            label.text = "Это всё"
            retryButton.isHidden = true
        case .failed:
            spinner.stopAnimating()
            label.text = "Не загрузилось."
            retryButton.isHidden = false
        }
    }
}
```

`UICollectionReusableView` — базовый класс для всего, что коллекция переиспользует. Лестница такая: `UICollectionViewCell → UICollectionReusableView → UIView`. Базовый класс добавляет к обычному view только то, что нужно для очереди переиспользования: `reuseIdentifier`, `prepareForReuse()` и приём атрибутов раскладки. Ячейка сверху добавляет своё: `contentView`, `backgroundView`, `selectedBackgroundView`, состояния `isSelected` и `isHighlighted`. У footer'а ничего этого нет — ни `contentView`, ни выделения (проверено рантаймом: `UICollectionReusableView` не отвечает на `contentView`), поэтому стек мы добавляем прямо в сам footer (`addSubview(stack)`), а в ячейке в 16.7 — в `contentView`. Полная карта семейства — глава 0, раздел 0.5.

Footer тоже берётся из очереди переиспользования, поэтому `apply(_:)` выставляет **все** свойства в каждом режиме: текст, спиннер и видимость кнопки. Если бы режим `.finished` не прятал кнопку, footer, который раньше показывал ошибку, остался бы с лишней кнопкой.

`UIActivityIndicatorView` по умолчанию прячется, когда не крутится (`hidesWhenStopped = true`), поэтому в режиме «Это всё» спиннер исчезает сам.

`label.font = .preferredFont(forTextStyle: .footnote)` вместе с `adjustsFontForContentSizeCategory = true` — поддержка Dynamic Type: пользователь увеличил шрифт в настройках системы, и подпись выросла без перезапуска.

`[weak self]` в `UIAction` у кнопки: кнопка держит action, action держит замыкание. Если замыкание сильно держит footer, а footer держит кнопку, получается кольцо, и никто из них не освободится.

Контроллер выбирает режим и обновляет footer:

```
import UIKit

extension GalleryViewController {
    func footerMode() -> FooterLoaderView.Mode {
        if lastLoadFailed { return .failed }
        return hasMore ? .loading : .finished
```

```
    }

    func reloadFooter() {
        let kind = UICollectionView.elementKindSectionFooter
        for case let footer as FooterLoaderView in
        ↪ collectionView.visibleSupplementaryViews(ofKind: kind) {
            footer.apply(footerMode())
        }
    }
}
```

`visibleSupplementaryViews(ofKind:)` отдаёт подвалы, которые сейчас на экране. Если footer не виден — обновлять нечего, при появлении он получит правильный режим в `viewForSupplementaryElementOfKind` (16.11). `for case let footer as FooterLoaderView in` — цикл только по элементам нужного типа, без принудительного приведения `as!`.

16.7 GalleryCell — асинхронная загрузка картинки

Ячейка — самое тонкое место галереи из-за переиспользования. На экране iPhone одновременно видно около 20 ячеек, а фото — тысяча. Коллекция не создаёт тысячу ячеек: те, что уехали за край экрана, складываются в очередь и снова выдаются через `dequeueReusableCell(withReuseIdentifier:for:)` для других фото. Как тарелки в кафе: помыли, поставили новое блюдо. Отсюда главное правило: всё, что ячейка показывала для прошлого фото, при повторном использовании надо сбросить.

```
import UIKit

final class GalleryCell: UICollectionViewCell {
    static let reuseID = "GalleryCell"

    private let imageView = UIImageView()
    private var currentURL: URL?
    private var loadTask: Task<Void, Never>?

    override init(frame: CGRect) {
        super.init(frame: frame)
        contentView.backgroundColor = .secondarySystemBackground
        contentView.layer.cornerRadius = 6
        contentView.clipsToBounds = true

        imageView.contentMode = .scaleAspectFill
        imageView.clipsToBounds = true
        imageView.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(imageView)
        NSLayoutConstraint.activate([
            imageView.topAnchor.constraint(equalTo: contentView.topAnchor),
            imageView.bottomAnchor.constraint(equalTo: contentView.bottomAnchor),
```

```
            imageView.leadingAnchor.constraint(equalTo: contentView.leadingAnchor),
            imageView.trailingAnchor.constraint(equalTo: contentView.trailingAnchor),
        ])
        isAccessibilityElement = true
        accessibilityTraits = .image
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func prepareForReuse() {
        super.prepareForReuse()
        loadTask?.cancel()
        loadTask = nil
        currentURL = nil
        imageView.image = nil
    }

    func configure(with photo: PicsumPhoto, pixelSide: Int) {
        accessibilityLabel = "Фото, автор \(photo.author)"
        guard let url = photo.thumbnailURL(side: pixelSide) else { return }
        currentURL = url

        if let cached = ImageCache.shared.cachedImage(for: url) {
            imageView.image = cached
            return
        }

        loadTask = Task { [weak self] in
            let image = await ImageCache.shared.image(for: url)
            guard let self, !Task.isCancelled, self.currentURL == url else { return }
            self.imageView.alpha = 0
            self.imageView.image = image
            UIView.animate(withDuration: 0.2) { self.imageView.alpha = 1 }
        }
    }
}
```

Заглушка вместо картинки. Пока превью не пришло, виден фон `contentView` цвета `.secondarySystemBackground` — светло-серый в светлой теме и тёмно-серый в тёмной. Это самая простая заглушка (placeholder — «место под будущее содержимое»). Хочешь бегущий отблеск (shimmer, «скелетон») — возьми `SkeletonView` из раздела 15.8 главы 15 и положи его под `imageView`.

`contentMode = .scaleAspectFill` — картинка заполняет квадрат целиком, лишнее обрезается (`clipsToBounds`). Превью у нас и так квадратные, но режим спасает, если сервер вернёт картинку другой формы.

`contentView`. Свои subviews ячейка коллекции держит не в себе, а в `contentView` — это документированное место для содержимого: ячейка кладёт его поверх `backgroundView` и

`selectedBackgroundView`, и у свежей ячейки `contentView` — единственный subview (проверено рантаймом). Добавишь картинку прямо в ячейку (`addSubview` на `self`) — она окажется под фоновыми view и не будет участвовать в анимациях выделения.

`prepareForReuse()` — коллекция вызывает этот метод перед тем, как `dequeueReusableCell` вернёт ячейку для другого фото (метод объявлен в `UICollectionReusableView`, поэтому есть и у footer'а). Здесь сбрасываем всё, что относится к прошлому фото: отменяем загрузку, забываем URL, убираем картинку. Без `imageView.image = nil` пользователь на долю секунды видел бы в ячейке чужое фото, пока грузится новое. Apple просит не назначать здесь новые данные — это работа источника данных в `cellForItemAt`; сброс в значения по умолчанию, как у нас, — ровно то, для чего метод предназначен. И обязательно вызывать `super.prepareForReuse()`: метод помечен `NS_REQUIRES_SUPER`.

Проверка `currentURL == url` — защита от гонки (race condition — ситуация, когда результат зависит от того, какая из двух операций закончится раньше). Сценарий:

1. Ячейка показывает фото A и начинает грузить картинку A.
2. Пользователь прокрутил, ячейка ушла в очередь и вернулась с фото B. Начинается загрузка B.
3. Картинка A приходит позже B (сеть непредсказуема).
4. Без проверки мы поставили бы A в ячейку, которая показывает B.

Сравнение URL гарантирует: картинка ставится, только если ячейка всё ещё ждёт именно её.

Зачем ещё и `loadTask?.cancel()`. Отмена задачи ячейки сообщает «результат мне больше не нужен», и `Task.isCancelled` после `await` вернёт `true`. Проверка URL закрывает ту же дыру, отмена — второй замок: например, если то же фото вернётся в ту же ячейку, URL совпадёт, а старая задача всё равно не должна ничего делать. Сама загрузка в `ImageCache` при этом **не** прерывается: ячейка ждёт через `await task.value`, а отмена ждущей задачи не отменяет ту, которую она ждёт. Мы сознательно так оставили: картинку могут ждать сразу две ячейки, и отмена одной не должна сорвать загрузку для другой, а скачанное попадёт в кеш. Если нужно рвать запросы при быстрой прокрутке, `ImageCache` придётся считать, сколько ячеек ждёт каждую загрузку, и отменять её, когда ждущих не осталось.

Сначала синхронный кеш. Если картинка уже в кеше, `cachedImage(for:)` отдаёт её сразу, в том же проходе раскладки. Без этой проверки даже закешированное фото показывалось бы через задачу — на один кадр позже, и при прокрутке назад ячейки мигали бы пустым фоном.

Плавное появление. `alpha = 0` и анимация до 1 за 0,2 секунды — картинка «проявляется» вместо резкой смены. Анимация делается только для пришедших из сети картинок; из кеша — сразу.

Сколько пикселей просить. Параметр `pixelSide` считает контроллер:

```
import UIKit

extension GalleryViewController {
    /// Сторона превью в пикселях: ширина ячейки в точках × масштаб экрана,
    /// округлённая вверх до сотни.
    var thumbnailPixelSide: Int {
        let width = collectionView.bounds.width
        let columns = CGFloat(Self.columns(forWidth: width))
```

```
        let points = width / columns
        let pixels = points * traitCollection.displayScale
        return Int((pixels / 100).rounded(.up)) * 100
    }
}
```

UIKit меряет всё в **точках** (points), а экран состоит из **пикселей**. На экранах @2x в одной точке 2×2 пикселя, на @3x — 3×3. `traitCollection.displayScale` — этот коэффициент для текущего экрана. Посчитаем для iPhone 16: ширина 393 точки, три колонки — примерно 131 точка на ячейку. Экран @3x: 131 × 3 = 393 пикселя. Округляем вверх до сотни — просим 400×400. Если попросить меньше (например, 300), картинку растянут до 393 пикселей, и она станет мыльной; если больше — лишний трафик и память.

Округление до сотни нужно ради кеша и сервера: у соседних устройств (393, 390, 402 точки ширины) получится один и тот же размер 400 и один и тот же URL. Так кеш срабатывает чаще, а сервер отдаёт уже подготовленные размеры. На iPad с 4 колонками и экраном @2x выйдет около 1024 / 4 × 2 = 512 пикселей, округляем до 600.

> **Упражнение 16.2.** Сделай так, чтобы превью начинали грузиться ещё до того, как ячейка появится на экране. У коллекции есть протокол `UICollectionViewDataSourcePrefetching`: метод `collectionView(_:prefetchItemsAt:)` сообщает номера элементов, которые скоро понадобятся. Подключи его к `GalleryViewController` и прогрей в нём `ImageCache`. Проверка: после этого при медленной прокрутке новые ряды появляются уже с картинками, без серых квадратов. Решение — в конце главы.

16.8 PhotoViewerViewController — pinch-to-zoom

При тапе на ячейку открываем полноэкранный просмотр. Растягивание двумя пальцами (pinch-to-zoom, «щипок») умеет сам `UIScrollView` — его нужно только правильно настроить.

```
import UIKit

final class PhotoViewerViewController: UIViewController {
    private let photo: PicsumPhoto
    private let scrollView = UIScrollView()
    private let imageView = UIImageView()
    private let spinner = UIActivityIndicatorView(style: .large)
    private var loadTask: Task<Void, Never>?

    init(photo: PicsumPhoto) {
        self.photo = photo
        super.init(nibName: nil, bundle: nil)
        modalPresentationStyle = .fullScreen
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
```

```
        super.viewDidLoad()
        view.backgroundColor = .black
        setupLayout()
        loadImage()
    }

    override func viewDidLayoutSubviews() {
        super.viewDidLayoutSubviews()
        // Пока фото не приближено — оно ровно в размер экрана.
        if scrollView.zoomScale == scrollView.minimumZoomScale {
            imageView.frame = scrollView.bounds
            scrollView.contentSize = scrollView.bounds.size
        }
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        if isBeingDismissed { loadTask?.cancel() }
    }

    private func setupLayout() {
        scrollView.frame = view.bounds
        scrollView.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        scrollView.minimumZoomScale = 1.0
        scrollView.maximumZoomScale = 4.0
        scrollView.bouncesZoom = true
        scrollView.showsVerticalScrollIndicator = false
        scrollView.showsHorizontalScrollIndicator = false
        scrollView.contentInsetAdjustmentBehavior = .never
        scrollView.delegate = self
        view.addSubview(scrollView)

        imageView.contentMode = .scaleAspectFit
        imageView.isAccessibilityElement = true
        imageView.accessibilityTraits = .image
        imageView.accessibilityLabel = "Фото, автор \(photo.author)"
        scrollView.addSubview(imageView)

        spinner.color = .white
        spinner.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(spinner)

        let closeButton = UIButton(type: .close)
        closeButton.overrideUserInterfaceStyle = .dark
        closeButton.accessibilityLabel = "Закрыть"
        closeButton.addAction(UIAction { [weak self] _ in self?.dismiss(animated: true) },
                              for: .touchUpInside)
        closeButton.translatesAutoresizingMaskIntoConstraints = false
```

```
        view.addSubview(closeButton)

        NSLayoutConstraint.activate([
            spinner.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            spinner.centerYAnchor.constraint(equalTo: view.centerYAnchor),
            closeButton.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪   constant: 8),
            closeButton.trailingAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.trailingAnchor,
                                                  constant: -16),
        ])

        let doubleTap = UITapGestureRecognizer(target: self, action:
        ↪  #selector(doubleTapped(_:)))
        doubleTap.numberOfTapsRequired = 2
        scrollView.addGestureRecognizer(doubleTap)
    }

    private func loadImage() {
        guard let url = photo.fullURL() else { return }
        spinner.startAnimating()
        loadTask = Task { [weak self] in
            let image = await ImageCache.shared.image(for: url)
            guard let self else { return }
            self.spinner.stopAnimating()
            self.imageView.image = image
        }
    }

    @objc private func doubleTapped(_ sender: UITapGestureRecognizer) {
        if scrollView.zoomScale > scrollView.minimumZoomScale {
            scrollView.setZoomScale(scrollView.minimumZoomScale, animated: true)
        } else {
            let point = sender.location(in: imageView)
            let zoom: CGFloat = 2.5
            let size = CGSize(width: scrollView.bounds.width / zoom,
                              height: scrollView.bounds.height / zoom)
            let rect = CGRect(x: point.x - size.width / 2,
                              y: point.y - size.height / 2,
                              width: size.width,
                              height: size.height)
            scrollView.zoom(to: rect, animated: true)
        }
    }
}

extension PhotoViewerViewController: UIScrollViewDelegate {
    func viewForZooming(in scrollView: UIScrollView) -> UIView? { imageView }
```

```
}
```

`UIScrollView` поддерживает щипок сам, если выполнены три условия:

1. `minimumZoomScale` меньше `maximumZoomScale`. У нас от 1 (обычный размер) до 4 (в четыре раза крупнее).
2. Делегат реализует `viewForZooming(in:)` и возвращает вид, который надо масштабировать.
3. Этот вид лежит **внутри** scroll view.

Почему кадры, а не Auto Layout. Во время зума scroll view не меняет `bounds` у `imageView` — он ставит ему преобразование масштаба (`transform`) и сам пересчитывает `contentSize`. Мы проверили на симуляторе: после `zoomScale = 2` у `imageView` `transform` стал `(a: 2, d: 2)`, `bounds` остались 393×852, а `contentSize` scroll view удвоился до 786×1704. А у view с `transform`, отличным от identity, значение `frame`, по документации Apple, **не определено** — его нельзя ни читать, ни присваивать. Поэтому `imageView` расставлен кадром, но только в одном случае: в `viewDidLayoutSubviews` мы трогаем `frame` лишь пока `zoomScale == minimumZoomScale`, то есть пока `transform` единичный и `frame` имеет смысл. Если пользователь приблизил фото и повернул телефон, кадр не трогаем — и потому что он не определён, и потому что иначе зум сбросится рывком. Ограничениями Auto Layout такой экран собрать тоже можно (через `contentLayoutGuide` scroll view), но для одной картинки на весь экран кадр короче. `contentMode = .scaleAspectFit` вписывает фото в экран целиком, с чёрными полосами по краям.

`scrollView.autoresizingMask = [.flexibleWidth, .flexibleHeight]` — scroll view растягивается вместе с `view` при повороте. Это старый механизм «пружинок» (autoresizing): маска говорит, какие стороны и размеры меняются вслед за superview. Он важен ещё по одной причине: в `viewDidLoad` размеры `view` ещё не окончательные, и `scrollView.frame = view.bounds` там — только стартовое значение; окончательный размер scroll view получит от маски, когда `view` примет свой настоящий размер. Как маска устроена и почему её нельзя смешивать с ограничениями на одном view — в главе 19, раздел 19.12.

`contentInsetAdjustmentBehavior = .never` — не сдвигать содержимое под безопасную зону (safe area — часть экрана, не закрытая чёлкой, островом и полоской «домой»). Фото должно занимать весь экран, до краёв.

`UIButton(type: .close)` — системная круглая кнопка с крестиком. `overrideUserInterfaceStyle = .dark` заставляет её рисоваться в тёмном стиле: на чёрном фоне кнопка светлой темы была бы почти не видна.

Двойной тап переключает «приблизить / вернуть». Если уже приближено — возвращаемся к масштабу 1. Если нет — приближаем **к точке тапа**: пользователь хочет рассмотреть именно то место, куда ткнул.

`zoom(to:animated:)` принимает прямоугольник в координатах `imageView` и масштабирует так, чтобы этот прямоугольник заполнил экран. Размер прямоугольника и задаёт силу зума: экран 393×852 точки, делим на 2,5 — прямоугольник 157×341 точка. Чтобы 157 точек заняли 393, их нужно растянуть в 2,5 раза — это и есть итоговый масштаб. Центр прямоугольника ставим в точку тапа: отнимаем половину ширины и половину высоты от координат точки.

`sender.location(in: imageView)` — координаты тапа в системе `imageView`. Масштаб scroll view применяет к `imageView` через преобразование, поэтому координаты внутри него остаются «нерастянутыми» — ровно такими, каких ждёт `zoom(to:)`.

Загрузка. Полное фото тоже идёт через `ImageCache`, так что при повторном открытии оно появится мгновенно. `viewDidDisappear` с проверкой `isBeingDismissed` отменяет ожидание, если просмотр закрыли до конца загрузки.

`modalPresentationStyle = .fullScreen` задан прямо в `init`: этот контроллер всегда открывается на весь экран, и вызывающему коду не нужно об этом помнить.

16.9 Context menu — долгое нажатие

```
import UIKit

extension GalleryViewController {
    func collectionView(_ collectionView: UICollectionView,
                        contextMenuConfigurationForItemAt indexPath: IndexPath,
                        point: CGPoint) -> UIContextMenuConfiguration? {
        let photo = photos[indexPath.item]
        return UIContextMenuConfiguration(identifier: nil, previewProvider: nil) { [weak
        ↪ self] _ in
            let open = UIAction(title: "Открыть",
                                image: UIImage(systemName:
↪ "arrow.up.left.and.arrow.down.right")) { _ in
                self?.openPhoto(at: indexPath.item)
            }
            let save = UIAction(title: "Сохранить в Фото",
                                image: UIImage(systemName: "square.and.arrow.down")) { _ in
                self?.savePhoto(photo)
            }
            let author = UIAction(title: "Автор: \(photo.author)",
                                  image: UIImage(systemName: "person.fill"),
                                  attributes: .disabled) { _ in }
            return UIMenu(children: [open, save, author])
        }
    }
}
```

`contextMenuConfigurationForItemAt` (iOS 13+) — метод делегата коллекции: на долгое нажатие по ячейке система поднимает ячейку над размытым фоном и показывает меню. Хорошее место для дополнительных действий, которым не хватает места в основном интерфейсе.

`previewProvider: nil` — в качестве превью система берёт саму ячейку. Можно вернуть свой контроллер (например, фото крупнее), но для сетки хватает ячейки.

Замыкание `actionProvider` вызывается при каждом показе меню; `[weak self]` в нём — чтобы меню не удерживало контроллер.

Пункт «Автор» — информационный, нажимать его незачем. `attributes: .disabled` рисует его серым и делает неактивным. Если оставить пункту пустой обработчик без `.disabled`, он выглядит как кнопка и закрывает меню по нажатию, ничего не сделав.

`openPhoto(at:)` и `savePhoto(_:)` — методы контроллера, их соберём в разделах 16.10 и 16.11.

16.10 «Сохранить в Фото» — какое разрешение нужно в действительности

Работа с медиатекой — место, где легко запросить лишнее. Apple даёт разные уровни доступа, и за каждым стоит свой системный диалог.

Что нужно приложению	Инструмент	Разрешение
Пользователь выбирает фото из медиатеки	`PHPickerViewController` (iOS 14+)	**Не нужно.** Выбор идёт в отдельном системном процессе, приложение получает только выбранные фото
Только добавить фото в медиатеку	`PHPhotoLibrary` с уровнем `.addOnly` (iOS 14+)	Ключ `NSPhotoLibraryAddUsageDescription` в Info.plist + диалог «разрешить добавлять фото»
Читать всю медиатеку (свою сетку, как в «Фото»)	`PHPhotoLibrary` с уровнем `.readWrite`	Ключ `NSPhotoLibraryUsageDescription` + диалог с вариантами «полный доступ» / «ограниченный»

Галерее нужно только **добавить** фото — значит, минимальный уровень `.addOnly`. Просить полный доступ ради сохранения одной картинки — лишний пугающий диалог, и пользователь чаще откажет. Выбор фото через `PHPickerViewController` мы сделаем в главе 19 (аватар в профиле).

Info.plist — файл настроек приложения, который система читает до запуска: имя, версия, разрешения и тексты к ним. Без строки `NSPhotoLibraryAddUsageDescription` приложение при запросе доступа **аварийно завершится**. В Xcode 26 ключ добавляется на вкладке **Info** цели приложения, в списке он называется «Privacy - Photo Library Additions Usage Description». Текст пиши о пользе для пользователя, он будет в системном диалоге: «Чтобы сохранить понравившееся фото в твою медиатеку».

```
import Photos
import UIKit

enum PhotoSaver {
    enum SaveError: Error {
        case notAllowed
    }

    static func save(_ image: UIImage) async throws {
        let status = await PHPhotoLibrary.requestAuthorization(for: .addOnly)
        guard status == .authorized else { throw SaveError.notAllowed }
        try await PHPhotoLibrary.shared().performChanges {
            PHAssetChangeRequest.creationRequestForAsset(from: image)
```

```
        }
    }
}
```

- `requestAuthorization(for: .addOnly)` при первом вызове показывает системный диалог, а при повторных сразу возвращает сохранённый ответ. Если пользователь раньше отказал, диалога больше не будет — только переход в Настройки.
- Для уровня `.addOnly` бывает только «разрешено» (`.authorized`) или «нет». Статус `.limited` («доступ к выбранным фото») относится к чтению медиатеки, поэтому здесь его не проверяем.
- `performChanges` — все изменения медиатеки делаются внутри такого блока: система выполняет их одной операцией. Асинхронная версия с `try await` доступна с iOS 15.
- `PHAssetChangeRequest.creationRequestForAsset(from:)` — «создай в медиатеке новый снимок из этой картинки».

Простой альтернативный путь — функция `UIImageWriteToSavedPhotosAlbum` из UIKit. Ей тоже нужен ключ `NSPhotoLibraryAddUsageDescription`, но результат приходит через селектор Objective-C, что с async-кодом неудобно.

Метод контроллера, который вызывает меню:

```
import UIKit

extension GalleryViewController {
    func savePhoto(_ photo: PicsumPhoto) {
        guard let url = photo.fullURL() else { return }
        Task { [weak self] in
            let message: String
            if let image = await ImageCache.shared.image(for: url) {
                do {
                    try await PhotoSaver.save(image)
                    message = "Фото сохранено в медиатеку."
                } catch PhotoSaver.SaveError.notAllowed {
                    message = "Нет доступа. Разреши добавление фото: Настройки →
↪  Конфиденциальность → Фото."
                } catch {
                    message = "Не удалось сохранить: \(error.localizedDescription)"
                }
            } else {
                message = "Не удалось скачать фото."
            }
            let alert = UIAlertController(title: nil, message: message, preferredStyle:
            ↪  .alert)
            alert.addAction(UIAlertAction(title: "OK", style: .default))
            self?.present(alert, animated: true)
        }
    }
}
```

Сохраняем полноразмерную версию (1200 точек по длинной стороне), а не превью: в медиатеке пользователь захочет нормальное фото. Если оно уже открывалось в просмотре, оно в кеше и сохранится сразу.

`catch PhotoSaver.SaveError.notAllowed` — отдельная ветка для отказа: пользователю нужно не «что-то пошло не так», а подсказка, где включить доступ.

16.11 Собираем GalleryViewController

Осталось само «тело» контроллера: хранимые свойства, создание коллекции и источник данных. Источник данных (data source) — объект, который отвечает коллекции на вопросы «сколько элементов?» и «какую ячейку показать под номером N?».

```
import UIKit

final class GalleryViewController: UIViewController {
    var photos: [PicsumPhoto] = []
    var nextPage = 1
    var hasMore = true
    var isLoadingPage = false
    var lastLoadFailed = false

    lazy var collectionView = UICollectionView(frame: .zero, collectionViewLayout:
     ↪  makeLayout())

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Галерея"
        view.backgroundColor = .systemBackground

        collectionView.backgroundColor = .systemBackground
        collectionView.dataSource = self
        collectionView.delegate = self
        collectionView.register(GalleryCell.self, forCellWithReuseIdentifier:
↪   GalleryCell.reuseID)
        collectionView.register(FooterLoaderView.self,
                                forSupplementaryViewOfKind:
↪   UICollectionView.elementKindSectionFooter,
                                withReuseIdentifier: FooterLoaderView.reuseID)
        collectionView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(collectionView)
        NSLayoutConstraint.activate([
            collectionView.topAnchor.constraint(equalTo: view.topAnchor),
            collectionView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            collectionView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            collectionView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])
```

```
        loadMore()
    }

    func openPhoto(at index: Int) {
        present(PhotoViewerViewController(photo: photos[index]), animated: true)
    }
}

extension GalleryViewController: UICollectionViewDataSource {
    func collectionView(_ collectionView: UICollectionView, numberOfItemsInSection
    ↪ section: Int) -> Int {
        photos.count
    }

    func collectionView(_ collectionView: UICollectionView,
                        cellForItemAt indexPath: IndexPath) -> UICollectionViewCell {
        let cell = collectionView.dequeueReusableCell(withReuseIdentifier:
        ↪ GalleryCell.reuseID,
                                                      for: indexPath)
        (cell as? GalleryCell)?.configure(with: photos[indexPath.item], pixelSide:
        ↪ thumbnailPixelSide)
        return cell
    }

    func collectionView(_ collectionView: UICollectionView,
                        viewForSupplementaryElementOfKind kind: String,
                        at indexPath: IndexPath) -> UICollectionReusableView {
        let view = collectionView.dequeueReusableSupplementaryView(
            ofKind: kind, withReuseIdentifier: FooterLoaderView.reuseID, for: indexPath
        )
        if let footer = view as? FooterLoaderView {
            footer.apply(footerMode())
            footer.onRetry = { [weak self] in self?.loadMore() }
        }
        return view
    }
}
```

Что здесь стоит разобрать:

- **Свойства без `private`.** Раскладка, пагинация и меню живут в расширениях в других файлах, а `private` в Swift виден только в пределах файла. Поэтому состояние открыто на уровне модуля (`internal`, по умолчанию). Если держишь весь контроллер в одном файле — можешь сделать их `private`.
- **`lazy var collectionView`** — коллекция создаётся при первом обращении. Нужно потому, что в инициализаторе свойства вызывается метод `makeLayout()`, а методы `self` до окончания инициализации звать нельзя. `lazy` откладывает создание до `viewDidLoad`.

- **`register`** — говорим коллекции, какой класс создавать для идентификатора `GalleryCell.reuseID`. Дальше `dequeueReusableCell` либо достаёт готовую ячейку из очереди, либо создаёт новую этого класса.
- **`(cell as? GalleryCell)?.configure`** вместо `as! GalleryCell`. Принудительное приведение уронит приложение, если однажды перепутаешь идентификатор. Мягкое приведение в худшем случае покажет пустую ячейку — ошибку видно, но приложение живо.
- **`dataSource` и `delegate` — слабые ссылки.** Оба свойства объявлены `weak` (так в заголовке `UICollectionView.h`): коллекция не держит контроллер, а контроллер держит коллекцию — кольца нет. Обратная сторона: если источником данных сделать отдельный объект и никуда его не сохранить, он освободится сразу, и `dataSource` станет `nil` (почему делегаты в UIKit всегда weak — глава 0, раздел 0.9).
- **Коллекция до верхнего края `view`.** Коллекция — это scroll view (`UICollectionView → UIScrollView → UIView`, глава 0, раздел 0.5), а у scroll view по умолчанию `contentInsetAdjustmentBehavior = .automatic`: система сама отступит сверху под навигационную панель и снизу под полоску «домой». Прокрученные фото красиво уезжают под полупрозрачную панель.
- **`onRetry`** в footer'е — `[weak self]`: footer живёт в коллекции, коллекция — в контроллере; сильная ссылка из footer'а на контроллер замкнула бы кольцо.

16.12 Бытовая аналогия

Галерея — это **бесконечная стена с открытками** в сувенирной лавке. Подходишь — видишь первые ряды. Идёшь дальше — продавец выносит ещё коробку открыток и развешивает её (пагинация). Раскладка подстраивается под стену: на широкой четыре колонки, на узкой три.

Рамок на стене меньше, чем открыток. Ты отошёл от рамки — в неё вставили другую открытку (переиспользование ячейки). Если ты заказал открытку для рамки, а её принесли, когда в рамке уже другая, — ставить не надо (проверка `currentURL == url`).

Лупа (щипок) увеличивает выбранную открытку. Двигает открытку не она сама, а **стекло над ней** — `UIScrollView`.

16.13 Что мы пропустили

- **Закрытие жестом вниз** — потянуть фото пальцем вниз, чтобы закрыть просмотр. Стандартный жест полноэкранных просмотрщиков, разобран в главе 37 (cookbook, photo viewer).
- **Анимация «из ячейки»** при открытии — миниатюра вырастает до полного экрана. Делается через `UIViewControllerAnimatedTransitioning`.
- **Diffable data source** — `UICollectionViewDiffableDataSource` сам вычисляет, что добавилось и удалилось, и анимирует изменения. Вместо ручного `insertItems` передаёшь ему «снимок» (snapshot) — полный список идентификаторов, который должен быть на экране.
- **Дисковый кеш.** `NSCache` живёт только в памяти, после перезапуска всё качается заново. `URLSession.shared` использует общий `URLCache` с небольшим объёмом; для настоящей галереи заводят свою `URLSession` с большим `URLCache` или пишут файлы в папку Caches.
- **Отмена сетевых загрузок** при быстрой прокрутке — подсчёт ждущих ячеек в `ImageCache` (см. 16.7).

Упражнение 16.3. Открой Галерею (розовая ячейка в лаунчере). Проверь по порядку: (1) при прокрутке вниз под сеткой ненадолго появляется «Загружаем ещё…» и приходят новые ряды; (2) тап открывает фото целиком, с сохранёнными пропорциями; (3) щипок приближает (в симуляторе — зажми Option и тяни мышью), двойной тап приближает к точке тапа, второй двойной тап возвращает; (4) долгое нажатие на ячейку показывает меню, пункт «Автор» серый; (5) «Сохранить в Фото» при первом вызове показывает системный запрос с твоим текстом из Info.plist. Затем выключи сеть на Mac, пролистай до конца загруженного — footer покажет «Не загрузилось» и кнопку «Повторить». Включи сеть, нажми кнопку — загрузка продолжится.

Ответы к упражнениям

Упражнение 16.1. Меняется только правило выбора колонок — раскладка и расчёт размера превью подхватят его сами, потому что оба зовут `columns(forWidth:)`:

```
static func columns(forWidth width: CGFloat) -> Int {
    switch width {
    case ..<350: return 2
    case ..<600: return 3
    default: return 4
    }
}
```

`case ..<350` — «всё, что меньше 350». Ветки проверяются по порядку, поэтому вторая ловит ширину от 350 до 599,99. Ширина ровно 600 теперь даёт 4 колонки, а в исходном правиле (`width > 600`) давала 3 — это граница, поставь её там, где тебе удобнее.

Упражнение 16.2. Подключаем протокол предзагрузки:

```
import UIKit

extension GalleryViewController: UICollectionViewDataSourcePrefetching {
    func collectionView(_ collectionView: UICollectionView, prefetchItemsAt indexPaths:
    ↪  [IndexPath]) {
        let side = thumbnailPixelSide
        for indexPath in indexPaths where indexPath.item < photos.count {
            guard let url = photos[indexPath.item].thumbnailURL(side: side) else {
            ↪  continue }
            Task { _ = await ImageCache.shared.image(for: url) }
        }
    }
}
```

И одна строка в `viewDidLoad`: `collectionView.prefetchDataSource = self`. Задача прогрева ничего не делает с результатом — ей важно, что `ImageCache` положит картинку в кеш. Когда ячейка появится, `cachedImage(for:)` отдаст её сразу. Если прогрев ещё идёт, ячейка подцепится к той же загрузке через `inflight` — второго запроса не будет. Проверка `indexPath.item < photos.count` — страховка: система может спросить про элементы, которых уже нет.

Что мы выучили

- picsum.photos — публичный API картинок без ключа. Превью — квадратные `thumbnailURL(side:)`, полное фото — `fullURL` с сохранением пропорций, иначе сервис обрежет его до квадрата.
- `CodingKeys` сопоставляет `download_url` из JSON со свойством `downloadURL`. Модель, которая уходит в другие потоки, — `nonisolated struct`.
- Последняя страница — та, в которой пришло меньше элементов, чем просили.
- `NSCache<NSURL, UIImage>` с лимитом в байтах сам освобождает память; картинка в памяти весит ширина × высота × 4 байта, а не размер JPEG.
- Словарь `inflight: [URL: Task]` убирает повторные запросы одной и той же картинки. `byPreparingForDisplay()` распаковывает JPEG заранее, вне главного потока.
- Compositional layout: item → group → section. Для сетки у item ширина `1/колонки`, высота группы через `fractionalWidth` даёт квадраты; правило колонок пересчитывается при повороте само. Раскладка — объект `NSObject`, не view: она только считает, где что стоит.
- `UICollectionViewCell → UICollectionReusableView → UIView`: footer и ячейка переиспользуются одинаково, но `contentView` и выделение есть только у ячейки; свои subviews — в `contentView`.
- При зуме scroll view ставит картинке `transform`, и её `frame` становится не определён — трогаем кадр только при единичном масштабе.
- Пагинация: `willDisplay` + запас в 6 ячеек + флаги `hasMore`, `isLoadingPage`, `nextPage`, `lastLoadFailed`. Новые элементы — `insertItems` в `performBatchUpdates`, не `reloadData()`.
- Footer — `UICollectionReusableView` с тремя состояниями, включая «Повторить» после ошибки.
- Ячейка: сброс в `prepareForReuse`, проверка `currentURL == url`, отмена задачи, сначала синхронный кеш. Размер превью = точки × `displayScale`, округлённые до сотни.
- `UIScrollView` + `viewForZooming` = щипок из коробки; `zoom(to:)` с прямоугольником в 2,5 раза меньше экрана приближает в 2,5 раза.
- Долгое нажатие — `contextMenuConfigurationForItemAt`; информационный пункт — `attributes: .disabled`.
- Сохранение в «Фото» — уровень `.addOnly` и ключ `NSPhotoLibraryAddUsageDescription`; выбор фото через `PHPickerViewController` разрешения не требует.

Apple Developer Documentation

- UICollectionView — коллекция с произвольной раскладкой; в Gallery — основной экран.
- UICollectionViewCompositionalLayout — раскладка из секций, групп и элементов (iOS 13+), пересчитывается от размера контейнера.
- NSCollectionLayoutSection — секция compositional layout; в неё кладём группу и `boundarySupplementaryItems`.
- UICollectionViewLayout — базовый класс раскладок: объект, который считает положение элементов, а не view.
- UICollectionReusableView — общий предок ячеек и дополнительных видов; здесь живёт `prepareForReuse()`.
- UICollectionViewCell — переиспользуемая ячейка; в `GalleryCell` сверяем `currentURL`.
- UICollectionViewCell.contentView — куда класть своё содержимое ячейки.
- UIView.frame — кадр в координатах superview; при `transform`, отличном от identity, значение не определено.

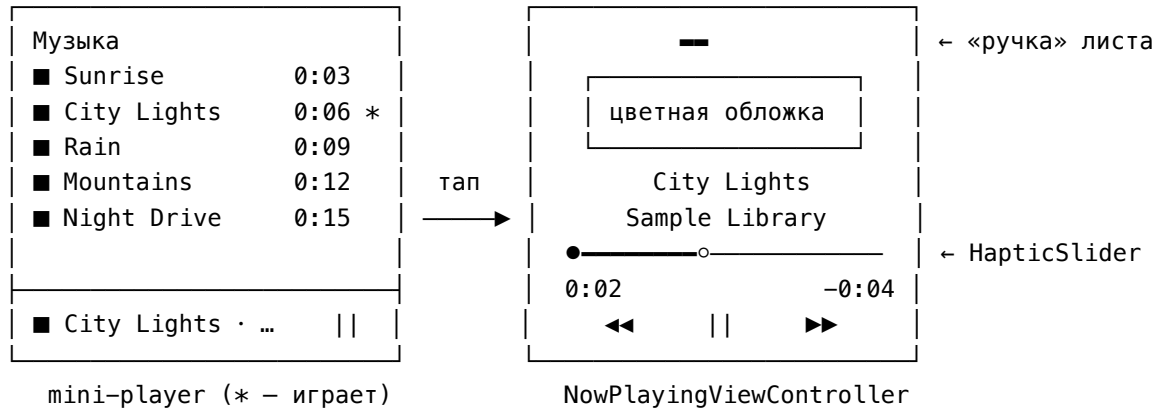
- UICollectionViewDiffableDataSource — источник данных на снимках, ступень после ручных `insertItems`.
- NSCache — кеш, который сам освобождает память; лимиты `countLimit` и `totalCostLimit`.
- preparingForDisplay() — семейство методов распаковки картинки заранее (iOS 15+), включая асинхронный `byPreparingForDisplay()`.
- zoom(to:animated:) — приближение scroll view к заданному прямоугольнику.
- UIContextMenuConfiguration — меню по долгому нажатию.
- PHPhotoLibrary — доступ к медиатеке: запрос разрешения и `performChanges`.
- PHAccessLevel.addOnly — уровень «только добавлять», которого хватает для сохранения.
- NSPhotoLibraryAddUsageDescription — ключ Info.plist с текстом для запроса на добавление фото.
- Delivering an enhanced privacy experience in your photos app — статья Apple о том, когда хватает `PHPicker`, а когда нужен доступ к медиатеке.
- HIG — Collections — рекомендации Apple по сеткам и коллекциям.
- HIG — Image views — подача изображений: пропорции, режимы заполнения, заглушки.

→ Глава 17. Music Player — AVPlayer, mini-player → full sheet, haptic slider

Глава 17. Music Player — AVPlayer, mini-player → full sheet, haptic slider

Музыкальный плеер собирает несколько важных тем сразу: воспроизведение звука по сети через `AVPlayer`, аудиосессию (как приложение договаривается с системой о звуке), одно общее состояние плеера, на которое подписаны несколько экранов, маленький плеер внизу списка (mini-player), который открывается в полноэкранный «Сейчас играет», и ползунок перемотки с тактильными щелчками.

Что строим:



Файлы:

```
Apps/Music/
├── Track.swift                     ← модель трека и библиотека
├── MusicPlayer.swift               ← единственный плеер + состояние
├── TimeFormat.swift                ← 75 секунд → "1:15"
├── CoverView.swift                 ← «обложка»: цвет + SF Symbol
├── MiniPlayerView.swift            ← полоска внизу списка
├── MusicLibraryViewController.swift  ← список треков
├── HapticSlider.swift              ← ползунок с щелчками
└── NowPlayingViewController.swift  ← полноэкранный плеер
```

> **Режим сборки** — как во введении: Swift 6, `Default Actor Isolation = MainActor`, iOS 15+. Код, который AVFoundation вызывает «со стороны», мы явно возвращаем на главный поток — в разделе 17.6 видно, как и почему.

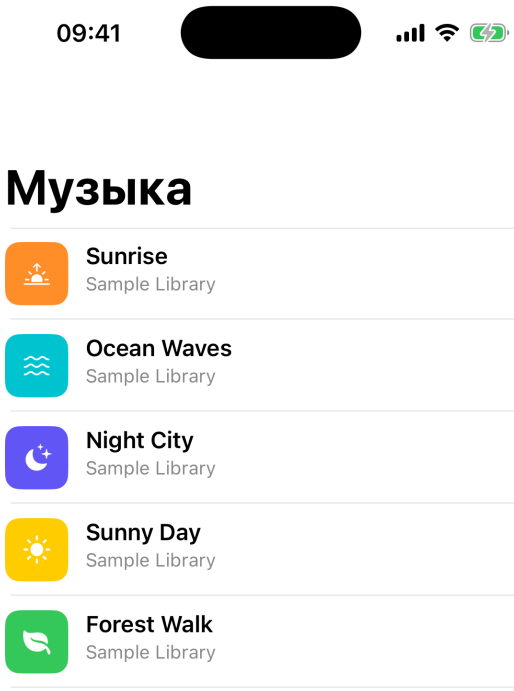


Рис. 13: Музыкальная библиотека с mini-player внизу

17.1 Треки — публичные mp3

Чтобы не класть аудиофайлы в проект, берём короткие публичные mp3 с сервиса **samplelib.com**. У него есть файлы длиной 3, 6, 9, 12 и 15 секунд с предсказуемыми адресами (мы проверили: все пять отвечают, от 52 КБ до 307 КБ).

```
import UIKit

struct Track: Hashable {
    let id: String
    let title: String
    let artist: String
    let durationSeconds: TimeInterval
    let url: URL
    let coverColor: UIColor
    let symbol: String
}

enum MusicLibrary {
    static let tracks: [Track] = [
        makeTrack(id: "1", title: "Sunrise", seconds: 3, color: .systemOrange, symbol:
↪  "sunrise.fill"),
        makeTrack(id: "2", title: "City Lights", seconds: 6, color: .systemIndigo, symbol:
↪  "building.2.fill"),
        makeTrack(id: "3", title: "Rain", seconds: 9, color: .systemTeal, symbol:
↪  "cloud.rain.fill"),
        makeTrack(id: "4", title: "Mountains", seconds: 12, color: .systemGreen, symbol:
↪  "mountain.2.fill"),
        makeTrack(id: "5", title: "Night Drive", seconds: 15, color: .systemPurple,
↪  symbol: "moon.stars.fill"),
    ]

    private static func makeTrack(id: String, title: String, seconds: Int,
                                  color: UIColor, symbol: String) -> Track {
        // URL собран из проверенного шаблона без пробелов и спецсимволов,
        // поэтому `!` здесь не выстрелит.
        Track(id: id,
              title: title,
              artist: "Sample Library",
              durationSeconds: TimeInterval(seconds),
              url: URL(string:
↪  "https://download.samplelib.com/mp3/sample-\(seconds)s.mp3")!,
              coverColor: color,
              symbol: symbol)
    }
}
```

`coverColor` + `symbol` — замена настоящей обложки: цветной квадрат с SF Symbol по центру. SF Symbols — встроенная в iOS библиотека значков, которые берутся по имени

(`UIImage(systemName: "sunrise.fill")`) и масштабируются вместе со шрифтом. Такая «обложка» — обычный запасной вариант (fallback) в настоящих приложениях, когда картинки нет.

`durationSeconds` — длительность, которую мы знаем заранее. Она нужна, чтобы показать длину трека в списке ещё до загрузки. Когда плеер прочитает настоящую длительность файла, мы подменим её (17.6).

Фабричный метод `makeTrack` убирает повтор: адрес строится из числа секунд, и опечатка в одном из пяти URL становится невозможной. Это единственное место главы, где стоит `!`: строка собрана из шаблона, в котором нет символов, запрещённых в URL.

17.2 AVPlayer vs AVAudioPlayer

Apple даёт два класса для воспроизведения звука:

- **`AVAudioPlayer`** (фреймворк AVFAudio) играет файл, который уже лежит на устройстве, или данные в памяти. Потоковое воспроизведение по сети он не умеет: сначала скачай файл целиком.
- **`AVPlayer`** (AVFoundation) играет и локальные файлы, и адреса в сети, начиная воспроизведение до окончания загрузки. Он же играет видео и потоки HLS.

Наши mp3 лежат в сети, поэтому берём `AVPlayer`. `AVAudioPlayer` удобен, когда файлы в приложении: у него проще интерфейс (громкость, повтор, уровни звука). А для совсем коротких звуков интерфейса (щелчок, «дзынь») есть ещё более лёгкий путь — `AudioServicesPlaySystemSound`.

17.3 Один плеер на всё приложение и аудиосессия

`MusicPlayer` — синглтон (singleton — класс с единственным общим экземпляром `shared`). Плеер должен быть один: если каждый экран заведёт свой `AVPlayer`, два трека заиграют одновременно.

```
import AVFoundation

@MainActor
final class MusicPlayer {
    static let shared = MusicPlayer()

    struct State {
        var current: Track?
        var isPlaying = false
        var progress: TimeInterval = 0
        var duration: TimeInterval = 0
    }

    private(set) var state = State()

    private let player = AVPlayer()
    private var timeObserver: Any?
```

```
    private var endObserver: NSObjectProtocol?
    private var observers: [(State) -> Bool] = []
    private var isSessionActive = false

    private init() {
        do {
            try AVAudioSession.sharedInstance().setCategory(.playback, mode: .default)
        } catch {
            print("Не удалось настроить аудиосессию: \(error)")
        }
        let interval = CMTime(seconds: 0.25, preferredTimescale: 600)
        timeObserver = player.addPeriodicTimeObserver(forInterval: interval, queue: .main)
↪   { [weak self] time in
            MainActor.assumeIsolated {
                self?.handleTick(time)
            }
        }
    }
}
```

`State` — всё, что нужно экранам: какой трек, играет ли, на какой секунде и какая длина. Экраны не лезут в `AVPlayer` сами, они получают готовый снимок состояния. `private(set)` — читать `state` может любой, менять — только сам плеер.

Один `AVPlayer` на всю жизнь. Можно создавать новый `AVPlayer` на каждый трек и каждый раз переставлять наблюдатель времени, но проще один раз создать плеер и менять в нём трек через `replaceCurrentItem(with:)` (17.6). Наблюдатель времени тогда тоже ставится один раз, в `init`, и его не нужно снимать — плеер живёт, пока живёт приложение.

`private init()` — никто, кроме самого класса, не создаст второй экземпляр. Все пользуются `MusicPlayer.shared`.

Аудиосессия

Аудиосессия (`AVAudioSession`) — посредник между приложением и системой звука iOS. Через неё приложение сообщает, **зачем** ему звук, а система решает, как с ним обращаться: глушить ли переключателем «Без звука», останавливать ли при блокировке экрана, приглушать ли музыку из другого приложения. Приложения делят один динамик, и аудиосессия — их договор о правилах.

По умолчанию (категория `.soloAmbient`) звук приложения глушится переключателем «Без звука» и замолкает при блокировке экрана — так документация Apple описывает поведение по умолчанию. Для плеера это не годится.

`setCategory(.playback)` говорит системе: «воспроизведение — главная функция приложения». После этого:

- звук играет, даже если переключатель стоит на «Без звука»;
- при старте воспроизведения музыка других приложений останавливается. Чтобы играть поверх чужой музыки, нужна опция `.mixWithOthers`;

- **в фоне** (свёрнутое приложение, заблокированный экран) звук продолжит играть, **только если** у приложения включён фоновый режим Audio. Одной категории для этого мало — подробности в разделе 17.13.

  Частое заблуждение — что `.playback` сам по себе даёт игру в фоне. По документации Apple («Configuring your app for media playback») категория `.playback` лишь *позволяет* играть в фоне, если включён фоновый режим «Audio, AirPlay, and Picture in Picture». Без него звук остановится, как только приложение уйдёт в фон.

Категорий в `AVAudioSession` семь (так в заголовках SDK):

Категория	Для чего	«Без звука»	Чужая музыка
`.ambient`	фоновые звуки игры, «дождь»	глушит	играет вместе
`.soloAmbient`	по умолчанию	глушит	останавливает
`.playback`	музыка, подкасты, видео	не глушит	останавливает (или `.mixWithOthers`)
`.record`	только запись	—	останавливает
`.playAndRecord`	звонки, голосовые сообщения	не глушит	останавливает
`.multiRoute`	вывод на несколько устройств сразу	не глушит	останавливает
`.audioProcessing`	устарела в iOS 10	—	—

Для музыкального плеера почти всегда `.playback`.

Когда активировать сессию. В `init` мы только задаём категорию, а `setActive(true)` откладываем до первого нажатия «играть» (метод `activateSessionIfNeeded` в 17.5). Apple советует именно так: активная сессия с категорией `.playback` останавливает музыку других приложений. Если активировать её при создании плеера, музыка пользователя оборвётся просто от того, что он открыл наш экран, ничего не нажав.

Периодический наблюдатель времени

`addPeriodicTimeObserver(forInterval:queue:using:)` вызывает замыкание каждые `interval` времени воспроизведения, а ещё когда воспроизведение стартует, останавливается или время прыгает (перемотка). Интервал 0,25 секунды — четыре обновления в секунду: этого хватает, чтобы полоса прогресса двигалась плавно, а счётчик «0:03» не опаздывал.

`CMTime` — время в AVFoundation. Это не число секунд, а дробь: `value / timescale`. `CMTime(seconds: 0.25, preferredTimescale: 600)` хранит 150/600 — ровно четверть секунды, без ошибок округления, как у `Double`. Число 600 выбрано потому, что делится на 24, 25, 30 и 60 — типичные частоты кадров видео, и любой кадр выражается в нём целым числом. Для звука подошло бы и другое, но 600 — привычный выбор.

`MainActor.assumeIsolated`. Замыкание наблюдателя AVFoundation объявляет как `@Sendable` — его разрешено вызывать из любого потока. Компилятор не знает, что мы передали `queue: .main`, и не пускает к `state` (он живёт на main actor). Если обратиться к `self.state` прямо в замыкании, Swift 6 выдаст предупреждения «main actor-isolated property ‘state’ can not be mutated

from a Sendable closure». `MainActor.assumeIsolated { }` говорит: «я гарантирую, что мы уже на главном потоке, пусти». Гарантия проверяется во время выполнения: если однажды замыкание придёт с другого потока, приложение остановится с понятной ошибкой, а не испортит данные молча. Такое обещание честно, потому что мы сами передали `queue: .main`.

17.4 Подписчики — как экраны узнают о переменах

Mini-player, экран «Сейчас играет» и список треков хотят знать текущее состояние. Делаем простую подписку: экран передаёт замыкание, плеер вызывает его при каждом изменении.

```
import AVFoundation

extension MusicPlayer {
    /// Подписывает `owner` на изменения. Подписка живёт, пока жив `owner`:
    /// как только он освобождён из памяти, обработчик сам выпадает из списка.
    func observe<Owner: AnyObject>(_ owner: Owner, _ handler: @escaping (Owner, State) ->
    ↪ Void) {
        let entry: (State) -> Bool = { [weak owner] state in
            guard let owner else { return false }
            handler(owner, state)
            return true
        }
        _ = entry(state)
        observers.append(entry)
    }

    func notify() {
        observers = observers.filter { $0(state) }
    }
}
```

Этот код живёт в том же файле `MusicPlayer.swift`, что и класс, поэтому расширение видит `private`-свойства вроде `observers`.

Почему нужна отписка. Представь `subscribe(_ handler:)` без отписки. Каждый раз, когда пользователь открывает «Сейчас играет», в массив добавлялся бы новый обработчик и не уходил никогда. Хуже того: если обработчик держит экран сильной ссылкой, закрытый экран не освобождается из памяти.

Как устроено у нас. Подписчик передаёт себя (`owner`) и обработчик, который получает этого `owner` параметром:

```
MusicPlayer.shared.observe(self) { vc, state in
    vc.apply(state: state)
}
```

Внутри плеер держит `owner` **слабо** (`[weak owner]`). Обработчику не нужно писать `[weak self]`: `vc` приходит параметром, а не захватывается. Когда экран закрыт и освобождён, `owner` становится `nil`, обёртка возвращает `false`, и `notify()` выкидывает её из списка (`filter` оставляет только те, что вернули `true`). Отписываться вручную не нужно — и забыть отписаться невозможно.

Почему не отписка в `deinit`? В Swift 6 `deinit` у класса на main actor не изолирован: из него нельзя вызвать метод плеера. Изолированный `isolated deinit` появился в Swift 6.2, но требует iOS 18.4 и новее, а у нас iOS 15. Слабая ссылка обходит проблему целиком.

`_ = entry(state)` — новый подписчик сразу получает текущее состояние. Экран открылся посреди трека — и сразу показывает правильное название и позицию, не дожидаясь очередного тика.

17.5 Play / Pause / Skip

```
import AVFoundation

extension MusicPlayer {
    func play(_ track: Track) {
        if state.current?.id != track.id {
            load(track)
        }
        resume()
    }

    func pause() {
        player.pause()
        state.isPlaying = false
        notify()
    }

    func toggle() {
        guard state.current != nil else { return }
        if state.isPlaying { pause() } else { resume() }
    }

    func skipForward() {
        step(by: 1)
    }

    func skipBackward() {
        // Как в «Музыке»: если трек играет дольше 3 секунд — сначала в его начало.
        if state.progress > 3 {
            seek(to: 0)
        } else {
            step(by: -1)
        }
    }

    private func resume() {
        guard state.current != nil else { return }
        activateSessionIfNeeded()
        player.play()
```

```
        state.isPlaying = true
        notify()
    }

    private func activateSessionIfNeeded() {
        guard !isSessionActive else { return }
        do {
            try AVAudioSession.sharedInstance().setActive(true)
            isSessionActive = true
        } catch {
            print("Аудиосессия не активировалась: \(error)")
        }
    }

    private func step(by offset: Int) {
        let tracks = MusicLibrary.tracks
        guard let current = state.current,
              let index = tracks.firstIndex(where: { $0.id == current.id }) else { return }
        let target = (index + offset + tracks.count) % tracks.count
        play(tracks[target])
    }
}
```

- **`play(_:)`** загружает трек, только если он другой. Тап по уже играющему треку не начинает его сначала.
- **`toggle()`** — пауза или продолжение. Общая часть «продолжить» вынесена в `resume()`, чтобы `play` и `toggle` не дублировали код.
- **`skipBackward()`** ведёт себя как системная «Музыка»: первое нажатие «назад» после третьей секунды возвращает в начало трека, и только в первые три секунды переключает на предыдущий.

`step(by:)` и остаток от деления. Переход по кругу: с последнего трека вперёд — на первый, с первого назад — на последний. Формула `(index + offset + count) % count` на числах, треков 5:

- вперёд с последнего: (4 + 1 + 5) % 5 = 10 % 5 = 0 — первый трек;
- назад с первого: (0 − 1 + 5) % 5 = 4 % 5 = 4 — последний;
- вперёд со второго: (1 + 1 + 5) % 5 = 7 % 5 = 2 — третий.

Зачем прибавлять `count`? В Swift остаток от отрицательного числа отрицательный: `-1 % 5` равно −1, и индекс −1 уронил бы приложение. Прибавка пяти делает число неотрицательным, а на результат деления по кругу не влияет.

17.6 `load` — смена трека

```
import AVFoundation

extension MusicPlayer {
```

```
    private func load(_ track: Track) {
        if let endObserver {
            NotificationCenter.default.removeObserver(endObserver)
        }
        let item = AVPlayerItem(url: track.url)
        player.replaceCurrentItem(with: item)

        state.current = track
        state.duration = track.durationSeconds
        state.progress = 0

        endObserver = NotificationCenter.default.addObserver(
            forName: AVPlayerItem.didPlayToEndTimeNotification,
            object: item,
            queue: .main
        ) { [weak self] _ in
            MainActor.assumeIsolated {
                self?.skipForward()
            }
        }
    }

    private func handleTick(_ time: CMTime) {
        guard state.current != nil else { return }
        state.progress = time.seconds
        if let itemDuration = player.currentItem?.duration.seconds,
           itemDuration.isFinite, itemDuration > 0 {
            state.duration = itemDuration
        }
        // Звонок, будильник, отключённые наушники ставят плеер на паузу
        // без нашего pause(). Сверяемся с самим плеером.
        state.isPlaying = player.timeControlStatus != .paused
        notify()
    }
}
```

1. **`AVPlayerItem(url:)`** — «что играть»: один конкретный файл со своим состоянием загрузки и длительностью. `AVPlayer` — «чем играть»: он умеет воспроизводить один item за раз.
2. **`replaceCurrentItem(with:)`** — поставить в тот же плеер другой трек. Старый item освобождается сам.
3. **Уведомление о конце трека.** `AVPlayerItem.didPlayToEndTimeNotification` приходит, когда трек доиграл. `object: item` — подписываемся только на **этот** item, иначе ловили бы окончание любого проигрывателя в приложении, например видео на соседнем экране.
4. **Снятие старого наблюдателя.** При смене трека прежнюю подписку убираем (`removeObserver`), и в каждый момент есть ровно одна.

Почему наблюдатель на замыкании, а не селектор. Привычная запись — `addObserver(self, selector: #selector(...))`. Но документация AVFoundation предупреждает, что уведомления

плеера могут приходить на другом потоке, а метод с `@objc` у нашего класса изолирован на main actor. Версия с замыканием и `queue: .main` гарантирует доставку на главную очередь, а `MainActor.assumeIsolated` объясняет это компилятору — так же, как в 17.3.

`handleTick` — вызывается четыре раза в секунду:

- `time.seconds` — текущая позиция в секундах (`CMTime` → `Double`);
- `currentItem?.duration` — настоящая длина файла. Пока файл не начал загружаться, длина неизвестна, и `duration.seconds` равен `NaN` («не число»). Проверка `isFinite` отсекает это значение, а пока настоящей длины нет, работает наша `durationSeconds` из библиотеки;
- `timeControlStatus` — что плеер делает в действительности: `.playing`, `.paused` или `.waitingToPlayAtSpecifiedRate` (ждёт, пока догрузятся данные). Если позвонили по телефону, система поставит плеер на паузу сама, и без этой строки кнопка продолжала бы показывать «пауза», хотя музыка молчит. Ожидание загрузки мы считаем «играет»: пользователь нажал play, звук вот-вот пойдёт.

17.7 Seek — перемотка

```
import AVFoundation

extension MusicPlayer {
    func seek(to seconds: TimeInterval) {
        let time = CMTime(seconds: seconds, preferredTimescale: 600)
        player.seek(to: time, toleranceBefore: .zero, toleranceAfter: .zero)
        state.progress = seconds
        notify()
    }
}
```

`seek(to:)` без допусков разрешает плееру прыгнуть в ближайшую удобную для декодера точку — для сжатого аудио это может быть на долю секунды в стороне. `toleranceBefore: .zero, toleranceAfter: .zero` — «ровно в эту точку». Точная перемотка чуть медленнее, но для коротких mp3 разницы не заметно, а ползунок не отскакивает.

`state.progress = seconds` сразу, не дожидаясь плеера: интерфейс показывает новую позицию мгновенно, а ближайший тик наблюдателя подтвердит её. Если перемотка ещё не закончилась, первый тик может на миг показать старую позицию — для нашего плеера это незаметно.

> **Упражнение 17.1.** Функция `formatPlaybackTime` (раздел 17.8) показывает время как «м:сс». Для подкаста на час с лишним «75:30» читается плохо. Напиши `formatLongPlaybackTime(_:)`, которая для 3725 секунд вернёт `"1:02:05"`, а для 75 секунд — по-прежнему `"1:15"`. Решение — в конце главы.

17.8 MusicLibraryViewController — список + mini-player

Сначала вспомогательные части: форматирование времени и «обложка».

```swift
import Foundation

/// 75.4 секунды → "1:15". Отрицательное и NaN показываем как "0:00".
func formatPlaybackTime(_ seconds: TimeInterval) -> String {
    guard seconds.isFinite, seconds > 0 else { return "0:00" }
    let total = Int(seconds.rounded(.down))
    return String(format: "%d:%02d", total / 60, total % 60)
}
```

75,4 секунды: отбрасываем дробную часть (`rounded(.down)`) — 75. Минуты — целая часть от деления на 60: 75 / 60 = 1. Секунды — остаток: 75 % 60 = 15. Формат `%02d` дописывает ведущий ноль до двух цифр, поэтому 5 секунд станут `05`, и получится `"1:05"`, а не `"1:5"`. Проверка `isFinite` нужна из-за `NaN`, о котором шла речь в 17.6.

```swift
import UIKit

/// Цветной квадрат с SF Symbol по центру — замена настоящей обложки.
final class CoverView: UIView {
    private let symbolView = UIImageView()

    init(symbolPointSize: CGFloat) {
        super.init(frame: .zero)
        layer.cornerRadius = 8
        symbolView.tintColor = .white
        symbolView.contentMode = .scaleAspectFit
        symbolView.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize:
↪   symbolPointSize,
                                                                              weight: .semibold)
        symbolView.translatesAutoresizingMaskIntoConstraints = false
        addSubview(symbolView)
        NSLayoutConstraint.activate([
            symbolView.centerXAnchor.constraint(equalTo: centerXAnchor),
            symbolView.centerYAnchor.constraint(equalTo: centerYAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func apply(_ track: Track?) {
        backgroundColor = track?.coverColor ?? .systemGray4
        symbolView.image = UIImage(systemName: track?.symbol ?? "music.note")
    }
}
```

Один класс для обложки в mini-player (значок 18 pt) и на большом экране (96 pt). `preferredSymbolConfiguratio` задаёт размер и толщину SF Symbol. `translatesAutoresizingMaskIntoConstraints = false` — обязательная строка для любого вида, который мы расставляем ограничениями (constraints) Auto Layout: у view, созданного в коде, это свойство по умолчанию `true`, и UIKit переводит

его маску autoresizing в собственные ограничения размера и положения; наши ограничения с ними конфликтуют. Исключение — arranged subviews стека (`coverView`, `titleLabel`, `playButton` ниже): `UIStackView` выставляет это свойство сам, и Apple прямо просит его там не трогать. Подробно о свойстве — глава 0, раздел 0.3, и глава 19, раздел 19.12.

Mini-player — полоска над нижним краем: обложка, название, кнопка «играть/пауза» и тонкая полоса прогресса сверху.

```
import UIKit

final class MiniPlayerView: UIView {
    var onTap: (() -> Void)?

    private let coverView = CoverView(symbolPointSize: 18)
    private let titleLabel = UILabel()
    private let playButton = UIButton(type: .system)
    private let progressBar = UIProgressView(progressViewStyle: .bar)

    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .secondarySystemBackground

        titleLabel.font = .preferredFont(forTextStyle: .subheadline)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.lineBreakMode = .byTruncatingTail

        playButton.addAction(UIAction { _ in MusicPlayer.shared.toggle() }, for:
↪  .touchUpInside)
        playButton.setPreferredSymbolConfiguration(UIImage.SymbolConfiguration(pointSize:
↪  22),
                                                   forImageIn: .normal)

        let row = UIStackView(arrangedSubviews: [coverView, titleLabel, playButton])
        row.spacing = 12
        row.alignment = .center

        [row, progressBar].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            addSubview($0)
        }
        NSLayoutConstraint.activate([
            progressBar.topAnchor.constraint(equalTo: topAnchor),
            progressBar.leadingAnchor.constraint(equalTo: leadingAnchor),
            progressBar.trailingAnchor.constraint(equalTo: trailingAnchor),

            row.topAnchor.constraint(equalTo: progressBar.bottomAnchor, constant: 8),
            row.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -8),
            row.leadingAnchor.constraint(equalTo: layoutMarginsGuide.leadingAnchor),
            row.trailingAnchor.constraint(equalTo: layoutMarginsGuide.trailingAnchor),
```

```
            coverView.widthAnchor.constraint(equalToConstant: 44),
            coverView.heightAnchor.constraint(equalToConstant: 44),
            playButton.widthAnchor.constraint(equalToConstant: 44),
            playButton.heightAnchor.constraint(equalToConstant: 44),
        ])

        addGestureRecognizer(UITapGestureRecognizer(target: self, action:
↪   #selector(tapped)))
        accessibilityHint = "Открывает полный плеер"
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    @objc private func tapped() { onTap?() }

    func apply(state: MusicPlayer.State) {
        guard let track = state.current else { return }
        coverView.apply(track)
        titleLabel.text = "\(track.title) · \(track.artist)"
        progressBar.progress = state.duration > 0 ? Float(state.progress / state.duration)
↪   : 0
        let symbol = state.isPlaying ? "pause.fill" : "play.fill"
        playButton.setImage(UIImage(systemName: symbol), for: .normal)
        playButton.accessibilityLabel = state.isPlaying ? "Пауза" : "Играть"
    }
}
```

- **Кнопка 44×44 точки** — минимальный размер области нажатия по рекомендациям Apple (HIG). Значок внутри меньше (22 pt), но пальцу есть куда попасть.
- **`setPreferredSymbolConfiguration(_:forImageIn:)`** — размер SF Symbol на кнопке. У `UIButton` это метод, а не свойство: присваивание `button.preferredSymbolConfigurationForImage = …` не скомпилируется.
- **Прогресс на числах:** трек 9 секунд, играет 4,5-я — 4,5 / 9 = 0,5, полоса заполнена наполовину. Деление только при `duration > 0`, иначе в самом начале вышло бы деление на ноль.
- **`layoutMarginsGuide`** — направляющая по внутренним полям вида (layout margins). У обычного `UIView` они по умолчанию 8 точек со всех сторон — так в документации `layoutMargins`, и так показал симулятор для view, вложенного в экран. Больше — системные минимальные поля (16 по бокам на iPhone, на симуляторе iPhone 16 мы сняли именно столько) плюс safe area — только у корневого `view` контроллера. Mini-player — обычный view, так что контент отступает от его краёв на 8 точек. Нужны поля как у экрана — задай `directionalLayoutMargins` сам или включи `preservesSuperviewLayoutMargins`.
- **Жест на весь вид.** `UITapGestureRecognizer` на полоске открывает полный плеер. Нажатие на кнопку «пауза» жест не перехватит: по правилу Apple («Attaching gesture recognizers to UIKit controls») события внутри границ контрола сначала обрабатывает сам контрол, и одиночный тап по `UIButton`, `UISwitch`, `UIStepper` или `UISegmentedControl` уходит в его

action, а не в распознаватель жестов на родительском view. Перебить это можно, только повесив жест прямо на кнопку — тогда уже он получит касание первым.

- **`accessibilityLabel`** у кнопки — что прочитает VoiceOver (экранный диктор для незрячих пользователей). Без него он прочитал бы имя картинки «pause.fill».

Экран списка:

```
import UIKit

final class MusicLibraryViewController: UIViewController {
    private let tableView = UITableView(frame: .zero, style: .plain)
    private let miniPlayer = MiniPlayerView()
    private let tracks = MusicLibrary.tracks
    private var playingTrackID: String?

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Музыка"
        view.backgroundColor = .systemBackground
        setupLayout()

        MusicPlayer.shared.observe(self) { vc, state in
            vc.miniPlayer.apply(state: state)
            vc.miniPlayer.isHidden = state.current == nil
            vc.updatePlayingMarker(state.current?.id)
        }
    }

    private func setupLayout() {
        tableView.dataSource = self
        tableView.delegate = self
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "TrackCell")

        miniPlayer.isHidden = true
        miniPlayer.onTap = { [weak self] in self?.presentNowPlaying() }

        // Стек сам «схлопывает» скрытый mini-player: таблица занимает его место.
        let stack = UIStackView(arrangedSubviews: [tableView, miniPlayer])
        stack.axis = .vertical
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: view.topAnchor),
            stack.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            stack.bottomAnchor.constraint(equalTo: view.safeAreaLayoutGuide.bottomAnchor),
        ])
    }
```

```
    private func updatePlayingMarker(_ id: String?) {
        guard id != playingTrackID else { return }
        let changed = [playingTrackID, id].compactMap { id in
            tracks.firstIndex { $0.id == id }.map { IndexPath(row: $0, section: 0) }
        }
        playingTrackID = id
        tableView.reloadRows(at: changed, with: .none)
    }

    private func presentNowPlaying() {
        let vc = NowPlayingViewController()
        vc.modalPresentationStyle = .pageSheet
        if let sheet = vc.sheetPresentationController {
            sheet.detents = [.large()]
            sheet.prefersGrabberVisible = true
        }
        present(vc, animated: true)
    }
}

extension MusicLibraryViewController: UITableViewDataSource, UITableViewDelegate {
    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        tracks.count
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "TrackCell", for:
        ↪ indexPath)
        let track = tracks[indexPath.row]
        var content = cell.defaultContentConfiguration()
        content.text = track.title
        content.secondaryText = "\(track.artist) ·
↪ \(formatPlaybackTime(track.durationSeconds))"
        content.image = UIImage(systemName: track.symbol)
        content.imageProperties.tintColor = track.coverColor
        cell.contentConfiguration = content
        let isPlaying = track.id == playingTrackID
        cell.accessoryView = isPlaying ? UIImageView(image: UIImage(systemName:
↪ "speaker.wave.2.fill")) : nil
        cell.accessibilityValue = isPlaying ? "Сейчас играет" : nil
        return cell
    }

    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        MusicPlayer.shared.play(tracks[indexPath.row])
    }
```

```
}
```

Вёрстка через `UIStackView`. Таблица и mini-player лежат в вертикальном стеке. У стека полезное свойство: скрытый (`isHidden = true`) элемент не занимает места — стек, по документации, перестраивает раскладку при каждой смене `isHidden` у arranged subview, а сам скрытый view из `arrangedSubviews` не выпадает (мы проверили: после скрытия второго элемента первый занял всю высоту, а в `arrangedSubviews` их по-прежнему два). Пока ничего не играет, таблица тянется до низа. Появился трек — mini-player показывается, и таблица сама укорачивается над ним. Если бы mini-player висел поверх таблицы, последняя строка пряталась бы под ним.

Низ стека привязан к `safeAreaLayoutGuide.bottomAnchor` — к краю безопасной зоны, над полоской «домой». Иначе кнопка mini-player оказалась бы под системным жестом.

Отметка играющего трека. Список показывает значок динамика у текущего трека. `updatePlayingMarker` перерисовывает только две строки — ту, что перестала играть, и ту, что начала. Проверка `guard id != playingTrackID` важна: обработчик вызывается четыре раза в секунду, и без неё мы бы перезагружали строки на каждом тике.

Переиспользование ячеек. В `cellForRowAt` `accessoryView` выставляется в **обеих** ветках: значок или `nil`. Если бы мы только ставили значок, ячейка, в которой он был, после переиспользования показала бы динамик у чужого трека.

`defaultContentConfiguration()` (iOS 14+) — стандартная раскладка ячейки: картинка, заголовок, подзаголовок. Заполняешь поля и присваиваешь `cell.contentConfiguration`.

17.9 NowPlaying — лист с «ручкой»

`presentNowPlaying()` из листинга выше открывает полный плеер листом (sheet — экран, который выезжает снизу и лежит поверх текущего):

- `modalPresentationStyle = .pageSheet` — стиль листа. На iPhone он почти во весь экран, сверху видно край предыдущего экрана; на iPad — карточка по центру.
- `sheetPresentationController` (iOS 15+) — настройки листа. Это не view и не view controller: `UISheetPresentationController → UIPresentationController → NSObject`, объект, который управляет показом (presentation) и живёт, пока экран показан (глава 0, раздел 0.6). `detents` — «фиксаторы», высоты, на которых лист может остановиться. `[.large()]` — только полная высота. Можно `[.medium(), .large()]`, и лист будет стоять на половине экрана, но для «Сейчас играет» половины мало: обложка и ползунок не помещаются.
- `prefersGrabberVisible = true` — серая «ручка» наверху листа, подсказка «потяни вниз, чтобы закрыть».

Контроллер не оборачиваем в `UINavigationController`: навигационная панель на этом экране не нужна, поэтому показываем его напрямую.

17.10 HapticSlider — ползунок с тактильными щелчками

Haptics — тактильная отдача: короткие вибрации моторчика Taptic Engine, которые ощущаются как щелчок. `UIImpactFeedbackGenerator` — генератор «удара» заданной силы.

```swift
import UIKit

final class HapticSlider: UISlider {
    private let haptic = UIImpactFeedbackGenerator(style: .light)
    private var lastTick: Float = 0

    override init(frame: CGRect) {
        super.init(frame: frame)
        addTarget(self, action: #selector(valueDidChange), for: .valueChanged)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func beginTracking(_ touch: UITouch, with event: UIEvent?) -> Bool {
        haptic.prepare()
        lastTick = tick(for: value)
        return super.beginTracking(touch, with: event)
    }

    private func tick(for value: Float) -> Float {
       let fraction = (value - minimumValue) / max(maximumValue - minimumValue, .ulpOfOne)
        return (fraction * 20).rounded()
    }

    @objc private func valueDidChange() {
        let current = tick(for: value)
        if current != lastTick {
            haptic.impactOccurred(intensity: 0.5)
            haptic.prepare()
            lastTick = current
        }
    }
}
```

`UISlider` — это `UIControl`. Лестница: `UISlider → UIControl → UIView`. От `UIControl` ползунок получает target-action, события `UIControl.Event` и состояния `isEnabled`/`isHighlighted`; именно поэтому у него есть `addTarget(_:action:for:)` и `addAction(_:for:)`, а у `UILabel` или `UIImageView` — нет (таблица «кто контрол, а кто нет» — глава 0, раздел 0.4). `.valueChanged` — событие контрола «значение изменилось»; у ползунка оно по умолчанию приходит непрерывно, пока палец тянет бегунок (`isContinuous = true`), а при `isContinuous = false` — один раз, когда палец отпущен. `beginTracking(_:with:)` — тоже метод `UIControl`: точка для подклассов, которую UIKit зовёт, когда касание входит в границы контрола.

Засечки на числах. Шкалу делим на 20 участков по 5%. `tick(for:)` переводит положение ползунка в номер засечки: значение 0,37 — это 37%, × 20 = 7,4, округляем — засечка 7. Сдвинул до 0,41: 8,2 → 8. Номер сменился — щелчок. Для трека в 15 секунд одна засечка — это 15 × 5% = 0,75 секунды перемотки.

`fraction` пересчитывает значение в долю от 0 до 1 для любого диапазона слайдера (не только 0...1). `max(..., .ulpOfOne)` — защита от деления на ноль, если минимум и максимум вдруг совпадут (`.ulpOfOne` — крошечное положительное число).

Без `lastTick` щелчок звучал бы на **каждое** событие `valueChanged`, а при перетаскивании их приходит десятки в секунду — сплошное жужжание вместо засечек.

`beginTracking` — UIKit вызывает его, когда палец лёг на ползунок. Здесь запоминаем текущую засечку: иначе `lastTick` остался бы 0, и первое же движение с середины шкалы дало бы лишний щелчок.

`prepare()` будит Taptic Engine: первый щелчок после паузы иначе может прийти с заметной задержкой. Apple советует вызывать `prepare()` незадолго до ожидаемого срабатывания — касание ползунка и есть такой момент. После каждого щелчка зовём `prepare()` снова, потому что очередной может понадобиться через долю секунды. Без `prepare()` первый щелчок запаздывает.

`intensity: 0.5` — половина силы стиля `.light`. Полная сила при частых щелчках утомляет.

`valueChanged` приходит только от **пальца**. Когда плеер сам двигает `slider.value` (17.11), событие не отправляется и щелчков нет. Это общее правило `UIControl`: программная установка значения (`value = …`, `setValue(_:animated:)`, `isOn = …` у `UISwitch`) не рассылает action'ы. Мы проверили на симуляторе: после двух программных присваиваний счётчик `.valueChanged` остался на нуле.

17.11 Scrubbing — перемотка ползунком

Scrubbing — перемотка «прокруткой» ползунка. Экран «Сейчас играет» целиком:

```
import UIKit

final class NowPlayingViewController: UIViewController {
    private let coverView = CoverView(symbolPointSize: 96)
    private let titleLabel = UILabel()
    private let artistLabel = UILabel()
    private let slider = HapticSlider()
    private let elapsedLabel = UILabel()
    private let remainingLabel = UILabel()
    private let playButton = UIButton(type: .system)
    private var isUserScrubbing = false

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        setupLayout()
        setupActions()
        MusicPlayer.shared.observe(self) { vc, state in
            vc.apply(state: state)
        }
    }
```

```
    private func setupLayout() {
        coverView.layer.cornerRadius = 16

        titleLabel.font = .preferredFont(forTextStyle: .title2)
        artistLabel.font = .preferredFont(forTextStyle: .body)
        artistLabel.textColor = .secondaryLabel
        for label in [titleLabel, artistLabel] {
            label.textAlignment = .center
            label.adjustsFontForContentSizeCategory = true
        }
        for label in [elapsedLabel, remainingLabel] {
            // Моноширинные цифры: "1:11" и "1:08" одной ширины, текст не дрожит.
            label.font = .monospacedDigitSystemFont(ofSize: 13, weight: .regular)
            label.textColor = .secondaryLabel
        }

       let times = UIStackView(arrangedSubviews: [elapsedLabel, UIView(), remainingLabel])

        let backButton = makeControlButton(symbol: "backward.fill", label: "Назад") {
            MusicPlayer.shared.skipBackward()
        }
        let forwardButton = makeControlButton(symbol: "forward.fill", label: "Вперёд") {
            MusicPlayer.shared.skipForward()
        }
        playButton.setPreferredSymbolConfiguration(UIImage.SymbolConfiguration(pointSize:
↪  44),
                                                   forImageIn: .normal)
        playButton.addAction(UIAction { _ in MusicPlayer.shared.toggle() }, for:
↪  .touchUpInside)

        let controls = UIStackView(arrangedSubviews: [backButton, playButton,
        ↪  forwardButton])
        controls.distribution = .equalSpacing

        let stack = UIStackView(arrangedSubviews: [coverView, titleLabel, artistLabel,
        ↪  slider, times, controls])
        stack.axis = .vertical
        stack.spacing = 12
        stack.setCustomSpacing(24, after: coverView)
        stack.setCustomSpacing(24, after: artistLabel)
        stack.setCustomSpacing(24, after: times)
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)

        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪  constant: 40),
```

```
            stack.leadingAnchor.constraint(equalTo: view.layoutMarginsGuide.leadingAnchor,
↪   constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪   view.layoutMarginsGuide.trailingAnchor, constant: -16),
            coverView.heightAnchor.constraint(equalTo: coverView.widthAnchor),
        ])
    }

    private func makeControlButton(symbol: String, label: String,
                                   action: @escaping () -> Void) -> UIButton {
        let button = UIButton(type: .system)
        button.setImage(UIImage(systemName: symbol), for: .normal)
        button.setPreferredSymbolConfiguration(UIImage.SymbolConfiguration(pointSize: 28),
                                               forImageIn: .normal)
        button.accessibilityLabel = label
        button.addAction(UIAction { _ in action() }, for: .touchUpInside)
        return button
    }

    private func setupActions() {
        slider.accessibilityLabel = "Позиция в треке"
        slider.addTarget(self, action: #selector(scrubbingStarted), for: .touchDown)
        slider.addTarget(self, action: #selector(scrubbingChanged), for: .valueChanged)
        slider.addTarget(self, action: #selector(scrubbingEnded),
                         for: [.touchUpInside, .touchUpOutside, .touchCancel])
    }

    @objc private func scrubbingStarted() {
        isUserScrubbing = true
    }

    @objc private func scrubbingChanged() {
        let duration = MusicPlayer.shared.state.duration
        let seconds = TimeInterval(slider.value) * duration
        elapsedLabel.text = formatPlaybackTime(seconds)
        remainingLabel.text = "−" + formatPlaybackTime(duration - seconds)
    }

    @objc private func scrubbingEnded() {
        isUserScrubbing = false
        let seconds = TimeInterval(slider.value) * MusicPlayer.shared.state.duration
        MusicPlayer.shared.seek(to: seconds)
    }

    private func apply(state: MusicPlayer.State) {
        coverView.apply(state.current)
        titleLabel.text = state.current?.title ?? "Ничего не играет"
        artistLabel.text = state.current?.artist
```

```
        let symbol = state.isPlaying ? "pause.circle.fill" : "play.circle.fill"
        playButton.setImage(UIImage(systemName: symbol), for: .normal)
        playButton.accessibilityLabel = state.isPlaying ? "Пауза" : "Играть"

        guard !isUserScrubbing else { return }
        slider.value = state.duration > 0 ? Float(state.progress / state.duration) : 0
        elapsedLabel.text = formatPlaybackTime(state.progress)
        remainingLabel.text = "−" + formatPlaybackTime(state.duration - state.progress)
    }
}
```

Перемотка идёт в три фазы. Все три события — из `UIControl.Event`, набора флагов (`OptionSet`), который ползунок наследует от `UIControl`; поэтому их можно перечислить массивом в одном `addTarget`, как `[.touchUpInside, .touchUpOutside, .touchCancel]`.

1. **`touchDown`** — палец лёг на ползунок, `isUserScrubbing = true`. С этого момента `apply(state:)` не трогает ползунок и подписи времени (строка `guard !isUserScrubbing`). Иначе четыре раза в секунду плеер возвращал бы ползунок в текущую позицию, и он прыгал бы между пальцем и музыкой.
2. **`valueChanged`** — палец двигается. Показываем **будущее** время в подписях, но плеер не перематываем: перемотка на каждое движение дёргала бы звук и сеть.
3. **`touchUpInside` / `touchUpOutside` / `touchCancel`** — палец отпущен (над ползунком, мимо него, или касание прервал, например, входящий звонок). Снимаем флаг и делаем один настоящий `seek(to:)`. Если забыть `touchCancel`, флаг мог бы остаться `true` навсегда, и ползунок перестал бы двигаться.

Время на числах. Трек 9 секунд, ползунок на 0,6: 0,6 × 9 = 5,4 секунды. Слева «0:05», справа осталось 9 − 5,4 = 3,6 — «-0:03».

Моноширинные цифры. В обычном шрифте цифра «1» уже «8», и при смене «0:11» на «0:18» подпись ширится и дёргается. `monospacedDigitSystemFont` делает все цифры одной ширины.

`coverView.heightAnchor == widthAnchor` — обложка всегда квадратная: ширину задаёт стек, высота равна ей.

`UIView()` посередине стека `times` — растягивающаяся «распорка». У пустого `UIView` нет собственного размера: `intrinsicContentSize` возвращает `UIView.noIntrinsicMetric` (это −1) по обеим осям, тогда как у подписей он равен размеру текста. Стек с распределением `.fill` отдаёт лишнее место тому, кому нечего «обнимать», — распорке, и подписи остаются прижатыми к краям.

> **Упражнение 17.2.** Добавь в плеер метод `jump(by:)`, который перематывает на заданное число секунд вперёд или назад, не выходя за начало и конец трека, и две кнопки «−15» и «+15» на экране «Сейчас играет» (SF Symbols `gobackward.15` и `goforward.15`). Проверка: на 12-секундном треке «+15» с 3-й секунды приводит в конец трека, а не за него; «−15» с 5-й секунды — в начало. Решение — в конце главы.

17.12 Автопереход и чужие паузы

Автопереход к очередному треку уже встроен в `load(_:)`: наблюдатель `didPlayToEndTimeNotification` вызывает `skipForward()`. Трек закончился — играет очередной, после последнего — снова первый.

Второй случай, о котором легко забыть, — пауза, которую поставили без нас. Входящий звонок, будильник, Siri, отключённые наушники: система останавливает воспроизведение сама, и наш `pause()` не вызывается. Поэтому `handleTick` сверяет `isPlaying` с `player.timeControlStatus` (17.6). Наблюдатель времени срабатывает и при остановке воспроизведения, так что кнопка сменится на «играть» сразу.

Возобновлять ли музыку после звонка — отдельное решение. Система присылает `AVAudioSession.interrupti` с флагом «можно продолжить» (`.shouldResume`). Приложения-плееры обычно продолжают, если флаг есть. Мы это пропустили, см. 17.15.

17.13 Фоновое воспроизведение — Background Modes

Пользователь включил трек и заблокировал экран. Чтобы звук не оборвался, одной категории `.playback` мало (17.3). Приложению нужен **фоновый режим** — разрешение системы продолжать работу в фоне ради конкретной задачи.

В Xcode 26: выбери цель приложения → вкладка **Signing & Capabilities** → кнопка **+ Capability** → **Background Modes** → отметь **Audio, AirPlay, and Picture in Picture**. Capability («возможность») — функция системы, которую приложение явно включает в настройках проекта. Xcode допишет в Info.plist ключ:

```
<key>UIBackgroundModes</key>
<array>
    <string>audio</string>
</array>
```

Этот режим разрешает жить в фоне, **пока играет звук**. Поставил на паузу — через некоторое время система приостановит приложение, как обычно. Правило App Store Review Guidelines 2.5.4 разрешает фоновые режимы только по прямому назначению: приложение с режимом `audio` должно действительно воспроизводить звук в фоне, иначе его могут отклонить.

Проверять фон удобнее на настоящем устройстве: включи трек, заблокируй экран — музыка продолжает играть. Уберёшь галочку — музыка замолчит при блокировке.

Кнопки управления на экране блокировки при этом **не появятся** сами: для них нужны `MPNowPlayingInfoCenter` (название, обложка, позиция) и `MPRemoteCommandCenter` (реакция на play/pause/перемотку с экрана блокировки и наушников). Это в списке пропущенного.

17.14 Бытовая аналогия

`MusicPlayer.shared` — **диджей за пультом**. Он знает, какая пластинка крутится, на какой секунде и играет ли она. Команды `play`, `pause`, `skipForward` — кнопки пульта.

Подписчики — **табло в зале**, на которых написано «сейчас играет». Табло подключены к пульту: пластинка сменилась — надписи обновились везде. Сняли табло со стены (экран закрылся) — провод от него просто перестаёт что-то значить, диджею ничего отключать не нужно (слабая ссылка).

Аудиосессия — **договор с управляющим клуба**: «у меня главная программа вечера, не выключайте звук, даже когда в зале просят тишины». А фоновый режим — **пропуск**, по которому можно играть и после закрытия зала.

Ползунок — **виниловый диск под пальцем**: взялся — музыка не мешает, крутишь — видишь, куда попадёшь, отпустил — играет с этого места. Щелчки — как засечки на ручке громкости старого усилителя.

17.15 Что мы пропустили

- **Экран блокировки и Пункт управления** — `MPNowPlayingInfoCenter` и `MPRemoteCommandCenter`. Без них в фоне музыка играет, но управлять ей с экрана блокировки нельзя.
- **Прерывания** — `AVAudioSession.interruptionNotification`: продолжить воспроизведение после звонка, если система разрешает.
- **Смена устройства вывода** — `AVAudioSession.routeChangeNotification`: выдернули наушники — принято ставить на паузу. Системный плеер так и делает; наш тоже встанет, но мы не подписаны на само событие.
- **AirPlay** — `AVRoutePickerView`, кнопка выбора колонки или телевизора.
- **Плавный переход между треками (crossfade)** — затухание одного трека одновременно с нарастанием другого; нужны два плеера.
- **Эквалайзер и визуализация** — `AVAudioEngine` с узлами обработки звука.
- **Тексты песен** с синхронизацией по времени (формат LRC).

Упражнение 17.3. Запусти «Музыкальный плеер». Проверь: (1) тап по треку — звук пошёл, внизу появился mini-player, у строки трека значок динамика, таблица укоротилась над mini-player; (2) тап по mini-player — открылся лист с «ручкой»; (3) веди ползунок пальцем — подписи времени меняются, звук не прыгает, а после отпускания играет с новой позиции; на iPhone ощущаются щелчки (в симуляторе вибрации нет); (4) дождись конца трека — сам включится очередной; (5) на устройстве с включённым Background Mode заблокируй экран — музыка продолжает играть.

Ответы к упражнениям

Упражнение 17.1.

```swift
import Foundation

func formatLongPlaybackTime(_ seconds: TimeInterval) -> String {
    guard seconds.isFinite, seconds > 0 else { return "0:00" }
    let total = Int(seconds.rounded(.down))
    let hours = total / 3600
    let minutes = (total % 3600) / 60
    let secs = total % 60
```

```
    if hours > 0 {
        return String(format: "%d:%02d:%02d", hours, minutes, secs)
    }
    return String(format: "%d:%02d", minutes, secs)
}
```

На 3725 секундах: часов 3725 / 3600 = 1; остаток 3725 % 3600 = 125 секунд, из них минут 125 / 60 = 2; секунд 3725 % 60 = 5. Итог `"1:02:05"`. Для 75 секунд часов 0 — работает короткая ветка, `"1:15"`.

Упражнение 17.2. Метод в плеере — отдельным расширением:

```
import AVFoundation

extension MusicPlayer {
    func jump(by delta: TimeInterval) {
        guard state.current != nil else { return }
        let target = min(max(state.progress + delta, 0), state.duration)
        seek(to: target)
    }
}
```

`max(..., 0)` не пускает раньше начала, `min(..., duration)` — дальше конца. 12-секундный трек, 3-я секунда, +15: 3 + 15 = 18, `min(18, 12)` = 12 — конец трека, и сработает автопереход. 5-я секунда, −15: 5 − 15 = −10, `max(-10, 0)` = 0 — начало.

Кнопки — в `setupLayout()` экрана «Сейчас играет», рядом с остальными:

```
let backJump = makeControlButton(symbol: "gobackward.15", label: "Назад на 15 секунд") {
    MusicPlayer.shared.jump(by: -15)
}
let forwardJump = makeControlButton(symbol: "goforward.15", label: "Вперёд на 15 секунд")
 ↪ {
    MusicPlayer.shared.jump(by: 15)
}
let controls = UIStackView(arrangedSubviews: [backJump, backButton, playButton,
 ↪ forwardButton, forwardJump])
```

Замени этим объявление `controls` — кнопок станет пять, и `.equalSpacing` сам расставит их с равными промежутками.

Что мы выучили

- `AVPlayer` — для звука по сети, `AVAudioPlayer` — для файлов на устройстве.
- Один `AVPlayer` на приложение, трек меняется через `replaceCurrentItem(with:)`; наблюдатель времени ставится один раз.
- Аудиосессия с категорией `.playback`: звук не глушится переключателем «Без звука». Активируем её при первом «играть», а не при запуске.

- Игра в фоне = `.playback` **и** Background Mode «Audio, AirPlay, and Picture in Picture» (`UIBackgroundModes` → `audio`).
- `CMTime` — время как дробь (`150/600` = 0,25 с). Замыкания AVFoundation возвращаем на главный поток через `queue: .main` + `MainActor.assumeIsolated`.
- Подписка со слабой ссылкой на владельца: закрытый экран выпадает из списка сам, отписываться не нужно.
- `timeControlStatus` показывает, играет ли плеер в действительности, — так кнопка не врёт после звонка.
- Переход по кругу: `(index + offset + count) % count`.
- Уведомление о конце трека — `didPlayToEndTimeNotification` с `object: item`, старое наблюдение снимаем.
- `UISlider → UIControl → UIView`: target-action и события `UIControl.Event`; программная установка `value` action'ы не рассылает.
- Перемотка ползунком: `touchDown` → флаг, `valueChanged` → только подписи, `touchUp/touchCancel` → один `seek`.
- `layoutMargins` обычного view — 8 точек со всех сторон; 16–20 по бокам только у корневого `view` контроллера.
- Тактильные засечки: 20 участков по 5%, щелчок при смене участка, `prepare()` при касании.
- Скрытый элемент `UIStackView` не занимает места — mini-player появляется без ручных ограничений.

Apple Developer Documentation

- AVPlayer — плеер для локальных и сетевых адресов.
- AVPlayerItem — конкретный трек или видео, который играет `AVPlayer`.
- AVAudioPlayer — проигрыватель файлов и данных на устройстве.
- AVAudioSession — аудиосессия приложения: категория, режим, активация.
- AVAudioSession.Category.playback — категория для приложений, где воспроизведение — главная функция.
- Configuring your app for media playback — статья Apple про категорию `.playback`, активацию сессии и фоновый режим.
- UIBackgroundModes — ключ Info.plist с фоновыми режимами, значение `audio`.
- addPeriodicTimeObserver(forInterval:queue:using:) — периодический вызов по ходу воспроизведения.
- didPlayToEndTimeNotification — уведомление об окончании трека.
- CMTime — время в виде дроби `value / timescale`.
- MPNowPlayingInfoCenter — информация «Сейчас играет» на экране блокировки.
- MPRemoteCommandCenter — команды с экрана блокировки и наушников.
- UISlider — ползунок, основа `HapticSlider`.
- UIControl.Event — события контролов: `.touchDown`, `.valueChanged`, `.touchUpInside` и другие.
- UIView.layoutMargins — внутренние поля view; по умолчанию 8 точек, у корневого view контроллера — системные минимальные.
- Attaching gesture recognizers to UIKit controls — почему тап по кнопке не достаётся жесту на родительском view.
- UIImpactFeedbackGenerator — тактильный «удар» для засечек.
- UISheetPresentationController — лист с фиксаторами высоты (iOS 15+).

- HIG: Playing audio — как приложение должно вести себя со звуком: «Без звука», прерывания, фон.

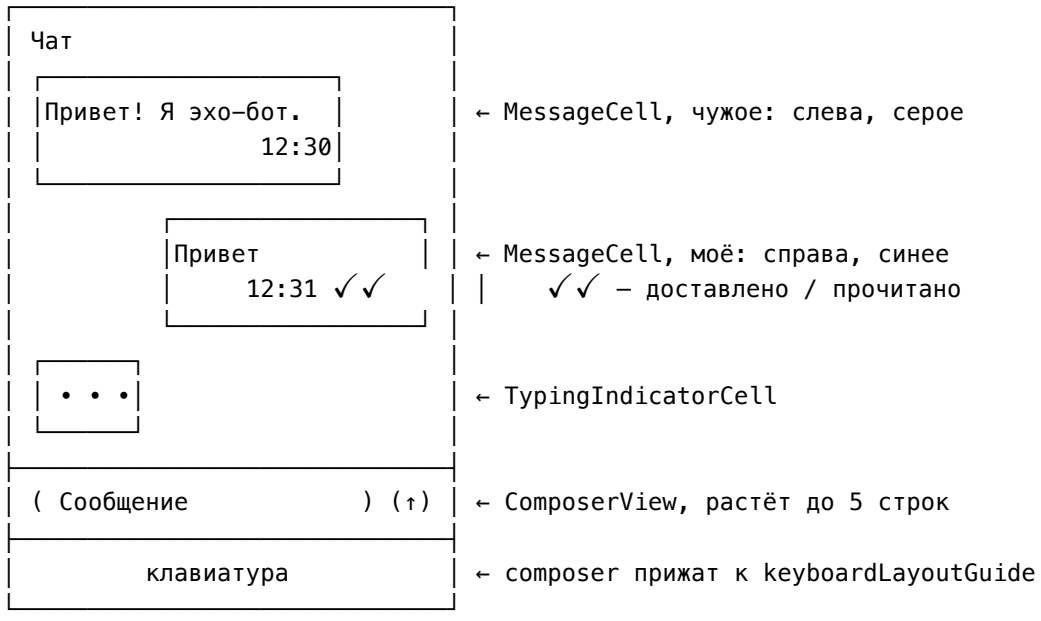
→ Глава 18. Chat — двусторонние ячейки, keyboardLayoutGuide, typing indicator

Глава 18. Chat — двусторонние ячейки, keyboardLayoutGuide, typing indicator

Чат — экран с одной из самых хитрых вёрсток в UIKit:

- сообщения разной высоты и с разных сторон: мои справа, чужие слева;
- сверху список сообщений, снизу поле ввода (composer);
- клавиатура поднимается и опускается — поле ввода должно ехать вместе с ней, а последние сообщения оставаться на виду;
- сообщения добавляются на лету, список надо докручивать до низа;
- статусы: отправляется → доставлено → прочитано;
- индикатор «печатает…» от собеседника.

Всё это собираем в этой главе. Собеседник — эхо-бот: ты отправляешь сообщение, он через секунду-две отвечает.



Файлы:

`Apps/Chat/`

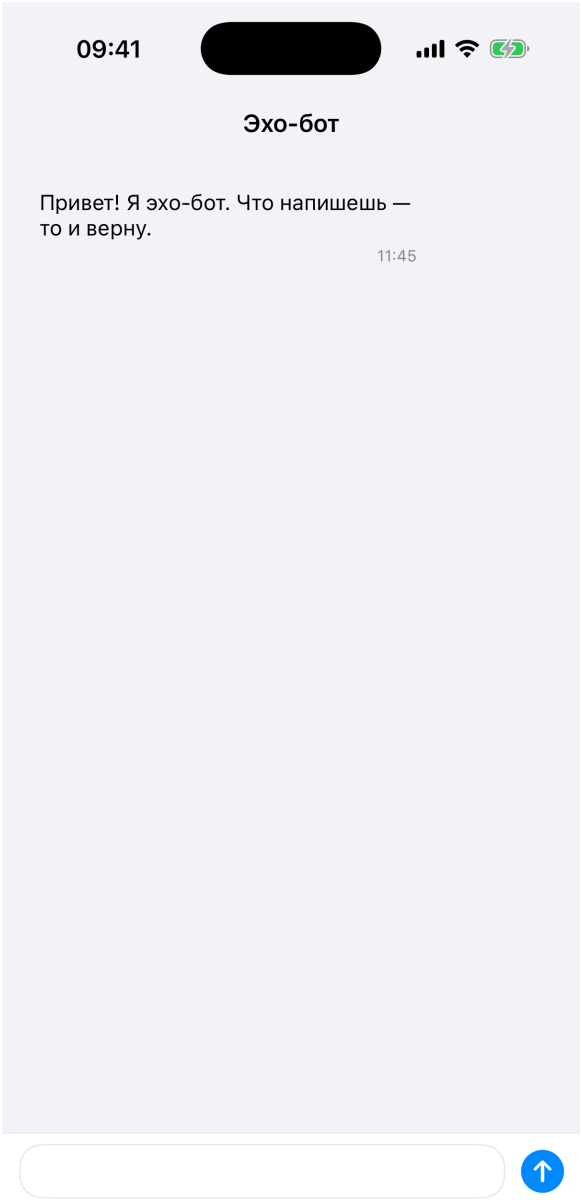


Рис. 14: Чат с эхо-ботом

```
├── ChatMessage.swift          ← модель сообщения
├── ChatBot.swift              ← эхо-бот с задержкой
├── MessageCell.swift          ← «пузырь» сообщения
├── TypingIndicatorCell.swift  ← три прыгающие точки
├── ComposerView.swift         ← поле ввода + кнопка
└── ChatViewController.swift   ← экран: таблица + composer + логика
```

> **Режим сборки** — как во введении: Swift 6, `Default Actor Isolation = MainActor`, iOS 15+. Весь код главы живёт на главном потоке; фоновая работа здесь только одна — ожидание ответа бота, и она уходит в `Task`.

18.1 Модель сообщения

```swift
import Foundation

struct ChatMessage: Hashable {
    enum Author { case me, them }
    enum Status { case sending, sent, delivered, read }

    let id: UUID
    let author: Author
    var text: String
    let date: Date
    var status: Status

    init(id: UUID = UUID(), author: Author, text: String,
         date: Date = Date(), status: Status = .sent) {
        self.id = id
        self.author = author
        self.text = text
        self.date = date
        self.status = status
    }
}
```

Минимум полей: id, кто автор, текст, время и статус.

`id: UUID` — уникальный номер сообщения. Искать сообщение по позиции в массиве ненадёжно: пока ждём ответа сервера, в массив могут добавиться другие сообщения. По `id` его всегда можно найти (так и сделаем в 18.3).

`Author` — чьё сообщение: `.me` рисуется справа, `.them` — слева.

`Status` — четыре стадии жизни моего сообщения:

- **`sending`** — показано в интерфейсе сразу, но сервер его ещё не подтвердил;
- **`sent`** — сервер принял (или сообщение сохранено локально);
- **`delivered`** — дошло до собеседника;
- **`read`** — собеседник открыл и прочитал.

В мессенджерах это обычно галочки: одна ✓ — отправлено, две ✓✓ — доставлено, выделенные ✓✓ — прочитано. У нас так же (18.9).

Явный `init` со значениями по умолчанию позволяет писать коротко: `ChatMessage(author: .me, text: "Привет")` — id, дата и статус подставятся сами.

18.2 ChatBot — эхо с задержкой

```
import Foundation

enum ChatBot {
    static func reply(to message: String) async -> String {
        let delay = UInt64.random(in: 800_000_000...1_800_000_000)   // 0,8–1,8 с
        try? await Task.sleep(nanoseconds: delay)
        let endings = ["", " :)", " Интересно.", " Расскажи ещё!"]
        return "Эхо: \(message)\(endings.randomElement() ?? "")"
    }
}
```

Бот возвращает «Эхо: твой текст» и случайное окончание. Задержка случайная, от 0,8 до 1,8 секунды: `Task.sleep` считает в наносекундах, в одной секунде их миллиард (1 000 000 000), поэтому 800 000 000 — это 0,8 секунды. Подчёркивания в числе — просто разделители разрядов для читаемости. `Task.sleep(nanoseconds:)` работает с iOS 13; вариант `Task.sleep(for: .seconds(1))` появился только в iOS 16.

Случайная задержка нужна, чтобы увидеть индикатор «печатает» и проверить, что ответы приходят в любом порядке и ничего не ломают.

Ответы — обычный текст, без эмодзи: в PDF-версии книги и в части шрифтов эмодзи превращаются в квадратики.

18.3 Optimistic UI — показать сразу, подтвердить потом

Optimistic UI («оптимистичный интерфейс») — показываем результат действия сразу, как будто сервер уже согласился, и тихо поправляем, если что-то пошло не так. Пользователь нажал «отправить» — его сообщение появляется мгновенно, с пометкой «отправляется».

```
import UIKit

extension ChatViewController {
    func handleSend(_ text: String) {
        let trimmed = text.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else { return }

        let message = ChatMessage(author: .me, text: trimmed, status: .sending)
        messages.append(message)
        tableView.insertRows(at: [IndexPath(row: messages.count - 1, section: 0)], with:
↪   .bottom)
```

```
        scrollToBottom(animated: true)

        Task { [weak self] in
            try? await Task.sleep(nanoseconds: 250_000_000)   // «сервер» подтвердил за
            ↪ 0,25 с
            self?.update(messageID: message.id, to: .delivered)
            self?.showTyping()
            let reply = await ChatBot.reply(to: trimmed)
            self?.receive(reply)
        }
    }

    func update(messageID: UUID, to status: ChatMessage.Status) {
        guard let row = messages.firstIndex(where: { $0.id == messageID }) else { return }
        messages[row].status = status
        tableView.reloadRows(at: [IndexPath(row: row, section: 0)], with: .none)
    }
}
```

Этот код лежит в `ChatViewController.swift`, в том же файле, что и сам класс (он будет в 18.11). Поэтому расширение видит его `private`-свойства: `messages`, `tableView` и другие.

По шагам:

1. **`trimmingCharacters(in: .whitespacesAndNewlines)`** — убираем пробелы и переносы по краям. Сообщение из одних пробелов не отправляем.
2. **Сразу** добавляем сообщение в массив со статусом `.sending` и вставляем строку в таблицу. `insertRows(... with: .bottom)` — строка въезжает снизу, а не появляется рывком, как при `reloadData()`.
3. **Через 0,25 секунды** «сервер» подтверждает, статус становится `.delivered`. Строку ищем по `id` (`firstIndex(where:)`), а не по номеру, который был при отправке.
4. Показываем «печатает…», ждём бота, добавляем ответ (18.4).

`[weak self]` в задаче — если пользователь закрыл чат, пока бот «думает», экран освободится, а оставшиеся вызовы через `self?.` просто не выполнятся.

Без optimistic UI пользователь нажал «отправить» — и четверть секунды (а на плохой сети — несколько секунд) ничего не происходит. Он решает, что не сработало, жмёт ещё раз — и получает дубль сообщения.

> **Частая ошибка.** Искать строку по номеру `optimisticIndex`, сохранённому при отправке, а после ответа бота вставлять и удалять строки в расчёте, что второе сообщение за это время не появится. Отправь два сообщения быстрее, чем за секунду, — и таблица падает с ошибкой «invalid number of rows». Как этого избежать — в 18.4.

18.4 Индикатор «бот печатает»

Индикатор — это ещё одна строка таблицы, после последнего сообщения. Её нет в массиве `messages`, она «приклеена» к концу, пока бот готовит хотя бы один ответ.

```
import UIKit

extension ChatViewController {
    func showTyping() {
        pendingReplies += 1
        guard pendingReplies == 1 else { return }   // строка «печатает» уже есть
       tableView.insertRows(at: [IndexPath(row: messages.count, section: 0)], with: .fade)
        scrollToBottom(animated: true)
    }

    func receive(_ reply: String) {
        pendingReplies -= 1
        let typingRow = messages.count   // строка «печатает» стоит сразу за последним
        ↪ сообщением

        let readRows = messages.indices.filter {
            messages[$0].author == .me && (messages[$0].status == .sent ||
↪   messages[$0].status == .delivered)
        }
        for row in readRows { messages[row].status = .read }
        messages.append(ChatMessage(author: .them, text: reply, status: .read))

        tableView.performBatchUpdates {
            tableView.reloadRows(at: readRows.map { IndexPath(row: $0, section: 0) },
↪   with: .none)
            tableView.insertRows(at: [IndexPath(row: typingRow, section: 0)], with: .fade)
            if !isBotTyping {
                tableView.deleteRows(at: [IndexPath(row: typingRow, section: 0)], with:
↪   .fade)
            }
        }
        scrollToBottom(animated: true)
    }
}
```

Счётчик вместо флага. Будь `isBotTyping` простым `Bool`, два быстрых сообщения — два ответа в пути — пытались бы вставить и удалить одну и ту же строку. `pendingReplies` считает, сколько ответов ещё в пути, а `isBotTyping` теперь вычисляется: «ответов в пути больше нуля». Строка «печатает» вставляется, только когда счётчик стал 1 (первый ответ пошёл), и удаляется, только когда стал 0 (последний пришёл).

Сколько строк в таблице. Таблица всегда спрашивает:

```
import UIKit

extension ChatViewController: UITableViewDataSource {
    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        messages.count + (isBotTyping ? 1 : 0)
    }
```

```
    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
     ↪  UITableViewCell {
        if isBotTyping && indexPath.row == messages.count {
            let cell = tableView.dequeueReusableCell(withIdentifier:
             ↪  TypingIndicatorCell.reuseID,
                                                     for: indexPath)
            (cell as? TypingIndicatorCell)?.startAnimating()
            return cell
        }
        let cell = tableView.dequeueReusableCell(withIdentifier: MessageCell.reuseID, for:
         ↪  indexPath)
        (cell as? MessageCell)?.configure(with: messages[indexPath.row])
        return cell
    }
}
```

Сообщений N — строк N, а пока бот печатает — N + 1, и последняя строка (номер N, считая с нуля) — индикатор.

Главное правило `insertRows`/`deleteRows`. После каждого изменения таблица проверяет арифметику: «было строк + вставлено − удалено» должно совпасть с тем, что теперь отвечает `numberOfRowsInSection`. Не совпало — приложение падает. Проверим `receive` на числах. До ответа: 3 сообщения + индикатор = 4 строки.

- **Ответов в пути больше не осталось** (`pendingReplies` стал 0). Сообщений стало 4, индикатора нет: 4 строки. В одном пакете (`performBatchUpdates`) удаляем старую строку 3 (индикатор) и вставляем новую строку 3 (ответ): 4 − 1 + 1 = 4. Сходится.
- **Ещё один ответ в пути** (`pendingReplies` стал 1). Сообщений 4 + индикатор = 5 строк. Вставляем ответ на место 3, индикатор сам съезжает на строку 4: 4 + 1 = 5. Сходится.

Внутри `performBatchUpdates` номера для удаления и перезагрузки считаются по **старому** состоянию таблицы, а для вставки — по **новому**. Поэтому одна и та же строка 3 и удаляется (старый индикатор), и вставляется (новый ответ).

Прочитано. Когда бот отвечает, он «прочитал» мои сообщения, которые уже дошли (`.sent` или `.delivered`). Сообщения в статусе `.sending` не трогаем: сервер их ещё не принял, прочитать их нельзя. Их строки перезагружаем в том же пакете — галочки поменяют цвет вместе с появлением ответа.

> **Упражнение 18.1.** Не запуская приложение, проследи счётчик `pendingReplies` и число строк-индикаторов, если отправить три сообщения подряд с интервалом в полсекунды. Потом проверь в симуляторе. Ответ — в конце главы.

18.5 TypingIndicatorCell — три точки в «пузыре»

```
import UIKit

final class TypingIndicatorCell: UITableViewCell {
```

```
    static let reuseID = "TypingIndicatorCell"

    private let bubble = UIView()
    private let dots = (0..<3).map { _ in UIView() }

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        backgroundColor = .clear

        bubble.backgroundColor = .secondarySystemBackground
        bubble.layer.cornerRadius = 16
        bubble.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(bubble)

        let stack = UIStackView(arrangedSubviews: dots)
        stack.spacing = 5
        stack.translatesAutoresizingMaskIntoConstraints = false
        bubble.addSubview(stack)

        for dot in dots {
            dot.backgroundColor = .secondaryLabel
            dot.layer.cornerRadius = 4
            dot.widthAnchor.constraint(equalToConstant: 8).isActive = true
            dot.heightAnchor.constraint(equalToConstant: 8).isActive = true
        }

        NSLayoutConstraint.activate([
            bubble.topAnchor.constraint(equalTo: contentView.topAnchor, constant: 4),
            bubble.bottomAnchor.constraint(equalTo: contentView.bottomAnchor, constant:
↪ -4),
            bubble.leadingAnchor.constraint(equalTo: contentView.leadingAnchor, constant:
↪ 12),
            stack.topAnchor.constraint(equalTo: bubble.topAnchor, constant: 14),
            stack.bottomAnchor.constraint(equalTo: bubble.bottomAnchor, constant: -14),
            stack.leadingAnchor.constraint(equalTo: bubble.leadingAnchor, constant: 14),
           stack.trailingAnchor.constraint(equalTo: bubble.trailingAnchor, constant: -14),
        ])

        isAccessibilityElement = true
        accessibilityLabel = "Бот печатает"
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func startAnimating() {
        let now = CACurrentMediaTime()
        for (index, dot) in dots.enumerated() {
```

```
            dot.layer.removeAllAnimations()
            let animation = CABasicAnimation(keyPath: "transform.translation.y")
            animation.fromValue = 0
            animation.toValue = -5
            animation.duration = 0.5
            animation.autoreverses = true
            animation.repeatCount = .infinity
            animation.beginTime = now + Double(index) * 0.15
            dot.layer.add(animation, forKey: "bounce")
        }
    }
}
```

Ячейка таблицы — это `UIView`. `UITableViewCell` наследует `UIView` напрямую, без промежуточного класса (у коллекции между ячейкой и `UIView` стоит `UICollectionReusableView` — глава 16, раздел 16.6). Инициализатор у неё свой — `init(style:reuseIdentifier:)`, его и переопределяем. Своё содержимое кладём в `contentView`: это документированное место, которое ячейка сдвигает сама при переходе в режим редактирования и держит поверх фоновых view. `backgroundColor = .clear` ставим самой ячейке, а не `contentView`: ячейка — отдельный view со своим фоном, и белый квадрат под пузырём рисовала бы она. Полная карта ячеек — глава 0, раздел 0.5.

Точка — это квадрат 8×8 со скруглением 4. Скругление, равное половине стороны, превращает квадрат в круг.

Анимация. `CABasicAnimation` — анимация Core Animation: она меняет одно свойство слоя (`CALayer`, «холст», на котором рисуется вид) от одного значения к другому, и всю работу делает система, без нашего кода на каждом кадре. Для каждой точки:

- `keyPath: "transform.translation.y"` — сдвиг по вертикали;
- от 0 до −5 точек: в UIKit ось Y направлена вниз, поэтому минус — это **вверх** на 5 точек;
- `duration = 0.5` — подъём за полсекунды;
- `autoreverses = true` — потом обратно вниз, ещё полсекунды: полный «прыжок» занимает секунду;
- `repeatCount = .infinity` — бесконечно;
- `beginTime = now + index * 0.15` — первая точка стартует сразу, вторая через 0,15 секунды, третья через 0,3. Получается бегущая волна.

`CACurrentMediaTime()` — текущее время часов Core Animation в секундах. `beginTime` задаётся по этим часам: если написать просто `0.15`, это будет момент «0,15 секунды после включения устройства», давно прошедший, и сдвиг пропадёт. Время берём один раз до цикла (`now`), чтобы у всех трёх точек была одна точка отсчёта.

`removeAllAnimations()` — страховка на случай переиспользования ячейки. Сам по себе повторный `add(_:forKey: "bounce")` второй анимации не создаст: по документации `add(_:forKey:)` на слое живёт только одна анимация на каждый ключ, и новая с тем же ключом заменяет старую. Но так анимация начинается заново с чистого слоя, что бы на нём ни висело раньше, и правило «сброс при переиспользовании» видно в коде явно.

Доступность. Вся ячейка — один элемент для VoiceOver с подписью «Бот печатает». Без этого он пытался бы озвучить три пустых вида.

18.6 keyboardLayoutGuide — поле ввода едет за клавиатурой

Layout guide («направляющая») — невидимый прямоугольник, к которому можно привязывать ограничения Auto Layout, как к обычному виду. При этом это не view: `UILayoutGuide → NSObject`, у неё нет слоя, она не входит в дерево subviews и не получает касаний — только даёт якоря (anchors) и `layoutFrame` (глава 0, раздел 0.8). Самый известный пример — `safeAreaLayoutGuide`, безопасная зона экрана. В iOS 15 появился `view.keyboardLayoutGuide` (класс `UIKeyboardLayoutGuide → UITrackingLayoutGuide → UILayoutGuide`) — направляющая, которая совпадает с клавиатурой: поднимается вместе с ней и опускается обратно.

Одно ограничение — и поле ввода всегда стоит над клавиатурой:

```
composer.bottomAnchor.constraint(equalTo: view.keyboardLayoutGuide.topAnchor)
```

(Полный код вёрстки — в 18.11.) Когда клавиатура спрятана, верх направляющей совпадает с низом безопасной зоны (документация: `topAnchor` направляющей равен `bottomAnchor` у `safeAreaLayoutGuide`), и поле ввода стоит над полоской «домой». Когда клавиатура выезжает, направляющая поднимается с той же анимацией, с той же скоростью и кривой, что и клавиатура. Никаких уведомлений, наблюдателей и ручных расчётов.

В iOS 17 у направляющей появились настройки: `usesBottomSafeArea` (учитывать ли безопасную зону, когда клавиатура спрятана) и `keyboardDismissPadding`. Нам хватает поведения по умолчанию, которое одинаково с iOS 15.

Как делали до iOS 15. Для сравнения — старый способ (при iOS 15+ он не нужен):

```
NotificationCenter.default.addObserver(self,
                                       selector: #selector(keyboardWillChange),
                                   name: UIResponder.keyboardWillChangeFrameNotification,
                                       object: nil)

@objc func keyboardWillChange(_ note: Notification) {
    guard let frame = note.userInfo?[UIResponder.keyboardFrameEndUserInfoKey] as? CGRect
     ↪  else { return }
    let overlap = view.bounds.height - view.convert(frame, from: nil).origin.y
    composerBottomConstraint.constant = -max(0, overlap - view.safeAreaInsets.bottom)
    UIView.animate(withDuration: 0.25) { self.view.layoutIfNeeded() }
}
```

Уведомление приносит кадр клавиатуры в координатах экрана, его надо перевести в координаты своего вида, вычесть безопасную зону и вручную анимировать. На числах: экран 852 точки в высоту, клавиатура начинается с 516-й точки — перекрытие 852 − 516 = 336 точек; минус 34 точки безопасной зоны снизу — поднимаем поле на 302 точки. А ещё нужно вытащить из уведомления длительность и кривую анимации клавиатуры: с захардкоженными 0,25 секунды поле ввода чуть отстаёт от клавиатуры или обгоняет её. `keyboardLayoutGuide` делает всё это сам.

Интерактивное закрытие. В 18.11 у таблицы стоит `keyboardDismissMode = .interactive`: потяни список вниз — клавиатура поедет за пальцем, как в «Сообщениях». Документация описывает направляющую как «отслеживающую положение клавиатуры», поэтому поле ввода опуска-

ется вслед за ней. Это движение управляется пальцем, а не анимацией, и проверять его удобнее на устройстве — в упражнении 18.3.

18.7 ComposerView — растущее поле ввода

```
import UIKit

final class ComposerView: UIView {
    var onSend: ((String) -> Void)?

    private let textView = UITextView()
    private let placeholderLabel = UILabel()
    private let sendButton = UIButton(type: .system)
    private var heightConstraint: NSLayoutConstraint!

    private let minHeight: CGFloat = 38
    private let maxHeight: CGFloat = 120

    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .systemBackground

        let separator = UIView()
        separator.backgroundColor = .separator

        textView.font = .preferredFont(forTextStyle: .body)
        textView.adjustsFontForContentSizeCategory = true
        textView.backgroundColor = .secondarySystemBackground
        textView.layer.cornerRadius = 18
        textView.textContainerInset = UIEdgeInsets(top: 8, left: 8, bottom: 8, right: 8)
        textView.isScrollEnabled = false
        textView.delegate = self
        textView.accessibilityLabel = "Сообщение"

        placeholderLabel.text = "Сообщение"
        placeholderLabel.font = textView.font
        placeholderLabel.textColor = .placeholderText
        placeholderLabel.isAccessibilityElement = false

        sendButton.setImage(UIImage(systemName: "arrow.up.circle.fill"), for: .normal)
        sendButton.setPreferredSymbolConfiguration(UIImage.SymbolConfiguration(pointSize:
↪  30),
                                                   forImageIn: .normal)
        sendButton.accessibilityLabel = "Отправить"
        sendButton.isEnabled = false
        sendButton.addAction(UIAction { [weak self] _ in self?.sendTapped() }, for:
↪  .touchUpInside)
```

```
        [separator, textView, placeholderLabel, sendButton].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            addSubview($0)
        }

        heightConstraint = textView.heightAnchor.constraint(equalToConstant: minHeight)

        NSLayoutConstraint.activate([
            separator.topAnchor.constraint(equalTo: topAnchor),
            separator.leadingAnchor.constraint(equalTo: leadingAnchor),
            separator.trailingAnchor.constraint(equalTo: trailingAnchor),
            separator.heightAnchor.constraint(equalToConstant: 1),

            textView.topAnchor.constraint(equalTo: topAnchor, constant: 8),
            textView.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -8),
            textView.leadingAnchor.constraint(equalTo: layoutMarginsGuide.leadingAnchor),
            heightConstraint,

            placeholderLabel.leadingAnchor.constraint(equalTo: textView.leadingAnchor,
↪   constant: 13),
            placeholderLabel.centerYAnchor.constraint(equalTo: textView.centerYAnchor),

            sendButton.leadingAnchor.constraint(equalTo: textView.trailingAnchor,
↪   constant: 8),
            sendButton.trailingAnchor.constraint(equalTo:
↪   layoutMarginsGuide.trailingAnchor),
            sendButton.bottomAnchor.constraint(equalTo: textView.bottomAnchor),
            sendButton.widthAnchor.constraint(equalToConstant: 38),
            sendButton.heightAnchor.constraint(equalToConstant: 38),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private func sendTapped() {
        onSend?(textView.text)
        textView.text = ""
        textDidChange()
    }

    fileprivate func textDidChange() {
        let isEmpty = textView.text.trimmingCharacters(in:
        ↪  .whitespacesAndNewlines).isEmpty
        sendButton.isEnabled = !isEmpty
        placeholderLabel.isHidden = !textView.text.isEmpty

        let fitting = textView.sizeThatFits(CGSize(width: textView.bounds.width,
```

```
                                                    height: .greatestFiniteMagnitude))
        heightConstraint.constant = min(max(minHeight, fitting.height), maxHeight)
        // Дошли до потолка — включаем прокрутку внутри поля, иначе нижние строки не видно.
        textView.isScrollEnabled = fitting.height > maxHeight
    }
}

extension ComposerView: UITextViewDelegate {
    func textViewDidChange(_ textView: UITextView) {
        textDidChange()
    }
}
```

Разберём.

- **`UITextView`, а не `UITextField`.** Текстовое поле однострочное, а сообщение может быть длинным, с переносами строк. Классы эти — дальние родственники: `UITextField → UIControl → UIView`, а `UITextView → UIScrollView → UIView`. Текстовое поле — контрол, у него есть `addAction(_:for:)` и событие `.editingChanged`. Текстовый вид — scroll view, не контрол: ни `addTarget`, ни `addAction` у него нет (проверено рантаймом), об изменениях он сообщает делегату (`textViewDidChange`) или уведомлением `UITextView.textDidChangeNotification`. Отсюда и `delegate = self` ниже (глава 0, разделы 0.4 и 0.5).
- **Подсказка «Сообщение».** У `UITextView`, в отличие от `UITextField`, нет свойства `placeholder`. Поэтому кладём поверх серый `UILabel` и прячем его, как только появился текст. Цвет `.placeholderText` — системный цвет подсказок, сам подстраивается под тёмную тему.
- **`sizeThatFits`** — «какой размер тебе нужен при такой ширине?». Ширину передаём текущую, высоту — бесконечную (`.greatestFiniteMagnitude`, самое большое число `CGFloat`): «сколько угодно в высоту». У `UITextView` этот метод возвращает размер текста независимо от прокрутки — мы проверили на симуляторе: для восьми строк он дал высоту 192 и при `isScrollEnabled = true`, и при `false`. Сам view он не меняет (документация: «does not actually resize view»), высоту мы ставим ограничением `heightConstraint`.
- **`isScrollEnabled = false`** нужен не для измерения, а для поведения: пока текст помещается, полю незачем прокручиваться внутри себя, и оно живёт как обычный растущий view. Заодно у него появляется `intrinsicContentSize` (с прокруткой он равен `noIntrinsicMetric`, −1, без прокрутки — размеру текста; тоже проверено), так что в другой вёрстке поле могло бы расти и без явного ограничения высоты.
- **Высота на числах.** `min(max(38, нужная), 120)`: не меньше 38 точек (одна строка шрифта body с отступами по 8 сверху и снизу) и не больше 120 (примерно пять строк). Нужно 20 — будет 38; нужно 80 — 80; нужно 200 — 120.
- **Прокрутка на потолке.** Когда текст выше 120 точек, включаем `isScrollEnabled`, и внутри поля можно прокрутить к нижним строкам. Без этого шестая строка просто прячется.
- **Кнопка отправки** выключена, пока текст пустой или из одних пробелов. Выключенная системная кнопка сама становится серой.
- **`textContainerInset`** — отступы текста внутри поля. По умолчанию они 8 сверху и снизу и 0 по бокам (проверено), поэтому без своих отступов текст прилипает к скруглённым краям слева и справа.

- **`addAction(UIAction { [weak self] ... })`** — обработчик замыканием вместо `@objc`-метода (iOS 14+). `[weak self]` разрывает кольцо «кнопка → action → замыкание → composer → кнопка».

Как высота передаётся наверх. Мы меняем `heightConstraint` у `textView`. Сам `ComposerView` высоты не задаёт: его верх и низ привязаны к `textView` с отступами 8. Выросло поле — вырос composer, а таблица над ним (её низ привязан к верху composer) стала ниже. Цепочка ограничений делает всё сама.

> **Упражнение 18.2.** Ограничь длину сообщения 1000 символами: при попытке вставить или напечатать больше лишнее не должно попадать в поле. Подсказка: у `UITextViewDelegate` есть метод `textView(_:shouldChangeTextIn:replacementText:)`. Проверка: вставь в поле текст из 1500 символов — поле не изменится. Решение — в конце главы.

18.8 MessageCell — пузырь слева или справа

```
import UIKit

final class MessageCell: UITableViewCell {
    static let reuseID = "MessageCell"

    private static let timeFormatter: DateFormatter = {
        let formatter = DateFormatter()
        formatter.timeStyle = .short
        formatter.dateStyle = .none
        return formatter
    }()

    private let bubble = UIView()
    private let messageLabel = UILabel()
    private let metaLabel = UILabel()
    private var leadingConstraint: NSLayoutConstraint!
    private var trailingConstraint: NSLayoutConstraint!

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        backgroundColor = .clear
        setupLayout()
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private func setupLayout() {
        bubble.layer.cornerRadius = 16
        messageLabel.numberOfLines = 0
        messageLabel.font = .preferredFont(forTextStyle: .body)
```

```
        messageLabel.adjustsFontForContentSizeCategory = true
        metaLabel.font = .preferredFont(forTextStyle: .caption2)
        metaLabel.adjustsFontForContentSizeCategory = true
        metaLabel.textAlignment = .right

        [bubble, messageLabel, metaLabel].forEach {
        ↪  $0.translatesAutoresizingMaskIntoConstraints = false }
        contentView.addSubview(bubble)
        bubble.addSubview(messageLabel)
        bubble.addSubview(metaLabel)

        leadingConstraint = bubble.leadingAnchor.constraint(equalTo:
↪  contentView.leadingAnchor, constant: 12)
        trailingConstraint = bubble.trailingAnchor.constraint(equalTo:
↪  contentView.trailingAnchor, constant: -12)

        NSLayoutConstraint.activate([
            bubble.topAnchor.constraint(equalTo: contentView.topAnchor, constant: 4),
            bubble.bottomAnchor.constraint(equalTo: contentView.bottomAnchor, constant:
↪  -4),
            bubble.widthAnchor.constraint(lessThanOrEqualTo: contentView.widthAnchor,
↪  multiplier: 0.75),

            messageLabel.topAnchor.constraint(equalTo: bubble.topAnchor, constant: 8),
            messageLabel.leadingAnchor.constraint(equalTo: bubble.leadingAnchor, constant:
↪  12),
            messageLabel.trailingAnchor.constraint(equalTo: bubble.trailingAnchor,
↪  constant: -12),

            metaLabel.topAnchor.constraint(equalTo: messageLabel.bottomAnchor, constant:
↪  2),
            metaLabel.leadingAnchor.constraint(greaterThanOrEqualTo: bubble.leadingAnchor,
↪  constant: 12),
            metaLabel.trailingAnchor.constraint(equalTo: bubble.trailingAnchor, constant:
↪  -12),
            metaLabel.bottomAnchor.constraint(equalTo: bubble.bottomAnchor, constant: -6),
        ])
    }

    func configure(with message: ChatMessage) {
        messageLabel.text = message.text
        let isMine = message.author == .me

        if isMine {
            bubble.backgroundColor = .systemBlue
            messageLabel.textColor = .white
            leadingConstraint.isActive = false
            trailingConstraint.isActive = true
```

```
        } else {
            bubble.backgroundColor = .secondarySystemBackground
            messageLabel.textColor = .label
            trailingConstraint.isActive = false
            leadingConstraint.isActive = true
        }

        let time = Self.timeFormatter.string(from: message.date)
        metaLabel.text = isMine ? "\(time) \(Self.glyph(for: message.status))" : time
        if isMine {
            metaLabel.textColor = message.status == .read ? .white :
↪   UIColor.white.withAlphaComponent(0.7)
        } else {
            metaLabel.textColor = .secondaryLabel
        }

        let author = isMine ? "Вы" : "Бот"
        accessibilityLabel = "\(author): \(message.text)"
        accessibilityValue = isMine ? "\(time), \(Self.spokenStatus(message.status))" :
↪   time
    }
}
```

(Методы `glyph(for:)` и `spokenStatus(_:)` — в 18.9.)

Два ограничения, активно одно. Пузырь должен прижиматься к правому краю для моих сообщений и к левому — для чужих. Создаём оба ограничения заранее (`leadingConstraint` — отступ 12 от левого края, `trailingConstraint` — от правого), а в `configure` включаем нужное и выключаем другое. Порядок в коде важен: **сначала выключаем** лишнее, потом включаем нужное. Если включены оба, система ограничений противоречива: «12 от левого края» и «12 от правого» вместе требуют ширину пузыря на 24 точки меньше ячейки, а третье ограничение говорит «не шире 75 %». Все три обязательные (приоритет 1000), выполнить их разом нельзя, и Auto Layout отбрасывает одно — какое именно, Apple не обещает. В нашей проверке на симуляторе пузырь остался у левого края и растянулся до 75 % ширины: не «на всю ширину», но и не по тексту. Ограничение добавляется в систему сразу при `isActive = true`, не дожидаясь прохода раскладки, поэтому даже мгновенный конфликт между двумя строками кода — лишний.

Ширина не больше 75%. `widthAnchor.constraint(lessThanOrEqualTo: ..., multiplier: 0.75)` — пузырь может быть уже, но не шире трёх четвертей ячейки. На экране 393 точки это 295 точек; длинное сообщение переносится на новые строки (`numberOfLines = 0` — без ограничения строк).

Пузырь по размеру текста. Ширину пузыря никто не задаёт жёстко: её определяет собственный размер текста (intrinsic content size — «сколько места нужно содержимому»). Короткое «Ок» даёт узкий пузырь, длинный абзац — широкий, но не шире 75%.

Высота ячейки складывается из ограничений: 4 сверху + 8 + высота текста + 2 + высота времени + 6 + 4 снизу. Таблица с `rowHeight = UITableView.automaticDimension` (18.11) сама посчитает её для каждого сообщения.

`DateFormatter` один на все ячейки. Не создавай форматтер при каждой настройке ячейки: создание `DateFormatter` — дорогая операция (он загружает правила локали), а `configure` вызывается при каждой прокрутке. `static let` создаётся один раз, лениво, при первом обращении. `timeStyle = .short` учитывает локаль устройства: «12:30» в России, «12:30 PM» в США.

Переиспользование. `configure` задаёт **всё**: текст, цвета, активные ограничения, подпись и данные для VoiceOver. Ячейка, которая была «моей», может прийти для чужого сообщения — и ни одно свойство не останется от прошлого.

Цвета и тёмная тема. `.systemBlue`, `.secondarySystemBackground`, `.label`, `.secondaryLabel` — системные динамические цвета: в тёмной теме они сами становятся подходящими оттенками. Белый текст на `.systemBlue` читается в обеих темах.

18.9 Статусы — галочки

```
import UIKit

extension MessageCell {
    static func glyph(for status: ChatMessage.Status) -> String {
        switch status {
        case .sending: return "…"
        case .sent: return "✓"
        case .delivered, .read: return "✓✓"
        }
    }

    static func spokenStatus(_ status: ChatMessage.Status) -> String {
        switch status {
        case .sending: return "отправляется"
        case .sent: return "отправлено"
        case .delivered: return "доставлено"
        case .read: return "прочитано"
        }
    }
}
```

Для моих сообщений под текстом — время и статус, для чужих — только время (чужие статусы нам не интересны).

- … — отправляется. Эмодзи песочных часов в PDF и на части шрифтов превращается в квадратик, поэтому берём обычный символ многоточия.
- ✓ — отправлено, ✓✓ — доставлено и прочитано.

Доставлено и прочитано отличаются **яркостью**: пока не прочитано, подпись полупрозрачная (70% непрозрачности — `withAlphaComponent(0.7)`), после прочтения — чисто белая. Красить «прочитано» в `.systemBlue` нельзя: синее на синем пузыре почти невидимо.

`spokenStatus` — то же самое словами для VoiceOver: галочки он прочитал бы как «галочка галочка». Метод отдельный, потому что на экране нужны значки, а в озвучке — слова.

18.10 Прокрутка к низу

```
import UIKit

extension ChatViewController {
    func scrollToBottom(animated: Bool) {
        let rows = tableView.numberOfRows(inSection: 0)
        guard rows > 0 else { return }
        let indexPath = IndexPath(row: rows - 1, section: 0)
        DispatchQueue.main.async { [weak self] in
            guard let self, indexPath.row < self.tableView.numberOfRows(inSection: 0) else
             ↪  { return }
            self.tableView.scrollToRow(at: indexPath, at: .bottom, animated: animated)
        }
    }
}
```

Сколько строк — спрашиваем у таблицы. `numberOfRows(inSection:)` отдаёт то, что таблица знает сейчас, с учётом индикатора. Считать это самим по `messages.count` и флагу — значит завести второй источник правды, который легко разъезжается с первым.

`DispatchQueue.main.async` — прокрутка откладывается до очередного прохода главного цикла (run loop — бесконечный цикл главного потока, который по очереди обрабатывает касания, таймеры и перерисовку). Сразу после `insertRows` таблица ещё не посчитала высоту новой строки, и прокрутка могла бы не доехать до конца. Через один проход раскладка уже свежая.

Повторная проверка внутри. За время ожидания строк могло стать меньше (например, индикатор удалили). Прокрутка к несуществующей строке — падение, поэтому перед `scrollToRow` проверяем ещё раз.

`scrollToRow(at:at:animated:)` с `.bottom` — прокрутить так, чтобы строка встала у нижнего края видимой области.

Клавиатура закрывает последние сообщения. Когда клавиатура выезжает, таблица становится ниже (её низ привязан к composer'у, а тот — к клавиатуре), но смещение прокрутки остаётся прежним — и последние сообщения уезжают под поле ввода. В `viewDidLayoutSubviews` (18.11) сравниваем высоту таблицы с прошлой: стала меньше — докручиваем к низу.

18.11 Собираем ChatViewController

```
import UIKit

final class ChatViewController: UIViewController {
    private let tableView = UITableView(frame: .zero, style: .plain)
    private let composer = ComposerView()

    private var messages: [ChatMessage] = [
        ChatMessage(author: .them, text: "Привет! Я эхо-бот. Напиши что-нибудь.", status:
 ↪  .read),
```

```
    ]
    /// Сколько ответов бота ещё в пути. Пока больше нуля — внизу строка «печатает».
    private var pendingReplies = 0
    private var isBotTyping: Bool { pendingReplies > 0 }
    private var lastTableHeight: CGFloat = 0

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Чат"
        view.backgroundColor = .systemBackground
        setupTable()
        setupLayout()
        composer.onSend = { [weak self] text in self?.handleSend(text) }
    }

    private func setupTable() {
        tableView.dataSource = self
        tableView.separatorStyle = .none
        tableView.allowsSelection = false
        tableView.rowHeight = UITableView.automaticDimension
        tableView.estimatedRowHeight = 60
        tableView.keyboardDismissMode = .interactive
        tableView.register(MessageCell.self, forCellReuseIdentifier: MessageCell.reuseID)
        tableView.register(TypingIndicatorCell.self, forCellReuseIdentifier:
↪   TypingIndicatorCell.reuseID)
    }

    private func setupLayout() {
        [tableView, composer].forEach {
            $0.translatesAutoresizingMaskIntoConstraints = false
            view.addSubview($0)
        }
        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            tableView.bottomAnchor.constraint(equalTo: composer.topAnchor),

            composer.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            composer.trailingAnchor.constraint(equalTo: view.trailingAnchor),
           composer.bottomAnchor.constraint(equalTo: view.keyboardLayoutGuide.topAnchor),
        ])
    }

    override func viewDidLayoutSubviews() {
        super.viewDidLayoutSubviews()
        // Клавиатура поднялась — таблица стала ниже. Держим последние сообщения на виду.
        let height = tableView.bounds.height
```

```
        if height < lastTableHeight {
            scrollToBottom(animated: false)
        }
        lastTableHeight = height
    }
}
```

Все методы из разделов 18.3, 18.4 и 18.10 — расширения этого класса в том же файле.

- **`rowHeight = UITableView.automaticDimension`** — высота строки из ограничений ячейки. `automaticDimension` — это константа −1, и она же значение `rowHeight` по умолчанию (проверено на свежей таблице), так что строка здесь ради ясности. `estimatedRowHeight = 60` — примерная высота, пока строка не посчитана: по ней таблица заранее оценивает общую длину списка и размер полосы прокрутки.
- **`separatorStyle = .none`** — в чате линии между строками не нужны.
- **`allowsSelection = false`** — сообщения не выделяются при тапе.
- **`keyboardDismissMode = .interactive`** — клавиатура уходит за пальцем, когда тянешь список вниз (18.6).
- **Таблица от верхнего края `view`.** Как и у коллекции в главе 16, scroll view сам отступит под навигационную панель, а сообщения при прокрутке будут уходить под неё.
- **`composer.onSend = { [weak self] ... }`** — composer хранит замыкание, контроллер хранит composer. Сильный `self` замкнул бы кольцо, и закрытый чат остался бы в памяти.
- **`lastTableHeight`** в `viewDidLayoutSubviews` — тот приём из 18.10: таблица стала ниже (выехала клавиатура или выросло поле ввода) — докручиваем к последнему сообщению. Без анимации, потому что движение уже идёт вместе с клавиатурой.

18.12 Бытовая аналогия

Чат — это **переписка записками через окошко**. Слева кладут записки от соседа, справа — твои. Сосед пишет — слышно шуршание карандаша (индикатор «печатает»). Сколько бы записок ты ни сунул в окошко, шуршание одно, и стихает, только когда сосед ответил на последнюю (счётчик `pendingReplies`).

Поле ввода — **блокнот на подставке**, а подставка стоит на клавиатуре. Клавиатура поднимается — блокнот едет вместе с ней (`keyboardLayoutGuide`), и ты не двигаешь его руками.

Галочки — **штампы на записке**: «принято в окошке», «сосед взял», «сосед прочитал». Свою записку ты видишь сразу, ещё до штампа (optimistic UI).

18.13 Что мы пропустили

- **Сохранение истории.** Сейчас при выходе из чата всё пропадает. В настоящем приложении — Core Data, SQLite или файлы JSON.
- **Настоящий сервер** — WebSocket для сообщений в реальном времени, очередь неотправленного на случай плохой сети и статус «ошибка» с кнопкой повтора.
- **Ответ на сообщение** с цитатой — обычно свайп вправо по пузырю.
- **Реакции** — долгое нажатие на сообщение и меню (`UIContextMenuConfiguration`, как в главе 16).

- **Вложения** — фото (через `PHPickerViewController`, см. главу 19), файлы, голосовые.
- **Группировка** — подряд идущие сообщения одного автора с меньшим отступом и одним «хвостиком», разделители дат.
- **Diffable data source** — таблица сама посчитает вставки и удаления, и ручная арифметика из 18.4 станет не нужна.
- **«Перевёрнутая» таблица** — приём больших мессенджеров: таблице задают `transform = CGAffineTransform(scaleX: 1, y: -1)`, такой же `transform` — каждой ячейке, новые сообщения кладут в **начало** массива, и «низ» чата оказывается в `contentOffset` ноль: ничего докручивать не нужно. Цена — у view с `transform`, отличным от identity, `frame` по документации Apple не определён, так что вёрстку таблицы ведут только через `bounds`, `center` или ограничения, а полосу прокрутки и отступы приходится зеркалить руками. Для чата с эхо-ботом это лишняя сложность.

Упражнение 18.3. Открой Чат (зелёная ячейка в лаунчере). Проверь: (1) сверху приветствие бота; (2) напиши «Привет» — сразу появится твой синий пузырь с «…», через четверть секунды «✓✓» (полупрозрачные), затем внизу прыгающие точки, затем ответ «Эхо: Привет» слева, и твои галочки станут ярко-белыми; (3) набери длинный текст — поле растёт до пяти строк, дальше внутри него появляется прокрутка; (4) на устройстве потяни список вниз — клавиатура уезжает за пальцем, а поле ввода опускается вслед за ней; (5) отправь три сообщения подряд очень быстро — приложение не падает, индикатор один (см. упражнение 18.1).

Ответы к упражнениям

Упражнение 18.1. Время отсчитываем от первой отправки, бот отвечает через 0,8–1,8 секунды после подтверждения:

Момент	Событие	`pendingReplies`	Строк «печатает»
0,00	отправлено сообщение 1	0	0
0,25	1 доставлено, `showTyping`	1	1 (вставлена)
0,50	отправлено 2	1	1
0,75	2 доставлено, `showTyping`	2	1 (уже есть)
1,00	отправлено 3	2	1
1,25	3 доставлено, `showTyping`	3	1
~1,3–2,1	ответ на 1, `receive`	2	1 (не удаляется)
…	ответ на 2	1	1
…	ответ на 3	0	0 (удалена)

Индикатор всегда один, исчезает с последним ответом. Ответы могут прийти не по порядку: задержка случайная, и ответ на второе сообщение иногда обгоняет первый. Счётчику это безразлично.

Упражнение 18.2. Отдельным расширением в `ComposerView.swift`:

```
import UIKit

extension ComposerView {
```

```
    func textView(_ textView: UITextView,
                  shouldChangeTextIn range: NSRange,
                  replacementText text: String) -> Bool {
        let current = textView.text as NSString
        let updated = current.replacingCharacters(in: range, with: text)
        return updated.count <= 1000
    }
}
```

Метод вызывается **до** изменения текста: `range` — какой кусок заменяется, `text` — на что (при вводе одной буквы — пустой кусок и одна буква, при вставке — выделенный фрагмент и вставляемый текст). Строим будущий текст и разрешаем правку, только если он не длиннее 1000 символов. `NSString` нужен потому, что `range` приходит как `NSRange` — в единицах UTF-16, как их считает `NSString`; у Swift-`String` свой способ индексации. `updated.count` — уже Swift-строка, и считаются видимые символы. Вставка 1500 символов целиком отклоняется, и поле не меняется.

Что мы выучили

- Optimistic UI: сообщение со статусом `.sending` показывается сразу, подтверждение меняет статус. Сообщение ищем по `id`, а не по сохранённому номеру строки.
- Индикатор «печатает» — лишняя строка после сообщений. Счётчик `pendingReplies` вместо флага не даёт двум ответам вставить и удалить её дважды.
- Арифметика `insertRows/deleteRows`: «было + вставлено − удалено = стало»; внутри `performBatchUpdates` удаления по старым номерам, вставки — по новым.
- Анимация точек — `CABasicAnimation` на `transform.translation.y`, `autoreverses`, бесконечный повтор, сдвиг `beginTime` от `CACurrentMediaTime()`.
- `view.keyboardLayoutGuide` (iOS 15+) — одно ограничение, и поле ввода едет за клавиатурой с её же анимацией, включая интерактивное закрытие.
- `UITextView → UIScrollView`, не контрол: об изменениях сообщает делегат, а не `addAction`. Растущее поле ввода: `sizeThatFits` + явная высота между 38 и 120 точками; прокрутка выключена, пока текст помещается, и включается на потолке.
- `UITableViewCell → UIView` напрямую; своё содержимое — в `contentView`. `UILayoutGuide → NSObject` — направляющая не view.
- Пузырь слева или справа — два ограничения, активно одно; сначала выключаем, потом включаем. Ширина ⩽ 75% ячейки.
- `DateFormatter` — один статический на все ячейки.
- Прочитано отличаем яркостью подписи, а VoiceOver получает статус словами.
- Прокрутка к низу — через `DispatchQueue.main.async`, число строк берём у самой таблицы; при подъёме клавиатуры докручиваем в `viewDidLayoutSubviews`.

Apple Developer Documentation

- UITableView — список сообщений с ячейками разной высоты.
- UITableView.automaticDimension — высота строки из ограничений ячейки.
- UIView.keyboardLayoutGuide — направляющая, которая следует за клавиатурой (iOS 15+).

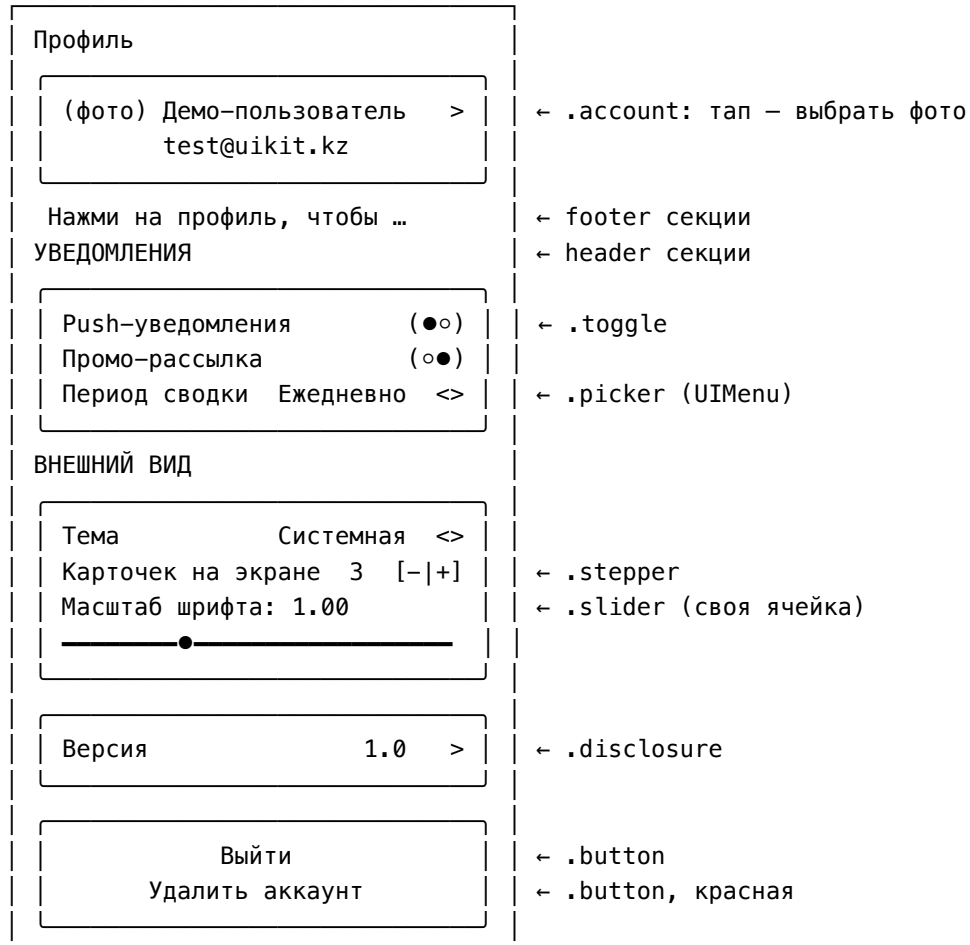
- UIKeyboardLayoutGuide — класс этой направляющей и её настройки (`usesBottomSafeArea` и другие появились в iOS 17).
- UIResponder.keyboardWillShowNotification — старый путь через уведомления, нужен только при поддержке iOS 14 и ниже.
- UITableViewCell — ячейка таблицы: наследник `UIView`, содержимое — в `contentView`.
- UILayoutGuide — направляющая: прямоугольник с якорями, но не view.
- UITextView — многострочное поле ввода; наследник `UIScrollView`.
- UIView.sizeThatFits(_:) — «какой размер тебе нужен»; сам view не меняет.
- CALayer.add(_:forKey:) — одна анимация на ключ: новая с тем же ключом заменяет старую.
- UITextViewDelegate.textViewDidChange(_:) — вызывается после каждого изменения текста.
- UIScrollView.contentInsetAdjustmentBehavior — как scroll view отступает под панели и безопасную зону.
- UIContextMenuConfiguration — меню по долгому нажатию, подходит для реакций на сообщения.
- CABasicAnimation — анимация одного свойства слоя, на ней прыгают точки.
- HIG: Layout — отступы, безопасные зоны и направляющие.

→ Глава 19. Profile / Settings — insetGrouped с разными типами ячеек

Глава 19. Profile / Settings — insetGrouped с разными типами ячеек

Экран настроек — самый «системный» из всех. Приложения повторяют устройство системных «Настроек»: таблица в стиле `insetGrouped` (секции-карточки со скруглёнными углами и отступом от краёв), заголовки и пояснения под секциями, разные виды строк — переключатель, счётчик «+/−», ползунок, выбор из списка, переход дальше, кнопка.

В этой главе строим такой экран. Все виды строк описываем **декларативно**: экран — это массив данных «какие секции и какие в них строки», а одна функция превращает каждую строку в ячейку.



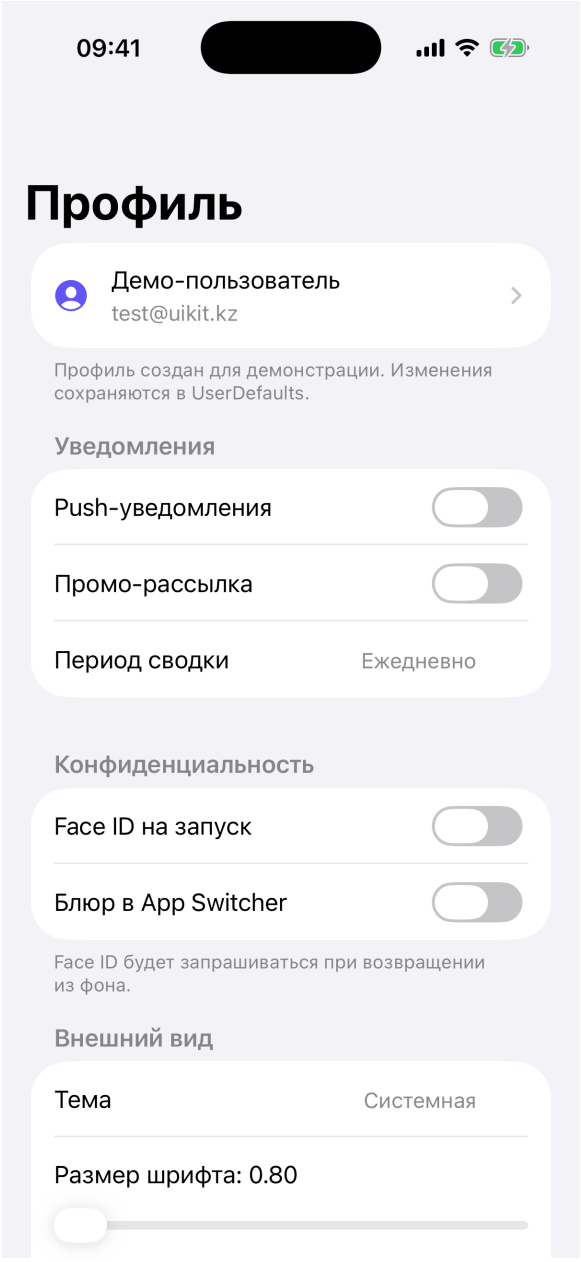


Рис. 15: Экран профиля и настроек

└──────────────────────────────────┘

Файлы:

```
Apps/Profile/
├── SettingsModel.swift          ← enum строк, секция, subscript(safe:)
├── SliderCell.swift             ← отдельная ячейка для ползунка
├── AvatarStore.swift            ← аватар в файле
└── ProfileViewController.swift  ← экран, ячейки, действия, PHPicker
```

`AuthStorage` — хранилище токена из главы 8 (раздел 8.3): у него есть `token` и `clear()`.

> **Режим сборки** — как во введении: Swift 6, `Default Actor Isolation = MainActor`, iOS 15+. Единственное место главы, где код приходит с фонового потока, — выбор фото в 19.11, там разберём, как вернуться на главный.

19.1 Декларативный массив секций

Главное — отделить **описание** экрана от **отрисовки**. Описание — массив значений, отрисовка — одна общая функция.

```swift
import UIKit

enum SettingsRow {
    case account(name: String, email: String, avatar: UIImage?)
    case toggle(title: String, key: String)
    case stepper(title: String, key: String, range: ClosedRange<Int>)
    case slider(title: String, key: String, range: ClosedRange<Float>)
    case picker(title: String, key: String, options: [String])
    case disclosure(title: String, value: String?, action: () -> Void)
    case button(title: String, isDestructive: Bool, action: () -> Void)
}

struct SettingsSection {
    let title: String?
    let footer: String?
    let rows: [SettingsRow]
}

extension Collection {
    /// Элемент по индексу или nil, если индекс за пределами — вместо падения.
    subscript(safe index: Index) -> Element? {
        indices.contains(index) ? self[index] : nil
    }
}
```

`SettingsRow` — перечисление со связанными значениями (associated values): у каждого варианта свои данные. Переключателю нужны заголовок и ключ, под которым хранится значение; счётчику — ещё и диапазон; кнопке — действие. Один вариант = один вид строки.

`key` — имя настройки в `UserDefaults`. UserDefaults — простое хранилище «ключ → значение» для пользовательских настроек, которое сохраняется между запусками приложения. Для токенов и паролей не годится (он хранится в открытом виде, для них — Keychain из главы 8), а для «включены ли push» — ровно то, что нужно.

`action: () -> Void` — замыкание прямо в данных. Строка сама знает, что делать при нажатии, и контроллеру не нужен отдельный `switch` по заголовкам.

`subscript(safe:)` — безопасный доступ по индексу: `options[safe: 5]` вернёт `nil`, если элементов меньше шести. Это не встроенный метод Swift: без этого объявления код не соберётся. Нам он нужен в выборе из списка: в `UserDefaults` может лежать номер варианта, которого в новой версии приложения уже нет.

Экран строит массив секций в одном месте:

```
import UIKit

extension ProfileViewController {
    func rebuildSections() {
        let email = AuthStorage.shared.token != nil ? "test@uikit.kz" : "—"
        let version = Bundle.main.object(forInfoDictionaryKey:
         ↪  "CFBundleShortVersionString") as? String

        sections = [
            SettingsSection(
                title: nil,
                footer: "Нажми на профиль, чтобы выбрать фото. Доступ ко всей медиатеке
↪  для этого не нужен.",
                rows: [.account(name: "Демо-пользователь", email: email, avatar:
↪  AvatarStore.load())]
            ),
            SettingsSection(title: "Уведомления", footer: nil, rows: [
                .toggle(title: "Push-уведомления", key: "settings.push"),
                .toggle(title: "Промо-рассылка", key: "settings.promo"),
                .picker(title: "Период сводки", key: "settings.digestPeriod",
                        options: ["Ежедневно", "Еженедельно", "Ежемесячно"]),
            ]),
            SettingsSection(title: "Внешний вид", footer: "Тема меняется сразу, без
↪  перезапуска.", rows: [
                .picker(title: "Тема", key: ProfileViewController.themeKey,
                        options: ["Системная", "Светлая", "Тёмная"]),
                .stepper(title: "Карточек на экране", key: "settings.cardsPerScreen",
                 ↪  range: 1...10),
                .slider(title: "Масштаб шрифта", key: "settings.fontScale", range:
                 ↪  0.8...1.4),
            ]),
            SettingsSection(title: "О приложении", footer: nil, rows: [
                .disclosure(title: "Версия", value: version) { [weak self] in
                 ↪  self?.showAbout() },
            ]),
```

```
            SettingsSection(
                title: nil,
                footer: "Удаление здесь демонстрационное. Как устроено настоящее — в главе
↪  40.",
                rows: [
                    .button(title: "Выйти", isDestructive: false) { [weak self] in
                    ↪  self?.logout() },
                   .button(title: "Удалить аккаунт", isDestructive: true) { [weak self] in
                        self?.confirmDelete()
                    },
                ]
            ),
        ]
        tableView.reloadData()
    }
}
```

Чтобы переставить строки или добавить новую настройку, меняешь только **массив**. Чтобы добавить новый **вид** строки, добавляешь вариант в `SettingsRow` и его отрисовку (компилятор сам напомнит: `switch` по перечислению без нового варианта не соберётся).

`CFBundleShortVersionString` — номер версии приложения из Info.plist («1.0»), тот, что видят пользователи в App Store.

`[weak self]` в действиях. Массив `sections` хранится в контроллере, а замыкания внутри него ссылаются на контроллер. Сильная ссылка замкнула бы кольцо «контроллер → массив → замыкание → контроллер», и экран настроек никогда не освободился бы из памяти.

> **Подсказка.** Такой подход похож на `Form` в SwiftUI. В классическом UIKit-коде обычно пишут `switch indexPath.section` в каждом методе таблицы. Декларативный массив чище: всё описание экрана в одном месте, а `numberOfSections`, `numberOfRowsInSection` и `cellForRowAt` просто читают его.

19.2 Строка аккаунта — картинка и два текста

Отрисовку разобьём на маленькие методы, по одному на вид строки. Общий `cellForRowAt` (19.12) только выбирает нужный.

```
import UIKit

extension ProfileViewController {
    func configureAccount(_ cell: UITableViewCell, name: String, email: String, avatar:
    ↪  UIImage?) {
        var content = cell.defaultContentConfiguration()
        content.text = name
        content.secondaryText = email
        content.image = avatar ?? UIImage(systemName: "person.crop.circle.fill")
        content.imageProperties.tintColor = .systemIndigo
        content.imageProperties.maximumSize = CGSize(width: 44, height: 44)
```

```
        content.imageProperties.reservedLayoutSize = CGSize(width: 44, height: 44)
        content.imageProperties.cornerRadius = 22
        cell.contentConfiguration = content
        cell.accessoryType = .disclosureIndicator
    }
}
```

`UIListContentConfiguration` (iOS 14+) — современный способ заполнить стандартную ячейку. Берёшь заготовку `cell.defaultContentConfiguration()`, заполняешь поля и присваиваешь `cell.contentConfiguration`. Раньше писали `cell.textLabel?.text` и `cell.detailTextLabel?.text`; в документации эти свойства помечены устаревшими начиная с iOS 27, а в SDK iOS 26.5 стоят с пометкой «будут устаревшими» — компилятор пока не предупреждает, но Apple прямо отправляет к content configuration. Смешивать нельзя: как только у ячейки есть `contentConfiguration`, `textLabel` и `detailTextLabel` возвращают `nil`.

Что это за объект. Конфигурация — не view, а описание содержимого: в Swift `UIListContentConfiguration` — структура (значение), которая реализует протокол `UIContentConfiguration` (глава 0, раздел 0.9). Присваивание `cell.contentConfiguration = content` заставляет ячейку попросить у конфигурации view через `makeContentView()` и поставить его на место своего `contentView`: мы проверили на симуляторе — после присваивания `cell.contentView` становится объектом класса `UIListContentView` с нашей подписью внутри. Повторное присваивание конфигурации того же типа не пересоздаёт этот view, а обновляет его; присваивание `nil` снова подменяет `contentView` на пустой. Поэтому конфигурацию можно присваивать при каждом `cellForRowAt`, не боясь накопить лишние view.

- `text` и `secondaryText` — основной текст и подпись под ним, мельче и серее.
- `image` — выбранное фото или системный значок человечка.
- `imageProperties.tintColor` — цвет значка (на фото не действует).
- `maximumSize` 44×44 — картинка не больше этого; фото 300×300 иначе раздуло бы строку.
- `reservedLayoutSize` — место под картинку, даже если она меньше. Без него текст сдвигался бы при смене значка на фото.
- `cornerRadius = 22` — половина стороны 44, картинка становится кругом.

`accessoryType = .disclosureIndicator` — серая стрелка «›» справа, знак «нажми, откроется что-то ещё».

19.3 Переключатель — UISwitch в accessoryView

```
import UIKit

extension ProfileViewController {
    func configureToggle(_ cell: UITableViewCell, title: String, key: String) {
        var content = cell.defaultContentConfiguration()
        content.text = title
        cell.contentConfiguration = content

        let toggle = UISwitch()
        toggle.isOn = UserDefaults.standard.bool(forKey: key)
```

```
        toggle.accessibilityLabel = title
        toggle.addAction(UIAction { action in
            guard let sender = action.sender as? UISwitch else { return }
            UserDefaults.standard.set(sender.isOn, forKey: key)
        }, for: .valueChanged)
        cell.accessoryView = toggle
        cell.selectionStyle = .none
    }
}
```

`accessoryView` — место справа в ячейке, куда можно положить любой вид. Переключатель — самый частый гость.

`UISwitch → UIControl → UIView`: это контрол, поэтому у него есть `addAction(_:for:)` и событие `.valueChanged`, которое приходит при каждом переключении пальцем. Программная установка `isOn` или `setOn(_:animated:)` action не рассылает (так в документации `setOn(_:animated:)`) — удобно, когда значение выставляешь из `UserDefaults`, как здесь: обработчик не сработает лишний раз (кто контрол, а кто нет — глава 0, раздел 0.4).

`addAction(UIAction { ... }, for: .valueChanged)` (iOS 14+) — обработчик замыканием, без `@objc`-метода. Значение берём из `action.sender` — элемента, который вызвал действие, — а не из переменной `toggle`. Если захватить `toggle` в замыкании, получится кольцо: переключатель держит action, action держит замыкание, замыкание держит переключатель. Каждый переключатель при прокрутке оставался бы в памяти. В списке захвата только `key` — строка, и `self` не нужен вовсе.

`as? UISwitch` с `guard` вместо `as! UISwitch` — если однажды обработчик окажется не на том элементе, ничего не случится, а не упадёт приложение.

`UserDefaults.standard.bool(forKey:)` вернёт `false`, если значение ещё ни разу не сохраняли, — для переключателя это разумное начальное положение.

`selectionStyle = .none` — строка не подсвечивается серым при тапе: реагирует только сам переключатель.

19.4 Счётчик «+/−» — UIStepper для целых чисел

```
import UIKit

extension ProfileViewController {
    func configureStepper(_ cell: UITableViewCell, title: String, key: String, range:
    ↪ ClosedRange<Int>) {
        let value = UserDefaults.standard.object(forKey: key) as? Int ?? range.lowerBound
        var content = cell.defaultContentConfiguration()
        content.text = title
        content.secondaryText = "\(value)"
        content.prefersSideBySideTextAndSecondaryText = true
        cell.contentConfiguration = content
```

```
        let stepper = UIStepper()
        stepper.minimumValue = Double(range.lowerBound)
        stepper.maximumValue = Double(range.upperBound)
        stepper.value = Double(value)
        stepper.accessibilityLabel = title
        stepper.addAction(UIAction { [weak cell] action in
            guard let sender = action.sender as? UIStepper else { return }
            let newValue = Int(sender.value)
            UserDefaults.standard.set(newValue, forKey: key)
            // Меняем текст на месте, а не перезагружаем строку: перезагрузка
            // заменила бы степпер под пальцем, и зажатый «+» перестал бы повторяться.
            guard let cell,
                  var updated = cell.contentConfiguration as? UIListContentConfiguration
                   ↪  else { return }
            updated.secondaryText = "\(newValue)"
            cell.contentConfiguration = updated
        }, for: .valueChanged)
        cell.accessoryView = stepper
        cell.selectionStyle = .none
    }
}
```

`UIStepper` — две кнопки «−» и «+». Само значение он не показывает, только меняет, поэтому число выводим в `secondaryText`.

Значение по умолчанию. `UserDefaults.standard.integer(forKey:)` вернул бы 0 для несохранённого ключа, а 0 вне нашего диапазона 1…10. Поэтому читаем `object(forKey:) as? Int`: `nil` значит «ещё не сохраняли», и тогда берём нижнюю границу диапазона.

`prefersSideBySideTextAndSecondaryText = true` — заголовок слева, значение справа в одну строку, как «Карточек на экране 3».

Обновление на месте. Напрашивается после каждого нажатия звать `tableView.reloadRows(at: [indexPath], with: .none)`. Но перезагрузка строки берёт для неё ячейку заново — и степпер под пальцем заменяется новым. Если зажать «+», степпер обычно повторяет нажатия (1, 2, 3, 4…), но после замены повтор обрывается на первом шаге. Поэтому мы меняем только текст: достаём текущую конфигурацию, правим `secondaryText` и присваиваем обратно. `[weak cell]` — степпер живёт внутри ячейки, и сильная ссылка из его обработчика на ячейку снова дала бы кольцо.

Значения на числах. `UIStepper` работает с `Double` (у него шаг может быть дробным), поэтому границы переводим `Double(1)`, `Double(10)`, а результат обратно `Int(sender.value)`. С шагом 1 по умолчанию дробей не бывает, и `Int(3.0)` даёт ровно 3.

19.5 Ползунок — отдельная ячейка

Ползунок широкий, в `accessoryView` он не помещается. Первое, что приходит в голову: в `cellForRowAt` удалять из `contentView` все виды и добавлять новые стек, метку и ползунок. У этого подхода три беды:

- ячейка общая для всех видов строк, а `contentView` у неё живёт по своим правилам: пока конфигурация того же типа присваивается снова, view остаётся тем же, и добавленный руками ползунок остаётся на нём (проверено на симуляторе — ползунок пережил повторное присваивание конфигурации); присвоишь `nil` или конфигурацию другого типа — ячейка подменит `contentView` целиком, и ползунок пропадёт вместе с ним. Угадывать, что из этого случится при очередном переиспользовании, — плохая идея;
- замыкание ползунка захватывало сам ползунок — кольцо, как в 19.3;
- каждая перерисовка создавала стек и ограничения заново.

Правильный путь в UIKit — **отдельный класс ячейки со своим идентификатором**. Ползунок создаётся один раз в `init`, а при переиспользовании меняются только значения.

```
import UIKit

final class SliderCell: UITableViewCell {
    static let reuseID = "SliderCell"

    private let titleLabel = UILabel()
    private let slider = UISlider()
    private var title = ""
    private var key = ""

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        titleLabel.font = .preferredFont(forTextStyle: .body)
        titleLabel.adjustsFontForContentSizeCategory = true
        titleLabel.numberOfLines = 0
        slider.addAction(UIAction { [weak self] _ in self?.valueChanged() }, for:
↪   .valueChanged)

        let stack = UIStackView(arrangedSubviews: [titleLabel, slider])
        stack.axis = .vertical
        stack.spacing = 8
        stack.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(stack)
        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: margins.topAnchor),
            stack.bottomAnchor.constraint(equalTo: margins.bottomAnchor),
            stack.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func configure(title: String, key: String, range: ClosedRange<Float>) {
        self.title = title
```

```
        self.key = key
        slider.minimumValue = range.lowerBound
        slider.maximumValue = range.upperBound
        slider.value = UserDefaults.standard.object(forKey: key) as? Float ??
↪   range.lowerBound
        slider.accessibilityLabel = title
        updateLabel()
    }

    private func valueChanged() {
        UserDefaults.standard.set(slider.value, forKey: key)
        updateLabel()
    }

    private func updateLabel() {
        titleLabel.text = "\(title): \(String(format: "%.2f", slider.value))"
    }
}
```

- **`layoutMarginsGuide`** — направляющая по внутренним полям `contentView` (layout margins). У обычного `UIView` они 8 точек со всех сторон, но ячейке таблица выставляет свои: в `insetGrouped` на iPhone с iOS 26.5 мы сняли 16 точек по бокам и 15 сверху и снизу. Это системные значения, они могут меняться от версии к версии — потому и привязываемся к направляющей, а не к числам: ползунок встаёт ровно по линии текста соседних строк, какими бы поля ни были.
- **Стек** — метка сверху, ползунок снизу, 8 точек между ними. Высота ячейки складывается из ограничений сама.
- **`[weak self]`** в обработчике — ползунок принадлежит ячейке, и сильная ссылка из обработчика на ячейку замкнула бы кольцо.
- **`key` и `title` хранятся в ячейке** — ячейку могут переиспользовать для другой настройки с другим ключом, и обработчик должен писать туда, куда указывает последняя `configure`, а не первая.
- **`String(format: "%.2f", ...)`** — число с двумя знаками после точки. `Float` не умеет хранить большинство десятичных дробей точно: 0,8 внутри хранится как 0,800000011920929, и без форматирования в подписи вылезали бы эти «хвосты». `%.2f` округляет до сотых: «0.80».
- **`numberOfLines = 0`** у метки — при крупном шрифте (Dynamic Type) подпись переносится, а не обрезается.

`UserDefaults` хранит `Float` как число, и `object(forKey:) as? Float` вернёт его обратно; если ключа нет — нижняя граница диапазона, 0.8.

> **Осторожно.** Правило переиспользования ячеек короткое: всё, что ячейка может показать, должно выставляться при **каждой** настройке или сбрасываться в `prepareForReuse`. Добавлять виды в `contentView` из `cellForRowAt` — почти всегда ошибка: через десять прокруток в ячейке окажется десять копий.

19.6 Выбор из списка — UIMenu на кнопке

```
import UIKit

extension ProfileViewController {
    func configurePicker(_ cell: UITableViewCell, title: String, key: String,
                         options: [String], indexPath: IndexPath) {
        let selectedIndex = UserDefaults.standard.integer(forKey: key)
        var content = cell.defaultContentConfiguration()
        content.text = title
        content.secondaryText = options[safe: selectedIndex] ?? options.first
        content.prefersSideBySideTextAndSecondaryText = true
        cell.contentConfiguration = content

        let actions = options.enumerated().map { index, option in
            UIAction(title: option, state: index == selectedIndex ? .on : .off) { [weak
↪   self] _ in
                UserDefaults.standard.set(index, forKey: key)
                if key == Self.themeKey { self?.applyTheme(index: index) }
                self?.tableView.reloadRows(at: [indexPath], with: .none)
            }
        }
        let button = UIButton(type: .system)
        button.menu = UIMenu(children: actions)
        button.showsMenuAsPrimaryAction = true
        button.setImage(UIImage(systemName: "chevron.up.chevron.down"), for: .normal)
        button.tintColor = .tertiaryLabel
        button.accessibilityLabel = title
        button.accessibilityValue = options[safe: selectedIndex]
        button.sizeToFit()   // у accessoryView должен быть размер, иначе кнопку не видно
        cell.accessoryView = button
        cell.selectionStyle = .none
    }

    static let themeKey = "settings.theme"

    func applyTheme(index: Int) {
        let styles: [UIUserInterfaceStyle] = [.unspecified, .light, .dark]
        view.window?.overrideUserInterfaceStyle = styles[safe: index] ?? .unspecified
    }
}
```

Вместо отдельного экрана со списком кладём в строку кнопку с меню `UIMenu` (iOS 14+). Тап — всплывает список вариантов, выбор сохраняется и строка обновляется.

- **`UIAction(title:state:)`** — пункт меню. `state: .on` ставит галочку у текущего варианта.
- **`button.menu` + `showsMenuAsPrimaryAction = true`** — меню открывается обычным тапом. Без второй строки оно открывалось бы только долгим нажатием.

- **`chevron.up.chevron.down`** — две стрелки, привычный значок «выпадающего списка».
- **`sizeToFit()`** — важная строка, которую легко забыть. `accessoryView` ставится с тем размером, который у вида уже есть. У `UISwitch` и `UIStepper` размер задан с рождения (на симуляторе iOS 26.5 — 63×28 и 94×32 точки), а `UIButton(type: .system)` — прямоугольник 0×0: кнопка была бы невидимой и ненажимаемой. `sizeToFit()` зовёт `sizeThatFits(_:)` и меняет `bounds` под картинку — для accessory view, которые таблица раскладывает по `frame`, а не по Auto Layout, это единственный способ.
- **`reloadRows` здесь уместен** — к моменту выбора меню уже закрыто, и заменить кнопку «под пальцем» нельзя, в отличие от степпера (19.4).
- **`indexPath` в замыкании.** Строки нашего экрана не двигаются, так что номер строки не устаревает. Если бы строки добавлялись и удалялись, номер пришлось бы искать заново в момент выбора.
- **`integer(forKey:)`** для номера варианта: 0 для несохранённого ключа — это первый вариант, «Ежедневно», и такое значение по умолчанию нас устраивает. `[safe:]` защищает от номера, которого нет в списке.

Тема. Выбор в строке «Тема» сразу меняет оформление: `overrideUserInterfaceStyle` у окна заставляет всё в нём рисоваться в светлой или тёмной теме. Окно — тоже view: `UIWindow → UIView`, корень дерева, в котором живут view всех экранов приложения; у него нет собственного изображения, оно только держит `rootViewController` и раздаёт события (глава 0, раздел 0.7). Поэтому свойство, которое есть у любого view, у окна действует на всё дерево разом. `.unspecified` — «как в системе». В playground окно общее для всех мини-приложений, так что тема сменится и у лаунчера. Выбранная тема применяется ещё и при появлении экрана (19.12) — иначе после перезапуска она бы не восстановилась.

> **Подсказка.** До iOS 14 выбор из списка делали переходом на отдельный экран с таблицей вариантов или `UIPickerView`. С `UIMenu` короче и привычнее для пользователя — так устроены многие строки системных «Настроек».

> **Упражнение 19.1.** Добавь в секцию «Внешний вид» строку «Язык интерфейса» с вариантами «Русский», «Қазақша», «English». Проверка: выбери «English», закрой и снова открой профиль — в строке по-прежнему «English». Решение — в конце главы.

19.7 Строка-переход — disclosure

```
import UIKit

extension ProfileViewController {
    func configureDisclosure(_ cell: UITableViewCell, title: String, value: String?) {
        var content = cell.defaultContentConfiguration()
        content.text = title
        content.secondaryText = value
        content.prefersSideBySideTextAndSecondaryText = true
        cell.contentConfiguration = content
        cell.accessoryType = .disclosureIndicator
    }
}
```

Строка-ссылка: тап открывает что-то новое — алерт, соседний экран, лист. Справа можно показать текущее значение («Версия — 1.0»).

Действие хранится в самой строке (`.disclosure(..., action:)`) и вызывается в методе делегата `didSelectRowAt`:

```
import UIKit

extension ProfileViewController: UITableViewDelegate {
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        switch sections[indexPath.section].rows[indexPath.row] {
        case .account:
            pickAvatar()
        case let .disclosure(_, _, action), let .button(_, _, action):
            action()
        default:
            break
        }
        tableView.deselectRow(at: indexPath, animated: true)
    }
}
```

`case let .disclosure(_, _, action), let .button(_, _, action):` — один `case` на два варианта: у обоих есть `action` одного типа, и Swift разрешает связать его в обеих ветках одним именем.

`deselectRow` после действия. Строка подсвечивается при тапе, и подсветку снимаем с анимацией — пользователь видит, какую строку нажал. Мы снимаем её **после** вызова действия: `logout()` (19.9) берёт выделенную строку, чтобы привязать к ней всплывающее окно на iPad.

19.8 Строка-кнопка — обычная и опасная

```
import UIKit

extension ProfileViewController {
    func configureButton(_ cell: UITableViewCell, title: String, isDestructive: Bool) {
        var content = cell.defaultContentConfiguration()
        content.text = title
        content.textProperties.color = isDestructive ? .systemRed : .systemBlue
        content.textProperties.alignment = .center
        cell.contentConfiguration = content
        cell.accessibilityTraits = .button
    }
}
```

Текст по центру, синий (обычное действие) или красный (опасное, destructive — «разрушительное», после которого данные пропадают). Никаких стрелок справа — вся строка работает как кнопка.

`textProperties.color` и `textProperties.alignment` — цвет и выравнивание текста внутри конфигурации. Со старым API это были `textLabel?.textColor` и `textLabel?.textAlignment`.

`accessibilityTraits = .button` — трейт (признак) для VoiceOver: он скажет «Выйти, кнопка», и пользователь поймёт, что строку можно нажать.

Центрированный цветной текст — стиль системных «Настроек» для действий вроде «Выйти» и «Удалить аккаунт».

19.9 Опасные действия — подтверждение

«Выйти» и «Удалить аккаунт» — действия, после которых данные теряются. Перед ними всегда спрашиваем подтверждение.

```
import UIKit

extension ProfileViewController {
    func showAbout() {
        let alert = UIAlertController(title: "UIKit Playground",
                                      message: "Демо-экран настроек из главы 19.",
                                      preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "OK", style: .default))
        present(alert, animated: true)
    }

    func logout() {
        let alert = UIAlertController(title: "Выйти?",
                                      message: "Понадобится войти заново.",
                                      preferredStyle: .actionSheet)
        alert.addAction(UIAlertAction(title: "Выйти", style: .destructive) { [weak self] _
↪  in
            AuthStorage.shared.clear()
            self?.rebuildSections()
        })
        alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
        // На iPad action sheet показывается всплывающим окном, и ему нужна точка привязки.
        if let popover = alert.popoverPresentationController {
            if let row = tableView.indexPathForSelectedRow, let cell =
             ↪  tableView.cellForRow(at: row) {
                popover.sourceView = cell
                popover.sourceRect = cell.bounds
            } else {
                popover.sourceView = view
                popover.sourceRect = CGRect(x: view.bounds.midX, y: view.bounds.midY,
↪  width: 0, height: 0)
            }
        }
        present(alert, animated: true)
    }
```

```
    func confirmDelete() {
        let alert = UIAlertController(title: "Удалить аккаунт?",
                                      message: "Это действие необратимо. Все данные
↪  удалятся.",
                                      preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
        alert.addAction(UIAlertAction(title: "Удалить", style: .destructive) { [weak self]
↪  _ in
            AuthStorage.shared.clear()
            AvatarStore.clear()
            self?.rebuildSections()
        })
        present(alert, animated: true)
    }
}
```

Два стиля `UIAlertController`:

- **`.actionSheet`** — список действий снизу экрана. Для решений полегче: выйти, поделиться, выбрать из вариантов.
- **`.alert`** — окно посередине, внимание сильнее. Для серьёзного: удалить, сбросить.

iPad. На iPad action sheet показывается всплывающим окном (popover) со стрелкой, и системе нужно знать, **на что** указывать стрелкой; без точки привязки показ на iPad падает. Откуда берётся `popoverPresentationController`: у `UIAlertController` со стилем `.actionSheet` свойство `modalPresentationStyle` равно `.popover`, а по документации обращение к `popoverPresentationController` при таком стиле создаёт объект `UIPopoverPresentationController` — на любом устройстве. Мы проверили на симуляторе iPhone: он **не** `nil`. На телефоне, в компактной ширине, popover сам адаптируется в лист снизу, и `sourceView` там просто не используется, так что `if let` — это не «только на iPad», а «если presentation controller есть», и выполняется он везде. Задать привязку на iPhone безвредно, а не задать на iPad — падение. `UIPopoverPresentationController → UIPresentationController → NSObject` — объект, управляющий показом, не view и не view controller; сам `UIAlertController` — обычный `UIViewController`, а `UIAlertAction` — `NSObject`, описание кнопки, не кнопка (глава 0, разделы 0.6 и 0.8). Привязываем окно к выбранной строке, а если строки нет — к центру экрана.

Стили кнопок. `style: .destructive` — красный текст, сигнал «осторожно». `style: .cancel` — отмена; система сама ставит её на привычное место, в каком бы порядке ты ни добавлял кнопки: в `.alert` с двумя кнопками отмена оказывается слева, в `.actionSheet` — отдельной кнопкой внизу. Кнопка `.cancel` в одном алерте может быть только одна.

После действия `rebuildSections()` перестраивает экран: почта в строке аккаунта станет «—», фото пропадёт.

> **Удаление аккаунта в App Store.** По правилу 5.1.1(v) App Store Review Guidelines приложение, где можно создать аккаунт, обязано давать и удалить его изнутри приложения — по-настоящему, с удалением данных на сервере, а не только выходом. Наша кнопка лишь стирает локальные данные. Как устроен настоящий процесс — в главе 40.

19.10 Заголовок и пояснение секции

```
import UIKit

extension ProfileViewController: UITableViewDataSource {
    func numberOfSections(in tableView: UITableView) -> Int {
        sections.count
    }

    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        sections[section].rows.count
    }

    func tableView(_ tableView: UITableView, titleForHeaderInSection section: Int) ->
    ↪ String? {
        sections[section].title
    }

    func tableView(_ tableView: UITableView, titleForFooterInSection section: Int) ->
    ↪ String? {
        sections[section].footer
    }
}
```

Все четыре метода только читают массив `sections` — вот где окупается декларативность. `nil` вместо заголовка или пояснения — секция без него.

Оформление система делает сама: заголовок — мелкий серый текст над карточкой, пояснение (footer) — мелкий серый текст под ней, с переносом строк.

- **Header** — название группы: «Уведомления», «Внешний вид».
- **Footer** — объяснение «зачем это»: «Тема меняется сразу, без перезапуска».

В системных «Настройках» пояснения под секциями встречаются на каждом шагу, и пользователи привыкли в них заглядывать. Пиши их там, где назначение настройки неочевидно.

19.11 Аватар через PHPicker — без доступа к медиатеке

Тап по строке аккаунта открывает системный выбор фото, `PHPickerViewController` (iOS 14+). Он работает в **отдельном процессе** системы: приложение не видит медиатеку, получает только то фото, которое пользователь выбрал. Поэтому **разрешение на доступ к фото не нужно** — ни системного диалога, ни ключа `NSPhotoLibraryUsageDescription` в Info.plist. (Сравнение уровней доступа к медиатеке — в таблице раздела 16.10 главы 16.)

```
import PhotosUI
import UIKit

extension ProfileViewController: PHPickerViewControllerDelegate {
    func pickAvatar() {
```

```
        var config = PHPickerConfiguration()
        config.filter = .images
        config.selectionLimit = 1
        let picker = PHPickerViewController(configuration: config)
        picker.delegate = self
        present(picker, animated: true)
    }

    func picker(_ picker: PHPickerViewController, didFinishPicking results:
    ↪  [PHPickerResult]) {
        picker.dismiss(animated: true)
        guard let provider = results.first?.itemProvider,
              provider.canLoadObject(ofClass: UIImage.self) else { return }

        provider.loadObject(ofClass: UIImage.self) { object, _ in
            // Сюда приходим на фоновой очереди.
            guard let image = object as? UIImage,
                  let small = image.preparingThumbnail(of: CGSize(width: 300, height:
                  ↪  300)) else { return }
            Task { @MainActor [weak self] in
                AvatarStore.save(small)
                self?.rebuildSections()
            }
        }
    }
}
```

- **`PHPickerConfiguration`** — настройки: `filter = .images` (только фото, без видео), `selectionLimit = 1` (одно фото; 0 значит «сколько угодно»).
- **`didFinishPicking`** вызывается и при выборе, и при отмене — тогда `results` пустой, и `guard` просто выходит. Закрывать picker нужно самому, в обоих случаях.
- **`itemProvider`** — «поставщик» выбранного файла. Картинку он отдаёт не сразу: оригинал может лежать в iCloud и скачиваться. `loadObject(ofClass: UIImage.self)` загружает и превращает в `UIImage`.
- **Фоновый поток.** Обработчик `loadObject` система вызывает на фоновой очереди — так написано в документации `NSItemProvider`, и замыкание помечено `@Sendable`. Трогать интерфейс и `self` (он на main actor) здесь нельзя. Уменьшаем фото прямо в фоне, а сохраняем и перерисовываем в `Task { @MainActor in ... }` — задаче, которая выполнится на главном потоке.
- **`preparingThumbnail(of:)`** (iOS 15+) — уменьшенная копия картинки. Фото с камеры — 4032×3024 пикселей: в распакованном виде 4032 × 3024 × 4 байта ≈ 49 МБ памяти. Для аватара в 44 точки (132 пикселя на экране @3x) хватает 300×300 — это 300 × 300 × 4 ≈ 0,36 МБ, в 135 раз меньше. Метод сохраняет пропорции: 300×300 — это рамка, в которую фото вписывается, а круглая маска в строке обрежет лишнее.

Хранилище аватара:

```
import UIKit
```

```
/// Аватар живёт файлом в Application Support, а не в UserDefaults:
/// картинка на сотни килобайт замедлила бы чтение всех настроек.
enum AvatarStore {
    private static var fileURL: URL? {
        guard let dir = FileManager.default.urls(for: .applicationSupportDirectory,
                                                 in: .userDomainMask).first else { return nil }
        try? FileManager.default.createDirectory(at: dir, withIntermediateDirectories:
        ↪  true)
        return dir.appendingPathComponent("avatar.jpg")
    }

    static func load() -> UIImage? {
        guard let url = fileURL, let data = try? Data(contentsOf: url) else { return nil }
        return UIImage(data: data)
    }

    static func save(_ image: UIImage) {
        guard let url = fileURL, let data = image.jpegData(compressionQuality: 0.8) else {
        ↪  return }
        try? data.write(to: url, options: .atomic)
    }

    static func clear() {
        guard let url = fileURL else { return }
        try? FileManager.default.removeItem(at: url)
    }
}
```

- **Application Support** — папка внутри песочницы приложения (sandbox — отдельный каталог, куда другие приложения доступа не имеют) для файлов, которые нужны приложению, но не видны пользователю в «Файлах». В отличие от Caches, система её не чистит. Папка создаётся не сразу, поэтому `createDirectory(..., withIntermediateDirectories: true)` — «создай, если нет; если есть — ничего страшного».
- **Почему не UserDefaults.** Весь `UserDefaults` загружается в память целиком при первом обращении. Картинка в нём замедляла бы чтение любой настройки.
- **`jpegData(compressionQuality: 0.8)`** — сжатие JPEG с качеством 80%: на глаз неотличимо, файл заметно меньше.
- **`.atomic`** — запись сначала во временный файл, потом подмена. Если приложение упадёт посреди записи, останется старый аватар, а не половина нового.

Упражнение 19.2. Сейчас тап по профилю сразу открывает выбор фото. Сделай так, чтобы при уже выбранном фото сначала появлялся action sheet «Выбрать другое фото / Удалить фото / Отмена», а без фото — сразу выбор, как раньше. Проверка: после «Удалить фото» в строке снова значок человечка. Решение — в конце главы.

19.12 Собираем ProfileViewController

```
import UIKit

final class ProfileViewController: UIViewController {
    let tableView = UITableView(frame: .zero, style: .insetGrouped)
    var sections: [SettingsSection] = []

    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Профиль"
        tableView.dataSource = self
        tableView.delegate = self
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "Cell")
        tableView.register(SliderCell.self, forCellReuseIdentifier: SliderCell.reuseID)
        tableView.frame = view.bounds
        tableView.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        view.addSubview(tableView)
        rebuildSections()
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        applyTheme(index: UserDefaults.standard.integer(forKey: Self.themeKey))
    }
}

extension ProfileViewController {
    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let row = sections[indexPath.section].rows[indexPath.row]

        if case let .slider(title, key, range) = row {
            let cell = tableView.dequeueReusableCell(withIdentifier: SliderCell.reuseID,
            ↪  for: indexPath)
            (cell as? SliderCell)?.configure(title: title, key: key, range: range)
            return cell
        }

        let cell = tableView.dequeueReusableCell(withIdentifier: "Cell", for: indexPath)
        // Ячейка могла прийти из-под строки другого вида — сбрасываем всё, что меняют
        ↪  методы ниже.
        cell.accessoryView = nil
        cell.accessoryType = .none
        cell.selectionStyle = .default
        cell.accessibilityTraits = .none

        switch row {
```

```
        case let .account(name, email, avatar):
            configureAccount(cell, name: name, email: email, avatar: avatar)
        case let .toggle(title, key):
            configureToggle(cell, title: title, key: key)
        case let .stepper(title, key, range):
            configureStepper(cell, title: title, key: key, range: range)
        case let .picker(title, key, options):
            configurePicker(cell, title: title, key: key, options: options, indexPath:
↪   indexPath)
        case let .disclosure(title, value, _):
            configureDisclosure(cell, title: title, value: value)
        case let .button(title, isDestructive, _):
            configureButton(cell, title: title, isDestructive: isDestructive)
        case .slider:
            break   // обработан выше своей ячейкой
        }
        return cell
    }
}
```

`.insetGrouped` (iOS 13+) — стиль таблицы настроек: каждая секция — отдельная скруглённая карточка с отступом от краёв экрана. Фон таблицы в этом стиле серый (`.systemGroupedBackground`), карточки — белые, в тёмной теме наоборот — система подбирает сама.

Два идентификатора ячеек. `"Cell"` — обычная `UITableViewCell` для всех видов строк, кроме ползунка; `SliderCell.reuseID` — своя ячейка. Ячейка ползунка никогда не придёт под переключатель, и наоборот.

Сброс перед настройкой. Общая ячейка переходит от строки к строке: была переключателем — стала кнопкой. Если не сбрасывать `accessoryView`, строка «Выйти», получив ячейку из-под «Push- уведомлений», покажет чужой переключатель. Четыре строки сброса перед `switch` выставляют то, что каждый вид строки меняет «по желанию»: `accessoryView`, `accessoryType`, `selectionStyle`, `accessibilityTraits`. `contentConfiguration` сбрасывать не нужно — каждый метод `configure…` присваивает его заново.

Свойства без `private`. Методы из 19.1–19.11 — расширения в других файлах, а `private` в Swift виден только внутри одного файла. Поэтому `tableView` и `sections` открыты на уровне модуля.

Тема в `viewDidAppear`. `view.window` появляется, когда `view` вставлена в дерево окна. Мы проверили порядок на симуляторе: в `viewDidLoad` и в `viewWillAppear` окно ещё `nil`, в `viewDidLayoutSubviews` и `viewDidAppear` — уже есть. Поэтому сохранённую тему применяем в `viewDidAppear`. Этот метод, в отличие от `viewDidLoad`, вызывается при каждом появлении экрана — для нас это безвредно: применить тему дважды не страшно.

Кадр и autoresizing. Таблица занимает весь `view`: задаём кадр `view.bounds` и `autoresizingMask`. Разберём, что здесь происходит, потому что это вторая система раскладки UIKit, старше Auto Layout.

`autoresizingMask` — набор флагов `UIView.AutoresizingMask`, который говорит, что делать с view, когда меняются `bounds` его superview. `.flexibleWidth` (битовое значение 2) — «растягивай ши-

рину», `.flexibleHeight` (16) — «растягивай высоту»; есть ещё четыре флага для гибких полей (`flexibleLeftMargin` = 1, `flexibleRightMargin` = 4, `flexibleTopMargin` = 8, `flexibleBottomMargin` = 32). Сумма наших двух — 18, и ровно такое значение стоит у `view` самого контроллера: UIKit растягивает корневой view вслед за окном тем же механизмом (проверено рантаймом). По умолчанию маска пустая (`[]`): view стоит на месте и не меняет размер.

Зачем маска, если кадр уже задан? Потому что в `viewDidLoad` размеры `view` ещё не окончательные — окно может быть другого размера, экран ещё не прошёл раскладку. `tableView.frame = view.bounds` даёт только стартовое значение; настоящий размер таблица получит, когда `view` примет свой, — через маску, без нашего участия. Без неё таблица осталась бы того размера, какой был в `viewDidLoad`, и при повороте не растянулась бы.

Почему это не смешивают с ограничениями на одном view. У view, созданного в коде, `translatesAutoresizingMaskIntoConstraints = true`: UIKit переводит маску и кадр в полный набор ограничений размера и положения. Добавь к такому view свои ограничения — получится конфликт, который Auto Layout разрешит, отбросив часть из них (так сказано в документации свойства). Правило простое: либо `frame` + `autoresizingMask` при `translates… = true`, либо ограничения при `translates… = false`. В главе 16 (16.8) scroll view для фото собран первым способом, поле ввода в главе 18 — вторым.

Для «одного вида на весь экран» первый способ короче, чем четыре ограничения. Отступы под навигационную панель и полоску «домой» таблица сделает сама (`contentInsetAdjustmentBehavior = .automatic`).

19.13 Бытовая аналогия

Экран настроек — **передняя панель старого усилителя**. Тумблеры (переключатели), кнопки «+/−» (степпер), ползунки, переключатель режимов (выбор из списка), красная кнопка сброса под колпачком (опасное действие с подтверждением) и подписи под каждой группой (footer).

Декларативный массив — **схема панели на бумаге**. Сначала рисуешь, где какой элемент, потом по схеме собирают панель. Хочешь переставить ручки — меняешь схему, а не перепаиваешь всё.

Переиспользование ячеек — **сменные накладки на одну и ту же кнопку**. Перед тем как надеть новую, старую снимаешь целиком, иначе на кнопке «Выйти» окажется остаток тумблера.

19.14 Что мы пропустили

- **`UIDatePicker` в строке** — выбор времени или даты. С iOS 14 стиль `.compact` помещается прямо в `accessoryView`.
- **Нестандартные элементы** — выбор цвета (`UIColorWell`, iOS 14+), кольцевой прогресс — как ползунок, своей ячейкой.
- **Редактирование** — `tableView.isEditing = true`, перестановка и удаление строк («Избранное», «Закладки»).
- **Поиск по настройкам** — `navigationItem.searchController`, как в системных «Настройках».
- **Diffable data source** — `UITableViewDiffableDataSource` для анимаций, когда секции появляются и исчезают (например, блок «Тихие часы» только при включённых push).

- **Биометрия** — строка «Входить по Face ID» с `LAContext` (фреймворк LocalAuthentication, глава 11).

  Упражнение 19.3. Открой Profile (фиолетовая ячейка). Сначала появится вход (auth gate из главы 8), войди с `test@uikit.kz`. Проверь: (1) переключи «Push-уведомления», закрой и открой профиль — положение сохранилось; (2) зажми «+» у «Карточек на экране» — число растёт само и останавливается на 10; (3) двигай ползунок — подпись меняется с двумя знаками после точки; (4) в строке «Тема» выбери «Тёмная» — весь экран сразу станет тёмным; (5) тап по профилю — откроется выбор фото без запроса разрешения, выбранное фото появится кругом в строке; (6) «Выйти» — снизу список действий, «Удалить аккаунт» — окно посередине с красной кнопкой; (7) быстро прокрути таблицу вверх-вниз — ни у одной кнопки не появится чужой переключатель.

Ответы к упражнениям

Упражнение 19.1. Одна строка в секции «Внешний вид» внутри `rebuildSections()`:

```
.picker(title: "Язык интерфейса", key: "settings.language",
        options: ["Русский", "Қазақша", "English"]),
```

Больше ничего менять не нужно: отрисовка, меню с галочкой и сохранение уже есть у вида строки `.picker`. Это и есть выигрыш от декларативного массива. Выбор сохраняется в `UserDefaults` под ключом `settings.language`, а при повторном открытии `configurePicker` прочитает его оттуда. (Язык самого приложения это не меняет: для этого нужна локализация строк — отдельная большая тема.)

Упражнение 19.2. Новый метод выбора действия:

```
import UIKit

extension ProfileViewController {
    func showAvatarOptions() {
        guard AvatarStore.load() != nil else {
            pickAvatar()
            return
        }
        let sheet = UIAlertController(title: nil, message: nil, preferredStyle:
        ↪  .actionSheet)
        sheet.addAction(UIAlertAction(title: "Выбрать другое фото", style: .default) {
↪  [weak self] _ in
            self?.pickAvatar()
        })
        sheet.addAction(UIAlertAction(title: "Удалить фото", style: .destructive) { [weak
↪  self] _ in
            AvatarStore.clear()
            self?.rebuildSections()
        })
        sheet.addAction(UIAlertAction(title: "Отмена", style: .cancel))
```

```
        if let popover = sheet.popoverPresentationController {
            popover.sourceView = tableView
            popover.sourceRect = tableView.rectForRow(at: IndexPath(row: 0, section: 0))
        }
        present(sheet, animated: true)
    }
}
```

И в `didSelectRowAt` заменить `case .account: pickAvatar()` на `case .account: showAvatarOptions()`. Для iPad привязываем окно к строке аккаунта: `rectForRow(at:)` даёт её прямоугольник в координатах таблицы.

Что мы выучили

- **Декларативный** массив `[SettingsSection]` с `enum SettingsRow`: экран описан данными, а методы таблицы только читают массив.
- **`UIListContentConfiguration`** (iOS 14+) — заполнение стандартной ячейки: `text`, `secondaryText`, `image`, `imageProperties`, `textProperties`. Это структура-описание (`UIContentConfiguration`), а не view; ячейка сама делает из неё `UIListContentView` и ставит его на место `contentView`. `textLabel`/`detailTextLabel` с ней не совмещаются.
- **Переключатель** — `UISwitch` в `accessoryView`, значение из `action.sender`, без захвата самого элемента.
- **Степпер** — значение меняем на месте, без `reloadRows`, иначе рвётся автоповтор зажатого «+».
- **Ползунок** — своя ячейка `SliderCell` со своим идентификатором; виды создаются один раз в `init`.
- **Выбор из списка** — `UIButton` с `UIMenu` и `showsMenuAsPrimaryAction`; кнопке в `accessoryView` нужен размер (`sizeToFit()`).
- **Общая ячейка** сбрасывает `accessoryView`, `accessoryType`, `selectionStyle` и трейты перед каждой настройкой.
- **Опасные действия** — подтверждение через `UIAlertController` (обычный `UIViewController`); у action sheet `modalPresentationStyle = .popover`, `popoverPresentationController` есть и на iPhone, а точка привязки обязательна на iPad.
- **`frame` + `autoresizingMask`** (`.flexibleWidth` = 2, `.flexibleHeight` = 16) — для одного view на весь экран; с ограничениями на том же view не смешивать: `translatesAutoresizingMaskIntoConstraints` либо `true`, либо `false`.
- **Удаление аккаунта** для App Store — настоящее, с сервера (правило 5.1.1(v), глава 40).
- **`PHPickerViewController`** — выбор фото без разрешения на медиатеку; результат приходит на фоновой очереди, на главный возвращаемся через `Task { @MainActor in }`; большое фото уменьшаем `preparingThumbnail(of:)`.
- Картинки — файлами в Application Support, настройки — в `UserDefaults`.

Apple Developer Documentation

- UITableView.Style.insetGrouped — стиль настроек: секции-карточки с отступом.

- UITableViewCell.CellStyle.value1 — классический «заголовок слева, значение справа»; сейчас вместо него `prefersSideBySideTextAndSecondaryText`.
- UIListContentConfiguration — конфигурация содержимого стандартной ячейки (iOS 14+).
- UIContentConfiguration — протокол конфигураций: `makeContentView()` и `updated(for:)`.
- UITableViewCell.contentConfiguration — как конфигурация подменяет `contentView` ячейки.
- UIViewController.popoverPresentationController — когда свойство создаётся и когда оно `nil`.
- UIView.autoresizingMask — флаги растяжения вслед за superview.
- UIView.translatesAutoresizingMaskIntoConstraints — почему маску и ограничения не смешивают на одном view.
- UISwitch — переключатель в `accessoryView`.
- UIStepper — кнопки «+/−» для чисел в диапазоне.
- UISlider — ползунок, в нашем экране — в своей ячейке.
- UIMenu — всплывающее меню с галочкой у выбранного пункта.
- UIButton — кнопка; свойства `menu` и `showsMenuAsPrimaryAction`.
- UIAction — обработчик-замыкание для `addAction(_:for:)` и пунктов меню.
- UIAlertController — подтверждения опасных действий, стили `.alert` и `.actionSheet`.
- UserDefaults — хранилище пользовательских настроек.
- PHPickerViewController — системный выбор фото без доступа к медиатеке.
- PHPickerViewControllerDelegate — получение выбранных фото.
- preparingThumbnail(of:) — уменьшенная копия картинки (iOS 15+).
- LAContext — Face ID / Touch ID, для будущей строки «Входить по Face ID».
- LAPolicy.deviceOwnerAuthenticationWithBiometrics — проверка только биометрией, без кода-пароля.
- HIG: Toggles — как вести себя переключателям и подписям к ним.

→ Глава 20. Custom Tab Bar — три стиля контейнера

Глава 20. Custom Tab Bar — три стиля контейнера

`UITabBarController` — стандартный таб-бар iOS: полоска с иконками внизу экрана, тап по иконке переключает раздел. На iPhone в ней помещается до пяти вкладок, лишние уходят в раздел «Ещё». Большие приложения почти всегда берут именно его, и правильно делают: он бесплатно даёт VoiceOver, бейджи, поддержку iPad и новый внешний вид при каждом обновлении iOS. В iOS 26 системный таб-бар сам стал плавающей «таблеткой» в стиле Liquid Glass и умеет сворачиваться при прокрутке (`tabBarMinimizeBehavior`, только iOS 26+), а на iPad с iOS 18 может превращаться в боковую панель (`mode = .tabSidebar`).

Но у стандартного контроллера есть рамки:

- Внешний вид настраивается через `UITabBarAppearance` (iOS 13+) — набор готовых параметров: фон, цвета иконок и подписей, бейджи. Форму, положение и поведение панели так не поменяешь.
- Переход между вкладками по умолчанию мгновенный. Анимацию можно добавить через делегат `UITabBarControllerDelegate` и собственный «аниматор», но это отдельная работа.
- На iOS 15–17 полоса всегда внизу, одна и та же на любом экране.

Иногда нужно своё: плавающая «таблетка» внизу на старых iOS, переключатель вкладок сверху с листанием пальцем, боковое выдвижное меню. Все три делаются своими руками через **container view controller**.

В этой главе строим все три. Попутно разберём сам паттерн «контейнер» — после него любая нестандартная навигация собирается из тех же трёх вызовов.

> **Режим компиляции.** Весь код главы проверен в режиме, на который книга рассчитана: Swift 6, `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` (настройка «Default Actor Isolation» в Build Settings), deployment target iOS 15.0. Все классы главы — наследники UIKit и так работают на главном потоке, поэтому ни одного `@MainActor` вручную писать не придётся.

20.1 Container view controller — паттерн

Напомним основу. **View controller** (VC) — объект, который управляет одним экраном: создаёт его view, реагирует на тапы, узнаёт о появлении и исчезновении экрана (методы жизненного цикла `viewDidLoad`, `viewWillAppear`, `viewDidAppear` и т.д., подробно — в главе 5). Сам он не view: `UIViewController → UIResponder → NSObject`, на экране рисуется только его `view` (глава 0, раздел 0.6).

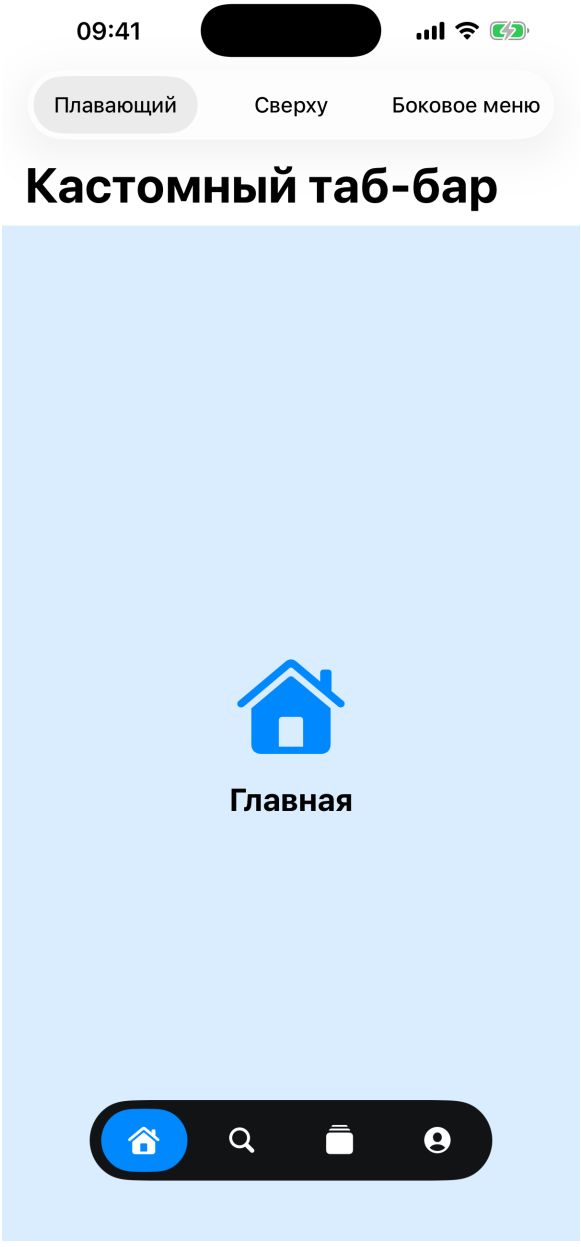


Рис. 16: Кастомный таб-бар с тремя стилями

Container view controller — VC, который показывает внутри себя другие VC. Он **владеет** ими как «детьми» (child view controllers), решает, какой из них сейчас на экране, и рисует вокруг своё «обрамление»: таб-бар, панель, шторку.

Бытовая картинка: контейнер — это рамка для фотографий со сменными вкладышами. Рамка одна (кнопки, панель), вкладыши (экраны-дети) меняются. Главное — вкладыш вставляется «по правилам», иначе он болтается.

Стандартные контейнеры iOS устроены именно так:

- `UINavigationController` — стек экранов с кнопкой «Назад».
- `UITabBarController` — вкладки.
- `UIPageViewController` — страницы, которые листают пальцем.
- `UISplitViewController` — две-три колонки, главным образом для iPad.

Все четыре — прямые наследники `UIViewController` (проверено рантаймом, см. главу 0, раздел 0.6); никакого отдельного «класса контейнера» в UIKit нет, контейнер — это роль, а не тип. Свой контейнер делается теми же вызовами, что используют они. Добавление — три шага:

1. `addChild(child)` — регистрируем ребёнка у родителя. Внутри UIKit сам вызывает `child.willMove(toParent: self)`.
2. `view.addSubview(child.view)` + constraints — кладём view ребёнка в иерархию и растягиваем.
3. `child.didMove(toParent: self)` — сообщаем ребёнку: «переезд закончен». Этот вызов UIKit за тебя **не** делает.

Удаление — в обратном порядке:

1. `child.willMove(toParent: nil)` — предупреждаем. Сам UIKit его не вызовет.
2. `child.view.removeFromSuperview()` — снимаем view.
3. `child.removeFromParent()` — разрываем связь; внутри UIKit вызывает `child.didMove(toParent: nil)`.

Асимметрия (при добавлении UIKit сам зовёт `willMove`, при удалении — сам зовёт `didMove`) описана прямо в заголовочном файле `UIViewController.h`: «addChildViewController: will call willMoveToParentViewController:self... However, it will not call didMoveToParentViewController:». Запоминать её не обязательно — хватит завернуть ритуал в две функции и пользоваться ими везде:

```
extension UIViewController {
    /// Встраивает `child` как дочерний экран и растягивает его view
    /// на весь `container` (по умолчанию — на собственный view).
    func embed(_ child: UIViewController, in container: UIView? = nil) {
        let host: UIView = container ?? view
        addChild(child)                                   // 1. регистрируем ребёнка
        child.view.translatesAutoresizingMaskIntoConstraints = false
        host.addSubview(child.view)                       // 2. кладём его view
        NSLayoutConstraint.activate([
            child.view.topAnchor.constraint(equalTo: host.topAnchor),
            child.view.bottomAnchor.constraint(equalTo: host.bottomAnchor),
            child.view.leadingAnchor.constraint(equalTo: host.leadingAnchor),
            child.view.trailingAnchor.constraint(equalTo: host.trailingAnchor),
```

```
        ])
        child.didMove(toParent: self)                    // 3. сообщаем «переезд закончен»
    }

    /// Вынимает этот экран из родителя — зеркально к `embed`.
    func unembed() {
        guard parent != nil else { return }
        willMove(toParent: nil)                          // 1. предупреждаем
        view.removeFromSuperview()                       // 2. снимаем view
        removeFromParent()                               // 3. разрываем связь
    }
}
```

Разберём неочевидные строки.

`let host: UIView = container ?? view` — если контейнер не передали, кладём ребёнка прямо в свой `view`. У `UIViewController` свойство `view` объявлено как `UIView!` (неявно развёрнутый опционал): при первом обращении UIKit сам создаёт view, поэтому оно не `nil`. Явный тип `UIView` слева нужен, чтобы `??` вернул обычный, а не опциональный view.

`translatesAutoresizingMaskIntoConstraints = false` — без этой строки UIKit превратит старую «маску автоматического ресайза» view в свои constraints, они столкнутся с нашими, и в консоли появится «Unable to simultaneously satisfy constraints». Напомним: **constraint** — правило Auto Layout вида «верх A равен верху B плюс 8 точек»; из набора таких правил система вычисляет рамки всех view.

Четыре constraints «верх-низ-лево-право равны контейнеру» — просто «растянуть во весь контейнер».

`guard parent != nil` в `unembed()` — защита от двойного удаления: если экран уже вынут, повторный вызов ничего не сломает.

Что будет, если ритуал пропустить? Мы проверили в симуляторе (iOS 26.5) три варианта ребёнка в одном родителе:

Как добавили	`viewWillAppear`/`viewDidAppear`	`parent`
`addChild` + `addSubview` + `didMove`	пришли	есть
только `addSubview`	пришли, но UIKit их не гарантирует	`nil`
`addChild` + `addSubview`, без `didMove`	пришли	есть

То есть первое, что ломается, — **не** жизненный цикл, а связи. У ребёнка, которого положили «только view», `parent == nil`, а значит, не работает всё, что UIKit ищет вверх по цепочке родителей: `navigationController`, `tabBarController`, `splitViewController` вернут `nil` (документация так и описывает их — «ближайший предок в иерархии view controller'ов»). Такой экран для UIKit — чужой: контейнер пересылает поворот (`viewWillTransition(to:with:)`) и прочие события только своим `children`. `viewWillAppear` в нашем замере всё же пришёл, но Apple этого не обещает — документация свойства `view` прямо требует вызвать `addChild(_:)` **до** добавления чужого view в свою иерархию. `didMove(toParent:)` — точка, где ребёнок узнаёт, что переезд

закончен; если он там что-то настраивает (например, анимацию появления), без вызова это просто не случится. Правильный ритуал ничего не стоит, поэтому всегда пиши его целиком.

А обратная ошибка — снять view, но забыть `removeFromParent()` — ещё коварнее: экран пропал с глаз, но остался в `children` родителя и живёт в памяти. Замер: после `e.view.removeFromSuperview()` у ребёнка `parent` всё ещё есть, а `parent.children.count` не уменьшился. Массив `children` держит детей сильными ссылками, так что пока экран в нём — он жив вместе со своим view, данными и подписками. Меняешь таб десять раз — десять «призраков» в памяти.

20.2 Tab model — общая для трёх стилей

Каждый таб — простая структура:

```
struct Tab {
    let title: String
    let icon: String      // имя SF Symbol
    let color: UIColor
}
```

`icon` — имя **SF Symbol**, иконки из встроенной библиотеки Apple (`UIImage(systemName: "house.fill")`). Их тысячи, все бесплатны, все масштабируются под размер шрифта.

`TabContentViewController` — заглушка для содержимого таба: иконка и подпись по центру на фоне цвета таба.

```
final class TabContentViewController: UIViewController {
    private let tabSpec: Tab

    init(tab: Tab) {
        self.tabSpec = tab
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = tabSpec.color.withAlphaComponent(0.15)

        let icon = UIImageView(image: UIImage(systemName: tabSpec.icon))
        icon.tintColor = tabSpec.color
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 64,
↪   weight: .semibold)

        let label = UILabel()
        label.text = tabSpec.title
        label.font = .preferredFont(forTextStyle: .title2)
        label.adjustsFontForContentSizeCategory = true
```

```
        let stack = UIStackView(arrangedSubviews: [icon, label])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 12
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.centerXAnchor),
            stack.centerYAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.centerYAnchor),
        ])
    }
}
```

`withAlphaComponent(0.15)` — тот же цвет, но непрозрачный на 15%: бледная заливка вместо кричащей.

`required init?(coder:)` с `fatalError` — этот инициализатор нужен только для загрузки из storyboard. Мы создаём экран кодом, поэтому честно падаем, если кто-то попробует иначе.

`.preferredFont(forTextStyle: .title2)` + `adjustsFontForContentSizeCategory` — шрифт из системы **Dynamic Type**: пользователь в «Настройки → Экран и яркость → Размер текста» делает текст крупнее, и подпись растёт вместе с ним, без перезапуска экрана.

Центрируем по `safeAreaLayoutGuide`. **Safe area** — часть экрана, которую ничто не закрывает: без «чёлки»/Dynamic Island сверху, полоски «домой» снизу и панелей навигации. Это пригодится в 20.3: мы расширим safe area ребёнка, чтобы плавающая панель не закрывала контент.

> **Имя `tabSpec`, а не `tab`.** В iOS 18 SDK у `UIViewController` появилось свойство `tab: UITab?` (только для чтения). Если в своём наследнике назвать свойство `tab`, Swift считает это переопределением, и сборка падает с ошибкой «overriding property must be as accessible as its enclosing type». Мы проверили это на Xcode 26.5. Отсюда и `tabSpec`.

20.3 Style 1 — Floating capsule

Внизу экрана — плавающая «таблетка» с иконками. Контент под ней виден целиком, панель не прибита к краю.

Сначала каркас контейнера:

```
final class FloatingTabBarContainer: UIViewController {
    private let tabs: [Tab]
    private var selectedIndex = 0
    private let contentContainer = UIView()
    private let barView = UIView()
    private var buttons: [UIButton] = []
    private var cachedChildren: [Int: UIViewController] = [:]   // кеш: индекс → экран
    private var currentChild: UIViewController?
```

```
    private let barHeight: CGFloat = 56
    private let barBottomGap: CGFloat = 8

    init(tabs: [Tab]) {
        self.tabs = tabs
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground

        contentContainer.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(contentContainer)
        NSLayoutConstraint.activate([
            contentContainer.topAnchor.constraint(equalTo: view.topAnchor),
            contentContainer.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            contentContainer.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            contentContainer.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])

        setupBar()
        showTab(at: 0, animated: false)
    }
}
```

`contentContainer` — отдельный view «под содержимое». Зачем, если можно класть детей прямо в `view`? Из-за **z-порядка** (какой view рисуется поверх какого): subview, добавленный позже, лежит выше. Панель мы добавим один раз, а детей будем менять много раз. Если класть их в `view`, каждый новый ребёнок ляжет **поверх** панели и закроет её. Внутри `contentContainer` дети меняются как угодно, а сам контейнер навсегда остаётся под панелью.

`cachedChildren` — словарь «номер таба → уже созданный экран». Зачем он, объясним в `showTab`.

Теперь панель:

```
private func setupBar() {
    barView.translatesAutoresizingMaskIntoConstraints = false
    barView.backgroundColor = UIColor.label.withAlphaComponent(0.92)
    barView.layer.cornerRadius = barHeight / 2
    view.addSubview(barView)

    let stack = UIStackView()
    stack.axis = .horizontal
    stack.distribution = .fillEqually
    stack.spacing = 8
    stack.translatesAutoresizingMaskIntoConstraints = false
```

```
    barView.addSubview(stack)

    NSLayoutConstraint.activate([
        barView.bottomAnchor.constraint(equalTo: view.safeAreaLayoutGuide.bottomAnchor,
                                        constant: -barBottomGap),
        barView.heightAnchor.constraint(equalToConstant: barHeight),
        barView.centerXAnchor.constraint(equalTo: view.centerXAnchor),

        stack.topAnchor.constraint(equalTo: barView.topAnchor, constant: 6),
        stack.bottomAnchor.constraint(equalTo: barView.bottomAnchor, constant: -6),
        stack.leadingAnchor.constraint(equalTo: barView.leadingAnchor, constant: 6),
        stack.trailingAnchor.constraint(equalTo: barView.trailingAnchor, constant: -6),
    ])

    // ... кнопки — ниже
}
```

`UIColor.label.withAlphaComponent(0.92)` — `label` это системный цвет текста: в светлой теме почти чёрный, в тёмной почти белый. Панель получается контрастной к фону в обеих темах сама, без проверок `if dark`.

`cornerRadius = barHeight / 2` — радиус скругления равен половине высоты: 56 / 2 = 28. Когда радиус ровно половина высоты, короткие стороны превращаются в полукруги — получается капсула.

`bottomAnchor = safeAreaLayoutGuide.bottomAnchor - 8` — низ панели на 8 точек выше нижней границы safe area. На iPhone без кнопки «Домой» safe area заканчивается над полоской «домой» (на iPhone 16 это 34 точки от края), так что панель висит в 34 + 8 = 42 точках от нижнего края экрана.

Ширину панели мы не задаём — её «распирает» содержимое: стек прибит к панели со всех сторон с отступом 6, а кнопки внутри имеют фиксированную ширину. Посчитаем на четырёх табах:

- 4 кнопки × 60 = 240 точек;
- 3 промежутка × 8 (`spacing`) = 24;
- отступы стека слева и справа: 6 + 6 = 12.

Итого 240 + 24 + 12 = 276 точек. На экране шириной 393 точки (iPhone 16) остаётся по (393 − 276) / 2 = 58,5 точек слева и справа — это и даёт ощущение «плавающей» панели. Высота кнопки: 56 − 6 − 6 = 44 точки — ровно минимальный размер тап-зоны по рекомендациям Apple (44×44).

`distribution = .fillEqually` — все кнопки одной ширины. Здесь ширина и так задана явно, но при другом количестве табов правило подстрахует.

Кнопки:

```
for (index, tab) in tabs.enumerated() {
    let button = UIButton(type: .system)
    button.setImage(UIImage(systemName: tab.icon), for: .normal)
    button.tintColor = .systemBackground
```

```
    button.layer.cornerRadius = 22
    button.accessibilityLabel = tab.title
    button.addAction(UIAction { [weak self] _ in
        self?.showTab(at: index, animated: true)
    }, for: .touchUpInside)
    button.widthAnchor.constraint(equalToConstant: 60).isActive = true
    stack.addArrangedSubview(button)
    buttons.append(button)
}
```

`tintColor = .systemBackground` — иконки цвета фона: на тёмной панели светлые, на светлой тёмные. Противоположность `label`.

`cornerRadius = 22` — половина высоты кнопки (44 / 2): подсветка выбранного таба тоже будет капсулой 60×44.

`accessibilityLabel = tab.title` — у кнопки только картинка, без текста. **VoiceOver** (экранный диктор для незрячих) без подписи прочитал бы «кнопка» или имя символа. С подписью — «Главная, кнопка».

`UIAction { [weak self] _ in ... }` — обработчик тапа в виде замыкания (`addAction(_:for:)`, iOS 14+). Он есть у кнопки, потому что `UIButton → UIControl`: target-action и события вроде `.touchUpInside` — имущество `UIControl`, у простого `UIView` или `UILabel` их нет (глава 0, раздел 0.4). `[weak self]` обязателен: кнопка живёт внутри `self`, а замыкание живёт внутри кнопки. Сильная ссылка на `self` замкнула бы круг «контейнер → кнопка → замыкание → контейнер», и контейнер никогда бы не освободился (retain cycle).

`index` захватывается по значению на каждой итерации цикла — каждая кнопка помнит свой номер.

Смена таба:

```
private func showTab(at index: Int, animated: Bool) {
    // Повторный тап по уже открытому табу ничего не делает.
    if index == selectedIndex, currentChild != nil { return }
    selectedIndex = index

    let child: UIViewController
    if let cached = cachedChildren[index] {
        child = cached
    } else {
        child = TabContentViewController(tab: tabs[index])
        cachedChildren[index] = child
    }

    let swap = {
        self.currentChild?.unembed()
        self.embed(child, in: self.contentContainer)
        // Контент не должен уходить под плавающую панель: 56 + 8 = 64 pt.
        child.additionalSafeAreaInsets.bottom = self.barHeight + self.barBottomGap
```

```
        self.currentChild = child
    }
    if animated {
        UIView.transition(with: contentContainer, duration: 0.2,
                          options: .transitionCrossDissolve, animations: swap)
    } else {
        swap()
    }
    updateSelection(animated: animated)
}
```

Пройдём по шагам.

Первая строка отсекает повторный тап по открытому табу. Условие `currentChild != nil` пропускает самый первый вызов из `viewDidLoad`, когда `selectedIndex` уже 0, но на экране ещё ничего нет.

Дальше — **кеш**. Экран таба создаём один раз и кладём в `cachedChildren`. При возврате на таб берём готовый экран: позиция прокрутки, введённый текст, загруженные данные — всё на месте. Так же ведёт себя стандартный `UITabBarController`: он держит экраны вкладок в памяти, пока жив сам.

Старый ребёнок вынимается **полным** ритуалом через `unembed()`. Короткий вариант `for sub in contentContainer.subviews { sub.removeFromSuperview() }` снимает только view, а сами VC остаются в `children` родителя. Каждый тап по табу добавлял бы ещё одного «призрака». Именно этот случай мы замеряли в 20.1.

`additionalSafeAreaInsets.bottom = 64` — **дополнительный отступ safe area** (iOS 11+). Мы говорим ребёнку: «считай, что снизу у тебя ещё 64 точки занято». 64 = 56 (высота панели) + 8 (зазор). Всё, что ребёнок привязал к своей safe area, — таблица, кнопка внизу экрана — само поднимется над панелью. Это лучше, чем знать в каждом экране про высоту чужой панели.

`UIView.transition(with:duration:options:animations:)` с `.transitionCrossDissolve` — плавная смена «было → стало» за 0,2 секунды: старое содержимое растворяется, новое проявляется. Вот та самая анимация переключения, которой нет у стандартного таб-бара.

Подсветка активной кнопки:

```
private func updateSelection(animated: Bool) {
    let apply = {
        for (i, button) in self.buttons.enumerated() {
            let active = i == self.selectedIndex
            button.backgroundColor = active ? self.tabs[i].color : .clear
            button.accessibilityTraits = active ? [.button, .selected] : .button
        }
    }
    if animated {
        UIView.animate(withDuration: 0.25, animations: apply)
    } else {
        apply()
    }
```

```
}
```

Выбранная кнопка получает цвет своего таба, остальные прозрачны. `UIView.animate(withDuration: 0.25)` перекрашивает фон за четверть секунды вместо мгновенного «прыжка». Первый показ (`animated: false`) рисуем сразу: анимировать появление экрана не нужно.

`accessibilityTraits` — **трейты доступности**, «ярлыки» элемента для VoiceOver. `.selected` на активной кнопке — VoiceOver скажет «Главная, выбрано, кнопка». У стандартного таб-бара это бесплатно, у своего — только если не забыть.

Замыкание `apply` обращается к `self` явно (`self.buttons`) — Swift требует этого внутри замыканий, чтобы захват `self` был виден глазами. Retain cycle тут нет: `UIView.animate` держит замыкание 0,25 секунды и отпускает.

> **Кеш или пересоздание?** Для большинства приложений кеш правильный: пользователь ждёт «вернуться туда же». Отказаться от него стоит для очень тяжёлых экранов — например, галерея с сотнями фото в памяти. Тогда при уходе с таба экран выбрасывают, а состояние (позицию прокрутки, фильтры) сохраняют отдельно.

Упражнение 20.1. В `FloatingTabBarContainer` закомментируй строку `child.additionalSafeAreaInsets.bottom = ...` и замени `TabContentViewController` на экран с таблицей (`UITableViewController` на 50 строк). Долистай до конца. Что видишь? Верни строку и сравни. Ответ — в конце главы.

20.4 Style 2 — Top Tabs

Переключатель вкладок сверху, под ним страницы, которые можно листать пальцем вбок. Так сделаны многие новостные ленты и магазины.

Хитрость: контейнером страниц берём готовый `UIPageViewController` (он уже умеет листание), а кнопки сверху просто синхронизируем с ним. То есть наш контейнер держит внутри себя другой контейнер.

```
final class TopTabsContainer: UIViewController,
                              UIPageViewControllerDataSource,
                              UIPageViewControllerDelegate {
    private let tabs: [Tab]
    private var selectedIndex = 0
    private let bar = UIStackView()
    private var buttons: [UIButton] = []
    private let indicatorLine = UIView()
    private var indicatorLeading: NSLayoutConstraint?
    private var pages: [UIViewController] = []
    private let pageVC = UIPageViewController(transitionStyle: .scroll,
                                              navigationOrientation: .horizontal)

    init(tabs: [Tab]) {
        self.tabs = tabs
        super.init(nibName: nil, bundle: nil)
    }
```

```
    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        setupTabsBar()
        setupPager()
        updateSelection(animated: false)
    }
}
```

Data source («источник данных») и **delegate** («делегат») — два протокола-«помощника» `UIPageViewController`. Data source отвечает на вопрос «какая страница слева/справа от этой?», делегат получает события «листание закончилось». Сам контроллер страниц ничего не знает о наших табах — он спрашивает нас.

`transitionStyle: .scroll` — страницы едут вбок, как лента. Второй вариант, `.pageCurl`, — загибающийся уголок бумажной страницы.

Полоска кнопок и бегунок-индикатор:

```
private func setupTabsBar() {
    bar.axis = .horizontal
    bar.distribution = .fillEqually
    bar.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(bar)

    for (i, tab) in tabs.enumerated() {
        let button = UIButton(type: .system)
        button.setTitle(tab.title, for: .normal)
        button.titleLabel?.font = .systemFont(ofSize: 14, weight: .semibold)
        button.addAction(UIAction { [weak self] _ in
            self?.select(index: i, animated: true)
        }, for: .touchUpInside)
        bar.addArrangedSubview(button)
        buttons.append(button)
    }

    indicatorLine.backgroundColor = .systemBlue
    indicatorLine.layer.cornerRadius = 1.5
    indicatorLine.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(indicatorLine)

    NSLayoutConstraint.activate([
        bar.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor),
        bar.leadingAnchor.constraint(equalTo: view.safeAreaLayoutGuide.leadingAnchor),
        bar.trailingAnchor.constraint(equalTo: view.safeAreaLayoutGuide.trailingAnchor),
        bar.heightAnchor.constraint(equalToConstant: 44),
```

```
        indicatorLine.topAnchor.constraint(equalTo: bar.bottomAnchor),
        indicatorLine.heightAnchor.constraint(equalToConstant: 3),
        // Ширина линии = ширине одной кнопки, а кнопки равны (fillEqually).
        indicatorLine.widthAnchor.constraint(equalTo: buttons[0].widthAnchor),
    ])
}
```

Полоска прибита к **safe area** сверху: сразу под навигационной панелью, а не за ней. Слева и справа — тоже к safe area, чтобы в альбомной ориентации кнопки не залезли под «чёлку».

Ширина линии — «равна ширине первой кнопки». Кнопки одинаковые (`.fillEqually`), так что это ширина любой из них. На iPhone 16 в портрете: 393 / 4 = 98,25 точки. В альбомной ориентации ширина safe area другая, но constraint пересчитается сам.

`cornerRadius = 1.5` при высоте 3 — опять «половина высоты»: линия с круглыми концами.

> **Почему не `view.bounds.width / 4`.** Так и тянет посчитать ширину линии и её сдвиг прямо в `viewDidLoad`. Это хрупко: в `viewDidLoad` размер view ещё может быть не окончательным, а после поворота экрана число устаревает, и линия съезжает. Constraint «равна ширине кнопки» и «левый край равен левому краю выбранной кнопки» пересчитываются Auto Layout при любом изменении размера сами.

Страницы:

```
private func setupPager() {
    pages = tabs.map { TabContentViewController(tab: $0) }
    pageVC.dataSource = self
    pageVC.delegate = self

    addChild(pageVC)
    pageVC.view.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(pageVC.view)
    NSLayoutConstraint.activate([
        pageVC.view.topAnchor.constraint(equalTo: indicatorLine.bottomAnchor),
        pageVC.view.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        pageVC.view.leadingAnchor.constraint(equalTo: view.leadingAnchor),
        pageVC.view.trailingAnchor.constraint(equalTo: view.trailingAnchor),
    ])
    pageVC.didMove(toParent: self)
    pageVC.setViewControllers([pages[0]], direction: .forward, animated: false)
}
```

Тот же ритуал «addChild → addSubview → didMove», но руками: здесь view ребёнка прибит не ко всему контейнеру, а под индикатор, поэтому готовый `embed` не подходит.

Все четыре страницы создаём заранее. Для четырёх лёгких экранов это нормально; для десятка тяжёлых лучше создавать их лениво в data source.

`setViewControllers([pages[0]], ...)` — какую страницу показать первой. Массив, потому что в режиме «разворот книги» на iPad страниц на экране две.

Тап по кнопке:

```
private func select(index: Int, animated: Bool) {
    guard index != selectedIndex else { return }
    let direction: UIPageViewController.NavigationDirection =
        index > selectedIndex ? .forward : .reverse
    selectedIndex = index
    pageVC.setViewControllers([pages[index]], direction: direction, animated: animated)
    updateSelection(animated: animated)
}
```

`direction` — в какую сторону ехать. С таба 1 на таб 3 — вперёд (новая страница приезжает справа), с 3 на 1 — назад (слева). Без этого переход «назад» выглядел бы как движение вперёд — мелочь, которую пользователь замечает.

Перестановка индикатора и цвета кнопок:

```
private func updateSelection(animated: Bool) {
    // Переставляем линию под выбранную кнопку: старое правило долой, новое — в бой.
    indicatorLeading?.isActive = false
    indicatorLeading = indicatorLine.leadingAnchor.constraint(
        equalTo: buttons[selectedIndex].leadingAnchor)
    indicatorLeading?.isActive = true

    for (i, button) in buttons.enumerated() {
        let active = i == selectedIndex
        button.tintColor = active ? .label : .secondaryLabel
        button.accessibilityTraits = active ? [.button, .selected] : .button
    }

    if animated {
        UIView.animate(withDuration: 0.25) { self.view.layoutIfNeeded() }
    }
}
```

Приём «анимировать constraint». Меняем правило (было «левый край линии = левому краю кнопки 0», стало «= левому краю кнопки 2»), а потом внутри `UIView.animate` вызываем `layoutIfNeeded()`. Этот вызов заставляет Auto Layout пересчитать рамки **прямо сейчас**, и раз мы внутри анимационного блока, переход от старой рамки к новой анимируется. Без `layoutIfNeeded()` внутри блока рамка обновилась бы на ближайшем проходе раскладки — мгновенно.

Числами: на iPhone 16 кнопки стоят с шагом 98,25 точки. Переход с таба 0 на таб 2 — линия проезжает 2 × 98,25 = 196,5 точки за 0,25 секунды.

Data source — те же два метода, что в онбординге (глава 6):

```
func pageViewController(_ pageViewController: UIPageViewController,
                        viewControllerBefore viewController: UIViewController) ->
                        ↪ UIViewController? {
    guard let i = pages.firstIndex(of: viewController), i > 0 else { return nil }
    return pages[i - 1]
```

```
}

func pageViewController(_ pageViewController: UIPageViewController,
                        viewControllerAfter viewController: UIViewController) ->
                         ↪  UIViewController? {
    guard let i = pages.firstIndex(of: viewController), i < pages.count - 1 else { return
     ↪  nil }
    return pages[i + 1]
}
```

`nil` означает «дальше страниц нет»: на первой странице нельзя листнуть влево, на последней — вправо. `firstIndex(of:)` работает, потому что `UIViewController` наследует `NSObject`, а тот умеет сравнение на равенство (по умолчанию — «тот же объект», глава 0, раздел 0.1).

Делегат — узнаём про листание пальцем:

```
func pageViewController(_ pageViewController: UIPageViewController,
                        didFinishAnimating finished: Bool,
                        previousViewControllers: [UIViewController],
                        transitionCompleted completed: Bool) {
    guard completed,
          let current = pageViewController.viewControllers?.first,
          let index = pages.firstIndex(of: current) else { return }
    selectedIndex = index
    updateSelection(animated: true)
}
```

`completed` — ключевой флаг. Пользователь мог начать листать и передумать — страница вернулась на место. Тогда анимация закончилась (`finished == true`), но переход **не состоялся** (`completed == false`), и двигать индикатор нельзя.

Получилась двусторонняя синхронизация: тап по кнопке → страница и индикатор; листание пальцем → индикатор и кнопки.

20.5 Style 3 — Drawer (боковое меню)

Главное содержимое во весь экран, кнопка-«гамбургер» (три полоски) в углу; тап — слева выезжает шторка со списком разделов. Такое меню встречается в почтовых клиентах и мессенджерах для рабочих команд. Apple в HIG предпочитает таб-бар или боковую колонку `UISplitViewController`, поэтому шторка — выбор для особых случаев, а не для каждого приложения.

Слои снизу вверх: контент → кнопка меню → затемнение (**overlay**, полупрозрачная чёрная «вуаль» поверх контента) → сама шторка.

```
final class DrawerContainer: UIViewController {
    private let tabs: [Tab]
    private var selectedIndex = -1          // -1: ещё ничего не показано
    private let contentHost = UIView()
```

```
    private let overlay = UIView()
    private let drawer = UIView()
    private let menuButton = UIButton(type: .system)
    private var itemButtons: [UIButton] = []
    private var currentChild: UIViewController?

    private var drawerLeading: NSLayoutConstraint!
    private let drawerWidth: CGFloat = 260
    private var isOpen = false
    private var panStartConstant: CGFloat = 0

    init(tabs: [Tab]) {
        self.tabs = tabs
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        setupContent()
        setupDrawer()
        setupGestures()
        showTab(at: 0)
    }
}
```

`drawerLeading: NSLayoutConstraint!` — неявно развёрнутый опционал: создать constraint можно только когда есть view, то есть в `setupDrawer()`, а не в инициализаторе. Обращаемся к нему только после `viewDidLoad`, так что падения не будет.

Контент, затемнение и кнопка меню:

```
private func setupContent() {
    for layerView in [contentHost, overlay] {
        layerView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(layerView)
        NSLayoutConstraint.activate([
            layerView.topAnchor.constraint(equalTo: view.topAnchor),
            layerView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            layerView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            layerView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])
    }
    overlay.backgroundColor = .black
    overlay.alpha = 0
    overlay.isUserInteractionEnabled = false   // закрытая шторка не ловит тапы
```

```
    var config = UIButton.Configuration.filled()
    config.image = UIImage(systemName: "line.3.horizontal")
    config.cornerStyle = .capsule
    menuButton.configuration = config
    menuButton.accessibilityLabel = "Меню разделов"
    menuButton.addAction(UIAction { [weak self] _ in
        self?.setDrawer(open: true)
    }, for: .touchUpInside)
    menuButton.translatesAutoresizingMaskIntoConstraints = false
    view.insertSubview(menuButton, belowSubview: overlay)
    NSLayoutConstraint.activate([
        menuButton.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor,
↪   constant: 8),
        menuButton.leadingAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.leadingAnchor, constant: 16),
        menuButton.widthAnchor.constraint(equalToConstant: 44),
        menuButton.heightAnchor.constraint(equalToConstant: 44),
    ])
}
```

Затемнение прозрачно (`alpha = 0`) и **не принимает касания** (`isUserInteractionEnabled = false`), пока шторка закрыта. Строго говоря, полностью прозрачный view касаний и так не получает: **hit-testing** (поиск view под пальцем, глава 0, раздел 0.3) пропускает view с `isHidden = true`, с `alpha` меньше 0,01 и с выключенным `isUserInteractionEnabled` — так написано в документации `hitTest(_:with:)`, и мы это проверили в рантайме: при `alpha = 0` `hitTest` возвращает view под затемнением. Но во время перетаскивания шторки `alpha` станет больше нуля (до 0,4), и без явного `isUserInteractionEnabled = false` вуаль начала бы перехватывать касания у контента ещё до того, как шторка открылась. Поэтому приём касаний включаем руками и только вместе со шторкой (в `setDrawer`). Прозрачный **фон** (`backgroundColor = .clear`) от касаний, кстати, не спасает — hit-testing смотрит на геометрию, а не на то, что нарисовано.

`insertSubview(menuButton, belowSubview: overlay)` — кнопка над контентом, но под затемнением: при открытой шторке она тоже тускнеет вместе с контентом.

`UIButton.Configuration.filled()` (iOS 15+) — готовый стиль «залитая кнопка», `.capsule` — скругление-капсула.

Шторка и главный приём — **constraint, который двигаем**:

```
private func setupDrawer() {
    drawer.backgroundColor = .secondarySystemBackground
    drawer.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(drawer)

    drawerLeading = drawer.leadingAnchor.constraint(equalTo: view.leadingAnchor,
                                                    constant: -drawerWidth)
    NSLayoutConstraint.activate([
        drawerLeading,
        drawer.topAnchor.constraint(equalTo: view.topAnchor),
```

```
        drawer.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        drawer.widthAnchor.constraint(equalToConstant: drawerWidth),
    ])

    let list = UIStackView()
    list.axis = .vertical
    list.spacing = 4
    list.translatesAutoresizingMaskIntoConstraints = false
    drawer.addSubview(list)
    NSLayoutConstraint.activate([
        list.topAnchor.constraint(equalTo: drawer.safeAreaLayoutGuide.topAnchor,
↪   constant: 16),
        list.leadingAnchor.constraint(equalTo: drawer.leadingAnchor, constant: 12),
        list.trailingAnchor.constraint(equalTo: drawer.trailingAnchor, constant: -12),
    ])

    for (index, tab) in tabs.enumerated() {
        var config = UIButton.Configuration.plain()
        config.title = tab.title
        config.image = UIImage(systemName: tab.icon)
        config.imagePadding = 12
        let button = UIButton(configuration: config)
        button.contentHorizontalAlignment = .leading
        button.tintColor = tab.color
        button.addAction(UIAction { [weak self] _ in
            self?.showTab(at: index)
            self?.setDrawer(open: false)
        }, for: .touchUpInside)
        list.addArrangedSubview(button)
        itemButtons.append(button)
    }
}
```

Шторка шириной 260 точек. Её левый край = левый край экрана **минус 260**: шторка целиком за левой границей, её правый край ровно в точке 0. Откроем — поставим `constant = 0`, и шторка займёт полосу от 0 до 260. На iPhone 16 (393 точки) справа останется 393 − 260 = 133 точки затемнённого контента — видно, что под шторкой «что-то есть», и в эту полосу удобно тапнуть, чтобы закрыть.

Шторка высотой во весь экран, а список внутри прибит к её safe area — пункты не залезут под статус-бар.

Открыть/закрыть с анимацией:

```
private func setDrawer(open: Bool) {
    isOpen = open
    drawerLeading.constant = open ? 0 : -drawerWidth
    overlay.isUserInteractionEnabled = open
    // VoiceOver: пока шторка открыта, читаем только её.
```

```
    drawer.accessibilityViewIsModal = open
    UIView.animate(withDuration: 0.3, delay: 0,
                   usingSpringWithDamping: 0.9, initialSpringVelocity: 0) {
        self.view.layoutIfNeeded()
        self.overlay.alpha = open ? 0.4 : 0
    }
    UIAccessibility.post(notification: .screenChanged, argument: open ? drawer : nil)
}
```

Тот же приём «поменяй constant → `layoutIfNeeded()` внутри анимации», что и с индикатором в 20.4.

`usingSpringWithDamping: 0.9` — **пружинная** анимация. Damping («затухание») — насколько пружина гасит колебания: 1,0 — движение приходит в цель плавно и без перелёта, чем меньше число, тем сильнее «перелетает и возвращается». 0,9 — почти без перелёта, но конец движения мягче, чем у обычной кривой. Для сравнения, 0,5 — заметный «отскок», для шторки слишком игриво. `initialSpringVelocity: 0` — начинаем с места, без начального толчка.

`overlay.alpha = 0.4` — затемнение на 40%: контент виден, но явно «на заднем плане».

`accessibilityViewIsModal = true` — для VoiceOver шторка становится «модальной»: диктор не прочитает контент под ней, пока она открыта. `UIAccessibility.post(notification: .screenChanged, argument: drawer)` — просим VoiceOver перевести фокус на шторку.

Ещё одна мелочь для VoiceOver — стандартный жест «назад» (два пальца рисуют букву «Z»):

```
override func accessibilityPerformEscape() -> Bool {
    guard isOpen else { return false }
    setDrawer(open: false)
    return true
}
```

Возвращаем `true` — «жест обработан».

Жесты

Три жеста: провести от левого края, чтобы открыть; потянуть шторку или затемнение влево, чтобы закрыть; тапнуть по затемнению.

```
private func setupGestures() {
    // Открыть: провести пальцем от левого края экрана.
    let edgePan = UIScreenEdgePanGestureRecognizer(target: self, action:
     ↪  #selector(handlePan(_:)))
    edgePan.edges = .left
    view.addGestureRecognizer(edgePan)

    // Закрыть: потянуть саму шторку или затемнение влево.
    drawer.addGestureRecognizer(UIPanGestureRecognizer(target: self, action:
↪   #selector(handlePan(_:))))
    overlay.addGestureRecognizer(UIPanGestureRecognizer(target: self, action:
↪   #selector(handlePan(_:))))
```

```
    // Закрыть: тап по затемнению.
    overlay.addGestureRecognizer(UITapGestureRecognizer(target: self, action:
↪   #selector(overlayTapped)))
}

@objc private func overlayTapped() {
    setDrawer(open: false)
}
```

Pan («панорамирование») — жест «положил палец и тянешь». `UIScreenEdgePanGestureRecognizer` — его вариант, который срабатывает только если палец начал движение у самого края экрана. Мы открываем шторку именно от края, а не любым горизонтальным свайпом: иначе жест конфликтовал бы с листанием внутри контента (каруселями, удалением ячеек свайпом).

Все три pan-жеста ведут в один обработчик — логика «тащим шторку» одинаковая.

Распознаватель жестов — не view (`UIGestureRecognizer → NSObject`, глава 0, раздел 0.8); он вешается на **одну** view (`gesture.view`) и получает те касания, которые hit-testing отдал этой view или любой её subview. У `edgePan` такая view — весь `view` контейнера, поэтому он срабатывает и когда палец начал движение над контентом ребёнка. Пан на `drawer` и `overlay` работает, только когда эти view сами принимают касания: у `overlay` это происходит лишь при открытой шторке.

```
@objc private func handlePan(_ gesture: UIPanGestureRecognizer) {
    let translation = gesture.translation(in: view).x
    switch gesture.state {
    case .began:
        panStartConstant = drawerLeading.constant
    case .changed:
        let target = panStartConstant + translation
        drawerLeading.constant = max(-drawerWidth, min(0, target))
        overlay.alpha = (drawerLeading.constant + drawerWidth) / drawerWidth * 0.4
    case .ended, .cancelled:
        let velocity = gesture.velocity(in: view).x          // точек в секунду
        let shouldOpen: Bool
        if velocity > 500 {
            shouldOpen = true                                // резкий бросок вправо
        } else if velocity < -500 {
            shouldOpen = false                               // резкий бросок влево
        } else {
            shouldOpen = drawerLeading.constant > -drawerWidth / 2
        }
        setDrawer(open: shouldOpen)
    default:
        break
    }
}
```

`translation(in:)` — на сколько точек палец сдвинулся **с начала жеста**. Вправо — положительное число, влево — отрицательное.

`.began` — запоминаем, где была шторка в момент касания. Дальше позиция = «где была» + «на сколько сдвинули палец». Первая версия главы вместо этого брала «0 или −260 в зависимости от `isOpen`»; если схватить шторку во время анимации (она на полпути), та прыгала к краю.

`max(-drawerWidth, min(0, target))` — **зажим** значения в диапазон от −260 до 0. `min(0, …)` не пускает шторку правее полностью открытого положения, `max(-260, …)` — левее полностью закрытого. Пример: шторка закрыта (−260), палец уехал вправо на 100 → target = −160, в пределах, шторка на −160. Палец уехал на 400 → target = 140, `min(0, 140)` = 0: шторка упёрлась, дальше не едет. (Эффект «резинки», когда шторка чуть тянется за предел и возвращается, тоже можно сделать, но это отдельная задача.)

Затемнение пропорционально открытости. Формула `(constant + 260) / 260 * 0.4` читается так: «какая доля шторки видна» × 40%. Числами:

- `constant = -260` (закрыта): (−260 + 260) / 260 = 0 → alpha 0;
- `constant = -130` (наполовину): 130 / 260 = 0,5 → alpha 0,5 × 0,4 = 0,2;
- `constant = 0` (открыта): 260 / 260 = 1 → alpha 0,4.

Когда палец отпустили (`.ended`), решаем, куда «докатить» шторку. Сначала смотрим на **скорость** жеста — `velocity(in:)`, в точках в секунду. 500 точек/с — это порог «резкого броска»: с такой скоростью шторка шириной 260 точек проехала бы целиком примерно за полсекунды (260 / 500 ≈ 0,52 с). Быстрый бросок вправо — открываем, даже если протащили всего на треть; быстро влево — закрываем. Без учёта скорости шторка ощущается «тугой»: пользователь смахнул, а она вернулась назад, потому что не дотянули до середины.

Если бросок медленный — решает положение: `constant > -130` (открыта больше чем наполовину) → открываем, иначе закрываем.

`.cancelled` обрабатываем так же, как `.ended`: жест могла прервать система (например, входящий звонок), шторка не должна застрять посередине.

Смена раздела — уже знакомый ритуал:

```
private func showTab(at index: Int) {
    guard index != selectedIndex else { return }
    selectedIndex = index
    currentChild?.unembed()
    let child = TabContentViewController(tab: tabs[index])
    embed(child, in: contentHost)
    currentChild = child
    for (i, button) in itemButtons.enumerated() {
        button.configuration?.baseForegroundColor = i == index ? tabs[i].color : .label
        button.accessibilityTraits = i == index ? [.button, .selected] : .button
    }
}
```

Здесь намеренно без кеша: в шторке разделы переключают реже, и на примере видно, что экран можно и пересоздавать. `selectedIndex` стартует с −1, чтобы самый первый `showTab(at: 0)` не отсёкся проверкой `guard`.

Упражнение 20.2. Поменяй в `handlePan` порог скорости с 500 на 2000 и попробуй резко смахнуть шторку от края, протащив её всего на четверть ширины. Что изменилось в поведении? Потом верни 500. Ответ — в конце главы.

20.6 Все три в одном демо — Segmented control

`CustomTabBarViewController` даёт выбрать стиль сегментированным переключателем в навигационной панели:

```
final class CustomTabBarViewController: UIViewController {
    private enum Style: Int, CaseIterable {
        case floating, topTabs, drawer

        var label: String {
            switch self {
            case .floating: return "Плавающий"
            case .topTabs:  return "Сверху"
            case .drawer:   return "Боковое меню"
            }
        }
    }

    private let segmented = UISegmentedControl(items: Style.allCases.map(\.label))
    private var currentContainer: UIViewController?

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
        segmented.selectedSegmentIndex = Style.floating.rawValue
        segmented.addTarget(self, action: #selector(styleChanged), for: .valueChanged)
        navigationItem.titleView = segmented
        switchTo(style: .floating)
    }

    @objc private func styleChanged() {
        guard let style = Style(rawValue: segmented.selectedSegmentIndex) else { return }
        switchTo(style: style)
    }

    private func switchTo(style: Style) {
        currentContainer?.unembed()

        let container: UIViewController
        switch style {
        case .floating: container = FloatingTabBarContainer(tabs: Self.makeTabs())
        case .topTabs:  container = TopTabsContainer(tabs: Self.makeTabs())
        case .drawer:   container = DrawerContainer(tabs: Self.makeTabs())
        }
```

```
        embed(container)
        currentContainer = container
    }
}

extension CustomTabBarViewController {
    static func makeTabs() -> [Tab] {
        [
            Tab(title: "Главная", icon: "house.fill", color: .systemBlue),
            Tab(title: "Поиск", icon: "magnifyingglass", color: .systemPurple),
            Tab(title: "Лента", icon: "rectangle.stack.fill", color: .systemPink),
            Tab(title: "Профиль", icon: "person.crop.circle.fill", color: .systemTeal),
        ]
    }
}
```

`enum Style: Int, CaseIterable` — `Int` даёт каждому случаю номер (0, 1, 2), который совпадает с номером сегмента. `CaseIterable` даёт `Style.allCases` — массив всех стилей, из которого строим подписи сегментов.

`map(\.label)` — «возьми у каждого элемента свойство `label`». Запись `\.label` — ссылка на свойство (key path), короче, чем `{ $0.label }`.

`Style(rawValue:)` возвращает опционал: номера 7 среди стилей нет. Отсюда `guard let`.

Контейнер сам становится ребёнком! Получается **контейнер контейнеров**: `CustomTabBarViewController` → `FloatingTabBarContainer` → `TabContentViewController`. Ритуал «add / will move → remove» работает на любой глубине: `embed` и `unembed` из 20.1 не знают и не хотят знать, что у ребёнка есть свои дети.

`segmented.addTarget(self, action: #selector(styleChanged), for: .valueChanged)` — классический target-action: `UISegmentedControl` → `UIControl`, событие `.valueChanged` приходит при смене выбранного сегмента (глава 0, раздел 0.4).

`navigationItem.titleView = segmented` — сегмент в центре навигационной панели вместо заголовка. `navigationItem` — свойство самого экрана, объект класса `UINavigationItem` (наследник `NSObject`, не view); навигационная панель лишь показывает item того экрана, который сейчас сверху стека (глава 0, разделы 0.6 и 0.8). Экран должен быть внутри `UINavigationController` — в playground'е так и есть. Экран без навигационной панели сегмент просто не покажет.

> **Как мы проверяли.** Все пять файлов главы (`Containment`, `Tab`, `FloatingTabBarContainer`, `TopTabsContainer`, `DrawerContainer`, `CustomTabBarViewController`) собраны в отдельном проекте командой `xcodebuild` для симулятора, iOS 15.0 deployment target, Swift 6 с MainActor по умолчанию — без ошибок и предупреждений.

20.7 Бытовая аналогия

Container VC — это **шкаф с полками**. Стандартный `UITabBarController` — шкаф из магазина: пять одинаковых полок, собирается за минуту, стоит в каждом доме. Если нужна верхняя сек-

ция для книг, средняя для журналов и выдвижной ящик для папок — заказываешь шкаф под себя. Дольше, дороже, зато ровно под твою комнату.

Floating capsule — **подставка для пульта** на диване. Маленькая, всегда под рукой, не загораживает телевизор.

Top tabs — **закладки в записной книжке**. Все разделы видны сверху, между соседними можно перелистывать.

Drawer — **выдвижной ящик**. По умолчанию закрыт и не мешает. Потянул за ручку — выехал; толкнул — закрылся, даже если толкнул несильно, но резко.

20.8 Что мы пропустили

- **Навигация внутри табов.** Обычно каждый таб — свой `UINavigationController`: тап по ячейке на «Главной» открывает детальный экран внутри «Главной», а не поверх всего приложения. С нашими контейнерами это делается так: вместо `TabContentViewController` кладём `UINavigationController(rootViewController: …)`. В playground'е весь контейнер уже стоит внутри навигационного контроллера из `showMain()` (глава 4), поэтому внешнюю панель тогда прячут (`navigationController?.setNavigationBarHidden(true, animated: false)`), иначе над табом окажутся две панели.
- **Бейджи.** Красный кружок с числом на иконке таба — отдельный маленький `UILabel` поверх кнопки.
- **Меню по долгому нажатию.** У `UIButton` есть свойство `menu` (iOS 14+): долгое нажатие на кнопку таба покажет список действий.
- **Состояние между запусками.** Какой таб был открыт, лучше сохранять (например, в `UserDefaults`) и восстанавливать при старте.
- **iPad.** На большом экране вместо таб-бара чаще используют `UISplitViewController` с боковой колонкой. Адаптивное приложение переключается по **size class** — грубой оценке ширины экрана («compact» на iPhone в портрете, «regular» на iPad).

Упражнение 20.3. Открой «Custom Tab Bar» (коричневая ячейка в лаунчере). Переключи сегмент на «Сверху», пролистай пальцем с «Главной» до «Профиля», потом тапни «Главная». Затем «Боковое меню»: открой шторку кнопкой, закрой тапом по затемнению, открой свайпом от левого края. Что должно происходить — в ответах.

Ответы к упражнениям

Упражнение 20.1. Без `additionalSafeAreaInsets` последние строки таблицы уходят под панель: долистав до конца, ты не увидишь последнюю строку целиком — её закрывает капсула. Со строкой таблица сама добавляет снизу 64 точки отступа (она уважает safe area ребёнка), и последняя строка останавливается над панелью. Для проверки используй экран:

```
final class LongListViewController: UITableViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "row")
    }
    override func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int)
    ↪  -> Int { 50 }
```

```
    override func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath)
    ↪  -> UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "row", for: indexPath)
        var content = cell.defaultContentConfiguration()
        content.text = "Строка \(indexPath.row + 1)"
        cell.contentConfiguration = content
        return cell
    }
}
```

и в `showTab` создавай `LongListViewController()` вместо `TabContentViewController(tab:)`.

Упражнение 20.2. С порогом 2000 точек/с «бросок» почти никогда не срабатывает (это экран шириной 393 точки за 0,2 секунды), и решает только положение: протащил на четверть — шторка вернётся назад, даже если смахнул резко. Ощущение «тугой», непослушной шторки. С порогом 500 короткий резкий свайп доводит шторку до конца — так ведут себя системные листы iOS.

Упражнение 20.3. В «Сверху» при листании пальцем синяя линия под кнопками переезжает под заголовок текущей страницы, а подпись выбранной кнопки темнеет (`.label`), остальные бледнеют. Тап по «Главная» с последней страницы пролистывает страницы **назад** (новая приезжает слева), и линия едет влево. В «Боковом меню» шторка выезжает слева на 260 точек, контент справа затемняется на 40%; тап по затемнению закрывает шторку; свайп от самого левого края тянет шторку за пальцем, и при отпускании больше чем на половину (или резким броском) она открывается.

Что мы выучили

- Container VC: `addChild` → `addSubview` + constraints → `didMove(toParent:)`. Удаление: `willMove(toParent: nil)` → `removeFromSuperview` → `removeFromParent`. Ритуал удобно спрятать в `embed/unembed`.
- Снять только view, не вызвав `removeFromParent()`, — утечка: экран остаётся в `children` родителя. Положить только view без `addChild` — экран без `parent`: `navigationController` и прочие «предки» у него `nil`.
- Hit-testing пропускает view с `isHidden`, `alpha < 0.01` и `isUserInteractionEnabled = false`; прозрачный фон от касаний не спасает. Приём касаний у затемнения включаем явно и только вместе со шторкой.
- Контейнеры вкладываются друг в друга на любую глубину.
- `additionalSafeAreaInsets` — способ сказать ребёнку «снизу занято 64 точки», не зная ничего о его вёрстке.
- **Floating capsule** — `cornerRadius` = половина высоты, ширина складывается из кнопок, отступов и промежутков.
- **Top tabs** — `UIPageViewController` + кнопки + индикатор, привязанный constraint'ом к выбранной кнопке; двусторонняя синхронизация тап ↔ свайп; смотрим на `completed`, а не на `finished`.
- **Drawer** — constraint `leading` от −260 до 0, анимация через `layoutIfNeeded()` внутри `UIView.animate`; пружина с затуханием 0,9 — мягко и без отскока.

- **Pan** — позиция = «где была в начале» + `translation`, зажим `max/min`, решение по скорости (±500 точек/с) и по половине ширины.
- Доступность своего таб-бара — на тебе: `accessibilityLabel`, трейт `.selected`, `accessibilityViewIsModal`, жест «назад».

Apple Developer Documentation

- UIViewController — Implementing a container view controller — официальный раздел про containment: `addChild(_:)`, `didMove(toParent:)`.
- Creating a custom container view controller — статья Apple с тем же ритуалом и примерами.
- addChild(_:) — присоединение ребёнка; пара к нему — `removeFromParent()`.
- willMove(toParent:) — уведомление перед удалением ребёнка; при удалении вызываешь его сам с `nil`.
- didMove(toParent:) — финальный шаг при добавлении ребёнка.
- additionalSafeAreaInsets — дополнительный отступ safe area для ребёнка.
- UITabBarController — стандартный контейнер для сравнения.
- UIPageViewController — листающие страницы для варианта «сверху».
- UIScreenEdgePanGestureRecognizer — жест от края экрана для открытия шторки.
- UIPanGestureRecognizer — `translation(in:)` и `velocity(in:)`.
- UISegmentedControl — переключатель стилей в `navigationItem.titleView`.
- HIG: Tab bars — сколько табов, как их подписывать, когда таб-бар уместен.
- HIG: Navigation bars — что можно класть в центр навигационной панели.

→ Глава 21. Complex Layouts — параллакс, sticky header, stretchy

Глава 21. Complex Layouts — параллакс, sticky header, stretchy

Сложные экраны отличают учебный пример от приложения, которое приятно держать в руках. Большая шапка, которая тянется вслед за пальцем, текст, который движется медленнее фона, заголовки разделов, прилипающие к верху, — всё это делается штатными средствами UIKit, без сторонних библиотек.

В этой главе строим экран в стиле «страница артиста» в музыкальном приложении: сверху градиентная шапка с аватаром и именем, под ней список разделов. Три эффекта:

- **Stretchy header** («тянущаяся шапка») — потянул список вниз дальше начала, шапка растягивается вместе с ним.
- При прокрутке вверх шапка уезжает за верхний край вместе с контентом.
- **Параллакс** — текст внутри шапки движется **с другой скоростью**, чем сама шапка. Разница скоростей создаёт ощущение глубины.

А в конце (21.11) — заголовки разделов, которые прилипают к верху, на `UICollectionViewCompositionalLayout` с расчётом всех размеров на числах.

Экран реализован в mini-app «Сложные экраны» (`StretchyHeaderViewController`).

> **Режим компиляции.** Весь код главы собран в проекте с deployment target iOS 15.0, Swift 6, Default Actor Isolation = MainActor. Числа в главе (координаты, размеры) не «на глаз»: мы запускали экран в симуляторе iPhone 16 на iOS 26.5 и печатали рамки view.

21.1 Архитектура

Сначала два понятия, на которых держится вся глава.

`contentOffset` — насколько содержимое scroll view прокручено. Это точка содержимого, которая сейчас стоит в верхнем левом углу scroll view. Прокрутил на 100 точек вниз по списку — `contentOffset.y = 100`. Под капотом это `bounds.origin` самого scroll view: сдвиг начала `bounds` сдвигает всё содержимое, а `frame` при этом не меняется (замер: `contentOffset.y = 120` → `bounds.origin = (0, 120)`, `frame.origin = (0, 0)`). Подробно о `UIScrollView` и его наследниках — глава 0, раздел 0.5.

`contentInset` — дополнительные поля вокруг содержимого, внутри которых тоже можно прокручивать. `contentInset.top = 280` значит: «над первой строкой есть 280 точек пустого места».

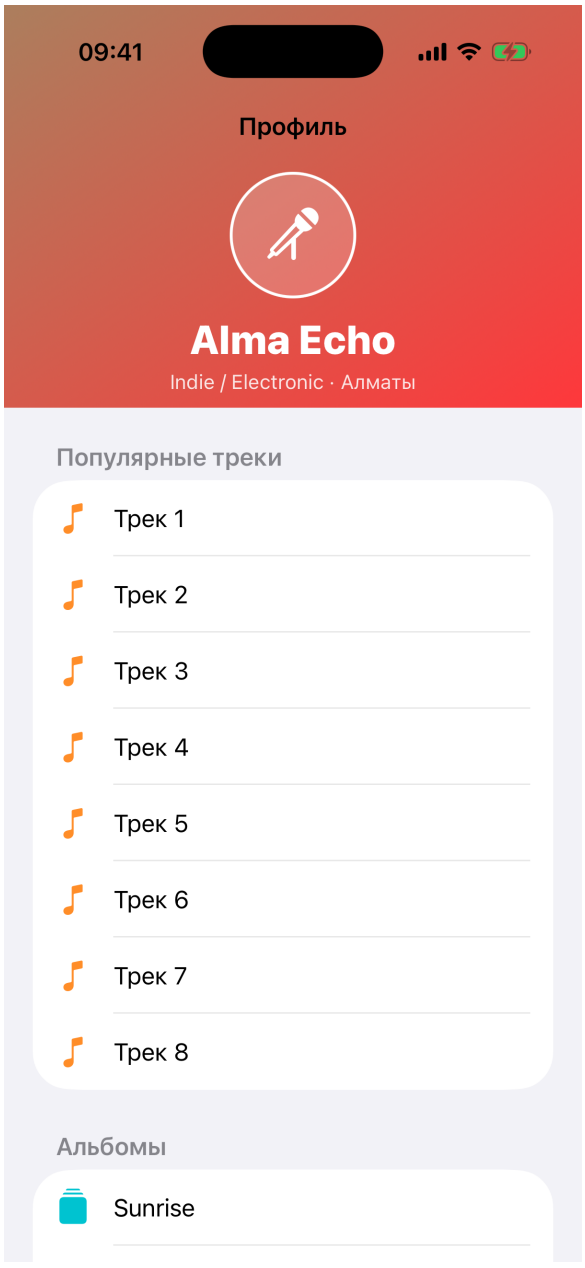
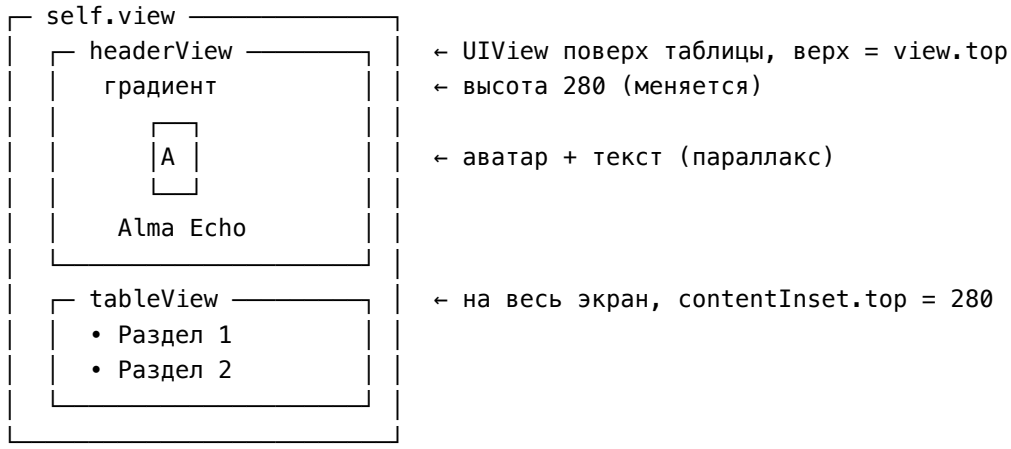


Рис. 17: Stretchy header с параллаксом

Бытовая аналогия: лента конвейера, у которой первые 280 сантиметров специально оставлены пустыми.

Идея экрана: **шапка — отдельный view**, не часть таблицы. Она лежит **поверх** таблицы и прибита к верху экрана. Таблица получает `contentInset.top = 280` — ровно высоту шапки, — поэтому первая строка в начальном положении стоит сразу под шапкой.



Когда пользователь прокручивает таблицу, мы ловим `scrollViewDidScroll` и меняем высоту и положение шапки через два constraint'а.

21.2 Сетап — два «слоя»

```
final class StretchyHeaderViewController: UIViewController {
    private let tableView = UITableView(frame: .zero, style: .insetGrouped)
    private let headerView = StretchyHeaderView()
    private let initialHeaderHeight: CGFloat = 280
    private var headerHeightConstraint: NSLayoutConstraint!
    private var headerTopConstraint: NSLayoutConstraint!

    private let sections: [(title: String, rows: [String])] = [
        ("О профиле", ["Артист · Алматы", "12 альбомов", "1,2 млн слушателей"]),
        ("Популярное", (1...8).map { "Трек \($0)" }),
        ("Альбомы", (1...6).map { "Альбом \($0)" }),
    ]

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemGroupedBackground
        setupHeader()
        setupTable()
    }

    private func setupHeader() {
        headerView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(headerView)
```

```
        headerHeightConstraint = headerView.heightAnchor.constraint(equalToConstant:
↪   initialHeaderHeight)
        headerTopConstraint = headerView.topAnchor.constraint(equalTo: view.topAnchor)
        NSLayoutConstraint.activate([
            headerTopConstraint,
            headerView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            headerView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            headerHeightConstraint,
        ])
    }

    private func setupTable() {
        tableView.translatesAutoresizingMaskIntoConstraints = false
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "row")
        tableView.dataSource = self
        tableView.delegate = self
        tableView.contentInset.top = initialHeaderHeight
        tableView.verticalScrollIndicatorInsets.top = initialHeaderHeight
        tableView.contentInsetAdjustmentBehavior = .never
        // Отступ сам по себе прокрутку не сдвигает — ставим начальную позицию явно.
        tableView.contentOffset.y = -initialHeaderHeight
        tableView.backgroundColor = .clear
        view.insertSubview(tableView, belowSubview: headerView)
        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])
    }

    // .never выключил и нижний автоматический отступ — возвращаем его руками,
    // иначе последняя строка спрячется под полоской «домой».
    override func viewSafeAreaInsetsDidChange() {
        super.viewSafeAreaInsetsDidChange()
        tableView.contentInset.bottom = view.safeAreaInsets.bottom
        tableView.verticalScrollIndicatorInsets.bottom = view.safeAreaInsets.bottom
    }
}
```

Ключевые моменты.

`sections` — данные экрана: массив кортежей «заголовок раздела + строки». Для учебного экрана кортежа хватает; в настоящем приложении здесь была бы модель из сети.

Стиль `.insetGrouped` — разделы в виде скруглённых карточек с отступами по бокам, как в «Настройках». Важнее другое: у сгруппированных стилей заголовки разделов **не прилипают** к верху при прокрутке. Почему это нам важно — в 21.7.

`contentInsetAdjustmentBehavior = .never` — по умолчанию scroll view сам добавляет к своим отступам **safe area** (полосу под статус-баром и навигационной панелью сверху, под полоской «домой» снизу). Итоговый отступ — «наш + автоматический» — лежит в `adjustedContentInset`. Мы выключаем автоматику, чтобы верхний отступ был ровно 280 и формулы ниже работали на простых числах.

`tableView.contentOffset.y = -initialHeaderHeight` — неочевидная, но обязательная строка. Отступ сам по себе прокрутку не сдвигает: таблица создаётся с `contentOffset.y = 0`, то есть первые 280 точек содержимого (раздел «О профиле» целиком) оказываются **под шапкой**. Мы поймали это в симуляторе: без строки экран открывался сразу с раздела «Популярное», а `contentOffset.y` был 0 вместо −280. Первая версия главы этой строки не имела — и первый раздел был спрятан, пока пользователь не начинал листать. Ставим начальную позицию явно: «видимая область начинается на 280 точек выше содержимого».

У `.never` есть обратная сторона, которую первая версия главы не учла: выключается и **нижний** автоматический отступ. Последние строки уходили под полоску «домой» (34 точки на iPhone 16). Поэтому `viewSafeAreaInsetsDidChange()` — UIKit вызывает его, когда safe area экрана меняется (при первом показе, при повороте) — возвращает нижний отступ руками: `contentInset.bottom = 34`.

`verticalScrollIndicatorInsets` (iOS 11.1+) — отступы полосы прокрутки. Иначе полоса начиналась бы с самого верха, «поверх» шапки.

`backgroundColor = .clear` у таблицы — фон даёт `view` (`systemGroupedBackground`), а таблица прозрачна. Так в момент растяжения под шапкой не мелькает другой цвет.

`view.insertSubview(tableView, belowSubview: headerView)` — таблица **под** шапкой по z-порядку (порядку наложения). Шапка рисуется поверх.

`headerTopConstraint` и **`headerHeightConstraint`** — два constraint'а, которыми будем «дышать»: менять `.constant` при прокрутке. Неявно развёрнутые опционалы (`!`), потому что создаются в `setupHeader()`, а не в инициализаторе; до `viewDidLoad` к ним никто не обращается.

21.3 ScrollViewDelegate — главная логика

`UITableView` — наследник `UIScrollView` (цепочку мы проверили в рантайме, см. главу 22 и главу 0, раздел 0.5), а протокол `UITableViewDelegate` наследует `UIScrollViewDelegate` — в заголовке `UITableView.h` так и написано: `@protocol UITableViewDelegate<NSObject, UIScrollViewDelegate>` (глава 0, раздел 0.9). Поэтому делегат таблицы получает и события прокрутки, а отдельного «делегата прокрутки» таблице назначать не нужно (и нельзя: `delegate` у неё один):

```
extension StretchyHeaderViewController: UITableViewDataSource, UITableViewDelegate {
    func numberOfSections(in tableView: UITableView) -> Int { sections.count }

    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        sections[section].rows.count
    }

    func tableView(_ tableView: UITableView, titleForHeaderInSection section: Int) ->
    ↪ String? {
        sections[section].title
```

```
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪ UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "row", for: indexPath)
        var content = cell.defaultContentConfiguration()
        content.text = sections[indexPath.section].rows[indexPath.row]
        cell.contentConfiguration = content
        return cell
    }

    func scrollViewDidScroll(_ scrollView: UIScrollView) {
        let offset = -(scrollView.contentOffset.y + scrollView.adjustedContentInset.top)
        if offset > 0 {
            // Тянут вниз: верх шапки стоит, шапка вытягивается.
            headerHeightConstraint.constant = initialHeaderHeight + offset
            headerTopConstraint.constant = 0
        } else {
            // Листают вверх: шапка целиком уезжает за верхний край.
            headerHeightConstraint.constant = initialHeaderHeight
            headerTopConstraint.constant = offset
        }
        headerView.applyParallax(offset: offset)
    }
}
```

Первые четыре метода — обычный data source таблицы: сколько разделов, сколько строк, заголовок раздела, ячейка. Главное — `scrollViewDidScroll`. Его вызывают при каждом изменении `contentOffset`; во время прокрутки это **каждый кадр** — до 60 раз в секунду (на экранах ProMotion — до 120), поэтому внутри только арифметика и два присваивания.

Разберём формулу на числах. У нас `contentInset.top = 280` и автоматика выключена, значит `adjustedContentInset.top` тоже 280.

`contentOffset.y` при таком отступе:

- **Начальное положение** (шапка целиком, первая строка сразу под ней): `contentOffset.y = -280`. Минус — потому что над содержимым 280 точек пустого поля, и видимая область начинается «выше нуля».
- **Потянули вниз** на 100 точек дальше начала: `contentOffset.y = -380`.
- **Прокрутили вверх** на 50 точек: `contentOffset.y = -230`.

`offset = -(contentOffset.y + 280)`:

- Начало: −(−280 + 280) = 0.
- Потянули на 100: −(−380 + 280) = −(−100) = **100** — положительный.
- Прокрутили на 50: −(−230 + 280) = −(50) = **−50** — отрицательный.

То есть `offset` — «насколько ушли от начального положения»: плюс — тянут вниз дальше начала, минус — листают вглубь списка. Внешний минус в формуле нужен только для удобного знака: «тянут вниз» = положительное число.

Тянут вниз (offset > 0)

Шапка **растягивается**. Верх стоит на месте, растёт высота:

- `headerHeightConstraint.constant = 280 + offset` → при `offset = 100` высота 380;
- `headerTopConstraint.constant = 0` → верх на верху экрана.

Замер в симуляторе: при `offset = 100` рамка шапки — `(0, 0, 393, 380)`, первая строка таблицы начинается на 380 + отступ секции — ровно под шапкой. Аватар и текст привязаны к **низу** шапки (21.6), поэтому уезжают вниз вместе с ним, а сверху открывается больше градиента.

Листают вверх (offset < 0)

Шапка **уезжает** наверх вместе с контентом:

- `headerHeightConstraint.constant = 280` — высота не меняется;
- `headerTopConstraint.constant = offset` → при `offset = −50` верх шапки на −50, то есть за верхним краем экрана.

Шапка занимает полосу от −50 до 230: верхние 50 точек ушли за край, низ шапки (230) совпадает с началом первой строки (280 − 50 = 230). Шапка и таблица двигаются как одно целое.

> **Ошибка, которую легко совершить.** Первая идея — при прокрутке вверх уменьшать высоту шапки до минимума (скажем, 64) и не двигать верх. Шапка съёживается и остаётся наверху, а `contentInset.top` по-прежнему 280 — между низом шапки и первой строкой образуется пустая полоса. Шапка должна **уехать**, а не съёжиться. (Сворачивающаяся шапка тоже бывает, но тогда одновременно меняют и `contentInset`, и это отдельный рецепт.)

21.4 Шапка не должна ловить касания

Шапка лежит **поверх** таблицы. Что будет, если пользователь положит палец на градиент и потянет вниз?

Когда палец касается экрана, UIKit ищет, какому view достался тап. Процесс называется **hit-testing** («проверка попадания»): окно вызывает у себя `hitTest(_:with:)`, тот проверяет `point(inside:with:)` и спускается в subviews — **с конца массива `subviews`**, то есть сначала к тому, что нарисовано выше, — и так до самого глубокого view, которое содержит точку. Кто первым нашёлся сверху, тот и получает касание. По документации `hitTest` пропускает view с `isHidden = true`, с `alpha` меньше 0,01 и с `isUserInteractionEnabled = false`, а точку за пределами `bounds` view не считает попаданием, даже если там торчит её subview (подробно — глава 0, раздел 0.3). Жест прокрутки таблицы (`UIPanGestureRecognizer` внутри `UIScrollView`) получает только те касания, которые достались самой таблице или её subview.

Замер в симуляторе: точка (100; 100) над таблицей, накрытой обычным `UIView` высотой 280, — `hitTest` возвращает накрывающий `UIView`. Шапка добавлена **позже** таблицы (она выше в `subviews`), и hit-testing доходит до неё первой. То есть тянуть таблицу за шапку **нельзя**: палец «застревает» на шапке, и потянуть таблицу вниз получится, только если начинать жест ниже шапки. Прозрачный фон не помог бы: hit-testing смотрит на геометрию, а не на то, что нарисовано.

Лечение — одна строка в шапке:

```
isUserInteractionEnabled = false
```

С ней тот же замер возвращает `UITableView`: касание проходит сквозь шапку к таблице. Шапке касания не нужны — в ней нет кнопок. Если кнопки в шапке появятся (например, «Подписаться»), понадобится переопределить `hitTest(_:with:)` у шапки: возвращать кнопку, если палец на ней, и `nil` для всего остального.

21.5 StretchyHeaderView — градиент, аватар, текст

```
final class StretchyHeaderView: UIView {
    // Слой самого view — сразу градиент. Размер он берёт от view без нашей помощи.
    override class var layerClass: AnyClass { CAGradientLayer.self }
    private var gradientLayer: CAGradientLayer { layer as! CAGradientLayer }

    private let avatar = UIView()
    private let avatarSymbol = UIImageView(image: UIImage(systemName: "music.mic"))
    private let titleLabel = UILabel()
    private let subtitleLabel = UILabel()

    override init(frame: CGRect) {
        super.init(frame: frame)
        clipsToBounds = true
        // Шапка — только картинка. Все касания проходят сквозь неё к таблице.
        isUserInteractionEnabled = false

        gradientLayer.startPoint = CGPoint(x: 0, y: 0)
        gradientLayer.endPoint = CGPoint(x: 1, y: 1)
        updateGradientColors()

        avatar.backgroundColor = UIColor.white.withAlphaComponent(0.25)
        avatar.layer.cornerRadius = 44
        avatarSymbol.tintColor = .white
        avatarSymbol.preferredSymbolConfiguration =
↪   UIImage.SymbolConfiguration(pointSize: 36, weight: .semibold)

        titleLabel.text = "Alma Echo"
        titleLabel.font = .systemFont(ofSize: 28, weight: .bold)
        titleLabel.textColor = .white
        subtitleLabel.text = "Артист · 1,2 млн слушателей"
        subtitleLabel.font = .systemFont(ofSize: 15, weight: .medium)
        subtitleLabel.textColor = UIColor.white.withAlphaComponent(0.85)

        for v in [avatar, avatarSymbol, titleLabel, subtitleLabel] {
            v.translatesAutoresizingMaskIntoConstraints = false
        }
        addSubview(avatar)
        avatar.addSubview(avatarSymbol)
```

```
        addSubview(titleLabel)
        addSubview(subtitleLabel)

        // ... constraints — в 21.6
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    // CGColor не умеет «перекрашиваться» под тему — делаем это сами.
    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateGradientColors()
        }
    }

    private func updateGradientColors() {
        gradientLayer.colors = [
            UIColor.systemBrown.resolvedColor(with: traitCollection).cgColor,
            UIColor.systemRed.resolvedColor(with: traitCollection).cgColor,
        ]
    }
}
```

`CAGradientLayer` — слой Core Animation, который рисует градиент. У каждого `UIView` есть ровно один **layer** (слой) — то, что реально рисуется на экране; view управляет им и добавляет касания и Auto Layout. Сам слой — не view и не responder: `CAGradientLayer → CALayer → NSObject`, у него нет constraints и он не получает касаний (глава 0, разделы 0.3 и 0.8).

`override class var layerClass` — говорим UIKit: «слой этого view — не обычный `CALayer`, а `CAGradientLayer`». UIKit читает это свойство один раз, при создании view, поэтому поменять класс слоя у готового view уже нельзя. Замер: у обычного `UIView` слой типа `CALayer`, у `StretchyHeaderView` — `CAGradientLayer`. Слой самого view всегда совпадает с ним по размеру (`frame` и `bounds` view — это и есть геометрия его слоя), поэтому при растяжении шапки градиент растягивается сам.

Градиент можно сделать **отдельным** слоем (`layer.insertSublayer(gradient, at: 0)`) и обновлять его размер в `layoutSubviews`. Так делают часто, но у отдельного слоя есть подвох: любое изменение его `frame` по умолчанию **неявно анимируется** за 0,25 секунды (`CATransaction.animationDuration()`). Замер: после изменения размера у такого слоя появились анимации `position` и `bounds`, у `layerClass`-варианта — ни одной. Причина в делегате: слой перед изменением спрашивает у делегата `action(forKey:)`, а делегат слоя view — сама view, и вне блока `UIView.animate` она отвечает «анимации нет» (замер: `view.layer.action(forKey: "bounds")` возвращает `nil`, у отдельного слоя — готовый `CABasicAnimation`). При растяжении шапки пальцем отдельный градиент отставал бы от краёв. `layerClass` убирает и `layoutSubviews`, и эту проблему.

`layer as! CAGradientLayer` — принудительное приведение типа. Обычно `as!` — повод насторожиться, но здесь класс слоя мы задали сами строкой выше, и приведение не может не сработать.

`clipsToBounds = true` — всё, что выходит за рамку шапки, обрезается. Параллакс сдвигает текст, и при сильном смещении он мог бы вылезти за шапку на таблицу.

Параметры градиента:

- `colors` — массив `CGColor`, не `UIColor`: Core Animation работает с более низкоуровневыми цветами.
- `startPoint` / `endPoint` — начало и конец градиента **в долях** слоя, а не в точках: (0; 0) — левый верхний угол, (1; 1) — правый нижний. Градиент идёт по диагонали; при любом размере шапки — из угла в угол.

Цвета и тёмная тема. `UIColor.systemBrown` — **динамический** цвет: в светлой и тёмной теме у него разные оттенки. А `CGColor` — «снимок» цвета на момент вызова, он не знает о темах. Первая версия главы брала `.cgColor` один раз в `init` — после переключения темы градиент оставался старым. Теперь `resolvedColor(with: traitCollection)` берёт вариант цвета для текущей темы, а `traitCollectionDidChange` перекрашивает градиент при смене. **Trait collection** — набор характеристик окружения view: тема, размер шрифта, size class. `hasDifferentColorAppearance` — «поменялось ли что-то, что влияет на цвета».

(В iOS 17 `traitCollectionDidChange` помечен устаревшим в пользу `registerForTraitChanges`. При deployment target iOS 15 компилятор не предупреждает, и метод продолжает работать; когда поднимешь минимальную версию до 17, перейди на новый API.)

21.6 Auto Layout внутри шапки

```
NSLayoutConstraint.activate([
    avatar.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -76),
    avatar.centerXAnchor.constraint(equalTo: centerXAnchor),
    avatar.widthAnchor.constraint(equalToConstant: 88),
    avatar.heightAnchor.constraint(equalToConstant: 88),
    avatarSymbol.centerXAnchor.constraint(equalTo: avatar.centerXAnchor),
    avatarSymbol.centerYAnchor.constraint(equalTo: avatar.centerYAnchor),

    titleLabel.topAnchor.constraint(equalTo: avatar.bottomAnchor, constant: 12),
    titleLabel.centerXAnchor.constraint(equalTo: centerXAnchor),

    subtitleLabel.topAnchor.constraint(equalTo: titleLabel.bottomAnchor, constant: 4),
    subtitleLabel.centerXAnchor.constraint(equalTo: centerXAnchor),
])
```

Всё привязано к **низу** шапки, а не к верху. Посчитаем положение элементов в шапке высотой 280:

- низ аватара — на 76 точек выше низа шапки: 280 − 76 = **204**;
- аватар 88×88, значит его верх: 204 − 88 = **116**; `cornerRadius = 44` (половина от 88) — круг;
- верх имени — на 12 ниже аватара: 204 + 12 = **216** (замер в симуляторе: `title.minY = 216`);

- подзаголовок — ещё на 4 точки ниже имени.

Имя и подзаголовок вместе занимают около 56 точек (две строки текста 28 и 15 точек плюс 4 между ними), так что под подзаголовком остаётся примерно 8 точек до низа шапки. Отсюда и число 76: 12 + 56 + 8.

Почему низ? Когда шапка растягивается до 380, верх остаётся на 0, а низ уезжает на 380. Если бы аватар был привязан к верху, он остался бы на 116, а внизу открылась бы пустая полоса градиента в 100 точек — между аватаром и первой строкой. С привязкой к низу аватар и текст едут вниз вместе с краем шапки и всё время стоят прямо над списком, а растягивается пустой градиент **сверху**. Замер: при растяжении на 100 точек имя переместилось с 216 на 296 — на 80, а не на 100. Недостающие 20 точек — это параллакс (21.7).

Можно центрировать содержимое по `centerY` — тогда при растяжении оно будет держаться посередине шапки. Другой характер: «подплывает», а не «прижат к списку».

`avatarSymbol` центрирован внутри аватара — это просто иконка SF Symbol поверх полупрозрачного белого круга.

Упражнение 21.1. Поменяй привязку аватара на `avatar.topAnchor.constraint(equalTo: topAnchor, constant: 116)` (то же положение в покое). Потяни список вниз на пару сантиметров. Что изменилось? Ответ — в конце главы.

21.7 Параллакс — текст «отстаёт» от шапки

```
func applyParallax(offset: CGFloat) {
    let limited = max(-150, min(150, offset))
    let shift = CGAffineTransform(translationX: 0, y: -limited * 0.2)
    titleLabel.transform = shift
    subtitleLabel.transform = shift
}
```

Параллакс — когда разные слои движутся с разной скоростью. Из окна поезда близкие столбы проносятся мимо, а дальние горы едва смещаются. Мозг читает разницу скоростей как глубину: что медленнее — то дальше.

Здесь низ шапки движется со скоростью пальца (коэффициент 1,0), а текст — на 20% медленнее, со скоростью 0,8. Текст как будто лежит **глубже** шапки.

Как это считается. `transform` — **аффинное преобразование**: сдвиг, поворот или масштаб, наложенный на view поверх Auto Layout. Constraints не меняются, view просто рисуется в другом месте: документация `UIView.transform` говорит, что Auto Layout считает рамку выравнивания по **непреобразованной** view, а `frame` при `transform`, отличном от identity, не определён и читать его не стоит. `center` и `bounds` при этом не трогаются (замер: сдвиг на −20 по вертикали — `center` тот же, `bounds` те же, `frame.origin.y` уехал на 20). `CGAffineTransform(translationX: 0, y: d)` — сдвиг на `d` точек по вертикали (плюс — вниз, минус — вверх).

На числах:

- **Тянем вниз на 100** (`offset = 100`): низ шапки и текст (через constraints) уехали вниз на 100. Сдвиг текста: −100 × 0,2 = −20, то есть на 20 вверх. Итого текст сместился на 100 − 20 = 80 — это 0,8 от пальца. Замер подтверждает: имя с 216 переехало на 296.
- **Листаем вверх на 50** (`offset = −50`): шапка уехала вверх на 50. Сдвиг текста: −(−50) × 0,2 = +10, вниз. Итого вверх на 50 − 10 = 40 — снова 0,8.

Коэффициент 0,2 — доля «отставания». Чем он **больше**, тем сильнее эффект: при 0,5 текст движется вдвое медленнее шапки — очень заметно; при 0,05 — едва различимо. 0,2 — ощутимо, но не укачивает. (Не путай: чем меньше коэффициент, тем слабее эффект, а не сильнее.)

`max(−150, min(150, offset))` — зажим в диапазон ±150. Пользователь может утянуть список на 500 точек; без зажима текст отстал бы на 500 × 0,2 = 100 точек, и сильно растянутая шапка выглядела бы странно. С зажимом отставание не больше 150 × 0,2 = 30 точек.

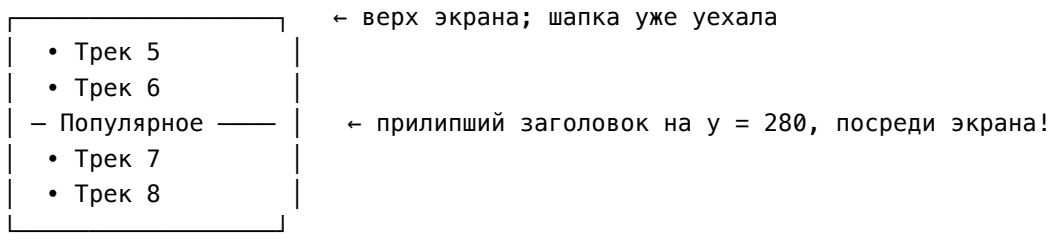
Аватар в параллаксе не участвует — он едет вместе с краем шапки. Разница скоростей аватара и имени и даёт глубину. Если сдвинуть всё одинаково, эффект пропадёт.

> **Когда параллакс работает.** Лучше всего — на **картинке с текстом поверх**: фото движется медленнее, текст быстрее (или наоборот). На однотонном фоне без деталей разницу скоростей почти не видно — глазу не за что зацепиться.
>
> И уважай настройку «Уменьшение движения»: если `UIAccessibility.isReduceMotionEnabled == true`, параллакс лучше выключить (передавать в `applyParallax` ноль).

21.8 Sticky-заголовки разделов — почему отказались

У таблицы в стиле `.plain` заголовки разделов **прилипают** (sticky header, «липкий заголовок»): пока раздел на экране, его заголовок висит у верхнего края таблицы. В контактах так висит буква текущего раздела — «А», «Б».

Где именно «верхний край»? Не у нулевой точки, а **под верхним отступом** таблицы. Мы проверили: `.plain`-таблица, `contentInset.top = 280`, автоматика выключена. В начальном положении заголовок первого раздела стоит на 302 точках от верха экрана: 280 отступа плюс 22 точки — верхний отступ заголовка, который у `.plain`-таблиц появился в iOS 15 (`sectionHeaderTopPadding`). Прокручиваем на 400 точек — шапка давно уехала, а заголовок **прилип на 280**, посреди экрана, и строки проезжают под ним.



Причина: прилипание считается от «видимого верха с учётом отступов», а наш отступ в 280 точек статичный — он не знает, что шапка уехала. Со стилем `.insetGrouped` заголовки не прилипают и просто уезжают вместе со строками (замер: при прокрутке на 400 заголовок первого раздела уже за экраном).

Если липкие заголовки нужны и со stretchy-шапкой — придётся менять `contentInset.top` вслед за шапкой или перейти на compositional layout (21.11), где прилипание настраивается явно.

21.9 Safe area и навигационная панель

Шапка идёт **от `view.topAnchor`**, а не от safe area — то есть залезает под статус-бар и под навигационную панель, если экран внутри `UINavigationController` (в playground'е mini-app показывается именно так). Градиент во весь верх и выглядит как продолжение панели.

Как выглядит сама панель, решает iOS. У навигационной панели два вида: `scrollEdgeAppearance` — когда край прокручиваемого контента совпадает с краем панели, — и `standardAppearance` — когда контент уехал под панель, с размытым полупрозрачным фоном. Если `scrollEdgeAppearance` не задан, UIKit берёт `standardAppearance` и делает ему **прозрачный** фон; до iOS 15 это касалось только панелей с большим заголовком, с iOS 15 — всех (документация `UINavigationBar.scrollEdgeAppearance`). На нашем экране в начальном положении панель прозрачная, кнопка «Назад» лежит прямо на градиенте; прокрутишь — появится размытый фон.

Проверь контраст: кнопка «Назад» по умолчанию цвета `tintColor` (обычно синего), а на коричнево-красном градиенте синий читается плохо. Поставь `navigationController?.navigationBar.tintCol = .white` на время показа экрана и верни прежний цвет при уходе.

Если нужна обычная непрозрачная панель, привяжи шапку к `view.safeAreaLayoutGuide.topAnchor`: тогда градиент начнётся под панелью, но и растяжение будет только в видимой области под ней.

Упражнение 21.2. Замени `.insetGrouped` на `.plain` и прокрути список вверх так, чтобы шапка уехала. Где висит заголовок «О профиле»? Ответ — в конце главы.

21.10 Бытовая аналогия

Сложный экран — **театральная сцена**.

- **Задник** — градиент шапки. Фон, на котором всё происходит.
- **Авансцена** — шапка целиком: меняет размер, уезжает и возвращается.
- **Актёры** — имя и подзаголовок. Двигаются по своей логике (параллакс), не приклеены намертво к декорациям.
- **Зрительный зал** — таблица с контентом, ради которого пользователь и пришёл.

А прозрачная шапка из 21.4 — это **стекло витрины**: всё видно, но руку просунуть можно только если стекла нет.

21.11 Липкие заголовки на compositional layout

Если нужна сетка карточек с разделами и **липкими** заголовками, удобнее `UICollectionView` с **compositional layout** (iOS 13+). Это способ описать раскладку коллекции «кубиками»: **item** (одна ячейка) → **group** (строка или столбец ячеек) → **section** (раздел из повторяющихся групп). Размеры задаются не в точках, а **относительно** контейнера:

- `.fractionalWidth(0.5)` — половина ширины контейнера;
- `.fractionalHeight(1.0)` — вся высота контейнера;
- `.absolute(44)` — ровно 44 точки;

- `.estimated(44)` — «примерно 44»: система посчитает настоящий размер через Auto Layout содержимого, 44 — лишь стартовая оценка.

«Контейнер» у каждого уровня свой: для item — группа, в которой он лежит; для группы — раздел (за вычетом отступов раздела).

```
static func makeLayout() -> UICollectionViewCompositionalLayout {
    // Ячейка: половина ширины группы, высота = половина ширины группы → квадрат.
    let itemSize = NSCollectionLayoutSize(widthDimension: .fractionalWidth(0.5),
                                          heightDimension: .fractionalWidth(0.5))
    let item = NSCollectionLayoutItem(layoutSize: itemSize)
    item.contentInsets = NSDirectionalEdgeInsets(top: 4, leading: 4, bottom: 4, trailing:
↪   4)

    // Группа — строка во всю ширину секции; в неё влезает 2 ячейки.
    let groupSize = NSCollectionLayoutSize(widthDimension: .fractionalWidth(1.0),
                                           heightDimension: .fractionalWidth(0.5))
    let group = NSCollectionLayoutGroup.horizontal(layoutSize: groupSize, subitems:
↪   [item])

    let section = NSCollectionLayoutSection(group: group)
    section.contentInsets = NSDirectionalEdgeInsets(top: 8, leading: 12, bottom: 16,
↪   trailing: 12)

    // Заголовок секции: ширина во всю строку, высота — «примерно 44, уточнит Auto Layout».
    let headerSize = NSCollectionLayoutSize(widthDimension: .fractionalWidth(1.0),
                                            heightDimension: .estimated(44))
    let header = NSCollectionLayoutBoundarySupplementaryItem(
        layoutSize: headerSize,
        elementKind: UICollectionView.elementKindSectionHeader,
        alignment: .top)
    header.pinToVisibleBounds = true          // прилипает к верху, пока секция на экране
    section.boundarySupplementaryItems = [header]

    return UICollectionViewCompositionalLayout(section: section)
}
```

Посчитаем размеры на iPhone 16 (ширина коллекции 393 точки), шаг за шагом.

1. **Раздел.** `section.contentInsets` — 12 слева и 12 справа. Ширина для групп: 393 − 12 − 12 = **369**.
2. **Группа.** Ширина `.fractionalWidth(1.0)` — вся ширина раздела: 369. Высота `.fractionalWidth(0.5)` — половина **ширины** раздела: 369 × 0,5 = **184,5**. (Да, высоту можно задать через ширину — так получаются квадраты и пропорции, не зависящие от экрана.)
3. **Ячейка.** Ширина `.fractionalWidth(0.5)` — половина ширины группы: 369 × 0,5 = 184,5. Высота `.fractionalWidth(0.5)` — тоже 184,5. В группу шириной 369 помещается 369 / 184,5 = 2 ячейки — горизонтальная группа сама повторяет item, пока хватает места.
4. **Отступы ячейки.** `item.contentInsets` по 4 со всех сторон **уменьшают** ячейку внутри отведённого ей места: 184,5 − 4 − 4 = **176,5** (на экране — 176,33, см. ниже). Первая ячейка

начинается на 12 (отступ раздела) + 4 (свой отступ) = **16** точек от левого края; между двумя ячейками 4 + 4 = **8** точек.

Замер в симуляторе (рамки из `layoutAttributesForItem`): первая ячейка `x = 16, ширина 176,33, высота 176,33`, вторая — `x = 200,33`, то есть 16 + 176,33 + 8. Откуда 176,33 вместо 176,5? Раскладка выравнивает рамки по **пиксельной сетке** экрана. У iPhone 16 экран @3x: одна точка — это 3 × 3 физических пикселя, и край view может стоять только на границе пикселя, то есть с шагом 1/3 точки (0, 0,33, 0,67, 1…). Число 176,5 на эту сетку не попадает (оно ровно посередине между 176,33 и 176,67), и UIKit берёт 176,33. На экране @2x шаг 0,5 точки, там осталось бы 176,5.

Заголовок — `NSCollectionLayoutBoundarySupplementaryItem`: «дополнительный view на границе раздела» (`alignment: .top` — над разделом). Ширина — вся ширина раздела, высота `.estimated(44)`: наш `SectionHeaderView` — это `UILabel` шрифта `.headline` с отступами 8 сверху и снизу, и система вычисляет его настоящую высоту сама — в симуляторе вышло 36,33 точки вместо оценочных 44. Поэтому первый ряд ячеек начинается на 36,33 + 8 (отступ раздела) + 4 (отступ ячейки) = 48,33 — так и показал замер. С крупным Dynamic Type заголовок вырастет, а не обрежется.

Одна тонкость из заголовочного файла UIKit: `contentInsets` у item **игнорируются по оси, где размер `.estimated`**. Для ячеек с оценочной высотой отступы сверху/снизу задавай расстоянием между группами (`section.interGroupSpacing`) или внутри самой ячейки.

`pinToVisibleBounds = true` — заголовок прилипает к верху видимой области, пока его раздел на экране; когда раздел уходит, заголовок нового раздела «выталкивает» предыдущий. Проверили: прокрутили коллекцию на 600 точек — заголовок второго раздела оказался на 659 = 600 + 59, то есть у верхнего края видимой области **под** safe area (59 точек — статус-бар и Dynamic Island на iPhone 16). У этой коллекции автоматические отступы не выключены, и прилипание их учитывает.

Для iOS 16+ есть `NSCollectionLayoutGroup.horizontal(layoutSize:repeatingSubitem:count:)` — «ровно N одинаковых ячеек»; старый вариант `subitem:count:` в iOS 16 объявлен устаревшим. Наш вариант с массивом `subitems: [item]` работает на всех версиях с iOS 13.

Полный экран-пример (`StickyGridViewController` с data source и `SectionHeaderView`) — в ответе к упражнению 21.3.

Упражнение 21.3. Сделай три ячейки в ряд вместо двух. Какие два числа в `makeLayout()` поменяются и какой получится видимая ширина ячейки на iPhone 16? Ответ — в конце главы.

21.12 Когда оно нужно

Stretchy header — продвинутый паттерн. Хорошо смотрится:

- на **экранах профиля** — артист, пользователь, компания;
- на **детальных экранах** — товар, рецепт, статья с большой фотографией.

Плохо смотрится:

- в **списках задач и настроек** — рабочие экраны, украшение отвлекает;
- в **формах** (вход, регистрация) — шапка съедает место под клавиатурой;
- в **чатах** — главное там лента сообщений и поле ввода.

Упражнение 21.4. Открой «Сложные экраны» (голубая ячейка). Положи палец **на шапку** и потяни вниз, отпусти. Потом прокрути вверх до конца. Что должно происходить — в ответах.

21.13 Что мы пропустили

- **Имя в навигационной панели.** Когда шапка полностью скрыта, в панели появляется «Alma Echo». Делается в том же `scrollViewDidScroll`: `navigationItem.title = offset < -initialHeaderHeight + 100 ? "Alma Echo" : nil`. Отдельное наблюдение за `contentOffset` не нужно — делегат и так вызывается на каждом кадре.
- **Фото вместо градиента.** `UIImageView` с `contentMode = .scaleAspectFill` вместо градиентного слоя и загрузка через кеш из главы 16.
- **Кнопка «Наверх»** после долгой прокрутки: `tableView.setContentOffset(CGPoint(x: 0, y: -initialHeaderHeight), animated: true)`.

Ответы к упражнениям

Упражнение 21.1. С привязкой к верху аватар стоит на месте (116), а при растяжении под именем появляется пустая полоса градиента между текстом и первой строкой. С привязкой к низу аватар и текст едут вниз вслед за краем шапки, а пустое место открывается сверху — так и задумано.

Упражнение 21.2. Заголовок «О профиле» прилипает на 280 точках от верха экрана (там, где в начале кончалась шапка), а строки проезжают под ним. С `.insetGrouped` он уезжает вместе с разделом.

Упражнение 21.3. Ширина ячейки — `.fractionalWidth(1.0 / 3.0)`; высоту группы (и ячейки) для квадратов ставим тоже `1.0 / 3.0`. Место под ячейку: 369 / 3 = 123 точки, видимая ширина 123 − 4 − 4 = **115** точек, промежуток между ячейками по-прежнему 8. Полный экран для экспериментов:

```
final class StickyGridViewController: UIViewController {
    private lazy var collectionView = UICollectionView(frame: .zero,
                                                       collectionViewLayout: Self.makeLayout())

    // static func makeLayout() — из раздела 21.11

    override func viewDidLoad() {
        super.viewDidLoad()
        collectionView.backgroundColor = .systemBackground
        collectionView.register(UICollectionViewCell.self, forCellWithReuseIdentifier:
↪   "cell")
        collectionView.register(SectionHeaderView.self,
                                forSupplementaryViewOfKind:
↪   UICollectionView.elementKindSectionHeader,
                                withReuseIdentifier: SectionHeaderView.reuseID)
        collectionView.dataSource = self
        collectionView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(collectionView)
```

```
        NSLayoutConstraint.activate([
            collectionView.topAnchor.constraint(equalTo: view.topAnchor),
            collectionView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            collectionView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            collectionView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])
    }
}

extension StickyGridViewController: UICollectionViewDataSource {
    func numberOfSections(in collectionView: UICollectionView) -> Int { 3 }

    func collectionView(_ collectionView: UICollectionView, numberOfItemsInSection
    ↪  section: Int) -> Int { 6 }

    func collectionView(_ collectionView: UICollectionView,
                        cellForItemAt indexPath: IndexPath) -> UICollectionViewCell {
        let cell = collectionView.dequeueReusableCell(withReuseIdentifier: "cell", for:
        ↪  indexPath)
        cell.contentView.backgroundColor = .systemTeal.withAlphaComponent(0.3)
        cell.contentView.layer.cornerRadius = 12
        return cell
    }

    func collectionView(_ collectionView: UICollectionView,
                        viewForSupplementaryElementOfKind kind: String,
                        at indexPath: IndexPath) -> UICollectionReusableView {
        let header = collectionView.dequeueReusableSupplementaryView(
            ofKind: kind, withReuseIdentifier: SectionHeaderView.reuseID, for: indexPath)
        (header as? SectionHeaderView)?.label.text = ["Популярное", "Альбомы",
        ↪  "Синглы"][indexPath.section]
        return header
    }
}

final class SectionHeaderView: UICollectionReusableView {
    static let reuseID = "SectionHeaderView"
    let label = UILabel()

    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .systemBackground
        label.font = .preferredFont(forTextStyle: .headline)
        label.adjustsFontForContentSizeCategory = true
        label.translatesAutoresizingMaskIntoConstraints = false
        addSubview(label)
        NSLayoutConstraint.activate([
            label.topAnchor.constraint(equalTo: topAnchor, constant: 8),
```

```
            label.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -8),
            label.leadingAnchor.constraint(equalTo: leadingAnchor, constant: 4),
            label.trailingAnchor.constraint(equalTo: trailingAnchor, constant: -4),
        ])
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }
}
```

У заголовка непрозрачный фон (`systemBackground`) — иначе прилипший заголовок был бы прозрачным, и ячейки просвечивали бы сквозь текст.

Упражнение 21.4. Палец на шапке тянет список: шапка растягивается, градиент увеличивается сверху, аватар и имя едут вниз, имя чуть медленнее аватара (параллакс). Отпустил — список пружинит обратно, шапка возвращается к 280. При прокрутке вверх шапка уезжает за верхний край вместе со списком, а заголовки разделов не прилипают.

Что мы выучили

- **Шапка — отдельный view поверх таблицы**, а не `tableHeaderView`; у таблицы `contentInset.top` = высоте шапки и `contentInsetAdjustmentBehavior = .never`. Нижний отступ под полоску «домой» после `.never` возвращаем руками.
- `offset = -(contentOffset.y + adjustedContentInset.top)`: 0 в начале, плюс — тянут вниз, минус — листают вверх.
  - `offset > 0` → высота = 280 + offset, верх = 0.
  - `offset < 0` → высота = 280, верх = offset.
- Hit-testing идёт по `subviews` с конца (верхние первыми) и пропускает `isHidden`, `alpha < 0.01`, `isUserInteractionEnabled = false`. Шапка поверх таблицы обязана пропускать касания: `isUserInteractionEnabled = false`, иначе тянуть за неё нельзя.
- `layerClass = CAGradientLayer` — градиент сам тянется за view, без неявной анимации отдельного слоя (view как делегат своего слоя отвечает на `action(forKey:) nil`); цвета под тему — через `resolvedColor`.
- `transform` не трогает constraints, `center` и `bounds`; `frame` при нём не определён.
- Содержимое шапки привязано к **низу** — растягивается пустой верх.
- **Параллакс** — `CGAffineTransform` со сдвигом `-offset × 0,2`: текст движется со скоростью 0,8; больший коэффициент — сильнее эффект.
- `.plain`-таблица прилипает заголовки под `contentInset.top` — при stretchy-шапке они зависают посреди экрана; `.insetGrouped` не прилипает.
- **Compositional layout**: item → group → section, размеры в долях контейнера; `contentInsets` item'а уменьшают его внутри места, на `.estimated`-оси игнорируются; `pinToVisibleBounds` — липкий заголовок.

Apple Developer Documentation

- UIScrollView — `contentOffset`, `contentInset`: основа всей stretchy-логики.
- scrollViewDidScroll(_:) — вызывается на каждом кадре прокрутки.
- contentOffset — текущая позиция прокрутки.

- adjustedContentInset — итоговый отступ с учётом safe area.
- contentInsetAdjustmentBehavior — `.never` выключает автоматические отступы.
- UITableView.Style.insetGrouped — стиль без прилипающих заголовков.
- layerClass — свой класс слоя для view.
- CAGradientLayer — градиент шапки.
- CGAffineTransform — сдвиг для параллакса.
- UIView.transform — почему `frame` при transform не определён и как это ладит с Auto Layout.
- CALayer.action(forKey:) — откуда у отдельного слоя берётся неявная анимация.
- UINavigationBar.scrollEdgeAppearance — прозрачная панель у края прокрутки, с iOS 15 для всех панелей.
- hitTest(_:with:) — как UIKit выбирает view для касания.
- UICollectionViewCompositionalLayout — раскладка item → group → section.
- NSCollectionLayoutDimension — `fractionalWidth`, `absolute`, `estimated`.
- NSCollectionLayoutBoundarySupplementaryItem — заголовки разделов и `pinToVisibleBounds`.
- HIG: Layout — отступы и safe area для шапок во весь экран.

→ Глава 22. Anatomy — тур по всем гейтам через modal preview

Глава 22. Anatomy — тур по всем гейтам через modal preview

Заключительный mini-app Части III. Anatomy — **учебный гид по гейтам**, которые мы построили в Части II. Напомним: **гейт** (gate, «ворота») — экран, который стоит на пути от запуска к основному содержимому: заставка, онбординг, запрос разрешений, вход, выбор региона. Пользователь открывает Anatomy, видит список гейтов, тапает любой — гейт показывается на весь экран. Можно посмотреть, как он выглядит, понажимать кнопки и закрыть.

Это удобно для **демонстрации** и **отладки**. Чтобы увидеть, скажем, запрос разрешения на камеру, не нужно сбрасывать приложение и проходить весь запуск заново — открыл Anatomy, тапнул «Permission primer».

В главе разбираем сам экран Anatomy, шаблон «показать отдельный экран модально», скрытое меню разработчика и — в конце — **анатомию самого UIKit**: от кого наследуются классы, которыми мы пользовались всю книгу. Цепочки наследования не по памяти: мы распечатали их в рантайме.

> **Режим компиляции.** Код главы собран в проекте с deployment target iOS 15.0, Swift 6, Default Actor Isolation = MainActor. Классы гейтов (`AnimatedSplashViewController`, `RegionPickerViewController`, `RegionStorage`, `AppRegistry`) — из глав 1, 2 и 10; для проверки мы подставляли их упрощённые заглушки с теми же инициализаторами.

22.1 Структура UI

```
final class AnatomyViewController: UIViewController {
    private struct GateEntry {
        let icon: String
        let title: String
        let subtitle: String
        let presenter: (UIViewController) -> Void
    }

    private static let cellID = "gate"
    private let tableView = UITableView(frame: .zero, style: .insetGrouped)
    private lazy var entries: [GateEntry] = makeEntries()

    override func viewDidLoad() {
```

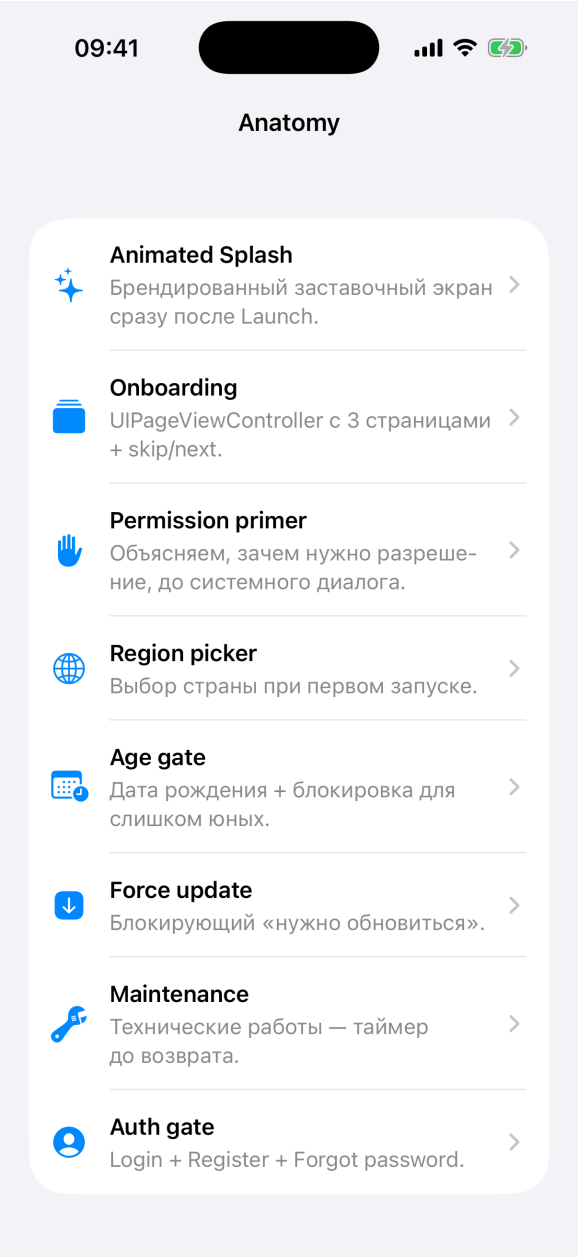


Рис. 18: Тур по гейтам

```
        super.viewDidLoad()
        title = "Anatomy"
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: Self.cellID)
        tableView.dataSource = self
        tableView.delegate = self
        tableView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(tableView)
        NSLayoutConstraint.activate([
            tableView.topAnchor.constraint(equalTo: view.topAnchor),
            tableView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            tableView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            tableView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
        ])
    }
}
```

Похоже на Profile (глава 19): декларативный массив описаний + одна функция отрисовки. Здесь всё проще — один тип строки на весь список.

`GateEntry` — описание одной строки: иконка, заголовок, подзаголовок и **`presenter`** — замыкание, которое умеет открыть этот гейт. Параметр `UIViewController` — экран-«якорь», с которого показываем гейт (в Anatomy это сам `AnatomyViewController`). Каждая строка знает, **как именно** показать свой гейт, а таблица об этом не знает ничего.

`register(_:forCellReuseIdentifier:)` — говорим таблице, какой класс ячейки создавать для идентификатора «gate». Без регистрации `dequeueReusableCell(withIdentifier:for:)` в 22.4 **роняет** приложение с сообщением «unable to dequeue a cell with identifier gate». Этот вариант метода требует регистрации — в отличие от старого `dequeueReusableCell(withIdentifier:)` без `for:`, который возвращает опционал.

`private lazy var entries` — `lazy` («ленивый»): массив создаётся при первом обращении, а не в инициализаторе. Иначе `makeEntries()` — метод экземпляра — нельзя было бы вызвать при инициализации свойства.

22.2 Entry — описание одного гейта

```
private func makeEntries() -> [GateEntry] {
    [
        GateEntry(
            icon: "sparkles",
            title: "Animated Splash",
            subtitle: "Брендированный заставочный экран сразу после Launch."
        ) { anchor in
            let manifest = AppRegistry.allApps.first { $0.id == "todo" } ??
            ↪ AppRegistry.allApps[0]
            let splash = AnimatedSplashViewController(manifest: manifest) { [weak anchor]
            ↪ in
                anchor?.dismiss(animated: true)
```

```
            }
            Self.presentFullScreen(splash, from: anchor)
        },
        GateEntry(
            icon: "globe",
            title: "Region picker",
            subtitle: "Выбор страны при первом запуске."
        ) { anchor in
            let picker = RegionPickerViewController(manifestId: "anatomy.demo",
                                                    brandColor: .systemBlue) { [weak anchor]
↪   _ in
                anchor?.dismiss(animated: true)
                RegionStorage.shared.reset(for: "anatomy.demo")
            }
            Self.presentFullScreen(UINavigationController(rootViewController: picker),
↪   from: anchor)
        },
        // ... остальные гейты — по тому же шаблону
    ]
}
```

Что внутри `presenter` зависит от гейта:

- **Animated Splash** — берём любой манифест (здесь «todo», а если его нет — первый), создаём `AnimatedSplashViewController`, в callback'е `onFinish` закрываем модалку. **Callback** («обратный вызов») — замыкание, которое экран вызовет сам, когда закончит работу.
- **Region picker** — создаём с выдуманным `manifestId` `"anatomy.demo"`, а после выбора **сбрасываем** сохранённый регион. Выбор хранится в `UserDefaults` (глава 10); без сброса при новом заходе в Anatomy гейт решил бы «регион уже выбран» и не показался.

Обрати внимание на захваты. Замыкания-presenter'ы **не трогают `self`**: всё нужное приходит параметром `anchor`, а показ идёт через статическую функцию `Self.presentFullScreen`. Это не случайность. Массив `entries` хранится внутри `AnatomyViewController`, замыкания хранятся в массиве. Если бы замыкание захватило `self` сильно, получился бы круг «экран → entries → замыкание → экран», и экран Anatomy никогда не освободился бы из памяти (retain cycle). Первая версия главы спасалась `[weak self]`; без захвата вообще — проще и надёжнее.

Внутренние замыкания (`onFinish` сплэша, выбор региона) хранятся уже в показанном гейте, а гейт показан **с** якоря. Якорь держит гейт (`presentedViewController`), гейт держит замыкание — поэтому внутри `[weak anchor]`: иначе снова круг.

22.3 Presenter — общая функция

```
private static func presentFullScreen(_ vc: UIViewController, from anchor:
↪   UIViewController) {
    vc.modalPresentationStyle = .fullScreen
    anchor.present(vc, animated: true)
}
```

Модальный показ (`present`) — экран появляется поверх текущего и закрывается `dismiss`. Стиль `.fullScreen` — на весь экран, без карточки сверху, как если бы гейт был корневым экраном окна. Так видно, как он выглядит в настоящем запуске. У `.fullScreen` есть и побочный эффект, про который документация `UIModalPresentationStyle.fullScreen` пишет прямо: после показа UIKit **убирает view показывающего экрана из иерархии**, и тот получает `viewWillDisappear`/`viewDidDisappear`; при закрытии — снова `viewWillAppear`. У листа нижний экран остаётся на месте и этих вызовов не получает (глава 0, раздел 0.6).

По умолчанию с iOS 13 модальные экраны показываются **листом**: карточка, из-под которой виден предыдущий экран, и её можно смахнуть вниз. Значение `modalPresentationStyle` по умолчанию — `.automatic`, и система сама подбирает стиль: до iOS 18 это `.pageSheet`, с iOS 18 — `.formSheet` (документация `UIModalPresentationStyle.automatic`; в рантайме iOS 26.5 свойство свежего контроллера читается как `.formSheet`). На iPhone разницы не видно — в компактной ширине form sheet выглядит как page sheet. Для Anatomy лист неправдоподобен: force-update и maintenance (глава 9) по смыслу **блокируют** приложение и смахиваться не должны.

`static` — функция не нужна экземпляру: всё, что ей требуется, приходит параметрами. Поэтому замыкания в 22.2 вызывают её через `Self.` без захвата `self`.

`UINavigationController(rootViewController: picker)` для региона — picker рассчитан на навигационную панель с заголовком и кнопкой, поэтому оборачиваем его, как в настоящем запуске.

22.4 TableView render

```
extension AnatomyViewController: UITableViewDataSource, UITableViewDelegate {
    func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
        entries.count
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
    ↪  UITableViewCell {
        let entry = entries[indexPath.row]
        let cell = tableView.dequeueReusableCell(withIdentifier: Self.cellID, for:
        ↪  indexPath)
        var content = cell.defaultContentConfiguration()
        content.text = entry.title
        content.secondaryText = entry.subtitle
        content.image = UIImage(systemName: entry.icon)
        content.imageProperties.tintColor = .systemBlue
        content.textProperties.font = .preferredFont(forTextStyle: .headline)
        content.secondaryTextProperties.color = .secondaryLabel
        cell.contentConfiguration = content
        cell.accessoryType = .disclosureIndicator
        return cell
    }

    func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
```

```
        tableView.deselectRow(at: indexPath, animated: true)
        entries[indexPath.row].presenter(self)
    }
}
```

Одна ячейка на все строки — `UIListContentConfiguration` (iOS 14+, подробно в главе 19): заголовок, подзаголовок, картинка слева. `defaultContentConfiguration()` даёт заготовку в системном стиле, мы меняем только нужное.

`textProperties.font = .preferredFont(forTextStyle: .headline)` — жирный системный стиль, который растёт с настройкой размера шрифта (Dynamic Type). Первая версия ставила `systemFont(ofSize: 16, weight: .semibold)` — такой шрифт не увеличивается.

`accessoryType = .disclosureIndicator` — серая стрелка «›» справа: «тап откроет что-то новое».

`deselectRow(at:animated:)` — снимаем серую подсветку строки после тапа. Без этого, закрыв гейт, пользователь увидит выделенную строку.

`presenter(self)` — передаём себя как якорь. Здесь `self` — просто аргумент вызова, а не захват в хранимое замыкание, так что круга нет.

22.5 Закрытие — не координатор, а callback

В `BootCoordinator` (глава 4) `onFinish` гейта ведёт к **очередному** гейту цепочки. В Anatomy — к `dismiss`. Это разные сценарии:

- настоящий запуск: заставка → онбординг → разрешения → … → основной экран;
- Anatomy: заставка → закрыли → снова список Anatomy.

Поэтому в presenter'е передаём свой callback:

```
let splash = AnimatedSplashViewController(manifest: manifest) { [weak anchor] in
    anchor?.dismiss(animated: true)
}
```

`onFinish` означает не «перейди к очередному гейту», а «закрой меня». Это работает, потому что заставка **не знает**, что будет дальше: для неё `onFinish` — просто `() -> Void`. Что делать после — решает тот, кто её создал: координатор или Anatomy.

`anchor?.dismiss(animated: true)` — вызываем `dismiss` у **показывающего** экрана, а не у самой заставки. Оба варианта работают: если вызвать `dismiss` у экрана, который сам ничего не показывает, UIKit перешлёт вызов тому, кто показал его. Но «закрывает тот, кто открыл» читается однозначнее.

> **Плюс callback-API.** Если бы заставка сама знала, что после неё идёт онбординг, её нельзя было бы показать в Anatomy отдельно. С абстрактным `onFinish` гейт переиспользуется где угодно.

22.6 Anatomy — учебный, а не рабочий экран

В обычном приложении пользователю не нужен «тур по экранам запуска». Anatomy полезна в двух сценариях:

1. **Учебная демонстрация** — как в нашей книге.
2. **Скрытое меню разработчика** (debug menu) — экран, где команда видит все гейты, переключает сервер, сбрасывает флаги. В сборке для App Store его прячут или вырезают совсем.

22.7 Скрытый debug-экран — паттерн

Два распространённых способа спрятать такой экран.

Секретный жест. Например, семь тапов по номеру версии на экране «О приложении»:

```
final class AboutViewController: UIViewController {
    private let versionLabel = UILabel()
    private var versionTapCount = 0

    override func viewDidLoad() {
        super.viewDidLoad()
        let version = Bundle.main.object(forInfoDictionaryKey:
        ↪  "CFBundleShortVersionString") as? String ?? "?"
        versionLabel.text = "Версия \(version)"
        versionLabel.isUserInteractionEnabled = true      // у UILabel по умолчанию false
        versionLabel.addGestureRecognizer(
            UITapGestureRecognizer(target: self, action: #selector(versionLabelTapped)))

        #if DEBUG
        navigationItem.rightBarButtonItem = UIBarButtonItem(
            image: UIImage(systemName: "wrench.and.screwdriver"),
            primaryAction: UIAction { [weak self] _ in self?.openAnatomy() })
        #endif
    }

    @objc private func versionLabelTapped() {
        versionTapCount += 1
        if versionTapCount >= 7 {
            versionTapCount = 0
            openAnatomy()
        }
    }

    private func openAnatomy() {
        present(UINavigationController(rootViewController: AnatomyViewController()),
↪   animated: true)
    }
}
```

`CFBundleShortVersionString` — ключ **Info.plist** (файла с описанием приложения внутри его пакета) с «маркетинговой» версией, той, что видна в App Store: «1.4.2».

`versionLabel.isUserInteractionEnabled = true` — без этой строки жест никогда не сработает: у `UILabel` касания по умолчанию выключены. Мы проверили это в симуляторе: у свежего `UILabel` и `UIImageView` `isUserInteractionEnabled == false`, у обычного `UIView` — `true`.

Семь тапов — привычка, пришедшая с Android: там семь тапов по «Номеру сборки» в настройках включают режим разработчика. В iOS такого системного жеста нет; число 7 просто достаточно большое, чтобы не набрать его случайно.

Секретный жест прячет экран, но **не защищает** его: кто знает жест, тот войдёт. Не клади в такое меню ничего опасного для обычного пользователя.

Условная компиляция. `#if DEBUG … #endif` — код внутри попадёт только в отладочную сборку. `DEBUG` — флаг, который шаблон проекта Xcode задаёт в конфигурации Debug (Build Settings → Active Compilation Conditions). В сборке Release, которая уходит в App Store, кнопки не будет **вообще** — её код даже не компилируется.

`UIBarButtonItem(image:primaryAction:)` — кнопка панели с действием-замыканием. В Swift это один инициализатор с параметрами по умолчанию, `init(title:image:primaryAction:menu:)` (iOS 14+, из Swift-обёртки UIKit): мы передаём только `image` и `primaryAction`. Записи вида `UIBarButtonItem(image: ..., ...) { ... }` (с замыканием в конце) не бывает — последний параметр здесь `UIAction?`, а не замыкание. Сам `UIBarButtonItem` — не view и не кнопка: `UIBarButtonItem → UIBarItem → NSObject`. Это описание кнопки, которое лежит в `navigationItem` экрана, а навигационная панель уже сама рисует по нему настоящий view (глава 0, раздел 0.8). Поэтому у него нет `addSubview`, `frame` или `isHidden`.

Упражнение 22.1. Добавь в `AboutViewController` сброс счётчика тапов, если между тапами прошло больше 2 секунд: семь тапов должны быть **подряд**. Подсказка: запоминай `Date()` последнего тапа. Решение — в конце главы.

22.8 Анатомия UIKit: кто от кого наследуется

Anatomy показывает гейты, а под ними — одни и те же «кирпичи» UIKit. Полезно один раз увидеть их родословную: многое в поведении объясняется тем, **кто родитель**. Полная карта классов — в главе 0; здесь — практическая проверка её утверждений на рантайме, тем же приёмом, что описан в разделе 0.10.

Цепочку предков можно не вспоминать, а спросить у рантайма Objective-C (глава 0, раздел 0.1):

```
/// «Родословная» класса: сам класс → родитель → … → корень.
func superclassChain(of cls: AnyClass) -> String {
    var names = [NSStringFromClass(cls)]
    var current: AnyClass? = class_getSuperclass(cls)
    while let next = current {
        names.append(NSStringFromClass(next))
        current = class_getSuperclass(next)
    }
    return names.joined(separator: " → ")
```

```
}
```

`class_getSuperclass` — функция рантайма: «дай родительский класс». У корня цепочки родителя нет — вернётся `nil`, и цикл закончится. `NSStringFromClass` — имя класса строкой. Вызов: `print(superclassChain(of: UITableView.self))`.

Результат в симуляторе iOS 26.5:

```
UIApplication        → UIResponder → NSObject
UIWindowScene        → UIScene → UIResponder → NSObject
UIWindow             → UIView → UIResponder → NSObject
UIViewController     → UIResponder → NSObject
UINavigationController → UIViewController → UIResponder → NSObject
UITableView          → UIScrollView → UIView → UIResponder → NSObject
UICollectionView     → UIScrollView → UIView → UIResponder → NSObject
UIButton             → UIControl → UIView → UIResponder → NSObject
UIRefreshControl     → UIControl → UIView → UIResponder → NSObject
UILabel              → UIView → UIResponder → NSObject
UIStackView          → UIView → UIResponder → NSObject
UIScreenEdgePanGestureRecognizer → UIPanGestureRecognizer → UIGestureRecognizer → NSObject
CAGradientLayer      → CALayer → NSObject
```

Что из этого следует на практике.

`UIResponder` — общий предок приложения, сцены, окна, экранов и всех view. **Responder** («отвечающий») — объект, который умеет получать события: касания, нажатия клавиш, жесты «встряхнуть», команды меню. Они выстраиваются в **responder chain** — цепочку, по которой событие идёт снизу вверх, пока кто-то его не обработает: view → его superview → … → view экрана → экран → окно → сцена → приложение. Каждое звено — это свойство `next` (замер: `vc.view.next === vc`, `window.next` — window scene). Так работал shake-жест в `PlaygroundWindow` (глава 3): окно — responder, и событие «встряхнули» доходит до него. Подробно — глава 0, раздел 0.2.

First responder («первый отвечающий») — тот, кто сейчас получает ввод с клавиатуры: например, текстовое поле, в котором мигает курсор. `becomeFirstResponder()` поднимает клавиатуру, `resignFirstResponder()` (или `view.endEditing(true)` из главы 23) — убирает.

`UIViewController` — не view. Экран наследует `UIResponder`, а не `UIView`: он **управляет** своим view (`view`), но сам на экране не рисуется. Поэтому у экрана нет `frame`, а у его `view` — есть.

`UITableView` и `UICollectionView` — это `UIScrollView`. Отсюда в главе 21 `contentOffset` и `contentInset` у таблицы, а в главе 23 — `refreshControl` у любого списка: они унаследованы от scroll view.

`UIRefreshControl` и `UIButton` — это `UIControl`. Поэтому у обоих одинаковое `addAction(_:for:)` и события вроде `.valueChanged` и `.touchUpInside`.

`UIScreenEdgePanGestureRecognizer` — это `UIPanGestureRecognizer`. Поэтому в главе 20 один обработчик `handlePan(_:)` принимает и жест от края, и обычное перетаскивание: у обоих есть `translation(in:)` и `velocity(in:)`.

`CAGradientLayer` — это `CALayer`, а не view: у слоя нет Auto Layout и касаний. Отсюда трюк с `layerClass` в главе 21.

`UIWindow` — это `UIView`, просто самый верхний. Поэтому в главе 23 overlay можно было положить прямо в окно через `addSubview`.

Упражнение 22.2. Распечатай цепочку для `UIAlertController`, `UISearchBar` и `UITextField`. От кого наследуется каждый и что это значит для их поведения? Ответ — в конце главы.

22.9 Бытовая аналогия

Anatomy — **витрина с инструментами в магазине**. Каждый инструмент (гейт) висит за стеклом, его можно взять в руки и попробовать. В квартире (рабочем приложении) инструменты лежат в ящике и достаются, только когда нужны.

Витрина нужна для **обучения** (посмотреть то, что в жизни срабатывает один раз — онбординг), для **отладки** (разработчик проверяет экран без полного запуска) и для **демонстрации** (показать архитектуру на собеседовании).

А раздел 22.8 — **родословная** инструментов: дрель и шуруповёрт — родня, поэтому у них одинаковый патрон.

22.10 Бонус-идеи

- **Переключатели гейтов.** Галочки «заставка, онбординг, разрешение, вход» — и Anatomy запускает цепочку гейтов с такой конфигурацией. Получится настоящий полигон для `BootCoordinator`.
- **Кнопка «Снимок экрана»** — рисует текущий экран в картинку через `UIGraphicsImageRenderer` и сохраняет в Фото (`UIImageWriteToSavedPhotosAlbum`). Для сохранения в Фото нужен ключ `NSPhotoLibraryAddUsageDescription` в Info.plist с объяснением для пользователя, иначе приложение упадёт при первом сохранении.
- **Список архитектурных компонентов** — `AppManifest`, `BootCoordinator`, `PlaygroundWindow` с описанием: Anatomy превращается в живую документацию.
- **Варианты одного гейта** — например, два дизайна онбординга рядом для сравнения (A/B-тест).

Ответы к упражнениям

Упражнение 22.1.

```
private var lastTapDate = Date.distantPast

@objc private func versionLabelTapped() {
    let now = Date()
    if now.timeIntervalSince(lastTapDate) > 2 {
        versionTapCount = 0          // слишком долгая пауза — начинаем сначала
    }
    lastTapDate = now
```

```
    versionTapCount += 1
    if versionTapCount >= 7 {
        versionTapCount = 0
        openAnatomy()
    }
}
```

`Date.distantPast` — дата «давным-давно»: первый тап всегда начинает отсчёт с нуля. `timeIntervalSince` — разница в секундах. Семь тапов с паузами по секунде откроют меню; пауза в три секунды после пятого тапа — сбросит счётчик.

Упражнение 22.2. В симуляторе iOS 26.5:

```
UIAlertController → UIViewController → UIResponder → NSObject
UISearchBar       → UIView → UIResponder → NSObject
UITextField       → UIControl → UIView → UIResponder → NSObject
```

`UIAlertController` — полноценный экран: его показывают через `present`, а не `addSubview`. `UISearchBar` — просто view, не контрол: у него нет `addAction(_:for:)`, события приходят через делегат `UISearchBarDelegate`. Зато внутри него живёт `searchTextField` (iOS 13+) — это `UITextField`, то есть уже контрол, и к нему `addAction` применим. `UITextField` — контрол, поэтому на него можно повесить `addAction(_, for: .editingChanged)` и узнавать о каждом изменении текста. Таблица «кто контрол, а кто нет» — глава 0, раздел 0.4.

Что мы выучили

- Anatomy — **учебный** mini-app; тот же приём — основа скрытого меню разработчика.
- Структура как в Profile: массив `[GateEntry]` + одна функция отрисовки. `dequeueReusableCell(withIdenti` требует `register`.
- `presenter: (UIViewController) -> Void` получает якорь параметром — тогда хранимым замыканиям не нужен `self`, и retain cycle невозможен.
- Гейт показывается `.fullScreen` — как в настоящем запуске, нижний экран при этом уходит из иерархии и получает `viewWillDisappear`; лист по умолчанию (`.automatic` → `.pageSheet` до iOS 18, `.formSheet` с iOS 18) можно смахнуть, блокирующим гейтам это не подходит.
- Гейты переиспользуются благодаря callback-API: один `AnimatedSplashViewController` работает и в координаторе, и в Anatomy.
- Гейтам, которые запоминают выбор в `UserDefaults`, в Anatomy делаем сброс.
- Скрытый доступ — `#if DEBUG` (кода нет в Release) или секретный жест (код есть, просто спрятан).
- `UIResponder` — общий предок приложения, сцены, окна, экранов и view; `UITableView`/`UICollectionView` — наследники `UIScrollView`, `UIButton` и `UIRefreshControl` — `UIControl`, экран — **не** view.

Apple Developer Documentation

- HIG: Patterns — обзор паттернов: запуск, онбординг, модальность.
- HIG: Launching — рекомендации Apple по первому запуску.
- HIG: Onboarding — объём и тон онбординга.

- HIG: Modality — когда полный экран, когда лист.
- present(_:animated:completion:) и dismiss(animated:completion:) — модальный показ и закрытие.
- UIModalPresentationStyle — `.fullScreen`, `.pageSheet`, `.automatic`.
- UIModalPresentationStyle.automatic — во что система превращает стиль по умолчанию: `.formSheet` с iOS 18, `.pageSheet` раньше.
- UIBarItem — общий предок `UIBarButtonItem` и `UITabBarItem`; не view.
- UIResponder — общий предок; responder chain и first responder.
- Using responders and the responder chain to handle events — как событие идёт по цепочке.
- class_getSuperclass(_:) — функция рантайма из раздела 22.8.
- Compiler Control Statements — `#if DEBUG` и условная компиляция в Swift.

Это конец Части III. Дальше — Часть IV, UI Cookbook. Это не mini-приложения, а **справочник UI-паттернов**: pull-to-refresh, skeleton, поиск, модалки, жесты, формы, анимации, haptics, доступность, темы. По несколько рецептов в главе. Открываешь, когда нужен конкретный паттерн, и забираешь в свой проект.

→ Глава 23. Cookbook — загрузка

Глава 23. Cookbook — загрузка

С этой главы начинается Часть IV — **UI Cookbook**. Это справочник. Каждый рецепт устроен одинаково: когда применять / минимальный код / частые ошибки / альтернативы. Читать подряд не нужно — открывай, когда нужен конкретный паттерн.

В этой главе — паттерны вокруг **загрузки**: как показать пользователю, что приложение не зависло, а работает. Правило за всеми рецептами одно: любое ожидание дольше доли секунды должно быть **видно**. Экран, который молча ждёт сеть, выглядит сломанным.

> **Режим компиляции.** Все листинги главы проверены компилятором в режиме книги: Swift 6, Default Actor Isolation = MainActor, deployment target iOS 15.0. Рецепты с `async/await` дополнительно проверены в языковом режиме Swift 5 (с тем же MainActor по умолчанию) — они собираются и там. Для компиляции нужны маленькие заглушки `Item`, `API.fetch(page:)` и `ImageCache` — в своём проекте на их месте твой сетевой слой (см. главу 15).

23.1 Pull-to-refresh

Pull-to-refresh («потяни, чтобы обновить») — список тянут пальцем вниз дальше начала, появляется крутящийся индикатор, данные перезагружаются.

Когда применять. Любой список или лента, где пользователь ожидает свежие данные «по требованию»: почта, новости, заказы. Системный контрол для этого, `UIRefreshControl`, есть в UIKit с iOS 6.

Минимальный код. Экран-лента (заготовка ниже продолжится в 23.2):

```
final class FeedViewController: UITableViewController {
    private var items: [Item] = []
    private var nextPage = 1
    private var isLoadingPage = false
    private var hasMore = true

    override func viewDidLoad() {
        super.viewDidLoad()
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "cell")

        let refresh = UIRefreshControl()
        refresh.attributedTitle = NSAttributedString(string: "Обновляем…")
        refresh.addAction(UIAction { [weak self] _ in
```

```
            Task {
                await self?.reload()
                self?.refreshControl?.endRefreshing()
            }
        }, for: .valueChanged)
        refreshControl = refresh

        loadMore()
    }

    private func reload() async {
        do {
            let fresh = try await API.fetch(page: 1)
            items = fresh
            nextPage = 2
            hasMore = fresh.count == API.pageSize
            tableView.reloadData()
        } catch {
            // Старые данные не трогаем — только сообщаем.
            showToast("Не удалось обновить")
        }
    }
}
```

`refreshControl = refresh` — у `UITableViewController` есть готовое свойство; у любого `UIScrollView` (а значит, и у `UITableView`, `UICollectionView`) — тоже, `scrollView.refreshControl` (iOS 10+). До iOS 10 контрол приходилось вставлять в иерархию view руками. Сам scroll view и становится ему superview (замер: после присваивания `refreshControl.superview` — `UITableView`).

`UIRefreshControl` — это `UIControl` (`UIRefreshControl → UIControl → UIView`, глава 0, раздел 0.4). Отсюда и `addAction(_:for:)`, и событие `.valueChanged` — тот же механизм target-action, что у `UIButton` или `UISwitch`; документация прямо советует слушать именно `.valueChanged`.

`.valueChanged` — событие «пользователь дотянул, пора обновлять». Контрол к этому моменту уже крутится сам (`isRefreshing == true`). Обратное неверно: `beginRefreshing()` только переводит контрол в состояние «обновляется» и показывает индикатор — по документации его вызывают, когда обновление запустило что-то внешнее (таймер, кнопка), а саму загрузку в этом случае стартуешь ты.

`Task { ... }` — запускаем асинхронную работу из синхронного обработчика. Экран живёт на главном потоке (main actor), и `Task`, созданный отсюда, **наследует** главный поток: код после `await` снова выполняется на нём. Поэтому `tableView.reloadData()` и `endRefreshing()` можно звать без `DispatchQueue.main.async`.

`[weak self]` в `UIAction` — контрол живёт внутри экрана, замыкание — внутри контрола. Без `weak` получился бы круг ссылок, и экран никогда бы не освободился.

`endRefreshing()` стоит **после** `await`, то есть вызывается и при успехе, и при ошибке: `reload()` ловит ошибку внутри себя и не бросает её наружу.

При ошибке список **не очищаем**: пользователь видел данные минуту назад, пусть видит их и дальше, плюс короткое сообщение (toast, см. 23.6).

Частые ошибки.

- **Забыли `endRefreshing()`** — индикатор крутится вечно. Если функция загрузки может бросить ошибку, ставь вызов в `defer { }` внутри `Task`: `defer` выполняется при любом выходе из блока.
- **Очистили список при ошибке.** Был список — стал пустой экран с ошибкой. Хуже, чем ничего не делать.
- **Свои `contentInset` и прокрутка.** Если экран сам управляет `contentInset.top` (как stretchy header в главе 21), индикатор появится в «неправильном» месте — над отступом. Проверь такой экран руками.

Альтернативы.

- **Кнопка «Обновить» в навигационной панели** — для редко меняющихся данных и для экранов без прокрутки.
- **Автообновление при возврате в приложение** — подписка на `UIScene.didActivateNotification` (сцена снова активна) и перезагрузка. Удобно для лент.

23.2 Infinite scroll (пагинация)

Пагинация — загрузка данных порциями («страницами») по 20–50 штук. **Infinite scroll** («бесконечная прокрутка») — очередная страница грузится сама, когда пользователь долистал почти до конца.

Когда применять. Длинные ленты, где данных больше, чем разумно грузить разом: соцсети, каталог магазина, история заказов.

Минимальный код. Продолжение `FeedViewController`:

```
override func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) ->
 ↪  Int {
    items.count
}

override func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
 ↪  UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "cell", for: indexPath)
    var content = cell.defaultContentConfiguration()
    content.text = items[indexPath.row].title
    cell.contentConfiguration = content
    return cell
}

override func tableView(_ tableView: UITableView,
                        willDisplay cell: UITableViewCell,
                        forRowAt indexPath: IndexPath) {
    if indexPath.row >= items.count - 6 {
        loadMore()
```

```
    }
}

private func loadMore() {
    guard hasMore, !isLoadingPage else { return }
    isLoadingPage = true
    Task {
        defer { isLoadingPage = false }
        do {
            let new = try await API.fetch(page: nextPage)
            let start = items.count
            items.append(contentsOf: new)
            nextPage += 1
            hasMore = new.count == API.pageSize        // неполная страница = последняя
            let paths = (start..<items.count).map { IndexPath(row: $0, section: 0) }
            tableView.performBatchUpdates {
                tableView.insertRows(at: paths, with: .automatic)
            }
        } catch {
            // Сеть упала — hasMore не трогаем: новый скролл попробует снова.
        }
    }
}
```

`willDisplay` — делегат сообщает «эта ячейка сейчас появится на экране». Условие `row >= items.count - 6` — «показываем одну из последних шести». Числами: загружено 20 строк (номера 0–19), триггер сработает на строке 14. Пока пользователь долистает оставшиеся пять-шесть строк, новая страница успеет прийти. Число 6 подбирается: на быстрой сети хватит 3, на медленной и с короткими ячейками — лучше 10.

Три переменные состояния:

- `nextPage` — номер страницы для очередного запроса.
- `isLoadingPage` — «запрос уже в пути». `willDisplay` вызывается для каждой появляющейся ячейки, и без флага за одну прокрутку ушло бы пять одинаковых запросов.
- `hasMore` — «есть ли смысл просить ещё».

`defer { isLoadingPage = false }` — флаг сбросится при любом исходе: успех, ошибка, ранний выход.

`hasMore = new.count == API.pageSize` — если сервер отдаёт по 20 штук, а пришло 13, это последняя страница, и ещё один запрос за пустым массивом не нужен. Если твоё API явно возвращает `hasNext` или общее количество — используй его, это надёжнее.

`performBatchUpdates { insertRows }` — добавляем только новые строки, а не перерисовываем всю таблицу. `reloadData()` здесь тоже бы сработал: позицию прокрутки он **не** сбрасывает (мы проверили: `contentOffset` до и после `reloadData()` одинаковый). Но `reloadData()` пересоздаёт все видимые ячейки без анимации, и при ячейках с оценочной высотой лента может «дёрнуться» на несколько точек. `insertRows` дешевле и спокойнее.

Частые ошибки.

- **Нет флага «уже грузим»** — дубли в ленте: две одинаковые страницы приехали одновременно.
- **Триггер на последней строке** (`count - 1`) — пользователь упирается в конец списка и ждёт. Запас в несколько строк убирает паузу.
- **При ошибке `hasMore = false`** — лента «кончилась» из-за одного сбоя сети. Ошибку показываем (например, строкой-футером «Не удалось загрузить. Повторить»), а `hasMore` не трогаем.
- **Смешали `reload` и `loadMore`.** Если пользователь потянул обновление, пока грузилась страница 5, ответ страницы 5 допишется к свежему списку после страницы 1. Надёжное решение — хранить `Task` загрузки и отменять его (`task.cancel()`) при обновлении.

Альтернативы.

- **Индикатор-футер** — `tableView.tableFooterView` со спиннером на время загрузки страницы; для `UICollectionView` — supplementary view типа footer (так сделано в галерее, глава 16).
- **Кнопка «Показать ещё»** — если автоматическая подгрузка раздражает или API медленное и дорогое.

Упражнение 23.1. В `API.fetch(page:)` верни 13 элементов на третьей странице вместо 20. Сколько всего запросов уйдёт за полную прокрутку ленты? А если убрать проверку `new.count == API.pageSize` и оставить только «пустой ответ = конец»? Ответ — в конце главы.

23.3 Skeleton-плейсхолдер

Skeleton («скелет») — серые прямоугольники на месте будущего текста и картинок, пока данные грузятся. Часто по ним пробегает светлый блик — **shimmer** («мерцание»). **Placeholder** («заполнитель») — общее слово для всего, что стоит на месте ещё не пришедшего контента.

Когда применять. Первое открытие экрана, пока данных нет. Скелет заранее показывает форму будущего экрана, и появление контента воспринимается как «проявление», а не как прыжок из пустоты.

Минимальный код.

```
final class SkeletonView: UIView {
    private let gradient = CAGradientLayer()

    override init(frame: CGRect) {
        super.init(frame: frame)
        layer.cornerRadius = 8
        clipsToBounds = true
        backgroundColor = UIColor.tertiaryLabel.withAlphaComponent(0.15)
        gradient.colors = [
            UIColor.white.withAlphaComponent(0).cgColor,
            UIColor.white.withAlphaComponent(0.35).cgColor,
            UIColor.white.withAlphaComponent(0).cgColor,
        ]
        gradient.startPoint = CGPoint(x: 0, y: 0.5)
        gradient.endPoint = CGPoint(x: 1, y: 0.5)
```

```
        gradient.locations = [-1, -0.5, 0]          // блик пока левее видимой области
        layer.addSublayer(gradient)
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    override func layoutSubviews() {
        super.layoutSubviews()
        CATransaction.begin()
        CATransaction.setDisableActions(true)        // без неявной анимации размера
        gradient.frame = bounds
        CATransaction.commit()
    }

    // Окно появилось (или вернулись в это окно) — запускаем заново.
    override func didMoveToWindow() {
        super.didMoveToWindow()
        if window != nil { startAnimating() }
    }

    func startAnimating() {
        guard !UIAccessibility.isReduceMotionEnabled else { return }   // «Уменьшить
        ↪ движение»
        let animation = CABasicAnimation(keyPath: "locations")
        animation.fromValue = [-1, -0.5, 0]
        animation.toValue = [1, 1.5, 2]
        animation.duration = 1.2
        animation.repeatCount = .infinity
        gradient.add(animation, forKey: "shimmer")
    }

    func stopAnimating() {
        gradient.removeAnimation(forKey: "shimmer")
    }
}
```

Фон — серый прямоугольник: `tertiaryLabel` с прозрачностью 15%. Системный цвет подстраивается под тёмную тему.

Блик — **градиент** из трёх цветов: прозрачный → белый на 35% → прозрачный. `startPoint (0; 0,5)` и `endPoint (1; 0,5)` — градиент идёт слева направо по средней линии. Координаты тут не в точках, а в долях слоя: 0 — левый край, 1 — правый.

`locations` — где вдоль этой линии стоят три цвета, тоже в долях ширины. `[–1, –0.5, 0]`: прозрачный на –1 (на целую ширину левее слоя), яркая середина блика на –0,5, снова прозрачный ровно на левом краю. Весь блик левее видимой области, прямоугольник просто серый.

Анимация двигает эти три числа к `[1, 1.5, 2]`: середина блика проходит путь от –0,5 до 1,5, то есть пересекает прямоугольник целиком (от 0 до 1) и уходит вправо. Путь — 2 ширины за

1,2 секунды, из них видимая часть — половина времени, 0,6 секунды. Потом `repeatCount = .infinity` запускает всё заново.

Почему анимируем `locations`, а не сдвиг слоя? Напрашивается двигать `transform.translation.x` от `-bounds.width` до `bounds.width`. Но если `startAnimating()` вызвать до раскладки — а так почти всегда и делают, сразу после создания, — `bounds.width` равна нулю, анимация едет «от 0 до 0», и блика нет. Доли ширины от размера не зависят: одна и та же анимация работает на прямоугольнике любой ширины и даже переживает поворот экрана.

`layoutSubviews` + `CATransaction.setDisableActions(true)` — `CALayer` — не view (`CALayer` → `NSObject`, глава 0, раздел 0.8) и в Auto Layout не участвует, поэтому размер отдельного слоя обновляем руками при каждой раскладке. А у отдельного (не «своего» для view) слоя любое изменение `frame` по умолчанию **неявно анимируется** за 0,25 секунды: блик бы «догонял» растущий прямоугольник. У слоя самой view такой анимации нет: view — делегат своего слоя и вне `UIView.animate` отвечает на `action(forKey:)` «ничего не анимировать». Транзакция с выключенными действиями даёт тот же ответ для чужого слоя. (Проверили в симуляторе: без неё у слоя после изменения размера появляются анимации `position` и `bounds`, с ней — ни одной.)

`didMoveToWindow` — когда view снят с экрана (ячейку переиспользовали, экран закрыли), Core Animation удаляет его анимации. Перезапуск при появлении в окне возвращает блик.

`UIAccessibility.isReduceMotionEnabled` — у пользователя включено «Настройки → Универсальный доступ → Движение → Уменьшение движения». Людям с вестибулярными нарушениями бегущие блики мешают; для них оставляем неподвижный серый скелет.

Анимация выполняется в Core Animation: главный поток только передаёт ей параметры, дальше кадры считаются отдельно. Поэтому сотня бликов не мешает прокрутке.

В ячейке таблицы скелет — это несколько `SkeletonView` на местах будущих картинки и строк текста. Пример — в главе 15 (Weather).

Частые ошибки.

- **Скелет не похож на экран.** Серые полоски должны стоять там, где будут заголовок, подзаголовок и картинка. Скелет «вообще» (три одинаковые полосы на любом экране) ничего не сообщает.
- **Текст «Загрузка…» внутри скелета.** Скелет — это форма без содержания; надпись делает его похожим на настоящий, но сломанный контент.
- **Слишком много скелетов.** 3–6 строк на экран достаточно. Две сотни одинаковых серых ячеек выглядят как зависание.
- **Скелет висит вечно при ошибке.** Если загрузка упала, скелет меняем на экран ошибки (глава 24), а не оставляем мерцать.

Альтернативы.

- **Спиннер по центру** — проще; хорош для коротких загрузок (меньше секунды) и экранов без чёткой формы.
- **Кеш сначала** — сразу показываем сохранённые в прошлый раз данные, а свежие подменяют их, когда придут. Лучше любого скелета, если кеш есть.

23.4 Spinner внутри кнопки

Spinner («спиннер») — крутящийся индикатор активности, `UIActivityIndicatorView`.

Когда применять. Кнопка, после нажатия которой идёт запрос: «Войти», «Сохранить», «Отправить».

Минимальный код.

```
final class LoginViewController: UIViewController {
    private let loginButton = UIButton(configuration: .filled())

    override func viewDidLoad() {
        super.viewDidLoad()
        loginButton.configuration?.title = "Войти"
        loginButton.addAction(UIAction { [weak self] _ in
            self?.login()
        }, for: .touchUpInside)
    }

    private func login() {
        startLoading()
        Task {
            defer { stopLoading() }
            try? await Task.sleep(nanoseconds: 1_000_000_000)   // вместо настоящего
            ↪ запроса
        }
    }

    private func startLoading() {
        view.endEditing(true)
        loginButton.isEnabled = false
        var cfg = loginButton.configuration
        cfg?.showsActivityIndicator = true
        cfg?.title = "Входим…"
        loginButton.configuration = cfg
    }

    private func stopLoading() {
        var cfg = loginButton.configuration
        cfg?.showsActivityIndicator = false
        cfg?.title = "Войти"
        loginButton.configuration = cfg
        loginButton.isEnabled = true
    }
}
```

`UIButton.Configuration` (iOS 15+) — описание внешнего вида кнопки одним значением: стиль, заголовок, картинка, отступы. `showsActivityIndicator = true` — встроенный спиннер.

По документации он показывается **на месте картинки** кнопки (слева от заголовка, если `imagePlacement` не менял).

`configuration` — это структура, значение, а не ссылка. Поэтому шаблон «скопировал → поменял → записал обратно»: `var cfg = loginButton.configuration`, правки, `loginButton.configuration = cfg`. Кнопка перерисуется при записи.

`view.endEditing(true)` — убираем клавиатуру: пользователь нажал «Войти», вводить больше нечего.

`isEnabled = false` — вторая половина рецепта, не менее важная, чем спиннер: пока идёт запрос, повторный тап невозможен.

`defer { stopLoading() }` в `Task` — кнопка вернётся в обычное состояние при любом исходе: успех, ошибка, отмена.

Частые ошибки.

- **Спиннер без блокировки.** Пользователь тапает пять раз — уходит пять запросов входа.
- **Кнопку не вернули после ошибки.** Кнопка навсегда «Входим…» и серая. Лечится `defer`.
- **Спиннер без текста.** Если убрать заголовок, кнопка сожмётся до размера спиннера, и вся раскладка вокруг «прыгнет». Меняй текст на «Входим…», а не убирай его.

Альтернативы.

- **Полноэкранный overlay** (23.7) — для долгих операций, когда трогать экран во время ожидания нельзя.
- **Оптимистичное действие** — показываем результат сразу («сообщение отправлено»), а при ошибке откатываем. Так работают мессенджеры.

23.5 Progressive image loading

Когда применять. Картинки из сети в списке или сетке. Пока картинка грузится — нейтральная заливка; пришла — плавно проявляется (**fade-in**, «проявление» от прозрачного к видимому).

Минимальный код. Самый частый случай — ячейка коллекции:

```
final class PhotoCell: UICollectionViewCell {
    private let imageView = UIImageView()
    private var loadTask: Task<Void, Never>?

    override init(frame: CGRect) {
        super.init(frame: frame)
        imageView.contentMode = .scaleAspectFill
        imageView.clipsToBounds = true
        imageView.backgroundColor = .secondarySystemFill      // заглушка, пока грузим
        imageView.frame = contentView.bounds
        imageView.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        contentView.addSubview(imageView)
    }
```

```
    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    func configure(with url: URL) {
        loadTask?.cancel()
        imageView.image = nil
        imageView.alpha = 0
        loadTask = Task { [weak self] in
            let image = await ImageCache.shared.image(for: url)
            guard !Task.isCancelled, let self else { return }   // ячейку уже отдали
             ↪  другому URL
            self.imageView.image = image
            UIView.animate(withDuration: 0.25) { self.imageView.alpha = 1 }
        }
    }

    override func prepareForReuse() {
        super.prepareForReuse()
        loadTask?.cancel()
        loadTask = nil
        imageView.image = nil
    }
}
```

Главная опасность здесь — **переиспользование ячеек**. Таблица и коллекция не создают ячейку на каждую строку: ушедшая за край экрана ячейка переиспользуется для новой строки. Сценарий бага: ячейка начала грузить фото A, пользователь пролистал, та же ячейка теперь показывает строку с фото B. Если A придёт позже B, оно перезапишет правильную картинку — в ленте «чужие» фото.

Защита — хранить задачу загрузки и отменять её:

- в `configure(with:)` перед новой загрузкой отменяем старую;
- в `prepareForReuse()` (UIKit зовёт его прямо перед переиспользованием) — тоже отменяем и чистим картинку;
- после `await` проверяем `Task.isCancelled`: отменённая задача не имеет права трогать ячейку.

`imageView.alpha = 0` + `UIView.animate(withDuration: 0.25) { alpha = 1 }` — картинка проявляется за четверть секунды поверх серой заливки `secondarySystemFill`.

`Task { [weak self] in ... }` создан внутри класса ячейки, а весь код книги живёт на главном потоке (MainActor по умолчанию) — задача **наследует** главный поток. Поэтому после `await` можно сразу менять `imageView`, без `await MainActor.run { }` — здесь это лишняя обёртка, которая только запутывает.

`[weak self]` — ячейку могут выбросить из памяти, пока картинка грузится; держать её сильной ссылкой ради картинки, которую никто не увидит, незачем.

`ImageCache` — кеш картинок из главы 16 (Gallery): сначала память, потом сеть.

Частые ошибки.

- **Нет отмены / проверки при переиспользовании** — «чужие» картинки в ячейках при быстрой прокрутке.
- **Не обнулили картинку.** Пока грузится новая, в ячейке видна старая картинка от прошлой строки.
- **Картинка на весь экран в ячейке 80×80.** Декодированная фотография 4000×3000 занимает около 48 МБ памяти (4000 × 3000 точек × 4 байта на пиксель). Для миниатюр проси у сервера маленькую версию или уменьшай картинку перед показом (`UIImage.preparingThumbnail(of:)`, iOS 15+).

Альтернативы.

- **BlurHash** — сервер вместе со ссылкой отдаёт строку в 20–30 символов, из которой мгновенно рисуется размытое «пятно» с цветами будущей фотографии. Нужна сторонняя библиотека декодирования.
- **Сначала маленькая, потом большая** — показываем миниатюру, поверх неё проявляется полная версия. Два запроса вместо одного.

Упражнение 23.2. Убери из `PhotoCell` строку `guard !Task.isCancelled, let self else { return }`, заменив на `guard let self else { return }`, и сделай так, чтобы `ImageCache` отвечал случайно от 0,1 до 2 секунд. Что увидишь при быстрой прокрутке сетки? Ответ — в конце главы.

23.6 Toast / Snackbar

Toast («тост») или **snackbar** — короткое сообщение внизу экрана, которое само появляется и само исчезает через пару секунд: «Сохранено», «Нет сети». В UIKit готового компонента нет, делаем сами.

Когда применять. Подтверждение завершённого действия или некритичная ошибка, на которую пользователю не нужно отвечать.

Минимальный код.

```
extension UIViewController {
    func showToast(_ text: String) {
        let label = PaddedLabel()
        label.text = text
        label.font = .preferredFont(forTextStyle: .subheadline)
        label.adjustsFontForContentSizeCategory = true
        label.numberOfLines = 0
        label.textAlignment = .center
        label.textColor = .white
        label.backgroundColor = UIColor.black.withAlphaComponent(0.85)
        label.layer.cornerRadius = 18
        label.layer.masksToBounds = true
        label.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(label)
        NSLayoutConstraint.activate([
            label.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            label.widthAnchor.constraint(lessThanOrEqualTo: view.widthAnchor, constant:
↪  -32),
```

```
            label.bottomAnchor.constraint(equalTo: view.safeAreaLayoutGuide.bottomAnchor,
↪   constant: 100),
        ])
        UIAccessibility.post(notification: .announcement, argument: text)   // VoiceOver
↪   прочитает

        UIView.animate(withDuration: 0.35) {
            label.transform = CGAffineTransform(translationX: 0, y: -120)
        }
        DispatchQueue.main.asyncAfter(deadline: .now() + 1.6) {
            UIView.animate(withDuration: 0.35, animations: {
                label.transform = .identity
                label.alpha = 0
            }, completion: { _ in
                label.removeFromSuperview()
            })
        }
    }
}

final class PaddedLabel: UILabel {
    private let inset = UIEdgeInsets(top: 10, left: 16, bottom: 10, right: 16)

    override func drawText(in rect: CGRect) {
        super.drawText(in: rect.inset(by: inset))
    }

    // Размер = размер текста в «урезанной» рамке + отступы обратно.
    override func textRect(forBounds bounds: CGRect, limitedToNumberOfLines
     ↪  numberOfLines: Int) -> CGRect {
        let inner = super.textRect(forBounds: bounds.inset(by: inset),
         ↪  limitedToNumberOfLines: numberOfLines)
        return inner.inset(by: UIEdgeInsets(top: -inset.top, left: -inset.left,
                                            bottom: -inset.bottom, right: -inset.right))
    }
}
```

Сначала геометрия, на числах. Низ плашки привязан на **100 точек ниже** нижней границы safe area — то есть плашка стартует за нижним краем видимой области. Затем `CGAffineTransform(translationX: 0, y: -120)` — сдвиг на 120 точек вверх. Итог: 100 − 120 = −20, плашка встаёт на 20 точек **выше** safe area. `transform` — это визуальный сдвиг поверх Auto Layout: constraints не меняются, view просто рисуется в другом месте.

Исчезновение — `transform = .identity` («без сдвига») и `alpha = 0` одновременно: плашка уезжает вниз и тает за 0,35 секунды. Затем `removeFromSuperview()` в `completion` — иначе прозрачные плашки копились бы в иерархии.

Время на экране: 0,35 с въезд, потом `asyncAfter(1.6)` отсчитывает 1,6 секунды от начала, то есть плашка стоит неподвижно около 1,25 секунды, затем 0,35 с уезжает. Короткое «Сохранено» так прочитать успевают. Для текста длиннее пяти-шести слов увеличь паузу.

`widthAnchor ≤ ширина экрана − 32` + `numberOfLines = 0` — длинный текст переносится на несколько строк и оставляет по 16 точек с краёв, а не уезжает за экран.

`PaddedLabel` — `UILabel` с внутренними отступами 10 сверху и снизу, 16 слева и справа. `drawText(in:)` рисует текст в рамке, уменьшенной на отступы. `textRect(forBounds:limitedToNumberOfLines:)` — метод, которым UILabel считает свой размер: берём размер текста в урезанной ширине и «наращиваем» отступы обратно (отрицательные inset'ы расширяют прямоугольник). Первая версия рецепта переопределяла `intrinsicContentSize` — это работает для одной строки, но многострочный текст считался бы по полной ширине и обрезался.

`layer.masksToBounds = true` — без него у `UILabel` углы не скруглятся. У обычного `UIView` фон хранится в `layer.backgroundColor`, и `cornerRadius` скругляет его сам, без обрезки. `UILabel` же рисует свой фон вместе с текстом в содержимое слоя (замер: после `label.backgroundColor = .red` `label.layer.backgroundColor` остаётся `nil`), а содержимое `cornerRadius` не трогает — его обрезает только `masksToBounds`. Это то же самое свойство, что `clipsToBounds` у view (замер: включил одно — включилось другое).

`UIAccessibility.post(notification: .announcement, argument: text)` — VoiceOver прочитает сообщение вслух. Иначе незрячий пользователь toast не узнает: он не получает фокус.

Касания: у `UILabel` по умолчанию `isUserInteractionEnabled == false` (проверили в симуляторе), поэтому тап по плашке **проходит сквозь** неё к тому, что под ней. Если нужна кнопка «Отменить» внутри toast'а — делай плашку из `UIView` с кнопкой внутри.

Частые ошибки.

- **Очередь сообщений.** Три toast'а подряд лягут друг на друга. Если сообщения могут идти пачкой — держи очередь и показывай по одному.
- **Toast для важного.** «Платёж не прошёл» не должен исчезать сам через две секунды. Для этого — алерт или экран ошибки.
- **Забыли про VoiceOver** — сообщение видят все, кроме тех, кто не видит.

Альтернативы.

- **`UIAlertController`** — для важного: требует явного «ОК», прерывает работу.
- **Баннер сверху поверх всего приложения** (как системное уведомление) — отдельное `UIWindow` с `windowLevel = .alert`, в которое кладётся плашка. Так баннер переживает смену экранов.

23.7 Loading overlay (полноэкранный)

Overlay — полупрозрачный слой поверх экрана, который блокирует касания.

Когда применять. Операция, во время которой пользователю нельзя ничего трогать: оплата, экспорт файла, долгая синхронизация.

Минимальный код.

```swift
extension UIViewController {
    @discardableResult
    func showOverlay(text: String) -> UIView {
        let overlay = UIView(frame: view.bounds)
        overlay.autoresizingMask = [.flexibleWidth, .flexibleHeight]   // растёт при
↪   повороте
        overlay.backgroundColor = UIColor.black.withAlphaComponent(0.5)
        overlay.alpha = 0
        overlay.accessibilityViewIsModal = true

        let spinner = UIActivityIndicatorView(style: .large)
        spinner.color = .white
        spinner.startAnimating()

        let label = UILabel()
        label.text = text
        label.textColor = .white
        label.font = .preferredFont(forTextStyle: .headline)
        label.adjustsFontForContentSizeCategory = true

        let stack = UIStackView(arrangedSubviews: [spinner, label])
        stack.axis = .vertical
        stack.spacing = 12
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        overlay.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: overlay.centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: overlay.centerYAnchor),
        ])

        view.addSubview(overlay)
        UIView.animate(withDuration: 0.2) { overlay.alpha = 1 }
        return overlay
    }

    func hideOverlay(_ overlay: UIView) {
        UIView.animate(withDuration: 0.2, animations: {
            overlay.alpha = 0
        }, completion: { _ in
            overlay.removeFromSuperview()
        })
    }
}
```

Чёрный с прозрачностью 50% затемняет экран наполовину: видно, что экран на месте, но «занят». Большой белый спиннер и подпись по центру.

`UIView(frame: view.bounds) + autoresizingMask = [.flexibleWidth, .flexibleHeight]` — overlay создан рамкой, а не constraints, поэтому ему нужна «маска авторесайза»: растягиваться вслед за родителем. Без маски при повороте экрана overlay остался бы старого размера, и половина экрана оказалась бы не закрыта (и доступна для тапов). Заметь: `translatesAutoresizingMaskIntoConstraints` здесь **не** выключаем — для view, которую раскладываем рамкой, значение по умолчанию `true` как раз и нужно (глава 0, раздел 0.3).

`UIView` по умолчанию принимает касания (`isUserInteractionEnabled == true`, замер) — поэтому overlay их и «съедает»: hit-testing находит его первым, ведь он добавлен последним и лежит выше всех в `subviews`, а до кнопок под ним поиск уже не доходит. Прозрачность 50% этому не мешает — касания пропускает только view с `alpha` ниже 0,01, скрытый или с выключенным `isUserInteractionEnabled`.

`accessibilityViewIsModal = true` — VoiceOver не будет читать элементы под затемнением.

Функция возвращает overlay, чтобы потом снять именно его через `hideOverlay(_:)`. `@discardableResult` — не ругаться, если результат не используют.

Показ и скрытие — 0,2 секунды на прозрачность: плавно, но без задержки.

Ограничение: overlay лежит внутри `view` текущего экрана. Если экран внутри `UINavigationController`, навигационная панель **не** закрыта — кнопка «Назад» останется нажимаемой. Если это недопустимо, клади overlay в `navigationController?.view` или в окно (`view.window`). Окно — это тоже view (`UIWindow → UIView → UIResponder`, глава 0, раздел 0.7), просто корень дерева: `addSubview` у него работает как у любой другой view, и z-порядок тот же — кто добавлен позже, тот выше. Overlay, положенный в окно поверх `rootViewController.view`, накрывает весь экран приложения вместе с навигационной панелью. Выше окна только другие окна (`UIWindow.Level`).

Частые ошибки.

- **Спиннер без подписи.** «Крутится — а что происходит?» Подпись «Готовим отчёт…» снимает вопрос.
- **Нет отмены.** Если операция может длиться минуту, дай кнопку «Отмена».
- **Overlay на каждую мелочь.** Блокировать экран ради запроса в 300 мс — мигание. Для коротких операций хватит спиннера в кнопке (23.4).

Альтернативы.

- **Прогресс-бар** (`UIProgressView`) — если известна доля выполнения: «Загружено 45%» честнее бесконечного спиннера.
- **Фоновая работа + уведомление** — для операций на минуты: пользователь уходит, приложение сообщает по готовности.

Упражнение 23.3. Вызови `showOverlay(text: "Готовим отчёт…")` на экране внутри `UINavigationController` и попробуй нажать «Назад». Потом поменяй `view.addSubview(overlay)` на добавление в окно. Что изменится? И что будет с overlay при повороте симулятора, если убрать `autoresizingMask`? Ответ — в конце главы.

Ответы к упражнениям

Упражнение 23.1. Страницы 1 и 2 приходят полными (по 20), третья — 13 элементов: `13 == 20` ложно, `hasMore` становится `false`. Итого **3 запроса**. Если оставить только «пустой ответ = конец», после страницы 3 уйдёт **четвёртый** запрос, вернёт пустой массив, и только тогда лента поймёт, что всё. Лишний запрос на каждую полную прокрутку каждого пользователя.

Упражнение 23.2. При быстрой прокрутке в ячейках сначала появляются правильные фото, а через долю секунды некоторые меняются на «чужие» — от строк, которые эта ячейка показывала раньше. Медленный ответ старого запроса пришёл после быстрого нового и перезаписал картинку. Вернёшь проверку `Task.isCancelled` — эффект пропадёт.

Упражнение 23.3. В первом варианте кнопка «Назад» в навигационной панели остаётся доступной: overlay закрывает только область экрана под панелью. Вариант с окном:

```
func showWindowOverlay(text: String) -> UIView? {
    guard let window = view.window else { return nil }
    let overlay = UIView(frame: window.bounds)
    overlay.autoresizingMask = [.flexibleWidth, .flexibleHeight]
    overlay.backgroundColor = UIColor.black.withAlphaComponent(0.5)
    window.addSubview(overlay)
    return overlay
}
```

— закрывает и панель, «Назад» не нажимается. Без `autoresizingMask` после поворота в альбомную ориентацию overlay остаётся портретного размера (393 точки в ширину на iPhone 16), правая часть экрана открыта и нажимается.

Что мы выучили

- **Pull-to-refresh** — `UIRefreshControl` (это `UIControl`: `addAction`, `.valueChanged`) в `refreshControl` любого scroll view, `endRefreshing()` при любом исходе, старые данные при ошибке не трогаем.
- **Пагинация** — триггер в `willDisplay` за N строк до конца, флаг «уже грузим», конец ленты по неполной странице, `insertRows` вместо `reloadData()`.
- **Skeleton** — серый фон + градиент-блик; анимируем `locations` (доли ширины), а не сдвиг в точках; без неявной анимации размера; перезапуск в `didMoveToWindow`; уважаем «Уменьшение движения».
- **Спиннер в кнопке** — `UIButton.Configuration.showsActivityIndicator` (iOS 15+) + `isEnabled = false` + `defer`.
- **Картинки в ячейках** — отмена `Task` в `prepareForReuse`, проверка `Task.isCancelled` после `await`, fade-in за 0,25 с.
- **Toast** — плашка за нижним краем + `transform` вверх; VoiceOver-объявление; `UILabel` пропускает касания сквозь себя; скругление фона `UILabel` требует `masksToBounds` (он же `clipsToBounds`).
- **Overlay** — `autoresizingMask`, блокировка касаний (view добавлен последним — hit-testing находит его первым), модальность для VoiceOver; навигационную панель закрывает только overlay на окне, а окно — тоже `UIView`.

Apple Developer Documentation

- `UIRefreshControl` — стандартный контрол pull-to-refresh.
- `UIScrollView.refreshControl` — слот для `UIRefreshControl` (iOS 10+) у любого scroll view.
- `UIControl` — родитель `UIRefreshControl`: target-action, `addAction(_:for:)`, события.
- `UIWindow` — окно как корневой `UIView` для overlay поверх всего.
- `tableView(_:willDisplay:forRowAt:)` — точка триггера пагинации.
- `performBatchUpdates(_:completion:)` — вставка новых строк без полной перезагрузки.
- `UIActivityIndicatorView` — системный спиннер.
- `UIButton.Configuration.showsActivityIndicator` — спиннер внутри кнопки (iOS 15+).
- `CAGradientLayer` и `CABasicAnimation` — основа блика для скелета.
- `UIAccessibility.isReduceMotionEnabled` — настройка «Уменьшение движения».
- `prepareForReuse()` — сброс ячейки перед переиспользованием.
- `URLSession.data(from:)` — загрузка данных через async/await (iOS 15+).
- HIG — Loading — рекомендации Apple по показу загрузки.

→ Глава 24. Cookbook — empty / error / offline states

Глава 24. Cookbook — empty / error / offline states

Загрузка закончилась — и не всегда списком данных. Возможных исходов три, кроме успеха: ничего не пришло (**empty state**, «пустое состояние»), пришла ошибка (**error state**) и нет интернета (**offline**). Каждый требует своего экрана. Белый пустой экран в любом из трёх случаев выглядит одинаково — как поломка.

> **Режим компиляции.** Листинги главы проверены компилятором: Swift 6, Default Actor Isolation = MainActor, deployment target iOS 15.0; код с `NWPathMonitor` и повторами запроса — ещё и в языковом режиме Swift 5.

24.1 EmptyStateView — общий компонент

Когда применять. Список без элементов: первый запуск, фильтр ничего не нашёл, пользователь всё удалил или всё выполнил.

Минимальный код. Иконка, заголовок, пояснение и необязательная кнопка — один компонент на всё приложение:

```
final class EmptyStateView: UIView {
    private let iconView = UIImageView()
    private let titleLabel = UILabel()
    private let bodyLabel = UILabel()
    private let actionButton = UIButton(configuration: .filled())
    private let stack = UIStackView()

    override init(frame: CGRect) {
        super.init(frame: frame)
        iconView.tintColor = .tertiaryLabel
        iconView.contentMode = .scaleAspectFit
        iconView.preferredSymbolConfiguration =
            UIImage.SymbolConfiguration(pointSize: 56, weight: .regular)
        iconView.isAccessibilityElement = false          // картинка-украшение

        titleLabel.font = .preferredFont(forTextStyle: .title3)
        titleLabel.textColor = .secondaryLabel
        titleLabel.textAlignment = .center
        titleLabel.numberOfLines = 0
```

```
        titleLabel.adjustsFontForContentSizeCategory = true

        bodyLabel.font = .preferredFont(forTextStyle: .subheadline)
        bodyLabel.textColor = .secondaryLabel
        bodyLabel.textAlignment = .center
        bodyLabel.numberOfLines = 0
        bodyLabel.adjustsFontForContentSizeCategory = true

        actionButton.configuration?.cornerStyle = .capsule
        actionButton.isHidden = true                        // по умолчанию кнопки нет

        for v in [iconView, titleLabel, bodyLabel, actionButton] {
            stack.addArrangedSubview(v)
        }
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 8
        stack.setCustomSpacing(16, after: iconView)
        stack.setCustomSpacing(24, after: bodyLabel)
        stack.translatesAutoresizingMaskIntoConstraints = false
        addSubview(stack)
        NSLayoutConstraint.activate([
            stack.centerYAnchor.constraint(equalTo: safeAreaLayoutGuide.centerYAnchor),
            stack.leadingAnchor.constraint(equalTo: layoutMarginsGuide.leadingAnchor,
↪   constant: 16),
            stack.trailingAnchor.constraint(equalTo: layoutMarginsGuide.trailingAnchor,
↪   constant: -16),
        ])
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }

    func configure(iconName: String, title: String, body: String) {
        iconView.image = UIImage(systemName: iconName)
        titleLabel.text = title
        bodyLabel.text = body
    }
}
```

Разберём, что тут важно.

`UIImage.SymbolConfiguration(pointSize: 56)` — SF Symbol рисуется в размере 56 точек: крупно, но не на пол-экрана. `.tertiaryLabel` — третий по яркости системный цвет текста: иконка видна, но не спорит с заголовком.

`isAccessibilityElement = false` на иконке — картинка-украшение. VoiceOver пропустит её и сразу прочитает заголовок; иначе он сказал бы «лоток, изображение» — бесполезный шум.

Шрифты — `preferredFont(forTextStyle:)` с `adjustsFontForContentSizeCategory`: это **Dynamic Type**, текст растёт вместе с настройкой размера шрифта в системе. Ранняя версия рецепта задавала `systemFont(ofSize: 18)` и `14` — такие шрифты не увеличиваются, и пользователю с крупным текстом пустой экран читать трудно. `numberOfLines = 0` у обоих текстов — при крупном шрифте заголовок переносится на вторую строку, а не обрезается троеточием.

Оба текста — `.secondaryLabel`. Соблазн взять для пояснения `.tertiaryLabel` лучше побороть: на белом фоне это серый с низким контрастом, который плохо читается. Для текста, который надо прочитать, не бери цвет бледнее `secondaryLabel`.

Стек (`UIStackView`) сам раскладывает элементы в столбик. `spacing = 8` — расстояние между элементами по умолчанию, `setCustomSpacing(16, after: iconView)` — после иконки отступ больше, `setCustomSpacing(24, after: bodyLabel)` — ещё больше перед кнопкой: кнопка — отдельное действие, а не продолжение текста. Скрытая кнопка (`isHidden = true`) в стеке не занимает места вовсе — стек пропускает скрытые элементы вместе с их отступами. Это штатное поведение: в заголовке `UIStackView.h` сказано, что стек перекладывает содержимое при смене `hidden` у любого из `arrangedSubviews`. Замер: три подписи по 20,33 точки с `spacing = 8` дают стеку высоту 77; скрыли одну — 48,67, то есть ушли и подпись, и один отступ. Сама скрытая view при этом остаётся и в `arrangedSubviews`, и в `subviews`.

Стек центрирован по вертикали и растянут по ширине между **layout margins** (внутренними полями view) плюс ещё 16 точек. У обычного `UIView` поля по умолчанию 8 точек с каждой стороны; у корневого view экрана они больше — там действуют системные минимальные поля плюс safe area (замер на iPhone 16: 16 точек по бокам, 59 сверху, 34 снизу). Подробно — глава 0, раздел 0.3. Наш `EmptyStateView` — обычный view, так что на iPhone 16 шириной 393 точки текст займёт примерно 393 − 2 × (8 + 16) = 345 точек — длинное пояснение перенесётся, а не прилипнет к краям.

Частые ошибки.

- **Один текст на все случаи.** «Ничего не найдено» подходит для пустого фильтра, но не для первого запуска. Разные ситуации — разные тексты.
- **Нет подсказки, что делать.** Пустой список — момент научить пользователя: «нажми плюс», «сбрось фильтры». Лучше всего — кнопкой (24.2).
- **Пустой экран во время загрузки.** Empty state показываем, только когда загрузка **закончилась** и данных правда нет. Пока грузится — скелет или спиннер (глава 23), иначе пользователь на секунду увидит «Пока пусто» при каждом открытии.

Варианты по контексту.

```
// Первый запуск
emptyView.configure(iconName: "tray", title: "Пока пусто",
                    body: "Добавь первую задачу — нажми «плюс» внизу.")

// Всё выполнено
emptyView.configure(iconName: "checkmark.seal.fill", title: "Всё выполнено!",
                    body: "Активных задач нет. Можно отдохнуть.")

// Фильтр ничего не нашёл
emptyView.configure(iconName: "magnifyingglass", title: "Не нашлось",
                    body: "Попробуй изменить запрос или сбросить фильтры.")
```

Три разных иконки и тона. «Пока пусто» — нейтрально и с инструкцией. «Всё выполнено» — похвала, а не пустота. «Не нашлось» — подсказка, как расширить поиск. (Эмодзи в строке вроде «Можно отдохнуть» не ставим: в интерфейсе эмодзи зависят от шрифта и плохо читаются VoiceOver'ом — лучше обойтись текстом или SF Symbol.)

24.2 Empty state с call-to-action

Call-to-action («призыв к действию») — кнопка, которая прямо говорит, что делать дальше: «Создать первую задачу».

Кнопка уже лежит в стеке `EmptyStateView` (скрытая). Добавим метод, который её показывает:

```
extension EmptyStateView {
    /// Кнопка «что делать дальше». Передай nil — кнопка спрячется.
    func setAction(title: String?, handler: (() -> Void)? = nil) {
        guard let title, let handler else {
            actionButton.isHidden = true
            return
        }
        actionButton.configuration?.title = title
        // Старый обработчик снимаем, иначе при повторном вызове их станет два.
        actionButton.removeAction(identifiedBy: UIAction.Identifier("empty.cta"), for:
 ↪  .touchUpInside)
        actionButton.addAction(UIAction(identifier: UIAction.Identifier("empty.cta")) { _
 ↪  in handler() },
                               for: .touchUpInside)
        actionButton.isHidden = false
    }
}
```

(В файле проекта метод лежит прямо в классе: `actionButton` объявлен `private`, а `private` в Swift виден и в расширениях того же файла.)

`UIAction.Identifier("empty.cta")` — у действия есть имя. Если один и тот же `EmptyStateView` сначала показал «Создать первую задачу», а потом ещё раз — без `removeAction(identifiedBy:)` на кнопке висело бы два обработчика, и тап открыл бы редактор дважды. Действия с одинаковым идентификатором UIKit и сам заменяет, но явное удаление читается понятнее.

Использование на экране списка:

```
final class TodoListDemoViewController: UIViewController {
    private let emptyView = EmptyStateView()

    func showFirstLaunch() {
        emptyView.configure(iconName: "tray", title: "Пока пусто",
                            body: "Добавь первую задачу — нажми «плюс» внизу.")
        emptyView.setAction(title: "Создать первую задачу") { [weak self] in
            self?.openEditor()
        }
    }
```

```
    func showAllDone() {
        emptyView.configure(iconName: "checkmark.seal.fill", title: "Всё выполнено!",
                            body: "Активных задач нет. Можно отдохнуть.")
        emptyView.setAction(title: nil)
    }

    private func openEditor() {}
}
```

`[weak self]` — экран держит `emptyView`, тот держит кнопку, кнопка — замыкание. Сильный `self` в замыкании замкнул бы круг, и экран не освободился бы после закрытия.

Кнопку нельзя создать снаружи и добавить в `stack`: `stack` объявлен `private`, из другого файла к нему не добраться, и такой код не соберётся. Поэтому кнопка — часть компонента, а снаружи ею управляют только через `setAction`.

Упражнение 24.1. Вызови `setAction(title: "Создать") { print("tap") }` дважды подряд, но закомментируй строку `removeAction(identifiedBy:)` и создавай действие **без** идентификатора: `UIAction { _ in handler() }`. Сколько раз напечатается «tap» на одно нажатие? Ответ — в конце главы.

24.3 Error state — иконка ошибки + retry

Retry — «повторить попытку».

Когда применять. Запрос упал: нет сети, сервер ответил ошибкой, данные не разобрались. Пользователю нужно понять, что случилось, и дать кнопку «Повторить».

Минимальный код.

```
func makeErrorState(onRetry: @escaping () -> Void) -> UIStackView {
    let icon = UIImageView(image: UIImage(systemName: "exclamationmark.icloud.fill"))
    icon.tintColor = .systemRed
    icon.contentMode = .scaleAspectFit
    icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 56)

    let title = UILabel()
    title.text = "Не удалось загрузить"
    title.font = .preferredFont(forTextStyle: .title2)
    title.adjustsFontForContentSizeCategory = true
    title.textAlignment = .center
    title.numberOfLines = 0

    let body = UILabel()
    body.text = "Проверь интернет и попробуй ещё раз."
    body.font = .preferredFont(forTextStyle: .body)
    body.adjustsFontForContentSizeCategory = true
    body.textColor = .secondaryLabel
```

```
    body.textAlignment = .center
    body.numberOfLines = 0

    var cfg = UIButton.Configuration.filled()
    cfg.title = "Повторить"
    cfg.cornerStyle = .capsule
    cfg.image = UIImage(systemName: "arrow.clockwise")
    cfg.imagePlacement = .leading
    cfg.imagePadding = 8
    let retry = UIButton(configuration: cfg, primaryAction: UIAction { _ in onRetry() })

    let stack = UIStackView(arrangedSubviews: [icon, title, body, retry])
    stack.axis = .vertical
    stack.alignment = .center
    stack.spacing = 12
    return stack
}
```

Красная иконка «облако с восклицательным знаком», заголовок, пояснение и кнопка с круговой стрелкой.

`UIButton(configuration:primaryAction:)` (iOS 15+) — кнопка и её действие одной строкой.

`imagePlacement = .leading` — картинка перед текстом («leading» — со стороны начала строки: слева в русском, справа в арабском и иврите). `imagePadding = 8` — 8 точек между стрелкой и словом.

`onRetry` — `@escaping`, потому что замыкание сохраняется внутри кнопки и вызывается позже, когда функция давно вернулась.

Текст ошибки человеческим языком — из типа ошибки:

```
func userMessage(for error: Error) -> String {
    guard let urlError = error as? URLError else { return "Что-то пошло не так. Попробуй
    ↪  ещё раз." }
    switch urlError.code {
    case .notConnectedToInternet, .networkConnectionLost:
        return "Нет интернета. Проверь Wi-Fi или мобильную сеть."
    case .timedOut:
        return "Сервер долго не отвечает. Попробуй чуть позже."
    default:
        return "Сервер недоступен. Попробуй чуть позже."
    }
}
```

`URLError` — ошибка сетевого уровня от `URLSession`. Код `.notConnectedToInternet` — это тот самый «NSURLErrorDomain −1009», который иначе увидел бы пользователь.

Повтор с растущей паузой. Если повторяешь запрос автоматически — не подряд. Паузы удваиваются: это **exponential backoff** («экспоненциальная задержка»):

```swift
// Повтор с растущей паузой: 1 с, 2 с — и после третьей неудачи сдаёмся.
func withRetry<T>(maxAttempts: Int = 3,
                  _ operation: () async throws -> T) async throws -> T {
    var delay: Double = 1
    for attempt in 1...maxAttempts {
        do {
            return try await operation()
        } catch {
            if attempt == maxAttempts { throw error }
            let jitter = Double.random(in: 0...0.3)          // разброс, чтобы клиенты не
            ↪ били синхронно
            try await Task.sleep(nanoseconds: UInt64((delay + jitter) * 1_000_000_000))
            delay *= 2
        }
    }
    fatalError("сюда не доходим: цикл либо вернул значение, либо бросил ошибку")
}
```

Проследим по шагам при трёх попытках:

1. Попытка 1 упала → ждём 1 с (+ случайные 0–0,3 с) → `delay` становится 2.
2. Попытка 2 упала → ждём 2 с (+ 0–0,3 с) → `delay` становится 4.
3. Попытка 3 упала → это последняя, бросаем ошибку наружу, показываем error state с кнопкой.

Всего пауз две, суммарно около 3 секунд. Если бы попыток было пять, паузы были бы 1, 2, 4, 8 — каждая вдвое длиннее предыдущей, вместе 15 секунд.

Jitter («дрожание») — случайная добавка к паузе. Представь: сервер упал, у десяти тысяч пользователей одновременно отвалился запрос. Без разброса все десять тысяч повторят ровно через 1 с, потом ровно через 2 с — синхронными волнами, которые мешают серверу подняться. Разброс в 0–0,3 с размазывает волну.

`Task.sleep(nanoseconds:)` — пауза в наносекундах (1 секунда = 1 000 000 000). Вариант `Task.sleep(for: .seconds(1))` удобнее, но доступен только с iOS 16, а книга поддерживает iOS 15.

`fatalError` в конце нужен только компилятору: он не может доказать, что цикл всегда либо вернёт значение, либо бросит ошибку.

Частые ошибки.

- **Технический текст.** «The operation couldn’t be completed. (NSURLErrorDomain error −1009.)» пользователю ничего не говорит.
- **Нет кнопки повтора.** Пользователь видит «ошибка» — и всё, остаётся только закрыть приложение.
- **Автоповтор без пауз.** Сервер ответил 500 → клиент тут же повторяет → 500 → повторяет. Тысячи клиентов в таком цикле сами создают нагрузку, которая не даёт серверу подняться. Растущая пауза и потолок попыток обязательны.

- **Повтор того, что нельзя повторять.** Запрос «списать деньги» повторять автоматически нельзя без защиты на сервере (ключ идемпотентности): первый запрос мог дойти, а упал только ответ.

24.4 Offline баннер — top-of-screen

Когда применять. Связи нет, но есть сохранённые данные (**кеш**). Экран работает, но пользователь должен знать, что видит не самое свежее.

Минимальный код.

```
final class OfflineBannerView: UIView {
    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .systemOrange
        let label = UILabel()
        label.text = "Нет подключения — показываем сохранённое"
        label.font = .preferredFont(forTextStyle: .footnote)
        label.adjustsFontForContentSizeCategory = true
        label.numberOfLines = 0
        label.textColor = .black
        let icon = UIImageView(image: UIImage(systemName: "wifi.slash"))
        icon.tintColor = .black
        icon.setContentHuggingPriority(.required, for: .horizontal)
        let stack = UIStackView(arrangedSubviews: [icon, label])
        stack.axis = .horizontal
        stack.spacing = 8
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        addSubview(stack)
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: topAnchor, constant: 8),
            stack.bottomAnchor.constraint(equalTo: bottomAnchor, constant: -8),
            stack.centerXAnchor.constraint(equalTo: centerXAnchor),
            stack.leadingAnchor.constraint(greaterThanOrEqualTo: leadingAnchor, constant:
↪  16),
        ])
        isUserInteractionEnabled = false     // тапы проходят сквозь баннер
        isAccessibilityElement = true
        accessibilityLabel = label.text
    }

    required init?(coder: NSCoder) { fatalError("Создаём только из кода") }
}
```

Ранняя версия объявляла только `init()` без `required init?(coder:)`. Такой класс **не собирается**. Правило Swift: как только подкласс объявляет свой назначенный (designated) инициализатор, он перестаёт наследовать инициализаторы родителя, а `init?(coder:)` у `UIView` помечен `required` (им UIKit создаёт view из storyboard и xib, протокол `NSCoding`) — поэтому его приходит-

ся написать явно, даже если он только падает. Назначенных инициализаторов у `UIView` два, `init(frame:)` и `init?(coder:)` (глава 0, раздел 0.3). Теперь — стандартная пара `init(frame:)` + `init?(coder:)`; создаётся как `OfflineBannerView()` (у `UIView` есть удобный `init()`, который зовёт `init(frame: .zero)`).

Цвет — оранжевый, а не красный: «нет сети» — предупреждение, а не авария; красный оставим для ошибок. Чёрный текст на `systemOrange` контрастнее белого.

`setContentHuggingPriority(.required, for: .horizontal)` — **hugging** («обнимание») — насколько view сопротивляется растягиванию шире своего естественного размера (`intrinsicContentSize`; у `UIImageView` это размер картинки). По умолчанию hugging у view равен 250, и у иконки, и у подписи (замер), так что стек не знал бы, кого растягивать. Приоритет 1000 у иконки означает: «растягивать нельзя, лишнее место отдайте тексту» (глава 0, раздел 0.3).

Стек по центру, но не ближе 16 точек к краю (`greaterThanOrEqualTo`): короткий текст по центру, длинный переносится.

`isUserInteractionEnabled = false` — баннер ничего не делает при тапе, пусть касания проходят к тому, что под ним: hit-testing пропускает такую view и идёт дальше вниз по `subviews` (глава 21, раздел 21.4).

`isAccessibilityElement = true` + `accessibilityLabel` — VoiceOver читает баннер одной фразой, а не «Wi-Fi, перечёркнуто» + текст по отдельности.

Баннер прикрепляют к `safeAreaLayoutGuide.topAnchor` экрана и показывают или прячут через `isHidden` или анимацию `alpha`. Узнать о сети помогает `NWPathMonitor` из фреймворка Network:

```
import Network

final class ConnectivityWatcher {
    private let monitor = NWPathMonitor()
    private let queue = DispatchQueue(label: "connectivity")

    /// Вызывается на главном потоке: true — сеть есть.
    var onChange: ((Bool) -> Void)?

    func start() {
        monitor.pathUpdateHandler = { [weak self] path in
            let isOnline = path.status == .satisfied
            // Обработчик приходит на фоновой очереди `queue` — прыгаем на главный.
            Task { @MainActor in
                self?.onChange?(isOnline)
            }
        }
        monitor.start(queue: queue)
    }

    func stop() {
        monitor.cancel()
    }
}
```

`NWPathMonitor` сообщает о каждом изменении «пути» в сеть: Wi-Fi появился, сотовая связь пропала. `monitor.start(queue:)` — обработчик будет вызываться на **нашей фоновой очереди**, а не на главном потоке. Трогать UI оттуда нельзя, поэтому внутри — `Task { @MainActor in … }`: «выполни это на главном потоке». В SDK обработчик объявлен как `@Sendable` — Swift 6 сам не даст забыть про переход между потоками.

`path.status == .satisfied` — «путь в сеть есть». Возможные значения: `.satisfied` (есть), `.unsatisfied` (нет), `.requiresConnection` (есть, но соединение надо поднять — например, VPN по запросу).

Честная оговорка: `.satisfied` означает «есть подключение к сети», а не «интернет работает». Wi-Fi в кафе без авторизации даст `.satisfied`, а запросы при этом будут падать. Поэтому баннер «нет сети» — подсказка, а окончательное слово за ошибкой реального запроса (24.3).

Частые ошибки.

- **Баннер не пропадает.** Показали при `.unsatisfied` и забыли спрятать при `.satisfied`.
- **UI из фонового обработчика.** `pathUpdateHandler` приходит не на главном потоке; прямой вызов `banner.isHidden = false` оттуда — гонка данных.
- **Баннер закрывает навигацию.** Если он лежит поверх навигационной панели и принимает касания, кнопка «Назад» перестаёт работать. Наш баннер касания не принимает — и всё равно лучше класть его под панелью.

24.5 Skeleton vs spinner vs blank — что когда

Сценарий	Что показать
Первое открытие, кеша нет	Skeleton (глава 23.3)
Повторное открытие, кеш есть	Кеш сразу + тихое обновление
Pull-to-refresh	Индикатор `UIRefreshControl`
Кнопка-действие (вход, отправка)	Спиннер в кнопке (23.4)
Долгая блокирующая операция	Overlay с подписью (23.7)
Загрузка закончилась, данных нет	EmptyStateView (24.1)
Запрос упал	Error state + «Повторить»
Нет сети, но есть кеш	Баннер + кеш (24.4)
Нет сети и нет кеша	Error state «Нет интернета»

Главное правило таблицы: состояние экрана — это **одно** значение. Удобно завести `enum` и рисовать экран по нему:

```
enum ScreenState<Value> {
    case loading
    case loaded(Value)
    case empty
    case failed(String)
}
```

Экран в каждый момент в ровно одном состоянии. Так не бывает «спиннер крутится поверх сообщения об ошибке» — классического бага, когда состояние разнесено по трём булевым флагам.

Упражнение 24.2. Напиши функцию `render(_ state: ScreenState<[String]>)` для экрана со списком, `EmptyStateView` и error state. Что она делает в каждом из четырёх случаев? Ответ — в конце главы.

24.6 Empty state для search

Особый случай — пустой результат **поиска**:

```
emptyView.configure(iconName: "magnifyingglass",
                    title: "Не нашлось «\(query)»",
                    body: "Попробуй изменить запрос или убрать фильтры.")
```

Покажи **сам запрос** в заголовке — пользователь видит, что искал, и замечает опечатку («ах, “кросовки” через одну “с”»). Строковая интерполяция `\(query)` вставляет текст запроса внутрь кавычек-«ёлочек».

Длинный запрос не сломает вёрстку: у заголовка `numberOfLines = 0`, он перенесётся.

24.7 iOS 17+: UIContentUnavailableConfiguration

В iOS 17 Apple добавила готовый пустой экран: `UIContentUnavailableConfiguration` и свойство `contentUnavailableConfiguration` у `UIViewController`. Система сама рисует иконку, заголовок, пояснение и кнопку в стиле iOS и сама центрирует. Есть готовые заготовки `.empty()`, `.loading()` и `.search()`.

Книга поддерживает iOS 15, поэтому код с проверкой версии:

```
final class ModernEmptyViewController: UIViewController {
    private var items: [String] = []

    override func viewDidLoad() {
        super.viewDidLoad()
        updateEmptyState()
    }

    private func updateEmptyState() {
        if #available(iOS 17.0, *) {
            if items.isEmpty {
                var config = UIContentUnavailableConfiguration.empty()
                config.image = UIImage(systemName: "tray")
                config.text = "Пока пусто"
                config.secondaryText = "Добавь первую задачу — нажми «плюс» внизу."
                contentUnavailableConfiguration = config
            } else {
                contentUnavailableConfiguration = nil
```

```
            }
        } else {
            // iOS 15–16: наш EmptyStateView из 24.1.
        }
    }
}
```

`if #available(iOS 17.0, *)` — ветка выполнится только на iOS 17 и новее; на iOS 15–16 работает `else` с нашим компонентом. Без проверки проект с deployment target 15.0 не соберётся: компилятор знает, что API нет на старых системах. `nil` в `contentUnavailableConfiguration` убирает пустой экран, когда данные появились.

Ответы к упражнениям

Упражнение 24.1. Два раза: на кнопке висят два обработчика, оба срабатывают на одно нажатие. С `removeAction(identifiedBy:)` и именованным действием — ровно один.

Упражнение 24.2. Например:

```
final class ListScreen: UIViewController {
    private let tableView = UITableView()
    private let emptyView = EmptyStateView()
    private let spinner = UIActivityIndicatorView(style: .large)
    private var errorView: UIView?
    private var rows: [String] = []

    func render(_ state: ScreenState<[String]>) {
        errorView?.removeFromSuperview()
        errorView = nil
        spinner.stopAnimating()
        tableView.isHidden = true
        emptyView.isHidden = true

        switch state {
        case .loading:
            spinner.startAnimating()
        case .loaded(let items):
            rows = items
            tableView.isHidden = false
            tableView.reloadData()
        case .empty:
            emptyView.configure(iconName: "tray", title: "Пока пусто",
                                body: "Добавь первую задачу — нажми «плюс» внизу.")
            emptyView.isHidden = false
        case .failed(let message):
            let error = makeErrorState { [weak self] in self?.render(.loading) }
            view.addSubview(error)
            errorView = error
```

```
            _ = message   // покажи message в заголовке ошибки
        }
    }
}
```

Сначала всё прячем, потом показываем ровно то, что нужно состоянию. Поэтому два состояния одновременно на экране не окажутся.

Что мы выучили

- **EmptyStateView** — иконка + заголовок + пояснение + необязательная кнопка; Dynamic Type, `numberOfLines = 0`, иконка скрыта от VoiceOver.
- Разные пустые состояния — разные тексты: первый запуск, всё выполнено, ничего не нашлось (с самим запросом в заголовке).
- **Error state** — человеческий текст по `URLError.code`, кнопка «Повторить».
- **Exponential backoff** — паузы 1, 2, 4… секунды с небольшим случайным разбросом и потолок попыток.
- **Offline-баннер** — `NWPathMonitor`, обработчик на фоновой очереди → `Task { @MainActor in }`; `.satisfied` не гарантирует работающий интернет.
- Наследник `UIView` со своим инициализатором обязан иметь `init?(coder:)`: подкласс с designated-инициализатором не наследует родительские, а `init?(coder:)` помечен `required`.
- Скрытый arranged subview не занимает места в стеке (ни сам, ни его отступ), но из `arrangedSubviews` не уходит.
- Состояние экрана — один `enum`, а не набор флагов.
- iOS 17+: `UIContentUnavailableConfiguration` под `#available`.

Apple Developer Documentation

- `UIContentUnavailableConfiguration` — системный пустой/загрузочный/поисковый экран (iOS 17+).
- `contentUnavailableConfiguration` — свойство контроллера для такого экрана (iOS 17+).
- `UIImage.SymbolConfiguration` — размер и вес SF Symbols.
- `URLError` — коды сетевых ошибок (`notConnectedToInternet`, `timedOut`…).
- `NWPathMonitor` — наблюдение за доступностью сети.
- `NWPath.Status` — `satisfied`, `unsatisfied`, `requiresConnection`.
- HIG — Loading — Apple о загрузке и переходных состояниях.
- HIG — Onboarding — первый запуск и подсказки в пустых экранах.

→ Глава 25. Cookbook — поиск и фильтры

Глава 25. Cookbook — поиск и фильтры

Пока список помещается на пару экранов, его листают. Когда строк становится сотни, человек перестаёт листать и начинает искать: вводит пару букв, сужает выдачу фильтром, меняет порядок сортировки. Эта глава собирает рецепты для всего этого: системная строка поиска, пауза перед поиском, области поиска, недавние запросы, фильтры-«таблетки», сортировка, поиск на сервере и подсветка найденного.

Как и вся часть IV, глава — справочник. Каждый рецепт устроен одинаково: когда применять, минимальный код, частые ошибки, альтернативы. Читать подряд не обязательно.

> **В каком режиме код.** Все листинги главы проверены компилятором в режиме Swift 6 с настройкой Default Actor Isolation = MainActor (так настроен учебный проект, см. Введение). В этом режиме каждый тип и функция без явных пометок принадлежат **главному актору** (main actor) — то есть выполняются на главном потоке, там же, где работает весь UIKit. Поэтому в коде ниже нет `DispatchQueue.main.async` вокруг обновления таблицы: мы и так на главном потоке.

Заготовка: список товаров

Все рецепты главы примеряем к одному экрану — списку товаров магазина. Вот его минимальная версия без поиска:

```
struct Product {
    let title: String
    let brand: String
    let price: Int          // цена в тенге, целым числом
}

final class ProductsViewController: UITableViewController {
    var allItems: [Product] = [
        Product(title: "Кофемашина Café Crème", brand: "Delonghi", price: 189_000),
        Product(title: "Кофемолка ручная", brand: "Hario", price: 22_500),
        Product(title: "Чайник электрический", brand: "Xiaomi", price: 14_990),
    ]
    var filtered: [Product] = []
    let searchController = UISearchController(searchResultsController: nil)
```

```
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Товары"
        tableView.register(UITableViewCell.self, forCellReuseIdentifier: "cell")
        filtered = allItems
        setupSearch()                       // напишем в 25.1
    }

    override func tableView(_ tableView: UITableView,
                            numberOfRowsInSection section: Int) -> Int {
        filtered.count
    }

    override func tableView(_ tableView: UITableView,
                            cellForRowAt indexPath: IndexPath) -> UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier: "cell", for: indexPath)
        let product = filtered[indexPath.row]
        var content = cell.defaultContentConfiguration()
        content.text = product.title
        content.secondaryText = product.brand
        cell.contentConfiguration = content
        return cell
    }
}
```

Главное здесь — **два массива**. `allItems` хранит все товары и никогда не меняется от поиска: это источник правды. `filtered` — то, что таблица показывает прямо сейчас. Поиск и фильтры только пересчитывают `filtered` из `allItems`, а таблица всегда читает `filtered`. Если держать один массив и выкидывать из него «лишнее», то после стирания запроса вернуть выкинутое будет неоткуда.

`dequeueReusableCell` берёт ячейку из очереди **переиспользования**: таблица не создаёт новую ячейку на каждую строку, а берёт ту, что только что уехала за край экрана, и перенастраивает её (подробно — глава 12). `defaultContentConfiguration()` — готовый набор «текст + подпись + картинка» для стандартной ячейки, о нём глава 27.

25.1 UISearchController

Когда применять. Любой список, в котором человек может что-то искать: контакты, заметки, товары, настройки.

`UISearchController` — готовый системный «поисковик»: строка ввода, кнопка «Отменить», анимация появления, затемнение фона. Тебе остаётся сказать ему, **кого звать** при каждом изменении текста, и отфильтровать список. Сам класс есть с iOS 8, а встраивать его прямо в навигационную панель через `navigationItem.searchController` можно с iOS 11.

Кто здесь кто. `UISearchController → UIViewController` — это view controller, а не view: своей рамки на экране у него нет, он управляет показом результатов и владеет строкой поиска `searchBar`. Сама строка — `UISearchBar → UIView`: view, но **не** `UIControl`, поэтому у неё нет `addAction(_:for:)`

и target-action, а о событиях она сообщает только через `UISearchBarDelegate`. Поле ввода внутри строки — `searchBar.searchTextField` (iOS 13+), класс `UISearchTextField → UITextField → UIControl`: вот у него `addAction` уже есть. Лестница классов целиком — глава 0, разделы 0.4 и 0.6.

Минимальный код.

```
extension ProductsViewController: UISearchResultsUpdating {
    func setupSearch() {
        searchController.searchResultsUpdater = self
        searchController.obscuresBackgroundDuringPresentation = false
        searchController.searchBar.placeholder = "Поиск товаров"
        navigationItem.searchController = searchController
        navigationItem.hidesSearchBarWhenScrolling = false
        definesPresentationContext = true
    }

    func updateSearchResults(for searchController: UISearchController) {
        let query = (searchController.searchBar.text ?? "")
            .trimmingCharacters(in: .whitespacesAndNewlines)
        filtered = query.isEmpty
            ? allItems
            : allItems.filter { $0.title.localizedStandardContains(query) }
        tableView.reloadData()
    }
}
```

Разберём по строкам.

`searchResultsUpdater = self` — назначаем экран «обновлятелем результатов». Это обычный **делегат**: объект, которому другой объект поручает часть работы. Похоже на секретаря, который звонит тебе каждый раз, когда приходит письмо: секретарь (`UISearchController`) сам следит за строкой, а решение «что показать» принимаешь ты в `updateSearchResults(for:)`. Документация называет два случая, когда UIKit его зовёт: строка поиска стала first responder (получила фокус и клавиатуру) и в строке изменился текст. Ссылка `searchResultsUpdater` слабая (weak), как и у всякого делегата: контроллер не удерживает твой экран, а объект-«обновлятель», созданный на лету и никем не удержанный, исчез бы сразу.

`UISearchController(searchResultsController: nil)` (в заготовке) — `nil` значит «отдельного экрана результатов нет, фильтруй в той же таблице». Если передать сюда другой view controller, поиск будет показывать результаты в нём, поверх списка. Так делают, когда результаты выглядят иначе, чем исходный список (например, в результатах смешаны товары и категории).

`obscuresBackgroundDuringPresentation = false` — не затемнять содержимое под строкой поиска, пока она активна. Значение по умолчанию зависит от того, с каким SDK собрано приложение: заголовок `UISearchController.h` говорит «default is NO for apps linked on iOS 15.0 and later, YES otherwise», и на iOS 26.5 свежий контроллер действительно отдаёт `false`. Для нашего проекта строка лишь закрепляет намерение, но она не лишняя: включённое затемнение нужно, когда результаты показывает отдельный `searchResultsController`, а у нас результат — та же таблица, и затемнение сделало бы её недоступной: тап по строке просто закрывал бы поиск.

`navigationItem.searchController = searchController` — встраиваем строку в навигационную панель, под заголовок. `navigationItem` — это объект класса `UINavigationItem` (потомок `NSObject`, не view), и он принадлежит **view controller'у**: панель лишь показывает табличку того экрана, который сейчас сверху (глава 26 и глава 0, раздел 0.6). Экран должен лежать внутри `UINavigationController`, иначе панели нет и строка не появится.

`hidesSearchBarWhenScrolling = false` — строка видна всегда. По умолчанию (`true`) iOS прячет её, когда список прокручен вниз, и показывает, если потянуть список от верха.

`definesPresentationContext = true` — говорит: «поиск живёт в границах этого экрана». `UISearchController` показывает себя собственной презентацией (`modalPresentationStyle` у него `.custom`), но накрывает он не всё окно, а **текущий контекст** (presentation context) — ту же область, что и стили `.currentContext`/`.overCurrentContext`. UIKit ищет, чей view накрыть: идёт от показывающего экрана вверх по иерархии view controller'ов до первого, у кого `definesPresentationContext == true`. У обычного `UIViewController` свойство по умолчанию `false`, а у `UINavigationController` — уже `true`, поэтому без этой строки контекстом станет весь контроллер навигации, и если во время поиска открыть карточку товара, строка поиска может остаться висеть поверх нового экрана.

Теперь фильтр. `trimmingCharacters(in: .whitespacesAndNewlines)` срезает пробелы по краям: запрос `"  кофе"` с случайным пробелом в начале иначе ничего бы не нашёл. `localizedStandardContains` ищет подстроку **без учёта регистра и диакритики** и с учётом языка устройства — так ищет Finder. Сравни:

- `"Кофемашина Café Crème".lowercased().contains("кофе")` → `true`;
- `"Кофемашина Café Crème".lowercased().contains("cafe")` → `false`, потому что é и e — разные символы;
- `"Кофемашина Café Crème".localizedStandardContains("cafe")` → `true`.

Частые ошибки.

- **Индекс из `filtered`, данные из `allItems`.** В `didSelectRowAt` пишут `allItems[indexPath.row]` — и при активном поиске открывается не тот товар. Таблица показывает `filtered`, значит и читать надо `filtered[indexPath.row]`.
- **Включили `obscuresBackgroundDuringPresentation = true`** при `searchResultsController: nil` (или проект собран со старым SDK, где это значение по умолчанию) — таблица под затемнением, строки не нажимаются.
- **Поиск с учётом регистра и диакритики.** `contains` ищет побайтово-точно: `"Apple"` не находит `"apple"`, `"cafe"` не находит `"café"`. Бери `localizedStandardContains` или `range(of:options: [.caseInsensitive, .diacriticInsensitive])`.
- **Тяжёлая фильтрация на каждое нажатие.** На паре сотен строк это незаметно. На десятках тысяч или при запросе к серверу — нужна пауза перед поиском, рецепт 25.2.

Альтернативы. Отдельный экран поиска (`searchResultsController` не `nil`) — когда результаты устроены иначе, чем список. Голый `UISearchBar` без контроллера — когда строка поиска должна стоять где-то внутри экрана, а не в навигационной панели; тогда показ, скрытие и кнопку «Отменить» делаешь сам, а о вводе узнаёшь из `UISearchBarDelegate` (`searchBar(_:textDidChange:)`), потому что target-action у строки нет. Ещё один живой пример поиска — глава 13 (Notes).

25.2 Debounced search

Когда применять. Поиск дорогой: большой объём данных, сложное сравнение или запрос к серверу.

Debounce (по-русски иногда говорят «антидребезг») — приём «подожди, пока поток событий затихнет, и только тогда действуй». Бытовая аналогия — лифт: он не уезжает, пока люди заходят, и каждый новый вошедший продлевает ожидание. Лифт трогается, только когда в дверях никого нет пару секунд. У нас «вошедший» — нажатие клавиши, а «поехать» — выполнить поиск.

Рядом живёт похожий приём — **throttle** («не чаще, чем раз в N секунд»): он пропускает первое событие сразу, а остальные — не чаще заданного интервала. Для поиска по вводу обычно нужен именно debounce: нам важен запрос, который человек допечатал, а не промежуточные.

```
final class DebouncedSearchViewController: UITableViewController,
                                           UISearchResultsUpdating {
    private var searchWorkItem: DispatchWorkItem?

    func updateSearchResults(for searchController: UISearchController) {
        let query = searchController.searchBar.text ?? ""
        searchWorkItem?.cancel()
        let work = DispatchWorkItem { [weak self] in
            self?.performSearch(query: query)
        }
        searchWorkItem = work
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.2, execute: work)
    }

    private func performSearch(query: String) {
        print("Ищем:", query)
        // фильтруем allItems или идём в сеть
    }
}
```

`DispatchWorkItem` — «задание в конверте»: кусок кода, который можно отдать очереди на выполнение и **отменить**, пока его не начали.

Как это работает, на числах. Пауза — `0.2` секунды, пятая доля секунды. Человек набирает «коф»:

Время	Событие	Что происходит
0,00 с	нажал «к»	задание «искать “к”» назначено на 0,20 с
0,12 с	нажал «о»	задание «к» отменено, «ко» назначено на 0,32 с
0,25 с	нажал «ф»	задание «ко» отменено, «коф» назначено на 0,45 с
0,45 с	тишина	выполняется поиск «коф» — **один** раз

Три нажатия — один поиск. Если между нажатиями дольше 0,2 с, поиск успевает выполниться и для промежуточного запроса, это нормально.

Строки, которые стоит разобрать:

- `searchWorkItem?.cancel()` — отменяем предыдущее задание. `cancel()` лишь помечает задание: если очередь до него ещё не дошла, она его пропустит. Уже начавшееся задание `cancel()` не прерывает.
- `let query = ...` снаружи замыкания — мы фиксируем текст **на момент нажатия**. Замыкание захватит именно это значение.
- `[weak self]` — задание живёт в очереди до 0,2 с. Если за это время экран закрыли, слабая ссылка не даст заданию удержать экран в памяти, а `self?.` просто ничего не сделает.
- `DispatchQueue.main.asyncAfter` — выполнить на главной очереди через заданное время. Главная — потому что `performSearch` потом обновляет таблицу.

Сколько ждать: 0,2–0,3 с для локального поиска, 0,3–0,5 с для сетевого. Меньше — поиск будет срабатывать посреди слова. Больше — выдача начнёт ощутимо «запаздывать» за вводом.

Частые ошибки.

- **Debounce не ускоряет сам поиск.** Он уменьшает **число** поисков, но каждый поиск по-прежнему выполняется на главном потоке. Если один проход по массиву занимает 100 мс, интерфейс всё так же подвиснет на 0,1 с после паузы. Лечится ускорением самого поиска (заранее приготовленные строки в нижнем регистре, индекс) или выносом его с главного потока.
- **Забыли `cancel()`** — тогда каждое нажатие запускает свой поиск, просто с опозданием на 0,2 с. Это уже не debounce, а задержка.

Альтернативы. То же самое через `Task` и `Task.sleep` — рецепт 25.7. Для поиска по сети этот вариант удобнее: отмена задачи автоматически отменяет и сетевой запрос.

Упражнение 25.1. Подключи `DebouncedSearchViewController` к `UISearchController` (как в 25.1) и быстро набери «кофе». Сколько раз напечатается «Ищем:»? Потом поставь паузу `1.0` и снова набери слово. Что изменилось для пользователя?

25.3 Scope buttons

Когда применять. Поиск по разным «полям»: контакты — все / по имени / по телефону; товары — везде / в названии / в бренде.

Scope bar (панель областей поиска) — ряд сегментов под строкой поиска. Выбранный сегмент задаёт, **где** искать.

```
enum SearchScope: Int, CaseIterable {
    case all, title, brand

    var buttonTitle: String {
        switch self {
        case .all:   return "Везде"
        case .title: return "Название"
        case .brand: return "Бренд"
        }
    }

    func matches(_ product: Product, query: String) -> Bool {
        switch self {
```

```
        case .all:
            return product.title.localizedStandardContains(query)
                || product.brand.localizedStandardContains(query)
        case .title:
            return product.title.localizedStandardContains(query)
        case .brand:
            return product.brand.localizedStandardContains(query)
        }
    }
}
```

Сначала — перечисление областей. `rawValue` типа `Int` совпадает с индексом сегмента: 0 — «Везде», 1 — «Название», 2 — «Бренд». Так индекс из `UISearchBar` превращается в область одной строкой. Логику «подходит ли товар» кладём прямо в перечисление: одна область — одно правило.

```
extension ProductsViewController: UISearchBarDelegate {
    func setupScopes() {
        searchController.searchBar.scopeButtonTitles =
            SearchScope.allCases.map(\.buttonTitle)
        searchController.searchBar.delegate = self
    }

    func searchBar(_ searchBar: UISearchBar,
                   selectedScopeButtonIndexDidChange selectedScope: Int) {
        applyFilter()
    }

    func applyFilter() {
        let bar = searchController.searchBar
        let query = (bar.text ?? "").trimmingCharacters(in: .whitespacesAndNewlines)
        let scope = SearchScope(rawValue: bar.selectedScopeButtonIndex) ?? .all
        filtered = query.isEmpty
            ? allItems
            : allItems.filter { scope.matches($0, query: query) }
        tableView.reloadData()
    }
}
```

`scopeButtonTitles` — подписи сегментов. Пока их нет, панели нет; по документации строка не покажет панель и при одном-единственном заголовке — нужно хотя бы два.

По умолчанию `UISearchController` сам показывает панель областей, когда строка поиска активна, и прячет, когда поиск закрыт (`automaticallyShowsScopeBar == true`). Если написать `searchBar.showsScopeBar = true`, контроллер переключится в ручной режим — документация прямо говорит, что установка `showsScopeBar` сбрасывает `automaticallyShowsScopeBar` в `false` (проверено на iOS 26.5): панель будет видна всегда, и прятать её придётся самому. С iOS 16 тот же переключатель называется `scopeBarActivation` (`.automatic` / `.manual` / `.onTextEntry` / `.onSearchActivation`).

`searchBar.delegate = self` — делегат строки поиска сообщает о нажатиях: смена сегмента, кнопка «Найти» на клавиатуре, «Отменить». `UISearchController` не занимает это свойство (у строки внутри свежего контроллера `delegate == nil`), так что экран может быть делегатом строки, а контроллер продолжит работать.

`selectedScopeButtonIndexDidChange` — смена сегмента. Текст не менялся, поэтому пересчитываем выдачу сами через `applyFilter()`. Из `updateSearchResults(for:)` тоже зови `applyFilter()` — тогда правило фильтрации живёт в одном месте.

Частые ошибки.

- **Больше трёх-четырёх сегментов.** На узком iPhone подписи обрежутся до «Наз…». Если областей много — выбирай область фильтром (25.5) или меню.
- **Сегмент поменяли, а выдача старая.** Забыли пересчитать фильтр в `selectedScopeButtonIndexDidChan`

25.4 Recent searches

Когда применять. Сложный поиск, где люди часто повторяют запросы: магазин, карты, справочник.

```
extension ProductsViewController {
    private static let recentKey = "recentSearches"

    var recentSearches: [String] {
        get { UserDefaults.standard.stringArray(forKey: Self.recentKey) ?? [] }
        set { UserDefaults.standard.set(Array(newValue.prefix(10)),
                                        forKey: Self.recentKey) }
    }

    func saveSearch(_ query: String) {
        let trimmed = query.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else { return }
        var list = recentSearches
        list.removeAll { $0.caseInsensitiveCompare(trimmed) == .orderedSame }
        list.insert(trimmed, at: 0)
        recentSearches = list
    }
}
```

`UserDefaults` — маленькое хранилище «ключ — значение» внутри приложения. Оно рассчитано на настройки и короткие списки; десяток строк истории — как раз его масштаб. Хранить там большие данные, пароли и токены нельзя: для секретов есть Keychain (глава 8).

`recentSearches` — вычисляемое свойство: читает и пишет прямо в `UserDefaults`. Сеттер отрезает список до 10 элементов: `prefix(10)` берёт первые десять, остальные пропадают. Новые запросы вставляются в начало, значит отрезаются самые старые.

`saveSearch` делает три вещи. Срезает пробелы и отбрасывает пустой запрос. Удаляет из истории **такой же** запрос без учёта регистра: `caseInsensitiveCompare` вернёт `.orderedSame` для «Кофе» и «кофе» — иначе история заполнится дублями. И вставляет запрос первым.

Когда сохранять. Не на каждое нажатие — иначе в истории окажутся «к», «ко», «коф». Сохраняй, когда запрос «состоялся»: человек нажал «Найти» на клавиатуре или открыл товар из результатов.

```
extension ProductsViewController {
    var showsRecent: Bool {
        searchController.isActive
            && (searchController.searchBar.text ?? "").isEmpty
            && !recentSearches.isEmpty
    }

    func searchBarSearchButtonClicked(_ searchBar: UISearchBar) {
        saveSearch(searchBar.text ?? "")
    }

    func didSelectRecent(at row: Int) {
        let query = recentSearches[row]
        searchController.searchBar.text = query
        applyFilter()
    }
}
```

`showsRecent` — «показываем историю, а не товары»: строка поиска активна, текст пуст, история не пуста. В `numberOfRowsInSection` и `cellForRowAt` проверяй этот флаг и отдавай либо `recentSearches.count` и текст запроса, либо товары. В `didSelectRowAt` при `showsRecent` зови `didSelectRecent(at:)`: он подставит запрос в строку и пересчитает выдачу.

`searchBarSearchButtonClicked` — метод `UISearchBarDelegate`, кнопка «Найти» на клавиатуре. Работает, потому что в 25.3 мы уже назначили экран делегатом строки.

Частые ошибки.

- **Нет кнопки «Очистить историю».** История поиска — личные данные. Дай способ её стереть: `recentSearches = []`.
- **История не обновляется на экране.** После `saveSearch` таблицу надо перезагрузить, если история сейчас видна.

25.5 Filter chips

Когда применять. Несколько фильтров одновременно: категория + цена + бренд. **Chips** («таблетки», «чипсы») — маленькие скруглённые кнопки в горизонтальную строку над списком. У каждой два состояния: фильтр не задан («Бренд» со стрелкой вниз) и задан («Hario» с крестиком).

Строку таблеток удобно сделать горизонтальным `UICollectionView`. Разметка:

```
func makeChipsLayout() -> UICollectionViewFlowLayout {
    let layout = UICollectionViewFlowLayout()
    layout.scrollDirection = .horizontal
    layout.estimatedItemSize = UICollectionViewFlowLayout.automaticSize
```

```
    layout.minimumLineSpacing = 8
    layout.sectionInset = UIEdgeInsets(top: 0, left: 16, bottom: 0, right: 16)
    return layout
}
```

`scrollDirection = .horizontal` — ячейки идут в ряд и листаются вбок.

`estimatedItemSize = automaticSize` — включает **самоподбор размера**: ширину каждой таблетки считает Auto Layout по её содержимому. «Все» получится узкой, «Кофемашины» — широкой.

`minimumLineSpacing = 8` — отступ **между таблетками**. Тут легко ошибиться. `UICollectionViewFlowLayout` раскладывает ячейки «линиями». При вертикальной прокрутке линия — это строка, а при горизонтальной — **столбец**. Наши таблетки стоят в одну строку, то есть каждая — отдельный столбец, и расстояние между ними задаёт `minimumLineSpacing`. А `minimumInteritemSpacing` при горизонтальной прокрутке — расстояние между ячейками **внутри одного столбца**, по вертикали; в строке таблеток оно ни на что не влияет.

`sectionInset` — поля по краям: первая таблетка начнётся в 16 точках от левого края экрана, как и текст в ячейках таблицы.

Сама таблетка — ячейка коллекции. `UICollectionViewCell` → `UICollectionReusableView` → `UIView`: это view, которую коллекция переиспользует, а своё содержимое кладут в её `contentView` (глава 0, раздел 0.5).

```
final class ChipCell: UICollectionViewCell {
    static let reuseID = "ChipCell"
    private let titleLabel = UILabel()
    private let iconView = UIImageView()

    override init(frame: CGRect) {
        super.init(frame: frame)
        contentView.layer.cornerRadius = 16
        titleLabel.font = .preferredFont(forTextStyle: .subheadline)
        titleLabel.adjustsFontForContentSizeCategory = true
        iconView.preferredSymbolConfiguration = UIImage.SymbolConfiguration(scale: .small)

        let stack = UIStackView(arrangedSubviews: [titleLabel, iconView])
        stack.spacing = 4
        stack.alignment = .center
        stack.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: contentView.topAnchor, constant: 6),
           stack.bottomAnchor.constraint(equalTo: contentView.bottomAnchor, constant: -6),
            stack.leadingAnchor.constraint(equalTo: contentView.leadingAnchor, constant:
↪  12),
            stack.trailingAnchor.constraint(equalTo: contentView.trailingAnchor, constant:
↪  -10),
        ])
        isAccessibilityElement = true
```

```
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func configure(title: String, isActive: Bool) {
        titleLabel.text = title
        iconView.image = UIImage(systemName: isActive ? "xmark.circle.fill" :
↪   "chevron.down")
        contentView.backgroundColor = isActive ? .systemBlue : .secondarySystemFill
        titleLabel.textColor = isActive ? .white : .label
        iconView.tintColor = titleLabel.textColor
        accessibilityLabel = title
        accessibilityTraits = isActive ? [.button, .selected] : .button
    }
}
```

Разбор:

- Стек «текст + иконка» прибит к `contentView` со всех четырёх сторон. Это обязательное условие самоподбора размера: Auto Layout должен по цепочке ограничений «дотянуться» от краёв ячейки до содержимого, иначе ему не из чего вычислить ширину.
- Высота на числах: у шрифта `subheadline` при стандартном размере текста строка около 20 точек, плюс 6 сверху и 6 снизу — таблетка высотой около 32 точек. `cornerRadius = 16` — половина высоты, края получаются полукруглыми.
- `preferredFont(forTextStyle:)` и `adjustsFontForContentSizeCategory` — поддержка **Dynamic Type**: если человек в настройках увеличил шрифт, таблетка вырастет вместе с текстом.
- `configure` — единственное место, где задаётся внешний вид. Ячейки переиспользуются, поэтому каждое свойство, которое зависит от состояния (текст, иконка, цвет), выставляется **в обе стороны**: и для активной, и для неактивной. Иначе переиспользованная ячейка унесёт синий фон из прошлой жизни.
- `accessibilityTraits` с `.selected` — VoiceOver (экранный диктор для незрячих) прочитает «Hario, выбрано, кнопка». Без этого разница между активной и неактивной таблеткой видна только глазами.

Поведение по тапу:

```
struct FilterChip {
    let name: String            // «Бренд»
    var value: String?          // «Hario» или nil, если фильтр не задан

    var displayTitle: String { value ?? name }
}

final class ChipsController: NSObject, UICollectionViewDelegate {
    var chips: [FilterChip] = [FilterChip(name: "Бренд"), FilterChip(name: "Цена")]
    var onPick: ((Int) -> Void)?        // показать выбор значения
    var onChange: (() -> Void)?         // пересчитать выдачу
```

```
    func collectionView(_ collectionView: UICollectionView,
                        didSelectItemAt indexPath: IndexPath) {
        if chips[indexPath.item].value == nil {
            onPick?(indexPath.item)
        } else {
            chips[indexPath.item].value = nil
            collectionView.reloadItems(at: [indexPath])
            onChange?()
        }
    }
}
```

Логика та же, что в любом магазине. У таблетки нет значения — тап открывает выбор (например, лист со списком брендов, глава 28). Значение есть — тап сбрасывает фильтр, таблетка перерисовывается, а выдача пересчитывается. `displayTitle` показывает на таблетке выбранное значение, а пока его нет — название фильтра.

Частые ошибки.

- **Нет видимой разницы «фильтр включён / выключен».** Человек думает, что фильтр сброшен, а выдача почему-то короткая. Активная таблетка должна отличаться цветом **и** значком, а не одним оттенком.
- **`minimumInteritemSpacing` вместо `minimumLineSpacing`** в горизонтальной строке — таблетки слипаются, и непонятно почему.
- **`itemSize = automaticSize`.** `automaticSize` — особое значение только для `estimatedItemSize`. Само по себе это `CGSize` с гигантскими сторонами (`CGFloat.greatestFiniteMagnitude`), и в `itemSize` раскладка примет его за буквальный размер ячейки.
- **Слишком много таблеток.** Если фильтров больше пяти-шести, строка уезжает далеко за край. Лучше одна кнопка «Фильтры» и отдельный экран со всеми фильтрами и кнопкой «Применить».

Альтернативы. Если таблеток три-четыре и они не меняются — `UIStackView` с кнопками внутри горизонтального `UIScrollView`: меньше кода, но без переиспользования ячеек. Для длинной строки переиспользование в `UICollectionView` уже окупается.

Упражнение 25.2. В строке шесть таблеток по 90 точек шириной, `minimumLineSpacing = 8`, `sectionInset` слева и справа по 16. Какая ширина у всей строки? Поместится ли она без прокрутки на экране шириной 393 точки (iPhone 16)?

25.6 Sort sheet

Когда применять. Дать выбор порядка: по дате, по названию, по цене. Кнопка «Сортировка» открывает список вариантов.

```
enum Sort: CaseIterable {
    case dateDesc, dateAsc, nameAsc, priceAsc

    var title: String {
        switch self {
```

```
        case .dateDesc: return "По дате (новые)"
        case .dateAsc:  return "По дате (старые)"
        case .nameAsc:  return "По названию (А–Я)"
        case .priceAsc: return "По цене (сначала дешёвые)"
        }
    }
}

final class SortableViewController: UIViewController {
    private let sortButton = UIButton(type: .system)
    private var currentSort: Sort = .dateDesc

    private func showSortSheet() {
        let alert = UIAlertController(title: "Сортировка", message: nil,
                                      preferredStyle: .actionSheet)
        for sort in Sort.allCases {
            alert.addAction(UIAlertAction(title: sort.title, style: .default) { [weak
↪   self] _ in
                self?.currentSort = sort
                self?.applySort()
            })
        }
        alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
        alert.popoverPresentationController?.sourceView = sortButton
        alert.popoverPresentationController?.sourceRect = sortButton.bounds
        present(alert, animated: true)
    }

    private func applySort() {
        // пересортировать данные и перезагрузить список
    }
}
```

Action sheet — список действий, который выезжает снизу экрана. Хорош для 3–6 вариантов: больше — и список начнёт прокручиваться.

Цикл по `Sort.allCases` добавляет по кнопке на каждый вариант. Кнопка со стилем `.cancel` («Отмена») встаёт отдельно, внизу — iOS ставит её туда сама, в каком бы порядке ты ни добавлял кнопки.

Две строки с `popoverPresentationController` обязательны. На iPad action sheet показывается не снизу, а **поповером** — облачком со стрелкой, указывающей на кнопку. Облачку нужно знать, откуда расти: `sourceView` — view-источник, `sourceRect` — прямоугольник в его координатах (здесь — вся кнопка). Без этих строк на iPad приложение **упадёт** при показе. На iPhone объект тоже есть: у action sheet стиль показа `.popover` (у свежего `UIAlertController` со стилем `.actionSheet` свойство `modalPresentationStyle` читается как `.popover`, проверено на iOS 26.5), просто в компактной ширине UIKit адаптирует поповер в лист, выезжающий снизу, и источник ему не нужен. Поэтому строки безвредны на любом устройстве, а `?.` страхует на случай, если стиль кто-то сменил.

У `UIAlertAction` нет публичного способа поставить галочку у текущего варианта. Встречается совет `setValue(true, forKey: "checked")` — это обращение к приватному свойству. Его не видно в документации, Apple может убрать его в любой версии iOS, а приложение с таким кодом рискует не пройти проверку App Review. Не используй.

Если нужна отметка текущего выбора — бери меню `UIMenu`:

```
extension SortableViewController {
    func setupSortMenu() {
        let actions = Sort.allCases.map { sort in
            UIAction(title: sort.title,
                     state: sort == currentSort ? .on : .off) { [weak self] _ in
                self?.currentSort = sort
                self?.applySort()
            }
        }
        sortButton.menu = UIMenu(title: "Сортировка",
                                 options: .singleSelection,
                                 children: actions)
        sortButton.showsMenuAsPrimaryAction = true
    }
}
```

`UIAction` — пункт меню с заголовком и замыканием. Свойство `state` со значением `.on` рисует у пункта галочку. И `UIAction`, и `UIMenu` — не view и не контролы (`UIAction → UIMenuElement → NSObject`): это описание пункта, а рисует и показывает его кнопка (глава 0, раздел 0.8).

`showsMenuAsPrimaryAction = true` — меню открывается обычным тапом по кнопке. Без этого — только долгим нажатием.

`options: .singleSelection` (iOS 15+) — «в меню может быть включён только один пункт». Без этой опции есть ловушка: `state` вычисляется **один раз**, в момент создания меню. Человек выбрал «По цене», `currentSort` поменялся, а галочка при новом открытии стоит на «По дате» — меню-то старое. С `.singleSelection` UIKit сам переносит галочку на выбранный пункт. Без неё пришлось бы пересобирать меню после каждого выбора (вызывать `setupSortMenu()` в замыкании).

`Sort` сравнивается через == без явного `Equatable`: Swift сам добавляет сравнение перечислениям без связанных значений.

Частые ошибки.

- **Action sheet без `sourceView` на iPad** — падение.
- **Сортировка «не держится».** После перезагрузки данных с сервера список приходит в серверном порядке. Применяй `currentSort` каждый раз, когда обновляешь данные, а не только в момент выбора.

25.7 Live API search (autocomplete)

Когда применять. Поиск на сервере: товары, адреса, люди в чате. Выдача меняется по мере ввода — **autocomplete** («автодополнение»).

```swift
struct ProductDTO: Decodable {
    let title: String
    let brand: String?
    let price: Double
}

struct SearchResponse: Decodable {
    let products: [ProductDTO]
}

enum ProductsAPI {
    static func search(query: String) async throws -> [ProductDTO] {
        var components = URLComponents(string: "https://dummyjson.com/products/search")
        components?.queryItems = [URLQueryItem(name: "q", value: query)]
        guard let url = components?.url else { throw URLError(.badURL) }
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode(SearchResponse.self, from: data).products
    }
}
```

Сначала — сетевой слой. `ProductDTO` описывает товар так, как его присылает сервер (DTO — «объект для передачи данных», форма ответа сервера, а не модель приложения). `brand` опциональный: у части товаров бренда в ответе нет, и без `?` расшифровка упала бы на первом же таком товаре.

`URLComponents` собирает адрес и сам **экранирует** запрос: пробел станет `%20`, кириллица — байтами вида `%D0%BA`. Склеивать адрес строками (`"...?q=" + query`) нельзя — на запросе «кофе машина» адрес сломается.

`URLSession.shared.data(from:)` — асинхронная загрузка (есть с iOS 15). `await` значит «здесь функция приостанавливается, пока не придёт ответ», главный поток при этом свободен и интерфейс не замирает.

Теперь сам поиск с паузой и отменой:

```swift
final class LiveSearchViewController: UITableViewController,
                                      UISearchResultsUpdating {
    private var results: [ProductDTO] = []
    private var searchTask: Task<Void, Never>?

    func updateSearchResults(for searchController: UISearchController) {
        let query = (searchController.searchBar.text ?? "")
            .trimmingCharacters(in: .whitespacesAndNewlines)
        searchTask?.cancel()

        guard query.count >= 2 else {
            results = []
            tableView.reloadData()
            return
        }
```

```
        searchTask = Task { [weak self] in
            try? await Task.sleep(nanoseconds: 300_000_000)   // 0,3 с
            guard !Task.isCancelled else { return }

            let found = try? await ProductsAPI.search(query: query)
            guard !Task.isCancelled, let found else { return }

            self?.results = found
            self?.tableView.reloadData()
        }
    }
}
```

`searchTask?.cancel()` — отменяем предыдущую задачу. Отмена в Swift **кооперативная**: `cancel()` не убивает задачу, а поднимает флажок «тебя отменили». Задача сама должна его проверить — отсюда `Task.isCancelled` ниже.

`guard query.count >= 2` — на одну букву сервер вернёт пол-каталога, а человек всё равно продолжит печатать. Пустой запрос очищает выдачу.

`Task { ... }` — новая асинхронная задача. В нашем режиме (весь код на главном акторе) она **наследует** главный актор от места создания. Это значит, что строки `self?.results = found` и `reloadData()` и так выполняются на главном потоке — оборачивать их в `MainActor.run` не нужно.

`Task.sleep(nanoseconds: 300_000_000)` — пауза 0,3 с (300 миллионов наносекунд; в секунде миллиард наносекунд). Это и есть debounce. Если за время паузы задачу отменили, `sleep` бросает ошибку отмены и заканчивается сразу; `try?` эту ошибку глотает. Более удобный `Task.sleep(for: .milliseconds(300))` появился только в iOS 16, а у нас минимальная версия — iOS 15.

Первый `guard !Task.isCancelled` — после паузы: если нас отменили, в сеть не идём.

Второй `guard !Task.isCancelled` — **после ответа сервера**. Без него возможна гонка. Пример на числах: запрос «ко» ушёл в 0,30 с, сервер отвечает 1,5 с. В 0,50 с человек допечатал «коф», запрос «коф» ушёл в 0,80 с и вернулся уже в 1,20 с. Ответ на «ко» придёт позже, в 1,80 с, и без проверки перезапишет свежую выдачу старой. `URLSession` при отмене задачи обычно прерывает запрос и бросает ошибку, но полагаться только на это не стоит: проверка флажка стоит одну строку.

`[weak self]` — задача может жить дольше экрана (медленная сеть). Слабая ссылка не удерживает закрытый экран в памяти.

Частые ошибки.

- **Нет проверки после `await`** — старые ответы перетирают новые.
- **Нет отмены при уходе с экрана.** Добавь в `viewWillDisappear` строку `searchTask?.cancel()`, чтобы не тратить трафик на выдачу, которую уже никто не увидит.
- **Нет состояния ошибки.** `try?` превращает ошибку сети в `nil`, и выдача просто не меняется. Для настоящего экрана покажи «Нет соединения» (глава 24).

25.8 Search highlight

Когда применять. Показать, **где** в результате нашлось совпадение: подсветить «кофе» в «Кофемашина Café Crème».

```
func highlight(_ text: String, query: String) -> NSAttributedString {
    let result = NSMutableAttributedString(string: text)
    let needle = query.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !needle.isEmpty else { return result }

    let source = text as NSString
    var searchRange = NSRange(location: 0, length: source.length)
    while true {
        let found = source.range(of: needle,
                                 options: [.caseInsensitive, .diacriticInsensitive],
                                 range: searchRange)
        if found.location == NSNotFound { break }
        result.addAttributes([
            .backgroundColor: UIColor.systemYellow.withAlphaComponent(0.4),
            .font: UIFont.preferredFont(forTextStyle: .body).withBoldTrait(),
        ], range: found)
        let next = found.location + found.length
        searchRange = NSRange(location: next, length: source.length - next)
    }
    return result
}

extension UIFont {
    func withBoldTrait() -> UIFont {
        guard let descriptor = fontDescriptor.withSymbolicTraits(.traitBold) else {
            return self
        }
        return UIFont(descriptor: descriptor, size: 0)
    }
}
```

`NSMutableAttributedString` — строка, у кусков которой могут быть свои **атрибуты**: цвет, шрифт, фон. Метки (`UILabel`) и ячейки умеют такую строку показывать.

Почему `NSString` и `NSRange`, а не обычный `String`: атрибуты задаются диапазонами в единицах UTF-16 — так строки хранит Foundation. `(text as NSString).range(of:)` сразу возвращает диапазон в тех же единицах, и его можно передать в `addAttributes` без пересчёта.

Цикл нужен, чтобы подсветить **все** вхождения, а не первое. На числах: в «Кофе и кофемолка» запрос «кофе» найдётся с позиции 0 длиной 4. Второй поиск начинается с позиции 4 и идёт до конца; второе «кофе» найдётся с позиции 7. Третий поиск вернёт `NSNotFound` — «не найдено», цикл закончится.

`.caseInsensitive` и `.diacriticInsensitive` — те же правила, что у поиска в 25.1: «Кофе» подсветится на запрос «кофе», «Café» — на «cafe». Подсветка должна совпадать с тем, по какому правилу искали, иначе строка найдётся, а подсвечено в ней ничего не будет.

Подсветка двумя способами — фоном **и** жирным шрифтом. Один только цвет плохо различают люди с нарушениями цветового зрения, а жирное начертание видно всем. `withBoldTrait()` — маленькое расширение: берёт описание шрифта, добавляет признак «жирный» и строит шрифт заново; `size: 0` значит «размер оставить как был». Шрифт `body` совпадает с тем, которым ячейка по умолчанию рисует основной текст.

В ячейке используй так:

```
var content = cell.defaultContentConfiguration()
content.attributedText = highlight(product.title, query: currentQuery)
cell.contentConfiguration = content
```

Частые ошибки.

- **Подсветка по другому правилу, чем поиск.** Искали без учёта диакритики, а подсвечиваем с учётом — «Café» в выдаче есть, а подсветки нет.
- **Диапазон из `String` в `NSRange` «на глаз».** Эмодзи и некоторые символы занимают в UTF-16 две единицы, и позиции из `String.count` съезжают. Бери диапазон из того же `NSString`, в котором искал.

Упражнение 25.3. Что вернёт `highlight("Кофе и кофемолка", query: "  КОФЕ ")`: сколько кусков будет подсвечено и с каких позиций они начинаются? А при `query: ""`?

Ответы к упражнениям

25.1. Если печатать быстро (между нажатиями меньше 0,2 с), «Ищем:» напечатается один раз — `Ищем: кофе`. Если где-то задержался дольше 0,2 с, появится и промежуточный запрос, например `Ищем: ко`. С паузой 1.0 промежуточных запросов почти не будет, но результат появится через целую секунду после последнего нажатия — для пользователя поиск станет «задумчивым».

25.2. Шесть таблеток: 6 × 90 = 540 точек. Промежутков между ними пять: 5 × 8 = 40. Поля: 16 + 16 = 32. Итого 540 + 40 + 32 = 612 точек. Это больше 393, так что строка прокручивается: видно примерно первые три-четыре таблетки, остальные — свайпом влево.

25.3. Запрос сначала обрезается до «КОФЕ». Поиск без учёта регистра найдёт два вхождения: с позиции 0 («Кофе») и с позиции 7 («кофе» в «кофемолка»), каждое длиной 4. Если запрос пустой (или из одних пробелов), `guard` вернёт строку без подсветки.

Что мы выучили

- **`UISearchController`** — view controller, а не view; его строка `UISearchBar` — view, но не `UIControl` (события — только через делегат), поле ввода внутри — `UISearchTextField` → `UITextField`.
- **`UISearchController`** встраивается в `navigationItem.searchController` (табличка принадлежит view controller'у); с `searchResultsController: nil` фильтруем ту же таблицу и держим

`obscuresBackgroundDuringPresentation = false` (для проектов на iOS 15 SDK это и так значение по умолчанию).
- **Два массива**: `allItems` — источник правды, `filtered` — то, что на экране. Индексы строк берём из `filtered`.
- **`localizedStandardContains`** ищет без учёта регистра и диакритики.
- **Debounce** — ждём паузу во вводе и отменяем прошлое задание: `DispatchWorkItem` + `cancel()` или `Task` + `Task.sleep` + `Task.isCancelled`. Debounce уменьшает число поисков, но не ускоряет каждый.
- **Scope bar** — сегменты «где искать»; смена сегмента требует пересчёта выдачи вручную.
- **Recent searches** — 10 последних запросов в `UserDefaults`, без дублей, сохраняются по «Найти», а не по каждой букве.
- **Filter chips** — горизонтальный collection view; отступ между таблетками задаёт `minimumLineSpacing`; активная таблетка отличается цветом, значком и признаком `.selected` для VoiceOver.
- **Сортировка** — action sheet (стиль показа `.popover`; на iPad обязательно `sourceView`, на iPhone поповер сам становится листом) или `UIMenu` с `.singleSelection`, чтобы галочка переезжала сама.
- **Live search** — отмена прошлой задачи и проверка `isCancelled` **после** ответа сервера, иначе старый ответ перетрёт новый.
- **Подсветка** — все вхождения, по тем же правилам, что и поиск, цветом и жирным шрифтом.

Apple Developer Documentation

- `UISearchController` — стандартный контроллер поиска; крепится к `navigationItem.searchController`.
- `UISearchBar` — сама строка ввода (scope buttons, placeholder, кнопка «Отменить»); `searchTextField` — поле ввода внутри неё (iOS 13+).
- `UIViewController.definesPresentationContext` — какой экран накрывает презентация «в текущем контексте».
- `UISearchResultsUpdating` — протокол обновления результатов при каждом изменении текста.
- `UISearchBarDelegate` — «Найти», «Отменить», смена сегмента области поиска.
- `UINavigationItem.searchController` — встраивание строки поиска в навигационную панель.
- `localizedStandardContains(_:)` — поиск подстроки без учёта регистра и диакритики.
- `UICollectionViewFlowLayout` — `minimumLineSpacing`, `estimatedItemSize`, `automaticSize`.
- `UIMenu.Options.singleSelection` — меню с одним выбранным пунктом (iOS 15+).
- `UIPopoverPresentationController` — `sourceView` / `sourceRect` для action sheet на iPad.
- `NSAttributedString.Key.backgroundColor` — атрибут подсветки совпадения.
- `Task.sleep(nanoseconds:)` и `Task.isCancelled` — пауза и проверка отмены.
- HIG — Searching — Apple про паттерны поиска, области поиска и недавние запросы.

→ Глава 26. Cookbook — навигация и заголовки

Глава 26. Cookbook — навигация и заголовки

Почти любое приложение — это стопка экранов: список, из него карточка, из карточки — настройки. Эта глава про всё, что происходит вокруг такой стопки: большой заголовок, который сжимается при прокрутке, бейджи на вкладках, индикатор шагов, «хлебные крошки», своя кнопка «назад», выбор между push и модальным показом, собственная анимация перехода и внешний вид навигационной панели.

> **В каком режиме код.** Как и во всей части IV, листинги проверены компилятором в режиме Swift 6 с Default Actor Isolation = MainActor: весь код без пометок выполняется на главном потоке (подробнее — в начале главы 25).

Несколько слов, без которых дальше будет трудно.

`UINavigationController` — контейнер, который держит экраны **стопкой** (стеком). Представь стопку карточек на столе: `push` кладёт новую карточку сверху, `pop` снимает верхнюю, и снова видна предыдущая. Видна всегда только верхняя.

Навигационная панель (navigation bar, класс `UINavigationBar → UIView`) — полоса сверху с заголовком и кнопками. Она одна на весь контроллер навигации.

`navigationItem` — «табличка» каждого экрана: заголовок, кнопки слева и справа, строка поиска. Это объект класса `UINavigationItem` (потомок `NSObject`, не view) и **свойство view controller'а**: UIKit создаёт его лениво при первом обращении, по одному на экран. Панель сама ничего не хранит — она показывает табличку того экрана, который сейчас сверху стопки, а кнопку «назад» берёт у таблички предыдущего. Поэтому кнопки экрана настраивают через его `navigationItem`, а не через панель напрямую. Сами кнопки в табличке — `UIBarButtonItem → UIBarItem → NSObject`: тоже не view, поэтому у них нет `frame`, `addSubview` и `addGestureRecognizer`; панель по описанию сама рисует для них настоящие кнопки. Лестница классов — глава 0, разделы 0.6 и 0.8.

26.1 Large titles + scroll fade

Когда применять. Главные экраны приложения, как в «Настройках» или «Почте»: сверху крупный заголовок, а при прокрутке списка он плавно сжимается в обычный, по центру панели.

```
// Там, где создаёшь стопку (SceneDelegate или координатор):
func makeOrdersFlow() -> UINavigationController {
    let nav = UINavigationController(rootViewController: OrdersViewController())
```

```
    nav.navigationBar.prefersLargeTitles = true
    return nav
}

final class OrdersViewController: UITableViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Заказы"
        navigationItem.largeTitleDisplayMode = .always
    }
}

final class OrderDetailsViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        navigationItem.largeTitleDisplayMode = .never
    }
}
```

Как это устроено:

- `prefersLargeTitles` — свойство **панели**, то есть всей стопки сразу. Включаем один раз, в момент создания контроллера навигации.
- `largeTitleDisplayMode` — свойство **таблички** конкретного экрана. `.always` — этот экран показывает большой заголовок. `.never` — обычный, маленький. `.automatic` (по умолчанию) — «как у предыдущего экрана в стопке».
- `title` — сам текст заголовка.

Типичная схема: главный список — `.always`, всё, что открывается из него, — `.never`. Если оставить карточке `.automatic`, она унаследует большой заголовок от списка, и длинное название заказа займёт полэкрана.

Сжатие при прокрутке iOS делает сама, но только если понимает, **за каким списком следить**. По документации view controller сам находит scroll view, анализируя иерархию view; когда экран — `UITableViewController` или таблица лежит первой в иерархии, обычно всё работает само. Если иерархия сложнее и между панелью и таблицей стоят другие view, укажи список явно (iOS 15+):

```
setContentScrollView(tableView, for: .top)
```

`for: .top` — «этот список отвечает за верхний край экрана»: панель будет сжиматься и менять фон по его прокрутке.

Частые ошибки.

- **`prefersLargeTitles` включают в каждом экране** в `viewWillAppear`. Это свойство общей панели — и при возврате назад заголовок дёргается между размерами. Включай один раз, а поведение отдельных экранов задавай через `largeTitleDisplayMode`.
- **Заголовок не сжимается.** Панель не нашла список: таблица не первая в иерархии. Лечится `setContentScrollView(_:for:)`.

- **Большой заголовок мешает своей шапке.** Если у тебя растягивающаяся шапка (stretchy header, глава 21), панель и шапка будут бороться за одно пространство. Ставь такому экрану `.never` и управляй размером заголовка сам.

26.2 Tab bar badges

Когда применять. Сообщить о новом внутри вкладки: непрочитанные сообщения, обновления, товары в корзине.

Бейдж (badge) — маленький кружок с числом у иконки вкладки. Рисует его сама iOS, красным, в правом верхнем углу иконки.

```
// Внутри экрана, который лежит во вкладке «Чаты»:
func updateUnreadBadge(unread: Int) {
    guard let item = navigationController?.tabBarItem ?? tabBarItem else { return }
    switch unread {
    case 0:     item.badgeValue = nil
    case 1...99: item.badgeValue = "\(unread)"
    default:    item.badgeValue = "99+"
    }
}
```

`badgeValue` — строка (`String?`). Обычно число, но можно и короткий текст вроде «new». `nil` убирает бейдж (это и значение по умолчанию). Сам `tabBarItem` — объект `UITabBarItem → UIBarItem → NSObject`, такое же свойство view controller'а, как `navigationItem`: не view, а описание вкладки, которое панель `UITabBar → UIView` рисует.

Почему `navigationController?.tabBarItem`: `UITabBarController` берёт иконку и бейдж у экрана, который **непосредственно** лежит во вкладке. Чаще всего это не твой экран, а `UINavigationController` вокруг него. Если поставить бейдж на `tabBarItem` самого экрана, он не появится: вкладка смотрит на табличку контроллера навигации. `?? tabBarItem` — запасной вариант для экрана, который лежит во вкладке без навигации.

Три ветки `switch`:

- `0` → `nil`. Бейдж «0» выглядит как «у тебя ноль новых» — это не новость, прячем.
- `1...99` → число как есть.
- больше 99 → «99+». Кружок растягивается под текст, и «1284» займёт полвкладки.

Цвет меняется через `badgeColor` (iOS 10+), если красный не подходит под дизайн.

Частые ошибки.

- **Бейдж «0».** См. выше: для нуля ставь `nil`.
- **Бейдж не гаснет.** Сбрасывай его, когда человек действительно увидел новое: открыл вкладку (`viewDidAppear`) или прочитал чат.
- **Бейдж ставят не на тот `tabBarItem`** — на экран вместо контроллера навигации вокруг него.

Альтернативы. Число на иконке приложения на домашнем экране — это другой бейдж, его ставит push-уведомление или API уведомлений (глава 41).

26.3 Step indicator

Когда применять. Мастер (wizard) или онбординг с понятными шагами: «Регистрация: шаг 2 из 4».

```
final class StepIndicatorView: UIView {
    private let totalSteps: Int
    private var dots: [UIView] = []

    init(totalSteps: Int) {
        self.totalSteps = totalSteps
        super.init(frame: .zero)
        setupDots()
        isAccessibilityElement = true
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private func setupDots() {
        let stack = UIStackView()
        stack.axis = .horizontal
        stack.spacing = 8
        stack.translatesAutoresizingMaskIntoConstraints = false
        for _ in 0..<totalSteps {
            let dot = UIView()
            dot.backgroundColor = .tertiaryLabel
            dot.layer.cornerRadius = 4
            dot.translatesAutoresizingMaskIntoConstraints = false
            dot.widthAnchor.constraint(equalToConstant: 8).isActive = true
            dot.heightAnchor.constraint(equalToConstant: 8).isActive = true
            dots.append(dot)
            stack.addArrangedSubview(dot)
        }
        addSubview(stack)
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: topAnchor),
            stack.bottomAnchor.constraint(equalTo: bottomAnchor),
            stack.centerXAnchor.constraint(equalTo: centerXAnchor),
            stack.leadingAnchor.constraint(greaterThanOrEqualTo: leadingAnchor),
        ])
    }

    func setCurrentStep(_ step: Int) {
        UIView.animate(withDuration: 0.2) {
            for (i, dot) in self.dots.enumerated() {
                dot.backgroundColor = i <= step ? .systemBlue : .tertiaryLabel
                dot.transform = i == step
                    ? CGAffineTransform(scaleX: 1.4, y: 1.4)
                    : .identity
```

```
            }
        }
        accessibilityLabel = "Шаг \(step + 1) из \(totalSteps)"
    }
}
```

Точки в ряд. Пройденные и текущая — синие, будущие — серые, а текущая ещё и крупнее.

Разберём то, что не бросается в глаза.

- Точка 8 × 8 точек с `cornerRadius = 4`: радиус скругления — половина стороны, поэтому квадрат превращается в круг.
- Стек прибит к `centerXAnchor` и к верху и низу. Ограничение `leadingAnchor ... greaterThanOrEqualTo` («не левее левого края») не даёт стеку вылезти за границы, если шагов окажется много. **Ограничение** (constraint) — правило Auto Layout вида «этот край равен тому краю плюс 8»; из набора таких правил система вычисляет положение и размер каждой view.
- `CGAffineTransform(scaleX: 1.4, y: 1.4)` — увеличить в 1,4 раза, то есть **на 40%**: точка 8 × 8 выглядит как 11,2 × 11,2. Трансформация меняет только отрисовку, а не место в раскладке, поэтому соседние точки не раздвигаются — крупная точка просто чуть «налезает» на промежутки. `.identity` — «без трансформации», исходный размер.
- `i <= step` — шаги нумеруются с нуля. При `step = 1` синими будут точки 0 и 1, то есть «пройден первый шаг, идёт второй».
- Один `UIView.animate` на все точки: изменения цвета и размера плавно проигрываются за 0,2 секунды.
- `accessibilityLabel` — VoiceOver прочитает «Шаг 2 из 4». Без этого незрячий человек услышит просто «элемент».

Вместо точек можно показать тонкую полосу прогресса:

```
let progress = UIProgressView(progressViewStyle: .bar)
progress.setProgress(Float(currentStep + 1) / Float(totalSteps), animated: true)
```

`progress` — число от 0 до 1, доля заполнения. На шаге 2 из 4 (`currentStep = 1`) получится (1 + 1) / 4 = 0,5 — полоса заполнена наполовину. `Float(...)` перед делением обязателен: `2 / 4` в целых числах даёт 0, и полоса останется пустой до последнего шага.

Частые ошибки.

- **Деление целых чисел** в прогрессе — см. выше.
- **Нумерация шагов с нуля в тексте.** Человеку нужно «Шаг 1 из 4», а не «Шаг 0 из 4»: к индексу прибавляй 1.

26.4 Breadcrumbs

Когда применять. Глубокая иерархия: «Каталог › Электроника › Смартфоны › Apple». **Хлебные крошки** — строка из уровней, по любому из которых можно вернуться. В iOS встречаются редко (стандартная кнопка «назад» ведёт только на уровень выше), но оправданы в файловых менеджерах и админках.

```
extension CatalogViewController {
    func makeBreadcrumbs(_ path: [String]) -> UIScrollView {
        let scrollView = UIScrollView()
        scrollView.showsHorizontalScrollIndicator = false
        let stack = UIStackView()
        stack.axis = .horizontal
        stack.spacing = 4
        stack.translatesAutoresizingMaskIntoConstraints = false
        scrollView.addSubview(stack)
        NSLayoutConstraint.activate([
            stack.leadingAnchor.constraint(equalTo:
↪   scrollView.contentLayoutGuide.leadingAnchor, constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪   scrollView.contentLayoutGuide.trailingAnchor, constant: -16),
            stack.topAnchor.constraint(equalTo: scrollView.contentLayoutGuide.topAnchor),
            stack.bottomAnchor.constraint(equalTo:
↪   scrollView.contentLayoutGuide.bottomAnchor),
            stack.heightAnchor.constraint(equalTo:
↪   scrollView.frameLayoutGuide.heightAnchor),
        ])

        for (level, name) in path.enumerated() {
            let isLast = level == path.count - 1
            let button = UIButton(type: .system)
            button.setTitle(name, for: .normal)
            button.titleLabel?.font = .preferredFont(forTextStyle: .footnote)
            button.isEnabled = !isLast
            button.addAction(UIAction { [weak self] _ in
                self?.navigateBack(toLevel: level)
            }, for: .touchUpInside)
            stack.addArrangedSubview(button)

            if !isLast {
                let separator = UILabel()
                separator.text = "›"
                separator.textColor = .secondaryLabel
                separator.isAccessibilityElement = false
                stack.addArrangedSubview(separator)
            }
        }
        return scrollView
    }

    func navigateBack(toLevel level: Int) {
        guard let stack = navigationController?.viewControllers,
              level < stack.count else { return }
        navigationController?.popToViewController(stack[level], animated: true)
    }
```

```
}
```

Горизонтально прокручиваемая строка кнопок. Тап по уровню — возврат на этот уровень.

- Стек прибит к `contentLayoutGuide` скролла — это «размер содержимого»: сколько места нужно всем кнопкам, столько и можно прокрутить. А высота стека приравнена к `frameLayoutGuide` — к видимой рамке скролла, иначе скроллу было бы непонятно, какой высоты содержимое, и появилась бы ещё и вертикальная прокрутка.
- `button.isEnabled = !isLast` — последний уровень — это экран, на котором ты сейчас. Возвращаться на него некуда, поэтому кнопка неактивна и выглядит серой.
- Разделитель «›» спрятан от VoiceOver (`isAccessibilityElement = false`): слушать «знак больше» между уровнями бесполезно.
- `navigateBack(toLevel:)` предполагает, что уровень крошки совпадает с позицией экрана в стопке: «Каталог» — нулевой экран, «Электроника» — первый и так далее. `popToViewController` снимает со стопки всё, что выше нужного экрана.
- `[weak self]` в замыкании кнопки: экран держит кнопку, кнопка — действие, действие — замыкание. Сильная ссылка на `self` замкнула бы круг, и экран никогда бы не освободился.

Частые ошибки.

- **Кликабельный текущий уровень** — тап ничего не делает, человек думает, что приложение зависло.
- **Уровни не совпадают со стопкой** (например, часть уровней открыта модально). Тогда `level` уже не индекс в `viewControllers`, и нужно искать экран по-другому.

26.5 Custom back button

Когда применять. Две разные задачи, которые часто путают.

Задача 1: кнопка «закрыть» на модальном экране. У первого экрана модального окна кнопки «назад» нет вовсе — возвращаться некуда, окно надо закрыть. Здесь своя левая кнопка — правильное решение:

```
final class EditorViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        navigationItem.leftBarButtonItem = UIBarButtonItem(
            image: UIImage(systemName: "xmark"),
            style: .plain,
            target: self,
            action: #selector(closeTapped)
        )
        navigationItem.leftBarButtonItem?.accessibilityLabel = "Закрыть"
    }

    @objc private func closeTapped() {
        dismiss(animated: true)
    }
}
```

`leftBarButtonItem` — кнопка в левом слоте таблички экрана. Напомню, что `UIBarButtonItem` — не view (`UIBarItem → NSObject`): это описание кнопки, и настоящую view для неё панель создаёт сама, поэтому `accessibilityLabel` мы ставим на сам item. Иконка `xmark` («крестик») — системный символ SF Symbols. У кнопки-картинки нет текста, поэтому VoiceOver нужно подсказать, что это «Закрыть».

`target: self, action: #selector(...)` — классический способ UIKit сказать «при нажатии вызови у этого объекта этот метод». Метод помечен `@objc`, потому что механизм селекторов пришёл из Objective-C.

Задача 2: поменять вид стандартной кнопки «назад» в стопке. Здесь `leftBarButtonItem` — **плохое** решение: своя левая кнопка заменяет системную «назад», и вместе с ней перестаёт работать свайп от левого края экрана. Правильнее настроить саму системную кнопку:

```
// Всё это — в экране, КУДА вернёмся (например, в списке):
navigationItem.backButtonTitle = "Список"        // текст кнопки «назад» на экране,
↪  открытом из списка
navigationItem.backButtonDisplayMode = .minimal  // iOS 14+: только стрелка, без текста
navigationItem.backBarButtonItem = UIBarButtonItem(title: "К списку") // старый способ

// Своя стрелка для всей панели:
let appearance = UINavigationBarAppearance()
appearance.configureWithDefaultBackground()
let arrow = UIImage(systemName: "arrow.left")
appearance.setBackIndicatorImage(arrow, transitionMaskImage: arrow)
navigationController?.navigationBar.standardAppearance = appearance
```

Главная хитрость: кнопка «назад» **принадлежит предыдущему экрану**. Когда поверх списка лежит карточка, панель берёт кнопку «назад» из таблички списка — из его `backBarButtonItem`, а если он `nil`, собирает её сама из `backButtonTitle`, `title` и `backButtonDisplayMode` того же списка. Логика такая: кнопка ведёт на список, значит и выглядеть должна так, как хочет список. Поэтому все три свойства ставят в экране, **куда** вернёмся, до того как он сделает push следующего (например, в `viewDidLoad`).

`backButtonDisplayMode` (iOS 14+) говорит, из чего собирать подпись: `.default` — заголовок предыдущего экрана, а если места мало — «Назад» или одна стрелка; `.generic` — только «Назад»; `.minimal` — одна стрелка. Документация оговаривает: режим действует, пока `backBarButtonItem` равен `nil`, — свой item важнее. У `backBarButtonItem` панель берёт заголовок и картинку, а `customView`, по той же документации, игнорирует; нажатие на кнопку «назад» всегда означает системный pop, своих `target`/`action` от неё не жди.

`setBackIndicatorImage(_:transitionMaskImage:)` меняет саму стрелку. Вторая картинка — маска, по которой текст кнопки «уезжает» под стрелку во время анимации перехода; чаще всего передают ту же картинку. Подробнее о `UINavigationBarAppearance` — в 26.9.

Все три способа сохраняют системную кнопку, а значит и свайп назад.

Частые ошибки.

- **Своя `leftBarButtonItem` в стопке ломает свайп назад.** Если без неё никак, восстанавливай жест правильно. Часто советуют `interactivePopGestureRecognizer?.delegate = nil`, но у этого хака есть ловушка: на **первом** экране стопки свайп от края тоже срабатывает,

возвращаться некуда, и навигация может перестать реагировать на касания. Надёжнее сделать контроллер навигации делегатом жеста и разрешать свайп, только когда в стопке больше одного экрана:

```
final class AppNavigationController: UINavigationController,
                                     UIGestureRecognizerDelegate {
    override func viewDidLoad() {
        super.viewDidLoad()
        interactivePopGestureRecognizer?.delegate = self
    }

    func gestureRecognizerShouldBegin(_ gestureRecognizer: UIGestureRecognizer) -> Bool {
        viewControllers.count > 1
    }
}
```

`gestureRecognizerShouldBegin` спрашивают прямо перед стартом жеста: «можно начинать?». Ответ «да, если в стопке больше одного экрана» отключает свайп на корневом экране и возвращает его на всех остальных, даже с кастомной левой кнопкой. Сам жест — экземпляр приватного подкласса `UIScreenEdgePanGestureRecognizer` (→ `UIPanGestureRecognizer` → `UIGestureRecognizer`), хотя свойство объявлено просто как `UIGestureRecognizer`. Честная оговорка: заголовок `UINavigationController.h` разрешает использовать это свойство только для настройки зависимостей между жестами (`require(toFail:)`); подмена делегата — распространённый приём, который работает много лет, но документацией не обещан. Где можно, оставляй системную кнопку. Подробнее о делегатах жестов — глава 29.

26.6 Modal vs push — когда какой

Push — положить экран в стопку навигации: он въезжает справа, слева появляется «назад». **Modal** (модальный показ) — открыть экран **поверх** текущего, как отдельное окно: он выезжает снизу и ложится поверх всей стопки навигации, а не внутрь неё, пока его не закроют.

Сценарий	Способ показа
Карточка из списка	push
Создание нового элемента	модальный лист (`.medium` или `.large`)
Редактирование одной настройки	push (внутри «Настроек»)
Авторизация	модальный на весь экран
Подтверждение действия	`UIAlertController`
«Поделиться»	`UIActivityViewController`
Быстрый выбор из нескольких опций	`UIMenu` или action sheet

Правило большого пальца. Если человек **смотрит и возвращается** (углубляется в иерархию, чтобы посмотреть подробности) — push. Если он **выполняет отдельную задачу и закрывает** её (создаёт, редактирует, соглашается, входит) — модальный показ. У модального окна должен быть явный выход: «Готово», «Отмена» или крестик.

Бытовая аналогия: push — ты листаешь папку и заходишь в подпапки; модальное окно — тебе выдали анкету, ты её заполнил и сдал.

Подробный разбор всех видов модального показа — глава 28.

26.7 Custom transitions

Когда применять. Стандартного въезда справа мало: например, миниатюра должна «вырасти» в полноэкранную карточку (такой переход называют **hero-анимацией**).

Переход между экранами в UIKit описывает объект-**аниматор**. Когда контроллер навигации собирается сделать push или pop, он спрашивает своего делегата: «есть у тебя аниматор для этого перехода?». Ответил `nil` — будет стандартная анимация.

```
final class ScaleTransition: NSObject, UIViewControllerAnimatedTransitioning {
    func transitionDuration(using transitionContext:
    ↪ UIViewControllerContextTransitioning?)
        -> TimeInterval {
        0.4
    }

    func animateTransition(using transitionContext: UIViewControllerContextTransitioning)
    ↪ {
        guard let toVC = transitionContext.viewController(forKey: .to),
              let toView = transitionContext.view(forKey: .to) else {
            transitionContext.completeTransition(false)
            return
        }
        let container = transitionContext.containerView
        toView.frame = transitionContext.finalFrame(for: toVC)
        container.addSubview(toView)
        toView.transform = CGAffineTransform(scaleX: 0.3, y: 0.3)
        toView.alpha = 0

        UIView.animate(withDuration: transitionDuration(using: transitionContext),
                       delay: 0,
                       usingSpringWithDamping: 0.8,
                       initialSpringVelocity: 0) {
            toView.transform = .identity
            toView.alpha = 1
        } completion: { _ in

↪ transitionContext.completeTransition(!transitionContext.transitionWasCancelled)
        }
    }
}
```

`UIViewControllerAnimatedTransitioning` — протокол аниматора, два обязательных метода:

- `transitionDuration` — длительность, 0,4 секунды.

- `animateTransition` — сама анимация. UIKit передаёт **контекст перехода**: откуда уходим (`.from`), куда приходим (`.to`) и `containerView` — «сцену», на которой разыгрывается переход.

Разбор по шагам:

1. `finalFrame(for:)` — где должен оказаться новый экран в конце. Без этой строки размер новой view не определён, и экран может появиться со смещением или не на весь размер.
2. Добавляем новый экран на сцену и уменьшаем до 30% (`scaleX: 0.3`), делаем прозрачным (`alpha = 0`).
3. Анимация с пружиной возвращает `transform` к `.identity` (100%) и `alpha` к 1. `usingSpringWithDamping: 0.8` — «гашение» пружины от 0 до 1: при 1 движение плавно останавливается без перелёта, при 0,8 экран чуть-чуть перелетит полный размер и вернётся, при 0,5 — заметно качнётся. `initialSpringVelocity: 0` — стартуем с места.
4. `completeTransition(...)` — **обязательно**. Это сигнал «переход закончен», после которого UIKit убирает старый экран и снова принимает касания. Передаём `!transitionWasCancelled`: если переход отменили (например, интерактивно), он не должен считаться состоявшимся.

Подключаем через делегата контроллера навигации:

```
final class HeroNavigationController: UINavigationController,
                                      UINavigationControllerDelegate {
    override func viewDidLoad() {
        super.viewDidLoad()
        delegate = self
    }

    func navigationController(_ navigationController: UINavigationController,
                              animationControllerFor operation:
                               ↪  UINavigationController.Operation,
                              from fromVC: UIViewController,
                              to toVC: UIViewController) ->
                               ↪  UIViewControllerAnimatedTransitioning? {
        operation == .push ? ScaleTransition() : nil
    }
}
```

Контроллер навигации сам себе делегат — это удобно: `delegate` хранится **слабой** ссылкой (`weak` в `UINavigationController.h`), и отдельный объект-делегат, созданный «на лету» и никем не удержанный, исчез бы сразу. Swift 6.3 на присваивание `nav.delegate = Del()` выдаёт предупреждение «weak reference will always be nil because the referenced object is deallocated here», но собирает. Свой контроллер никуда не денется, пока жива стопка.

Аниматор отдаём только для `.push`. Для `.pop` возвращаем `nil`, и тогда работает стандартный возврат — **вместе со свайпом от края**. Если отдавать аниматор и на pop, системный интерактивный свайп перестаёт работать: UIKit ждёт от тебя ещё и **интерактивный контроллер** (`UIPercentDrivenInteractiveTransition`), который переводит движение пальца в процент перехода.

Для **модальных** экранов схема та же, но делегат другой — `transitioningDelegate` у показываемого экрана (глава 28, раздел 28.12).

Частые ошибки.

- **Забыли `completeTransition`** — анимация доиграла, а приложение «зависло»: UIKit ждёт сигнала и не принимает касания.
- **Забыли `finalFrame(for:)`** — экран появляется не того размера.
- **Аниматор на pop без интерактивного контроллера** — пропал свайп назад.
- **Делегат не удержан.** `navigationController.delegate = SomeDelegate()` — объект тут же освободится, и анимация станет стандартной; компилятор предупредит, но приложение запустится.

26.8 Title view с двумя строками

Когда применять. Заголовок и подзаголовок в панели: имя в чате и статус «в сети», название документа и «изменён 5 минут назад».

В iOS 26 для этого появилось готовое свойство. Для более ранних версий собираем подзаголовок сами:

```
extension ChatViewController {
    func setTitle(name: String, status: String) {
        if #available(iOS 26.0, *) {
            navigationItem.title = name
            navigationItem.subtitle = status
        } else {
            navigationItem.titleView = makeTwoLineTitle(name: name, status: status)
        }
    }

    private func makeTwoLineTitle(name: String, status: String) -> UIView {
        let titleLabel = UILabel()
        titleLabel.text = name
        titleLabel.font = .preferredFont(forTextStyle: .headline)
        titleLabel.textAlignment = .center

        let subtitleLabel = UILabel()
        subtitleLabel.text = status
        subtitleLabel.font = .preferredFont(forTextStyle: .caption1)
        subtitleLabel.textColor = .secondaryLabel
        subtitleLabel.textAlignment = .center

        let stack = UIStackView(arrangedSubviews: [titleLabel, subtitleLabel])
        stack.axis = .vertical
        stack.alignment = .center
        stack.isAccessibilityElement = true
        stack.accessibilityLabel = "\(name), \(status)"
        stack.accessibilityTraits = .header
```

```
        return stack
    }
}
```

`#available(iOS 26.0, *)` — проверка версии во время работы: на iOS 26 и новее используем системный подзаголовок, на iOS 15–18 — свою сборку. Минимальная версия проекта — iOS 15, поэтому без этой проверки код с `subtitle` не соберётся.

`titleView` принимает любую view и ставит её на место заголовка; по документации панель центрирует её и при необходимости подгоняет размер. Размер панель спрашивает у самой view через Auto Layout, а стек с двумя метками умеет его назвать: ширину — по самой длинной строке, высоту — по сумме двух строк. Поэтому оборачивать его в дополнительный контейнер не нужно.

Шрифты `headline` и `caption1` — стили Dynamic Type: они совпадают с тем, чем iOS рисует обычный заголовок и мелкие подписи, и увеличиваются вместе с системным размером текста.

Для VoiceOver стек объявлен одним элементом с меткой «Анна, в сети» и признаком заголовка — иначе диктор прочитает две строки как две разрозненные метки.

Частые ошибки.

- **Жёсткие размеры шрифта** (`systemFont(ofSize: 16)`) — при крупном системном тексте подзаголовок останется мелким.
- **`titleView` без размеров.** Если отдать в `titleView` пустой `UIView`, внутри которого стек не прибит ко всем краям, контейнер окажется нулевого размера, и заголовок не будет виден.

26.9 Nav bar appearance — кастомизация

Когда применять. Панель должна быть фирменного цвета, с другим шрифтом заголовка или без тени снизу.

С iOS 13 внешний вид панели описывает объект `UINavigationBarAppearance`:

```
let appearance = UINavigationBarAppearance()
appearance.configureWithOpaqueBackground()
appearance.backgroundColor = .systemBlue
appearance.titleTextAttributes = [.foregroundColor: UIColor.white]
appearance.largeTitleTextAttributes = [.foregroundColor: UIColor.white]

let bar = navigationController?.navigationBar
bar?.standardAppearance = appearance
bar?.scrollEdgeAppearance = appearance
bar?.compactAppearance = appearance
bar?.tintColor = .white
```

Первая строка после создания — «базовая заготовка» фона:

- `configureWithOpaqueBackground()` — сплошной непрозрачный фон.

- `configureWithDefaultBackground()` — системный полупрозрачный фон с размытием (как в «Настройках»).
- `configureWithTransparentBackground()` — полностью прозрачный, без тени.

`titleTextAttributes` и `largeTitleTextAttributes` — оформление обычного и большого заголовка (цвет, шрифт).

Панель бывает в разных **состояниях**, и на каждое есть свой слот:

- `standardAppearance` — обычное состояние, когда под панелью прокручен контент.
- `scrollEdgeAppearance` — когда список прокручен **к самому верху** (контент «упирается» в край панели). Это состояние особенно заметно с большими заголовками.
- `compactAppearance` — **компактная** панель, пониже: на iPhone в горизонтальной ориентации.
- `compactScrollEdgeAppearance` (iOS 15+) — компактная и у края прокрутки одновременно.

Что будет, если какой-то слот не задать. `compactAppearance` возьмёт значение из `standardAppearance` — это безопасно. А вот `scrollEdgeAppearance` без явного значения берёт `standardAppearance`, **изменённый на прозрачный фон** — так написано в документации; до iOS 15 это касалось только панелей с большими заголовками, с iOS 15 — всех панелей. Если задать только `standardAppearance`, панель будет синей при прокрутке и прозрачной, когда список наверху. Поэтому задавай как минимум `standardAppearance` и `scrollEdgeAppearance`.

`tintColor` — цвет **кнопок** панели («назад», кнопки справа). Он не входит в `UINavigationBarAppearance`. Без этой строки на синей панели останутся синие кнопки, и их не будет видно.

Внешний вид можно задать и отдельному экрану — через `navigationItem.standardAppearance` и `navigationItem.scrollEdgeAppearance`. Тогда он действует, только пока этот экран сверху стопки, и не «протекает» на остальные.

Частые ошибки.

- **Задан только `standardAppearance`** — панель прозрачная в верхней точке списка.
- **Забыт `tintColor`** — кнопки сливаются с фоном.
- **Настройка панели из дочернего экрана** через `navigationController?.navigationBar` меняет её для всей стопки, и при возврате назад цвет «прилипает» к предыдущим экранам. Для одного экрана используй `navigationItem`.
- **Белый текст на светлом фоне** — проверь контраст: для обычного текста относительная яркость светлого цвета должна быть хотя бы в 4,5 раза больше, чем тёмного (в формуле есть небольшая поправка, подробно — глава 34).

Упражнение 26.1. Настрой синюю панель так, как в листинге выше, но задай только `standardAppearance`. Открой список, прокрути его вниз и снова к самому верху. Что происходит с цветом панели? Исправь.

Упражнение 26.2. В приложении три вкладки, вторая — «Чаты» внутри `UINavigationController`. Из `ChatsViewController` ты пишешь `tabBarItem.badgeValue = "3"`, но бейдж не появляется. Почему и как починить?

Ответы к упражнениям

26.1. Пока список прокручен вниз, панель синяя (работает `standardAppearance`). Когда список доходит до самого верха, панель переключается на `scrollEdgeAppearance`. Он не задан, и с iOS 15 фон в этом состоянии прозрачный: синий исчезает, а белый заголовок оказывается на белом фоне. Лечение — добавить `bar?.scrollEdgeAppearance = appearance`.

26.2. Вкладка берёт бейдж у экрана, который непосредственно лежит в `UITabBarController`, — у `UINavigationController`, а не у `ChatsViewController`. Бейдж надо ставить на `navigationController?.tabBarItem = "3"` (или использовать функцию `updateUnreadBadge` из 26.2, которая это учитывает).

Что мы выучили

- **`UINavigationController`** — стопка экранов; у каждого экрана своя табличка `navigationItem` (`UINavigationItem → NSObject`, свойство view controller'а); `UIBarButtonItem` и `UITabBarItem` — `UIBarItem`, не view: панель рисует их сама.
- **Large titles** — `prefersLargeTitles` один раз на всю стопку, `largeTitleDisplayMode` на каждом экране; для сжатия панель должна знать список (`setContentScrollView`).
- **Бейдж вкладки** ставится на `tabBarItem` того контроллера, что лежит во вкладке; `0` → `nil`, больше 99 → «99+».
- **Step indicator** — точки с `CGAffineTransform` (1,4 = на 40% крупнее) или `UIProgressView` с дробным делением.
- **Breadcrumbs** — скролл с кнопками, возврат через `popToViewController`; текущий уровень неактивен.
- **Кнопка «назад» принадлежит предыдущему экрану**: для модального окна — своя `leftBarButtonItem`; для стопки — `backBarButtonItem`, `backButtonTitle`, `backButtonDisplayMode` в экране, куда вернёмся, и `setBackIndicatorImage` для стрелки, чтобы не потерять свайп назад.
- **Push или modal**: смотрю и возвращаюсь — push; выполняю задачу и закрываю — modal.
- **Custom transition** — аниматор с `finalFrame(for:)` и обязательным `completeTransition`; только для push, чтобы сохранить системный свайп назад.
- **Два уровня заголовка** — `navigationItem.subtitle` в iOS 26, стек в `titleView` для более ранних версий.
- **`UINavigationBarAppearance`** — задавай минимум `standardAppearance` и `scrollEdgeAppearance`, плюс `tintColor` для кнопок.

Apple Developer Documentation

- `UINavigationController` — стопка экранов с навигационной панелью.
- `UINavigationBar` — сама панель, `prefersLargeTitles`, слоты внешнего вида.
- `UINavigationBarAppearance` — описание внешнего вида; `configureWith…Background`, `setBackIndicatorImage(_:transitionMaskImage:)`.
- `UINavigationItem.largeTitleDisplayMode` — `.always` / `.never` / `.automatic`.
- `UIViewController.navigationItem` — табличка экрана: создаётся лениво, принадлежит view controller'у.
- `UINavigationItem.backBarButtonItem` — кнопка «назад», которую этот экран отдаёт следующему.

- `UINavigationItem.backButtonDisplayMode` — из чего собирать подпись кнопки «назад» (iOS 14+), ставится там же.
- `UINavigationItem.titleView` — слот для произвольной view на месте заголовка.
- `UINavigationItem.subtitle` — подзаголовок в панели (iOS 26+).
- `UIViewController.setContentScrollView(_:for:)` — какой список отслеживает панель (iOS 15+).
- `UITabBarItem.badgeValue` — текст бейджа вкладки.
- `UIViewControllerAnimatedTransitioning` — аниматор перехода с `animateTransition(using:)`.
- `UINavigationControllerDelegate` — выдаёт аниматоры для push и pop.
- `UIPercentDrivenInteractiveTransition` — интерактивный переход, управляемый пальцем.
- `UINavigationController.interactivePopGestureRecognizer` — системный свайп назад от края.
- HIG — Navigation bars — Apple про заголовки, кнопки и иерархию.
- HIG — Modality — когда показывать экран модально.

→ Глава 27. Cookbook — типы ячеек

Глава 27. Cookbook — типы ячеек

Экран настроек, профиль, форма заказа — всё это таблица из ячеек разных видов: с переключателем, со степпером, со слайдером, с выбором даты, с полем ввода. Эта глава — каталог таких ячеек: как сделать каждую и где у каждой прячутся ошибки.

> **В каком режиме код.** Листинги проверены компилятором в режиме Swift 6 с Default Actor Isolation = MainActor (подробнее — в начале главы 25). Короткие фрагменты вида `var content = ...` живут внутри `tableView(_:cellForRowAt:)`: там уже есть `cell` и `indexPath`.

Два понятия, на которых держится вся глава.

Ячейка (cell) — одна строка таблицы, объект `UITableViewCell`. По родословной это `UITableViewCell → UIView` напрямую: не `UIControl` (подсветку и выделение делает таблица, а не target-action) и не view controller. Внутри у ячейки несколько слоёв: снизу `backgroundView` и `selectedBackgroundView`, над ними `contentView` — единственное место для твоего содержимого, а справа может стоять **аксессуар** (accessory) — стрелка, галочка или любая твоя view (переключатель, кнопка). Аксессуар лежит **вне** `contentView`: когда он появляется или таблица входит в режим редактирования, ячейка сдвигает и сжимает именно `contentView`, поэтому своё добавляют только туда. Лестница классов — глава 0, раздел 0.5.

Переиспользование (reuse). Таблица не создаёт по ячейке на каждую строку данных. Ячеек примерно столько, сколько помещается на экране, плюс пара запасных. Строка уехала за верхний край — её ячейка отправляется в очередь, и таблица тут же берёт её для строки, появляющейся снизу. Это как гардероб с десятью номерками на сотню посетителей: номерок сдали — его тут же выдали новому гостю. Отсюда главное правило: в `cellForRowAt` **каждое** свойство ячейки, которое зависит от данных, надо выставлять заново, иначе ячейка покажет остатки прошлой строки. Подробно — глава 12.

27.1 Default content configuration

Когда применять. Стандартная ячейка: текст, подпись, иконка, стрелка справа. Самая частая.

```
var content = cell.defaultContentConfiguration()
content.text = "Уведомления"
content.secondaryText = "Включены"
content.image = UIImage(systemName: "bell.fill")
content.imageProperties.tintColor = .systemRed
content.imageProperties.maximumSize = CGSize(width: 28, height: 28)
content.textProperties.font = .preferredFont(forTextStyle: .body)
```

```
content.secondaryTextProperties.color = .secondaryLabel
content.prefersSideBySideTextAndSecondaryText = true
cell.contentConfiguration = content
cell.accessoryType = .disclosureIndicator
```

Content configuration (конфигурация содержимого, iOS 14+) — это **описание** того, что должно быть в ячейке, а не сами метки и картинки. Ты заполняешь структуру — текст, подпись, картинку, их цвета и шрифты, — отдаёшь её ячейке, и ячейка сама создаёт и раскладывает нужные view.

Разберём по строкам.

- `defaultContentConfiguration()` — заготовка, уже подходящая под стиль таблицы (отступы, шрифты по умолчанию).
- `var content` — это **структура**, то есть значение, а не ссылка. Изменения в `content` ничего не меняют на экране, пока ты не присвоишь её обратно: `cell.contentConfiguration = content`. Забыть эту строку — частая ошибка «я поменял текст, а он не поменялся».
- `imageProperties.maximumSize` — картинка не больше 28 × 28 точек. Без ограничения большая картинка раздует ячейку.
- `textProperties.font` — шрифт основного текста. Стиль `.body` — стиль **Dynamic Type**: он растёт вместе с системным размером текста, который человек выбирает в настройках.
- `prefersSideBySideTextAndSecondaryText = true` — основной текст слева, подпись справа, в одну строку, как в «Настройках». По умолчанию (`false`) подпись стоит **под** текстом. Если при крупном шрифте в строку не помещается, ячейка сама переставит подпись вниз.
- `accessoryType = .disclosureIndicator` — серая стрелка «›» справа: «тап откроет новый экран». Другие стандартные варианты: `.checkmark` (галочка), `.detailButton` (кнопка «i»), `.detailDisclosureButton` («i» и стрелка вместе), `.none`.

Что происходит при присваивании `contentConfiguration`: ячейка **заменяет** свой `contentView` на view конфигурации. На iOS 26.5 это видно напрямую — до присваивания `cell.contentView` имеет класс `UITableViewCellContentView`, после — `UIListContentView` с метками внутри. Поэтому свои subviews в `contentView` и конфигурацию в одной ячейке не смешивают: контейнер, в который ты их добавил, конфигурация подменит.

Этот API заменил старые свойства `cell.textLabel`, `cell.detailTextLabel` и `cell.imageView`. В заголовке `UITableViewCell.h` они помечены «this property will be deprecated in a future release». Смешивать старое и новое нельзя по той же причине: конфигурация подменяет содержимое ячейки и затрёт то, что ты написал в `textLabel`.

Есть и готовые варианты конфигурации для типовых ячеек: `UIListContentConfiguration.valueCell()` — текст слева, значение справа; `.subtitleCell()` — подпись под текстом. Их используют, когда ячейку создают без `defaultContentConfiguration()`.

Частые ошибки.

- **Изменили `content`, но не присвоили** его обратно в `cell.contentConfiguration`.
- **`cell.textLabel?.text = ...` вместе с конфигурацией** — одно перетирает другое.
- **Жёсткий размер шрифта** (`systemFont(ofSize: 16)`) — ячейка не реагирует на Dynamic Type.

27.2 Switch cell

Когда применять. Настройка «вкл/выкл»: уведомления, тёмная тема, автозагрузка.

```
var content = cell.defaultContentConfiguration()
content.text = "Уведомления"
cell.contentConfiguration = content

let toggle = UISwitch()
toggle.isOn = notificationsEnabled
toggle.addAction(UIAction { [weak self] action in
    guard let toggle = action.sender as? UISwitch else { return }
    self?.setNotificationsEnabled(toggle.isOn)
}, for: .valueChanged)
cell.accessoryView = toggle
cell.selectionStyle = .none
```

`accessoryView` — своя view на месте аксессуара; когда она задана, ячейка игнорирует `accessoryType`. `UISwitch` при создании сразу получает свой стандартный размер (на iOS 26.5 — 63 × 28 точек), поэтому его можно класть в аксессуар без настройки рамки.

`UIAction` — действие-замыкание, которое контрол вызовет при событии `.valueChanged` (переключатель сменил положение). `addAction(_:for:)` есть у `UISwitch` потому, что он `UIControl`; у `UILabel` или `UIImageView` такого метода нет (глава 0, раздел 0.4).

Самая важная строка — `action.sender as? UISwitch`. Кажется, проще написать `self?.setNotificationsEnabled` обратившись к `toggle` напрямую. Но тогда получится **цикл удержания** (retain cycle): переключатель держит своё действие, действие держит замыкание, а замыкание держит переключатель. Три объекта держат друг друга по кругу и никогда не освободятся. На каждую перерисовку строки в памяти будет оставаться по «мёртвому» переключателю. `action.sender` — это тот контрол, который вызвал действие, и через него мы получаем переключатель без сильной ссылки.

`[weak self]` — по той же причине, но для экрана: экран держит таблицу, таблица — ячейку, ячейка — переключатель, и дальше по цепочке до замыкания.

`selectionStyle = .none` — тап по самой строке ничего не делает (переключается только переключатель), поэтому строка не должна подсвечиваться при нажатии. Подробный пример экрана настроек с переключателями — глава 19 (Profile).

Частые ошибки.

- **Замыкание захватывает сам контрол** — цикл удержания. Бери контрол из `action.sender`.
- **Состояние переключателя не из данных.** `toggle.isOn` надо выставлять в `cellForRowAt` каждый раз из модели. Иначе при прокрутке переключатель «перепрыгнет» в чужое положение.

27.3 Stepper cell

Когда применять. Небольшое целое число: количество, уровень, число повторов. **Степпер** (`UIStepper`) — пара кнопок «−» и «+».

```
var content = cell.defaultContentConfiguration()
content.text = "Уровень"
content.secondaryText = "\(level)"
content.prefersSideBySideTextAndSecondaryText = true
cell.contentConfiguration = content

let stepper = UIStepper()
stepper.minimumValue = 0
stepper.maximumValue = 10
stepper.value = Double(level)
stepper.addAction(UIAction { [weak self, weak cell] action in
    guard let stepper = action.sender as? UIStepper, let cell else { return }
    let newLevel = Int(stepper.value)
    self?.level = newLevel
    var updated = content
    updated.secondaryText = "\(newLevel)"
    cell.contentConfiguration = updated
}, for: .valueChanged)
cell.accessoryView = stepper
cell.selectionStyle = .none
```

`value` у степпера — `Double`, поэтому на входе `Double(level)`, а на выходе `Int(stepper.value)`. Степпер шагает по 1 (`stepValue`), от 0 до 10 включительно.

Число справа («5») надо обновить при каждом нажатии. Первое, что приходит в голову, — перезагрузить строку (`reloadRows`). Но перезагрузка создаст **новый** степпер, а старый, под пальцем, уйдёт с экрана. Если человек зажал «+», чтобы число бежало само, это автоповторение оборвётся. Поэтому обновляем только текст: копируем конфигурацию (`var updated = content`), меняем подпись и присваиваем ячейке.

`[weak cell]` — замыкание не должно удерживать ячейку: ячейка держит степпер, степпер держит замыкание — снова круг.

27.4 Slider cell (полный label + slider)

Когда применять. Значение из непрерывного диапазона: размер шрифта, громкость, яркость. Слайдер занимает всю ширину, и в аксессуар он не помещается — нужна своя раскладка внутри ячейки.

Для такой ячейки правильнее завести свой класс:

```
final class SliderCell: UITableViewCell {
    static let reuseID = "SliderCell"
    var onChange: ((Float) -> Void)?

    private let titleLabel = UILabel()
    private let slider = UISlider()
    private var title = ""
```

```
    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        titleLabel.font = .preferredFont(forTextStyle: .body)
        titleLabel.adjustsFontForContentSizeCategory = true

        slider.addAction(UIAction { [weak self] _ in
            guard let self else { return }
            self.updateLabel()
            self.onChange?(self.slider.value)
        }, for: .valueChanged)

        let stack = UIStackView(arrangedSubviews: [titleLabel, slider])
        stack.axis = .vertical
        stack.spacing = 8
        stack.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(stack)
        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: margins.topAnchor),
            stack.bottomAnchor.constraint(equalTo: margins.bottomAnchor),
            stack.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func configure(title: String, value: Float, range: ClosedRange<Float>) {
        self.title = title
        slider.minimumValue = range.lowerBound
        slider.maximumValue = range.upperBound
        slider.value = value
        updateLabel()
    }

    override func prepareForReuse() {
        super.prepareForReuse()
        onChange = nil
    }

    private func updateLabel() {
        titleLabel.text = "\(title): \(Int(slider.value.rounded())) pt"
    }
}
```

Разбор:

- Слайдер и метка создаются **один раз**, в `init`. При переиспользовании ячейка приходит уже с ними, и `configure` только выставляет значения. Ничего не добавляется повторно.
- Стек прибит к `layoutMarginsGuide` — к **внутренним полям** ячейки. Это те же отступы, что у текста в стандартных ячейках (обычно 16–20 точек по бокам), поэтому слайдер ровно выстраивается с соседними строками.
- `slider.value` — `Float`. Метка показывает округлённое значение: при `value = 17.46` будет «Размер шрифта: 17 pt», при `17.5` — «18 pt» (`rounded()` округляет половину вверх). Без округления на экране мелькало бы «17.462963».
- `onChange` — замыкание, через которое ячейка сообщает экрану новое значение. Экран задаёт его в `cellForRowAt`.
- `prepareForReuse()` — таблица вызывает этот метод перед тем, как вернуть ячейку из `dequeueReusableCell`. Мы обнуляем `onChange`, чтобы старое замыкание (от прошлой строки) не сработало на новой. Документация просит сбрасывать здесь только то, что не относится к содержимому (alpha, выделение, вот такие колбэки), а само содержимое всегда выставлять заново в `cellForRowAt` — что мы и делаем в `configure`.
- В замыкании слайдера `[weak self]`: ячейка держит слайдер, слайдер — замыкание, и сильный `self` замкнул бы круг.

Использование в `cellForRowAt`:

```
let cell = tableView.dequeueReusableCell(withIdentifier: SliderCell.reuseID,
                                         for: indexPath) as! SliderCell
cell.configure(title: "Размер шрифта", value: fontSize, range: 12...28)
cell.onChange = { [weak self] value in
    self?.fontSize = value
}
return cell
```

`as! SliderCell` — принудительное приведение типа. Здесь оно оправдано: если ячейка под этим идентификатором не `SliderCell`, это ошибка программиста, и лучше упасть сразу при разработке. Класс регистрируется один раз в `viewDidLoad`: `tableView.register(SliderCell.self, forCellReuseIdentifier: SliderCell.reuseID)`.

Встречается и более короткий способ: в `cellForRowAt` обнулить `contentConfiguration` (чтобы ячейка вернула обычный `contentView`, см. 27.1), удалить из `contentView` все старые subviews и добавить новые. Он работает, но на каждую прокрутку создаёт и удаляет view, и легко забыть удаление — тогда при переиспользовании в ячейке окажется два слайдера друг на друге. Свой класс надёжнее.

Частые ошибки.

- **Замыкание слайдера захватывает слайдер** — цикл удержания (см. 27.2).
- **Добавление subviews в `cellForRowAt` без удаления** — наложение view при переиспользовании.
- **Не обнулили колбэк в `prepareForReuse`** — изменение слайдера в строке 5 записывается в данные строки 1.

Упражнение 27.1. Найди проблему в этом коде и исправь её:

```
let slider = UISlider()
slider.addAction(UIAction { _ in
```

```
    print(slider.value)
}, for: .valueChanged)
cell.accessoryView = slider
```

27.5 Inline picker (UIMenu)

Когда применять. Выбор из нескольких вариантов прямо в строке: тема оформления, язык, единицы измерения.

```
var content = cell.defaultContentConfiguration()
content.text = "Тема"
cell.contentConfiguration = content

let actions = themes.map { theme in
    UIAction(title: theme,
             state: theme == currentTheme ? .on : .off) { [weak self] _ in
        self?.setTheme(theme)
    }
}
var config = UIButton.Configuration.plain()
config.contentInsets = .zero
let button = UIButton(configuration: config)
button.menu = UIMenu(children: actions)
button.showsMenuAsPrimaryAction = true
button.changesSelectionAsPrimaryAction = true
button.sizeToFit()
cell.accessoryView = button
cell.selectionStyle = .none
```

Тап по кнопке открывает меню со всеми вариантами, текущий отмечен галочкой.

- `UIAction(title:state:)` — пункт меню. `.on` у текущей темы рисует галочку.
- `showsMenuAsPrimaryAction = true` — меню по обычному тапу, а не по долгому нажатию.
- `changesSelectionAsPrimaryAction = true` (iOS 15+) превращает кнопку во **всплывающую кнопку выбора** (pop-up button): она сама показывает заголовок выбранного пункта со значком «^⌄» и сама переносит галочку при выборе. Поэтому подпись `secondaryText` в ячейке не нужна — значение видно на кнопке.
- `sizeToFit()` — обязательно. `accessoryView` использует тот размер, который у view уже есть, а кнопка, созданная в коде, имеет нулевой размер. Без этой строки кнопка будет невидимой.

Частые ошибки.

- **Кнопка в аксессуаре без размера** — ячейка выглядит пустой, хотя код «точно добавил кнопку».
- **Галочка не переезжает.** Если собрать меню без `changesSelectionAsPrimaryAction` (или без опции `.singleSelection`, см. 25.6), состояние `state` зафиксировано в момент создания меню. После выбора меню надо пересобрать или перезагрузить строку.

27.6 Disclosure value cell

Когда применять. Строка, которая ведёт на другой экран и показывает текущее значение: «Язык › Русский», «О приложении › 1.0.3».

```
let version = Bundle.main.infoDictionary?["CFBundleShortVersionString"] as? String ?? "—"
var content = cell.defaultContentConfiguration()
content.text = "О приложении"
content.secondaryText = version
content.prefersSideBySideTextAndSecondaryText = true
cell.contentConfiguration = content
cell.accessoryType = .disclosureIndicator
```

`CFBundleShortVersionString` — номер версии из настроек проекта (Marketing Version), то самое «1.0.3», которое видно в App Store. Берём его из `Info.plist` приложения, а не пишем руками, иначе после обновления в ячейке останется старый номер. Если ключа нет, покажем тире.

Стрелка `.disclosureIndicator` обещает «тап откроет экран». Не ставь её на строки, которые ничего не открывают: человек будет тапать и удивляться.

27.7 Multi-line cell

Когда применять. Текст, который не помещается в одну строку: описание, адрес, отзыв.

```
var content = cell.defaultContentConfiguration()
content.text = "Доставка"
content.textProperties.numberOfLines = 0
content.secondaryText = "Курьер привезёт заказ завтра с 10:00 до 14:00. Перед приездом
↪  позвонит."
content.secondaryTextProperties.numberOfLines = 0
cell.contentConfiguration = content
```

`numberOfLines = 0` — «сколько угодно строк». По умолчанию у текста конфигурации тоже `0`, но явная строка защищает от сюрприза, если кто-то в другом месте ограничил число строк.

Высоту ячейки таблица вычислит сама: с iOS 11 у `UITableView` по умолчанию `rowHeight = UITableView.automaticDimension` — «высота по содержимому». Если высота всё равно не растёт, проверь, не задал ли кто-то `tableView.rowHeight = 44` или метод `heightForRowAt` с фиксированным числом.

27.8 Segmented cell

Когда применять. Выбор из 2–4 коротких вариантов, которые хочется видеть сразу: «День / Неделя / Месяц».

Сегментированный контрол (`UISegmentedControl`) — ряд кнопок-сегментов, из которых выбран ровно один.

```
final class SegmentedCell: UITableViewCell {
    static let reuseID = "SegmentedCell"
```

```
    var onChange: ((Int) -> Void)?
    private let segmented = UISegmentedControl()

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        segmented.addAction(UIAction { [weak self] action in
            guard let control = action.sender as? UISegmentedControl else { return }
            self?.onChange?(control.selectedSegmentIndex)
        }, for: .valueChanged)
        segmented.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(segmented)
        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            segmented.topAnchor.constraint(equalTo: margins.topAnchor),
            segmented.bottomAnchor.constraint(equalTo: margins.bottomAnchor),
            segmented.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            segmented.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func configure(items: [String], selected: Int) {
        segmented.removeAllSegments()
        for (index, title) in items.enumerated() {
            segmented.insertSegment(withTitle: title, at: index, animated: false)
        }
        segmented.selectedSegmentIndex = selected
    }

    override func prepareForReuse() {
        super.prepareForReuse()
        onChange = nil
    }
}
```

Та же схема, что у слайдера: контрол создаётся один раз, `configure` заново выставляет сегменты и выбранный индекс, `prepareForReuse` отвязывает колбэк. `removeAllSegments()` перед вставкой нужен, чтобы переиспользованная ячейка не набрала сегменты от прошлой строки плюс новые.

Сегменты делят ширину поровну: на iPhone шириной 393 точки с полями по 20 точек на три сегмента приходится (393 − 40) / 3 ≈ 117 точек каждому. «Неделя» помещается, «За последние 30 дней» — уже нет. Больше четырёх сегментов или длинные подписи — повод взять меню (27.5).

27.9 Picker cell (full width)

Когда применять. Выбор даты и времени в строке настроек или формы.

```
var content = cell.defaultContentConfiguration()
content.text = "Напомнить"
cell.contentConfiguration = content

let datePicker = UIDatePicker()
datePicker.datePickerMode = .dateAndTime
datePicker.preferredDatePickerStyle = .compact
datePicker.date = reminderDate
datePicker.addAction(UIAction { [weak self] action in
    guard let picker = action.sender as? UIDatePicker else { return }
    self?.reminderDate = picker.date
}, for: .valueChanged)
datePicker.sizeToFit()
cell.accessoryView = datePicker
cell.selectionStyle = .none
```

`UIDatePicker` умеет выглядеть по-разному, стиль задаёт `preferredDatePickerStyle` (iOS 13.4+):

- `.compact` (iOS 14+) — маленькие «таблетки» с датой и временем прямо в строке. Тап по таблетке открывает поверх экрана календарь или выбор времени. Лучший вариант для строки таблицы.
- `.inline` (iOS 14+) — полный календарь прямо на экране, занимает много места по высоте. Хорош, когда выбор даты — главное на экране.
- `.wheels` — классические «барабаны», которые крутят пальцем. Они тоже стоят прямо на экране (это не отдельное окно), около 200 точек в высоту.
- `.automatic` — стиль выбирает система.

`sizeToFit()` — подгоняем рамку под содержимое. Пикер, в отличие от кнопки из 27.5, получает размер уже при создании (на iOS 26.5 компактный пикер с датой и временем — около 200 × 36 точек; ширина зависит от локали и самой даты), но после смены режима и стиля рамку надёжнее пересчитать: аксессуар берёт тот размер, который у view есть в момент присваивания.

27.10 Inline editing cell

Когда применять. Короткое поле ввода прямо в строке: имя, никнейм, название — без отдельного экрана редактирования.

```
final class TextFieldCell: UITableViewCell {
    static let reuseID = "TextFieldCell"
    var onChange: ((String) -> Void)?

    private let titleLabel = UILabel()
    private let textField = UITextField()

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
```

```
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        selectionStyle = .none
        titleLabel.font = .preferredFont(forTextStyle: .body)
        textField.font = .preferredFont(forTextStyle: .body)
        textField.textAlignment = .right
        textField.clearButtonMode = .whileEditing
        textField.addAction(UIAction { [weak self] action in
            guard let field = action.sender as? UITextField else { return }
            self?.onChange?(field.text ?? "")
        }, for: .editingChanged)

        titleLabel.setContentHuggingPriority(.required, for: .horizontal)
        titleLabel.setContentCompressionResistancePriority(.required, for: .horizontal)
        textField.setContentHuggingPriority(.defaultLow, for: .horizontal)

        let stack = UIStackView(arrangedSubviews: [titleLabel, textField])
        stack.spacing = 16
        stack.translatesAutoresizingMaskIntoConstraints = false
        contentView.addSubview(stack)
        let margins = contentView.layoutMarginsGuide
        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: margins.topAnchor),
            stack.bottomAnchor.constraint(equalTo: margins.bottomAnchor),
            stack.leadingAnchor.constraint(equalTo: margins.leadingAnchor),
            stack.trailingAnchor.constraint(equalTo: margins.trailingAnchor),
        ])
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    func configure(title: String, text: String, placeholder: String) {
        titleLabel.text = title
        textField.text = text
        textField.placeholder = placeholder
    }

    override func prepareForReuse() {
        super.prepareForReuse()
        onChange = nil
    }
}
```

Метка слева, поле справа тянется до края. Тут работают два правила Auto Layout, которые часто путают.

Intrinsic content size («собственный размер») — размер, который view хочет иметь сама: метка «Имя» — ширину своего текста, кнопка — ширину заголовка с полями. Когда в стеке места

больше или меньше, чем сумма собственных размеров, Auto Layout решает, кого растянуть, а кого сжать. Для этого у каждой view две настройки:

- **Content hugging** («обнимание») — насколько view **не хочет растягиваться** больше собственного размера. Высокий приоритет: «обниму свой текст и не вырасту». Низкий: «могу растянуться».
- **Compression resistance** («сопротивление сжатию») — насколько view **не хочет сжиматься** меньше собственного размера. Высокий приоритет: «не обрежьте мне текст».

Приоритеты — числа от 1 до 1000: `.defaultLow` = 250, `.defaultHigh` = 750, `.required` = 1000.

В нашей ячейке ширина 353 точки (393 минус поля по 20). Метка «Имя» хочет, скажем, 40 точек, между ней и полем 16. Остаётся 297 точек, и Auto Layout решает, кому их отдать. У метки hugging 1000, у поля 250 — растягивается поле. А если поле с длинным текстом начнёт «давить», у метки compression resistance 1000 — она не сожмётся до «Им…».

Одно только hugging защищает от растягивания, но не от сжатия. Для «не сжимаемой» метки нужна именно compression resistance.

`textAlignment = .right` — текст в поле прижат к правому краю, как значения в «Настройках». `clearButtonMode = .whileEditing` — крестик «очистить» появляется во время ввода.

Частые ошибки.

- **Перепутали hugging и compression resistance** — метка обрезается при длинном вводе.
- **Данные не сохраняются при прокрутке.** Если не записывать текст в модель на каждое изменение (`.editingChanged`), то после прокрутки переиспользованная ячейка покажет старое значение из модели.

Упражнение 27.2. В ячейке `TextFieldCell` поменяй приоритеты местами: метке — hugging `.defaultLow`, полю — `.required`. Что изменится на экране? Опиши словами, без запуска.

27.11 Variable-height cells

Когда применять. Ячейки разной высоты: отзывы, сообщения, заметки.

```
override func viewDidLoad() {
    super.viewDidLoad()
    tableView.rowHeight = UITableView.automaticDimension
    tableView.estimatedRowHeight = 80
}
```

`rowHeight = automaticDimension` — высоту каждой ячейки вычисляет Auto Layout по её содержимому. С iOS 11 это значение по умолчанию, строка здесь для ясности. Чтобы расчёт сработал, содержимое ячейки должно быть прибито к `contentView` сверху и снизу непрерывной цепочкой ограничений (как стеки выше).

`estimatedRowHeight` — **прикидка** высоты строки. Таблица не меряет все ячейки заранее — это дорого. Она меряет только видимые, а для остальных берёт прикидку, чтобы вычислить общую высоту списка и размер полосы прокрутки. На числах: 1000 строк × 80 = 80 000 точек предполагаемой высоты. Если реальные ячейки в среднем по 200 точек, прикидка ошибает-

ся в 2,5 раза: полоса прокрутки будет «прыгать», а список — дёргаться при быстрой прокрутке. Ставь прикидку близко к средней реальной высоте.

27.12 Cell selection style

```
cell.selectionStyle = .none     // не подсвечивать при нажатии
cell.selectionStyle = .default  // системная подсветка (серая)
```

Для строк с переключателем, слайдером, полем ввода — `.none`: тап по строке ничего не делает. Для строк, которые открывают экран или выполняют действие, — `.default`.

В перечислении есть ещё `.blue` и `.gray`. Они не устарели формально, но `.blue` давно не синий — даёт обычную системную подсветку. На практике используют `.default` и `.none`.

Подсветка после тапа должна гаснуть. Если ты открываешь экран из `didSelectRowAt`, сними выделение сам:

```
override func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
    tableView.deselectRow(at: indexPath, animated: true)
    openDetails(for: indexPath)
}
```

У `UITableViewController` это частично происходит само (`clearsSelectionOnViewWillAppear`) при возврате назад, но у обычного `UIViewController` с таблицей строка останется серой.

Свой цвет подсветки:

```
let background = UIView()
background.backgroundColor = UIColor.systemBlue.withAlphaComponent(0.1)
cell.selectedBackgroundView = background
```

`selectedBackgroundView` — view, которую ячейка добавляет subview'ом только пока выделена: сразу над `backgroundView`, если он есть, иначе под всем остальным. `withAlphaComponent(0.1)` — синий на 10% непрозрачности: лёгкий голубой оттенок, через который виден текст. Эти две view и `backgroundConfiguration` взаимоисключающи: задал `selectedBackgroundView` — конфигурация фона сбросится в `nil`.

27.13 Avatar в ячейке

Когда применять. Список людей: контакты, участники чата, комментарии.

```
var content = cell.defaultContentConfiguration()
content.text = user.name
content.secondaryText = user.email
content.image = avatarCache[user.id] ?? UIImage(systemName: "person.crop.circle.fill")
content.imageProperties.maximumSize = CGSize(width: 36, height: 36)
content.imageProperties.reservedLayoutSize = CGSize(width: 36, height: 36)
content.imageProperties.cornerRadius = 18
cell.contentConfiguration = content
```

- `cornerRadius = 18` при размере 36 — половина стороны, поэтому картинка обрезается в круг.
- `reservedLayoutSize` — место под картинку резервируется всегда, даже пока аватар грузится. Иначе текст в ячейке «прыгнет» вправо, когда картинка появится.
- Пока настоящего фото нет, показываем системный символ-заглушку.

Настоящие фото грузятся из сети, и здесь главная ловушка переиспользования. Пока фото летит, человек прокрутил список, ячейку отдали другому пользователю — и пришедшее фото Анны окажется в строке Бориса. Надёжная схема: картинку кладём в кэш по идентификатору пользователя, а ячейку не трогаем напрямую — просим таблицу заново настроить строку, где этот пользователь сейчас:

```
func loadAvatarIfNeeded(for user: User) {
    guard avatarCache[user.id] == nil else { return }
    Task { [weak self] in
        guard let image = await AvatarLoader.image(for: user.avatarURL),
              let self else { return }
        avatarCache[user.id] = image
        if let row = users.firstIndex(where: { $0.id == user.id }) {
            tableView.reconfigureRows(at: [IndexPath(row: row, section: 0)])
        }
    }
}
```

`reconfigureRows(at:)` (iOS 15+) заново вызывает `cellForRowAt` для строки, у которой сейчас есть ячейка (видимой или предзагруженной), и `dequeueReusableCell` возвращает **ту же** ячейку — без `prepareForReuse` и без новой view. `cellForRowAt` возьмёт фото из `avatarCache`. Если у строки ячейки сейчас нет, вызов ничего не сделает — фото подхватится из кэша, когда строка снова появится.

Искать строку по `user.id` в момент прихода фото, а не запоминать `indexPath` заранее, нужно потому, что за время загрузки список мог измениться: строку удалили, добавили новую сверху.

Частые ошибки.

- **Фото кладут прямо в ячейку из замыкания загрузки** — фото оказывается у чужого пользователя.
- **`cell.imageView?.image = ...` при конфигурации** — конфигурация перетрёт картинку при ближайшем обновлении (см. 27.1).

27.14 Drag handles (editing mode)

Когда применять. Ручная сортировка: плейлист, порядок виджетов, избранное.

```
override func viewDidLoad() {
    super.viewDidLoad()
    tableView.isEditing = true
}

override func tableView(_ tableView: UITableView,
```

```
                        canMoveRowAt indexPath: IndexPath) -> Bool { true }

override func tableView(_ tableView: UITableView,
                        moveRowAt source: IndexPath, to destination: IndexPath) {
    let item = items.remove(at: source.row)
    items.insert(item, at: destination.row)
}

override func tableView(_ tableView: UITableView,
                        editingStyleForRowAt indexPath: IndexPath) ->
                         ↪  UITableViewCell.EditingStyle {
    .none
}

override func tableView(_ tableView: UITableView,
                      shouldIndentWhileEditingRowAt indexPath: IndexPath) -> Bool { false }
```

`isEditing = true` включает **режим редактирования**: справа у строк появляются «ручки» (≡), за которые строку можно перетащить.

- `canMoveRowAt` — какие строки можно двигать. Здесь все.
- `moveRowAt` — таблица уже переставила строку на экране, и ты обязан повторить перестановку в данных. Иначе при очередной перезагрузке строки вернутся на старые места.
- `editingStyleForRowAt` возвращает `.none` — без красного кружка «удалить» слева: только перестановка.
- `shouldIndentWhileEditingRowAt` — `false`, чтобы содержимое не сдвигалось вправо, освобождая место под кружок, которого нет.

Как `remove` + `insert` двигают элемент, на числах. Было `[A, B, C]`, тянем A (строка 0) на место C (строка 2). `remove(at: 0)` даёт `[B, C]`, `insert(A, at: 2)` — `[B, C, A]`. Именно это человек и видит на экране.

Ответы к упражнениям

27.1. Замыкание захватывает `slider` сильной ссылкой, а слайдер держит своё действие — цикл удержания, слайдер никогда не освободится (на симуляторе с iOS 26 это легко проверить слабой ссылкой: после выхода из области видимости она так и остаётся не `nil`). Исправление — взять слайдер из `action.sender` (а ещё лучше сделать ячейку своим классом, как `SliderCell`):

```
let slider = UISlider()
slider.addAction(UIAction { action in
    guard let slider = action.sender as? UISlider else { return }
    print(slider.value)
}, for: .valueChanged)
cell.accessoryView = slider
```

27.2. Теперь метка готова растягиваться (hugging 250), а поле — нет (hugging 1000). Всё свободное место отдаётся метке: «Имя» займёт почти всю ширину, а поле ввода сожмётся до соб-

ственного размера — ширины плейсхолдера или введённого текста — и прижмётся к правому краю. Тапать в узкое поле неудобно, и при пустом плейсхолдере его почти не видно.

Что мы выучили

- **Ячейка** — `UITableViewCell → UIView` (не `UIControl`); своё — только в `contentView`, аксессуар лежит вне его; `contentConfiguration` подменяет `contentView` на view конфигурации.
- **Переиспользование**: в `cellForRowAt` выставляй каждое зависящее от данных свойство заново; в своих классах ячеек отвязывай колбэки в `prepareForReuse`.
- **Content configuration** — описание содержимого (`text`, `secondaryText`, `image`, свойства); изменённую структуру присваивай обратно в `contentConfiguration`.
- **Контролы в аксессуаре** (`UISwitch`, `UIStepper`, `UIDatePicker`, кнопка): контрол бери из `action.sender`, иначе цикл удержания; view без собственного размера — `sizeToFit()`.
- **Слайдер, сегменты, поле ввода** — свой класс ячейки: view создаются один раз в `init`, `configure` только выставляет значения.
- **Меню выбора** — `changesSelectionAsPrimaryAction` (iOS 15+) делает кнопку, которая сама показывает выбранное.
- **Hugging** — «не растягиваться», **compression resistance** — «не сжиматься»; у метки рядом с полем нужны оба.
- **Высота по содержимому** — `automaticDimension` по умолчанию, `estimatedRowHeight` близко к средней реальной высоте.
- **Асинхронные аватары** — кэш по идентификатору и `reconfigureRows`, а не запись в ячейку из замыкания загрузки.
- **Перестановка строк** — `isEditing`, `moveRowAt` с обновлением данных, `.none` вместо кнопки удаления.

Apple Developer Documentation

- `UITableViewCell` — базовая ячейка таблицы, `accessoryView`, `selectionStyle`, `prepareForReuse()`; `contentView` — куда класть своё содержимое.
- `UIListContentConfiguration` — `defaultContentConfiguration()`, `valueCell()`, `subtitleCell()` (iOS 14+).
- `UIListContentConfiguration.ImageProperties` — `cornerRadius`, `maximumSize`, `reservedLayoutSize`, `tintColor`.
- `UICollectionViewListCell` — ячейка-строка для списков на основе collection view (iOS 14+).
- `UISwitch`, `UIStepper`, `UISlider`, `UISegmentedControl` — контролы для ячеек настроек.
- `UIDatePicker.Style` — `.compact`, `.inline`, `.wheels`, `.automatic`.
- `UIButton.changesSelectionAsPrimaryAction` — всплывающая кнопка выбора (iOS 15+).
- `UIView.setContentHuggingPriority(_:for:)` и `setContentCompressionResistancePriority(_:for:)` — кто растягивается, а кто не сжимается.
- `UITableView.automaticDimension` — высота ячейки по Auto Layout.
- `UITableView.reconfigureRows(at:)` — перенастроить видимые строки без пересоздания ячеек (iOS 15+).

→ Глава 28. Cookbook — модалки и листы

Глава 28. Cookbook — модалки и листы

Всё, что показывается **поверх** текущего экрана: лист снизу, окно на весь экран, поповер со стрелкой, системный алерт, собственное окно подтверждения, баннер сверху, короткое «Сохранено» по центру. Эта глава — каталог таких способов и ошибок, которые с ними случаются.

> **В каком режиме код.** Листинги проверены компилятором в режиме Swift 6 с Default Actor Isolation = MainActor и минимальной версией iOS 15 (подробнее — в начале главы 25). Поэтому всё, что появилось позже iOS 15, в коде обёрнуто в проверку `#available`.

Модальный показ (modal presentation) — открыть экран поверх текущего так, что пока его не закроют, работать можно только с ним. Показывает его метод `present(_:animated:)`, закрывает — `dismiss(animated:)`. Тот, кто показал, называется **presenting** (показывающий) экран, тот, кого показали, — **presented** (показанный). У обоих есть ссылки друг на друга: `presentedViewController` у показывающего и `presentingViewController` у показанного. Закрывать обязан показывающий; если вызвать `dismiss` у показанного экрана, UIKit по документации просто перешлёт вызов показывающему — поэтому `dismiss(animated: true)` внутри модального экрана работает.

Как именно модальный экран выглядит — лист, весь экран, поповер — задаёт его `modalPresentationStyle`. Значение по умолчанию — `.automatic`: с iOS 18 UIKit разворачивает его в `.formSheet`, в iOS 13–17 — в `.pageSheet`; на iPhone оба выглядят одинаково — листом. Читается свойство уже развёрнутым: у свежего `UIViewController` на iOS 26.5 оно отдаёт `.formSheet`, и сравнение `modalPresentationStyle == .automatic` даёт `false`.

Показом управляет отдельный объект — **presentation controller** (`UIPresentationController → NSObject`). Это не view и не view controller: у него нет своей рамки и жизненного цикла появления, он решает, куда поставить показанный экран, что сделать с нижним, затемнять ли фон. UIKit создаёт его сам под выбранный стиль; `sheetPresentationController` и `popoverPresentationController` из этой главы — его подклассы (глава 0, разделы 0.6 и 0.8).

28.1 Bottom sheet с детентами

Когда применять. «Лёгкая» задача, после которой хочется вернуться к тому, что было под ней: новая заметка, фильтр, выбор адреса. **Лист** (sheet) выезжает снизу, под ним виден краешек предыдущего экрана, а высоту листа можно менять, потянув за верхний край.

Детент (detent, дословно «фиксатор») — высота, на которой лист может остановиться. Похоже на выдвижной ящик с защёлками: его можно выдвинуть наполовину или до конца, и он «щёлкнет» в одном из этих положений.

```
let editor = EditorViewController()
let nav = UINavigationController(rootViewController: editor)

if let sheet = nav.sheetPresentationController {
    sheet.detents = [.medium(), .large()]
    sheet.prefersGrabberVisible = true
    sheet.preferredCornerRadius = 24
}

present(nav, animated: true)
```

`sheetPresentationController` (iOS 15+) — объект, который управляет листом (`UISheetPresentationController` → `UIPresentationController`, не view). По документации он есть у экрана со стилем показа `.pageSheet` или `.formSheet`, а стиль по умолчанию `.automatic` разворачивается в один из них, поэтому ничего дополнительно выставлять не нужно: у свежего `UIViewController` на iOS 26.5 свойство уже не `nil`. У экрана со стилем `.fullScreen` оно `nil`.

- `detents` — список допустимых высот. `[.medium(), .large()]` — примерно половина экрана и почти весь экран. `[.medium()]` — только половина. `.medium()` не работает в **компактной высоте** (iPhone повёрнут горизонтально): там лист сразу раскрывается полностью.
- `prefersGrabberVisible = true` — серая полоска-«ручка» сверху листа: подсказка, что его можно тянуть.
- `preferredCornerRadius` — радиус скругления верхних углов, 24 точки. Без него — системное значение.
- Экран обёрнут в `UINavigationController`, чтобы у листа была навигационная панель с кнопками «Отмена» и «Готово».

Ещё одно свойство часто понимают неправильно: `prefersScrollingExpandsWhenScrolledToEdge`. По умолчанию оно `true`. Это значит: если в листе есть список, лист стоит на `.medium`, а список прокручен к самому верху, то свайп вверх по списку сначала **раскрывает лист** до `.large`, и только потом начинает прокручивать список. Если лист должен оставаться на половине, а список — прокручиваться (например, лист поверх карты), ставь `false`.

Свои детенты (iOS 16+). В iOS 15 есть только `.medium()` и `.large()`. Произвольная высота появилась в iOS 16, и при минимальной версии iOS 15 её нужно обернуть в проверку:

```
if #available(iOS 16.0, *) {
    let small = UISheetPresentationController.Detent.Identifier("small")
    sheet.detents = [
        .custom(identifier: small) { _ in 200 },
        .custom { context in context.maximumDetentValue * 0.7 },
        .large(),
    ]
    sheet.largestUndimmedDetentIdentifier = small
} else {
    sheet.detents = [.medium(), .large()]
```

```
}
```

`.custom { ... }` — детент, высоту которого считает замыкание. Замыкание получает **контекст**, а в нём `maximumDetentValue` — самая большая возможная высота листа на этом экране.

- `{ _ in 200 }` — лист высотой 200 точек. Эти 200 точек отсчитываются **внутри безопасной области**: по документации Apple, у листа, прижатого к низу экрана, к ним добавляется нижний отступ (на iPhone с полоской «домой» — 34 точки). Твоему содержимому при этом достанутся ровно 200. Итоговая высота на экране зависит от версии iOS: на симуляторе iPhone 16 с iOS 26, где лист «парит» с отступами от краёв экрана, он получился около 224 точек.
- `maximumDetentValue * 0.7` — 70% максимальной высоты. Если максимум 800 точек, лист остановится на 560.
- `identifier` — имя детента. Оно нужно, чтобы ссылаться на него из других свойств. Параметр необязательный: без имени UIKit назначит детенту своё.

`largestUndimmedDetentIdentifier = small` — «до высоты `small` включительно фон не затемнять». Пока лист маленький, экран под ним остаётся ярким и **нажимаемым** — как лист с маршрутом поверх карты. Если лист поднять выше, фон затемнится.

`else` — на iOS 15 откатываемся к системным детентам. Без `#available` проект с минимальной версией iOS 15 просто не соберётся: компилятор знает, что `.custom` на iOS 15 нет.

Сменить высоту листа из кода — например, раскрыть его, когда человек начал вводить текст, — можно с анимацией:

```
sheet.animateChanges {
    sheet.selectedDetentIdentifier = .large
}
```

`selectedDetentIdentifier` — текущая высота. `animateChanges` проигрывает её смену плавно, а не рывком.

Защита от случайного закрытия. Лист закрывается свайпом вниз. Если в нём несохранённый текст, это обидно. Запрети свайп и спроси:

```
final class NoteEditorViewController: UIViewController,
                                       UIAdaptivePresentationControllerDelegate {
    private var hasChanges = false {
        didSet { isModalInPresentation = hasChanges }
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        navigationController?.presentationController?.delegate = self
    }

    func presentationControllerDidAttemptToDismiss(_ presentationController:
    ↪  UIPresentationController) {
        let alert = UIAlertController(title: nil, message: nil, preferredStyle:
        ↪  .actionSheet)
```

```
        alert.addAction(UIAlertAction(title: "Удалить изменения", style: .destructive) {
↪   [weak self] _ in
            self?.dismiss(animated: true)
        })
        alert.addAction(UIAlertAction(title: "Продолжить редактирование", style: .cancel))
        alert.popoverPresentationController?.sourceView = view
        alert.popoverPresentationController?.sourceRect = CGRect(x: view.bounds.midX, y:
↪   0, width: 1, height: 1)
        present(alert, animated: true)
    }
}
```

`isModalInPresentation = true` — «этот экран нельзя закрыть свайпом»: по документации UIKit игнорирует события вне границ экрана и запрещает интерактивное закрытие. Мы включаем его, только пока есть изменения (`didSet` у `hasChanges`). Лист при попытке свайпа упруго вернётся на место, а UIKit вызовет `presentationControllerDidAttemptToDismiss` — тут мы спрашиваем, что делать. В параметре метода приходит тот самый `UIPresentationController`, который управляет показом (см. начало главы).

Делегата назначаем у `navigationController?.presentationController`: показан был не сам редактор, а контроллер навигации вокруг него. Назначаем в `viewDidAppear` — к этому моменту экран уже внутри показанного контроллера.

Частые ошибки.

- **`.custom` детенты без `#available`** — проект с iOS 15 не собирается.
- **Ждут `viewWillAppear` у экрана под листом.** Когда лист закрывается, экран под ним **не** получает `viewWillAppear` и `viewDidAppear`: он всё время оставался на экране, просто частично закрытый. Документация `.pageSheet` и `.formSheet` говорит это прямо: где нижний экран остаётся видимым, его `viewWillDisappear` и `viewDidDisappear` не вызываются (а значит, и парные «appear» при закрытии). Если после закрытия листа нужно обновить данные, сообщи об этом явно — делегатом или замыканием `onSave`.
- **Лист с несохранённым вводом закрывается свайпом** — нет `isModalInPresentation`.

28.2 Full-screen modal

Когда применять. Экран, который должен полностью отгородить человека от остального приложения: вход в аккаунт, обязательное обновление, просмотр фото.

```
let login = LoginViewController()
let nav = UINavigationController(rootViewController: login)
nav.modalPresentationStyle = .fullScreen
present(nav, animated: true)
```

`.fullScreen` — модальный экран занимает весь экран, а view нижнего экрана после окончания анимации **убираются** из иерархии (так в документации стиля). Поэтому у нижнего экрана вызываются `viewWillDisappear` и `viewDidDisappear`, а при закрытии — `viewWillAppear` и `viewDidAppear`. Свайпом вниз такой экран не закрывается: нужна кнопка.

`.overFullScreen` — тоже на весь экран, но view нижнего экрана **остаются** в иерархии, под модальным. Если у модального экрана прозрачный или полупрозрачный фон, нижний будет виден сквозь него. Методы `viewWillDisappear` у нижнего экрана при этом не вызываются. Этот стиль нужен для затемнений и своих окон подтверждения (28.9).

За «убрать или оставить» отвечает presentation controller: у него есть свойство `shouldRemovePresentersView`. На iOS 26.5 оно `true` только у контроллера стиля `.fullScreen`; у `.pageSheet`, `.formSheet`, `.automatic` и `.overFullScreen` — `false`, и у базового `UIPresentationController` по умолчанию тоже `false`.

Сравнение стилей (по документации `UIModalPresentationStyle` и проверке на iOS 26.5):

Стиль	Что под ним	`viewWillDisappear` нижнего	Свайп вниз
`.automatic`	то же, что `.formSheet` (iOS 18+) или `.pageSheet` (до 18)	нет	закрывает
`.pageSheet`	iPhone: нижний виден полоской сверху; iPad: карточка размером со страницу поверх затемнения	нет *	закрывает
`.formSheet`	iPhone: тот же лист; iPad: окно по центру, размер — `preferredContentSize`	нет *	закрывает
`.fullScreen`	убран из иерархии	да	нет
`.overFullScreen`	остаётся, виден сквозь прозрачные места	нет	нет
`.popover`	iPad: облачко со стрелкой; iPhone: ведёт себя как `.formSheet`	нет	iPhone: закрывает
`.custom`	решает твой presentation controller (`shouldRemovePresentersView`, по умолчанию `false`)	нет по умолчанию	нет

* В **компактной высоте** (iPhone, повёрнутый горизонтально) `.pageSheet` и `.formSheet` по документации ведут себя как `.fullScreen`: нижний экран убирается, и `viewWillDisappear` у него вызывается. Если логика экрана опирается на эти вызовы, проверь её и в ландшафте. Свайп вниз у листов отключает `isModalInPresentation` (28.1).

28.3 Page sheet и стиль по умолчанию

```
vc.modalPresentationStyle = .pageSheet
```

С iOS 13 модальные экраны по умолчанию показываются не на весь экран, а **листом**: новый экран выезжает снизу, нижний слегка уменьшается и виден полоской сверху, закрыть можно свайпом вниз. Стиль по умолчанию — `.automatic`, и документация говорит точно, во что он разворачивается: в iOS 18 и новее — в `.formSheet`, в iOS 13–17 — в `.pageSheet`; некоторые системные контроллеры выбирают своё (у `UIAlertController` это `.custom` и `.popover`). На iPhone оба листа выглядят одинаково (см. 28.4). Разница заметна в коде: свойство отдаёт уже развёрнутое значение, так что на iOS 26.5 у свежего view controller'а `modalPresentationStyle == .formSheet`, и проверка «стиль ещё `.automatic`?» всегда ложна.

`.pageSheet` — это тот же лист, что в 28.1. Детенты — просто его настройки: без них лист раскрыт на `.large`.

28.4 Form sheet (iPad)

```
vc.modalPresentationStyle = .formSheet
vc.preferredContentSize = CGSize(width: 540, height: 620)
```

На iPad `.formSheet` — окно по центру экрана поверх приглушённого фона (в iOS 26 это слой с эффектом Liquid Glass), как диалог настроек; по умолчанию оно меньше, чем `.pageSheet`. На iPhone разницы с `.pageSheet` нет: окну некуда «встать по центру», и документация описывает для компактной ширины тот же лист с полоской нижнего экрана сверху.

`preferredContentSize` — желаемый размер окна, 540 × 620 точек. iPad учитывает его для `.formSheet` (это документированный способ задать размер), а iPhone игнорирует.

28.5 Popover

Когда применять. Небольшое окошко рядом с конкретной кнопкой: выбор цвета, короткие настройки. **Поповер** (popover) — «облачко» со стрелкой, указывающей на кнопку, из которой оно выросло. Это типичный элемент iPad.

```
let picker = ColorPickerViewController()
picker.modalPresentationStyle = .popover
picker.preferredContentSize = CGSize(width: 280, height: 320)

if let popover = picker.popoverPresentationController {
    popover.sourceView = colorButton
    popover.sourceRect = colorButton.bounds
    popover.permittedArrowDirections = [.up, .down]
}

present(picker, animated: true)
```

- `sourceView` и `sourceRect` — откуда растёт стрелка: view и прямоугольник в её координатах. `colorButton.bounds` — вся кнопка.
- `permittedArrowDirections` — куда может смотреть стрелка. `[.up, .down]` — поповер встанет под кнопкой или над ней, но не сбоку.
- `preferredContentSize` — размер облачка, 280 × 320 точек.

На iPhone экран узкий, и поповер по умолчанию **адаптируется**: документация говорит, что с iOS 13 в компактной ширине или высоте `.popover` ведёт себя как `.formSheet`, то есть показывается не облачком, а модальным листом. Это нормальное поведение, и часто оно лучше облачка. С iOS 15 этот лист можно настроить, например дать ему половинную высоту:

```
if let popover = picker.popoverPresentationController {
    let sheet = popover.adaptiveSheetPresentationController
    sheet.detents = [.medium()]
```

```
    sheet.prefersGrabberVisible = true
}
```

`adaptiveSheetPresentationController` — тот лист, в который поповер превратится на iPhone. На iPad эти настройки ни на что не влияют.

Если нужно **настоящее** облачко и на iPhone, запрети адаптацию через делегата:

```
extension PaletteViewController: UIPopoverPresentationControllerDelegate {
    func adaptivePresentationStyle(for controller: UIPresentationController,
                                   traitCollection: UITraitCollection) ->
                                    ↪ UIModalPresentationStyle {
        .none   // не адаптировать: остаться поповером
    }
}

// при показе:
// picker.popoverPresentationController?.delegate = self
```

`adaptivePresentationStyle` — UIKit спрашивает: «экран стал узким, во что превратить поповер?». `.none` означает «ни во что, оставь как есть» — документация разрешает это значение только как ответ отсюда, назначать его в `modalPresentationStyle` нельзя. Делегат хранится слабой ссылкой (`weak` в `UIPopoverPresentationController.h`), поэтому им обычно делают сам показывающий экран.

Частые ошибки.

- **Нет `sourceView`** — на iPad показ поповера падает с ошибкой: UIKit не знает, куда направить стрелку.
- **Облачко на iPhone с большим содержимым.** Поповер 280 × 320 на экране шириной 393 — нормально, 380 × 600 — уже почти лист, только неудобный. Не запрещай адаптацию без нужды.

28.6 UIAlertController — alert

Когда применять. Подтверждение опасного действия, сообщение об ошибке, выбор из двух-трёх вариантов. **Алерт** — системное окошко по центру экрана, которое нельзя не заметить.

```
let alert = UIAlertController(
    title: "Удалить заметку?",
    message: "Это действие нельзя отменить.",
    preferredStyle: .alert
)
alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
alert.addAction(UIAlertAction(title: "Удалить", style: .destructive) { [weak self] _ in
    self?.deleteNote()
})
present(alert, animated: true)
```

Три стиля кнопки:

- `.cancel` — отмена. Выделяется системно и одна на алерт: на второй кнопке `.cancel` метод `addAction` бросает исключение `NSInternalInconsistencyException` «UIAlertController can only have one action with a style of UIAlertActionStyleCancel» (проверено на iOS 26.5), и приложение падает.
- `.destructive` — красная подпись: действие удаляет или необратимо меняет данные.
- `.default` — обычная кнопка.

Где окажется «Отмена», решает iOS. Когда кнопок две, они стоят рядом, и отмена — **слева**. Когда кнопок три и больше, они выстраиваются столбиком, и отмена — **внизу**. Порядок вызовов `addAction` на положение отмены не влияет.

`[weak self]` в замыкании кнопки: алерт показан экраном, и сильная ссылка удержала бы экран, пока жив алерт. Утечки здесь обычно нет (алерт закроется), но привычка брать `weak` в замыканиях UIKit избавляет от раздумий «а тут можно?».

Если одна из кнопок — ожидаемый ответ, выдели её: `alert.preferredAction = someAction`. Её подпись станет жирной (если есть кнопка «Отмена», выделение переедет с неё), и она сработает по клавише Return на подключённой клавиатуре. Два условия из документации: действие должно быть уже добавлено через `addAction` (назначить его раньше — ошибка программиста), и работает это только для стиля `.alert`, action sheet свойство игнорирует.

Формулировка по рекомендациям Apple (HIG): заголовок — короткий вопрос о действии («Удалить заметку?»), кнопка — глагол этого действия («Удалить»), а не «Да» и «ОК».

28.7 UIAlertController — action sheet

Когда применять. Выбор из нескольких действий над объектом: «Поделиться», «Сохранить», «Удалить».

```
let sheet = UIAlertController(title: nil, message: "Что сделать с фото?",
                              preferredStyle: .actionSheet)
sheet.addAction(UIAlertAction(title: "Поделиться", style: .default))
sheet.addAction(UIAlertAction(title: "Сохранить", style: .default))
sheet.addAction(UIAlertAction(title: "Удалить", style: .destructive))
sheet.addAction(UIAlertAction(title: "Отмена", style: .cancel))

sheet.popoverPresentationController?.sourceView = moreButton
sheet.popoverPresentationController?.sourceRect = moreButton.bounds

present(sheet, animated: true)
```

На iPhone action sheet выезжает снизу, «Отмена» — отдельной кнопкой в самом низу. На iPad он показывается **поповером** рядом с кнопкой, а кнопки «Отмена» там нет вовсе: чтобы отменить, тапают мимо облачка.

Две строки с `popoverPresentationController` на iPad **обязательны**: без источника показ action sheet падает. Почему: у `UIAlertController` со стилем `.actionSheet` стиль показа — `.popover` (так читается `modalPresentationStyle` на iOS 26.5), а поповеру нужно знать, откуда расти. На iPhone свойство тоже **не** `nil` — объект есть, просто в компактной ширине поповер адаптируется в лист

снизу, и источник ему не нужен; строки безвредны. Так что пиши их всегда, даже если пока тестируешь только на iPhone.

28.8 Alert с UITextField

Когда применять. Короткий ввод без отдельного экрана: название папки, переименование.

```
let alert = UIAlertController(title: "Новая папка", message: nil, preferredStyle: .alert)
alert.addTextField { field in
    field.placeholder = "Название"
    field.autocapitalizationType = .sentences
    field.returnKeyType = .done
}

let create = UIAlertAction(title: "Создать", style: .default) { [weak self, weak alert] _
↪  in
    let name = alert?.textFields?.first?.text ?? ""
    self?.createFolder(named: name)
}
create.isEnabled = false
alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
alert.addAction(create)
alert.preferredAction = create

alert.textFields?.first?.addAction(UIAction { [weak create] action in
    let text = (action.sender as? UITextField)?.text ?? ""
    create?.isEnabled = !text.trimmingCharacters(in: .whitespaces).isEmpty
}, for: .editingChanged)

present(alert, animated: true)
```

- `addTextField` добавляет поле в алерт. Замыкание настраивает его: плейсхолдер, заглавная первая буква (`.sentences`), кнопка «Готово» на клавиатуре.
- Доступ к полю — через `alert.textFields`. В замыкании кнопки `alert` захвачен **слабо** (`weak alert`): алерт держит кнопку, кнопка — замыкание, и сильный захват алерта замкнул бы круг.
- `create.isEnabled = false` — пока поле пустое, «Создать» неактивна. Действие на `.editingChanged` включает кнопку, как только в поле появился хотя бы один непробельный символ. Папку с именем из трёх пробелов создать не получится.

28.9 Custom alert (без UIAlertController)

Когда применять. Стандартный алерт не подходит: нужна картинка, фирменные цвета, нестандартная раскладка.

Своё окно лучше делать **отдельным экраном**, показанным поверх остальных, а не view, добавленной в текущий экран. Тогда оно закроет и навигационную панель, и вкладки, и сработает из любого места:

```
final class ConfirmViewController: UIViewController {
    var onConfirm: (() -> Void)?
    private let card = UIView()

    init() {
        super.init(nibName: nil, bundle: nil)
        modalPresentationStyle = .overFullScreen
        modalTransitionStyle = .crossDissolve
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = UIColor.black.withAlphaComponent(0.4)
        view.accessibilityViewIsModal = true

        card.backgroundColor = .systemBackground
        card.layer.cornerRadius = 20
        card.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(card)

        let icon = UIImageView(image: UIImage(systemName: "gift.fill"))
        icon.tintColor = .systemPink
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 44)
        let label = UILabel()
        label.text = "Забрать бонус 500 ₸?"
        label.font = .preferredFont(forTextStyle: .headline)
        label.numberOfLines = 0
        label.textAlignment = .center
        let button = UIButton(configuration: .filled())
        button.setTitle("Забрать", for: .normal)
        button.addAction(UIAction { [weak self] _ in
            self?.dismiss(animated: true) { self?.onConfirm?() }
        }, for: .touchUpInside)

        let stack = UIStackView(arrangedSubviews: [icon, label, button])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 16
        stack.translatesAutoresizingMaskIntoConstraints = false
        card.addSubview(stack)

        NSLayoutConstraint.activate([
            card.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            card.centerYAnchor.constraint(equalTo: view.centerYAnchor),
            card.widthAnchor.constraint(equalTo: view.widthAnchor, multiplier: 0.8),
            stack.topAnchor.constraint(equalTo: card.topAnchor, constant: 24),
```

```
            stack.bottomAnchor.constraint(equalTo: card.bottomAnchor, constant: -24),
            stack.leadingAnchor.constraint(equalTo: card.leadingAnchor, constant: 20),
            stack.trailingAnchor.constraint(equalTo: card.trailingAnchor, constant: -20),
        ])
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        card.transform = CGAffineTransform(scaleX: 0.9, y: 0.9)
        UIView.animate(withDuration: 0.35, delay: 0,
                       usingSpringWithDamping: 0.7, initialSpringVelocity: 0) {
            self.card.transform = .identity
        }
    }
}
```

Показ: `present(ConfirmViewController(), animated: true)`.

Разбор:

- `.overFullScreen` — экран поверх всего, а нижний остаётся виден (28.2). Полупрозрачный чёрный фон (`alpha 0.4` — 40% черноты) затемняет его.
- `.crossDissolve` — появление «проявлением», без выезда снизу.
- Карточка по центру шириной 80% экрана: на iPhone шириной 393 точки это около 314 точек. Высоту карточке задаёт стек через отступы сверху и снизу.
- Анимация карточки: стартуем с 90% размера (`scaleX: 0.9`) и пружиной возвращаемся к 100%. `usingSpringWithDamping: 0.7` — «чуть перелетит 100% и вернётся»: карточка на мгновение станет немного больше и тут же сядет на место. Длительность 0,35 с.
- `onConfirm` вызывается **после** закрытия (в completion `dismiss`), чтобы новый экран, если действие его откроет, не столкнулся с ещё не закрытым окном.
- `accessibilityViewIsModal = true` — VoiceOver не «провалится» в элементы экрана под окном, а будет читать только его содержимое.

Частые ошибки.

- **Окно как view внутри текущего экрана.** Не закрывает навигационную панель и вкладки, а при повороте рамка затемнения, заданная через `view.bounds`, остаётся старого размера.
- **Нет способа закрыть**, кроме главной кнопки. Добавь «Не сейчас» или закрытие по тапу на затемнение.

28.10 In-app banner

Когда применять. Ненавязчивое сообщение сверху, которое не блокирует работу: «Новое сообщение», «Загрузка завершена», «Нет соединения».

```
extension UIViewController {
    func showBanner(_ text: String, duration: TimeInterval = 3) {
        guard let window = view.window else { return }
```

```
        let label = UILabel()
        label.text = text
        label.textColor = .white
        label.numberOfLines = 0
        label.font = .preferredFont(forTextStyle: .subheadline)

        let banner = UIView()
        banner.backgroundColor = .systemBlue
        banner.layer.cornerRadius = 14
        banner.translatesAutoresizingMaskIntoConstraints = false
        label.translatesAutoresizingMaskIntoConstraints = false
        banner.addSubview(label)
        window.addSubview(banner)

        NSLayoutConstraint.activate([
            label.topAnchor.constraint(equalTo: banner.topAnchor, constant: 12),
            label.bottomAnchor.constraint(equalTo: banner.bottomAnchor, constant: -12),
            label.leadingAnchor.constraint(equalTo: banner.leadingAnchor, constant: 16),
           label.trailingAnchor.constraint(equalTo: banner.trailingAnchor, constant: -16),
            banner.topAnchor.constraint(equalTo: window.safeAreaLayoutGuide.topAnchor,
↪   constant: 8),
            banner.leadingAnchor.constraint(equalTo: window.leadingAnchor, constant: 12),
            banner.trailingAnchor.constraint(equalTo: window.trailingAnchor, constant:
↪   -12),
        ])
        window.layoutIfNeeded()

        let hiddenOffset = -(banner.frame.maxY + 20)
        banner.transform = CGAffineTransform(translationX: 0, y: hiddenOffset)
        UIView.animate(withDuration: 0.4, delay: 0,
                       usingSpringWithDamping: 0.8, initialSpringVelocity: 0) {
            banner.transform = .identity
        }
        UIAccessibility.post(notification: .announcement, argument: text)

        DispatchQueue.main.asyncAfter(deadline: .now() + duration) {
            UIView.animate(withDuration: 0.3, animations: {
                banner.transform = CGAffineTransform(translationX: 0, y: hiddenOffset)
            }, completion: { _ in
                banner.removeFromSuperview()
            })
        }
    }
}
```

Баннер выезжает сверху, три секунды висит и уезжает обратно.

- Баннер добавляется в **окно** (`view.window`, а окно — тоже view, `UIWindow → UIView`), а не в экран: новый subview ложится поверх всего, что в окне сейчас, — навигационной панели, вкладок и уже показанных модальных экранов. Экран, который покажут модально **после**, лягет уже над баннером.
- Ограничения: баннер на 8 точек ниже **безопасной области** (safe area — часть экрана, не занятая «чёлкой», Dynamic Island и статус-баром), с полями по 12 точек слева и справа. Высота — по тексту: метка многострочная, и длинное сообщение сделает баннер выше.
- `window.layoutIfNeeded()` — просим Auto Layout посчитать рамки **сейчас**, чтобы узнать, где баннер окажется.
- `hiddenOffset` — насколько сдвинуть баннер вверх, чтобы он целиком ушёл за экран. На числах: безопасная область на iPhone 16 начинается примерно на 59 точках, плюс 8 — баннер начинается на 67; при высоте 42 (однострочный текст, замерено на симуляторе) его нижний край —
  109. Сдвиг на −(109 + 20) = −129 точек уводит его с запасом за верх экрана. Число 100 «на глаз», как часто пишут, оставило бы нижний край баннера видимым.
- Пружина с гашением 0,8 — баннер выезжает с едва заметным «доводом», без раскачки.
- `UIAccessibility.post(notification: .announcement, ...)` — VoiceOver прочитает текст баннера вслух. Без этого незрячий человек о нём не узнает.
- `DispatchQueue.main.asyncAfter` — через `duration` секунд запустить исчезновение. Замыкание держит `banner` до конца анимации, а после `removeFromSuperview()` баннер освобождается.

Частые ошибки.

- **Сдвиг «на −100»** — баннер не прячется целиком на телефонах с высоким статус-баром.
- **Баннеры наслаиваются.** Если показать три подряд, они лягут друг на друга. Для частых сообщений заведи очередь: новый баннер — после ухода предыдущего.

28.11 HUD — Heads-Up Display

Когда применять. Короткое подтверждение: «Сохранено», «Скопировано». **HUD** — блок с иконкой и текстом по центру экрана, который сам исчезает через секунду-две. Название пришло из авиации: так называют данные, выведенные прямо на стекло перед пилотом.

```
extension UIViewController {
    func showHUD(_ text: String, symbol: String = "checkmark.circle.fill") {
        let hud = UIView()
        hud.backgroundColor = UIColor.black.withAlphaComponent(0.85)
        hud.layer.cornerRadius = 18
        hud.translatesAutoresizingMaskIntoConstraints = false

        let icon = UIImageView(image: UIImage(systemName: symbol))
        icon.tintColor = .systemGreen
        icon.preferredSymbolConfiguration = UIImage.SymbolConfiguration(pointSize: 40)
        let label = UILabel()
        label.text = text
        label.textColor = .white
        label.font = .preferredFont(forTextStyle: .headline)
```

```
        let stack = UIStackView(arrangedSubviews: [icon, label])
        stack.axis = .vertical
        stack.alignment = .center
        stack.spacing = 12
        stack.translatesAutoresizingMaskIntoConstraints = false
        hud.addSubview(stack)
        view.addSubview(hud)

        NSLayoutConstraint.activate([
            stack.topAnchor.constraint(equalTo: hud.topAnchor, constant: 24),
            stack.bottomAnchor.constraint(equalTo: hud.bottomAnchor, constant: -24),
            stack.leadingAnchor.constraint(equalTo: hud.leadingAnchor, constant: 32),
            stack.trailingAnchor.constraint(equalTo: hud.trailingAnchor, constant: -32),
            hud.centerXAnchor.constraint(equalTo: view.centerXAnchor),
            hud.centerYAnchor.constraint(equalTo: view.centerYAnchor),
        ])

        hud.alpha = 0
        UIView.animate(withDuration: 0.2) { hud.alpha = 1 }
        UIAccessibility.post(notification: .announcement, argument: text)
        DispatchQueue.main.asyncAfter(deadline: .now() + 1.2) {
            UIView.animate(withDuration: 0.3, animations: { hud.alpha = 0 }) { _ in
                hud.removeFromSuperview()
            }
        }
    }
}
```

Хронология на числах: 0,2 с проявление, около 1 с HUD виден полностью, в 1,2 с начинается исчезновение, к 1,5 с его уже нет. Этого хватает, чтобы прочитать одно слово, и не мешает продолжать работу.

HUD не принимает ввод и не требует нажатия. Если на подтверждение нужно ответить — это уже алерт (28.6).

Упражнение 28.1. Поменяй в `showBanner` расчёт `hiddenOffset` на `-100` и запусти на iPhone 16 с двухстрочным текстом баннера (высота около 64 точек). Виден ли баннер до и после анимации? Посчитай, какой сдвиг нужен в действительности.

28.12 Modal flow с UIViewControllerTransitioningDelegate

Когда применять. Своя анимация модального показа: проявление, выезд сбоку, «рост» из миниатюры.

Схема похожа на переходы в стопке навигации (26.7): нужен **аниматор**, который умеет анимировать, и **делегат**, который выдаёт аниматор по запросу UIKit.

```
final class FadeTransition: NSObject, UIViewControllerAnimatedTransitioning {
    let isPresenting: Bool
```

```
    init(isPresenting: Bool) { self.isPresenting = isPresenting }

    func transitionDuration(using ctx: UIViewControllerContextTransitioning?) ->
    ↪  TimeInterval {
        0.3
    }

    func animateTransition(using ctx: UIViewControllerContextTransitioning) {
        let duration = transitionDuration(using: ctx)
        if isPresenting {
            guard let toVC = ctx.viewController(forKey: .to),
                  let toView = ctx.view(forKey: .to) else {
                ctx.completeTransition(false)
                return
            }
            toView.frame = ctx.finalFrame(for: toVC)
            toView.alpha = 0
            ctx.containerView.addSubview(toView)
            UIView.animate(withDuration: duration, animations: {
                toView.alpha = 1
            }, completion: { _ in
                ctx.completeTransition(!ctx.transitionWasCancelled)
            })
        } else {
            guard let fromView = ctx.view(forKey: .from) else {
                ctx.completeTransition(false)
                return
            }
            UIView.animate(withDuration: duration, animations: {
                fromView.alpha = 0
            }, completion: { _ in
                ctx.completeTransition(!ctx.transitionWasCancelled)
            })
        }
    }
}

final class FadeTransitioningDelegate: NSObject, UIViewControllerTransitioningDelegate {
    func animationController(forPresented presented: UIViewController,
                             presenting: UIViewController,
                             source: UIViewController) ->
                              ↪  UIViewControllerAnimatedTransitioning? {
        FadeTransition(isPresenting: true)
    }

    func animationController(forDismissed dismissed: UIViewController)
        -> UIViewControllerAnimatedTransitioning? {
        FadeTransition(isPresenting: false)
```

```
    }
}
```

Один аниматор на обе стороны: флаг `isPresenting` говорит, показываем мы экран или закрываем.

- При показе новый экран получает итоговую рамку (`finalFrame`), становится прозрачным, добавляется на сцену и за 0,3 с проявляется.
- При закрытии уходящий экран за 0,3 с растворяется. Удалять его из иерархии вручную не нужно — это сделает UIKit после `completeTransition`.
- В обеих ветках `guard` в случае неудачи **всё равно** вызывает `completeTransition(false)`. Если просто выйти через `return`, переход никогда не закончится, и приложение перестанет реагировать на касания.

Делегат отвечает на два вопроса UIKit: «чем анимировать показ?» (`forPresented`) и «чем анимировать закрытие?» (`forDismissed`).

Подключение:

```
final class GalleryViewController: UIViewController {
    private let fadeDelegate = FadeTransitioningDelegate()

    func openPhoto() {
        let photo = UIViewController()
        photo.view.backgroundColor = .black
        photo.modalPresentationStyle = .custom
        photo.transitioningDelegate = fadeDelegate
        present(photo, animated: true)
    }
}
```

Главная ловушка — в первой строке класса. `transitioningDelegate` хранится **слабой** ссылкой (`weak` в `UIViewController.h`). Если написать `photo.transitioningDelegate = FadeTransitioningDelegate()`, объект делегата никто не удержит, он исчезнет сразу после присваивания, и UIKit покажет стандартную анимацию. Компилятор Swift 6.3 это замечает и выдаёт предупреждение «weak reference will always be nil because the referenced object is deallocated here» — но только предупреждение, код собирается и запускается. Поэтому делегат лежит в свойстве показывающего экрана.

`.custom` — «стандартного оформления показа не надо, анимирую сам». Документация добавляет: при этом стиле UIKit берёт у `transitioningDelegate` не только аниматоры, но и presentation controller; если делегат его не отдаёт (как наш), используется базовый `UIPresentationController`, у которого `shouldRemovePresentersView == false`. Поэтому нижний экран остаётся в иерархии, как при `.overFullScreen`.

Через эту схему делают hero-анимацию, выезд с любой стороны, переворот карточки — всё, что можно описать анимацией двух view на одной сцене.

Частые ошибки.

- **Делегат не удержан** — анимация «не работает»; единственный сигнал — предупреждение компилятора, которое легко не заметить.
- **`return` без `completeTransition`** — зависший интерфейс.
- **Забыли `.custom`** — для стилей вроде `.pageSheet` UIKit может использовать своё оформление показа вместо твоего.

Упражнение 28.2. В `GalleryViewController` замени свойство `fadeDelegate` на прямое присваивание `photo.transitioningDelegate = FadeTransitioningDelegate()`. Что увидит пользователь? Что скажет компилятор и почему приложение всё равно запустится?

Ответы к упражнениям

28.1. Баннер начинается на 59 + 8 = 67 точках и при высоте 64 заканчивается на 131. Сдвиг на −100 поднимет его нижний край до 31 — это ещё внутри экрана, под статус-баром. До появления и после ухода нижняя часть баннера будет торчать сверху. Нужно сдвинуть минимум на 131 точку, с запасом — на −(131 + 20) = −151; ровно это и считает формула `-(banner.frame.maxY + 20)`.

28.2. Пользователь увидит стандартный модальный показ, как будто никакого делегата нет (при `.custom` — просто появление без твоей анимации). `transitioningDelegate` — слабая ссылка, созданный объект тут же освобождается, и к моменту показа свойство уже `nil`. Компилятор это видит: Swift 6.3 выдаёт на такую строку предупреждение «weak reference will always be nil because the referenced object is deallocated here» (проверено сборкой с `-swift-version 6`). Но это предупреждение, а не ошибка — присвоить новый объект слабому свойству законно, поэтому проект собирается и запускается, а анимации нет.

Что мы выучили

- **Показом управляет presentation controller** (`UIPresentationController → NSObject`) — не view и не view controller; `dismiss` у показанного экрана UIKit пересылает показывающему.
- **Стиль по умолчанию `.automatic`** разворачивается в `.formSheet` (iOS 18+) или `.pageSheet` (iOS 13–17); свойство читается уже развёрнутым, на iPhone оба — лист.
- **Лист** (`UISheetPresentationController`, iOS 15+) — детенты `.medium()` / `.large()`, ручка, скругление; свои высоты `.custom` — только iOS 16+ и только под `#available`.
- **Высота `.custom`** считается внутри безопасной области; `200` — это высота содержимого, сам лист на экране выше.
- **`largestUndimmedDetentIdentifier`** — лист без затемнения, под которым можно работать.
- **`isModalInPresentation`** + `presentationControllerDidAttemptToDismiss` — защита несохранённого ввода от свайпа.
- **Экран под листом не получает `viewWillAppear`** после закрытия листа; у `.fullScreen` — получает (`shouldRemovePresentersView` там `true`); в компактной высоте листы ведут себя как `.fullScreen`.
- **Поповер** на iPhone превращается в лист (`adaptiveSheetPresentationController` для настройки); `.none` в делегате оставляет облачко.
- **Алерт**: одна `.cancel`, она слева при двух кнопках и внизу при трёх и больше; `preferredAction` — только после `addAction` и только для `.alert`; action sheet показывается стилем `.popover` и на iPad требует `sourceView` (на iPhone `popoverPresentationController` тоже не `nil`).

- **Своё окно** — отдельный экран `.overFullScreen` + `.crossDissolve` с `accessibilityViewIsModal`.
- **Баннер и HUD** — в окно или экран, прячем на реально посчитанное расстояние, объявляем VoiceOver через `.announcement`.
- **Своя анимация модала** — аниматор + `transitioningDelegate`, который надо удержать сильной ссылкой.

Apple Developer Documentation

- `UISheetPresentationController` — лист с детентами (iOS 15+).
- `UISheetPresentationController.Detent` — `.medium()`, `.large()`, `.custom(identifier:resolver:)` (iOS 16+).
- `largestUndimmedDetentIdentifier` — до какой высоты не затемнять фон.
- `UIViewController.isModalInPresentation` — запрет закрытия свайпом.
- `UIAdaptivePresentationControllerDelegate` — `presentationControllerDidAttemptToDismiss(_:)` и адаптация стилей.
- `UIModalPresentationStyle` — `.automatic`, `.pageSheet`, `.formSheet`, `.fullScreen`, `.overFullScreen`, `.popover`, `.custom`; `UIViewController.modalPresentationStyle` — само свойство и его значение по умолчанию.
- `UIPresentationController` — объект, управляющий показом; `shouldRemovePresentersView`.
- `UIViewController.dismiss(animated:completion:)` — закрывает показывающий; вызов у показанного пересылается ему.
- `UIPopoverPresentationController` — `sourceView`/`sourceRect`, `adaptiveSheetPresentationController`.
- `UIAlertController` — `.alert` и `.actionSheet`, `preferredAction`, `addTextField(configurationHandler:)`.
- `UIAlertAction.Style` — `.default`, `.cancel`, `.destructive`.
- `UIAccessibility.Notification.announcement` — объявить текст для VoiceOver.
- `UIViewControllerTransitioningDelegate` и `UIViewControllerAnimatedTransitioning` — свои модальные переходы.
- HIG — Sheets, HIG — Alerts, HIG — Popovers — Apple о том, когда какой способ показа уместен.

→ Глава 29. Cookbook — жесты

Глава 29. Cookbook — жесты

Тап, двойной тап, долгое нажатие, свайп, перетаскивание, щипок — всё это жесты. UIKit распознаёт их сам, тебе остаётся подключить нужный «распознаватель» и решить, что делать, когда жест случился. Эта глава — каталог жестов, математика перетаскивания (смещение и скорость на числах) и главное — что делать, когда два жеста спорят за одно касание.

> **В каком режиме код.** Листинги проверены компилятором в режиме Swift 6 с Default Actor Isolation = MainActor и минимальной версией iOS 15 (подробнее — в начале главы 25).

Распознаватель жестов (gesture recognizer) — объект, который смотрит на поток касаний одной view и решает: «это был тап» или «это перетаскивание». Похоже на датчики в умном доме: один реагирует на хлопок, другой на движение, третий на открытую дверь. Каждый распознаватель — подкласс `UIGestureRecognizer`: `UITapGestureRecognizer`, `UIPanGestureRecognizer` и так далее. Его добавляют к view методом `addGestureRecognizer(_:)`, а о результате он сообщает через пару «цель — действие» (`target` + `action`, метод с пометкой `@objc`); `addAction(UIAction)` у него нет — это не `UIControl`.

Кто он такой. `UIGestureRecognizer → NSObject`: не view и не responder. У него нет `frame`, его нельзя добавить через `addSubview`, и в responder chain он не участвует (так пишет документация) — он стоит **рядом** с цепочкой и получает касания, прошедшие hit-testing в его view и её subviews, раньше самой view. Привязан он ровно к **одной** view: она доступна как `gesture.view`, а view держит распознаватель сильной ссылкой. Если добавить тот же объект к другой view, он переедет: на iOS 26.5 после `v2.addGestureRecognizer(g)` у `v1.gestureRecognizers` пусто, а `g.view === v2`. Нужен один жест на двух view — создай два распознавателя. Значения по умолчанию: `cancelsTouchesInView = true`, `delaysTouchesBegan = false`, `delaysTouchesEnded = true`, `isEnabled = true`, `delegate = nil` (глава 0, разделы 0.2 и 0.8).

У распознавателя есть **состояние** (`state`); пока жест не начался, оно `.possible`. Жесты бывают двух видов:

- **Дискретные** («разовые») — тап, свайп. Случились один раз и всё: распознаватель сразу переходит в `.ended` (оно же `.recognized`) и вызывает действие один раз.
- **Непрерывные** — перетаскивание (pan), щипок (pinch), долгое нажатие. Действие вызывается много раз: `.began` в начале, `.changed` на каждое движение пальца, `.ended` при отпускании. Если жест прервала система (например, пришёл звонок), будет `.cancelled`.

Есть ещё `.failed` — «это был не мой жест». Например, распознаватель двойного тапа переходит в `.failed`, если второго тапа не последовало.

29.1 UIContextMenu — long-press с превью

Когда применять. Долгое нажатие на элемент показывает меню действий и увеличенное превью — как на фото в «Фото» или на ссылке в Safari. **Контекстное меню** — это меню, которое относится к конкретному элементу, на котором его вызвали.

```
final class PhotoViewController: UIViewController, UIContextMenuInteractionDelegate {
    private let photoView = UIImageView()

    override func viewDidLoad() {
        super.viewDidLoad()
        photoView.isUserInteractionEnabled = true
        photoView.addInteraction(UIContextMenuInteraction(delegate: self))
    }

    func contextMenuInteraction(_ interaction: UIContextMenuInteraction,
                                configurationForMenuAtLocation location: CGPoint)
        -> UIContextMenuConfiguration? {
        UIContextMenuConfiguration(identifier: nil, previewProvider: nil) { [weak self] _
↪   in
            UIMenu(children: [
                UIAction(title: "В избранное", image: UIImage(systemName: "heart")) { _ in
                    self?.toggleFavorite()
                },
                UIAction(title: "Поделиться", image: UIImage(systemName:
↪   "square.and.arrow.up")) { _ in
                    self?.share()
                },
                UIAction(title: "Удалить", image: UIImage(systemName: "trash"),
                         attributes: .destructive) { _ in
                    self?.delete()
                },
            ])
        }
    }

    private func toggleFavorite() {}
    private func share() {}
    private func delete() {}
}
```

`UIContextMenuInteraction` — не распознаватель, а **взаимодействие** (interaction): объект `NSObject`, реализующий протокол `UIInteraction`, то есть готовое поведение целиком — долгое нажатие, лёгкая отдача вибрацией, размытие фона, превью, меню. Подключается через `addInteraction`, и, как распознаватель, живёт при одной view (`interaction.view`). `isUserInteractionEnabled = true` нужен, потому что `UIImageView` по умолчанию касания не принимает (29.5).

Делегат отвечает на один вопрос: «какое меню показать в этой точке?». `location` — точка нажатия, если меню должно зависеть от того, куда именно нажали. Вернуть `nil` — меню не будет.

- `previewProvider: nil` — превью будет увеличенной копией самой view. Если передать замыкание, возвращающее view controller, в превью покажется этот экран (например, открытая заметка).
- Третий параметр — замыкание, которое строит меню. Оно получает список «предложенных» системой действий (мы его не используем, _).
- `attributes: .destructive` — красный пункт.

В таблице и коллекции взаимодействие подключать не нужно — у их делегатов есть готовый метод:

```
override func tableView(_ tableView: UITableView,
                        contextMenuConfigurationForRowAt indexPath: IndexPath,
                        point: CGPoint) -> UIContextMenuConfiguration? {
    let item = items[indexPath.row]
    return UIContextMenuConfiguration(identifier: nil, previewProvider: nil) { [weak self]
     ↪  _ in
        UIMenu(children: [
           UIAction(title: "Скопировать", image: UIImage(systemName: "doc.on.doc")) { _ in
                UIPasteboard.general.string = item
            },
            UIAction(title: "Удалить", image: UIImage(systemName: "trash"),
                     attributes: .destructive) { _ in
                self?.remove(item)
            },
        ])
    }
}
```

`let item = items[indexPath.row]` берём **сразу**, а не внутри пунктов меню. Пока меню открыто, список может измениться (например, придёт обновление с сервера), и `indexPath` будет указывать уже на другую строку. Сам элемент — надёжнее.

29.2 Swipe actions (table)

Когда применять. Действия по свайпу строки: удалить, архивировать, пометить прочитанным — как в «Почте».

```
override func tableView(_ tableView: UITableView,
                        trailingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
    let item = items[indexPath.row]

    let delete = UIContextualAction(style: .destructive, title: "Удалить") { [weak self]
     ↪  _, _, done in
        guard let self, let row = self.items.firstIndex(of: item) else {
            done(false)
```

```
            return
        }
        self.items.remove(at: row)
        tableView.deleteRows(at: [IndexPath(row: row, section: 0)], with: .automatic)
        done(true)
    }
    delete.image = UIImage(systemName: "trash")

    let archive = UIContextualAction(style: .normal, title: "Архив") { [weak self] _, _,
    ↪ done in
        self?.archive(item)
        done(true)
    }
    archive.image = UIImage(systemName: "archivebox")
    archive.backgroundColor = .systemOrange

    return UISwipeActionsConfiguration(actions: [delete, archive])
}
```

`trailingSwipeActions...` — кнопки, которые открываются свайпом **влево**, с правого края строки. `leadingSwipeActions...` — свайпом вправо, с левого края (обычно «прочитано» или «закрепить»).

Порядок в массиве: **первое** действие стоит **ближе всего к краю**. У нас `[delete, archive]` — «Удалить» у самого правого края, «Архив» левее.

Стили:

- `.destructive` — красный фон. Смысл «действие удаляет строку»: после `done(true)` таблица ожидает, что строка исчезнет.
- `.normal` — серый фон по умолчанию; цвет можно сменить через `backgroundColor`.

Полный свайп — если протянуть строку до конца, выполнится **первое** действие без нажатия на кнопку. Это зависит не от стиля, а от свойства `performsFirstActionWithFullSwipe` конфигурации (по умолчанию `true`). Чтобы случайно ничего не удалить, можно поставить первым безопасное действие или выключить полный свайп.

`done(...)` — обработчик завершения. Вызвать его **обязательно**: `true` — «действие выполнено», `false` — «не получилось, верни строку как было». Не вызовешь — строка останется полуоткрытой.

Почему строку ищем заново через `firstIndex(of: item)`: замыкание выполнится позже, когда человек нажмёт кнопку. За это время список мог измениться, и старый `indexPath.row` указал бы на чужую строку. Для `firstIndex(of:)` элементы должны быть `Equatable` — у нас это строки.

29.3 Pinch to zoom

Когда применять. Увеличение фото, карты, схемы двумя пальцами.

```swift
final class ZoomViewController: UIViewController, UIScrollViewDelegate {
    private let scrollView = UIScrollView()
    private let imageView = UIImageView()

    override func viewDidLoad() {
        super.viewDidLoad()
        scrollView.minimumZoomScale = 1.0
        scrollView.maximumZoomScale = 4.0
        scrollView.bouncesZoom = true
        scrollView.delegate = self
        scrollView.frame = view.bounds
        scrollView.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        imageView.frame = scrollView.bounds
        imageView.contentMode = .scaleAspectFit
        view.addSubview(scrollView)
        scrollView.addSubview(imageView)
    }

    func viewForZooming(in scrollView: UIScrollView) -> UIView? { imageView }
}
```

`UIScrollView` умеет масштабирование щипком **сам**. Нужно три вещи: `minimumZoomScale` меньше `maximumZoomScale`, делегат и метод `viewForZooming`, который говорит, **что** увеличивать.

`maximumZoomScale = 4.0` — увеличить можно в 4 раза: фото шириной 393 точки на экране станет 1572 точки, и видна будет четверть его ширины. `bouncesZoom = true` — если растянуть сильнее, картинка «пружинит» и возвращается к пределу.

Полный просмотрщик с двойным тапом для увеличения — глава 16 (Gallery) и глава 37 (Photo viewer).

Если масштабировать нужно не картинку в скролле, а произвольную view, есть `UIPinchGestureRecognizer`:

```swift
@objc private func handlePinch(_ gesture: UIPinchGestureRecognizer) {
    guard let target = gesture.view else { return }
    target.transform = target.transform.scaledBy(x: gesture.scale, y: gesture.scale)
    gesture.scale = 1
}
```

`gesture.view` — та единственная view, к которой жест привязан; благодаря этому один обработчик годится для любой view. `gesture.scale` — во сколько раз пальцы разошлись с начала жеста: 1,0 — не двигались, 2,0 — расстояние между пальцами выросло вдвое, 0,5 — сократилось вдвое. Трюк с `gesture.scale = 1` делает шаги **относительными**: каждый вызов умножает текущий масштаб на изменение **с прошлого вызова**, и счётчик обнуляется. Без обнуления масштаб бы «разгонялся»: на втором кадре 1,1 × 1,2, на третьем уже × 1,3 поверх и так далее.

29.4 Pan для draggable view

Когда применять. Перетаскивание пальцем: карточка, стикер, плавающая кнопка. **Pan** — жест «ведение пальцем».

```
final class DragViewController: UIViewController {
    private let card = UIView(frame: CGRect(x: 40, y: 200, width: 120, height: 80))
    private var dragStart = CGPoint.zero

    override func viewDidLoad() {
        super.viewDidLoad()
        card.backgroundColor = .systemBlue
        view.addSubview(card)
        let pan = UIPanGestureRecognizer(target: self, action: #selector(handlePan(_:)))
        card.addGestureRecognizer(pan)
    }

    @objc private func handlePan(_ gesture: UIPanGestureRecognizer) {
        switch gesture.state {
        case .began:
            dragStart = card.center
        case .changed:
            let translation = gesture.translation(in: view)
            card.center = CGPoint(x: dragStart.x + translation.x,
                                  y: dragStart.y + translation.y)
        case .ended, .cancelled:
            let velocity = gesture.velocity(in: view)
            let target = CGPoint(x: card.center.x + project(velocity.x),
                                 y: card.center.y + project(velocity.y))
            UIView.animate(withDuration: 0.3) {
                self.card.center = self.clampedToScreen(target)
            }
        default:
            break
        }
    }

    /// Сколько точек карточка «докатилась» бы по инерции.
    private func project(_ velocity: CGFloat) -> CGFloat {
        let deceleration: CGFloat = 0.998
        return (velocity / 1000) * deceleration / (1 - deceleration)
    }

    private func clampedToScreen(_ point: CGPoint) -> CGPoint {
        let area = view.bounds.inset(by: view.safeAreaInsets)
        let halfW = card.bounds.width / 2
        let halfH = card.bounds.height / 2
        return CGPoint(x: min(max(point.x, area.minX + halfW), area.maxX - halfW),
                       y: min(max(point.y, area.minY + halfH), area.maxY - halfH))
```

```
    }
}
```

Две величины, на которых стоит любой pan.

Смещение (`translation(in:)`) — насколько палец сдвинулся **от точки начала жеста**, в координатах указанной view. Палец начал в (100, 300) и сейчас в (160, 280) — смещение (60, −20): 60 точек вправо и 20 вверх (ось Y в UIKit направлена **вниз**, поэтому движение вверх даёт минус). Отсюда схема: в `.began` запоминаем, где был центр карточки, а в `.changed` ставим центр = старт + смещение. Смещение меряем в координатах `view` — неподвижного родителя. Если мерить в координатах самой карточки, которая при этом движется, отсчёт будет ехать вместе с ней.

Скорость (`velocity(in:)`) — как быстро движется палец в момент вызова, в **точках в секунду**. (800, 0) — вправо со скоростью 800 точек в секунду, то есть за секунду палец прошёл бы примерно два экрана iPhone по ширине. Скорость нужна в `.ended`: отпустили медленно — карточка остаётся под пальцем, «швырнули» — она должна пролететь дальше.

Инерция — функция `project`. Она отвечает: «если бы карточка тормозила как обычный список iOS, сколько бы она ещё проехала?». 0,998 — коэффициент торможения обычной прокрутки (`UIScrollView.DecelerationRate.normal`): за каждую миллисекунду скорость умножается на 0,998, то есть теряет 0,2%. Сложив все эти убывающие шаги, получаем формулу из функции. На числах: при скорости 1000 точек в секунду карточка докатится ещё на 1000 / 1000 × 0,998 / 0,002 ≈ 499 точек, при 200 точках в секунду — на 100. Получается простое правило: путь по инерции — примерно половина скорости.

`clampedToScreen` не даёт карточке улететь за экран: берём безопасную область, отступаем от её краёв на половину размера карточки (ведь `center` — это середина) и зажимаем координату между границами через `min(max(...))`. Точка x = 500 на экране шириной 393 превратится в 393 − 60 = 333.

Частые ошибки.

- **Смещение прибавляют к текущему центру** на каждом `.changed`, а не к стартовому. Смещение считается от начала жеста, поэтому карточка «разгоняется» и убегает из-под пальца. Лечение — стартовая точка, как в листинге, или обнулять смещение после каждого шага: `gesture.setTranslation(.zero, in: view)`.
- **Не обработан `.cancelled`** — после входящего звонка карточка остаётся посреди экрана в промежуточном положении.

Упражнение 29.1. Карточку отпустили со скоростью (−1500, 400). Куда она докатится по инерции (в точках по каждой оси), если центр в момент отпускания был (200, 400)? Что сделает `clampedToScreen` на экране 393 × 852 с безопасной областью сверху 59 и снизу 34 и карточкой 120 × 80?

29.5 Tap gesture

Когда применять. Тап по view, которая сама не кнопка: аватар, картинка, метка.

```
final class ProfileHeaderViewController: UIViewController {
    private let avatarView = UIImageView()
```

```
    override func viewDidLoad() {
        super.viewDidLoad()
        let tap = UITapGestureRecognizer(target: self, action: #selector(avatarTapped))
        avatarView.addGestureRecognizer(tap)
        avatarView.isUserInteractionEnabled = true
        avatarView.accessibilityTraits = .button
        avatarView.accessibilityLabel = "Сменить фото профиля"
        avatarView.isAccessibilityElement = true
    }

    @objc private func avatarTapped() {
        // открыть выбор фото
    }
}
```

`isUserInteractionEnabled = true` — **обязательно** для `UIImageView` и `UILabel`: у них это свойство по умолчанию `false` (в заголовках `UILabel.h` и `UIImageView.h` так и написано: «default is NO»), и hit-testing их пропускает — касание достаётся той view, что окажется под ними, обычно superview. Распознаватель получает только касания, прошедшие hit-testing в его view, поэтому на такой view он просто молчит, без ошибок.

Три строки доступности превращают картинку в «кнопку» для VoiceOver: иначе незрячий человек не узнает, что аватар нажимается. Если элемент выглядит и работает как кнопка, часто проще сразу сделать его `UIButton` — доступность и подсветка при нажатии будут бесплатно.

Двойной тап и конфликт с одинарным:

```
let singleTap = UITapGestureRecognizer(target: self, action: #selector(toggleControls))
let doubleTap = UITapGestureRecognizer(target: self, action: #selector(zoomIn))
doubleTap.numberOfTapsRequired = 2
singleTap.require(toFail: doubleTap)
photoView.addGestureRecognizer(singleTap)
photoView.addGestureRecognizer(doubleTap)
```

Без последней настройки двойной тап вызовет **оба** действия: первый касание распознается как одинарный тап (интерфейс скроется), а второе — как двойной (фото увеличится).

`singleTap.require(toFail: doubleTap)` — «одинарный тап срабатывает, только когда двойной **провалился**». Одинарный теперь ждёт: последует ли второй тап? Если за короткий промежуток второго касания не было, двойной переходит в `.failed`, и только тогда срабатывает одинарный. Цена — небольшая задержка реакции на одинарный тап, её видно, если присмотреться. Поэтому `require(toFail:)` ставят только там, где двойной тап действительно есть.

29.6 Long press

Когда применять. Своё действие по долгому нажатию, когда контекстное меню (29.1) не подходит: например, начать запись голоса, пока палец держит кнопку.

```
@objc private func handleLongPress(_ gesture: UILongPressGestureRecognizer) {
    switch gesture.state {
    case .began:
        startRecording()
    case .ended:
        stopRecording(send: true)
    case .cancelled, .failed:
        stopRecording(send: false)
    default:
        break
    }
}

// подключение:
// let longPress = UILongPressGestureRecognizer(target: self, action:
↪  #selector(handleLongPress(_:)))
// longPress.minimumPressDuration = 0.3
// micButton.addGestureRecognizer(longPress)
```

`minimumPressDuration` — сколько держать палец, чтобы жест начался. По умолчанию 0,5 с, здесь 0,3 с — запись должна начинаться быстро.

Долгое нажатие — **непрерывный** жест. `.began` приходит, когда палец продержался нужное время, а не в момент касания. `.ended` — когда палец отпустили. Пока палец держат, можно двигать его: если сдвиг больше `allowableMovement` (по умолчанию 10 точек) до срабатывания, жест провалится.

Для меню действий по долгому нажатию бери `UIContextMenuInteraction` (29.1): там уже есть вибрация, превью и поддержка доступности.

29.7 Swipe gesture (не actions)

Когда применять. Короткий быстрый взмах для переключения: другая фотография, другой день в календаре.

```
let swipeLeft = UISwipeGestureRecognizer(target: self, action: #selector(showNext))
swipeLeft.direction = .left
view.addGestureRecognizer(swipeLeft)

let swipeRight = UISwipeGestureRecognizer(target: self, action: #selector(showPrevious))
swipeRight.direction = .right
view.addGestureRecognizer(swipeRight)
```

Свайп — **дискретный** жест: он срабатывает один раз, когда палец быстро прошёл достаточное расстояние в нужном направлении. Медленное ведение пальцем — это уже pan (29.4).

Почему два распознавателя, а не один с `direction = [.left, .right]`: распознаватель с двумя направлениями сработает на оба, но **не скажет, какое именно** было. Два отдельных распознавателя — два разных действия.

Свайп не даёт «прилипания» к пальцу: экран не едет вслед за движением. Если нужен листающийся ряд страниц, где соседняя страница видна во время жеста, — это `UIPageViewController` или скролл с `isPagingEnabled`.

29.8 Pan-to-dismiss modal

Когда применять. Свой полноэкранный модальный экран (например, просмотр фото), который закрывается движением вниз.

```
final class DismissableViewController: UIViewController {
    init() {
        super.init(nibName: nil, bundle: nil)
        modalPresentationStyle = .overFullScreen
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .black
        let pan = UIPanGestureRecognizer(target: self, action:
         ↪  #selector(handleDragDown(_:)))
        view.addGestureRecognizer(pan)
    }

    @objc private func handleDragDown(_ gesture: UIPanGestureRecognizer) {
        let dy = max(0, gesture.translation(in: view).y)

        switch gesture.state {
        case .changed:
            view.transform = CGAffineTransform(translationX: 0, y: dy)
            let progress = min(dy / 300, 1)
            view.alpha = 1 - progress * 0.5
        case .ended, .cancelled:
            let velocityY = gesture.velocity(in: view).y
            if gesture.state == .ended && (dy > 150 || velocityY > 1000) {
                dismiss(animated: true)
            } else {
                UIView.animate(withDuration: 0.35, delay: 0,
                               usingSpringWithDamping: 0.8, initialSpringVelocity: 0) {
                    self.view.transform = .identity
                    self.view.alpha = 1
                }
            }
        default:
            break
        }
    }
```

```
}
```

Тянем экран вниз: он едет за пальцем и бледнеет. Отпустили далеко или резко — закрываем, иначе пружиной возвращаем на место.

- `max(0, translation.y)` — движение вверх не учитываем: смещение зажато снизу нулём. Не пиши вместо этого `guard translation > 0 else { return }`: если потянуть вниз, а потом вернуть палец выше стартовой точки и отпустить, `.ended` так и не обработается, и экран останется сдвинутым.
- **Прозрачность на числах**: `progress` — доля пути от 0 до 1, где 1 — это 300 точек. Потянули на 150 — progress 0,5, `alpha = 1 - 0,5 × 0,5 = 0,75` (экран на четверть прозрачный). Потянули на 300 и дальше — `alpha = 0,5`, прозрачнее не станет.
- **Порог закрытия** — два условия через «или»: оттащили дальше 150 точек **или** отпустили со скоростью больше 1000 точек в секунду вниз. Второе нужно для короткого резкого взмаха: палец прошёл всего 60 точек, но так быстро, что намерение закрыть не вызывает сомнений.
- `gesture.state == .ended` — при `.cancelled` (жест прервала система) не закрываем, а возвращаем экран.
- `.overFullScreen` — чтобы под экраном оставался предыдущий: когда экран становится полупрозрачным и съезжает, сквозь него видно, куда мы вернёмся. При `.fullScreen` предыдущий экран убран из иерархии, и сквозь полупрозрачный экран будет виден чёрный фон окна.

Для системного листа (`.pageSheet`, глава 28) всё это работает само, писать ничего не нужно. Вариант для просмотрщика фото — глава 37, раздел 37.3.

Упражнение 29.2. Человек потянул экран вниз на 90 точек и отпустил со скоростью 1200 точек в секунду. Закроется ли экран? А если потянул на 200 точек, остановил палец (скорость около 0) и отпустил? Какой будет `alpha` в момент отпускания во втором случае?

29.9 Edge swipe (back gesture)

`UINavigationController` уже содержит распознаватель свайпа от левого края — `interactivePopGestureRecogni`
Он возвращает на предыдущий экран стопки. Свойство объявлено как `UIGestureRecognizer`, но на деле это экземпляр приватного подкласса `UIScreenEdgePanGestureRecognizer` (проверено на iOS 26.5), а тот — наследник `UIPanGestureRecognizer`: pan, который начинается только у края экрана.

Самый частый вопрос — как его **сохранить** или **восстановить**, если ты поставил свою левую кнопку. Правильный способ с подклассом контроллера навигации разобран в главе 26, раздел 26.5: контроллер становится делегатом жеста и разрешает его, только когда в стопке больше одного экрана.

С iOS 26 у стопки появился ещё один системный жест — `interactiveContentPopGestureRecognizer`: возврат назад свайпом вправо **из любой точки содержимого**, а не только от края. Если на экране есть свой горизонтальный жест (карусель, слайдер, перетаскиваемая карточка), он может конфликтовать с этим возвратом. Apple прямо пишет, что это свойство предназначено для настройки зависимостей, например:

```
if #available(iOS 26.0, *),
   let contentPop = navigationController?.interactiveContentPopGestureRecognizer {
    contentPop.require(toFail: carouselPan)
}
```

«Возврат из контента начнётся, только если мой жест карусели провалился» — пока палец листает карусель, экран назад не уедет.

Свой жест от края экрана — `UIScreenEdgePanGestureRecognizer`:

```
let edgePan = UIScreenEdgePanGestureRecognizer(target: self, action:
↪  #selector(openMenu(_:)))
edgePan.edges = .right
view.addGestureRecognizer(edgePan)
```

`UIScreenEdgePanGestureRecognizer` → `UIPanGestureRecognizer`: это pan, который начинается только у указанного края экрана (здесь правого), учитывает только первое касание и отдаёт те же `translation(in:)` и `velocity(in:)`. Удобно для выдвижного бокового меню. Обработчик — такой же, как у pan в 29.4. `edges` нужно задать до того, как жест добавлен к view, — так просит документация.

29.10 Конфликт gesture'ов

Чаще всего жесты ломаются не поодиночке, а когда их несколько: pan внутри скролла, тап внутри ячейки, свайп назад поверх карусели.

Правило по умолчанию. Если на одном касании могут сработать два распознавателя (на одной view или на view и её родителе), UIKit разрешает сработать **только одному** — документация `shouldRecognizeSimultaneouslyWith` формулирует это как «no two gestures can be recognized simultaneously»: первый распознавший жест побеждает, остальные переходят в `.failed`. Отсюда инструменты:

1. Направление — `gestureRecognizerShouldBegin`. Классика: горизонтальный pan на ячейке вертикального списка. Список прокручивается вертикальным pan'ом, твоя ячейка должна уезжать вбок. Разрешим своему жесту начаться, только если движение **больше горизонтальное, чем вертикальное**:

```
extension SwipeCell {
    override func gestureRecognizerShouldBegin(_ gestureRecognizer: UIGestureRecognizer)
    ↪  -> Bool {
        guard let pan = gestureRecognizer as? UIPanGestureRecognizer else { return true }
        let velocity = pan.velocity(in: self)
        return abs(velocity.x) > abs(velocity.y)
    }
}
```

`abs` — модуль числа, без знака. Сравниваем, какая составляющая скорости больше. Скорость (300, −80): по горизонтали 300, по вертикали 80 — жест горизонтальный, ячейка поедет. Скорость (40, 500) — палец идёт вниз, наш жест не начнётся, и касание достанется списку.

Обрати внимание: метод объявлен с `override`. У `UIView` уже есть свой `gestureRecognizerShouldBegin(_:)` (по умолчанию возвращает `true`), и ячейка — это view (`UITableViewCell → UIView`). Для жестов, которые привязаны к самой ячейке, UIKit спросит именно этот метод в момент, когда распознаватель ещё в `.possible` и готов перейти в `.began`; `false` переводит его в `.failed`. Так, по документации, `UISlider` отсекает свайпы вдоль себя. Делегата назначать не обязательно — но метод работает только для жестов на этой view, а не на её subviews.

2. Одновременность — `shouldRecognizeSimultaneouslyWith`. Иногда оба жеста должны работать вместе: щипок и поворот фотографии двумя пальцами, свой pan поверх скролла для эффекта параллакса.

```
final class StickerViewController: UIViewController, UIGestureRecognizerDelegate {
    private let pinch = UIPinchGestureRecognizer()
    private let rotation = UIRotationGestureRecognizer()

    override func viewDidLoad() {
        super.viewDidLoad()
        pinch.delegate = self
        rotation.delegate = self
    }

    func gestureRecognizer(_ gestureRecognizer: UIGestureRecognizer,
                           shouldRecognizeSimultaneouslyWith other: UIGestureRecognizer) ->
                             ↪ Bool {
        (gestureRecognizer === pinch && other === rotation)
            || (gestureRecognizer === rotation && other === pinch)
    }
}
```

Отвечаем «да» **только** для нужной пары — щипок с поворотом. Голое `return true` разрешило бы одновременность со всем подряд, включая системный свайп назад и прокрутку, и тогда при вращении стикера заодно ехал бы экран.

`===` — сравнение **объектов** («это тот же самый распознаватель»), а не значений.

3. Очерёдность — `require(toFail:)`. Уже встречался в 29.5: «мой жест ждёт, пока тот провалится». Работает и между разными видами жестов: `swipe.require(toFail: pan)` — свайп сработает, только если pan не распознан.

Сводка:

Ситуация	Инструмент
Жесты по разным направлениям	`gestureRecognizerShouldBegin` + скорость
Оба жеста должны работать вместе	`shouldRecognizeSimultaneouslyWith`, точечно
Один жест — частный случай другого (1 и 2 тапа)	`require(toFail:)`

Упражнение 29.3. В ячейке горизонтальный pan с проверкой из листинга выше. Человек повёл пальцем почти по диагонали, скорость (250, 260). Что произойдёт: поедет ячейка или про-

крутится список? Как изменить условие, чтобы ячейка реагировала только на явно горизонтальное движение — например, когда горизонтальная скорость хотя бы вдвое больше вертикальной?

29.11 Gesture inside UITableView

Тап по ячейке обрабатывает `didSelectRowAt`. Если нужно отдельное действие по тапу на **часть** ячейки — например, на аватарку, — распознаватель ставится в **самой ячейке**, один раз:

```
final class CommentCell: UITableViewCell {
    var onAvatarTap: (() -> Void)?
    let avatarView = UIImageView()

    override init(style: UITableViewCell.CellStyle, reuseIdentifier: String?) {
        super.init(style: style, reuseIdentifier: reuseIdentifier)
        avatarView.isUserInteractionEnabled = true
        let tap = UITapGestureRecognizer(target: self, action: #selector(avatarTapped))
        avatarView.addGestureRecognizer(tap)
        contentView.addSubview(avatarView)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    @objc private func avatarTapped() { onAvatarTap?() }

    override func prepareForReuse() {
        super.prepareForReuse()
        onAvatarTap = nil
    }
}

// в cellForRowAt:
// cell.onAvatarTap = { [weak self] in self?.openProfile(of: comment.author) }
```

Почему не в `cellForRowAt`: `cellForRowAt` вызывается при каждом появлении строки. Если добавлять распознаватель там, то переиспользованная ячейка накопит их десяток, и один тап вызовет действие десять раз. В `init` ячейки распознаватель добавляется ровно один раз за её жизнь, а в `cellForRowAt` меняется только замыкание `onAvatarTap` — оно и знает, чей это аватар.

Тап по аватарке **не** выделит строку: у распознавателя по умолчанию `cancelsTouchesInView = true` — распознав тап, он не отдаёт view оставшиеся касания, а уже доставленные отзывает через `touchesCancelled(_:with:)`; таблица не видит «нажатия на строку». Если нужно и то, и другое, поставь `tap.cancelsTouchesInView = false` — тогда view получит всю последовательность касаний целиком.

Альтернатива без жестов — положить в ячейку `UIButton` с картинкой аватара. Это проще и сразу доступно для VoiceOver. Для кнопки «подробнее» справа есть стандартный аксессуар `.detailButton` и метод делегата `accessoryButtonTappedForRowWith`.

29.12 Бытовая аналогия

Распознаватели — это **датчики в комнате**. Один срабатывает на хлопок (тап), другой на долгое касание выключателя (long press), третий на взмах рукой (swipe), четвёртый — когда двигают мебель (pan), пятый — когда разводят руками (pinch). Каждый датчик запускает своё действие.

Когда датчики стоят рядом и видят одно и то же движение, их нужно договорить: либо дать им приоритет («сначала убедись, что это не двойной хлопок» — `require(toFail:)`), либо разрешить работать вместе (`shouldRecognizeSimultaneouslyWith`), либо научить отличать движения по направлению (`gestureRecognizerShouldBegin`).

Ответы к упражнениям

29.1. Инерция — примерно половина скорости: по X −1500 / 1000 × 0,998 / 0,002 ≈ −748 точек, по Y ≈ +200 точек. Цель (200 − 748, 400 + 200) = (−548, 600). `clampedToScreen`: безопасная область по X — от 0 до 393, по Y — от 59 до 852 − 34 = 818. Отступаем на половину карточки (60 по X, 40 по Y): X зажимается в диапазон 60…333, Y — в 99…778. −548 меньше 60, поэтому X = 60; 600 внутри диапазона. Карточка уедет в (60, 600) — прижмётся к левому краю.

29.2. В первом случае закроется: 90 точек меньше порога 150, но скорость 1200 больше 1000, а условия соединены «или». Во втором тоже закроется: 200 больше 150. `alpha` в момент отпускания: progress = 200 / 300 ≈ 0,67, `alpha = 1 − 0,67 × 0,5 ≈ 0,67`.

29.3. 250 < 260, условие `abs(x) > abs(y)` ложно — ячейка не поедет, прокрутится список. Для «явно горизонтального» движения:

```
return abs(velocity.x) > 2 * abs(velocity.y)
```

Скорость (250, 260) — не пройдёт, (600, 200) — пройдёт (600 > 400).

Что мы выучили

- **Распознаватель жестов** — `UIGestureRecognizer → NSObject`: не view и не responder, привязан ровно к одной view (`gesture.view`), получает её касания раньше неё; дискретные жесты (тап, свайп) срабатывают один раз, непрерывные (pan, pinch, long press) проходят `.began → .changed → .ended`.
- **`UIContextMenuInteraction`** — долгое нажатие с меню и превью; в таблице — `contextMenuConfigurationFor` элемент берём сразу.
- **Swipe actions** — первое действие ближе к краю и выполняется полным свайпом (`performsFirstActionWithFullSwipe`); `done(...)` обязателен; строку ищем заново в момент нажатия.
- **Pinch** — `UIScrollView` + `viewForZooming`; для своей view — `scaledBy` и `gesture.scale = 1`.
- **Pan** — смещение от начала жеста + стартовая точка; скорость в точках в секунду; инерция ≈ половина скорости; зажим в безопасную область.
- **Тап** по `UIImageView/UILabel` — только с `isUserInteractionEnabled = true`; одинарный с двойным — `require(toFail:)`.
- **Pan-to-dismiss** — `max(0, dy)`, порог по расстоянию **или** скорости, `.overFullScreen`, пружина при возврате.

- **Конфликты** — направление через `gestureRecognizerShouldBegin`, одновременность только для нужной пары, очерёдность через `require(toFail:)`; в iOS 26 — зависимость от `interactiveContentPopGestureRecognizer`. Системный свайп назад — `UIScreenEdgePanGestureRecognizer → UIPanGestureRecognizer`.
- **Жест в ячейке** — добавляем в `init` ячейки, а не в `cellForRowAt`; `cancelsTouchesInView` решает, выделится ли строка.

Apple Developer Documentation

- `UIGestureRecognizer` — базовый класс, состояния, `require(toFail:)`, `cancelsTouchesInView`.
- `UIGestureRecognizer.view` — единственная view, к которой привязан жест.
- `UIGestureRecognizerDelegate` — `gestureRecognizerShouldBegin(_:)`, `gestureRecognizer(_:shouldRecogniz`
- `UIView.gestureRecognizerShouldBegin(_:)` — тот же вопрос, но самой view, без делегата.
- `UIPanGestureRecognizer` — `translation(in:)`, `velocity(in:)`, `setTranslation(_:in:)`.
- `UITapGestureRecognizer`, `UILongPressGestureRecognizer`, `UIPinchGestureRecognizer`, `UIRotationGestureRecognizer`, `UISwipeGestureRecognizer`, `UIScreenEdgePanGestureRecognizer` — стандартные распознаватели.
- `UIContextMenuInteraction` и `UIContextMenuConfiguration` — контекстное меню с превью.
- `UISwipeActionsConfiguration` и `UIContextualAction` — действия по свайпу строки.
- `UIScrollView.DecelerationRate` — коэффициенты торможения прокрутки (`.normal` = 0,998).
- `UINavigationController.interactivePopGestureRecognizer` и `interactiveContentPopGestureRecognizer` — системные жесты возврата (второй — iOS 26+).
- HIG — Gestures — Apple о стандартных жестах и их смысле.

→ Глава 30. Cookbook — формы и валидация

Глава 30. Cookbook — формы и валидация

Вход, регистрация, адрес доставки, заявка — любая форма сводится к одному и тому же: поля ввода, проверка того, что в них написано, кнопка отправки и клавиатура, которая всё это норовит закрыть. Эта глава собирает рецепты для каждой части: проверку на лету, маску телефона, силу пароля, растущее поле для длинного текста, пошаговую форму, черновик, работу с клавиатурой и автозаполнение паролей и кодов из SMS.

> **В каком режиме код.** Листинги проверены компилятором в режиме Swift 6 с Default Actor Isolation = MainActor и минимальной версией iOS 15 (подробнее — в начале главы 25).

Пара терминов, которые встретятся везде.

First responder («первый отвечающий») — объект, которому UIKit первым отдаёт события клавиатуры. Поле ввода может им стать, потому что `UITextField` и `UITextView` — view, а всякий `UIView` — наследник `UIResponder` (глава 0, раздел 0.2). Когда ты тапаешь в поле, оно становится first responder, и iOS показывает клавиатуру. `becomeFirstResponder()` — сделать поле активным из кода, `resignFirstResponder()` — отпустить, после чего клавиатура уедет.

Два поля ввода устроены по-разному, и это видно по API. `UITextField` — контрол (`UITextField → UIControl → UIView`), поэтому у него есть target-action: `addAction(_:for:)`, события `.editingChanged` и `.editingDidEnd`. `UITextView` — не контрол, а scroll view (`UITextView → UIScrollView → UIView`): событий `UIControl.Event` у него нет, об изменениях сообщает делегат `UITextViewDelegate` (подробно — глава 0, разделы 0.4 и 0.5).

Валидация — проверка введённого: похоже ли это на email, хватает ли длины пароля, все ли цифры телефона на месте.

30.1 Inline validation

Когда применять. Поле показывает подсказку «правильно / нет» до нажатия кнопки отправки.

Главный вопрос — **когда** показывать ошибку. Если кричать «неверный email» после первой же буквы, форма выглядит так, будто ругает человека за то, что он ещё не допечатал. Хорошая схема:

- пока человек печатает — ошибку **не** показываем, а если она уже висит, убираем, как только поле стало правильным;

- когда он ушёл из поля (перешёл в другое) — проверяем и, если нужно, показываем ошибку;
- при нажатии «Отправить» — проверяем всё.

```
final class SignUpViewController: UIViewController {
    private let emailField = UITextField()
    private let hintLabel = UILabel()

    override func viewDidLoad() {
        super.viewDidLoad()
        hintLabel.font = .preferredFont(forTextStyle: .footnote)
        hintLabel.numberOfLines = 0
        emailField.addAction(UIAction { [weak self] _ in
            self?.validateEmail(showErrors: false)
        }, for: .editingChanged)
        emailField.addAction(UIAction { [weak self] _ in
            self?.validateEmail(showErrors: true)
        }, for: .editingDidEnd)
    }

    @discardableResult
    private func validateEmail(showErrors: Bool) -> Bool {
        let text = (emailField.text ?? "").trimmingCharacters(in: .whitespaces)
        let isValid = isValidEmail(text)
        if isValid {
            hintLabel.text = "Email выглядит правильно"
            hintLabel.textColor = .systemGreen
        } else if showErrors && !text.isEmpty {
            hintLabel.text = "Нужен формат you@example.kz"
            hintLabel.textColor = .systemRed
        } else if !showErrors {
            hintLabel.text = nil
        }
        return isValid
    }
}

func isValidEmail(_ text: String) -> Bool {
    let pattern = #"^[^\s@]+@[^\s@]+\.[^\s@]{2,}$"#
    return text.range(of: pattern, options: .regularExpression) != nil
}
```

Два события `UITextField` (оба — `UIControl.Event`, они есть у поля именно потому, что оно контрол):

- `.editingChanged` — на каждое изменение текста (нажатие клавиши, вставка, автозаполнение). Здесь `showErrors: false`: подтверждаем, если стало правильно, и молча убираем подсказку, если нет.

- `.editingDidEnd` — поле перестало быть активным (перешли в другое или закрыли клавиатуру). Здесь уже можно сказать «не то».

`validateEmail` возвращает `Bool` — пригодится при отправке формы. `@discardableResult` говорит компилятору: «результат можно не использовать», иначе на вызовах из действий были бы предупреждения.

Проверка email — **регулярное выражение** (шаблон для текста): `[^\s@]+` — «один или больше символов, кроме пробела и @», потом `@`, потом снова такие символы, точка и хотя бы два символа в конце. `a@b.kz` пройдёт, `a@b`, `a b@c.kz` и `@c.kz` — нет. Это **нестрогая** проверка, и так задумано: по-настоящему адрес проверяет только письмо, дошедшее до получателя. Строгие «правильные» регулярки для email занимают экран и всё равно отвергают часть реальных адресов.

Цвет — не единственный признак: ошибка ещё и написана словами. Людям, которые плохо различают красный и зелёный, одного цвета мало.

Частые ошибки.

- **Ошибка на первой букве** — см. выше.
- **Подсказка не для VoiceOver.** Когда появляется ошибка, объяви её: `UIAccessibility.post(notification: .announcement, argument: text)`.
- **Проверка без обрезки пробелов.** Автозаполнение и вставка иногда приносят пробел в конце, и правильный адрес «не проходит».

30.2 Submit button enabled только при валидной форме

Когда применять. Короткие формы (вход, 2–3 поля): кнопка неактивна, пока данных не хватает.

```
final class LoginFormViewController: UIViewController {
    private let emailField = UITextField()
    private let passwordField = UITextField()
    private let submitButton = UIButton(configuration: .filled())

    override func viewDidLoad() {
        super.viewDidLoad()
        submitButton.setTitle("Войти", for: .normal)
        for field in [emailField, passwordField] {
            field.addAction(UIAction { [weak self] _ in
                self?.updateSubmitState()
            }, for: .editingChanged)
        }
        updateSubmitState()
    }

    private func updateSubmitState() {
        let emailOk = isValidEmail(emailField.text ?? "")
        let passwordOk = (passwordField.text ?? "").count >= 8
        submitButton.isEnabled = emailOk && passwordOk
```

```
    }
}
```

Одно действие на оба поля: любое изменение пересчитывает состояние кнопки. В `viewDidLoad` вызываем `updateSubmitState()` сразу — чтобы кнопка была неактивной с самого начала, а не только после первого нажатия клавиши.

Кнопка создана через `UIButton.Configuration` (iOS 15+): у неё уже есть системный неактивный вид — приглушённый цвет. Вручную менять `alpha`, как делают со старыми кнопками, не нужно.

У этого приёма есть обратная сторона: неактивная кнопка не объясняет, **что** не так. Для длинных форм лучше оставить кнопку активной, а по нажатию подсветить все проблемные поля (30.1 с `showErrors: true`). Пример экрана входа целиком — глава 8 (Auth gate).

30.3 Password show/hide

Когда применять. Поле пароля с «глазом»: нажал — пароль виден, нажал ещё раз — снова точки.

```
final class PasswordField: UITextField {
    private let eyeButton = UIButton(type: .system)

    override init(frame: CGRect) {
        super.init(frame: frame)
        isSecureTextEntry = true
        textContentType = .password
        autocapitalizationType = .none
        autocorrectionType = .no

        var config = UIButton.Configuration.plain()
        config.image = UIImage(systemName: "eye.slash")
        config.contentInsets = NSDirectionalEdgeInsets(top: 0, leading: 8, bottom: 0,
↪   trailing: 8)
        eyeButton.configuration = config
        eyeButton.tintColor = .secondaryLabel
        eyeButton.accessibilityLabel = "Показать пароль"
        eyeButton.addAction(UIAction { [weak self] _ in
            self?.toggleVisibility()
        }, for: .touchUpInside)
        rightView = eyeButton
        rightViewMode = .always
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    private func toggleVisibility() {
        isSecureTextEntry.toggle()
        let visible = !isSecureTextEntry
```

```
        eyeButton.configuration?.image = UIImage(systemName: visible ? "eye" : "eye.slash")
        eyeButton.accessibilityLabel = visible ? "Скрыть пароль" : "Показать пароль"
    }
}
```

- `isSecureTextEntry = true` — вместо символов точки. Скопировать текст из такого поля нельзя.
- `rightView` — view внутри поля справа. `rightViewMode = .always` — видна всегда (есть ещё `.whileEditing`, `.unlessEditing`, `.never`).
- Кнопка лежит в свойстве поля, а в замыкании — только `[weak self]`. Если бы замыкание обращалось к кнопке через захват (`eye.setImage(...)` внутри действия кнопки `eye`), получился бы цикл удержания: кнопка держит действие, действие — кнопку (подробно — глава 27, 27.2).
- `contentInsets` — отступ 8 точек слева и справа от значка, чтобы он не прилипал к краю поля.
- `accessibilityLabel` меняется вместе со значком: VoiceOver должен говорить, что **сделает** кнопка.

Частые ошибки.

- **Текст стирается при повторном вводе.** Когда поле снова переключают в скрытый режим и начинают печатать, iOS по умолчанию стирает всё введённое ранее — так ведут себя все поля паролей. Это защитная особенность, а не ошибка твоего кода, но о ней стоит знать до того, как тестировщик заведёт баг.
- **«Глаз» без подписи для VoiceOver** — незрячий человек услышит «кнопка» и не поймёт, что она делает.

30.4 Phone mask +7 (___) ___-__-__

Когда применять. Поле телефона, которое само расставляет скобки, пробелы и дефисы: человек вводит только цифры.

Маска — шаблон, по которому форматируется ввод. Логику форматирования держим отдельно от поля: так её легко проверить без интерфейса.

```
enum PhoneFormatter {
    /// Десять цифр номера после кода страны.
    static func nationalDigits(from raw: String) -> String {
        var digits = raw.filter(\.isNumber)
        if raw.hasPrefix("+7") {
            digits.removeFirst()                    // «7» из «+7» — это код страны
        } else if digits == "8" {
            digits = ""                             // привычная «8» в начале номера
        } else if digits.count == 11, let first = digits.first, first == "7" || first ==
        ↪  "8" {
            digits.removeFirst()                    // вставили номер целиком: «8 701 …»
        }
        return String(digits.prefix(10))
    }
```

```
    /// «7011234567» → «+7 (701) 123-45-67»; неполный номер — неполная маска.
    static func format(_ national: String) -> String {
        guard !national.isEmpty else { return "" }
        var result = "+7 ("
        for (index, digit) in national.enumerated() {
            switch index {
            case 3: result += ") "
            case 6, 8: result += "-"
            default: break
            }
            result.append(digit)
        }
        return result
    }

    /// Текст поля после правки: что было, какой диапазон заменили и на что.
    static func apply(to current: String, range: NSRange, replacement: String) -> String {
        let text = current as NSString
        var range = range
        let removed = text.substring(with: range)
        if replacement.isEmpty, range.length > 0, !removed.contains(where: \.isNumber) {
            // Стёрли только разделитель («)», «-», пробел) — сотрём и цифру перед ним.
            let head = text.substring(to: range.location) as NSString
            let lastDigit = head.rangeOfCharacter(from: .decimalDigits, options:
             ↪  .backwards)
            if lastDigit.location != NSNotFound {
                range = NSRange(location: lastDigit.location,
                                length: range.location + range.length - lastDigit.location)
            }
        }
        let raw = text.replacingCharacters(in: range, with: replacement)
        return format(nationalDigits(from: raw))
    }
}
```

Три функции — три шага.

`nationalDigits` достаёт из любой строки цифры номера без кода страны. Разберём ветки:

- Текст начинается с «+7» (в поле уже есть маска) — первая цифра «7» — это код страны, убираем её.
- Поле было пустым, и человек нажал «8» — по привычке, как при звонке по Казахстану или России с «восьмёрки». Эту «8» не считаем цифрой номера.
- Вставили номер целиком, 11 цифр, начинается с 7 или 8 («8 701 123 45 67» из заметок) — отрезаем первую цифру.
- `prefix(10)` — номер после кода страны содержит ровно 10 цифр, лишнее отбрасываем.

`format` расставляет разделители. Индексы — позиции цифр с нуля: перед цифрой №3 (четвёртой) закрываем скобку, перед №6 и №8 — дефис. Для «7011234567»: «+7 (» + «701» + «) » + «123» + «-» + «45» + «-» + «67» = «+7 (701) 123-45-67». Пустой номер даёт пустую строку, поэтому поле можно очистить до конца.

`apply` — что получится после правки. Есть тонкость со стиранием. Если курсор стоит сразу после «) » и человек жмёт «стереть», стирается пробел — цифры не изменились, и форматирование вернуло бы тот же текст. Кнопка «стереть» перестала бы работать. Поэтому, если удалили только разделитель, мы расширяем диапазон назад до ближайшей цифры и стираем её тоже.

Проверка на числах (эти примеры прогнаны через код выше):

Ввод	Результат
набрали по одной `7011234567`	`+7 (701) 123-45-67`
набрали по одной `87011234567`	`+7 (701) 123-45-67`
вставили `8 701 123 45 67`	`+7 (701) 123-45-67`
набрали `701123456789` (12 цифр)	`+7 (701) 123-45-67`
стёрли пробел в `+7 (701) 2`	`+7 (702`

Поле, которое использует форматтер:

```
final class PhoneField: UITextField, UITextFieldDelegate {
    override init(frame: CGRect) {
        super.init(frame: frame)
        keyboardType = .phonePad
        textContentType = .telephoneNumber
        placeholder = "+7 (___) ___-__-__"
        delegate = self
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    var nationalNumber: String { PhoneFormatter.nationalDigits(from: text ?? "") }

    func textField(_ textField: UITextField,
                   shouldChangeCharactersIn range: NSRange,
                   replacementString string: String) -> Bool {
        textField.text = PhoneFormatter.apply(to: textField.text ?? "",
                                              range: range, replacement: string)
        let end = textField.endOfDocument
        textField.selectedTextRange = textField.textRange(from: end, to: end)
        textField.sendActions(for: .editingChanged)
        return false
    }
}
```

`textField(_:shouldChangeCharactersIn:replacementString:)` — метод делегата, который UIKit спрашивает **перед** каждым изменением: «вот диапазон, вот новый текст — применить?». Мы

считаем результат сами, записываем его в поле и отвечаем `false`: «сам не меняй, я уже всё сделал».

Две строки после записи текста легко забыть:

- Курсор ставится в конец. Когда текст задают из кода, курсор может оказаться где угодно, а маска рассчитана на ввод по порядку.
- `sendActions(for: .editingChanged)` — когда делегат возвращает `false` и текст меняется из кода, событие `.editingChanged` само **не приходит**. Без этой строки проверка формы из 30.2 не узнает, что телефон изменился, и кнопка «Отправить» не включится.

`keyboardType = .phonePad` — клавиатура с цифрами, `*` и `#` (документация `UIKeyboardType.phonePad`); знак «+» в нашей маске подставляет форматтер, набирать его не нужно. `textContentType = .telephoneNumber` — iOS предложит номер из карточки владельца телефона в «Контактах».

С iOS 26 у делегата есть и метод с несколькими диапазонами, `shouldChangeCharactersInRanges`, а метод с одним диапазоном помечен как «будет объявлен устаревшим». Для проекта с минимальной версией iOS 15 он по-прежнему основной.

Частые ошибки.

- **Нет `sendActions(for: .editingChanged)`** — валидация «не видит» ввод.
- **Не работает «стереть»** рядом с разделителем — нет расширения диапазона.
- **Храним форматированную строку.** На сервер отправляй `nationalNumber` (или «+7» + цифры), а не «+7 (701) 123-45-67»: у маски может поменяться вид, у номера — нет.

Упражнение 30.1. Что вернёт `PhoneFormatter.format("70112")`? А `PhoneFormatter.nationalDigits(from: "+7 (701) 12")`? Посчитай в уме, потом проверь в playground.

30.5 Password strength

Когда применять. Регистрация или смена пароля: показываем, насколько пароль надёжен, пока его печатают.

```
enum PasswordStrength {
    case weak, medium, strong

    var title: String {
        switch self {
        case .weak:   return "Слабый"
        case .medium: return "Средний"
        case .strong: return "Надёжный"
        }
    }

    var color: UIColor {
        switch self {
        case .weak:   return .systemRed
        case .medium: return .systemOrange
        case .strong: return .systemGreen
        }
```

```
    }
}

func passwordStrength(_ password: String) -> PasswordStrength {
    var score = 0
    if password.count >= 8  { score += 1 }
    if password.count >= 12 { score += 1 }
    if password.contains(where: \.isNumber) { score += 1 }
    if password.contains(where: \.isUppercase) && password.contains(where: \.isLowercase)
    ↪ {
        score += 1
    }
    if password.contains(where: { !$0.isLetter && !$0.isNumber }) { score += 1 }

    switch score {
    case 0...1: return .weak
    case 2...3: return .medium
    default:    return .strong
    }
}
```

Пять признаков, за каждый — балл: длина от 8, длина от 12, есть цифра, есть и заглавные, и строчные буквы, есть спецсимвол. Сумма от 0 до 5 переводится в оценку: 0–1 — слабый, 2–3 — средний, 4–5 — надёжный.

На числах:

- `qwerty` — короче 8, нет цифр, нет заглавных, нет спецсимволов: 0 баллов, «Слабый».
- `Almaty2026` — длина 10 (⩾ 8: +1), цифры (+1), заглавная и строчные (+1): 3 балла, «Средний».
- `Almaty-2026!` — длина 12 (+2), цифры (+1), регистры (+1), спецсимволы (+1): 5 баллов, «Надёжный».

У `enum` есть и `title`, и `color`: под полем показываем слово и цвет вместе (цвет в одиночку не все различают).

Это грубая эвристика. Она не знает, что `Password1!` — один из самых популярных паролей в мире, и поставит ему «Надёжный». Настоящие оценщики сверяют пароль со словарями утёкших паролей; пример такого алгоритма — zxcvbn от Dropbox (написан на JavaScript, есть порты на другие языки). Главную проверку всё равно делает сервер.

30.6 Auto-grow UITextView

Когда применять. Многострочное поле, которое растёт по мере ввода: комментарий, сообщение в чате, отзыв.

```
final class AutoGrowTextView: UITextView, UITextViewDelegate {
    var minHeight: CGFloat = 44
    var maxHeight: CGFloat = 120
    private lazy var heightConstraint = heightAnchor.constraint(equalToConstant:
    ↪ minHeight)
```

```
    override init(frame: CGRect, textContainer: NSTextContainer?) {
        super.init(frame: frame, textContainer: textContainer)
        font = .preferredFont(forTextStyle: .body)
        adjustsFontForContentSizeCategory = true
        isScrollEnabled = false
        delegate = self
        heightConstraint.isActive = true
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override var text: String! {
        didSet { updateHeight() }
    }

    override func layoutSubviews() {
        super.layoutSubviews()
        updateHeight()
    }

    func textViewDidChange(_ textView: UITextView) {
        updateHeight()
    }

    private func updateHeight() {
        guard bounds.width > 0 else { return }
        let fitting = sizeThatFits(CGSize(width: bounds.width, height:
        ↪  .greatestFiniteMagnitude))
        let newHeight = min(max(minHeight, fitting.height), maxHeight)
        isScrollEnabled = fitting.height > maxHeight
        if heightConstraint.constant != newHeight {
            heightConstraint.constant = newHeight
        }
    }
}
```

Как это работает:

- `sizeThatFits` спрашивает у текстового поля: «какой высоты ты хочешь быть при этой ширине?». Ширину даём текущую, высоту — «сколько угодно» (`greatestFiniteMagnitude`, самое большое число `CGFloat`).
- Результат зажимаем между 44 и 120 точками: `max(44, h)` не даёт стать ниже одной строки, `min(..., 120)` — выше примерно пяти строк. На числах: нужно 30 → будет 44; нужно 80 → 80; нужно 200 → 120.
- `isScrollEnabled`. `UITextView` — наследник `UIScrollView`, и пока прокрутка включена, он ведёт себя как любой scroll view: его `intrinsicContentSize` равен `UIView.noIntrinsicMetric` (−1 × −1 в замере на симуляторе), размер ему должен дать кто-то снаружи. С `isScrollEnabled`

`= false` он начинает сообщать размер по содержимому (в замере — 80 × 30 точек для одной короткой строки). Пока текст помещается, прокрутка выключена; когда нужно больше 120, прокрутку **включаем**, иначе текст ниже пятой строки нельзя было бы увидеть: длинный текст просто обрезался бы.

- Ограничение высоты создаётся в `init` — `lazy var` откладывает его создание до первого обращения, когда `self` уже готов. Никаких `NSLayoutConstraint!`, которые упадут, если их забыли задать снаружи.
- Высота пересчитывается в трёх местах: при вводе (`textViewDidChange` — метод делегата, потому что у `UITextView` нет события `.editingChanged`), при смене текста из кода (`didSet` у `text` — делегат в этом случае не вызывается) и при смене ширины (`layoutSubviews`, например после поворота экрана). `guard bounds.width > 0` — до первой раскладки ширина нулевая, и считать нечего.

Полный пример в строке сообщения чата — глава 18 (Chat).

30.7 Form wizard (multi-step)

Когда применять. Длинная регистрация, которую удобнее пройти по шагам: данные → телефон → код из SMS → согласия. **Мастер** (wizard) — форма из нескольких экранов с «Далее» и индикатором прогресса.

```
final class WizardViewController: UIViewController {
    var onFinish: (() -> Void)?

    private let pages: [UIViewController]
    private var currentIndex = 0
    private let pageVC = UIPageViewController(transitionStyle: .scroll,
                                              navigationOrientation: .horizontal)
    private let progressView = UIProgressView(progressViewStyle: .bar)

    init(pages: [UIViewController]) {
        precondition(!pages.isEmpty, "Мастеру нужен хотя бы один шаг")
        self.pages = pages
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) { fatalError("init(coder:) не используется") }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground

        addChild(pageVC)
        pageVC.view.translatesAutoresizingMaskIntoConstraints = false
        progressView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(progressView)
        view.addSubview(pageVC.view)
        NSLayoutConstraint.activate([
```

```
            progressView.topAnchor.constraint(equalTo:
↪   view.safeAreaLayoutGuide.topAnchor),
            progressView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            progressView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            pageVC.view.topAnchor.constraint(equalTo: progressView.bottomAnchor),
            pageVC.view.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            pageVC.view.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            pageVC.view.bottomAnchor.constraint(equalTo: view.bottomAnchor),
        ])
        pageVC.didMove(toParent: self)

        pageVC.setViewControllers([pages[0]], direction: .forward, animated: false)
        updateProgress()
    }

    func goNext() {
        guard currentIndex < pages.count - 1 else {
            onFinish?()
            return
        }
        currentIndex += 1
        pageVC.setViewControllers([pages[currentIndex]], direction: .forward, animated:
↪   true)
        updateProgress()
    }

    func goBack() {
        guard currentIndex > 0 else { return }
        currentIndex -= 1
        pageVC.setViewControllers([pages[currentIndex]], direction: .reverse, animated:
↪   true)
        updateProgress()
    }

    private func updateProgress() {
        progressView.setProgress(Float(currentIndex + 1) / Float(pages.count), animated:
↪   true)
    }
}
```

`UIPageViewController` — container view controller (контейнер), который показывает по одному экрану и умеет анимировать переход к другому; семейство контейнеров — глава 0, раздел 0.6. Он встроен в мастер как **дочерний контроллер** (child view controller): `addChild`, добавить его view, прибить ограничениями, `didMove(toParent:)`. Эти три шага обязательны — без `addChild` дочерний экран не получает событий жизненного цикла (`viewWillAppear` и другие), потому что UIKit пересылает их только по цепочке `parent → children`.

Обрати внимание: у `pageVC` **нет источника данных** (`dataSource`). Документация к этому свойству говорит прямо: если оно `nil`, навигация жестами отключена. Поэтому листать шаги пальцем нельзя — только кнопками «Далее» и «Назад», которые вызывают `goNext()` и `goBack()`. Для формы это правильно: иначе человек проскочит шаг, не заполнив его, свайпом.

Прогресс на числах: четыре шага; на первом `(0 + 1) / 4 = 0,25` — полоса заполнена на четверть, на последнем `4 / 4 = 1` — полностью.

Каждый шаг — отдельный экран со своей проверкой полей. Кнопка «Далее» внутри шага вызывает мастер через замыкание или делегата: `step.onNext = { [weak wizard] in wizard?.goNext() }`.

30.8 Save as draft

Когда применять. Длинная форма: заявка, отзыв, отчёт. Если человек случайно закроет экран или приложение выгрузится из памяти, написанное не должно пропасть.

```
struct ReportDraft: Codable {
    var text: String
    var savedAt: Date
}

final class ReportViewController: UIViewController, UITextViewDelegate {
    private static let draftKey = "draft.report"
    private let textView = UITextView()
    private var saveWorkItem: DispatchWorkItem?

    override func viewDidLoad() {
        super.viewDidLoad()
        textView.delegate = self
        if let data = UserDefaults.standard.data(forKey: Self.draftKey),
           let draft = try? JSONDecoder().decode(ReportDraft.self, from: data) {
            textView.text = draft.text
        }
    }

    func textViewDidChange(_ textView: UITextView) {
        saveWorkItem?.cancel()
        let work = DispatchWorkItem { [weak self] in
            self?.saveDraft()
        }
        saveWorkItem = work
        DispatchQueue.main.asyncAfter(deadline: .now() + 1.0, execute: work)
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        saveWorkItem?.cancel()
        saveDraft()
```

```
    }

    private func saveDraft() {
        let draft = ReportDraft(text: textView.text, savedAt: Date())
        guard let data = try? JSONEncoder().encode(draft) else { return }
        UserDefaults.standard.set(data, forKey: Self.draftKey)
    }

    func didSubmitSuccessfully() {
        saveWorkItem?.cancel()
        UserDefaults.standard.removeObject(forKey: Self.draftKey)
    }
}
```

- **Сохранение с паузой** — тот же debounce, что в поиске (глава 25, 25.2): каждое изменение отменяет прошлое задание и назначает новое через 1 секунду. Пока человек печатает, запись не идёт; замолчал на секунду — черновик сохранён.
- `viewWillDisappear` — при уходе с экрана сохраняем **сразу**, не дожидаясь секунды: иначе последние слова, набранные прямо перед закрытием, пропадут вместе с отменённым заданием.
- `Codable` — черновик превращается в JSON (`JSONEncoder`) и обратно (`JSONDecoder`). `UserDefaults` хранит получившиеся байты (`Data`).
- `didSubmitSuccessfully` — после успешной отправки черновик удаляем. Иначе в другой раз форма откроется со старым, уже отправленным текстом.
- `super.viewDidLoad()` — не пропускай вызов родителя в методах жизненного цикла.

`UserDefaults` подходит для черновика в несколько килобайт. Если в черновике фотографии или десятки страниц текста — сохраняй в файл (глава 13, FileManager).

30.9 Keyboard avoidance

Когда применять. Всегда, когда на экране есть поля ввода: клавиатура занимает около 40% высоты iPhone и закрывает всё, что было в нижней части экрана.

С iOS 15 у каждой view есть **направляющая клавиатуры** (`keyboardLayoutGuide`). Это не view, а layout guide — объект класса `UIKeyboardLayoutGuide → UITrackingLayoutGuide → UILayoutGuide → NSObject` (цепочка снята на симуляторе): невидимый прямоугольник с якорями, к которым можно привязывать констрейнты (о layout guide — глава 0, раздел 0.3). Пока клавиатура пристыкована к нижнему краю экрана, направляющая совпадает с ней; когда клавиатуры нет, её верх (`topAnchor`) совпадает с низом `safeAreaLayoutGuide`. Достаточно привязать к ней низ формы:

```
final class AddressFormViewController: UIViewController {
    private let scrollView = UIScrollView()
    private let stack = UIStackView()

    override func viewDidLoad() {
        super.viewDidLoad()
```

```
        view.backgroundColor = .systemBackground
        scrollView.keyboardDismissMode = .interactive
        scrollView.translatesAutoresizingMaskIntoConstraints = false
        stack.axis = .vertical
        stack.spacing = 12
        stack.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(scrollView)
        scrollView.addSubview(stack)

        NSLayoutConstraint.activate([
            scrollView.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor),
            scrollView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
            scrollView.trailingAnchor.constraint(equalTo: view.trailingAnchor),
            scrollView.bottomAnchor.constraint(equalTo:
↪   view.keyboardLayoutGuide.topAnchor),

            stack.topAnchor.constraint(equalTo: scrollView.contentLayoutGuide.topAnchor,
↪   constant: 16),
            stack.bottomAnchor.constraint(equalTo:
↪   scrollView.contentLayoutGuide.bottomAnchor, constant: -16),
            stack.leadingAnchor.constraint(equalTo:
↪   scrollView.frameLayoutGuide.leadingAnchor, constant: 16),
            stack.trailingAnchor.constraint(equalTo:
↪   scrollView.frameLayoutGuide.trailingAnchor, constant: -16),
        ])
    }
}
```

Главная строка — `scrollView.bottomAnchor ... keyboardLayoutGuide.topAnchor`: низ прокручиваемой области всегда стоит над клавиатурой. Клавиатура выехала — область стала короче, и нижние поля можно докрутить до видимой части. Анимация сжатия идёт синхронно с клавиатурой, без единой строки кода. Пример с полем сообщения в чате — глава 18, раздел 18.6.

Форма лежит в `UIScrollView` не случайно. Если просто прибить кнопку «Отправить» к клавиатуре без прокрутки, на маленьком экране поля окажутся **под** верхним краем: для формы из восьми полей по 44 точки плюс промежутки нужно около 450 точек, а над клавиатурой на iPhone SE остаётся около 300.

- `stack` прибит к `contentLayoutGuide` сверху и снизу — это задаёт высоту прокручиваемого содержимого.
- По бокам — к `frameLayoutGuide` (видимой рамке скролла): ширина содержимого равна ширине экрана, горизонтальной прокрутки нет.
- `keyboardDismissMode = .interactive` — клавиатуру можно «утащить» вниз пальцем, прокручивая форму, как в «Сообщениях».

Обычно UIKit сам докручивает scroll view так, чтобы активное поле оказалось над клавиатурой, но в документации это поведение не обещано. Если поле всё же осталось под клавиатурой или закрывает что-то своё (например, подсказку под полем), докрути вручную:

```
func textFieldDidBeginEditing(_ textField: UITextField) {
    let rect = textField.convert(textField.bounds, to: scrollView)
        .insetBy(dx: 0, dy: -40)
    scrollView.scrollRectToVisible(rect, animated: true)
}
```

`convert(_:to:)` переводит рамку поля в координаты скролла. `insetBy(dx: 0, dy: -40)` **расширяет** прямоугольник на 40 точек вверх и вниз (отрицательный отступ — наружу), чтобы вместе с полем была видна и подсказка под ним.

В старом коде ты встретишь другой способ — подписку на уведомления клавиатуры:

```
@objc func keyboardWillChange(_ note: Notification) {
    guard let value = note.userInfo?[UIResponder.keyboardFrameEndUserInfoKey] as? NSValue,
          let duration = note.userInfo?[UIResponder.keyboardAnimationDurationUserInfoKey]
          ↪ as? Double
    else { return }
    let keyboardFrame = view.convert(value.cgRectValue, from: nil)
    let overlap = max(0, view.bounds.maxY - keyboardFrame.minY -
    ↪ view.safeAreaInsets.bottom)
    bottomConstraint.constant = -overlap
    UIView.animate(withDuration: duration) { self.view.layoutIfNeeded() }
}
```

Его подключают через `NotificationCenter` на `keyboardWillChangeFrameNotification`. Рамка клавиатуры приходит в координатах экрана (так сказано в документации `keyboardFrameEndUserInfoKey`); `convert(_:from: nil)` переводит из координат окна в координаты view — у полноэкранного приложения координаты окна и экрана совпадают, а в Split View и Stage Manager на iPad уже нет, и это ещё один довод в пользу `keyboardLayoutGuide`. **Перекрытие** — сколько клавиатура залезла на view снизу: экран высотой 852, клавиатура начинается на 516 — перекрытие 336 точек, минус нижний отступ безопасной области 34 — поднимаем содержимое на 302. При минимальной версии iOS 15 этот код не нужен: `keyboardLayoutGuide` делает то же самое сам, включая поворот и смену высоты при переключении языка.

Одна оговорка про iPad. Отстыкованную, разделённую или плавающую клавиатуру направляющая по умолчанию **не** отслеживает: свойство `followsUndockedKeyboard` равно `false`, и в этих режимах она просто стоит у низа safe area — так описано в заголовке `UIKeyboardLayoutGuide.h`. Хочешь, чтобы форма уходила и от плавающей клавиатуры, — включи `followsUndockedKeyboard = true`.

30.10 Return key flow

Когда применять. Форма из нескольких полей: кнопка на клавиатуре переводит в соседнее поле, на последнем — отправляет.

```
final class CredentialsViewController: UIViewController, UITextFieldDelegate {
    private let emailField = UITextField()
    private let passwordField = UITextField()
```

```
    override func viewDidLoad() {
        super.viewDidLoad()
        emailField.returnKeyType = .next
        emailField.delegate = self
        passwordField.returnKeyType = .go
        passwordField.delegate = self
    }

    func textFieldShouldReturn(_ textField: UITextField) -> Bool {
        if textField === emailField {
            passwordField.becomeFirstResponder()
        } else {
            textField.resignFirstResponder()
            submit()
        }
        return false
    }

    private func submit() {}
}
```

`returnKeyType` — надпись на синей кнопке клавиатуры: `.next` («Далее»), `.go` («Перейти»), `.done` («Готово»), `.send`, `.search`. Надпись не меняет поведения — поведение пишешь сам в `textFieldShouldReturn`.

`textFieldShouldReturn` вызывается при нажатии этой кнопки. На поле email делаем активным поле пароля: `becomeFirstResponder()`, и клавиатура не прячется, а просто «переезжает». На поле пароля отпускаем фокус (клавиатура уедет) и отправляем форму. Возвращаемое значение, по документации, — «выполнять ли полю стандартную обработку кнопки Return». У однострочного `UITextField` заметной стандартной обработки нет (перевод строки в него не вставишь), так что главное — то, что ты сделал внутри метода; `false` здесь читается как «обработал сам».

`===` сравнивает **объекты**: это то же самое поле, а не поле с таким же текстом.

30.11 TextField suggestions / textContentType

Когда применять. Любое поле, для которого iOS умеет что-то подсказать: логин, пароль, код из SMS, адрес, телефон, имя.

```
func configureAutoFill() {
    loginField.textContentType = .username
    loginField.keyboardType = .emailAddress
    passwordField.textContentType = .password
    passwordField.isSecureTextEntry = true

    newPasswordField.textContentType = .newPassword
    newPasswordField.isSecureTextEntry = true
    newPasswordField.passwordRules = UITextInputPasswordRules(
```

```
        descriptor: "required: lower; required: upper; required: digit; minlength: 8;"
    )

    codeField.textContentType = .oneTimeCode
    codeField.keyboardType = .numberPad

    phoneField.textContentType = .telephoneNumber
}
```

`textContentType` говорит системе, **что** в поле, и от этого зависит подсказка над клавиатурой (строка QuickType):

- `.username` — логин. Если логин — это email, всё равно ставь `.username`, а email-клавиатуру задай отдельно через `keyboardType`: так рекомендует Apple, иначе автозаполнение паролей хуже понимает, что это форма входа.
- `.password` — существующий пароль: iOS предложит сохранённый.
- `.newPassword` — новый пароль при регистрации: iOS предложит сгенерированный надёжный пароль и сохранит его. `passwordRules` описывает требования сервера — строчная и заглавная буквы, цифра, не короче 8 символов, — и сгенерированный пароль им будет соответствовать.
- `.oneTimeCode` — одноразовый код. Когда приходит SMS с кодом, iOS показывает его над клавиатурой: один тап — и код в поле. Подсказка живёт недолго — несколько минут после прихода сообщения, потом исчезает.
- `.telephoneNumber` — номер из карточки владельца в «Контактах».

Что нужно, чтобы это работало в полную силу:

- **Связанный домен** (associated domains). Сохранённые пароли для твоего сайта iOS предлагает сразу в строке подсказок, а надёжный пароль для `.newPassword` генерирует, только если приложение связано со своим доменом: в проекте включена возможность Associated Domains с записью `webcredentials:example.kz`, а на сайте лежит файл `apple-app-site-association`. Без связи человек всё равно может выбрать пароль вручную, через значок ключа.
- **Для кода из SMS** ничего дополнительно настраивать не нужно, но текст сообщения должен быть таким, чтобы iOS распознала в нём код. Проверить просто: отправь такое SMS себе. Если код в нём подчёркнут и по тапу есть «Скопировать код», система его понимает.
- **Не снимай first responder, когда приложение становится неактивным.** Когда человек выбирает пароль из подсказки, iOS просит Face ID, и приложение на мгновение теряет активность. Если твоя заглушка (как в главе 11) в этот момент вызывает `endEditing(true)` или убирает форму из иерархии view, поле перестаёт быть first responder, и автозаполнению некуда вставить данные. Накладка поверх формы, которая саму форму не трогает, этому не мешает.

30.12 Disable autocorrect

```
func configureLoginField(_ field: UITextField) {
    field.autocapitalizationType = .none
    field.autocorrectionType = .no
```

```
    field.spellCheckingType = .no
    field.smartQuotesType = .no
    field.smartDashesType = .no
}
```

По умолчанию `UITextField` делает первую букву предложения заглавной (`autocapitalizationType = .sentences` — так и записано в заголовке `UITextInputTraits.h`) и исправляет «опечатки». В обычном тексте это удобно, а в логине — вредно: «ivanov» превращается в «Ivanov», и вход не проходит, хотя человек уверен, что ввёл всё правильно.

Выключаем для email, логина, пароля, промокода, адреса сайта:

- `autocapitalizationType = .none` — без автоматических заглавных. Email-клавиатура (`keyboardType = .emailAddress`) эту настройку сама **не** меняет — выключай явно.
- `autocorrectionType = .no` — без автоисправления.
- `spellCheckingType = .no` — без красного подчёркивания «ошибок».
- `smartQuotesType`, `smartDashesType` (iOS 11+) — без замены прямых кавычек на «ёлочки» и двух дефисов на тире: в пароле "--" должно остаться "--".

Упражнение 30.2. Собери экран регистрации из трёх полей: email, новый пароль, повтор пароля. Какие `textContentType`, `keyboardType`, `returnKeyType` и настройки автоисправления ты поставишь каждому полю? Какое условие включает кнопку «Создать аккаунт»?

Ответы к упражнениям

30.1. `format("70112")`: цифры с индексами 0–4. Перед индексом 3 ставится «) », поэтому результат — «+7 (701) 12». `nationalDigits(from: "+7 (701) 12")`: все цифры — «770112», строка начинается с «+7», поэтому первая «7» (код страны) отрезается — остаётся «70112».

30.2. Возможный вариант:

```
emailField.textContentType = .username
emailField.keyboardType = .emailAddress
emailField.returnKeyType = .next
emailField.autocapitalizationType = .none
emailField.autocorrectionType = .no

passwordField.textContentType = .newPassword
passwordField.isSecureTextEntry = true
passwordField.returnKeyType = .next

repeatField.textContentType = .newPassword
repeatField.isSecureTextEntry = true
repeatField.returnKeyType = .done

let canSubmit = isValidEmail(emailField.text ?? "")
    && (passwordField.text ?? "").count >= 8
    && passwordField.text == repeatField.text
```

Оба поля пароля — `.newPassword`: тогда iOS подставит сгенерированный пароль сразу в оба. Кнопка включается, когда email похож на адрес, пароль не короче 8 символов и повтор совпадает с паролем. В `textFieldShouldReturn` первые два поля передают фокус дальше, а последнее отправляет форму (как в 30.10).

Что мы выучили

- **Проверка на лету**: `.editingChanged` — только подтверждать и убирать ошибку, `.editingDidEnd` и отправка — показывать ошибку; цвет плюс текст.
- **Email** проверяем нестрого (`something@domain.xx`); настоящая проверка — письмо.
- **Кнопка отправки** неактивна, пока форма неполна; `UIButton.Configuration` сам рисует неактивный вид.
- **Показ пароля** — `isSecureTextEntry.toggle()`, кнопка в `rightView` без захвата самой себя в замыкании.
- **Маска телефона** — отдельный форматтер (`nationalDigits`, `format`, `apply`), `return false` в делегате, курсор в конец и `sendActions(for: .editingChanged)`.
- **Сила пароля** — баллы за длину и разнообразие символов; это подсказка, главная проверка на сервере.
- **Растущее поле** — `sizeThatFits`, зажим между минимумом и максимумом, прокрутка включается выше максимума.
- **Мастер** — `UIPageViewController` без `dataSource`, дочерний контроллер через `addChild`/`didMove`, прогресс `(шаг + 1) / всего`.
- **Черновик** — сохранение с паузой 1 с, немедленное при уходе с экрана, удаление после отправки.
- **Два поля ввода**: `UITextField` — контрол (target-action, `.editingChanged`), `UITextView` — scroll view (делегат, `intrinsicContentSize` только при `isScrollEnabled = false`).
- **Клавиатура** — форма в `UIScrollView`, низ скролла к `keyboardLayoutGuide.topAnchor` (iOS 15+); направляющая — layout guide, а не view; плавающую клавиатуру iPad отслеживает только с `followsUndockedKeyboard = true`.
- **Return** — `returnKeyType` + `textFieldShouldReturn` для перехода между полями.
- **Автозаполнение** — `.username` (даже для email), `.password`, `.newPassword` + `passwordRules`, `.oneTimeCode`; для паролей нужен связанный домен.
- **Автоисправление** выключаем для логинов, паролей и email явно.

Apple Developer Documentation

- `UITextField` — однострочное поле, `rightView`, `rightViewMode`, `isSecureTextEntry`.
- `UITextFieldDelegate` — `textField(_:shouldChangeCharactersIn:replacementString:)`, `textFieldShouldReturn(_:)`.
- `UITextContentType` — `.username`, `.password`, `.newPassword`, `.oneTimeCode`, `.telephoneNumber`.
- Password AutoFill — как работает автозаполнение паролей и зачем связанный домен.
- Enabling Password AutoFill on a text input view — какие `textContentType` ставить в форме входа и регистрации.
- `UITextInputPasswordRules` — требования к сгенерированному паролю.
- `UITextInputTraits` — `autocapitalizationType`, `autocorrectionType`, `spellCheckingType`, `returnKeyType`, `keyboardType`.
- `UITextView` и `UITextViewDelegate` — многострочный ввод.

- `UIPageViewController` — контейнер шагов мастера.
- `UIView.keyboardLayoutGuide` — направляющая клавиатуры (iOS 15+).
- `UIKeyboardLayoutGuide.followsUndockedKeyboard` — отслеживать ли плавающую и разделённую клавиатуру iPad (по умолчанию `false`).
- `UIPageViewController.dataSource` — без источника данных навигация жестами отключена.
- `UIResponder.keyboardWillChangeFrameNotification` — уведомление о смене рамки клавиатуры (старый способ).
- HIG — Text fields — Apple о полях ввода и подсказках.

→ Глава 31. Cookbook — дата, время, деньги

Глава 31. Cookbook — дата, время, деньги

Всё, что показывает человеку `Date` или сумму денег, проходит через **форматтер** — объект, который превращает значение в строку по правилам конкретного языка и страны. В этой главе собраны рецепты для дат, длительностей, чисел и денег, и главное — реальный вывод этих рецептов для трёх локалей, с которыми ты скорее всего столкнёшься: `ru_RU`, `ru_KZ` и `kk_KZ`.

Два слова, без которых дальше никуда.

Локаль (`Locale`) — набор региональных правил: язык, страна, разделитель тысяч, запятая или точка в дробях, какая валюта «своя», с какого дня начинается неделя. Идентификатор складывается из языка и страны: `ru_KZ` — русский язык по правилам Казахстана, `kk_KZ` — казахский язык в Казахстане, `ru_RU` — русский в России. Язык один, страны разные — и форматтер покажет разную валюту.

Часовой пояс (`TimeZone`) — сдвиг местного времени относительно **UTC**, всемирного координированного времени (это «время по нулевому меридиану», от которого считают все пояса). Алматы сейчас живёт в UTC+5: когда в Лондоне по UTC 09:30, в Алматы 14:30.

> **Как проверены примеры.** Все строки вида `// "..."` в этой главе — настоящий вывод, снятый программой на симуляторе iOS 26.5. На Mac те же форматтеры иногда печатают другое (другие версии данных о локалях), поэтому проверяй на симуляторе или устройстве. Код главы проверен компилятором в режиме Swift 6 с изоляцией `MainActor` по умолчанию (о режимах компилятора — во Введении).

31.1 DateFormatter — база

Когда применять. Показать дату или время в интерфейсе: «12 мая 2026 г.», «14:30», «вторник».

```
let formatter = DateFormatter()
formatter.locale = Locale(identifier: "ru_RU")
formatter.dateStyle = .medium
formatter.timeStyle = .short
let text = formatter.string(from: date)
// "12 мая 2026 г., 14:30"
```

Разбор:

- `locale` — по чьим правилам писать. Если не задать, возьмётся `Locale.current` — локаль, выбранная пользователем в Настройках (обычно тебе именно это и нужно; жёсткий `ru_RU` здесь только чтобы пример был воспроизводимым).
- `dateStyle` и `timeStyle` — готовые уровни подробности. Отдельно для даты и для времени, `.none` выключает часть.
- `string(from:)` — сама конвертация.

Четыре уровня для даты 12 мая 2026 года выглядят так (вывод симулятора):

Стиль	`ru_RU` и `ru_KZ`	`kk_KZ`	`en_US`
`.short`	12.05.2026	12.05.26	5/12/26
`.medium`	12 мая 2026 г.	2026 ж. 12 мам.	May 12, 2026
`.long`	12 мая 2026 г.	2026 ж. 12 мамыр	May 12, 2026
`.full`	вторник, 12 мая 2026 г.	2026 ж. 12 мамыр, сейсенбі	Tuesday, May 12, 2026

Обрати внимание: в русских локалях `.medium` и `.long` совпадают и оба добавляют «г.», а казахская локаль ставит год **первым**. Именно поэтому стили лучше ручных форматов: ты не знаешь заранее, в каком порядке человек привык читать дату.

Кастомный формат — когда нужен конкретный вид:

```
formatter.dateFormat = "d MMMM yyyy"   // "12 мая 2026"
formatter.dateFormat = "HH:mm"         // "14:30"
formatter.dateFormat = "EEEE"          // "вторник"
formatter.dateFormat = "yyyy-MM-dd"    // "2026-05-12"
```

Буквы в шаблоне — это поля даты: `d` — день, `MMMM` — месяц словом, `yyyy` — год, `HH` — часы от 00 до 23, `mm` — минуты, `EEEE` — день недели словом. Две ловушки:

- **Жёсткий формат не переставляет поля под локаль.** `"d MMMM yyyy"` в `en_US` даст «12 May 2026», хотя американец ждёт «May 12». Если нужен «набор полей», а порядок пусть решает локаль, используй шаблон: `formatter.setLocalizedDateFormatFromTemplate("dMMMM")` — для `ru_RU` получится «12 мая», для `en_US` — «May 12».
- **Месяц без дня — буква `L`, а не `M`.** В русском месяц склоняется: «12 мая», но «май 2026». `"MMMM yyyy"` печатает «мая 2026» (родительный падеж), а `"LLLL yyyy"` — правильное «май 2026». `L` означает «самостоятельная форма месяца».

  Кешируй форматтеры. Создание `DateFormatter` — сравнительно дорогая операция: он загружает данные локали. Для ячейки таблицы, которая форматирует дату при каждом показе, заведи один экземпляр и используй его повторно.

```
enum Formatters {
    static let mediumDateTime: DateFormatter = {
        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "ru_RU")
        formatter.dateStyle = .medium
        formatter.timeStyle = .short
        return formatter
```

```
    }()
}

// В ячейке:
dateLabel.text = Formatters.mediumDateTime.string(from: note.createdAt)
```

`static let` в Swift ленивый: замыкание выполнится один раз, при первом обращении, дальше все берут готовый объект. Пустой `enum` здесь — просто «папка» для констант, у него нельзя создать экземпляр.

31.2 RelativeDateTimeFormatter — «5 минут назад»

Когда применять. Ленты, чаты, комментарии — там, где важнее «как давно», чем точная дата.

```
let formatter = RelativeDateTimeFormatter()
formatter.locale = Locale(identifier: "ru_RU")
formatter.unitsStyle = .full
let text = formatter.localizedString(for: someDate, relativeTo: Date())
// someDate на 5 минут раньше → "5 минут назад"
```

`localizedString(for:relativeTo:)` берёт две даты и описывает разницу между ними: первая — событие, вторая — «сейчас». Склонения форматтер делает сам: «5 минут назад», но «21 минуту назад».

Стили `unitsStyle` для трёх случаев (−5 минут, +2 дня, −21 минута):

Стиль	`ru_RU`	`kk_KZ`
`.full`	5 минут назад · через 2 дня · 21 минуту назад	5 минут бұрын · 2 күннен кейін · 21 минут бұрын
`.short`	5 мин. назад · через 2 дн. · 21 мин. назад	5 мин. бұрын · 2 күннен кейін · 21 мин. бұрын
`.abbreviated`	-5 мин · +2 дн · -21 мин	5 мин. бұрын · 2 күннен кейін · 21 мин. бұрын
`.spellOut`	пять минут назад · через два дня · двадцать один минуту назад	бес минут бұрын · …

Последняя строка — не опечатка в книге: `.spellOut` на iOS 26.5 печатает «двадцать один минуту» (род не согласован). Прописью по-русски этот стиль лучше не показывать.

Полезная настройка — `dateTimeStyle = .named`. Тогда вместо «1 день назад» будет «вчера», вместо «через 1 день» — «завтра», а для нулевой разницы — «сейчас» (без неё форматтер пишет «через 0 секунд»).

`RelativeDateTimeFormatter` появился в iOS 13, так что для нашего iOS 15 доступен без проверок.

31.3 DateComponentsFormatter — длительности

Когда применять. Длительность — это не момент, а отрезок: «трек длится 3:25», «до конца 1 ч 23 мин».

```
let formatter = DateComponentsFormatter()
formatter.allowedUnits = [.hour, .minute, .second]
formatter.unitsStyle = .abbreviated
formatter.string(from: 4980)   // 4980 секунд → "1 ч 23 мин" (ru)
```

Число `4980` — секунды: 4980 = 3600 (час) + 1380, а 1380 секунд — это 23 минуты ровно, поэтому секунд в ответе нет. `allowedUnits` задаёт, какими единицами можно пользоваться.

Вывод для 4980 секунд в `ru_RU`:

- `.positional` — «1:23:00» (как на часах плеера);
- `.abbreviated` и `.short` — «1 ч 23 мин»;
- `.full` — «1 час 23 минуты»;
- `.spellOut` — «один час двадцать три минуты».

У этого форматтера нет свойства `locale`: язык берётся из его `calendar`. Чтобы задать язык явно, создай календарь с нужной локалью и присвой `formatter.calendar = calendar`. В приложении обычно этого не делают — пользователю показывают его собственный язык.

Для таймера плеера пригодится ещё одна настройка:

```
formatter.allowedUnits = [.minute, .second]
formatter.unitsStyle = .positional
formatter.zeroFormattingBehavior = .pad
formatter.string(from: 65)     // "01:05"
formatter.string(from: 3725)   // "62:05"
```

`.pad` добивает нули слева: 65 секунд — «01:05», а не «1:05». Во втором примере часов в `allowedUnits` нет, поэтому 1 час 2 минуты 5 секунд превращаются в «62:05».

> В iOS 16 появился тип `Duration` со своим форматированием (`Duration.seconds(4980).formatted(...)`). На iOS 15 его нет — компилятор с deployment target 15 откажется собирать такой код без `if #available(iOS 16, *)`.

31.4 Календарные операции

Календарь (`Calendar`) — это правила счёта дней: сколько дней в месяце, какой год високосный, когда переводят часы. `Calendar.current` — календарь пользователя, с его часовым поясом.

```
let cal = Calendar.current
let now = Date()

// Начало текущего дня (00:00 в часовом поясе календаря)
let today = cal.startOfDay(for: now)

// Та же минута завтра
```

```swift
let tomorrow = cal.date(byAdding: .day, value: 1, to: now)

// День недели: 1 — воскресенье, 2 — понедельник, …, 7 — суббота
let weekday = cal.component(.weekday, from: now)

// Сегодня ли?
let isToday = cal.isDateInToday(someDate)

// Полных лет
let age = cal.dateComponents([.year], from: birthDate, to: now).year ?? 0
```

Несколько тонкостей, о которые спотыкаются:

- Нумерация `weekday` **не зависит от локали**: воскресенье всегда 1. От локали зависит другое — `cal.firstWeekday`, первый день недели в календарной сетке. У `ru_RU` и `kk_KZ` он равен 2 (понедельник), у `en_US` — 1 (воскресенье).
- `dateComponents([.year], from:to:)` считает **полные** годы. Для рождённого 13 мая 2000 года на 12 мая 2026-го получится 25, а не 26: день рождения ещё не наступил. Проверено на симуляторе.
- `date(byAdding:)` возвращает опционал: теоретически не каждую дату можно сдвинуть (например, на несуществующий час при переводе часов).

Почему нельзя `date.addingTimeInterval(86400)`. 86 400 — это секунд в сутках: 24 × 60 × 60. Но не каждые сутки длятся 24 часа. В странах с летним временем (DST, daylight saving time — «перевод часов на час вперёд весной») в день перевода в сутках 23 часа. Живой пример с симулятора: в Нью-Йорке 7 марта 2026 года в 12:00 прибавляем 86 400 секунд и получаем 8 марта **13:00**, а `cal.date(byAdding: .day, value: 1, to:)` даёт 8 марта **12:00**.

В Казахстане летнего времени нет с 2005 года, но зато в 2024 году сменился сам пояс: до 1 марта 2024 года Алматы жил в UTC+6, после — в UTC+5. Календарь и `TimeZone` эти правила знают (база часовых поясов приходит с обновлениями iOS), а твоя ручная арифметика — нет.

31.5 NumberFormatter — деньги

Когда применять. Любая сумма на экране: цена, итог корзины, баланс.

```swift
let formatter = NumberFormatter()
formatter.numberStyle = .currency
formatter.locale = Locale(identifier: "ru_RU")
formatter.string(from: NSNumber(value: 1234.5))  // "1 234,50 ₽"

formatter.locale = Locale(identifier: "ru_KZ")
formatter.string(from: NSNumber(value: 1234.5))  // "1 234,50 ₸"
```

`numberStyle = .currency` включает сразу три вещи: разделитель тысяч, два знака после запятой и символ валюты, **которую локаль считает своей**. Для `ru_KZ` и `kk_KZ` это тенге (₸), для `ru_RU` — рубль.

Пробелы в этих строках не обычные. Между «1» и «234» и перед «₸» стоит **неразрывный пробел** (символ U+00A0): он выглядит как пробел, но не даёт строке переломиться посреди суммы. Отсюда классическая ошибка в тестах: `XCTAssertEqual(text, "1 234,50 ₸")` с обычным пробелом не проходит, хотя на глаз строки одинаковые.

Валюта отдельно от локали:

```
formatter.locale = Locale(identifier: "ru_RU")
formatter.currencyCode = "USD"
formatter.string(from: NSNumber(value: 1234.5))  // "1 234,50 $"

formatter.currencyCode = "KZT"
formatter.string(from: NSNumber(value: 1234.5))  // "1 234,50 KZT"
```

`currencyCode` меняет валюту, а оформление (где запятая, где символ) остаётся по правилам локали. Поэтому доллар в `ru_RU` стоит **после** числа: «1 234,50 $». Американский вид «$1,234.50» получится только с `Locale(identifier: "en_US")`.

Вторая строка важна для приложений в Казахстане: с локалью `ru_RU` тенге выводится кодом **«KZT»**, а не знаком ₸. Знак ₸ форматтер знает только для локалей, где тенге — «своя» валюта (`ru_KZ`, `kk_KZ`). Если пользователь с российскими региональными настройками открывает казахстанский магазин, он увидит «KZT». Реши заранее, что тебе нужно: либо форматируешь по `ru_KZ`, либо явно задаёшь `formatter.currencySymbol = "₸"`.

Тенге обычно показывают без тиынов (сотых долей). Для этого:

```
enum Money {
    static let tenge: NumberFormatter = {
        let formatter = NumberFormatter()
        formatter.numberStyle = .currency
        formatter.locale = Locale(identifier: "ru_KZ")
        formatter.currencyCode = "KZT"
        formatter.maximumFractionDigits = 0
        return formatter
    }()
}

Money.tenge.string(from: 1234.5)   // "1 234 ₸"
Money.tenge.string(from: 1235.5)   // "1 236 ₸"
```

Почему 1234,5 превратилось в 1234, а 1235,5 — в 1236? По умолчанию `NumberFormatter` округляет **банковским способом** (`.halfEven`): ровно половину он округляет к **чётному** соседу. У 1234,5 соседи 1234 и 1235, чётный — 1234. У 1235,5 соседи 1235 и 1236, чётный — 1236. Так делают, чтобы при сложении тысяч округлённых сумм ошибки «вверх» и «вниз» гасили друг друга. Если бизнес требует «школьное» округление (половина всегда вверх), поставь `formatter.roundingMode = .halfUp` — тогда 2,5 станет 3.

Сами деньги при этом храни не в `Double`, а в `Decimal` — почему, разобрано в разделе 31.13.

31.6 NumberFormatter — другие стили

```
formatter.numberStyle = .percent
formatter.string(from: NSNumber(value: 0.42))  // "42 %"

formatter.numberStyle = .decimal
formatter.maximumFractionDigits = 2
formatter.string(from: NSNumber(value: 1234567.89))  // "1 234 567,89"

formatter.numberStyle = .spellOut
formatter.string(from: NSNumber(value: 42))  // "сорок два"

formatter.numberStyle = .ordinal
formatter.string(from: NSNumber(value: 3))   // "3"
```

Разбор (локаль `ru_RU`):

- `.percent` умножает на 100: 0.42 — это 42 процента. В русских локалях между числом и знаком стоит неразрывный пробел («42 %»), в `kk_KZ` пробела нет («42%»).
- `.decimal` — просто число с разделителями. `maximumFractionDigits` ограничивает хвост после запятой.
- `.spellOut` — прописью. Для договоров и чеков: `1234` в `ru_RU` — «одна тысяча двести тридцать четыре», в `kk_KZ` — «мың екі жүз отыз төрт».
- `.ordinal` — порядковое число. В `en_US` получится «3rd», в `kk_KZ` — «3-ші», а в русских локалях просто «3»: окончание («3-й», «3-я», «3-е») зависит от рода и падежа слова, которого форматтер не знает.

31.7 Анимация числа

«Растущий счётчик» — сумма на экране итогов плавно бежит от 0 до 12 500. Первая мысль — расставить 30 отложенных задач:

```
func animateCount(from start: Int, to end: Int, duration: TimeInterval = 1.0, label:
↪  UILabel) {
    let steps = 30
    let stepDuration = duration / Double(steps)
    for step in 0...steps {
        let delay = stepDuration * Double(step)
        let progress = Double(step) / Double(steps)
        let value = Int(Double(start) + Double(end - start) * progress)
        DispatchQueue.main.asyncAfter(deadline: .now() + delay) {
            label.text = "\(value)"
        }
    }
}
```

Математика словами: секунду делим на 30 шагов, каждый шаг длится 1/30 ≈ 0,033 секунды. На шаге номер `step` мы прошли долю пути `step / 30`: на 15-м шаге — половину, и при счёте от 0

до 100 на экране будет 50. Значение считаем как «старт плюс пройденная доля разницы»: 0 + 100 × 0,5 = 50.

Недостатки такого подхода: 31 задача ставится в очередь сразу и отменить их нельзя. Вызовешь функцию второй раз, пока первая не закончилась, — два набора задач будут перебивать друг друга, и число на экране запрыгает.

Аккуратнее через **`CADisplayLink`** — таймер, который срабатывает ровно перед каждой перерисовкой экрана (обычно 60 раз в секунду):

```
final class CountAnimator {
    private var displayLink: CADisplayLink?
    private var startValue: Double = 0
    private var endValue: Double = 0
    private var duration: Double = 1.0
    private var startTime: CFTimeInterval = 0
    var update: ((Double) -> Void)?

    func animate(from: Double, to: Double, duration: Double = 1.0) {
        startValue = from
        endValue = to
        self.duration = duration
        startTime = CACurrentMediaTime()
        displayLink?.invalidate()
        let link = CADisplayLink(target: self, selector: #selector(tick))
        link.add(to: .main, forMode: .common)
        displayLink = link
    }

    func stop() {
        displayLink?.invalidate()
        displayLink = nil
    }

    @objc private func tick() {
        let elapsed = CACurrentMediaTime() - startTime
        let progress = min(elapsed / duration, 1.0)
        let value = startValue + (endValue - startValue) * progress
        update?(value)
        if progress >= 1.0 { stop() }
    }
}
```

Что здесь происходит:

- `CACurrentMediaTime()` — монотонные часы в секундах. Их не собьёт смена времени в Настройках, поэтому для анимаций берут их, а не `Date()`.
- В `tick()` мы не считаем кадры, а смотрим, **сколько времени прошло**. Если прошло 0,25 секунды из 1, `progress` = 0,25, и счётчик показывает четверть пути. Если какой-то кадр пропустится, очередной всё равно покажет правильное значение — число не отстаёт.

- `min(..., 1.0)` не даёт перелететь за конец: последний кадр может прийти на 1,01 секунде.
- `link.add(to: .main, forMode: .common)` — подписка на главный run loop (цикл обработки событий главного потока). Режим `.common` нужен, чтобы счётчик не замирал, пока пользователь скроллит: во время скролла run loop переходит в режим отслеживания касаний, и таймеры в режиме `.default` стоят.
- `displayLink?.invalidate()` в начале `animate` — повторный запуск отменяет предыдущий, та проблема из первого варианта ушла.

Про память. `CADisplayLink` держит свой `target` сильной ссылкой (документация `init(target:selector:)`: «the newly constructed display link retains the target»; на симуляторе target жив, пока жив link, и освобождается после `invalidate()`), а наш аниматор держит `displayLink`. Это цикл удержания: пока ссылка не остановлена, аниматор не освободится. Цикл разрывает `stop()` — он вызывается сам в конце анимации. Если экран закрывают раньше, вызови `stop()` вручную:

```
final class TotalsViewController: UIViewController {
    private let label = UILabel()
    private let animator = CountAnimator()

    override func viewDidLoad() {
        super.viewDidLoad()
        animator.update = { [weak self] value in
            self?.label.text = "\(Int(value))"
        }
        animator.animate(from: 0, to: 100, duration: 1.0)
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        animator.stop()
    }
}
```

`[weak self]` в замыкании `update` — чтобы аниматор (которым владеет контроллер) не держал контроллер в ответ.

Для денег подставь в `update` форматтер из 31.5: `self?.label.text = Money.tenge.string(from: value as NSNumber)`.

31.8 UIDatePicker

Когда применять. Пользователь выбирает дату или время: дата рождения, время напоминания, срок задачи.

```
let picker = UIDatePicker()
picker.datePickerMode = .date   // .time / .dateAndTime / .countDownTimer
if #available(iOS 14, *) {
    picker.preferredDatePickerStyle = .compact
}
picker.minimumDate = Calendar.current.date(byAdding: .year, value: -100, to: Date())
```

```
picker.maximumDate = Date()
picker.locale = Locale(identifier: "ru_RU")
picker.addTarget(self, action: #selector(dateChanged(_:)), for: .valueChanged)

@objc private func dateChanged(_ picker: UIDatePicker) {
    print(picker.date)
}
```

Разбор:

- `datePickerMode` — что выбираем: только дату, только время, оба сразу или длительность таймера. По умолчанию — `.dateAndTime`.
- `minimumDate` / `maximumDate` — границы. Для даты рождения: не раньше, чем 100 лет назад, и не позже сегодняшнего дня. Даты за границами выбрать нельзя (в календаре они серые), а если присвоить `date` за пределами диапазона из кода, пикер подтянет её к ближайшей границе — проверено на симуляторе. В режиме `.countDownTimer` границы игнорируются.
- `addTarget(_:action:for: .valueChanged)` — target-action. Он есть у пикера потому, что `UIDatePicker` — контрол: `UIDatePicker → UIControl → UIView` (глава 0, раздел 0.4); с iOS 14 подойдёт и `addAction(UIAction { … }, for: .valueChanged)`. `picker.date` — это `Date`, то есть момент времени; показывать его пользователю снова надо через форматтер. Исключение — режим `.countDownTimer`: там `date` не используется, а выбранная длительность лежит в `countDownDuration` (секунды, хотя барабаны показывают часы и минуты).
- `#available(iOS 14, *)` здесь формальность: стиль `.compact` появился в iOS 14, а у нас минимум iOS 15. Проверку оставили, чтобы код можно было перенести в проект с более старым минимумом.

Стили:

- `.compact` — маленькая «таблетка» с датой; по тапу открывается календарь во всплывающем окне. Хорошо ложится в ячейку формы.
- `.inline` — календарь открыт прямо на экране всё время. Для экранов, где выбор даты — главное действие (бронирование).
- `.wheels` — классические «барабаны», как в iOS 13 и раньше.
- `.automatic` — выбор за системой, зависит от режима и платформы. На iPhone с iOS 26.5 для `.date` это `.compact`, для `.countDownTimer` — `.wheels` (замер свойства `datePickerStyle`, которое, в отличие от `preferredDatePickerStyle`, всегда возвращает конкретный стиль).

31.9 Часовые пояса

`Date` — это **момент времени**: число секунд от 1 января 2001 года по UTC. У него нет часового пояса. Пояс появляется, только когда мы превращаем момент в строку для человека:

```
let formatter = DateFormatter()
formatter.dateFormat = "HH:mm"
formatter.timeZone = TimeZone(identifier: "Asia/Almaty")
```

Один и тот же момент `2026-05-12T09:30:00Z` (буква `Z` в конце значит «по UTC») форматтер с поясом `Asia/Almaty` покажет как 14:30, с поясом `Europe/Moscow` — как 12:30. Момент один, подписи разные.

Правило для работы с сервером: **передавай и храни в UTC, показывай в поясе пользователя**.

```
// Разбор строки ISO 8601 с сервера
let isoFormatter = ISO8601DateFormatter()
isoFormatter.formatOptions = [.withInternetDateTime]
guard let date = isoFormatter.date(from: "2026-05-12T10:30:00Z") else { return }
// date — момент, от часового пояса не зависит

// Отображение для пользователя
let display = DateFormatter()
display.dateStyle = .medium
display.timeStyle = .short
// timeZone по умолчанию = TimeZone.current → человек видит своё время
let text = display.string(from: date)
```

ISO 8601 — международный стандарт записи даты: `2026-05-12T10:30:00Z`. Сначала год, потом месяц и день, после `T` — время, в конце пояс. Сервера почти всегда отдают даты так.

Ловушки, которые реально случаются:

- **Дробные секунды.** Если сервер присылает `2026-05-12T10:30:00.123Z`, форматтер с опциями `[.withInternetDateTime]` вернёт `nil` (проверено). Нужно добавить `.withFractionalSeconds`. А раз результат опциональный, не ставь `!` на разборе серверной строки: формат однажды поменяется, и приложение упадёт у всех пользователей сразу. `guard let` выше как раз для этого.
- **Свой формат для сервера — только с `en_US_POSIX`.** Если сервер шлёт `"2026-05-12 10:30:00"`, форматтеру нужно задать `locale = Locale(identifier: "en_US_POSIX")` и `timeZone = TimeZone(identifier: "UTC")`. POSIX-локаль — «никакая» локаль, которая не меняется от настроек пользователя. Без неё у человека с 12-часовым форматом времени разбор может сломаться.
- **`YYYY` вместо `yyyy`.** Большая `Y` — это «год недели» (по ISO неделям), а не календарный год. На симуляторе дата 29 декабря 2025 года с форматом `"YYYY-MM-dd"` печатается как «2026-12-29»: эта неделя уже считается первой неделей 2026-го. Всегда пиши маленькую `yyyy`.

С iOS 15 есть и короткий путь для разбора: `try Date("2026-05-12T10:30:00Z", strategy: .iso8601)`.

31.10 Тонкости русского плюрала

Плюрал (от plural — множественное число) — выбор формы слова по числу. В английском форм две (1 minute, 2 minutes), в русском три: «1 минута», «2 минуты», «5 минут». Правило:

- если две последние цифры числа от 11 до 14 — форма «минут» (11 минут, 112 минут);
- иначе смотрим на последнюю цифру: 1 — «минута», 2–4 — «минуты», всё остальное (0, 5–9) — «минут».

```
func plural(_ n: Int, forms: (one: String, few: String, many: String)) -> String {
    let n = abs(n)
    let mod100 = n % 100
```

```
    let mod10 = n % 10
    if mod100 >= 11 && mod100 <= 14 { return forms.many }
    switch mod10 {
    case 1: return forms.one
    case 2, 3, 4: return forms.few
    default: return forms.many
    }
}

plural(1, forms: ("минута", "минуты", "минут"))    // "минута"
plural(2, forms: ("минута", "минуты", "минут"))    // "минуты"
plural(5, forms: ("минута", "минуты", "минут"))    // "минут"
plural(21, forms: ("минута", "минуты", "минут"))   // "минута"
plural(12, forms: ("минута", "минуты", "минут"))   // "минут" (исключение 11–14)
plural(111, forms: ("минута", "минуты", "минут"))  // "минут"
```

`n % 100` — остаток от деления на 100, то есть две последние цифры: для 112 это 12. `n % 10` — последняя цифра. `abs(n)` добавлен, потому что в Swift остаток от отрицательного числа отрицательный (−21 % 10 = −1), и без него «−21 минута» ушла бы в ветку `default`.

Как это делают в настоящем проекте. Функция выше хороша для одного-двух мест. Если приложение переводится на несколько языков, формы слов кладут в **String Catalog** (файл `Localizable.xcstrings`): у строки `"%lld minutes"` включается вариант «Vary by Plural», и для русского Xcode сам предложит формы one / few / many. В коде остаётся `String(localized: "\(minutes) minutes")`, а правила выбора формы знает система — для каждого языка свои.

В iOS 15 появилось ещё автоматическое согласование (`AttributedString` с разметкой `^[5 минута](inflect: true)`). На симуляторе iOS 26.5 для русского оно не сработало: «1 минута», «2 минута», «5 минута» — строка осталась как есть. Для русского на него не рассчитывай.

31.11 Размер файлов

```
let formatter = ByteCountFormatter()
formatter.allowedUnits = [.useMB, .useGB]
formatter.countStyle = .file
formatter.string(fromByteCount: 1234567890)  // "1,23 GB"
```

`allowedUnits` ограничивает единицы: с `[.useMB, .useGB]` 456 000 байт станут «0,5 MB», а не «456 KB».

`countStyle` решает, сколько байт в килобайте:

- `.file` и `.decimal` — десятичные единицы, 1 KB = 1000 байт. Так размеры файлов показывает Finder, поэтому для «размер загрузки» бери `.file`: 1 234 567 890 байт = «1,23 GB».
- `.memory` и `.binary` — двоичные, 1 KB = 1024 байта. Так принято считать оперативную память. То же число станет «1,15 GB»: гигабайт здесь больше (1024 × 1024 × 1024 = 1 073 741 824 байта), поэтому «гигабайтов» меньше.

Две детали из прогона на симуляторе. У `ByteCountFormatter` нет свойства `locale`: запятая берётся из текущей локали устройства, а единицы остаются английскими («GB»). Форматирование в стиле iOS 15 переводит и единицы:

```
Int64(1_234_567_890).formatted(.byteCount(style: .file))
// "1,23 ГБ" при русской локали устройства
```

31.12 Сравнение строк с учётом локали

При сортировке списка по имени:

```
// Плохо — сравнение по кодам символов
items.sorted { $0.name < $1.name }

// Хорошо — сравнение по правилам языка
items.sorted { $0.name.localizedStandardCompare($1.name) == .orderedAscending }
```

Что получилось на симуляторе для списка «Яблоко, Ёлка, Ель, Жук, Айва, Ёж, Еда»:

- через <: Ёж, Ёлка, Айва, Еда, Ель, Жук, Яблоко. Буква «Ё» в Unicode стоит **перед** «А» (код U+0401 против U+0410), поэтому все слова на «Ё» улетают в начало списка.
- через `localizedStandardCompare`: Айва, Еда, Ёж, Ёлка, Ель, Жук, Яблоко. «Е» и «Ё» сравниваются как одна буква, и «Ёлка» встаёт между «Ёж» и «Ель», как в словаре.

`localizedStandardCompare` — то сравнение, которым сортирует Finder: без учёта регистра, с учётом правил языка и с «умными» числами — «file2» окажется раньше «file10» (при посимвольном сравнении было бы наоборот, потому что «1» меньше «2»).

31.13 Деньги: Decimal вместо Double

Когда применять. Всегда, когда число — это деньги: цена, скидка, налог, итог.

`Double` хранит число в **двоичной** системе. Большинство десятичных дробей в двоичной записи бесконечны, как 1/3 = 0,333… в десятичной. Поэтому 0,1 хранится приблизительно (в памяти это 0,1000000000000000055…), и ошибки накапливаются:

```
print(0.1 + 0.2)          // 0.30000000000000004
print(0.1 + 0.2 == 0.3)   // false
```

Десять раз прибавить 0,1 — получится 0,9999999999999999, а не 1. Для длины линии на экране это неважно, для суммы в чеке — ошибка на копейку, которую увидит бухгалтер.

`Decimal` хранит число в десятичной системе, поэтому 0,1 для него — ровно одна десятая:

```
let a = Decimal(string: "0.1")!
let b = Decimal(string: "0.2")!
print(a + b)                              // 0.3
print(a + b == Decimal(string: "0.3")!)   // true
```

Главная ловушка — **создание `Decimal` из дробного литерала**:

```
print(Decimal(1234.56))             // 1234.5599999999997952
print(Decimal(string: "1234.56")!)  // 1234.56
```

`Decimal(1234.56)` сначала превращает литерал в `Double` (уже с ошибкой), а потом переносит эту ошибку в `Decimal`. Создавай `Decimal` из строки (`Decimal(string:)` — например, из ответа сервера), из целого числа или храни суммы в **минимальных единицах**: цена 1999,99 ₸ = 199 999 тиынов, целое `Int`, никаких дробей.

Операции `+`, `-`, `*`, `/` у `Decimal` обычные. Для округления в Foundation есть функция `NSDecimalRound`; удобнее завернуть её в метод:

```
extension Decimal {
    func rounded(scale: Int, _ mode: NSDecimalNumber.RoundingMode = .plain) -> Decimal {
        var value = self
        var result = Decimal()
        NSDecimalRound(&result, &value, scale, mode)
        return result
    }
}

let price = Decimal(string: "1999.99")!
let total = price * 3                          // 5999.97
let tax = total * Decimal(string: "0.16")!     // 959.9952
tax.rounded(scale: 2)                          // 960 (то есть 960,00)
```

Числа словами: три товара по 1999,99 — ровно 5999,97. Налог 16% — это умножить на 0,16, получается 959,9952: четыре знака после запятой. В деньгах их два, поэтому округляем до сотых (`scale: 2`). Третья цифра после запятой — 5, значит округляем вверх: 959,9952 → 960,00. Вывод `960`, а не `960.00`, потому что `Decimal` не хранит незначащие нули — два знака добавит форматтер при показе.

Режимы округления на числе 2,5:

- `.plain` — «школьное»: половина вверх, 2,5 → 3;
- `.bankers` — к чётному: 2,5 → 2, но 3,5 → 4 (о смысле — в 31.5);
- `.up` / `.down` — всегда вверх / всегда вниз.

Какой выбрать, решает не программист, а бухгалтерия или закон страны. Важно одно: округлять один раз, в оговорённом месте (например, налог по каждой строке чека или по итогу — это разные суммы).

Чтобы показать `Decimal` через `NumberFormatter`, передай его как `NSDecimalNumber`: `Money.tenge.string(from: total as NSDecimalNumber)`.

31.14 FormatStyle — современный синтаксис (iOS 15+)

С iOS 15 у дат и чисел есть метод `.formatted(...)`. Он не требует создавать и кешировать форматтер — стиль описывается значением прямо в месте вызова:

```
let kz = Locale(identifier: "ru_KZ")

date.formatted(date: .long, time: .shortened)
// "12 мая 2026 г. в 14:30" (при русской локали устройства)

date.formatted(.dateTime.day().month(.wide).year().locale(kz))
// "12 мая 2026 г."

let money = Decimal(string: "1234.5")!
money.formatted(.currency(code: "KZT").locale(kz))
// "1 234,50 ₸"
money.formatted(.currency(code: "KZT").precision(.fractionLength(0)).locale(kz))
// "1 234 ₸"

0.42.formatted(.percent.locale(kz))     // "42 %"

["яблоки", "груши", "сливы"].formatted(.list(type: .and).locale(kz))
// "яблоки, груши и сливы"
```

Разбор:

- `.dateTime.day().month(.wide).year()` — перечисляешь **поля**, а порядок и знаки препинания выберет локаль. Это то же, что шаблон в `setLocalizedDateFormatFromTemplate`, только проще.
- `.currency(code:)` у `Decimal` — прямой путь для денег, без `NSDecimalNumber`. Правила про знак ₸ те же, что в 31.5: с локалью `ru_RU` вместо ₸ будет «KZT».
- `.precision(.fractionLength(0))` — сколько знаков после запятой. Округление по умолчанию тоже банковское: 2,5 → 2, 3,5 → 4.
- `.list(type: .and)` — список через запятую с «и» перед последним элементом, по правилам языка.

Что выбрать: для нового кода с минимумом iOS 15 — `formatted`, он короче и не требует кеша. `DateFormatter` и `NumberFormatter` остаются для разбора строк с сервера и для кода, который уже на них написан.

Упражнения

Упражнение 31.1. Напиши функцию `priceText(_ amount: Decimal) -> String`, которая печатает сумму в тенге без тиынов по правилам `ru_KZ`. Проверь: `priceText(Decimal(string: "1234.5")!)` и `priceText(Decimal(string: "1235.5")!)`. Объясни, почему результаты «округлились в разные стороны».

Упражнение 31.2. Функция `plural` из 31.10 возвращает только слово. Напиши `minutesText(_ n: Int) -> String`, которая возвращает число вместе со словом: `minutesText(21)` → «21 минута», `minutesText(114)` → «114 минут». Какие ещё числа из диапазона 100–125 стоит проверить?

Упражнение 31.3. Сервер присылает `"2026-05-12T10:30:00.500Z"`. Напиши разбор этой строки в `Date` и покажи время в поясе `Asia/Almaty` в формате `HH:mm`. Что должно получиться?

Ответы к упражнениям

31.1.

```
func priceText(_ amount: Decimal) -> String {
    amount.formatted(
        .currency(code: "KZT")
            .precision(.fractionLength(0))
            .locale(Locale(identifier: "ru_KZ"))
    )
}

priceText(Decimal(string: "1234.5")!)   // "1 234 ₸"
priceText(Decimal(string: "1235.5")!)   // "1 236 ₸"
```

Округление банковское: ровно половина идёт к чётному соседу. У 1234,5 чётный сосед — 1234, у 1235,5 — 1236. Если нужно «половина всегда вверх», добавь `.rounded(rule: .toNearestOrAwayFromZero)` перед `.locale(...)` — тогда 1234,5 станет «1 235 ₸».

31.2.

```
func minutesText(_ n: Int) -> String {
    "\(n) \(plural(n, forms: ("минута", "минуты", "минут")))"
}

minutesText(21)    // "21 минута"
minutesText(114)   // "114 минут"
```

В диапазоне 100–125 стоит проверить границы правил: 101 («минута»), 102 («минуты»), 111–114 («минут», исключение по двум последним цифрам), 115 («минут»), 121 («минута»), 122 («минуты»).

31.3.

```
let iso = ISO8601DateFormatter()
iso.formatOptions = [.withInternetDateTime, .withFractionalSeconds]

let time = DateFormatter()
time.dateFormat = "HH:mm"
time.timeZone = TimeZone(identifier: "Asia/Almaty")

if let date = iso.date(from: "2026-05-12T10:30:00.500Z") {
    print(time.string(from: date))   // "15:30"
}
```

10:30 по UTC плюс 5 часов разницы Алматы — 15:30. Без `.withFractionalSeconds` разбор вернул бы `nil`, и `if let` просто не выполнил бы печать.

Что мы выучили

- **Локаль** решает всё оформление: порядок полей даты, разделители, символ валюты. Для примеров — фиксированная локаль, в приложении — `Locale.current`.
- **DateFormatter**: стили `.short`…`.full` лучше ручного формата; для «полей без порядка» — шаблон; месяц без дня — `LLLL`. Кешируй.
- **RelativeDateTimeFormatter** — «5 минут назад», с `.named` — «вчера». `.spellOut` по-русски ошибается в роде.
- **DateComponentsFormatter** — длительности, `.positional` + `.pad` для таймера «01:05».
- **Календарь** вместо арифметики с 86 400 секундами: перевод часов и смена поясов (Алматы: UTC+6 → UTC+5 в 2024-м).
- **NumberFormatter** `.currency`: знак ₸ только в локалях `ru_KZ` / `kk_KZ`, в `ru_RU` — «KZT»; `currencyCode` меняет валюту, но не оформление; неразрывные пробелы; банковское округление по умолчанию.
- **CADisplayLink** — счётчик по прошедшему времени, режим `.common`, не забыть `stop()`.
- **Часовые пояса**: `Date` — момент без пояса; храни UTC, показывай локальное; `en_US_POSIX` для серверных форматов; `yyyy`, а не `YYYY`.
- **Плюрал**: три формы, исключение 11–14; в больших проектах — String Catalog с «Vary by Plural».
- **ByteCountFormatter**: `.file` = 1000, `.memory` = 1024.
- **`localizedStandardCompare`** — сортировка как в словаре и Finder.
- **Деньги — `Decimal`** из строки или целые тиыны; округление через `NSDecimalRound` один раз в оговорённом месте.
- **FormatStyle** (iOS 15+) — `.formatted(...)` без кеша форматтеров.

Apple Developer Documentation

- UIDatePicker — UIKit-контрол выбора даты/времени.
- UIDatePicker.countDownDuration — значение в режиме `.countDownTimer` (секунды).
- UIDatePickerStyle — `.compact` / `.inline` / `.wheels` / `.automatic`.
- DateFormatter — форматирование `Date` ↔ `String` с учётом локали.
- RelativeDateTimeFormatter — «5 минут назад» / «через 2 дня», iOS 13+.
- DateComponentsFormatter — длительности вида «1 ч 23 мин».
- ISO8601DateFormatter — разбор и запись ISO 8601.
- Date.ISO8601FormatStyle — ISO 8601 в стиле FormatStyle, iOS 15+.
- Date.FormatStyle — `date.formatted(...)`, iOS 15+.
- NumberFormatter — `.currency` / `.percent` / `.decimal` / `.ordinal`.
- Decimal — точная десятичная арифметика для денег.
- Calendar — календарные операции вместо ручной арифметики с `TimeInterval`.
- TimeZone — часовые пояса при отображении дат.
- ByteCountFormatter — размеры файлов «1,23 GB».
- CADisplayLink — таймер, синхронный с обновлением экрана.
- Localizing and varying text with a string catalog — String Catalog и формы множественного числа.

→ Глава 32. Cookbook — анимации

Глава 32. Cookbook — анимации

`UIView.animate`, `CABasicAnimation`, `CAEmitterLayer` — и когда что использовать. Все числа про кривые и пружины в этой главе не взяты из головы: они сняты на симуляторе iOS 26.5 — квадрат двигали на 100 точек и 60 раз в секунду записывали, где он находится.

Сначала — что вообще происходит, когда iOS «анимирует».

Анимация в UIKit — это не покадровое рисование руками. Ты говоришь «было `alpha = 0`, стало `alpha = 1`, за 0,3 секунды», а система сама вычисляет все промежуточные значения. Это называется **интерполяцией**: между двумя известными точками достраиваются промежуточные. Считает и рисует их **Core Animation** — слой системы, который работает с `CALayer`. У каждого `UIView` внутри есть ровно один свой слой (`view.layer`), и именно слой отображается на экране. Слой — не view: `CALayer → NSObject`, он не responder, не получает касаний и не участвует в Auto Layout (глава 0, разделы 0.3 и 0.8).

Две вещи, которые важно держать в голове:

- **Модель и то, что видно.** Когда ты пишешь `view.alpha = 1` внутри `animate`, свойство меняется **сразу**. На экране же 0,3 секунды рисуется промежуточное значение из так называемого **presentation layer** — «слоя-отображения», копии слоя с текущими промежуточными значениями (`view.layer.presentation()`). Наши замеры читали именно его.
- **Анимация не блокирует код.** `UIView.animate` возвращается мгновенно, а строка после него выполняется, пока анимация ещё идёт. Чтобы сделать что-то **после**, есть `completion`.

32.1 UIView.animate — базовый

Когда применять. Появление, исчезновение, сдвиг, изменение цвета — 90% анимаций в обычном приложении.

```
UIView.animate(withDuration: 0.3) {
    view.alpha = 1.0
    view.transform = .identity
}
```

`withDuration: 0.3` — длительность в секундах, 0,3 с = 300 мс. Ориентиры: 0,2–0,3 с — для мелких изменений (кнопка, подсказка), 0,35–0,5 с — для крупных (карточка въезжает, экран меняется). Дольше полусекунды интерфейс начинает казаться медленным.

`transform = .identity` — «никаких преобразований», исходный размер и положение. Обычно так заканчивают анимацию, которая началась из сдвинутого или уменьшенного состояния.

Что можно анимировать через `UIView.animate` — документация `UIView` перечисляет шесть свойств:

- `frame`, `bounds`, `center` — положение и размер;
- `transform` — сдвиг, масштаб, поворот;
- `alpha` — прозрачность;
- `backgroundColor`.

Плюс `constant` у констрейнта — не напрямую, а через `layoutIfNeeded()` (раздел 32.8). `isHidden` не анимируется — прячут через `alpha` (замер на симуляторе: после `isHidden = true` внутри блока у слоя не появилось ни одной анимации).

Со свойствами слоя (`view.layer.cornerRadius`, `shadowOpacity`, `borderColor`…) всё тоньше, и здесь ошибаются в обе стороны. Снаружи анимационного блока они меняются рывком: `UIView` — делегат своего слоя и отключает ему неявные анимации (у отдельного `CALayer`, который ты создал сам и добавил через `addSublayer`, они есть — 0,25 с по умолчанию; замер: смена `opacity` у такого слоя сразу вешает на него анимацию, у `view.layer` — нет). Внутри `UIView.animate` на iOS 26.5 замер показал, что `cornerRadius`, `borderWidth`, `borderColor`, `shadowOpacity` и `shadowRadius` у `view.layer` получают анимацию наравне с `alpha`; для `cornerRadius` Apple показала это ещё в iOS 11. Но в документации этот список не зафиксирован, а на слой, добавленный через `addSublayer`, `UIView.animate` не действует вовсе (замер: ноль анимаций). Отсюда правило: шесть свойств view — через `UIView.animate`; свойства своих слоёв, бесконечные и точно управляемые анимации — через `CABasicAnimation` (раздел 32.6).

С действием в конце:

```
UIView.animate(withDuration: 0.3, animations: {
    view.alpha = 0
}) { _ in
    view.removeFromSuperview()
}
```

Параметр замыкания `completion` — `Bool`: `true`, если анимация дошла до конца, `false`, если её прервали (например, запустили новую анимацию того же свойства).

Кривые: разгон и торможение

По умолчанию `UIView.animate` использует кривую **ease-in-out** — «плавный разгон и плавное торможение». Кривая отвечает на вопрос: какая доля пути пройдена к такой-то доле времени. Замер на симуляторе, квадрат едет 100 точек за 1 секунду:

Кривая (`options`)	через 0,25 с	через 0,5 с	через 0,75 с
`.curveEaseInOut` (по умолчанию)	~12 точек	~49	~86
`.curveEaseIn` — только разгон	~8	~30	~60
`.curveEaseOut` — только торможение	~35	~66	~89
`.curveLinear` — ровно	~23	~48	~73

(У линейной кривой вместо 25/50/75 получилось чуть меньше: замер отстаёт от анимации примерно на кадр.)

Читается так. `.curveEaseInOut` первую четверть времени едет медленно (12% пути), в середине разгоняется и последнюю четверть снова тормозит. `.curveEaseOut` стартует быстро — треть пути уже за первую четверть времени — и мягко останавливается: так ведут себя появляющиеся элементы, они «влетают и садятся». `.curveEaseIn` наоборот, разгоняется к концу: подходит для уходящих элементов, которые «улетают» с экрана. Линейную кривую в интерфейсе почти не используют — движение выглядит механическим; она нужна для бесконечного вращения спиннера.

```
UIView.animate(withDuration: 0.3, delay: 0, options: [.curveEaseOut]) {
    card.transform = .identity
}
```

32.2 Spring — пружина

Когда применять. Когда элемент должен «приехать» живо: карточка возвращается на место, кнопка «отпрыгивает» после нажатия.

```
UIView.animate(
    withDuration: 0.5,
    delay: 0,
    usingSpringWithDamping: 0.6,
    initialSpringVelocity: 0.3,
    options: []
) {
    view.transform = .identity
}
```

Кривую в `options` здесь не указываем: форму движения задаёт пружина.

`usingSpringWithDamping` — затухание, число от 0 до 1. Проще всего понять через «перелёт»: насколько элемент проскочит цель, прежде чем вернуться. Замер на симуляторе, квадрат едет на 100 точек вниз:

damping	Перелёт	Как ощущается
0.5	~16 точек (16% пути)	заметно «прыгает»
0.6	~9 точек	живо, с отскоком
0.7	~5 точек	чуть перелетит и вернётся
0.85	меньше 1 точки	почти без отскока, но «мягкий» финиш
1.0	0	без пружины, плавная остановка

Чем меньше damping, тем больше колебаний. Для интерфейса обычно берут 0.7–0.85: движение живое, но не игрушечное. 0.5 и ниже — для игровых и праздничных эффектов.

`initialSpringVelocity` — начальная скорость, но в необычных единицах: это доля всего пути **в секунду**. Если элемент должен проехать 200 точек, то `initialSpringVelocity: 1` значит «стартовать со скоростью 200 точек в секунду», а `0.3` — 60 точек в секунду. Ноль — старт с места. Зачем это нужно: если пользователь «бросил» карточку пальцем со скоростью 800 точек в секунду, а

до места осталось 200 точек, передай `800 / 200 = 4` — и анимация продолжит движение пальца без рывка.

С iOS 17 есть запись попроще — длительность и «прыгучесть» (0 — без отскока, 0.3 — заметный):

```
if #available(iOS 17, *) {
    UIView.animate(springDuration: 0.5, bounce: 0.3) {
        view.transform = .identity
    }
}
```

У нас минимум iOS 15, поэтому без `#available` такой код не соберётся.

32.3 Последовательность

Когда применять. Несколько элементов появляются по очереди — «лесенкой».

```
private func animateSequence() {
    for (i, dot) in [dot1, dot2, dot3].enumerated() {
        dot.alpha = 0
        dot.transform = CGAffineTransform(translationX: 0, y: 20)
        UIView.animate(withDuration: 0.4, delay: 0.15 * Double(i),
                       usingSpringWithDamping: 0.7, initialSpringVelocity: 0) {
            dot.alpha = 1
            dot.transform = .identity
        }
    }
}
```

Разбор:

- Сначала ставим начальное состояние без анимации: невидимо и на 20 точек ниже (`translationX: 0, y: 20` — сдвиг по вертикали вниз, ось y в iOS смотрит вниз).
- `delay: 0.15 * Double(i)` — задержки 0, 0,15 и 0,3 секунды. Каждая анимация длится 0,4 с, значит последняя закончится через 0,3 + 0,4 = 0,7 секунды после старта.
- В анимации возвращаем `alpha = 1` и `.identity` — точки «всплывают» на место с лёгким отскоком (damping 0.7 — перелёт около 5%).

Альтернатива — ключевые кадры:

```
UIView.animateKeyframes(withDuration: 1.0, delay: 0, options: []) {
    UIView.addKeyframe(withRelativeStartTime: 0.0, relativeDuration: 0.3) {
        dot1.alpha = 1
    }
    UIView.addKeyframe(withRelativeStartTime: 0.3, relativeDuration: 0.3) {
        dot2.alpha = 1
    }
    UIView.addKeyframe(withRelativeStartTime: 0.6, relativeDuration: 0.3) {
        dot3.alpha = 1
```

```
    }
}
```

Ключевой кадр — отрезок общей анимации. Время задаётся долями от общей длительности: при `withDuration: 1.0` кадр с `relativeStartTime: 0.3` и `relativeDuration: 0.3` идёт с 0,3 по 0,6 секунды. Удобство в том, что всё остаётся одной анимацией: поменяешь 1.0 на 2.0 — все кадры растянутся пропорционально, а один `completion` сработает в конце.

32.4 Появление ячеек

Когда применять. Первый показ списка — строки «въезжают» снизу одна за другой.

```
private var animatedFirstScreen = false

func tableView(_ tableView: UITableView, willDisplay cell: UITableViewCell,
               forRowAt indexPath: IndexPath) {
    guard !animatedFirstScreen else { return }
    let visibleCount = tableView.indexPathsForVisibleRows?.count ?? 0
    if indexPath.row >= visibleCount { animatedFirstScreen = true; return }

    cell.alpha = 0
    cell.transform = CGAffineTransform(translationX: 0, y: 20)
    UIView.animate(withDuration: 0.4, delay: 0.05 * Double(indexPath.row),
                   usingSpringWithDamping: 0.85, initialSpringVelocity: 0) {
        cell.alpha = 1
        cell.transform = .identity
    }
}
```

`willDisplay` — метод делегата таблицы, который вызывается прямо перед тем, как ячейка появится на экране. Это последний момент поменять её вид.

Почему нужен флаг. Без него задержка считается от номера строки: `0.05 × row`. На первом экране это приятно (строки 0…10 — задержки от 0 до 0,5 с). Но при прокрутке до строки 200 задержка становится 0.05 × 200 = 10 секунд: ячейка выезжает на экран пустой и появляется через 10 секунд. Поэтому анимируем только первый экран — пока номер строки меньше количества видимых строк — а потом флаг выключает эффект навсегда.

Про **переиспользование ячеек**: таблица не создаёт новую ячейку для каждой строки, а берёт ушедшую за край экрана и наполняет новыми данными. Если анимацию прервали на полпути, в переиспользованной ячейке мог остаться `alpha = 0.4`. Здесь это не случится — анимация всегда доводит до `1` и `.identity`, — но если меняешь вид ячейки в `willDisplay`, всегда ставь **конечное** состояние явно.

И учти настройку «Уменьшение движения» (Reduce Motion, см. главу 34.5): такие декоративные эффекты при ней лучше выключать совсем.

32.5 Анимации SF Symbols (iOS 17+)

SF Symbols — системная библиотека иконок Apple (`UIImage(systemName:)`). С iOS 17 эти иконки умеют анимироваться сами:

```
imageView.image = UIImage(systemName: "heart.fill")
if #available(iOS 17, *) {
    imageView.addSymbolEffect(.bounce, options: .speed(1.5))   // подпрыгнуть
    imageView.addSymbolEffect(.pulse)                          // пульсировать
    imageView.addSymbolEffect(.scale.up)                       // увеличиться и держать
}
```

- `.bounce` — один «подскок», `.speed(1.5)` — в полтора раза быстрее обычного.
- `.pulse` — прозрачность плавно «дышит», пока эффект не снимут.
- `.scale.up` — иконка увеличивается и остаётся увеличенной до снятия эффекта.

Снять всё — `imageView.removeAllSymbolEffects()`. На iOS 15–16 этих методов нет, поэтому код обязательно в `#available(iOS 17, *)`; на старых системах иконка просто останется статичной.

32.6 CABasicAnimation — анимация слоя

Когда применять. Бесконечные анимации и свойства `CALayer`, которые `UIView.animate` не умеет.

```
let pulse = CABasicAnimation(keyPath: "transform.scale")
pulse.fromValue = 1.0
pulse.toValue = 1.4
pulse.duration = 0.8
pulse.autoreverses = true
pulse.repeatCount = .infinity
view.layer.add(pulse, forKey: "pulse")
```

Разбор:

- `keyPath: "transform.scale"` — какое свойство слоя анимировать, записанное строкой. Опечатка в строке компилятором не ловится: анимация просто ничего не сделает.
- `fromValue: 1.0` → `toValue: 1.4` — масштаб от исходного до «на 40% больше». Круг диаметром 60 точек вырастет до 84.
- `duration: 0.8` + `autoreverses = true` — 0,8 с растёт и ещё 0,8 с возвращается, полный «вдох-выдох» — 1,6 с.
- `repeatCount = .infinity` — без конца.
- `forKey: "pulse"` — имя, по которому анимацию потом снимают. `layer.add(_:forKey:)` **копирует** объект анимации (документация: «this object is copied by the render tree, not referenced») — менять `pulse` после добавления бессмысленно.

Сама анимация — не слой и не view, а объект-описание: `CABasicAnimation` → `CAPropertyAnimation` → `CAAnimation` → `NSObject` (глава 0, раздел 0.8).

Чем это отличается от `UIView.animate`: `CABasicAnimation` **не меняет модель**. Свойство слоя остаётся 1.0 (замер: после `add` `layer.opacity` для аналогичной анимации прозрачности так и

равен 1.0), анимация лишь рисует промежуточные значения в presentation layer. Когда анимация закончится или её снимут, экран вернётся к модельному значению — тот самый «отскок». Поэтому для «один раз увеличить и оставить» сначала присваивают новое значение слою, а потом добавляют анимацию от старого к новому:

```
let grow = CABasicAnimation(keyPath: "transform.scale")
grow.fromValue = 1.0
grow.toValue = 1.4
grow.duration = 0.3
view.layer.transform = CATransform3DMakeScale(1.4, 1.4, 1)  // модель
view.layer.add(grow, forKey: "grow")                         // картинка
```

Два свойства, которыми часто «чинят» отскок, — `isRemovedOnCompletion` и `fillMode`. По умолчанию анимация после окончания снимается со слоя (`isRemovedOnCompletion = true`, `fillMode = .removed` — оба значения из документации), и экран возвращается к модели. Если поставить `isRemovedOnCompletion = false` и `fillMode = .forwards`, картинка застынет в конечном состоянии, но модель не изменится: в замере на симуляторе после такой анимации `layer.opacity` остался 1.0, а `layer.presentation()?.opacity` — 0. Слой выглядит прозрачным, а для твоего кода, хит-тестинга и следующих анимаций он непрозрачный; к тому же законченная анимация так и висит на слое. Это не способ «оставить результат», а способ получить расхождение между моделью и экраном. Оставлять результат нужно так, как в листинге выше.

Когда `CABasicAnimation`, а не `UIView.animate`:

- **Бесконечная анимация** (пульс, мерцание скелетона) — `repeatCount = .infinity`.
- **Свойства своих слоёв**: `strokeEnd` у `CAShapeLayer`, цвета градиента в `CAGradientLayer` — на sublayer `UIView.animate` не действует (раздел 32.1); а также `cornerRadius` и `shadowOpacity` у `view.layer`, если нужна гарантия одинакового поведения на всех версиях iOS.
- **Плавная смена картинки** — `keyPath: "contents"`.

Снять: `view.layer.removeAnimation(forKey: "pulse")`.

Бесконечную анимацию Core Animation может снять сама — например, когда view убирают из окна. Надёжный приём — запускать её в `didMoveToWindow()` (метод `UIView`, вызывается при появлении в окне) и при возвращении приложения из фона, а не один раз в `init`.

32.7 CAKeyframeAnimation — несколько точек

Когда применять. Движение по нескольким значениям подряд: тряска поля с ошибкой, «покачивание» иконки.

```
let animation = CAKeyframeAnimation(keyPath: "transform.translation.x")
animation.values = [-12, 12, -10, 10, -6, 6, 0]
animation.duration = 0.4
view.layer.add(animation, forKey: "shake")
```

Эффект «неверный пароль»: поле качается влево-вправо и успокаивается. Числа в `values` — сдвиг по горизонтали в точках: 12 влево, 12 вправо, 10 влево… Размах уменьшается 12 → 10 → 6 → 0 — так выглядит затухающее колебание, как у задетой струны. Последнее значение 0 — возврат на место (иначе после анимации поле «прыгнет» обратно).

Семь значений — это шесть промежутков. За 0,4 с каждый длится 0,4 / 6 ≈ 0,067 с: быстрая, но различимая тряска. Хочешь медленнее — увеличь `duration`, промежутки растянутся одинаково.

32.8 Анимация констрейнта

Констрейнт (constraint) — правило Auto Layout вроде «высота = 120» или «отступ слева = 16». Анимировать его напрямую нельзя: меняется число, а кадры view пересчитываются при ближайшей раскладке.

```
heightConstraint.constant = 200
UIView.animate(withDuration: 0.3) {
    self.view.layoutIfNeeded()
}
```

Меняем `constant` снаружи, а внутри анимации вызываем `layoutIfNeeded()` — «разложи view прямо сейчас». Раскладка случится внутри анимационного блока, поэтому все новые `frame` поедут плавно: высота со 120 до 200 точек за 0,3 с.

Вызывать `layoutIfNeeded()` нужно у **общего предка** всех view, чьи рамки изменятся, — обычно это `self.view` контроллера. Документация описывает метод так: «используя view-получателя как корень, он раскладывает поддерево начиная с этого корня». Рамку каждого view выставляет `layoutSubviews` его superview, поэтому `changedView.layoutIfNeeded()` разложит только то, что внутри `changedView`; его собственная высота и соседи, которые от неё зависят, переедут рывком при следующем обычном проходе layout.

32.9 Смена одного view на другой

```
UIView.transition(from: oldView, to: newView,
                  duration: 0.3,
                  options: [.transitionCrossDissolve]) { _ in
    // oldView уже удалён из иерархии
}
```

Cross-dissolve — «наплыв»: старое изображение растворяется, новое проявляется. Этот вариант метода сам удаляет `oldView` из родителя и добавляет `newView` на его место — не забудь, что у `newView` должны быть свои констрейнты или `frame`. Если оба view уже в иерархии и нужно лишь спрятать одно и показать другое, добавь опцию `.showHideTransitionViews`: тогда метод переключит `isHidden`, а не переставит view (документация). Контроллеры метод не трогает — он работает только с иерархией view.

Второй вариант — анимировать всё содержимое контейнера:

```
UIView.transition(with: container, duration: 0.3,
                  options: .transitionCrossDissolve,
                  animations: {
    container.subviews.first?.removeFromSuperview()
    container.addSubview(newView)
```

```
})
```

Система снимает «снимок» контейнера до и после блока `animations` и плавно переводит один в другой. Так же `BootCoordinator.setRoot` меняет корневой экран окна (см. главу 4.5).

32.10 Конфетти через CAEmitterLayer

Когда применять. Праздник: выполнил все задачи, прошёл уровень, оформил первый заказ.

```
private func launchConfetti() {
    let emitter = CAEmitterLayer()
    emitter.emitterPosition = CGPoint(x: view.bounds.midX, y: -20)
    emitter.emitterShape = .line
    emitter.emitterSize = CGSize(width: view.bounds.width, height: 1)
    let colors: [UIColor] = [.systemRed, .systemOrange, .systemYellow,
                             .systemGreen, .systemBlue, .systemPurple]
    emitter.emitterCells = colors.map { color in
        let image = makeConfettiImage(color: color)
        let cell = CAEmitterCell()
        cell.birthRate = 6
        cell.lifetime = 6
        cell.velocity = 180
        cell.velocityRange = 40
        cell.emissionLongitude = .pi   // вниз
        cell.emissionRange = 0.4
        cell.spin = 2
        cell.spinRange = 3
        cell.contents = image.cgImage
        cell.contentsScale = image.scale
        return cell
    }
    view.layer.addSublayer(emitter)
    DispatchQueue.main.asyncAfter(deadline: .now() + 1.0) {
        emitter.birthRate = 0   // новых частиц больше не рождать
        DispatchQueue.main.asyncAfter(deadline: .now() + 6.0) {
            emitter.removeFromSuperlayer()
        }
    }
}

private func makeConfettiImage(color: UIColor) -> UIImage {
    let size = CGSize(width: 14, height: 6)
    return UIGraphicsImageRenderer(size: size).image { _ in
        color.setFill()
        UIBezierPath(roundedRect: CGRect(origin: .zero, size: size), cornerRadius:
 ↪  1.5).fill()
    }
}
```

Система частиц — генератор множества мелких картинок, у каждой своя скорость, вращение и время жизни. `CAEmitterLayer` — «пушка», `CAEmitterCell` — описание одного вида частиц. Параметры словами:

- `emitterPosition` y = −20, форма `.line` шириной в экран — частицы рождаются на линии чуть выше верхнего края.
- `birthRate = 6` — шесть частиц в секунду **каждого** цвета; цветов шесть, итого около 36 в секунду.
- `lifetime = 6` — каждая живёт 6 секунд.
- `velocity = 180`, `velocityRange = 40` — скорость 180 ± 40 точек в секунду, то есть от 140 до 220: за 6 секунд частица пролетает примерно 850–1300 точек, больше высоты экрана.
- `emissionLongitude = .pi` — направление вылета. Угол в радианах (π радиан = 180°, подробнее в 32.12); на симуляторе с этим значением частицы летят вниз. `emissionRange = 0.4` — разброс ±0,4 радиана, примерно ±23°: конфетти летит веером, а не строем.
- `spin = 2`, `spinRange = 3` — вращение 2 ± 3 радиана в секунду: кто-то крутится быстро, кто-то медленно и в другую сторону.

Частая ошибка — `cell.scale = 0.1` без `contentsScale`. Скриншот с симулятора показывает крошечные цветные точки вместо конфетти: картинка 14 × 6 точек, уменьшенная до 10%, — это 1,4 × 0,6 точки. `contentsScale = image.scale` говорит слою, что в картинке 3 пикселя на точку (экран @3x), и частица рисуется ровно 14 × 6 точек. Без неё, наоборот, частица получается в три раза крупнее — 42 × 18 точек.

`emitter.birthRate = 0` через секунду — «перестань рождать новые»: уже летящие частицы долетают. Их `lifetime` 6 секунд, поэтому слой удаляем ещё через 6 секунд, а не раньше — иначе конфетти исчезнет посреди экрана.

32.11 Hero-анимация (перелёт элемента между экранами)

Hero-анимация — элемент «перелетает» с одного экрана на другой: миниатюра в списке вырастает в полноэкранное фото. Классический путь — собственный переход через `UIViewControllerAnimatedTransitioning` (протокол «аниматора перехода»; подробно в главе 26.7):

```
func animateTransition(using ctx: UIViewControllerContextTransitioning) {
    let container = ctx.containerView
    // 1. Найти исходный view и конечный view
    // 2. Снять snapshot (снимок) исходного
    // 3. Анимировать снимок между двумя позициями
    // 4. Скрыть оригиналы во время анимации
    // 5. Показать конечный экран, удалить снимок
}
```

Это каркас, полная рабочая версия для фото — в главе 37.7. Реализовать переход правильно непросто: нужно пересчитать координаты между экранами, учесть поворот и отмену жестом.

С iOS 18 в UIKit появился готовый zoom-переход:

```swift
let thumbnail = cell.imageView   // миниатюра, из которой «вырастет» экран
if #available(iOS 18, *) {
    detail.preferredTransition = .zoom { _ in thumbnail }
}
navigationController?.pushViewController(detail, animated: true)
```

Замыкание возвращает view, **из которого** вырастает новый экран. Миниатюру кладём в локальную константу, чтобы замыкание не захватывало весь контроллер через `self`. На iOS 15–17 условие не выполнится, и экран откроется обычным переходом.

32.12 Вращение

```swift
UIView.animate(withDuration: 1.0) {
    view.transform = CGAffineTransform(rotationAngle: .pi)
}

// Бесконечно
let rotate = CABasicAnimation(keyPath: "transform.rotation")
rotate.fromValue = 0
rotate.toValue = 2 * Double.pi
rotate.duration = 1.0
rotate.repeatCount = .infinity
view.layer.add(rotate, forKey: "spin")
```

Углы в iOS задаются в **радианах**, не в градусах. Перевод простой: π (≈ 3,14) радиан — это полоборота, 180°; 2π — полный оборот, 360°; π / 2 — четверть, 90°. Чтобы перевести градусы в радианы, умножь на π и раздели на 180: 45° = π / 4 ≈ 0,785.

Две особенности `UIView.animate` с поворотом:

- Он анимирует от старого `transform` к новому **кратчайшим путём**. Поворот на 2π даёт ту же матрицу, что и 0, — анимация не сделает ничего. Поэтому бесконечное вращение делают через `CABasicAnimation` с `transform.rotation`: там анимируется само число от 0 до 2π.
- При повороте ровно на π оба направления одинаково «коротки», и сторону выбирает система. Нужно гарантированно по часовой — поверни на чуть меньший угол (`.pi * 0.999`) или используй `CABasicAnimation`.

В координатах iOS ось y смотрит вниз, поэтому положительный угол — поворот **по часовой стрелке**.

32.13 UIViewPropertyAnimator — анимация под управлением

```swift
let animator = UIViewPropertyAnimator(duration: 0.3, curve: .easeOut) {
    view.alpha = 1
}
animator.startAnimation()
```

```
// Можно поставить на паузу, перемотать и продолжить
animator.pauseAnimation()
animator.fractionComplete = 0.5
animator.continueAnimation(withTimingParameters: nil, durationFactor: 1.0)
```

`UIViewPropertyAnimator` (iOS 10+) — анимация-объект: обычный `NSObject` (глава 0, раздел 0.8), который живёт столько, сколько ты его держишь. В отличие от `UIView.animate`, его можно хранить в свойстве и управлять:

- `pauseAnimation()` — заморозить;
- `fractionComplete = 0.5` — перемотать на середину: `alpha` станет ровно 0,5. По умолчанию перемотка линейная — `scrubsLinearly = true` (iOS 11+), кривая при ручной перемотке не применяется; замер на симуляторе с кривой `.easeOut` дал presentation-значение `opacity` 0,5. Поставь `scrubsLinearly = false`, если хочешь, чтобы палец «чувствовал» кривую;
- `isReversed = true` — пустить назад;
- `continueAnimation(...)` — доиграть с текущего места. Вызывать можно только у активного и поставленного на паузу аниматора (документация). `durationFactor` — множитель к исходной длительности, по которому считается оставшееся время: 1.0 — в темпе исходной анимации, 0.5 — вдвое быстрее.

Главное применение — **интерактивные переходы**: пользователь тянет пальцем, и анимация идёт вслед за ним (`fractionComplete` = пройденная пальцем доля), а если отпустил не дотянув — `isReversed = true`, и всё возвращается. Так устроен свайп-закрытие листа, который можно «передумать» на середине.

32.14 Бытовая аналогия

`UIView.animate` — это **дверь с доводчиком**. Ты говоришь, где она должна оказаться («закрыта»), а доводчик сам ведёт её туда за нужное время — с разгоном и торможением.

Пружина — та же дверь на тугой петле: немного проскочит упор и вернётся. Чем слабее демпфер (меньше damping), тем больше «хлопает».

`CABasicAnimation` с `repeatCount = .infinity` — **маятник часов**: запустил, и он качается сам, пока не остановишь. Положение маятника при этом ничего не меняет в механизме — как анимация слоя не меняет модельное значение.

`CAEmitterLayer` — **хлопушка**: одна точка вылета, сотни конфетти, у каждого своя траектория и время падения.

Упражнения

Упражнение 32.1. Сделай функцию `shake(_ view: UIView)` с амплитудой 8 точек, пятью «качаниями» и длительностью 0,3 с, которая возвращает view точно на место. Сколько значений будет в `values` и сколько длится каждый промежуток?

Упражнение 32.2. Кнопка «В корзину» при нажатии уменьшается до 90% и пружиной возвращается. Напиши `pressAnimation(_ button: UIButton)`: сжатие за 0,1 с, возврат — пружина с перелётом около 5% за 0,4 с. Какое значение damping взять?

Ответы к упражнениям

32.1.

```
func shake(_ view: UIView) {
    let animation = CAKeyframeAnimation(keyPath: "transform.translation.x")
    animation.values = [-8, 8, -6, 6, -3, 0]
    animation.duration = 0.3
    view.layer.add(animation, forKey: "shake")
}
```

Пять движений (влево, вправо, влево, вправо, влево) и возврат в ноль — шесть значений, пять промежутков. Каждый длится 0,3 / 5 = 0,06 с. Ноль в конце обязателен: без него на последнем кадре поле стоит сдвинутым на 3 точки, а потом «прыгает» на место.

32.2.

```
func pressAnimation(_ button: UIButton) {
    UIView.animate(withDuration: 0.1, animations: {
        button.transform = CGAffineTransform(scaleX: 0.9, y: 0.9)
    }) { _ in
        UIView.animate(withDuration: 0.4, delay: 0,
                       usingSpringWithDamping: 0.7, initialSpringVelocity: 0) {
            button.transform = .identity
        }
    }
}
```

`scaleX: 0.9, y: 0.9` — 90% от исходного размера, то есть на 10% меньше. Перелёт около 5% по замеру из 32.2 даёт damping 0.7. Возврат запускается в `completion` первой анимации — после того, как сжатие закончилось.

Что мы выучили

- **`UIView.animate`** — задаёшь конечное значение, система интерполирует. Модель меняется сразу, на экране — presentation layer. Документированные свойства — `frame`, `bounds`, `center`, `transform`, `alpha`, `backgroundColor`; `isHidden` — нет; свойства `view.layer` на практике анимируются, свои sublayer — нет.
- **Кривые**: ease-in-out (по умолчанию) — разгон и торможение; ease-out — для появления; ease-in — для ухода; linear — для вращения.
- **Пружина**: damping 0.5 — перелёт ~16%, 0.7 — ~5%, 1.0 — без перелёта. Velocity — доля пути в секунду. В iOS 17+ — `animate(springDuration:bounce:)`.
- **Последовательность** — через `delay` или `animateKeyframes` с долями общего времени.
- **Появление ячеек** — только первый экран, иначе задержка растёт с номером строки.

- **SF Symbol effects** (iOS 17+) — только внутри `#available`.
- **`CABasicAnimation`** — бесконечные анимации и свойства слоя; не меняет модель. «Оставить результат» — выставить модельное значение и добавить анимацию; `fillMode = .forwards` без модели — расхождение экрана и кода.
- **`CAKeyframeAnimation`** — тряска с затухающей амплитудой и нулём в конце.
- **Констрейнты** — `constant` снаружи, `layoutIfNeeded()` общего предка внутри анимации.
- **`UIView.transition`** — наплыв между view.
- **`CAEmitterLayer`** — конфетти; `contentsScale` у частиц, удалять слой после `lifetime`.
- **Радианы**: π = 180°; бесконечное вращение — через `CABasicAnimation`.
- **`UIViewPropertyAnimator`** — пауза, перемотка (линейная, `scrubsLinearly`), реверс для интерактивных анимаций. Zoom-переход — iOS 18+.

Apple Developer Documentation

- UIView.animate(withDuration:animations:) — базовая блочная анимация.
- UIView.animate(withDuration:delay:usingSpringWithDamping:initialSpringVelocity:options:animations:co — пружинная анимация.
- UIView.animateKeyframes(withDuration:delay:options:animations:completion:) — последовательность через ключевые кадры.
- UIView.transition(with:duration:options:animations:completion:) — наплыв и переворот между view.
- UIViewPropertyAnimator — пауза / перемотка / реверс, iOS 10+.
- UIViewPropertyAnimator.scrubsLinearly — линейная перемотка `fractionComplete` (по умолчанию `true`, iOS 11+).
- UIView.layoutIfNeeded() — раскладывает поддерево от view-получателя; почему вызывать у общего предка.
- UISpringTimingParameters — параметры пружины для `UIViewPropertyAnimator`.
- UICubicTimingParameters — собственные кривые разгона и торможения.
- CABasicAnimation — анимация свойств `CALayer`.
- CALayer.add(_:forKey:) — анимация копируется при добавлении.
- CAAnimation.isRemovedOnCompletion — по умолчанию `true`; вместе с `fillMode` объясняет «отскок».
- CAKeyframeAnimation — несколько ключевых значений (тряска и др.).
- CAEmitterLayer — система частиц (конфетти).
- CAEmitterCell — параметры одного вида частиц.
- UIViewControllerTransitioningDelegate — точка входа для собственного перехода.
- UIViewControllerAnimatedTransitioning — протокол анимации перехода.
- UIViewController.Transition — готовый zoom-переход, iOS 18+.
- UIImageView/addSymbolEffect(_:options:animated:) — анимации SF Symbols, iOS 17+.

→ Глава 33. Cookbook — haptics

Глава 33. Cookbook — haptics

Haptics (тактильная отдача) — короткие вибрации, которые пользователь чувствует пальцем: «щелчок» переключателя, «тик» барабана выбора даты, двойной толчок при успешной оплате. Это не звонковый вибромотор, а **Taptic Engine** — линейный моторчик внутри iPhone, который умеет делать очень короткие и точные импульсы.

В этой главе — три стандартных вида отдачи, когда какой уместен и как не переборщить.

Сначала о том, где отдача вообще работает, — иначе ты потратишь час, тыкая в симулятор:

- **Симулятор** вибрацию не воспроизводит. Вызовы проходят без ошибок и без эффекта. Проверять haptics можно только на устройстве.
- **iPhone.** Отдачу через `UIFeedbackGenerator` дают модели с Taptic Engine нового поколения — iPhone 7 и новее. На более старых вызовы просто ничего не делают.
- **iPad и iPod touch** встроенной тактильной отдачи не имеют — это прямо сказано в документации Core Haptics («Some devices don't support haptic feedback, including iPad, iPod touch, and Apple Vision Pro»). Приложение на iPad не падает, отдача просто молчит.
- **Пользователь может выключить** системную отдачу: Настройки → Звуки, тактильные сигналы → «Системные тактильные». Тогда стандартные генераторы тоже молчат.
- **Во время записи звука** (приложение пишет с микрофона) система по умолчанию глушит отдачу, чтобы вибрация не попала в запись. Разрешить можно через `AVAudioSession.setAllowHapticsAndSystemSoundsDuringRecording(true)`.

Отсюда главное правило: **отдача — всегда дополнение**. Любое событие, которое сопровождается вибрацией, должно быть понятно и без неё — анимацией, цветом, текстом.

33.1 Impact (удар)

Когда применять. Физическое событие в интерфейсе: элемент «встал на место», карточка «ударилась» о край, кнопку нажали.

```
let haptic = UIImpactFeedbackGenerator(style: .medium)
haptic.impactOccurred()

// С силой от 0 до 1 (iOS 13+)
haptic.impactOccurred(intensity: 0.6)
```

`UIImpactFeedbackGenerator` — **генератор** отдачи: объект, который знает, какой именно «удар» воспроизвести. `impactOccurred()` — «удар произошёл, сыграй».

Это не view и не контрол: `UIImpactFeedbackGenerator → UIFeedbackGenerator → NSObject` (цепочка снята на симуляторе; глава 0, раздел 0.8). Генератор никуда не добавляют в иерархию и ничем не рисуют — просто держат объект и зовут метод. Сам `UIFeedbackGenerator` — абстрактный класс: документация прямо просит не создавать его экземпляры, а брать один из наследников — impact, selection, notification и, с iOS 17.5, canvas для рисования.

Стили:

- **`.light`** — лёгкий, как касание мелкого предмета.
- **`.medium`** — средний, самый ходовой.
- **`.heavy`** — тяжёлый, «глухой».
- **`.soft`** (iOS 13+) — мягкий, упругий, как удар о подушку.
- **`.rigid`** (iOS 13+) — жёсткий, короткий, как щелчок.

`intensity: 0.6` — 60% силы выбранного стиля. Для частых событий (перетаскивание, прокрутка) берут 0,4–0,6: полная сила на каждое событие быстро утомляет.

> В SDK iOS 26.5 инициализатор `init(style:)` помечен как «будет устаревшим» (`API_TO_BE_DEPRECATED` в заголовке `UIImpactFeedbackGenerator.h`) в пользу `init(style:view:)` из iOS 17.5 — генератора, привязанного к конкретному view: с этой версии генератор ещё и `UIInteraction`, который прикрепляется к view и знает, где на экране случилось событие. У нас минимум iOS 15, поэтому пишем `init(style:)`; когда поднимешь минимум до 17.5, переходи на новый вариант.

33.2 Notification (успех / предупреждение / ошибка)

Когда применять. Сообщить **результат** действия: платёж прошёл, форма не отправилась, лимит почти исчерпан.

```
let notification = UINotificationFeedbackGenerator()
notification.notificationOccurred(.success)  // или .warning / .error
```

Три типа — три разных узора вибрации. Система играет их одинаково во всех приложениях, и пользователь со временем узнаёт их без экрана. Поэтому HIG Apple просит использовать их **по смыслу** («Use system-provided haptic patterns according to their documented meanings»): не ставь `.error` на «добавлено в избранное», даже если узор тебе больше нравится.

33.3 Selection (короткий тик)

Когда применять. Значение меняется по шагам: барабан выбора, сегменты, перетаскивание по засечкам.

```
let selection = UISelectionFeedbackGenerator()
selection.selectionChanged()
```

Очень короткий и тонкий тик. Подходит для **частых** событий: при прокрутке барабана — на каждое новое значение. По HIG стандартные переключатели, ползунки и барабаны выбора (`UISwitch`, `UISlider`, `UIPickerView`) играют системную отдачу сами — добавлять её к ним не нужно.

33.4 prepare() — прогрев

Taptic Engine в покое «спит», чтобы беречь батарею. Первый импульс после сна приходит с небольшой задержкой — на глаз это рассинхрон «анимация уже случилась, а вибрация чуть позже».

```
let haptic = UIImpactFeedbackGenerator(style: .medium)
haptic.prepare()  // прогрев

// ... через долю секунды или пару секунд:
haptic.impactOccurred()  // с минимальной задержкой
```

Что говорит документация Apple про `prepare()`:

- Генератор остаётся «прогретым» **недолго — обычно секунды**. Потом Taptic Engine снова засыпает.
- Вызвать `prepare()` и **сразу** `impactOccurred()` бесполезно: движку нужно время на подготовку.
- После срабатывания движок возвращается в покой; если отдача может понадобиться снова в ближайшие секунды — вызови `prepare()` ещё раз.

Когда вызывать:

- Когда событие **ожидается**. Пользователь начал тянуть ползунок — `prepare()`. Через секунду он дотянет до засечки — отдача без задержки.
- В начале жеста (`.began` у распознавателя), если в его конце будет отдача.

Когда можно обойтись: одиночный тап по кнопке — задержка там незаметна на фоне самого нажатия.

33.5 Один генератор или новый каждый раз

```
// Один на экран — лучше
final class ChatViewController: UIViewController {
    private let haptic = UIImpactFeedbackGenerator(style: .light)

    private func sendTapped() {
        haptic.impactOccurred()
        haptic.prepare()   // новое сообщение может уйти через секунду
    }
}

// Каждый раз новый — хуже
private func sendTapped() {
    let haptic = UIImpactFeedbackGenerator(style: .light)  // создали и выбросили
    haptic.impactOccurred()  // без прогрева
}
```

Генератор — лёгкий объект, создавать его не дорого. Смысл хранения в свойстве — в **прогреве**: `prepare()` действует на конкретный экземпляр, и у генератора, созданного «на один раз», прогреть нечего.

33.6 Свои узоры — Core Haptics (iOS 13+)

Core Haptics — низкоуровневый фреймворк для собственных узоров отдачи: нарастающая вибрация «бомба сейчас взорвётся», ритм сердца, отдача в такт музыке.

```
import CoreHaptics

final class HapticEngine {
    private var engine: CHHapticEngine?
    private let supportsHaptics = CHHapticEngine.capabilitiesForHardware().supportsHaptics

    init() {
        guard supportsHaptics else { return }
        do {
            let engine = try CHHapticEngine()
            engine.resetHandler = { [weak engine] in
                try? engine?.start()
            }
            engine.stoppedHandler = { reason in
                print("Haptic engine stopped: \(reason.rawValue)")
            }
            try engine.start()
            self.engine = engine
        } catch {
            print("Haptic engine failed: \(error)")
        }
    }

    func playCustom() {
        guard let engine else { return }
        let intensity = CHHapticEventParameter(parameterID: .hapticIntensity, value: 1.0)
        let sharpness = CHHapticEventParameter(parameterID: .hapticSharpness, value: 0.5)
        let event = CHHapticEvent(eventType: .hapticTransient,
                                  parameters: [intensity, sharpness],
                                  relativeTime: 0)
        do {
            try engine.start()
            let pattern = try CHHapticPattern(events: [event], parameters: [])
            let player = try engine.makePlayer(with: pattern)
            try player.start(atTime: CHHapticTimeImmediate)
        } catch {
            print("Failed to play haptic: \(error)")
        }
    }
```

```
}
```

Разбор по шагам:

- `capabilitiesForHardware().supportsHaptics` — умеет ли устройство. На iPad и в симуляторе — `false`, и мы вообще не создаём движок. Проверяй возможность, а не модель телефона.
- `CHHapticEngine` — связь приложения с моторчиком. Без запущенного движка узор не сыграть.
- `resetHandler` — система перезапустила свой сервер отдачи (такое бывает); движок нужно запустить снова. `[weak engine]` — чтобы замыкание, которое хранит сам движок, не держало его сильной ссылкой.
- `stoppedHandler` — движок остановили: звонок, уход приложения в фон. Это нормальная часть жизни движка, поэтому в `playCustom()` перед проигрыванием мы на всякий случай вызываем `engine.start()` — для уже запущенного движка это безопасно.
- `CHHapticEvent` — одно событие узора. `.hapticTransient` — короткий удар; бывает ещё `.hapticContinuous` — длительная вибрация с `duration`.
- Параметры от 0 до 1: **intensity** — сила, **sharpness** — «острота» (0 — мягкий, округлый толчок, 1 — чёткий, «механический» щелчок).
- `relativeTime: 0` — когда внутри узора случится событие, в секундах от начала. Три события с временем 0, 0,1 и 0,2 — три удара с интервалом в десятую долю секунды.
- `makePlayer(with:)` + `start(atTime: CHHapticTimeImmediate)` — сыграть прямо сейчас.

Core Haptics — мощный, но многословный инструмент. Бери его, только если трёх стандартных генераторов действительно не хватает.

33.7 Когда haptics не нужны

- **Обычная прокрутка списка** — система там отдачу не даёт, и добавленная тобой будет раздражать.
- **Каждое нажатие клавиши** — у клавиатуры своя отдача, пользователь включает её в Настройках.
- **Долгие операции** — не вибрируй «в процессе». Одна отдача в конце: `.success` или `.error`.
- **Дублирование стандартных контролов** — `UISwitch`, барабаны пикера уже вибрируют сами.
- **Без выключателя.** HIG Apple советует делать отдачу отключаемой («Make haptics optional»). Системный переключатель выключает стандартные генераторы, а для игр и Core Haptics-эффектов заведи свой пункт в настройках приложения:

```
enum HapticsSettings {
    static var isEnabled: Bool {
        UserDefaults.standard.object(forKey: "settings.haptics") as? Bool ?? true
    }
}

if HapticsSettings.isEnabled {
    haptic.impactOccurred()
}
```

`object(forKey:) as? Bool ?? true` — если пользователь ещё ничего не выбирал, ключа нет, и мы считаем отдачу включённой. Обычный `bool(forKey:)` вернул бы для отсутствующего ключа `false`.

33.8 Отдача для ползунка

```
final class HapticSlider: UISlider {
    private let haptic = UIImpactFeedbackGenerator(style: .light)
    private var lastTick: Float = 0

    override init(frame: CGRect) {
        super.init(frame: frame)
        addTarget(self, action: #selector(check), for: .valueChanged)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    @objc private func check() {
        let bucket = (value * 20).rounded()
        if bucket != lastTick {
            haptic.impactOccurred(intensity: 0.5)
            lastTick = bucket
        }
    }
}
```

Математика словами: значение ползунка от 0 до 1. Умножаем на 20 и округляем — получаем номер «засечки» от 0 до 20, то есть шкала разбита на 20 шагов по 5%. Значение 0,33 даёт 6,6 → засечка 7; 0,36 даёт 7,2 → тоже 7, отдачи нет; 0,38 даёт 7,6 → 8, щелчок. Отдача срабатывает только при **смене** засечки, а не на каждое событие `valueChanged` (их при перетаскивании десятки в секунду).

`required init?(coder:)` обязателен: `UISlider → UIControl → UIView`, а `UIView` реализует протокол `NSCoding` и объявляет `init?(coder:)` как `required` (глава 0, разделы 0.3 и 0.9). Как только ты написал свой назначенный `init(frame:)`, компилятор требует явно написать и его, иначе код не соберётся.

События `.valueChanged` при перетаскивании приходят десятки раз в секунду, потому что у ползунка `isContinuous = true` по умолчанию (заголовок `UISlider.h`); поставь `false` — и событие придёт один раз, когда палец отпустят, но тогда и засечки по пути не прощёлкают.

По смыслу здесь подходит и `UISelectionFeedbackGenerator` (33.3): засечки — это смена значения по шагам. Разница на ощупь: selection — тоньше, impact `.light` — чуть «плотнее». Полная версия с плеером — в главе 17.10.

33.9 Отдача при достижении цели

«Шагомер: 10 000 шагов сегодня!»

```
func didReachGoal() {
    let notification = UINotificationFeedbackGenerator()
    notification.notificationOccurred(.success)

    // Вместе с отдачей — «подпрыгивание» иконки
    UIView.animate(withDuration: 0.2, animations: {
        self.goalIcon.transform = CGAffineTransform(scaleX: 1.3, y: 1.3)
    }) { _ in
        UIView.animate(withDuration: 0.3, delay: 0,
                       usingSpringWithDamping: 0.5, initialSpringVelocity: 0) {
            self.goalIcon.transform = .identity
        }
    }
}
```

Иконка за 0,2 с вырастает на 30% (`1.3` — 130% размера), потом пружиной с damping 0.5 возвращается — с заметным отскоком (около 16% перелёта, см. главу 32.2). Отдачу запускаем **в тот же момент**, что и анимацию: вибрация и визуальный «подскок» воспринимаются как одно событие. Отдача после анимации ощущалась бы запоздалой.

33.10 Отдача при перетаскивании

Пользователь долгим нажатием «берёт» элемент, тащит и отпускает:

```
@objc private func handleLongPress(_ gesture: UILongPressGestureRecognizer) {
    switch gesture.state {
    case .began:
        UIImpactFeedbackGenerator(style: .medium).impactOccurred()
        // начать перетаскивание
    case .ended:
        UIImpactFeedbackGenerator(style: .light).impactOccurred()
        // положить элемент на новое место
    default:
        break
    }
}
```

`.began` — долгое нажатие распознано (по умолчанию после 0,5 с удержания — `minimumPressDuration`, значение из заголовка и рантайма), `.ended` — палец отпущен. «Взять» — `.medium`: важное событие, должно ощущаться. «Положить» — `.light`: подтверждение, без нажима. Здесь мы создаём генераторы на месте, без прогрева, и это осознанное упрощение: между «взять» и «положить» проходят секунды, а для производственного кода храни генератор в свойстве и вызывай `prepare()` в `.began` (раздел 33.4).

Упражнения

Упражнение 33.1. Для экрана оплаты подбери отдачу: а) нажатие кнопки «Оплатить»; б) платёж прошёл; в) банк отклонил карту; г) пользователь листает барабан выбора срока рассрочки (`UIPickerView`). Что выбрать в каждом случае и где отдачу добавлять не нужно?

Упражнение 33.2. Поменяй `HapticSlider` так, чтобы засечек было 10 (каждые 10%) и чтобы генератор прогревался, когда пользователь касается ползунка. Подсказка: событие `.touchDown`.

Ответы к упражнениям

33.1. а) `UIImpactFeedbackGenerator(style: .medium)` — или вообще без отдачи: нажатие кнопки и так видно. Результата ещё нет, поэтому не `.success`. б) `UINotificationFeedbackGenerator` с `.success`. в) Тот же генератор с `.error` — и обязательно текст ошибки на экране. г) Ничего не добавляй: `UIPickerView` даёт тиковую отдачу сам.

33.2.

```
final class HapticSlider: UISlider {
    private let haptic = UIImpactFeedbackGenerator(style: .light)
    private var lastTick: Float = 0

    override init(frame: CGRect) {
        super.init(frame: frame)
        addTarget(self, action: #selector(check), for: .valueChanged)
        addTarget(self, action: #selector(warmUp), for: .touchDown)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    @objc private func warmUp() {
        lastTick = (value * 10).rounded()
        haptic.prepare()
    }

    @objc private func check() {
        let bucket = (value * 10).rounded()
        if bucket != lastTick {
            haptic.impactOccurred(intensity: 0.5)
            haptic.prepare()
            lastTick = bucket
        }
    }
}
```

`value * 10` — десять шагов по 10%. `.touchDown` приходит, когда палец коснулся ползунка, ещё до перетаскивания, — лучшее время для `prepare()`. В `warmUp` заодно запоминаем текущую засеч-

ку, чтобы первое же микродвижение не дало ложный щелчок. `prepare()` после каждого удара держит движок прогретым для очередной засечки.

Что мы выучили

- **Где работает**: только на устройстве; iPhone 7 и новее; на iPad и в симуляторе — тишина; пользователь может выключить; при записи звука отдача глушится.
- **Генераторы — `NSObject`**, не view и не контролы: держи объект в свойстве, в иерархию ничего не добавляй; базовый класс абстрактный.
- **`UIImpactFeedbackGenerator`** — стили `.light` / `.medium` / `.heavy` / `.soft` / `.rigid`, `intensity` от 0 до 1.
- **`UINotificationFeedbackGenerator`** — `.success` / `.warning` / `.error`, строго по смыслу.
- **`UISelectionFeedbackGenerator`** — тик для пошаговой смены значения.
- **`prepare()`** — за доли секунды или секунды до события, не вплотную; действует недолго.
- **Генератор в свойстве** — чтобы было что прогревать.
- **Core Haptics** — свои узоры; проверка `supportsHaptics`, перезапуск движка в `resetHandler` и перед проигрыванием.
- **Не нужны** при прокрутке, наборе текста, поверх стандартных контролов; делай отдачу отключаемой.
- **Ползунок с засечками** — отдача только при смене засечки (`isContinuous = true` даёт десятки событий в секунду); `required init?(coder:)` обязателен из-за `NSCoding` у `UIView`.
- **Синхронно с анимацией** — отдача в момент начала анимации.

Apple Developer Documentation

- UIFeedbackGenerator — абстрактный базовый класс тактильной отдачи (`NSObject`).
- UIFeedbackGenerator.init(view:) — генератор, привязанный к view, iOS 17.5+.
- UIFeedbackGenerator.prepare() — прогрев Taptic Engine и сколько он длится.
- UIImpactFeedbackGenerator — «удар» при действиях в интерфейсе.
- UIImpactFeedbackGenerator.FeedbackStyle — `.light` / `.medium` / `.heavy` / `.soft` / `.rigid`.
- UINotificationFeedbackGenerator — `.success` / `.warning` / `.error`.
- UISelectionFeedbackGenerator — короткий тик при смене значения.
- Preparing your app to play haptics — проверка `supportsHaptics`, какие устройства не поддерживают отдачу, перезапуск движка.
- CHHapticEngine — движок Core Haptics, iOS 13+.
- CHHapticPattern — описание собственного узора.
- HIG — Playing haptics — когда отдача уместна и почему её делают отключаемой.

→ Глава 34. Cookbook — accessibility

Глава 34. Cookbook — accessibility

Accessibility (доступность) — это возможность пользоваться приложением людям с особенностями зрения, слуха, моторики: незрячему — через экранный диктор, слабовидящему — с крупным шрифтом, человеку с тремором — с крупными кнопками. iOS даёт для этого десятки встроенных инструментов, и почти все они работают со стандартными компонентами UIKit «бесплатно». Ломаются они там, где ты рисуешь что-то своё: иконка без текста, карточка из трёх подписей, свой ползунок.

Обязательна ли доступность? В правилах App Review отдельного пункта «приложение обязано поддерживать VoiceOver» нет. Но с 2025 года в App Store Connect можно заполнить **Accessibility Nutrition Labels** — метки на странице приложения «поддерживает VoiceOver, крупный текст, …», и пользователи их видят. А в ряде стран доступность цифровых сервисов требует закон (например, European Accessibility Act в ЕС действует с июня 2025 года для многих видов услуг). И главное — это твои живые пользователи.

Ключевые термины главы:

- **VoiceOver** — экранный диктор iOS. Пользователь водит пальцем по экрану или смахивает влево-вправо, VoiceOver переходит от элемента к элементу и произносит, что это. Двойное касание — «нажать».
- **Accessibility element** (элемент доступности) — то, на чём VoiceOver может остановиться. У каждого есть **label** (имя), **traits** (что это за штука: кнопка, заголовок, картинка), **value** (текущее значение) и **hint** (подсказка).
- **Dynamic Type** — системный размер шрифта, который пользователь выбирает в Настройках. Приложение должно под него подстраиваться.

34.1 VoiceOver — accessibilityLabel и hint

Когда применять. У каждого интерактивного элемента должно быть понятное имя.

```
button.accessibilityLabel = "Удалить заметку"
button.accessibilityHint = "Удаляет заметку без возможности восстановления"
```

`accessibilityLabel` — **что это**, коротко. VoiceOver прочитает «Удалить заметку, кнопка»: слово «кнопка» он добавит сам по трейту, поэтому в label его не пиши («Кнопка удалить» прозвучит как «Кнопка удалить, кнопка»). У кнопки с текстом label берётся из заголовка автоматически.

Откуда эти свойства у кнопки. `accessibilityLabel`, `accessibilityHint`, `accessibilityValue`, `accessibilityTraits` и `isAccessibilityElement` объявлены не в `UIView`, а в категории `NSObject (UIAccessibility)` (заголовок `UIAccessibility.h`) — то есть они есть у любого объекта UIKit,

в том числе у `UIBarButtonItem` и `UITabBarItem`, которые не view (глава 0, разделы 0.1 и 0.8). `isAccessibilityElement` по умолчанию `false`, а у стандартных контролов UIKit — `true` (комментарий в заголовке и документация).

`accessibilityHint` — **что произойдёт**, если нажать. Необязательна, VoiceOver читает её после паузы, и многие пользователи подсказки отключают. Apple в документации к `accessibilityHint` советует описывать результат глаголом без подлежащего («Удаляет заметку», «Воспроизводит песню») и не описывать жест («Дважды коснитесь, чтобы ...»): жест пользователь VoiceOver знает лучше тебя, а на Apple Watch или с клавиатурой он другой.

Иконки без текста — главный источник проблем:

```
let deleteButton = UIButton(type: .system)
deleteButton.setImage(UIImage(systemName: "trash"), for: .normal)
deleteButton.accessibilityLabel = "Удалить"
```

Без явного label VoiceOver попытается назвать кнопку по картинке: у SF Symbols есть системное описание, у твоих ресурсов — имя файла, а если и этого нет, прозвучит просто «кнопка». Ни один из вариантов не гарантирует понятное слово на языке интерфейса — задавай label явно.

34.2 Скрыть от VoiceOver

```
decorativeView.isAccessibilityElement = false
decorativeView.accessibilityElementsHidden = true
```

Два свойства делают разное:

- `isAccessibilityElement = false` — **сам** view не элемент, но его дочерние view VoiceOver по-прежнему найдёт. У простого `UIView` и `UIImageView` это и так значение по умолчанию (элементами по умолчанию являются только стандартные контролы — документация `isAccessibilityElement`).
- `accessibilityElementsHidden = true` — спрятать **всё, что внутри** view: заголовок `UIAccessibility.h` описывает свойство как «marks all the accessible elements contained within as hidden». Сам контейнер оно элементом не делает и не отменяет; для декоративного блока обычно ставят оба свойства. Нужно, когда блок состоит из подписей и картинок, и ни одна из них не должна звучать (например, фоновая иллюстрация с надписью «Добро пожаловать» поверх настоящего заголовка).

Скрывай декор (фоновые картинки, разделители, иконку рядом с текстом, который и так всё говорит), но никогда — то, что несёт смысл.

34.3 Группировка элементов

```
card.isAccessibilityElement = true
card.accessibilityLabel = "iPhone 17 Pro, 649 990 тенге"
card.accessibilityTraits = [.button]
```

Карточка товара из двух подписей «iPhone 17 Pro» и «649 990 ₸» без группировки — это два отдельных элемента: пользователь смахивает, слышит название, смахивает ещё раз, слышит цену. В длинном списке это вдвое больше жестов. Когда сама карточка стала элементом (`isAccessibilityElement = true`), её дочерние подписи VoiceOver больше не видит — слышно одно объявление: «iPhone 17 Pro, 649 990 тенге, кнопка».

Обрати внимание на «тенге» словом: значок «₸» VoiceOver может прочитать не так, как ожидаешь. Для денег надёжнее собрать label из текста. Трейт `.button` — потому что по карточке можно нажать и перейти к товару.

34.4 Dynamic Type

Когда применять. Всегда, для всего текста интерфейса.

```
label.font = UIFont.preferredFont(forTextStyle: .body)
label.adjustsFontForContentSizeCategory = true
```

`preferredFont(forTextStyle:)` — системный шрифт для **стиля текста** («основной текст», «заголовок», «подпись»), размер которого зависит от выбора пользователя: Настройки → Экран и яркость → Размер текста, а самые крупные размеры — в Универсальном доступе → Дисплей и размер текста → Увеличенный текст.

`adjustsFontForContentSizeCategory = true` — пересчитывать шрифт, когда пользователь меняет размер, прямо на лету. Без этого новый размер применится только к экранам, созданным после смены.

Насколько крупно бывает — замер на симуляторе, размер стиля `.body` в точках:

Настройка	`.body`	`.largeTitle`	`.caption1`
Самый мелкий (XS)	14	31	11
По умолчанию (L)	17	34	12
Самый крупный обычный (XXXL)	23	40	18
Самый крупный «универсальный доступ» (AX5)	53	60	43

При максимальной настройке основной текст в **три с лишним раза** крупнее обычного (53 против 17). Строка, которая помещалась в одну линию, займёт три-четыре. Поэтому Dynamic Type — это не только шрифт, но и вёрстка: `numberOfLines = 0`, вертикальные стеки вместо горизонтальных на больших размерах (проверка — `traitCollection.preferredContentSizeCategory.isAccessibilityC` ячейки с автоматической высотой (34.12).

Стили:

- `.largeTitle` — крупный заголовок экрана;
- `.title1` / `.title2` / `.title3` — заголовки разделов;
- `.headline` — выделенная строка; `.body` — основной текст; `.callout`, `.subheadline` — второстепенный;
- `.footnote`, `.caption1`, `.caption2` — мелкие подписи.

Свой шрифт:

```
let base = UIFont(name: "Avenir-Book", size: 16) ?? .systemFont(ofSize: 16)
label.font = UIFontMetrics(forTextStyle: .body).scaledFont(for: base)
label.adjustsFontForContentSizeCategory = true
```

`UIFontMetrics` масштабирует твой шрифт так же, как система масштабирует стиль `.body`: 16 точек при настройке по умолчанию, 21 при XXXL, 45 при AX5 (замер на симуляторе). `??` `.systemFont` вместо `!` — если шрифта с таким именем нет (опечатка, шрифт не добавлен в проект), приложение не упадёт.

34.5 Reduce Motion — «Уменьшение движения»

Людей с вестибулярными нарушениями от резких движений на экране укачивает: параллакс, пружины, масштабирование на весь экран вызывают головокружение. В Настройках → Универсальный доступ → Движение есть переключатель «Уменьшение движения».

```
if UIAccessibility.isReduceMotionEnabled {
    // Спокойный вариант — только прозрачность
    UIView.animate(withDuration: 0.2) { view.alpha = 1 }
} else {
    // Полный вариант — пружина со сдвигом
    UIView.animate(withDuration: 0.5, delay: 0,
                   usingSpringWithDamping: 0.6,
                   initialSpringVelocity: 0.3) {
        view.transform = .identity
    }
}
```

Правило: движение заменяем на **растворение** (изменение прозрачности), а не выключаем анимацию совсем — иначе пропадает сама подсказка «что-то изменилось». Декоративные эффекты (лесенка ячеек из главы 32.4, конфетти) при включённой настройке лучше не показывать вовсе.

Чтобы среагировать на переключение, не перезапуская экран, подпишись на `UIAccessibility.reduceMotionS`

34.6 Зона нажатия — минимум 44 × 44 точки

Apple Human Interface Guidelines: минимальный размер элемента, по которому нажимают пальцем, — **44 × 44 точки** (около 7 мм на экране iPhone). Меньше — промахи, особенно у людей с тремором и у всех остальных на ходу.

```
button.heightAnchor.constraint(greaterThanOrEqualToConstant: 44).isActive = true
button.widthAnchor.constraint(greaterThanOrEqualToConstant: 44).isActive = true
```

`greaterThanOrEqualToConstant` — «не меньше 44», но может быть больше, если содержимое крупнее (например, при большом Dynamic Type).

Маленькую иконку можно оставить маленькой визуально, увеличив отступы внутри кнопки:

```
var cfg = UIButton.Configuration.plain()
cfg.image = UIImage(systemName: "xmark")
cfg.contentInsets = NSDirectionalEdgeInsets(top: 12, leading: 12, bottom: 12, trailing:
↪  12)
let button = UIButton(configuration: cfg)
```

Арифметика на глаз: иконка «xmark» около 17 × 15 точек, плюс отступ 12 с каждой стороны — примерно 41 × 39, меньше нормы. Замер на симуляторе дал другое: `button.intrinsicContentSize` — 46 × 44 точки, потому что конфигурация кнопки добавляет к картинке ещё немного своего поля. Вывод: не считай размер в уме, а смотри `intrinsicContentSize` (или рамку элемента в Accessibility Inspector) и страхуй констрейнтами «не меньше 44» из примера выше.

34.7 Контраст и Increase Contrast

Контраст — насколько текст отличается по яркости от фона. Стандарт WCAG (международные правила доступности веб-контента, на них ссылается и Apple) требует для обычного текста **4.5:1**, для крупного (от 18 точек, или от 14 жирным) и для значков — **3:1**. В HIG та же норма сведена в таблицу попроще, по ней проверяет Accessibility Inspector: текст до 17 pt — 4.5:1, от 18 pt — 3:1, жирный любого размера — 3:1 (эта таблица — в чек-листе главы 43). Мелкий жирный текст надёжнее всё же держать на 4.5:1: так он проходит оба правила.

Что значит «4.5:1» словами. У каждого цвета считают «светлоту» — число от 0 (чёрный) до 1 (белый). Зелёный канал вносит в неё больше всего (около 72%), красный — около 21%, синий — всего 7%: глаз гораздо чувствительнее к зелёному. Потом берут светлоту более светлого цвета и более тёмного, к обеим прибавляют 0,05 и делят одно на другое. Чёрный на белом: (1 + 0,05) / (0 + 0,05) = 21, максимум. Одинаковые цвета: 1:1, текста не видно. Порог 4.5:1 — это, например, серый `#767676` на белом (4,54:1): чуть светлее — и уже не проходит.

Системные цвета — не гарантия. Замер на симуляторе iOS 26.5, цвет текста на `systemBackground`:

Цвет	Светлая тема	Тёмная тема	Светлая + Increase Contrast
`.label`	21:1	21:1	21:1
`.secondaryLabel`	**3,4:1**	6,4:1	6,0:1
`.tertiaryLabel`	**1,7:1**	**2,2:1**	4,5:1
`.systemBlue`	**3,5:1**	6,5:1	4,6:1
`.systemGreen`	**2,2:1**	10,4:1	4,5:1

Выводы. `.secondaryLabel` в светлой теме — 3,4:1: для мелких подписей это ниже нормы 4.5:1, для крупного текста — проходит. `.tertiaryLabel` годится только для совсем второстепенного (плейсхолдеры, неактивное). Зелёный текст на белом (2,2:1) читается плохо — зелёный хорош для значков и заливок, но не для текста. При этом в режиме **Increase Contrast** («Увеличение контраста», Настройки → Универсальный доступ → Дисплей и размер текста) система сама темнеет цвета, и почти все доходят до 4.5:1. Своим цветам ты должен дать такую же возможность.

Как узнать, что режим включён:

```
if traitCollection.accessibilityContrast == .high {
    label.textColor = .label
} else {
    label.textColor = .secondaryLabel
}
```

`accessibilityContrast` — свойство **trait collection** (набора характеристик окружения: тема, размер шрифта, контраст; подробно — в главе 35). Есть и глобальный флаг `UIAccessibility.isDarkerSystemColor` с уведомлением `darkerSystemColorsStatusDidChangeNotification`.

Где читать `traitCollection`. Пока view не в окне, его trait collection предварительная: в замере на симуляторе у свободного `UIView` и у только что созданного контроллера size class равен `unspecified`. Надёжные места — обработчик `registerForTraitChanges` (iOS 17) или `traitCollectionDidChange` (iOS 15–16), `layoutSubviews`, а в контроллере — `viewIsAppearing`, где, по документации, трейты и геометрия уже актуальны. Подробно — глава 35, вводная часть и раздел 35.6.

Проще всего — не писать условия руками. Для своих цветов в Asset Catalog у набора цветов включи **High Contrast**: рядом с вариантами «Any» и «Dark» появятся высококонтрастные, и iOS подставит их сама.

34.8 Accessibility traits

```
button.accessibilityTraits = [.button]
imageView.accessibilityTraits = [.image]
headerLabel.accessibilityTraits = [.header]
customSlider.accessibilityTraits = [.adjustable]  // VoiceOver: смахивание вверх/вниз
↪  меняет значение

// Несколько сразу
tabButton.accessibilityTraits = [.button, .selected]  // выбранная вкладка
```

Трейт — «что это за элемент и в каком он состоянии». VoiceOver читает трейт после label: «Настройки, кнопка», «Профиль, заголовок», «Громкость, регулируемый элемент».

- `.header` — заголовок раздела. Незрячие пользователи часто перемещаются **по заголовкам** (через ротор — «колёсико» VoiceOver, которое переключает режим навигации поворотом двух пальцев), поэтому заголовки экранов и секций помечай обязательно.
- `.adjustable` — элемент со значением, которое меняют смахиванием вверх/вниз. Для своего контрола тогда нужно переопределить `accessibilityIncrement()` и `accessibilityDecrement()`.
- `.selected` — выбранное состояние (вкладка, сегмент, отмеченная строка).

Стандартные `UIButton`, `UISlider`, `UISwitch` проставляют трейты сами. Свой view, собранный из `UIView` с жестом нажатия, — нет: без трейтов VoiceOver не поймёт, что по нему можно нажать.

34.9 accessibilityValue — текущее значение

```
ratingView.accessibilityValue = "4 из 5 звёзд"
// VoiceOver: "Оценка, 4 из 5 звёзд, регулируемый элемент"
```

Value — то, что **меняется**, в отличие от label, который постоянен: label «Громкость», value «50 процентов». Стандартные `UISlider` и `UIStepper` сообщают значение сами. Задавать value вручную нужно для своих контролов: рейтинг звёздами, круговой прогресс, свой переключатель.

34.10 Проверка через VoiceOver

В симуляторе VoiceOver нет. Сочетание Cmd+F5 на Mac включает VoiceOver **самого Mac**, а не iOS внутри симулятора — это другой диктор с другими жестами. В симуляторе проверяй разметку через Accessibility Inspector (34.11).

На устройстве:

- Настройки → Универсальный доступ → VoiceOver → включить;
- удобнее — **быстрая команда**: Настройки → Универсальный доступ → Быстрая команда → VoiceOver. Тогда тройное нажатие боковой кнопки включает и выключает диктор.

Базовые жесты: смахивание вправо/влево одним пальцем — очередной / предыдущий элемент, двойное касание — нажать, смахивание вверх/вниз — изменить регулируемый элемент или выбрать действие (34.14).

Пройди с VoiceOver главный сценарий приложения — вход, основной экран, покупку — перед каждым крупным релизом. Это 15 минут, которые находят больше, чем любой автоматический аудит.

34.11 Accessibility Inspector (Xcode)

`Xcode → Open Developer Tool → Accessibility Inspector`. Инструмент подключается к симулятору, и при наведении на элемент показывает его label, value, traits и место в иерархии. Кнопка **Audit** прогоняет экран через автоматические проверки: низкий контраст, маленькие зоны нажатия, элементы без label, текст, который не масштабируется под Dynamic Type.

С iOS 17 такой же аудит можно запускать в UI-тестах: `try app.performAccessibilityAudit()` — тест упадёт, если на экране появится элемент без имени.

34.12 Dynamic Type в ячейках таблицы

```
tableView.rowHeight = UITableView.automaticDimension
tableView.estimatedRowHeight = 44

// В ячейке
label.numberOfLines = 0
label.font = UIFont.preferredFont(forTextStyle: .body)
```

```
label.adjustsFontForContentSizeCategory = true
```

`automaticDimension` — высота ячейки считается из её констрейнтов, `estimatedRowHeight` — примерная высота для расчёта полосы прокрутки до того, как ячейку разложат. Первая строка — значение по умолчанию: у свежего `UITableView` и `rowHeight`, и `estimatedRowHeight` равны `automaticDimension` (это −1; замер на симуляторе), так что пишут её для явности, а вот явная оценка 44 помогает таблице точнее считать полосу прокрутки. `numberOfLines = 0` — сколько угодно строк. Вместе с `preferredFont` получаем ячейку, которая при размере AX5 вырастает в высоту, а не обрезает текст многоточием. Работает, только если подпись привязана констрейнтами к верху **и** к низу `contentView` — иначе таблице не из чего вычислить высоту.

34.13 Картинки

```
productImage.isAccessibilityElement = true
productImage.accessibilityLabel = "Красные кроссовки, вид сбоку"

// Декоративная картинка (фон) — ничего не нужно:
// у UIImageView isAccessibilityElement по умолчанию false
```

Содержательная картинка (фото товара, график, схема) должна быть элементом с описанием **того, что на ней**, а не «картинка» или имя файла. Декоративная — наоборот, остаётся невидимой для VoiceOver: у `UIImageView` по умолчанию `isAccessibilityElement == false` (элементами по умолчанию документация объявляет только стандартные контролы, а `UIImageView → UIView` — не контрол).

34.14 Custom actions — свои действия

Смахивания в ячейках (swipe actions) и контекстные меню незрячему пользователю неудобны или недоступны. Для них есть **custom actions** — действия, которые VoiceOver предлагает прямо на элементе:

```
let noteID = note.id
cell.accessibilityCustomActions = [
    UIAccessibilityCustomAction(name: "Удалить") { [weak self] _ in
        self?.deleteNote(id: noteID)
        return true
    },
    UIAccessibilityCustomAction(name: "В архив") { [weak self] _ in
        self?.archiveNote(id: noteID)
        return true
    },
]
```

На такой ячейке VoiceOver скажет «Доступны действия». Смахивание вверх/вниз перебирает их («Удалить», «В архив»), двойное касание выполняет. Замыкание возвращает `true`, если действие удалось.

Две детали в коде:

- `[weak self]` — ячейку держит таблица, таблицу — контроллер; если действие будет держать контроллер сильно, получится цикл удержания.
- Захватываем **идентификатор заметки**, а не `indexPath`. Номер строки меняется после удаления соседних строк, а ячейку со старыми действиями могут не успеть перенастроить, — и «Удалить» удалит не ту заметку.

34.15 Бытовая аналогия

Accessibility — как **таблички в здании**. Зрячий человек найдёт кабинет по номеру на двери. Незрячий — по табличке со шрифтом Брайля (`accessibilityLabel`). Человек на коляске — по пандусу вместо ступенек (крупные зоны нажатия). Пожилой — по крупным цифрам на указателях (Dynamic Type).

Здание без табличек формально работает — стены, двери, всё на месте, — но половина посетителей в нём заблудится. Табличку повесить дёшево, если делать это сразу, и дорого, если потом обходить всё здание заново.

Упражнения

Упражнение 34.1. В ячейке заметки три подписи: заголовок «Купить молоко», дата «12 мая» и значок-скрепка «есть вложение» (SF Symbol `paperclip`). Сделай так, чтобы VoiceOver читал ячейку одним объявлением. Что должно прозвучать?

Упражнение 34.2. Кнопка «Закрыть» — иконка `xmark` 16 точек без текста. Перечисли, что с ней нужно сделать, чтобы она прошла Accessibility Inspector Audit, и напиши код.

Ответы к упражнениям

34.1.

```
func configureAccessibility(of cell: UITableViewCell,
                            title: String, dateText: String, hasAttachment: Bool) {
    cell.isAccessibilityElement = true
    var parts = [title, dateText]
    if hasAttachment { parts.append("есть вложение") }
    cell.accessibilityLabel = parts.joined(separator: ", ")
}
```

Прозвучит: «Купить молоко, 12 мая, есть вложение». Если по ячейке можно нажать и открыть заметку, добавь `cell.accessibilityTraits .insert(.button)` — тогда VoiceOver допишет «кнопка». Иконку скрепки мы озвучили словами, а сама по себе она больше не звучит: ячейка стала элементом, и её дочерние view VoiceOver не видит.

34.2. Нужно: понятное имя на языке интерфейса и зона нажатия не меньше 44 × 44 точек.

```
func makeCloseButton() -> UIButton {
    var cfg = UIButton.Configuration.plain()
    cfg.image = UIImage(systemName: "xmark")
    cfg.contentInsets = NSDirectionalEdgeInsets(top: 12, leading: 12,
                                                bottom: 12, trailing: 12)
    let button = UIButton(configuration: cfg)
    button.accessibilityLabel = "Закрыть"
    button.translatesAutoresizingMaskIntoConstraints = false
    NSLayoutConstraint.activate([
        button.widthAnchor.constraint(greaterThanOrEqualToConstant: 44),
        button.heightAnchor.constraint(greaterThanOrEqualToConstant: 44),
    ])
    return button
}
```

С отступами 12 такая кнопка на симуляторе получилась 46 × 44 точки (см. 34.6), а констрейнты «не меньше 44» страхуют, если иконку заменят на меньшую или уберут отступы. Трейт `.button` у `UIButton` уже есть.

Что мы выучили

- **Label** — «что это», без слова «кнопка»; у иконок — обязательно и по-русски. **Hint** — «что произойдёт», глаголом, без описания жеста.
- **Accessibility-свойства объявлены на `NSObject`** — есть у любого объекта UIKit, включая `UIBarButtonItem`; `isAccessibilityElement` по умолчанию `true` только у контролов.
- **`isAccessibilityElement = false`** скрывает сам view, **`accessibilityElementsHidden`** — всё его содержимое.
- **`traitCollection`** читать после входа в иерархию: `registerForTraitChanges` / `traitCollectionDidChange`, `layoutSubviews`, `viewIsAppearing`.
- **Группировка**: карточка — один элемент с общим label.
- **Dynamic Type**: `preferredFont` + `adjustsFontForContentSizeCategory`; размер `.body` от 14 до 53 точек; свои шрифты — через `UIFontMetrics`.
- **Reduce Motion** — движение заменяем растворением.
- **Зона нажатия** не меньше 44 × 44 точек.
- **Контраст** 4.5:1 для текста, 3:1 для крупного и значков; `.secondaryLabel` в светлой теме — 3,4:1; High Contrast в Asset Catalog.
- **Traits**: `.header` для навигации по заголовкам, `.adjustable`, `.selected`.
- **Value** — меняющееся значение своих контролов.
- **VoiceOver** проверяется только на устройстве; в симуляторе — Accessibility Inspector и `performAccessibilityAudit()` (iOS 17).
- **Custom actions** — замена свайпам, захватывать ID, а не `indexPath`.

Apple Developer Documentation

- UIAccessibility — глобальные настройки доступности и уведомления.

- UIAccessibility.isVoiceOverRunning — VoiceOver включён.
- UIAccessibility.isReduceMotionEnabled — пользователь просит уменьшить движение.
- UIAccessibility.isDarkerSystemColorsEnabled — включено Increase Contrast.
- UITraitCollection.accessibilityContrast — контраст как характеристика окружения.
- isAccessibilityElement — по умолчанию `false`, у контролов UIKit — `true`.
- UIViewController.viewIsAppearing(_:) — момент, когда trait collection и геометрия уже актуальны.
- accessibilityLabel — короткое имя элемента для VoiceOver.
- accessibilityHint — подсказка о результате действия.
- accessibilityValue — текущее значение элемента.
- accessibilityTraits — `.button`, `.header`, `.adjustable` и т. д.
- UIAccessibilityCustomAction — свои действия для VoiceOver.
- UIContentSizeCategory — текущий размер Dynamic Type.
- UIFontMetrics — масштабирование своих шрифтов под Dynamic Type.
- UIFont.preferredFont(forTextStyle:) — системный шрифт под Dynamic Type.
- XCUIApplication.performAccessibilityAudit — автоматический аудит в UI-тестах, iOS 17+.
- HIG — Accessibility — гайдлайн Apple по доступности.

→ Глава 35. Cookbook — темы и цвета

Глава 35. Cookbook — темы и цвета

Светлая и тёмная тема, системные «смысловые» цвета, своя палитра бренда и выбор темы в настройках приложения.

В основе всего лежит **trait collection** (`UITraitCollection`) — набор характеристик окружения, в котором сейчас показывается view: тема (светлая или тёмная), размер шрифта Dynamic Type, повышенный контраст, **size class** (грубая оценка ширины и высоты: «компактная» на iPhone в портрете, «обычная» на iPad) и другие. Каждый view и контроллер знает свою trait collection (`traitCollection`) и получает её от родителя: окно → контроллер → его view → дочерние view. Когда пользователь переключает тему, по всему этому дереву проходит изменение, и адаптивные цвета перерисовываются.

`UITraitCollection` — обычный объект (→ `NSObject`), неизменяемый снимок характеристик. Свойство `traitCollection` есть у всех, кто реализует протокол `UITraitEnvironment`: `UIScreen`, `UIWindow`, `UIViewController`, `UIView`, `UIPresentationController` (проверено `conforms(to:)` на симуляторе; глава 0, разделы 0.8 и 0.9). Читать его имеет смысл, когда объект уже в иерархии: у свободного `UIView` или только что созданного контроллера size class ещё `unspecified` (замер). Поэтому цвета и раскладку, зависящие от трейтов, обновляют не в `init`/`viewDidLoad`, а там, где UIKit сам сообщает об изменениях: `registerForTraitChanges` (iOS 17) или `traitCollectionDidChange` (до iOS 17), `layoutSubviews`, в контроллере — `viewIsAppearing` (раздел 35.6).

Аналогия: trait collection — это «погода» в комнате, где висит view. Цвет, подобранный под погоду, как хамелеон: сам меняется, когда «погода» сменилась.

35.1 Системные цвета (адаптивные)

```
view.backgroundColor = .systemBackground       // белый в светлой / чёрный в тёмной
label.textColor = .label                       // чёрный / белый
label.textColor = .secondaryLabel              // серый, приглушённый
label.textColor = .tertiaryLabel               // ещё бледнее

// Фоны
view.backgroundColor = .secondarySystemBackground
view.backgroundColor = .tertiarySystemBackground
view.backgroundColor = .systemGroupedBackground     // для таблиц в стиле grouped

// Заливки (фон полей, кнопок, прогресс-баров)
```

```
view.backgroundColor = .systemFill
view.backgroundColor = .secondarySystemFill
view.backgroundColor = .tertiarySystemFill
view.backgroundColor = .quaternarySystemFill

// Разделители
view.backgroundColor = .separator
view.backgroundColor = .opaqueSeparator
```

Эти цвета называют **семантическими** (смысловыми): имя говорит не «какой цвет», а «для чего» — «основной текст», «фон второго уровня». Конкретное значение подставит система по trait collection. `.label` в светлой теме — чёрный, в тёмной — белый; при повышенном контрасте — ещё контрастнее.

Отсюда правило: для фонов и текста интерфейса бери семантические цвета, а не `.white` и `.black`. Белый фон, заданный жёстко, в тёмной теме останется белым — с белым же текстом `.label` на нём. `.white` уместен там, где цвет не должен меняться: белый текст на красной кнопке, белая иконка поверх фото.

Уровни фона. `systemBackground` → `secondarySystemBackground` → `tertiarySystemBackground` — это «слои»: экран, карточка на экране, поле внутри карточки. В светлой теме слои различаются оттенком серого, в тёмной — становятся чуть светлее с каждым уровнем.

Есть и тонкость, о которой узнают случайно: в тёмной теме у модальных листов и поповеров фон **светлее** чёрного. Система поднимает «уровень» интерфейса (`traitCollection.userInterfaceLevel = .elevated`), и тот же `.systemBackground` становится тёмно-серым, чтобы лист отделялся от экрана под ним. Если в листе ты задал фон жёстким чёрным, это отделение пропадёт.

Полный список — в HIG, раздел Color: Apple Human Interface Guidelines — Color.

35.2 Tint color — цвет акцента

```
view.tintColor = .systemIndigo
button.tintColor = .systemRed  // переопределить для одной кнопки
```

`tintColor` — цвет «интерактивности»: системные кнопки, иконки в навбаре, переключатели, выделение в пикере. Правило наследования — из документации `UIView.tintColor`: свойство возвращает первое **не-default** значение в цепочке superview, начиная с самого view; если никто выше не задал — системный цвет (`systemBlue`). Задал окну — подхватит всё приложение; задал одной кнопке — только она. Замер на симуляторе: `parent.tintColor = .systemRed` — у вложенного view и кнопки внутри `tintColor` стал красным; `child.tintColor = .systemGreen` — кнопка внутри `child` взяла зелёный, ближайший. Свой view, который рисует что-то цветом `tintColor`, переопределяет `tintColorDidChange()` — UIKit зовёт его у subview, когда цвет сменился у предка (проверено: сработал у дочернего view после смены цвета родителя):

```
final class TintedRing: UIView {
    override func tintColorDidChange() {
        super.tintColorDidChange()
        layer.borderColor = tintColor.cgColor
```

```
    }
}
```

Есть и `tintAdjustmentMode`: переведи его в `.dimmed` у корневого view — и `tintColor` всего поддерева будет возвращаться приглушённым (так гасят интерфейс под модальным окном); при смене режима `tintColorDidChange()` у subview тоже вызывается — оба правила из заголовка `UIView.h`.

Когда один и тот же вид нужен **всем** экземплярам класса, есть appearance proxy. `UIAppearance` — протокол (глава 0, раздел 0.9), который реализует `UIView` и, например, `UIBarButtonItem` (проверено `conforms(to:)`); `appearance()` возвращает прокси-объект класса, и всё, что ты ему присвоишь, UIKit применит к экземплярам:

```
UISwitch.appearance().onTintColor = .systemIndigo
UILabel.appearance(whenContainedInInstancesOf: [UITableViewHeaderFooterView.self])
    .textColor = .secondaryLabel
```

Вторая форма — «только внутри таких контейнеров». Две особенности, обе из документации и подтверждены на симуляторе: прокси срабатывает, когда view **входит в окно** (у свежего `UISwitch` до `addSubview` в окно `onTintColor` ещё старый, после — новый), и уже показанные view не перекрашивает — их надо убрать из иерархии и вернуть. А `tintColor` через прокси задавать нельзя: заголовок `UIAppearance.h` прямо говорит, что он несовместим с наследованием по дереву, — для него есть окно.

`.systemBlue`, `.systemRed`, `.systemGreen` и остальные тоже адаптивные: в тёмной теме и при повышенном контрасте у каждого свой оттенок. Apple подстраивает их от версии к версии: в iOS 26.5 `systemBlue` в светлой теме — `#0088FF`, в тёмной — `#0091FF` (замер на симуляторе). Поэтому не записывай системные цвета в код числами.

35.3 Свой адаптивный цвет в коде

```
let cardBackground = UIColor { traitCollection in
    if traitCollection.userInterfaceStyle == .dark {
        return UIColor(white: 0.15, alpha: 1.0)
    } else {
        return UIColor(white: 0.98, alpha: 1.0)
    }
}
```

`UIColor(dynamicProvider:)` (iOS 13+) — цвет, который определяется замыканием. UIKit вызывает его каждый раз, когда ему нужно конкретное значение, и передаёт текущую trait collection. `white: 0.15` — серый, в котором 15% «белизны»: почти чёрный для тёмной темы. `0.98` — почти белый для светлой.

Замыкание вызывается часто — держи его быстрым и без побочных эффектов: никаких сетевых запросов и записей в `UserDefaults`.

35.4 Цвета в Asset Catalog

Это рекомендуемый способ для палитры бренда. В `Assets.xcassets`:

1. `+` → Color Set, назови `BrandPrimary`.
2. В инспекторе справа, в Appearances, выбери **Any, Dark** — появятся два квадрата.
3. Задай цвета: Any (светлая) — `#1A73E8`, Dark — `#4285F4`.
4. Если нужен вариант для повышенного контраста — там же включи **High Contrast** (см. главу 34.7).

В коде:

```
let primary = UIColor(named: "BrandPrimary") ?? .systemBlue
view.backgroundColor = primary
```

Один набор цветов сам работает в обеих темах. `UIColor(named:)` возвращает опционал: при опечатке в имени получишь `nil`. `?? .systemBlue` даёт запасной цвет; если хочешь, чтобы опечатку было невозможно пропустить, лучше упасть сразу с понятным сообщением (как в 35.8), чем молча показывать не тот цвет.

35.5 Принудительная тема

Конкретный экран всегда тёмный:

```
override func viewDidLoad() {
    super.viewDidLoad()
    overrideUserInterfaceStyle = .dark  // этот контроллер, его view и дочерние контроллеры
}
```

Так сделан Калькулятор (глава 14.9) — всегда тёмный, независимо от системы. Так же — просмотрщик фото (глава 37).

Всё приложение:

```
// В SceneDelegate
window?.overrideUserInterfaceStyle = .dark
```

Окно — корень иерархии, поэтому тема окна достаётся всем экранам, включая модальные. Если приложение **вообще** не поддерживает тёмную тему, можно вместо кода добавить в Info.plist ключ `UIUserInterfaceStyle` со значением `Light` (в Xcode он называется «Appearance»). Но сделать тёмную тему обычно дешевле, чем объяснять пользователям, почему её нет.

35.6 Реакция на смену темы

Семантические и `dynamicProvider`-цвета, присвоенные свойствам `UIView` (`backgroundColor`, `textColor`, `tintColor`), меняются сами. Не меняются цвета, которые ушли в **`CALayer`**: у слоя цвет — `CGColor`, объект Core Graphics с уже вычисленными компонентами, без привязки к теме. Когда ты пишешь `UIColor.systemBackground.cgColor`, UIKit один раз разрешает адаптивный цвет по `UITraitCollection.current` — trait collection «текущего контекста», которую UIKit выставляет на время `layoutSubviews`, `draw(_:)`, `traitCollectionDidChange` и обработчиков трейтов

(документация `UITraitCollection.current`), — и отдаёт слою готовое значение. Сменилась тема — слой остался со старым цветом.

Типичные жертвы: `layer.borderColor`, `layer.shadowColor`, цвета `CAGradientLayer` и `CAShapeLayer`. Их обновляют вручную при смене темы.

На iOS 17+ — регистрация на изменение нужной характеристики:

```
final class GradientView: UIView {
    private let gradient = CAGradientLayer()

    override init(frame: CGRect) {
        super.init(frame: frame)
        layer.addSublayer(gradient)
        updateColors()
        if #available(iOS 17, *) {
            registerForTraitChanges([UITraitUserInterfaceStyle.self]) { (self: Self, _) in
                self.updateColors()
            }
        }
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    // iOS 15–16: старый способ (в iOS 17 помечен устаревшим)
    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if #available(iOS 17, *) { return }
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateColors()
        }
    }

    override func layoutSubviews() {
        super.layoutSubviews()
        gradient.frame = bounds
    }

    private func updateColors() {
        let tc = traitCollection
        gradient.colors = [
            UIColor.systemBlue.resolvedColor(with: tc).cgColor,
            UIColor.systemPurple.resolvedColor(with: tc).cgColor,
        ]
    }
}
```

Разбор:

- `registerForTraitChanges([UITraitUserInterfaceStyle.self])` — «сообщи, когда поменяется тема»; метод протокола `UITraitChangeObservable`, iOS 17. Замыкание получает сам view как параметр (`self: Self`), поэтому `[weak self]` не нужен: UIKit не создаёт цикла. Снимать регистрацию не нужно — UIKit сам чистит её в конце жизни объекта (документация). В `init` view ещё не в окне, и его trait collection предварительная; когда view попадёт в окно с другой темой, трейт сменится и обработчик вызовется — поэтому стартовый `updateColors()` плюс подписка дают верный цвет в любом случае.
- `traitCollectionDidChange` — способ для iOS 15–16. В iOS 17 он помечен устаревшим (заголовок: `API_DEPRECATED(... ios(8.0, 17.0))`, с отсылкой к `UITraitChangeObservable`), но вызываться не перестал, поэтому внутри мы выходим сразу, если работает новая регистрация, — иначе на iOS 17+ цвета обновлялись бы дважды.
- `hasDifferentColorAppearance(comparedTo:)` — изменилось ли то, что влияет на цвета (тема, контраст, уровень интерфейса). Смена размера шрифта сюда не попадёт — лишней перерисовки не будет (проверено: тёмная против светлой — `true`, размер шрифта L против XL — `false`).
- `resolvedColor(with: tc)` — «какой конкретный цвет у адаптивного цвета для этой trait collection». Надёжнее, чем голый `.cgColor`, который смотрит на `UITraitCollection.current`.

Без этого кода градиент «застрянет» в теме, в которой экран открылся: пользователь переключил тему в Пункте управления — а у тебя на тёмном экране висит светлый градиент.

35.7 Выбор темы в настройках приложения

```
enum AppTheme: Int, CaseIterable {
    case system, light, dark

    var displayName: String {
        switch self {
        case .system: return "Как в системе"
        case .light: return "Светлая"
        case .dark: return "Тёмная"
        }
    }

    var userInterfaceStyle: UIUserInterfaceStyle {
        switch self {
        case .system: return .unspecified
        case .light: return .light
        case .dark: return .dark
        }
    }

    static var saved: AppTheme {
        AppTheme(rawValue: UserDefaults.standard.integer(forKey: "settings.theme")) ??
↪   .system
```

```
    }

    func save() {
        UserDefaults.standard.set(rawValue, forKey: "settings.theme")
    }
}
```

Разбор:

- `Int` в качестве сырого значения — чтобы хранить выбор в `UserDefaults` числом. `system` = 0, `light` = 1, `dark` = 2.
- `.unspecified` — «ничего не навязывать», тема будет как в системе.
- `saved` читает `integer(forKey:)`. Если пользователь ещё ничего не выбирал, ключа нет и метод вернёт 0 — это как раз `.system`. Удачно совпало не случайно: вариант «по умолчанию» поставлен первым.

Применение — при запуске, в `SceneDelegate`, до показа окна:

```
func scene(_ scene: UIScene, willConnectTo session: UISceneSession,
           options connectionOptions: UIScene.ConnectionOptions) {
    guard let windowScene = scene as? UIWindowScene else { return }
    let window = UIWindow(windowScene: windowScene)
    window.overrideUserInterfaceStyle = AppTheme.saved.userInterfaceStyle
    window.rootViewController = RootViewController()
    window.makeKeyAndVisible()
    self.window = window
}
```

И при выборе в настройках — сохранить и применить к окну текущей сцены (`view.window`):

```
private func themeSelected(_ theme: AppTheme) {
    theme.save()
    view.window?.overrideUserInterfaceStyle = theme.userInterfaceStyle
}
```

Меню выбора — `UIMenu` на кнопке или action sheet (см. главу 28.7).

35.8 Палитра бренда

```
enum Palette {
    static let primary = color("BrandPrimary")
    static let secondary = color("BrandSecondary")
    static let accent = color("BrandAccent")
    static let success = color("BrandSuccess")
    static let warning = color("BrandWarning")
    static let danger = color("BrandDanger")

    private static func color(_ name: String) -> UIColor {
```

```
        guard let color = UIColor(named: name) else {
            fatalError("Нет цвета \(name) в Assets.xcassets")
        }
        return color
    }
}

view.backgroundColor = Palette.primary
button.tintColor = Palette.accent
```

Это расширенная версия `Palette` из главы 5 (`Common/DesignSystem.swift`): в проекте playground'а замени ей ту, а не заводи вторую, иначе компилятор скажет, что `Palette` объявлен дважды. Имена `success` и `danger` сохранены, так что код, который ими уже пользуется, не сломается; вместо `tint` из главы 5 здесь `accent`.

Все цвета бренда — в одном месте. Код ссылается на `Palette.primary`, а не на строку `"BrandPrimary"` в сорока местах, — опечатка возможна только внутри `Palette`, и `fatalError` с понятным текстом покажет её при первом же запуске экрана.

Дизайнер сказал «поменяй основной с синего на бирюзовый» — меняешь цвет в color set `BrandPrimary`, код не трогаешь. Приложение при этом **пересобирается**: Asset Catalog компилируется вместе с кодом, и новый цвет попадёт к пользователям только с новой версией в App Store. Менять цвета «на лету», без релиза, можно лишь загружая их с сервера — это отдельная задача.

35.9 Контраст цветов

Требования WCAG (подробно и с числами — в главе 34.7):

- уровень **AA**: 4.5:1 для обычного текста, 3:1 для крупного и для значков;
- уровень **AAA**: 7:1 для обычного текста, 4.5:1 для крупного.

«Крупный» в терминах WCAG — от 18 пунктов обычным начертанием или от 14 жирным (в пикселях веба — 24 и примерно 19). Для iOS это грубо соответствует стилям `.title3` и крупнее.

У Apple в HIG правило мягче: 3:1 хватает и жирному тексту любого размера. Разницу между двумя стандартами разбирает глава 34, раздел 34.7; надёжнее держать мелкий жирный текст на 4.5:1.

Системные цвета **не все** проходят AA. Замер на iOS 26.5: `.secondaryLabel` на белом — 3,4:1, `.systemGreen` на белом — 2,2:1. Со своей палитрой проверяй каждую пару «текст/фон» в обеих темах: Accessibility Inspector (кнопка Audit) или любой калькулятор контраста по HEX-кодам.

Реакция на повышенный контраст — лучше всего через Asset Catalog (вариант High Contrast у color set). Если нужен код:

```
let secondaryText = UIColor { traits in
    traits.accessibilityContrast == .high ? .label : .secondaryLabel
}
label.textColor = secondaryText
```

Цвет — адаптивный, поэтому сам перерисуется, когда пользователь включит «Увеличение контраста»: `accessibilityContrast` — трейт, и UIKit перерешит цвет при его смене. Можно и руками — глобальный флаг `UIAccessibility.isDarkerSystemColorsEnabled` плюс подписка на `darkerSystemColorsStatusDidChangeNotification`, — но это два механизма вместо одного.

35.10 Цвета в Storyboard

Цвета из Asset Catalog доступны в списке цветов Interface Builder — выбираешь `BrandPrimary`, и он работает в обеих темах. Системные семантические цвета там тоже есть (System Background Color, Label Color).

Цвет, заданный в Storyboard числами (`RGB 123, 45, 67`), не адаптивный: в тёмной теме он останется таким же. В этой книге Storyboard используется только для LaunchScreen, но правило то же: на экране запуска — цвета из Asset Catalog, иначе тёмная тема начнётся с белой вспышки.

35.11 Плавная смена темы

```
extension UIWindow {
    func setTheme(_ theme: AppTheme) {
        UIView.transition(with: self, duration: 0.3,
                          options: .transitionCrossDissolve,
                          animations: {
            self.overrideUserInterfaceStyle = theme.userInterfaceStyle
        })
    }
}

// Использование, например, в настройках
view.window?.setTheme(.dark)
```

Без анимации всё приложение перекрашивается мгновенно — резкая вспышка. `transitionCrossDissolve` за 0,3 с растворяет старую картинку окна в новой (подробнее о переходах — в главе 32.9).

35.12 Тема у дочерних и модальных экранов

`overrideUserInterfaceStyle` у контроллера, по документации Apple, действует на сам контроллер, всю его иерархию view и **встроенные** дочерние контроллеры (children — те, что добавлены через `addChild`, например экраны внутри `UINavigationController`); на родителей и приложение — нет (заголовок `UIViewController.h`: «this view controller and its children... does not affect the application or any parent view controllers»). Замер на симуляторе: после `vc.overrideUserInterfaceStyle = .dark` `traitCollection` у `vc`, его `view` и добавленного через `addChild` контроллера стал `.dark`.

Модальный экран (`present`) дочерним не является — он показывается поверх в своём контейнере и берёт тему **окна**. Если ты сделал тёмным один контроллер через override, а из него показываешь лист, лист будет в теме системы. Хочешь, чтобы он тоже был тёмным, — задай

override и ему перед `present`. А тема, заданная окну (35.5), действует на все экраны сразу, включая модальные: заголовок `UIView.h` говорит об этом прямо — у окна свойство действует на `rootViewController` со всей иерархией «и на presentations внутри окна».

Упражнения

Упражнение 35.1. У карточки есть рамка: `card.layer.borderColor = UIColor.separator.cgColor`. Пользователь переключает тему, и рамка становится то слишком яркой, то невидимой. Объясни почему и исправь для iOS 15+.

Упражнение 35.2. Сделай адаптивный цвет «фон ценника»: светлая тема — `#FFF4E5`, тёмная — `#3A2A14`, а при повышенном контрасте в обеих темах — `.systemBackground`. Используй `UIColor(dynamicProvider:)`.

Ответы к упражнениям

35.1. `.cgColor` превращает адаптивный цвет в конкретный в момент присваивания; слой не знает о темах. Обновляем цвет при смене характеристик:

```
final class CardView: UIView {
    override init(frame: CGRect) {
        super.init(frame: frame)
        layer.borderWidth = 1
        updateBorder()
        if #available(iOS 17, *) {
            registerForTraitChanges([UITraitUserInterfaceStyle.self]) { (self: Self, _) in
                self.updateBorder()
            }
        }
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if #available(iOS 17, *) { return }
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateBorder()
        }
    }

    private func updateBorder() {
        layer.borderColor = UIColor.separator.resolvedColor(with: traitCollection).cgColor
    }
```

```
}
```

35.2.

```
extension UIColor {
    convenience init(hex: UInt32) {
        self.init(red: CGFloat((hex >> 16) & 0xFF) / 255,
                  green: CGFloat((hex >> 8) & 0xFF) / 255,
                  blue: CGFloat(hex & 0xFF) / 255,
                  alpha: 1)
    }
}

let priceTagBackground = UIColor { traits in
    if traits.accessibilityContrast == .high {
        return .systemBackground
    }
    return traits.userInterfaceStyle == .dark
        ? UIColor(hex: 0x3A2A14)
        : UIColor(hex: 0xFFF4E5)
}
```

`hex >> 16 & 0xFF` вынимает из числа `0xFFF4E5` первые две шестнадцатеричные цифры — красный канал `FF` = 255; `>> 8` — зелёный `F4` = 244; последние две — синий `E5` = 229. Делим на 255, потому что `UIColor` ждёт доли от 0 до 1. Проверку контраста ставим первой: она важнее темы.

Что мы выучили

- **Trait collection** — характеристики окружения (тема, контраст, размер шрифта, size class), передаются от окна вниз по иерархии; `UITraitCollection → NSObject`, есть у всех `UITraitEnvironment`. Читать после входа в иерархию: `registerForTraitChanges`, `layoutSubviews`, `viewIsAppearing`.
- **Семантические цвета** (`.label`, `.systemBackground`) — для фонов и текста; `.white`/`.black` — только где цвет не должен меняться. В тёмной теме модальные листы «приподняты» и светлее.
- **`tintColor`** — первое не-default значение по цепочке superview, иначе `systemBlue`; свои view реагируют в `tintColorDidChange()`; системные цвета меняются от версии iOS.
- **`UIAppearance`** — протокол; `appearance()` даёт прокси класса, применяется при входе view в окно; `tintColor` через прокси нельзя.
- **`UIColor(dynamicProvider:)`** — свой адаптивный цвет, быстрое замыкание.
- **Asset Catalog** — Any / Dark / High Contrast; `UIColor(named:)` опционален; смена цвета требует новой сборки.
- **`overrideUserInterfaceStyle`** — у контроллера и его children, у окна — для всего; модальные экраны берут тему окна.
- **`CGColor` в слоях не адаптивный** — обновлять через `registerForTraitChanges` (iOS 17+) или `traitCollectionDidChange`.

- **Выбор темы** — `enum` с `.unspecified`, `UserDefaults`, применение в `SceneDelegate` и плавная смена через `UIView.transition`.
- **Контраст** — AA 4.5:1 / 3:1; системные цвета проходят не все.

Apple Developer Documentation

- UITraitCollection — характеристики окружения view.
- UITraitCollection.userInterfaceStyle — текущая тема.
- UIUserInterfaceStyle — `.unspecified` / `.light` / `.dark`.
- UIViewController.overrideUserInterfaceStyle — тема контроллера, его иерархии и дочерних контроллеров.
- UIColor.init(dynamicProvider:) — адаптивный цвет через замыкание, iOS 13+.
- UIColor.resolvedColor(with:) — конкретный цвет для trait collection.
- UITraitChangeObservable.registerForTraitChanges — подписка на изменение характеристик, iOS 17+.
- UITraitEnvironment.traitCollectionDidChange(_:) — старый способ, устарел в iOS 17.
- UITraitCollection.hasDifferentColorAppearance(comparedTo:) — изменилось ли то, что влияет на цвета.
- UITraitCollection.current — trait collection текущего контекста; по ней `.cgColor` разрешает адаптивный цвет.
- UIViewController.viewIsAppearing(_:) — трейты и геометрия уже актуальны.
- UIView.tintColor — правило «первое не-default значение по цепочке superview».
- UIView.tintColorDidChange() — реакция своего view на смену tint color.
- UIAppearance — протокол appearance proxy; применяется при входе view в окно.
- UI element colors — семантические цвета (`.label`, `.systemBackground`, заливки, разделители).
- HIG — Dark Mode — гайдлайн Apple по тёмной теме.
- HIG — Color — системные и семантические цвета.

→ Глава 36. Cookbook — индикаторы статуса

Глава 36. Cookbook — индикаторы статуса

Маленькие элементы, которые сообщают состояние: зелёная точка «в сети», три прыгающие точки «печатает», красный кружок с числом непрочитанных, полоска и кольцо прогресса, галочки доставки.

У всех рецептов главы две общие ловушки, и они проверены на симуляторе:

- **Цвета слоёв не следят за темой.** Рамка, обводка, цвет `CAShapeLayer` — это `CGColor`, и при переключении темы они остаются прежними (почему — в главе 35.6). Поэтому каждый компонент ниже сам обновляет такие цвета.
- **Бесконечные анимации слоя снимаются**, когда view уходит из окна. Замер: пульсирующей точке убрали и вернули родителя — список анимаций слоя стал пустым, точка замерла. Поэтому анимации запускаются в `didMoveToWindow()`, а не один раз в `init`.

И одно правило доступности: **статус нельзя передавать только цветом**. Около 8% мужчин плохо различают красный и зелёный, а VoiceOver цвет вообще не произносит. Цвет + текст, цвет + форма, цвет + `accessibilityLabel`.

36.1 Точка «в сети»

Зелёный кружок в углу аватарки:

```
final class OnlineDotView: UIView {
    var isOnline = false {
        didSet {
            backgroundColor = isOnline ? .systemGreen : .systemGray
        }
    }

    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .systemGray
        layer.cornerRadius = 6
        layer.borderWidth = 2
        updateBorder()
        if #available(iOS 17, *) {
            registerForTraitChanges([UITraitUserInterfaceStyle.self]) { (self: Self, _) in
```

```
                self.updateBorder()
            }
        }
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if #available(iOS 17, *) { return }
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateBorder()
        }
    }

    private func updateBorder() {
        layer.borderColor = UIColor.systemBackground.resolvedColor(with:
↪ traitCollection).cgColor
    }
}
```

Размещение рядом с аватаркой:

```
let dot = OnlineDotView()
dot.translatesAutoresizingMaskIntoConstraints = false
container.addSubview(avatar)
container.addSubview(dot)   // не в avatar: у круглой аватарки clipsToBounds = true
NSLayoutConstraint.activate([
    dot.widthAnchor.constraint(equalToConstant: 12),
    dot.heightAnchor.constraint(equalToConstant: 12),
    dot.trailingAnchor.constraint(equalTo: avatar.trailingAnchor),
    dot.bottomAnchor.constraint(equalTo: avatar.bottomAnchor),
])
dot.isOnline = true
avatar.accessibilityLabel = "Айгерим, в сети"
```

Разбор:

- `cornerRadius = 6` при размере 12 × 12 — половина стороны, поэтому квадрат становится кругом.
- **Рамка цвета фона** (2 точки, `systemBackground`) — «вырезает» точку из аватарки, иначе зелёное сливается с фото. В светлой теме рамка белая, в тёмной — чёрная; обновляется при смене темы через `updateBorder()`. Если задать цвет рамки один раз через `.cgColor`, после переключения на тёмную тему вокруг точки останется белое кольцо.

- **Точка добавлена в общий контейнер, а не в аватарку.** Круглую аватарку обычно делают через `cornerRadius` + `clipsToBounds = true`. Всё, что внутри неё, обрезается по кругу, — точка в правом нижнем углу окажется за пределами круга и пропадёт.
- `translatesAutoresizingMaskIntoConstraints = false` — без этой строки UIKit создаст для view свои констрейнты из `frame`, и они будут конфликтовать с нашими.
- Статус продублирован в `accessibilityLabel` аватарки — для VoiceOver (глава 34).

Цвета статусов, к которым привыкли пользователи мессенджеров: `.systemGreen` — в сети, `.systemYellow` или `.systemOrange` — отошёл, `.systemGray` — не в сети, `.systemRed` — «не беспокоить».

36.2 «Печатает...» — три прыгающие точки

```
final class TypingIndicatorView: UIView {
    private let dots: [UIView] = (0..<3).map { _ in UIView() }

    override init(frame: CGRect) {
        super.init(frame: frame)
        let stack = UIStackView(arrangedSubviews: dots)
        stack.spacing = 4
        stack.translatesAutoresizingMaskIntoConstraints = false
        addSubview(stack)
        for dot in dots {
            dot.backgroundColor = .secondaryLabel
            dot.layer.cornerRadius = 4
            dot.widthAnchor.constraint(equalToConstant: 8).isActive = true
            dot.heightAnchor.constraint(equalToConstant: 8).isActive = true
        }
        NSLayoutConstraint.activate([
            stack.centerXAnchor.constraint(equalTo: centerXAnchor),
            stack.centerYAnchor.constraint(equalTo: centerYAnchor),
        ])
        isAccessibilityElement = true
        accessibilityLabel = "Собеседник печатает"
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func didMoveToWindow() {
        super.didMoveToWindow()
        if window != nil { startAnimating() }
    }

    private func startAnimating() {
        for (i, dot) in dots.enumerated() {
            let animation = CABasicAnimation(keyPath: "transform.translation.y")
```

```
            animation.fromValue = 0
            animation.toValue = -5
            animation.duration = 0.5
            animation.autoreverses = true
            animation.repeatCount = .infinity
            animation.beginTime = CACurrentMediaTime() + Double(i) * 0.15
            dot.layer.add(animation, forKey: "bounce")
        }
    }
}
```

Разбор:

- Каждая точка поднимается на 5 точек вверх (`-5`: ось y в iOS смотрит вниз, поэтому «вверх» — минус) за 0,5 с и за 0,5 с возвращается: полный прыжок — 1 секунда.
- `beginTime = CACurrentMediaTime() + i × 0.15` — вторая точка стартует на 0,15 с позже первой, третья — на 0,3 с. Получается «бегущая волна». `CACurrentMediaTime()` нужен потому, что `beginTime` задаётся во времени Core Animation, а не как задержка «от сейчас».
- `didMoveToWindow()` вызывается, когда view попадает в окно или уходит из него. Мы запускаем анимацию каждый раз, когда view снова в окне: ячейку таблицы переиспользовали, экран закрыли и открыли — точки продолжают прыгать. `add(_:forKey:)` с тем же ключом заменяет старую анимацию, дублей не будет.
- Для VoiceOver весь индикатор — один элемент с понятным текстом.

Полная версия в «пузыре» чата — в главе 18.5.

36.3 Бейдж с числом

На вкладке таб-бара бейдж встроенный:

```
// внутри экрана, который сидит на вкладке
tabBarItem.badgeValue = "3"
tabBarItem.badgeColor = .systemRed
```

`tabBarItem` — свойство **view controller'а**, а не панели: класс `UITabBarItem` наследует `UIBarItem` → `NSObject`, это не view, а описание вкладки (глава 0, разделы 0.6 и 0.8). Панель `UITabBar` лишь показывает эти объекты: `tabBarController?.tabBar.items?[0]` — тот же самый экземпляр, что `viewControllers[0].tabBarItem` (проверено на симуляторе: `===` даёт `true`), поэтому через него тоже работает, но менять массив `items` у панели, которой управляет `UITabBarController`, нельзя — Apple пишет, что это вызовет исключение. `badgeValue = nil` — убрать бейдж. Строка может быть любой, но длинные тексты система обрежет — для счётчика ограничь сам: «99+».

Свой бейдж на любом view:

```
final class BadgeLabel: UILabel {
    override init(frame: CGRect) {
        super.init(frame: frame)
        font = .systemFont(ofSize: 11, weight: .bold)
        textColor = .white
```

```
        backgroundColor = .systemRed
        textAlignment = .center
        layer.cornerRadius = 9
        layer.masksToBounds = true
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    // Поля по бокам: "99+" не должен касаться краёв
    override var intrinsicContentSize: CGSize {
        let size = super.intrinsicContentSize
        return CGSize(width: max(size.width + 10, 18), height: 18)
    }

    func setCount(_ count: Int) {
        isHidden = count == 0
        text = count > 99 ? "99+" : "\(count)"
        accessibilityLabel = "\(count) непрочитанных"
    }
}
```

И размещение в правом верхнем углу иконки:

```
let badge = BadgeLabel()
badge.translatesAutoresizingMaskIntoConstraints = false
iconView.addSubview(badge)
NSLayoutConstraint.activate([
    badge.centerXAnchor.constraint(equalTo: iconView.trailingAnchor, constant: -2),
    badge.centerYAnchor.constraint(equalTo: iconView.topAnchor, constant: 2),
])
badge.setCount(128)   // покажет "99+"
```

Разбор:

- Высота 18, `cornerRadius = 9` — половина высоты: у однозначного числа получается круг, у многозначного — «таблетка» со скруглёнными торцами.
- `intrinsicContentSize` — «естественный» размер view, который оно само сообщает Auto Layout. Для `UILabel` это размер текста: на симуляторе «99+» жирным 11 pt занимает 22,7 × 13,3 точки, а «7» — 7 × 13,3. Auto Layout превращает этот размер в необязательные констрейнты (hugging с приоритетом 250 и compression resistance с 750), поэтому явный констрейнт на ширину или высоту его перекроет (подробно — глава 0, раздел 0.3). Мы добавляем 10 точек (по 5 с каждой стороны) и не даём стать уже 18 — иначе у «7» бейдж был бы не кругом, а тонким овалом. Без полей «99+» упирается в края.
- `masksToBounds = true` — обязателен именно для `UILabel`. `cornerRadius` скругляет только то, что рисует сам слой: его `backgroundColor` и `border`. У обычного `UIView` `backgroundColor` и есть `layer.backgroundColor`, поэтому там хватает одного `cornerRadius`. А `UILabel` рисует фон вместе с текстом в содержимое своего слоя (`_UILabelLayer`): после

`label.backgroundColor = .systemRed` свойство `label.layer.backgroundColor` остаётся `nil` (проверено на симуляторе). Содержимое слоя `cornerRadius` не трогает, его обрезает только маска — `masksToBounds`, то же самое, что `clipsToBounds` у view (глава 0, раздел 0.3).

- Центр бейджа ставим **на угол** иконки, со сдвигом 2 точки внутрь: половина бейджа выходит за границы иконки. Поэтому у `iconView` не должно быть `clipsToBounds = true`, иначе вылезающая половина обрежется (та же ловушка, что с точкой в 36.1).
- `isHidden = count == 0` — бейдж «0» никто не показывает.

Контраст. Белый текст на `.systemRed` — 3,6:1 (замер на iOS 26.5), ниже нормы 4.5:1 для мелкого текста. Так выглядят бейджи и в самой iOS, пользователи к ним привыкли, но для важных чисел добавь `accessibilityLabel`, а не надейся на цвет.

36.4 Полоса прогресса

```
let progress = UIProgressView(progressViewStyle: .default)
progress.progress = 0.42  // 42%
progress.trackTintColor = .systemFill
progress.progressTintColor = .systemBlue
```

`progress` — число от 0 до 1: 0,42 — заполнено 42% полосы. Если у тебя «скачано 3 МБ из 12», это 3 / 12 = 0,25. `trackTintColor` — цвет пустой части, `progressTintColor` — заполненной.

`UIProgressView` — наследник `UIView`, а не `UIControl`: он только показывает значение, пользователь на него не нажимает, и `addAction` у него нет (кто контрол, а кто нет — глава 0, раздел 0.4). Свой размер он сообщает через `intrinsicContentSize`: высота 4 точки, ширина — `UIView.noIntrinsicMetric` (на iOS 26.5), так что ширину задай констрейнтом, а высоту можно не трогать.

Стили: `.default` — обычная полоса, `.bar` — в заголовке SDK помечен «for use in a toolbar»: вариант для тулбара и панели навигации, отличается только оформлением дорожки.

Плавное изменение:

```
progress.setProgress(0.75, animated: true)
```

`animated: true` — полоса доедет от 0,42 до 0,75 плавно. VoiceOver читает стандартный `UIProgressView` как индикатор прогресса; для своего рисования (36.5) `accessibilityValue` придётся задавать самому.

36.5 Круговой прогресс

```
final class CircularProgressView: UIView {
    private let trackLayer = CAShapeLayer()
    private let progressLayer = CAShapeLayer()

    var progress: CGFloat = 0 {
        didSet {
            progressLayer.strokeEnd = min(max(progress, 0), 1)
```

```
            accessibilityValue = "\(Int((progress * 100).rounded())) процентов"
        }
    }

    override init(frame: CGRect) {
        super.init(frame: frame)
        layer.addSublayer(trackLayer)
        layer.addSublayer(progressLayer)
        for shape in [trackLayer, progressLayer] {
            shape.lineWidth = 8
            shape.fillColor = UIColor.clear.cgColor
        }
        progressLayer.lineCap = .round
        progressLayer.strokeEnd = 0
        updateColors()
        isAccessibilityElement = true
        accessibilityLabel = "Прогресс"
        if #available(iOS 17, *) {
            registerForTraitChanges([UITraitUserInterfaceStyle.self]) { (self: Self, _) in
                self.updateColors()
            }
        }
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func traitCollectionDidChange(_ previousTraitCollection: UITraitCollection?)
    ↪ {
        super.traitCollectionDidChange(previousTraitCollection)
        if #available(iOS 17, *) { return }
        if traitCollection.hasDifferentColorAppearance(comparedTo:
        ↪ previousTraitCollection) {
            updateColors()
        }
    }

    override func layoutSubviews() {
        super.layoutSubviews()
        let center = CGPoint(x: bounds.midX, y: bounds.midY)
        let radius = min(bounds.width, bounds.height) / 2 - 4
        let path = UIBezierPath(arcCenter: center, radius: radius,
                                startAngle: -.pi / 2,
                                endAngle: 1.5 * .pi,
                                clockwise: true)
        trackLayer.path = path.cgPath
        progressLayer.path = path.cgPath
```

```
    }

    private func updateColors() {
        trackLayer.strokeColor = UIColor.tertiarySystemFill.resolvedColor(with:
↪   traitCollection).cgColor
        progressLayer.strokeColor = UIColor.systemBlue.resolvedColor(with:
↪   traitCollection).cgColor
    }
}
```

Идея: два **`CAShapeLayer`** — слоя, которые рисуют заданную линию. Нижний (дорожка) — полная серая окружность, верхний (прогресс) — та же окружность синим, но нарисованная не целиком. Сколько нарисовать, решает `strokeEnd`: 0 — ничего, 0,5 — половина, 1 — вся линия. Значения по умолчанию из `CAShapeLayer.h`: `fillColor` — **непрозрачный чёрный**, `strokeColor` — `nil`, `lineWidth` — 1, `strokeEnd` — 1. Поэтому `fillColor = UIColor.clear.cgColor` в цикле обязателен — иначе вместо кольца получится чёрный диск.

`CAShapeLayer → CALayer → NSObject` — это не view: слой не участвует в Auto Layout, не получает касаний и не знает про `traitCollection` (глава 0, раздел 0.8). Отсюда две особенности класса выше: геометрию слоёв мы считаем сами в `layoutSubviews()`, а цвета обновляем по трейтам view, которое ими владеет.

Геометрия словами:

- Радиус — половина меньшей стороны минус 4. Толщина линии 8, и линия рисуется **по центру** пути: 4 точки внутрь, 4 наружу. Без `- 4` внешняя половина линии вылезала бы за границы view.
- Углы в радианах (π = 180°, см. главу 32.12). Ноль радиан — это «3 часа» на циферблате, а углы в UIKit растут **по часовой стрелке** (ось y смотрит вниз). Поэтому `−π/2` — это «12 часов», верх круга. Конечный угол `1.5π = −π/2 + 2π` — снова 12 часов, но уже после полного оборота. Прогресс 0,25 заполнит дугу от 12 до 3 часов.
- `lineCap = .round` — скруглённые концы дуги.

Путь пересчитывается в `layoutSubviews()` — UIKit вызывает его при каждом изменении `bounds` view (в том числе при повороте экрана), при добавлении subview и после `setNeedsLayout()` — в следующем проходе раскладки, а не сразу (глава 0, раздел 0.3). А смена `backgroundColor`, например, раскладку не трогает: на симуляторе `layoutSubviews` после неё не вызвался. Цвета поэтому **не** в `layoutSubviews`: это метод про геометрию, и UIKit не обещает вызывать его при смене темы. Для них отдельный `updateColors()` с подпиской на изменение трейтов.

Бесплатный бонус: `progressLayer` — самостоятельный слой, не слой какого-то view. У таких слоёв смена свойства анимируется автоматически (так называемая **неявная анимация**; длительность берётся из `CATransaction.animationDuration()`, по умолчанию 0,25 с). У `view.layer` этого нет: UIKit отключает неявные анимации собственного слоя view, их включает только `UIView.animate`. Поэтому `progress = 0.7` плавно «докрутит» кольцо без единой строки анимационного кода. Если нужно мгновенно — отключи действия внутри своей транзакции:

```
CATransaction.begin()
CATransaction.setDisableActions(true)
progressLayer.strokeEnd = 0.7
```

```
CATransaction.commit()
```

Без `begin/commit` вызов `setDisableActions(true)` действует на неявную транзакцию до конца текущего прохода run loop — и отключит анимацию у всех слоёв, которые ты в этот момент меняешь.

36.6 Состояния экрана — один enum

```
final class ContentStateView: UIView {
    enum State { case loading, loaded, error, empty }

    private let spinner = UIActivityIndicatorView(style: .large)
    private let emptyView = UILabel()
    private let errorView = UILabel()
    let contentView = UIView()

    var state: State = .loading {
        didSet { update() }
    }

    private func update() {
        if state == .loading { spinner.startAnimating() } else { spinner.stopAnimating() }
        emptyView.isHidden = state != .empty
        errorView.isHidden = state != .error
        contentView.isHidden = state != .loaded
    }
}
```

Спиннеру отдельная строка с `isHidden` не нужна: у `UIActivityIndicatorView` свойство `hidesWhenStopped` по умолчанию `true` (заголовок SDK), поэтому `stopAnimating()` сам ставит `isHidden = true`, а `startAnimating()` — снимает. На симуляторе свежий спиннер до `startAnimating()` уже скрыт. Как и `UIProgressView`, это `UIView`, а не `UIControl` — он ничего не «нажимает», только крутится.

State machine (конечный автомат) — объект, который в каждый момент находится ровно в одном из заранее перечисленных состояний. Здесь их четыре, и `update()` из одного места решает, что видно. Нельзя случайно показать одновременно спиннер и ошибку: такое сочетание просто не выразить через `enum`. Добавил новое состояние — компилятор заставит учесть его в каждом `switch`.

Какое состояние когда показывать (скелетон, спиннер или пустой экран) — подробно в главе 24.5.

36.7 Бегущее число

```
private func animateCount(from: Int, to: Int, label: UILabel) {
```

```
    let duration = 0.6
    let steps = 30
    for step in 0...steps {
        let progress = Double(step) / Double(steps)
        let value = Int(Double(from) + Double(to - from) * progress)
        DispatchQueue.main.asyncAfter(deadline: .now() + duration * progress) {
            label.text = "\(value)"
        }
    }
}
```

Быстрый вариант: 31 отложенная задача за 0,6 с, по одной каждые 0,02 с. На шаге 15 из 30 прошла половина времени (`progress` = 0,5), и счётчик от 0 до 200 показывает 100.

У такого подхода есть минусы (задачи нельзя отменить, повторный вызов «перемешивает» числа). Вариант на `CADisplayLink`, без этих проблем, — в главе 31.7.

36.8 Цвета и иконки статусов

```
enum Status {
    case success, warning, error, info, neutral

    var color: UIColor {
        switch self {
        case .success: return .systemGreen
        case .warning: return .systemOrange
        case .error: return .systemRed
        case .info: return .systemBlue
        case .neutral: return .systemGray
        }
    }

    var icon: String {
        switch self {
        case .success: return "checkmark.circle.fill"
        case .warning: return "exclamationmark.triangle.fill"
        case .error: return "xmark.circle.fill"
        case .info: return "info.circle.fill"
        case .neutral: return "circle.fill"
        }
    }

    var spokenName: String {
        switch self {
        case .success: return "Успешно"
        case .warning: return "Внимание"
        case .error: return "Ошибка"
```

```
        case .info: return "Информация"
        case .neutral: return "Без статуса"
        }
    }
}

let status = Status.warning
let iconView = UIImageView(image: UIImage(systemName: status.icon))
iconView.tintColor = status.color
iconView.isAccessibilityElement = true
iconView.accessibilityLabel = status.spokenName
```

Один `enum` на все индикаторы приложения: цвет, иконка SF Symbols и слово для VoiceOver рядом. Иконки специально разной **формы** — галочка, треугольник, крестик, «i», — чтобы статус различался и без цвета. Цвет иконки задаёт `tintColor`: SF Symbols по умолчанию рисуются как шаблон и красятся в цвет акцента.

36.9 «Подпрыгивание» бейджа при смене числа

```
private func updateBadge(_ count: Int) {
    badge.setCount(count)
    UIView.animate(withDuration: 0.15, animations: {
        self.badge.transform = CGAffineTransform(scaleX: 1.3, y: 1.3)
    }) { _ in
        UIView.animate(withDuration: 0.15) {
            self.badge.transform = .identity
        }
    }
}
```

За 0,15 с бейдж вырастает на 30% (`1.3` — 130% размера: бейдж 18 точек становится 23,4), ещё за 0,15 с возвращается. Итого 0,3 с — достаточно, чтобы глаз заметил «число изменилось», и не мешает. `transform` не трогает Auto Layout: Apple пишет, что с iOS 8 раскладка считает alignment rect по нетрансформированному view, констрейнты не пересчитываются. Масштаб применяется относительно `center` (точнее, `layer.anchorPoint`, по умолчанию это центр), поэтому бейдж растёт вокруг своей середины. На симуляторе у бейджа 18 × 18 в точке (100, 100) после масштаба 1,3 `bounds` и `center` не изменились, а `frame` стал (97,3; 97,3; 23,4 × 23,4) — и при `transform` ≠ identity на `frame` полагаться нельзя, Apple называет его «undefined» (глава 0, раздел 0.3).

36.10 Индикатор страниц

```
let pageControl = UIPageControl()
pageControl.numberOfPages = 4
pageControl.currentPage = 0
pageControl.currentPageIndicatorTintColor = .systemBlue
```

```
pageControl.pageIndicatorTintColor = .quaternaryLabel
pageControl.addAction(UIAction { [weak pageControl] _ in
    guard let page = pageControl?.currentPage else { return }
    print("Пользователь выбрал страницу \(page)")
}, for: .valueChanged)
```

`UIPageControl` — ряд точек «страница N из M» под листаемым контентом. В отличие от `UIProgressView` и спиннера это **контрол** (`UIPageControl → UIControl → UIView`, глава 0, раздел 0.4): тап по точкам меняет `currentPage` и посылает событие `.valueChanged`, которое ловят через target-action или `addAction(_:for:)` (iOS 14+). Протягивание по точкам — **continuous interaction** (iOS 14+): если зажать палец и вести в сторону, страницы перелистываются одна за другой, как ползунком. Это поведение включено по умолчанию (`allowsContinuousInteraction` в заголовке SDK — «Default is YES», на симуляторе читается `true`); выключить его можно, поставив `false`.

Как связать точки с `UIPageViewController` — в главе 6.5.

36.11 Галочки доставки

```
enum MessageStatus {
    case sending, sent, delivered, read

    var glyph: String? {
        switch self {
        case .sending: return nil          // вместо текста — иконка часов
        case .sent: return "✓"
        case .delivered, .read: return "✓✓"
        }
    }

    var symbolName: String? {
        self == .sending ? "clock" : nil
    }

    var color: UIColor {
        self == .read ? .systemBlue : .secondaryLabel
    }

    var spokenName: String {
        switch self {
        case .sending: return "отправляется"
        case .sent: return "отправлено"
        case .delivered: return "доставлено"
        case .read: return "прочитано"
        }
    }
}
```

Одна галочка — ушло на сервер, две — доставлено собеседнику, две синие — прочитано. «Доставлено» и «прочитано» отличаются **только цветом**, поэтому `spokenName` для VoiceOver здесь обязателен. «Отправляется» показываем SF Symbol `clock`, а не эмодзи песочных часов: эмодзи в интерфейсе выглядят по-разному на разных версиях системы и плохо красятся в цвет текста.

Живой пример с оптимистичной отправкой — в главе 18.

36.12 Качество связи

```
enum ConnectionQuality {
    case excellent, good, fair, poor, offline

    var color: UIColor {
        switch self {
        case .excellent, .good: return .systemGreen
        case .fair: return .systemYellow
        case .poor: return .systemOrange
        case .offline: return .systemRed
        }
    }

    var bars: Int {
        switch self {
        case .excellent: return 4
        case .good: return 3
        case .fair: return 2
        case .poor: return 1
        case .offline: return 0
        }
    }
}
```

Индикатор в видеозвонках: четыре полоски разной высоты, из них закрашено `bars` штук цветом `color`, остальные — серые. «Отлично» — 4 зелёных, «плохо» — 1 оранжевая и 3 серые. Здесь снова работает правило «не только цветом»: число закрашенных полосок читается и без него.

Упражнения

Упражнение 36.1. В `CircularProgressView` прогресс рисуется от «12 часов» по часовой стрелке. Как изменить параметры дуги, чтобы прогресс начинался снизу, с «6 часов»? Чему равны `startAngle` и `endAngle`?

Упражнение 36.2. Сделай бейдж, который показывает «99+» для чисел больше 99, прячется при 0 и для VoiceOver говорит «Нет непрочитанных» / «1 непрочитанное» / «5 непрочитанных». Используй функцию `plural` из главы 31.10.

Ответы к упражнениям

36.1. «6 часов» — это четверть оборота **после** «3 часов» по часовой стрелке, то есть угол +π/2. Полный круг — ещё 2π:

```
let path = UIBezierPath(arcCenter: center, radius: radius,
                        startAngle: .pi / 2,
                        endAngle: .pi / 2 + 2 * .pi,
                        clockwise: true)
```

`endAngle` = π/2 + 2π = 2,5π. Прогресс 0,25 теперь заполнит дугу от 6 до 9 часов.

36.2.

```
func plural(_ n: Int, forms: (one: String, few: String, many: String)) -> String {
    let n = abs(n)
    let mod100 = n % 100
    let mod10 = n % 10
    if mod100 >= 11 && mod100 <= 14 { return forms.many }
    switch mod10 {
    case 1: return forms.one
    case 2, 3, 4: return forms.few
    default: return forms.many
    }
}

final class UnreadBadge: UILabel {
    func setCount(_ count: Int) {
        isHidden = count == 0
        text = count > 99 ? "99+" : "\(count)"
        accessibilityLabel = count == 0
            ? "Нет непрочитанных"
            : "\(count) \(plural(count, forms: ("непрочитанное", "непрочитанных",
            ↪  "непрочитанных")))"
    }
}
```

Для 1 и 21 — «непрочитанное», для 2–4 и остальных — «непрочитанных» (в этом слове формы few и many совпадают). Обрати внимание: для VoiceOver мы говорим настоящее число, даже если на экране «99+».

Что мы выучили

- **Цвета слоёв** (`borderColor`, `strokeColor`) обновляем при смене темы; **бесконечные анимации** перезапускаем в `didMoveToWindow()`.
- **Статус не только цветом**: форма, текст, `accessibilityLabel`.
- **Точка «в сети»** — круг с рамкой цвета фона, в общем контейнере, а не внутри круглой аватарки.
- **«Печатает»** — три точки с `beginTime` со сдвигом 0,15 с.

- **Бейдж вкладки** — `tabBarItem` view controller'а (`UITabBarItem` — не view); **свой бейдж** — `UILabel` с полями через `intrinsicContentSize`, «99+», скрыт при 0, `masksToBounds` обязателен (фон лейбла — в содержимом слоя); у родителя не должно быть `clipsToBounds`.
- **`UIProgressView`** и **`UIActivityIndicatorView`** — view, не контролы; доля от 0 до 1, `setProgress(_:animated:)`; `hidesWhenStopped` по умолчанию `true`.
- **Круговой прогресс** — два `CAShapeLayer`, `strokeEnd`, `fillColor` обнулить (по умолчанию чёрный), старт в `-π/2`, радиус минус половина толщины линии, неявная анимация; выключать её — `CATransaction.begin()` / `setDisableActions(true)` / `commit()`.
- **Состояния экрана** — один `enum` и один `update()`.
- **Бегущее число** — быстрый вариант здесь, правильный — в 31.7.
- **Статусы, галочки, качество связи** — `enum` с цветом, иконкой и словом для VoiceOver.
- **`UIPageControl`** — контрол: `.valueChanged` и `addAction`; continuous interaction (iOS 14+) включён по умолчанию.
- **`transform`** не трогает Auto Layout и `center`/`bounds`, но делает `frame` неопределённым.

Apple Developer Documentation

- UIProgressView — линейный прогресс с `progress` / `setProgress(_:animated:)`.
- UIViewController.tabBarItem — вкладка принадлежит view controller'у, панель её только показывает.
- UIView.intrinsicContentSize — «естественный» размер view для Auto Layout.
- UIView.transform — масштаб вокруг центра; не влияет на Auto Layout с iOS 8.
- UIProgressView.Style — `.default` и `.bar`.
- UIPageControl — индикатор страниц и `allowsContinuousInteraction` (iOS 14+).
- UIActivityIndicatorView — стандартный спиннер `.medium` / `.large`.
- UITabBarItem.badgeValue — текст бейджа на вкладке.
- UITabBarItem.badgeColor — цвет бейджа.
- CAShapeLayer — `strokeEnd` для кругового прогресса.
- UIBezierPath — дуга для кругового индикатора.
- CABasicAnimation — бесконечная анимация точек «печатает».
- UIView.didMoveToWindow() — момент, когда view попал в окно или покинул его.

→ Глава 37. Cookbook — photo viewer

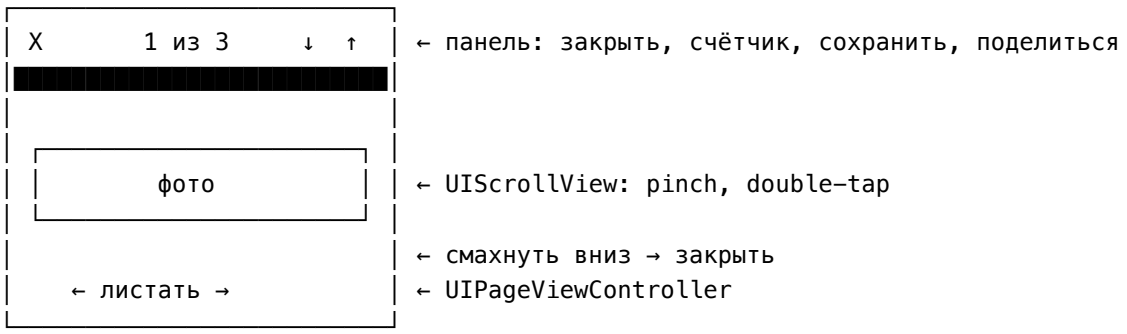
Глава 37. Cookbook — photo viewer

Photo viewer (просмотрщик фото) — полноэкранный экран, куда попадаешь по тапу на миниатюру: фото на чёрном фоне, приближение двумя пальцами, двойной тап, листание соседних фото, «смахни вниз, чтобы закрыть», кнопки «Поделиться» и «Сохранить».

В главе 16 (Галерея) у нас уже был простой просмотр одного фото с зумом. Здесь собираем полную версию из трёх классов:

- `PhotoViewerViewController` — одна страница: одно фото, зум, жесты;
- `PhotoPagerViewController` — листание страниц, панель с кнопками, счётчик «1 из 3»;
- `PhotoZoomTransition` — «вырастание» миниатюры в полноэкранное фото.

Весь код главы собран в настоящий проект (минимум iOS 15, Swift 6, изоляция `MainActor` по умолчанию) и запущен на симуляторе iPhone 16 с iOS 26.5; числа в разборах — из этого прогона. Загрузку картинок делает `ImageCache` из главы 16.3 — в этом проекте лежит его копия.



37.1 Одна страница: фото с зумом

Основа — **`UIScrollView`**. Это не только «прокручиваемая область»: у него встроено масштабирование. Представь лист бумаги (`imageView`) под лупой (`scrollView`): scroll view может увеличивать лист и двигать его под лупой.

Как устроен зум внутри. `UIScrollView → UIView` (глава 0, раздел 0.5) масштабирует не себя, а **один** свой subview — тот, что делегат вернёт из `viewForZooming(in:)`. Scroll view ставит этому view `transform` с нужным масштабом и сам пересчитывает `contentSize`. На симуляторе iOS 26.5: `imageView` 393 × 262 при `setZoomScale(2.5)` получает `transform` = scale 2,5, его `bounds` остаются 393 × 262, а `frame` и `contentSize` scroll view становятся 982,5 × 655. Отсюда два правила этой главы: `imageView.transform` принадлежит scroll view (37.3), а `contentSize` после зума читаем, не задаём

(37.1, `centerImage`). Зум включается, только когда `maximumZoomScale` больше `minimumZoomScale` — по умолчанию оба равны 1,0 (заголовок `UIScrollView.h`).

```
import UIKit
import Photos

final class PhotoViewerViewController: UIViewController {
    let imageURL: URL
    let scrollView = UIScrollView()
    let imageView = UIImageView()
    private let spinner = UIActivityIndicatorView(style: .large)
    private let doubleTap = UITapGestureRecognizer()
    private let pan = UIPanGestureRecognizer()
    var onSingleTap: (() -> Void)?

    init(imageURL: URL) {
        self.imageURL = imageURL
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .black

        scrollView.frame = view.bounds
        scrollView.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        scrollView.contentInsetAdjustmentBehavior = .never
        scrollView.showsHorizontalScrollIndicator = false
        scrollView.showsVerticalScrollIndicator = false
        scrollView.minimumZoomScale = 1.0
        scrollView.maximumZoomScale = 4.0
        scrollView.delegate = self
        view.addSubview(scrollView)

        imageView.contentMode = .scaleAspectFit
        scrollView.addSubview(imageView)

        spinner.color = .white
        spinner.center = CGPoint(x: view.bounds.midX, y: view.bounds.midY)
        spinner.autoresizingMask = [.flexibleLeftMargin, .flexibleRightMargin,
                                    .flexibleTopMargin, .flexibleBottomMargin]
        view.addSubview(spinner)

        setupGestures()
        loadImage()
```

```
    }
```

Разбор:

- `imageURL` не `private`: пейджер (37.6) по нему узнаёт, какое фото на странице.
- `required init?(coder:)` обязателен: как только у контроллера свой `init`, компилятор требует и этот (он нужен для Storyboard); без него код не соберётся.
- `autoresizingMask = [.flexibleWidth, .flexibleHeight]` — scroll view растягивается вместе с экраном (например, при повороте). Для такого простого случая это короче констрейнтов. Это же спасает от ловушки `viewDidLoad`: `view.bounds` здесь ещё не окончательные — view только что создано и не разложено (глава 0, раздел 0.6), — но благодаря маске scroll view подтянется под настоящий размер в первом же проходе раскладки.
- `contentInsetAdjustmentBehavior = .never` — не сдвигать содержимое под **safe area** (зону экрана без «чёлки» и полоски жестов внизу): фото мы центрируем сами.
- `minimumZoomScale = 1.0`, `maximumZoomScale = 4.0` — от «фото целиком» до «в 4 раза крупнее». Строка с минимумом — для наглядности: 1,0 и так значение по умолчанию; а вот без максимума больше 1,0 pinch не заработает.
- Спиннер держится по центру: гибкие отступы со всех сторон.

Загрузка и раскладка:

```
    private func loadImage() {
        spinner.startAnimating()
        Task {
            let image = await ImageCache.shared.image(for: imageURL)
            spinner.stopAnimating()
            imageView.image = image
            layoutImage()
        }
    }

    override func viewDidLayoutSubviews() {
        super.viewDidLayoutSubviews()
        if scrollView.zoomScale == scrollView.minimumZoomScale {
            layoutImage()
        }
    }

    private func layoutImage() {
        guard let size = imageView.image?.size, size.width > 0, size.height > 0 else {
        ↪  return }
        let bounds = scrollView.bounds.size
        let scale = min(bounds.width / size.width, bounds.height / size.height)
        let fitted = CGSize(width: size.width * scale, height: size.height * scale)
        imageView.frame = CGRect(origin: .zero, size: fitted)
        scrollView.contentSize = fitted
        centerImage()
    }
```

```
    private func centerImage() {
        let bounds = scrollView.bounds.size
        let content = scrollView.contentSize
        let dx = max(0, (bounds.width - content.width) / 2)
        let dy = max(0, (bounds.height - content.height) / 2)
        scrollView.contentInset = UIEdgeInsets(top: dy, left: dx, bottom: dy, right: dx)
    }

    override func viewWillTransition(to size: CGSize,
                                     with coordinator:
                                      ↪ UIViewControllerTransitionCoordinator) {
        super.viewWillTransition(to: size, with: coordinator)
        scrollView.setZoomScale(scrollView.minimumZoomScale, animated: false)
    }
}

extension PhotoViewerViewController: UIScrollViewDelegate {
    func viewForZooming(in scrollView: UIScrollView) -> UIView? { imageView }

    func scrollViewDidZoom(_ scrollView: UIScrollView) {
        centerImage()
    }
}
```

`Task { ... }` в контроллере наследует главный актор, поэтому после `await` можно сразу трогать `imageView` — без `MainActor.run`.

`layoutImage()` на числах. Фото 1200 × 800 пикселей, экран iPhone 16 — 393 × 852 точки. Во сколько раз уменьшить, чтобы фото влезло целиком? По ширине 393 / 1200 ≈ 0,33, по высоте 852 / 800 ≈ 1,07. Берём **меньшее** (`min`), иначе фото вылезет по ширине: 0,33. Получается 393 × 262 точки — то, что показал прогон на симуляторе (`imageFrame: 393.0 × 262.0`). Соотношение сторон сохранилось: 1200 / 800 = 1,5 и 393 / 262 = 1,5.

`contentSize` — размер «листа» внутри scroll view. Мы делаем `imageView` ровно по размеру вписанного фото, а не на весь экран. Если `imageView` займёт весь экран с `.scaleAspectFit`, при зуме можно уехать в чёрные поля над и под фото — лупа приблизит вместе с фото пустоту вокруг.

`centerImage()` на числах. Лист 262 точки в высоту, экран 852. Остаётся 852 − 262 = 590 точек пустоты, по 295 сверху и снизу. `contentInset` — отступ вокруг листа внутри scroll view: с ним фото стоит ровно посередине (в прогоне: `inset top: 295, bottom: 295`). При зуме ×2,5 лист становится 262 × 2,5 = 655 точек — это новое `contentSize`, которое scroll view выставил сам, нам его только прочитать. Пустоты остаётся (852 − 655) / 2 = 98,5 — и прогон показал `98.5`. Поэтому центрирование вызывается в `scrollViewDidZoom`: при каждом изменении масштаба. Когда фото больше экрана, `max(0, …)` обнуляет отступ, и фото прокручивается от края до края.

`viewForZooming(in:)` — метод делегата, который отвечает: «какой view увеличивать». Без него pinch-жест ничего не делает: в заголовке SDK так и написано — «if delegate returns nil, nothing happens». **Делегат** — объект, которому scroll view задаёт вопросы и сообщает о событиях; здесь это наш контроллер. Как у всех делегатов UIKit, `scrollView.delegate` — слабая ссылка (`weak` в

`UIScrollView.h`): контроллер владеет scroll view, а не наоборот, и цикла не возникает (глава 0, раздел 0.9).

`viewWillTransition` — экран поворачивается. Сбрасываем зум, и в `viewDidLayoutSubviews` фото впишется заново под новые размеры.

37.2 Двойной тап — приблизить к точке

```
extension PhotoViewerViewController: UIGestureRecognizerDelegate {
    private func setupGestures() {
        doubleTap.addTarget(self, action: #selector(doubleTapped(_:)))
        doubleTap.numberOfTapsRequired = 2
        scrollView.addGestureRecognizer(doubleTap)

        let singleTap = UITapGestureRecognizer(target: self, action:
        ↪  #selector(singleTapped))
        singleTap.require(toFail: doubleTap)
        scrollView.addGestureRecognizer(singleTap)

        pan.addTarget(self, action: #selector(handlePan(_:)))
        pan.delegate = self
        view.addGestureRecognizer(pan)
    }

    @objc private func singleTapped() {
        onSingleTap?()
    }

    @objc private func doubleTapped(_ sender: UITapGestureRecognizer) {
        if scrollView.zoomScale > scrollView.minimumZoomScale {
            scrollView.setZoomScale(scrollView.minimumZoomScale, animated: true)
        } else {
            let point = sender.location(in: imageView)
            let targetScale: CGFloat = 2.5
            let size = CGSize(width: scrollView.bounds.width / targetScale,
                              height: scrollView.bounds.height / targetScale)
            let rect = CGRect(x: point.x - size.width / 2,
                              y: point.y - size.height / 2,
                              width: size.width, height: size.height)
            scrollView.zoom(to: rect, animated: true)
        }
    }
```

Если фото уже приближено — возвращаем к целому. Если нет — приближаем **к точке тапа**: человек дважды тапнул по лицу на фото — он хочет видеть лицо, а не центр кадра.

Как `zoom(to:)` превращается в масштаб. Метод говорит scroll view: «сделай так, чтобы этот прямоугольник занял весь экран». Прямоугольник шириной 393 / 2,5 ≈ 157 точек растягивается на

393 точки — это и есть увеличение в 2,5 раза. Прогон подтвердил: `zoomScale after zoom(to:) = 2.5`. Хочешь сильнее — уменьши прямоугольник: при `targetScale = 3` он станет 131 точку.

`sender.location(in: imageView)` — точка тапа в координатах самой картинки, потому что `zoom(to:)` ждёт прямоугольник в координатах увеличиваемого view.

Одиночный тап (прячет панели, 37.9) ждёт, пока двойной «провалится»: `singleTap.require(toFail: doubleTap)`. Без этого первый же тап двойного нажатия прятал бы панель. Цена — одиночный тап срабатывает с небольшой задержкой: система ждёт, не будет ли второго.

37.3 Смахнуть вниз, чтобы закрыть

```
    func gestureRecognizerShouldBegin(_ gestureRecognizer: UIGestureRecognizer) -> Bool {
        guard gestureRecognizer === pan else { return true }
        guard scrollView.zoomScale <= scrollView.minimumZoomScale + 0.01 else { return
        ↪ false }
        let velocity = pan.velocity(in: view)
        return velocity.y > abs(velocity.x)
    }

    @objc private func handlePan(_ gesture: UIPanGestureRecognizer) {
        let translation = gesture.translation(in: view)
        let dismissDistance: CGFloat = 150
        switch gesture.state {
        case .changed:
            let down = max(translation.y, 0)
            let scale = max(1 - down / 1000, 0.85)
            scrollView.transform = CGAffineTransform(translationX: translation.x, y:
↪   translation.y)
                .scaledBy(x: scale, y: scale)
           view.backgroundColor = UIColor.black.withAlphaComponent(max(0, 1 - down / 400))
        case .ended, .cancelled:
            let velocity = gesture.velocity(in: view).y
            if gesture.state == .ended && (translation.y > dismissDistance || velocity >
            ↪ 1000) {
                dismiss(animated: true)
            } else {
                UIView.animate(withDuration: 0.3, delay: 0,
                               usingSpringWithDamping: 0.8, initialSpringVelocity: 0) {
                    self.scrollView.transform = .identity
                    self.view.backgroundColor = .black
                }
            }
        default:
            break
        }
    }
}
```

Когда жест вообще начинать (`gestureRecognizerShouldBegin`, метод делегата распознавателя):

- только при исходном масштабе — если фото приближено, движение пальца должно двигать фото, а не закрывать экран;
- только если палец идёт **больше вниз, чем вбок**: `velocity.y > |velocity.x|`. Иначе горизонтальное листание (37.6) и закрытие спорили бы за один и тот же жест. Скорость — в точках в секунду: палец идёт вниз со скоростью 400 и вбок 100 — закрываем, вбок 600 — отдаём жест листанию.

Что происходит, пока тянешь (`.changed`), на числах:

- `translation` — насколько сдвинулся палец от начала жеста. Фото едет вместе с пальцем.
- Масштаб `1 - down / 1000`: на каждые 100 точек вниз фото уменьшается на 10%, но не меньше 0,85 (85%). Сдвинул на 150 — масштаб 0,85: фото на 15% меньше.
- Прозрачность фона `1 - down / 400`: на 100 точках фон на 25% прозрачнее, на 150 — осталось 62,5% черноты, на 400 и дальше — фон полностью прозрачный.

Когда отпустил (`.ended`): закрываем, если утянул дальше 150 точек **или** бросил быстро — быстрее 1000 точек в секунду (это короткий резкий взмах, даже на 60 точек). Иначе фото пружиной возвращается на место (damping 0.8 — почти без отскока, см. главу 32.2). Проверка `gesture.state == .ended` не даёт закрыть экран, если жест прервала система (`.cancelled`, например входящий звонок).

Три частые ошибки в таком рецепте:

1. Двигать `imageView.transform`. Но `transform` у `imageView` занят самим scroll view: именно через него работает зум (см. начало 37.1). Scroll view о твоём изменении не узнает: на симуляторе после ручного `imageView.transform = translation(0, 100)` `zoomScale` остался 1,0 и `contentSize` не сдвинулся, а при следующем pinch scroll view перезапишет transform своим. Двигать нужно весь `scrollView`.
2. Ставить проверку «тянем вниз» в начало обработчика: тогда жест, начатый вбок, вообще не сбрасывает состояние. Делегат с `gestureRecognizerShouldBegin` решает это до начала жеста.
3. Прозрачный фон бессмыслен, если под экраном ничего не видно. При показе со стилем `.fullScreen` UIKit **убирает** view экрана-источника из иерархии, как только анимация показа закончилась (за это отвечает `shouldRemovePresentersView` у `UIPresentationController`, глава 0, раздел 0.6). Чтобы при смахивании просвечивала галерея, экран показывают со стилем `.overFullScreen` — он оставляет источник на месте (см. 37.7).

37.4 Кнопка «Поделиться»

```
@objc private func shareTapped(_ sender: UIBarButtonItem) {
    guard let image = currentViewer?.imageView.image else { return }
    let activity = UIActivityViewController(activityItems: [image],
     ↪  applicationActivities: nil)
    activity.popoverPresentationController?.barButtonItem = sender
    present(activity, animated: true)
}
```

`UIActivityViewController` — стандартное системное меню «Поделиться»: AirDrop, сообщения, мессенджеры, «Сохранить в Файлы». Передаём картинку, остальное система сделает сама.

`popoverPresentationController?.barButtonItem = sender` — на iPad это меню показывается **поповером** (всплывающим окошком со стрелкой), и ему надо знать, к чему прикрепить стрелку. Без этой строки на iPad приложение падает. На iPhone объект тоже есть: у свежего `UIActivityViewController` на симуляторе iPhone `modalPresentationStyle` читается как `.popover`, и `popoverPresentationController` не `nil`. Просто в компактной ширине поповер **адаптируется** в лист снизу, и якорь не используется — строка безвредна, а код один для обоих устройств. `UIPopoverPresentationController → UIPresentationController → NSObject` — это не view и не view controller, а объект, который управляет показом (глава 0, раздел 0.6). У него два якоря: `sourceView` для обычных view и отдельный `barButtonItem` — потому что `UIBarButtonItem` не view (`UIBarItem → NSObject`, раздел 0.8), и `convert` или `frame` у него нет. Этот метод и те, что ниже, живут в `PhotoPagerViewController` (37.6), у которого есть панель с кнопками.

37.5 Сохранить в Фото

```
    @objc private func saveTapped() {
        guard let image = currentViewer?.imageView.image else { return }
        Task {
            do {
                try await PHPhotoLibrary.shared().performChanges {
                    PHAssetChangeRequest.creationRequestForAsset(from: image)
                }
                showMessage("Сохранено в Фото")
            } catch {
                showMessage("Не удалось сохранить")
            }
        }
    }

    private func showMessage(_ text: String) {
        let alert = UIAlertController(title: nil, message: text, preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "OK", style: .default))
        present(alert, animated: true)
    }
```

PhotoKit (фреймворк `Photos`) — доступ к медиатеке. `performChanges` выполняет изменение, `PHAssetChangeRequest.creationRequestForAsset` — «создай новый снимок из этой картинки». Асинхронная версия `performChanges` с `try await` есть с iOS 15, поэтому код читается сверху вниз: получилось — одно сообщение, ошибка — другое. Возвращаться на главный поток через `DispatchQueue.main.async` не нужно: `Task` в контроллере выполняется на главном акторе, и после `await` мы снова на нём.

Info.plist обязателен. В Info.plist (файл настроек приложения) нужен ключ `NSPhotoLibraryAddUsageDescripti` с текстом, зачем приложению доступ, например «Чтобы сохранять понравившиеся фото». При первом сохранении система покажет запрос с этим текстом. Без ключа приложение

аварийно завершится при первой же попытке. В Xcode этот ключ называется «Privacy - Photo Library Additions Usage Description».

Есть и старый способ — C-функция `UIImageWriteToSavedPhotosAlbum`:

```
UIImageWriteToSavedPhotosAlbum(image, self,
                               #selector(image(_:didFinishSavingWithError:contextInfo:)),
                               nil)

@objc func image(_ image: UIImage, didFinishSavingWithError error: Error?,
                 contextInfo: UnsafeRawPointer) {
    showMessage(error == nil ? "Сохранено в Фото" : "Не удалось сохранить")
}
```

Результат приходит в метод с особой сигнатурой через селектор — это наследие Objective-C. Ключ в Info.plist нужен тот же. Для нового кода удобнее `performChanges`.

37.6 Листание фото — UIPageViewController

`UIPageViewController` — контейнер, который листает экраны («страницы») свайпом, как книгу. Каждая страница — отдельный `PhotoViewerViewController`. Подробно о нём — в главе 6 (Onboarding).

```
final class PhotoPagerViewController: UIViewController {
    private let urls: [URL]
    private var currentIndex: Int
    private let pageVC = UIPageViewController(transitionStyle: .scroll,
                                              navigationOrientation: .horizontal,
                                              options: [.interPageSpacing: 16])
    private let counterLabel = UILabel()
    private var isUIHidden = false

    init(urls: [URL], startIndex: Int) {
        self.urls = urls
        self.currentIndex = startIndex
        super.init(nibName: nil, bundle: nil)
    }

    required init?(coder: NSCoder) {
        fatalError("init(coder:) не используется")
    }

    var currentViewer: PhotoViewerViewController? {
        pageVC.viewControllers?.first as? PhotoViewerViewController
    }

    override var preferredStatusBarStyle: UIStatusBarStyle { .lightContent }
    override var prefersStatusBarHidden: Bool { isUIHidden }
```

```
    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .clear
        overrideUserInterfaceStyle = .dark

        addChild(pageVC)
        pageVC.view.frame = view.bounds
        pageVC.view.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        view.addSubview(pageVC.view)
        pageVC.didMove(toParent: self)
        pageVC.dataSource = self
        pageVC.delegate = self
        pageVC.setViewControllers([makeViewer(at: currentIndex)],
                                  direction: .forward, animated: false)

        setupNavigationBar()
        updateCounter()
        preloadNeighbors()
    }

    private func makeViewer(at index: Int) -> PhotoViewerViewController {
        let viewer = PhotoViewerViewController(imageURL: urls[index])
        viewer.onSingleTap = { [weak self] in self?.toggleUI() }
        return viewer
    }
```

Разбор:

- `.interPageSpacing: 16` — чёрная щель 16 точек между фото при листании, как в «Фото».
- `addChild` → `addSubview` → `didMove(toParent:)` — три шага встраивания **дочернего контроллера**: сообщаем UIKit, что `pageVC` — наш ребёнок, чтобы он получал события жизненного цикла (`viewWillAppear` и другие) и повороты. `addChild` сам вызывает `willMove(toParent:)`, а `didMove(toParent:)` после `addSubview` — наша обязанность (containment — глава 0, раздел 0.6).
- `dataSource` и `delegate` у `UIPageViewController` — слабые ссылки (`weak` в заголовке SDK); без `dataSource` листание жестом вообще выключено.
- `view.backgroundColor = .clear` — фон пейджера прозрачный, чёрный фон рисует каждая страница: так при смахивании вниз (37.3) сквозь него видна галерея.
- `overrideUserInterfaceStyle = .dark` — тёмная тема для просмотрщика независимо от системной (см. главу 35.5).
- `[weak self]` в `onSingleTap`: страницу держит `pageVC`, его — пейджер; если бы страница держала пейджер сильно, получился бы цикл.

Источник данных — соседние страницы:

```
extension PhotoPagerViewController: UIPageViewControllerDataSource,
↪  UIPageViewControllerDelegate {
    func pageViewController(_ pageViewController: UIPageViewController,
                            viewControllerBefore viewController: UIViewController) ->
                             ↪  UIViewController? {
```

```
        guard let viewer = viewController as? PhotoViewerViewController,
              let index = urls.firstIndex(of: viewer.imageURL),
              index > 0 else { return nil }
        return makeViewer(at: index - 1)
    }

    func pageViewController(_ pageViewController: UIPageViewController,
                            viewControllerAfter viewController: UIViewController) ->
                             ↪  UIViewController? {
        guard let viewer = viewController as? PhotoViewerViewController,
              let index = urls.firstIndex(of: viewer.imageURL),
              index < urls.count - 1 else { return nil }
        return makeViewer(at: index + 1)
    }

    func pageViewController(_ pageViewController: UIPageViewController,
                            didFinishAnimating finished: Bool,
                            previousViewControllers: [UIViewController],
                            transitionCompleted completed: Bool) {
        guard completed,
              let viewer = currentViewer,
              let index = urls.firstIndex(of: viewer.imageURL) else { return }
        currentIndex = index
        updateCounter()
        preloadNeighbors()
    }
}
```

Data source («источник данных») — объект, у которого контейнер спрашивает: «какая страница перед этой?», «какая после?». `nil` — «страниц больше нет», дальше не листается.

`didFinishAnimating ... transitionCompleted` — метод **делегата**: листание закончилось. Проверяем `completed`: пользователь мог начать листать и передумать, тогда страница не сменилась, и счётчик трогать нельзя.

`urls.firstIndex(of:)` ищет номер фото по адресу. Если в галерее одно фото повторяется дважды, номер будет найден неверно; тогда храни индекс прямо в `PhotoViewerViewController`.

37.7 Hero-анимация: миниатюра вырастает в фото

Hero-анимация — миниатюра из сетки «перелетает» и вырастает в полноэкранное фото, а при закрытии улетает обратно в свою ячейку. Делается через собственный переход: объект-аниматор (`UIViewControllerAnimatedTransitioning`) и делегат перехода, который его выдаёт (о протоколах — в главе 26.7).

```
final class PhotoZoomTransition: NSObject, UIViewControllerAnimatedTransitioning {
    private let isPresenting: Bool
    private let thumbnail: UIImageView
```

```
    private let fullImageView: () -> UIImageView?

    init(isPresenting: Bool, thumbnail: UIImageView, fullImageView: @escaping () ->
    ↪  UIImageView?) {
        self.isPresenting = isPresenting
        self.thumbnail = thumbnail
        self.fullImageView = fullImageView
    }

    func transitionDuration(using ctx: UIViewControllerContextTransitioning?) ->
    ↪  TimeInterval {
        0.35
    }

    func animateTransition(using ctx: UIViewControllerContextTransitioning) {
        let container = ctx.containerView
        let thumbFrame = thumbnail.convert(thumbnail.bounds, to: container)
        let flying = UIImageView(image: thumbnail.image)
        flying.contentMode = .scaleAspectFill
        flying.clipsToBounds = true
        let duration = transitionDuration(using: ctx)

        if isPresenting {
            guard let toVC = ctx.viewController(forKey: .to),
                  let toView = ctx.view(forKey: .to) else {
                ctx.completeTransition(false)
                return
            }
            toView.frame = ctx.finalFrame(for: toVC)
            toView.alpha = 0
            container.addSubview(toView)

            let imageSize = thumbnail.image?.size ?? thumbFrame.size
            let endFrame = Self.aspectFitFrame(for: imageSize, in: container.bounds)
            flying.frame = thumbFrame
            container.addSubview(flying)
            thumbnail.isHidden = true

            UIView.animate(withDuration: duration, delay: 0,
                           usingSpringWithDamping: 0.9, initialSpringVelocity: 0) {
                flying.frame = endFrame
                toView.alpha = 1
            } completion: { _ in
                flying.removeFromSuperview()
                self.thumbnail.isHidden = false
                ctx.completeTransition(!ctx.transitionWasCancelled)
            }
        } else {
```

```
            guard let fromView = ctx.view(forKey: .from) else {
                ctx.completeTransition(false)
                return
            }
            let full = fullImageView()
            flying.frame = full.map { $0.convert($0.bounds, to: container) } ??
↪   container.bounds
            container.addSubview(flying)
            full?.isHidden = true
            thumbnail.isHidden = true

            UIView.animate(withDuration: duration, delay: 0,
                           usingSpringWithDamping: 0.9, initialSpringVelocity: 0) {
                flying.frame = thumbFrame
                fromView.alpha = 0
            } completion: { _ in
                flying.removeFromSuperview()
                self.thumbnail.isHidden = false
                full?.isHidden = false
                ctx.completeTransition(!ctx.transitionWasCancelled)
            }
        }
    }

    static func aspectFitFrame(for imageSize: CGSize, in bounds: CGRect) -> CGRect {
        guard imageSize.width > 0, imageSize.height > 0 else { return bounds }
        let scale = min(bounds.width / imageSize.width, bounds.height / imageSize.height)
       let size = CGSize(width: imageSize.width * scale, height: imageSize.height * scale)
        return CGRect(x: bounds.midX - size.width / 2, y: bounds.midY - size.height / 2,
                      width: size.width, height: size.height)
    }
}
```

Идея: настоящие экраны не двигаются. Вместо них летит **копия картинки** (`flying`) — временный `UIImageView`, который живёт только во время анимации.

По шагам для открытия:

1. `containerView` — «сцена» перехода, общий view, в котором на время анимации лежат оба экрана.
2. `thumbnail.convert(thumbnail.bounds, to: container)` — где миниатюра находится в координатах сцены. У миниатюры свой `frame` внутри ячейки (например, x = 20 в ячейке), а нам нужна её позиция на всём экране. `convert(_:to:)` берёт прямоугольник **в своей системе координат** (в `bounds` получателя) и возвращает его в системе координат другого view; оба должны быть в одном окне, а `to: nil` даёт координаты окна. Поэтому передаём `bounds`, а не `frame`: на симуляторе для миниатюры (20, 10, 100 × 100) в ячейке на y = 300 `convert(bounds, to:)` даёт правильные (20, 310), а `convert(frame, to:)` — (40, 320), сдвинутые на лишний origin (глава 0, раздел 0.3). Если у view есть `transform`, результат —

описывающий прямоугольник (bounding box): миниатюра 100 × 100, повёрнутая на 45°, вернётся как 141 × 141.
3. Новый экран кладём на сцену прозрачным (`alpha = 0`).
4. Конечная рамка — фото, вписанное в экран, тем же расчётом, что в 37.1: для фото 1200 × 800 это 393 × 262 точки по центру.
5. Прячем настоящую миниатюру (иначе на её месте осталась бы копия) и анимируем: копия растёт, новый экран проявляется.
6. В конце удаляем копию, возвращаем миниатюру и **обязательно** вызываем `completeTransition` — без него UIKit считает переход незаконченным, и экран перестаёт реагировать на касания.

Закрытие — то же в обратную сторону: копия берёт рамку текущего большого фото и улетает в рамку миниатюры. `contentMode = .scaleAspectFill` у копии — чтобы на финише она совпала с квадратной обрезанной миниатюрой.

Прогон на симуляторе: открытие и закрытие прошли, после закрытия экран снят (`dismissed: true`), миниатюра видима (`thumb hidden: false`).

Делегат, который выдаёт аниматор, и сам показ из галереи:

```
final class PhotoZoomTransitioningDelegate: NSObject,
↪ UIViewControllerTransitioningDelegate {
    private let thumbnail: UIImageView
    private let fullImageView: () -> UIImageView?

    init(thumbnail: UIImageView, fullImageView: @escaping () -> UIImageView?) {
        self.thumbnail = thumbnail
        self.fullImageView = fullImageView
    }

    func animationController(forPresented presented: UIViewController,
                             presenting: UIViewController,
                             source: UIViewController) ->
                              ↪ UIViewControllerAnimatedTransitioning? {
        PhotoZoomTransition(isPresenting: true, thumbnail: thumbnail, fullImageView:
↪ fullImageView)
    }

    func animationController(forDismissed dismissed: UIViewController) ->
     ↪ UIViewControllerAnimatedTransitioning? {
        PhotoZoomTransition(isPresenting: false, thumbnail: thumbnail, fullImageView:
↪ fullImageView)
    }
}

final class PhotoNavigationController: UINavigationController {
    override var childForStatusBarStyle: UIViewController? { topViewController }
    override var childForStatusBarHidden: UIViewController? { topViewController }
}
```

```
// В контроллере галереи:
final class GalleryViewController: UIViewController {
    private var urls: [URL] = []
    private var thumbnails: [UIImageView] = []
    private var zoomDelegate: PhotoZoomTransitioningDelegate?

    private func openPhoto(at index: Int) {
        let pager = PhotoPagerViewController(urls: urls, startIndex: index)
        let nav = PhotoNavigationController(rootViewController: pager)
        nav.modalPresentationStyle = .overFullScreen
        nav.modalPresentationCapturesStatusBarAppearance = true
        nav.overrideUserInterfaceStyle = .dark
        let delegate = PhotoZoomTransitioningDelegate(thumbnail: thumbnails[index]) {
         ↪  [weak pager] in
            pager?.currentViewer?.imageView
        }
        zoomDelegate = delegate
        nav.transitioningDelegate = delegate
        present(nav, animated: true)
    }
}
```

Разбор деталей, на которых спотыкаются:

- `zoomDelegate = delegate` — `transitioningDelegate` у контроллера хранится **слабой** ссылкой. Если не держать делегата в свойстве, он исчезнет сразу после `present`, и закрытие пройдёт без нашей анимации.
- `fullImageView` — замыкание, а не готовый view: к моменту закрытия пользователь мог перелистать на третье фото, и лететь обратно должно то, что на экране сейчас. `[weak pager]` — чтобы делегат, который живёт в галерее, не удерживал закрытый просмотрщик.
- `.overFullScreen` — экран-источник остаётся под новым, поэтому работает прозрачность при смахивании (37.3).
- `PhotoNavigationController` и `modalPresentationCapturesStatusBarAppearance` — чтобы цветом статус-бара (часы, батарея) управлял просмотрщик. Обычный `UINavigationController` решает это по своей панели, и в прогоне на iOS 26.5 время показывалось чёрным по чёрному. Подкласс отдаёт решение верхнему экрану, а тот просит светлый текст.

Хочешь закрытие вслед за пальцем, с возможностью «передумать», — поверх этого аниматора добавляют `UIPercentDrivenInteractiveTransition`. А с iOS 18 похожий эффект даёт готовый zoom-переход (глава 32.11), но для iOS 15–17 нужен код выше.

37.8 Панель навигации над фото

```
    private func setupNavigationBar() {
        let appearance = UINavigationBarAppearance()
        appearance.configureWithOpaqueBackground()
        appearance.backgroundColor = .black
```

```
        appearance.shadowColor = .clear
        appearance.titleTextAttributes = [.foregroundColor: UIColor.white]
        navigationItem.standardAppearance = appearance
        navigationItem.scrollEdgeAppearance = appearance

        navigationItem.leftBarButtonItem = UIBarButtonItem(
            barButtonSystemItem: .close, target: self, action: #selector(closeTapped))
        navigationItem.rightBarButtonItems = [
            UIBarButtonItem(barButtonSystemItem: .action, target: self, action:
↪   #selector(shareTapped(_:))),
            UIBarButtonItem(image: UIImage(systemName: "square.and.arrow.down"),
                            style: .plain, target: self, action: #selector(saveTapped)),
        ]
        navigationItem.titleView = counterLabel
        counterLabel.font = .systemFont(ofSize: 15, weight: .semibold)
        counterLabel.textColor = .white
    }

    @objc private func closeTapped() {
        dismiss(animated: true)
    }
```

`UINavigationBarAppearance` (iOS 13+) — описание внешнего вида панели: фон, тень, цвет заголовка. Задаём его **через `navigationItem`** этого экрана, а не через `navigationBar` — так настройка действует только пока этот экран наверху и не «протекает» на другие экраны того же навигационного стека. Почему так работает: `navigationItem` — свойство **view controller'а** (класс `UINavigationItem` → `NSObject`, не view), панель `UINavigationBar` показывает `navigationItem` верхнего экрана как свой `topItem` (на симуляторе `vc.navigationItem === navigationBar.topItem`), и Apple пишет, что `standardAppearance` у item'а перекрывает одноимённое свойство панели (глава 0, разделы 0.6 и 0.8). `standardAppearance` — обычное состояние, `scrollEdgeAppearance` — когда контент прокручен к самому верху. `shadowColor = .clear` убирает тонкую линию под панелью.

Почему чёрный фон, а не прозрачный. Классический рецепт — `configureWithTransparentBackground()`, чтобы фото было видно и под панелью. Мы проверили его на симуляторе iOS 26.5: при показе `.overFullScreen` сквозь прозрачную (и даже полупрозрачную) панель проступала панель **галереи** под просмотрщиком — заголовок «Галерея» наезжал на «1 из 3». С непрозрачным чёрным фоном картинка чистая. На фото это почти не влияет: панель чёрная, как и фон вокруг фото, и её можно спрятать тапом (37.9).

Кнопки системные: `.close` — крестик, `.action` — стандартный значок «Поделиться», для сохранения — SF Symbol `square.and.arrow.down`. Сами `UIBarButtonItem` — тоже не view (`UIBarItem` → `NSObject`): их нельзя добавить через `addSubview` или повесить на них жест, панель сама рисует для каждого кнопку по описанию.

37.9 Спрятать панель тапом

```
    private func toggleUI() {
```

```
        isUIHidden.toggle()
        navigationController?.setNavigationBarHidden(isUIHidden, animated: true)
        UIView.animate(withDuration: 0.2) {
            self.navigationController?.setNeedsStatusBarAppearanceUpdate()
        }
    }
```

Тап по фото (одиночный, из 37.2) прячет панель и статус-бар — фото остаётся одно на чёрном. Ещё тап — всё возвращается.

- `setNavigationBarHidden(_:animated:)` — панель уезжает вверх с анимацией.
- Статус-бар прячется через `prefersStatusBarHidden`, который мы переопределили в пейджере (37.6). Просто поменять `isUIHidden` мало: UIKit должен узнать, что ответ изменился, — `setNeedsStatusBarAppearanceUpdate()`. Вызов внутри `UIView.animate` делает исчезновение плавным.

37.10 Счётчик «N из M»

```
    private func updateCounter() {
        counterLabel.text = "\(currentIndex + 1) из \(urls.count)"
        counterLabel.sizeToFit()
    }
```

`currentIndex + 1` — номера в массиве идут с нуля, а люди считают с одного: первое фото — «1 из 3». `sizeToFit()` подгоняет размер подписи под текст: без него у `titleView` может остаться нулевая ширина. В прогоне заголовок показал «1 из 3».

Подпись создаётся **один раз** (`counterLabel` — свойство), а обновляется только текст. Создавать новый `UILabel` при каждом листании — лишняя работа на каждой странице.

Для небольшого числа фото вместо счётчика можно поставить внизу `UIPageControl` с точками (глава 36.10), но при 30 фото точки уже не помещаются — счётчик универсальнее.

37.11 Подгрузка соседних фото

```
    private func preloadNeighbors() {
        for index in [currentIndex - 1, currentIndex + 1] where
        ↪ urls.indices.contains(index) {
            let url = urls[index]
            Task { _ = await ImageCache.shared.image(for: url) }
        }
    }
}
```

Пока пользователь смотрит фото номер 5, заранее грузим 4 и 6 в кеш. Когда он перелистнёт, страница возьмёт картинку из кеша мгновенно, без спиннера. Дальние фото не грузим: при галерее из 500 снимков загружать всё сразу — лишний трафик и память.

- `urls.indices.contains(index)` — у первого фото нет «предыдущего» (индекс −1), у последнего нет «после него». Проверка отсекает несуществующие номера без падения.
- `_ = await` — результат нам не нужен, важен сам факт: картинка легла в кеш. `ImageCache` (глава 16.3) к тому же склеивает одинаковые запросы: если страница попросит то же фото, пока идёт подгрузка, второго скачивания не будет.

37.12 Что должно быть в хорошем просмотрщике

1. **Чёрный фон** и тёмная тема — фото ничто не перебивает.
2. **Три жеста**: pinch, двойной тап к точке, смахивание вниз для закрытия (только при исходном масштабе).
3. **Фото вписано и по центру** — `contentSize` по размеру фото, центрирование через `contentInset` при каждом зуме.
4. **Тап прячет интерфейс** — панель и статус-бар.
5. **«Поделиться» и «Сохранить»** — с поповером для iPad и ключом `NSPhotoLibraryAddUsageDescription`.
6. **«N из M»**, если фото несколько.
7. **Подгрузка соседних** фото.
8. **Спиннер**, пока фото грузится.
9. **Поворот экрана** — фото вписывается заново.
10. **Доступность**: у кнопок системные подписи уже есть; самому фото дай `accessibilityLabel` с описанием, если оно известно (глава 34.13).

Упражнения

Упражнение 37.1. Сделай порог закрытия не фиксированным (150 точек), а равным 20% высоты экрана. Сколько это точек на iPhone 16 (высота 852 точки) и на iPad с высотой 1024 точки в альбомной ориентации?

Упражнение 37.2. Двойной тап сейчас увеличивает в 2,5 раза. Сделай, чтобы он приближал **до максимума** (`maximumZoomScale`, то есть в 4 раза). Какого размера будет прямоугольник для `zoom(to:)` на экране 393 × 852?

Ответы к упражнениям

37.1.

```
func dismissThreshold(for view: UIView) -> CGFloat {
    view.bounds.height * 0.2
}
```

В `handlePan` вместо `let dismissDistance: CGFloat = 150` пиши `let dismissDistance = dismissThreshold(for: view)`. На iPhone 16: 852 × 0,2 = 170,4 точки. На iPad в альбомной ориентации: 1024 × 0,2 = 204,8 точки. Порог в процентах ведёт себя одинаково на всех экранах — на маленьком телефоне не приходится тянуть «через весь экран».

37.2.

```
func zoomRect(around point: CGPoint, in scrollView: UIScrollView) -> CGRect {
    let targetScale = scrollView.maximumZoomScale
    let size = CGSize(width: scrollView.bounds.width / targetScale,
                      height: scrollView.bounds.height / targetScale)
    return CGRect(x: point.x - size.width / 2,
                  y: point.y - size.height / 2,
                  width: size.width, height: size.height)
}
```

В `doubleTapped` замени расчёт на `scrollView.zoom(to: zoomRect(around: point, in: scrollView), animated: true)`. Прямоугольник: 393 / 4 ≈ 98 точек в ширину и 852 / 4 = 213 в высоту. Именно он растянется на весь экран — увеличение в 4 раза.

Что мы выучили

- **`UIScrollView` + `viewForZooming`** — зум из коробки: scroll view ставит `transform` одному subview и сам пересчитывает `contentSize`; `bounds` subview не меняются; зум включён, только если `maximumZoomScale` > `minimumZoomScale` (по умолчанию оба 1,0). `imageView` по размеру вписанного фото, а не на весь экран.
- **Вписывание**: масштаб — меньшее из двух отношений сторон; 1200 × 800 на экране 393 × 852 → 393 × 262.
- **Центрирование** через `contentInset` в `scrollViewDidZoom`.
- **Двойной тап** — `zoom(to:)` с прямоугольником «экран / масштаб» вокруг точки тапа; одиночный тап ждёт `require(toFail:)`.
- **Смахивание вниз** — двигаем `scrollView`, а не `imageView`; жест только при исходном масштабе и вертикальном движении; порог 150 точек или 1000 точек в секунду.
- **`.overFullScreen`**, чтобы под просмотрщиком была видна галерея.
- **Поделиться** — `UIActivityViewController`; `popoverPresentationController` есть и на iPhone (там поповер адаптируется в лист), якорь нужен iPad.
- **`convert(_:to:)`** — прямоугольник в своих `bounds` → координаты другого view того же окна; с `transform` возвращает bounding box.
- **`navigationItem`** и **`UIBarButtonItem`** — не view: описания, которые панель показывает для верхнего экрана.
- **Сохранить** — `PHPhotoLibrary.performChanges` (async с iOS 15) и ключ `NSPhotoLibraryAddUsageDescription`
- **`UIPageViewController`** — data source с соседями, счётчик по `transitionCompleted`.
- **Hero-переход** — летит копия картинки; делегат перехода держим в свойстве; `completeTransition` обязателен.
- **Панель** — через `navigationItem`, на iOS 26.5 — непрозрачная чёрная; статус-бар — подкласс навигационного контроллера.
- **Подгрузка соседей** — в кеш, с проверкой границ массива.

Apple Developer Documentation

- UIScrollView — прокрутка и масштабирование, основа просмотрщика.
- UIScrollView.zoomScale — текущий масштаб содержимого; по умолчанию 1.0.
- UIScrollView.zoom(to:animated:) — приблизить к прямоугольнику.

- UIView.convert(_:to:) — пересчёт прямоугольника из своих `bounds` в координаты другого view.
- UIPopoverPresentationController — якоря `sourceView` / `barButtonItem`, адаптация в компактной ширине.
- UIViewController.navigationItem — `UINavigationItem` принадлежит view controller'у, панель показывает его как `topItem`.
- UIScrollViewDelegate.viewForZooming(in:) — какой view масштабировать.
- UIScrollViewDelegate.scrollViewDidZoom(_:) — реакция на изменение масштаба (центрирование).
- UIPanGestureRecognizer — основа закрытия смахиванием.
- UIGestureRecognizerDelegate.gestureRecognizerShouldBegin(_:) — разрешить или запретить начало жеста.
- UITapGestureRecognizer — двойной и одиночный тап; см. `require(toFail:)`.
- UIPageViewController — листание страниц.
- UIViewControllerTransitioningDelegate — выдаёт аниматор перехода.
- UIViewControllerAnimatedTransitioning — протокол аниматора.
- UIPercentDrivenInteractiveTransition — интерактивный переход вслед за пальцем.
- UIModalPresentationStyle.overFullScreen — показ, при котором экран-источник остаётся под новым.
- UIActivityViewController — системное меню «Поделиться».
- PHPhotoLibrary — изменения медиатеки через `performChanges`.
- NSPhotoLibraryAddUsageDescription — ключ Info.plist для сохранения в Фото.
- UIImageWriteToSavedPhotosAlbum(::::) — старый способ сохранения.
- UINavigationBarAppearance — внешний вид панели навигации.

Это конец Части IV (UI Cookbook). Дальше — Часть V, чек-лист перед релизом: скриншоты, Privacy Manifest, удаление аккаунта, push и deep links, виджеты и App Intents, аудит доступности. Это не код, а методичка «что подготовить перед первым релизом».

→ Глава 38. Production: screenshots для App Store

Глава 38. Production: screenshots для App Store

Часть V — последняя. Здесь меньше кода и больше правил: что подготовить перед первым релизом, какие требования у Apple на сегодня и где на этих требованиях обычно спотыкаются. Каждое правило я привязываю к первоисточнику — к пункту App Review Guidelines (правила, по которым сотрудник Apple проверяет приложение перед публикацией, дальше просто «Guidelines») или к справке App Store Connect. Требования меняются, поэтому в конце каждого раздела есть ссылка, по которой их стоит перепроверить перед отправкой.

Цифры и формулировки в этой главе сверены с документацией Apple в сентябре 2026 года (Guidelines от 8 июня 2026 года, таблица Screenshot specifications в справке App Store Connect).

Эта глава — про скриншоты и видео на странице приложения.

38.1 Зачем скриншоты

Страница приложения в App Store (Apple называет её *product page*) — это витрина: иконка, название, подзаголовок, скриншоты, видео, описание, оценки. Из всего этого скриншоты занимают больше всего места на экране и решают, нажмёт ли человек «Загрузить».

Apple пишет на странице Creating your product page, что в результатах поиска показываются первые **от одного до трёх** скриншотов (зависит от ориентации: вертикальных влезает три, горизонтальный один), если у приложения нет видео-превью. Отсюда практическое правило: первые три кадра должны сами по себе объяснять, что делает приложение. Остальные человек увидит, только если откроет страницу и пролистает.

Аналогия — обложка книги в магазине. Покупатель видит корешок и обложку, листать начинает только если обложка зацепила.

38.2 Требования Apple: размеры и количество

Сначала два слова о единицах. Интерфейс iOS меряется в **точках** (points, pt), а экран состоит из **пикселей**. У современных iPhone в одной точке 3×3 пикселя (это и есть «@3x»). iPhone 17 Pro Max в точках — 440 × 956, в пикселях — 440 × 3 = 1320 на 956 × 3 = 2868. App Store Connect просит скриншоты именно в пикселях.

Из таблицы Screenshot specifications (сентябрь 2026):

Количество. На каждый размер экрана и каждый язык — от **1 до 10** скриншотов. Форматы — `.png`, `.jpg`, `.jpeg`. Прозрачность (альфа-канал) не допускается: PNG с прозрачным фоном App Store Connect не примет.

iPhone 6,9” (iPhone Air, 18 Pro Max, 17 Pro Max, 16 Pro Max, 16 Plus, 15 Pro Max, 15 Plus, 14 Pro Max). Принимаются три размера в портрете:

- 1320 × 2868 — iPhone 16/17/18 Pro Max;
- 1290 × 2796 — iPhone 14 Pro Max, 15 Pro Max, 15 Plus, 16 Plus;
- 1260 × 2736 — iPhone Air.

В ландшафте — те же числа, переставленные местами.

iPhone 6,5” (iPhone 14 Plus, 11–13 Pro Max, 11, XS Max, XR): 1284 × 2778 или 1242 × 2688. В таблице у этого класса написано «Required if app runs on iPhone and screenshots for 6.9” display aren’t provided» — обязателен, если приложение работает на iPhone и **нет** скриншотов 6,9”.

Отсюда главное правило для iPhone: нужен **один** набор — либо 6,9”, либо 6,5”. На практике берут 6,9”: он крупнее, и Apple сама уменьшит его для всех меньших экранов (6,3”, 6,1”, 5,5” и т.д.). У каждого младшего класса в таблице так и написано: если своих скриншотов нет, используются уменьшенные от старшего.

iPad 13” (iPad Pro M4/M5, iPad Air M2–M4, старшие iPad Pro 12,9”): 2064 × 2752 или 2048 × 2732 в портрете. Требование — «Required if app runs on iPad». Если твоё приложение только для iPhone (в настройках таргета в Supported Destinations нет iPad), iPad-скриншоты не нужны. Если iPad поддерживается — 13” обязателен, меньшие iPad снова масштабируются из него.

> **Про новые устройства.** В таблице уже есть строка «iPhone Duo» с пометкой, что загрузка для него появится в App Store Connect позже. Таблица обновляется вместе с линейкой устройств, поэтому перед каждым релизом открывай её заново, а не полагайся на цифры из книги.

Если размер «почти подходит» (например, ты обрезал картинку на пару пикселей), App Store Connect выдаст ошибку при загрузке. Размер должен совпасть с одним из значений таблицы точно.

Упражнение. Посчитай в уме пиксельный размер скриншота для iPhone с экраном 402 × 874 точки и множителем @3x. Найди этот размер в таблице Apple — к какому классу экрана он относится и нужен ли он при наличии 6,9”? Ответ — в конце главы.

38.3 Как делать скриншоты

Симулятор — самый предсказуемый способ: нет чужих уведомлений, случайного процента батареи и бликов.

1. Запусти приложение на симуляторе iPhone 17 Pro Max (это класс 6,9”).
2. Открой нужный экран.
3. В Simulator нажми `Cmd+S` (File → Save Screen). PNG по умолчанию сохраняется на рабочий стол, размер — 1320 × 2868, ровно как требует таблица.

То же самое из терминала (удобно для скриптов и CI — сервера, который собирает приложение автоматически):

```
xcrun simctl io booted screenshot ~/Desktop/todo-list.png
```

`booted` — «тот симулятор, что сейчас запущен». Если запущено несколько, вместо `booted` подставь UDID нужного из `xcrun simctl list devices`.

Реальное устройство тоже подходит: боковая кнопка + громкость вверх. Минус — в кадр попадает реальный статус-бар (оператор, батарея 23%), и снимок с iPhone 16 Pro (6,3”) для 6,9” не годится — размер другой.

Автоматизация. Когда языков несколько, снимать руками долго. Сторонний инструмент `fastlane snapshot` прогоняет UI-тесты (`XCUITest` — тесты, которые тыкают в интерфейс как пользователь) на нужных симуляторах и языках и складывает скриншоты по папкам. Это не часть Xcode, его ставят отдельно.

38.4 Статус-бар 9:41

На маркетинговых фото Apple в статус-баре почти всегда 9:41, полная батарея и полный сигнал. Симулятор умеет подменить статус-бар одной командой:

```
xcrun simctl status_bar booted override \
    --time "9:41" \
    --batteryState charged --batteryLevel 100 \
    --cellularBars 4 --wifiBars 3
```

Разбор:

- `status_bar booted override` — переопредели статус-бар запущенного симулятора;
- `--time "9:41"` — время в статус-баре (часы самого Mac не меняются);
- `--batteryState charged --batteryLevel 100` — «заряжено, 100%»;
- `--cellularBars 4 --wifiBars 3` — полный сигнал сотовой сети и Wi-Fi.

Вернуть как было — `xcrun simctl status_bar booted clear`, посмотреть текущие подмены — `xcrun simctl status_bar booted list`. Если после перезапуска симулятора статус-бар снова «живой», просто выполни команду ещё раз.

Никакого правила «время должно быть 9:41» у Apple нет — это традиция с презентации первого iPhone. Но аккуратный одинаковый статус-бар на всех кадрах выглядит заметно лучше, чем «14:07, 12% батареи» на одном и «09:15, 80%» на другом.

38.5 Что Guidelines говорят о содержимом скриншотов

Главные пункты раздела 2.3 «Accurate Metadata» (метаданные — всё, что описывает приложение в магазине: название, описание, скриншоты, видео):

- **2.3.3** — скриншоты должны показывать приложение **в работе**, а не только заставку, экран входа или splash (экран-заставку при запуске). Текстовые подписи и наложения поверх скриншота разрешены.
- **2.3.7** — в названии, подзаголовке, скриншотах и видео не должно быть цен и посторонних фраз «для ранжирования». Подзаголовок не должен содержать непроверяемых заявлений — «лучшее приложение №1» сюда относится.

- **2.3.8** — скриншоты и видео должны подходить для рейтинга 4+, даже если само приложение 17+.
- **2.3.9** — права на все материалы на скриншотах должны быть у тебя, а данные в кадре — вымышленные, не реального человека. Никаких настоящих телефонов и email твоих знакомых.
- **2.3.10** — не показывай названия, иконки и изображения других мобильных платформ (Android-телефон в рамке — прямой путь к отказу).
- **2.3.2** — если на скриншоте что-то, за что надо доплатить (премиум, подписка), это должно быть понятно из описания и скриншотов.

И общее для всего раздела 2.3: метаданные должны соответствовать тому, что приложение делает сейчас. Показал на скриншоте функцию, которой в сборке нет, — получишь отказ по 2.3.1.

Рамки устройств Apple раздаёт сама в Apple Design Resources (шаблоны устройств, «product bezels»), а правила использования изображений её устройств описаны в App Store Marketing Guidelines. Если рисуешь рамку iPhone вокруг скриншота — бери её оттуда, а не с первого попавшегося сайта.

38.6 Стиль скриншотов

Три варианта от слабого к сильному.

Слабо: голый скриншот

Просто кадр экрана. Человек видит обычный список и не понимает, чем приложение отличается от десятка похожих. Доля тех, кто после просмотра страницы нажал «Загрузить» (конверсия), у таких страниц обычно ниже.

Лучше: скриншот + подпись

Кадр плюс крупный заголовок сверху или снизу:

```
┌──────────────────────┐
│  «Все задачи под рукой» │
│                      │
│  [скриншот приложения]  │
└──────────────────────┘
```

Человек сразу читает, **что** он видит.

Сильнее всего: рамка устройства + подпись + фон

Скриншот в рамке устройства на фоне фирменного цвета и короткая подпись:

```
┌──────────────────────┐
│   фон в цветах бренда   │
│                      │
│  [iPhone с экраном      │
│   приложения внутри]    │
```

```
┌──────────────────────────┐
│                          │
│  «Список дел простой,    │
│   как лист бумаги»       │
└──────────────────────────┘
```

Это работа для дизайнера, но она окупается. Итоговая картинка всё равно должна иметь ровно пиксельный размер из таблицы — 1320 × 2868 для 6,9". Если в Figma ты рисуешь макет в точках (440 × 956), экспортируй с множителем 3x.

Инструменты: Figma или Sketch (рамки из Apple Design Resources), сторонние генераторы вроде AppMockUp Studio. Встроенного «генератора маркетинговых скриншотов» в Xcode нет.

Если приложение поддерживает тёмную тему, Apple на странице Creating your product page советует показать хотя бы один скриншот в тёмном оформлении.

38.7 Что показывать

Три–пять главных сценариев — по одному на кадр:

1. **Главная функция** — то, ради чего приложение качают.
2. **Отличие** — чем ты лучше соседей по поиску.
3. **Простота** — как быстро начать.
4. **Доверие** — «без рекламы», «данные остаются на телефоне». Только если это правда и совпадает с анкетой App Privacy (глава 39).
5. **Дополнительная ценность** — виджет, тёмная тема, iPad.

Для Todo из главы 12 это могло бы выглядеть так:

1. Список задач с отмеченными и неотмеченными ячейками — «Все задачи под рукой».
2. Редактор задачи — «Название, заметка, срок».
3. Фильтр — «Сегодня / На неделе / Все».
4. Pull-to-refresh (обновление жестом «потяни вниз») — «Синхронизация одним жестом».
5. Пустой список с кнопкой — «Первая задача за 5 секунд».

Кадр 5 — единственный, где показан «пустой» экран, и он всё равно показывает действие, а не заставку. Это важно для 2.3.3.

Упражнение. Составь такие же пять подписей для Weather из главы 15. Проверь каждую по пунктам 38.5: нет ли цены, «лучшего приложения», чужой платформы и данных реального человека. Пример — в ответах.

38.8 Локализация скриншотов

Если приложение переведено на несколько языков, для каждого языка (в App Store Connect это «локализация») можно загрузить свои скриншоты.

Порядок в App Store Connect: страница приложения → нужная версия → справа вверху выпадающий список языка → раздел App Previews and Screenshots → View All Sizes in Media Manager. Там для каждого размера и языка отдельная «ячейка», куда перетаскиваются файлы.

Если для какого-то языка скриншоты не загружены, App Store показывает скриншоты основного языка приложения (Primary Language в App Information). Для видео справка Apple фор-

мулирует это так: если превью для языка витрины нет, показывается превью «ближайшего подходящего языка» (next best available language).

Практический вывод: если основной язык русский, а приложение продаётся и в других странах — английские скриншоты стоит сделать отдельно, иначе англоязычный пользователь увидит русские подписи.

38.9 Видео-превью (App Preview)

Кроме скриншотов можно загрузить короткое видео. Требования из App preview specifications:

- **длина** — от 15 до 30 секунд;
- **количество** — до трёх видео на размер экрана и язык;
- **разрешение** для iPhone 6,9” и 6,5” — **886 × 1920** (портрет) или 1920 × 886 (ландшафт); для iPad 13” — своё значение из той же таблицы;
- **формат** — `.mov`, `.m4v`, `.mp4` (H.264) или `.mov` (ProRes 422 HQ);
- **частота кадров** — не больше 30 кадров в секунду;
- **размер файла** — до 500 МБ.

886 × 1920 — не «родное» разрешение экрана, а уменьшенное примерно в полтора раза (1320 / 886 ≈ 1,49). Запись с симулятора придёт в полном разрешении экрана, так что её придётся перекодировать в 886 × 1920 (iMovie, Final Cut или `ffmpeg`).

По Guidelines **2.3.4** превью может содержать **только записи экрана самого приложения**. Разрешено добавить закадровый голос, текстовые подписи и видео-наложения, чтобы объяснить непонятное. А вот снятая на камеру рука с телефоном или кадры «из жизни» — нет.

Ещё два факта из справки: видео на странице всегда стоят **перед** скриншотами, и на странице они запускаются сами без звука. Поэтому первые две-три секунды должны быть понятны без звука, а не начинаться с заставки.

Записать экран симулятора:

```
xcrun simctl io booted recordVideo --codec=h264 demo.mov
# ... выполняешь действия в симуляторе ...
# Ctrl+C — остановить запись
```

`--codec=h264` обязателен: без него `simctl` пишет в HEVC (H.265), а App Store Connect из этих двух кодеков принимает только H.264. После записи видео нужно обрезать до 15–30 секунд и уменьшить до 886 × 1920.

Видео особенно помогает там, где скриншоты не передают движение: игры, редакторы, приложения с жестами. Для простого списка дел оно необязательно.

38.10 A/B-тесты страницы (Product Page Optimization)

В App Store Connect можно сравнить разные варианты страницы — Apple называет это Product Page Optimization.

Как это устроено по документации Apple:

- до **трёх** альтернативных вариантов (treatments) против текущей страницы; меняются иконка, скриншоты и видео;
- ты выбираешь долю трафика: например, 40% на тест с двумя вариантами — каждый вариант увидят 20% посетителей, а 60% увидят обычную страницу;
- варианты видят люди на **iOS и iPadOS 15 и новее**;
- тест идёт до **90 дней** или пока ты его не остановишь; одновременно может идти только один тест;
- новые скриншоты для вариантов проходят проверку App Review;
- результаты — в App Analytics; Apple советует ждать, пока вариант покажет разницу с уверенностью не ниже 90%.

«Уверенность 90%» простыми словами: если бы в действительности варианты были одинаковыми, такая разница в результатах случайно получалась бы реже, чем в 1 случае из 10. Поэтому не останавливай тест через два дня, когда у варианта B «+15%» по трём сотням показов — это ещё шум.

Есть и родственная функция — **Custom Product Pages**: дополнительные версии страницы со своими скриншотами и отдельной ссылкой, например для рекламной кампании.

38.11 Где обычно ошибаются

- **Тестовые данные в кадре** — «Тест 1», «Lorem ipsum», «asdf». Заполни приложение правдоподобными вымышленными данными (2.3.9).
- **Скриншот логина или splash первым кадром** — нарушение 2.3.3 и потерянный первый кадр в поиске.
- **Старый дизайн** — скриншоты с прошлой версии, где интерфейс уже другой. По 2.3 метаданные должны соответствовать текущей версии. Обновить скриншоты можно только в новой версии приложения: после одобрения они «замораживаются».
- **Уведомления и «12% батареи» в статус-баре** — решается командой из 38.4.
- **PNG с прозрачностью** — не загрузится.
- **Нет ни 6,9”, ни 6,5”** — App Store Connect не даст отправить версию на проверку, пока не заполнен обязательный размер.
- **Цены и «скидка 50%» на скриншоте** — 2.3.7. Для акций есть In-App Events и Promotional Text.

38.12 Ответы к упражнениям

38.2. 402 × 3 = 1206, 874 × 3 = 2622 — скриншот 1206 × 2622. В таблице это класс **6,3”** (iPhone 16 Pro, 17, 17 Pro, 18 Pro). Если у тебя есть 6,9”, отдельные 6,3” не нужны: у класса 6,3” в таблице написано, что без своих скриншотов используются уменьшенные. Загружать их стоит, только если на 6,3” интерфейс выглядит заметно иначе.

38.7. Пример для Weather:

1. «Погода там, где ты» — главный экран с текущей температурой.
2. «Почасовой прогноз на сутки» — лента по часам.
3. «Работает и без сети» — экран с баннером «офлайн» и сохранёнными данными.
4. «Потяни, чтобы обновить» — момент pull-to-refresh.

5. «Любой город мира» — поиск города.

Проверка: цен нет, превосходных степеней нет, чужих платформ нет, город и данные вымышленные или общедоступные (прогноз, а не чьи-то личные данные). Кадр 3 — честное обещание, только если офлайн-режим из главы 15 действительно есть в сборке.

Что мы выучили

- Для iPhone нужен **один** набор скриншотов: **6,9”** (1320 × 2868, 1290 × 2796 или 1260 × 2736) **или** 6,5”. Меньшие экраны App Store получает уменьшением.
- Для iPad, если он поддерживается, обязателен **13”** (2064 × 2752 или 2048 × 2732).
- **От 1 до 10** скриншотов на размер и язык; PNG/JPEG без прозрачности; размер — ровно из таблицы.
- Первые 1–3 скриншота видны в поиске, если нет видео.
- **2.3.3**: показывай приложение в работе, не splash и не логин. **2.3.7**: без цен и «лучшего приложения». **2.3.9**: только вымышленные данные. **2.3.10**: без чужих платформ.
- **App Preview**: 15–30 секунд, до трёх штук, 886 × 1920 для iPhone, только запись экрана самого приложения (2.3.4).
- `xcrun simctl status_bar booted override --time "9:41"` — аккуратный статус-бар на всех кадрах.
- **Product Page Optimization**: до трёх вариантов, до 90 дней, iOS 15+ у зрителей.

Apple Developer Documentation

- Screenshot specifications — таблица размеров и количества скриншотов для всех устройств.
- App preview specifications — длина, разрешения, кодеки и размер файла для видео.
- Upload app previews and screenshots — куда загружать, Media Manager, локализация.
- App Review Guidelines — 2.3 Accurate Metadata — правила для скриншотов, видео и описания.
- Creating your product page — рекомендации Apple по иконке, скриншотам и превью.
- Product page optimization — A/B-тесты страницы приложения.
- Apple Design Resources — официальные рамки устройств и шаблоны.
- App Store Marketing Guidelines — как можно использовать изображения устройств Apple и бейдж App Store.
- Running your app on simulated or physical devices — работа с симулятором, из которого снимаются скриншоты.

→ Глава 39. Production: App Privacy + Privacy Manifest

Глава 39. Production: App Privacy + Privacy Manifest

Прежде чем приложение попадёт в App Store, ты дважды рассказываешь Apple, что оно делает с данными пользователя. Первый раз — в анкете **App Privacy** в App Store Connect: из неё на странице приложения собирается блок «Конфиденциальность приложения», который люди называют «этикеткой конфиденциальности» (Apple — *Privacy Nutrition Label*, по аналогии с этикеткой состава на продуктах). Второй раз — в файле `PrivacyInfo.xcprivacy` внутри самого приложения (*privacy manifest*, «манифест конфиденциальности»).

Анкета — для людей: что ты собираешь и зачем. Манифест — для Apple и Xcode: какие «чувствительные» системные API трогает код и какие данные уходят с устройства. Если одно не совпадает с другим или с поведением приложения — это повод для отказа на проверке.

Все требования ниже сверены с документацией Apple в сентябре 2026 года. Пункты App Review Guidelines (дальше «Guidelines») указаны номерами.

39.1 Анкета App Privacy в App Store Connect

Анкета находится в App Store Connect: страница приложения → App Privacy. Без неё новое приложение и обновление отправить на проверку нельзя. При этом ответы можно менять в любой момент **без** выпуска новой версии — например, если ты подключил аналитику на сервере.

Главное слово анкеты — **collect** («собирать»). По определению Apple на странице App privacy details «собирать» — значит **передавать данные с устройства** так, что ты или твои партнёры (сторонние SDK — чужие библиотеки внутри твоего приложения: аналитика, реклама, крэш-репорты) можете получить к ним доступ дольше, чем нужно, чтобы ответить на запрос прямо сейчас.

Два примера, чтобы почувствовать границу:

- Приложение «Погода» (глава 15) отправляет координаты на сервер прогноза, сервер отвечает и **ничего не сохраняет** — это не сбор. Если сервер пишет координаты в лог и хранит месяц — это сбор (Location).
- Заметки хранятся только на телефоне и никуда не уходят — не сбор, даже если это личные тексты.

Для каждого типа данных, который ты собираешь, анкета спрашивает три вещи:

1. **Для чего** используются данные (можно выбрать несколько целей).

2. **Связаны ли с личностью** (*linked to the user*) — через аккаунт, устройство или другие детали. Apple прямо пишет, что «персональные данные» по законам о приватности считаются связанными.
3. **Используются ли для трекинга** (*tracking*) — о нём в 39.7.

Категории данных по актуальному списку Apple:

Категория	Что внутри (примеры)
Contact Info	имя, email, телефон, адрес
Health & Fitness	медицинские данные, данные о тренировках
Financial Info	платёжные данные, кредитная история, доходы
Location	точная и приблизительная геопозиция
Sensitive Info	этническая принадлежность, взгляды, биометрия и т.п.
Contacts	адресная книга, социальный граф
User Content	сообщения, фото и видео, аудио, игровые сохранения, обращения в поддержку
Browsing History	что человек смотрел вне приложения (сайты)
Search History	поисковые запросы внутри приложения
Identifiers	User ID, Device ID (в том числе рекламный идентификатор)
Purchases	история покупок
Usage Data	нажатия, запуски, просмотренная реклама
Diagnostics	крэши, скорость запуска, расход энергии
Surroundings	сканирование окружения (AR)
Body	движения рук и головы (Apple Vision Pro)
Other Data	всё остальное

Граница «точная / приблизительная геопозиция» у Apple задана числом: точная — координаты с **тремя и более** знаками после запятой. Три знака широты — это примерно 100 метров (один градус широты ≈ 111 км, тысячная доля ≈ 111 м). Координаты `43.238, 76.945` уже указывают на квартал в Алматы, а `43.2, 76.9` — на район в несколько километров.

Цели (*purposes*):

- **Third-Party Advertising** — чужая реклама в твоём приложении или передача данных тем, кто её показывает;
- **Developer's Advertising or Marketing** — твоя реклама и рассылки;
- **Analytics** — изучение поведения пользователей;
- **Product Personalization** — рекомендации, персональная лента;
- **App Functionality** — вход, работа функций, защита от мошенничества, крэш-репорты, поддержка;
- **Other Purposes** — остальное.

Некоторые данные можно не указывать (*optional disclosure*), только если выполнены **все** условия сразу: данные не для трекинга и не для рекламы, собираются редко, не входят в основную функцию, человек сам каждый раз заполняет форму и видит, что отправляет. Типичный пример — необязательная форма обратной связи. Если выполнено не всё — указывать нужно.

Ещё пара правил, на которых часто ошибаются:

- **Данные SDK — тоже твои.** Если Firebase Crashlytics отправляет крэш-логи, в анкете должна быть Diagnostics → Crash Data.
- **IP-адрес** указывается по тому, как ты его используешь: для геолокации — как Location, для диагностики — как Diagnostics.
- **Чат между пользователями** — это Emails or Text Messages, даже если это не SMS.

Отвечать за точность анкеты по Guidelines обязан ты: раздел **2.3** требует, чтобы метаданные, «включая информацию о конфиденциальности», соответствовали поведению приложения, а раздел **5.1.2** запрещает передавать персональные данные без разрешения человека. Приложения, которые делятся данными без согласия, по 5.1.2(i) могут снять с продажи.

Упражнение. Заполни анкету «на бумаге» для Todo из главы 12: задачи хранятся в `UserDefaults`, при первом запуске загружаются демо-задачи с `dummyjson.com`, аналитики нет. Что будет в анкете? Ответ — в конце главы.

39.2 Privacy Manifest — `PrivacyInfo.xcprivacy`

Манифест — это файл в формате property list (тот же формат, что у `Info.plist`: XML со словарями и массивами), который лежит внутри бандла приложения. Бандл (*bundle*) — папка `.app`, в которую Xcode складывает бинарник, картинки, `Info.plist` и остальные ресурсы.

В манифесте четыре ключа верхнего уровня:

Ключ	Что описывает
`NSPrivacyTracking`	используется ли что-то в приложении для трекинга
`NSPrivacyTrackingDomains`	домены, к которым обращается код для трекинга
`NSPrivacyCollectedDataTypes`	какие данные собираются (то же, что в анкете 39.1)
`NSPrivacyAccessedAPITypes`	какие required reason API используются и по какой причине

Требование к манифесту приходит от App Store, а не от версии iOS. В документации Describing use of required reason API сказано: с **1 мая 2024 года** App Store Connect **не принимает** приложения, которые используют required reason API и не описали причину в манифесте. Минимальная версия iOS у приложения значения не имеет: приложение с iOS 15 обязано иметь манифест так же, как с iOS 26.

Как создать в Xcode 26: **File** → **New** → **File from Template…** → раздел **Resource** → **App Privacy**. Имя по умолчанию — `PrivacyInfo`, файл получит расширение `.xcprivacy`. Убедись, что галочка стоит у таргета приложения: файл должен попасть в бандл. Xcode открывает его в табличном редакторе; на исходный XML можно переключиться через контекстное меню файла → Open As → Source Code.

Минимальный манифест для нашего playground-приложения:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
```

```
    <key>NSPrivacyTracking</key>
    <false/>
    <key>NSPrivacyTrackingDomains</key>
    <array/>
    <key>NSPrivacyCollectedDataTypes</key>
    <array/>
    <key>NSPrivacyAccessedAPITypes</key>
    <array>
        <dict>
            <key>NSPrivacyAccessedAPIType</key>
            <string>NSPrivacyAccessedAPICategoryUserDefaults</string>
            <key>NSPrivacyAccessedAPITypeReasons</key>
            <array>
                <string>CA92.1</string>
            </array>
        </dict>
        <dict>
            <key>NSPrivacyAccessedAPIType</key>
            <string>NSPrivacyAccessedAPICategoryFileTimestamp</string>
            <key>NSPrivacyAccessedAPITypeReasons</key>
            <array>
                <string>C617.1</string>
            </array>
        </dict>
    </array>
</dict>
</plist>
```

Разбор:

- `NSPrivacyTracking = false` — мы никого не трекаем.
- `NSPrivacyTrackingDomains` — пустой массив: трекинговых доменов нет. Если бы `NSPrivacyTracking` был `true`, сюда пришлось бы перечислить домены рекламных сетей. iOS блокирует обращения к этим доменам, пока человек не разрешил трекинг (39.7).
- `NSPrivacyCollectedDataTypes` — пустой массив: с устройства ничего не собираем (в смысле 39.1).
- `NSPrivacyAccessedAPITypes` — два словаря. Первый: мы пользуемся `UserDefaults` (Todo, настройки Profile), причина `CA92.1`. Второй: Notes из главы 13 читает даты изменения файлов в своей папке, причина `C617.1`. Что значат эти коды — в 39.3.

Проверить, что XML не сломан, можно командой `plutil -lint PrivacyInfo.xcprivacy` — она ответит `OK` или покажет строку с ошибкой.

39.3 Required reason API — что это и какие коды выбрать

Некоторые системные API сами по себе безобидны, но их можно использовать для **fingerprinting** — «снятия отпечатка»: сложить много мелких признаков устройства (вре-

мя загрузки системы, объём диска, список клавиатур) в уникальный номер и узнавать человека без его согласия. Apple пишет прямо: fingerprinting запрещён, **даже если** человек разрешил трекинг.

Поэтому для таких API нужно объявить причину — короткий код из фиксированного списка Apple. Причина ограничивает, что ты имеешь право делать с данными. Например, `DDA9.1` разрешает показывать даты файлов пользователю, но запрещает отправлять их с устройства.

Актуальные категории и коды (сентябрь 2026, из справки ключа NSPrivacyAccessedAPIType):

`NSPrivacyAccessedAPICategoryUserDefaults` — `UserDefaults`.

- `CA92.1` — читаешь и пишешь данные, доступные только самому приложению. Это наш случай почти всегда.
- `1C8F.1` — данные общие для приложений, расширений и App Clip одной App Group (например, приложение и виджет из главы 42).
- `C56D.1` — только для сторонних SDK, которые дают обёртку над `UserDefaults`.
- `AC6B.1` — чтение настроек MDM (управление корпоративными устройствами).

`NSPrivacyAccessedAPICategoryFileTimestamp` — даты создания и изменения файлов: `FileAttributeKey.creatio` `URLResourceKey.creationDateKey`/`.contentModificationDateKey`, `UIDocument.fileModificationDate`, низкоуровневые `stat`, `getattrlist` и родственные.

- `DDA9.1` — показать даты файлов пользователю; с устройства не отправлять.
- `C617.1` — даты, размер и другие метаданные файлов **внутри контейнера приложения**, контейнера App Group или CloudKit-контейнера приложения.
- `3B52.1` — метаданные файлов, к которым пользователь сам дал доступ (например, выбрал через document picker).
- `0A2A.1` — только для SDK-обёрток.

`NSPrivacyAccessedAPICategorySystemBootTime` — время с загрузки системы: `ProcessInfo.systemUptime`, `mach_absolute_time()`.

- `35F9.1` — измерить время между событиями в приложении или сделать таймер. Отправлять с устройства можно только сами интервалы.
- `8FFB.1` — вычислить абсолютное время событий в приложении (например, событий UIKit).
- `3D61.1` — включить в необязательный баг-репорт, который человек отправляет сам.

`NSPrivacyAccessedAPICategoryDiskSpace` — свободное и общее место: `volumeAvailableCapacityKey` и родственные ключи, `statfs`, `statvfs`.

- `85F4.1` — показать место на диске пользователю.
- `E174.1` — проверить, хватит ли места записать файл, или почистить кэш при нехватке. Поведение приложения должно заметно меняться.
- `7D9E.1` — в необязательный баг-репорт.
- `B728.1` — только для приложений медицинских исследований.

`NSPrivacyAccessedAPICategoryActiveKeyboards` — список активных клавиатур (`UITextInputMode.activeInputMo`

- `3EC4.1` — ты сам делаешь приложение-клавиатуру.
- `54BD.1` — подстроить интерфейс под активную клавиатуру.

Как понять, что код трогает эти API. Вот знакомые строки из разных глав книги, и каждая попадает в свою категорию:

```
import UIKit

func requiredReasonExamples(noteURL: URL) {
    // 1. UserDefaults — категория UserDefaults, причина CA92.1
    UserDefaults.standard.set(true, forKey: "onboarding.done")

    // 2. Дата изменения файла — категория FileTimestamp
    let attributes = try? FileManager.default.attributesOfItem(atPath: noteURL.path)
    let modified = attributes?[.modificationDate] as? Date
    print("Заметка изменена: \(String(describing: modified))")

    // 3. Время с момента загрузки системы — категория SystemBootTime
    let uptime = ProcessInfo.processInfo.systemUptime
    print("Прошло с загрузки: \(uptime) с")

    // 4. Свободное место — категория DiskSpace
    let home = URL(fileURLWithPath: NSHomeDirectory())
    let values = try? home.resourceValues(forKeys:
     ↪  [.volumeAvailableCapacityForImportantUsageKey])
    print("Свободно: \(values?.volumeAvailableCapacityForImportantUsage ?? 0) байт")
}
```

Разбор по строкам:

- `UserDefaults.standard.set` — любой вызов `UserDefaults`, даже просто флаг «онбординг показан», уже требует записи в манифесте.
- `attributesOfItem(atPath:)` сам по себе возвращает много атрибутов, но как только мы достаём `.modificationDate` — это FileTimestamp. Если заметки лежат в папке приложения и дата нужна для сортировки — `C617.1`; если дату показываем в списке — подходит и `DDA9.1`. Можно указать обе.
- `systemUptime` — например, чтобы мерить, сколько секунд пользователь был на экране. Это `35F9.1`.
- `volumeAvailableCapacityForImportantUsageKey` — перед скачиванием большого файла проверяем место. Это `E174.1`.

Если причина твоего использования в списке не найдена — значит, так использовать API нельзя. Apple предлагает отправить запрос на новую причину через форму, ссылка на неё есть в той же статье.

> **Где проверить весь проект.** После Product → Archive открой Organizer, выбери архив и в контекстном меню — **Generate Privacy Report**. Xcode соберёт манифесты приложения и всех SDK в один PDF-отчёт. По нему удобно сверять анкету App Privacy.

Упражнение. Какие записи нужны в манифесте для такого кода: приложение сохраняет тему оформления в `UserDefaults(suiteName: "group.kz.example.playground")`, которую читает и виджет? Ответ — в конце главы.

39.4 Собираемые данные в манифесте

Если приложение всё-таки собирает данные (например, email для входа в главе 8), в манифесте появляется словарь на каждый тип:

```
<key>NSPrivacyCollectedDataTypes</key>
<array>
    <dict>
        <key>NSPrivacyCollectedDataType</key>
        <string>NSPrivacyCollectedDataTypeEmailAddress</string>
        <key>NSPrivacyCollectedDataTypeLinked</key>
        <true/>
        <key>NSPrivacyCollectedDataTypeTracking</key>
        <false/>
        <key>NSPrivacyCollectedDataTypePurposes</key>
        <array>
            <string>NSPrivacyCollectedDataTypePurposeAppFunctionality</string>
        </array>
    </dict>
</array>
```

- `NSPrivacyCollectedDataType` — тип данных. Имена повторяют анкету: `...Name`, `...EmailAddress`, `...PhoneNumber`, `...PreciseLocation`, `...CrashData` и т.д.
- `NSPrivacyCollectedDataTypeLinked` — связаны ли с личностью. Email для входа — связан.
- `NSPrivacyCollectedDataTypeTracking` — используются ли для трекинга.
- `NSPrivacyCollectedDataTypePurposes` — цели: `...PurposeAppFunctionality`, `...PurposeAnalytics`, `...PurposeProductPersonalization`, `...PurposeDeveloperAdvertising`, `...PurposeThirdPartyAdvertising`, `...PurposeOther`.

Содержимое этого блока должно совпадать с анкетой App Store Connect. Анкету Apple сама из манифеста не заполняет, но Privacy Report из 39.3 помогает ничего не забыть.

39.5 Манифесты сторонних SDK

Требование к SDK написано на странице Third-party SDK requirements. Apple опубликовала список часто используемых SDK: Alamofire, AFNetworking, Firebase (Core, Auth, Crashlytics, Messaging, Firestore и другие), Facebook SDK, OneSignal, SDWebImage, Kingfisher и ещё несколько десятков. Если в новом приложении или в обновлении, которое **добавляет** такой SDK, у него нет своего манифеста, App Store Connect сборку не примет. Для SDK, подключённых как готовый бинарник (`.xcframework`), нужна ещё и **подпись** разработчика SDK.

Для SDK не из списка манифест пока не обязателен, но Apple призывает авторов его добавлять. И главное правило со страницы Apple: за код стороннего SDK в твоём приложении отвечаешь ты.

Как проверить: в навигаторе проекта раскрой Package Dependencies, найди пакет и посмотри, есть ли в нём `PrivacyInfo.xcprivacy`. Надёжнее — тот же Privacy Report из 39.3: SDK без манифеста там просто не будет. Если у SDK из списка манифеста нет — обнови его до свежей версии, почти все популярные библиотеки добавили манифесты ещё в 2024 году.

39.6 Usage description в Info.plist

Отдельно от манифеста, для каждого разрешения нужна строка-объяснение в `Info.plist` (подробно — глава 7, раздел 7.7). Эту строку человек видит в системном диалоге «Разрешить доступ к камере?».

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Чтобы показать погоду в твоём городе.</string>

<key>NSPhotoLibraryUsageDescription</key>
<string>Чтобы вставить фото в заметку.</string>

<key>NSCameraUsageDescription</key>
<string>Чтобы сделать фото для аватарки.</string>

<key>NSMicrophoneUsageDescription</key>
<string>Чтобы записать голосовое сообщение в чат.</string>

<key>NSContactsUsageDescription</key>
<string>Чтобы найти друзей, которые уже пользуются приложением.</string>

<key>NSFaceIDUsageDescription</key>
<string>Face ID защищает вход в раздел «Профиль».</string>
```

Что будет, если ключа нет: для камеры, микрофона, фото, контактов и геолокации система **завершает приложение** в момент запроса доступа — в консоли Xcode будет сообщение о нарушении приватности с именем недостающего ключа. Для Face ID документация ключа `NSFaceIDUsageDescription` называет его обязательным, если приложение пользуется Face ID.

Guidelines **5.1.1(ii)** требует, чтобы строки «ясно и полностью» описывали использование данных. «Нужно для работы приложения» — плохо: непонятно зачем. «Чтобы вставить фото в заметку» — хорошо: конкретная функция.

И ещё два пункта рядом: **5.1.1(iii)** — просить доступ только к тому, что нужно основной функции, и по возможности пользоваться системными пикерами (`PHPickerViewController` не требует доступа ко всей фотобиблиотеке). **5.1.1(iv)** — уважать отказ: если человек не дал геопозицию, предложи ввести город вручную, а не блокируй приложение.

39.7 App Tracking Transparency (ATT)

Трекинг по определению Apple — это связывание данных о человеке из твоего приложения с данными **других компаний** (чужих приложений, сайтов) для таргетированной рекламы или измерения рекламы, а также передача данных брокерам данных. Показывать свою рекламу по данным своего же приложения — не трекинг.

Если приложение трекает, нужно разрешение через фреймворк AppTrackingTransparency (Guidelines **5.1.2(i)**: «You must receive explicit permission from users via the App Tracking Transparency APIs to track their activity»). Пока человек не разрешил, рекламный идентификатор устройства (IDFA) возвращается из одних нулей: `00000000-0000-0000-0000-000000000000`.

```swift
import AppTrackingTransparency
import AdSupport

func askForTracking() async {
    // Спрашиваем только если ещё не спрашивали
    guard ATTrackingManager.trackingAuthorizationStatus == .notDetermined else { return }
    let status = await ATTrackingManager.requestTrackingAuthorization()
    switch status {
    case .authorized:
        let idfa = ASIdentifierManager.shared().advertisingIdentifier
        print("IDFA доступен: \(idfa)")
    case .denied, .restricted, .notDetermined:
        print("Трекинга нет — IDFA из одних нулей")
    @unknown default:
        break
    }
}
```

Разбор:

- `trackingAuthorizationStatus` — текущий статус без показа диалога. Если человек уже ответил, второй раз система диалог не покажет.
- `await requestTrackingAuthorization()` — async-версия (iOS 14+). Версия с замыканием тоже есть, но Swift импортирует её замыкание как `@Sendable`: оно не привязано к главному потоку. Если внутри написать `label.text = ...`, компилятор Swift 6.3 предупредит «main actor-isolated property 'text' can not be mutated from a Sendable closure» — и будет прав. Async-версия снимает вопрос: после `await` мы снова в контексте вызывающего кода (в нашем проекте — на главном акторе), и интерфейс можно трогать сразу.
- `ASIdentifierManager.shared().advertisingIdentifier` — сам IDFA из фреймворка AdSupport.
- `.restricted` — трекинг запрещён на уровне устройства (например, детский аккаунт).
- `.notDetermined` после запроса — система закрыла диалог без ответа; документация советует спросить снова позже.

Нюансы из документации метода:

- Диалог показывается, **только если приложение активно** (состояние `UIApplication.State.active`). Вызов из `application(_:didFinishLaunchingWithOptions:)` часто приходится на момент, когда приложение ещё не активно, — тогда диалога не будет, а статус останется прежним. Вызывай после того, как первый экран появился.
- Если в Настройках → Конфиденциальность и безопасность → Отслеживание выключено «Трекинг-запросы от приложений», диалога не будет вовсе.
- В ЕС после ответа повторно спросить можно через год.

Ключ в `Info.plist` обязателен, без него диалог не покажется:

```
<key>NSUserTrackingUsageDescription</key>
<string>Чтобы показывать рекламу, которая тебе интересна. Можно отказаться — приложение
 ↪  будет работать так же.</string>
```

Если трекинга нет — не вызывай ATT вообще. Диалог без причины только пугает людей, а по 5.1.2(i) приложение не может требовать разрешения на трекинг ради доступа к функциям или наградам.

39.8 Где хранить данные на устройстве

Это не требование Apple, а практика, которая напрямую влияет на то, что ты пишешь в анкете и манифесте.

Keychain (связка ключей — защищённое системное хранилище, глава 8) — для токенов, паролей и другой чувствительной мелочи. Содержимое шифруется ключами устройства. На практике записи Keychain **остаются после удаления приложения** и видны ему после переустановки — Apple не обещает это как гарантию, поэтому на это нельзя опираться, но и забыть про это нельзя: при удалении аккаунта токены нужно стирать явно (глава 40).

UserDefaults — для настроек, флагов и мелких данных интерфейса. Хранится в обычном plist-файле внутри песочницы (sandbox — отдельной папки приложения, куда другие приложения не имеют доступа). Файл не зашифрован отдельно, поэтому секреты туда класть нельзя. И, как мы видели в 39.3, `UserDefaults` — required reason API.

`Documents/` — пользовательский контент: заметки, фото, документы. Эта папка попадает в резервную копию iCloud и Finder, поэтому после восстановления из копии на новом iPhone заметки вернутся.

`Library/Caches/` — кэш, который можно потерять без последствий. В резервную копию не попадает, и iOS может очистить эту папку, когда на устройстве мало места.

`tmp/` — временные файлы на время одной операции. iOS может удалить их в любой момент, когда приложение не запущено.

39.9 Законы: GDPR, CCPA, казахстанский закон о персональных данных

Guidelines — это правила App Store. Поверх них действуют законы страны, где живут твои пользователи. Несколько примеров:

- **GDPR** — регламент ЕС о защите данных. Требует законного основания для обработки (часто — согласие), права на доступ к своим данным, на исправление и удаление.
- **CCPA/CPRA** — закон Калифорнии: право узнать, какие данные собраны, и запретить их продажу.
- **Закон Республики Казахстан «О персональных данных и их защите»** (№ 94-V от 21 мая 2013 года) — согласие на сбор и обработку, требование хранить базы с персональными данными граждан на территории Казахстана. Детали — у юриста, закон регулярно дополняют.

Что из этого точно нужно в приложении, независимо от страны, — потому что этого требует Apple:

- **Privacy Policy** — ссылка на политику конфиденциальности в App Store Connect **и** внутри приложения на видном месте (5.1.1(i)). Политика должна описывать, какие данные соби-

раются и зачем, кому передаются, сколько хранятся и как отозвать согласие или удалить данные.
- **Отзыв согласия** — понятный способ передумать (5.1.1(ii)).
- **Удаление аккаунта** внутри приложения, если в нём можно создать аккаунт (5.1.1(v), глава 40).

Для страницы в App Store есть ещё необязательное поле **Privacy Choices URL** — ссылка на страницу, где человек управляет своими данными.

Playground из книги ничего не собирает, поэтому баннер согласия ему не нужен. В реальном приложении с аналитикой и рекламой — обсуди с юристом.

39.10 Что запрещено

- **Fingerprinting** — вычислять устройство по набору признаков. Запрещено всегда, даже при разрешённом трекинге (статья о required reason API).
- **Скрытое построение профиля** из данных, которые ты называешь «анонимными», и попытки деанонимизировать людей (5.1.2(iii)).
- **Сбор списка установленных приложений** для аналитики и рекламы (5.1.2(iv)).
- **Передача данных третьим сторонам без явного согласия** — в том числе в сторонние AI-сервисы (5.1.2(i) это прямо упоминает).
- **Требовать включить пуши, геолокацию или трекинг** в обмен на доступ к функциям или награды (5.1.2(i)).

39.11 App Transport Security

App Transport Security (ATS) — механизм iOS, который по умолчанию разрешает сетевые запросы только по HTTPS с современным TLS. Отключить его для всех доменов можно так:

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
</dict>
```

В production так не делай. Исключение для одного домена выглядит так:

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSExceptionDomains</key>
    <dict>
        <key>legacy.example.com</key>
        <dict>
            <key>NSExceptionAllowsInsecureHTTPLoads</key>
            <true/>
        </dict>
    </dict>
</dict>
```

По документации Preventing insecure network connections **оба** варианта требуют обоснования при отправке в App Store и могут вызвать дополнительную проверку: и `NSAllowsArbitraryLoads`, и `NSExceptionAllowsInsecureHTTPLoads` для отдельного домена (а также `NSAllowsArbitraryLoadsForMedia`, `NSAllowsArbitraryLoadsInWebContent`, `NSExceptionMinimumTLSVersion`). Примеры допустимых причин от Apple: сервер чужой компании без HTTPS, устройства в сети, которые нельзя обновить. Правильное решение почти всегда — включить HTTPS на сервере.

39.12 Ответы к упражнениям

39.1 (анкета для Todo). Задачи хранятся только в `UserDefaults` на устройстве и никуда не отправляются — это не сбор. Демо-задачи мы **получаем** с `dummyjson.com`, но ничего о человеке туда не **передаём** (кроме технического запроса). Аналитики нет. Итог: в анкете ответ «Data Not Collected». На странице App Store появится «Разработчик не собирает данные».

Проверка себя: если позже ты добавишь Firebase Crashlytics, ответ изменится — Diagnostics → Crash Data, цель App Functionality; связаны ли с пользователем — зависит от того, передаёшь ли ты в Crashlytics User ID.

39.3 (App Group). Категория `NSPrivacyAccessedAPICategoryUserDefaults` с причиной `1C8F.1` — данные общие для приложения и расширений одной App Group. Если приложение пользуется и обычным `UserDefaults.standard`, в том же словаре указывают обе причины: `CA92.1` и `1C8F.1`. Манифест нужен и приложению, и расширению виджета — у каждого исполняемого модуля свой бандл.

```
<dict>
    <key>NSPrivacyAccessedAPIType</key>
    <string>NSPrivacyAccessedAPICategoryUserDefaults</string>
    <key>NSPrivacyAccessedAPITypeReasons</key>
    <array>
        <string>CA92.1</string>
        <string>1C8F.1</string>
    </array>
</dict>
```

Что мы выучили

- **Анкета App Privacy** в App Store Connect обязательна для отправки на проверку. «Собирать» = передавать с устройства и хранить дольше, чем нужно для ответа. Три вопроса на тип данных: цель, связь с личностью, трекинг.
- **`PrivacyInfo.xcprivacy`** — манифест в бандле: `NSPrivacyTracking`, `NSPrivacyTrackingDomains`, `NSPrivacyCollectedDataTypes`, `NSPrivacyAccessedAPITypes`. С 1 мая 2024 без причин для required reason API сборку не принимают.
- **Required reason API**: UserDefaults (`CA92.1`, `1C8F.1`), FileTimestamp (`DDA9.1`, `C617.1`, `3B52.1`), SystemBootTime (`35F9.1`, `8FFB.1`, `3D61.1`), DiskSpace (`85F4.1`, `E174.1`, `7D9E.1`), ActiveKeyboards (`3EC4.1`, `54BD.1`).
- **SDK из списка Apple** обязаны иметь манифест (и подпись, если это бинарник). Privacy Report в Organizer собирает всё в один отчёт.

- **Usage description** в `Info.plist` — конкретно «зачем» (5.1.1(ii)); без ключа доступ к камере, фото, микрофону завершает приложение.
- **ATT** — только если действительно трекаешь; async-версия запроса; диалог появляется только в активном приложении.
- **Privacy Policy** — ссылка в App Store Connect и в приложении (5.1.1(i)).
- **ATS-исключения**, включая исключение для одного домена, требуют обоснования при проверке.

Apple Developer Documentation

- App privacy details on the App Store — определения «collect», «linked», «tracking», полный список типов данных и целей.
- Privacy manifest files — структура `PrivacyInfo.xcprivacy` и как его создать.
- Describing use of required reason API — правила для required reason API и дата 1 мая 2024.
- NSPrivacyAccessedAPIType — список категорий, API и кодов причин.
- Third-party SDK requirements — список SDK, которым обязательны манифест и подпись.
- App Review Guidelines — 5.1 Privacy — политика конфиденциальности, согласие, минимизация данных, трекинг.
- App Tracking Transparency — фреймворк ATT.
- `requestTrackingAuthorization(completionHandler:)` — когда показывается диалог и когда нет.
- Preventing insecure network connections — ATS и исключения, требующие обоснования.
- Information property list — все ключи `Info.plist`, включая `NS…UsageDescription`.

→ Глава 40. Production: account deletion flow

Глава 40. Production: account deletion flow

С 30 июня 2022 года Apple требует: если в приложении можно **создать** аккаунт, в нём же должно быть можно этот аккаунт **удалить**. Не выйти, не «деактивировать», а инициировать полное удаление аккаунта вместе с данными.

Это правило самой Apple, а не пересказ закона. Оно записано в App Review Guidelines (дальше «Guidelines»), пункт **5.1.1(v) Account Sign-In**:

> If your app supports account creation, you must also offer account deletion within the app.

Подробности Apple разобрала на отдельной странице Offering account deletion in your app — дальше я ссылаюсь на неё как на «страницу Apple про удаление». Законы вроде европейского GDPR (статья 17, «право на удаление») требуют похожего, но Apple прямо пишет: если удаление есть только для жителей ЕС или Калифорнии, этого **недостаточно**, удалять аккаунт должны мочь все пользователи, где бы они ни жили.

Требование касается и «гостевых» аккаунтов, которые приложение создаёт само, без регистрации: их тоже должно быть можно удалить. И даже если регистрация идёт через сайт в браузере, кнопка удаления всё равно нужна в приложении.

40.1 Что нужно реализовать

По странице Apple про удаление:

1. **Кнопку легко найти.** Обычно — в настройках аккаунта. В нашем Profile из главы 19 — внизу экрана, под «Выйти».
2. **Удалить можно весь аккаунт** вместе с персональными данными. Можно дополнительно предложить «временно отключить», но **только** отключения недостаточно.
3. **Без звонков, писем и тикетов в поддержку.** Исключение — приложения из строго регулируемых отраслей (банки, медицина и т.п., пункт 5.1.1(ix)): им можно добавить шаги через поддержку.
4. **Можно подтверждать.** Разрешено спросить «вы уверены?», попросить ввести пароль или код из письма, чтобы аккаунт не удалил кто-то другой. Но процесс не должен быть «неоправданно сложным» — иначе отказ на проверке.
5. **Держи человека в курсе.** Если удаление занимает время (например, ручная обработка), скажи, сколько, и сообщи, когда закончено.

6. **Если нужен сайт** для завершения удаления — дай прямую ссылку на нужную страницу, а не на главную.

40.2 Кнопка в Profile

В главе 19 мы описывали экран настроек массивом секций. Добавляем секцию с двумя кнопками:

```
Section(title: nil, footer: nil, rows: [
    .button(title: "Выйти из аккаунта", isDestructive: false) { [weak self] in
        self?.logout()
    },
    .button(title: "Удалить аккаунт", isDestructive: true) { [weak self] in
        self?.confirmDelete()
    },
]),
```

`Row.button` и `Section` — из раздела 19.1: строка-кнопка с заголовком, флагом «разрушительное действие» (красный текст) и замыканием. `[weak self]` нужен, потому что массив секций хранит контроллер (`self.sections`), а замыкания лежат внутри этого массива. Без `weak` контроллер держал бы замыкание, а замыкание — контроллер: цикл сильных ссылок, и экран никогда не освободился бы из памяти.

«Выйти» — обычная синяя кнопка: после выхода аккаунт остаётся, войти можно снова. «Удалить» — красная: действие необратимое.

40.3 Подтверждение и удаление

Весь поток в коде. Сначала — вспомогательные типы. Сетевой клиент для удаления:

```
import UIKit
import StoreKit

enum APIError: Error {
    case unauthorized
    case server(Int)
}

nonisolated struct DeletionReceipt: Decodable {
    /// Когда данные будут стёрты окончательно; nil — уже стёрты.
    let purgeDate: Date?
}

enum AccountAPI {
    /// DELETE /account — сервер удаляет аккаунт или ставит его в очередь на удаление.
    static func deleteAccount(token: String) async throws -> DeletionReceipt {
        guard let url = URL(string: "https://api.example.com/account") else {
            throw APIError.server(0)
```

```
        }
        var request = URLRequest(url: url)
        request.httpMethod = "DELETE"
        request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")

        let (data, response) = try await URLSession.shared.data(for: request)
        guard let http = response as? HTTPURLResponse else { throw APIError.server(0) }

        switch http.statusCode {
        case 200, 202:
            let decoder = JSONDecoder()
            decoder.dateDecodingStrategy = .iso8601
            return try decoder.decode(DeletionReceipt.self, from: data)
        case 401:
            throw APIError.unauthorized
        default:
            throw APIError.server(http.statusCode)
        }
    }
}
```

Разбор:

- `DELETE /account` с токеном в заголовке `Authorization` — сервер понимает, чей аккаунт удалять, из токена. Никаких `userId` в теле запроса: иначе злоумышленник мог бы подставить чужой.
- Ответ `200 OK` — удалено сразу. `202 Accepted` («принято к исполнению») — сервер поставил удаление в очередь; в теле он возвращает дату, когда данные будут стёрты окончательно.
- `nonisolated struct DeletionReceipt` — в нашем проекте весь код по умолчанию на главном акторе (введение, раздел 0.1). Модель, которую декодируют из JSON, помечаем `nonisolated`, чтобы её можно было спокойно передавать между потоками.
- `dateDecodingStrategy = .iso8601` — сервер присылает дату строкой вида `"2026-10-27T00:00:00Z"`.
- `guard let url` вместо `URL(string:)!` — восклицательный знак (force unwrap) уронил бы приложение, если бы в адресе случайно оказался недопустимый символ.

Локальная очистка — отдельной функцией:

```
enum LocalData {
    static func wipe() {
        // 1. Токен из Keychain (глава 8)
        AuthStorage.shared.clear()

        // 2. Все настройки приложения в UserDefaults
        if let bundleID = Bundle.main.bundleIdentifier {
            UserDefaults.standard.removePersistentDomain(forName: bundleID)
        }

        // 3. Файлы пользователя: Documents и Caches
```

```
        let fm = FileManager.default
        let folders = [
            fm.urls(for: .documentDirectory, in: .userDomainMask).first,
            fm.urls(for: .cachesDirectory, in: .userDomainMask).first,
        ].compactMap { $0 }

        for folder in folders {
            let items = (try? fm.contentsOfDirectory(at: folder,
            ↪  includingPropertiesForKeys: nil)) ?? []
            for item in items {
                try? fm.removeItem(at: item)
            }
        }
    }
}
```

- `AuthStorage.shared.clear()` — наш Keychain-класс из главы 8. Токен нужно стереть явно: записи Keychain на практике переживают даже удаление приложения.
- `removePersistentDomain(forName:)` — удаляет **все** ключи `UserDefaults.standard` этого приложения разом. Домен (*domain*) здесь — «раздел» настроек; у приложения он называется по bundle ID.
- Папки мы не удаляем, а **очищаем**: `Documents` и `Caches` — системные папки песочницы, их содержимое можно удалять, а саму папку лучше оставить.

Теперь сам контроллер:

```
final class ProfileViewController: UIViewController {

    /// Координатор (глава 4) подставляет сюда переход на экран входа.
    var onAccountDeleted: (() -> Void)?

    func confirmDelete() {
        let alert = UIAlertController(
            title: "Удалить аккаунт?",
            message: "Мы удалим профиль, заметки и фото с сервера и с этого устройства.
↪   Отменить это нельзя.",
            preferredStyle: .alert
        )
        alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
        alert.addAction(UIAlertAction(title: "Удалить", style: .destructive) { [weak self]
↪   _ in
            self?.performDelete()
        })
        present(alert, animated: true)
    }

    private func performDelete() {
        guard let token = AuthStorage.shared.token else {
            // Не вошёл — удалять на сервере нечего, чистим устройство
```

```
            LocalData.wipe()
            onAccountDeleted?()
            return
        }

        let spinner = UIActivityIndicatorView(style: .large)
        spinner.center = CGPoint(x: view.bounds.midX, y: view.bounds.midY)
        view.addSubview(spinner)
        spinner.startAnimating()
        view.isUserInteractionEnabled = false

        Task { [weak self] in
            do {
                let receipt = try await AccountAPI.deleteAccount(token: token)
                LocalData.wipe()
                spinner.removeFromSuperview()
                self?.view.isUserInteractionEnabled = true
                self?.showDone(receipt: receipt)
            } catch {
                spinner.removeFromSuperview()
                self?.view.isUserInteractionEnabled = true
                self?.showError()
            }
        }
    }

    private func showDone(receipt: DeletionReceipt) {
        var message = "Аккаунт удалён."
        if let date = receipt.purgeDate {
            let day = date.formatted(date: .long, time: .omitted)
            message = "Аккаунт отключён. Все данные будут окончательно стёрты \(day)."
        }
        let alert = UIAlertController(title: "Готово", message: message, preferredStyle:
        ↪  .alert)
        alert.addAction(UIAlertAction(title: "OK", style: .default) { [weak self] _ in
            self?.onAccountDeleted?()
        })
        present(alert, animated: true)
    }

    private func showError() {
        let alert = UIAlertController(
            title: "Не получилось удалить аккаунт",
            message: "Проверь интернет и попробуй ещё раз. Данные на устройстве мы не
↪  трогали.",
            preferredStyle: .alert
        )
        alert.addAction(UIAlertAction(title: "OK", style: .cancel))
```

```
        present(alert, animated: true)
    }
}
```

Разбор по шагам:

- `confirmDelete()` — одно подтверждение. Текст говорит, **что именно** удалится и что это необратимо. Кнопка «Удалить» в стиле `.destructive` (красная), «Отмена» — `.cancel`.
- `performDelete()` сначала проверяет токен. Нет токена — человек не вошёл, серверного аккаунта нет, чистим только устройство.
- Центр спиннера — середина **`bounds`** нашего view, а не `view.center`. `center` — это центр `frame`, он задан в координатах superview; `bounds` — своя система координат view, и именно в ней живут subviews (глава 0, раздел 0.3). У корневого view экрана на весь экран эти точки совпадают, поэтому ошибка `spinner.center = view.center` долго не замечается. Но стоит вложить этот экран как child view controller со сдвигом — и спиннер уедет: на симуляторе у view с `frame.origin.y = 400` и высотой 452 `center.y` = 626, а `bounds.midY` = 226; спиннер, поставленный в `center`, оказался за нижней границей view.
- Спиннер и `isUserInteractionEnabled = false` — пока идёт запрос, касания к этому view и всем его subviews не доставляются (глава 0, раздел 0.3), и человек не нажмёт «Удалить» дважды. Панель навигации в это не входит: `UINavigationBar` лежит не внутри `view` экрана, а рядом, в контейнере `UINavigationController` (на симуляторе `navigationBar.isDescendant(of: view)` даёт `false`), так что кнопка «Назад» останется живой — если это важно, на время запроса поставь `navigationItem.hidesBackButton = true`.
- `Task { ... }` — запускаем асинхронную работу. Контроллер в нашем режиме на главном акторе, поэтому код внутри `Task` тоже выполняется на главном потоке, и трогать `view` после `await` безопасно.
- `[weak self]` в `Task` — если пользователь закроет экран, пока идёт запрос, контроллер не будет удерживаться до конца запроса.
- **Порядок важен:** сначала сервер, потом устройство. Если запрос упал, мы **не** стираем локальные данные и показываем ошибку. С `try? await API.deleteAccount()` и сразу очисткой при отсутствии сети человек думал бы, что аккаунт удалён, хотя на сервере всё осталось.
- `showDone` различает два исхода: удалено сразу или поставлено в очередь с датой. Это требование страницы Apple: сказать, сколько займёт удаление.
- `onAccountDeleted` — вместо `dismiss` переход на экран входа отдаём координатору. Контроллер не знает, кто его показал и куда вести дальше.

Упражнение. Добавь в `showError` кнопку «Повторить», которая снова вызывает `performDelete()`. Подумай, нужен ли в ней `[weak self]`. Решение — в конце главы.

40.4 Серверная часть: что значит «удалить»

Серверный код — за рамками книги, но от него зависит, пройдёт ли приложение проверку. По странице Apple про удаление:

- **Удалить нужно аккаунт и все связанные данные**, которые ты не обязан хранить по закону.

- **Контент, который видели другие, тоже удаляется**: фото, видео, посты, отзывы. «Люди ожидают, что все данные, связанные с аккаунтом, будут удалены». Если закон обязывает что-то хранить — предупреди об этом пользователя.
- **Удаление не обязано быть мгновенным.** Если процесс ручной или занимает время — это допустимо, если ты сообщил срок и уведомил о завершении. Срок должен соответствовать местным законам.

Что обычно **остаётся** на законных основаниях:

- **Финансовые документы** — чеки и транзакции: бухгалтерию хранят по закону несколько лет.
- **Журналы безопасности** — в объёме, который требует закон.
- **Обезличенная статистика** — если она действительно не связана с человеком (например, «в сентябре было 1200 заказов»).

Что удаляется точно: запись пользователя, имя, email, телефон, контент, настройки, все активные сессии и токены, токены пушей (глава 41), подписки на рассылки, сохранённые токены карт у платёжного провайдера.

40.5 Мягкое и жёсткое удаление

Жёсткое удаление (*hard delete*) — строки из базы стираются сразу.

Мягкое удаление (*soft delete*) — в строке пользователя ставится отметка времени `deleted_at`, аккаунт перестаёт работать, а через N дней фоновая задача стирает его окончательно:

```
-- В момент запроса: аккаунт отключён, вход невозможен
UPDATE users SET deleted_at = NOW() WHERE id = 42;

-- Раз в сутки: окончательно стираем тех, кто удалён больше 30 дней назад
DELETE FROM users WHERE deleted_at < NOW() - INTERVAL '30 days';
```

На числах: человек удалил аккаунт 1 октября в 10:00. Проверка «прошло больше 30 дней» станет истинной после 31 октября, 10:00, и ближайший ночной запуск задачи сотрёт запись.

Apple **не требует** и **не рекомендует** ни один из вариантов. Мягкое удаление с отсрочкой допустимо, если:

- аккаунт сразу перестаёт работать;
- ты **сказал** человеку, когда данные будут стёрты окончательно (наш `purgeDate` в `showDone`);
- по истечении срока данные действительно удаляются.

Не путай это с «деактивацией»: если после «удаления» данные хранятся вечно, это и есть деактивация, а её одной, по странице Apple, недостаточно.

40.6 Восстановление в течение отсрочки

Если у тебя мягкое удаление, можно дать передумать. Сервер при попытке входа удалённого, но ещё не стёртого аккаунта отвечает особым кодом, например `409 Conflict` с датой окончательного удаления, а приложение показывает:

```
┌──────────────────────────────┐
│   Этот аккаунт удалён        │
│                              │
│   Данные будут стёрты        │
│   27 октября 2026 года.      │
│   До этого его можно         │
│   восстановить.              │
│                              │
│   [ Восстановить ]           │
│   [ Создать новый ]          │
└──────────────────────────────┘
```

После даты окончательного удаления сервер восстановить уже не может — данных нет.

40.7 Sign in with Apple — отзыв токенов

Если в приложении есть вход через Apple (Sign in with Apple), страница Apple про удаление требует при удалении аккаунта **отозвать токены** через Sign in with Apple REST API. После отзыва приложение пропадает из списка «Приложения, использующие Apple ID» в настройках человека, и при новом входе он увидит экран согласия заново, как новый пользователь.

Отзыв делает **сервер**, а не приложение: запрос `POST https://appleid.apple.com/auth/revoke` с параметрами `client_id` (твой идентификатор приложения), `client_secret` (JWT, подписанный ключом Sign in with Apple — секрет живёт только на сервере), `token` (refresh-токен или access-токен пользователя) и `token_type_hint`.

Где сервер возьмёт токен пользователя? При входе приложение получает от Apple **authorization code** (`ASAuthorizationAppleIDCredential.authorizationCode`) и отправляет его на сервер, а сервер обменивает код на refresh-токен и хранит его. Если сервер этого не делал, приложению придётся перед удалением попросить человека ещё раз войти через Apple — чтобы получить свежий authorization code и отдать его серверу для обмена и отзыва.

В приложении можно дополнительно проверить, не отозвал ли человек доступ сам (через настройки Apple ID):

```swift
import AuthenticationServices

func checkAppleCredential(userID: String) async -> Bool {
    let provider = ASAuthorizationAppleIDProvider()
    do {
        let state = try await provider.credentialState(forUserID: userID)
        return state == .authorized
    } catch {
        return false
    }
}
```

`credentialState(forUserID:)` возвращает `.authorized`, `.revoked` или `.notFound`. Если `.revoked` — человек сам отключил приложение от Apple ID; сессию в приложении стоит завершить. Этот метод **не отзывает** токены — он только читает состояние.

40.8 Подписки

Если у человека активная автовозобновляемая подписка через App Store, удаление аккаунта её **не отменяет**: платит он Apple, а не тебе. Страница Apple про удаление говорит так: предупреди, что списания продолжатся через Apple, и попроси отменить подписку **до** удаления. Можно предложить запланировать удаление на дату окончания подписки, но только если **есть и вариант удалить сразу**.

Как открыть системный экран управления подписками (iOS 15+):

```
extension ProfileViewController {

    func hasActiveSubscription() async -> Bool {
        for await result in Transaction.currentEntitlements {
            if case .verified(let transaction) = result,
               transaction.productType == .autoRenewable,
               transaction.revocationDate == nil {
                return true
            }
        }
        return false
    }

    func openSubscriptions() {
        guard let scene = view.window?.windowScene else { return }
        Task {
            do {
                try await AppStore.showManageSubscriptions(in: scene)
            } catch {
                if let url = URL(string: "https://apps.apple.com/account/subscriptions") {
                    await UIApplication.shared.open(url)
                }
            }
        }
    }

    func warnAboutSubscription() {
        let alert = UIAlertController(
            title: "Подписка активна",
            message: "Удаление аккаунта не отменяет подписку: списания через Apple
↪  продолжатся. Сначала отмени её, потом удаляй аккаунт.",
            preferredStyle: .alert
        )
        alert.addAction(UIAlertAction(title: "Управлять подписками", style: .default) {
↪  [weak self] _ in
            self?.openSubscriptions()
        })
        alert.addAction(UIAlertAction(title: "Удалить всё равно", style: .destructive) {
↪  [weak self] _ in
```

```
            self?.confirmDelete()
        })
        alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
        present(alert, animated: true)
    }
}
```

Разбор:

- `Transaction.currentEntitlements` (StoreKit 2, iOS 15+) — асинхронная последовательность текущих прав пользователя: активные подписки и купленные товары. Перебираем через `for await`.
- `.verified(let transaction)` — StoreKit проверил подпись транзакции. Непроверенные (`.unverified`) пропускаем.
- `productType == .autoRenewable` и `revocationDate == nil` — это автовозобновляемая подписка, и Apple её не отзывала (например, после возврата денег).
- `AppStore.showManageSubscriptions(in:)` — системный лист управления подписками прямо поверх приложения. Ему нужна сцена окна (`UIWindowScene`), берём её у `view.window`. Сцена — это не view: `UIWindowScene → UIScene → UIResponder`, она владеет окнами (`windows`) и экраном (`screen`), а окно `UIWindow → UIView` знает свою сцену через `windowScene` (глава 0, раздел 0.7). Пока view не в окне, `view.window` — `nil`, и `guard` просто выйдет.
- Если лист показать не удалось, открываем ссылку `https://apps.apple.com/account/subscriptions` — её Apple приводит на странице про удаление. Схема `itms-apps://` тоже откроет App Store, но официальная ссылка — https.
- `warnAboutSubscription()` показывается **вместо** `confirmDelete()`, если `hasActiveSubscription()` вернул `true`. Вариант «Удалить всё равно» обязателен — нельзя заставлять человека сначала отменять подписку.

Для возврата денег за покупки у StoreKit есть `Transaction.beginRefundRequest(for:in:)` (iOS 15+) — системный лист запроса возврата. Предлагать его при удалении не обязательно, Apple упоминает его как возможность.

40.9 Что Apple не пропустит

По Guidelines 5.1.1(v) и странице Apple про удаление:

- **«Напишите в поддержку»**, «позвоните», «отправьте email» — нельзя (кроме строго регулируемых отраслей).
- **Только деактивация** без удаления данных — недостаточно.
- **«Удалите приложение»** — это не удаление аккаунта, данные остаются на сервере.
- **Удаление только для отдельных стран** — недостаточно.
- **Неоправданно сложный путь**: пять экранов, капча, опрос «почему уходите» без кнопки «пропустить».

Что можно:

- Подтверждение, повторный ввод пароля, код из письма.
- Ссылка на страницу сайта, если удаление завершается там, — **прямая** ссылка на эту страницу, а начинается процесс всё равно в приложении.

- Отсрочка с понятной датой окончательного удаления.

Про оформление есть раздел HIG (Human Interface Guidelines — руководство Apple по дизайну интерфейсов) Managing accounts. Из него стоит взять три мысли: ссылку на удаление не прятать в политику конфиденциальности или условия использования; удаление в приложении и на сайте должно быть одинаково простым; если закон обязывает что-то хранить (например, медицинские записи), прямо объясни это человеку.

40.10 Тестирование

Чек-лист перед отправкой:

- ☐ Кнопка «Удалить аккаунт» видна в настройках аккаунта без поиска.
- ☐ Подтверждение показывается, «Отмена» ничего не удаляет.
- ☐ Запрос к серверу отправляется с токеном; на сервере аккаунт перестаёт работать сразу.
- ☐ Нет сети → ошибка, локальные данные на месте.
- ☐ После успеха: токен в Keychain удалён, `UserDefaults` пуст, `Documents` и `Caches` пусты, приложение на экране входа.
- ☐ Повторный вход удалённым аккаунтом → понятное сообщение (или экран восстановления из 40.6).
- ☐ Для входа через Apple: после удаления приложение пропало из списка в настройках Apple ID.
- ☐ С активной подпиской: показывается предупреждение и открывается экран управления подписками.
- ☐ В заметках для проверяющего (App Review Notes) — тестовый аккаунт, который не жалко удалить, и где искать кнопку. Проверяющий действительно нажмёт «Удалить».

40.11 Политика конфиденциальности

Guidelines **5.1.1(i)** требует, чтобы политика конфиденциальности объясняла, как долго хранятся данные, как их удалить и как отозвать согласие. Для удаления аккаунта напиши там:

- где в приложении кнопка удаления;
- сколько времени занимает удаление и что происходит с данными в это время;
- что остаётся после удаления и почему (например, чеки — по налоговому закону).

40.12 Ответы к упражнениям

40.3. Кнопка «Повторить»:

```
private func showError() {
    let alert = UIAlertController(
        title: "Не получилось удалить аккаунт",
        message: "Проверь интернет и попробуй ещё раз. Данные на устройстве мы не
↪   трогали.",
        preferredStyle: .alert
    )
    alert.addAction(UIAlertAction(title: "Отмена", style: .cancel))
```

```
    alert.addAction(UIAlertAction(title: "Повторить", style: .default) { [weak self] _ in
        self?.performDelete()
    })
    present(alert, animated: true)
}
```

`[weak self]` здесь нужен не из-за цикла: алерт не хранится в контроллере, так что цикла ссылок нет. Но замыкание кнопки живёт, пока живёт алерт, а с `weak` нажатие «Повторить» после закрытия экрана просто ничего не сделает, вместо того чтобы отправлять запрос от имени уже ненужного контроллера. Это дешёвая привычка, и в книге мы её держим во всех замыканиях `UIAlertAction`.

Что мы выучили

- **5.1.1(v)**: если можно создать аккаунт, в приложении должно быть можно его удалить. Требование с 30 июня 2022 года, для всех стран, включая гостевые аккаунты.
- Только деактивации мало; звонки, письма и поддержка — нельзя (кроме регулируемых отраслей); подтверждение и повторный вход — можно.
- **Сначала сервер, потом устройство.** Упал запрос — ничего не стираем и показываем ошибку.
- Спиннер ставим в середину `bounds`, а не в `view.center` (`center` — в координатах superview); `isUserInteractionEnabled = false` закрывает view и его subviews, но не панель навигации.
- Локально: Keychain, `removePersistentDomain(forName:)`, содержимое `Documents` и `Caches`.
- Удаляется и контент, который видели другие. Храним только то, что требует закон, и сообщаем об этом.
- Отсрочка допустима, если человек знает дату окончательного удаления.
- **Sign in with Apple**: сервер отзывает токены через `POST https://appleid.apple.com/auth/revoke`. `credentialState` только читает состояние.
- **Подписки** не отменяются удалением аккаунта: предупреди и открой `AppStore.showManageSubscription`, но оставь вариант удалить сразу.

Apple Developer Documentation

- Offering account deletion in your app — требования и FAQ Apple по удалению аккаунта.
- App Review Guidelines — 5.1.1 Data Collection and Storage — пункт (v) об удалении аккаунта и (i) о политике конфиденциальности.
- HIG — Managing accounts — рекомендации по интерфейсу входа и удаления аккаунта.
- Revoke tokens — серверный запрос отзыва токенов Sign in with Apple.
- `ASAuthorizationAppleIDProvider` — проверка состояния учётных данных Apple ID.
- `showManageSubscriptions(in:)` — системный экран управления подписками.
- `Transaction.currentEntitlements` — текущие подписки и покупки пользователя.
- `beginRefundRequest(for:in:)` — системный лист запроса возврата.
- `removePersistentDomain(forName:)` — очистка всех настроек приложения.
- UIView.center — центр `frame` в координатах superview; для subviews используй середину `bounds`.
- Keychain services — удаление токенов и паролей из связки ключей.

→ Глава 41. Production: push notifications + deep links

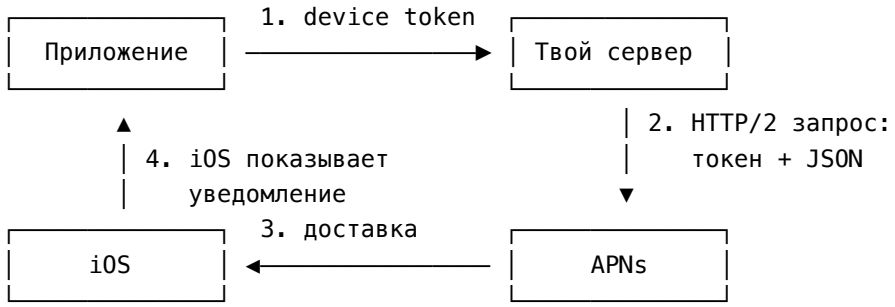
Глава 41. Production: push notifications + deep links

Два связанных сценария. **Push-уведомление** (дальше просто «пуш») — сообщение, которое твой сервер отправляет на телефон через сервис Apple, даже когда приложение закрыто. **Deep link** («глубокая ссылка») — ссылка, которая открывает не просто приложение, а конкретный экран внутри него: чат №1234, профиль пользователя, товар. Связаны они потому, что тап по пушу почти всегда должен вести на конкретный экран — то есть пуш несёт в себе deep link.

Факты о сервисе пушей сверены с документацией Apple (раздел User Notifications) в сентябре 2026 года.

41.1 Как устроена доставка пуша

Участников четыре:



- **APNs** (Apple Push Notification service) — сервис Apple, через который идут все пуши на устройства Apple. Напрямую на телефон твой сервер ничего отправить не может.
- **Device token** («токен устройства») — адрес твоего приложения на конкретном телефоне. Его выдаёт APNs при регистрации. Токен уникален для пары «приложение + устройство»: у двух твоих приложений на одном iPhone токены разные.
- **Provider server** — в терминах Apple это твой сервер, который формирует пуш и отправляет его в APNs.

Аналогия: APNs — почта, device token — почтовый адрес, твой сервер — отправитель. Адрес ты узнаёшь от самого получателя (приложение сообщает токен серверу), а доставляет всегда почта.

41.2 Настройка в Apple Developer и ключ `.p8`

1. Открой developer.apple.com/account → **Certificates, Identifiers & Profiles** → **Identifiers**, выбери App ID приложения и включи **Push Notifications**. (Если включишь capability в Xcode при автоматической подписи, Xcode сделает это сам.)
2. Там же, в разделе **Keys**, создай ключ с галочкой **Apple Push Notifications service (APNs)**. Apple выдаст:
   - файл ключа с расширением `.p8` — его можно скачать **только один раз**;
   - **Key ID** — строка из 10 символов;
   - понадобится ещё **Team ID** — идентификатор твоей команды разработчика (виден в Membership details).
3. Файл `.p8`, Key ID, Team ID и bundle ID приложения отдай тем, кто делает сервер. Ключ — секрет: кто им владеет, может слать пуши твоим пользователям.

Что сервер делает с ключом. Он подписывает им короткий JSON-токен (JWT — JSON Web Token), в котором Team ID и время создания, и кладёт его в заголовок `authorization: bearer <токен>` каждого запроса к APNs. По документации Establishing a token-based connection to APNs этот JWT нужно обновлять **не реже раза в 60 минут и не чаще раза в 20 минут**: APNs отклоняет токен старше часа с ошибкой `ExpiredProviderToken (403)`.

Сам ключ `.p8` не истекает — его можно только отозвать. Сейчас Apple предлагает два вида ключей:

- **Team-scoped** — на все приложения команды, но для **одной** среды (Sandbox или Production), максимум два ключа на среду;
- **Topic-specific** — только для выбранных приложений.

Старые ключи, которые работают сразу в обеих средах, продолжают работать, но Apple советует разделять ключи по средам.

Альтернатива ключу — **сертификат** `.p12` для конкретного приложения. Он действует **год**, потом его нужно выпускать заново, иначе пуши перестанут доходить. Поэтому для новых проектов бери ключ.

41.3 Две среды APNs: sandbox и production

У APNs два сервера:

- **development (sandbox)** — `api.sandbox.push.apple.com:443`;
- **production** — `api.push.apple.com:443`.

Какая среда у сборки, решает entitlement `aps-environment`. **Entitlement** — это разрешение, вшитое в подпись приложения: «этому приложению можно получать пуши, в такой-то среде». По документации ключа aps-environment Xcode ставит значение по профилю подписи (provisioning profile — файл Apple, который связывает приложение, команду и устройства):

Как установлена сборка	`aps-environment`	Сервер APNs
Запуск из Xcode на своём iPhone	`development`	sandbox
TestFlight	`production`	production
App Store	`production`	production

TestFlight — сервис Apple для бета-тестирования: сборку загружают в App Store Connect, а тестировщики ставят её через приложение TestFlight ещё до выхода в App Store.

Ошибка номер один на практике: сборка из TestFlight регистрируется в production, а сервер шлёт в sandbox — или наоборот. Токен из одной среды в другой не работает, APNs отвечает ошибкой `BadDeviceToken`. Поэтому вместе с токеном отправляй на сервер и среду (например, по флагу сборки `DEBUG`).

41.4 Xcode: capability

В Xcode → таргет приложения → **Signing & Capabilities** → **+ Capability**:

1. **Push Notifications** — обязательно. Добавляет в файл `.entitlements` ключ `aps-environment`.
2. **Background Modes** → галочка **Remote notifications** — **только** если нужны тихие (фоновые) пуши из 41.9. Для обычных видимых пушей это не требуется.

Capability («возможность») в Xcode — переключатель, который одновременно добавляет entitlement в подпись и включает соответствующую функцию у App ID.

41.5 Регистрация и device token

Регистрация и запрос разрешения — две **разные** вещи:

- `registerForRemoteNotifications()` — получить device token. Работает без разрешения пользователя (тихие пуши приходят и без него).
- `requestAuthorization(options:)` — спросить разрешение **показывать** уведомления: баннер, звук, бейдж.

По документации Registering your app with APNs регистрироваться нужно **при каждом запуске**: APNs выдаёт новый токен после восстановления из резервной копии, установки на новое устройство и переустановки системы. Там же прямо сказано: не кэшируй токен локально.

```
import UIKit
import UserNotifications

enum API {
    static func registerDevice(token: String) async throws { /* POST на сервер */ }
}

final class AppDelegate: UIResponder, UIApplicationDelegate {

    func application(_ application: UIApplication,
                     didFinishLaunchingWithOptions launchOptions:
                     ↪  [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        // Делегат ставим до конца запуска — иначе тап по пушу,
        // который запустил приложение, может потеряться
        UNUserNotificationCenter.current().delegate = self
        // Токен — при каждом запуске
        application.registerForRemoteNotifications()
        return true
```

```
    }

    /// Вызываем из экрана-объяснения (primer), а не при запуске
    func requestPushPermission() async -> Bool {
        do {
            return try await UNUserNotificationCenter.current()
                .requestAuthorization(options: [.alert, .sound, .badge])
        } catch {
            print("Ошибка запроса разрешения: \(error)")
            return false
        }
    }

    func application(_ application: UIApplication,
                     didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
        let token = deviceToken.map { String(format: "%02x", $0) }.joined()
        Task {
            try? await API.registerDevice(token: token)
        }
    }

    func application(_ application: UIApplication,
                     didFailToRegisterForRemoteNotificationsWithError error: Error) {
        print("Не удалось зарегистрироваться в APNs: \(error)")
    }
}
```

Разбор:

- `UNUserNotificationCenter.current().delegate = self` — в `didFinishLaunching`. В документации свойства `delegate` сказано, что делегат нужно установить до того, как приложение закончит запуск — в `application(_:willFinishLaunchingWithOptions:)` или `application(_:didFinishLaunchingWithOptions:)`: иначе событие «пользователь тапнул уведомление, и оно запустило приложение» некому будет получить. Сам делегат, как и в UIKit, хранится слабой ссылкой (`weak` в `UNUserNotificationCenter.h`); `AppDelegate` живёт всё время работы приложения, поэтому проблем с этим нет.
- `registerForRemoteNotifications()` — асинхронный запрос к APNs. Результат придёт в один из двух методов ниже.
- `requestPushPermission()` — отдельно, чтобы вызвать его в удачный момент, после объяснения (41.6). `async`-версия возвращает `true`, если человек разрешил.
- `deviceToken: Data` — 32 байта (сейчас; Apple не гарантирует длину). `map { String(format: "%02x", $0) }` превращает каждый байт в две шестнадцатеричные цифры: байт 255 → `"ff"`, байт 10 → `"0a"`. `joined()` склеивает — получается строка из 64 символов, которую ждёт сервер.
- `didFailToRegister...` — на симуляторе без поддержки удалённых пушей, без сети или без entitlement сюда приходит ошибка. Документация советует запомнить это и попробовать позже.

Режим Swift 6 и `MainActor`. Протокол `UNUserNotificationCenterDelegate` в заголовке SDK **не** помечен `@MainActor`. Наш `AppDelegate` — на главном акторе (режим по умолчанию из введения), поэтому Swift 6.2+ делает его соответствие протоколу **изолированным**: методы делегата рассчитаны на вызов с главного потока. Код ниже собирается в режиме Swift 6 без ошибок. Если бы система вызвала делегат не с главного потока, в режиме Swift 6 проверка изоляции остановила бы приложение, а не дала бы тихую гонку данных. Хочешь убедиться у себя — добавь в `willPresent` строку `print(Thread.isMainThread)` и отправь пуш командой из 41.15.

41.6 Объяснение перед запросом разрешения

Apple не требует показывать собственный экран-объяснение (*primer*, глава 7) перед системным диалогом пушей. Но системный диалог показывается **один раз**: если человек нажал «Не разрешать», вернуть разрешение он сможет только в Настройках. Поэтому сначала объясни выгоду своими словами:

«Включи уведомления, чтобы: - знать о новых сообщениях в чате; - получать напоминания о задачах со сроком; - узнать, когда закончилась долгая загрузка».

И только после «Включить» на твоём экране — `requestPushPermission()`.

Требования Guidelines:

- **4.5.4** — пуши **не должны быть обязательными** для работы приложения и не должны нести конфиденциальную информацию. Рекламу и маркетинг пушами можно слать, только если человек **явно согласился** на это в интерфейсе приложения и может там же отказаться.
- **5.1.2(i)** — нельзя требовать включить пуши в обмен на доступ к функциям или награды.

Для рекламных пушей это значит: отдельный переключатель «Акции и скидки» в настройках (как в Profile, глава 19), выключенный по умолчанию.

41.7 Содержимое пуша (payload)

Сервер отправляет в APNs JSON — *payload* («полезная нагрузка»):

```
{
  "aps": {
    "alert": {
      "title": "Новое сообщение",
      "body": "Айдос: Привет! Когда встречаемся?"
    },
    "badge": 5,
    "sound": "default"
  },
  "deep_link": "myapp://chat/1234"
}
```

- `aps` — словарь, который читает iOS. Все системные ключи — внутри него.
- `aps.alert.title` и `aps.alert.body` — заголовок и текст уведомления.
- `aps.badge` — число на иконке приложения. `0` убирает бейдж.

- `aps.sound` — `"default"` для системного звука или имя файла из бандла.
- `deep_link` — наш собственный ключ **вне** `aps`. iOS его не трогает, приложение прочитает его из `userInfo`.

Ограничение размера — **4 КБ** (4096 байт) на весь JSON. Картинки в пуш не кладут: кладут ссылку, а скачивает её расширение из 41.12.

Кроме тела, сервер указывает заголовки HTTP/2-запроса:

- `apns-topic` — bundle ID приложения;
- `apns-push-type` — `alert` для видимого пуша, `background` для тихого. Документация просит отправлять его с каждым пушем;
- `apns-priority` — `10` (доставить сразу) или `5` (можно отложить для экономии батареи). Для `background` — только `5`.

41.8 Получение: приложение открыто и тап по уведомлению

```
extension AppDelegate: UNUserNotificationCenterDelegate {

    // Пуш пришёл, когда приложение на экране
    func userNotificationCenter(_ center: UNUserNotificationCenter,
                                willPresent notification: UNNotification,
                                withCompletionHandler completionHandler: @escaping
                                 ↪  (UNNotificationPresentationOptions) -> Void) {
        completionHandler([.banner, .list, .sound, .badge])
    }

    // Пользователь тапнул уведомление (или кнопку в нём)
    func userNotificationCenter(_ center: UNUserNotificationCenter,
                                didReceive response: UNNotificationResponse,
                                withCompletionHandler completionHandler: @escaping () ->
                                 ↪  Void) {
        let userInfo = response.notification.request.content.userInfo
        if let deepLink = userInfo["deep_link"] as? String,
           let url = URL(string: deepLink) {
            DeepLinkRouter.shared.open(url)
        }
        completionHandler()
    }
}
```

`willPresent` вызывается, только если приложение **на экране** (foreground). По умолчанию в этом состоянии iOS пуш **не показывает** — считается, что человек и так видит приложение. Параметр `completionHandler` решает, что сделать:

- `.banner` — показать баннер сверху (iOS 14+);
- `.list` — оставить в Центре уведомлений (iOS 14+);
- `.sound`, `.badge` — звук и число на иконке;

- `[]` — ничего не показывать. Удобно, если человек сейчас в том же чате, куда пришло сообщение.

`didReceive` вызывается, когда человек **тапнул** уведомление: на заблокированном экране, в Центре уведомлений или баннер. Если приложение было закрыто, iOS сначала запустит его, а потом вызовет этот метод — вот почему делегат ставится в `didFinishLaunching`.

`DeepLinkRouter` — наш маршрутизатор ссылок из 41.10: одна точка, куда сходятся ссылки из пушей, URL-схем и Universal Links.

`completionHandler()` — обязательно вызвать, когда обработка закончена.

В SDK есть и `async`-варианты этих методов. Мы берём версии с `completionHandler`, чтобы явно видеть момент, когда сообщаем системе «готово».

Упражнение. Сделай так, чтобы пуш про чат `1234` **не показывался**, если человек сейчас открыт именно в этом чате. Подсказка: где-то должно храниться «текущий открытый чат». Решение — в конце главы.

41.9 Тихие (фоновые) пуши

Пуш без баннера и звука — чтобы разбудить приложение и обновить данные:

```
{
  "aps": {
    "content-available": 1
  },
  "type": "sync",
  "task_id": "123"
}
```

По документации Pushing background updates to your app:

- в `aps` только `content-available: 1` — никаких `alert`, `sound`, `badge`;
- заголовки `apns-push-type: background` и `apns-priority: 5`;
- в приложении нужен Background Modes → Remote notifications (41.4);
- система считает такие пуши низкоприоритетными: доставка **не гарантирована**, их могут придержать и объединить. Apple советует не больше **двух-трёх в час**;
- если человек принудительно закрыл приложение смахиванием, отложенный тихий пуш система выбросит.

iOS будит приложение и вызывает:

```
extension AppDelegate {
    func application(_ application: UIApplication,
                     didReceiveRemoteNotification userInfo: [AnyHashable: Any],
                     fetchCompletionHandler completionHandler: @escaping
                      ↪ (UIBackgroundFetchResult) -> Void) {
        Task {
            let changed = await syncData()
            completionHandler(changed ? .newData : .noData)
        }
```

```
    }

    func syncData() async -> Bool {
        // Загрузить изменения с сервера и сохранить локально
        return true
    }
}
```

На всю работу у приложения **30 секунд**. `completionHandler` нужно вызвать обязательно и честно: `.newData` — что-то загрузили, `.noData` — нечего было, `.failed` — ошибка. Система учитывает, сколько времени и энергии тратит приложение на фоновую работу, и может реже будить тех, кто работает долго.

41.10 Deep links через URL-схему

URL-схема — префикс ссылки вида `myapp://`, который ты регистрируешь за своим приложением. Регистрация в `Info.plist`:

```
<key>CFBundleURLTypes</key>
<array>
    <dict>
        <key>CFBundleURLName</key>
        <string>kz.example.playground</string>
        <key>CFBundleURLSchemes</key>
        <array>
            <string>myapp</string>
        </array>
    </dict>
</array>
```

Теперь ссылка `myapp://chat/1234` из Safari, заметок или другого приложения откроет наше. Проверить на симуляторе:

```
xcrun simctl openurl booted "myapp://chat/1234"
```

Разбор ссылки — отдельный тип, чтобы не размазывать `if` по делегатам:

```
enum DeepLink: Equatable {
    case chat(id: String)
    case profile(id: String)
    case todos

    init?(url: URL) {
        let parts = url.pathComponents.filter { $0 != "/" }
        switch url.scheme {
        case "myapp":
            // myapp://chat/1234 → host = "chat", parts = ["1234"]
            switch (url.host, parts.count) {
```

```
            case ("chat", 1):  self = .chat(id: parts[0])
            case ("todos", _): self = .todos
            default: return nil
            }
        case "https":
            // https://example.com/share/chat/1234 → parts = ["share", "chat", "1234"]
            guard url.host == "example.com" else { return nil }
            switch (parts.first, parts.count) {
            case ("profile", 2): self = .profile(id: parts[1])
            case ("share", 3) where parts[1] == "chat": self = .chat(id: parts[2])
            default: return nil
            }
        default:
            return nil
        }
    }
}
```

Разбор на примерах:

- `myapp://chat/1234`: схема `myapp`, **хост** (то, что после `://`) — `chat`, путь — `/1234`. `pathComponents` для пути `/1234` возвращает `["/", "1234"]`: первый элемент — сам корневой слеш. Мы выкидываем его фильтром и получаем `["1234"]`.
- `myapp://chat/` и `myapp://chat` — частей пути нет, `parts.count == 0`, ссылка не распознана, `init` вернёт `nil`. Проверка `components.last != "/"` тоже сработала бы, но читается хуже.
- `https://example.com/share/chat/77` — это Universal Link из 41.11, разбирается той же функцией: `.chat(id: "77")`.
- `https://evil.com/profile/5` — чужой домен, `nil`.

Мы прогнали эти адреса (и ещё `myapp://todos/all`, `https://example.com/profile/5`, `https://example.com/share`) через `DeepLink(url:)` в отдельной программе и получили ровно такие результаты: `.todos`, `.profile(id: "5")` и `nil`.

Сам переход — в маршрутизаторе:

```
final class DeepLinkRouter {
    static let shared = DeepLinkRouter()
    weak var navigationController: UINavigationController?

    func open(_ url: URL) {
        guard let link = DeepLink(url: url) else {
            print("Неизвестная ссылка, игнорируем: \(url)")
            return
        }
        guard let nav = navigationController else { return }
        nav.popToRootViewController(animated: false)
        switch link {
        case .chat(let id):
            let vc = UIViewController()
            vc.title = "Чат \(id)"
```

```
            nav.pushViewController(vc, animated: true)
        case .profile(let id):
            let vc = UIViewController()
            vc.title = "Профиль \(id)"
            nav.pushViewController(vc, animated: true)
        case .todos:
            break // корневой экран и есть список
        }
    }
}
```

`weak var navigationController` — маршрутизатор живёт всё время работы приложения (`static let shared`), а контроллер навигации — только пока показано окно. Сильная ссылка удерживала бы его в памяти. Вместо `UIViewController()` в реальном коде — твои экраны чата и профиля.

И два места в `SceneDelegate`, откуда приходят ссылки:

```
final class SceneDelegate: UIResponder, UIWindowSceneDelegate {

    var window: UIWindow?

    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        guard let windowScene = scene as? UIWindowScene else { return }
        let nav = UINavigationController(rootViewController: UIViewController())
        let window = UIWindow(windowScene: windowScene)
        window.rootViewController = nav
        window.makeKeyAndVisible()
        self.window = window
        DeepLinkRouter.shared.navigationController = nav

        // Холодный старт: приложение не работало, ссылка пришла вместе с запуском
        if let url = connectionOptions.urlContexts.first?.url {
            DeepLinkRouter.shared.open(url)
        }
        if let activity = connectionOptions.userActivities.first(where: {
            $0.activityType == NSUserActivityTypeBrowsingWeb
        }), let url = activity.webpageURL {
            DeepLinkRouter.shared.open(url)
        }
    }

    // Приложение уже запущено: пришла ссылка по URL-схеме
    func scene(_ scene: UIScene, openURLContexts URLContexts: Set<UIOpenURLContext>) {
        guard let url = URLContexts.first?.url else { return }
        DeepLinkRouter.shared.open(url)
    }
```

```
    // Приложение уже запущено: пришла Universal Link (41.11)
    func scene(_ scene: UIScene, continue userActivity: NSUserActivity) {
        guard userActivity.activityType == NSUserActivityTypeBrowsingWeb,
              let url = userActivity.webpageURL else { return }
        DeepLinkRouter.shared.open(url)
    }
}
```

Что здесь строится в `scene(_:willConnectTo:options:)`. Сцена (`UIWindowScene → UIScene → UIResponder`) — не view, а «сеанс» приложения на экране; она владеет окнами. Окно — `UIWindow → UIView`, корень дерева view: `UIWindow(windowScene:)` привязывает его к сцене, `rootViewController` даёт окну единственный верхний subview — `nav.view`, а `makeKeyAndVisible()` показывает окно (снимает `isHidden`) и делает его **ключевым** — тем, что получает события клавиатуры и стоит в начале responder chain. Сильная ссылка `self.window = window` обязательна: это свойство требует протокол `UIWindowSceneDelegate`, и именно оно держит окно живым — сцена свои окна не удерживает. Проверено на симуляторе: окно, созданное в локальной переменной и показанное через `makeKeyAndVisible()`, после выхода из области видимости освободилось, а `windowScene.windows` опустел. Подробно про окно, сцену и их делегатов — глава 0, раздел 0.7.

Главная ловушка — **холодный старт**. Если приложение не было запущено, iOS **не** вызывает `scene(_:openURLContexts:)` — ссылка приезжает в `connectionOptions.urlContexts` внутри `scene(_:willConnectTo:options:)`. Аналогично Universal Link приезжает в `connectionOptions.userActivities`. Кто обрабатывает ссылки только в `openURLContexts`, получает баг «по ссылке приложение открывается, но на главном экране». Для Universal Links это прямо написано в документации Supporting universal links in your app: не запущенное приложение получает ссылку в `scene(_:willConnectTo:options:)`, запущенное — в `scene(_:continue:)`. С URL-схемой то же самое: `UIScene.ConnectionOptions.urlContexts` — ссылки, с которыми сцену создали. Проверить у себя: закрой приложение в симуляторе и выполни `xcrun simctl openurl booted "myapp://chat/1234"` (симулятор может спросить «Открыть в ...?»), затем повтори при открытом приложении.

Слабое место URL-схем: **любое** приложение может объявить ту же схему `myapp`, и iOS не гарантирует, какое откроется. Поэтому через схему нельзя передавать ничего секретного (токены, коды входа).

41.11 Universal Links

Universal Link («универсальная ссылка») — обычная https-ссылка на твой сайт, например `https://example.com/share/chat/1234`. Если приложение установлено — iOS откроет его и передаст ссылку. Если нет — откроется сайт в Safari. Подделать нельзя: связь приложения и домена подтверждает файл на твоём сервере, а сервер контролируешь только ты.

Настройка из трёх частей.

1. Файл `apple-app-site-association` на сервере. Имя — ровно такое, **без** расширения `.json`. Адрес — `https://example.com/.well-known/apple-app-site-association`. Отдавать по **https** с действительным сертификатом и **без редиректов**. Для каждого поддомена (`www.example.com`, `m.example.com`) нужен свой файл. Содержимое в актуальном формате (iOS 13+):

```
{
  "applinks": {
    "details": [
      {
        "appIDs": [ "ABCDE12345.kz.example.playground" ],
        "components": [
          { "/": "/share/*", "comment": "Все ссылки, которыми делятся из приложения" },
          { "/": "/profile/*" },
          { "/": "/profile/settings", "exclude": true, "comment": "Эту страницу открываем
          ↪  в браузере" }
        ]
      }
    ]
  }
}
```

- `appIDs` — список приложений в формате `<Team ID>.<bundle ID>`. `ABCDE12345` — Team ID из 41.2.
- `components` — шаблоны адресов. Ключ `"/"` — путь, `*` — любая последовательность символов, `?` — ровно один символ. Есть ключи `"?"` для параметров запроса и `"#"` для фрагмента.
- `"exclude": true` — «эти адреса **не** открывать в приложении». Правила проверяются по порядку, срабатывает первое подходящее, поэтому исключения ставят **выше** общих правил, которые их перекрывают. В нашем примере `/profile/settings` стоит **после** `/profile/*`, и исключение не сработает — это задание для упражнения ниже.

В старых статьях встречается формат `"apps": []` + `"appID"` + `"paths"` — это прежний формат для iOS 12 и старше. Раз у нас iOS 15+, нужен только новый.

С iOS 14 устройства не скачивают этот файл с твоего сервера напрямую: его забирает CDN Apple (сеть серверов Apple) — в течение 24 часов после изменения, а устройства проверяют обновления примерно раз в неделю после установки приложения. Значит, сервер должен быть доступен из интернета, а правки файла доходят не мгновенно. Для разработки есть режим в обход CDN: в entitlement пишут `applinks:example.com?mode=developer`, а на устройстве включают Настройки → Разработчик → Associated Domains Development.

2. Entitlement в Xcode. Signing & Capabilities → **+ Capability** → **Associated Domains** → добавь строку `applinks:example.com`. Без `https://`, без пути и без слеша в конце.

3. Обработка в `SceneDelegate` — уже написана в 41.10: метод `scene(_:continue:)` для запущенного приложения и `connectionOptions.userActivities` для холодного старта. Тип активности — `NSUserActivityTypeBrowsingWeb`, адрес — `webpageURL`.

Документация Supporting universal links in your app предупреждает: Universal Link — такой же вход в приложение извне, как любой другой. Проверяй все параметры, отбрасывай кривые ссылки и не давай ссылке **действовать** без подтверждения — например, удалять данные.

Когда Universal Link **не** откроет приложение: если человек ввёл адрес вручную в Safari, и если ссылка ведёт на тот же домен, на странице которого он уже находится в Safari.

Упражнение. Исправь `components` так, чтобы `/profile/settings` действительно открывался в браузере, а остальные `/profile/...` — в приложении. Ответ — в конце главы.

41.12 Что когда использовать

Сценарий	Способ
Вход через внешний сервис (OAuth)	`ASWebAuthenticationSession`, схема для возврата
Ссылка «посмотри профиль», которой делятся	Universal Link
Пуш, ведущий на экран	deep link в payload
Переход из своего второго приложения	URL-схема
QR-код на товаре	Universal Link
Ссылка в рекламе и письмах	Universal Link

Universal Links предпочтительнее почти везде: их нельзя перехватить, и они работают без приложения (откроется сайт). Для входа через сторонний сервис Apple предлагает `ASWebAuthenticationSession`: он показывает страницу входа и возвращает результат прямо в приложение, перехват схемы другим приложением ему не страшен.

41.13 Кнопки в уведомлении: категории

К уведомлению можно добавить кнопки. Кнопки описываются заранее, при запуске, и объединяются в **категорию**:

```
import UserNotifications

func registerNotificationCategories() {
    let accept = UNNotificationAction(
        identifier: "ACCEPT",
        title: "Принять",
        options: [.foreground]
    )
    let decline = UNNotificationAction(
        identifier: "DECLINE",
        title: "Отклонить",
        options: [.destructive]
    )
    let invitation = UNNotificationCategory(
        identifier: "INVITATION",
        actions: [accept, decline],
        intentIdentifiers: [],
        options: []
    )
    UNUserNotificationCenter.current().setNotificationCategories([invitation])
}

func handleInvitation(_ response: UNNotificationResponse) {
    switch response.actionIdentifier {
    case "ACCEPT":
```

```
        print("Принял приглашение")
    case "DECLINE":
        print("Отклонил приглашение")
    case UNNotificationDefaultActionIdentifier:
        print("Просто тапнул уведомление")
    default:
        break
    }
}
```

- `options: [.foreground]` — нажатие «Принять» открывает приложение.
- `options: [.destructive]` — кнопка «Отклонить» красная, приложение не открывается: действие обрабатывается в фоне.
- `setNotificationCategories` вызывают при каждом запуске, например из `didFinishLaunching`.
- `UNNotificationDefaultActionIdentifier` — тап по самому уведомлению, а не по кнопке.

В payload указывается категория:

```
{
  "aps": {
    "alert": { "title": "Приглашение", "body": "Айгерим зовёт тебя в группу «Бег»" },
    "category": "INVITATION"
  },
  "invitation_id": "abc123"
}
```

Кнопки появляются, когда человек раскрывает уведомление (долгое нажатие или свайп вниз). Нажатие прилетает в тот же `didReceive` из 41.8 — там вызываешь `handleInvitation(response)`.

41.14 Notification Service Extension

Расширение, которое получает пуш **до** показа и может изменить его: расшифровать текст, скачать картинку и прикрепить её. **Расширение** (*app extension*) — отдельный небольшой исполняемый модуль внутри твоего приложения со своим таргетом, который система запускает сама.

1. Xcode → File → New → Target → **Notification Service Extension**.
2. В payload — `"mutable-content": 1` и обязательно `alert`: расширение вызывается только для видимых уведомлений.

```
{
  "aps": { "alert": { "title": "Новое фото", "body": "…" }, "mutable-content": 1 },
  "image_url": "https://example.com/p/1.jpg"
}
```

Код расширения, по мотивам шаблона Xcode 26:

```
import UserNotifications

final class NotificationService: UNNotificationServiceExtension {
```

```
    private var contentHandler: ((UNNotificationContent) -> Void)?
    private var bestAttempt: UNMutableNotificationContent?

    override func didReceive(_ request: UNNotificationRequest,
                             withContentHandler contentHandler: @escaping
                              ↪ (UNNotificationContent) -> Void) {
        self.contentHandler = contentHandler
        guard let content = request.content.mutableCopy() as? UNMutableNotificationContent
         ↪ else {
            contentHandler(request.content)
            return
        }
        bestAttempt = content
        content.title = "Расшифровано: \(content.title)"
        contentHandler(content)
    }

    override func serviceExtensionTimeWillExpire() {
        // Время почти вышло — отдаём то, что успели
        if let contentHandler, let bestAttempt {
            contentHandler(bestAttempt)
        }
    }
}
```

- `mutableCopy() as? UNMutableNotificationContent` — копия содержимого, которую можно менять. Не пиши `as!`: если приведение не удастся, расширение упадёт, и уведомление покажется без изменений. С `guard` мы явно отдаём оригинал.
- `contentHandler(content)` — «показывай вот это». Вызвать нужно ровно один раз.
- `serviceExtensionTimeWillExpire()` — у расширения ограниченное время (около 30 секунд). Если скачивание картинки не успело, система вызовет этот метод, и мы покажем хотя бы изменённый текст.

Режим изоляции в расширении. Настройка `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` есть в шаблоне **приложения** Xcode 26, а в шаблонах расширений её нет: код расширения по умолчанию не привязан к главному актору. Если ты для единообразия включишь MainActor и в расширении, этот класс перестанет собираться: «main actor-isolated instance method 'didReceive(_:withContentHandler:)' has different actor isolation from nonisolated overridden declaration». Лечится пометкой `nonisolated final class NotificationService`. Мы проверили оба варианта компилятором.

41.15 Тестирование пушей

Симулятор. Команда `simctl push` доставляет пуш прямо в симулятор, без APNs и без сервера:

```
xcrun simctl push booted kz.example.playground payload.apns
```

Файл `payload.apns` — тот же JSON, что и для APNs. Bundle ID можно не писать в команде, если добавить в JSON ключ верхнего уровня:

```
{
  "Simulator Target Bundle": "kz.example.playground",
  "aps": {
    "alert": { "title": "Тест", "body": "Проверяем deep link" },
    "sound": "default"
  },
  "deep_link": "myapp://chat/1234"
}
```

Ограничения по справке `simctl push`: не больше 4096 байт и только обычные пуши приложения (VoIP и другие типы не поддерживаются). Ещё удобнее — перетащить файл `.apns` на окно симулятора.

Реальное устройство. В Apple Developer есть веб-инструмент **Push Notifications Console** (из раздела CloudKit Console): вставляешь device token, собираешь payload и отправляешь — без своего сервера. Там же журнал доставки. Не забудь выбрать среду: сборка из Xcode — Development, из TestFlight — Production (41.3).

41.16 Практика хороших пушей

- **Только то, что важно человеку.** Не каждый лайк стоит пуша.
- **Настройки по типам.** Дай выключить отдельные виды уведомлений, а не только все сразу. Рекламные — отдельно и выключены по умолчанию (4.5.4).
- **Тихие часы.** Не шли несрочное ночью по часовому поясу человека: 22:00 в Алматы (UTC+5) — это 17:00 по UTC, поэтому сервер должен хранить часовой пояс пользователя, а не считать «ночь» по своему времени.
- **Provisional authorization** (iOS 12+): опция `.provisional` в `requestAuthorization`. Системный диалог не показывается, уведомления тихо попадают в Центр уведомлений, и человек сам решает там, «Оставить» их или «Выключить».

41.17 Ответы к упражнениям

41.8 — не показывать пуш открытого чата. Храним открытый чат в маршрутизаторе и сравниваем в `willPresent`:

```
extension DeepLinkRouter {
    // В реальном коде экран чата выставляет id в viewWillAppear и сбрасывает в
    ↪ viewWillDisappear
    static var currentChatID: String?
}

extension AppDelegate {
    func presentationOptions(for notification: UNNotification) ->
    ↪ UNNotificationPresentationOptions {
        let userInfo = notification.request.content.userInfo
```

```
        if let link = (userInfo["deep_link"] as? String).flatMap(URL.init(string:)),
           case .chat(let id)? = DeepLink(url: link),
           id == DeepLinkRouter.currentChatID {
            return []           // человек уже в этом чате
        }
        return [.banner, .list, .sound, .badge]
    }
}
```

В `willPresent` вызываем `completionHandler(presentationOptions(for: notification))`. Проверка: открой «чат 1234», отправь `simctl push` с `myapp://chat/1234` — баннера нет; с `myapp://chat/77` — баннер есть.

`static var` в классе на главном акторе компилятор Swift 6 пропускает: переменная изолирована главным актором, гонки нет.

41.11 — исключение. Исключение ставим **перед** общим правилом:

```
"components": [
  { "/": "/profile/settings", "exclude": true },
  { "/": "/share/*" },
  { "/": "/profile/*" }
]
```

Теперь `/profile/settings` совпадает с первым правилом и открывается в браузере, а `/profile/42` проходит мимо него и совпадает с третьим.

Что мы выучили

- **APNs** доставляет пуши; сервер подписывает запросы ключом `.p8` (JWT обновлять раз в 20–60 минут). Ключ не истекает, сертификат `.p12` живёт год.
- **Две среды**: sandbox для сборок из Xcode, production для TestFlight и App Store. Токены из разных сред не взаимозаменяемы.
- **Регистрация** при каждом запуске, токен не кэшировать. Разрешение на показ — отдельно, после объяснения.
- **4.5.4**: пуши не обязательны для работы, реклама — только с явного согласия и с возможностью отказаться.
- **Payload** до 4 КБ; свои ключи — вне `aps`; заголовки `apns-push-type`, `apns-priority`, `apns-topic`.
- **`willPresent`** — приложение на экране, **`didReceive`** — тап. Делегат ставить в `didFinishLaunching`.
- **Тихие пуши**: `content-available: 1`, priority 5, 30 секунд работы, доставка не гарантирована.
- **Deep links**: один `DeepLink`-парсер и один маршрутизатор; ссылки на холодном старте приходят в `connectionOptions`.
- **Окно и сцена**: `UIWindow` — view, корень дерева; `UIWindowScene` — responder, не view; `makeKeyAndVisible()` показывает окно и делает его ключевым; окно держим сильной ссылкой в делегате сцены.

- **Universal Links**: `apple-app-site-association` без расширения в `/.well-known/`, формат `appIDs` + `components`, https без редиректов, Associated Domains `applinks:домен`.
- **Notification Service Extension**: `mutable-content: 1`, без `as!`; расширения по умолчанию не на главном акторе.
- **Тесты**: `xcrun simctl push`, Push Notifications Console для устройства.

Apple Developer Documentation

- Registering your app with APNs — регистрация, device token, почему не кэшировать токен.
- Establishing a token-based connection to APNs — ключ `.p8`, team-scoped и topic-specific ключи, обновление JWT.
- Sending notification requests to APNs — серверы sandbox/production, заголовки, лимит 4 КБ.
- Generating a remote notification — ключи словаря `aps`.
- Pushing background updates to your app — тихие пуши и их ограничения.
- `aps-environment` — какая среда APNs у какой сборки.
- `UNUserNotificationCenter.delegate` — почему делегат ставят до конца запуска.
- `UNUserNotificationCenterDelegate` — `willPresent` и `didReceive`.
- `UIWindow.makeKeyAndVisible()` — показать окно и сделать его ключевым.
- `UNNotificationCategory` — кнопки в уведомлениях.
- `UNNotificationServiceExtension` — изменение содержимого пуша перед показом.
- Testing notifications using the Push Notification Console — отправка тестовых пушей на устройство.
- Supporting associated domains — файл `apple-app-site-association`, CDN Apple, режим разработчика.
- Supporting universal links in your app — обработка Universal Links в приложении со сценами.
- Defining a custom URL scheme for your app — регистрация URL-схемы и её ограничения.
- App Review Guidelines — 4.5.4 — правила для push-уведомлений.

→ Глава 42. Production: widgets + App Intents

Глава 42. Production: widgets + App Intents

Виджеты и App Intents — два способа, которыми приложение работает **вне** своего окна: на главном экране и экране блокировки, в Spotlight, в Siri, в приложении «Команды» (Shortcuts).

Это обзорная глава: у каждого фреймворка десятки возможностей, мы разбираем минимально рабочий путь для Todo из главы 12 — виджет со списком задач, действие «Создать задачу» для Siri и кнопку-галочку прямо на виджете. Весь код ниже проверен компилятором Swift 6.3 (Xcode 26.5) в режиме Swift 6 с минимальной версией iOS 15; где API новее, это отмечено `@available`.

Версии iOS, на которые стоит опираться (по документации Apple):

Возможность	Минимальная iOS
Виджеты на главном экране (WidgetKit)	14
Виджеты на экране блокировки (`accessory…`)	16
App Intents (Siri, Команды, Spotlight)	16
Live Activities	16.1
`Activity.request(…content:pushType:)`	16.2
Интерактивные виджеты (`Button(intent:)`)	17
Настраиваемые виджеты на App Intents	17
StandBy (виджеты при зарядке в альбомной ориентации)	17
Виджеты-переключатели в Пункте управления (Controls)	18

42.1 WidgetKit — что такое виджет

Виджет — маленькая карточка на главном экране или экране блокировки, которая показывает данные твоего приложения. Главное, что нужно понять: виджет — **не** работающее мини-приложение. Это заранее подготовленная картинка. Система время от времени просит твой код: «дай, что показывать сейчас и в ближайшие часы», сохраняет ответ и рисует его, не запуская приложение.

Аналогия — табло расписания на вокзале. Табло не спрашивает у каждого поезда, где он сейчас; диспетчер заранее вывешивает расписание, а табло просто показывает нужную строку по времени.

Интерфейс виджета пишется **только на SwiftUI**, даже если приложение целиком на UIKit, как у нас. Пугаться не надо: виджет — это несколько `Text` и `VStack`, их синтаксис читается с первого взгляда.

Виды виджетов:

- **Главный экран** (iOS 14+) — размеры `systemSmall`, `systemMedium`, `systemLarge`; на iPad ещё `systemExtraLarge`.
- **Экран блокировки** (iOS 16+) — `accessoryCircular`, `accessoryRectangular`, `accessoryInline`.
- **StandBy** (iOS 17+) — iPhone на зарядке в альбомной ориентации показывает `systemSmall`-виджеты крупно.
- **Live Activities** (iOS 16.1+) — «живая» карточка на экране блокировки и в Dynamic Island (вырез-пилюля вверху экрана на iPhone 14 Pro и новее) для событий «прямо сейчас»: доставка, такси, счёт матча.

42.2 Таргет виджета

Xcode → File → New → Target → **Widget Extension**. Xcode создаст новый таргет — **расширение** (*app extension*): отдельный исполняемый модуль, который живёт внутри твоего приложения, но работает в своём процессе.

Про режим Swift. Настройка `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` из введения есть только в шаблоне **приложения** Xcode 26; в шаблоне Widget Extension её нет. Значит, код виджета по умолчанию не привязан к главному актору. Это пригодится в 42.4, когда один файл будет входить в оба таргета.

Минимальная версия iOS у таргета расширения может быть выше, чем у приложения. Если приложение поддерживает iOS 15, а виджет ты хочешь делать только для iOS 17+, у расширения ставишь 17 — на iOS 15 и 16 приложение будет работать, просто без виджета. В коде ниже мы держим расширение на iOS 15 и отмечаем новые API через `@available`, чтобы показать, как это выглядит.

42.3 Модель, запись в таймлайне и провайдер

Сначала общая модель задачи и хранилище в App Group (что это — в 42.4):

```
import WidgetKit
import SwiftUI

// Этот файл входит в оба таргета — приложение и виджет
nonisolated struct Todo: Codable, Identifiable, Hashable {
    var id = UUID()
    var title: String
    var isDone: Bool = false
}

extension Todo {
    static let previewList = [
        Todo(title: "Забрать посылку"),
```

```
        Todo(title: "Купить продукты", isDone: true),
    ]
}

nonisolated enum SharedTodoStore {
    static let appGroup = "group.kz.example.playground"
    static let key = "todos"

    static func load() -> [Todo] {
        guard let defaults = UserDefaults(suiteName: appGroup),
              let data = defaults.data(forKey: key),
              let todos = try? JSONDecoder().decode([Todo].self, from: data)
        else { return [] }
        return todos
    }

    static func save(_ todos: [Todo]) {
        guard let defaults = UserDefaults(suiteName: appGroup),
              let data = try? JSONEncoder().encode(todos) else { return }
        defaults.set(data, forKey: key)
    }
}
```

- `Identifiable` нужен SwiftUI, чтобы строить список через `ForEach`: у каждой задачи есть стабильный `id`.
- `nonisolated` у `Todo` и `SharedTodoStore` — потому что файл входит в **оба** таргета. В приложении по умолчанию всё на главном акторе, в виджете — нет. Без `nonisolated` в приложении эти типы стали бы «главноакторными», и код интента из 42.8, который выполняется вне главного актора, не смог бы их вызвать: компилятор ответил бы «main actor-isolated static method ‘load()’ cannot be called from outside of the actor». Мы поймали эту ошибку на проверке.
- `UserDefaults(suiteName:)` — общие настройки группы приложений, а не личные настройки одного таргета.
- Никаких `!`: если группа не настроена, `load()` вернёт пустой массив, и виджет покажет «Задач нет», а не упадёт.

Дальше — **запись таймлайна** (*timeline entry*) и **провайдер**:

```
struct TodoEntry: TimelineEntry {
    let date: Date
    let todos: [Todo]
}

struct TodoProvider: TimelineProvider {
    func placeholder(in context: Context) -> TodoEntry {
        TodoEntry(date: Date(), todos: Todo.previewList)
    }

    func getSnapshot(in context: Context, completion: @escaping (TodoEntry) -> Void) {
```

```
        if context.isPreview {
            completion(TodoEntry(date: Date(), todos: Todo.previewList))
        } else {
            completion(TodoEntry(date: Date(), todos: SharedTodoStore.load()))
        }
    }

    func getTimeline(in context: Context, completion: @escaping (Timeline<TodoEntry>) ->
    ↪ Void) {
        let now = Date()
        let entry = TodoEntry(date: now, todos: SharedTodoStore.load())
        let nextUpdate = Calendar.current.date(byAdding: .minute, value: 30, to: now)
            ?? now.addingTimeInterval(30 * 60)
        completion(Timeline(entries: [entry], policy: .after(nextUpdate)))
    }
}
```

Таймлайн (*timeline*) — то самое «расписание» из аналогии: массив записей, у каждой — дата, с которой её показывать. Провайдер отвечает на три вопроса системы:

- `placeholder` — «что нарисовать, пока данных ещё нет». Система покажет это размытым силуэтом. Работать должно мгновенно, без чтения диска.
- `getSnapshot` — «один кадр прямо сейчас». Вызывается, в частности, в галерее выбора виджетов. `context.isPreview == true` — это как раз галерея: показываем красивые демо-задачи. Если бы мы читали реальные данные, у нового пользователя в галерее был бы пустой прямоугольник, и он прошёл бы мимо.
- `getTimeline` — «расписание вперёд». Мы отдаём одну запись «сейчас» и политику `.after(nextUpdate)`: «спроси меня снова не раньше чем через 30 минут».

`Calendar.current.date(byAdding:...)` возвращает опционал: в теории календарь может не суметь прибавить. Вместо `!` у нас запасной вариант — прибавить 30 × 60 = 1800 секунд.

Виджет не обновляется точно по расписанию. По документации Keeping a widget up to date у каждого виджета есть дневной **бюджет** обновлений: для виджета, который человек часто видит, — обычно **от 40 до 70** обновлений в сутки. На числах: сутки — 1440 минут; 1440 / 70 ≈ 20 минут, 1440 / 40 = 36 минут. Отсюда ориентир Apple «примерно раз в 15–60 минут». Просить обновление каждую минуту бессмысленно — бюджет кончится к обеду.

Не расходует бюджет перезагрузка, которую приложение просит, пока открыто, — об этом в 42.4.

42.4 Общие данные — App Group

Виджет — отдельный процесс со своей песочницей (изолированной папкой на диске). Он **не видит** `UserDefaults.standard` и папку `Documents` приложения. Мостик между ними — **App Group** («группа приложений»): общая папка и общий `UserDefaults` для приложений и расширений одной команды.

1. Таргет приложения → Signing & Capabilities → **+ Capability** → **App Groups** → добавь `group.kz.example.playground`.
2. То же самое у таргета виджета — **та же** группа.

Приложение сохраняет задачи в группу и просит WidgetKit перерисовать виджет:

```
import WidgetKit

// Этот код — в таргете приложения
final class TodoStorage {
    static let shared = TodoStorage()
    private(set) var todos: [Todo] = SharedTodoStore.load()

    func add(_ todo: Todo) {
        todos.append(todo)
        SharedTodoStore.save(todos)
        WidgetCenter.shared.reloadTimelines(ofKind: "TodoWidget")
    }
}
```

`WidgetCenter.shared.reloadTimelines(ofKind:)` — «данные изменились, перезапроси таймлайн у виджета с таким `kind`». Без этого вызова новая задача появится на виджете, только когда подойдёт очередь по расписанию — через полчаса.

Для файлов (например, картинок) у App Group есть общая папка:

```
func sharedFileURL(named name: String) -> URL? {
    FileManager.default
        .containerURL(forSecurityApplicationGroupIdentifier:
        ↪  "group.kz.example.playground")?
        .appendingPathComponent(name)
}
```

`containerURL(...)` возвращает `nil`, если группа не подключена к таргету. Опциональная цепочка `?.` вместо `!` превращает забытую настройку в понятный `nil`, а не в падение.

И про Privacy Manifest (глава 39): `UserDefaults(suiteName:)` — это тот же required reason API. Для App Group причина — `1C8F.1`, а манифест нужен и приложению, и расширению.

42.5 Интерфейс виджета

```
struct TodoWidgetView: View {
    let entry: TodoEntry
    @Environment(\.widgetFamily) private var family

    private var limit: Int {
        switch family {
        case .systemSmall: return 3
        case .systemMedium: return 4
```

```
        default: return 8
        }
    }

    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Список дел")
                .font(.headline)
            if entry.todos.isEmpty {
                Text("Задач нет")
                    .font(.subheadline)
                    .foregroundColor(.secondary)
            } else {
                ForEach(entry.todos.prefix(limit)) { todo in
                    Text(todo.title)
                        .font(.subheadline)
                        .strikethrough(todo.isDone)
                        .lineLimit(1)
                }
            }
            Spacer(minLength: 0)
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity, alignment: .topLeading)
        .widgetBackground(Color(.systemBackground))
        .widgetURL(URL(string: "myapp://todos"))
    }
}

extension View {
    // На iOS 17+ фон виджета задаётся через containerBackground,
    // иначе система покажет заглушку вместо виджета. На iOS 14–16 — обычный фон.
    @ViewBuilder
    func widgetBackground(_ color: Color) -> some View {
        if #available(iOS 17.0, *) {
            containerBackground(color, for: .widget)
        } else {
            padding().background(color)
        }
    }
}
```

Для тех, кто SwiftUI не видел:

- `VStack(alignment: .leading, spacing: 4)` — вертикальная стопка с выравниванием по левому краю и отступом 4 точки, аналог вертикального `UIStackView`.
- `@Environment(\.widgetFamily)` — размер, в котором система сейчас рисует виджет. В маленький влезает 3 строки, в средний — 4.
- `ForEach(entry.todos.prefix(limit))` — первые `limit` задач.

- `.strikethrough(todo.isDone)` — зачеркнуть выполненные.
- `widgetBackground` — наш помощник. С iOS 17 виджет **обязан** задать фон через `containerBackground(_:for: .widget)`: система сама решает, когда фон показать, а когда убрать (например, в StandBy). Без него на iOS 17+ вместо виджета будет сообщение с просьбой перейти на новый API. Этот API есть только с iOS 17, поэтому ветка `if #available`.
- `.widgetURL(...)` — куда вести при тапе. Об этом в 42.7.

И сам виджет:

```
@main
struct TodoWidget: Widget {
    var body: some WidgetConfiguration {
        StaticConfiguration(kind: "TodoWidget", provider: TodoProvider()) { entry in
            TodoWidgetView(entry: entry)
        }
        .configurationDisplayName("Список дел")
        .description("Ближайшие задачи из приложения")
        .supportedFamilies([.systemSmall, .systemMedium, .systemLarge])
    }
}
```

- `@main` — точка входа расширения. Если виджетов несколько, `@main` ставят на `WidgetBundle` (42.9).
- `StaticConfiguration` — виджет без пользовательских настроек. `kind` — строковый идентификатор, тот же, что в `reloadTimelines(ofKind:)`.
- `configurationDisplayName` и `description` — название и описание в галерее виджетов.
- `supportedFamilies` — какие размеры предлагать.

Упражнение. Добавь в заголовок виджета счётчик «Список дел · 3», где 3 — число невыполненных задач. Ответ — в конце главы.

42.6 Как посмотреть виджет

1. Выбери схему таргета виджета (Xcode создаёт её вместе с таргетом) и запусти на симуляторе. Xcode установит приложение с расширением.
2. На симуляторе: долгое нажатие на главном экране → «Изменить» → «Добавить виджет» → найди своё приложение.

Для быстрой отладки внешнего вида в Xcode есть превью (`#Preview(as: .systemSmall)`, iOS 17+). Для проверки таймлайна полезно запустить приложение, добавить задачу и убедиться, что виджет обновился сразу — это и есть работа `reloadTimelines`.

42.7 Тап по виджету — deep link

Тап по виджету открывает приложение. Куда именно — задаётся ссылкой:

- `.widgetURL(URL)` — одна ссылка на весь виджет. Для `systemSmall` это **единственный** вариант: маленький виджет — одна зона нажатия.

- `Link(destination:) { ... }` — отдельные зоны в среднем и большом виджете: тап по конкретной задаче ведёт в неё.

```
Link(destination: URL(string: "myapp://todos") ?? URL(fileURLWithPath: "/")) {
    Text("Все задачи")
}
```

Ссылка `myapp://todos` приходит в приложение как обычная URL-схема: `scene(_:openURLContexts:)` или, при холодном старте, `connectionOptions.urlContexts` — всё это разобрано в главе 41 (41.10), вместе с `DeepLink` и `DeepLinkRouter`.

42.8 App Intents — действия для Siri и «Команд»

App Intent («намерение») — действие твоего приложения, которое система может вызвать сама: из Siri, из приложения «Команды», из Spotlight, с кнопки на виджете. Ты описываешь действие структурой, а система строит вокруг неё интерфейс. Фреймворк App Intents появился в **iOS 16**.

```
import AppIntents

@available(iOS 16.0, *)
struct CreateTodoIntent: AppIntent {
    static let title: LocalizedStringResource = "Создать задачу"
    static let description = IntentDescription("Добавить задачу в список дел")

    @Parameter(title: "Текст задачи")
    var todoText: String

    @MainActor
    func perform() async throws -> some IntentResult & ProvidesDialog {
        let text = todoText.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !text.isEmpty else {
            throw $todoText.needsValueError("Какую задачу добавить?")
        }
        TodoStorage.shared.add(Todo(title: text))
        return .result(dialog: "Создано: \(text)")
    }
}
```

Разбор:

- `@available(iOS 16.0, *)` — у нас приложение с iOS 15, а App Intents — с 16. На iOS 15 этого действия просто не будет.
- `static let title` — название действия в «Командах». Именно `let`, а не `var`: в Swift 6 изменяемое статическое свойство — общее изменяемое состояние, и компилятор откажется его собирать («static property ‘title’ is not concurrency-safe»).
- `static let description` — тип `IntentDescription`, а не строка. Кроме текста у него есть `categoryName` (группировка в «Командах») и массив `searchKeywords` (ключевые слова для поиска) — оба с iOS 17.

- `@Parameter(title:)` — входной параметр. Если его не передали, Siri спросит голосом, «Команды» — полем ввода.
- `@MainActor func perform()` — сама работа. Протокол `AppIntent` не привязывает `perform()` к главному актору, а наш `TodoStorage` в приложении — главноакторный. Если `perform()` зовёт `TodoStorage.shared.add` без пометки, Swift 6 выдаёт ошибку «expression is 'async' but is not marked with 'await'». `@MainActor` на методе решает это: вся работа идёт на главном потоке, как и остальной код приложения.
- `some IntentResult & ProvidesDialog` — результат, в котором есть фраза для Siri. Без `& ProvidesDialog` вызов `.result(dialog:)` не соберётся.
- `$todoText.needsValueError(...)` — «параметр пустой, переспроси». Siri задаст этот вопрос вслух.

Чтобы действие было доступно голосом **без** ручной настройки в «Командах», его объявляют в `AppShortcutsProvider`:

```
@available(iOS 16.0, *)
struct PlaygroundShortcuts: AppShortcutsProvider {
    static var appShortcuts: [AppShortcut] {
        AppShortcut(
            intent: CreateTodoIntent(),
            phrases: [
                "Создать задачу в \(.applicationName)",
                "Добавить дело в \(.applicationName)",
            ],
            shortTitle: "Создать задачу",
            systemImageName: "plus.circle"
        )
    }
}
```

- `phrases` — фразы для Siri. Каждая **обязана** содержать `\(.applicationName)` — название приложения: так Siri понимает, к какому приложению обращаются.
- `shortTitle` и `systemImageName` — подпись и иконка SF Symbols на плашке в «Командах» и Spotlight.
- Провайдер ничего не нужно регистрировать: Xcode при сборке извлекает метаданные интентов из кода, и система узнаёт о них после установки.

Проверка: запусти приложение на симуляторе iOS 16+, открой «Команды» → вкладка приложений → найди своё приложение. Там будет «Создать задачу». Голосом — на реальном устройстве с Siri.

42.9 Экран блокировки (iOS 16+)

Для экрана блокировки — отдельные размеры `accessory…`. Их интерфейс **монохромный**: система перекрашивает виджет в цвет экрана блокировки.

```
@available(iOS 16.0, *)
struct TodoCountAccessoryView: View {
    let entry: TodoEntry
```

```
    @Environment(\.widgetFamily) private var family

    private var left: Int { entry.todos.filter { !$0.isDone }.count }

    var body: some View {
        switch family {
        case .accessoryCircular:
            ZStack {
                AccessoryWidgetBackground()
                Text("\(left)")
                    .font(.title2.bold())
            }
        case .accessoryRectangular:
            VStack(alignment: .leading) {
                Text("Осталось задач: \(left)")
                    .font(.headline)
                    .widgetAccentable()
                Text(entry.todos.first(where: { !$0.isDone })?.title ?? "Всё сделано")
                    .privacySensitive()
            }
        default:
            Text("\(left)")
        }
    }
}

@available(iOS 16.0, *)
struct TodoLockWidget: Widget {
    var body: some WidgetConfiguration {
        StaticConfiguration(kind: "TodoLockWidget", provider: TodoProvider()) { entry in
            TodoCountAccessoryView(entry: entry)
                .widgetBackground(Color.clear)
        }
        .configurationDisplayName("Задачи на экране блокировки")
        .description("Сколько задач осталось")
        .supportedFamilies([.accessoryCircular, .accessoryRectangular])
    }
}

@main
struct PlaygroundWidgets: WidgetBundle {
    var body: some Widget {
        TodoWidget()
        if #available(iOS 16.1, *) {
            TodoLockWidget()
        }
    }
}
```

- `accessoryCircular` — кружок (как кольца активности), `accessoryRectangular` — прямоугольник на две-три строки.
- `AccessoryWidgetBackground()` — стандартная полупрозрачная подложка кружка.
- `.widgetAccentable()` — эта часть окрасится акцентным цветом.
- `.privacySensitive()` — текст задачи **скрывается**, пока телефон заблокирован. Название задачи может быть личным («Записаться к врачу»), а экран блокировки видит кто угодно. Это правильный ответ на вопрос «как защитить данные в виджете»; ключ `NSWidgetWantsLocation` про другое — он нужен виджетам, которые запрашивают геопозицию.
- `WidgetBundle` — несколько виджетов в одном расширении; `@main` теперь стоит на нём, а у `TodoWidget` его нужно убрать.
- `if #available(iOS 16.1, *)` — внутри `WidgetBundle` компилятор требует именно 16.1 или новее: на `16.0` он предупреждает, что код «may crash on earlier versions of the OS». Мы получили это предупреждение и исправили. На iOS 16.0 виджет экрана блокировки не появится — это цена поддержки старых версий.

42.10 Интерактивный виджет (iOS 17+)

С iOS 17 в виджете можно поставить кнопку или переключатель, которые выполняют действие **без открытия приложения**. Действие — это App Intent:

```
import AppIntents
import SwiftUI
import WidgetKit

// Файл входит в оба таргета: приложение и виджет
@available(iOS 17.0, *)
struct ToggleTodoIntent: AppIntent {
    static let title: LocalizedStringResource = "Отметить задачу"

    @Parameter(title: "ID задачи")
    var todoID: String

    init() {}

    init(todoID: UUID) {
        self.todoID = todoID.uuidString
    }

    func perform() async throws -> some IntentResult {
        var todos = SharedTodoStore.load()
        if let index = todos.firstIndex(where: { $0.id.uuidString == todoID }) {
            todos[index].isDone.toggle()
            SharedTodoStore.save(todos)
        }
        return .result()
    }
}
```

```
@available(iOS 17.0, *)
struct TodoRow: View {
    let todo: Todo

    var body: some View {
        Button(intent: ToggleTodoIntent(todoID: todo.id)) {
            Label(todo.title, systemImage: todo.isDone ? "checkmark.circle.fill" :
↪  "circle")
                .strikethrough(todo.isDone)
        }
        .buttonStyle(.plain)
    }
}
```

- `init()` — пустой инициализатор обязателен для `AppIntent`: система создаёт интент сама, а потом заполняет параметры.
- `init(todoID:)` — наш удобный инициализатор для кнопки.
- `@Parameter var todoID: String` — храним `UUID` строкой: параметры интентов поддерживают ограниченный набор типов, а строка есть всегда.
- `perform()` здесь **без** `@MainActor`: интент выполняется в процессе виджета, и он трогает только `SharedTodoStore`, помеченный `nonisolated` (42.3).
- После выполнения интента, запущенного кнопкой виджета, система сама перезагружает таймлайн — виджет перерисуется с новой галочкой.
- `Button(intent:)` — кнопка SwiftUI, которая вместо замыкания запускает интент. Работает с iOS 17.

Чтобы использовать строку, в `TodoWidgetView` вместо `Text(todo.title)` на iOS 17+ показываешь `TodoRow(todo: todo)`.

42.11 Live Activities (iOS 16.1+)

Live Activity — карточка «здесь и сейчас» на экране блокировки и в Dynamic Island. Данные делятся на **постоянные** (номер заказа) и **меняющиеся** (этап, минуты до доставки).

Описание данных — общий файл для приложения и расширения:

```
import ActivityKit
import Foundation

@available(iOS 16.1, *)
nonisolated struct DeliveryAttributes: ActivityAttributes {
    public struct ContentState: Codable, Hashable {
        var stage: String          // «Готовится», «В пути», «Доставлено»
        var minutesRemaining: Int
    }
    var orderId: String
}
```

Управление из приложения:

```
@available(iOS 16.2, *)
final class DeliveryTracker {
    private var activity: Activity<DeliveryAttributes>?

    func start(orderId: String) {
        guard ActivityAuthorizationInfo().areActivitiesEnabled else {
            print("Live Activities выключены в настройках")
            return
        }
        do {
            activity = try Activity.request(
                attributes: DeliveryAttributes(orderId: orderId),
                content: ActivityContent(
                    state: .init(stage: "Готовится", minutesRemaining: 25),
                    staleDate: nil
                ),
                pushType: nil
            )
        } catch {
            print("Не удалось запустить Live Activity: \(error)")
        }
    }

    func update(stage: String, minutes: Int) async {
        await activity?.update(
            ActivityContent(state: .init(stage: stage, minutesRemaining: minutes),
                            staleDate: nil)
        )
    }

    func finish() async {
        await activity?.end(
            ActivityContent(state: .init(stage: "Доставлено", minutesRemaining: 0),
                            staleDate: nil),
            dismissalPolicy: .default
        )
        activity = nil
    }
}
```

- `ActivityAuthorizationInfo().areActivitiesEnabled` — человек мог выключить Live Activities для приложения в настройках.
- `Activity.request(attributes:content:pushType:)` — с iOS 16.2. `pushType: nil` — обновляем только из приложения; с `.token` можно обновлять пушами с сервера (тип пуша `liveactivity`, глава 41).
- `staleDate` — момент, после которого данные считаются устаревшими (`nil` — не устаревают).

- `end(_:dismissalPolicy: .default)` — финальное состояние «Доставлено» остаётся на экране блокировки какое-то время, чтобы человек увидел итог.

Ограничения из документации ActivityKit:

- в `Info.plist` приложения нужен ключ `NSSupportsLiveActivities = YES`;
- данные (постоянные + меняющиеся) — не больше **4 КБ**;
- активность живёт до **8 часов**, после чего система её завершает; на экране блокировки она может провисеть ещё до 4 часов — максимум 12 часов суммарно.

И внешний вид — в расширении виджета:

```
import ActivityKit
import SwiftUI
import WidgetKit

@available(iOS 16.1, *)
struct DeliveryLiveActivity: Widget {
    var body: some WidgetConfiguration {
        ActivityConfiguration(for: DeliveryAttributes.self) { context in
            // Экран блокировки и баннер на iPhone без Dynamic Island
            HStack {
                Text(context.state.stage).font(.headline)
                Spacer()
                Text("\(context.state.minutesRemaining) мин")
            }
            .padding()
        } dynamicIsland: { context in
            DynamicIsland {
                DynamicIslandExpandedRegion(.leading) {
                    Text("Заказ \(context.attributes.orderId)")
                }
                DynamicIslandExpandedRegion(.trailing) {
                    Text("\(context.state.minutesRemaining) мин")
                }
                DynamicIslandExpandedRegion(.bottom) {
                    Text(context.state.stage)
                }
            } compactLeading: {
                Image(systemName: "bag")
            } compactTrailing: {
                Text("\(context.state.minutesRemaining)м")
            } minimal: {
                Image(systemName: "bag")
            }
        }
    }
}
```

У Dynamic Island три вида: **expanded** (развёрнутый, по долгому нажатию), **compact** (две половинки слева и справа от камеры) и **minimal** (маленький кружок, когда активностей несколько). Эту структуру добавляешь в `WidgetBundle` из 42.9 (с проверкой `if #available(iOS 16.2, *)`).

42.12 Что когда использовать

Задача	Решение
Данные на главном экране	Виджет (WidgetKit)
Действие голосом через Siri	App Intent + App Shortcut
Кнопка в «Командах»	App Intent
Галочка прямо на виджете	App Intent + `Button(intent:)` (iOS 17+)
Поиск своего контента в Spotlight	Core Spotlight; с iOS 18 — ещё и App Intents для сущностей
Отслеживание заказа в реальном времени	Live Activity
Статус на экране блокировки	`accessory`-виджет

42.13 Приватность виджетов

Виджет видят люди рядом с тобой: на главном экране, на экране блокировки, в StandBy на тумбочке. Правила:

- **`.privacySensitive()`** на всём, что личное: суммы, сообщения, названия задач. На заблокированном устройстве система заменит такой текст заглушкой.
- **Privacy Manifest** — отдельный у расширения, если оно пользуется required reason API (App Group `UserDefaults` — это `1C8F.1`).
- **Геопозиция в виджете** — если виджет сам запрашивает местоположение, в `Info.plist` расширения нужен ключ `NSWidgetWantsLocation = YES`, а у приложения — разрешение на геопозицию.

42.14 Нужен ли виджет

Если приложение **функциональное** — список дел, заметки, трекер привычек, погода, — виджет часто становится главным способом им пользоваться: человек видит данные, не открывая приложение.

Если приложение **сессионное** — игра, магазин, утилита «открыл и закрыл», — виджет не обязателен. Начни с App Intent: одно действие для Siri и «Команд» стоит дешевле и полезнее «витринного» виджета.

42.15 Ответы к упражнениям

42.5 — счётчик в заголовке.

```
struct TodoWidgetHeader: View {
    let todos: [Todo]
```

```
    var body: some View {
        let left = todos.filter { !$0.isDone }.count
        Text("Список дел · \(left)")
            .font(.headline)
    }
}
```

В `TodoWidgetView` заменяешь `Text("Список дел")` на `TodoWidgetHeader(todos: entry.todos)`. Проверка: для `Todo.previewList` (одна задача выполнена, одна нет) заголовок в галерее виджетов — «Список дел · 1».

Что мы выучили

- **WidgetKit** (iOS 14+): виджет — расписание снимков, а не работающая программа. Интерфейс — на SwiftUI.
- **`TimelineProvider`**: `placeholder`, `getSnapshot` (с `context.isPreview`), `getTimeline` с политикой `.after`.
- **Бюджет** — обычно 40–70 обновлений в сутки; после изменения данных в приложении — `WidgetCenter.shared.reloadTimelines(ofKind:)`.
- **App Group** — общий `UserDefaults` и папка для приложения и виджета. Общие типы в файлах для двух таргетов — `nonisolated`.
- **iOS 17+**: фон через `containerBackground(_:for: .widget)`, интерактивные виджеты через `Button(intent:)`.
- **Тап**: `widgetURL` (единственный вариант для `systemSmall`) и `Link`.
- **App Intents** (iOS 16+): `static let title`, `@Parameter`, `perform()`; `@MainActor` на `perform`, если трогаешь главноакторный код; `AppShortcutsProvider` с `\(.applicationName)` в каждой фразе.
- **Экран блокировки** (iOS 16+): `accessoryCircular`, `accessoryRectangular`, `.privacySensitive()`.
- **Live Activities** (iOS 16.1+): `ActivityAttributes`, `Activity.request(...content:pushType:)` с iOS 16.2, `NSSupportsLiveActivities`, до 8 часов и 4 КБ данных.

Apple Developer Documentation

- WidgetKit — фреймворк виджетов.
- Creating a widget extension — таргет виджета, провайдер, конфигурация.
- `TimelineProvider` — `placeholder`, `getSnapshot`, `getTimeline`.
- Keeping a widget up to date — бюджет обновлений и `reloadTimelines`.
- Adding interactivity to widgets and Live Activities — `Button(intent:)` и `Toggle(isOn:intent:)`.
- Creating accessory widgets and watch complications — виджеты экрана блокировки.
- `widgetURL(_:)` — ссылка при тапе по виджету.
- App Intents — фреймворк действий для Siri, «Команд» и Spotlight.
- `AppShortcutsProvider` — фразы Siri для твоих интентов.
- Displaying live data with Live Activities — ActivityKit, Dynamic Island, лимиты.
- Configuring App Groups — общая группа для приложения и расширений.

→ Глава 43. Production: accessibility audit перед релизом

Глава 43. Production: accessibility audit перед релизом

Финальная глава книги. Перед каждым релизом стоит пройти этот чек-лист. **Accessibility** («доступность») — это способность приложения работать для людей с разными возможностями: незрячих, слабовидящих, с тремором рук, с дальтонизмом, с непереносимостью резкой анимации. И заодно для всех остальных в неудобной ситуации: на ярком солнце, одной рукой в автобусе, без звука.

Что с требованиями. App Review Guidelines не отклоняют приложение только за плохую доступность. Но с 2025 года в App Store Connect есть **Accessibility Nutrition Labels** — «этикетка доступности» на странице приложения (раздел 43.1), и Apple прямо пишет, что со временем заполнять её станет **обязательно** для новых приложений и обновлений. Так что аудит доступности постепенно переходит из «хорошего тона» в требование.

Код отдельных приёмов (`accessibilityLabel`, группировка, Dynamic Type, Reduce Motion, 44 точки) разобран в главе 34 (Cookbook — accessibility). Здесь — как проверить всё приложение целиком.

43.1 Accessibility Nutrition Labels

В App Store Connect (страница приложения → раздел про доступность) ты отмечаешь, какие функции доступности приложение поддерживает в **основных сценариях**. По справке Overview of Accessibility Nutrition Labels их девять:

Функция	Что это значит простыми словами
VoiceOver	всем можно пользоваться с экранным диктором
Voice Control	всем можно управлять голосом («нажми Добавить»)
Larger Text	текст увеличивается до 200% и больше, и ничего не ломается
Dark Interface	есть тёмная тема
Differentiate Without Color Alone	важное различается не только цветом, но и формой или текстом
Sufficient Contrast	текст и значки достаточно контрастны
Reduced Motion	анимации, от которых укачивает, можно уменьшить
Captions	у видео и аудио есть субтитры
Audio Descriptions	у видео есть звуковое описание происходящего

Пока заполнение **добровольное**, этикетка видна в App Store на устройствах с iOS 26 и новее. Отмечать функцию стоит, только если она работает во **всех основных сценариях** приложения: для каждой у Apple есть отдельная страница с критериями оценки (evaluation criteria). Чек-лист ниже устроен так, чтобы после него можно было честно заполнить эту этикетку.

43.2 Полный чек-лист

VoiceOver

VoiceOver — экранный диктор iOS: читает вслух то, что под пальцем, а управление идёт жестами (свайп вправо — к очередному элементу, двойной тап — нажать).

- ☐ У каждого интерактивного элемента есть понятный `accessibilityLabel` (глава 34.1).
- ☐ Кнопки-иконки без текста подписаны: не «кнопка», а «Добавить задачу».
- ☐ Декоративные картинки скрыты: `isAccessibilityElement = false`.
- ☐ Ячейка списка читается **одной фразой** («Купить продукты, выполнено»), а не тремя отдельными элементами (34.3).
- ☐ Swipe-действия продублированы через `accessibilityCustomActions` (34.14): незрячий человек не увидит, что ячейку можно смахнуть.
- ☐ Порядок чтения логичный (сверху вниз, слева направо); если нет — `accessibilityElements` у контейнера.
- ☐ Модальный экран закрывается жестом «Z» (двумя пальцами зигзагом) — для этого `accessibilityPerformEscape()` или стандартный `UINavigationController`/`UIAlertController`.

Dynamic Type

Dynamic Type — системный размер шрифта, который человек выбирает в настройках. Приложение должно подхватывать его.

- ☐ Все тексты — `UIFont.preferredFont(forTextStyle:)` или свои шрифты через `UIFontMetrics.scaledFont(f` (34.4).
- ☐ `adjustsFontForContentSizeCategory = true` — тогда шрифт меняется без перезапуска. Свойство объявлено в протоколе `UIContentSizeCategoryAdjusting`, который по заголовкам SDK реализуют только `UILabel`, `UITextField` и `UITextView`; по умолчанию оно `false`. У `UIButton` его нет — включай у `button.titleLabel`. Работает только с шрифтом из `preferredFont(forTextStyle:)` или `UIFontMetrics`.
- ☐ Ячейки таблиц с автоматической высотой и `numberOfLines = 0` (34.12).
- ☐ На самых крупных размерах текст не обрезается, горизонтальные стопки перестраиваются в вертикальные.

Контраст

- ☐ Текст до 17 pt — контраст не ниже **4.5:1**; текст от 18 pt и жирный текст любого размера — не ниже **3:1**. Так в разделе HIG — Accessibility (с опорой на WCAG уровня AA). Сам WCAG к жирному строже: крупным он считает жирный текст от 14 pt (34.7).
- ☐ Для неосновного текста — значков, границ полей, состояния чекбокса — рекомендуемый минимум **3:1** (критерии Sufficient Contrast).
- ☐ Проверено в светлой **и** тёмной теме.

- ☐ Если в какой-то теме контраста не хватает — при включённом «Увеличение контраста» (`UIAccessibility.isDarkerSystemColorsEnabled`) он становится достаточным (34.7).

Зоны нажатия

- ☐ Всё, что нажимается, не меньше **44 × 44 точек** — рекомендуемый размер из HIG. Абсолютный минимум по HIG для iOS — 28 × 28, но это для редких исключений.
- ☐ Маленькая иконка 20 × 20 получает увеличенную зону нажатия (34.6), а не увеличенную картинку.

Движение

- ☐ При `UIAccessibility.isReduceMotionEnabled` пружинные и масштабирующие анимации заменены на плавное появление или мгновенный переход (34.5).
- ☐ Параллакс (глава 21) отключается.
- ☐ Автовоспроизведение видео не стартует само.

Не только цветом

- ☐ Ошибка в форме — красная рамка **и** иконка или текст «Неверный email».
- ☐ Статусы (онлайн/офлайн, выполнено/нет) различаются формой или подписью, а не только цветом.
- ☐ У видео с речью есть субтитры.

Клавиатура и Switch Control

Switch Control — управление одной-двумя кнопками-переключателями для людей, которые не могут пользоваться сенсорным экраном: система по очереди подсвечивает элементы, человек нажимает «выбрать».

- ☐ На iPad с внешней клавиатурой до всех элементов можно добраться клавишей Tab.
- ☐ Порядок перехода логичный.
- ☐ Выделенный элемент видно (есть рамка фокуса).

43.3 Accessibility Inspector

`Xcode → Open Developer Tool → Accessibility Inspector`.

Сверху слева выбираешь цель — запущенный симулятор. Три режима:

- **Inspection** — наведи курсор на элемент и увидишь его label, value, traits (роль: кнопка, заголовок, выбрано), hint.
- **Audit** — автоматическая проверка текущего экрана: низкий контраст, маленькие зоны нажатия, элементы без описания, текст, который не масштабируется с Dynamic Type, обрезанный текст. Каждую находку можно подсветить на экране.
- **Settings** — переключатели прямо для симулятора: размер шрифта, увеличение контраста, уменьшение движения, инверсия цветов. Удобно проверять Dynamic Type без похода в Настройки.

Важно: **в симуляторе нет VoiceOver.** Сочетание `Cmd+F5` включает VoiceOver самого **Mac**, а не симулятора. Проверка «на слух» — только на реальном устройстве; в симуляторе вместо неё — Inspector.

Ещё быстрее — **Environment Overrides** в Xcode: во время отладки на панели отладки есть кнопка с переключателями, там меняется Dynamic Type, тёмная тема и часть настроек доступности прямо на запущенном приложении.

43.4 Автоматический аудит в UI-тестах (iOS 17+)

С Xcode 15 и iOS 17 тот же аудит, что в Inspector, можно запускать из UI-теста — и тогда он проверяет экраны на каждой сборке:

```
import XCTest

final class AccessibilityAuditTests: XCTestCase {

    @MainActor
    func testTodoListPassesAccessibilityAudit() throws {
        guard #available(iOS 17.0, *) else {
            throw XCTSkip("Аудит доступности в UI-тестах есть с iOS 17")
        }
        let app = XCUIApplication()
        app.launch()
        app.cells.firstMatch.tap()        // открыть экран, который проверяем
        app.navigationBars.buttons.firstMatch.tap()

        try app.performAccessibilityAudit(for: [.contrast, .dynamicType, .hitRegion,
        ↪  .sufficientElementDescription]) { issue in
            // Вернуть true — «эту находку игнорируем»
            issue.auditType == .contrast && issue.element?.label == "Логотип"
        }
    }
}
```

Разбор:

- UI-тест (`XCUITest`) — тест, который запускает приложение и нажимает на элементы как пользователь. Таргет UI-тестов создаётся через File → New → Target → UI Testing Bundle.
- `guard #available(iOS 17.0, *)` — у нас минимальная версия iOS 15, а аудит появился в iOS 17. На старом симуляторе тест будет пропущен (`XCTSkip`), а не упадёт.
- `@MainActor` у метода — действия с `XCUIApplication` выполняются на главном потоке. Таргет UI-тестов в Xcode 26 создаётся без «MainActor по умолчанию», поэтому пометку ставим сами.
- `performAccessibilityAudit(for:)` — запустить проверки. Мы берём контраст, Dynamic Type, зоны нажатия и наличие описаний; без параметра проверяется всё.
- Замыкание получает каждую находку и возвращает `true`, если её надо **пропустить**. Здесь игнорируем контраст у логотипа — это картинка бренда, а не текст. Всё, что не пропущено, валит тест.

Мы проверили этот код компилятором; запускать его нужно в таргете UI-тестов на симуляторе iOS 17+.

43.5 Сценарии для VoiceOver на устройстве

Включить: Настройки → Универсальный доступ → VoiceOver. Удобнее назначить быстрое включение: Настройки → Универсальный доступ → Быстрая команда → VoiceOver, дальше тройное нажатие боковой кнопки.

Минимальный прогон:

1. **Запуск** — VoiceOver называет экран и первый элемент.
2. **Главное действие** — например, создать задачу. Получается ли пройти весь путь, не глядя на экран?
3. **Список и детальный экран** — листание тремя пальцами, открытие элемента двойным тапом.
4. **Модальный экран** — закрывается ли жестом «Z» или кнопкой «Закрыть».
5. **Форма** — поля названы, ошибки зачитываются.

Где VoiceOver сказал что-то непонятное («кнопка», «image 3») — нужен `accessibilityLabel` или `accessibilityHint`.

43.6 Dynamic Type: крайние размеры

Путь в iOS 26: Настройки → Универсальный доступ → Дисплей и размер текста → **Увеличенный текст**. Там ползунок из 7 обычных размеров; включив «Увеличенные размеры», получаешь ещё 5.

Размеры в числах — в таблице раздела 34.4: `.body` растёт от 14 pt на самом мелком размере до 53 pt на самом крупном, то есть 53 / 17 ≈ 3,12 — в три с лишним раза крупнее стандартных 17 pt. Строка, в которую при 17 pt помещалось 30 символов, при 53 pt вместит около десяти. Вот почему на крупных размерах ломаются горизонтальные стопки «иконка + текст + кнопка» — их нужно перестраивать в вертикальные.

Критерий Larger Text для этикетки из 43.1 — текст увеличивается **до 200% и больше**. Для `.body` это 34 pt и выше — то есть без «Увеличенных размеров» критерий не выполнить.

Что проверить:

- **Минимум** — текст всё ещё читается.
- **Стандарт** — дизайн как задуман.
- **Максимум обычных** и **максимум увеличенных** — текст не обрезается, кнопки растут вместе с текстом, ячейки становятся выше, экран прокручивается.

Упражнение. Открой Todo (глава 12) в симуляторе, в Accessibility Inspector → Settings выставь самый крупный размер. Запиши, что сломалось. Типичные находки и исправления — в конце главы.

43.7 Reduce Motion

Настройки → Универсальный доступ → Движение → **Уменьшение движения**.

Проверь:

- пружинные и масштабирующие переходы заменились на плавное появление или мгновенную смену;
- параллакс не работает;
- видео не запускается само;
- модальные экраны появляются без «вылета» снизу, если твой переход собственный.

Код переключения — глава 34.5: ветка по `UIAccessibility.isReduceMotionEnabled` и подписка на `UIAccessibility.reduceMotionStatusDidChangeNotification`.

43.8 Цветовые фильтры

Настройки → Универсальный доступ → Дисплей и размер текста → **Светофильтры**. Режимы:

- **Оттенки серого** — полностью монохромная картинка;
- **Фильтр красного/зелёного** (протанопия);
- **Фильтр зелёного/красного** (дейтеранопия);
- **Фильтр синего/жёлтого** (тританопия);
- **Оттенок** — окраска всего экрана.

Самый строгий тест — оттенки серого: если в сером приложение понятно, цвет нигде не единственный носитель смысла. Проверь:

- кнопки отличаются от текста;
- ошибки видны без красного;
- графики и статусы различимы.

43.9 На реальном устройстве

Симулятор — приближение. На телефоне:

- **VoiceOver** — только здесь (43.3).
- **Зоны нажатия** — палец намного толще курсора мыши; то, что легко нажать мышкой в симуляторе, может быть мучением пальцем.
- **Скорость** — анимации на Mac с мощным процессором плавнее, чем на телефоне трёхлетней давности.

Минимум — прогони основной сценарий на самом старом iPhone, который поддерживает твоя минимальная версия iOS.

43.10 Локализация

Если приложение на нескольких языках:

- ☐ Длинные слова (немецкий, казахский, русский) не обрезаются: там, где «Save» — 4 буквы, «Сохранить» — 9.
- ☐ Языки справа налево (арабский, иврит): отступы через `NSDirectionalEdgeInsets` (leading/trailing — «в начале/в конце строки»), а не `UIEdgeInsets` (left/right). Тогда при арабском интерфейсе отступы зеркалятся сами.
- ☐ Числа, даты и деньги — через форматтеры с локалью (глава 31): «1 234,50 ₸» в Казахстане и «₸1,234.50» в США — это одно число.
- ☐ Проверено в Xcode: схема → Run → Options → App Language, в том числе псевдоязыки «Double-Length Pseudolanguage» (удвоенные строки) и «Right-to-Left Pseudolanguage».

43.11 Производительность

Не про доступность напрямую, но из того же списка «проверить перед релизом»:

- ☐ **Зависания.** Инструменты Apple начинают сообщать о проблеме, когда главный поток не отвечает дольше **250 мс** (документация Understanding hangs in your app). При этом задержку больше **100 мс** в ответ на нажатие человек уже замечает.
- ☐ **Плавность прокрутки.** При 60 кадрах в секунду на один кадр остаётся 1000 / 60 ≈ **16,7 мс**; на экранах 120 Гц — около 8,3 мс. Всё, что дольше, — пропущенный кадр, «дёрганье».
- ☐ **Запуск.** Чем меньше работы в `didFinishLaunching`, тем лучше: UIKit вызывает его на главном потоке, и всё, что там выполняется, прибавляется ко времени запуска. Слишком долгий запуск система прерывает (watchdog).
- ☐ **Память и энергия** — датчики CPU, Memory и Energy Impact на вкладке Debug navigator в Xcode во время работы приложения.

Инструменты: Instruments → **Time Profiler** (на что уходит время), **Allocations** и **Leaks** (память и утечки), **Hangs** (зависания). После релиза те же метрики с реальных устройств — в Xcode Organizer.

43.12 Финальный чек-лист перед отправкой

Сборка

- ☐ Архив в конфигурации Release.
- ☐ Отладочные `print` и тестовые меню убраны или закрыты флагом `#if DEBUG`.
- ☐ Номер версии (Version) и номер сборки (Build) увеличены.

Разрешения

- ☐ Все `NS…UsageDescription` в `Info.plist` объясняют конкретную функцию (глава 39.6, Guidelines 5.1.1(ii)).
- ☐ Перед системными диалогами — экраны-объяснения (глава 7).
- ☐ Отказ в разрешении не ломает приложение: есть запасной путь (5.1.1(iv)).

Страница в App Store

- ☐ Скриншоты (глава 38).
- ☐ Анкета App Privacy (глава 39).

- ☐ `PrivacyInfo.xcprivacy` у приложения и расширений (глава 39).
- ☐ Ссылка на политику конфиденциальности — в App Store Connect и в приложении (5.1.1(i)).
- ☐ Описание, ключевые слова, Support URL.
- ☐ Accessibility Nutrition Labels — пока добровольно (43.1).

Аккаунт и данные

- ☐ Удаление аккаунта в приложении (глава 40, 5.1.1(v)).
- ☐ Выход из аккаунта работает.

Сеть

- ☐ Только HTTPS (глава 39.11).
- ☐ Обработаны таймауты, ошибки 4xx и 5xx.
- ☐ Без сети — понятный экран или баннер (глава 24.4).

Проверка

- ☐ Сборка прошла через TestFlight — бета-тестирование Apple на реальных устройствах.
- ☐ В App Review Notes — **тестовый аккаунт** и пароль, если без входа приложение не открыть. Guidelines 2.1 требует дать проверяющему рабочий демо-аккаунт или полноценный демо-режим.
- ☐ Если какие-то функции работают только с особым оборудованием или в определённой стране — объяснение в Review Notes.

43.13 После релиза

- **Крэши** — Xcode Organizer (раздел Crashes) собирает отчёты с устройств, где пользователи разрешили отправку аналитики; или сторонний сервис (Firebase Crashlytics) — не забудь указать его в анкете App Privacy.
- **Метрики** — Organizer: время запуска, зависания, расход энергии.
- **A/B-тесты страницы** — Product Page Optimization (глава 38.10).
- **Отзывы** — отвечай на них в App Store Connect: ответ разработчика виден всем.

43.14 Ответы к упражнениям

43.6 (Todo на максимальном размере). Типичные находки и что делать:

- Заголовок задачи обрезан многоточием — у лейбла ячейки `numberOfLines = 1`. Ставим `0`, высота ячейки автоматическая.
- Галочка и текст налезают друг на друга — горизонтальная стопка. Для категорий `accessibility…` меняем ось стопки на вертикальную: проверяем `traitCollection.preferredContentSize` и выставляем `stackView.axis = .vertical`.
- Текст не меняется, пока не перезапустишь экран — не хватает `adjustsFontForContentSizeCategory = true`.

- Шрифт не растёт вовсе — где-то `UIFont.systemFont(ofSize: 17)` вместо `preferredFont(forTextStyle: .body)`.

Проверка: после исправлений Audit в Accessibility Inspector не находит на экране обрезанного текста и текста без поддержки Dynamic Type.

43.15 Что мы прошли за книгу

От пустого Xcode-проекта до приложения, которое не стыдно отправить в App Store:

- **Фундамент** (часть I) — AppManifest, BootCoordinator, PlaygroundWindow, анимированный splash, жизненный цикл.
- **Launch-гейты** (часть II) — онбординг, экран-объяснение перед разрешениями, вход и регистрация с Keychain, принудительное обновление и техработы, регион и возраст, размытие и биометрия.
- **Мини-приложения** (часть III) — Todo, Notes, Calculator, Weather, Gallery, Music, Chat, Profile, Custom Tab Bar, Complex Layouts, Anatomy.
- **UI Cookbook** (часть IV) — загрузка, пустые и ошибочные состояния, поиск, навигация, ячейки, модальные экраны, жесты, формы, даты и деньги, анимации, haptics, доступность, темы, индикаторы, просмотр фото.
- **Production** (часть V) — скриншоты, приватность и манифест, удаление аккаунта, пуши и ссылки, виджеты и App Intents, аудит доступности.

Что мы выучили в этой главе

- **Accessibility Nutrition Labels** — девять функций; пока добровольно, в будущем обязательно. Отмечать только то, что работает во всех основных сценариях.
- **Чек-лист**: VoiceOver, Dynamic Type, контраст (4.5:1 до 17 pt, 3:1 от 18 pt и для жирного), зоны 44 × 44, Reduce Motion, не только цветом, клавиатура.
- **Accessibility Inspector**: Inspection, Audit, Settings. В симуляторе нет VoiceOver — только на устройстве.
- **`performAccessibilityAudit`** в UI-тестах (iOS 17+) — аудит на каждой сборке.
- **Dynamic Type**: `.body` от 14 до 53 pt; критерий Larger Text — 200% и больше; `adjustsFontForContentSizeCa` есть только у `UILabel`, `UITextField`, `UITextView` (у кнопки — на `titleLabel`).
- **Светофильтры**, особенно оттенки серого.
- **Производительность**: зависание — от 250 мс, кадр при 60 Гц — 16,7 мс; Time Profiler, Allocations, Hangs.
- **Финальный чек-лист**: сборка, разрешения, страница, аккаунт, сеть, TestFlight и демо-аккаунт для проверяющего (2.1).

Apple Developer Documentation

- Overview of Accessibility Nutrition Labels — девять функций доступности на странице App Store.
- HIG — Accessibility — контраст, размеры элементов, движение, VoiceOver.
- Accessibility Inspector — инспектор и аудит в Xcode.
- `performAccessibilityAudit(for:_:)` — аудит доступности в UI-тестах.

- `UIFontMetrics` — масштабирование своих шрифтов под Dynamic Type.
- `adjustsFontForContentSizeCategory` — автообновление шрифта; только для `UILabel`, `UITextField`, `UITextView`.
- `UIAccessibility` — флаги и уведомления настроек доступности.
- `accessibilityElements` — порядок чтения элементов для VoiceOver.
- Understanding hangs in your app — что считается зависанием и как его найти.
- Reducing your app's launch time — что влияет на время запуска.
- App Review Guidelines — 2.1 App Completeness — демо-аккаунт и Review Notes.

Это конец основного текста. Спасибо, что прошёл этот путь. iOS-разработка — большой мир, эта книга — карта основных районов. Дальше — практика на своём проекте. Нашёл ошибку или неясное место — открой issue в репозитории книги sirserik/uikit-playground-book.

Удачи в App Store!

→ Приложение. Словарь терминов

Приложение. Словарь терминов

Здесь собраны термины книги: всё, что относится к iOS, UIKit и экосистеме Apple, немного Swift Concurrency, программистский жаргон и математика, которая встречается в главах. Термины разбиты на темы, внутри каждой темы — по алфавиту, сначала русские названия, потом латинские. У каждого — объяснение в одну-две фразы и номера глав: первая — где термин появляется и объясняется впервые, остальные — где он разобран подробнее. Встретил незнакомое слово — загляни сюда, а за полным объяснением с примерами иди в указанную главу.

Карта UIKit: классы и наследование

Ветка ответчиков (`UIResponder`) — левая половина карты UIKit: приложение, сцена, окно, экраны и все view. Всё, что получает события. Правая половина — объекты, наследующие `NSObject` напрямую: слои, жесты, элементы панелей, цвета, constraints. (гл. 0)

Делегат и источник данных (delegate, data source) — два протокола, через которые класс UIKit общается с твоим кодом: делегату сообщает о событиях и задаёт вопросы, у источника данных запрашивает содержимое. Обе ссылки слабые, протоколы наследуют `NSObjectProtocol`. (гл. 0, 4, 12)

Иерархия классов (class hierarchy) — кто от кого наследуется. В UIKit почти всё стоит на лестнице `NSObject → UIResponder → UIView → UIControl → конкретный класс`; каждая ступень добавляет умения. Проверяется в рантайме через `class_getSuperclass`. (гл. 0, 22)

Контрол (`UIControl`) — view, которое умеет target-action и имеет состояния `isEnabled` / `isSelected` / `isHighlighted`. Контролы: `UIButton`, `UISwitch`, `UISlider`, `UITextField`, `UIRefreshControl`, `UIPageControl`, `UIDatePicker`. Не контролы: `UILabel`, `UIImageView`, `UISearchBar`, `UITextView`. (гл. 0, 12, 14)

Корневой класс (`NSObject`) — предок каждого класса UIKit. Даёт равенство по адресу, `description`, селекторы (`#selector` требует `@objc`) и возможность реализовать протоколы делегатов. Структуры `CGRect`, `IndexPath`, `UIControl.Event` его не наследуют. (гл. 0)

Ответчик (`UIResponder`) — объект, умеющий получать события (касания, клавиши, встряхивание) и передавать их по цепочке через `next`. Ступень между `NSObject` и `UIView`; на ней же стоят `UIViewController`, `UIWindow`, `UIScene`, `UIApplication`. (гл. 0, 3)

Первый ответчик (first responder) — объект, с которого начинается цепочка для событий без координат: клавиатура, встряхивание, команды меню. `canBecomeFirstResponder` по умолчанию `false`, у текстовых полей `true`; `becomeFirstResponder()` поднимает клавиатуру. (гл. 0, 3, 8)

Правило «нет `frame` — не view» — если у объекта нет `frame`, его нельзя `addSubview`, он не получает касаний и не участвует в Auto Layout: `CALayer`, `UIGestureRecognizer`, `UIBarButtonItem`, `UINavigationItem`, `UIAction`, `NSLayoutConstraint`, `UIPresentationController`. (гл. 0)

Ступень наследования — один класс в цепочке предков и набор умений, которые он добавляет: `NSObject` — личность, `UIResponder` — события, `UIView` — прямоугольник и раскладка, `UIControl` — target-action. (гл. 0)

Фреймворки под `import UIKit` — Foundation (`NSObject`, `NotificationCenter`), Core Graphics (`CGRect`, `CGFloat` — структуры), Core Animation (`CALayer`, `CAAnimation`) и сам UIKit. Префикс имени (`NS`, `CG`, `CA`, `UI`) подсказывает, откуда класс. (гл. 0)

Цепочка ответчиков (responder chain) — порядок передачи необработанного события: view → его view controller → superview → … → окно → сцена → `UIApplication` → `AppDelegate`. Правило «кто следующий» задаёт свойство `next`. (гл. 0, 3)

Экран — не view (`UIViewController`) — view controller наследует `UIResponder`, а не `UIView`: у него нет `frame` и `addSubview`, он управляет своим `view`, который создаётся лениво через `loadView`. (гл. 0, 1, 22)

`UIBarItem` — общий предок `UIBarButtonItem` и `UITabBarItem`: описание кнопки для панели, наследник `NSObject`, не view. Рисует его панель (`UINavigationBar`, `UITabBar`, `UIToolbar`). (гл. 0, 22, 26)

`UIScrollView` и наследники — `UITableView`, `UICollectionView` и `UITextView` наследуют scroll view, поэтому у всех трёх есть `contentOffset`, `contentInset`, `refreshControl`, а их делегаты наследуют `UIScrollViewDelegate`. (гл. 0, 12, 21)

Приложение и жизненный цикл

Восстановление состояния (state restoration) — вернуть пользователя на тот экран, где он был, после того как система выгрузила приложение из памяти. В приложении со сценами делается через `NSUserActivity` сцены; в книге не используется. (гл. 5)

Главный экран mini-app (main) — экран, ради которого mini-app существует, после всех гейтов: список дел, калькулятор, чат. (введение, гл. 4)

Делегат приложения (`AppDelegate`) — объект, которому система сообщает о событиях всего процесса: запуск, получение device token для пушей, конфигурация новых сцен. Экраны и окно в нём больше не создают. (гл. 5)

Делегат сцены (`SceneDelegate`) — объект событий одной сцены: подключилась, стала активной, ушла в фон. В `scene(_:willConnectTo:options:)` он создаёт окно и первый экран. (гл. 5)

Деплоймент-таргет (deployment target) — самая старая версия iOS, на которую можно установить приложение. В книге это iOS 15.0; всё новее обёрнуто в `@available` или `if #available`. (введение, гл. 12)

Жизненный цикл (lifecycle) — порядок состояний, через которые проходит приложение или экран, и методы, которые iOS вызывает на каждом переходе: «запустилось → активно → в фоне». У экрана свой цикл: `viewDidLoad` → `viewWillAppear` → `viewDidAppear` → `viewWillDisappear` → `viewDidDisappear`. (гл. 1, 5)

Ключевое окно (key window) — окно, которое получает ввод с клавиатуры и события без конкретной точки касания. Окно становится ключевым и видимым после `makeKeyAndVisible()`. (гл. 3, 5)

Корневой экран (`rootViewController`) — view controller, который окно показывает целиком. Окно — рама картины, корневой экран — сама картина: её меняют, рама остаётся. (гл. 3, 4)

Окно (`UIWindow`) — самый верхний контейнер интерфейса: всё, что видно на экране, рисуется внутри окна. Клавиатура и системные алерты живут в своих окнах поверх твоего. (гл. 3)

Снимок для переключателя приложений (snapshot) — картинка интерфейса, которую UIKit делает после ухода в фон и показывает в переключателе приложений. Хранится на диске, поэтому пароли и номера карт перед снимком закрывают размытием. (гл. 11)

Состояния сцены — **active** (на экране, принимает касания), **inactive** (видна, но касаний нет: открыт переключатель приложений, Центр уведомлений, пришёл звонок) и **background** (не видна, через несколько секунд iOS обычно её приостанавливает). (гл. 5, 11)

Сцена (`UIScene`, `UIWindowScene`) — один «экземпляр» интерфейса приложения со своими окнами. На iPhone она обычно одна, на iPad их может быть несколько: две заметки рядом — две сцены. Появились в iOS 13, а по заметке Apple TN3187 приложение без сцен после iOS 26 перестанет запускаться. (гл. 3, 5, 11)

Фоновые задачи (`BGTaskScheduler`) — механизм, который периодически будит свёрнутое приложение, чтобы обновить данные. Регистрацию делают до конца запуска; в книге не используется. (гл. 5)

Холодный старт (cold start) — запуск приложения, которого нет в памяти: процесс создаётся с нуля. На старом телефоне он доходит до секунды-двух, и всё это время виден только launch screen. Ссылка, открывшая приложение при холодном старте, приходит в `connectionOptions`. (гл. 1, 5, 41)

Application Scene Manifest (`UIApplicationSceneManifest`) — ключ `Info.plist` с настройками сцен: сколько окон можно открыть и какой класс сделать делегатом сцены. Имя конфигурации в нём должно совпадать с тем, что возвращает `AppDelegate`, буква в букву. (введение, гл. 5)

Bundle (бандл) — папка `.app`, в которую Xcode складывает бинарник, картинки, `Info.plist` и другие ресурсы. Файлы вроде `PrivacyInfo.xcprivacy` лежат прямо в ней. (гл. 7, 39)

Bundle ID — идентификатор приложения из настроек таргета, например `kz.waid.uikitplayground`. По нему `simctl` сбрасывает разрешения, а Keychain и APNs отличают твоё приложение от чужих. (гл. 7, 8, 41)

`didFinishLaunchingWithOptions` — метод `AppDelegate`: «процесс запущен, приложение готово к работе». Возвращает `true`, если всё в порядке. (гл. 5)

Info.plist — файл-анкета приложения: название, версия, минимальная iOS, строки-пояснения разрешений, настройки сцен. iOS читает его до запуска твоего кода; часть ключей Xcode 26 собирает из настроек сборки `INFOPLIST_KEY_…`. (введение, гл. 5, 7)

Launch screen — системная заставка, которую iOS показывает с первого кадра, пока приложение запускается. Твой код в этот момент ещё не работает, поэтому экран описывают storyboard'ом `LaunchScreen` или ключом `UILaunchScreen` (iOS 14+), и iOS кэширует его снимок. (гл. 1)

Splash — заставка, которую приложение показывает само после launch screen: анимированная иконка, название, индикатор. В отличие от launch screen, это обычный view controller с кодом. (введение, гл. 1)

`UIApplication` — объект процесса, один на всё приложение: верхний уровень схемы «приложение → сцена → окно → экран». Через `UIApplication.shared.open(url)` открывают ссылки. (гл. 5, 9)

`UISceneSession` — «карточка» сцены, которую хранит система: тип, конфигурация, сохранённое состояние. Сцену могут выгрузить из памяти, а по карточке её потом восстановят. (гл. 5)

`viewDidLoad` — метод view controller'а, который UIKit вызывает один раз, когда view экрана создан. Здесь собирают интерфейс; экран ещё не на экране, для запуска анимаций это рано — их запускают в `viewDidAppear`. (гл. 1, 5)

`willConnectTo` — метод `scene(_:willConnectTo:options:)` делегата сцены: сцена создана, пора собрать окно и первый экран. Вызывается один раз на сцену, при возвращении из фона — нет. (гл. 5)

Экраны и навигация

Аниматор перехода (`UIViewControllerAnimatedTransitioning`) — объект, который описывает свою анимацию перехода между экранами: длительность (`transitionDuration`) и саму анимацию. Контроллер навигации или модальный показ спрашивает у делегата, есть ли такой аниматор. (гл. 26, 28)

Детент (detent) — «фиксатор», высота, на которой может остановиться лист. В iOS 15 есть `.medium()` (примерно половина экрана) и `.large()` (почти весь экран), произвольная высота `.custom` — с iOS 16. (гл. 12, 17, 28)

Дочерний экран (child view controller) — экран, который контейнер держит внутри себя. Добавление — три шага: `addChild`, положить `child.view` в иерархию с constraints и вызвать `didMove(toParent:)` — последний вызов UIKit за тебя не делает. (гл. 6, 20)

Контейнер (container view controller) — view controller, который показывает внутри себя другие экраны и рисует вокруг «обрамление»: `UINavigationController`, `UITabBarController`, `UIPageViewController`. Как рамка со сменными вкладышами. (гл. 6, 20)

Лист (sheet) — модальный экран, который выезжает снизу и оставляет видным край предыдущего. Его тянут вниз, чтобы закрыть, а высоту задают детентами через `sheetPresentationController` (iOS 15+). (гл. 12, 17, 28)

Лист «Поделиться» (`UIActivityViewController`) — стандартный share sheet: передаёшь массив объектов, и iOS сама показывает подходящие приложения и действия — «Сообщения», «Почта», AirDrop, «Скопировать». (гл. 13, 37)

Модальный показ (modal) — экран показывают поверх текущего вызовом `present`, а закрывают `dismiss`. Правило: человек выполняет отдельную задачу и закрывает её («новая заметка») — модально; смотрит подробности и возвращается — push. (гл. 4, 26, 28)

Навигационная панель (navigation bar) — полоса сверху `UINavigationController` с заголовком и кнопками. Её вид задаёт `UINavigationBarAppearance`: `scrollEdgeAppearance` — когда список в самом начале, `standardAppearance` — когда контент уехал под панель. (гл. 4, 21, 26)

Поповер (popover) — всплывающее окно со стрелкой на iPad. Ему нужно знать, на что указывать стрелкой (`popoverPresentationController?.sourceView` или `barButtonItem`), иначе показ листа «Поделиться» на iPad упадёт. (гл. 13, 19, 28)

Стек навигации (push / pop) — `UINavigationController` держит экраны стопкой: push кладёт новый сверху (он выезжает справа), кнопка «Назад» или свайп от левого края снимают верхний. (гл. 4, 8, 26)

Экран (view controller) — объект-наследник `UIViewController`, который управляет одним экраном: создаёт его содержимое, реагирует на появление, исчезновение и нажатия. Экран — комната, view controller — её хозяин. (гл. 1)

Action sheet — стиль `UIAlertController` со списком действий снизу экрана для решений полегче: выйти, поделиться, выбрать вариант. Для серьёзного («удалить») берут `.alert` посередине экрана. (гл. 19, 28)

Grabber — серая полоска-«ручка» вверху листа (`prefersGrabberVisible = true`): подсказка, что лист можно тянуть. (гл. 12, 17)

`isModalInPresentation` — запрет закрывать лист свайпом вниз. Ставят, когда в листе несохранённый текст, и спрашивают подтверждение в `presentationControllerDidAttemptToDismiss`. (гл. 28)

Large title — крупный заголовок, как в «Настройках», который при прокрутке сжимается в обычный. Включается `prefersLargeTitles = true` у навигационной панели. (гл. 5, 26)

`modalPresentationStyle` — как показать модальный экран: `.pageSheet` (лист, стиль по умолчанию), `.fullScreen` (на весь экран), `.formSheet` (карточка по центру на iPad) и другие. (гл. 16, 17, 28)

`UIAlertController` — системное окно с заголовком, текстом и кнопками в стиле `.alert` или `.actionSheet`. Можно добавить в него текстовое поле. (гл. 19, 28)

`UIMenu` — всплывающее меню действий у кнопки. С `showsMenuAsPrimaryAction = true` оно открывается обычным тапом, без него — только долгим нажатием. (гл. 13, 19, 25)

`UIPageControl` — ряд точек-индикаторов страниц: активная точка ярче. Сам по себе интерактивный: тап справа от активной точки листает вперёд. (гл. 6, 36)

`UIPageViewController` — готовый контейнер для страниц, которые листают пальцем: ты отдаёшь ему страницы «до» и «после» текущей через data source. (гл. 6, 37)

`UISearchController` — системный контроллер поиска: строка поиска с кнопкой «Отменить» и логикой её появления и скрытия. Встраивается в навигационную панель через `navigationItem.searchController`. (гл. 13, 25)

`UISplitViewController` — контейнер на две-три колонки, главным образом для iPad: список слева, содержимое справа. (гл. 20)

`UITabBarController` — стандартный таб-бар: полоса с иконками внизу, тап переключает раздел. На iPhone помещается до пяти вкладок, лишние уходят в «Ещё»; бейджи и VoiceOver даёт бесплатно. (гл. 20, 26)

Back item (кнопка «Назад») — панель навигации берёт кнопку «назад» из `navigationItem` экрана **под** верхним: `backBarButtonItem`, `backButtonTitle`, `backButtonDisplayMode` настраивают в экране, куда вернёмся. (гл. 0, 26)

Container view controller (контейнер) — view controller без своего содержимого, который держит дочерние экраны в `children`: `UINavigationController`, `UITabBarController`, `UIPageViewController`, свой контейнер в главе 20. Порядок встраивания: `addChild` → view в иерархию → `didMove(toParent:)`. (гл. 0, 6, 20)

Presentation controller (`UIPresentationController`) — объект-`NSObject`, который ведёт модальный показ: размер, затемнение, лист или popover. Наследники — `UISheetPresentationController` и `UIPopoverPresentationController`. Не view и не view controller. (гл. 0, 17, 28)

Presenting / presented view controller — показывающий и показанный экраны. `dismiss` у показанного UIKit пересылает показывающему. (гл. 0, 22, 28)

Presentation context — экран, чью view накрывает показ в стиле `.currentContext` / `.overCurrentContext` и `UISearchController`; задаётся `definesPresentationContext`, у `UINavigationController` по умолчанию `true`. (гл. 13, 25)

`shouldRemovePresentersView` — убирать ли view нижнего экрана при показе; `true` только у `.fullScreen`, поэтому лист не вызывает `viewWillDisappear` у нижнего экрана. (гл. 28, 37)

`UISearchTextField` — поле ввода внутри `UISearchBar` (`searchTextField`, iOS 13+): наследник `UITextField`, то есть контрол, хотя сам `UISearchBar` — нет. (гл. 22, 25)

`loadView` / `isViewLoaded` / `viewIfLoaded` — `view` экрана создаётся лениво при первом обращении; `loadView` создаёт его (не вызывать самому), `isViewLoaded` отвечает, создан ли, не создавая. (гл. 0, 1)

`viewIsAppearing(_:)` — метод экрана (iOS 13+, объявлен в SDK iOS 17), который приходит после `viewWillAppear`, когда view уже в иерархии и у него верные размеры и трейты. (гл. 0, 34, 35)

`clearsSelectionOnViewWillAppear` — свойство `UITableViewController`, по умолчанию `true`: при возврате на экран выделение строки снимается само. (гл. 4)

Раскладка

Внутренний размер (intrinsic content size) — размер, который view хочет занимать по своему содержимому: у надписи — размер текста, у картинки — размер иконки. Auto Layout берёт его, если ширина или высота не заданы. (гл. 7, 12, 27)

Дробный размер (`fractionalWidth`, `absolute`, `estimated`) — размеры в compositional layout: `.fractionalWidth(0.5)` — половина ширины контейнера, `.absolute(60)` — ровно 60 точек, `.estimated(44)` — «примерно 44», настоящий размер посчитает Auto Layout. (гл. 16, 21)

Направляющая (layout guide, `UILayoutGuide`) — невидимый прямоугольник с якорями, к которому привязывают constraints, как к обычному view; наследник `NSObject`, не view. Самые известные — `safeAreaLayoutGuide`, `layoutMarginsGuide`, `readableContentGuide` и `keyboardLayoutGuide`. (гл. 0, 1, 15, 18, 30)

Направляющая клавиатуры (`keyboardLayoutGuide`) — направляющая iOS 15+, которая совпадает с клавиатурой, а без клавиатуры — с низом safe area. Одна строка `textView.bottomAnchor`

`== keyboardLayoutGuide.topAnchor` заменяет ручную подписку на уведомления клавиатуры. (гл. 13, 18, 30)

Параллакс (parallax) — элементы при прокрутке движутся с разной скоростью, и это даёт ощущение глубины. В главе 21 текст шапки сдвигается на `-offset × 0,2`, то есть едет со скоростью 0,8 от шапки. (гл. 21)

Поля (layout margins) — стандартные отступы по краям view: на iPhone в портрете обычно 16 или 20 точек. Текст, прижатый к полям (`layoutMarginsGuide`), не липнет к краю экрана. (гл. 1, 6)

Приоритет constraint (priority) — число от 1 до 1000: 1000 (`.required`) — обязательно, 750 (`.defaultHigh`) — «очень хочу, но могу уступить», 250 (`.defaultLow`) — «если получится». Необязательный constraint уступает, вместо того чтобы сломать раскладку. (гл. 14, 27)

Прокрутка (`UIScrollView`) — view, содержимое которого больше рамки и сдвигается пальцем. Внутри две направляющие: `frameLayoutGuide` — видимая рамка на экране, `contentLayoutGuide` — всё прокручиваемое содержимое. (гл. 6, 8, 15, 21)

Рамка и границы (`frame` и `bounds`) — `frame` — прямоугольник view в координатах родителя, `bounds` — тот же прямоугольник в собственных координатах view. У окна iPhone 16 `bounds` — (0, 0, 393, 852) точки. (гл. 1, 11)

Сдвиг прокрутки (`contentOffset`) — насколько содержимое прокручено: точка содержимого, которая сейчас стоит в левом верхнем углу. Прокрутил список на 100 точек вниз — `contentOffset.y = 100`; при ширине страницы 390 сдвиг 780 значит третью страницу (780 / 390 = 2, счёт с нуля). (гл. 6, 21)

Сопротивление сжатию (compression resistance) — приоритет, с которым view не даёт сжать себя меньше внутреннего размера. У температуры «23°» он максимальный, поэтому на узком экране сожмётся описание, а не число. (гл. 15, 27)

Стек (`UIStackView`) — контейнер, который сам выстраивает вложенные view в столбик или строку с промежутком `spacing`. Constraints задают только самому стеку; `distribution = .fillEqually` делает все элементы одинаковыми. (гл. 2, 14)

Сторона начала и конца (leading / trailing) — «от начала» и «до конца» строки вместо «слева» и «справа». Для языков с письмом справа налево (RTL — арабский, иврит) iOS зеркалит интерфейс, и `leading` сам становится правой стороной. (гл. 9, 16)

Точка (point, pt) — единица размеров в UIKit, не пиксель. На экране @3x в одной точке 3 × 3 = 9 пикселей, на @2x — 2 × 2 = 4. iPhone 17 Pro Max шириной 440 точек — это 440 × 3 = 1320 пикселей. (гл. 1, 16, 38)

Тянущаяся шапка (stretchy header) — шапка над списком, которая растягивается, если потянуть список вниз дальше начала, а при прокрутке вверх уезжает вместе с контентом. (гл. 21)

Удобная для чтения ширина (`readableContentGuide`) — полоса, которая на iPhone почти совпадает с полями, а на широком iPad ограничивает ширину текста, чтобы строки не тянулись на весь экран. (гл. 2)

Уступчивость к растяжению (content hugging) — приоритет, с которым view «обнимает» своё содержимое и не даёт растянуть себя шире внутреннего размера. Лишнее место в ряду достаётся тому, у кого hugging ниже. (гл. 7, 15, 27)

Auto Layout — система правил, по которым UIKit сам вычисляет размеры и положение элементов: «лейбл на 20 точек ниже иконки, по центру». Раскладка получается верной на любом iPhone, iPad и в любой ориентации. (введение, гл. 1)

Autoresizing mask — старый способ раскладки через `frame`: `[.flexibleWidth, .flexibleHeight]` растягивает view вслед за родителем. Нужен для view, созданных рамкой, а не constraints. (гл. 11, 23)

Compositional layout (`UICollectionViewCompositionalLayout`) — раскладка коллекции из трёх уровней: item (ячейка) → group (ряд или колонка из ячеек) → section (раздел из повторяющихся групп). Появилась в iOS 13. (гл. 16, 21)

Constraint (ограничение) — одно правило Auto Layout: «ширина = ширина соседа × 2 + 12», «низ стека = верх клавиатуры». Правила включают пачкой через `NSLayoutConstraint.activate`. Если правила противоречат друг другу, в консоли появится «Unable to simultaneously satisfy constraints». (введение, гл. 1, 14)

`contentInset` — внутренний отступ прокрутки: `contentInset.bottom = 88` даёт прокрутить список на 88 точек дальше, чтобы последняя строка вышла из-под плавающей кнопки. `adjustedContentInset` — тот же отступ плюс то, что добавила система под safe area. (гл. 12, 21)

`contentMode` — как картинка вписывается в свой view: `.scaleAspectFill` заполняет рамку целиком и обрезает лишнее (вместе с `clipsToBounds = true`), `.scaleAspectFit` вписывает целиком с полями. (гл. 16, 21)

`layoutIfNeeded()` — заставляет Auto Layout пересчитать рамки прямо сейчас. Внутри `UIView.animate` это превращает смену constraint в плавную анимацию. (гл. 13, 20, 32)

`layoutSubviews` / `viewDidLayoutSubviews` — методы, которые UIKit вызывает после раскладки view или экрана. Здесь подгоняют то, что не участвует в Auto Layout: размер отдельного слоя, рамку картинки. (гл. 16, 21, 23)

Safe area (безопасная зона) — часть экрана, которую ничего не перекрывает: ни «чёлка» или Dynamic Island сверху, ни полоска «домой» снизу. На iPhone 16 Pro нижний отступ зоны — 34 точки, и кнопка с отступом 20 окажется в 34 + 20 = 54 точках от края. (гл. 1, 12, 21)

Size class — грубая оценка ширины и высоты экрана: «compact» на iPhone в портрете, «regular» на iPad. Адаптивное приложение переключает раскладку по нему. (гл. 20, 35)

Sticky header («липкий» заголовок) — заголовок раздела, который при прокрутке прилипает к верху, пока его раздел на экране. Так ведут себя заголовки таблицы в стиле `.plain`; в compositional layout это `pinToVisibleBounds`. (гл. 21)

Supplementary view (дополнительный вид) — элемент коллекции, который не является ячейкой с данными: заголовок раздела, подвал с индикатором загрузки, значок. (гл. 16, 21)

`translatesAutoresizingMaskIntoConstraints = false` — обязательная строка для view, которое расставляешь constraints в коде. Без неё UIKit создаст правила из `frame`, они столкнутся с твоими, и раскладка сломается. (гл. 1, 6)

@2x и @3x — множитель экрана: сколько пикселей в одной точке по каждой стороне. Для превью шириной 131 точка на экране @3x нужна картинка 131 × 3 = 393 пикселя; текущий множитель — `traitCollection.displayScale`. (гл. 1, 16, 38)

Layout margins (поля) — `layoutMargins` / `layoutMarginsGuide`: у обычного view 8 точек с каждой стороны, у корневого view экрана — системные минимальные с учётом safe area (на iPhone 16: 16 по бокам, 59 сверху, 34 снизу). (гл. 0, 1, 17, 19)

`followsUndockedKeyboard` — флаг `keyboardLayoutGuide`: отслеживать ли отстыкованную или плавающую клавиатуру iPad; по умолчанию `false`, тогда направляющая стоит у низа safe area. (гл. 30)

Compression resistance (сопротивление сжатию) — приоритет, с которым view не даёт сжать себя меньше `intrinsicContentSize`; по умолчанию 750. Hugging — сопротивление растяжению, 250. (гл. 0, 7, 15)

Arranged subview — view из `arrangedSubviews` стека: для неё стек сам строит constraints и сам ставит `translatesAutoresizingMaskIntoConstraints = false`. Подмножество `subviews`; `removeArrangedSubview` не удаляет из иерархии; скрытая не занимает места. (гл. 2, 14, 17)

Distribution и alignment стека — правило деления места вдоль оси (`.fill`, `.fillEqually`, `.fillProportionally`, `.equalSpacing`, `.equalCentering`) и расстановки поперёк (`.fill`, `.leading`/`.top`, `.center`, `.trailing`/`.bottom`, `.firstBaseline`/`.lastBaseline`). (гл. 14)

Non-rendering view — view, которая только расставляет subviews и ничего не рисует сама: `UIStackView` запрещает переопределять `layerClass` и `draw(_:)`; фон у стека появился с iOS 14. (гл. 15)

Списки и ячейки

Индекс строки (`IndexPath`) — пара «секция, строка», которой таблица и коллекция называют конкретную ячейку. Все их методы данных и событий получают его параметром. (гл. 4, 12)

Источник данных (data source) — объект, который отвечает списку на вопросы «сколько строк?» и «что показать в строке N?». Таблица сама данные не хранит, она спрашивает. (гл. 4, 6)

Переиспользование ячеек (reuse) — таблица не создаёт ячейку на каждую строку: при тысяче строк и десяти видимых ячеек будет около десяти. Строка ушла за край — её ячейка вернулась в запас и пришла заново для другой строки, поэтому в `configure` задают все свойства, включая «пустые». (гл. 4, 10, 12)

Пул ячеек и идентификатор (`reuseIdentifier`) — имя, под которым класс ячейки регистрируют в таблице (`register(_:forCellReuseIdentifier:)`), а потом просят готовую ячейку через `dequeueReusableCell(withIdentifier:for:)`. (гл. 4, 10)

Пустой фон таблицы (`backgroundView`) — view, которое таблица рисует под всеми строками. Удобное место для надписи «пусто»: оно само растягивается на всю таблицу. (гл. 12, 24)

Самоподстройка высоты (self-sizing cells) — `rowHeight = UITableView.automaticDimension`: высоту строки считает Auto Layout по содержимому. `estimatedRowHeight` — примерная высота, по которой таблица заранее оценивает длину списка. (гл. 18, 27)

Свайп-действия (swipe actions) — кнопки, которые открываются смахиванием строки: trailing — у правого края, палец ведёшь справа налево («Удалить»), leading — у левого края, палец слева направо («Готово»). Каждая кнопка — `UIContextualAction`. (гл. 12, 29)

Снимок данных (snapshot) — в diffable data source полный список разделов и элементов, который ты отдаёшь источнику данных вместо ручных `insertItems`. Источник сам сравнит его со старым и анимирует разницу. (гл. 16)

Таблица (`UITableView`) — вертикальный прокручиваемый список строк с секциями и переиспользуемыми ячейками. `UITableViewController` — готовый экран, у которого корневой view — таблица. (гл. 4, 12)

Accessory (аксессуар ячейки) — значок справа в строке: `disclosureIndicator` (серая стрелка «›» — «откроется новый экран»), `checkmark` (галочка) или свой view через `accessoryView`. (гл. 4, 10, 19)

Content configuration (`UIListContentConfiguration`) — готовая раскладка содержимого стандартной ячейки (iOS 14+): картинка, основной и дополнительный текст. `defaultContentConfiguration()` даёт заготовку, заполняешь поля и присваиваешь в `cell.contentConfiguration`. (гл. 4, 10, 27)

Diffable data source — источник данных, который сам вычисляет разницу между старым и новым списком и анимирует вставки и удаления. Требует, чтобы элементы были `Hashable`: по идентификатору он узнаёт «ту же» строку. (гл. 12, 16)

`.insetGrouped` — стиль таблицы, где строки собраны в скруглённые «карточки» с отступами по бокам на сером фоне, как в «Настройках». (гл. 4, 10, 19)

`performBatchUpdates` — пакетное изменение коллекции или таблицы: «в конец добавлено 30 элементов». Видимые ячейки не трогаются, новые появляются с анимацией; к моменту вызова массив данных уже должен быть обновлён. (гл. 16, 18)

Prefetching (предзагрузка) — `UICollectionViewDataSourcePrefetching` заранее сообщает номера элементов, которые скоро понадобятся, чтобы начать грузить их картинки до появления на экране. (гл. 16)

`prepareForReuse()` — метод ячейки, который система вызывает прямо перед тем, как отдать её для новой строки. Здесь сбрасывают старое: замыкание, картинку, начатую загрузку. (гл. 12, 16, 27)

`reloadData()` — таблица заново спрашивает data source обо всём и перерисовывает видимые строки. Для одной-двух изменившихся строк дешевле `reloadRows(at:with:)`. (гл. 10, 12)

`UICollectionView` — вид для коллекции элементов с любой раскладкой: сетка, карусель, список. Где стоит каждая ячейка, решает отдельный объект — layout. (гл. 6, 16)

`UIRefreshControl` — стандартный pull-to-refresh: крутилка, которая появляется, если потянуть список вниз. Это `UIControl`, поэтому его событие — `.valueChanged`. (гл. 15, 22, 23)

`willDisplay` — метод делегата «эта ячейка сейчас появится на экране». Для пагинации: загружено 20 строк, условие `row >= count - 6` сработает на строке 14, и новая страница успеет прийти, пока пользователь долистает оставшиеся. (гл. 16, 23)

Self-sizing cells — ячейки, высоту которых считает Auto Layout: `rowHeight` и `estimatedRowHeight` по умолчанию равны `UITableView.automaticDimension` (это −1), содержимое прибито к `contentView` сверху и снизу. (гл. 0, 12, 27)

`contentView` ячейки — штатный superview содержимого: ячейка меняет его размер в режиме редактирования и под аксессуар. Всё своё кладут туда, а не в саму ячейку. (гл. 0, 12, 27)

Layout object (объект раскладки коллекции) — `UICollectionViewLayout`, `UICollectionViewFlowLayout`, `UICollectionViewCompositionalLayout`: наследник `NSObject`, считает положение элементов и не рисует. (гл. 0, 16)

Ввод, касания и жесты

Дискретный и непрерывный жест — дискретный (тап, свайп) срабатывает один раз и сразу переходит в `.ended`. Непрерывный (pan, pinch, долгое нажатие) шлёт `.began`, много `.changed` и `.ended`; если его прервала система, будет `.cancelled`. (гл. 29)

Инерция жеста — сколько карточка ещё проедет после отпускания, если тормозить как обычный список iOS. При скорости 1000 точек в секунду получается около 499 точек: путь по инерции — примерно половина скорости. (гл. 29)

Клавиатура по типу поля (`keyboardType`) — какую клавиатуру показать: `.emailAddress` (с `@` и точкой на виду), `.phonePad`, `.decimalPad`. `returnKeyType` меняет только надпись на синей кнопке («Далее», «Готово»), поведение пишешь сам в `textFieldShouldReturn`. (гл. 8, 30)

Контекстное меню (`UIContextMenuConfiguration`) — меню с превью, которое открывается долгим нажатием на ячейку или view. (гл. 16, 29)

Минимальная область касания — 44 × 44 точки по Human Interface Guidelines: в меньшую цель трудно попасть пальцем. Значок может быть 22 точки, а «мишень» вокруг него — 44. (гл. 8, 12, 34)

Отклик вибромотором (haptics) — короткие вибрации Taptic Engine, которые ощущаются как щелчок. Работает только на реальном iPhone; симулятор и iPad молчат, а пользователь может выключить системные тактильные сигналы, поэтому отклик — всегда дополнение к видимому результату. (введение, гл. 14, 17, 33)

Первый ответчик (first responder) — объект, которому событие достаётся первым: для касания — view под пальцем, для клавиатуры — поле, где мигает курсор. `becomeFirstResponder()` ставит курсор в поле и показывает клавиатуру, `resignFirstResponder()` убирает её. (гл. 3, 12)

Порог жеста — граница, после которой жест «засчитан». Шторка из главы 20 открывается, если бросок быстрее 500 точек в секунду или открыта больше чем наполовину; модальный экран закрывается, если его оттащили дальше 150 точек или отпустили быстрее 1000 точек в секунду. (гл. 20, 29)

Проверка попадания (hit-testing) — поиск view, которому достался тап: система идёт от верхних view к нижним и отдаёт касание самому верхнему, который лежит под пальцем и принимает касания. Невидимый view с `isUserInteractionEnabled = true` поверх контента «съест» все тапы. (гл. 20, 21)

Распознаватель жестов (`UIGestureRecognizer`) — объект, который следит за касаниями на view и сообщает, что случился жест: `UITapGestureRecognizer`, `UIPanGestureRecognizer`, `UIPinchGestureRecognizer`, `UILongPressGestureRecognizer`. Добавляется `addGestureRecognizer(_:)`, о результате сообщает через target-action. (гл. 29)

Скорость жеста (`velocity(in:)`) — как быстро движется палец, в точках в секунду. (800, 0) — вправо со скоростью 800 точек в секунду: за секунду палец прошёл бы примерно два экрана iPhone по ширине. (гл. 20, 29)

Смещение жеста (`translation(in:)`) — на сколько точек палец сдвинулся с начала жеста: вправо и вниз — плюс, влево и вверх — минус. Позиция в `.changed` = «где было в `.began`» + смещение. (гл. 20, 29)

Цепочка ответчиков (responder chain) — порядок, в котором необработанное событие передаётся дальше через свойство `next`: view → родительский view → view controller → окно → сцена → `UIApplication` → `AppDelegate`. Ответчик (responder) — любой наследник `UIResponder`. (гл. 3, 22)

Щипок (pinch) — жест двумя пальцами для масштаба. Pinch-to-zoom у фотографии умеет сам `UIScrollView` через `viewForZooming` и `maximumZoomScale`. (гл. 16, 29, 37)

`isUserInteractionEnabled` — принимает ли view касания. У `UILabel` и `UIImageView` по умолчанию `false`, поэтому жест на надписи молча не сработает, пока не включишь. (гл. 14, 20)

Pan — жест «положил палец и тянешь». `UIScreenEdgePanGestureRecognizer` — его вариант, который срабатывает, только если палец начал движение у самого края экрана. (гл. 20, 29)

Password AutoFill — подсказка сохранённого логина и пароля над клавиатурой. Включается правильным `textContentType`; сгенерировать надёжный пароль для `.newPassword` iOS предложит, только если приложение связано с доменом через Associated Domains. (гл. 8, 30)

`prepare()` у генератора отклика — «разбудить» Taptic Engine заранее: первый отклик после паузы иначе может опоздать. Вызывают незадолго до ожидаемого срабатывания, например когда палец коснулся ползунка. (гл. 14, 17, 33)

`require(toFail:)` — «этот жест срабатывает, только когда другой провалился». `singleTap.require(toFail: doubleTap)` не даёт одинарному тапу сработать на первом касании двойного; цена — небольшая задержка одинарного. (гл. 29, 37)

`shouldRecognizeSimultaneouslyWith` — метод делегата жеста, который разрешает двум жестам работать вместе: щипок и поворот фото, свой pan поверх прокрутки. (гл. 29)

Target-action — старый способ подписки на событие: «при `.touchUpInside` вызови у объекта метод `checkboxTapped`». Метод помечают `@objc`, потому что механизм пришёл из Objective-C и ищет метод по имени. С iOS 14 то же делает замыкание: `addAction(UIAction { … }, for:)`. (гл. 12, 14, 29)

`textContentType` — подсказка системе, что это за поле: `.username`, `.password`, `.newPassword`, `.oneTimeCode`, `.telephoneNumber`. По ней iOS предлагает сохранённые пароли, код из SMS или номер из «Контактов». (гл. 8, 30)

`UIButton.Configuration` — описание кнопки целиком (iOS 15+): текст, цвета, скругление, отступы, спиннер внутри (`showsActivityIndicator`). Конфигурацию нельзя поправить на месте: копируешь в переменную, меняешь и присваиваешь обратно. (гл. 6, 8)

`UIControl` — базовый класс элементов с событиями: `UIButton`, `UISwitch`, `UISlider`, `UITextField`, `UIRefreshControl`. У всех одинаковые `addAction(_:for:)` и события вроде `.valueChanged` и `.touchUpInside`. (гл. 17, 22)

`UIDatePicker` — выбор даты: барабаны (`.wheels`) или компактная кнопка с календарём (`.compact`). «Пустым» он быть не умеет, поэтому «без даты» выражают отдельным переключателем. (гл. 10, 12, 31)

`UIImpactFeedbackGenerator` — генератор отклика-«удара» заданной силы: `.light`, `.medium`, `.heavy`, `.soft`, `.rigid`. `impactOccurred(intensity: 0.6)` — удар на 60% силы. (гл. 14, 17, 33)

`UINotificationFeedbackGenerator` — отклик-«сообщение» трёх видов: `.success`, `.warning`, `.error`. `.error` ощущается как быстрое «тук-тук-тук», его трудно спутать с обычным нажатием. (гл. 14, 33)

`UISelectionFeedbackGenerator` — самый тонкий «тик» для смены выбора: прокрутка барабана, засечки ползунка. (гл. 14, 33)

`UITextField` и `UITextView` — однострочное поле ввода и многострочное с прокруткой. У `UITextView` нет `placeholder`, подсказку «Сообщение» кладут поверх отдельной надписью. (гл. 8, 13, 18)

Hit-testing — поиск view под касанием: `hitTest(_:with:)` идёт от окна вглубь, обходит `subviews` с конца (верхние первыми) и пропускает view с `isHidden`, `alpha < 0.01`, `isUserInteractionEnabled == false`; точка вне `bounds` родителя — не попадание. (гл. 0, 20, 21)

`isContinuous` — у `UISlider` и `UIDatePicker`: слать `.valueChanged` непрерывно во время перетаскивания (по умолчанию `true`) или один раз при отпускании. (гл. 17, 33)

Continuous interaction — протягивание пальцем по точкам `UIPageControl` листает страницы подряд; `allowsContinuousInteraction` включено по умолчанию. (гл. 36)

`lineFragmentPadding` — 5 точек слева и справа от строк внутри text container у `UITextView`; добавляются к `textContainerInset` (по умолчанию 8 сверху и снизу). (гл. 13, 18)

`countDownDuration` — значение `UIDatePicker` в режиме `.countDownTimer` в секундах; `date` и границы `minimumDate`/`maximumDate` в этом режиме не используются. (гл. 31)

Графика, цвет и анимации

Адаптивный цвет (dynamic color) — цвет с вариантами для светлой и тёмной темы, повышенного контраста: `UIColor { traits in … }` или Color Set в `Assets.xcassets` с вариантами Any/Dark. Сам перерисовывается при смене темы. (гл. 15, 35)

Блик (shimmer) — светлая полоса, которая бежит по серым заготовкам skeleton'а. Делается `CAGradientLayer` из трёх цветов (серый → светлее → серый) и анимацией его `locations`; при Reduce Motion блик выключают. (гл. 15, 23)

Вращение и радианы — углы в iOS задаются в радианах: π (примерно 3,14) — пол-оборота, 180°; 2π — полный оборот; π/2 — 90°. Из градусов: умножь на π и раздели на 180, 45° = π/4 ≈ 0,785. Ось y смотрит вниз, поэтому положительный угол — по часовой стрелке. (гл. 32, 36)

Кривая анимации (easing) — как распределить путь по времени. `.curveEaseInOut` (по умолчанию) — разгон и торможение: за первую четверть времени квадрат проходит около 12% пути; `.curveEaseOut` — быстрый старт, мягкая остановка, для появления; `.curveEaseIn` — разгон, для ухода; `.curveLinear` — ровно, для вращения спиннера. (гл. 1, 32)

Модельный и отображаемый слой (presentation layer) — внутри `UIView.animate` свойство меняется сразу, а на экране 0,3 секунды рисуется промежуточное значение из presentation layer — копии слоя с текущими значениями (`layer.presentation()`). (гл. 32)

Неявная анимация (implicit animation) — у отдельного слоя, который не принадлежит view, любое изменение `frame` по умолчанию анимируется за 0,25 секунды. Отключается `CATransaction.setDisableActions(true)`. (гл. 21, 23)

Поочерёдное появление (stagger) — элементы появляются друг за другом с задержками, например 0,05 → 0,35 → 0,7 секунды, а не одновременно. (гл. 1)

Преобразование (transform, `CGAffineTransform`) — как нарисовать view поверх его места в раскладке: масштаб, поворот, сдвиг. Масштаб 0,6 — 60% от обычного (иконка 80 × 80 станет 48 × 48), масштаб 1,4 — на 40% больше; constraints при этом не меняются. `.identity` — «без преобразования». (гл. 1, 21, 32)

Пружина (spring) — анимация, которая может перелететь цель и вернуться. `damping` (затухание) от 0 до 1: при 0,5 квадрат, едущий на 100 точек, перелетает примерно на 16 точек, при 0,7 — на 5 («чуть перелетит и вернётся»), при 1,0 — без перелёта. `initialSpringVelocity` — стартовая скорость в долях пути в секунду: для пути 200 точек значение 1 — это 200 точек в секунду. (гл. 1, 32)

Семантический цвет (semantic color) — системный цвет, имя которого говорит «для чего», а не «какой»: `.systemBackground` — фон (белый в светлой теме, чёрный в тёмной), `.label` — основной текст, `.secondaryLabel` — второстепенный. Тёмная тема с ними работает сама. (гл. 2, 35)

Скелет (skeleton) — серые плашки на месте будущего текста и картинок, пока данные грузятся. Повторяет форму настоящей строки, поэтому появление данных выглядит спокойнее, чем пустой экран или спиннер. (введение, гл. 15, 23, 24)

Слой (layer, `CALayer`) — то, что реально рисуется на экране: у каждого view есть слой, а view добавляет к нему касания и Auto Layout. На слое задают тень, скругление, рамку; цвет у слоя — `CGColor`. (гл. 12, 21, 35)

Тёмная тема (dark mode) — системное оформление с тёмным фоном. Семантические и адаптивные цвета подстраиваются сами, а `CGColor` в слоях — «застывший» цвет, его перекрашивают вручную при смене темы. (гл. 15, 35)

Характеристики окружения (trait collection, `UITraitCollection`) — «погода» вокруг view: тема, размер шрифта Dynamic Type, повышенный контраст, size class. Передаётся от окна вниз по иерархии; на смену подписываются через `traitCollectionDidChange` или, с iOS 17, `registerForTraitChanges`. (гл. 15, 21, 35)

`alpha` — непрозрачность view: 0 — полностью прозрачный, 1 — полностью видимый, 0,5 — наполовину. Анимация `alpha` от 0 до 1 — плавное появление (fade-in). (гл. 1, 32)

`CABasicAnimation` — анимация Core Animation, которая меняет одно свойство слоя от одного значения к другому; всю работу на каждом кадре делает система. Нужна для бесконечных анимаций (`repeatCount = .infinity`) и свойств, которые `UIView.animate` не умеет. (гл. 18, 32)

`CADisplayLink` — таймер, который срабатывает ровно перед каждой перерисовкой экрана, обычно 60 раз в секунду (на ProMotion — до 120). Подходит для «бегущего» числа. (гл. 31, 36)

`CAEmitterLayer` — слой-излучатель частиц: конфетти, снег, искры. `emissionLongitude = .pi` — направление вылета, `emissionRange = 0,4` — разброс ±0,4 радиана, примерно ±23°. (гл. 32)

`CAGradientLayer` — слой, который рисует градиент. Цвета — массив `CGColor`, начало и конец — в долях слоя: (0; 0) — левый верхний угол, (1; 1) — правый нижний. (гл. 15, 21)

`CAKeyframeAnimation` — анимация по нескольким значениям подряд: тряска поля с ошибкой, «покачивание» иконки. (гл. 32)

`cgColor` — цвет в форме Core Animation. При переводе адаптивный `UIColor` выбирает один вариант для текущей темы и дальше не меняется; правильный вариант берут через `resolvedColor(with: traitCollection)`. (гл. 15, 21, 35)

Core Animation — слой ниже UIKit, который рисует и анимирует слои (`CALayer`). Анимацию отрисовывает системный процесс, и твоему коду не нужно ничего делать на каждом кадре. (гл. 12, 15, 32)

Fade-in — плавное проявление: прозрачность растёт от 0 до 1. **`.transitionCrossDissolve`** — перекрёстное растворение: старое содержимое за доли секунды гаснет, новое одновременно проявляется. (гл. 1, 4, 20)

Hero-анимация — элемент «перелетает» между экранами: миниатюра в сетке вырастает в полноэкранное фото. Делается своим аниматором перехода (`UIViewControllerAnimatedTransitioning`). (гл. 26, 32, 37)

SF Symbols — несколько тысяч векторных иконок Apple, встроенных в систему. Берутся по имени — `UIImage(systemName: "checklist")`, рисуются как буква шрифта и перекрашиваются в `tintColor`. (гл. 1, 2)

`tintColor` — «фирменный» цвет view, который наследуют вложенные элементы: кнопки панели навигации, галочка ячейки, SF Symbol. (гл. 1, 4, 10)

`UIActivityIndicatorView` (спиннер) — системный крутящийся индикатор «что-то происходит». Подходит, когда длительность неизвестна и форму будущего экрана показать нельзя. (гл. 1, 8, 23)

`UIFontMetrics` — масштабирует свой шрифт так же, как система масштабирует текстовый стиль: 16 точек при обычной настройке станут 21 при XXXL и 45 при AX5. (гл. 15, 34)

`UIView.animate` — базовая анимация: в замыкании пишешь, каким view должен стать, а UIKit плавно проводит его из текущего состояния в новое за `withDuration` секунд. Метод возвращается сразу, строка после него выполняется, пока анимация идёт. (гл. 1, 32)

`UIViewPropertyAnimator` — анимация под управлением: её можно поставить на паузу, прокрутить к доле `fractionComplete` пальцем или развернуть назад. (гл. 32)

`UIVisualEffectView` — view с системным эффектом, например размытием `UIBlurEffect` «как в системе». Им закрывают экран перед уходом в фон. (гл. 11)

Animatable property — свойство, изменение которого внутри `UIView.animate` UIKit интерполирует; у `UIView` документированы `frame`, `bounds`, `center`, `transform`, `alpha`, `backgroundColor`; `isHidden` не анимируется. (гл. 32)

`isRemovedOnCompletion` / `fillMode` — снимать ли CA-анимацию после окончания (по умолчанию `true`) и что показывать вне её интервала (`.removed`). Анимация не меняет модельное значение слоя — его присваивают отдельно. (гл. 15, 32)

`scrubsLinearly` — у `UIViewPropertyAnimator` перемотка через `fractionComplete` линейная, кривая не применяется; по умолчанию `true`. (гл. 32)

`action(forKey:)` — вопрос слоя к делегату, анимировать ли изменение свойства; view-делегат вне блока анимации отвечает `nil`, отдельный слой получает неявную анимацию 0,25 с. (гл. 21, 23)

Содержимое слоя (layer contents) — то, что слой рисует внутри себя; у `UILabel` фон и текст рисуются в содержимом `_UILabelLayer`, поэтому `cornerRadius` без `masksToBounds` фон не скругляет. (гл. 23, 36)

Bounding box (описывающий прямоугольник) — `frame` view с `transform` ≠ identity и результат `convert(_:to:)` для неё: прямоугольник, в который вписана повёрнутая или увеличенная view. (гл. 0, 37)

`UITraitEnvironment` — протокол объектов с `traitCollection`: `UIScreen`, `UIWindow`, `UIViewController`, `UIView`, `UIPresentationController`. `UITraitCollection.current` — трейты текущего контекста, по ним разрешается `cgColor`. (гл. 0, 35)

`UIAppearance` / **appearance proxy** — протокол и прокси-объект класса (`UINavigationBar.appearance()`) для настройки вида всех экземпляров; применяется, когда view попадает в окно. (гл. 0, 35)

`tintColorDidChange()` — метод `UIView`, который UIKit зовёт у subview при смене `tintColor` или `tintAdjustmentMode` у предка. (гл. 35)

Конкурентность и память

Актор (actor) — объект со «своей очередью»: к его данным обращаются строго по одному, поэтому гонки внутри него нет. (введение, гл. 5)

Главный поток (main thread) — единственный «сотрудник», которому разрешено трогать интерфейс. Если лейбл поменяет фоновый поток, приложение может нарисовать мусор или упасть — и не всегда, а раз в сто запусков. (введение, гл. 5)

Гонка данных (data race) — два потока одновременно читают и меняют одни данные. Ошибка проявляется редко и случайно, отладчиком её ловить мучительно; режим Swift 6 превращает большинство таких мест в ошибки компиляции. (введение, гл. 5)

Жетон подписки — значение, которое вернул `NotificationCenter.addObserver(forName:…)`. Его сохраняют и отдают в `removeObserver`, иначе замыкание продолжит вызываться. (гл. 5, 11)

Кооперативная отмена — `task.cancel()` не обрывает задачу, а ставит ей флаг «тебя отменили». `Task.sleep` на флаг реагирует и заканчивается раньше, а свой код проверяет `Task.isCancelled` после `await`. (гл. 9, 25)

Отсоединённая задача (`Task.detached`) — задача, которая ничего не наследует от места создания и выполняется не на главном потоке. Интерфейс из неё трогают только через `await MainActor.run { … }`. (гл. 5, 11)

Слабая ссылка (`weak`) — ссылка, которая не продлевает объекту жизнь: когда его отпустят, она станет `nil`. `[weak self]` в замыкании — список захвата, который не даёт замыканию удерживать экран или координатор. (гл. 1, 3, 4)

Утечка памяти (memory leak) — объект висит в памяти, хотя уже не нужен: например, окно держит замыкание, а замыкание сильно держит координатор. Ищут через Instruments → Leaks. (гл. 3, 43)

Цикл сильных ссылок (retain cycle) — объекты держат друг друга и не освобождаются никогда: лаунчер → координатор → замыкание → лаунчер. Разрывают его слабой ссылкой в одном из звеньев. (гл. 4, 5)

Approachable Concurrency — настройка Xcode 26 («доступная конкурентность»), которая делает правила Swift 6 предсказуемее. Главное: асинхронная функция без пометок выполняется там же, откуда её вызвали, а не «где-то в фоне». (введение, гл. 5)

ARC (автоматический подсчёт ссылок) — Swift считает сильные ссылки на объект и освобождает его, когда их стало ноль. Поэтому `CLLocationManager`, созданный в локальной переменной, освободится сразу после выхода из функции, и его делегат не сработает. (гл. 4, 7)

`await` — точка ожидания: пока асинхронная функция работает, главный поток не заблокирован и рисует интерфейс, а после результата задача продолжается. (гл. 5)

`@concurrent` — пометка для асинхронной `nonisolated`-функции «выполняй в фоновом пуле потоков». Без неё при Approachable Concurrency функция выполнится там, откуда её вызвали. (гл. 5)

Default Actor Isolation = MainActor — настройка Xcode 26: всё, что объявлено в модуле без явной пометки (свои классы, структуры, функции), считается `@MainActor`. Гонка «обновил лейбл из фона» становится ошибкой компиляции. (введение, гл. 5)

`deinit` — метод, который Swift вызывает прямо перед освобождением объекта. Удобен, чтобы проверить, что экран действительно ушёл из памяти; `isolated deinit` для кода на главном акторе требует Swift 6.2 и iOS 18.4. (гл. 2, 12, 17)

`DispatchQueue.main.async` / `asyncAfter` — выполнить код на главном потоке «как только сможешь» или через заданное время: `asyncAfter(deadline: .now() + 1.4)` — через 1,4 секунды. (гл. 1, 15)

`DispatchWorkItem` — кусок работы, который отдают очереди и могут отменить, пока его не начали. На нём строят debounce: каждое нажатие отменяет прошлое задание и назначает новое. (гл. 13, 25)

`MainActor` — особый актор, чья «очередь» — главный поток. Пометка `@MainActor` значит «этот код выполняется только на главном потоке»; весь UIKit Apple уже пометила так. (введение, гл. 5)

`MainActor.assumeIsolated` — утверждение для компилятора «мы уже на главном потоке, выполни синхронно». Проверяется во время работы: если это не так, приложение упадёт с понятной ошибкой. Мост к старым API вроде `addObserver(…queue: .main)`. (гл. 5, 11)

`MainActor.run` — «выполни это замыкание на главном потоке»; `await` перед ним — потому что главный поток может быть занят. (гл. 5)

`nonisolated` — «не привязан ни к какому актору, можно вызывать откуда угодно». Им помечают чистые функции и модели, которые разбирают в фоне: сортировку, форматирование, `Codable`-структуры. (введение, гл. 5, 15)

`NotificationCenter` — общая «доска объявлений» приложения: одни объекты вешают объявления («сцена сейчас станет неактивной»), другие подписываются. Отправитель не знает подписчиков, подписчик не знает отправителя. (гл. 3, 5, 11)

`Sendable` — «значение можно безопасно передать в другой поток». Замыкание `@Sendable` может выполниться на любом потоке, поэтому из него нельзя напрямую звать метод экрана. (гл. 5, 10, 12)

Swift 6 (языковой режим) — строгий режим языка: большинство нарушений правил конкурентности — ошибки, а не предупреждения. Шаблон Xcode 26.5 ставит Swift 5, книга переключает на Swift 6. (введение)

`Task { … }` — запуск асинхронной работы. Созданная в коде на главном потоке задача наследует эту привязку, поэтому после `await` в ней можно менять лейбл. (гл. 5, 9)

`Task.sleep` — асинхронная пауза, которая не блокирует поток: `Task.sleep(nanoseconds: 600_000_000)` — 0,6 секунды. Удобная запись `Task.sleep(for: .milliseconds(600))` появилась только в iOS 16. (гл. 8, 9)

`Timer` — таймер, который вызывает код через интервал. Перед новым вызывают `invalidate()` у старого, а в замыкании берут `[weak self]`: таймер держит своё замыкание, пока работает. (гл. 9)

`withCheckedContinuation` — мост между API, которое отвечает через делегат или замыкание, и `async`-функцией. Continuation одноразовая: `resume` дважды — падение, ни разу — функция будет ждать вечно. (гл. 7)

`withTaskGroup` — группа параллельных задач: запросы по пяти городам уходят в сеть одновременно, а результаты приходят в `for await` по мере готовности, в любом порядке. (гл. 15)

Preconcurrency (`@preconcurrency`) — для API, импортированных из Objective-C, нарушения `@MainActor` Swift 6 показывает предупреждениями, а не ошибками; с чистыми Swift-функциями та же конструкция — ошибка. (гл. 4, 5)

Designated / required инициализатор — подкласс со своим назначенным `init` не наследует родительские; `init?(coder:)` у `UIView` и `UIViewController` помечен `required` из-за `NSCoding`, поэтому его приходится объявлять. (гл. 0, 24, 33)

Данные, файлы и сеть

Атомарная запись (`.atomic`) — данные сначала пишутся во временный файл, а потом одним действием подменяют старый. Если приложение упадёт посреди записи, на диске останется старая полная версия, а не половина новой. (гл. 13, 19)

Дедупликация запросов — два экрана одновременно попросили погоду одного города и получили результат одного запроса. Хранилище держит словарь задач «в полёте» (`inflight`) и отдаёт вторую подписку на уже идущую задачу. (гл. 15)

Доменная модель — тип, с которым работает само приложение, в отличие от DTO, повторяющего ответ сервера. Маппинг переводит одно в другое: переименовывает поля, склеивает массивы в понятные структуры. (гл. 15)

Ключ идемпотентности — метка запроса, по которой сервер узнаёт повтор и не выполняет действие дважды. Без неё платёж нельзя повторять автоматически: первый запрос мог дойти, а упал только ответ. (гл. 24)

Кэш (cache) — копия данных, которую держат, чтобы не качать заново: погода на диске на случай офлайна, картинки в памяти для быстрой прокрутки, конфиг сервера в `UserDefaults`. (гл. 9, 15, 16)

Миграция данных — перевод сохранённых данных в новый формат при обновлении приложения. Версия в ключе (`todo.items.v1`) даёт для неё место: новая версия читает `v1`, переводит и пишет `v2`. (гл. 12)

Оптимистичный интерфейс (optimistic UI) — результат действия показывают сразу, как будто сервер уже согласился, а при ошибке тихо откатывают. Так работают мессенджеры: сообщение появляется в чате до ответа сервера. (гл. 18, 23)

Офлайн-режим (offline) — приложение без сети не показывает пустой экран, а берёт последние сохранённые данные и сообщает баннером, что они могут быть устаревшими. (гл. 15, 24)

Пагинация (pagination) — загрузка данных порциями («страницами») по 20–50 штук. Бесконечная прокрутка (infinite scroll) грузит очередную страницу, когда пользователь докрутил почти до конца. (гл. 16, 23)

Песочница (sandbox) — личная папка приложения на устройстве: другие приложения в неё не заглянут, а при удалении приложения она стирается вместе со всем содержимым. (гл. 6, 39)

Повтор с растущей паузой (exponential backoff) — автоматический повтор неудачного запроса с паузой, которая каждый раз удваивается: 1 с, потом 2 с, потом 4 с. Случайная добавка (jitter) 0–0,3 с разводит во времени тысячи клиентов, чтобы они не били упавший сервер синхронными волнами. (гл. 24)

Подавление дребезга (debounce) — «подожди, пока поток событий затихнет, и только тогда действуй»: поиск запускается через 200–300 мс после последней нажатой буквы, а не на каждую. Как лифт, который не уезжает, пока люди заходят. (гл. 13, 25)

Прореживание (throttle) — «не чаще, чем раз в N секунд»: первое событие проходит сразу, остальные — не чаще интервала. Для поиска по вводу обычно нужен debounce, а не throttle. (гл. 25)

Codable — протокол, с которым тип умеет превращаться в JSON (`JSONEncoder`) и обратно (`JSONDecoder`). Если в новой версии появится обязательное поле, старые данные перестанут читаться. (гл. 12, 15)

`CodingKeys` — перечисление, которое сопоставляет имена свойств с ключами JSON: `download_url` (snake_case, стиль сервера) ↔ `downloadURL` (camelCase, стиль Swift). Сопоставление точное и регистрозависимое. (гл. 15, 16)

Documents, Caches, tmp — папки песочницы. `Documents` — пользовательский контент, попадает в резервную копию; `Library/Caches` — то, что можно скачать заново, система может очистить её при нехватке места; `tmp` — файлы на одну операцию. (гл. 13, 15, 39)

DTO (data transfer object) — «объект для передачи данных», структура, которая повторяет JSON сервера один в один. Приложение с ней напрямую не работает, её переводят в доменную модель. (гл. 15, 25)

Endpoint (эндпоинт) — адрес на сервере, на который приложение шлёт запрос: `/config`, `/todos`. Вместе с полями запросов и ответов адреса образуют API сервера. (гл. 9)

HTTP-статус — трёхзначный код ответа сервера: `200..<300` — успех, 404 — не найдено, 500 — ошибка сервера. `URLSession` не считает 404 и 500 ошибкой, проверять код приходится самому. (гл. 12, 15)

Mock (мок) — ненастоящая реализация сервиса для разработки: вместо сервера — функция с задержкой 0,6 секунды и заранее заданными ответами. Когда появится сервер, мок заменяют, не трогая экраны. (гл. 8, 9)

`NSCache` — словарь «ключ → значение» для кэша, который сам выбрасывает содержимое, когда памяти мало. Обычный словарь картинок держал бы все 500 пролистанных фото до конца жизни приложения. (гл. 16)

`NWPathMonitor` — наблюдатель за доступностью сети из фреймворка Network: сообщает, есть ли путь в интернет, чтобы показать офлайн-баннер. Его замыкание приходит не на главном потоке. (гл. 15, 24)

`timeoutInterval` — сколько секунд ждать ответа: при `timeoutInterval = 10` запрос, на который сервер молчит больше 10 секунд, завершится ошибкой. (гл. 12, 15)

`URLComponents` — сборщик адреса из частей, который сам экранирует параметры: пробелы и кириллицу превращает в `%20`, `%D0%…`. Склеивать URL строками опасно: `"?q=" + "молоко и хлеб"` даст неверный адрес. (гл. 12, 15)

`URLSession` — системный клиент HTTP. `URLSession.shared.data(for:)` (iOS 15+) выполняет запрос асинхронно; настоящая сетевая работа всё равно идёт в фоне без твоего участия. (гл. 12, 15)

UserDefaults — маленькое встроенное хранилище настроек «ключ → значение», которое переживает перезапуск. Годится для флагов («онбординг показан»), но не для секретов: файл не зашифрован отдельно и попадает в резервные копии. (гл. 2, 6, 8, 39)

Дата, время, числа и локализация

Банковское округление (`.halfEven`, `.bankers`) — ровно половину округляют к чётному соседу: 2,5 → 2, 3,5 → 4, 1234,5 → 1234, 1235,5 → 1236. При сложении тысяч сумм ошибки «вверх» и «вниз» гасят друг друга. `NumberFormatter` и `formatted` округляют так по умолчанию; «школьное» — `.halfUp` или `.plain`. (гл. 31)

Значащие цифры — цифры числа без ведущих нулей. `Double` хранит около 16 значащих цифр, дисплей калькулятора показывает 12; числа от 10^{12} и больше (или меньше 10^{-8}) калькулятор пишет экспоненциально: `1.5e20` — 1,5 умножить на 10 в двадцатой степени. (гл. 14)

Локаль (locale) — набор региональных правил устройства: язык, регион, разделитель тысяч, запятая или точка в дробях, «своя» валюта, первый день недели. `ru_KZ` — русский язык по правилам Казахстана: деньги в «1 234,50 ₸». (гл. 10, 31)

Минимальные единицы — хранить деньги целым числом самых мелких единиц: 1999,99 ₸ = 199 999 тиынов, никаких дробей. (гл. 31)

Неразрывный пробел — символ U+00A0, выглядит как пробел, но не даёт строке переломиться посреди суммы. `NumberFormatter` ставит его в «1 234,50 ₸», поэтому тест с обычным пробелом не проходит, хотя строки на глаз одинаковые. (гл. 31)

Плюрал (plural) — выбор формы слова по числу. В русском три формы: «1 минута», «2 минуты», «5 минут»; числа с двумя последними цифрами 11–14 всегда «минут» (112 минут). В файлах локализации это категории Unicode CLDR one / few / many (и other для дробных). (гл. 10, 31)

Разделитель дробной части — в России и Казахстане запятая («2,5»), в США — точка. Движок калькулятора всегда считает с точкой, а заменять её на запятую для показа — задача экрана по `Locale.current.decimalSeparator`. (гл. 14, 31)

Сравнение строк по правилам языка — `localizedStandardCompare` сортирует как словарь: «Ё» рядом с «Е», «file2» раньше «file10». Обычное `<` сравнивает коды символов, и все слова на «Ё» улетают в начало (U+0401 меньше U+0410). (гл. 31)

Часовой пояс (time zone) — смещение местного времени от UTC: Алматы — UTC+5. `Date` пояса не имеет, пояс появляется только при превращении момента в текст; один момент 09:30 UTC в Алматы — 14:30, в Москве — 12:30. (гл. 10, 12, 31)

Эпоха Apple и Unix-время — `Date` внутри хранит секунды от 1 января 2001 года, 00:00 UTC; Unix-время считает секунды от 1 января 1970 года. Полночь 27 сентября 2026 года по UTC — это 812 160 000 секунд в эпохе Apple и 1 790 467 200 в Unix-времени. `JSONEncoder` по умолчанию пишет дату числом в эпохе Apple, поэтому для сервера ставят `.iso8601`. (гл. 12, 15, 31)

`Calendar.current` — календарь и часовой пояс, выбранные на телефоне. `startOfDay(for:)` отбрасывает время суток, а `dateComponents([.year], from:to:)` считает полные годы: родился 28.09.2010, сегодня 27.09.2026 — это 15 лет, а не 16. (гл. 10, 12, 31)

`DateComponentsFormatter` — форматтер длительностей: превращает секунды в текст на языке устройства, например 3723 секунды — «1 ч 2 мин 3 с». (гл. 9, 31)

`DateFormatter` — превращает `Date` в текст и обратно по правилам локали. Создание форматтера сравнительно дорогое, поэтому его делают один раз (`static let`), а не в каждой ячейке. (гл. 12, 13, 31)

`Decimal` — число в десятичной системе: 0,1 + 0,2 у него ровно 0,3. Выбор для денег; создают из строки или целого, потому что `Decimal(1234.56)` сначала станет неточным `Double` (1234.5599999999997952). (гл. 14, 31)

`Double` и 0,1 + 0,2 — `Double` хранит дробь в двоичной системе, где 0,1 — бесконечная дробь, как 1/3 в десятичной. Поэтому `0.1 + 0.2` даёт `0.30000000000000004`, а десять раз по 0,1 — 0,9999999999999999. Для размеров на экране это неважно, для денег — ошибка на копейку. (гл. 14, 31)

`en_US_POSIX` — «техническая» локаль, которая не меняется от настроек пользователя: точка как десятичный разделитель, никаких пробелов в тысячах, латинские цифры. С ней разбирают даты и числа от сервера, иначе у человека с 12-часовым форматом времени разбор может сломаться. (гл. 14, 31)

`FormatStyle` (`.formatted`) — современное форматирование (iOS 15+) без создания и кэширования форматтера: `date.formatted(date: .long, time: .shortened)`, `money.formatted(.currency(code: "KZT"))`. (гл. 31)

ISO 3166 — стандарт двухбуквенных кодов стран: `KZ`, `RU`, `UZ`. Этот же код iOS возвращает в локали, поэтому по нему удобно сравнивать регион. (гл. 10)

ISO 8601 — международная запись даты: `2026-05-12T10:30:00Z` — год, месяц, день, после `T` время, в конце пояс. Сервера почти всегда отдают даты так; с дробными секундами нужна опция `.withFractionalSeconds`. (гл. 12, 31)

`NSLocalizedString` / `String(localized:)` — достать переведённую строку по ключу из ресурсов приложения; с числом система сама выберет нужную форму плюрала. (гл. 10, 31)

`NumberFormatter` — форматирование чисел: `.currency` сразу даёт разделитель тысяч, два знака после запятой и символ «своей» валюты локали. С `ru_RU` тенге выводится кодом «KZT», знак ₸ — только для `ru_KZ` и `kk_KZ`. (гл. 14, 31)

`RelativeDateTimeFormatter` — «N минут назад» по правилам языка: «3 минуты назад», «21 минуту назад». Для лент и чатов, где важнее «как давно», чем точная дата. (гл. 15, 31)

`setLocalizedDateFormatFromTemplate` — просишь поля даты («день и сокращённый месяц», `"d MMM"`), а порядок и знаки расставит локаль: для русской получится «27 сент.». (гл. 12, 15)

String Catalog (`.xcstrings`) — файл переводов Xcode 15+. У строки можно включить Vary by Plural и заполнить формы, а выбор формы по числу берёт на себя система. (гл. 10, 31)

UTC — всемирное координированное время, нулевой часовой пояс; буква `Z` в конце даты («Zulu») значит «по UTC». Правило: передавай и храни в UTC, показывай в поясе пользователя. (гл. 10, 13, 31)

`YYYY` и `yyyy` — в формате даты большая `Y` — «год недели» по ISO, маленькая `y` — календарный год. 29 декабря 2025 года с `"YYYY-MM-dd"` напечатается как «2026-12-29», поэтому всегда пиши `yyyy`. (гл. 31)

Безопасность и разрешения

Биометрия (Face ID, Touch ID) — проверка владельца по лицу или отпечатку. Само распознавание делает Secure Enclave, но приложение получает лишь «да» или «нет», и на взломанном телефоне этот `if success` можно подменить. (гл. 11)

Взломанное устройство (jailbreak) — iPhone со снятыми ограничениями iOS: там можно прочитать файлы любого приложения, включая `UserDefaults`, и подменить решения его кода. Поэтому токен кладут в Keychain. (гл. 8, 11)

Размытие для приватности (privacy blur) — перед уходом в фон (на `willDeactivate`) окно накрывают размытием, чтобы в переключателе приложений и в снимке экрана не было чувствительных данных. (гл. 11)

Разрешение (permission) — доступ к личным данным и железу: геолокация, фото, камера, микрофон, контакты, уведомления. Приложение не может взять их само, iOS показывает системный вопрос, и ответ можно спросить только один раз. (гл. 7)

Регулярное выражение (regex) — шаблон для текста. Для email книга берёт нестрогий шаблон «символы без пробела и @, потом @, потом снова символы, точка и хотя бы два символа»: `a@b.kz` проходит, `a@b` — нет. (гл. 8, 30)

Статус разрешения — `notDetermined` (ещё не спрашивали), `denied` (отказал), `restricted` (запрещено ограничениями устройства, например родительским контролем), `limited` (для фото — доступ только к выбранным снимкам). После отказа повторного диалога не будет, остаётся ссылка в Настройки через `openSettingsURLString`. (гл. 7)

Строка-пояснение (purpose string, usage description) — текст в `Info.plist`, который iOS показывает в системном вопросе о разрешении: `NSCameraUsageDescription`, `NSLocationWhenInUseUsageDescriptio` Без неё приложение падает при первом запросе. (гл. 7, 39)

Токен (token) — строка, которую сервер выдаёт после успешного входа: «пропуск», который приложение прикладывает к каждому запросу. Пароль после входа хранить не нужно, хранится только токен — в Keychain. (гл. 8, 11)

Трекинг (tracking) — по определению Apple, связывание данных о человеке из твоего приложения с данными других компаний для рекламы или её измерения, а также передача данных брокерам. Своя реклама по данным своего же приложения — не трекинг. (гл. 39)

Экран-подводка к разрешению (permission primer) — свой экран, который объясняет, зачем нужно разрешение, ещё до системного вопроса. Кнопка на нём одна — «Продолжить», и ведёт она к системному диалогу: пункт 5.1.1(iv) Guidelines запрещает манипулировать человеком или принуждать его к согласию. (введение, гл. 7)

App Tracking Transparency (ATT) — фреймворк и системный вопрос «Разрешить отслеживание?». Нужен, только если приложение связывает данные о человеке с данными других компаний для рекламы (это и есть трекинг); без разрешения рекламный идентификатор IDFA — одни нули. (гл. 7, 39)

App Transport Security (ATS) — механизм iOS, который по умолчанию разрешает сеть только по HTTPS с современным TLS. `NSAllowsArbitraryLoads` выключает его для всех доменов — в production так не делают, исключение пишут для одного домена. (гл. 39)

Associated Domains — capability, которая связывает приложение с твоим доменом. Нужна для Universal Links и для того, чтобы iOS предлагала пароли от сайта и генерировала надёжный пароль в приложении. (гл. 8, 30, 41)

IDFA — рекламный идентификатор устройства. Пока человек не разрешил отслеживание через ATT, он возвращается из одних нулей: `00000000-0000-0000-0000-000000000000`. (гл. 39)

Keychain (связка ключей) — системное зашифрованное хранилище секретов на устройстве: пароли Safari, Wi-Fi и токены приложений. Запись в нём определяют «сервис» (обычно bundle id плюс назначение) и «аккаунт» (имя слота). (гл. 8, 11, 39)

`kSecAttrAccessible` — когда запись Keychain доступна. `WhenUnlockedThisDeviceOnly` — только при разблокированном телефоне и без переезда на новое устройство из резервной копии; `AfterFirstUnlock` — после первой разблокировки с момента включения, для работы в фоне. (гл. 8, 11)

`LAContext` — главный класс фреймворка LocalAuthentication. `canEvaluatePolicy` проверяет, доступна ли биометрия, и заполняет `biometryType`; `evaluatePolicy` показывает окно Face ID с твоим текстом `localizedReason`. (гл. 11)

`LAPolicy` — что проверять: `.deviceOwnerAuthenticationWithBiometrics` — только лицо или палец; `.deviceOwnerAuthentication` — биометрия, а если не вышло, код-пароль устройства. (гл. 11)

LocalAuthentication — фреймворк Apple для проверки владельца устройства: Face ID, Touch ID или код-пароль. Для Face ID нужна строка `NSFaceIDUsageDescription` в `Info.plist`. (гл. 11, 19)

Passkeys — вход вообще без пароля: ключ хранится в связке ключей и подтверждается Face ID. HIG советует passkeys, если в приложении нет Sign in with Apple. (гл. 8)

PhotoKit (`PHPhotoLibrary`) — доступ к медиатеке. Уровень `.addOnly` — только добавлять снимки, не видя библиотеку; `.readWrite` — полный доступ со своим диалогом. (гл. 7, 16, 37)

`PHPickerViewController` — системное окно выбора фото (iOS 14+). Работает в отдельном процессе iOS, приложение получает только выбранные снимки, и разрешение на медиатеку не нужно. (гл. 7, 16, 19)

`SecAccessControl` — правило доступа к записи Keychain. С флагом `.biometryCurrentSet` iOS отдаст запись только после Face ID или Touch ID с текущим набором лиц и пальцев: добавил кто-то своё лицо — запись недоступна. Мягче — `.biometryAny` и `.userPresence`. (гл. 11)

Secure Enclave — отдельный защищённый сопроцессор, который хранит биометрические данные и сам сравнивает лицо или отпечаток. Подделать распознавание нельзя, поэтому привязка секрета к биометрии надёжнее своего `if`. (гл. 11)

Sign in with Apple — вход через Apple ID. При удалении аккаунта токены Sign in with Apple нужно отозвать на сервере Apple. (гл. 8, 40)

TCC — подсистема iOS, которая следит за доступом к личным данным. Если нет строки-пояснения, TCC роняет приложение при первом запросе разрешения. (гл. 7)

Доступность

Доступность (accessibility) — возможность пользоваться приложением людям с особенностями зрения, слуха, моторики: незрячему — с экранным диктором, слабовидящему — с крупным шрифтом, человеку с тремором — с крупными кнопками. Стандартные компоненты UIKit доступны «бесплатно», ломается своё. (гл. 34, 43)

Контраст (contrast ratio) — во сколько раз светлота текста больше светлоты фона (к обеим прибавляют 0,05). Чёрный на белом — 21:1, максимум; серый `#767676` на белом — 4,54:1, ровно на грани. По WCAG обычному тексту нужно не меньше 4.5:1, крупному (от 18 пунктов или от 14 жирным) и значкам — 3:1. В таблице HIG, по которой проверяет Accessibility Inspector, проще: текст до 17 pt — 4.5:1, от 18 pt и жирный любого размера — 3:1; мелкий жирный текст надёжнее держать на 4.5:1, тогда он проходит оба правила. (гл. 34, 35, 43)

Ротор (rotor) — «колёсико» VoiceOver: поворотом двух пальцев переключают режим навигации, например «по заголовкам». Поэтому заголовки экранов и секций помечают трейтом `.header`. (гл. 34)

Текстовый стиль (`preferredFont(forTextStyle:)`) — шрифт не фиксированного размера, а по роли: `.title1`, `.body`, `.headline`. Размер берётся из настройки Dynamic Type. (гл. 1, 6)

Трейт (accessibility trait) — признак элемента для VoiceOver: `.button` («кнопка»), `.header`, `.selected` («выбрано»), `.adjustable` (значение меняют смахиванием вверх и вниз). (гл. 14, 19, 34)

Уведомление VoiceOver (`UIAccessibility.post`) — сообщение диктору: `.screenChanged` — «экран сильно изменился», фокус переходит на указанный элемент; `.announcement` — прочитать фразу. (гл. 10, 20)

Элемент доступности (accessibility element) — то, на чём останавливается VoiceOver. `isAccessibilityElement = true` на карточке читает её одной фразой «Ветер: 1 м/с», а `false` на декоративной иконке убирает её из чтения. (гл. 15, 24, 34)

Accessibility Inspector — инструмент Xcode: показывает label, value и трейты элемента под курсором, а кнопка Audit проверяет экран на низкий контраст, мелкие зоны касания и пропущенные подписи. (гл. 34, 43)

Accessibility Nutrition Labels — «этикетка доступности» на странице приложения в App Store: какие из девяти функций (VoiceOver, крупный текст, тёмная тема и другие) поддерживаются. Заполняется в App Store Connect. (гл. 34, 43)

`accessibilityCustomActions` — свои действия для VoiceOver. Ими дублируют свайп-действия и контекстные меню, которые незрячий человек не увидит. (гл. 34, 43)

`accessibilityLabel`, `accessibilityValue`, `accessibilityHint` — что прочитает VoiceOver: label — имя («Удалить»), value — текущее значение («4 из 5 звёзд»), hint — что произойдёт при нажатии, читается после паузы. Кнопка без label с одной картинкой прозвучит просто как «кнопка». (гл. 12, 34)

Dynamic Type — системная настройка размера текста (Настройки → Экран и яркость → Размер текста). Текст на `preferredFont(forTextStyle:)` с `adjustsFontForContentSizeCategory = true` меняется на лету; на максимальном размере AX5 основной текст — 53 пункта вместо 17, в 3,12 раза крупнее. (гл. 1, 34, 43)

Increase Contrast (Увеличение контраста) — настройка, при которой системные цвета становятся контрастнее. Своя палитра реагирует на неё через вариант High Contrast у Color Set или `traits.accessibilityContrast == .high`. (гл. 34, 35)

Reduce Motion (Уменьшение движения) — настройка универсального доступа для людей, которых укачивает от анимаций. Проверяется `UIAccessibility.isReduceMotionEnabled`: бегущий блик и параллакс выключают, пружины заменяют плавным появлением. (гл. 15, 21, 34, 43)

Switch Control и Voice Control — управление для тех, кто не может пользоваться сенсорным экраном. В Switch Control система по очереди подсвечивает элементы, а человек нажимает «выбрать» одной-двумя кнопками; в Voice Control всем управляют голосом («нажми Добавить»). (гл. 43)

VoiceOver — экранный диктор iOS для незрячих: читает вслух элемент под пальцем, свайп вправо — к очередному элементу, двойной тап — нажатие. В симуляторе его нет, проверяют на устройстве. (гл. 12, 34, 43)

WCAG — международные правила доступности веб-контента, на которые ссылается Apple. Уровень AA: 4.5:1 для обычного текста и 3:1 для крупного и значков; AAA: 7:1 и 4.5:1. (гл. 1, 34, 35)

`accessibilityElementsHidden` — скрыть от VoiceOver все элементы внутри view; сам контейнер элементом не становится. (гл. 34)

Публикация, пуши и расширения

Анкета App Privacy — раздел App Store Connect, где ты описываешь, какие данные приложение собирает (передаёт с устройства) и зачем. Из неё на странице приложения собирается «этикетка конфиденциальности» (Privacy Nutrition Label); ответы можно менять без новой версии. (гл. 39)

Бюджет обновлений виджета — сколько раз в сутки система согласится обновить виджет: обычно 40–70. Сутки — 1440 минут, 1440 / 70 ≈ 20 минут, 1440 / 40 = 36 минут, отсюда ориентир «раз в 15–60 минут». Перезагрузка по просьбе открытого приложения бюджет не тратит. (гл. 42)

Виджет (widget) — карточка на главном экране или экране блокировки с данными приложения. Это не работающая программа, а заранее подготовленная картинка по расписанию; интерфейс пишется только на SwiftUI, даже если приложение на UIKit. (гл. 42)

Глубокая ссылка (deep link) — ссылка, которая открывает приложение сразу на нужном экране: `myapp://chat/1234`. В книге все ссылки — из пушей, URL-схем, Universal Links и виджетов — сходятся в один маршрутизатор `DeepLinkRouter`. (гл. 41, 42)

Мягкое и жёсткое удаление (soft / hard delete) — жёсткое стирает строки из базы сразу; мягкое ставит пометку «удалён» и стирает по-настоящему после отсрочки, например через 30 дней, давая время передумать. (гл. 40)

Пуш-уведомление (push notification) — сообщение, которое сервер присылает на телефон, даже когда приложение закрыто. Доставляет его только APNs, напрямую твой сервер на телефон ничего отправить не может. (гл. 41)

Расширение (app extension) — отдельный исполняемый модуль со своим таргетом, который живёт внутри приложения, но работает в своём процессе и запускается системой: виджет, Notification Service Extension. (гл. 41, 42)

Скриншоты App Store — от 1 до 10 на размер экрана и язык, в пикселях и без прозрачности. Для iPhone достаточно одного набора 6,9” (например, 1320 × 2868 — это 440 × 956 точек × 3) — меньшие экраны Apple уменьшит сама. (гл. 38)

Таймлайн (timeline) — расписание виджета: массив записей (`TimelineEntry`), у каждой дата, с которой её показывать. Провайдер (`TimelineProvider`) отвечает системе на `placeholder`, `getSnapshot` и `getTimeline` и задаёт политику обновления, например `.after(через 30 минут)`. (гл. 42)

Тихий пуш (silent push) — пуш без баннера и звука с `"content-available": 1`: будит приложение в фоне, чтобы обновить данные. Приходит и без разрешения на уведомления. (гл. 41)

Удаление аккаунта — по пункту 5.1.1(v) Guidelines приложение, где можно создать аккаунт, обязано давать и удалить его изнутри приложения — по-настоящему, с данными на сервере, а не только выйти. (гл. 8, 19, 40)

APNs (Apple Push Notification service) — сервис Apple, через который идут все пуши на устройства Apple. У него две среды: development (sandbox) для сборок из Xcode и production для TestFlight и App Store; среду сборки задаёт entitlement `aps-environment`. (гл. 41)

App Group — общая папка и общий `UserDefaults` для приложения и его расширений одной команды. Виджет — отдельный процесс со своей песочницей и без App Group не видит данных приложения. (гл. 12, 42)

App Intent — действие приложения, которое система вызывает сама: из Siri, «Команд», Spotlight, с кнопки на виджете. Описываешь его структурой с `title`, `@Parameter` и `perform()`, а интерфейс вокруг строит система. Фреймворк App Intents — с iOS 16. (гл. 42)

App Preview — видео на странице App Store: 15–30 секунд, до трёх на размер экрана и язык, не больше 30 кадров в секунду и до 500 МБ. (гл. 38)

App Review Guidelines — правила проверки App Store с номерами пунктов: 2.3.3 — скриншоты показывают приложение в работе, 4.8 — вход через сторонние сервисы, 5.1.1 — сбор данных и разрешения, 5.1.2 — трекинг. (гл. 7, 8, 38, 39, 40)

App Store Connect — сайт Apple для публикации: карточка приложения, скриншоты, анкета App Privacy, загрузка сборок и отзывы. Автоматическая проверка сборки ловит, например, отсутствующие строки-пояснения. (гл. 7, 38, 39)

Apple-app-site-association (AASA) — файл на твоём сервере по адресу `https://домен/.well-known/apple-app-site-association` без расширения, по https и без редиректов. Подтверждает связь домена с приложением для Universal Links и паролей. (гл. 30, 41)

`ASWebAuthenticationSession` — системное окно входа через внешний сервис (OAuth): показывает страницу входа и возвращает результат прямо в приложение, так что перехват URL-схемы чужим приложением ему не страшен. (гл. 41)

Capability («возможность») — функция, которую включают на вкладке Signing & Capabilities: Push Notifications, App Groups, Associated Domains. Включение добавляет в подпись приложения entitlement. (гл. 10, 41)

Device token (токен устройства) — адрес твоего приложения на конкретном телефоне, который выдаёт APNs при регистрации. Уникален для пары «приложение + устройство»; приложение отправляет его своему серверу. (гл. 5, 41)

Dynamic Island — вырез-«пилюля» вверху экрана на iPhone 14 Pro и новее. Safe area его обходит, а Live Activity может показывать в нём свои данные. (гл. 1, 42)

Entitlement — разрешение, вшитое в подпись приложения: «этому приложению можно получать пуши, в такой-то среде». Появляется, когда включаешь capability. (гл. 10, 41)

Fingerprinting («снятие отпечатка») — вычислить устройство по набору мелких признаков (время загрузки системы, объём диска, список клавиатур) и узнавать человека без согласия. Apple запрещает это даже при разрешённом трекинге. (гл. 39)

Live Activity — карточка «здесь и сейчас» на экране блокировки и в Dynamic Island: доставка, такси, счёт матча (iOS 16.1+, фреймворк ActivityKit). Данных — не больше 4 КБ, живёт до 8 часов. (гл. 42)

Notification Service Extension — расширение, которое получает пуш до показа и может его изменить: расшифровать текст, скачать и прикрепить картинку. Срабатывает, если в payload есть `"mutable-content": 1`. (гл. 41)

`.p8`-ключ APNs — ключ из Apple Developer, которым сервер подписывает токен (JWT) для запросов к APNs. Вместе с Key ID, Team ID и bundle ID его отдают разработчикам сервера; кто владеет ключом, может слать пуши твоим пользователям. (гл. 41)

Payload («полезная нагрузка») — JSON пуша, который сервер отправляет в APNs: в словаре `aps` — заголовок, текст, бейдж, звук, рядом — свои поля вроде `deep_link`. Размер — до 4 КБ. (гл. 41)

Privacy manifest (`PrivacyInfo.xcprivacy`) — файл в бандле, где приложение объявляет трекинг, домены трекинга, собираемые данные и required reason API. С 1 мая 2024 года App Store Connect не принимает сборки без нужных причин в манифесте, независимо от минимальной версии iOS. (гл. 12, 39)

Provisional authorization — «временное» разрешение на уведомления (iOS 12+): системного диалога нет, уведомления тихо попадают в Центр уведомлений, и человек сам решает там, оставить их или выключить. (гл. 41)

Provisioning profile (профиль подписи) — файл Apple, который связывает приложение, команду разработчика и устройства. По нему Xcode ставит значения entitlements, например среду APNs. (гл. 41)

Required reason API — системные API, которые можно использовать для fingerprinting (`UserDefaults`, даты файлов, время загрузки, свободное место, список клавиатур). Для них в privacy manifest указывают причину кодом из списка Apple, например `CA92.1` для своих `UserDefaults`. (гл. 12, 13, 39)

Team ID — идентификатор твоей команды разработчика в Apple Developer (виден в Membership details). Нужен серверу для пушей и в файле AASA, где приложение записано как `<Team ID>.<bundle ID>`. (гл. 41)

TestFlight — бета-тестирование Apple на реальных устройствах. Сборка из TestFlight регистрируется в production-среде APNs, а не в sandbox — частая причина «пуши не доходят». (гл. 41, 43)

Universal Link — обычная https-ссылка на твой сайт, которую iOS открывает в приложении, если оно установлено, и в Safari, если нет. Подделать нельзя: связь подтверждает файл AASA на сервере, который контролируешь только ты. (гл. 5, 41)

URL-схема (URL scheme) — префикс ссылки вида `myapp://`, который приложение регистрирует за собой в `Info.plist` (`CFBundleURLTypes`). Схему может занять и чужое приложение, поэтому для ссылок наружу лучше Universal Links. (гл. 41, 42)

WidgetKit — фреймворк виджетов (iOS 14+): главный экран — `systemSmall`, `systemMedium`, `systemLarge`; экран блокировки (iOS 16+) — `accessoryCircular`, `accessoryRectangular`, `accessoryInline`; интерактивные кнопки на виджете — с iOS 17. (гл. 42)

Xcode Organizer — окно Xcode с отчётами о падениях (Crashes), зависаниях, времени запуска и расходе энергии, собранными с устройств пользователей после релиза. (гл. 43)

Жаргон и паттерны

Бейдж (badge) — красный кружок с числом на иконке вкладки или приложения: «3 непрочитанных». У вкладки задаётся `tabBarItem.badgeValue`, у иконки — полем `badge` пуша. (гл. 26, 36, 41)

Гейт (gate) — экран, который пользователь проходит до main-экрана mini-app, как турникет на входе в офис: onboarding, вход в аккаунт, проверка возраста, Face ID при возврате. (введение, гл. 2, 4)

Делегат (delegate) — объект, которому другой объект сообщает о событиях и у которого спрашивает решения: таблица сообщает «тапнули строку», система — «приложение ушло в фон». Связь хранят слабой ссылкой, чтобы не получить цикл. (гл. 4, 5, 8)

Заглушка (placeholder) — временная замена настоящего содержимого: экран «СКОРО» на месте неготового mini-app, серая подсказка в пустом поле ввода, скелет на месте данных. (гл. 2, 8, 23)

Запасной вариант (fallback) — что взять, если основное не сработало: `?? .current`, если имя часового пояса незнакомо; системная иконка, если обложки нет. (гл. 15, 17)

Коллбэк (callback) — функция обратного вызова: замыкание, которое передают объекту, чтобы он «позвонил», когда закончит. Splash вызывает `onFinish`, а что дальше, решает координатор; как курьеру оставить номер «позвони, когда доставишь». (введение, гл. 1, 22)

Конечный автомат (state machine) — у объекта конечный набор состояний, и каждое событие переводит его из одного в другое. Так устроены калькулятор (каждое нажатие меняет ситуацию) и координатор гейтов. (гл. 4, 14, 36)

Координатор (coordinator) — объект, который решает, какой экран показать дальше, чтобы сами экраны об этом не знали. `BootCoordinator` ведёт mini-app через гейты к main-экрану. (гл. 1, 4)

Лаунчер (launcher) — главный экран playground'а со списком mini-app: запускает их, как рабочий стол iPhone запускает приложения. (введение, гл. 4)

Наблюдатель (observer) — «кто-то, кто хочет знать об изменениях». Хранилище держит список наблюдателей и зовёт их после каждого изменения, а экраны перерисовываются. (гл. 12, 17)

Онбординг (onboarding) — несколько вводных экранов при первом запуске: «вот что умеет приложение». Флаг «уже показан» хранится в `UserDefaults` отдельно для каждого mini-app. (введение, гл. 6)

Пустое, ошибочное и офлайн-состояние (empty / error / offline state) — три исхода загрузки, кроме успеха: ничего не пришло, пришла ошибка, нет интернета. У каждого свой экран с иконкой, текстом и действием («Обновить»); белый экран в любом случае выглядит как поломка. (гл. 24)

Синглтон (singleton) — класс с единственным общим экземпляром `static let shared`. Музыкальный плеер должен быть один: если каждый экран заведёт свой `AVPlayer`, два трека заиграют одновременно. (гл. 12, 17)

Удалённая конфигурация (remote config) — настройки, которые приложение получает с сервера при запуске: минимальная поддерживаемая версия, режим техработ. Позволяет менять поведение без выпуска новой версии. (гл. 9)

Фабрика (factory) — замыкание или функция, которая создаёт объект по запросу: не сам товар, а цех, который выпускает товар по заказу. `makeMain` создаёт main-экран заново при каждом входе в mini-app. (гл. 2)

Шторка (drawer) — боковое меню, которое выезжает от края экрана поверх контента; под ней — затемнение (overlay), полупрозрачная чёрная «вуаль», тап по которой закрывает шторку. (гл. 20)

API — договор между сервером и приложением: какие адреса у сервера и какие поля в запросах и ответах. Изменение, после которого старые версии приложения перестают понимать ответ, называют breaking change («ломающее изменение»). (гл. 9)

`AVPlayer` и аудиосессия — `AVPlayer` играет звук и видео из файла или по сети, начиная до конца загрузки. Аудиосессия (`AVAudioSession`) — договор приложения с системой звука: категория `.playback` не даёт переключателю «Без звука» и блокировке экрана заглушить музыку. (гл. 17)

Bottom sheet — лист снизу с детентами для «лёгкой» задачи, после которой хочется вернуться к тому, что под ним: новая заметка, фильтр, выбор адреса. (гл. 28)

Call-to-action («призыв к действию») — главная кнопка пустого или ошибочного экрана, которая говорит, что делать дальше: «Создать первую заметку», «Повторить». (гл. 24)

Chips («таблетки») — маленькие скруглённые кнопки-фильтры в ряд: «до 10 000 ₸», «Apple», «В наличии». (гл. 25)

Dependency injection («внедрение зависимости») — объект не сам решает, с чем работать, ему это передают снаружи: хранилище получает `UserDefaults` параметром, и в тестах можно подсунуть другое. (гл. 4, 12)

FAB (floating action button) — «плавающая кнопка действия»: круглая кнопка «плюс» поверх списка в углу экрана. (гл. 12)

Force update (принудительное обновление) — гейт «обновите приложение»: сервер сообщает минимальную версию, и старое приложение показывает блокатор с кнопкой в App Store. (гл. 9)

HUD (Heads-Up Display) — блок с иконкой и текстом по центру экрана для короткого подтверждения: «Сохранено», «Скопировано». Сам исчезает через секунду-две. (гл. 28)

Human Interface Guidelines (HIG) — продуктовые правила Apple о том, как должны выглядеть и вести себя приложения: launch screen, разрешения, размеры касаний, доступность. (введение, гл. 1)

Maintenance (режим обслуживания) — гейт «ведутся технические работы»: сервер сообщает, что сейчас недоступен, а приложение показывает экран с таймером и повтором. (гл. 9)

Mini-app — одно мини-приложение в playground'е книги со своим splash, фирменным цветом и гейтами: «Список дел», «Погода», «Чат». (введение, гл. 2)

Mini-player — маленький плеер внизу списка, который играет, пока ты листаешь, и по тапу открывается в полноэкранный «Сейчас играет». (гл. 17)

Photo viewer (просмотрщик фото) — показ фото на весь экран с увеличением пальцами, двойным тапом, листанием и закрытием смахиванием вниз. (введение, гл. 16, 37)

Playground (полигон) — учебный проект книги `UIKitPlayground`: один Xcode-проект, где на главном экране список мини-приложений, и каждое стартует как отдельное приложение. (введение)

Pull-to-refresh — «потяни список вниз, чтобы обновить»: жест, после которого появляется крутилка и данные перезагружаются. (введение, гл. 15, 23)

Semantic versioning (версия «мажорная.минорная.патч») — версия `2.10.1` из трёх чисел. Сравнивать её как строку нельзя: посимвольно «1.10» меньше «1.9», потому что «1» меньше «9», поэтому части сравнивают как числа. (гл. 9)

Storyboard — файл Xcode, где экраны не пишут кодом, а рисуют мышкой в Interface Builder. В книге он нужен только для `LaunchScreen`. (введение, гл. 1)

Toast / Snackbar — короткое сообщение внизу экрана, которое само появляется и исчезает через пару секунд: «Сохранено», «Нет сети». (гл. 23)

Typing indicator — три прыгающие точки «печатает…» в чате, пока собеседник набирает ответ. (гл. 18, 36)

`xcrun simctl` — командная строка симулятора: сбросить разрешения (`privacy booted reset`), найти папку приложения, подменить статус-бар на «9:41» для скриншотов. (гл. 7, 13, 38)